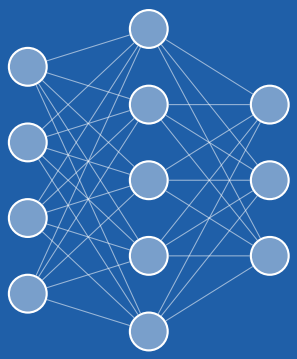
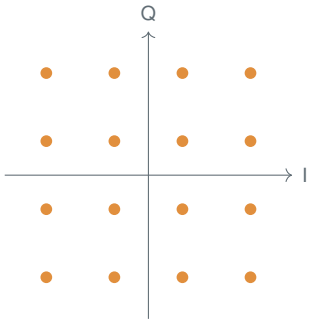
Neural Networks and Deep Learning from Absolute Zero



with Applications to Wireless Communications

Intuition first. Every equation derived.
Every concept visualised.

From vectors and derivatives to backpropagation, CNNs, LSTMs, Transformers and reinforcement learning — then the physical layer as an autoencoder, modulation recognition, channel decoding, model-free end-to-end learning, MIMO, CSI prediction, indoor localization and dynamic spectrum access, with hands-on labs.



Prepared for Sufi · Edition of October 2026

Preface

This book was written for one reader: an engineer who wants to understand neural networks and deep learning *completely* — the mathematics, the intuition behind it, and the practice of building models — and then to see how these tools are being used to rethink wireless communication systems.

It begins from absolute zero. Part I teaches only the mathematics you need, with pictures before symbols. Part II derives the neural network from a single neuron to backpropagation, and computes everything by hand. Part III makes you build networks yourself, first in NumPy (with gradient checking, so you *know* your maths is right) and then in PyTorch. Part IV covers the deep architectures — convolutional, recurrent and attention-based networks, generative models — and reinforcement learning, which several of the wireless papers rely on.

Part V is where the two fields meet. After a wireless primer written for a reader with no radio background, each chapter teaches one research paper: what problem it solves, the system model, the network architecture drawn as a block diagram, the training procedure, the results the authors report, and *our own small-scale reproduction* with code you can run. Part VI contains four Kaggle-style labs on modulation classification in increasingly hostile channels, and Part VII shows how to design, debug and deploy a model of your own.

Every figure in this book was produced by a Python script that was actually run while the book was being written; the numbers quoted in the reproductions are the numbers those scripts produced. Where a reproduction did *not* match the paper, the text says so and explains why — that is part of learning to read research critically. Where a detail of a paper could not be verified from the paper itself, the text says that too.

Read it with a pencil. Do the worked examples yourself. Change one number in each script and predict what will happen before you run it. By the end you will be able to derive, build, train, debug and deploy your own models — and read the wireless-ML literature as a peer.

How the boxes work.

Intuition: blue boxes

the picture to hold in your head. Read these first.

Definition: teal boxes

precise statements of new terms.

Key equation: purple boxes

the equations worth memorising.

Worked example: green boxes

worked numerical examples — redo them by hand.

Wireless connection: orange boxes

how a general deep-learning idea shows up in radio systems.

Pitfall: red boxes

common mistakes and how to recognise them.

Paper at a glance: gold boxes

a one-glance summary of a research paper.

Contents

Preface **i**

1 How to Use This Book **1**

1.1 Who this book is for . . . 1
1.2 The map of the book . . . 2
1.3 How to study each chapter . . . 2
1.4 Software setup . . . 3
1.5 Notation used throughout . . . 4
1.6 Suggested schedules . . . 4

I Mathematical Foundations **5**

2 Vectors, Matrices and Linear Maps **6**

2.1 Scalars and vectors . . . 6
2.2 The dot product . . . 8
2.3 Matrices: machines that transform space . . . 10
2.4 Matrix multiplication . . . 12
2.5 Transpose, identity, inverse . . . 13
2.6 The determinant: how much area is scaled . . . 14
2.7 Linear independence and rank . . . 14
2.8 Eigenvalues and eigenvectors . . . 15
2.9 The singular value decomposition . . . 16
2.10 Broadcasting and tensors . . . 18
2.11 Complex numbers, vectors and matrices . . . 18

3 Derivatives, Gradients and the Chain Rule **24**

3.1 Functions . . . 24
3.2 The derivative: slope at a point . . . 24
3.3 Differentiation rules, derived . . . 26
3.4 Numerical derivatives and why centred differences win . . . 28
3.5 Many inputs: partial derivatives and the gradient . . . 29
3.6 The chain rule . . . 31
3.7 Computational graphs and backpropagation by hand . . . 32
3.8 Taylor expansions, the Hessian and curvature . . . 33
3.9 Matrix calculus essentials . . . 35
3.10 A note on complex variables (Wirtinger derivatives) . . . 36

4 Probability, Statistics and Information **40**
4.1 Events and the rules of probability . . . 40
4.2 Random variables and their distributions . . . 41
4.3 Expectation and variance . . . 41
4.4 The distributions you need . . . 42
4.5 Joint, marginal and conditional probability . . . 44
4.6 Sums of random variables: the facts behind weight initialisation . . . 45
4.7 The law of large numbers and the central limit theorem . . . 46
4.8 Exponentials and logarithms . . . 47
4.9 Maximum likelihood estimation . . . 48
4.10 Information theory . . . 49
4.11 Channel capacity . . . 52

II Neural Networks from First Principles **56**

5 What It Means to Learn from Data **57**
5.1 Rules versus learning . . . 57
5.2 Datasets, features and labels . . . 58
5.3 The families of machine learning . . . 59
5.4 Generalization: the real goal . . . 61
5.5 The universal recipe . . . 63
5.6 Our first model, end to end: linear regression . . . 63
5.7 A completely different learner: k-nearest neighbours . . . 67
5.8 Hand-made features versus learned representations . . . 69
5.9 Putting it together: a map of a learning project . . . 69

6 The Neuron, the Perceptron and Activation Functions **73**
6.1 From biological to artificial neurons . . . 73
6.2 The geometry of a neuron . . . 74
6.3 The perceptron . . . 75
6.4 Logistic regression: a neuron that outputs a probability . . . 78
6.5 A catalogue of activation functions . . . 79
6.6 Softmax: an activation for many classes . . . 83
6.7 Why nonlinearity is essential . . . 85

7 Layers, Multilayer Perceptrons and Forward Propagation **90**
7.1 From one neuron to a layer . . . 90
7.2 The multilayer perceptron . . . 91
7.3 Solving XOR by hand . . . 93
7.4 What can an MLP represent? Universal approximation . . . 94
7.5 How hidden layers bend and fold space . . . 96

7.6 Depth versus width . . . 96
7.7 Counting FLOPs . . . 98
7.8 Wireless application: an MLP soft demapper for 16-QAM . . . 98

8 Loss Functions and Maximum Likelihood 104
8.1 The loss as a measure of wrongness . . . 104
8.2 The maximum likelihood principle . . . 105
8.3 Regression losses . . . 106
8.4 Binary classification: binary cross-entropy . . . 108
8.5 Multi-class classification: categorical cross-entropy . . . 110
8.6 Numerically stable implementations . . . 111
8.7 Variations used in practice . . . 113
8.8 Losses in communication systems . . . 114

9 Gradient Descent and the Learning Rate 119
9.1 The foggy mountain . . . 119
9.2 A complete worked example in one dimension . . . 120
9.3 When does gradient descent converge? The stability condition . . . 122
9.4 The shape of real loss landscapes . . . 124
9.5 Batch, mini-batch and stochastic gradient descent . . . 127
9.6 Learning-rate schedules . . . 129
9.7 Wireless highlight: the LMS equalizer is stochastic gradient descent . . . 131
9.8 Putting it together: a checklist . . . 134

10 Backpropagation, Derived and Computed by Hand 138
10.1 The relay race in reverse . . . 138
10.2 Computational graphs . . . 139
10.3 The backpropagation equations for a layer . . . 140
10.4 The complete hand example . . . 142
10.5 A whole network: the backward pass as an algorithm . . . 145
10.6 Forward mode versus reverse mode . . . 147
10.7 An autograd engine in 59 lines . . . 147
10.8 Wireless: backpropagating through the channel . . . 150

11 Making Training Work: Gradients, Initialization and Optimizers 155
11.1 Vanishing and exploding gradients . . . 155
11.2 Initialization . . . 157
11.3 Optimizers . . . 160
11.4 Gradient clipping . . . 165
11.5 Residual connections: gradient highways . . . 165
11.6 Learning-rate warm-up, revisited . . . 166
11.7 Training on limited-precision hardware . . . 167

12 Generalization: Overfitting, Regularization, Normalization and Evaluation 172
12.1 Underfitting and overfitting . . . 172
12.2 The bias–variance decomposition . . . 174
12.3 Regularization . . . 175
12.4 Normalization layers . . . 181
12.5 Splitting data honestly . . . 182
12.6 Hyperparameter tuning . . . 184
12.7 Classification metrics . . . 184
12.8 Regression metrics . . . 187
12.9 Wireless metrics . . . 187

III Building Networks Yourself 192

13 Lab: A Neural Network from Scratch in NumPy 193
13.1 The plan: data in, gradients out . . . 193
13.2 The data: three interleaved spirals . . . 194
13.3 Parameters: He initialisation . . . 196
13.4 The forward pass . . . 196
13.5 The loss: stable softmax cross-entropy . . . 197
13.6 The backward pass . . . 199
13.7 Training: mini-batch SGD and prediction . . . 200
13.8 The gradient check: auditing backprop . . . 202
13.9 Training the spiral network . . . 204
13.10 Experiments: predict, then run . . . 206
13.11 Wireless mini-lab: a learned 16-QAM detector . . . 208
13.12 From this lab to frameworks . . . 212

14 Lab: PyTorch and an End-to-End Project 215
14.1 Tensors: NumPy arrays with superpowers . . . 215
14.2 Autograd: backpropagation for free . . . 216
14.3 Modules, losses and optimisers . . . 217
14.4 The canonical training loop . . . 218
14.5 The spiral network, again: PyTorch versus NumPy . . . 219
14.6 Gradient checking in PyTorch . . . 220
14.7 End-to-end project: a handwritten-digit classifier . . . 221
14.8 Saving, loading and deploying . . . 227
14.9 Reproducibility, devices and speed . . . 230
14.10 Common PyTorch bugs . . . 230
14.11 Wireless: a Dataset for complex IQ frames . . . 230

IV Deep Architectures and Reinforcement Learning **237**

15 Convolutional Neural Networks **238**

15.1 Why not just use an MLP? . . . 238
15.2 1-D convolution: the sliding dot product . . . 239
15.3 2-D convolution and multiple channels . . . 240
15.4 Padding, stride and dilation . . . 243
15.5 Pooling . . . 244
15.6 Receptive field . . . 245
15.7 Backpropagation through a convolution . . . 246
15.8 Training CNNs on real data . . . 248
15.9 Efficient convolutions: 1×1 and depthwise-separable . . . 252
15.10 Classic architectures as block diagrams . . . 252
15.11 CNNs on IQ samples . . . 253

16 Recurrent Networks: RNN, LSTM and GRU **257**

16.1 Sequence data and why we need memory . . . 257
16.2 The vanilla RNN . . . 258
16.3 Backpropagation through time (BPTT) . . . 259
16.4 Vanishing and exploding gradients . . . 260
16.5 Long Short-Term Memory (LSTM) . . . 262
16.6 The Gated Recurrent Unit (GRU) . . . 264
16.7 Bidirectional and stacked RNNs . . . 265
16.8 Experiments: watching memory fail and succeed . . . 265
16.9 The CLDNN: convolution + LSTM + dense . . . 267
16.10 Wireless experiment: predicting a fading channel . . . 268

17 Attention and Transformers **274**

17.1 The idea of attention . . . 274
17.2 Scaled dot-product attention . . . 275
17.3 Multi-head attention . . . 278
17.4 Where am I? Positional encodings . . . 279
17.5 The Transformer block . . . 281
17.6 Encoders, decoders and cross-attention . . . 282
17.7 The cost of looking at everything . . . 284
17.8 Vision Transformers . . . 285
17.9 Lab: a tiny Transformer that reverses and sorts . . . 285
17.10 Transformers in the physical layer . . . 288

18 Autoencoders, VAEs, GANs, Diffusion and Transfer Learning **293**

18.1 Autoencoders . . . 293
18.2 Variational autoencoders . . . 296

18.3 Generative adversarial networks . . . 301
18.4 Diffusion models . . . 304
18.5 Transfer learning . . . 308

19 Reinforcement Learning: From Bandits to Deep Q-Networks and Policy Gradients 315
19.1 Learning from consequences . . . 315
19.2 Multi-armed bandits: RL without state . . . 316
19.3 Markov decision processes . . . 320
19.4 Model-free learning: Monte Carlo, TD, Q-learning, SARSA . . . 324
19.5 Function approximation and Deep Q-Networks . . . 327
19.6 Partial observability: POMDPs and belief states . . . 331
19.7 Policy gradients . . . 333
19.8 A map of RL methods and their wireless uses . . . 337

V Deep Learning for Wireless Communications 342

20 A Wireless Communications Primer for Deep Learning 343
20.1 Radio waves and the communication system . . . 343
20.2 Passband and complex baseband . . . 345
20.3 Digital modulation . . . 348
20.4 The AWGN channel and optimal detection . . . 351
20.5 Real channels: path loss, shadowing and fading . . . 355
20.6 Hardware impairments . . . 363
20.7 Equalization, OFDM and channel estimation . . . 364
20.8 MIMO: multiple antennas . . . 367
20.9 Channel coding . . . 369
20.10 Medium access, spectrum and networks . . . 373
20.11 5G NR context for the practitioner . . . 375
20.12 Where each later chapter plugs in . . . 376

21 An Introduction to Deep Learning for the Physical Layer 381
21.1 The problem and why the classical approach can fall short . . . 382
21.2 Background you need . . . 382
21.3 System model: communication as an autoencoder . . . 383
21.4 Training procedure as reported . . . 386
21.5 Results reported in the paper . . . 386
21.6 Our small-scale reproduction . . . 390
21.7 Limitations and follow-up work . . . 395
21.8 From paper to practice in 5G NR and 6G . . . 396

22 Deep Architectures for Modulation Recognition 400
22.1 The problem and why classical methods fall short . . . 401

22.2 Background you need . . . 401
22.3 The RadioML 2016.10a data set . . . 402
22.4 The architectures . . . 403
22.5 Training setup as reported . . . 405
22.6 Results reported in the paper . . . 405
22.7 Our small-scale reproduction . . . 406
22.8 Limitations and follow-up work . . . 411
22.9 From paper to practice in 5G NR and 6G . . . 411

23 On Deep Learning-Based Channel Decoding 415
23.1 The problem and why it is hard . . . 416
23.2 Background you need . . . 416
23.3 System model . . . 418
23.4 The neural decoder and its training . . . 419
23.5 Results reported in the paper . . . 421
23.6 Our small-scale reproduction . . . 422
23.7 Limitations and follow-up work . . . 426
23.8 From paper to practice in 5G NR and 6G . . . 427

24 End-to-End Learning of Communications Systems Without a Channel Model 430
24.1 The problem: you cannot differentiate the air . . . 430
24.2 Background: gradients of averages you cannot differentiate . . . 432
24.3 System model and the stochastic transmitter . . . 433
24.4 The alternating training algorithm . . . 436
24.5 Architectures reported in the paper . . . 436
24.6 Results reported in the paper . . . 437
24.7 Our small-scale reproduction . . . 438
24.8 Limitations and follow-ups . . . 443
24.9 From paper to practice in 5G NR/6G . . . 443
24.10 Recap, exercises and quiz . . . 443

25 Deep Reinforcement Learning Autoencoder with Noisy Feedback 447
25.1 The problem: the feedback travels through a channel too . . . 447
25.2 Background: what noise does to a stochastic gradient . . . 448
25.3 System model: losses corrupted by Gaussian noise . . . 449
25.4 Analysis: unbiased, but noisier . . . 449
25.5 The proposed approach: learn the feedback link too . . . 451
25.6 Results reported in the paper . . . 453
25.7 Our small-scale reproduction . . . 453
25.8 Limitations and follow-ups . . . 457
25.9 From paper to practice in 5G NR/6G . . . 457
25.10 Recap, exercises and quiz . . . 458

26 Deep Learning Based MIMO Communications 461
26.1 The problem and the intuition . . . 462
26.2 Background: MIMO in one sitting . . . 462
26.3 System model of the paper . . . 464
26.4 Architectures . . . 465
26.5 Results reported in the paper . . . 465
26.6 Our small-scale reproduction . . . 466
26.7 Limitations and follow-ups . . . 470
26.8 From paper to practice in 5G NR/6G . . . 471
26.9 Recap, exercises and quiz . . . 471

27 Channel State Information Prediction for 5G: A Deep Learning Approach 474
27.1 The problem: CSI is always late . . . 475
27.2 Background: from autoregression to CNN+LSTM . . . 476
27.3 System model of OCEAN . . . 477
27.4 Results reported in the paper . . . 479
27.5 Our small-scale reproduction . . . 480
27.6 Limitations and follow-ups . . . 485
27.7 From paper to practice in 5G NR and 6G . . . 486

28 Predicting Wireless Channel Features using Neural Networks 490
28.1 The problem: features you cannot see . . . 491
28.2 Background: channel features . . . 492
28.3 System model and problem formulation . . . 493
28.4 The neural network and its training . . . 494
28.5 Results reported in the paper . . . 496
28.6 Our small-scale reproduction . . . 497
28.7 Limitations and follow-ups . . . 502
28.8 From paper to practice in 5G NR and 6G . . . 502

29 An Improved WiFi Indoor Localization Method Combining CSI and RSS 506
29.1 Indoor positioning: the landscape . . . 507
29.2 RSS: one number per access point . . . 507
29.3 CSI: a channel gain per subcarrier . . . 508
29.4 Matching: from nearest neighbours to deep networks . . . 510
29.5 Our small-scale reproduction . . . 512
29.6 Limitations and follow-ups . . . 516
29.7 From paper to practice in 5G NR and 6G . . . 516

30 Deep Reinforcement Learning for Dynamic Multichannel Access 520
30.1 The problem and the intuition . . . 521
30.2 Background you need . . . 521

30.3 System model . . . 522
30.4 Classic baselines: myopic and Whittle index . . . 523
30.5 The method: a DQN that reads the history . . . 525
30.6 Results as reported . . . 528
30.7 Our small-scale reproduction . . . 529
30.8 Limitations and follow-up work . . . 535
30.9 In practice: cognitive radio, NR-U and Wi-Fi channel selection . . . 535
30.10 Recap, exercises and quiz . . . 536

31 Online Learning in BitTorrent Systems 539
31.1 BitTorrent from zero . . . 539
31.2 The paper's formulation . . . 542
31.3 Results as reported . . . 544
31.4 Our small-scale reproduction . . . 545
31.5 Limitations and follow-ups . . . 550
31.6 Why this matters for wireless networks . . . 550
31.7 Recap, exercises and quiz . . . 551

VI Kaggle-Style Labs: Modulation Classification 554

32 Lab A: Feature-Based Modulation Classification in AWGN 555
32.1 The competition . . . 555
32.2 Two schools: likelihood versus features . . . 557
32.3 Moments and cumulants from scratch . . . 559
32.4 Why Gaussian noise vanishes above order two . . . 562
32.5 The enemy at high SNR: estimator variance . . . 563
32.6 Instantaneous and spectral features . . . 564
32.7 Feature extraction code . . . 565
32.8 Classifiers . . . 566
32.9 Results . . . 567
32.10 Kaggle notes for Lab A . . . 571

33 Lab B: Deep Learning-Based Modulation Classification in AWGN 574
33.1 The competition, recapped . . . 574
33.2 Input representations . . . 574
33.3 Three architectures . . . 575
33.4 Training . . . 576
33.5 Results . . . 578
33.6 Ablations . . . 582
33.7 Leaderboard tricks . . . 582
33.8 Inference cost . . . 583

34 Lab C: Deep Learning-Based Modulation Classification in Rayleigh Multipath Channels 586
34.1 The competition . . . 586
34.2 Multipath from first principles . . . 586
34.3 What multipath does to the signal and its features . . . 588
34.4 Experiments . . . 591
34.5 Results . . . 595
34.6 Which classes become confusable, and why . . . 595

35 Lab D: Deep Learning-Based Modulation Classification in IIR-Type Channels 600
35.1 The competition . . . 600
35.2 IIR filters from first principles . . . 600
35.3 Where do IIR-type channels arise? . . . 602
35.4 Why IIR is harder than FIR multipath . . . 603
35.5 Experiments . . . 604
35.6 Results . . . 608
35.7 Part VI finale: four labs side by side . . . 611

VII Your Own Model 615

36 Designing, Training, Debugging and Deploying Your Own Model 616
36.1 Framing the task before touching a network . . . 616
36.2 Choosing the model from the data type . . . 617
36.3 Building the dataset . . . 618
36.4 A recipe for training, with every check demonstrated . . . 622
36.5 Debugging: symptoms, causes and fixes . . . 625
36.6 Evaluation that you can trust . . . 627
36.7 Deployment . . . 629
36.8 Capstone projects . . . 633

A Equation Cheat Sheet 638
A.1 Mathematical foundations . . . 638
A.2 Neurons, losses and backpropagation . . . 639
A.3 Optimisation and regularisation . . . 641
A.4 Architectures . . . 642
A.5 Generative models and reinforcement learning . . . 643
A.6 Wireless communications . . . 644
A.7 Evaluation and deployment . . . 646

B Study Plan, Resources and Paper Index 648
B.1 A twenty-week study plan . . . 648
B.2 Books . . . 650

B.3 Courses and videos 650
B.4 Tools and datasets for wireless machine learning 651
B.5 Index of the wireless papers taught in this book 652
B.6 Landmark deep-learning papers 653
B.7 Where to go next 655

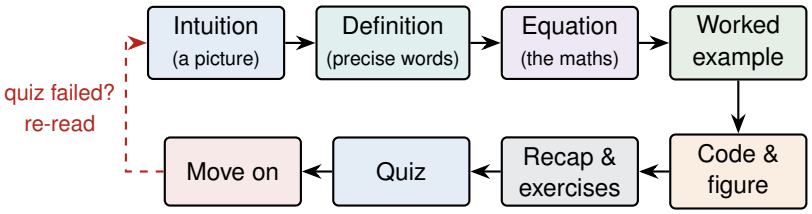
C Glossary **656**

Chapter 1

How to Use This Book

This book takes you from *nothing* — no calculus, no linear algebra, no programming in machine learning — to the point where you can (i) derive every equation inside a neural network by hand, (ii) see each concept as a picture in your head, (iii) build, train, debug and deploy your own models, and (iv) read and reproduce research papers that apply deep learning to wireless communications.

It is written to be read *offline, on paper*. Every chapter is self-contained enough that you can carry a few printed pages with you, and every idea is presented in the same order:



1.1 Who this book is for

You are curious, willing to do arithmetic with a pencil, and want to *really understand* — not just call a library. If you already know some engineering (signals, radio, embedded systems) you will move quickly through Part I and find Part V (deep learning for wireless) especially rewarding, because there the two worlds meet: the communication engineer's models of channels, codes and constellations, and the deep learning engineer's universal function approximators trained by gradient descent.

1.2 The map of the book

Part I — Mathematical foundations. Vectors, matrices, derivatives, gradients, the chain rule, probability, logarithms, entropy. Only what you need, taught from zero.

Part II — Neural networks from first principles. The neuron, activations, layers, loss functions, gradient descent, backpropagation derived and computed by hand, initialization, optimizers, regularization, normalization, metrics.

Part III — Building networks yourself. A network in pure NumPy verified by gradient checking; then PyTorch and a complete end-to-end image-classification project.

Part IV — Deep architectures & reinforcement learning. CNNs, RNN/LSTM/GRU, attention and Transformers, autoencoders, GANs, diffusion, transfer learning; Markov decision processes, Q-learning, DQN, policy gradients, bandits.

Part V — Deep learning for wireless communications. A wireless primer from zero, then one chapter per research paper: the physical layer as an autoencoder, modulation recognition, channel decoding, model-free end-to-end learning, noisy feedback, MIMO, CSI prediction, channel-feature prediction, indoor localization, dynamic spectrum access, online learning in BitTorrent.

Part VI — Kaggle-style labs: modulation classification. Feature-based (cumulants) and deep-learning classifiers in AWGN, Rayleigh multipath and IIR-type channels — with full code and real results.

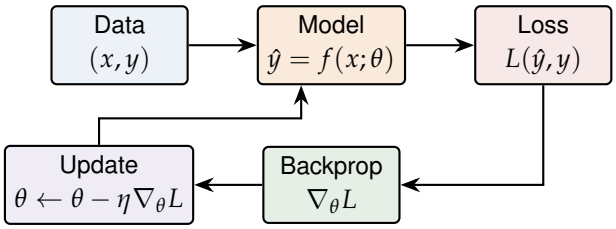
Part VII — Your own model. Designing, training, debugging and deploying a model on your own data; a debugging checklist; a wireless capstone.

1.3 How to study each chapter

1. **Read the intuition box first.** Close your eyes and try to see the picture it describes. If you cannot, read it again.
2. **Redo every worked example with a pencil.** Do not just read the numbers — produce them. Arithmetic done by your own hand is the fastest route to understanding.
3. **Run the code and change one number.** Every figure in this book was produced by a short Python script; the script is printed next to the figure (or in the chapter's lab section). Change the learning rate, the noise level or the number of neurons and predict what will happen *before* you look.
4. **Do the exercises before peeking at the answers.** Testing yourself ("retrieval practice") builds memory far better than re-reading.
5. **Pass the quiz before moving on.** If you cannot answer a quiz question without looking, go back to the section it came from.

Intuition: the whole subject in one sentence

A neural network is a stack of very simple steps — *multiply by a matrix, add a bias, bend the result with a simple nonlinear function* — repeated many times. Training means: measure how wrong the network is (the *loss*), use the chain rule to find how each knob affects that wrongness (*backpropagation*), and turn every knob a tiny bit in the direction that reduces it (*gradient descent*). Everything else in this book is a refinement of that loop.



1.4 Software setup

All code is Python 3.10 or newer. We use **NumPy** first, because it hides nothing, then **PyTorch**, the framework used by most research and courses. Figures use **matplotlib**; a few labs use **scikit-learn** and **SciPy**.

```
pip install numpy scipy matplotlib scikit-learn torch torchvision pillow
```

A first test that everything works:

```
import numpy as np, matplotlib.pyplot as plt
x = np.linspace(-5, 5, 200)
plt.plot(x, x**2); plt.title("My first plot: y = x^2"); plt.grid(True); plt.show()
```

Pitfall: printed code

Code printed in a book can pick up typographical characters (curly quotes, long dashes). If a copied line fails with a strange syntax error, retype the quotes and minus signs by hand. Every script in this book is short enough to type.

1.5 Notation used throughout

Symbol	Meaning
x, $\mathbf{x}$	a single number (scalar); a vector (bold, lower case)
$\mathbf{X}$, $\mathbf{W}$	a matrix (bold, upper case). A batch of N inputs with d features is $\mathbf{X} \in \mathbb{R}^{N \times d}$
$\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{b}$	one layer, "batch-first" row convention (same as the code). $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$
f, σ, ReLU	activation functions; σ is the sigmoid
$\hat{y}$, y	prediction ("y-hat") and true label
L, $\mathcal{L}$	loss; $\nabla_\theta L$ its gradient with respect to the parameters θ
η	learning rate (Greek "eta")
$\odot$	element-wise (Hadamard) multiplication
$\mathbb{E}[\cdot]$	expectation (average under a probability distribution)
$\mathcal{N}(\mu, \sigma^2)$, $\mathcal{CN}(0, \sigma^2)$	real Gaussian; circularly-symmetric complex Gaussian
$\mathbf{A}^{\mathsf{T}}$, $\mathbf{A}^{\mathsf{H}}$	transpose; conjugate (Hermitian) transpose
SNR, E_b/N_0	signal-to-noise ratio; energy per bit to noise density (Part V)

1.6 Suggested schedules

At 8–10 hours per week the whole book takes about 18–20 weeks; full-time readers can halve that. A reader with an engineering background can compress Part I into one week.

Weeks	Chapters	Deliverable
1–2	Part I (Ch. 2–4)	matrix products and chain-rule exercises by hand
3–5	Part II (Ch. 5–12)	backprop of a 2-2-1 network on paper
6–7	Part III (Ch. 13–14)	NumPy net passing a gradient check; PyTorch classifier
8–11	Part IV (Ch. 15–19)	CNN, LSTM, tiny Transformer, a DQN agent
12–16	Part V (Ch. 20–31)	reproduce one figure from three of the papers
17–18	Part VI (Ch. 32–35)	modulation classifier robust to multipath
19–20	Part VII (Ch. 36)	your own capstone model, exported and deployed

Chapter recap

- Each chapter goes intuition → definition → equation → worked example → code/figure → recap → exercises → quiz.
- Do the arithmetic yourself; run the code; change one number at a time.
- The core loop of deep learning: forward pass, loss, backpropagation, update. Everything else refines it.

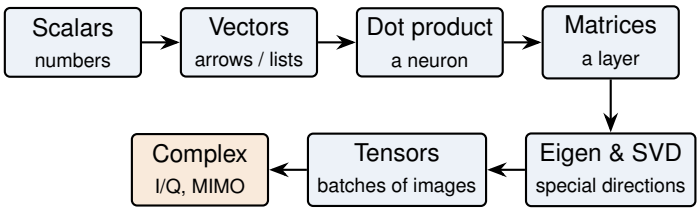
Part I

Mathematical Foundations

Chapter 2

Vectors, Matrices and Linear Maps

A neural network, stripped of its mystique, is a long sequence of two operations: *multiply a list of numbers by a table of numbers*, then *bend the result*. The "list" is a vector, the "table" is a matrix, and the branch of mathematics that studies them is called **linear algebra**. In this chapter you will learn it from nothing. By the end you will be able to multiply matrices by hand, see a matrix as a machine that stretches and rotates space, understand why some directions are special (eigenvectors), and decompose any matrix into rotate–stretch–rotate (the SVD). These ideas reappear everywhere: every layer of a network is a matrix product; vanishing and exploding gradients in deep and recurrent networks are explained by eigenvalues; and in wireless, the received signal of a multi-antenna (MIMO) link is literally $\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}$, a matrix–vector product plus noise, whose best use is found by the SVD.



2.1 Scalars and vectors

2.1.1 Scalars

A **scalar** is just a single ordinary number: 3, -0.5, π. We write scalars in plain italic letters such as x, a, λ. The set of all real numbers is written $\mathbb{R}$, so "$x \in \mathbb{R}$" reads "x is a real number".

2.1.2 Vectors: two pictures of the same thing

Intuition: a vector is an arrow and a list at the same time

Stand at a crossroads. "Walk 3 blocks east and 2 blocks north" is an instruction with a *direction* and a *length*: draw it as an arrow from where you stand to where you end up. The same instruction can be written as a list of two numbers, $[3, 2]$. These are the two pictures of a vector: a geometric **arrow** (good for intuition) and an ordered **list of numbers** (good for computing). A grayscale image of 28×28 pixels is also a vector: a list of 784 brightness values, i.e. an arrow in a 784-dimensional space. We cannot draw that space, but every rule we learn in 2-D works there unchanged.

Definition: vector

A **vector** with n components is an ordered list of n real numbers, written in bold:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n.$$

x_i ("x sub i") is the i-th **component**; n is the **dimension**. By convention vectors are *columns*; to save space we often write $\mathbf{x} = [x_1, \ldots, x_n]^\mathsf{T}$, where the little $^\mathsf{T}$ ("transpose") turns the row into a column.

2.1.3 Addition and scaling

Two vectors of the same size are added *component by component*, and multiplying by a scalar multiplies *every* component:

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 \\ -1 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}, \qquad 2.5 \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2.5 \\ 5 \end{bmatrix}.$$

Geometrically, adding is "walk the first arrow, then the second" (tip-to-tail), and scaling by c stretches the arrow by the factor c (and flips it if $c < 0$). Subtraction $\mathbf{a} - \mathbf{b}$ gives the arrow pointing *from* the tip of $\mathbf{b}$ *to* the tip of $\mathbf{a}$ — this is how we measure the error between a prediction and a target.

2.1.4 Length: the norm

How long is the arrow $[3, 4]$? It is the hypotenuse of a right triangle with sides 3 and 4, so by Pythagoras its length is $\sqrt{3^2 + 4^2} = \sqrt{25} = 5$. In n dimensions we simply keep adding squares.

Definition: Euclidean norm

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2} = \sqrt{\textstyle\sum_{i=1}^{n} x_i^2}. \tag{2.1}$$

The symbol $\sum_{i=1}^{n}$ (capital Greek "sigma") means "add up the following expression for $i = 1, 2, \ldots, n$". A vector of length 1 is a **unit vector**; dividing any nonzero vector by its norm, $\mathbf{x} / \|\mathbf{x}\|$, gives the unit vector pointing the same way ("normalising").

Worked example: the norm of $[1, 2, 3]$

$\|[1, 2, 3]\| = \sqrt{1^2 + 2^2 + 3^2} = \sqrt{1 + 4 + 9} = \sqrt{14} \approx 3.742$. The unit vector in that direction is $[1, 2, 3]/3.742 \approx [0.267,\ 0.535,\ 0.802]$; check: $0.267^2 + 0.535^2 + 0.802^2 \approx 1$.

Other norms exist. The **L1 norm** $\|\mathbf{x}\|_1 = \sum_i |x_i|$ ("Manhattan distance": walking along the street grid) gives $1 + 2 + 3 = 6$ for the vector above. It reappears in Part II as L1 regularization. When we write $\|\cdot\|$ without a subscript we always mean the Euclidean (L2) norm. Note also that $\|\mathbf{x}\|^2 = \sum_i x_i^2$ — the squared norm is the *energy* of a signal and the *sum of squared errors* of a prediction.

2.2 The dot product

2.2.1 Algebraic definition

Definition: dot product

For two vectors of the same dimension,

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^{\mathsf{T}}\mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n = \sum_{i=1}^{n} a_i b_i. \tag{2.2}$$

Multiply matching components, then add. The answer is a single scalar.

Worked example: $[1,2,3] \cdot [4,5,6]$

$1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 4 + 10 + 18 = 32.$

Notice that $\mathbf{a} \cdot \mathbf{a} = \sum_i a_i^2 = \|\mathbf{a}\|^2$: the dot product of a vector with itself is its squared length.

2.2.2 Geometric meaning: how much two arrows agree

Intuition: the dot product measures alignment

Push a shopping trolley with force $\mathbf{a}$ while it rolls along direction $\mathbf{b}$. Only the part of your push that lies *along* $\mathbf{b}$ does useful work; the sideways part is wasted. The dot product is exactly "(useful part of $\mathbf{a}$ along $\mathbf{b}$) $\times$ (length of $\mathbf{b}$)". It is large and positive when the arrows point the same way, zero when they are perpendicular, and negative when they point in opposite directions.

Key equation: geometric dot product

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos\theta, \tag{2.3}$$

where θ is the angle between the two arrows.

Derivation. Why do the algebraic formula (2.2) and the geometric formula (2.3) agree? Draw the triangle formed by $\mathbf{a}$, $\mathbf{b}$ and the third side $\mathbf{a} - \mathbf{b}$. The *law of cosines* from school trigonometry says, for a triangle with sides of lengths A, B, C and angle θ between sides A and B: $C^2 = A^2 + B^2 - 2AB\cos\theta$. Here $A = \|\mathbf{a}\|$, $B = \|\mathbf{b}\|$, $C = \|\mathbf{a} - \mathbf{b}\|$, so

$$\|\mathbf{a} - \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\,\|\mathbf{a}\|\,\|\mathbf{b}\|\cos\theta.$$

Now expand the left side using components:

$$\|\mathbf{a} - \mathbf{b}\|^2 = \sum_i (a_i - b_i)^2 = \sum_i \left(a_i^2 - 2a_i b_i + b_i^2\right) = \|\mathbf{a}\|^2 - 2\sum_i a_i b_i + \|\mathbf{b}\|^2.$$

Comparing the two lines, the $\|\mathbf{a}\|^2$ and $\|\mathbf{b}\|^2$ terms cancel, leaving $-2\sum_i a_i b_i = -2\,\|\mathbf{a}\|\,\|\mathbf{b}\|\cos\theta$, i.e. $\sum_i a_i b_i = \|\mathbf{a}\|\,\|\mathbf{b}\|\cos\theta$. □

2.2.3 Cosine similarity and projection

Dividing (2.3) by the two lengths isolates the angle:

$$\cos\theta = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\,\|\mathbf{b}\|} \in [-1,1]. \tag{2.4}$$

This number is the **cosine similarity**. It ignores length and keeps only direction: $+1$ means "same direction", 0 "unrelated" (perpendicular, also called **orthogonal**), -1 "opposite". Search engines, recommendation systems and word embeddings and attention (Part IV) compare vectors this way.

Worked example: cosine similarity of $[1,2,3]$ and $[4,5,6]$

$\|[4,5,6]\| = \sqrt{16+25+36} = \sqrt{77} \approx 8.775$. So $\cos\theta = 32/(3.742 \times 8.775) = 32/32.83 \approx 0.975$, and $\theta = \arccos(0.975) \approx 12.9°$. The two vectors point in almost the same direction.

The **projection** of $\mathbf{a}$ onto $\mathbf{b}$ is the "shadow" that $\mathbf{a}$ casts on the line through $\mathbf{b}$ when light shines perpendicular to that line. Its signed length is $\|\mathbf{a}\|\cos\theta = \mathbf{a}\cdot\mathbf{b}/\|\mathbf{b}\|$, and the shadow vector itself is that length times the unit vector $\mathbf{b}/\|\mathbf{b}\|$:

$$\text{proj}_{\mathbf{b}}\,\mathbf{a} = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{b}\|^2}\,\mathbf{b}. \tag{2.5}$$

Figure 2.1 shows this with $\mathbf{a} = [1,3]$, $\mathbf{b} = [4,1]$: $\mathbf{a}\cdot\mathbf{b} = 7$, $\|\mathbf{b}\|^2 = 17$, so the shadow is $\frac{7}{17}[4,1] \approx [1.65, 0.41]$ of length $7/\sqrt{17} \approx 1.70$.

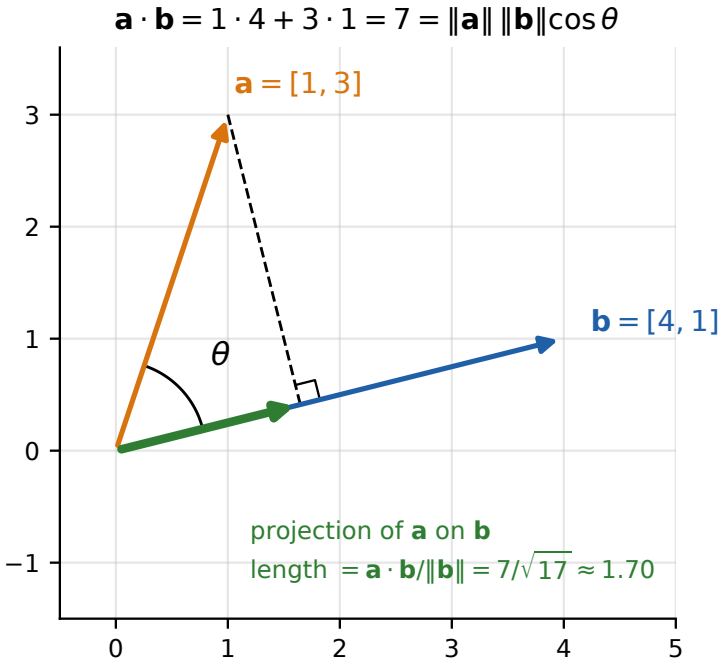
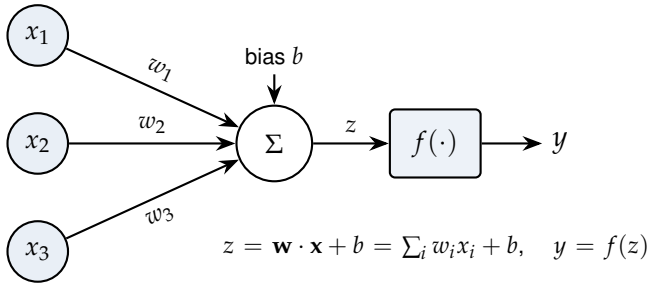


Figure 2.1: The dot product as a projection. The green arrow is the shadow of $\mathbf{a}$ on the line of $\mathbf{b}$; the dashed line meets that line at a right angle. The dot product equals (length of the shadow) $\times$ (length of $\mathbf{b}$). If you rotate $\mathbf{a}$ until it is perpendicular to $\mathbf{b}$ the shadow shrinks to zero, and so does the dot product.

2.2.4 A neuron is a dot product

Intuition: a neuron is a template detector

A neuron holds a weight vector $\mathbf{w}$ and receives an input $\mathbf{x}$. It computes $z = \mathbf{w} \cdot \mathbf{x} + b$ and then applies a simple bending function. Because of (2.3), $\mathbf{w} \cdot \mathbf{x}$ is large when the input points in the same direction as the weights: the weight vector is a *template*, and the neuron fires when the input "looks like" the template. Training a network means sculpting these templates.



Worked example: one neuron by hand

Let $\mathbf{w} = [0.5, -1, 2]$, $b = 0.5$, $\mathbf{x} = [1, 2, 3]$. Then $z = 0.5 \cdot 1 + (-1) \cdot 2 + 2 \cdot 3 + 0.5 = 0.5 - 2 + 6 + 0.5 = 5$. With the ReLU bending function $f(z) = \max(0, z)$ the output is $y = 5$.

Wireless connection: matched filters and correlators are dot products

A 5G receiver searching for the primary synchronisation signal (PSS) slides a known sequence $\mathbf{s}$ along the received samples and computes $\left|\mathbf{s}^{\mathsf{H}}\mathbf{r}\right|$ at every offset. This *correlator* is a (complex) dot product: it peaks where the received window is most aligned with the template. A neuron in the first layer of a network trained on raw I/Q samples learns exactly such templates — a learned matched filter. Cosine similarity is the normalised correlation coefficient you already use to set detection thresholds independent of signal power.

2.3 Matrices: machines that transform space

Definition: matrix

A **matrix** $\mathbf{A} \in \mathbb{R}^{m \times n}$ is a rectangular table of numbers with m rows and n columns. Its entry in row i, column j is a_{ij} (or A_{ij}). The pair (m, n) is its **shape** — always "rows first".

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \in \mathbb{R}^{2 \times 3}, \qquad a_{12} = 2,\ a_{21} = 4.$$

2.3.1 Matrix times vector

Multiplying a matrix by a vector, $\mathbf{y} = \mathbf{A}\mathbf{x}$, means: *take the dot product of each row of* $\mathbf{A}$ *with* $\mathbf{x}$. Row i produces component y_i:

$$y_i = \sum_{j=1}^{n} a_{ij} x_j, \qquad i = 1, \ldots, m. \tag{2.6}$$

So $\mathbf{A} \in \mathbb{R}^{m\times n}$ eats a vector with n components and outputs one with m components. A layer of m neurons, each a dot product with its own weight row, is exactly one matrix–vector product.

Worked example: matrix times vector

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 1\cdot 1 + 2\cdot 0 + 3\cdot(-1) \\ 4\cdot 1 + 5\cdot 0 + 6\cdot(-1) \end{bmatrix} = \begin{bmatrix} -2 \\ -2 \end{bmatrix}.$$

There is a second, equally important reading of (2.6): $\mathbf{Ax} = x_1(\text{column } 1) + x_2(\text{column } 2) + \ldots$, a *weighted sum of the columns*. Check with the example: $1\cdot[1,4] + 0\cdot[2,5] + (-1)\cdot[3,6] = [-2,-2]$. This "column picture" leads to the geometric view.

2.3.2 The geometric view

Intuition: a matrix is a machine that moves every point of space

Draw a sheet of graph paper. Feed every point into the machine $\mathbf{x} \mapsto \mathbf{Ax}$ and plot where it lands. Straight lines stay straight, the origin stays put, and evenly spaced grid lines stay evenly spaced — that is what "linear" means. The grid is stretched, sheared, rotated or flipped as a whole. And you only need to know where the two basic arrows $\mathbf{e}_1 = [1,0]$ and $\mathbf{e}_2 = [0,1]$ land: by the column picture, $\mathbf{Ae}_1$ is the first column and $\mathbf{Ae}_2$ the second. *The columns of a matrix are where the basis arrows go.*

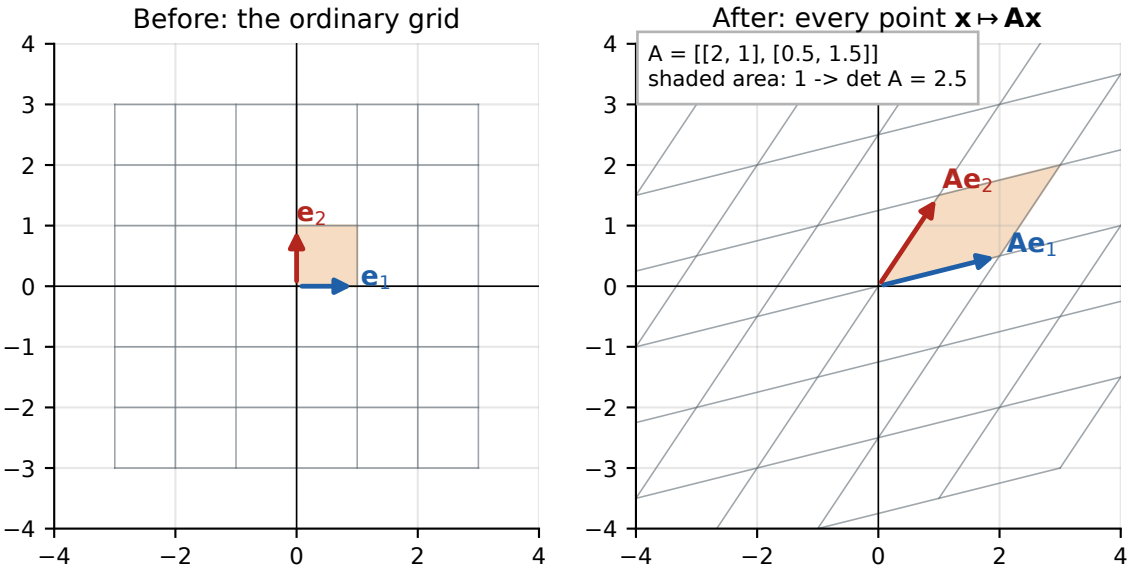


Figure 2.2: A matrix transforms the whole plane. Left: the ordinary grid with basis arrows $\mathbf{e}_1$ (blue) and $\mathbf{e}_2$ (red) and the unit square (orange). Right: after applying $\mathbf{A} = \left[\begin{smallmatrix} 2 & 1 \\ 0.5 & 1.5 \end{smallmatrix}\right]$. The blue arrow lands on the first column $[2, 0.5]$, the red on the second column $[1, 1.5]$. Grid lines remain straight, parallel and evenly spaced; the unit square becomes a parallelogram of area 2.5, which is the determinant of $\mathbf{A}$ (Section 2.6).

A few famous 2×2 machines: $\left[\begin{smallmatrix} c & 0 \\ 0 & c \end{smallmatrix}\right]$ scales everything by c; $\left[\begin{smallmatrix} \cos\varphi & -\sin\varphi \\ \sin\varphi & \cos\varphi \end{smallmatrix}\right]$ rotates by angle φ; $\left[\begin{smallmatrix} 1 & k \\ 0 & 1 \end{smallmatrix}\right]$ shears (slides the top of the square sideways); $\left[\begin{smallmatrix} 1 & 0 \\ 0 & -1 \end{smallmatrix}\right]$ reflects in the horizontal axis.

Definition: linear map

A function T from vectors to vectors is **linear** if $T(\mathbf{u}+\mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$ and $T(c\mathbf{u}) = cT(\mathbf{u})$ for all vectors and scalars. Every matrix defines a linear map, and every linear map between $\mathbb{R}^n$ and $\mathbb{R}^m$ is a matrix.

Pitfall: linear layers alone are not enough

Applying two matrices in a row, $\mathbf{B}(\mathbf{A}\mathbf{x})$, is again a single linear map (the matrix $\mathbf{BA}$). So a "deep" network made only of matrix multiplications collapses to one matrix and can only draw straight decision boundaries. The nonlinear bending functions between layers (Part II) are what make depth useful.

2.4 Matrix multiplication

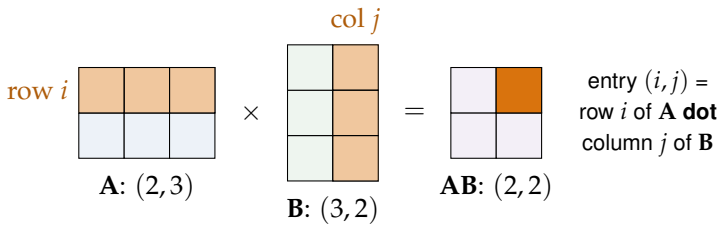
2.4.1 Row-by-column

To multiply $\mathbf{A} \in \mathbb{R}^{m\times n}$ by $\mathbf{B} \in \mathbb{R}^{n\times p}$, treat each column of $\mathbf{B}$ as a vector and multiply it by $\mathbf{A}$. Equivalently, entry (i, j) of the product is the dot product of *row i of* $\mathbf{A}$ with *column j of* $\mathbf{B}$:

Key equation: matrix product

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{n} a_{ik}\, b_{kj}, \qquad \underbrace{(m,n)}_{\mathbf{A}} \times \underbrace{(n,p)}_{\mathbf{B}} \to \underbrace{(m,p)}_{\mathbf{AB}}. \tag{2.7}$$

The **shape rule**: the *inner* dimensions must match (both n, because a row of $\mathbf{A}$ and a column of $\mathbf{B}$ must have the same length to be dotted) and they disappear; the *outer* dimensions give the shape of the result. Checking shapes is the single most useful debugging habit in deep learning.



Worked example: a 2×3 times a 3×2

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}.$$

Shapes: $(2,3) \times (3,2) \to (2,2)$. The four entries:

$$\begin{aligned} (1,1) &: 1\cdot 7 + 2\cdot 9 + 3\cdot 11 = 7 + 18 + 33 = 58, \\ (1,2) &: 1\cdot 8 + 2\cdot 10 + 3\cdot 12 = 8 + 20 + 36 = 64, \\ (2,1) &: 4\cdot 7 + 5\cdot 9 + 6\cdot 11 = 28 + 45 + 66 = 139, \\ (2,2) &: 4\cdot 8 + 5\cdot 10 + 6\cdot 12 = 32 + 50 + 72 = 154. \end{aligned}$$

So $\mathbf{AB} = \begin{bmatrix} 58 & 64 \\ 139 & 154 \end{bmatrix}$.

```python
import numpy as np
A = np.array([[1, 2, 3], [4, 5, 6]])
B = np.array([[7, 8], [9, 10], [11, 12]])
print(A @ B)          # [[ 58  64] [139 154]]   shape (2, 2)
print((B @ A).shape)  # (3, 3) -- a completely different object!
```

2.4.2 Order matters: $\mathbf{AB} \neq \mathbf{BA}$

With numbers, $3 \times 5 = 5 \times 3$. With matrices this fails. In the example above, $\mathbf{BA}$ has shape $(3,2) \times (2,3) \to (3,3)$, so it is not even the same size as $\mathbf{AB}$. Even for square matrices of the same size:

$$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 2 \\ 1 & 1 \end{bmatrix}, \qquad \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 1 & 3 \end{bmatrix}.$$

Geometrically, $\mathbf{ABx} = \mathbf{A}(\mathbf{Bx})$ means "first apply $\mathbf{B}$, then $\mathbf{A}$" (read right to left, like function composition). Putting on socks then shoes is not the same as shoes then socks. Matrix multiplication *is* associative, $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$, and distributive, $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$.

2.4.3 A whole layer for a whole batch

In code we process N examples at once. Stack them as the rows of $\mathbf{X} \in \mathbb{R}^{N \times d}$. A layer with d inputs and h neurons stores $\mathbf{W} \in \mathbb{R}^{d \times h}$ (column j = weights of neuron j) and computes

$$\mathbf{Z} = \mathbf{XW} + \mathbf{b}, \qquad (N,d) \times (d,h) \to (N,h).$$

Row n of $\mathbf{Z}$ holds the h neuron outputs for example n. One matrix product computes $N \times h$ dot products — which is why GPUs, built for exactly this, make deep learning fast.

2.5 Transpose, identity, inverse

2.5.1 Transpose

The **transpose** $\mathbf{A}^\mathsf{T}$ flips a matrix over its main diagonal: rows become columns, $(\mathbf{A}^\mathsf{T})_{ij} = a_{ji}$, and the shape (m,n) becomes (n,m). A column vector transposed is a row vector, which is why $\mathbf{a}^\mathsf{T}\mathbf{b}$ (a $1 \times n$ times an $n \times 1$) is the 1×1 dot product. A matrix with $\mathbf{S}^\mathsf{T} = \mathbf{S}$ is **symmetric**.

Key equation: transpose of a product reverses the order

$$(\mathbf{AB})^\mathsf{T} = \mathbf{B}^\mathsf{T}\mathbf{A}^\mathsf{T}. \tag{2.8}$$

Derivation with indices. Entry (i,j) of the left side is entry (j,i) of $\mathbf{AB}$: $\sum_k a_{jk} b_{ki}$. Entry (i,j) of the right side is row i of $\mathbf{B}^\mathsf{T}$ dotted with column j of $\mathbf{A}^\mathsf{T}$. Row i of $\mathbf{B}^\mathsf{T}$ is column i of $\mathbf{B}$, with entries b_{ki}; column j of $\mathbf{A}^\mathsf{T}$ is row j of $\mathbf{A}$, with entries a_{jk}. Their dot product is $\sum_k b_{ki} a_{jk}$, the same sum. $\square$ (Shapes confirm it: if $\mathbf{A}$ is (m,n) and $\mathbf{B}$ is (n,p), then $\mathbf{B}^\mathsf{T}\mathbf{A}^\mathsf{T}$ is $(p,n)(n,m) \to (p,m)$, the shape of $(\mathbf{AB})^\mathsf{T}$; the other order would not even fit.) This rule is used constantly in backpropagation (Chapter 3 and Part II).

2.5.2 Identity and inverse

The **identity matrix I** has ones on the diagonal and zeros elsewhere; it is the "do nothing" machine: $\mathbf{Ix} = \mathbf{x}$ and $\mathbf{IA} = \mathbf{AI} = \mathbf{A}$. The **inverse** $\mathbf{A}^{-1}$ of a square matrix is the machine that undoes $\mathbf{A}$: $\mathbf{A}^{-1}\mathbf{A} = \mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$. If $\mathbf{Ax} = \mathbf{y}$ then $\mathbf{x} = \mathbf{A}^{-1}\mathbf{y}$. For a 2×2 matrix there is a closed formula:

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \implies \mathbf{A}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}. \tag{2.9}$$

Verification: $\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} = \begin{bmatrix} ad - bc & -ab + ba \\ cd - dc & -cb + da \end{bmatrix} = (ad - bc)\mathbf{I}$, so dividing by $ad - bc$ gives $\mathbf{I}$.

Worked example: inverting a 2×2

$\mathbf{A} = \left[\begin{smallmatrix} 4 & 7 \\ 2 & 6 \end{smallmatrix}\right]$: $ad - bc = 24 - 14 = 10$, so $\mathbf{A}^{-1} = \frac{1}{10}\left[\begin{smallmatrix} 6 & -7 \\ -2 & 4 \end{smallmatrix}\right] = \left[\begin{smallmatrix} 0.6 & -0.7 \\ -0.2 & 0.4 \end{smallmatrix}\right]$. Check one entry of $\mathbf{A}\mathbf{A}^{-1}$: row 1 times column $1 = 4(0.6) + 7(-0.2) = 2.4 - 1.4 = 1$. ✓

The number $ad - bc$ in the denominator is the determinant. When it is zero, no inverse exists.

2.6 The determinant: how much area is scaled

Intuition: determinant = area magnification

Look again at Fig. 2.2: the unit square (area 1) became a parallelogram. Every little square of the grid is scaled by the *same* factor, because the map is linear. That factor is the **determinant** $\det \mathbf{A}$. If it is 2.5 the matrix multiplies all areas by 2.5. If it is negative, the plane has also been flipped (like a mirror image). If it is *zero*, the plane has been squashed onto a line or a point — information is destroyed and cannot be recovered, so there is no inverse.

For a 2×2 matrix, $\det \left[\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right] = ad - bc$. *Why area?* The columns $[a, c]$ and $[b, d]$ span the parallelogram. Take the case $a, b, c, d > 0$ with $[a, c]$ below $[b, d]$; enclose the parallelogram in the rectangle of width $a + b$ and height $c + d$; the corner pieces are two triangles of area $\frac{1}{2}ac$, two of area $\frac{1}{2}bd$ and two small rectangles of area bc. Subtracting: $(a + b)(c + d) - ac - bd - 2bc = ad - bc$. For Fig. 2.2: $2 \cdot 1.5 - 1 \cdot 0.5 = 2.5$. Two useful facts: $\det(\mathbf{AB}) = \det \mathbf{A} \cdot \det \mathbf{B}$ (scale by one factor then the other), and $\det \mathbf{A}$ equals the product of the eigenvalues (Section 2.8). In n dimensions the determinant scales *volumes*.

2.7 Linear independence and rank

Intuition: redundant arrows

If you are given the arrows $[1, 2]$ and $[2, 4]$, the second is just twice the first: it points the same way and adds no new direction. Combining them ($c_1[1, 2] + c_2[2, 4]$) can only ever reach points on one line. The arrows $[1, 2]$ and $[1, 0]$ point in genuinely different directions, and combining them reaches every point of the plane.

Definition: independence, span, rank

Vectors $\mathbf{v}_1, \ldots, \mathbf{v}_k$ are **linearly independent** if none of them can be written as a weighted sum of the others; equivalently, $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$ only when all $c_i = 0$. The set of all weighted sums is their **span**. The **rank** of a matrix is the number of independent columns (which always equals the number of independent rows) — the dimension of the space its outputs can reach.

$\left[\begin{smallmatrix}1 & 2\\ 2 & 4\end{smallmatrix}\right]$ has rank 1 (its determinant $4 - 4 = 0$ confirms the squash onto a line). A square $n \times n$ matrix is invertible exactly when its rank is n ("full rank"), exactly when $\det \neq 0$. Rank matters in deep learning (low-rank adapters such as LoRA approximate a big weight update by a product of two thin matrices) and enormously in wireless: the rank of a MIMO channel matrix is the number of independent data streams it can carry.

2.8 Eigenvalues and eigenvectors

Intuition: the directions a matrix only stretches

Watch Fig. 2.3. Most arrows are *turned* by the matrix as well as stretched. But a few special directions come out pointing exactly where they went in — only longer, shorter or reversed. These are the **eigenvectors** (German *eigen* = "own"), and the stretch factor along each is its **eigenvalue**. Along an eigenvector a complicated matrix behaves like a plain number.

Definition: eigenvector, eigenvalue

A nonzero vector $\mathbf{v}$ is an eigenvector of the square matrix $\mathbf{A}$, with eigenvalue λ (Greek "lambda"), if

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}. \tag{2.10}$$

How to find them. Rewrite (2.10) as $\mathbf{A}\mathbf{v} - \lambda\mathbf{I}\mathbf{v} = \mathbf{0}$, i.e. $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$. We want a *nonzero* $\mathbf{v}$ that the matrix $\mathbf{A} - \lambda\mathbf{I}$ squashes to zero. That is only possible if $\mathbf{A} - \lambda\mathbf{I}$ destroys information, i.e. its determinant is zero:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0 \quad \text{(the characteristic equation)}. \tag{2.11}$$

Solve it for λ, then for each λ solve $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$ for $\mathbf{v}$.

Worked example: eigen-decomposition of a 2×2

$\mathbf{S} = \begin{bmatrix}2 & 1\\ 1 & 2\end{bmatrix}$. Then $\mathbf{S} - \lambda\mathbf{I} = \begin{bmatrix}2-\lambda & 1\\ 1 & 2-\lambda\end{bmatrix}$ and

$$\det(\mathbf{S} - \lambda\mathbf{I}) = (2-\lambda)^2 - 1\cdot 1 = \lambda^2 - 4\lambda + 3 = (\lambda - 3)(\lambda - 1) = 0,$$

so $\lambda_1 = 3$, $\lambda_2 = 1$.
For $\lambda_1 = 3$: $\left[\begin{smallmatrix}-1 & 1\\ 1 & -1\end{smallmatrix}\right]\mathbf{v} = \mathbf{0}$ gives $-v_1 + v_2 = 0$, so $v_1 = v_2$: $\mathbf{v}_1 = [1, 1]$. Check: $\mathbf{S}[1, 1]^\mathsf{T} = [3, 3]^\mathsf{T} = 3[1, 1]^\mathsf{T}$. ✓
For $\lambda_2 = 1$: $\left[\begin{smallmatrix}1 & 1\\ 1 & 1\end{smallmatrix}\right]\mathbf{v} = \mathbf{0}$ gives $v_1 = -v_2$: $\mathbf{v}_2 = [1, -1]$. Check: $\mathbf{S}[1, -1]^\mathsf{T} = [1, -1]^\mathsf{T}$. ✓
Note $\lambda_1\lambda_2 = 3 = \det\mathbf{S} = 4 - 1$, and $\lambda_1 + \lambda_2 = 4 =$ sum of diagonal entries (the *trace*).
Because $\mathbf{S}$ is symmetric, its eigenvectors are perpendicular: $[1, 1] \cdot [1, -1] = 0$.

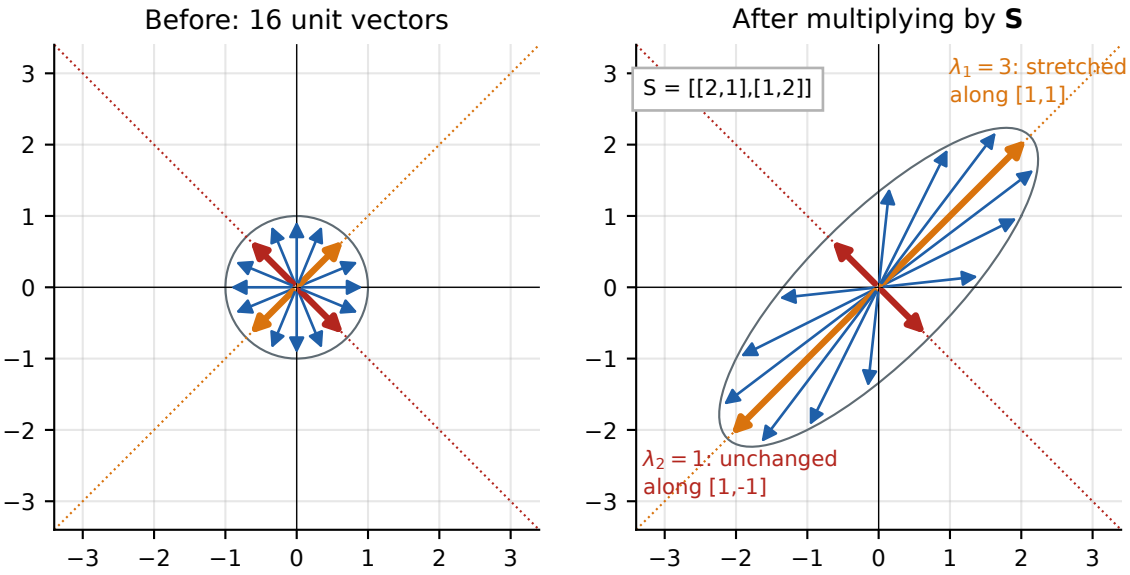


Figure 2.3: Sixteen unit arrows before (left) and after (right) multiplication by $\mathbf{S} = \left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$. The circle becomes an ellipse. Almost every arrow is rotated towards the $[1, 1]$ diagonal — except the thick orange arrows along $[1, 1]$ (stretched by $\lambda_1 = 3$, still on their dotted line) and the thick red arrows along $[1, -1]$ (left unchanged, $\lambda_2 = 1$). The ellipse's axes are the eigenvector directions and its semi-axis lengths are the eigenvalues.

2.8.1 Why eigenvalues matter: repeated multiplication

If $\mathbf{A}$ has independent eigenvectors, any vector can be written as $\mathbf{x} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2$. Applying $\mathbf{A}$ k times then gives, by linearity and (2.10) used k times,

$$\mathbf{A}^k\mathbf{x} = c_1\lambda_1^k\mathbf{v}_1 + c_2\lambda_2^k\mathbf{v}_2. \tag{2.12}$$

Components along eigenvalues with $|\lambda| > 1$ *explode*; those with $|\lambda| < 1$ *vanish*. Even mild values compound brutally: $0.9^{50} \approx 0.005$ while $1.1^{50} \approx 117$.

Pitfall: vanishing and exploding gradients

In a deep network, and especially in a recurrent network that applies the *same* weight matrix at each of T time steps, the backward-flowing gradient is multiplied by (roughly) the same kind of matrix again and again. By (2.12) it shrinks to nothing or blows up depending on whether the relevant eigenvalues (more precisely, singular values) are below or above 1. This is the root cause analysed in Part II (weight initialisation) and Part IV (why LSTMs were invented).

For a **symmetric** matrix, the eigenvectors can always be chosen perpendicular and of unit length, and stacking them as columns of $\mathbf{Q}$ gives $\mathbf{S} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\mathsf{T}$ with $\mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ (the **spectral theorem**). Covariance matrices (Chapter 4) and Hessians (Chapter 3) are symmetric, so this decomposition is everywhere.

2.9 The singular value decomposition

Eigenvectors need a square matrix, and a non-symmetric matrix may turn every direction. The SVD works for *every* matrix, of any shape.

Intuition: every matrix is rotate, stretch, rotate

Any linear map, however messy, does three simple things in sequence: (1) **rotate** (or reflect) the input space so that certain special input directions line up with the axes; (2) **stretch** along each axis by a non-negative factor (some may be zero); (3) **rotate** the result into position in the output space. A circle therefore always becomes an ellipse, and the SVD tells you its axes.

Key equation: singular value decomposition

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\mathsf{T}}, \tag{2.13}$$

where $\mathbf{U}$ ($m \times m$) and $\mathbf{V}$ ($n \times n$) are **orthogonal** (columns perpendicular unit vectors, $\mathbf{U}^{\mathsf{T}}\mathbf{U} = \mathbf{I}$: a pure rotation/reflection), and $\boldsymbol{\Sigma}$ ($m \times n$) is zero except for the **singular values** $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ on its diagonal.

Where the pieces come from. Compute

$$\mathbf{A}^{\mathsf{T}}\mathbf{A} = (\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\mathsf{T}})^{\mathsf{T}}(\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\mathsf{T}}) = \mathbf{V}\boldsymbol{\Sigma}^{\mathsf{T}}\mathbf{U}^{\mathsf{T}}\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\mathsf{T}} = \mathbf{V}(\boldsymbol{\Sigma}^{\mathsf{T}}\boldsymbol{\Sigma})\mathbf{V}^{\mathsf{T}},$$

using (2.8) and $\mathbf{U}^{\mathsf{T}}\mathbf{U} = \mathbf{I}$. This is the spectral decomposition of the symmetric matrix $\mathbf{A}^{\mathsf{T}}\mathbf{A}$: so the columns of $\mathbf{V}$ are its eigenvectors and σ_i^2 are its eigenvalues. Then $\mathbf{u}_i = \mathbf{A}\mathbf{v}_i/\sigma_i$.

Worked example: SVD of a 2×2 by hand

$\mathbf{A} = \left[\begin{smallmatrix} 3 & 0 \\ 4 & 5 \end{smallmatrix}\right]$. First $\mathbf{A}^{\mathsf{T}}\mathbf{A} = \left[\begin{smallmatrix} 3 & 4 \\ 0 & 5 \end{smallmatrix}\right]\left[\begin{smallmatrix} 3 & 0 \\ 4 & 5 \end{smallmatrix}\right] = \left[\begin{smallmatrix} 25 & 20 \\ 20 & 25 \end{smallmatrix}\right]$. Its characteristic equation $(25-\lambda)^2 - 400 = 0$ gives $25 - \lambda = \pm 20$, so $\lambda = 45$ and 5, with eigenvectors $[1,1]/\sqrt{2}$ and $[1,-1]/\sqrt{2}$ (same pattern as the previous example). Hence $\sigma_1 = \sqrt{45} \approx 6.708$ and $\sigma_2 = \sqrt{5} \approx 2.236$. Check: $\sigma_1\sigma_2 = \sqrt{225} = 15 = |\det \mathbf{A}| = |15 - 0|$. ✓
Then $\mathbf{u}_1 = \mathbf{A}\mathbf{v}_1/\sigma_1 = \frac{1}{\sqrt{2}\sqrt{45}}[3,9] = [1,3]/\sqrt{10}$ and $\mathbf{u}_2 = \mathbf{A}\mathbf{v}_2/\sigma_2 = \frac{1}{\sqrt{2}\sqrt{5}}[3,-1] = [3,-1]/\sqrt{10}$, which are indeed perpendicular unit vectors.

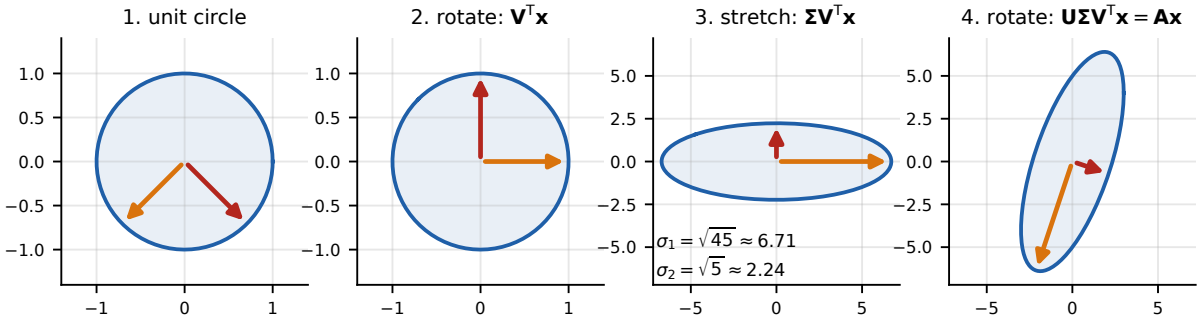


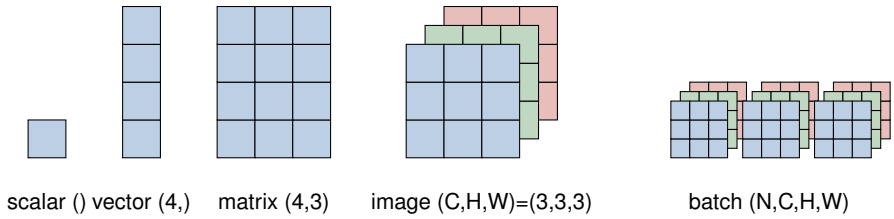
Figure 2.4: The SVD of $\mathbf{A} = \left[\begin{smallmatrix} 3 & 0 \\ 4 & 5 \end{smallmatrix}\right]$ applied to the unit circle in three stages. The orange and red arrows are the right singular vectors $\mathbf{v}_1, \mathbf{v}_2$. Stage 2: $\mathbf{V}^{\mathsf{T}}$ rotates them onto the axes (the circle looks the same). Stage 3: $\boldsymbol{\Sigma}$ stretches the axes by $\sigma_1 \approx 6.71$ and $\sigma_2 \approx 2.24$, producing an ellipse. Stage 4: $\mathbf{U}$ rotates the ellipse into its final position. Note the axis scales change between panels 2 and 3.

The largest singular value σ_1 is the maximum factor by which $\mathbf{A}$ can lengthen any vector; the ratio σ_1/σ_n (the **condition number**) says how close the matrix is to squashing some direction — large condition numbers make inversion numerically dangerous, a fact well known to anyone who has implemented zero-forcing equalisation. Keeping only the largest few singular values gives the best low-rank approximation of a matrix (used for compression and in PCA).

2.10 Broadcasting and tensors

2.10.1 Tensors

A scalar has 0 indices, a vector 1, a matrix 2. A **tensor** is the general name for an array with any number of indices ("axes"). Deep learning code is all tensors. A batch of colour images is a 4-axis tensor of shape (N, C, H, W): N images, C colour channels, height H, width W. For example $(32, 3, 28, 28)$ holds $32 \times 3 \times 28 \times 28 = 75{,}264$ numbers; the entry `X[n,c,i,j]` is the brightness of channel c at row i, column j of image n.



2.10.2 Broadcasting

In $\mathbf{Z} = \mathbf{XW} + \mathbf{b}$ we added a vector $\mathbf{b}$ of shape $(h,)$ to a matrix of shape (N, h). Strictly, you cannot add objects of different shapes; **broadcasting** is the convention (used by NumPy and PyTorch) that silently copies the smaller array along the missing axes. The rule: compare shapes from the right; two sizes are compatible if they are equal or one of them is 1 (or missing); the result takes the larger size.

```
X = np.ones((4, 3)); b = np.array([10., 20., 30.])
print((X + b).shape)          # (4, 3): b is added to every row
img = np.zeros((32, 3, 28, 28))
mean = np.array([0.5, 0.4, 0.3]).reshape(1, 3, 1, 1)  # one value per channel
print((img - mean).shape)     # (32, 3, 28, 28)
```

Pitfall: silent broadcasting bugs

If a target vector has shape $(N,)$ and a prediction has shape $(N, 1)$, then `pred - y` broadcasts to shape (N, N) — a matrix of every prediction minus every target. The code runs, the loss is wrong, and training mysteriously fails. Print shapes whenever in doubt.

2.11 Complex numbers, vectors and matrices

Wireless signals are naturally described by complex numbers, and Part V of this book uses them throughout. Here is everything we need, from zero.

2.11.1 Complex numbers as arrows in the plane

Intuition: a complex number is a 2-D arrow that knows how to multiply

Invent a number j with $j^2 = -1$ (engineers write j; mathematicians write i). A **complex number** $z = a + jb$ is a pair of real numbers: the **real part** a and the **imaginary part** b. Plot it as the point (a, b) in the *complex plane*. So far it is just a 2-D vector. The magic is multiplication:

multiplying by j *rotates by* 90° ($j \cdot (a+jb) = -b + ja$), and in general multiplying two complex numbers *multiplies their lengths and adds their angles*.

Definition: modulus, phase, conjugate

For $z = a + jb$: the **modulus** $|z| = \sqrt{a^2 + b^2}$ (length of the arrow), the **phase** $\varphi = \operatorname{atan2}(b, a)$ (its angle), and the **conjugate** $z^* = a - jb$ (mirror image in the real axis). Note $zz^* = (a+jb)(a-jb) = a^2 - j^2b^2 = a^2 + b^2 = |z|^2$, a real number.

Arithmetic follows ordinary algebra with $j^2 = -1$:

$$(1+2j)(3-j) = 3 - j + 6j - 2j^2 = 3 + 5j + 2 = 5 + 5j.$$

Euler's formula $e^{j\varphi} = \cos\varphi + j\sin\varphi$ gives the **polar form** $z = |z|\, e^{j\varphi}$. (The formula follows from the Taylor series of Chapter 3; geometrically, $e^{j\varphi}$ is the unit arrow at angle φ.) In polar form, multiplication is obvious: $r_1 e^{j\varphi_1} \cdot r_2 e^{j\varphi_2} = r_1 r_2 e^{j(\varphi_1+\varphi_2)}$. For instance $z = 3 + 4j$ has $|z| = 5$ and $\varphi = \operatorname{atan2}(4, 3) \approx 53.1^\circ$; multiplying by $e^{j\pi/3}$ rotates it by 60° without changing its length (Fig. 2.5).

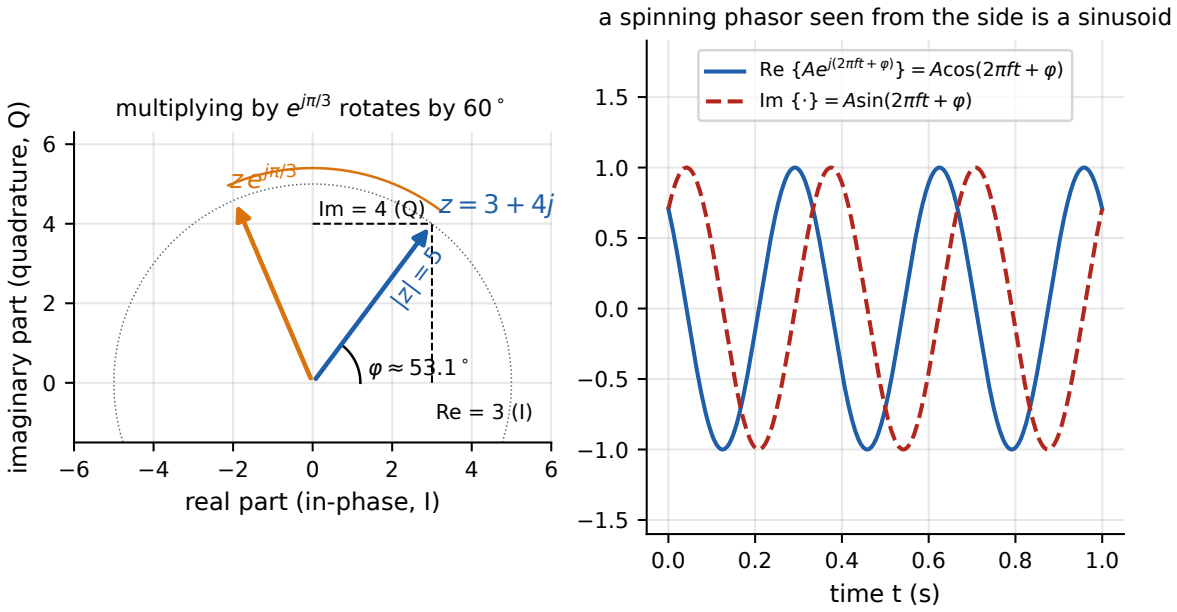


Figure 2.5: Left: the complex number $z = 3 + 4j$ as an arrow with modulus 5 and phase 53.1°; multiplying by $e^{j\pi/3}$ rotates it by 60° (orange). Right: the phasor $Ae^{j(2\pi ft+\varphi)}$ spins around the circle f times per second; its real part (projection on the horizontal axis) is the cosine you see on an oscilloscope, its imaginary part the sine. The pair (I,Q) is just (real, imaginary).

Wireless connection: I/Q samples, phase rotation and the one-tap equaliser

A baseband sample is a complex number $x = I + jQ$. A flat-fading channel multiplies it by a complex gain $h = |h|\, e^{j\theta}$: the constellation is scaled by $|h|$ and *rotated* by θ. A carrier frequency offset Δf multiplies sample n by $e^{j2\pi\Delta f n T_s}$: the constellation spins. Equalisation $\hat{x} = y/h = y\,h^*/\,|h|^2$ is a rotation back by $-\theta$ and a rescaling. Feeding I and Q as two real input channels to a neural network (as Part VI does) is legitimate precisely because a complex number *is* a 2-D real vector — but the network must then learn for itself that rotations are

special.

2.11.2 Complex vectors, the conjugate transpose and the Hermitian inner product

A complex vector $\mathbf{x} \in \mathbb{C}^n$ is a list of n complex numbers. The naive dot product fails as a length: for $\mathbf{h} = [1, j]$, $\mathbf{h}^{\mathsf{T}}\mathbf{h} = 1 + j^2 = 0$, yet $\mathbf{h}$ is clearly not the zero vector. The fix is to conjugate one of the factors.

Definition: conjugate transpose and Hermitian inner product

The **conjugate (Hermitian) transpose** $\mathbf{A}^{\mathsf{H}}$ transposes and conjugates every entry: $(\mathbf{A}^{\mathsf{H}})_{ij} = a_{ji}^*$. The **Hermitian inner product** of $\mathbf{u}, \mathbf{v} \in \mathbb{C}^n$ is

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^{\mathsf{H}}\mathbf{v} = \sum_{i=1}^{n} u_i^* v_i, \qquad \|\mathbf{u}\|^2 = \mathbf{u}^{\mathsf{H}}\mathbf{u} = \sum_i |u_i|^2 \geq 0. \tag{2.14}$$

Note $\mathbf{v}^{\mathsf{H}}\mathbf{u} = (\mathbf{u}^{\mathsf{H}}\mathbf{v})^*$: swapping the order conjugates the result. A matrix with $\mathbf{A}^{\mathsf{H}} = \mathbf{A}$ is **Hermitian** (the complex analogue of symmetric); its eigenvalues are real. A matrix with $\mathbf{U}^{\mathsf{H}}\mathbf{U} = \mathbf{I}$ is **unitary** (the complex analogue of orthogonal).

Worked example: Hermitian inner product

$\mathbf{h} = [1, j]$: $\mathbf{h}^{\mathsf{H}}\mathbf{h} = 1^* \cdot 1 + j^* \cdot j = 1 + (-j)(j) = 1 + 1 = 2$, so $\|\mathbf{h}\| = \sqrt{2}$. For $\mathbf{u} = [1 + j, 2]$, $\mathbf{v} = [j, 1 - j]$: $\mathbf{u}^{\mathsf{H}}\mathbf{v} = (1 - j)(j) + 2(1 - j) = j + 1 + 2 - 2j = 3 - j$, and $\mathbf{v}^{\mathsf{H}}\mathbf{u} = 3 + j$, its conjugate. ✓

All the real results carry over with ^{T} replaced by ^{H}: $(\mathbf{AB})^{\mathsf{H}} = \mathbf{B}^{\mathsf{H}}\mathbf{A}^{\mathsf{H}}$, the Cauchy–Schwarz inequality $|\mathbf{u}^{\mathsf{H}}\mathbf{v}| \leq \|\mathbf{u}\|\,\|\mathbf{v}\|$, and the SVD $\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathsf{H}}$ with unitary $\mathbf{U}, \mathbf{V}$ and real non-negative singular values.

Wireless connection: receive beamforming is a dot product

A receiver with n antennas sees $\mathbf{y} = \mathbf{h}\,s + \mathbf{n}$, where s is the transmitted symbol, $\mathbf{h} \in \mathbb{C}^n$ the vector of channel gains to each antenna and $\mathbf{n}$ noise. A *combiner* (beamformer) $\mathbf{w}$ forms $z = \mathbf{w}^{\mathsf{H}}\mathbf{y} = (\mathbf{w}^{\mathsf{H}}\mathbf{h})s + \mathbf{w}^{\mathsf{H}}\mathbf{n}$ — a Hermitian inner product. With $\|\mathbf{w}\| = 1$ the noise power is unchanged, so we want $|\mathbf{w}^{\mathsf{H}}\mathbf{h}|$ as large as possible. By Cauchy–Schwarz it is at most $\|\mathbf{h}\|$, reached when $\mathbf{w}$ points along $\mathbf{h}$: $\mathbf{w} = \mathbf{h}/\|\mathbf{h}\|$, **maximum-ratio combining**. Example: $\mathbf{h} = [1, j]$. MRC gives $\mathbf{w}^{\mathsf{H}}\mathbf{h} = \frac{1}{\sqrt{2}}(1 \cdot 1 + (-j)(j)) = \sqrt{2}$, power gain 2 (3 dB). The naive equal-weight combiner $\mathbf{w} = [1, 1]/\sqrt{2}$ gives $\mathbf{w}^{\mathsf{H}}\mathbf{h} = (1 + j)/\sqrt{2}$, power gain only 1: the second antenna's signal arrives 90° out of phase and is wasted. Beamforming is just "aligning two complex arrows".

2.11.3 MIMO: $\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}$ and eigen-channels

With n_t transmit and n_r receive antennas, each receive antenna hears a weighted sum of all transmitted symbols, so

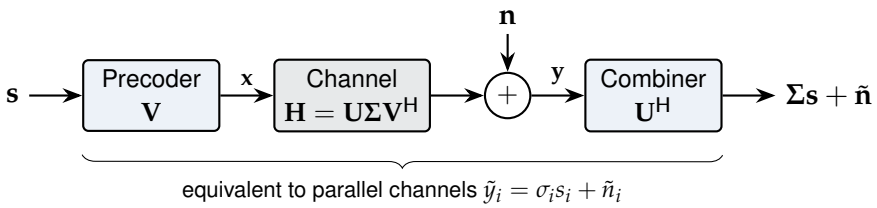
$$\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}, \qquad \mathbf{H} \in \mathbb{C}^{n_r \times n_t}, \tag{2.15}$$

where h_{ij} is the complex gain from transmit antenna j to receive antenna i. Each output mixes all inputs — interference. The SVD untangles it. Write $\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathsf{H}}$. If the transmitter *precodes* its

data symbols $\mathbf{s}$ as $\mathbf{x} = \mathbf{V}\mathbf{s}$ and the receiver computes $\tilde{\mathbf{y}} = \mathbf{U}^{\mathsf{H}}\mathbf{y}$, then

$$\tilde{\mathbf{y}} = \mathbf{U}^{\mathsf{H}}(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathsf{H}}\mathbf{V}\mathbf{s} + \mathbf{n}) = \mathbf{\Sigma}\mathbf{s} + \mathbf{U}^{\mathsf{H}}\mathbf{n},$$

because $\mathbf{U}^{\mathsf{H}}\mathbf{U} = \mathbf{I}$ and $\mathbf{V}^{\mathsf{H}}\mathbf{V} = \mathbf{I}$. Since $\mathbf{\Sigma}$ is diagonal, $\tilde{y}_i = \sigma_i s_i + \tilde{n}_i$: the MIMO channel becomes $\operatorname{rank}(\mathbf{H})$ *independent parallel scalar channels* (eigen-channels) with gains σ_i, and the noise $\mathbf{U}^{\mathsf{H}}\mathbf{n}$ is statistically unchanged because a unitary matrix merely rotates it.



Wireless connection: eigen-channels with numbers

Take (for simplicity a real) $\mathbf{H} = \left[\begin{smallmatrix} 3 & 0 \\ 4 & 5 \end{smallmatrix}\right]$, whose SVD we computed above. The two eigen-channels have amplitude gains $\sigma_1 \approx 6.71$ and $\sigma_2 \approx 2.24$, i.e. power gains $\sigma_1^2 = 45$ and $\sigma_2^2 = 5$. A rank-1 channel such as $\left[\begin{smallmatrix} 1 & 2 \\ 2 & 4 \end{smallmatrix}\right]$ (typical of line-of-sight with closely spaced antennas) has $\sigma_2 = 0$ and supports only one stream, however many antennas you add. The 5G rank indicator (RI) and precoding matrix indicator (PMI) that a UE reports are exactly estimates of "how many useful σ_i" and "an approximation of $\mathbf{V}$". Chapter 4 turns these gains into capacity, and Part V shows networks that learn precoders.

Chapter recap

- A vector is both an arrow and a list; its length is $\|\mathbf{x}\| = \sqrt{\sum x_i^2}$ (e.g. $\|[1,2,3]\| = \sqrt{14} \approx 3.742$).
- The dot product $\mathbf{a} \cdot \mathbf{b} = \sum a_i b_i = \|\mathbf{a}\|\,\|\mathbf{b}\| \cos\theta$ measures alignment; it gives cosine similarity and projections. A neuron computes $\mathbf{w} \cdot \mathbf{x} + b$.
- A matrix is a linear machine; its columns are where the basis arrows land. $(m,n)(n,p) \to (m,p)$; entry (i,j) = row i dot column j. $\mathbf{AB} \neq \mathbf{BA}$; $(\mathbf{AB})^{\mathsf{T}} = \mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}$.
- $\det \mathbf{A}$ is the area/volume scale factor; $\det = 0 \iff$ not invertible $\iff$ not full rank.
- Eigenvectors are only stretched: $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$, found from $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$. Repeated multiplication makes $|\lambda| > 1$ explode and $|\lambda| < 1$ vanish.
- SVD: every matrix is rotate–stretch–rotate, $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathsf{T}}$.
- Tensors are multi-axis arrays, e.g. (N, C, H, W); broadcasting copies along missing axes.
- Complex numbers are arrows that multiply by scaling and rotating. Use ^{H} and $\mathbf{u}^{\mathsf{H}}\mathbf{v} = \sum u_i^* v_i$. Beamforming is an inner product; MIMO $\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}$ splits into parallel eigen-channels via the SVD.

Exercises

1. Compute $\|[2,-1,2]\|$, the unit vector in its direction, and its L1 norm.
2. Find the angle between $\mathbf{a} = [1,0,1]$ and $\mathbf{b} = [0,1,1]$, and the projection of $\mathbf{a}$ onto $\mathbf{b}$.

3. Compute $\begin{bmatrix} 2 & 0 & 1 \\ -1 & 3 & 2 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 4 & 0 \end{bmatrix}$. What are the shapes? Is the product in the other order defined, and what shape would it have?
4. A layer receives a batch $\mathbf{X}$ of shape $(64, 784)$ and outputs $\mathbf{Z}$ of shape $(64, 128)$. What shape must $\mathbf{W}$ and $\mathbf{b}$ have? How many parameters does the layer have?
5. Find the determinant and inverse of $\left[\begin{smallmatrix} 3 & 1 \\ 5 & 2 \end{smallmatrix}\right]$. What does the determinant tell you geometrically?
6. Find the eigenvalues and eigenvectors of $\left[\begin{smallmatrix} 4 & 1 \\ 2 & 3 \end{smallmatrix}\right]$. Check that their product equals the determinant.
7. For $\mathbf{u} = [1, j]$ and $\mathbf{v} = [j, 1]$ compute $\mathbf{u}^{\mathsf{H}}\mathbf{v}$, $\|\mathbf{u}\|$ and $\|\mathbf{v}\|$. Are they orthogonal?
8. (Conceptual) A 4×4 MIMO channel has singular values $[5, 2, 0.1, 0]$. What is its rank? How many streams would you actually send, and why?

Answers

1. $\sqrt{4+1+4} = \sqrt{9} = 3$. Unit vector $[2/3, -1/3, 2/3]$. L1 norm $2+1+2 = 5$.
2. $\mathbf{a} \cdot \mathbf{b} = 0+0+1 = 1$; $\|\mathbf{a}\| = \|\mathbf{b}\| = \sqrt{2}$. $\cos\theta = 1/2$, so $\theta = 60°$. Projection $= \frac{1}{2}[0, 1, 1] = [0, 0.5, 0.5]$.
3. Shapes $(2,3)(3,2) \to (2,2)$. Row 1: $[2\cdot 1 + 0 + 1\cdot 4,\ 2\cdot 2 + 0 + 0] = [6, 4]$. Row 2: $[-1 + 0 + 8,\ -2 + 3 + 0] = [7, 1]$. Result $\left[\begin{smallmatrix} 6 & 4 \\ 7 & 1 \end{smallmatrix}\right]$. The other order is $(3,2)(2,3) \to (3,3)$: defined, but a different object.
4. $(64, 784)(784, 128) \to (64, 128)$, so $\mathbf{W} \in \mathbb{R}^{784\times 128}$ and $\mathbf{b} \in \mathbb{R}^{128}$ (broadcast over the 64 rows). Parameters: $784 \cdot 128 + 128 = 100{,}352 + 128 = 100{,}480$.
5. $\det = 3\cdot 2 - 1\cdot 5 = 1$. Inverse $= \frac{1}{1}\left[\begin{smallmatrix} 2 & -1 \\ -5 & 3 \end{smallmatrix}\right]$. Check: $\left[\begin{smallmatrix} 3 & 1 \\ 5 & 2 \end{smallmatrix}\right]\left[\begin{smallmatrix} 2 & -1 \\ -5 & 3 \end{smallmatrix}\right] = \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]$. Determinant 1: areas are preserved (the map shears/rotates without changing area, and does not flip).
6. $\det\left[\begin{smallmatrix} 4-\lambda & 1 \\ 2 & 3-\lambda \end{smallmatrix}\right] = (4-\lambda)(3-\lambda) - 2 = \lambda^2 - 7\lambda + 10 = (\lambda - 5)(\lambda - 2)$. $\lambda = 5$: $-v_1 + v_2 = 0 \Rightarrow \mathbf{v} = [1, 1]$. $\lambda = 2$: $2v_1 + v_2 = 0 \Rightarrow \mathbf{v} = [1, -2]$. Product $5\cdot 2 = 10 = 4\cdot 3 - 1\cdot 2$. ✓ (The matrix is not symmetric, and the eigenvectors are not perpendicular.)
7. $\mathbf{u}^{\mathsf{H}}\mathbf{v} = 1^* \cdot j + j^* \cdot 1 = j - j = 0$. $\|\mathbf{u}\|^2 = 1 + 1 = 2$, $\|\mathbf{v}\|^2 = 2$, so both norms are $\sqrt{2}$. Since the inner product is 0 they are orthogonal (even though the plain $\mathbf{u}^{\mathsf{T}}\mathbf{v} = 2j \neq 0$).
8. Rank $= 3$ (three nonzero singular values). The third eigen-channel has power gain 0.01, i.e. $20\log_{10}(5/0.1) \approx 34$ dB weaker than the first; at practical SNR it carries almost nothing, so you would send 2 streams (this is what rank adaptation does).

Self-test quiz

1. What is $[1, 2, 3] \cdot [4, 5, 6]$? *32.*
2. Two vectors have cosine similarity 0. What is the angle between them? $90°$ *(orthogonal).*
3. Can a $(5, 3)$ matrix multiply a $(5, 3)$ matrix? *No; inner dims 3 and 5 differ.* $\mathbf{A}^{\mathsf{T}}\mathbf{B}$ *works:* $(3, 3)$.

4. What does $\det \mathbf{A} = 0$ mean geometrically? *Space is squashed to lower dimension; no inverse.*
5. An RNN multiplies its gradient by a matrix with largest eigenvalue magnitude 0.8 at each of 100 steps. What happens? *It vanishes ($0.8^{100} \approx 2 \times 10^{-10}$).*
6. Why must we conjugate in the complex inner product? *So that $\mathbf{u}^{\mathsf{H}}\mathbf{u} = \sum |u_i|^2$ is a real, non-negative length.*

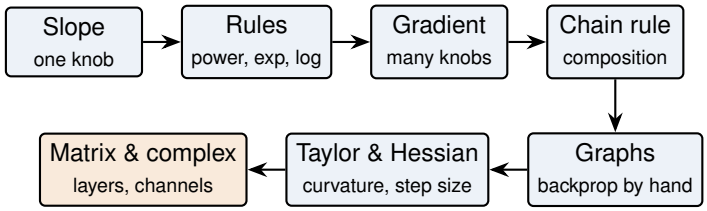
Further reading

- G. Strang, *Introduction to Linear Algebra*, 5th ed., Wellesley–Cambridge Press, 2016. The classic, intuitive first course; the column picture and the four subspaces.
- 3Blue1Brown (G. Sanderson), *Essence of Linear Algebra* video series. Animated versions of every geometric picture in this chapter.
- M. P. Deisenroth, A. A. Faisal, C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020, Chapters 2–4.
- D. Tse and P. Viswanath, *Fundamentals of Wireless Communication*, Cambridge University Press, 2005, Chapter 7 (MIMO and the SVD-based parallel channel decomposition).

Chapter 3

Derivatives, Gradients and the Chain Rule

Training a neural network is a search: among millions of possible settings of its weights, find the ones that make the error (the *loss*) small. The only tool that makes this search feasible is the **derivative**, which answers one question: *if I nudge this knob a tiny bit, how much does the output change, and in which direction?* With millions of knobs, the answer is a vector called the **gradient**; and because a network is a long chain of simple functions, the gradient is computed with the **chain rule** — applied systematically, this is *backpropagation*. This chapter builds all three ideas from nothing: slopes, limits, the differentiation rules (each derived), partial derivatives and gradients, the chain rule, computational graphs, Taylor expansions and curvature, and the handful of matrix-calculus formulas that Part II uses to derive backpropagation for whole layers. We close with derivatives of real functions of complex variables, which is what you need to train anything that touches I/Q samples or channel coefficients.



3.1 Functions

A **function** is a rule that takes an input and returns exactly one output. We write $y = f(x)$: "y is f of x". Examples: $f(x) = x^2$ (square it), $g(x) = 3x + 1$ (multiply by 3, add 1), $\sigma(z) = 1/(1 + e^{-z})$ (the sigmoid, which squeezes any number into $(0, 1)$). A function can take several inputs, $f(x, y) = x^2 y + 3y$, or a whole vector, $L(\mathbf{w})$. A neural network is a function: input image in, class probabilities out. Its loss is a function of its weights. Everything in this chapter is about how outputs respond to small changes of inputs.

3.2 The derivative: slope at a point

Intuition: a speedometer

Driving a car, your position $s(t)$ changes with time. The average speed over an hour is (distance covered)/(time taken). But the speedometer shows something subtler: the speed *right now*. You can estimate it by measuring the distance covered over a shorter and shorter interval — a minute, a second, a millisecond. The number this ratio settles down to is the **derivative**: the instantaneous rate of change. On a graph, it is the steepness (slope) of the curve at one point — the slope of the straight line that just touches the curve there, the **tangent line**.

3.2.1 From average slope to instantaneous slope

The slope of a straight line through two points (x_1, y_1) and (x_2, y_2) is "rise over run", $(y_2 - y_1)/(x_2 - x_1)$. For a curve, pick the point x and a nearby point $x + h$, where h ("step") is a small number. The line through $(x, f(x))$ and $(x + h, f(x + h))$ is a **secant**, with slope

$$\frac{f(x+h) - f(x)}{h}.$$

As h shrinks, the second point slides towards the first, and the secant swings into the tangent (Fig. 3.1).

Definition: derivative

$$f'(x) = \frac{\mathrm{d}f}{\mathrm{d}x} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}. \tag{3.1}$$

"$\lim_{h \to 0}$" (the **limit**) means: the value the expression approaches as h gets as close to zero as we like (we never set $h = 0$ exactly, which would give $0/0$). The notation $\frac{\mathrm{d}f}{\mathrm{d}x}$ ("dee f dee x") reminds us it is a ratio of a tiny change in f to a tiny change in x. If the limit exists, f is **differentiable** at x.

Worked example: derivative of x^2 from the definition

$$\frac{(x+h)^2 - x^2}{h} = \frac{x^2 + 2xh + h^2 - x^2}{h} = \frac{2xh + h^2}{h} = 2x + h \xrightarrow{h \to 0} 2x.$$

So $\frac{\mathrm{d}}{\mathrm{d}x} x^2 = 2x$. At $x = 1$ the secant slope is $2 + h$: 4 for $h = 2$, 3 for $h = 1$, 2.1 for $h = 0.1$, approaching 2.

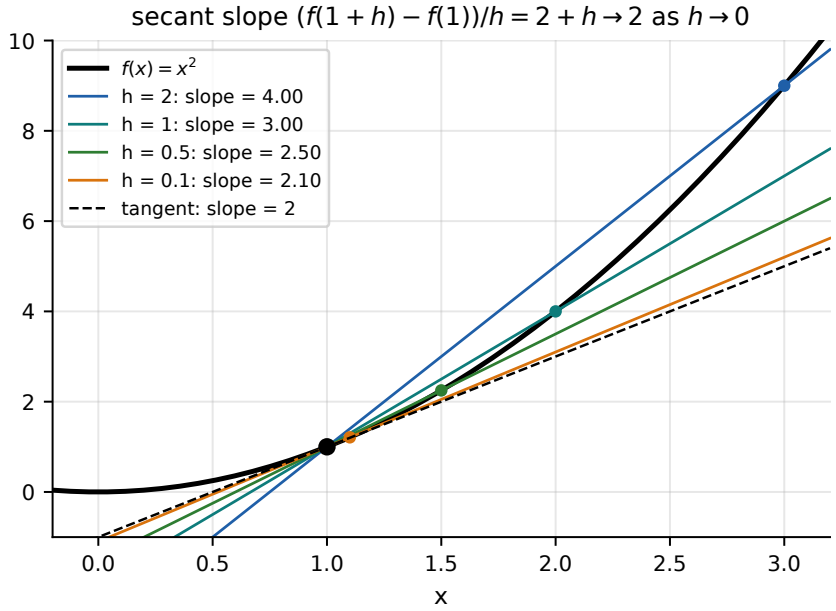


Figure 3.1: Secants of $f(x) = x^2$ through $(1, 1)$ and $(1 + h, (1 + h)^2)$ for shrinking h. Their slopes $2 + h$ approach the slope 2 of the dashed tangent line. The derivative is the limit of this process.

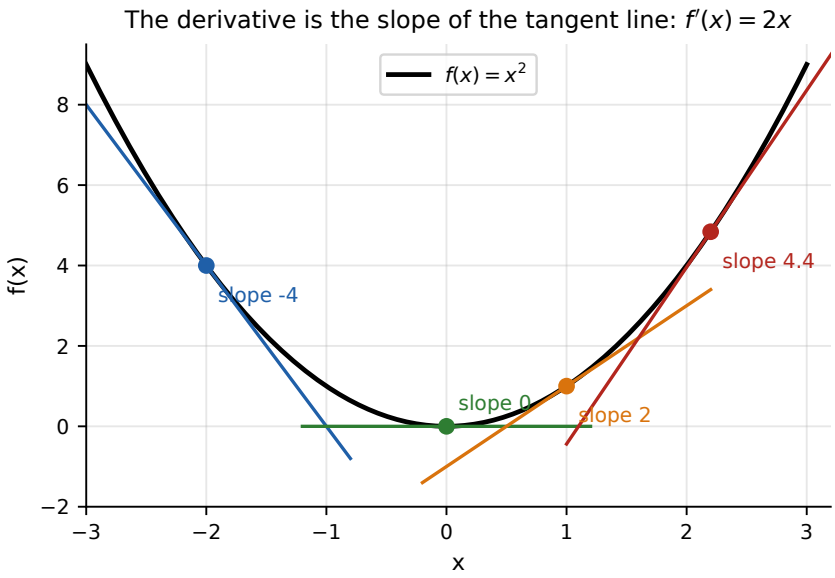


Figure 3.2: The derivative $f'(x) = 2x$ read off as tangent slopes: negative on the left (the curve goes downhill), zero at the bottom, positive and growing on the right. A minimum is where the slope is zero — this is why "set the derivative to zero" finds optima, and why gradient descent stops at the bottom.

The derivative as a local linear approximation. Rearranging (3.1) for small h gives the single most useful formula in this book:

$$f(x+h) \approx f(x) + f'(x)\,h. \tag{3.2}$$

In words: *near x, the function behaves like a straight line with slope $f'(x)$.* If $f'(x) = 3$, moving the input by 0.01 moves the output by about 0.03. If $f'(x) < 0$, increasing x decreases f. Gradient descent (Part II) is built entirely on this: to decrease f, move x by $h = -\eta f'(x)$ for a small $\eta > 0$; then $f(x+h) \approx f(x) - \eta f'(x)^2$, which is smaller.

3.3 Differentiation rules, derived

Computing limits every time would be painful. A handful of rules, each proven once from (3.1), cover every function in this book.

Constants and sums. The derivative of a constant is 0 (a flat line has zero slope). Derivatives respect sums and constant multiples: $(af + bg)' = af' + bg'$, because the difference quotient of $af + bg$ is a times that of f plus b times that of g, and limits respect sums.

Power rule: $\frac{\mathrm{d}}{\mathrm{d}x} x^n = nx^{n-1}$. For a positive integer n, expand $(x+h)^n$ with the binomial theorem: $(x+h)^n = x^n + nx^{n-1}h + (\text{terms containing } h^2, h^3, \dots)$. Then

$$\frac{(x+h)^n - x^n}{h} = nx^{n-1} + (\text{terms containing } h, h^2, \dots) \xrightarrow{h \to 0} nx^{n-1}.$$

The rule holds for every real power: $\frac{\mathrm{d}}{\mathrm{d}x}\sqrt{x} = \frac{\mathrm{d}}{\mathrm{d}x} x^{1/2} = \frac{1}{2}x^{-1/2}$, $\frac{\mathrm{d}}{\mathrm{d}x}\frac{1}{x} = -x^{-2}$.

Exponential: $\frac{\mathrm{d}}{\mathrm{d}x}e^x = e^x$**.** The number $e \approx 2.71828$ is *defined* as the base for which the exponential's slope at 0 is exactly 1: $\lim_{h\to 0}(e^h - 1)/h = 1$. Then

$$\frac{e^{x+h} - e^x}{h} = e^x \frac{e^h - 1}{h} \xrightarrow{h\to 0} e^x \cdot 1 = e^x.$$

The exponential is the function whose growth rate equals its current value — compound interest, chain reactions, and the reason softmax uses e^z.

Natural logarithm: $\frac{\mathrm{d}}{\mathrm{d}x}\ln x = \frac{1}{x}$**.** The logarithm undoes the exponential: if $y = \ln x$ then $x = e^y$. Near this point, a small change Δy produces $\Delta x \approx e^y \Delta y = x\,\Delta y$ (by the exponential rule). So $\Delta y/\Delta x \approx 1/x$. Logs turn products into sums and are everywhere in losses (Chapter 4).

Product rule: $(fg)' = f'g + fg'$**.** Think of $f(x)g(x)$ as the area of a rectangle with sides f and g. Nudge x by h: the sides grow by $\Delta f \approx f'h$ and $\Delta g \approx g'h$. The new area is $(f + \Delta f)(g + \Delta g) = fg + \Delta f\, g + f\,\Delta g + \Delta f\,\Delta g$. The extra area is two thin strips plus a tiny corner $\Delta f \Delta g \approx f'g'h^2$. Dividing by h and letting $h \to 0$, the corner vanishes (it has h^2) and we get $f'g + fg'$.

Quotient rule: $(f/g)' = (f'g - fg')/g^2$**.** Write $q = f/g$, so $f = qg$. By the product rule $f' = q'g + qg'$. Solve: $q' = (f' - qg')/g = (f' - \frac{f}{g}g')/g = (f'g - fg')/g^2$.

Worked example: the sigmoid's derivative

$\sigma(z) = \dfrac{1}{1+e^{-z}}$. Quotient rule with $f = 1$ ($f' = 0$) and $g = 1 + e^{-z}$ ($g' = -e^{-z}$, by the chain rule of Section 3.6):

$$\sigma'(z) = \frac{0 - 1\cdot(-e^{-z})}{(1+e^{-z})^2} = \frac{e^{-z}}{(1+e^{-z})^2} = \underbrace{\frac{1}{1+e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1+e^{-z}}}_{1-\sigma(z)}.$$

So $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. At $z = 0$: $\sigma = 0.5$, $\sigma' = 0.25$ (its maximum). At $z = 2$: $\sigma \approx 0.881$, $\sigma' \approx 0.105$. Far from zero the slope is nearly 0 — the sigmoid "saturates", one cause of vanishing gradients.

Table 3.1: The derivatives used in this book.

$f(x)$	$f'(x)$	where it appears
c	0	biases' constant terms
x^n	nx^{n-1}	squared error e^2
e^x	e^x	softmax, sigmoid
$\ln x$	$1/x$	cross-entropy, log-likelihood
$\sigma(x)$	$\sigma(x)(1-\sigma(x))$	logistic output
$\tanh x$	$1 - \tanh^2 x$	RNNs
$\mathrm{ReLU}(x) = \max(0, x)$	1 if $x > 0$, 0 if $x < 0$	hidden layers
$\sin x,\ \cos x$	$\cos x,\ -\sin x$	phases, oscillators

3.4 Numerical derivatives and why centred differences win

A computer cannot take a limit, but it can use a small h. Two obvious choices:

$$\text{forward: } \frac{f(x+h)-f(x)}{h}, \qquad \text{centred: } \frac{f(x+h)-f(x-h)}{2h}.$$

Worked example: centred difference for x^2 at $x = 3$

With $h = 0.001$: $f(3.001) = 9.006001$ and $f(2.999) = 8.994001$, so

$$\frac{9.006001 - 8.994001}{0.002} = \frac{0.012}{0.002} = 6.0,$$

exactly the true derivative $2 \cdot 3 = 6$. The forward difference gives $(9.006001 - 9)/0.001 = 6.001$, off by h.

Why the centred formula is better. Use the Taylor expansion of Section 3.8 (the second-order version of (3.2), with a cubic correction):

$$f(x+h) = f(x) + f'(x)h + \tfrac{1}{2}f''(x)h^2 + \tfrac{1}{6}f'''(x)h^3 + \dots$$
$$f(x-h) = f(x) - f'(x)h + \tfrac{1}{2}f''(x)h^2 - \tfrac{1}{6}f'''(x)h^3 + \dots$$

The forward difference is $[f(x+h) - f(x)]/h = f'(x) + \frac{1}{2}f''(x)h + \dots$: error proportional to h, written $O(h)$. Subtracting the two lines, the even terms cancel: $f(x+h) - f(x-h) = 2f'(x)h + \frac{1}{3}f'''(x)h^3 + \dots$, so the centred difference is $f'(x) + \frac{1}{6}f'''(x)h^2 + \dots$: error $O(h^2)$. Halving h quarters the error. For $f = x^2$ the third derivative is zero, so the centred difference is exact (up to rounding), as we saw.

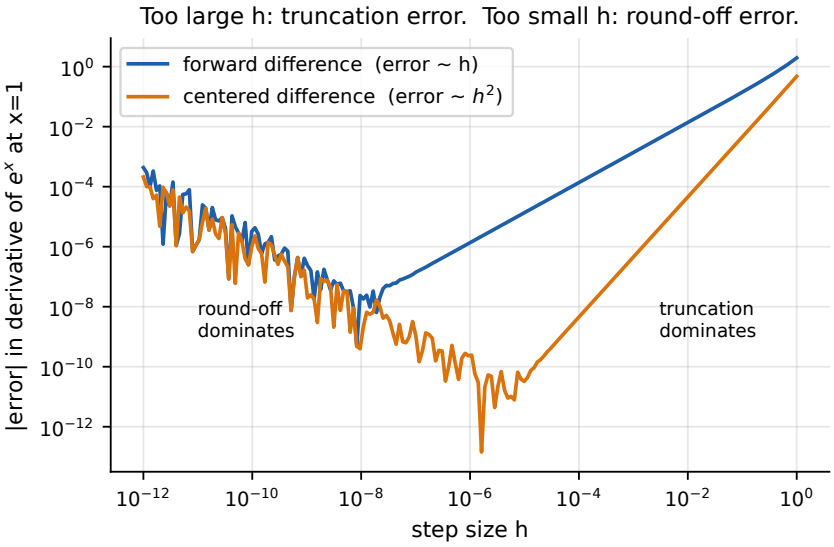


Figure 3.3: Error of numerical derivatives of e^x at $x = 1$ versus step h (log–log axes). For large h the error falls along straight lines of slope 1 (forward) and 2 (centred), confirming $O(h)$ and $O(h^2)$. For very small h, floating-point round-off (subtracting two nearly equal numbers) takes over. The centred difference reaches about 10^{-10} at $h \approx 10^{-5}$ — the usual choice for gradient checking.

This is the basis of **gradient checking** (Chapter 13): after deriving and coding a gradient by hand, compare it with centred differences; agreement to about 10^{-7} relative error means your backpropagation is correct.

3.5 Many inputs: partial derivatives and the gradient

Intuition: standing on a hillside

Imagine standing on a hill, where the height is $f(x, y)$ and x, y are your east and north coordinates. Step a little east, keeping north fixed, and note how fast you climb: that is the **partial derivative** $\frac{\partial f}{\partial x}$. Step north instead: $\frac{\partial f}{\partial y}$. Collect the two numbers into an arrow, $[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}]$. This arrow, the **gradient**, points in the direction of steepest climb, and its length is how steep that climb is. To go downhill fastest — to reduce a loss — walk the opposite way.

Definition: partial derivative, gradient

The **partial derivative** of $f(x_1, \ldots, x_n)$ with respect to x_i, written $\frac{\partial f}{\partial x_i}$ ("partial f partial x_i"), is the ordinary derivative in x_i while treating all other inputs as constants. The **gradient** is the vector of all partial derivatives:

$$\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n}\right]^{\mathsf{T}}. \tag{3.3}$$

(∇ is read "nabla" or "del".) It has the same shape as the input.

Worked example: gradient of $x^2y + 3y$ at $(2, 1)$

Treating y as a constant: $\frac{\partial f}{\partial x} = 2xy$. Treating x as a constant: $\frac{\partial f}{\partial y} = x^2 + 3$. At $(2, 1)$: $\frac{\partial f}{\partial x} = 2 \cdot 2 \cdot 1 = 4$, $\frac{\partial f}{\partial y} = 4 + 3 = 7$. So $\nabla f(2, 1) = [4, 7]$. Check with centred differences ($h = 10^{-3}$): $[f(2.001, 1) - f(1.999, 1)]/0.002 = [7.004001 - 6.996001]/0.002 = 4.000$. ✓

The multivariable version of (3.2) is: if every input moves a little, $\Delta\mathbf{x} = [\Delta x_1, \ldots, \Delta x_n]$, then the output changes by the sum of the individual effects,

$$f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + \sum_i \frac{\partial f}{\partial x_i}\Delta x_i = f(\mathbf{x}) + \nabla f \cdot \Delta\mathbf{x}. \tag{3.4}$$

A dot product (Chapter 2) appears: the change in f is the gradient dotted with the step.

3.5.1 Why the gradient points uphill

Take a step of fixed small length ϵ in the direction of a unit vector $\mathbf{u}$: $\Delta\mathbf{x} = \epsilon\mathbf{u}$. By (3.4) the change is $\epsilon\, \nabla f \cdot \mathbf{u}$. The rate $\nabla f \cdot \mathbf{u}$ is the **directional derivative**. Which direction maximises it? By the geometric dot product,

$$\nabla f \cdot \mathbf{u} = \|\nabla f\|\ \|\mathbf{u}\| \cos\theta = \|\nabla f\| \cos\theta,$$

where θ is the angle between $\mathbf{u}$ and the gradient. Since $\cos\theta \le 1$ (this is the Cauchy–Schwarz inequality $|\mathbf{a} \cdot \mathbf{b}| \le \|\mathbf{a}\|\,\|\mathbf{b}\|$), the rate is at most $\|\nabla f\|$, achieved at $\theta = 0$: *when* $\mathbf{u}$ *points along the gradient*. The steepest descent is at $\theta = 180°$, along $-\nabla f$, with rate $-\|\nabla f\|$. At $\theta = 90°$ the rate is zero: moving perpendicular to the gradient keeps f constant — you are walking along a **contour line**.

Key equation: gradient descent

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \, \nabla L(\mathbf{w}), \tag{3.5}$$

with a small learning rate $\eta > 0$. By (3.4) each step changes the loss by about $-\eta \, \|\nabla L\|^2 \leq 0$.

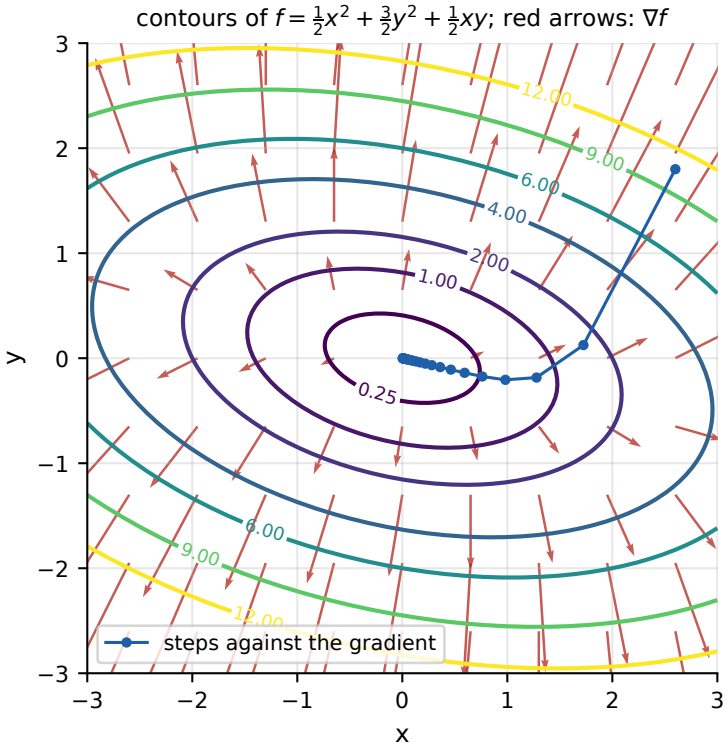


Figure 3.4: A contour map (lines of equal height, like a hiking map) of $f(x, y) = \frac{1}{2}x^2 + \frac{3}{2}y^2 + \frac{1}{2}xy$. The red arrows are gradients: they always cross contours at right angles and point uphill, longer where contours are tightly packed (steep). The blue path is gradient descent with $\eta = 0.25$ from $(2.6, 1.8)$: it first drops steeply in y (the steep direction), then crawls along the shallow valley towards the minimum at the origin.

Wireless connection: gradients in classical receivers

You have used gradient descent long before neural networks. The LMS adaptive equaliser updates its taps by $\mathbf{w} \leftarrow \mathbf{w} + \mu \, e^* \mathbf{x}$, which is exactly (3.5) applied to the instantaneous squared error $|e|^2$ (we derive this in Section 3.10). Phase-locked loops and timing-recovery loops are gradient descent on a phase or timing error. A neural receiver trained with backpropagation is the same idea with far more taps and nonlinearities.

3.6 The chain rule

Intuition: gears

Three gears are meshed: gear A drives B, B drives C. If B turns 3 times for each turn of A, and C turns 2 times for each turn of B, then C turns $3 \times 2 = 6$ times per turn of A. Rates of change in a chain *multiply*. A neural network is a long gear train: input $\to$ layer 1 $\to$ layer 2 $\to \cdots \to$ loss. The sensitivity of the loss to an early weight is the product of the local sensitivities along the way.

Key equation: chain rule, one variable

If $y = f(u)$ and $u = g(x)$, so that $y = f(g(x))$, then

$$\frac{\mathrm{d}y}{\mathrm{d}x} = \frac{\mathrm{d}y}{\mathrm{d}u} \cdot \frac{\mathrm{d}u}{\mathrm{d}x} = f'(g(x))\, g'(x). \tag{3.6}$$

Derivation. A small change Δx causes $\Delta u \approx g'(x)\Delta x$ (by (3.2)). That change in u causes $\Delta y \approx f'(u)\Delta u \approx f'(u)g'(x)\Delta x$. Dividing by Δx and letting it shrink gives (3.6).

Worked example: $y = (3x+1)^2$

Inner $u = 3x + 1$ with $\frac{\mathrm{d}u}{\mathrm{d}x} = 3$; outer $y = u^2$ with $\frac{\mathrm{d}y}{\mathrm{d}u} = 2u$. So $\frac{\mathrm{d}y}{\mathrm{d}x} = 2u \cdot 3 = 6(3x+1)$. At $x = 1$: $6 \cdot 4 = 24$. Check by expanding: $y = 9x^2 + 6x + 1$, $y' = 18x + 6 = 24$ at $x = 1$. ✓

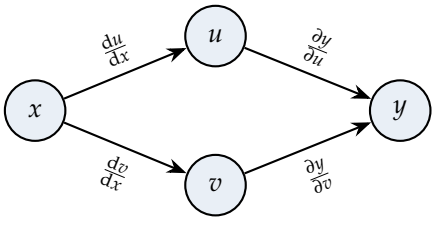
3.6.1 Several paths: the multivariable chain rule

Often one variable influences the output through several routes. Suppose x affects both $u = g(x)$ and $v = h(x)$, and $y = f(u, v)$. A nudge Δx moves u by $\frac{\mathrm{d}u}{\mathrm{d}x}\Delta x$ and v by $\frac{\mathrm{d}v}{\mathrm{d}x}\Delta x$; by (3.4) the effect on y is the sum of both effects:

$$\frac{\mathrm{d}y}{\mathrm{d}x} = \frac{\partial y}{\partial u}\frac{\mathrm{d}u}{\mathrm{d}x} + \frac{\partial y}{\partial v}\frac{\mathrm{d}v}{\mathrm{d}x}. \tag{3.7}$$

Key equation: sum over paths

The derivative of an output with respect to an input equals the **sum, over every path** from the input to the output, of the **product of the local derivatives** along that path.



$$\frac{\mathrm{d}y}{\mathrm{d}x} = \underbrace{\frac{\partial y}{\partial u}\frac{\mathrm{d}u}{\mathrm{d}x}}_{\text{upper path}} + \underbrace{\frac{\partial y}{\partial v}\frac{\mathrm{d}v}{\mathrm{d}x}}_{\text{lower path}}$$

Worked example: two paths

$y = uv$ with $u = x^2$, $v = \sin x$. Paths: $\frac{\partial y}{\partial u}\frac{\mathrm{d}u}{\mathrm{d}x} = v \cdot 2x$ and $\frac{\partial y}{\partial v}\frac{\mathrm{d}v}{\mathrm{d}x} = u\cos x$. Sum: $2x\sin x + x^2\cos x$ — the product rule, recovered as a special case.

3.6.2 The Jacobian

When a function maps a vector $\mathbf{x} \in \mathbb{R}^n$ to a vector $\mathbf{y} \in \mathbb{R}^m$ (like a layer of a network), we need every partial derivative $\frac{\partial y_i}{\partial x_j}$. Arrange them in an $m \times n$ table: the **Jacobian matrix**

$$\mathbf{J} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}, \qquad \Delta \mathbf{y} \approx \mathbf{J}\, \Delta \mathbf{x}. \tag{3.8}$$

The Jacobian is the best linear (matrix) approximation of the function near a point. For a linear map $\mathbf{y} = \mathbf{A}\mathbf{x}$, $y_i = \sum_j a_{ij} x_j$ so $\frac{\partial y_i}{\partial x_j} = a_{ij}$: the Jacobian is $\mathbf{A}$ itself. For an element-wise function $y_i = \sigma(x_i)$ it is diagonal, $\operatorname{diag}(\sigma'(x_1), \ldots, \sigma'(x_n))$. The chain rule for vector functions becomes *matrix multiplication of Jacobians*: if $\mathbf{z} = \mathbf{f}(\mathbf{y})$ and $\mathbf{y} = \mathbf{g}(\mathbf{x})$, then $\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{z}}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ — each entry of that product is exactly the sum over paths of (3.7). A deep network's input–output Jacobian is a product of many layer Jacobians; by Chapter 2's eigenvalue argument, such products tend to vanish or explode.

3.7 Computational graphs and backpropagation by hand

Intuition: an assembly line, run forwards then backwards

Break a complicated formula into elementary steps, each a node in a graph. The **forward pass** pushes numbers through the nodes, left to right, computing the output. The **backward pass** then walks right to left: each node receives "how much the final output cares about my output" from downstream, multiplies it by its own *local* derivative, and passes the result upstream. By the chain rule this produces every derivative of the output with respect to every input, in one sweep, at a cost comparable to the forward pass. This is backpropagation.

We work a complete example: a single neuron with a sigmoid output and squared-error loss,

$$z = wx + b, \qquad a = \sigma(z), \qquad e = a - y, \qquad L = e^2,$$

with $x = 1.5$, $w = 2$, $b = -1$, target $y = 1$. We want $\frac{\partial L}{\partial w}$ and $\frac{\partial L}{\partial b}$ to know how to adjust the weight and bias.

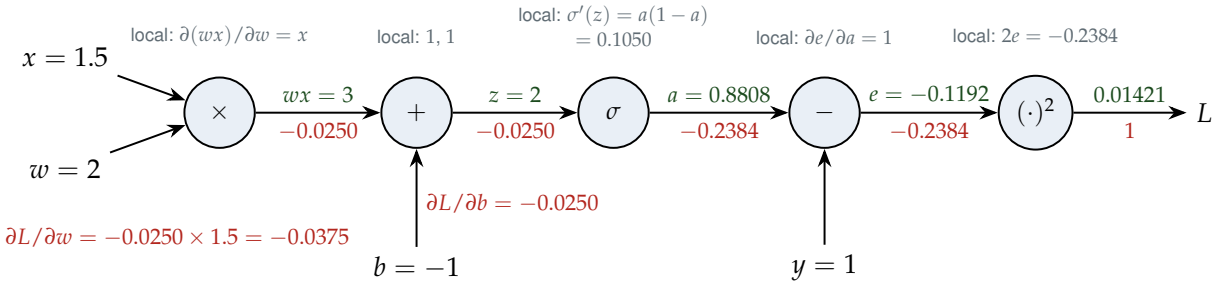


Figure 3.5: A computational graph for $L = (\sigma(wx + b) - y)^2$. Green numbers above the arrows: forward values. Red numbers below: the backward "upstream gradient" $\partial L/\partial$(that wire), obtained by multiplying the gradient arriving from the right by the node's local derivative (grey labels). The final red numbers are the gradients with respect to the parameters.

Worked example: forward and backward pass by hand

Forward (left to right):

$$wx = 2 \times 1.5 = 3, \qquad z = 3 + (-1) = 2,$$
$$a = \sigma(2) = \frac{1}{1+e^{-2}} = \frac{1}{1.1353} = 0.8808, \qquad e = 0.8808 - 1 = -0.1192,$$
$$L = (-0.1192)^2 = 0.01421.$$

Backward (right to left), each step = upstream gradient $\times$ local derivative:

$$\frac{\partial L}{\partial L} = 1 \qquad \text{(start)}$$
$$\frac{\partial L}{\partial e} = 1 \times 2e = 2(-0.1192) = -0.2384 \qquad \text{(local: } \tfrac{d}{de}e^2 = 2e\text{)}$$
$$\frac{\partial L}{\partial a} = -0.2384 \times 1 = -0.2384 \qquad \text{(local: } \frac{\partial(a-y)}{\partial a} = 1\text{)}$$
$$\frac{\partial L}{\partial z} = -0.2384 \times a(1-a) = -0.2384 \times 0.1050 = -0.02503 \qquad \text{(local: } \sigma'(z)\text{)}$$
$$\frac{\partial L}{\partial b} = -0.02503 \times 1 = -0.02503 \qquad \text{(local: } \frac{\partial z}{\partial b} = 1\text{)}$$
$$\frac{\partial L}{\partial w} = -0.02503 \times x = -0.02503 \times 1.5 = -0.03755 \qquad \text{(local: } \frac{\partial(wx)}{\partial w} = x\text{)}$$

Both gradients are negative: increasing w or b would decrease the loss (push a closer to the target 1). A gradient step with $\eta = 1$ gives $w \leftarrow 2 + 0.0375$, $b \leftarrow -1 + 0.0250$. A centred-difference check gives -0.037547 and -0.025031, matching to every digit shown.

Two things to notice. First, the backward pass reused forward values (e, a, x): this is why frameworks store activations during training (and why training needs more memory than inference). Second, the quantity $\frac{\partial L}{\partial z}$ was computed once and then shared by both $\frac{\partial L}{\partial w}$ and $\frac{\partial L}{\partial b}$. In a big network these shared intermediate results are what make backpropagation efficient: one backward sweep gives all million gradients, instead of a million separate forward passes for numerical differences.

3.8 Taylor expansions, the Hessian and curvature

Intuition: zooming in on a curve

Zoom far enough into any smooth curve and it looks like a straight line ((3.2)). Zoom out a little and the line is no longer good enough; the curve bends. The best *parabola* through the point captures that bend. The Taylor expansion is this sequence of ever-better polynomial approximations: constant, line, parabola, cubic… each adding one more derivative's worth of information.

Key equation: Taylor expansion around x_0

$$f(x_0 + \Delta) = f(x_0) + f'(x_0)\Delta + \frac{1}{2}f''(x_0)\Delta^2 + \frac{1}{6}f'''(x_0)\Delta^3 + \cdots = \sum_{k=0}^{\infty} \frac{f^{(k)}(x_0)}{k!}\Delta^k, \qquad (3.9)$$

where f'' is the derivative of f' (the **second derivative**), and $k! = 1 \cdot 2 \cdots k$.

Where the coefficients come from. Suppose $f(x_0 + \Delta) = c_0 + c_1\Delta + c_2\Delta^2 + c_3\Delta^3 + \ldots$. Set $\Delta = 0$: $c_0 = f(x_0)$. Differentiate once in Δ: $f'(x_0 + \Delta) = c_1 + 2c_2\Delta + 3c_3\Delta^2 + \ldots$; at $\Delta = 0$, $c_1 = f'(x_0)$. Differentiate again: $f'' = 2c_2 + 6c_3\Delta + \ldots$, so $c_2 = f''(x_0)/2$. Again: $c_3 = f'''(x_0)/6$. The pattern is $c_k = f^{(k)}(x_0)/k!$.

Worked example: Taylor series of e^x around 0

Every derivative of e^x is e^x, equal to 1 at 0. So $e^\Delta = 1 + \Delta + \frac{\Delta^2}{2} + \frac{\Delta^3}{6} + \ldots$. At $\Delta = 0.5$: true $e^{0.5} = 1.6487$; first order 1.5 (error 0.149); second order 1.625 (error 0.024). Substituting $\Delta = j\varphi$ and separating the real terms ($1 - \varphi^2/2 + \cdots = \cos\varphi$) from the imaginary ones ($\varphi - \varphi^3/6 + \cdots = \sin\varphi$) proves Euler's formula $e^{j\varphi} = \cos\varphi + j\sin\varphi$ used in Chapter 2.

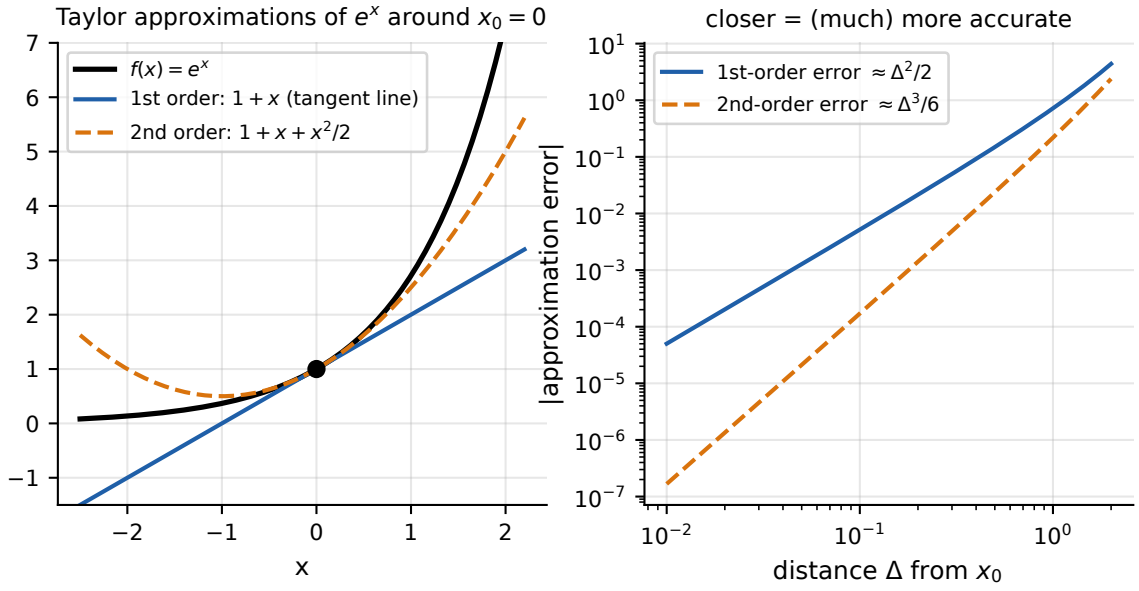


Figure 3.6: Left: e^x (black) with its first-order (tangent line) and second-order (parabola) Taylor approximations around 0. Both are excellent near 0 and drift apart further away; the parabola stays close for longer. Right: approximation error versus distance Δ on log–log axes. The lines have slopes 2 and 3: halving the distance cuts the first-order error by 4 and the second-order error by 8.

3.8.1 Second derivative = curvature

$f''(x)$ is the rate of change of the slope. If $f'' > 0$ the slope increases as you move right: the curve bends upwards like a bowl (*convex*); if $f'' < 0$, like a dome. At a point where $f' = 0$, $f'' > 0$ means a minimum and $f'' < 0$ a maximum.

For functions of many variables, the second derivatives form the **Hessian** matrix $\mathbf{H}$ with entries $H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$ (differentiate with respect to x_j, then x_i; for smooth functions the order does not matter, so $\mathbf{H}$ is symmetric). The second-order Taylor expansion becomes

$$f(\mathbf{x} + \Delta\mathbf{x}) \approx f(\mathbf{x}) + \nabla f \cdot \Delta\mathbf{x} + \tfrac{1}{2}\Delta\mathbf{x}^\mathsf{T}\mathbf{H}\,\Delta\mathbf{x}. \tag{3.10}$$

For the bowl of Fig. 3.4, $f = \frac{1}{2}x^2 + \frac{3}{2}y^2 + \frac{1}{2}xy$: $\frac{\partial f}{\partial x} = x + \frac{1}{2}y$, $\frac{\partial f}{\partial y} = 3y + \frac{1}{2}x$, so $\mathbf{H} = \left[\begin{smallmatrix} 1 & 0.5 \\ 0.5 & 3 \end{smallmatrix}\right]$. Its eigenvalues (Chapter 2) are about 0.88 and 3.12: the curvature along the steepest valley direction is about 3.5 times that along the shallowest.

3.8.2 Curvature limits the step size

Consider the one-dimensional loss $L(w) = \frac{1}{2}hw^2$ with curvature $L'' = h > 0$. Then $L'(w) = hw$ and a gradient step gives

$$w \leftarrow w - \eta h w = (1 - \eta h)\, w.$$

Each step multiplies w by the factor $(1 - \eta h)$. If $0 < \eta < 1/h$ the factor is between 0 and 1: smooth convergence. If $1/h < \eta < 2/h$ it is between -1 and 0: w flips sign each step but still shrinks (zig-zag). If $\eta > 2/h$ then $|1 - \eta h| > 1$ and w *grows* — divergence. The ideal $\eta = 1/h$ jumps to the minimum in one step (this is Newton's method).

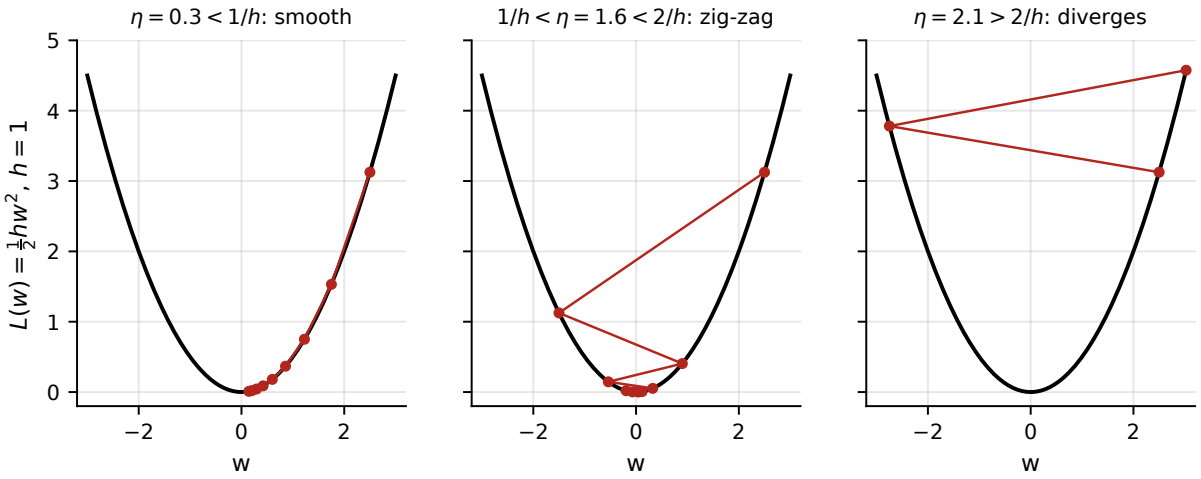


Figure 3.7: Gradient descent on $L = \frac{1}{2}w^2$ (curvature $h = 1$) from $w = 2.5$. Left: $\eta = 0.3$ glides down. Middle: $\eta = 1.6$ overshoots and zig-zags but converges (factor -0.6 per step). Right: $\eta = 2.1$ overshoots more each time and diverges (factor -1.1).

In many dimensions the same analysis applies along each Hessian eigenvector: the step must satisfy $\eta < 2/\lambda_{\max}$ for stability (for our bowl, $2/3.12 \approx 0.64$), but the progress along the flattest direction is governed by $\eta\lambda_{\min}$. When $\lambda_{\max}/\lambda_{\min}$ (the condition number) is large, gradient descent zig-zags across a narrow valley while crawling along it. Momentum, Adam and normalisation layers (Part II) are all, in part, remedies for this.

3.9 Matrix calculus essentials

Part II derives backpropagation for whole layers at once. That needs only a few results, each obtained by writing the expression with indices and applying the ordinary rules. Our convention: the gradient of a scalar with respect to a vector or matrix has the *same shape* as that vector or matrix.

1. $\nabla_{\mathbf{w}}(\mathbf{w}^{\mathsf{T}}\mathbf{x}) = \mathbf{x}$. $\mathbf{w}^{\mathsf{T}}\mathbf{x} = \sum_k w_k x_k$. Differentiating with respect to one component w_i, only the $k = i$ term contains w_i, giving x_i. Stacking for all i: the gradient is $\mathbf{x}$. (Exactly like $\frac{\mathrm{d}}{\mathrm{d}w}(wx) = x$ in one dimension.)

2. $\nabla_{\mathbf{w}} \|\mathbf{w}\|^2 = 2\mathbf{w}$. $\|\mathbf{w}\|^2 = \sum_k w_k^2$; $\frac{\partial}{\partial w_i}$ gives $2w_i$. This is the gradient of L2 weight decay.

3. $\nabla_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = 2\mathbf{X}^\mathsf{T}(\mathbf{X}\mathbf{w} - \mathbf{y})$. Let $\mathbf{r} = \mathbf{X}\mathbf{w} - \mathbf{y}$ (the residual), with $r_n = \sum_k X_{nk} w_k - y_n$. Then $\frac{\partial r_n}{\partial w_i} = X_{ni}$, i.e. the Jacobian $\frac{\partial \mathbf{r}}{\partial \mathbf{w}} = \mathbf{X}$ (result for linear maps above). By the chain rule (sum over paths, through every r_n):

$$\frac{\partial}{\partial w_i} \sum_n r_n^2 = \sum_n 2 r_n \frac{\partial r_n}{\partial w_i} = 2 \sum_n X_{ni} r_n = 2(\mathbf{X}^\mathsf{T}\mathbf{r})_i.$$

Setting this to zero gives the least-squares solution $\mathbf{X}^\mathsf{T}\mathbf{X}\mathbf{w} = \mathbf{X}^\mathsf{T}\mathbf{y}$ (the "normal equations").

4. The layer gradient: $\nabla_{\mathbf{W}} L = \mathbf{X}^\mathsf{T} \nabla_{\mathbf{Z}} L$ for $\mathbf{Z} = \mathbf{X}\mathbf{W}$. This is the workhorse of backpropagation. Let $\mathbf{X} \in \mathbb{R}^{N \times d}$ (batch of inputs), $\mathbf{W} \in \mathbb{R}^{d \times h}$, $\mathbf{Z} = \mathbf{X}\mathbf{W} \in \mathbb{R}^{N \times h}$, and suppose the loss L depends on $\mathbf{Z}$, with known upstream gradient $\mathbf{G} = \nabla_{\mathbf{Z}} L$ (so $G_{nj} = \frac{\partial L}{\partial Z_{nj}}$). With indices, $Z_{nj} = \sum_k X_{nk} W_{kj}$. The weight W_{ij} appears in Z_{nj} for every example n (column j only), with $\frac{\partial Z_{nj}}{\partial W_{ij}} = X_{ni}$. Sum over paths:

$$\frac{\partial L}{\partial W_{ij}} = \sum_{n=1}^{N} \frac{\partial L}{\partial Z_{nj}} \frac{\partial Z_{nj}}{\partial W_{ij}} = \sum_n G_{nj} X_{ni} = \sum_n (\mathbf{X}^\mathsf{T})_{in} G_{nj} = (\mathbf{X}^\mathsf{T}\mathbf{G})_{ij}.$$

Key equation: backprop through a linear layer

$$\mathbf{Z} = \mathbf{X}\mathbf{W} \quad \Longrightarrow \quad \nabla_{\mathbf{W}} L = \mathbf{X}^\mathsf{T} \nabla_{\mathbf{Z}} L, \qquad \nabla_{\mathbf{X}} L = \nabla_{\mathbf{Z}} L \, \mathbf{W}^\mathsf{T}. \tag{3.11}$$

The second formula follows the same way: X_{nk} appears in Z_{nj} for every j with derivative W_{kj}, so $\frac{\partial L}{\partial X_{nk}} = \sum_j G_{nj} W_{kj} = (\mathbf{G}\mathbf{W}^\mathsf{T})_{nk}$. **Shape check**, the easiest way to remember them: $\nabla_{\mathbf{W}} L$ must be (d, h); $\mathbf{X}^\mathsf{T}$ is (d, N) and $\mathbf{G}$ is (N, h), so $\mathbf{X}^\mathsf{T}\mathbf{G}$ is (d, h) ✓. $\nabla_{\mathbf{X}} L$ must be (N, d); $\mathbf{G}\mathbf{W}^\mathsf{T}$ is $(N, h)(h, d)$ ✓. The sum over n in $\mathbf{X}^\mathsf{T}\mathbf{G}$ means the weight gradient automatically *adds up the contributions of every example in the batch.*

Worked example: layer gradient with numbers

$\mathbf{X} = \left[\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right]$ (two examples, two features), upstream $\mathbf{G} = \left[\begin{smallmatrix} 1 & 0 \\ 0.5 & -1 \end{smallmatrix}\right]$. Then $\nabla_{\mathbf{W}} L = \mathbf{X}^\mathsf{T}\mathbf{G} = \left[\begin{smallmatrix} 1 & 3 \\ 2 & 4 \end{smallmatrix}\right] \left[\begin{smallmatrix} 1 & 0 \\ 0.5 & -1 \end{smallmatrix}\right] = \left[\begin{smallmatrix} 1{+}1.5 & 0{-}3 \\ 2{+}2 & 0{-}4 \end{smallmatrix}\right] = \left[\begin{smallmatrix} 2.5 & -3 \\ 4 & -4 \end{smallmatrix}\right]$. Entry $(1,1)$ is $X_{11}G_{11} + X_{21}G_{21} = 1 \cdot 1 + 3 \cdot 0.5$: feature 1 of each example times that example's upstream gradient for neuron 1, summed over the batch.

3.10 A note on complex variables (Wirtinger derivatives)

In wireless we often minimise a real loss of complex parameters, for example the squared error $L(h) = |y - hx|^2$ when estimating a channel gain h from a known pilot x and received sample y. A real-valued function of a complex variable is never differentiable in the strict complex sense (except when constant), but we do not need that: we only need to know which way to move $h = h_R + jh_I$ to decrease L. Treat L as a real function of the two real numbers h_R, h_I; the steepest-ascent direction in the complex plane is $\frac{\partial L}{\partial h_R} + j\frac{\partial L}{\partial h_I}$.

The **Wirtinger** shortcut computes this without splitting into real and imaginary parts. Define

$$\frac{\partial}{\partial h^*} = \frac{1}{2}\left(\frac{\partial}{\partial h_R} + j\frac{\partial}{\partial h_I}\right),$$

and use the rule: *differentiate with respect to h^* treating h as a constant*. Then the steepest-ascent direction is $2\,\frac{\partial L}{\partial h^*}$.

Worked example: gradient of $|y - hx|^2$

Write $L = (y - hx)(y - hx)^* = (y - hx)(y^* - h^* x^*)$. Treating h as constant, only the second factor contains h^*, with derivative $-x^*$:

$$\frac{\partial L}{\partial h^*} = (y - hx)(-x^*) = -x^*(y - hx) = -x^* e, \qquad e = y - hx.$$

Gradient descent: $h \leftarrow h - \mu \cdot 2\frac{\partial L}{\partial h^*} = h + 2\mu x^* e$. This is exactly the **LMS** update (with step 2μ absorbed into μ, and $\mathbf{w}^{\mathsf{H}}\mathbf{x}$ in place of hx for a multi-tap filter). Setting $\frac{\partial L}{\partial h^*} = 0$ gives $e = 0$, i.e. $h = y/x$. Numbers: $x = 1 + 2j$, $y = 3 - j$, $h = 0.5 + 0.5j$. Then $hx = 0.5 + j + 0.5j - 1 = -0.5 + 1.5j$, $e = 3.5 - 2.5j$, and $2\frac{\partial L}{\partial h^*} = -2(1 - 2j)(3.5 - 2.5j) = -2(3.5 - 2.5j - 7j - 5) = -2(-1.5 - 9.5j) = 3 + 19j$. A numerical check of $\frac{\partial L}{\partial h_R} + j\frac{\partial L}{\partial h_I}$ gives $3.000 + 19.000j$. ✓

Wireless connection: training on complex signals

PyTorch supports complex tensors and, for a real loss, its autograd returns the conjugate Wirtinger gradient scaled so that `h -= lr * h.grad` descends, exactly as above. Whether you use complex layers or split I and Q into two real channels, the result is the same gradient — the Wirtinger calculus is only bookkeeping. Part V uses this when networks learn channel estimates, precoders and constellations end-to-end.

Wireless connection: the derivative of capacity

The AWGN capacity (Chapter 4) is $C = \log_2(1 + \mathrm{SNR})$ bits per channel use. Since $\log_2 u = \ln u / \ln 2$, the chain rule gives $\frac{\mathrm{dC}}{\mathrm{dSNR}} = \frac{1}{(1+\mathrm{SNR})\ln 2}$. At SNR $= 1$ (0 dB) one extra unit of SNR buys 0.72 bit; at SNR $= 10$ (10 dB) only 0.13 bit. This diminishing return — the slope falls like $1/\mathrm{SNR}$ — is why "water-filling" power allocation over MIMO eigen-channels (Chapter 2) gives more power to good channels only up to a point, and it is derived by setting exactly such derivatives equal.

Chapter recap

- Derivative = instantaneous slope $= \lim_{h\to 0}[f(x+h) - f(x)]/h$; locally $f(x+h) \approx f(x) + f'(x)h$.
- Rules: power nx^{n-1}, $e^x \to e^x$, $\ln x \to 1/x$, product $f'g + fg'$, quotient $(f'g - fg')/g^2$; $\sigma' = \sigma(1 - \sigma)$.
- Centred differences have $O(h^2)$ error (e.g. x^2 at 3 gives exactly 6.0); used for gradient checking.
- Gradient ∇f = vector of partial derivatives; it points to steepest ascent (Cauchy–Schwarz), is perpendicular to contours; gradient descent steps along $-\nabla f$.
- Chain rule: rates multiply along a path and add across paths; for vector functions, Jacobians multiply.
- Backprop = forward pass storing values, backward pass multiplying upstream gradient by local derivatives.

- Taylor: $f(x_0+\Delta) \approx f + f'\Delta + \frac{1}{2}f''\Delta^2$; the Hessian's eigenvalues are curvatures; stability needs $\eta < 2/\lambda_{\max}$.
- Matrix calculus: $\nabla_{\mathbf{w}} \mathbf{w}^\top \mathbf{x} = \mathbf{x}$, $\nabla \|\mathbf{w}\|^2 = 2\mathbf{w}$, $\mathbf{Z} = \mathbf{X}\mathbf{W} \Rightarrow \nabla_{\mathbf{W}} L = \mathbf{X}^\top \nabla_{\mathbf{Z}} L$, $\nabla_{\mathbf{X}} L = \nabla_{\mathbf{Z}} L\, \mathbf{W}^\top$.
- Complex: descend along $-\frac{\partial L}{\partial h^*}$; for $|y - hx|^2$ this gives the LMS update $h \leftarrow h + \mu x^* e$.

Exercises

1. Use the limit definition to find the derivative of $f(x) = 3x^2 - x$, and evaluate it at $x = 2$.
2. Differentiate: (a) $x^3 e^x$; (b) $\ln(1+x^2)$; (c) $\tanh x = \dfrac{e^x - e^{-x}}{e^x + e^{-x}}$, showing that $\tanh'(x) = 1 - \tanh^2 x$.
3. Estimate the derivative of $f(x) = x^3$ at $x = 2$ with forward and centred differences, $h = 0.01$. Compare the errors with the predictions $\frac{1}{2}f''h$ and $\frac{1}{6}f'''h^2$.
4. For $f(x,y) = x^2 + xy + 2y^2$, compute ∇f at $(1,1)$. What is the rate of increase per unit step in the direction $[1,0]$? In the direction of the gradient? Give a direction in which f does not change.
5. Use the chain rule to differentiate $L = (\sigma(wx) - y)^2$ with respect to w. Evaluate at $w = 0$, $x = 2$, $y = 1$.
6. Draw the computational graph of $f = (a+b)\cdot c$ with $a = 1$, $b = 2$, $c = -3$. Run the forward and backward passes to find $\frac{\partial f}{\partial a}$, $\frac{\partial f}{\partial b}$, $\frac{\partial f}{\partial c}$.
7. A loss has Hessian eigenvalues 0.1 and 50. What is the largest stable learning rate for gradient descent? With that rate, about how many steps does it take to shrink the error along the flat direction by a factor $e \approx 2.7$?
8. For $\mathbf{Z} = \mathbf{X}\mathbf{W}$ with $\mathbf{X}$ of shape $(32, 10)$ and $\mathbf{W}$ of shape $(10, 4)$, give the shapes of $\nabla_{\mathbf{Z}} L$, $\nabla_{\mathbf{W}} L$, $\nabla_{\mathbf{X}} L$ and the formulas.

Answers

1. $\frac{3(x+h)^2 - (x+h) - 3x^2 + x}{h} = \frac{6xh + 3h^2 - h}{h} = 6x + 3h - 1 \to 6x - 1$. At $x = 2$: 11.
2. (a) Product rule: $3x^2 e^x + x^3 e^x = x^2 e^x (3 + x)$. (b) Chain rule with $u = 1 + x^2$: $\frac{1}{u} \cdot 2x = \frac{2x}{1+x^2}$. (c) Quotient rule with $f = e^x - e^{-x}$, $g = e^x + e^{-x}$, $f' = g$, $g' = f$: $\tanh' = \frac{g\cdot g - f\cdot f}{g^2} = 1 - \frac{f^2}{g^2} = 1 - \tanh^2 x$.
3. True value $3 \cdot 4 = 12$. Forward: $(2.01^3 - 8)/0.01 = (8.120601 - 8)/0.01 = 12.0601$, error $0.0601 \approx \frac{1}{2}(12)(0.01) = 0.06$ (since $f'' = 6x = 12$). Centred: $(8.120601 - 7.880599)/0.02 = 12.0001$, error $0.0001 = \frac{1}{6}(6)(0.01)^2$ (since $f''' = 6$).
4. $\nabla f = [2x + y,\ x + 4y] = [3, 5]$ at $(1,1)$. Along $[1,0]$: $\nabla f \cdot [1,0] = 3$. Along the gradient: $\|\nabla f\| = \sqrt{34} \approx 5.83$ (the maximum). Perpendicular to the gradient, e.g. $[5, -3]/\sqrt{34}$: rate $15 - 15 = 0$.
5. $\frac{\mathrm{d}L}{\mathrm{d}w} = 2(\sigma(wx) - y)\cdot \sigma'(wx) \cdot x$. At $w = 0$: $\sigma(0) = 0.5$, $\sigma'(0) = 0.25$, so $2(0.5 - 1)(0.25)(2) = -0.5$.

6. Forward: $s = a + b = 3$, $f = s \cdot c = -9$. Backward: $\frac{\partial f}{\partial s} = c = -3$, $\frac{\partial f}{\partial c} = s = 3$; the sum node passes gradients unchanged, so $\frac{\partial f}{\partial a} = \frac{\partial f}{\partial b} = -3$.
7. $\eta < 2/50 = 0.04$. Along the flat direction the error is multiplied by $1 - \eta\lambda = 1 - 0.04 \cdot 0.1 = 0.996$ per step; shrinking by e needs about $1/0.004 = 250$ steps — the ill-conditioning problem.
8. $\nabla_{\mathbf{Z}} L$: $(32, 4)$. $\nabla_{\mathbf{W}} L = \mathbf{X}^{\mathsf{T}} \nabla_{\mathbf{Z}} L$: $(10, 32)(32, 4) \to (10, 4)$. $\nabla_{\mathbf{X}} L = \nabla_{\mathbf{Z}} L\, \mathbf{W}^{\mathsf{T}}$: $(32, 4)(4, 10) \to (32, 10)$.

Self-test quiz

1. What is the derivative of x^2 at $x = 3$, and what does it mean? *6; output changes about 6 times as fast as input.*
2. Why is the centred difference more accurate than the forward one? *Even Taylor terms cancel: error $O(h^2)$ not $O(h)$.*
3. What is ∇f for $f(x, y) = x^2 y + 3y$ at $(2, 1)$? $[4, 7]$.
4. In which direction does f decrease fastest? $-\nabla f$.
5. $y = (3x + 1)^2$. What is dy/dx at $x = 1$? *24.*
6. What is $\nabla_{\mathbf{W}} L$ for $\mathbf{Z} = \mathbf{X}\mathbf{W}$? $\mathbf{X}^{\mathsf{T}} \nabla_{\mathbf{Z}} L$.

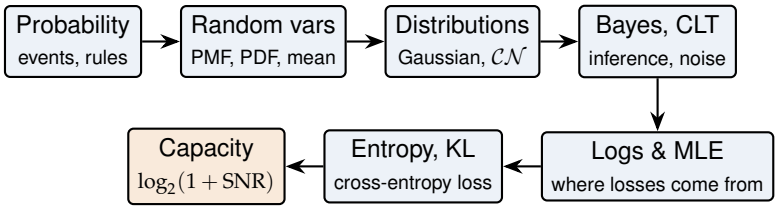
Further reading

- G. Strang, *Calculus*, 3rd ed., Wellesley–Cambridge Press, 2017 (earlier editions are freely available from MIT OpenCourseWare).
- 3Blue1Brown (G. Sanderson), *Essence of Calculus* video series — animated intuition for derivatives, the chain rule and Taylor series.
- K. B. Petersen and M. S. Pedersen, *The Matrix Cookbook*, Technical University of Denmark, 2012 — a reference table of matrix derivatives.
- T. Parr and J. Howard, "The Matrix Calculus You Need for Deep Learning", 2018, arXiv preprint 1802.01528.
- S. Haykin, *Adaptive Filter Theory*, 5th ed., Pearson, 2014 — LMS and complex gradients (Wirtinger calculus) from the signal-processing side.

Chapter 4

Probability, Statistics and Information

A neural network never knows anything for certain. A classifier does not say "this is a cat"; it says "cat with probability 0.92". A wireless receiver does not see the transmitted symbol; it sees the symbol plus random noise. Training data are a random sample from a much larger world, and the training procedure itself (random initialisation, random mini-batches, dropout) is random. The language for all of this is **probability**. On top of it sit **statistics** (learning about the world from samples) and **information theory** (measuring surprise and uncertainty in bits). These three subjects explain where loss functions come from (cross-entropy is not an arbitrary choice; it falls out of maximum likelihood), why we normalise inputs and how to initialise weights, why noise in a radio receiver is Gaussian, and the ultimate limit on how many bits per second any channel — or any learned autoencoder transceiver — can carry.



4.1 Events and the rules of probability

Intuition: probability as long-run frequency, or as degree of belief

Roll a fair die many times. About one roll in six shows a 4. The **probability** of rolling a 4 is that long-run fraction, $1/6$. Probability is also used for one-off uncertain statements ("the probability that this received symbol was a 1 is 0.8"); there it measures a degree of belief. Both readings obey the same simple rules, so we never need to choose.

Definition: sample space, event, probability

The **sample space** Ω (Greek "omega") is the set of all possible outcomes of an experiment; for a die $\Omega = \{1,2,3,4,5,6\}$. An **event** is a set of outcomes, e.g. $A = \{\text{even}\} = \{2,4,6\}$. A **probability** P assigns to each event a number obeying three **axioms**:

1. $P(A) \geq 0$ for every event (no negative probabilities);
2. $P(\Omega) = 1$ (something certainly happens);
3. if A and B cannot happen together (*disjoint*), $P(A \text{ or } B) = P(A) + P(B)$.

Everything else follows. For a fair die each outcome has probability $1/6$, so $P(\text{even}) = 3/6 = 0.5$. The **complement** "not A" has $P(\text{not } A) = 1 - P(A)$, because A and not-A are disjoint and together make Ω. For events that *can* overlap, $P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$: adding the two counts the overlap twice, so we subtract it once.

4.2 Random variables and their distributions

Definition: random variable

A **random variable** (written with a capital letter, X) is a number whose value depends on the outcome of a random experiment: the face of a die, the noise voltage at a sampling instant, the pixel value of a randomly chosen image. A specific value it takes is written in lower case, x.

4.2.1 Discrete variables: the PMF

If X takes a list of separate values, its **probability mass function** (PMF) is $p(x) = P(X = x)$. The values are non-negative and sum to 1: $\sum_x p(x) = 1$. For a fair die, $p(x) = 1/6$ for $x = 1, \dots, 6$.

4.2.2 Continuous variables: the PDF and CDF

Intuition: density is probability per unit length

The noise voltage can be any real number. The probability that it is *exactly* $0.31415\ldots$ volts is zero — there are infinitely many possible values. What makes sense is the probability of an *interval*. A **probability density** $p(x)$ is like the density of a metal rod: the mass of a short piece of length Δx near x is about $p(x)\Delta x$. Probability is the *area* under the density curve.

Definition: PDF and CDF

A continuous random variable has a **probability density function** (PDF) $p(x) \geq 0$ with

$$P(a \leq X \leq b) = \int_a^b p(x)\,dx, \qquad \int_{-\infty}^{\infty} p(x)\,dx = 1.$$

The integral sign $\int_a^b$ means "the area under the curve between a and b" (the continuous version of $\sum$). The **cumulative distribution function** (CDF) is $F(x) = P(X \leq x) = \int_{-\infty}^{x} p(t)\,dt$: it rises from 0 to 1, and its slope is the density, $F'(x) = p(x)$.

A density can exceed 1 (a very narrow peak must be tall to have area 1); a probability cannot.

4.3 Expectation and variance

Definition: expectation

The **expected value** (mean) of X is its probability-weighted average:

$$\mu = \mathbb{E}[X] = \sum_x x\,p(x) \quad \text{(discrete)}, \qquad \mathbb{E}[X] = \int x\,p(x)\,dx \quad \text{(continuous)}. \tag{4.1}$$

More generally $\mathbb{E}[g(X)] = \sum_x g(x)p(x)$ for any function g. Expectation is **linear**: $\mathbb{E}[aX + bY + c] = a\mathbb{E}[X] + b\mathbb{E}[Y] + c$, always (it is just a weighted sum).

Worked example: the fair die

$\mathbb{E}[X] = \frac{1}{6}(1+2+3+4+5+6) = \frac{21}{6} = 3.5$. No single roll gives 3.5; it is the long-run average — the balance point of the PMF.

Definition: variance and standard deviation

The **variance** measures spread: the average squared distance from the mean,

$$\sigma^2 = \mathrm{Var}(X) = \mathbb{E}\big[(X-\mu)^2\big] = \mathbb{E}[X^2] - \mu^2. \tag{4.2}$$

The **standard deviation** $\sigma = \sqrt{\mathrm{Var}(X)}$ has the same units as X.

Derivation of the shortcut: expand the square and use linearity: $\mathbb{E}[(X-\mu)^2] = \mathbb{E}[X^2 - 2\mu X + \mu^2] = \mathbb{E}[X^2] - 2\mu\mathbb{E}[X] + \mu^2 = \mathbb{E}[X^2] - 2\mu^2 + \mu^2 = \mathbb{E}[X^2] - \mu^2$. For the die, $\mathbb{E}[X^2] = \frac{1+4+9+16+25+36}{6} = \frac{91}{6}$, so Var $= \frac{91}{6} - 3.5^2 = 15.167 - 12.25 = 2.917$ and $\sigma \approx 1.708$. Scaling: $\mathrm{Var}(aX+b) = a^2\,\mathrm{Var}(X)$ (shifting does not change spread; scaling by a scales distances by a and squared distances by a^2).

Worked example: mean and variance of a data set

Data: $[2,4,4,4,5,5,7,9]$ ($n = 8$ values). Mean: $(2+4+4+4+5+5+7+9)/8 = 40/8 = 5$. Squared deviations from 5: $9,1,1,1,0,0,4,16$, summing to 32. Variance: $32/8 = 4$. Standard deviation: $\sqrt{4} = 2$. (Dividing by $n-1 = 7$ instead gives the "unbiased sample variance" 4.571; NumPy's `np.var` divides by n by default, PyTorch's `torch.var` by $n-1$ — a classic source of tiny mismatches.)

4.3.1 Standardisation

Subtracting the mean and dividing by the standard deviation,

$$x' = \frac{x-\mu}{\sigma}, \tag{4.3}$$

gives a variable with mean 0 and variance 1 (by the scaling rules: mean $(\mu-\mu)/\sigma = 0$, variance $\sigma^2/\sigma^2 = 1$). For MNIST handwritten digits, with pixel values scaled to $[0,1]$, the training-set mean is 0.1307 and the standard deviation 0.3081; a black pixel (0) becomes $(0-0.1307)/0.3081 = -0.424$ and a white pixel (1) becomes 2.822. Inputs on a common, centred scale make gradient descent far better conditioned (Chapter 3): no single feature dominates the curvature. Batch normalisation (Part II) applies the same idea inside the network.

4.4 The distributions you need

Bernoulli. One yes/no trial: $X = 1$ with probability p, $X = 0$ with probability $1-p$. $\mathbb{E}[X] = p$; $\mathbb{E}[X^2] = p$ so $\mathrm{Var}(X) = p - p^2 = p(1-p)$. A binary classifier's sigmoid output is the p of a Bernoulli; a received bit is a Bernoulli.

Categorical. One trial with K outcomes having probabilities $p_1, \ldots, p_K$, $\sum_k p_k = 1$. A softmax output layer produces exactly such a vector. The true label is usually written **one-hot**: e.g. class 2 of 3 is $[0,1,0]$.

Uniform. On $[a, b]$ the density is flat, $p(x) = 1/(b-a)$. Mean $(a+b)/2$, variance $(b-a)^2/12$ (so $U(-1,1)$ has variance $4/12 = 1/3$). Random phases of radio paths are uniform on $[0, 2\pi)$.

Gaussian (normal). The bell curve, with mean μ and variance σ^2:

Key equation: Gaussian density $\mathcal{N}(\mu, \sigma^2)$

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right). \tag{4.4}$$

Read it from the inside: $(x-\mu)^2/\sigma^2$ is the squared distance from the centre measured in standard deviations; the exponential of minus half of that decays rapidly; the constant in front makes the area equal to 1. About 68.3% of the probability lies within $\pm 1\sigma$ of the mean, 95.4% within $\pm 2\sigma$, 99.7% within $\pm 3\sigma$.

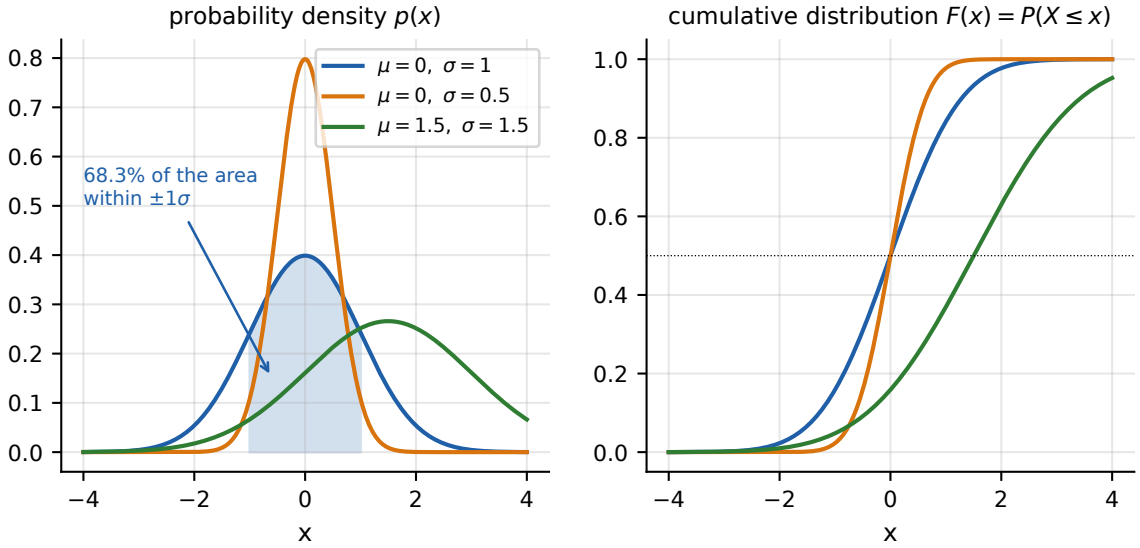


Figure 4.1: Left: Gaussian densities. μ shifts the bell, σ sets its width; the narrow one ($\sigma = 0.5$) is taller because its area must still be 1. Shaded: the 68.3% of probability within one standard deviation. Right: the corresponding CDFs, rising from 0 to 1; each crosses 0.5 at its mean, and narrow distributions rise more steeply.

Circularly-symmetric complex Gaussian $\mathcal{CN}(0, \sigma^2)$. Baseband noise has an in-phase and a quadrature part. We write $N = N_I + jN_Q$ with N_I and N_Q independent, each $\mathcal{N}(0, \sigma^2/2)$. The total power is $\mathbb{E}[|N|^2] = \mathbb{E}[N_I^2] + \mathbb{E}[N_Q^2] = \sigma^2/2 + \sigma^2/2 = \sigma^2$. Multiplying the two real densities (allowed because they are independent, Section 4.5):

$$p(n) = \frac{1}{\sqrt{\pi\sigma^2}} e^{-n_I^2/\sigma^2} \cdot \frac{1}{\sqrt{\pi\sigma^2}} e^{-n_Q^2/\sigma^2} = \frac{1}{\pi\sigma^2} \exp\left(-\frac{|n|^2}{\sigma^2}\right).$$

The density depends only on $|n|$, not on the angle: rotating the noise (multiplying by $e^{j\theta}$) does not change its distribution. That is what "circularly symmetric" means, and it is why a unitary matrix leaves noise statistics unchanged (Chapter 2).

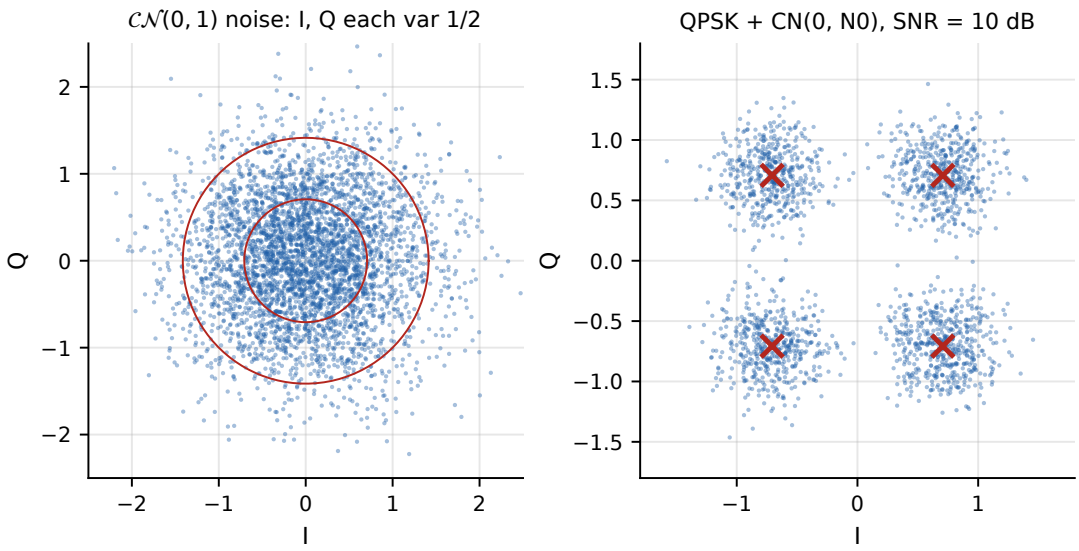


Figure 4.2: Left: 4000 samples of $\mathcal{CN}(0,1)$ noise. The cloud is round (circular symmetry); circles at one and two per-component standard deviations ($\sqrt{1/2}$ and $2\sqrt{1/2}$) are drawn in red. Right: QPSK symbols (red crosses, unit average energy) after adding $\mathcal{CN}(0, N_0)$ noise at SNR $= 1/N_0 = 10$ dB: each symbol becomes a small round cloud. A detector, classical or neural, must decide which cross each dot came from.

4.5 Joint, marginal and conditional probability

With two random variables we need their **joint** distribution $p(x,y) = P(X = x \text{ and } Y = y)$. From it:

- the **marginal** $p(x) = \sum_y p(x,y)$ (sum out what you do not care about);
- the **conditional** $p(y \mid x) = \dfrac{p(x,y)}{p(x)}$, read "probability of y given x": restrict attention to the cases where $X = x$ and renormalise.

Rearranging gives the **product rule** $p(x,y) = p(x)\,p(y \mid x)$. X and Y are **independent** if knowing one tells you nothing about the other: $p(y \mid x) = p(y)$, equivalently $p(x,y) = p(x)p(y)$. A neural classifier models a conditional distribution $p(\text{label} \mid \text{input})$; a channel is described by $p(y \mid x)$, the probability of receiving y when x was sent.

4.5.1 Bayes' rule

Intuition: updating beliefs with evidence

You start with a belief (the *prior*). You observe evidence. Bayes' rule tells you your new belief (the *posterior*): weight each hypothesis by how well it predicted the evidence, and renormalise. Hypotheses that predicted the evidence well gain probability; those that did not, lose it.

Writing the product rule both ways, $p(x)p(y \mid x) = p(y)p(x \mid y)$, and dividing by $p(y)$:

Key equation: Bayes' rule

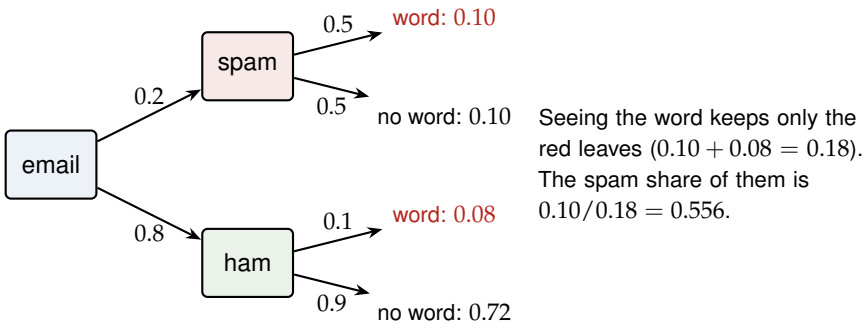
$$p(x \mid y) = \frac{p(y \mid x)\, p(x)}{p(y)}, \qquad p(y) = \sum_{x'} p(y \mid x')\, p(x'). \tag{4.5}$$

Worked example: a spam filter

20% of emails are spam. The word "winner" appears in 50% of spam and 10% of legitimate ("ham") emails. An email contains "winner". Is it spam?

$$\begin{aligned} P(\text{word}) &= P(\text{word} \mid \text{spam})P(\text{spam}) + P(\text{word} \mid \text{ham})P(\text{ham}) \\ &= 0.5 \cdot 0.2 + 0.1 \cdot 0.8 = 0.10 + 0.08 = 0.18, \\ P(\text{spam} \mid \text{word}) &= \frac{0.10}{0.18} \approx 0.556. \end{aligned}$$

The evidence raised the probability of spam from 20% to 55.6% — substantial, but far from certain, because ham is so much more common.



Wireless connection: the MAP detector and soft bits are Bayes' rule

A receiver observes $y = x + n$ with $x \in \{-1, +1\}$ (BPSK) and $n \sim \mathcal{N}(0, \sigma^2)$. Bayes gives $P(x = +1 \mid y) \propto p(y \mid +1)P(+1)$. With equal priors, the ratio of posteriors is $\exp(-(y-1)^2/2\sigma^2) / \exp(-(y+1)^2/2\sigma^2) = \exp(2y/\sigma^2)$ (expand the squares: the y^2 and 1 terms cancel, leaving $4y/2\sigma^2$). Its logarithm, the **log-likelihood ratio** $\text{LLR} = 2y/\sigma^2$, is the "soft bit" that LDPC and polar decoders consume. A neural demapper trained with cross-entropy learns to output exactly this posterior (Section 4.9).

4.6 Sums of random variables: the facts behind weight initialisation

For *any* X, Y: $\mathbb{E}[X+Y] = \mathbb{E}[X] + \mathbb{E}[Y]$. For *independent* X, Y, two more facts hold.

1. $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$. Discrete proof: $\mathbb{E}[XY] = \sum_{x,y} xy\, p(x)p(y) = \big(\sum_x x\, p(x)\big)\big(\sum_y y\, p(y)\big)$.

2. $\operatorname{Var}(X+Y) = \operatorname{Var}(X) + \operatorname{Var}(Y)$. Let $\tilde{X} = X - \mathbb{E}X$, $\tilde{Y} = Y - \mathbb{E}Y$ (centred, independent). Then $\operatorname{Var}(X+Y) = \mathbb{E}[(\tilde{X}+\tilde{Y})^2] = \mathbb{E}[\tilde{X}^2] + 2\mathbb{E}[\tilde{X}\tilde{Y}] + \mathbb{E}[\tilde{Y}^2]$, and by fact 1 the middle term is $2\mathbb{E}[\tilde{X}]\mathbb{E}[\tilde{Y}] = 0$. Variances of independent variables add (standard deviations do not!).

3. If also $\mathbb{E}X = \mathbb{E}Y = 0$: $\mathrm{Var}(XY) = \mathrm{Var}(X)\,\mathrm{Var}(Y)$. $\mathrm{Var}(XY) = \mathbb{E}[X^2Y^2] - (\mathbb{E}[XY])^2 = \mathbb{E}[X^2]\mathbb{E}[Y^2] - 0 = \mathrm{Var}(X)\,\mathrm{Var}(Y)$, using fact 1 for X^2, Y^2 (also independent) and $\mathbb{E}[X^2] = \mathrm{Var}(X)$ when the mean is zero.

Worked example: why weights are initialised with variance $1/n$

A neuron computes $z = \sum_{i=1}^{n} w_i x_i$ with n inputs. Suppose the w_i and x_i are all independent, zero-mean, with $\mathrm{Var}(w_i) = v_w$ and $\mathrm{Var}(x_i) = v_x$. By fact 3 each term has variance $v_w v_x$; by fact 2 the n terms add: $\mathrm{Var}(z) = n\,v_w v_x$. For $n = 784$ inputs and $v_w = 1$, the output variance is 784 times the input variance; after ten such layers it would be 784^{10}. Choosing $v_w = 1/n$ keeps $\mathrm{Var}(z) = v_x$, layer after layer. This is the core of LeCun/Xavier initialisation; He initialisation uses $2/n$ to compensate for ReLU zeroing half its inputs (Part II).

4.7 The law of large numbers and the central limit theorem

Law of large numbers. Average n independent samples with the same mean μ and variance σ^2: $\bar{X} = \frac{1}{n}\sum_i X_i$. Its mean is μ (linearity). Its variance is $\frac{1}{n^2} \cdot n\sigma^2 = \sigma^2/n$ (scaling rule, then fact 2). As n grows, the spread of the average shrinks like $\sigma/\sqrt{n}$: averages settle down to the true mean. This is why a mini-batch gradient is a reasonable estimate of the full gradient, and why quadrupling the batch size halves its noise.

Central limit theorem (CLT). More surprisingly, the *shape* of the distribution of a sum becomes Gaussian, whatever the shape of the individual terms (provided they are independent and have finite variance, none dominating):

Key equation: central limit theorem

If $X_1, \ldots, X_n$ are independent with mean μ and variance σ^2, then $\dfrac{\sum_i X_i - n\mu}{\sigma\sqrt{n}}$ is approximately $\mathcal{N}(0,1)$ for large n.

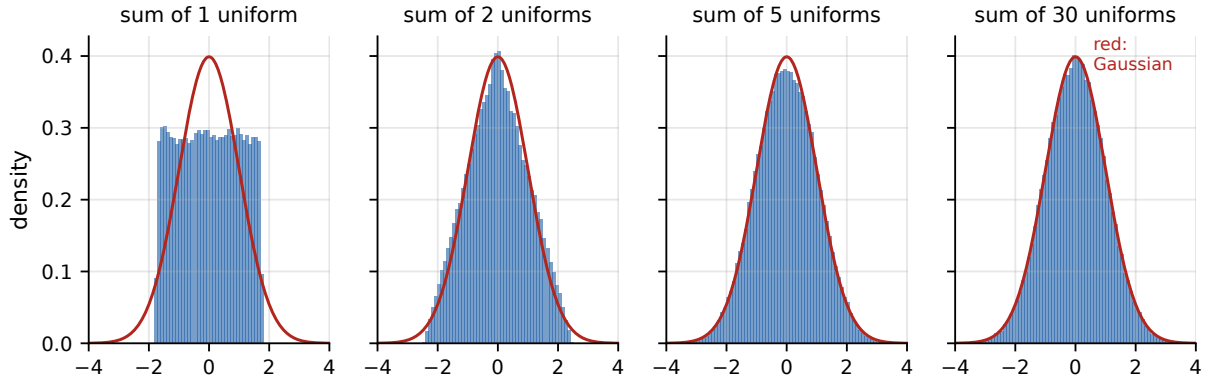


Figure 4.3: The central limit theorem in action: histograms of standardised sums of n independent uniform variables (100 000 trials each), against the $\mathcal{N}(0,1)$ density (red). One uniform is flat; the sum of two is a triangle; by five it is already nearly indistinguishable from a bell; by thirty it is a Gaussian. The terms themselves were nothing like Gaussian.

Wireless connection: why thermal noise and Rayleigh fading are Gaussian

Thermal noise is the sum of tiny voltage contributions from an astronomical number of electrons moving randomly; by the CLT it is Gaussian, which is why AWGN is the default noise model. *Rayleigh fading* arises when a signal reaches the receiver through many scattered paths, each with its own amplitude a_ℓ and random phase ϕ_ℓ: $h = \sum_\ell a_\ell e^{j\phi_\ell}$. Both the real and imaginary parts are sums of many independent terms, so h is approximately $\mathcal{CN}(0, \Omega)$, and its envelope $|h|$ follows a Rayleigh distribution (Fig. 4.4). When one strong line-of-sight path dominates, the CLT's "no single term dominates" condition fails, and the envelope becomes Rician instead.

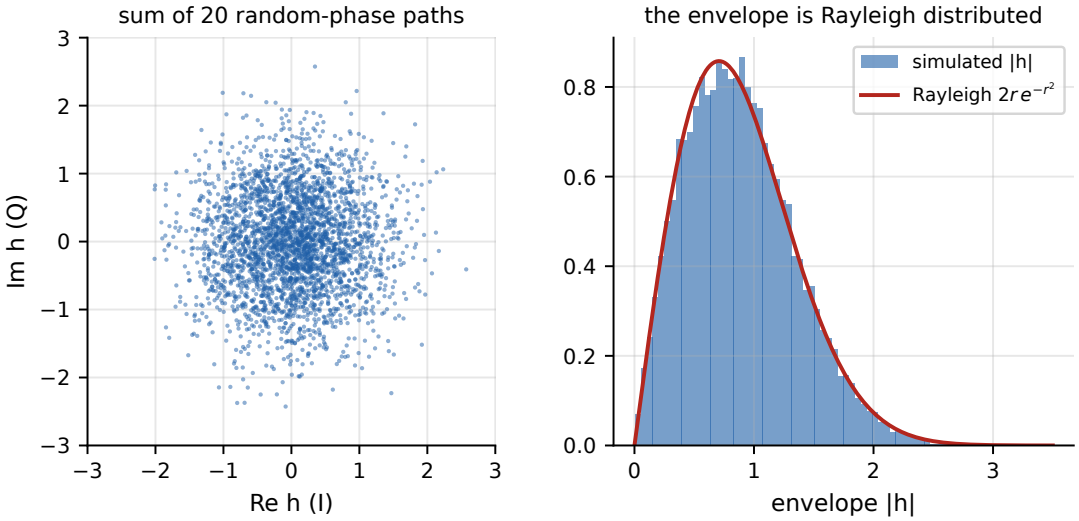


Figure 4.4: Our simulation of multipath: each channel coefficient is the sum of 20 paths with random amplitudes (uniform on $[0.5, 1.5]$) and uniformly random phases, normalised to unit power. Left: the coefficients form a round complex Gaussian cloud. Right: the histogram of $|h|$ matches the Rayleigh density $2re^{-r^2}$ (for $\mathbb{E}\,|h|^2 = 1$).

4.8 Exponentials and logarithms

The **logarithm** answers "what power?": $\log_b x = y$ means $b^y = x$. So $\log_{10} 1000 = 3$, $\log_2 2^5 = 5$. The natural log ln uses base $e \approx 2.718$. Rules (all from the exponent laws $b^u b^v = b^{u+v}$ and $(b^u)^k = b^{uk}$):

$$\log(ab) = \log a + \log b, \quad \log(a/b) = \log a - \log b, \quad \log(a^k) = k \log a, \quad \log_b x = \frac{\ln x}{\ln b}.$$

Logs turn products into sums, and they are *increasing* (bigger input, bigger output), so maximising $\log f$ is the same as maximising f. Decibels are logs: $\mathrm{SNR}_{\mathrm{dB}} = 10 \log_{10} \mathrm{SNR}$.

4.8.1 Why we work with log-probabilities

The probability of a data set of n independent examples is a product $\prod_i p_i$ (capital pi: "multiply together"). With $n = 10{,}000$ factors around 0.1 the product is 10^{-10000}, which underflows to zero in floating point. Its logarithm, $\sum_i \ln p_i \approx -23{,}000$, is perfectly representable. Sums are also much easier to differentiate than products.

4.8.2 The log-sum-exp trick

Softmax and many likelihoods need $\ln \sum_k e^{z_k}$. For $z = [1000, 1001, 1002]$, e^{1000} overflows to infinity. Let $m = \max_k z_k$. Then

$$\ln \sum_k e^{z_k} = \ln \left(e^m \sum_k e^{z_k - m} \right) = \ln e^m + \ln \sum_k e^{z_k - m} = m + \ln \sum_k e^{z_k - m}. \tag{4.6}$$

Now every exponent is ≤ 0, so nothing overflows, and the largest term is $e^0 = 1$, so the sum is at least 1 and its log cannot underflow. Example: $1002 + \ln(e^{-2} + e^{-1} + e^0) = 1002 + \ln(0.1353 + 0.3679 + 1) = 1002 + \ln 1.5032 = 1002.408$. Every deep learning library uses this inside its cross-entropy loss — one reason to feed raw scores ("logits") to `CrossEntropyLoss` rather than probabilities.

4.9 Maximum likelihood estimation

Intuition: choose the explanation that makes the data least surprising

You flip a coin 10 times and see 7 heads. What is the coin's probability of heads? Each candidate value p assigns some probability to what you saw. $p = 0.1$ makes 7 heads absurdly unlikely; $p = 0.7$ makes it quite likely. **Maximum likelihood** picks the parameter under which the observed data were most probable. Training a neural network with cross-entropy is exactly this, with the network's weights as the parameters.

Definition: likelihood

Given independent data $x_1, \ldots, x_n$ and a model $p(x; \theta)$ with parameters θ, the **likelihood** is $\mathcal{L}(\theta) = \prod_i p(x_i; \theta)$, the **log-likelihood** $\ell(\theta) = \sum_i \ln p(x_i; \theta)$, and the maximum likelihood estimate is $\hat{\theta} = \arg\max_\theta \ell(\theta)$. Equivalently we *minimise the negative log-likelihood* (NLL) — a loss.

Worked example: Bernoulli MLE

Data: k heads in n flips. $\ell(p) = k \ln p + (n - k) \ln(1 - p)$. Set the derivative to zero:

$$\frac{\mathrm{d}\ell}{\mathrm{d}p} = \frac{k}{p} - \frac{n-k}{1-p} = 0 \implies k(1-p) = (n-k)p \implies k = np \implies \hat{p} = \frac{k}{n}.$$

For 7 heads in 10, $\hat{p} = 0.7$: the intuitive answer, now justified. Note the NLL per flip, $-[y \ln p + (1 - y) \ln(1 - p)]$ for an outcome $y \in \{0, 1\}$, is precisely the **binary cross-entropy loss**.

Worked example: Gaussian MLE

Data $x_1, \ldots, x_n$ modelled as $\mathcal{N}(\mu, \sigma^2)$. Taking the log of (4.4) for each point and summing:

$$\ell(\mu, \sigma^2) = -\frac{n}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_i (x_i - \mu)^2.$$

Mean: only the second term contains μ; $\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2} \sum_i (x_i - \mu) = 0$ gives $\sum_i x_i = n\mu$, so $\hat{\mu} = \frac{1}{n} \sum_i x_i$, the sample average. *Variance:* write $v = \sigma^2$; $\frac{\partial \ell}{\partial v} = -\frac{n}{2v} + \frac{1}{2v^2} \sum_i (x_i - \mu)^2 = 0$; multiply by $2v^2$: $-nv + \sum_i (x_i - \mu)^2 = 0$, so $\hat{\sigma}^2 = \frac{1}{n} \sum_i (x_i - \hat{\mu})^2$. For $[2, 4, 4, 4, 5, 5, 7, 9]$: $\hat{\mu} = 5$, $\hat{\sigma}^2 = 4$.

Crucially, for fixed σ maximising the likelihood is the same as *minimising the sum of squared errors*: **mean-squared-error loss is maximum likelihood under Gaussian noise**.

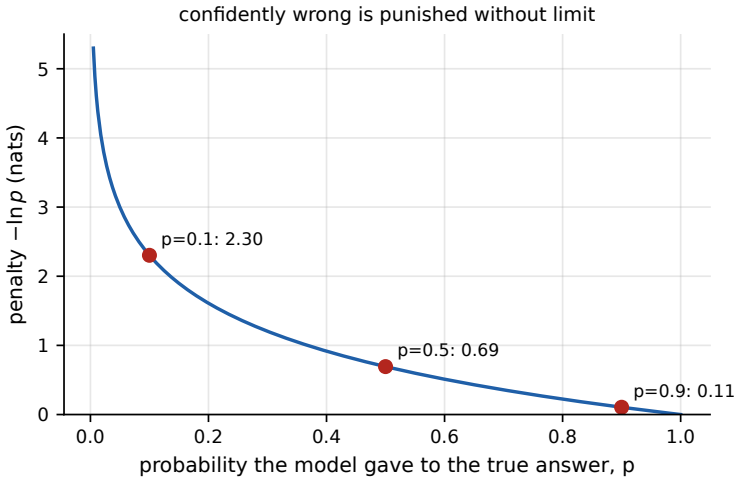


Figure 4.5: The per-example penalty $-\ln p$ that maximum likelihood (cross-entropy) imposes, as a function of the probability the model assigned to the correct answer. Confident and right ($p = 0.9$): penalty 0.11. Unsure ($p = 0.5$): 0.69. Confident and wrong ($p = 0.1$): 2.30, and it goes to infinity as $p \to 0$. This steep wall is what drives a classifier to fix its worst mistakes first.

4.10 Information theory

Intuition: information is surprise

"The sun rose this morning" tells you almost nothing; you expected it. "It snowed in Abu Dhabi" tells you a lot. Information measures *surprise*: rare events carry more of it. And the information of two independent events should add. The only function that turns "multiply probabilities" into "add information" is the logarithm.

Definition: self-information and entropy

The **self-information** of an outcome with probability p is $I = -\log_2 p$ bits (with ln the unit is nats; 1 nat $= 1/\ln 2 \approx 1.443$ bits). The **entropy** of a random variable is its average self-information — the average surprise, or uncertainty:

$$H(X) = -\sum_x p(x) \log_2 p(x) = \mathbb{E}[-\log_2 p(X)]. \tag{4.7}$$

A fair coin: $H = -2 \cdot \frac{1}{2} \log_2 \frac{1}{2} = 1$ bit. A fair die: $\log_2 6 \approx 2.585$ bits. A coin with $P(\text{heads}) = 0.9$: $H = -(0.9 \log_2 0.9 + 0.1 \log_2 0.1) = 0.137 + 0.332 = 0.469$ bit — less uncertain, less to learn from each flip. A certain outcome has $H = 0$. Among all distributions on K outcomes, the uniform one has the largest entropy, $\log_2 K$. Shannon's source coding theorem gives entropy an operational meaning: it is the minimum average number of bits per symbol needed to compress the source.

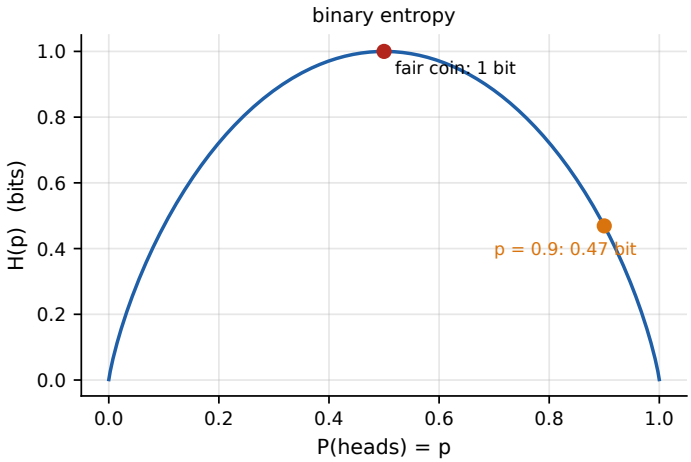


Figure 4.6: Binary entropy $H(p) = -p\log_2 p - (1-p)\log_2(1-p)$. It is 0 when the outcome is certain ($p = 0$ or 1), and peaks at 1 bit for a fair coin. A biased coin with $p = 0.9$ carries only 0.47 bit per flip.

4.10.1 Cross-entropy: the classification loss

Suppose the truth is distributed as p, but we encode or predict using a model q. The average surprise we experience is the **cross-entropy**

$$H(p, q) = -\sum_x p(x)\log q(x). \tag{4.8}$$

For a single training example with one-hot label (true class c), p puts all mass on c, so $H(p, q) = -\log q(c)$: exactly the negative log-likelihood of Fig. 4.5. If the true class is the first of three and the network outputs $q = [0.7, 0.2, 0.1]$, the loss is $-\ln 0.7 = 0.357$ nats. *Cross-entropy loss = NLL = maximum likelihood* — three names for one idea.

4.10.2 KL divergence: the extra cost of being wrong

Definition: Kullback–Leibler divergence

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_x p(x)\log\frac{p(x)}{q(x)} = H(p, q) - H(p). \tag{4.9}$$

It is the extra average surprise (extra bits) incurred by using q when the truth is p.

Since $H(p)$ does not depend on the model, minimising cross-entropy over q is the same as minimising the KL divergence from the data distribution to the model.

KL is never negative. We use the inequality $\ln u \le u - 1$ (the curve $\ln u$ lies below its tangent line at $u = 1$; Chapter 3). With natural logs,

$$-D_{\mathrm{KL}}(p\|q) = \sum_x p(x)\ln\frac{q(x)}{p(x)} \le \sum_x p(x)\left(\frac{q(x)}{p(x)} - 1\right) = \sum_x q(x) - \sum_x p(x) = 1 - 1 = 0.$$

So $D_{\mathrm{KL}} \ge 0$, with equality only when $q = p$ everywhere (where $\ln u = u - 1$, i.e. $u = 1$). The best possible model has zero extra cost; any mismatch costs bits. KL is *not* symmetric: $D(p\|q) \ne D(q\|p)$ in general, so it is not a distance.

Worked example: entropy, cross-entropy and KL by hand

$p = [0.5, 0.25, 0.25]$, $q = [\frac{1}{3}, \frac{1}{3}, \frac{1}{3}]$. $H(p) = 0.5(1) + 0.25(2) + 0.25(2) = 1.5$ bits. $H(p,q) = -\log_2 \frac{1}{3} = 1.585$ bits. $D(p\|q) = 1.585 - 1.5 = 0.085$ bit. The other way, $D(q\|p) = \frac{1}{3}\log_2 \frac{1/3}{1/2} + \frac{2}{3}\log_2 \frac{1/3}{1/4} = -0.195 + 0.277 = 0.082$ bit: close here, but not equal.

For two Gaussians the sum becomes an integral with a closed form (obtained by substituting (4.4) into the KL integral and using $\mathbb{E}_p[(X-\mu_2)^2] = \sigma_1^2 + (\mu_1 - \mu_2)^2$):

$$D_{\mathrm{KL}}\big(\mathcal{N}(\mu_1, \sigma_1^2) \,\|\, \mathcal{N}(\mu_2, \sigma_2^2)\big) = \ln \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2} \text{ nats.} \tag{4.10}$$

This formula is the regularisation term of the variational autoencoder (Part IV).

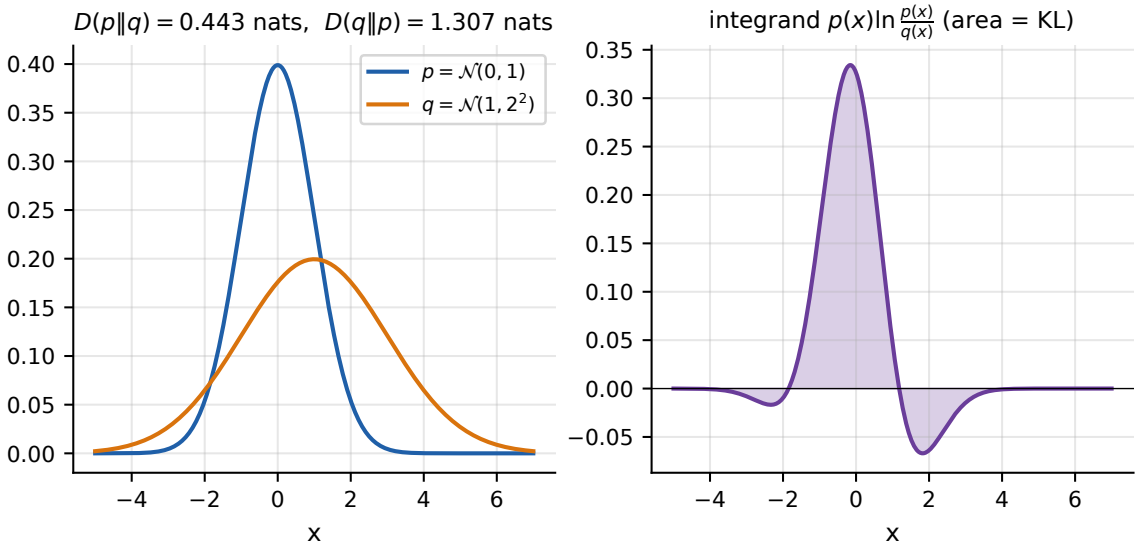


Figure 4.7: Left: $p = \mathcal{N}(0,1)$ and $q = \mathcal{N}(1, 2^2)$. By (4.10), $D(p\|q) = \ln 2 + \frac{1+1}{8} - \frac{1}{2} = 0.443$ nats but $D(q\|p) = \ln \frac{1}{2} + \frac{4+1}{2} - \frac{1}{2} = 1.307$ nats: describing the wide q with the narrow p is far more costly, because p assigns tiny probability to q's tails. Right: the integrand $p \ln(p/q)$. It is negative where $q > p$, but the total area is positive, as the proof guarantees.

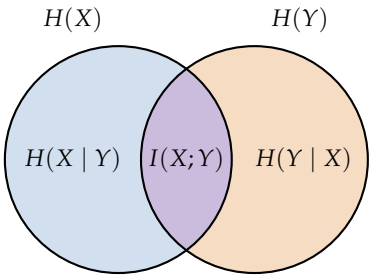
4.10.3 Mutual information

Definition: mutual information

How much does observing Y reduce our uncertainty about X?

$$I(X;Y) = H(X) - H(X \mid Y) = D_{\mathrm{KL}}\big(p(x,y) \,\|\, p(x)p(y)\big), \tag{4.11}$$

where $H(X \mid Y) = -\sum_{x,y} p(x,y) \log p(x \mid y)$ is the uncertainty remaining about X after seeing Y. It is symmetric, non-negative (it is a KL divergence), and zero exactly when X and Y are independent.

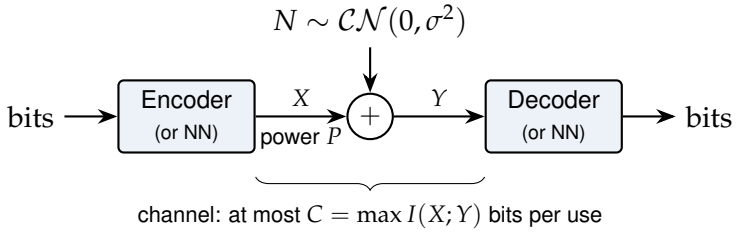


Each circle is the uncertainty in one variable. The overlap is what they share: the information about X that observing Y provides. For a channel, X is what was sent and Y what was received.

Worked example: a binary symmetric channel

Send a fair bit X; the channel flips it with probability 0.1. Then $H(X) = 1$ bit. Given Y, the bit is wrong with probability 0.1, so $H(X \mid Y) = H_b(0.1) = 0.469$ bit. Hence $I(X;Y) = 1 - 0.469 = 0.531$ bit per use: noise destroys almost half the information.

4.11 Channel capacity



Definition: capacity

The **capacity** of a channel is the largest mutual information between input and output over all allowed input distributions: $C = \max_{p(x)} I(X;Y)$. Shannon's channel coding theorem says that any rate below C can be achieved with arbitrarily small error probability (with long enough codes), and no rate above C can.

For the complex AWGN channel $Y = X + N$ with $N \sim \mathcal{CN}(0, \sigma^2)$ and input power $\mathbb{E}\,|X|^2 \le P$:

Key equation: AWGN capacity

$$C = \log_2\left(1 + \frac{P}{\sigma^2}\right) = \log_2(1 + \mathrm{SNR}) \text{ bits per complex channel use.} \tag{4.12}$$

Derivation sketch. For continuous variables, entropy is replaced by **differential entropy** $h(X) = -\int p \log p$. For a real Gaussian, $-\ln p(x) = \frac{1}{2}\ln(2\pi\sigma^2) + \frac{(x-\mu)^2}{2\sigma^2}$, whose expectation is $\frac{1}{2}\ln(2\pi\sigma^2) + \frac{1}{2} = \frac{1}{2}\ln(2\pi e\sigma^2)$ nats. A $\mathcal{CN}(0,\sigma^2)$ variable is two independent real Gaussians of variance $\sigma^2/2$, so $h = 2 \cdot \frac{1}{2}\log_2(2\pi e\,\sigma^2/2) = \log_2(\pi e\sigma^2)$ bits. Now $I(X;Y) = h(Y) - h(Y \mid X)$, and given X the only uncertainty in Y is the noise: $h(Y \mid X) = h(N) = \log_2(\pi e\sigma^2)$. The output has power $P + \sigma^2$ (variances of independent signals add), and among all distributions with a given power the Gaussian has the largest entropy (a fact we accept here), so $h(Y) \le \log_2(\pi e(P+\sigma^2))$, with equality when X itself is $\mathcal{CN}(0, P)$. Therefore

$$C = \log_2(\pi e(P+\sigma^2)) - \log_2(\pi e\sigma^2) = \log_2 \frac{P+\sigma^2}{\sigma^2} = \log_2(1 + \mathrm{SNR}).$$

Intuition: how many distinguishable levels?

Over a long block, the received vector lies in a "sphere" whose size is set by signal-plus-noise power, and noise smears each codeword over a smaller sphere set by the noise power alone. The number of non-overlapping small spheres that fit in the big one grows like $(1 + \mathrm{SNR})$ per complex dimension. Taking $\log_2$ counts bits. Doubling the power at high SNR adds one more "level" of the log: one extra bit.

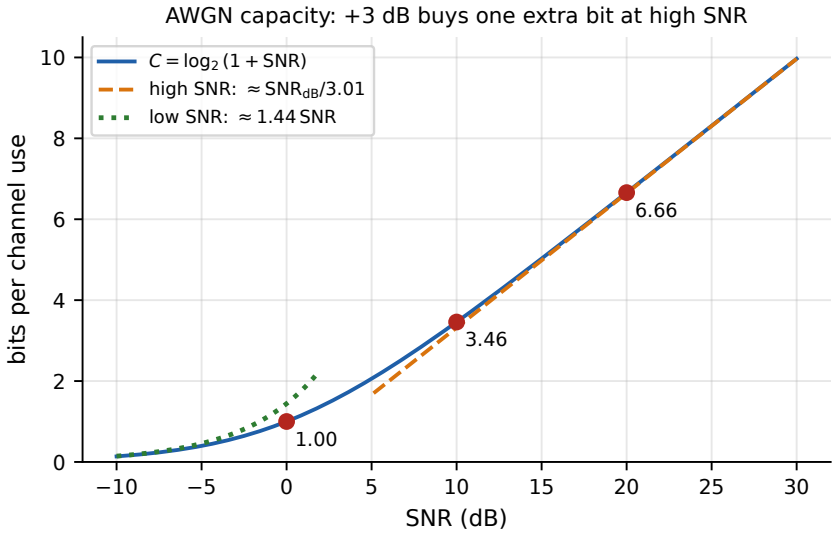


Figure 4.8: AWGN capacity versus SNR. At 0 dB (SNR $= 1$): 1 bit per use; at 10 dB: 3.46; at 20 dB: 6.66. At high SNR the curve approaches $\log_2 \mathrm{SNR}$, gaining one bit per 3 dB. At low SNR it is linear, $C \approx \mathrm{SNR}/\ln 2$: capacity is power-limited. No modulation, code or neural transceiver can operate above this curve.

Wireless connection: capacity, MIMO and learned transceivers

With the MIMO eigen-channels of Chapter 2, $\tilde{y}_i = \sigma_i s_i + \tilde{n}_i$, capacity is simply the sum over the parallel channels: $C = \sum_i \log_2(1 + \sigma_i^2 P_i/\sigma^2)$, maximised by water-filling the powers P_i. For $\mathbf{H} = \left[\begin{smallmatrix} 3 & 0 \\ 4 & 5 \end{smallmatrix}\right]$ with $P_i/\sigma^2 = 1$, $C = \log_2 46 + \log_2 6 \approx 5.52 + 2.58 = 8.11$ bits per use, versus $\log_2 46 = 5.52$ for the best single stream. In Part V, an *autoencoder* learns an encoder and decoder end-to-end by minimising cross-entropy between transmitted and decoded messages. Since cross-entropy bounds mutual information, such training is implicitly pushing $I(X;Y)$ towards capacity, and (4.12) is the yardstick against which those learned systems are judged.

Chapter recap

- Probability obeys three axioms; random variables have a PMF (discrete) or PDF (continuous; probability = area) and a CDF.
- $\mathbb{E}[X]$ is the weighted average (fair die: 3.5); Var $= \mathbb{E}[X^2] - \mu^2$. Example $[2,4,4,4,5,5,7,9]$: mean 5, variance 4, std 2. Standardise with $(x - \mu)/\sigma$ (MNIST: 0.1307, 0.3081).
- Key distributions: Bernoulli, categorical, uniform, Gaussian $\mathcal{N}(\mu, \sigma^2)$, and $\mathcal{CN}(0, \sigma^2)$ with I and Q each of variance $\sigma^2/2$.

- Conditional $p(y|x) = p(x,y)/p(x)$; Bayes $p(x|y) = p(y|x)p(x)/p(y)$ (spam: $0.1/0.18 \approx 0.556$).
- Independent: variances add; zero-mean products multiply variances $\Rightarrow$ initialise weights with variance $\sim 1/n$.
- LLN: averages converge ($\sigma/\sqrt{n}$). CLT: sums become Gaussian $\Rightarrow$ thermal noise and Rayleigh fading.
- Use logs and log-sum-exp for numerical safety. MLE: Bernoulli $\hat{p} = k/n$; Gaussian $\hat{\mu} =$ mean, $\hat{\sigma}^2 =$ mean squared deviation. MSE = Gaussian NLL; cross-entropy = categorical NLL.
- Entropy $H = -\sum p \log p$; cross-entropy $H(p,q)$; $D_{\mathrm{KL}} = H(p,q) - H(p) \geq 0$; mutual information $I(X;Y) = H(X) - H(X|Y)$; AWGN capacity $C = \log_2(1 + \mathrm{SNR})$.

Exercises

1. A biased die has $P(6) = 0.5$ and the other faces equally likely. Find $\mathbb{E}[X]$ and $\mathrm{Var}(X)$.
2. Compute the mean, variance (divide by n) and standard deviation of $[1, 3, 5, 7]$. Standardise the value 7.
3. A disease affects 1% of people. A test detects it 95% of the time when present and gives a false positive 5% of the time when absent. A person tests positive. What is the probability they have the disease?
4. $X \sim \mathcal{N}(0, 4)$ and $Y \sim \mathcal{N}(0, 9)$ are independent. What are $\mathrm{Var}(X + Y)$, $\mathrm{Var}(X - Y)$ and $\mathrm{Var}(XY)$? What is the standard deviation of $X + Y$?
5. A layer has 256 inputs of unit variance. What weight variance keeps the pre-activation variance at 1? What standard deviation is that?
6. Compute $\ln(e^{-500} + e^{-501})$ stably with the log-sum-exp trick.
7. A three-class classifier outputs $q = [0.2, 0.5, 0.3]$ and the true class is the third. What is the cross-entropy loss in nats? What output would give zero loss?
8. (Conceptual) A link runs at 20 dB SNR. By how much does the AWGN capacity increase if the SNR improves by 3 dB? If the bandwidth is doubled at the same total power (so SNR halves)? Explain the difference.

Answers

1. Faces 1–5 each have 0.1. $\mathbb{E}X = 0.1(1 + 2 + 3 + 4 + 5) + 0.5 \cdot 6 = 1.5 + 3 = 4.5$. $\mathbb{E}X^2 = 0.1(1 + 4 + 9 + 16 + 25) + 0.5 \cdot 36 = 5.5 + 18 = 23.5$. Var $= 23.5 - 20.25 = 3.25$.
2. Mean $16/4 = 4$. Squared deviations $9, 1, 1, 9$, sum 20, variance 5, std $\sqrt{5} \approx 2.236$. Standardised 7: $(7 - 4)/2.236 = 1.342$.
3. $P(+) = 0.95 \cdot 0.01 + 0.05 \cdot 0.99 = 0.0095 + 0.0495 = 0.059$. $P(\text{disease} \mid +) = 0.0095/0.059 \approx 0.161$. Only 16%: the disease is rare, so most positives are false positives.
4. $\mathrm{Var}(X + Y) = 4 + 9 = 13$; $\mathrm{Var}(X - Y) = 4 + (-1)^2 \cdot 9 = 13$; $\mathrm{Var}(XY) = 4 \cdot 9 = 36$. Std of $X + Y$ is $\sqrt{13} \approx 3.61$, not $2 + 3 = 5$.

5. $256\,v_w \cdot 1 = 1 \Rightarrow v_w = 1/256$, standard deviation $1/16 = 0.0625$.
6. $m = -500$: $-500 + \ln(1 + e^{-1}) = -500 + \ln 1.3679 = -500 + 0.3133 = -499.687$.
7. $-\ln 0.3 = 1.204$ nats. Zero loss needs $q = [0, 0, 1]$ (in practice approached but never reached with softmax).
8. At 20 dB (SNR $= 100$), $C = \log_2 101 = 6.658$. At 23 dB (SNR ≈ 199.5), $C = \log_2 200.5 \approx 7.648$: about $+1$ bit per use. Doubling bandwidth gives twice as many uses per second at SNR $= 50$: $2\log_2 51 = 2 \times 5.672 = 11.34$ bits per original time unit, versus 6.658 — a far larger gain. At high SNR, capacity grows linearly with bandwidth but only logarithmically with power.

Self-test quiz

1. What is the expected value of a fair die roll? *3.5.*
2. What are the variances of the I and Q parts of $\mathcal{CN}(0, \sigma^2)$ noise? $\sigma^2/2$ *each.*
3. Why is thermal noise Gaussian? *CLT: sum of very many small independent contributions.*
4. Minimising MSE is maximum likelihood under which noise model? *Gaussian.*
5. Can the KL divergence be negative? *No;* ≥ 0, *zero only when* $p = q$.
6. What is the AWGN capacity at SNR $= 0$ dB? $\log_2 2 = 1$ *bit per use.*

Further reading

- J. K. Blitzstein and J. Hwang, *Introduction to Probability*, 2nd ed., CRC Press, 2019 — intuitive and thorough; free online edition.
- T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006 — entropy, KL, mutual information and the Gaussian channel (Ch. 9).
- D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*, Cambridge University Press, 2003 — links information theory to learning; free online.
- C. E. Shannon, "A Mathematical Theory of Communication", *Bell System Technical Journal*, 27, 1948.
- C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, Chapters 1–2 — maximum likelihood and Bayesian inference.

Part II

Neural Networks from First Principles

Chapter 5

What It Means to Learn from Data

Part I gave you the mathematical tools: vectors, matrices, derivatives, gradients and probability. In this chapter we put them to work for the first time. We ask a deceptively simple question — *what does it mean for a computer to "learn"?* — and answer it so concretely that by the end you will have built, by hand and in code, your first learning machine from start to finish.

You will learn the vocabulary every later chapter relies on (dataset, feature, label, model, parameter, loss, optimizer, training set, test set), the four big families of machine learning (supervised, unsupervised, self-supervised and reinforcement learning), and the one idea that the whole field is built around: *generalization*. We then solve a tiny regression problem twice — once exactly with the *normal equations* and once iteratively with *gradient descent* — and meet a second, very different kind of learner, the k-nearest-neighbour classifier. Along the way we map every idea onto a wireless problem you already know, from modulation classification to dynamic spectrum access.

5.1 Rules versus learning

Intuition: two ways to build a BPSK detector

Suppose you must write a program that decides whether a received sample r carried the bit 0 or the bit 1 of a BPSK signal. The *rule-based* way: you know the transmitter maps $0 \mapsto -1$ and $1 \mapsto +1$, that the noise is Gaussian and symmetric, so you write `bit = 1 if r > 0 else 0`. You, the engineer, did the thinking; the program only executes your rule.
The *learning* way: you do not write the rule. You collect thousands of pairs (received sample, transmitted bit), hand them to a program with an adjustable threshold t, and let the program move t until it makes as few mistakes as possible on those pairs. If the channel is clean and symmetric, the program will discover $t \approx 0$ by itself. If the receiver has a DC offset of $+0.3$ that you did not know about, the program will discover $t \approx 0.3$ — something your hand-written rule would have missed.

This little story contains the whole philosophy of machine learning. Classical engineering writes the *rule* and feeds in data to get answers. Machine learning feeds in data *and the answers* and gets back the rule (Fig. 5.1). Learning is worth its cost when (i) the rule is too complicated to write down (recognizing a face, a voice, a modulation format under multipath), (ii) the rule depends on conditions we cannot model accurately (hardware impairments, unknown interference), or (iii) the rule changes over time and must be re-derived automatically.

Definition: learning

A program *learns* from experience if its performance at some task, as measured by a chosen score, improves as it is given more experience. In machine learning the "experience" is a *dataset*, the "task" is a mapping we want (input $\rightarrow$ output, or input $\rightarrow$ action), and the "score" is usually expressed as a *loss* we want to make small.

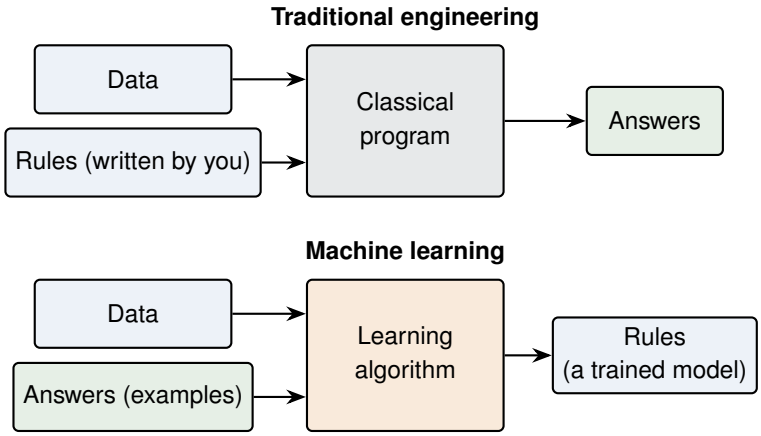


Figure 5.1: Classical programming versus machine learning. In classical programming, a human writes the rule and the computer produces answers. In machine learning, the computer is given examples of data *with* answers and produces the rule — the *trained model* — which is then used like any other program on new data.

This definition (a paraphrase of the classic one by Tom Mitchell) is important because it forces you to name three things before starting any project: *what task*, *what experience*, and *how is success measured*. A surprising number of failed ML projects never answered the third question.

5.2 Datasets, features and labels

Definition: dataset, example, feature, label

A **dataset** is a collection of N **examples** (also called samples or data points). Each example i has

- a **feature vector** $\mathbf{x}_i \in \mathbb{R}^d$: the d numbers that describe the input (the things we measure);
- in supervised learning, a **label** (or target) y_i: the answer we want the model to produce for that input.

Stacking the feature vectors as rows gives the **design matrix** $\mathbf{X} \in \mathbb{R}^{N \times d}$; stacking the labels gives the vector $\mathbf{y} \in \mathbb{R}^N$.

Rows are examples, columns are features — this "batch-first" convention is the one used by NumPy, PyTorch and the rest of this book. Table 5.1 shows a toy wireless dataset written this way.

Table 5.1: A toy supervised dataset for modulation classification. Each row is one received burst (one example); the first four columns are hand-made features computed from the I/Q samples; the last column is the label. Real datasets have thousands to millions of rows.

example i	SNR (dB)	amplitude var.	$\lvert C_{42} \rvert$	phase var.	label y_i
1	12.1	0.02	2.00	0.31	BPSK
2	8.4	0.03	0.98	0.80	QPSK
3	15.0	0.19	0.68	0.79	16-QAM
4	10.7	0.04	1.01	0.81	QPSK
⋮	⋮	⋮	⋮	⋮	⋮

A few words that recur constantly:

- **Model** $f(\mathbf{x};\theta)$: a function with adjustable numbers θ (Greek "theta") that maps a feature vector to a prediction $\hat{y}$.
- **Parameters** θ: the adjustable numbers *learned* from data (a slope and an intercept; later, millions of neural-network weights).
- **Hyperparameters**: numbers *you* choose before learning starts (the learning rate, the number of neighbours k, the number of layers). They are not learned by the same procedure.
- **Training**: the process of choosing θ from the data. **Inference**: using the trained model on new inputs.

5.3 The families of machine learning

What kind of answers does the data contain? That single question sorts machine learning into four families (Fig. 5.2).

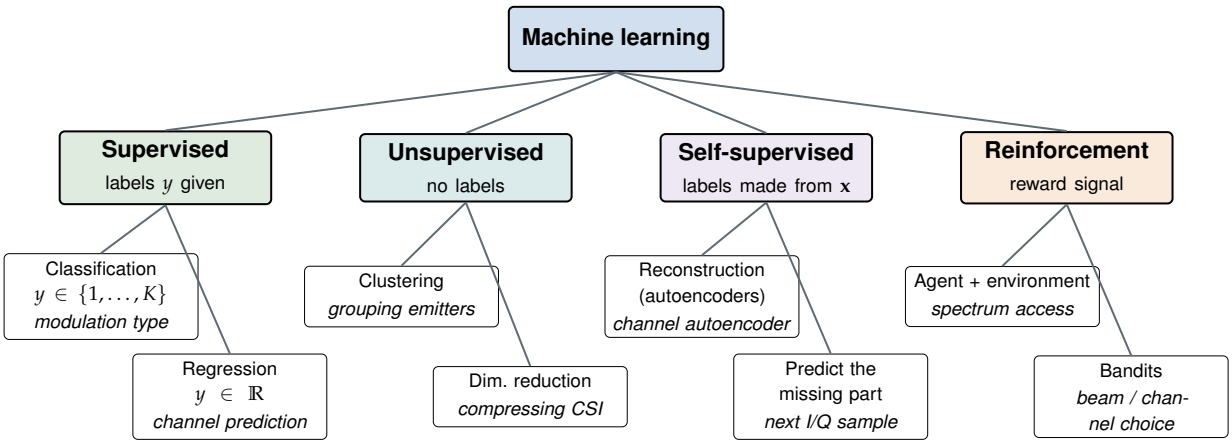


Figure 5.2: A taxonomy of machine learning. The four families differ in *what kind of feedback* the learner receives: correct answers (supervised), nothing (unsupervised), answers manufactured from the input itself (self-supervised), or a scalar reward for actions (reinforcement). Italic text gives a wireless example of each leaf.

5.3.1 Supervised learning

Every training example comes with the correct answer y_i, as if a teacher (a "supervisor") had labelled it. The goal is a function f with $f(\mathbf{x}) \approx y$ for *new* inputs.

- **Classification**: y is one of K discrete categories (BPSK / QPSK / 16-QAM; cat / dog; bit 0 / bit 1). With $K = 2$ it is *binary* classification.
- **Regression**: y is a real number or vector (tomorrow's temperature; the channel gain h five slots ahead; the position of a user).

The difference matters because it decides the model's output and the loss function: classification outputs probabilities over classes and is trained with cross-entropy, regression outputs numbers and is usually trained with mean squared error (both are derived in Chapter 8).

Wireless connection: supervised learning: modulation classification and channel prediction

Classification. In automatic modulation classification, a receiver sees a short burst of I/Q samples and must decide which modulation was used. A labelled dataset is easy to create in simulation — you generate bursts of each format through a channel model and record the label — so this is the textbook supervised problem. The input is either hand-made features (as in Table 5.1) or the raw I/Q samples, and the output is one of K classes. Parts V and VI of this book build such classifiers.

Regression. In channel prediction, the input is the sequence of past channel estimates $h[n-L+1], \ldots, h[n]$ and the label is a future value $h[n+\Delta]$ (a complex number, i.e. two real targets). Labels come "for free": if you log channel estimates over time, the future value of each window is already in the log. Outdated CSI is a real 5G problem (the channel ages between the SRS measurement and the PDSCH transmission), and prediction is a natural regression task.

5.3.2 Unsupervised learning

Only the inputs $\mathbf{x}_i$ are available — nobody tells us the answers. The goal is to discover *structure*:

- **Clustering**: group similar examples together (customers with similar purchases; received pulses that come from the same radar).
- **Dimensionality reduction**: describe each example with fewer numbers while keeping the important information (principal component analysis, which you met through the SVD in Part I, is the classic method).
- **Density estimation**: learn how probable each input is, so that unusual inputs (anomalies) can be flagged.

Wireless connection: unsupervised learning: clustering emitters and interference

A spectrum monitor records thousands of pulses or bursts per second. For each it measures features such as centre frequency, bandwidth, pulse width, pulse-repetition interval and angle of arrival. Nobody labels them. Clustering these feature vectors groups the bursts into "probably the same transmitter" groups — this is called *emitter de-interleaving*. Similarly, clustering the spectrograms of interference observed by a gNB can reveal a handful of recurring interference types (a neighbouring TDD cell out of sync, an intermodulation product, a faulty repeater) without anyone having labelled them in advance. Rogue-base-station detection often starts this way: cluster the normal cells, then flag what does not fit any cluster.

5.3.3 Self-supervised learning

A clever middle ground: the data has no human labels, but we *manufacture* labels from the input itself. Hide part of the input and ask the model to predict it (the next word of a sentence, the next sample of a waveform, a masked patch of an image), or ask the model to reproduce its input through a narrow bottleneck (an *autoencoder*). Because labels cost nothing, self-supervised learning can use enormous unlabelled datasets; it is how large language models are pre-trained.

Wireless connection: self-supervised learning: the channel autoencoder

In the "physical layer as an autoencoder" idea (O'Shea and Hoydis, 2017; Chapter 21 onward) a neural transmitter maps a message $s \in \{1, \dots, M\}$ to channel symbols, a channel model adds noise, and a neural receiver tries to output the same s. The "label" is simply the input message: the system is trained to *reproduce its own input* across a noisy channel. No human labelling is needed — the transmitter and receiver learn a constellation and a detector jointly. This is self-supervision in its purest form, and it is one of the main threads of Part V.

5.3.4 Reinforcement learning

Here there is no dataset of correct answers at all. An **agent** interacts with an **environment**: at each time step it observes a **state** s_t, takes an **action** a_t, and receives a scalar **reward** r_t and a new state s_{t+1}. The goal is a **policy** (a rule state $\to$ action) that maximizes the total reward collected over time (Fig. 5.3). The reward tells the agent *how good* an action was, but never *what the best action would have been* — that is the key difference from supervised learning. Rewards may also arrive late: a move in chess is judged only when the game ends.

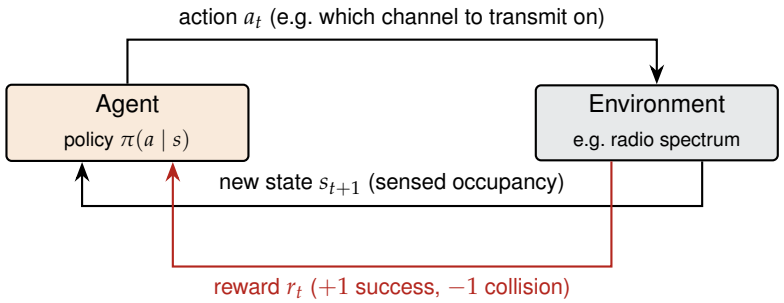


Figure 5.3: The reinforcement-learning loop. The agent acts, the environment responds with a new state and a scalar reward, and the agent adjusts its policy to collect more reward over time. The labels show the dynamic-spectrum-access example.

Wireless connection: reinforcement learning: dynamic spectrum access

A secondary (unlicensed) user wants to transmit in one of C channels that licensed primary users occupy according to unknown, time-varying patterns. At each slot the agent picks a channel (action), transmits, and observes an ACK ($r = +1$) or a collision ($r = -1$ or 0). Nobody can give it the "correct channel" as a label, because that depends on the future behaviour of other users; the agent must learn by trial and error which channels tend to be free, and when. This is the problem of the dynamic-spectrum-access paper studied in Part V, and the same framework is used for beam selection, power control and scheduling.

5.4 Generalization: the real goal

Intuition: the student who memorized the answers

Two students prepare for an exam using last year's paper. One memorizes the answers: "question 3 is 42". The other learns the method. On last year's paper both score 100%. On *this* year's paper, with different numbers, only the second student does well. A model that

only memorizes its training data is the first student. What we want is the second: good performance on data it has *never seen*. That ability is called **generalization**.

Fitting the training data perfectly is easy — a lookup table does it. The difficulty is doing well on new data from the same source. To *measure* generalization we must therefore keep some data hidden from the learner:

Definition: training set and test set

Before training, split the dataset at random into a **training set** (typically 70–80%), used to choose the parameters, and a **test set** (the rest), which is locked away and used *only once at the end* to estimate performance on new data. The error on the training set is the *training error*; the error on the test set is the *test error* (or an estimate of the *generalization error*).

Figure 5.4 shows the classic picture. Twelve noisy samples are taken from a smooth curve and fitted by polynomials of increasing degree. The straight line (degree 1) is too simple: it **underfits**, and both training and test errors are high. Degree 3 captures the shape: both errors are low. Degree 10 has so much freedom that it threads every training point, including their noise; its training error is almost zero but between and beyond the points it wiggles wildly, and its test error grows. That is **overfitting**. The right-hand panel summarizes the pattern you will see again and again in this book: training error keeps falling as the model gets more flexible, while test error first falls and then rises.

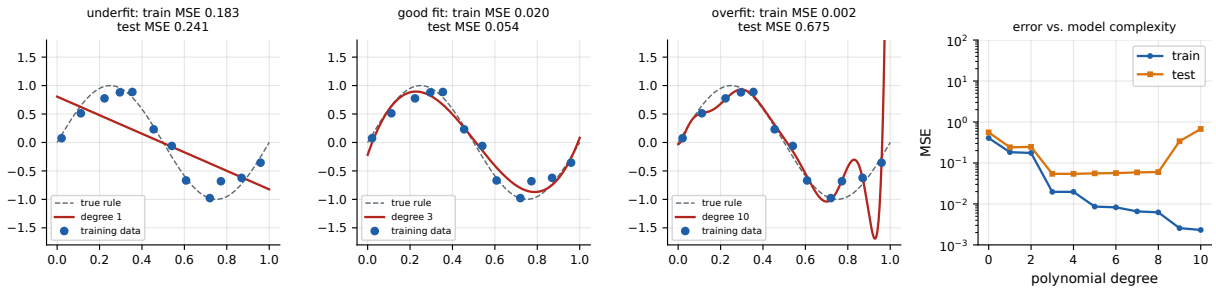


Figure 5.4: Underfitting, good fit and overfitting. Twelve noisy samples (dots) of a sine wave (dashed) are fitted by polynomials of degree 1, 3 and 10. Look at the titles: training error always falls with degree, but test error (measured on 500 fresh samples) is smallest for a moderate degree. Right: the full curves of training and test MSE versus degree — the gap between them is the signature of overfitting. (Script: `code/ch05/fig_generalization.py`.)

Pitfall: never tune on the test set

If you try ten models and keep the one with the best *test* score, the test set has become part of training and its score is now optimistic. Use a third split — a **validation set** — carved out of the training data to choose hyperparameters and compare models, and touch the test set only once. Chapter 12 returns to validation, cross-validation and metrics in detail.

Pitfall: train and test must come from the same world

A modulation classifier trained on AWGN-only simulations and tested on the same simulator may show 99% accuracy and then collapse on over-the-air captures with multipath and carrier offset. A test set only measures generalization to data *like itself*. Make your test data

look like deployment data.

5.5 The universal recipe

Almost every learning system in this book — from the two-parameter line below to a 100-million-parameter Transformer — is assembled from the same five ingredients (Fig. 5.5).

1. **Data** $\{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, split into training, validation and test sets.
2. **Model** $\hat{y} = f(\mathbf{x}; \theta)$: a family of candidate functions, one for every value of the parameters θ. Choosing the model family is choosing what kind of rules the learner is *able* to find.
3. **Loss** $L(\theta)$: one number that measures how wrong the model is on the training data, e.g. the mean squared error $L = \frac{1}{N}\sum_i (\hat{y}_i - y_i)^2$.
4. **Optimizer**: a procedure that changes θ to make L smaller — solving equations exactly when possible, or gradient descent.
5. **Evaluation**: measure performance on held-out data to check generalization.

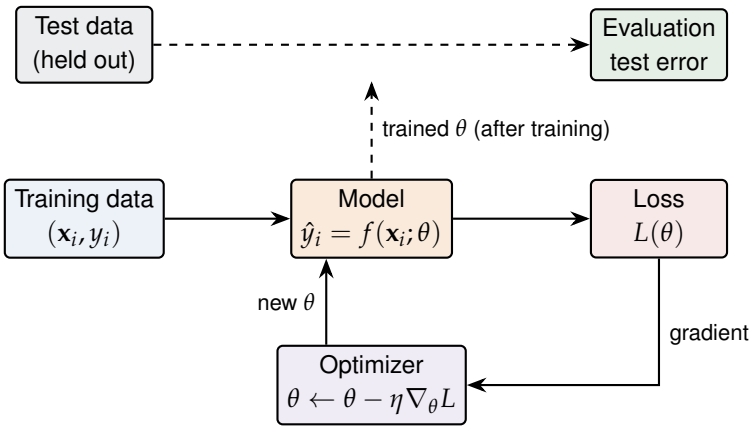


Figure 5.5: The universal recipe of machine learning. The inner loop (solid arrows) repeatedly predicts, measures the loss, and updates the parameters. Only when training is finished does the trained model meet the held-out test data (dashed arrows) to estimate how well it generalizes.

5.6 Our first model, end to end: linear regression

We now follow the recipe for the simplest possible case, doing every step by hand.

Data. Four examples with one feature each:

$$x = (0,\ 1,\ 2,\ 3), \qquad y = (1,\ 3,\ 4,\ 6).$$

Think of x as transmit power setting and y as measured received level, or simply as four dots on graph paper (Fig. 5.6a).

Model. A straight line with two parameters, slope w and intercept b:

$$\hat{y} = f(x; w, b) = wx + b. \tag{5.1}$$

In machine-learning language w is a *weight* and b a *bias*; this tiny model is exactly one artificial neuron without an activation function, as you will see in Chapter 6.

Loss. For each example the *residual* (error) is $e_i = \hat{y}_i - y_i = wx_i + b - y_i$. We square the residuals (so positive and negative errors cannot cancel, and big errors count extra) and average them:

$$L(w, b) = \frac{1}{N} \sum_{i=1}^{N} (wx_i + b - y_i)^2. \tag{5.2}$$

This is the **mean squared error (MSE)**. Every pair (w, b) gives one number L; plotting L over the (w, b) plane gives the *loss landscape* of Fig. 5.6b, a smooth bowl. Learning means finding the bottom of the bowl.

5.6.1 Optimizer 1: the exact solution (normal equations)

At the bottom of a smooth bowl the surface is flat in every direction, so both partial derivatives are zero. We compute them with the chain rule: the derivative of $(\text{something})^2$ is $2(\text{something})$ times the derivative of the something, and $\frac{\partial}{\partial w}(wx_i + b - y_i) = x_i$, $\frac{\partial}{\partial b}(wx_i + b - y_i) = 1$:

$$\frac{\partial L}{\partial w} = \frac{1}{N} \sum_i 2\,(wx_i + b - y_i)\, x_i = \frac{2}{N} \sum_i e_i x_i, \tag{5.3}$$

$$\frac{\partial L}{\partial b} = \frac{1}{N} \sum_i 2\,(wx_i + b - y_i) \cdot 1 = \frac{2}{N} \sum_i e_i. \tag{5.4}$$

Setting (5.4) to zero and dividing by 2:

$$\sum_i (wx_i + b - y_i) = 0 \implies w \sum_i x_i + Nb = \sum_i y_i \implies b = \bar{y} - w\bar{x},$$

where $\bar{x} = \frac{1}{N}\sum_i x_i$ and $\bar{y} = \frac{1}{N}\sum_i y_i$ are the averages. So the best line always passes through the "centre of mass" $(\bar{x}, \bar{y})$ of the data. Substituting $b = \bar{y} - w\bar{x}$ into (5.3)$= 0$:

$$\sum_i (wx_i + \bar{y} - w\bar{x} - y_i) x_i = 0 \implies w \sum_i (x_i - \bar{x}) x_i = \sum_i (y_i - \bar{y}) x_i.$$

Because $\sum_i (x_i - \bar{x}) = 0$, we may replace the trailing x_i by $(x_i - \bar{x})$ on both sides without changing either sum (we subtract $\bar{x}\sum_i (x_i - \bar{x}) = 0$ and $\bar{x}\sum_i (y_i - \bar{y}) = 0$). Hence

$$w^\star = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}}, \qquad b^\star = \bar{y} - w^\star \bar{x}. \tag{5.5}$$

Worked example: closed-form fit of the four points

$\bar{x} = (0 + 1 + 2 + 3)/4 = 1.5$ and $\bar{y} = (1 + 3 + 4 + 6)/4 = 3.5$.
Deviations: $x_i - \bar{x} = (-1.5, -0.5, 0.5, 1.5)$, $y_i - \bar{y} = (-2.5, -0.5, 0.5, 2.5)$.
$S_{xy} = (-1.5)(-2.5) + (-0.5)(-0.5) + (0.5)(0.5) + (1.5)(2.5) = 3.75 + 0.25 + 0.25 + 3.75 = 8$.
$S_{xx} = 2.25 + 0.25 + 0.25 + 2.25 = 5$.
So $w^\star = 8/5 = 1.6$ and $b^\star = 3.5 - 1.6 \times 1.5 = 1.1$. The fitted line is $\hat{y} = 1.6x + 1.1$.
Predictions: $(1.1,\ 2.7,\ 4.3,\ 5.9)$; residuals $e = (0.1, -0.3, 0.3, -0.1)$; minimum loss $L^\star = (0.01 + 0.09 + 0.09 + 0.01)/4 = 0.05$. Check: the residuals sum to zero, as (5.4)$= 0$ demands, and $\sum e_i x_i = 0 - 0.3 + 0.6 - 0.3 = 0$, as (5.3)$= 0$ demands. For a new input $x = 4$ the model predicts $\hat{y} = 7.5$.

The matrix form. With many features the same derivation is neater in matrix notation. Absorb the bias into the weights by adding a constant feature 1 to every example: the design matrix becomes $\mathbf{X} = [\mathbf{1}\ \ \mathbf{x}] \in \mathbb{R}^{N\times 2}$ and $\theta = (b, w)^{\mathsf{T}}$, so all predictions at once are $\hat{\mathbf{y}} = \mathbf{X}\theta$. The loss is $L = \frac{1}{N}\,\|\mathbf{X}\theta - \mathbf{y}\|^2 = \frac{1}{N}(\mathbf{X}\theta - \mathbf{y})^{\mathsf{T}}(\mathbf{X}\theta - \mathbf{y})$. Expanding,

$$NL = \theta^{\mathsf{T}}\mathbf{X}^{\mathsf{T}}\mathbf{X}\theta - 2\theta^{\mathsf{T}}\mathbf{X}^{\mathsf{T}}\mathbf{y} + \mathbf{y}^{\mathsf{T}}\mathbf{y}.$$

Using the gradient rules of Part I, $\nabla_\theta(\theta^{\mathsf{T}}\mathbf{A}\theta) = 2\mathbf{A}\theta$ for symmetric $\mathbf{A}$ and $\nabla_\theta(\theta^{\mathsf{T}}\mathbf{c}) = \mathbf{c}$, the gradient is $\frac{2}{N}(\mathbf{X}^{\mathsf{T}}\mathbf{X}\theta - \mathbf{X}^{\mathsf{T}}\mathbf{y})$. Setting it to zero gives the **normal equations**:

Key equation: normal equations of least squares

$$\mathbf{X}^{\mathsf{T}}\mathbf{X}\,\theta^\star = \mathbf{X}^{\mathsf{T}}\mathbf{y} \qquad \Longrightarrow \qquad \theta^\star = (\mathbf{X}^{\mathsf{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathsf{T}}\mathbf{y}.$$

The name comes from geometry: at the optimum the residual vector $\mathbf{X}\theta^\star - \mathbf{y}$ is *normal* (perpendicular) to every column of $\mathbf{X}$, i.e. $\mathbf{X}^{\mathsf{T}}(\mathbf{X}\theta^\star - \mathbf{y}) = \mathbf{0}$.

Worked example: the same fit with matrices

$$\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad \mathbf{X}^{\mathsf{T}}\mathbf{X} = \begin{bmatrix} 4 & 6 \\ 6 & 14 \end{bmatrix}, \quad \mathbf{X}^{\mathsf{T}}\mathbf{y} = \begin{bmatrix} 1+3+4+6 \\ 0+3+8+18 \end{bmatrix} = \begin{bmatrix} 14 \\ 29 \end{bmatrix}.$$

The determinant is $4\cdot 14 - 6\cdot 6 = 20$, so $(\mathbf{X}^{\mathsf{T}}\mathbf{X})^{-1} = \frac{1}{20}\begin{bmatrix} 14 & -6 \\ -6 & 4 \end{bmatrix}$ and

$$\theta^\star = \frac{1}{20}\begin{bmatrix} 14\cdot 14 - 6\cdot 29 \\ -6\cdot 14 + 4\cdot 29 \end{bmatrix} = \frac{1}{20}\begin{bmatrix} 22 \\ 32 \end{bmatrix} = \begin{bmatrix} 1.1 \\ 1.6 \end{bmatrix}.$$

Same answer: $b^\star = 1.1$, $w^\star = 1.6$.

5.6.2 Optimizer 2: gradient descent

The normal equations are perfect for this model, but they do not exist for neural networks: there the condition "gradient $= 0$" is a huge system of nonlinear equations that nobody can solve exactly. What *always* works is to walk downhill.

Intuition: walking down a foggy valley

You stand on a hillside in thick fog and want to reach the valley floor. You cannot see the bottom, but you can feel the slope under your feet. So you take a small step in the steepest downhill direction, feel the slope again, and repeat. The gradient ∇L points steepest *uphill*; stepping along $-\nabla L$ goes steepest downhill. The step size η (the *learning rate*) says how big each step is.

Key equation: gradient descent update

$$w \leftarrow w - \eta\,\frac{\partial L}{\partial w}, \qquad b \leftarrow b - \eta\,\frac{\partial L}{\partial b}, \qquad \text{with } \frac{\partial L}{\partial w} = \frac{2}{N}\sum_i e_i x_i, \quad \frac{\partial L}{\partial b} = \frac{2}{N}\sum_i e_i.$$

Worked example: two gradient-descent steps by hand, $\eta = 0.1$, start at $w = b = 0$

Step 1. With $w = b = 0$ every prediction is 0, so $e = \hat{y} - y = (-1, -3, -4, -6)$. Loss: $L = (1 + 9 + 16 + 36)/4 = 15.5$. Gradients: $\sum e_i x_i = (-1)(0) + (-3)(1) + (-4)(2) + (-6)(3) = -29$, so $\frac{\partial L}{\partial w} = \frac{2}{4}(-29) = -14.5$; $\sum e_i = -14$, so $\frac{\partial L}{\partial b} = \frac{2}{4}(-14) = -7$. Update: $w = 0 - 0.1(-14.5) = 1.45$, $b = 0 - 0.1(-7) = 0.70$.

Step 2. Predictions $1.45x + 0.7 = (0.70, 2.15, 3.60, 5.05)$, errors $e = (-0.30, -0.85, -0.40, -0.95)$. Loss: $(0.09 + 0.7225 + 0.16 + 0.9025)/4 = 0.46875$ — down from 15.5 after a single step! $\sum e_i x_i = 0 - 0.85 - 0.80 - 2.85 = -4.5 \Rightarrow \frac{\partial L}{\partial w} = -2.25$; $\sum e_i = -2.5 \Rightarrow \frac{\partial L}{\partial b} = -1.25$. Update: $w = 1.45 + 0.225 = 1.675$, $b = 0.70 + 0.125 = 0.825$; the new loss is 0.0834.

Both gradients were negative, so both parameters increased — the line was too low and too flat, and gradient descent raised and tilted it. Continuing, (w, b) converges to $(1.6, 1.1)$, the exact answer (after 2000 steps it agrees to 15 digits).

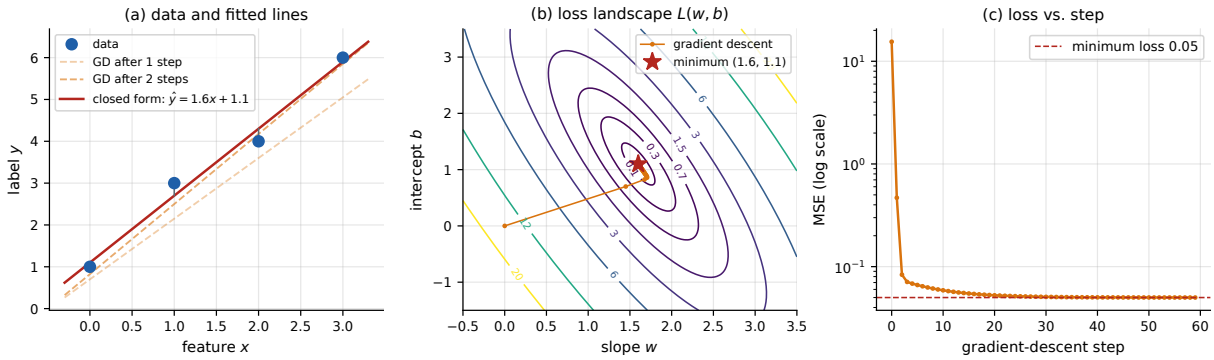


Figure 5.6: Linear regression on four points. (a) The data, the exact least-squares line $\hat{y} = 1.6x + 1.1$ with its residuals (vertical grey segments), and the lines after one and two gradient-descent steps from $w = b = 0$. (b) The loss landscape $L(w, b)$ as contour lines: an elongated bowl whose bottom (star) is the closed-form solution; orange dots are the gradient-descent path, which first jumps toward the valley and then creeps along its floor. (c) Loss versus step on a log scale: a fast drop, then slow convergence to the minimum 0.05. (Script: `code/ch05/fig_regression.py`.)

Look at Fig. 5.6b. The bowl is not round but stretched along a diagonal valley, because w and b trade off against each other (a steeper slope can be compensated by a lower intercept). Gradient descent drops quickly into the valley and then crawls along its floor. This slow crawl in elongated valleys is a central theme of Chapters 9 and 11, where momentum and Adam are introduced to speed it up. The code is only a few lines:

```
import numpy as np
x = np.array([0., 1., 2., 3.]); y = np.array([1., 3., 4., 6.]); N = len(x)
# closed form (normal equations)
X = np.c_[np.ones(N), x]
b_star, w_star = np.linalg.solve(X.T @ X, X.T @ y)    # -> 1.1, 1.6
# gradient descent
w, b, eta = 0.0, 0.0, 0.1
for step in range(2000):
    e = w * x + b - y                      # residuals
    gw, gb = 2/N * np.sum(e * x), 2/N * np.sum(e)
    w, b = w - eta * gw, b - eta * gb      # -> converges to 1.6, 1.1
```

Pitfall: the learning rate

Too small an η and you need thousands of steps; too large and every step overshoots the valley floor, so the loss *grows*. For this dataset any η above about 0.24 diverges (try it; Exercise 7 derives the exact limit). Choosing the learning rate is the single most important tuning decision in deep learning (Chapter 9).

5.7 A completely different learner: k-nearest neighbours

Linear regression *compresses* the training data into two numbers and throws the data away. The opposite extreme keeps all the data and *uses it directly* at prediction time.

Intuition: ask your neighbours

To guess whether a house is expensive, look at the k most similar houses you know about and take a majority vote (or, for a price, their average). No training at all — just memory and a notion of distance.

Definition: k-nearest-neighbour classifier

Given training examples $(\mathbf{x}_i, y_i)$, a query point $\mathbf{q}$ and an integer k: compute the distances $d_i = \|\mathbf{x}_i - \mathbf{q}\|$ (usually Euclidean), find the k examples with the smallest d_i, and predict the most common label among them. For regression, predict the average of their labels.

Worked example: kNN by hand

Training points of class 0: $(1,1)$, $(2,1)$. Class 1: $(4,4)$, $(5,4)$, $(4,5)$. Query $\mathbf{q} = (3, 2.4)$. Squared distances: to $(1,1)$: $2^2 + 1.4^2 = 5.96$; to $(2,1)$: $1 + 1.96 = 2.96$; to $(4,4)$: $1 + 2.56 = 3.56$; to $(5,4)$: $4 + 2.56 = 6.56$; to $(4,5)$: $1 + 6.76 = 7.76$. Sorted: $(2,1)$ [class 0], $(4,4)$ [1], $(1,1)$ [0], $(5,4)$ [1], $(4,5)$ [1].

- $k = 1$: nearest is class $0 \Rightarrow$ predict 0.
- $k = 3$: votes 0, 1, 0 $\Rightarrow$ predict 0.
- $k = 5$: votes 0, 1, 0, 1, 1 $\Rightarrow$ predict 1.

The answer depends on k, a hyperparameter we must choose.

Figure 5.7 shows the decision regions of kNN on a noisy two-class dataset ("two moons"). With $k = 1$ every training point owns a small island; the boundary is jagged and memorizes noise — training accuracy is 100% but test accuracy only 86%. With $k = 15$ the boundary is smooth and follows the true shape: 90% on new data. With $k = 100$ (half of all training points vote!) the boundary becomes almost a straight line and underfits. Figure 5.8 plots training and test accuracy against k: it is the polynomial picture of Fig. 5.4 again, reflected (here small k means a *complex* model).

kNN deserves a place in your toolbox as a **baseline**: it takes two lines of code, has one hyperparameter, and tells you quickly whether a problem is easy. If your elaborate neural network cannot beat kNN, something is wrong. Its weaknesses are just as instructive: prediction is slow (distance to *every* training point), it needs all the data stored, and in high dimensions distances become nearly equal for all points (the "curse of dimensionality"), so neighbours stop being meaningful. A kNN on raw 1024-sample I/Q bursts works poorly; on good features it can

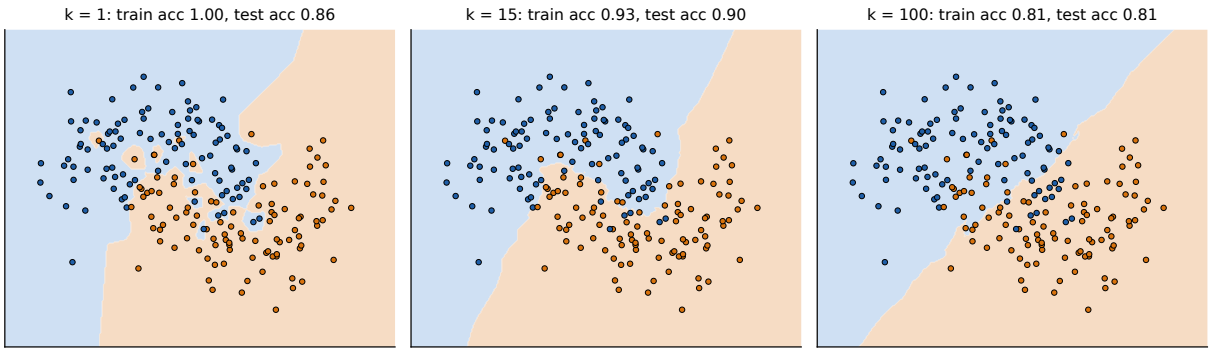


Figure 5.7: Decision regions of k-nearest-neighbour classifiers trained on the same 200 noisy points. Each background colour shows the class that would be predicted for a query at that location. Small k (left) gives a ragged boundary that overfits; large k (right) gives an over-smooth one that underfits. Test accuracy is measured on 1000 fresh points. (Script: `code/ch05/fig_knn.py`.)

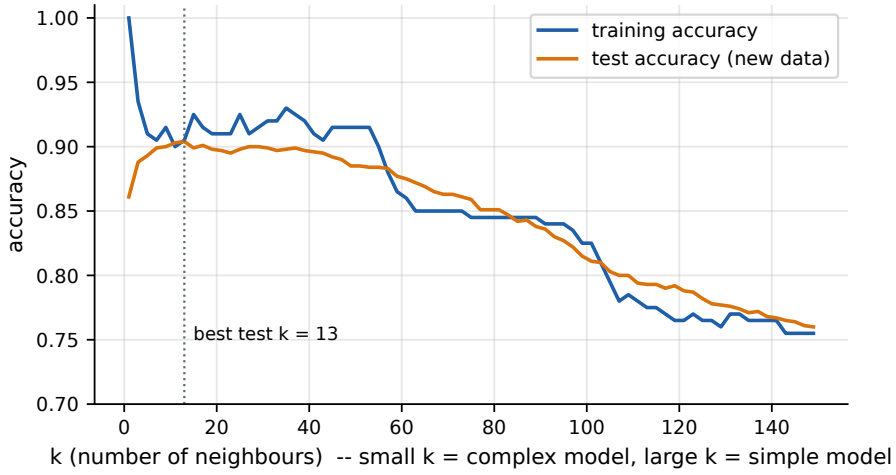


Figure 5.8: Training versus test accuracy of kNN as a function of k. Training accuracy is 100% at $k = 1$ (every point is its own nearest neighbour) and falls steadily; test accuracy peaks at a moderate k (here 13). The best hyperparameter is found on held-out data, never on the training data.

work well. That brings us to features.

5.8 Hand-made features versus learned representations

Both models so far operate on whatever features we give them. For decades the craft of machine learning was largely *feature engineering*: an expert decides which numbers to compute from the raw signal, and a simple classifier does the rest. In modulation classification, the experts' features are higher-order cumulants (e.g. $|C_{42}|$ is about 2 for BPSK, 1 for QPSK and 0.68 for square 16-QAM), amplitude and phase statistics, spectral peaks of the signal raised to the 2nd or 4th power, and so on.

Deep learning changes the division of labour (Fig. 5.9). A neural network takes the *raw* input — I/Q samples, pixels — and its early layers learn their own features ("representations") as part of training, guided only by the loss. Learned representations often beat hand-made ones when data is plentiful, because the network can discover cues the expert never thought of; hand-made features win when data is scarce or when physics gives us a provably good statistic (a matched filter is still the optimal detector in AWGN, and no network will beat it there).

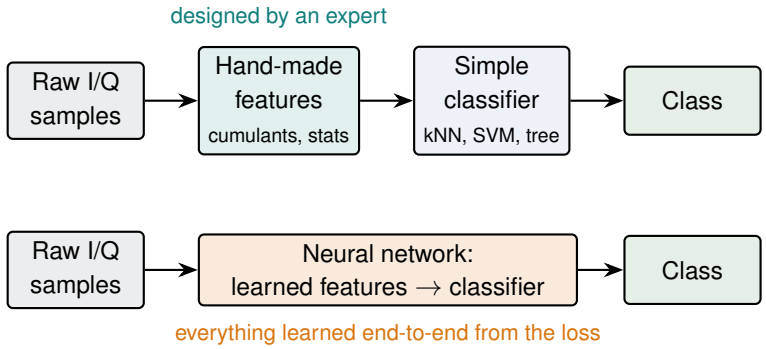


Figure 5.9: Two pipelines for modulation classification. Top: classical ML — an expert designs the features, a simple model learns only the final decision. Bottom: deep learning — the network learns its own features from raw samples, jointly with the decision, by minimizing one loss.

5.9 Putting it together: a map of a learning project

Table 5.2: The recipe instantiated for the wireless problems in this chapter.

Problem	Family	Data (input → target)	Loss / score
Modulation classification	supervised, classification	I/Q burst → format label	cross-entropy / accuracy
Channel prediction	supervised, regression	past CSI → future CSI	MSE / NMSE
Emitter de-interleaving	unsupervised, clustering	pulse features, no labels	cluster compactness
Channel autoencoder	self-supervised	message → same message	cross-entropy / BLER
Dynamic spectrum access	reinforcement	state, action → reward	total reward / throughput

Chapter recap

- Machine learning produces rules from examples. It pays off when rules are hard to write, depend on unmodelled conditions, or change.
- A dataset is a matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ (rows = examples, columns = features) plus, in supervised learning, labels $\mathbf{y}$.
- Families: *supervised* (classification, regression), *unsupervised* (clustering, dimensionality reduction), *self-supervised* (labels made from the input), *reinforcement* (agent, actions, rewards).
- The goal is *generalization*: low error on unseen data. Measure it on a held-out test set; tune on a validation set. Too simple $\Rightarrow$ underfit; too flexible $\Rightarrow$ overfit.
- Recipe: data, model, loss, optimizer, evaluation.
- Linear regression: $\hat{y} = wx + b$, MSE loss. Exact solution $\theta^\star = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\mathbf{y}$; for our data $\hat{y} = 1.6x + 1.1$. Gradient descent $\theta \leftarrow \theta - \eta \nabla L$ reaches the same answer iteratively and is the method that scales to neural networks.
- kNN: store the data, vote among the k nearest; a useful baseline whose k trades overfitting for underfitting.
- Deep learning learns the features themselves, end to end from the loss.

Exercises

1. **(Classify the problem.)** For each task say which family (and, if supervised, classification or regression) applies: (a) estimating the SNR of a received burst from its samples, given a simulated dataset with known SNRs; (b) grouping a city's cell sites into "similar traffic profile" groups; (c) a base station learning by trial and error which beam to serve a user on, rewarded by the achieved throughput; (d) training a network to fill in deliberately erased OFDM subcarriers of recorded signals.
2. **(Closed form.)** Fit $\hat{y} = wx + b$ to the points $x = (1, 2, 3)$, $y = (2, 2, 5)$ using (5.5). Give $w^\star$, $b^\star$, the residuals, and the minimum MSE.
3. **(Gradient descent.)** For the data of Exercise 2, start at $w = b = 0$ with $\eta = 0.1$ and perform one gradient-descent step. Give the gradients and the new (w, b).
4. **(Normal equations.)** Show that if all x_i are equal, $\mathbf{X}^\mathsf{T}\mathbf{X}$ is singular. Explain in words why the line cannot be determined.
5. **(kNN.)** With the five training points of the kNN worked example, classify the query $(3, 3)$ for $k = 1$ and $k = 3$. Is there a tie at $k = 1$? How would you break it?
6. **(Generalization.)** A colleague reports that her modulation classifier reaches 99.8% accuracy. Write three questions you would ask before believing that it will work in the field.
7. **(Learning rate.)** For the four-point dataset, the loss is a quadratic bowl. Using code (or the observation that gradient descent diverges when η exceeds $2/\lambda_{\max}$, where $\lambda_{\max}$ is the largest eigenvalue of $\frac{2}{N}\mathbf{X}^\mathsf{T}\mathbf{X}$), find the largest stable learning rate.

Answers

1. (a) Supervised regression (target is a real number, the SNR). (b) Unsupervised clustering (no labels). (c) Reinforcement learning (actions and rewards; in its simplest stateless form a multi-armed bandit). (d) Self-supervised (labels are the erased values, which we know because we erased them).

2. $\bar{x} = 2$, $\bar{y} = 3$. Deviations x: $(-1, 0, 1)$; y: $(-1, -1, 2)$. $S_{xy} = 1 + 0 + 2 = 3$, $S_{xx} = 2$, so $w^\star = 1.5$, $b^\star = 3 - 1.5 \cdot 2 = 0$. Predictions $(1.5, 3, 4.5)$; residuals $(-0.5, 1, -0.5)$; MSE $= (0.25 + 1 + 0.25)/3 = 0.5$.

3. $e = (-2, -2, -5)$. $\sum e_i x_i = -2 - 4 - 15 = -21$, $\frac{\partial L}{\partial w} = \frac{2}{3}(-21) = -14$; $\sum e_i = -9$, $\frac{\partial L}{\partial b} = \frac{2}{3}(-9) = -6$. New $w = 1.4$, $b = 0.6$.

4. If $x_i = c$ for all i, $\mathbf{X}^\mathsf{T}\mathbf{X} = \begin{bmatrix} N & Nc \\ Nc & Nc^2 \end{bmatrix}$ with determinant $N^2c^2 - N^2c^2 = 0$. All points lie on one vertical line, so any line through $(c, \bar{y})$ has the same loss: the slope is undetermined. (In the scalar formula, $S_{xx} = 0$.)

5. Squared distances from $(3, 3)$: $(1, 1)$: 8; $(2, 1)$: 5; $(4, 4)$: 2; $(5, 4)$: 5; $(4, 5)$: 5. $k = 1$: $(4, 4)$ is uniquely nearest $\Rightarrow$ class 1, no tie. $k = 3$: the nearest is $(4, 4)$, then three points tie at distance $\sqrt{5}$, one of class 0 and two of class 1; whichever two are chosen, at least one more vote goes to class 1, so the majority is class 1. Ties are usually broken by preferring the closer neighbour, by random choice, or by choosing odd k for two classes.

6. For example: Was the test set held out and never used for tuning? Does the test data come from the same channel conditions as deployment (multipath, frequency offset, real hardware, the full SNR range)? What is the accuracy at low SNR, and per class — is 99.8% dominated by easy high-SNR examples? How big was the test set (is the number statistically meaningful)?

7. $\frac{2}{N}\mathbf{X}^\mathsf{T}\mathbf{X} = \frac{1}{2}\begin{bmatrix} 4 & 6 \\ 6 & 14 \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 3 & 7 \end{bmatrix}$. Its eigenvalues solve $\lambda^2 - 9\lambda + 5 = 0$: $\lambda = (9 \pm \sqrt{61})/2$, i.e. 0.595 and 8.405. Stability needs $\eta < 2/8.405 \approx 0.238$. (The slow direction, $\lambda = 0.595$, explains the slow crawl along the valley.) A quick experiment confirms divergence for $\eta = 0.24$ and convergence for $\eta = 0.23$.

Self-test quiz

1. What is the difference between a parameter and a hyperparameter?
   Parameters are learned from the data by the optimizer (e.g. w, b); hyperparameters are chosen by you before training (e.g. η, k).

2. Why can't we use the test set to choose k in kNN?
   The test score would then be biased upward; choose k on a validation set and use the test set once.

3. What does the reward in reinforcement learning not tell the agent?
   What the best action would have been — only how good the chosen action was.

4. At the least-squares solution, what are $\sum_i e_i$ and $\sum_i e_i x_i$?
   Both are zero (the two partial derivatives vanish).

5. Training error is 0.1%, test error 25%. Underfitting or overfitting?
   Overfitting: the model memorized the training data.

6. Why is the channel autoencoder called self-supervised?
   Its target is its own input message; no human labels are needed.

Further reading

- T. M. Mitchell, *Machine Learning*, McGraw-Hill, 1997 — chapter 1 gives the classic definition of learning paraphrased here.
- T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009 — chapters 2–3 cover least squares, nearest neighbours and the bias–variance trade-off in depth.
- I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016 — chapter 5, "Machine Learning Basics".
- R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- T. O'Shea and J. Hoydis, "An introduction to deep learning for the physical layer", *IEEE Trans. Cognitive Communications and Networking*, 2017 — the channel autoencoder and modulation classification.

Chapter 6

The Neuron, the Perceptron and Activation Functions

Every neural network, however large, is built from one tiny component repeated millions of times: the artificial *neuron*. A neuron does just two things: it computes a weighted sum of its inputs, then passes that sum through a simple nonlinear function called the *activation*. In this chapter we take that component apart. We will see what a single neuron computes *geometrically* (it splits space with a flat cut), how the oldest learning rule in the field — the *perceptron* rule of 1958 — adjusts its weights, why one neuron can compute AND but can never compute XOR, and how a sigmoid neuron turns into a probability machine, *logistic regression*. The second half is a catalogue of the activation functions used in practice (sigmoid, tanh, ReLU and its relatives, GELU, SiLU, softmax), each with its formula, derivative, range and failure modes. We end with the single most important fact about activations: without them, a deep network is no more powerful than a single layer.

For the wireless engineer there are good friends here: a sigmoid neuron *is* the optimal BPSK detector, a tanh looks very much like a saturating power amplifier, and softmax is how a modulation classifier reports its belief over the formats.

6.1 From biological to artificial neurons

Intuition: a committee that votes with different strengths

A biological neuron receives electrical signals from thousands of other neurons through its *dendrites*. Each connection (a *synapse*) has a strength: some excite, some inhibit. The cell body adds up all the incoming influence, and if the total exceeds a threshold, the neuron *fires* a spike down its *axon* to the next neurons. Think of a committee where each member votes, but some members' votes count more than others (and some count negatively). If the weighted vote passes a threshold, the motion is carried.

The artificial neuron copies exactly that cartoon — and nothing more (Fig. 6.1). Real neurons are vastly more complex (timing, chemistry, dendritic computation); the analogy gave the field its name and its first idea, but the rest of this book uses only the mathematics.

Definition: artificial neuron

A neuron with input $\mathbf{x} = (x_1, \dots, x_d)^\mathsf{T} \in \mathbb{R}^d$, weight vector $\mathbf{w} \in \mathbb{R}^d$, bias $b \in \mathbb{R}$ and activation function $f : \mathbb{R} \to \mathbb{R}$ computes

$$z = \mathbf{w}^\mathsf{T}\mathbf{x} + b = \sum_{j=1}^{d} w_j x_j + b, \qquad a = f(z). \tag{6.1}$$

z is the **pre-activation** (or *logit*), a the **activation** (the output). The weights and bias are the neuron's parameters.

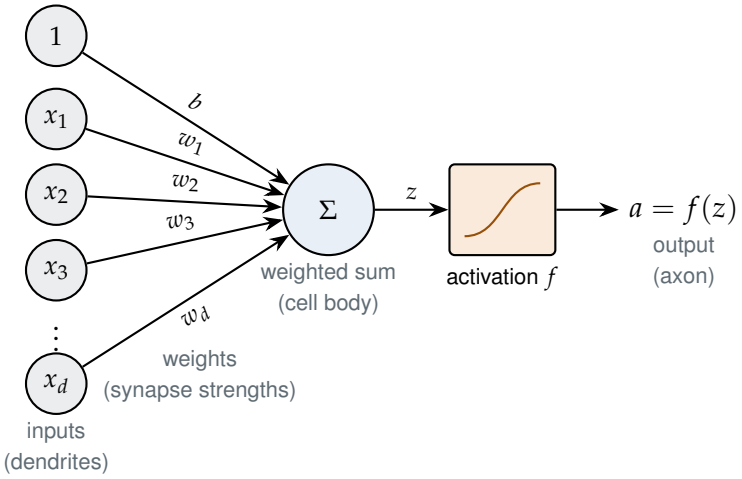


Figure 6.1: An artificial neuron. Inputs $x_1, \ldots, x_d$ are multiplied by weights $w_1, \ldots, w_d$, summed together with a bias b (drawn as the weight of a constant input 1), giving the *pre-activation* z. The activation function f then produces the output a. Grey labels give the loose biological counterparts.

Worked example: one neuron by hand

$\mathbf{x} = (1, 2)$, $\mathbf{w} = (2, -1)$, $b = 0.5$. Then $z = 2 \cdot 1 + (-1) \cdot 2 + 0.5 = 0.5$. With a sigmoid activation $f(z) = 1/(1 + e^{-z})$: $a = 1/(1 + e^{-0.5}) = 1/(1 + 0.6065) = 0.6225$. With ReLU, $a = \max(0, 0.5) = 0.5$. With a step, $a = 1$.

6.2 The geometry of a neuron

The formula $z = \mathbf{w}^\mathsf{T}\mathbf{x} + b$ hides a beautiful picture. Take $d = 2$ so we can draw it. For every point $\mathbf{x} = (x_1, x_2)$ in the plane, the neuron computes a number z. Where is z exactly zero?

$$w_1 x_1 + w_2 x_2 + b = 0$$

is the equation of a **straight line**. In three dimensions it would be a plane, and in d dimensions it is called a **hyperplane** (a flat surface of dimension $d - 1$). On one side of the line $z > 0$, on the other $z < 0$. A neuron with a step activation therefore cuts the input space into two half-spaces with a single flat cut, and says "1" on one side and "0" on the other.

Key equation: the three geometric facts

1. The decision boundary $\{\mathbf{x} : \mathbf{w}^\mathsf{T}\mathbf{x} + b = 0\}$ is a hyperplane.
2. $\mathbf{w}$ is **normal** (perpendicular) to it and points toward the side where $z > 0$.
3. Its signed distance from the origin is $-b/\|\mathbf{w}\|$; more generally the signed distance of any point $\mathbf{x}$ from the boundary is $z/\|\mathbf{w}\|$.

Why $\mathbf{w}$ is perpendicular. Take two points $\mathbf{p}$ and $\mathbf{q}$ both *on* the boundary: $\mathbf{w}^\mathsf{T}\mathbf{p} + b = 0$ and $\mathbf{w}^\mathsf{T}\mathbf{q} + b = 0$. Subtracting, $\mathbf{w}^\mathsf{T}(\mathbf{p} - \mathbf{q}) = 0$. The vector $\mathbf{p} - \mathbf{q}$ points *along* the boundary, and its dot product with $\mathbf{w}$ is zero, so $\mathbf{w}$ is perpendicular to every direction lying in the boundary. Moving from the boundary by a small step $\epsilon\mathbf{w}$ changes z by $\epsilon\mathbf{w}^\mathsf{T}\mathbf{w} = \epsilon\|\mathbf{w}\|^2 > 0$: so $\mathbf{w}$ points toward positive z.

Why the distance is $z/\|\mathbf{w}\|$. Start from a point $\mathbf{x}$ and walk perpendicular to the boundary, i.e. along the unit vector $\mathbf{u} = \mathbf{w}/\|\mathbf{w}\|$, a distance t: z changes by $t\,\mathbf{w}^{\mathsf{T}}\mathbf{u} = t\,\|\mathbf{w}\|$. To reach $z = 0$ from value z we need $t = -z/\|\mathbf{w}\|$. So the point sits at distance $z/\|\mathbf{w}\|$ from the boundary (positive on the $\mathbf{w}$ side). For the origin, $z = b$, which gives the distance $-b/\|\mathbf{w}\|$ of the boundary from the origin in the direction of $\mathbf{w}$.

In words: *the weights set the orientation of the cut, the bias slides it*, and z is a scaled *distance* to the cut — far from the boundary $|z|$ is large (the neuron is "confident"), near it $z \approx 0$. Figure 6.2 shows all three facts for $\mathbf{w} = (2, 1)$.

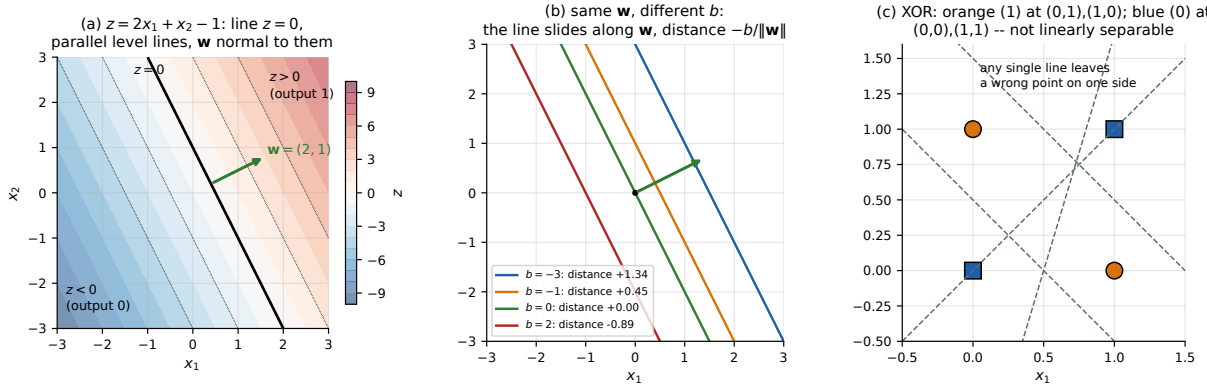


Figure 6.2: Geometry of one neuron. (a) Colour shows $z = 2x_1 + x_2 - 1$ over the plane. The thick line is the decision boundary $z = 0$; dotted lines are levels $z = \pm 2, \pm 4$, parallel and equally spaced; the green arrow $\mathbf{w} = (2, 1)$ is perpendicular to them and points toward increasing z. (b) Keeping $\mathbf{w}$ and changing b slides the boundary along $\mathbf{w}$ without rotating it; the legend gives the signed distance $-b/\|\mathbf{w}\|$ from the origin. (c) The XOR problem: no single straight line can put both orange points on one side and both blue points on the other. (Script: `code/ch06/fig_neuron_geometry.py`.)

Wireless connection: the neuron as a matched filter and correlator

$z = \mathbf{w}^{\mathsf{T}}\mathbf{x}$ is a *correlation* of the input with the template $\mathbf{w}$ — the same operation as a matched filter or a correlator in a receiver. A PSS/SSS detector in 5G correlates the received samples with a known sequence and compares with a threshold: that is literally one neuron with fixed weights (the sequence) and a step activation (the threshold, set by the bias). What learning adds is that the template and threshold are chosen from data rather than from a standard document.

6.3 The perceptron

In 1958 Frank Rosenblatt proposed the *perceptron*: a single neuron with a step activation,

$$\hat{y} = \mathrm{step}(\mathbf{w}^{\mathsf{T}}\mathbf{x} + b), \qquad \mathrm{step}(z) = \begin{cases} 1 & z > 0 \\ 0 & z \le 0 \end{cases}$$

together with a strikingly simple rule for learning $\mathbf{w}$ and b from labelled examples with $y \in \{0, 1\}$.

6.3.1 Logic gates by hand

Before learning anything, let us check that one neuron *can* represent a logic gate. Take AND with $\mathbf{w} = (1, 1)$, $b = -1.5$:

x_1	x_2	$z = x_1 + x_2 - 1.5$	$\hat{y} = \text{step}(z)$	AND	correct?
0	0	-1.5	0	0	yes
0	1	-0.5	0	0	yes
1	0	-0.5	0	0	yes
1	1	$+0.5$	1	1	yes

All four inputs are correct. Geometrically, the line $x_1 + x_2 = 1.5$ cuts off the corner $(1,1)$ from the other three. Changing only the bias to $b = -0.5$ moves the line to $x_1 + x_2 = 0.5$, which separates $(0,0)$ from the rest: that is OR. And $w = -1$, $b = 0.5$ on a single input gives NOT. So one neuron is a programmable logic gate — the observation that made McCulloch and Pitts (1943) argue that networks of neurons could compute anything a computer can.

6.3.2 The perceptron learning rule

Intuition: nudge the boundary toward the mistakes

Present one example at a time. If the perceptron is right, do nothing. If it says 0 but the answer is 1, the example is on the wrong side of the line: add a bit of the example to $\mathbf{w}$, which rotates $\mathbf{w}$ toward $\mathbf{x}$ and increases z for this input next time. If it says 1 but the answer is 0, subtract a bit of the example. The bias is treated as the weight of a constant input 1.

Key equation: perceptron rule

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y - \hat{y})\,\mathbf{x}, \qquad b \leftarrow b + \eta\,(y - \hat{y}).$$

The factor $(y - \hat{y})$ is 0 when correct, $+1$ for a missed positive and -1 for a false positive, so the rule does exactly what the intuition says. Why does it help? After an update on a missed positive, the new pre-activation for the same input is

$$z_{\text{new}} = (\mathbf{w} + \eta\mathbf{x})^{\mathsf{T}}\mathbf{x} + (b + \eta) = z_{\text{old}} + \eta(\|\mathbf{x}\|^2 + 1) > z_{\text{old}},$$

so that example is pushed toward the correct side (and symmetrically for false positives).

Worked example: training a perceptron on AND from zero

Start with $\mathbf{w} = (0,0)$, $b = 0$, $\eta = 1$, and cycle through the four examples in the order $(0,0), (0,1), (1,0), (1,1)$. Recall $\hat{y} = 1$ only if $z > 0$. Table 6.1 lists every step.

Check the final perceptron $\mathbf{w} = (2,1)$, $b = -2$: $z(0,0) = -2$, $z(0,1) = -1$, $z(1,0) = 0$, $z(1,1) = 1$, so the outputs are $0,0,0,1$ — AND. Notice that $(1,0)$ sits exactly *on* the boundary ($z = 0$) and is classified 0 only because our step outputs 0 at $z = 0$. The perceptron stops as soon as there are no mistakes; it does not try to put the boundary in the "middle". This lack of a safety margin is one of the reasons we will soon prefer a smooth neuron trained with a loss function. Figure 6.3 shows how the boundary jumps around during training.

Definition: linear separability

A labelled dataset is **linearly separable** if some hyperplane $\mathbf{w}^{\mathsf{T}}\mathbf{x} + b = 0$ has all examples of class 1 strictly on one side and all examples of class 0 strictly on the other.

Table 6.1: Perceptron training trace for AND ($\eta = 1$, start at zero). Each row presents one example; "update" is $y - \hat{y}$. Rows with a non-zero update change the parameters (shown after the update). Epoch 6 (not shown) makes no mistakes, so training stops with $\mathbf{w} = (2, 1)$, $b = -2$.

epoch	$\mathbf{x}$	y	$z = \mathbf{w}^\top \mathbf{x} + b$	$\hat{y}$	update	new $(\mathbf{w};\ b)$
1	(0,0)	0	0	0	0	(0,0; 0)
1	(0,1)	0	0	0	0	(0,0; 0)
1	(1,0)	0	0	0	0	(0,0; 0)
1	(1,1)	1	0	0	+1	(1,1; 1)
2	(0,0)	0	1	1	−1	(1,1; 0)
2	(0,1)	0	1	1	−1	(1,0; −1)
2	(1,0)	0	0	0	0	(1,0; −1)
2	(1,1)	1	0	0	+1	(2,1; 0)
3	(0,0)	0	0	0	0	(2,1; 0)
3	(0,1)	0	1	1	−1	(2,0; −1)
3	(1,0)	0	1	1	−1	(1,0; −2)
3	(1,1)	1	−1	0	+1	(2,1; −1)
4	(0,0)	0	−1	0	0	(2,1; −1)
4	(0,1)	0	0	0	0	(2,1; −1)
4	(1,0)	0	1	1	−1	(1,1; −2)
4	(1,1)	1	0	0	+1	(2,2; −1)
5	(0,0)	0	−1	0	0	(2,2; −1)
5	(0,1)	0	1	1	−1	(2,1; −2)
5	(1,0)	0	0	0	0	(2,1; −2)
5	(1,1)	1	1	1	0	(2,1; −2)

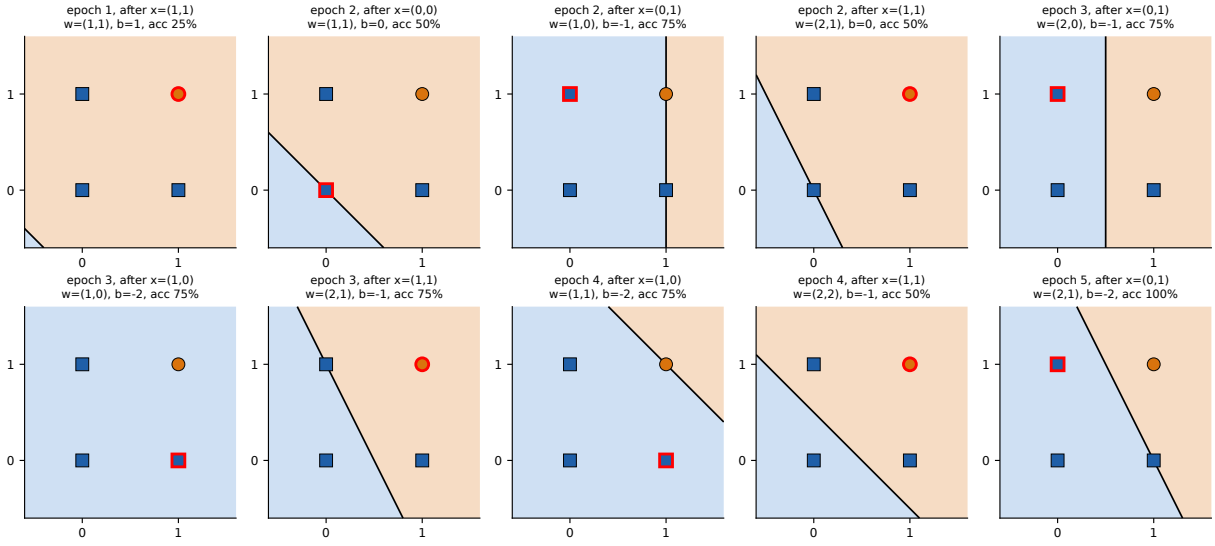


Figure 6.3: The perceptron learning AND (squares: label 0; circles: label 1). Each panel shows the boundary and the predicted regions (orange = 1) right after one of the ten updates of Table 6.1; the example that triggered the update is outlined in red. The boundary swings back and forth while the mistakes are corrected and settles after the tenth update at $2x_1 + x_2 - 2 = 0$ with 100% accuracy. (Script: `code/ch06/fig_perceptron.py`.)

Perceptron convergence theorem. If the data are linearly separable, the perceptron rule finds a separating hyperplane after a *finite* number of updates, whatever the order of presentation (Novikoff, 1962). If the data are not separable, the rule never settles: it keeps cycling forever. That leads to the most famous negative result in the history of the field.

6.3.3 XOR: what one neuron cannot do

XOR outputs 1 when exactly one input is 1: $(0,0) \mapsto 0$, $(0,1) \mapsto 1$, $(1,0) \mapsto 1$, $(1,1) \mapsto 0$. In Fig. 6.2c the two classes sit on opposite diagonals of the square; try placing a ruler so that both orange points are on one side and both blue points on the other — you cannot. Here is the proof.

Key equation: XOR is not linearly separable

Suppose w_1, w_2, b produce XOR, i.e. $z > 0$ exactly for the inputs with output 1. Then

$$\begin{aligned} (0,0) \mapsto 0: &\quad b \le 0, & (1,0) \mapsto 1: &\quad w_1 + b > 0, \\ (0,1) \mapsto 1: &\quad w_2 + b > 0, & (1,1) \mapsto 0: &\quad w_1 + w_2 + b \le 0. \end{aligned}$$

Adding the two middle inequalities: $w_1 + w_2 + 2b > 0$, i.e. $w_1 + w_2 + b > -b \ge 0$ (using $b \le 0$). This contradicts the last inequality. So no such neuron exists. ■

Minsky and Papert's 1969 book *Perceptrons* made limitations of this kind famous and contributed to a long decline of neural-network research. The way out — *stacking* neurons in layers so that a hidden layer re-maps the inputs into a space where they *are* separable — is the subject of Chapter 7, where we will solve XOR by hand with two hidden ReLU neurons.

6.4 Logistic regression: a neuron that outputs a probability

The step function has two defects. It throws away information: $z = 0.01$ and $z = 100$ both give "1", although the second is far more certain. And it is flat almost everywhere, so its derivative is zero and calculus cannot tell us how to improve the weights. Replace the step by a smooth S-shaped curve, the **sigmoid** (or logistic function):

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \tag{6.2}$$

It goes smoothly from 0 (as $z \to -\infty$) through $\sigma(0) = 0.5$ to 1 (as $z \to +\infty$). A neuron with a sigmoid activation is called **logistic regression** (despite the name, it is a classifier), and its output is read as a probability:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^{\mathsf{T}}\mathbf{x} + b), \qquad P(y = 0 \mid \mathbf{x}) = 1 - \sigma(\mathbf{w}^{\mathsf{T}}\mathbf{x} + b).$$

Why is that a sensible probability? The log-odds. The *odds* of an event with probability p are $p/(1-p)$ ("3 to 1" means $p = 0.75$). Solve $p = \sigma(z)$ for z:

$$p = \frac{1}{1+e^{-z}} \;\Rightarrow\; 1 + e^{-z} = \frac{1}{p} \;\Rightarrow\; e^{-z} = \frac{1-p}{p} \;\Rightarrow\; z = \ln \frac{p}{1-p}.$$

So logistic regression assumes that the *log-odds* of class 1 are a linear function of the input. The pre-activation z has a meaning: it is the log-odds, or *logit*. Each unit increase in z multiplies the odds by $e \approx 2.718$. The decision boundary $p = 0.5$ is still the hyperplane $z = 0$, but now the

distance from it (z) tells us *how confident* the model is. In the example of Section 6.1, $z = 0.5$ gives $p = 0.6225$: odds 1.65 to 1.

How are $\mathbf{w}$ and b learned? By choosing them to maximize the probability of the observed labels, which leads to the *binary cross-entropy* loss and the wonderfully simple gradient $\frac{\partial L}{\partial z} = \sigma(z) - y$. We derive both in Chapter 8 and use gradient descent (Chapter 9) to minimize it.

Wireless connection: a single neuron is the optimal BPSK detector

BPSK sends $s \in \{-1, +1\}$ (equally likely) over AWGN: $r = s + n$ with $n \sim \mathcal{N}(0, \sigma_n^2)$. The posterior probability of $s = +1$ follows from Bayes' rule:

$$P(s = +1 \mid r) = \frac{p(r \mid +1)}{p(r \mid +1) + p(r \mid -1)} = \frac{1}{1 + \dfrac{p(r \mid -1)}{p(r \mid +1)}} = \frac{1}{1 + e^{-\Lambda(r)}},$$

where $\Lambda(r) = \ln \frac{p(r|+1)}{p(r|-1)}$ is the log-likelihood ratio (LLR). With Gaussian densities $p(r \mid s) \propto e^{-(r-s)^2/(2\sigma_n^2)}$,

$$\Lambda(r) = \frac{-(r-1)^2 + (r+1)^2}{2\sigma_n^2} = \frac{4r}{2\sigma_n^2} = \frac{2}{\sigma_n^2} r.$$

So $P(s = +1 \mid r) = \sigma\left(\frac{2}{\sigma_n^2} r\right)$: *exactly* one sigmoid neuron with weight $w = 2/\sigma_n^2$ and bias $b = 0$, and its pre-activation z is the channel LLR that an LDPC decoder consumes. Example: $\sigma_n^2 = 0.5$, $r = 0.3$ gives $z = 1.2$ and $P(s = +1 \mid r) = 0.7685$. Thresholding at 0.5 is the familiar sign detector. If the receiver has an unknown DC offset d, the means become $\pm 1 + d$, the LLR becomes $\frac{2}{\sigma_n^2}(r - d)$, and the optimal neuron has bias $b = -wd$ — a neuron trained on data would learn it automatically (Fig. 6.4a,b). The weight also grows as the noise shrinks: at high SNR the sigmoid becomes almost a step.

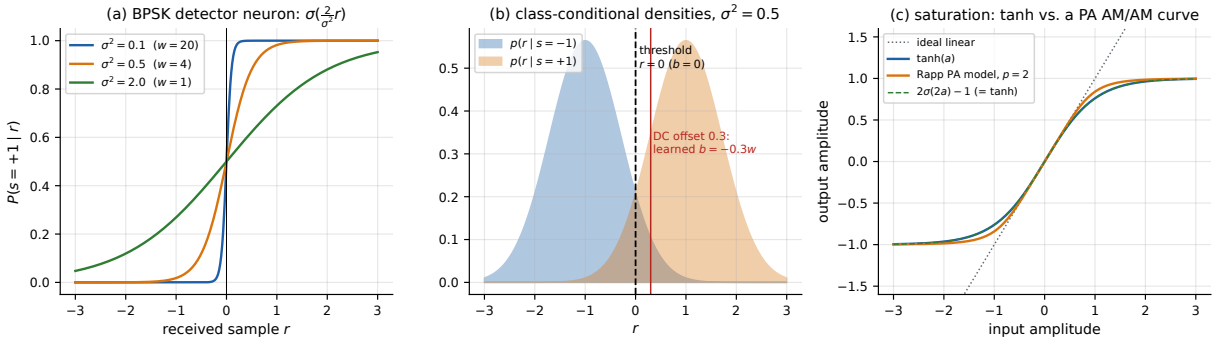


Figure 6.4: (a) The optimal BPSK detector is a sigmoid neuron $\sigma(wr)$ with $w = 2/\sigma_n^2$: the lower the noise, the steeper the curve. (b) The two class-conditional Gaussians at $\sigma_n^2 = 0.5$; the decision threshold is where they cross ($r = 0$, i.e. $b = 0$). A DC offset moves the crossing; a learned bias follows it. (c) tanh compared with the Rapp AM/AM model of a solid-state power amplifier ($p = 2$): both are linear for small inputs and saturate smoothly for large ones. (Script: `code/ch06/fig_logistic_bpsk.py`.)

6.5 A catalogue of activation functions

Inside a network, each hidden neuron applies an activation function f to its pre-activation. During training (Chapter 10) the gradient of the loss flows backward through every neuron and gets multiplied by $f'(z)$ at each one. So two properties of f matter most: its *shape* (what it can

represent) and its *derivative* (how well gradients pass through it). Figure 6.5 plots every function in this section together with its derivative; keep it open while reading.

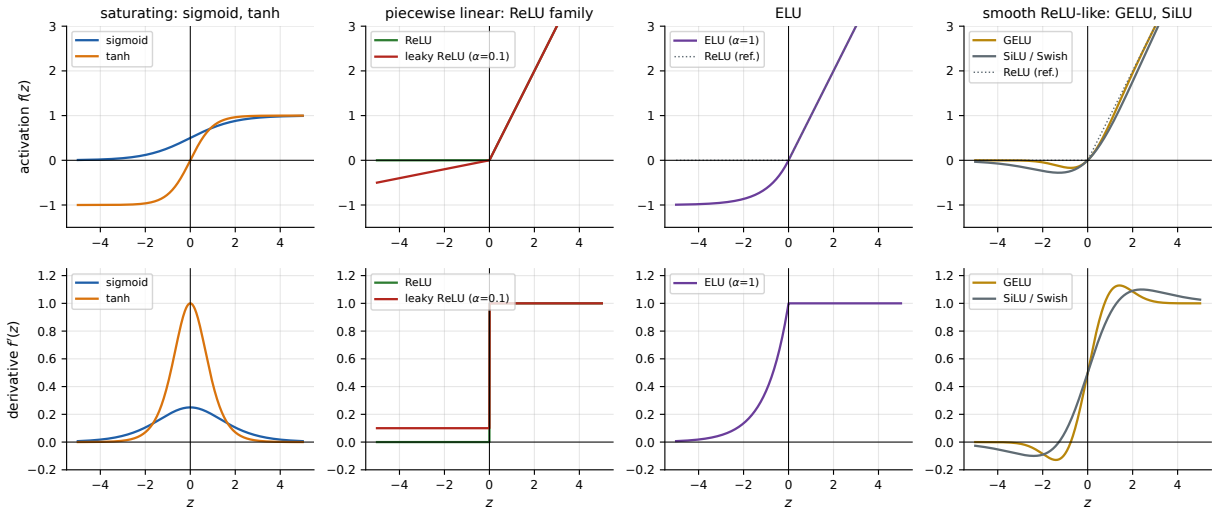


Figure 6.5: Activation functions (top) and their derivatives (bottom). Saturating functions (sigmoid, tanh) have derivatives that vanish for large $|z|$; the ReLU family has derivative exactly 1 for $z > 0$; GELU and SiLU are smooth versions of ReLU that dip slightly below zero for negative inputs. Note the sigmoid's maximum slope is only 0.25. (Script: `code/ch06/fig_activations.py`.)

6.5.1 Sigmoid

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \text{range } (0,1).$$

Derivative. Write $\sigma(z) = (1+e^{-z})^{-1}$ and use the chain rule (derivative of u^{-1} is $-u^{-2}u'$, with $u = 1+e^{-z}$, $u' = -e^{-z}$):

$$\sigma'(z) = -(1+e^{-z})^{-2}\cdot(-e^{-z}) = \frac{e^{-z}}{(1+e^{-z})^2} = \underbrace{\frac{1}{1+e^{-z}}}_{\sigma(z)}\cdot\underbrace{\frac{e^{-z}}{1+e^{-z}}}_{1-\sigma(z)}.$$

The second factor is $1-\sigma$ because $1-\frac{1}{1+e^{-z}} = \frac{1+e^{-z}-1}{1+e^{-z}}$. Hence

Key equation: sigmoid derivative

$$\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big).$$

This is convenient: once the forward pass has computed $a = \sigma(z)$, the derivative is just $a(1-a)$, with no extra exponentials.

Worked example: sigmoid saturation in numbers

$\sigma(0) = 0.5$, slope $0.5 \times 0.5 = 0.25$ — the steepest it ever gets. $\sigma(2) = 0.8808$, slope 0.105. $\sigma(5) = 1/(1+e^{-5}) = 1/1.006738 = 0.9933$, slope $0.9933 \times 0.0067 = 0.0066$. At $z = 5$ the gradient is 38 times smaller than at $z = 0$; at $z = 10$ it is about 4.5×10^{-5}. A neuron pushed far into either tail barely learns: it is *saturated*.

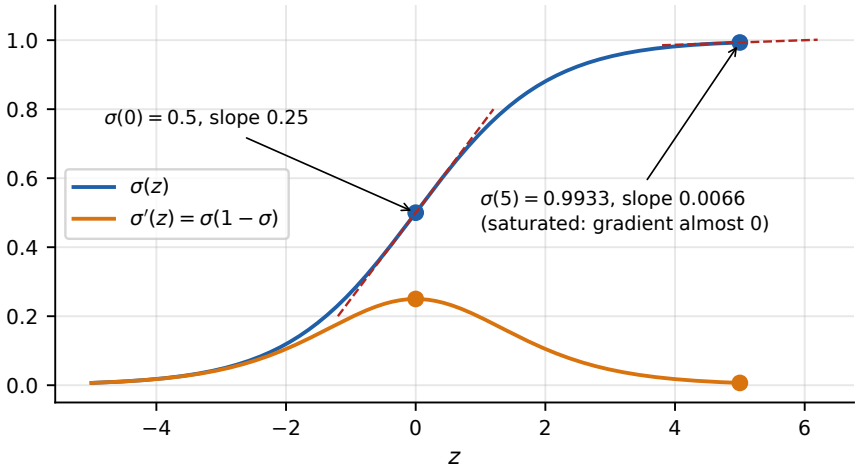


Figure 6.6: The sigmoid and its derivative, with tangent lines at $z = 0$ and $z = 5$. At the centre the slope is 0.25; at $z = 5$ the curve is almost flat (slope 0.0066), so almost no gradient flows through a saturated sigmoid.

Failure modes. (i) **Vanishing gradients**: the derivative is at most 0.25, so a gradient passing backward through n sigmoid layers is multiplied by at most 0.25^n — 10^{-6} after ten layers. (ii) **Not zero-centred**: outputs are always positive, which makes the gradients of the next layer's weights all share one sign and causes zig-zag updates. (iii) An exponential per neuron (minor). Today sigmoid is used almost only at the *output* of binary classifiers and in *gates* (LSTM, attention gating), where a value in $(0, 1)$ is genuinely wanted.

6.5.2 Hyperbolic tangent

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1, \qquad \text{range } (-1, 1).$$

The identity: $2\sigma(2z) - 1 = \frac{2}{1+e^{-2z}} - 1 = \frac{1-e^{-2z}}{1+e^{-2z}}$; multiply top and bottom by e^z to get $\frac{e^z-e^{-z}}{e^z+e^{-z}}$. So tanh is a sigmoid stretched to $(-1, 1)$ and made zero-centred.

Derivative. Quotient rule with $u = e^z - e^{-z}$, $v = e^z + e^{-z}$, $u' = v$, $v' = u$:

$$\tanh'(z) = \frac{u'v - uv'}{v^2} = \frac{v^2 - u^2}{v^2} = 1 - \frac{u^2}{v^2} = 1 - \tanh^2(z).$$

Key equation: tanh derivative

$$\tanh'(z) = 1 - \tanh^2(z), \qquad \tanh'(0) = 1.$$

Numbers: $\tanh(1) = 0.7616$ (slope 0.420), $\tanh(2) = 0.9640$ (slope 0.071), $\tanh(3) = 0.9951$ (slope 0.0099). The peak slope of 1 (four times the sigmoid's) and zero-centred outputs made tanh the default hidden activation before 2010; it still saturates, so deep tanh networks also suffer vanishing gradients. It is used inside RNN/LSTM cells and wherever bounded symmetric outputs are needed (e.g. a transmitter network whose output must stay within an amplitude limit).

Wireless connection: tanh, sigmoid and power-amplifier saturation

A power amplifier is linear for small signals and *compresses* as the input approaches saturation — its AM/AM curve flattens at the maximum output amplitude. The Rapp model of a solid-state PA, $A_{\text{out}} = A/(1 + (A/A_{\text{sat}})^{2p})^{1/(2p)}$, and tanh are almost indistinguishable for moderate p (Fig. 6.4c). The analogy works both ways. Driving a sigmoid or tanh neuron hard into saturation, like driving a PA into compression, flattens its response and destroys small-signal information (for the neuron, the gradient). And when we want a neural network to *model* a PA — for digital predistortion, or to include a realistic PA inside an end-to-end learned transmitter — tanh-type activations are a natural building block. A sigmoid can likewise model a soft limiter or the AGC-plus-ADC clipping of a receiver front end.

6.5.3 ReLU (rectified linear unit)

$$\text{ReLU}(z) = \max(0, z), \qquad \text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}, \qquad \text{range } [0, \infty).$$

(At $z = 0$ the derivative is undefined; software simply uses 0.) ReLU is the default hidden activation of modern deep learning since around 2011. Its advantages: no saturation for positive inputs (derivative exactly 1, so gradients pass through unchanged), trivially cheap (a comparison), and *sparse* activity (typically half the neurons output exactly 0 for a given input). A network of ReLUs computes a *piecewise linear* function; Chapter 7 shows how such pieces can approximate any curve.

Failure mode: dying ReLU. If a neuron's bias becomes very negative (e.g. after a too-large update), $z < 0$ for *every* input; then its output and its gradient are always zero and it can never recover. In a badly trained network a large fraction of neurons can "die". Remedies: sensible initialization (Chapter 11), moderate learning rates, or the variants below.

6.5.4 Leaky ReLU and ELU

$$\text{LeakyReLU}(z) = \begin{cases} z & z > 0 \\ \alpha z & z \le 0 \end{cases} \ (\alpha \approx 0.01\text{–}0.2), \qquad \text{ELU}(z) = \begin{cases} z & z > 0 \\ \alpha(e^z - 1) & z \le 0 \end{cases}.$$

Leaky ReLU gives negative inputs a small slope α, so the gradient is never exactly zero and neurons cannot die; its derivative is 1 or α. (PReLU learns α.) ELU uses a smooth exponential for $z < 0$; it saturates at $-\alpha$ for very negative inputs, which makes mean activations closer to zero. Its derivative for $z \le 0$ is $\alpha e^z = \text{ELU}(z) + \alpha$, again computable from the output. Numbers: $\text{ELU}(-1) = e^{-1} - 1 = -0.632$.

6.5.5 GELU

The Gaussian error linear unit multiplies the input by the probability that a standard Gaussian variable is below it:

$$\text{GELU}(z) = z\,\Phi(z) = \frac{z}{2}\Big(1 + \text{erf}\big(z/\sqrt{2}\big)\Big),$$

where Φ is the standard normal CDF. Intuition: ReLU multiplies z by a hard gate (1 if $z > 0$, else 0); GELU multiplies by a *soft* gate $\Phi(z)$ that is near 1 for large positive z, near 0 for large negative

z and 0.5 at zero. The derivative (product rule, $\Phi' = \phi$, the Gaussian density) is

$$\mathrm{GELU}'(z) = \Phi(z) + z\,\phi(z), \qquad \phi(z) = \tfrac{1}{\sqrt{2\pi}} e^{-z^2/2}.$$

Because erf used to be slow, a tanh approximation is common:

$$\mathrm{GELU}(z) \approx \frac{z}{2}\Big(1 + \tanh\Big[\sqrt{2/\pi}\,\big(z + 0.044715\,z^3\big)\Big]\Big).$$

Numbers: $\mathrm{GELU}(1) = 0.8413$ (approx. 0.8412), $\mathrm{GELU}(-1) = -0.1587$ (approx. -0.1588), $\mathrm{GELU}(2) = 1.9545$. GELU is the activation of BERT, GPT and most Transformers.

6.5.6 SiLU / Swish

$$\mathrm{SiLU}(z) = z\,\sigma(z), \qquad \mathrm{SiLU}'(z) = \sigma(z) + z\,\sigma(z)(1-\sigma(z)) = \sigma(z)\big(1 + z(1-\sigma(z))\big).$$

The same soft-gate idea with a sigmoid gate (Swish adds a learnable scale β: $z\,\sigma(\beta z)$). Like GELU it is smooth, unbounded above, slightly negative for moderate negative inputs (minimum about -0.278 at $z \approx -1.28$), and close to ReLU far from zero. Numbers: $\mathrm{SiLU}(1) = 0.7311$, $\mathrm{SiLU}(-1) = -0.2689$. It is popular in vision networks (EfficientNet) and in modern LLM feed-forward blocks (SwiGLU).

Table 6.2: Activation functions at a glance.

name	$f(z)$	$f'(z)$	range	typical use / failure mode
sigmoid	$1/(1+e^{-z})$	$\sigma(1-\sigma)$, max 0.25	$(0,1)$	binary output, gates / saturates
tanh	$\frac{e^z-e^{-z}}{e^z+e^{-z}}$	$1-\tanh^2$, max 1	$(-1,1)$	RNNs, bounded outputs / saturates
ReLU	$\max(0,z)$	1 or 0	$[0,\infty)$	default hidden / dying units
leaky ReLU	$\max(\alpha z, z)$	1 or α	$\mathbb{R}$	avoids dying units
ELU	z or $\alpha(e^z-1)$	1 or $f+\alpha$	$(-\alpha,\infty)$	smoother, near-zero mean
GELU	$z\Phi(z)$	$\Phi + z\phi$	$\approx[-0.17,\infty)$	Transformers
SiLU	$z\sigma(z)$	$\sigma(1+z(1-\sigma))$	$\approx[-0.28,\infty)$	CNNs, LLM blocks

6.6 Softmax: an activation for many classes

Logistic regression handles two classes. For K classes (BPSK, QPSK, 8PSK, 16-QAM, …) we let the last layer produce K pre-activations $z_1, \ldots, z_K$ (one score per class, also called *logits*) and convert them into a probability distribution with the **softmax**:

Key equation: softmax

$$p_i = \mathrm{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{k=1}^{K} e^{z_k}}, \qquad i = 1, \ldots, K.$$

Exponentiation makes every score positive and amplifies differences; dividing by the sum makes them add up to 1. Unlike the activations above, softmax is not applied to each neuron separately: every output depends on *all* inputs.

Worked example: softmax of $\mathbf{z} = (2,\ 1,\ 0.1)$

$e^2 = 7.3891$, $e^1 = 2.7183$, $e^{0.1} = 1.1052$; sum $= 11.2125$. $p = (7.3891,\ 2.7183,\ 1.1052)/11.2125 = (0.659,\ 0.242,\ 0.099)$. They sum to 1, and the order of the scores is preserved. Adding a constant to every logit leaves p unchanged (the factor e^c cancels): $\text{softmax}(\mathbf{z}) = \text{softmax}(\mathbf{z} - c)$. Implementations use $c = \max_k z_k$ so that no exponential overflows (Chapter 8).

Two sanity checks connect softmax to what we already know. With $K = 2$: $p_1 = \frac{e^{z_1}}{e^{z_1}+e^{z_2}} = \frac{1}{1+e^{-(z_1-z_2)}} = \sigma(z_1 - z_2)$, so softmax over two classes is a sigmoid of the score difference. And the name: softmax is a "soft" (differentiable) version of the *argmax*, which would put probability 1 on the largest score.

6.6.1 The softmax Jacobian

To train through softmax we need $\frac{\partial p_i}{\partial z_j}$ for every pair (i, j). Write $S = \sum_k e^{z_k}$, so $p_i = e^{z_i}/S$ and $\frac{\partial S}{\partial z_j} = e^{z_j}$.

Case $i = j$. Quotient rule:

$$\frac{\partial p_i}{\partial z_i} = \frac{e^{z_i}S - e^{z_i}e^{z_i}}{S^2} = \frac{e^{z_i}}{S} - \left(\frac{e^{z_i}}{S}\right)^2 = p_i - p_i^2 = p_i(1 - p_i).$$

Case $i \neq j$. The numerator e^{z_i} does not depend on z_j:

$$\frac{\partial p_i}{\partial z_j} = \frac{0 \cdot S - e^{z_i}e^{z_j}}{S^2} = -\frac{e^{z_i}}{S}\frac{e^{z_j}}{S} = -p_i p_j.$$

Both cases combine with the Kronecker delta ($\delta_{ij} = 1$ if $i = j$, else 0):

Key equation: softmax Jacobian

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j), \qquad \text{i.e.} \qquad \mathbf{J} = \text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\mathsf{T}.$$

For $\mathbf{p} = (0.659, 0.242, 0.099)$:

$$\mathbf{J} = \begin{bmatrix} 0.225 & -0.160 & -0.065 \\ -0.160 & 0.184 & -0.024 \\ -0.065 & -0.024 & 0.089 \end{bmatrix}.$$

Each row sums to zero (raising one logit takes probability from the others, the total stays 1), and the diagonal entries $p_i(1 - p_i)$ are the sigmoid's derivative again. In Chapter 8 this Jacobian combines with the cross-entropy loss to give the gradient $\mathbf{p} - \mathbf{y}$.

6.6.2 Temperature

Dividing the logits by a **temperature** $T > 0$ before softmax, $p_i = e^{z_i/T} / \sum_k e^{z_k/T}$, controls how peaked the distribution is (Fig. 6.7). For $\mathbf{z} = (2, 1, 0.1)$: $T = 0.5$ gives $(0.864, 0.117, 0.019)$; $T = 1$ gives $(0.659, 0.242, 0.099)$; $T = 2$ gives $(0.502, 0.304, 0.194)$; $T = 5$ gives $(0.400, 0.327, 0.273)$. As $T \to 0$ softmax becomes argmax (one-hot); as $T \to \infty$ it becomes uniform. Temperature is used to sample more or less adventurously from language models, to *calibrate* over-confident classifiers

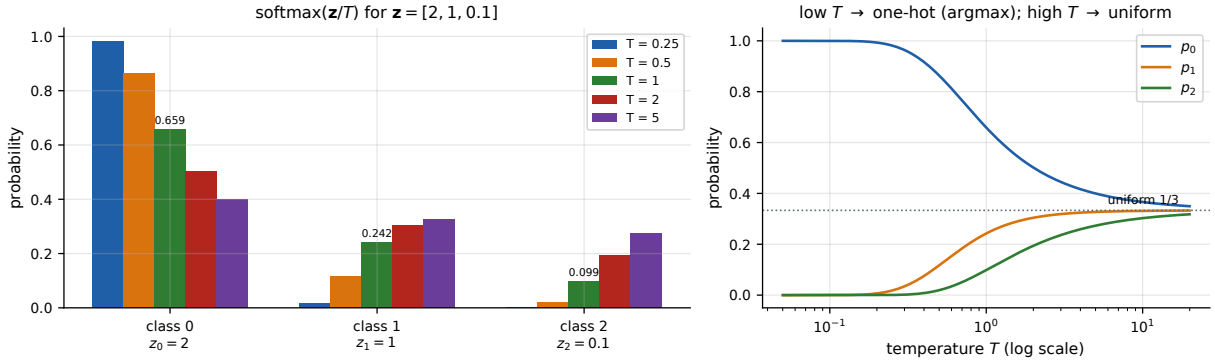


Figure 6.7: Softmax temperature. Left: the probabilities for logits $(2, 1, 0.1)$ at five temperatures (the $T = 1$ values are labelled). Right: each probability as a continuous function of T; low temperatures concentrate all mass on the largest logit, high temperatures flatten toward $1/3$. The ranking never changes. (Script: `code/ch06/fig_softmax_temp.py`.)

(temperature scaling), and in knowledge distillation, where a high T exposes how a teacher network ranks the wrong classes.

Wireless connection: softmax over modulation classes

A modulation classifier ends in a layer of K neurons, one per format, followed by softmax (Fig. 6.8). Its output is a posterior distribution, e.g. $(P_{\text{BPSK}}, P_{\text{QPSK}}, P_{\text{8PSK}}, P_{\text{16QAM}}) = (0.02, 0.71, 0.21, 0.06)$, not just a decision. That matters in a real receiver: if the top probability is low (QPSK vs. 8PSK confusion at low SNR is typical), a cognitive radio can wait for more samples, or a blind receiver can try to demodulate under the two most likely hypotheses and let the CRC decide. Temperature scaling, fitted on a validation set, makes these probabilities *calibrated* — a predicted 0.7 should be right 70% of the time — which is essential if they feed a downstream decision.

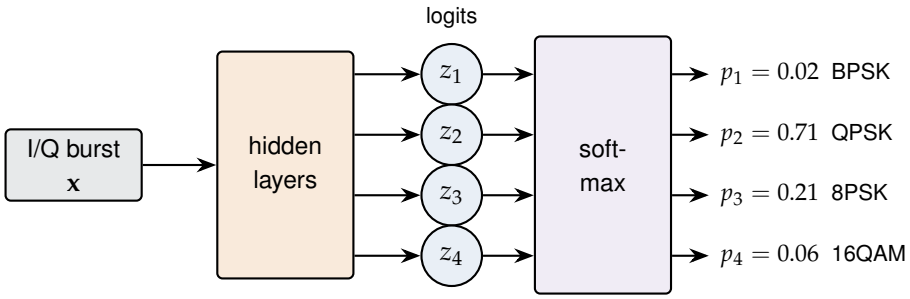


Figure 6.8: The output end of a modulation classifier. The last linear layer produces one logit per class; softmax turns the logits into a probability distribution over the $K = 4$ formats.

6.7 Why nonlinearity is essential

Why not skip activations altogether and stack linear layers? Because it buys nothing. Two linear layers in a row compute

$$\mathbf{h} = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1, \qquad \mathbf{y} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2 = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \underbrace{(\mathbf{W}_2\mathbf{W}_1)}_{\mathbf{W}}\mathbf{x} + \underbrace{(\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2)}_{\mathbf{b}},$$

which is again a single linear layer with weight matrix $\mathbf{W} = \mathbf{W}_2\mathbf{W}_1$ and bias $\mathbf{b}$. By induction, *any* number of linear layers collapses to one: a 100-layer linear network can draw only a straight line (hyperplane) and still cannot compute XOR. (The column-vector convention is used here for readability; the batch-first form $\mathbf{XW}$ of Chapter 7 gives the same conclusion.)

Intuition: folding paper

Linear maps can only stretch, rotate and shear a sheet of paper; however many you apply, straight lines stay straight, and two classes that a straight cut could not separate remain inseparable. A nonlinearity like ReLU is a *fold*: it flattens half of the space onto a crease. Alternate stretching and folding a few times and you can bring any two regions of the sheet onto opposite sides of a single cut. That is what hidden layers do, and Chapter 7 shows it with real pictures.

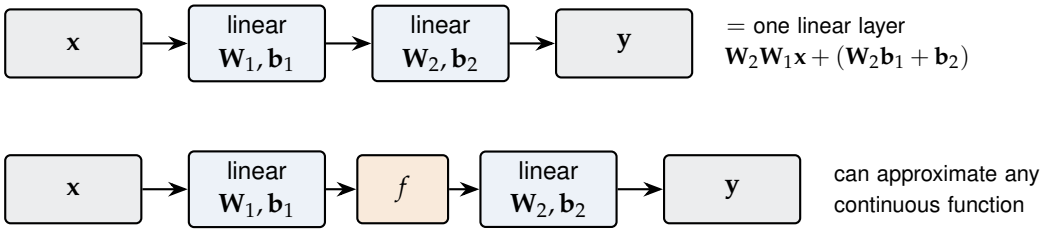


Figure 6.9: Top: stacked linear layers collapse into a single linear layer. Bottom: inserting a nonlinear activation f between them makes the composition genuinely more powerful — wide enough, it can approximate any continuous function (Chapter 7).

Pitfall: forgetting the activation

A common bug in hand-written networks is to leave out the activation between two `Linear` layers, or to put a ReLU on the final layer of a regression network (so it can never predict negative values), or to apply softmax twice (once in the model and again inside the loss function). Rule of thumb: nonlinear activations on hidden layers; on the output layer use *identity* for regression, *sigmoid* for binary or multi-label outputs, *softmax* for one-of-K classification — and in PyTorch let the loss function apply sigmoid/softmax for you (Chapter 8).

Chapter recap

- A neuron computes $z = \mathbf{w}^\mathsf{T}\mathbf{x} + b$ and $a = f(z)$. Its decision boundary $z = 0$ is a hyperplane with normal vector $\mathbf{w}$ at signed distance $-b/\|\mathbf{w}\|$ from the origin; $z/\|\mathbf{w}\|$ is a point's distance from it.
- Perceptron: step activation, rule $\mathbf{w} \leftarrow \mathbf{w} + \eta(y - \hat{y})\mathbf{x}$. AND: $\mathbf{w} = (1, 1)$, $b = -1.5$. Converges iff the data are linearly separable; XOR is provably not.
- Logistic regression $P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\mathsf{T}\mathbf{x} + b)$: z is the log-odds. The optimal BPSK detector is a sigmoid neuron with $w = 2/\sigma_n^2$; its z is the LLR.
- $\sigma' = \sigma(1 - \sigma)$ (max 0.25; $\sigma(5) = 0.9933$, slope 0.0066), $\tanh' = 1 - \tanh^2$; saturating activations cause vanishing gradients. ReLU (default; may die), leaky ReLU, ELU, GELU $z\Phi(z)$, SiLU $z\sigma(z)$.

- Softmax $p_i = e^{z_i} / \sum_k e^{z_k}$; $(2, 1, 0.1) \mapsto (0.659, 0.242, 0.099)$; Jacobian $p_i(\delta_{ij} - p_j)$; temperature controls sharpness.
- Without nonlinear activations, any stack of layers is a single linear map.

Exercises

1. **(Geometry.)** A neuron has $\mathbf{w} = (3, 4)$, $b = -10$. (a) Where does its boundary cross the axes? (b) What is the distance of the boundary from the origin? (c) What is the signed distance of the point $(3, 3)$ from the boundary, and which output does a step neuron give there?
2. **(Gates.)** Find weights and a bias for a perceptron computing NAND (output 0 only for $(1, 1)$). Verify all four inputs.
3. **(Perceptron trace.)** Train a perceptron for OR from $\mathbf{w} = (0, 0)$, $b = 0$, $\eta = 1$, presenting $(0, 0), (0, 1), (1, 0), (1, 1)$ in order, with $\hat{y} = 1$ iff $z > 0$. Give the parameters after each update and the final solution.
4. **(Derivatives.)** Compute $\sigma(-2)$, $\sigma'(-2)$, $\tanh(0.5)$ and $\tanh'(0.5)$. Show that $\tanh'(z) = 4\sigma'(2z)$.
5. **(Softmax.)** Compute $\text{softmax}(1, 1, 1, 1)$ and $\text{softmax}(3, 0)$. Show that the second equals $(\sigma(3), \sigma(-3))$.
6. **(Jacobian.)** Show that every row of the softmax Jacobian sums to zero, and explain why that must be so.
7. **(BPSK neuron.)** At $\sigma_n^2 = 0.25$, what weight does the optimal BPSK neuron have? What is $P(s = +1 \mid r = -0.1)$? If there is a known DC offset $d = 0.2$, what bias should the neuron have?
8. **(Linearity.)** Prove that a network $\mathbf{y} = \mathbf{W}_3(\mathbf{W}_2(\mathbf{W}_1\mathbf{x}))$ with no biases and no activations cannot compute $y = x_1 x_2$ for scalar output, by finding two inputs where linearity would be violated.

Answers

1. (a) $3x_1 + 4x_2 = 10$: crosses the x_1-axis at $10/3 \approx 3.33$ and the x_2-axis at 2.5. (b) $\|\mathbf{w}\| = 5$, distance $= -b/\|\mathbf{w}\| = 10/5 = 2$. (c) $z = 9 + 12 - 10 = 11$, signed distance $11/5 = 2.2$ on the $\mathbf{w}$ side; output 1.
2. E.g. $\mathbf{w} = (-1, -1)$, $b = 1.5$: $z = 1.5, 0.5, 0.5, -0.5 \Rightarrow 1, 1, 1, 0$. (It is AND with all signs flipped.)
3. Epoch 1: $(0,0)$: $z = 0, \hat{y} = 0$, OK. $(0,1)$: $z = 0, \hat{y} = 0, y = 1 \Rightarrow \mathbf{w} = (0,1), b = 1$. $(1,0)$: $z = 1$, OK. $(1,1)$: $z = 2$, OK. Epoch 2: $(0,0)$: $z = 1, \hat{y} = 1, y = 0 \Rightarrow \mathbf{w} = (0,1), b = 0$. $(0,1)$: $z = 1$, OK. $(1,0)$: $z = 0, \hat{y} = 0, y = 1 \Rightarrow \mathbf{w} = (1,1), b = 1$. $(1,1)$: OK. Epoch 3: $(0,0)$: $z = 1$, wrong $\Rightarrow \mathbf{w} = (1,1), b = 0$. Then $(0,1)$: $z = 1$ OK; $(1,0)$: $z = 1$ OK; $(1,1)$: $z = 2$ OK. Epoch 4: $(0,0)$: $z = 0, \hat{y} = 0$, OK; all others OK. Final $\mathbf{w} = (1,1), b = 0$ (with $(0,0)$ on the boundary).
4. $\sigma(-2) = 1/(1 + e^2) = 0.1192$; $\sigma'(-2) = 0.1192 \times 0.8808 = 0.1050$. $\tanh(0.5) = 0.4621$; $\tanh'(0.5) = 1 - 0.2135 = 0.7864$. Since $\tanh(z) = 2\sigma(2z) - 1$, differentiating with the

chain rule gives $\tanh'(z) = 2 \cdot \sigma'(2z) \cdot 2 = 4\sigma'(2z)$.

5. $(0.25, 0.25, 0.25, 0.25)$. $\text{softmax}(3, 0) = (e^3, 1)/(e^3 + 1) = (0.9526, 0.0474)$; and $\frac{e^3}{e^3+1} = \frac{1}{1+e^{-3}} = \sigma(3)$, $\frac{1}{e^3+1} = \sigma(-3)$.
6. Row i: $\sum_j p_i(\delta_{ij} - p_j) = p_i - p_i \sum_j p_j = p_i - p_i = 0$. Because $\sum_i p_i = 1$ for every $\mathbf{z}$, the total cannot change when any z_j changes, so $\sum_i \frac{\partial p_i}{\partial z_j} = 0$ (column sums; the Jacobian is symmetric so row sums vanish too).
7. $w = 2/0.25 = 8$. $z = 8 \times (-0.1) = -0.8$, $P = \sigma(-0.8) = 0.3100$. With offset, $z = 8(r - 0.2)$, so $b = -8 \times 0.2 = -1.6$.
8. The composition is a linear map $y = \mathbf{v}^\mathsf{T}\mathbf{x}$ with $\mathbf{v}^\mathsf{T} = \mathbf{W}_3\mathbf{W}_2\mathbf{W}_1$. Linearity requires $y(\mathbf{a} + \mathbf{b}) = y(\mathbf{a}) + y(\mathbf{b})$. Take $\mathbf{a} = (1, 0)$, $\mathbf{b} = (0, 1)$: x_1x_2 gives 0 and 0, but 1 for $\mathbf{a} + \mathbf{b} = (1, 1)$. $0 + 0 \neq 1$, so no linear map computes x_1x_2.

Self-test quiz

1. What do the weights and the bias each control geometrically?
   The weights set the orientation (normal direction) of the boundary; the bias shifts it parallel to itself.
2. Why can't a single perceptron compute XOR?
   XOR's classes are not linearly separable; the four inequalities contradict each other.
3. What is the maximum derivative of the sigmoid, and where?
   0.25, at $z = 0$.
4. What does the pre-activation z of logistic regression represent?
   The log-odds $\ln \frac{p}{1-p}$ of class 1 (for BPSK, the LLR).
5. What happens to softmax as the temperature goes to zero?
   It approaches a one-hot argmax.
6. Why are activation functions necessary in hidden layers?
   Without them, the composition of linear layers is a single linear map.

Further reading

- F. Rosenblatt, "The perceptron: a probabilistic model for information storage and organization in the brain", *Psychological Review*, 65(6), 1958.
- M. Minsky and S. Papert, *Perceptrons*, MIT Press, 1969.
- V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines", ICML 2010; X. Glorot, A. Bordes and Y. Bengio, "Deep sparse rectifier neural networks", AISTATS 2011.
- D.-A. Clevert, T. Unterthiner and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units (ELUs)", ICLR 2016.
- D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)", arXiv:1606.08415, 2016.
- P. Ramachandran, B. Zoph and Q. V. Le, "Searching for activation functions", arXiv preprint 1710.05941, 2017 (Swish).

- C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, “On calibration of modern neural networks”, ICML 2017 (temperature scaling).

Chapter 7

Layers, Multilayer Perceptrons and Forward Propagation

One neuron draws one straight cut (Chapter 6). That is not enough even for XOR. In this chapter we put many neurons side by side to form a *layer*, and stack layers to form a *multilayer perceptron* (MLP), the basic "fully connected" neural network. You will learn to write a whole layer as one matrix multiplication, to track the shape of every array, and to run the *forward pass* — the computation from input to output — both by hand and in code. We then answer the big question: *what can such networks represent?* We will solve XOR with two hidden ReLUs, build a sine wave out of ReLU "hinges" to see why the *universal approximation theorem* holds, watch trained hidden layers bend and fold the input space, and compare deep and wide networks. Finally we count parameters and FLOPs, the two numbers that decide whether a network fits on your hardware.

The wireless application is a real one: a small MLP that replaces the 16-QAM soft demapper, learning to output the bit log-likelihood ratios that a channel decoder needs.

7.1 From one neuron to a layer

Intuition: a panel of detectors

Think of a layer as a panel of m feature detectors looking at the same input. Each neuron has its own weight vector (its own template) and its own bias, so each one asks a different question of the input: "is there energy at this frequency?", "is the amplitude roughly constant?". The layer's output is the vector of all m answers. The next layer then looks at *those answers* instead of the raw input, so it can ask more abstract questions.

Take an input $\mathbf{x} \in \mathbb{R}^d$ (one example, written as a *row* vector to match the batch-first convention of the book) and m neurons. Neuron j computes $z_j = \sum_{i=1}^{d} x_i W_{ij} + b_j$. Collect the weights in a matrix $\mathbf{W} \in \mathbb{R}^{d \times m}$ whose *column* j holds neuron j's weights, and the biases in a row $\mathbf{b} \in \mathbb{R}^m$. Then all m pre-activations are one vector–matrix product:

$$\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b}, \qquad \mathbf{a} = f(\mathbf{z}) \ \text{(applied element by element)}.$$

Definition: fully connected (dense) layer

A dense layer with d_{in} inputs and d_{out} outputs has parameters $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$. For a batch of N inputs stacked as rows of $\mathbf{X} \in \mathbb{R}^{N \times d_{\text{in}}}$ it computes

$$\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{b} \in \mathbb{R}^{N \times d_{\text{out}}}, \qquad \mathbf{A} = f(\mathbf{Z}). \tag{7.1}$$

Every output is connected to every input, hence "fully connected". In PyTorch this is `nn.Linear(d_in, d_out)`.[a]

[a] PyTorch stores the weight as $d_{\text{out}} \times d_{\text{in}}$ and computes $\mathbf{X}\mathbf{W}^{\mathsf{T}} + \mathbf{b}$; the mathematics is identical.

7.1.1 Shapes and broadcasting

Shapes are the grammar of deep learning; most bugs are shape bugs. In (7.1), $\mathbf{XW}$ is $(N \times d_{\text{in}})(d_{\text{in}} \times d_{\text{out}}) = (N \times d_{\text{out}})$: the inner dimensions must match and disappear. Row n of $\mathbf{Z}$ is the pre-activation vector of example n; column j is neuron j's value across the batch.

But $\mathbf{b}$ has shape (d_{out}), not $(N \times d_{\text{out}})$. Adding them uses **broadcasting**: the bias row is (conceptually) copied N times, once for each example, so that every example gets the same bias. NumPy and PyTorch do this automatically whenever the trailing dimensions agree (Fig. 7.1). Broadcasting is convenient but dangerous: if you accidentally give $\mathbf{b}$ the shape $(N \times 1)$, it will *also* broadcast — along the wrong axis — without any error.

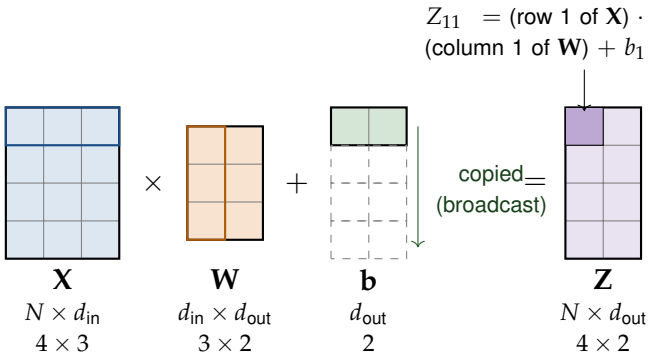


Figure 7.1: The shapes in one dense layer, for a batch of $N = 4$ examples with $d_{\text{in}} = 3$ features and $d_{\text{out}} = 2$ neurons. Each entry of $\mathbf{Z}$ is the dot product of one example (a row of $\mathbf{X}$) with one neuron's weights (a column of $\mathbf{W}$) plus that neuron's bias. The bias row is broadcast down the batch.

7.2 The multilayer perceptron

Stack L dense layers, each feeding the next, with a nonlinearity after every layer except (usually) the last. That is a **multilayer perceptron**. The layers between input and output are *hidden layers*, because their values are neither given in the data nor reported as output — the network invents them.

Definition: MLP and forward propagation

With $\mathbf{A}^{(0)} = \mathbf{X}$ and layer sizes $d_0, d_1, \ldots, d_L$, an MLP computes, for $\ell = 1, \ldots, L$,

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}, \qquad \mathbf{A}^{(\ell)} = f^{(\ell)}\big(\mathbf{Z}^{(\ell)}\big), \tag{7.2}$$

with $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d_{\ell-1} \times d_\ell}$ and $\mathbf{b}^{(\ell)} \in \mathbb{R}^{d_\ell}$. Hidden layers use a nonlinear f (ReLU, GELU, …); the output activation $f^{(L)}$ is chosen for the task (identity for regression, sigmoid for binary, softmax for multi-class). Computing (7.2) from $\ell = 1$ to L is **forward propagation**. The whole network is the function composition

$$\hat{\mathbf{Y}} = f^{(L)}\Big(\cdots f^{(2)}\big(f^{(1)}(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)})\mathbf{W}^{(2)} + \mathbf{b}^{(2)}\big)\cdots\Big).$$

The forward pass is just a loop:

```
import numpy as np
def relu(z):    return np.maximum(z, 0)
def softmax(z):
```

```
    e = np.exp(z - z.max(axis=1, keepdims=True))    # subtract max for stability
    return e / e.sum(axis=1, keepdims=True)

def forward(X, params):
    """params = [(W1, b1), (W2, b2), ...]; ReLU hidden layers, softmax output."""
    A = X
    for l, (W, b) in enumerate(params):
        Z = A @ W + b                                  # (N, d_in) @ (d_in, d_out) + (d_out,)
        A = softmax(Z) if l == len(params) - 1 else relu(Z)
    return A
```

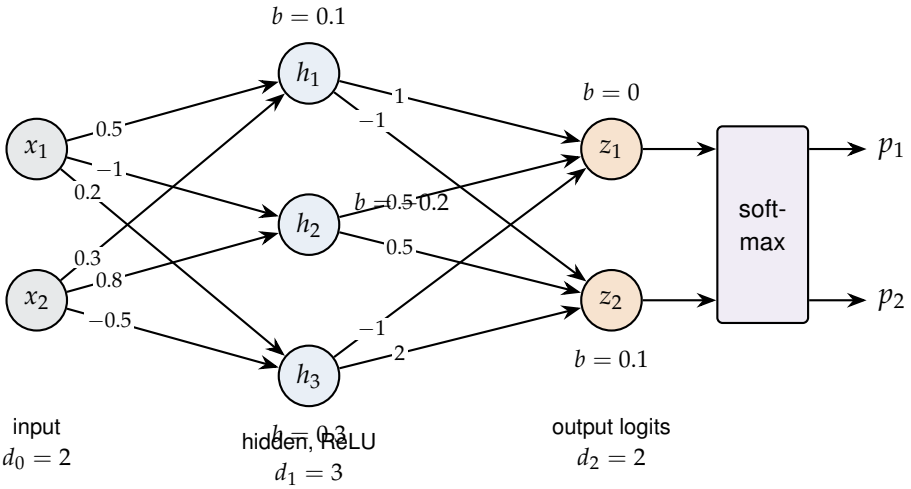


Figure 7.2: The 2-3-2 network used in the worked example, with every weight written on its edge and every bias next to its neuron. Hidden neurons use ReLU; the two output logits go through softmax.

Worked example: a complete forward pass through a 2-3-2 network

Parameters (Fig. 7.2):

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & -1.0 & 0.2 \\ 0.3 & 0.8 & -0.5 \end{bmatrix}, \qquad \mathbf{b}^{(1)} = (0.1,\ -0.2,\ 0.3),$$

$$\mathbf{W}^{(2)} = \begin{bmatrix} 1.0 & -1.0 \\ 0.5 & 0.5 \\ -1.0 & 2.0 \end{bmatrix}, \qquad \mathbf{b}^{(2)} = (0,\ 0.1).$$

Input $\mathbf{x} = (1,\ 2)$.

Layer 1, pre-activation. Each entry is $\mathbf{x}$ times a column of $\mathbf{W}^{(1)}$, plus a bias:

$$\begin{aligned} z_1^{(1)} &= 1(0.5) + 2(0.3) + 0.1 = 1.2, \\ z_2^{(1)} &= 1(-1.0) + 2(0.8) - 0.2 = 0.4, \\ z_3^{(1)} &= 1(0.2) + 2(-0.5) + 0.3 = -0.5. \end{aligned}$$

Layer 1, ReLU. $\mathbf{h} = \max(0, \mathbf{z}^{(1)}) = (1.2,\ 0.4,\ 0)$. The third neuron is silent for this input.

Layer 2, logits.

$$\begin{aligned} z_1^{(2)} &= 1.2(1.0) + 0.4(0.5) + 0(-1.0) + 0 = 1.4, \\ z_2^{(2)} &= 1.2(-1.0) + 0.4(0.5) + 0(2.0) + 0.1 = -0.9. \end{aligned}$$

Softmax. $e^{1.4} = 4.0552$, $e^{-0.9} = 0.4066$, sum 4.4618; so $\mathbf{p} = (0.9089,\ 0.0911)$. The network is 91% confident in class 1.

The same in batch form. Add a second example $(0, -1)$: $\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 0 & -1 \end{bmatrix}$. Then

$$\mathbf{Z}^{(1)} = \begin{bmatrix} 1.2 & 0.4 & -0.5 \\ -0.2 & -1.0 & 0.8 \end{bmatrix}, \qquad \mathbf{H} = \begin{bmatrix} 1.2 & 0.4 & 0 \\ 0 & 0 & 0.8 \end{bmatrix},$$
$$\mathbf{Z}^{(2)} = \begin{bmatrix} 1.4 & -0.9 \\ -0.8 & 1.7 \end{bmatrix}, \qquad \mathbf{P} = \begin{bmatrix} 0.9089 & 0.0911 \\ 0.0759 & 0.9241 \end{bmatrix}.$$

Each row is processed independently; one matrix multiplication handles the whole batch. (Verified in `code/ch07/verify_numbers.py`.)

7.2.1 Shapes and parameter count for a 784-128-10 network

The classic MNIST digit classifier flattens a 28×28 grey-scale image into $d_0 = 784$ inputs, uses one hidden layer of 128 ReLU units and 10 softmax outputs (one per digit). For a batch of $N = 64$ images:

array	shape	meaning	parameters
$\mathbf{X}$	64×784	batch of flattened images	—
$\mathbf{W}^{(1)}, \mathbf{b}^{(1)}$	$784 \times 128, 128$	hidden layer	$100{,}352 + 128 = 100{,}480$
$\mathbf{Z}^{(1)}, \mathbf{H}$	64×128	hidden pre-activations / activations	—
$\mathbf{W}^{(2)}, \mathbf{b}^{(2)}$	$128 \times 10, 10$	output layer	$1{,}280 + 10 = 1{,}290$
$\mathbf{Z}^{(2)}, \mathbf{P}$	64×10	logits / class probabilities	—
total			**101,770**

The general rule: a dense layer has $d_{\text{in}} d_{\text{out}} + d_{\text{out}}$ parameters. Note that 99% of them sit in the first layer — dense layers on raw high-dimensional input are expensive, which is one motivation for the convolutional layers of Part IV.

7.3 Solving XOR by hand

We proved in Chapter 6 that no single neuron computes XOR. Two hidden ReLU neurons suffice:

Key equation: an exact XOR network

$$h_1 = \text{ReLU}(x_1 + x_2), \qquad h_2 = \text{ReLU}(x_1 + x_2 - 1), \qquad y = h_1 - 2h_2.$$

In matrix form $\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$, $\mathbf{b}^{(1)} = (0, -1)$, $\mathbf{W}^{(2)} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$, $b^{(2)} = 0$. Check all four inputs:

(x_1, x_2)	$s = x_1 + x_2$	$h_1 = \text{ReLU}(s)$	$h_2 = \text{ReLU}(s-1)$	$y = h_1 - 2h_2$	XOR
(0,0)	0	0	0	0	0
(0,1)	1	1	0	1	1
(1,0)	1	1	0	1	1
(1,1)	2	2	1	0	0

Figure 7.3 shows how it works. Both hidden units look only at $s = x_1 + x_2$. h_1 rises with slope 1 from $s = 0$; h_2 switches on at $s = 1$. Subtracting $2h_2$ turns the rising line around at $s = 1$, giving a "tent" that is 1 at $s = 1$ and 0 at $s = 0$ and $s = 2$. In the input plane, the output is large in a diagonal band between the lines $x_1 + x_2 = 0$ and $x_1 + x_2 = 2$, peaked along $x_1 + x_2 = 1$ — exactly where the two "1" inputs live. And in the *hidden* space (h_1, h_2) the four inputs land at $(0,0)$, $(1,0)$ [twice] and $(2,1)$: now a single straight line separates the classes. The hidden layer has *re-represented* the data so that the output neuron's simple linear cut suffices. That is the essence of deep learning.

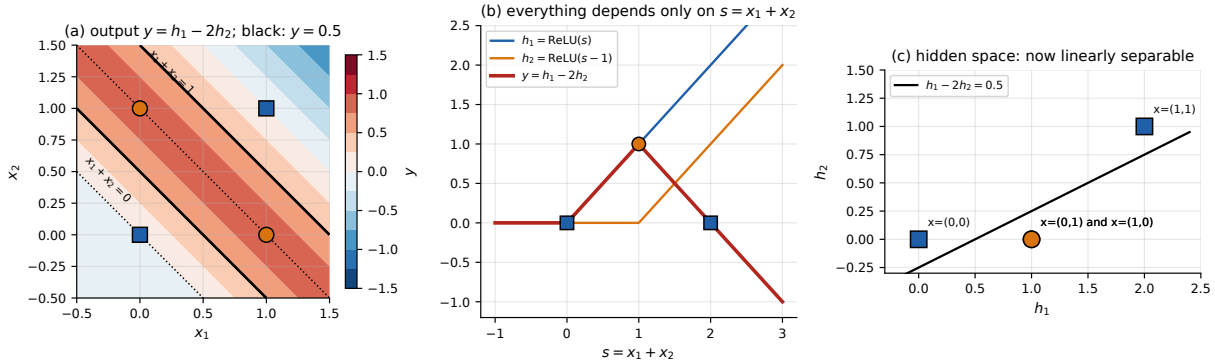


Figure 7.3: XOR solved by two hidden ReLUs. (a) The network output over the input plane: red where y is large; the black contours $y = 0.5$ enclose the band that contains the two class-1 points (circles). Dotted: the lines where the hidden units switch on. (b) Along $s = x_1 + x_2$: h_1, h_2 and their combination, a tent. (c) The four inputs in hidden space (h_1, h_2): the points $(0,1)$ and $(1,0)$ both map to $(1,0)$, and a straight line now separates the classes. (Script: `code/ch07/fig_xor_regions.py`.)

7.4 What can an MLP represent? Universal approximation

XOR needed two hidden units. How far does this go? Remarkably far.

Key equation: universal approximation theorem (one hidden layer)

Let f be any continuous function on a closed bounded set $K \subset \mathbb{R}^d$ (for example the cube $[0,1]^d$), and let $\varepsilon > 0$ be any tolerance. If the activation function is continuous and *not a polynomial* (sigmoid, tanh, ReLU, GELU all qualify), then there exist a width m and parameters $\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{w}^{(2)}, b^{(2)}$ such that the one-hidden-layer network

$$g(\mathbf{x}) = \sum_{j=1}^{m} w_j^{(2)}\, \sigma\big(\mathbf{x}\mathbf{W}_{:,j}^{(1)} + b_j^{(1)}\big) + b^{(2)}$$

satisfies $|g(\mathbf{x}) - f(\mathbf{x})| < \varepsilon$ for *every* $\mathbf{x} \in K$.

(Cybenko 1989 for sigmoids; Hornik 1991 for general bounded activations; Leshno et al. 1993 for the "non-polynomial" condition, which covers ReLU.) Read the statement carefully — it is as important to know what it does *not* say:

- It is an *existence* result. It says good weights exist, not that gradient descent will find them.
- It says nothing about how large m must be. For some functions the required width grows exponentially with the input dimension d.

- It says nothing about *generalization*: fitting a function on the training points is not the same as learning the right function from finite noisy data (Chapter 5).
- It applies on a bounded region K; outside it the network can do anything.

7.4.1 A constructive picture: building a sine wave from ReLU hinges

For one input and ReLU activations the theorem can be proved with a picture. A single hidden unit $c\ \mathrm{ReLU}(x-k)$ is a *hinge*: zero to the left of the knot k and a straight line of slope c to the right. A sum of hinges with knots $k_1 < k_2 < \dots$ is a piecewise-linear function whose slope changes by c_i at knot k_i. So to draw any polyline through points $(k_0, y_0), (k_1, y_1), \dots, (k_m, y_m)$:

1. set the output bias to y_0;
2. for the first segment, use a hinge at k_0 with $c_1 = s_1$, the slope of segment 1;
3. at each further knot k_i add a hinge with $c_{i+1} = s_{i+1} - s_i$, the *change* in slope.

That gives

$$g(x) = y_0 + \sum_{i=0}^{m-1} c_{i+1}\ \mathrm{ReLU}(x - k_i), \tag{7.3}$$

a one-hidden-layer network with m units, all input weights 1, biases $-k_i$ and output weights c_{i+1}. Any continuous function on an interval can be approximated by a polyline with enough knots, and the error of linear interpolation shrinks like $1/m^2$ for smooth functions.

Worked example: five hinges for $\sin x$ on $[0, 2\pi]$

Knots at $k_i = 2\pi i/5$: 0, 1.257, 2.513, 3.770, 5.027, 6.283, with values $\sin k_i =$ 0, 0.951, 0.588, −0.588, −0.951, 0. Segment slopes $s_i = (\sin k_i - \sin k_{i-1})/1.257$: 0.757, −0.289, −0.935, −0.289, 0.757. Output weights (changes of slope): $c_1 = 0.757$, $c_2 = -0.289 - 0.757 = -1.046$, $c_3 = -0.646$, $c_4 = +0.646$, $c_5 = +1.046$. Figure 7.4 shows the five weighted hinges and their running sum; the maximum error is 0.18.

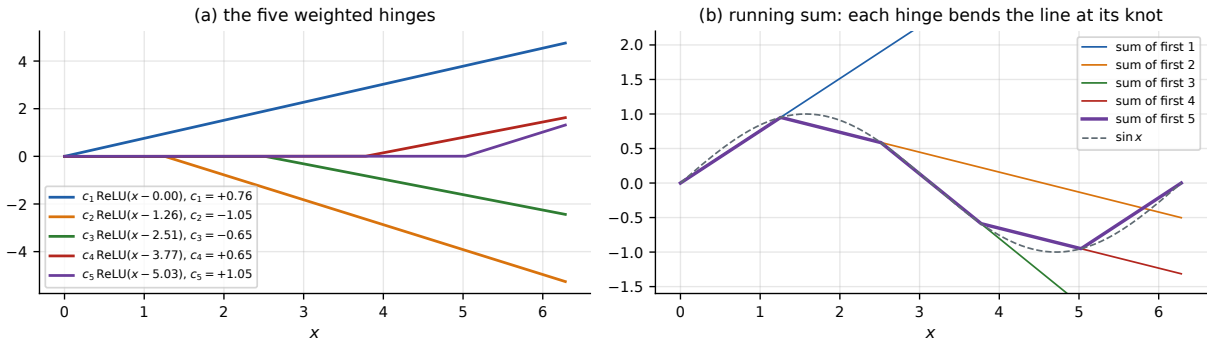


Figure 7.4: Building $\sin x$ from five ReLU hinges. (a) Each hidden unit contributes $c_i\,\mathrm{ReLU}(x - k_i)$: zero left of its knot, a ramp to the right. (b) Adding them one at a time: the first hinge draws the first segment, each further hinge bends the line by c_i at its knot. The final sum (thick purple) is the polyline through six points of the sine.

In more dimensions the same idea works with "bumps": a few ReLUs combine into a localized tent that is non-zero only in a small region, and a sum of many bumps of different heights can paint any continuous surface — the network becomes a kind of adaptive lookup table. Training

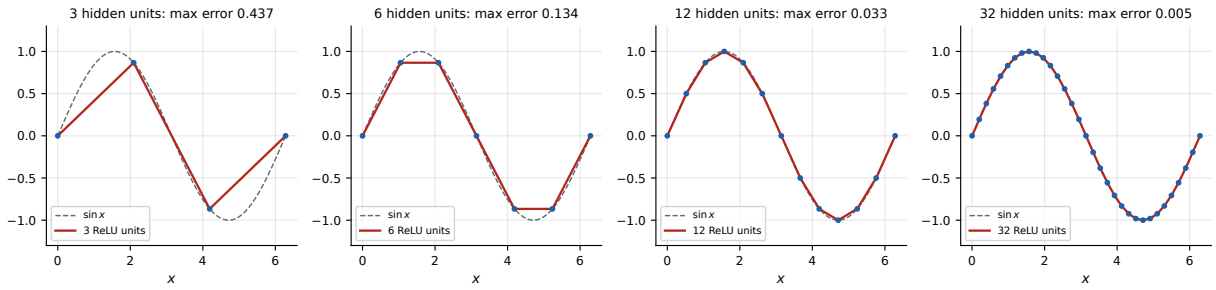


Figure 7.5: More hidden units, better approximation. One hidden layer with m ReLU units built by (7.3) approximates $\sin x$ with maximum error 0.44 ($m = 3$), 0.13 ($m = 6$), 0.033 ($m = 12$) and 0.005 ($m = 32$): roughly $1/m^2$, as interpolation theory predicts. (Script: `code/ch07/fig_uat_relu.py`.)

does not usually find such tidy constructions, but it is reassuring to know that the network *can* represent the answer.

7.5 How hidden layers bend and fold space

Intuition: a rubber sheet

Draw the training data on a rubber sheet with a square grid. Each layer stretches, rotates and shifts the sheet (the matrix $\mathbf{W}$ and bias $\mathbf{b}$), then bends it with the activation (tanh squashes everything into a box; ReLU folds negative half-planes flat onto the axes). After a few layers the sheet has been deformed so that the two classes lie on opposite sides of a straight line, and the output neuron only has to make that last straight cut.

To see this literally, we train a deliberately narrow network — 2 inputs, three hidden tanh layers of only 2 units each, and one sigmoid output — on the "two moons" dataset, which no straight line can separate. Because every hidden layer has two units, we can plot the data *in each layer's coordinates*, together with the image of the original square grid (Fig. 7.6). Layer by layer the moons are pulled apart, the grid gets curved and squashed, and the last hidden representation is linearly separable. Notice also what the network throws away: by the second layer it has collapsed the data onto a thin curve, keeping only the one direction that matters for the decision.

7.6 Depth versus width

The universal approximation theorem needs only one hidden layer. Why, then, does "deep" learning use many? Because depth can be exponentially more *efficient*. Consider the "tent" layer $t(u) = 2\,\mathrm{ReLU}(u) - 4\,\mathrm{ReLU}(u - \frac{1}{2})$, which uses two ReLU units and maps $[0, 1]$ onto $[0, 1]$ going up then down (two linear pieces). Composing it with itself, $t(t(u))$, makes four pieces; k compositions make 2^k pieces (Fig. 7.7). A deep network with k layers of two units ($2k$ neurons) thus produces a sawtooth with 2^k linear pieces, whereas a single hidden layer needs about 2^k units for the same function, because each ReLU in one layer adds at most one knot. Every layer *folds* the output of the previous one, and folds multiply.

In practice the trade-off looks like this:

- **Width** adds capacity in parallel; wide shallow nets are easy to train but parameter-hungry.
- **Depth** builds *hierarchical* features — edges → textures → parts → objects in images; samples

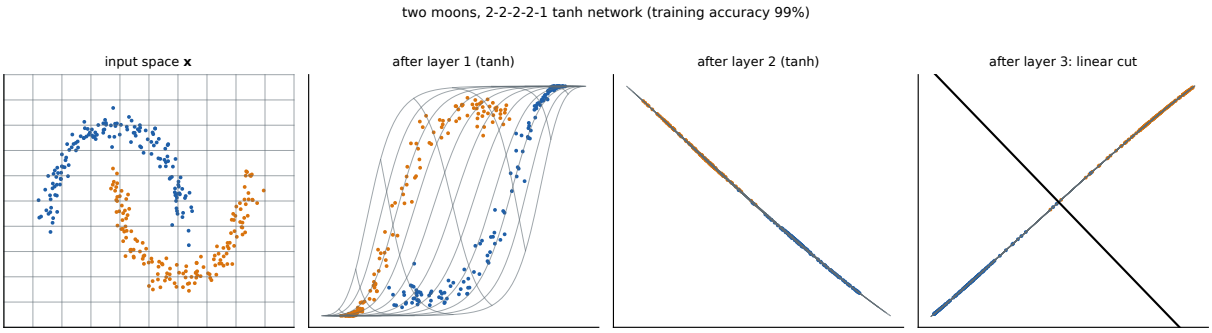


Figure 7.6: Hidden layers transform space. A 2-2-2-2-1 tanh network trained on the two-moons data (our own small experiment, 99% training accuracy). Left: the input space with a square grid. Then the data and the image of the grid after each hidden layer. Layer 1 bends the grid and pulls the moons apart; layers 2 and 3 squeeze the data onto an almost one-dimensional curve along which the two classes are *ordered*, so the output neuron's straight black line (right) separates them. Squeezing away directions that do not matter for the task is typical of trained networks. (Script: `code/ch07/fig_space_folding.py`.)

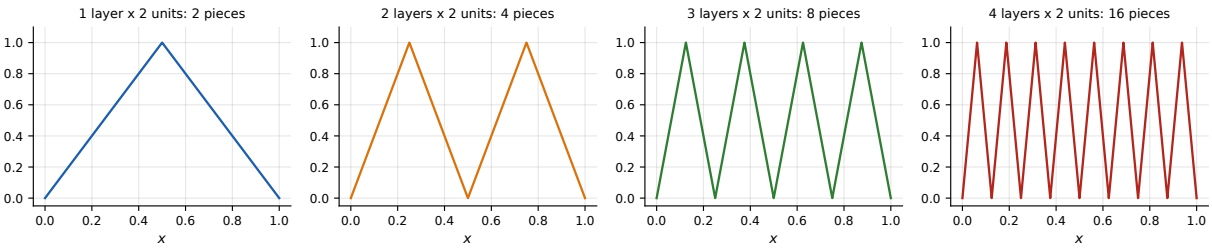


Figure 7.7: Depth multiplies, width adds. Composing a two-unit ReLU "tent" layer k times produces 2^k linear pieces with only $2k$ neurons. A single hidden layer would need about 2^k neurons for the same sawtooth. (Script: `code/ch07/fig_depth_sawtooth.py`.)

→ symbols → codewords in a receiver — and can represent many functions with far fewer parameters. But deep plain MLPs are harder to train (vanishing gradients, Chapter 10); initialization, normalization and residual connections (Chapters 11–12 and Part IV) are the tools that make depth work.

- For small tabular or signal-feature problems, 2–4 hidden layers of 32–512 units is a sensible starting point.

7.7 Counting FLOPs

How much computation does a forward pass cost? Each output of a dense layer needs d_{in} multiplications and d_{in} additions (counting the bias), i.e. d_{in} *multiply–accumulates* (MACs). A layer therefore costs $d_{\text{in}} d_{\text{out}}$ MACs per example, or about $2 d_{\text{in}} d_{\text{out}}$ floating-point operations (FLOPs), plus d_{out} cheap activation evaluations.

Worked example: FLOPs of the 784-128-10 network

MACs: $784 \times 128 + 128 \times 10 = 100{,}352 + 1{,}280 = 101{,}632$ per image; FLOPs $\approx 2 \times 101{,}632 \approx 2.0 \times 10^5$. A batch of 64 images costs 1.3×10^7 FLOPs — trivial for a laptop CPU, which sustains around 10^{10}–10^{11} FLOP/s. Training costs roughly three times the forward pass per example (the backward pass needs about twice the forward work, Chapter 10). Note that for dense layers MACs $\approx$ parameters: each weight is used once per example.

Wireless connection: FLOPs per bit on a DSP

These counts decide whether a neural receiver block is deployable. A 5G NR carrier with 273 PRBs and 14 OFDM symbols per slot carries $273 \times 12 \times 14 = 45{,}864$ resource elements per slot — every 0.5 ms at 30 kHz subcarrier spacing, i.e. about 9.2×10^7 REs per second per layer. A neural demapper that costs 1,000 MACs per RE therefore needs about 9×10^{10} MAC/s per spatial layer — far beyond a small DSP core, while a 50-MAC network is affordable. On fixed-point hardware such as a CEVA-class DSP, also count memory: weights (in 8- or 16-bit) must fit in local memory, and every activation must be quantized. Always compute MACs per RE (or per bit) before proposing a network for the physical layer.

7.8 Wireless application: an MLP soft demapper for 16-QAM

A modern receiver does not decide the bits of a QAM symbol directly. It computes, for every bit, a *log-likelihood ratio* (LLR) and passes those soft values to the LDPC or polar decoder. The *demapper* that computes them is a fixed function of the received sample — exactly the kind of mapping an MLP can learn. Here we define the exact target, train a small MLP to reproduce it, and compare the two.

7.8.1 The max-log LLR

16-QAM sends one of 16 points $s \in \mathcal{S}$, each labelled with 4 bits (b_0, b_1, b_2, b_3). We use Gray mapping per axis: bits (b_0, b_1) select the in-phase level and (b_2, b_3) the quadrature level by $00 \mapsto -3$, $01 \mapsto -1$, $11 \mapsto +1$, $10 \mapsto +3$, all scaled by $1/\sqrt{10}$ so that the average symbol energy is $E_s = 1$. Over AWGN, $y = s + n$ with $n \sim \mathcal{CN}(0, N_0)$, so $p(y \mid s) = \frac{1}{\pi N_0} e^{-|y-s|^2/N_0}$.

Definition: bit LLR and its max-log approximation

The LLR of bit k is

$$L_k(y) = \ln \frac{P(b_k = 0 \mid y)}{P(b_k = 1 \mid y)} = \ln \frac{\sum_{s \in \mathcal{S}_k^0} e^{-|y-s|^2/N_0}}{\sum_{s \in \mathcal{S}_k^1} e^{-|y-s|^2/N_0}},$$

where $\mathcal{S}_k^b$ is the set of the 8 constellation points whose bit k equals b (equally likely symbols assumed). Replacing each sum of exponentials by its largest term ($\ln \sum_i e^{a_i} \approx \max_i a_i$) gives the **max-log LLR**

$$L_k(y) \approx \frac{1}{N_0} \left[\min_{s \in \mathcal{S}_k^1} |y - s|^2 - \min_{s \in \mathcal{S}_k^0} |y - s|^2 \right]. \tag{7.4}$$

Positive L_k means bit k is more likely 0; $|L_k|$ is the confidence.

Worked example: max-log LLRs for one received sample

$y = 0.4 + 0.1j$, $N_0 = 0.1$ ($E_s/N_0 = 10$ dB); write $d = 1/\sqrt{10} = 0.3162$. The nearest point overall is $d(1 + j) = 0.316 + 0.316j$ (bits 1111), at squared distance 0.0538. For b_0 (sign of the I level), the nearest point with $b_0 = 0$ is $d(-1 + j)$ at squared distance 0.5597: $L_0 = (0.0538 - 0.5597)/0.1 = -5.06$ (confidently 1). For b_1 (inner versus outer I level), the nearest point with $b_1 = 0$ is $d(3 + j)$ at 0.3478: $L_1 = -2.94$. Similarly $L_2 = -1.26$ (the sample is close to the I axis, so the sign of Q is uncertain) and $L_3 = -6.74$.

For square QAM with Gray mapping the max-log LLRs are piecewise-linear functions of y_I and y_Q separately: for example, for b_1, $L_1 = \frac{4d}{N_0}(|y_I| - 2d)$. Piecewise-linear functions are exactly what ReLU networks produce, so a small ReLU MLP should learn this demapper easily. Because $N_0 L_k$ does not depend on the noise level, we train the network to output $N_0 L_k$ and divide by N_0 afterwards; one network then serves all SNRs.

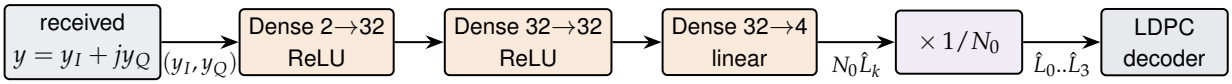


Figure 7.8: The neural soft demapper. Two real inputs, two hidden ReLU layers of 32 units, four linear outputs (one per bit) trained to match $N_0 L_k$; scaling by $1/N_0$ gives LLRs for the decoder. The network has 1,284 parameters and costs 1,216 MACs per symbol.

The training script draws 20,000 received samples at random E_s/N_0 between 0 and 20 dB, computes the targets with (7.4), and fits the 2-32-32-4 network of Fig. 7.8 by minimizing the mean squared error with Adam (both explained in later chapters):

```
# ---- training data: received samples at a range of SNRs (target N0*LLR is SNR-independent)
N = 20000
s = const[rng.integers(0, 16, N)]
N0_tr = 10 ** (-rng.uniform(0, 20, N) / 10)
y = s + np.sqrt(N0_tr / 2) * (rng.standard_normal(N) + 1j * rng.standard_normal(N))
X = torch.tensor(np.c_[y.real, y.imag], dtype=torch.float32)
T = torch.tensor(maxlog_llr_N0(y), dtype=torch.float32)

net = nn.Sequential(nn.Linear(2, 32), nn.ReLU(), nn.Linear(32, 32), nn.ReLU(), nn.Linear(32, 4))
opt = torch.optim.Adam(net.parameters(), lr=3e-3)
```

```
sched = torch.optim.lr_scheduler.StepLR(opt, 800, 0.3)
for step in range(2400):
    idx = torch.randint(0, N, (1024,))
    opt.zero_grad(); loss = ((net(X[idx]) - T[idx]) ** 2).mean(); loss.backward();
     opt.step(); sched.step()
    if step % 600 == 0 or step == 2399: print(step, 'mse', round(loss.item(), 5))
```

Figure 7.9 shows the result at $E_s/N_0 = 10$ dB. Along a horizontal cut through the constellation the MLP's LLRs for b_0 and b_1 lie on top of the max-log target, which itself is very close to the exact log-MAP LLR (the max-log approximation errs only near decision boundaries, where two terms of the sum are comparable). On 5,000 test symbols the RMS LLR error is 0.12 against an RMS LLR magnitude of 9.5, and the MLP's hard decisions agree with max-log's for 99.7% of bits (the disagreements are LLRs very close to zero). The right panel shows the learned LLR of b_1 over the whole I/Q plane: a function of y_I only, negative between the inner columns and positive outside, with zero crossings at $y_I = \pm 2d$ — the network has discovered the Gray-mapping structure by itself.

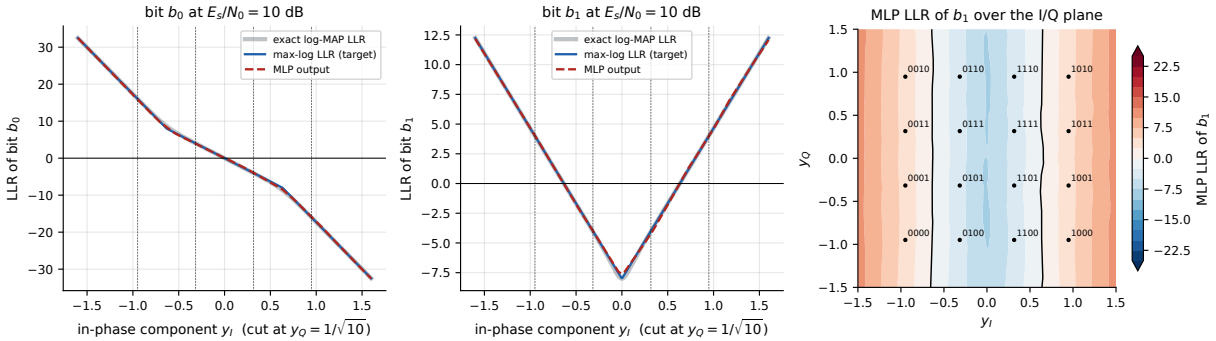


Figure 7.9: Our MLP soft demapper for Gray 16-QAM at $E_s/N_0 = 10$ dB. Left and centre: LLRs of bits b_0 (sign of y_I) and b_1 (inner/outer column) along the cut $y_Q = 1/\sqrt{10}$; the MLP output (dashed red) is indistinguishable from the max-log target (blue), and both track the exact log-MAP LLR (thick grey). Dotted verticals mark the constellation's I levels. Right: the MLP's LLR of b_1 over the I/Q plane with the bit labels of the 16 points; the black contour is the learned decision boundary $\hat{L}_1 = 0$. (Script: `code/ch07/mlp_demapper.py`.)

Wireless connection: why learn a demapper we can write down?

For Gray QAM over AWGN the max-log demapper is cheap and exact, so nobody would replace it with an MLP. The point of this exercise is the *method*: the same network and training loop work when the closed form is unavailable or wrong — with phase noise or I/Q imbalance distorting the constellation, non-Gaussian interference, nonlinear PA distortion, or a *learned* constellation from an end-to-end autoencoder (Part V), where no textbook formula exists. It also hints at training the demapper directly for decoding performance (bit-wise binary cross-entropy, Chapter 8) instead of imitating max-log. And the cost matters: our 1,216 MACs per symbol is far more than a max-log demapper needs (a few comparisons and multiplications per bit) — see the FLOP box above.

Pitfall: common forward-pass bugs

(i) Transposed weights: an `(N, d_in) @ (d_out, d_in)` error, or worse a silent bug when $d_{\text{in}} = d_{\text{out}}$. (ii) Bias of the wrong shape broadcasting across the batch. (iii) A missing

activation between layers (the network collapses to linear, Chapter 6). (iv) Applying softmax and then a loss that applies it again. (v) Unscaled inputs: features of size 1000 next to features of size 0.001 make some neurons saturate immediately — standardize inputs to zero mean and unit variance.

Chapter recap

- A dense layer: $\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{b}$, $\mathbf{X} \in \mathbb{R}^{N \times d_{\text{in}}}$, $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$, bias broadcast over the batch; $d_{\text{in}} d_{\text{out}} + d_{\text{out}}$ parameters.
- An MLP alternates dense layers and nonlinearities; the forward pass applies them in order. 784-128-10 has 101,770 parameters and about 2×10^5 FLOPs per example.
- Worked 2-3-2 example: $\mathbf{x} = (1, 2) \to \mathbf{h} = (1.2, 0.4, 0) \to \mathbf{z} = (1.4, -0.9) \to \mathbf{p} = (0.909, 0.091)$.
- XOR: $h_1 = \text{ReLU}(x_1 + x_2)$, $h_2 = \text{ReLU}(x_1 + x_2 - 1)$, $y = h_1 - 2h_2$; the hidden layer makes the data linearly separable.
- Universal approximation: one hidden layer with enough non-polynomial units approximates any continuous function on a bounded set — existence only. ReLU hinges make the construction explicit.
- Depth composes folds: k layers can make 2^k linear pieces; width adds pieces only linearly.
- An MLP can learn the 16-QAM max-log demapper almost exactly; always count MACs per symbol before deploying.

Exercises

1. **(Shapes.)** A network maps 64 inputs through hidden layers of 256 and 128 units to 8 outputs. Give the shape of every weight matrix and bias, the total number of parameters, and the shape of every activation for a batch of 32.
2. **(Forward pass.)** Using the 2-3-2 network of the worked example, compute $\mathbf{h}$, $\mathbf{z}^{(2)}$ and $\mathbf{p}$ for $\mathbf{x} = (-1, 1)$.
3. **(XOR variants.)** Modify the XOR network so that it computes XNOR (1 when the inputs are equal). Hint: you only need to change the output layer.
4. **(Hinges.)** Write $g(x) = |x|$ on $[-1, 1]$ as a one-hidden-layer ReLU network with two hidden units. Then build a network for the "tent" that is 0 for $x \le 0$, rises to 1 at $x = 1$, and returns to 0 at $x = 2$ (and stays 0 after).
5. **(Linear collapse.)** Show that if the hidden activation in the XOR network were the identity, the output would be $y = -(x_1 + x_2) + 2$, and that this cannot compute XOR.
6. **(FLOPs.)** How many MACs per example does the network of Exercise 1 need? If it processes every RE of a 273-PRB, 14-symbol slot every 0.5 ms, how many MAC/s is that?
7. **(Max-log LLR.)** For Gray 16-QAM as defined in Section 7.8, compute the max-log LLR of b_0 for $y = -0.1 + 0.9j$ at $N_0 = 0.2$. Which constellation points determine it?

8. **(Concept.)** The universal approximation theorem guarantees a one-hidden-layer network can fit any continuous function. Give three reasons why practitioners still use deeper networks and still worry about whether a network will work.

Answers

1. $\mathbf{W}^{(1)}$: 64×256, $\mathbf{b}^{(1)}$: 256; $\mathbf{W}^{(2)}$: 256×128, $\mathbf{b}^{(2)}$: 128; $\mathbf{W}^{(3)}$: 128×8, $\mathbf{b}^{(3)}$: 8. Parameters: $16{,}384 + 256 + 32{,}768 + 128 + 1{,}024 + 8 = 50{,}568$. Activations: $\mathbf{X}$ 32×64, $\mathbf{H}^{(1)}$ 32×256, $\mathbf{H}^{(2)}$ 32×128, output 32×8.

2. $\mathbf{z}^{(1)} = (-0.5 + 0.3 + 0.1,\ 1 + 0.8 - 0.2,\ -0.2 - 0.5 + 0.3) = (-0.1,\ 1.6,\ -0.4)$; $\mathbf{h} = (0, 1.6, 0)$. $\mathbf{z}^{(2)} = (0.8,\ 0.8 + 0.1) = (0.8, 0.9)$. $\mathbf{p} = \mathrm{softmax}(0.8, 0.9) = (0.475, 0.525)$.

3. XNOR $= 1 -$ XOR: use $y = 1 - h_1 + 2h_2$ (output weights $(-1, 2)$, output bias 1). Check: $(0,0) \to 1$, $(0,1) \to 0$, $(1,0) \to 0$, $(1,1) \to 1 - 2 + 2 = 1$.

4. $|x| = \mathrm{ReLU}(x) + \mathrm{ReLU}(-x)$ (input weights $+1, -1$, biases 0, output weights 1, 1). Tent: $\mathrm{ReLU}(x) - 2\,\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2)$: slope $+1$ from 0, changes by -2 at 1 (to -1), by $+1$ at 2 (to 0). Three hidden units.

5. With identity activations $h_1 = x_1 + x_2$, $h_2 = x_1 + x_2 - 1$, $y = h_1 - 2h_2 = -(x_1 + x_2) + 2$. Values: $(0,0) \to 2$, $(0,1), (1,0) \to 1$, $(1,1) \to 0$. It is a linear function of $s = x_1 + x_2$, monotone in s, so it cannot be high at $s = 1$ and low at both $s = 0$ and $s = 2$; no threshold reproduces XOR.

6. MACs $= 64 \cdot 256 + 256 \cdot 128 + 128 \cdot 8 = 16{,}384 + 32{,}768 + 1{,}024 = 50{,}176$. REs per second $= 45{,}864 / 0.0005 = 9.17 \times 10^7$; total $\approx 4.6 \times 10^{12}$ MAC/s — impossible on a DSP, a reminder to keep PHY networks tiny.

7. b_0 is the sign of the I level ($b_0 = 0$: I $\in \{-3d, -d\}$; $b_0 = 1$: I $\in \{d, 3d\}$) and does not depend on Q, so take the best Q level ($3d = 0.949$, closest to 0.9) for both. Nearest $b_0 = 0$ point: $d(-1 + 3j) = -0.316 + 0.949j$, squared distance $0.216^2 + 0.049^2 = 0.0491$. Nearest $b_0 = 1$ point: $d(1 + 3j) = 0.316 + 0.949j$, $0.416^2 + 0.049^2 = 0.1756$. $L_0 = (0.1756 - 0.0491)/0.2 = +0.63$: weakly favours bit 0. (Equivalently $L_0 = -4d\, y_I / N_0 = 0.63$ in the inner region.)

8. (i) The required width can be astronomically large; depth represents many functions with exponentially fewer units (sawtooth example). (ii) Existence of weights says nothing about whether gradient descent finds them. (iii) Fitting the training data says nothing about generalizing to new data; inductive biases of the architecture (depth, convolution, attention) strongly affect generalization.

Self-test quiz

1. In $\mathbf{Z} = \mathbf{XW} + \mathbf{b}$, what does column j of $\mathbf{W}$ hold?
   The input weights of neuron j.

2. How many parameters does a dense layer from 100 to 50 units have?
   $100 \times 50 + 50 = 5{,}050$.

3. What does broadcasting do with the bias?
   Adds the same bias row to every example (row) of the batch.

4. In the XOR network, where do the inputs $(0,1)$ and $(1,0)$ land in hidden space?
   Both at $(h_1, h_2) = (1,0)$.
5. What does the universal approximation theorem *not* guarantee?
   A practical width, that training finds the weights, or generalization.
6. Why does a ReLU network fit max-log LLRs of Gray QAM so well?
   Those LLRs are piecewise-linear in y_I, y_Q, which is exactly what ReLU networks compute.

Further reading

- G. Cybenko, "Approximation by superpositions of a sigmoidal function", *Mathematics of Control, Signals and Systems*, 2(4), 1989.
- K. Hornik, "Approximation capabilities of multilayer feedforward networks", *Neural Networks*, 4(2), 1991.
- M. Leshno, V. Ya. Lin, A. Pinkus and S. Schocken, "Multilayer feedforward networks with a nonpolynomial activation function can approximate any function", *Neural Networks*, 6(6), 1993.
- M. Telgarsky, "Benefits of depth in neural networks", COLT 2016 — the sawtooth construction.
- C. Olah, "Neural networks, manifolds, and topology", blog post, 2014 — beautiful pictures of layers deforming space.
- O. Shental and J. Hoydis, "Machine LLRning: learning to softly demodulate", IEEE Globecom Workshops, 2019 — learned demappers for LLR computation.
- S. Dörner, S. Cammerer, J. Hoydis and S. ten Brink, "Deep learning based communication over the air", *IEEE J. Sel. Topics Signal Process.*, 2018.

Chapter 8

Loss Functions and Maximum Likelihood

A network's forward pass produces a prediction. Training needs one more ingredient: a single number that says *how wrong* that prediction is, so that gradient descent has something to push downhill. That number is the *loss*. Choosing it is not a matter of taste: almost every loss used in practice follows from one principle, *maximum likelihood*, applied to an assumption about how the data are generated. Assume Gaussian noise and you get the mean squared error; assume coin flips and you get binary cross-entropy; assume a die with K faces and you get categorical cross-entropy. In this chapter we derive each of them, compute their gradients by hand (including the two most important results in deep learning, $\frac{\partial L}{\partial z} = a - y$ and $\frac{\partial L}{\partial z} = \mathbf{p} - \mathbf{y}$), see why the "obvious" choice of MSE for classification fails, and learn the numerically stable ways that every real implementation uses. We finish with practical refinements: multi-label outputs, label smoothing, class weights and the focal loss.

For a communications engineer this chapter is a homecoming. Cross-entropy is a mutual-information bound, bit-wise cross-entropy trains networks whose outputs *are* LLRs, and MSE is the familiar channel-estimation criterion.

8.1 The loss as a measure of wrongness

Intuition: a fine for every mistake

Imagine a teacher who fines the student for every answer: a small fine for a small mistake, a big fine for a big one, nothing for a perfect answer. The student's goal is to minimize the total fine. The *fining rule* decides what the student learns to care about: fine squared errors and the student avoids large blunders at all costs; fine absolute errors and the student tolerates the occasional blunder but aims for the median; fine confident wrong answers enormously and the student learns to be honest about uncertainty.

Definition: loss and cost

A **loss function** $\ell(\hat{y}, y) \geq 0$ measures the error of one prediction $\hat{y}$ against its target y. The training objective (also called the cost or empirical risk) is the average over the N training examples,

$$L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell\big(f(\mathbf{x}_i; \theta),\, y_i\big).$$

Training means $\theta^\star = \arg\min_\theta L(\theta)$.

A good loss must be (i) small when predictions are good, (ii) *differentiable* (almost everywhere) with useful gradients, because gradient descent only sees the slope, and (iii) aligned with what we actually care about. Point (iii) is subtle: we often care about accuracy or bit-error rate, but these are step functions of the parameters (a tiny weight change either flips a decision or does nothing), so their gradient is zero almost everywhere. We therefore train on a smooth *surrogate* loss and *report* the metric we care about (Fig. 8.1).

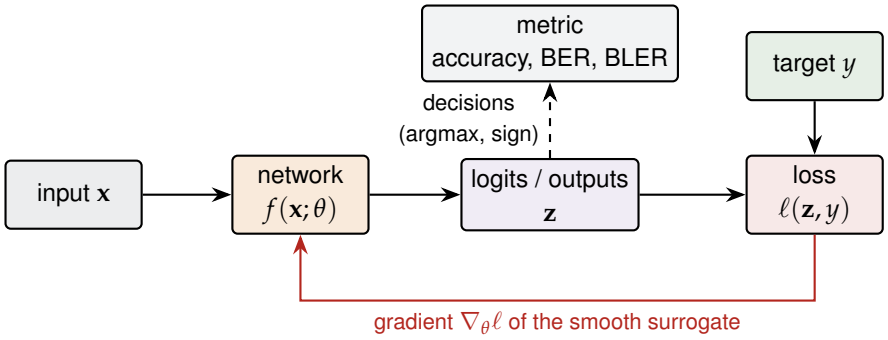


Figure 8.1: The loss is the smooth quantity we differentiate and minimize; the metric (accuracy, BER, BLER) is what we report. Metrics depend on hard decisions and have zero gradient almost everywhere, so they cannot drive training directly.

8.2 The maximum likelihood principle

Where do good losses come from? Treat the network as a *probabilistic model*: for each input $\mathbf{x}$ it specifies a probability distribution over possible targets, $p(y \mid \mathbf{x}; \theta)$. Different parameters make the observed training labels more or less probable.

Definition: likelihood and maximum likelihood

For independent training examples, the **likelihood** of the parameters is the probability the model assigns to the observed labels,

$$\mathcal{L}(\theta) = \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i; \theta).$$

The **maximum likelihood (ML) estimate** is the θ that makes the data most probable: $\theta_{\mathrm{ML}} = \arg\max_\theta \mathcal{L}(\theta)$.

Products of thousands of numbers below 1 underflow to zero on a computer, and products are awkward to differentiate. The logarithm fixes both: it is increasing (so it does not move the maximum) and it turns products into sums. Maximizing the log-likelihood is the same as minimizing its negative, averaged over examples:

Key equation: negative log-likelihood (NLL)

$$\theta_{\mathrm{ML}} = \arg\min_\theta \underbrace{-\frac{1}{N}\sum_{i=1}^{N} \ln p(y_i \mid \mathbf{x}_i; \theta)}_{\text{NLL loss}}.$$

So the recipe for a loss is: *(1) decide what distribution the targets follow given the network output; (2) write down* $-\ln p$. Each choice of distribution produces a famous loss (Table 8.1 and Fig. 8.2).

Wireless connection: maximum likelihood is already your friend

ML is not new to you: the ML detector picks the symbol $\hat{s} = \arg\max_s p(\mathbf{y} \mid s)$, which in AWGN is the nearest constellation point; ML channel estimation from pilots in Gaussian noise is least squares. Training a neural network by ML is the same idea applied to the *parameters*

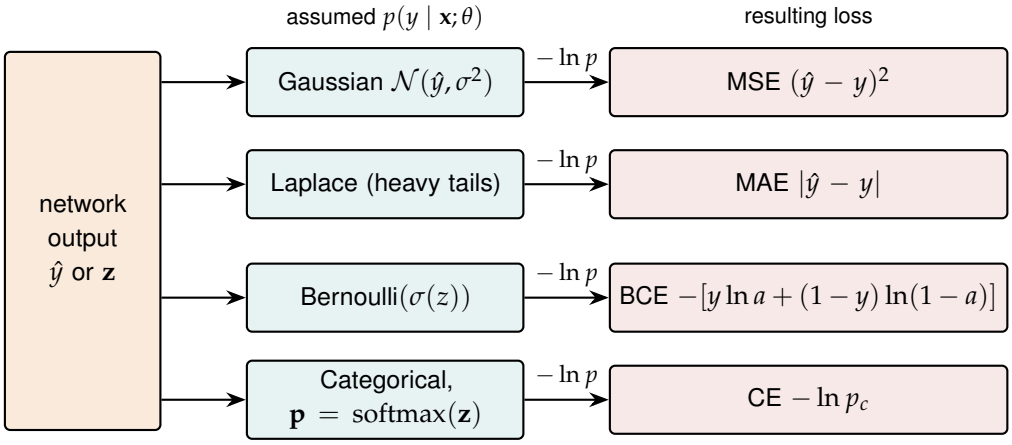


Figure 8.2: The maximum-likelihood recipe. The network output parameterizes a probability distribution over the target; taking the negative log of that distribution at the observed target gives the loss. Every standard loss in this chapter is produced this way.

instead of the transmitted symbols: choose the weights under which the observed training labels are most probable.

8.3 Regression losses

8.3.1 Mean squared error from Gaussian noise

Assume the target is the network output plus Gaussian noise: $y = \hat{y} + \epsilon$ with $\epsilon \sim \mathcal{N}(0,\sigma^2)$ and $\hat{y} = f(\mathbf{x};\theta)$. Then

$$p(y \mid \mathbf{x};\theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\!\Big(-\frac{(y-\hat{y})^2}{2\sigma^2}\Big), \qquad -\ln p(y \mid \mathbf{x};\theta) = \frac{(y-\hat{y})^2}{2\sigma^2} + \frac{1}{2}\ln(2\pi\sigma^2).$$

The second term does not depend on θ, and the factor $1/(2\sigma^2)$ only scales the loss. Dropping both:

Key equation: MSE is the Gaussian NLL

$$L_{\mathrm{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2, \qquad \frac{\partial L_{\mathrm{MSE}}}{\partial \hat{y}_i} = \frac{2}{N}(\hat{y}_i - y_i).$$

The gradient is proportional to the error: big errors get big corrections. The MSE minimizer, for a fixed input, is the *mean* of the targets (set $\frac{\mathrm{d}}{\mathrm{d}c}\sum_i (c-y_i)^2 = 2\sum_i (c-y_i) = 0$, so $c = \bar{y}$). For vector targets, sum the squares over components ($\|\hat{\mathbf{y}} - \mathbf{y}\|^2$), which corresponds to independent Gaussian noise on each component.

Worked example: MSE and its gradient

Predictions $\hat{\mathbf{y}} = (2.5,\ 0,\ 2.1)$, targets $\mathbf{y} = (3,\ -0.5,\ 2)$. Residuals $(-0.5,\ 0.5,\ 0.1)$; MSE $= (0.25 + 0.25 + 0.01)/3 = 0.17$. Gradient $\frac{2}{3}(-0.5, 0.5, 0.1) = (-0.333,\ 0.333,\ 0.067)$: gradient descent raises the first prediction, lowers the second, and barely touches the third. The MAE (next) is $(0.5 + 0.5 + 0.1)/3 = 0.367$.

8.3.2 Mean absolute error and Huber

If the noise has heavier tails — a Laplace distribution $p(\epsilon) = \frac{1}{2b} e^{-|\epsilon|/b}$ — the NLL is $|y - \hat{y}|/b +$ const, giving the **mean absolute error** $L_{\text{MAE}} = \frac{1}{N} \sum_i |\hat{y}_i - y_i|$ with gradient $\frac{1}{N} \operatorname{sign}(\hat{y}_i - y_i)$. Every example pushes with the same force, whatever the size of its error, so a few wild outliers cannot dominate. Its minimizer is the *median*. The downside: the gradient does not shrink near the optimum, so training jitters around it, and it is not differentiable at zero.

The **Huber loss** combines the two: quadratic for small residuals, linear for large ones,

$$\ell_\delta(r) = \begin{cases} \frac{1}{2} r^2 & |r| \le \delta \\ \delta\left(|r| - \frac{1}{2}\delta\right) & |r| > \delta \end{cases}, \qquad \ell'_\delta(r) = \operatorname{clip}(r, -\delta, \delta).$$

The two pieces meet with equal value ($\frac{1}{2}\delta^2$) and equal slope (δ) at $|r| = \delta$, so the loss is smooth. Numbers ($\delta = 1$): $\ell(0.5) = 0.125$, $\ell(3) = 1 \cdot (3 - 0.5) = 2.5$ (MSE would give 9, or 4.5 with the $\frac{1}{2}$). Its gradient is the residual *clipped* at $\pm\delta$: it behaves like MSE near the optimum and like MAE for outliers (Fig. 8.3a,b). In PyTorch, `SmoothL1Loss` is Huber with $\delta = 1$ divided by δ.

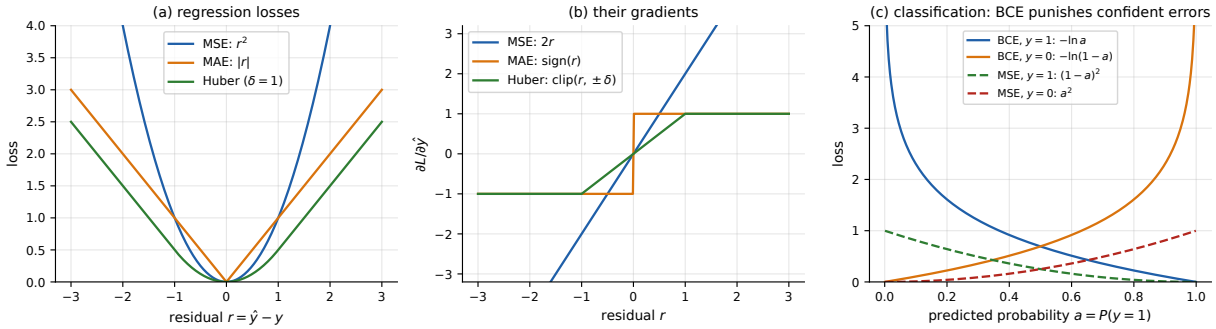


Figure 8.3: (a) Regression losses as a function of the residual $r = \hat{y} - y$: MSE grows quadratically, MAE linearly, Huber is quadratic inside $|r| \le 1$ and linear outside. (b) Their gradients: MSE's grows without bound, MAE's has constant magnitude, Huber's is the residual clipped to ± 1. (c) Classification losses as a function of the predicted probability a of class 1: binary cross-entropy (solid) goes to infinity for confident wrong predictions; MSE (dashed) never exceeds 1. (Script: `code/ch08/fig_losses.py`.)

Figure 8.4 shows the practical difference: fitting a line to 30 points of which 3 are gross outliers. MSE is dragged far off the true line; MAE and Huber barely notice.

Wireless connection: MSE for channel estimation and CSI prediction

Channel estimation and prediction are regression problems on complex numbers: the network outputs $\hat{h}$ (as two real numbers, real and imaginary part, per resource element or antenna), and the natural loss is $\frac{1}{N} \sum |\hat{h}_i - h_i|^2$ — the MSE you already use to judge LS and LMMSE estimators. Two details matter in practice. (i) Channel power varies by tens of dB between users and slots, so a plain MSE is dominated by the strong channels; the field therefore reports and often trains on the *normalized* MSE, $\text{NMSE} = \mathbb{E}\left[\left\|\hat{\mathbf{h}} - \mathbf{h}\right\|^2 / \|\mathbf{h}\|^2\right]$, usually in dB. (ii) If the estimates will be used for precoding, the error that matters is in the beamforming gain, not in every coefficient; losses based on the cosine similarity $|\hat{\mathbf{h}}^{\mathsf{H}}\mathbf{h}| / (\|\hat{\mathbf{h}}\| \, \|\mathbf{h}\|)$ are common in CSI-feedback work. When the impairments are impulsive (bursty interference), a Huber loss gives a more robust estimator than MSE.

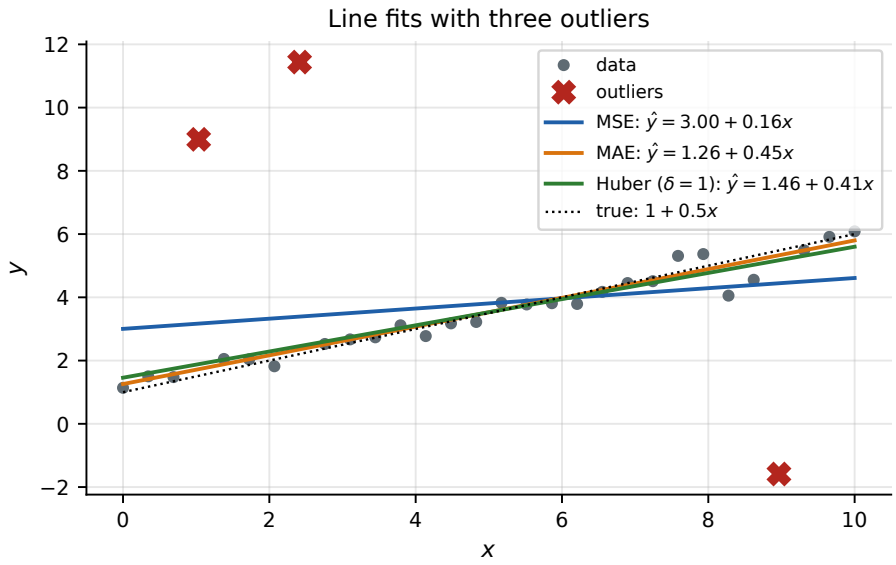


Figure 8.4: Robustness to outliers. Fitting $\hat{y} = b + wx$ by minimizing MSE, MAE or Huber loss on data from $y = 1 + 0.5x$ plus small noise, with three gross outliers (red crosses). The squared error lets the outliers pull the MSE line badly off course (slope 0.16 instead of 0.5); MAE and Huber recover slopes 0.45 and 0.41. (Script: `code/ch08/fig_losses.py`.)

8.4 Binary classification: binary cross-entropy

8.4.1 Derivation from the Bernoulli likelihood

For a yes/no label $y \in \{0, 1\}$, the network outputs one logit z and the probability $a = \sigma(z) = P(y = 1 \mid \mathbf{x})$ (logistic regression, Chapter 6). The label is a biased coin flip, a *Bernoulli* variable:

$$p(y \mid \mathbf{x}) = a^y (1 - a)^{1-y} \qquad \text{(this is } a \text{ if } y = 1,\ 1 - a \text{ if } y = 0\text{)}.$$

Taking $-\ln$:

Key equation: binary cross-entropy (BCE)

$$\ell_{\text{BCE}}(a, y) = -\big[y \ln a + (1 - y) \ln(1 - a)\big], \qquad a = \sigma(z).$$

Only one term is active per example: $-\ln a$ when $y = 1$, $-\ln(1 - a)$ when $y = 0$. If the model says $a = 0.9$ and $y = 1$, the loss is $-\ln 0.9 = 0.105$; if it says $a = 0.01$ and $y = 1$, the loss is $-\ln 0.01 = 4.6$; as $a \to 0$ with $y = 1$ the loss goes to infinity (Fig. 8.3c). Confident mistakes are punished without limit.

8.4.2 The gradient: $\frac{\partial \ell}{\partial z} = a - y$

Use the chain rule through $a = \sigma(z)$, with $\frac{\mathrm{d}a}{\mathrm{d}z} = a(1 - a)$ (Chapter 6):

$$\frac{\partial \ell}{\partial a} = -\frac{y}{a} + \frac{1 - y}{1 - a} = \frac{-y(1 - a) + (1 - y)a}{a(1 - a)} = \frac{a - y}{a(1 - a)}.$$

Multiplying by $\frac{\mathrm{d}a}{\mathrm{d}z} = a(1 - a)$, the denominators cancel exactly:

Key equation: BCE gradient with respect to the logit

$$\frac{\partial \ell_{\text{BCE}}}{\partial z} = \frac{a - y}{a(1-a)} \cdot a(1-a) = a - y.$$

This is one of the most beautiful results in machine learning. The gradient is simply "prediction minus target": at most 1 in magnitude, largest when the model is most wrong, and zero when it is exactly right. (It has the same form as the MSE gradient for a linear output — not a coincidence: both are "canonical link" pairs of generalized linear models.) For a neuron $z = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b$, the chain rule continues to $\frac{\partial \ell}{\partial \mathbf{w}} = (a - y)\mathbf{x}$ and $\frac{\partial \ell}{\partial b} = a - y$.

Worked example: BCE numbers

$z = 2$, so $a = \sigma(2) = 0.8808$. If $y = 1$: $\ell = -\ln 0.8808 = 0.127$, $\frac{\partial \ell}{\partial z} = -0.119$ (push z up a little). If $y = 0$: $\ell = -\ln(1 - 0.8808) = -\ln 0.1192 = 2.127$, $\frac{\partial \ell}{\partial z} = +0.881$ (push z down hard).

8.4.3 Why not MSE with a sigmoid?

It is tempting to treat the probability a as a number and use $\ell = (a - y)^2$. Chain rule again:

$$\frac{\partial \ell_{\text{MSE}}}{\partial z} = 2(a - y) \cdot \frac{\mathrm{d}a}{\mathrm{d}z} = 2(a - y)\, a(1 - a).$$

The extra factor $a(1 - a)$ is the sigmoid's slope, which vanishes when the sigmoid saturates. Consider a *confidently wrong* prediction, $y = 1$ but $z = -4$: $a = 0.018$, so

$$\text{BCE: } \frac{\partial \ell}{\partial z} = a - 1 = -0.982, \qquad \text{MSE: } \frac{\partial \ell}{\partial z} = 2(-0.982)(0.018)(0.982) = -0.035.$$

BCE pushes 28 times harder exactly when the model most needs correcting. With MSE, a neuron that starts out confidently wrong is stuck on a flat plateau and learns very slowly (Fig. 8.5a). The right panel shows the effect in a real (tiny) experiment: a one-feature logistic regression initialized with the wrong sign of the weight reaches 90% accuracy after 10 steps with BCE but needs 47 with MSE. In deeper networks the plateau can last indefinitely. This is why classification networks use cross-entropy, never MSE on probabilities.

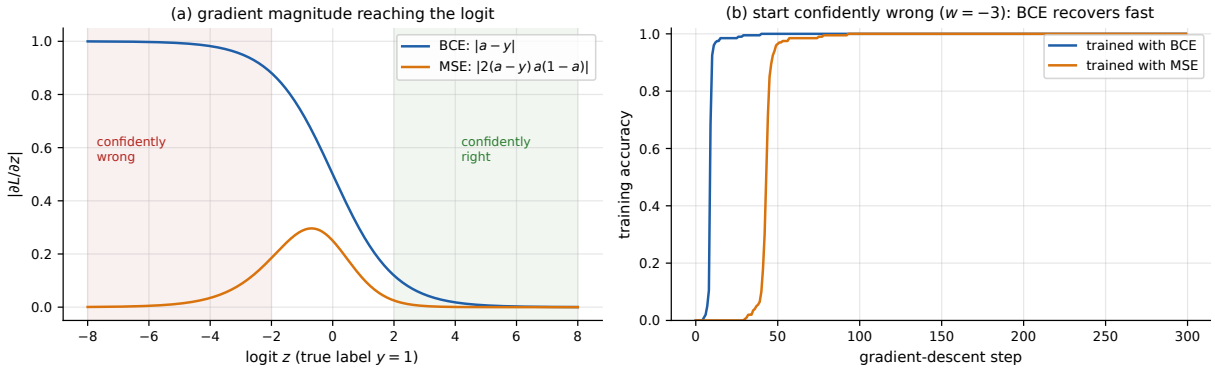


Figure 8.5: Why cross-entropy, not MSE, for classification. (a) Magnitude of the gradient reaching the logit z for a positive example ($y = 1$). BCE gives a gradient near 1 when the prediction is confidently wrong (left); MSE through a sigmoid gives almost none there, because of the factor $a(1 - a)$. (b) Training accuracy of a 1-D logistic regression started with $w = -3$ (confidently wrong), learning rate 0.5: BCE recovers within about 10 steps, MSE needs about 47. (Script: `code/ch08/fig_gradients.py`.)

8.5 Multi-class classification: categorical cross-entropy

8.5.1 Derivation from the categorical likelihood

With K classes, the network outputs logits $\mathbf{z} \in \mathbb{R}^K$ and probabilities $\mathbf{p} = \text{softmax}(\mathbf{z})$. Encode the label as a **one-hot** vector $\mathbf{y} \in \{0,1\}^K$ with a single 1 at the true class c. The label follows a categorical distribution (a K-sided die):

$$p(\mathbf{y} \mid \mathbf{x}) = \prod_{k=1}^{K} p_k^{y_k} = p_c.$$

Taking $-\ln$:

Key equation: categorical cross-entropy (CE)

$$\ell_{\text{CE}}(\mathbf{p}, \mathbf{y}) = -\sum_{k=1}^{K} y_k \ln p_k = -\ln p_c.$$

Only the probability of the true class matters; the loss is zero only if $p_c = 1$. With $K = 2$ it reduces to BCE.

Why "cross-entropy"? For two distributions $\mathbf{q}$ (true) and $\mathbf{p}$ (model), the cross-entropy $H(\mathbf{q}, \mathbf{p}) = -\sum_k q_k \ln p_k$ equals $H(\mathbf{q}) + D_{\text{KL}}(\mathbf{q} \| \mathbf{p})$, the entropy of $\mathbf{q}$ plus the Kullback–Leibler divergence (Part I). With a one-hot $\mathbf{q}$, $H(\mathbf{q}) = 0$, so minimizing CE minimizes the KL divergence from the label distribution to the model's distribution. In bits (using $\log_2$) it is the average number of bits a code designed for $\mathbf{p}$ needs to describe labels drawn from $\mathbf{q}$.

8.5.2 The gradient: $\frac{\partial \ell}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}$

We need $\frac{\partial \ell}{\partial z_j}$ for each logit. Since $\ell = -\sum_k y_k \ln p_k$, the chain rule gives $\frac{\partial \ell}{\partial z_j} = -\sum_k \frac{y_k}{p_k} \frac{\partial p_k}{\partial z_j}$. Insert the softmax Jacobian $\frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j)$ from Chapter 6:

$$\frac{\partial \ell}{\partial z_j} = -\sum_k \frac{y_k}{p_k} p_k(\delta_{kj} - p_j) = -\sum_k y_k \delta_{kj} + p_j \sum_k y_k = -y_j + p_j,$$

using $\sum_k y_k \delta_{kj} = y_j$ and $\sum_k y_k = 1$. Hence

Key equation: softmax + cross-entropy gradient

$$\frac{\partial \ell_{\text{CE}}}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}.$$

Again "prediction minus target", now as a vector: the true class's logit is pushed up by $1 - p_c$, every other logit is pushed down by its probability p_k. The components sum to zero.

Worked example: $\mathbf{z} = (2, 1, 0.1)$, true class 0

From Chapter 6, $\mathbf{p} = (0.659, 0.242, 0.099)$. Loss: $\ell = -\ln 0.659 = 0.417$. Gradient: $\mathbf{p} - \mathbf{y} = (0.659 - 1,\ 0.242,\ 0.099) = (-0.341,\ 0.242,\ 0.099)$. A gradient step $\mathbf{z} \leftarrow \mathbf{z} - \eta(\mathbf{p} - \mathbf{y})$ raises z_0 and lowers the other two, the more so for the more probable wrong class. If instead the true

class were 2, the loss would be $-\ln 0.099 = 2.32$ and the gradient $(0.659, 0.242, -0.901)$.

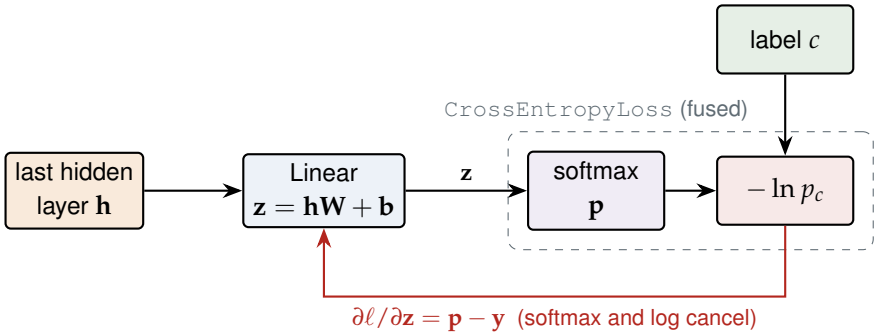


Figure 8.6: The output end of a classifier. Softmax and the negative log are fused into one operation in practice: its input is the logit vector $\mathbf{z}$ and its gradient is the simple $\mathbf{p} - \mathbf{y}$. In PyTorch the model outputs logits and `nn.CrossEntropyLoss` applies log-softmax internally.

8.5.3 The expected initial loss: $\ln K$

A freshly initialized network knows nothing, so its logits are small and nearly equal and $\mathbf{p} \approx (1/K, \ldots, 1/K)$. Then $\ell = -\ln(1/K) = \ln K$: 0.693 for 2 classes, 2.303 for 10 classes, 6.91 for 1000. This is a free and powerful debugging check: *the very first loss printed by your training loop should be close to* $\ln K$. Much higher means the initial logits are too large (the network is confidently wrong before seeing any data); much lower may mean a bug that leaks the labels. Figure 8.7 checks it on a real network: 30 random initializations of a small MLP on the 10-class digits data give losses between 2.27 and 2.39, around $\ln 10$; scaling the output weights by 30 gives losses around 10.

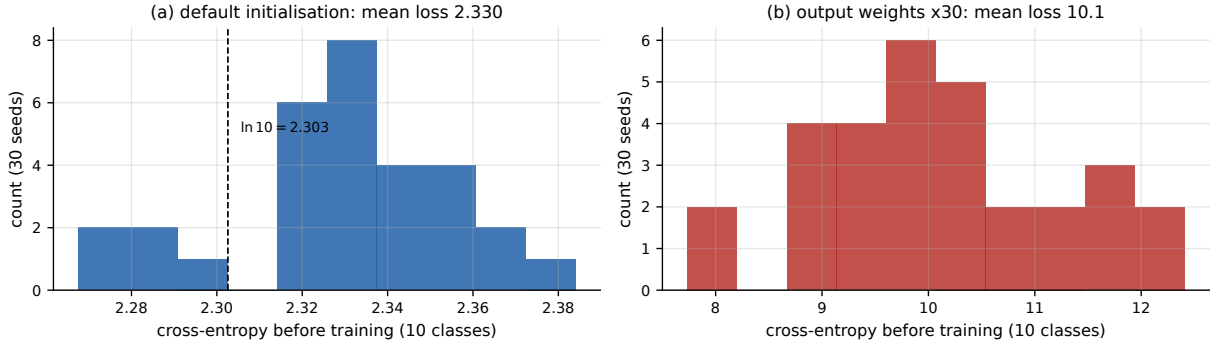


Figure 8.7: The first loss should be about $\ln K$. Cross-entropy of an untrained 64-64-10 MLP on the 10-class digits dataset, for 30 random initializations. (a) With PyTorch's default initialization the losses cluster at $\ln 10 = 2.303$. (b) With output weights multiplied by 30 the untrained network is over-confident and the loss is around 10 — a sign of a bad initialization. (Script: `code/ch08/fig_initial_loss.py`.)

8.6 Numerically stable implementations

Written naively, these losses break on a computer. In 32-bit floating point e^{89} overflows to `inf`, and $\sigma(-120)$ underflows to 0, so $\ln \sigma(-120) = \ln 0 = -\infty$. Logits of size 100 are not unusual during training. Every framework therefore computes the losses *from the logits* with two identities.

8.6.1 Log-sum-exp for cross-entropy

$$\ell_{\text{CE}} = -\ln \frac{e^{z_c}}{\sum_k e^{z_k}} = \ln \sum_k e^{z_k} - z_c.$$

The term $\text{LSE}(\mathbf{z}) = \ln \sum_k e^{z_k}$ is computed by first subtracting the maximum $m = \max_k z_k$:

Key equation: log-sum-exp trick

$$\text{LSE}(\mathbf{z}) = m + \ln \sum_k e^{z_k - m}, \qquad m = \max_k z_k.$$

Proof: $\sum_k e^{z_k} = e^m \sum_k e^{z_k - m}$; take the log. Now every exponent is ≤ 0 (no overflow) and the largest term is $e^0 = 1$ (so the sum is ≥ 1 and its log is safe). For $\mathbf{z} = (2, 1, 0.1)$: $m = 2$, $\sum e^{z_k - 2} = 1 + 0.3679 + 0.1496 = 1.5175$, $\text{LSE} = 2 + 0.4171 = 2.417$, and $\ell = 2.417 - 2 = 0.417$ for class 0 — same as before. For $\mathbf{z} = (1000, 999, 998)$ the naive formula returns `nan`; the trick returns $1000.408 - 1000 = 0.408$.

8.6.2 BCE with logits

Express BCE through z instead of $a = \sigma(z)$. Using $\ln \sigma(z) = -\ln(1 + e^{-z})$ and $\ln(1 - \sigma(z)) = \ln \sigma(-z) = -z - \ln(1 + e^{-z})$ [since $1 - \sigma(z) = \frac{e^{-z}}{1 + e^{-z}}$]:

$$\ell = y \ln(1 + e^{-z}) + (1 - y)\big(z + \ln(1 + e^{-z})\big) = z - zy + \ln(1 + e^{-z}).$$

This is fine for $z \ge 0$, but for very negative z the term e^{-z} overflows. For $z < 0$ use $\ln(1 + e^{-z}) = \ln\big(e^{-z}(e^z + 1)\big) = -z + \ln(1 + e^z)$, which gives $\ell = -zy + \ln(1 + e^z)$. Both cases combine into one expression with no overflow:

Key equation: stable BCE with logits

$$\ell_{\text{BCE}}(z, y) = \max(z, 0) - zy + \ln\big(1 + e^{-|z|}\big).$$

Check: for $z \ge 0$, $\max = z$ and $|z| = z$ gives the first form; for $z < 0$, $\max = 0$ and $-|z| = z$ gives the second. The argument of the exponential is never positive. Numbers: $z = 3$, $y = 0$: $3 - 0 + \ln(1 + e^{-3}) = 3.0486$; $z = -30$, $y = 1$: $0 + 30 + \ln(1 + e^{-30}) = 30.000$, where the naive formula gives $-\ln \sigma(-30)$, which is fine in float64 but $-\ln 0 = \infty$ in float32 once $|z|$ exceeds about 88. This is exactly `nn.BCEWithLogitsLoss` (implementations use `log1p` for $\ln(1 + u)$ to stay accurate when u is tiny).

Pitfall: double softmax, double sigmoid

`nn.CrossEntropyLoss` expects *raw logits* and applies log-softmax itself; `nn.BCEWithLogitsLoss` expects logits and applies the sigmoid itself. Adding a softmax or sigmoid at the end of your model and then using these losses applies it twice: the code runs, the loss decreases, but training is slow and the probabilities are wrong. Use `nn.NLLLoss` only after an explicit `log_softmax`, and `nn.BCELoss` only on probabilities — and prefer the fused versions.

```
import torch, torch.nn as nn, torch.nn.functional as F
z = torch.tensor([[2.0, 1.0, 0.1]], requires_grad=True)   # logits, batch of 1
y = torch.tensor([0])                                      # class index
```

```
loss = F.cross_entropy(z, y)          # log-sum-exp inside
loss.backward()
print(loss.item(), z.grad)            # 0.4170  tensor([[-0.3410, 0.2424, 0.0986]])
# binary / multi-label: one logit per output, targets in {0,1}
zb = torch.tensor([3.0, -30.0]); yb = torch.tensor([0.0, 1.0])
print(F.binary_cross_entropy_with_logits(zb, yb, reduction='none'))  # [3.0486,
    30.0]
```

8.7 Variations used in practice

8.7.1 Multi-class versus multi-label

Multi-class: exactly one of K classes is true (a burst is BPSK *or* QPSK *or* ...). Use softmax + CE: the probabilities compete and sum to 1. *Multi-label*: any subset can be true (a spectrogram may contain LTE *and* Wi-Fi *and* a radar pulse; an image may contain a cat and a dog). Use K independent sigmoids and the sum (or mean) of K BCE losses, $\ell = \sum_k \ell_{\mathrm{BCE}}(z_k, y_k)$, with a multi-hot target. Each output then answers its own yes/no question and the probabilities need not sum to 1.

8.7.2 Label smoothing

With one-hot targets, CE is minimized only when $p_c \to 1$, i.e. when the true logit exceeds the others by an infinite margin. The network keeps growing its logits even after it classifies everything correctly, becoming over-confident (and poorly calibrated). **Label smoothing** replaces the target by a mixture with the uniform distribution:

$$\tilde{\mathbf{y}} = (1-\varepsilon)\mathbf{y} + \frac{\varepsilon}{K}\mathbf{1}, \qquad \ell = -\sum_k \tilde{y}_k \ln p_k, \qquad \frac{\partial \ell}{\partial \mathbf{z}} = \mathbf{p} - \tilde{\mathbf{y}}.$$

The gradient vanishes when $\mathbf{p} = \tilde{\mathbf{y}}$, i.e. at $p_c = 1 - \varepsilon + \varepsilon/K$: a *finite* optimum. With $K = 3$, $\varepsilon = 0.1$: $\tilde{\mathbf{y}} = (0.933, 0.033, 0.033)$ and the optimal logit margin is $\ln(0.933/0.033) = 3.33$. For our $\mathbf{z} = (2, 1, 0.1)$ the smoothed loss is 0.514 and the gradient $(-0.274, 0.209, 0.065)$ — a gentler push than before (Fig. 8.8a). Typical values are $\varepsilon = 0.05$–0.1.

8.7.3 Class weighting and focal loss

When classes are imbalanced — 95% "no signal", 5% "signal present" — a network can reach 95% accuracy by always saying "no signal", and the plain CE barely objects because the rare class contributes little to the average. Two remedies:

- **Class weights**: multiply each example's loss by a weight α_c for its class, e.g. inversely proportional to class frequency: $\alpha = 1/(K \cdot \mathrm{freq})$ gives 0.53 for the common and 10 for the rare class in the 95/5 example. (`weight=` in `CrossEntropyLoss`, `pos_weight=` in `BCEWithLogitsLoss`.)
- **Focal loss** (Lin et al., 2017): $\ell = -(1-p_t)^\gamma \ln p_t$, where p_t is the probability assigned to the true class. The factor $(1-p_t)^\gamma$ is tiny for easy, well-classified examples and close to 1 for hard ones, so training concentrates on the hard examples (Fig. 8.8b). With $\gamma = 2$: an easy example with $p_t = 0.9$ contributes $0.01 \times 0.105 = 0.00105$ instead of 0.105 (100 times less), while a hard one with $p_t = 0.1$ contributes $0.81 \times 2.303 = 1.87$.

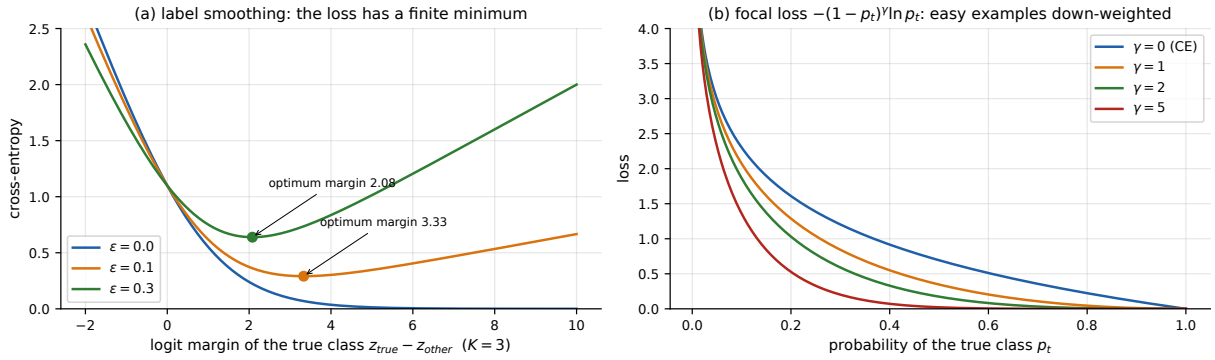


Figure 8.8: (a) Label smoothing. Cross-entropy of a 3-class prediction as a function of the true class's logit margin. Without smoothing ($\varepsilon = 0$) the loss keeps decreasing forever, rewarding ever larger logits; with smoothing the loss has a minimum at a finite margin (3.33 for $\varepsilon = 0.1$, 2.08 for $\varepsilon = 0.3$) and rises again for over-confident predictions. (b) Focal loss for several γ: the larger γ, the more the loss of well-classified examples (p_t near 1) is suppressed. (Script: `code/ch08/fig_smoothing.py`.)

8.7.4 Choosing the output head

Table 8.1: Matching the output layer to the loss. "Logits" means the raw output of the last linear layer.

task	assumed distribution	output activation	loss (PyTorch)	gradient at logits
regression	Gaussian	identity	MSE `MSELoss`	$\frac{2}{N}(\hat{y} - y)$
robust regression	Laplace / mixed	identity	MAE, Huber `L1Loss`, `HuberLoss`	sign / clipped residual
binary	Bernoulli	sigmoid (in loss)	BCE `BCEWithLogitsLoss`	$\sigma(z) - y$
multi-label (K yes/no)	K Bernoullis	K sigmoids (in loss)	sum of BCE	$\sigma(\mathbf{z}) - \mathbf{y}$
multi-class (one of K)	categorical	softmax (in loss)	CE `CrossEntropyLoss`	$\mathbf{p} - \mathbf{y}$

8.8 Losses in communication systems

Wireless connection: symbol-wise cross-entropy for the channel autoencoder

In the channel autoencoder (Chapter 5, Part V) a message of k bits is one of $M = 2^k$ symbols. The receiver network ends in M logits and a softmax, and is trained with categorical CE on the message index — a 2^k-class classification problem. For $k = 4$, $M = 16$, the initial loss should be $\ln 16 = 2.77$ nats ($= 4$ bits: the receiver knows nothing about 4 bits). CE also has an information-theoretic meaning: for any receiver, $I(S;\mathbf{Y}) \geq H(S) - \mathbb{E}[\ell_{\text{CE}}]$ (in bits), so with uniform messages ($H(S) = k$ bits) minimizing the CE *maximizes a lower bound on the mutual information* between the transmitted message and the received signal. A trained system with CE $= 0.1$ bits per message achieves at least 3.9 of the 4 bits per channel use. The drawback: M outputs grow exponentially with k, which is why bit-wise training is preferred for larger blocks.

Wireless connection: bit-wise BCE, LLRs and bit-metric decoding

Modern receivers are *bit-interleaved*: the demapper outputs one soft value per bit, and the LDPC decoder treats bits independently — *bit-metric decoding*. The matching loss is the sum of k binary cross-entropies, one per bit, on k logits. Its minimizer has a remarkable property: since BCE is minimized when $\sigma(z_k) = P(b_k = 1 \mid \mathbf{y})$, the optimal logit is

$$z_k^\star = \ln \frac{P(b_k = 1 \mid \mathbf{y})}{P(b_k = 0 \mid \mathbf{y})},$$

exactly the bit LLR (with the opposite sign to the convention $\ln \frac{P(0)}{P(1)}$ of Chapter 7; flip the sign before feeding a decoder that uses the 0-over-1 convention). So a network trained with bit-wise BCE learns to output LLRs *without ever being shown an LLR*. Moreover, $k - \sum_k \mathbb{E}[\ell_{\mathrm{BCE},k}]/\ln 2$ is an achievable rate for bit-metric decoding (it equals the bit-wise mutual information, BMI, when the logits are the true LLRs, and lower-bounds it otherwise).
We checked this with a small experiment (`code/ch08/bitwise_bce.py`): the 2-32-32-4 MLP of Chapter 7 trained on 16-QAM at $E_s/N_0 = 8$ dB with bit-wise BCE only. On fresh test data it reaches a BCE of 0.234 nats/bit versus 0.232 for the exact LLRs, i.e. an estimated 2.65 of the 2.66 bits/symbol achievable with exact LLRs, and the same BER (0.101 vs 0.100). Figure 8.9 compares its logits with exact LLRs: they agree closely near zero, where decisions are made, while very confident LLRs ($|L| > 6$) are underestimated — those examples are already almost perfectly classified, so BCE (whose gradient is $\sigma(z) - y \approx 0$ there) no longer pushes them. Such mild under-confidence costs almost nothing in decoding.

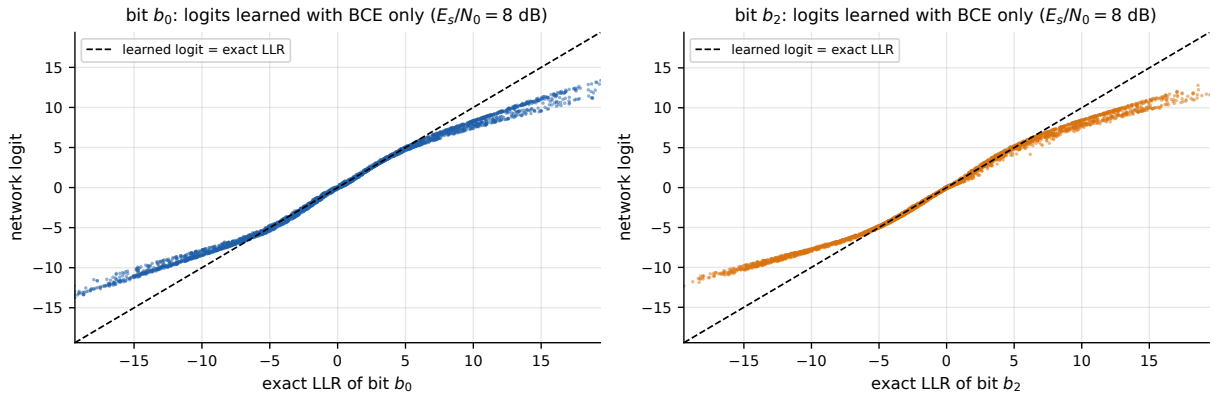


Figure 8.9: Our small-scale check that BCE-trained logits are LLRs. Each dot is one received 16-QAM symbol at $E_s/N_0 = 8$ dB: horizontal axis, the exact LLR $\ln \frac{P(b_k=1|y)}{P(b_k=0|y)}$; vertical axis, the logit of an MLP trained only with bit-wise BCE on the transmitted bits. Points follow the diagonal for small and moderate LLRs and bend below it for very confident ones. (Script: `code/ch08/bitwise_bce.py`.)

Wireless connection: BER, BLER and cross-entropy: what to train, what to report

A link is judged by its bit-error rate (BER) and, above all, its block-error rate (BLER, the fraction of transport blocks failing CRC), because HARQ and link adaptation run on BLER targets such as 10%. Neither can be trained directly: both count hard errors, with zero gradient almost everywhere, and BLER also depends on the decoder. Cross-entropy is the smooth surrogate: symbol-wise CE bounds the mutual information the receiver extracts, and bit-wise BCE is tied to the rate a bit-metric decoder can achieve, so lowering it generally

lowers the BLER after decoding. In practice: *train on (B)CE, select models on validation BLER (or BMI), report BER/BLER versus SNR curves*. Do not be surprised if two models with similar BCE differ in BLER at a given SNR: BLER depends on the tails (rare confident errors), which an average loss does not capture fully.

Chapter recap

- A loss is the smooth number we minimize; metrics (accuracy, BER, BLER) are reported, not differentiated.
- Maximum likelihood: minimize the negative log-likelihood $-\frac{1}{N}\sum \ln p(y_i \mid \mathbf{x}_i; \theta)$. Gaussian $\Rightarrow$ MSE; Laplace $\Rightarrow$ MAE; Bernoulli $\Rightarrow$ BCE; categorical $\Rightarrow$ CE.
- MSE gradient $\frac{2}{N}(\hat{y} - y)$; Huber = MSE near zero, MAE for outliers (gradient clipped to $\pm\delta$).
- BCE with sigmoid: $\frac{\partial \ell}{\partial z} = a - y$. MSE with sigmoid: $2(a-y)a(1-a)$, which vanishes when confidently wrong (e.g. 0.035 vs 0.982 at $z = -4$).
- CE with softmax: $\frac{\partial \ell}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}$; $\mathbf{z} = (2, 1, 0.1)$, class 0: $\ell = 0.417$, gradient $(-0.341, 0.242, 0.099)$. Initial loss $\approx \ln K$ (2.303 for $K = 10$).
- Stable forms: $\mathrm{LSE}(\mathbf{z}) = m + \ln \sum e^{z_k - m}$; $\mathrm{BCE} = \max(z, 0) - zy + \ln(1 + e^{-|z|})$. Feed logits to fused losses.
- Multi-label: independent sigmoids + BCE. Label smoothing gives a finite optimum; class weights and focal loss handle imbalance.
- Wireless: symbol-wise CE bounds mutual information; bit-wise BCE makes logits LLRs and measures an achievable BMD rate; MSE/NMSE for channel estimation.

Exercises

1. **(ML for a constant.)** You observe $y = (1, 2, 6)$ and model all three as c plus noise. Find the c that minimizes MSE and the c that minimizes MAE. Which is more affected if 6 becomes 60?
2. **(BCE by hand.)** For $z = -1$ compute a, the BCE for $y = 1$ and $y = 0$, and $\frac{\partial \ell}{\partial z}$ in both cases.
3. **(MSE vs BCE.)** For $y = 0$ and $z = 5$ compute the gradient with respect to z for BCE and for MSE on the probability. By what factor do they differ?
4. **(CE by hand.)** Logits $\mathbf{z} = (0.5, 2.0, -1.0, 0.0)$, true class 1. Compute $\mathbf{p}$, the loss, and the gradient. Verify that the gradient sums to zero.
5. **(Stability.)** Evaluate the CE for $\mathbf{z} = (50, -50)$, true class 1, with the log-sum-exp trick. What would the naive formula do in float32?
6. **(Stable BCE.)** Show that $\max(z, 0) - zy + \ln(1 + e^{-|z|})$ is symmetric in the sense $\ell(z, 1) = \ell(-z, 0)$, and explain why that makes sense.
7. **(Label smoothing.)** For $K = 10$ and $\varepsilon = 0.1$, what is the smoothed target for the true class and for each other class? What logit margin minimizes the smoothed loss?
8. **(Autoencoder.)** A channel autoencoder with $k = 8$ bits per message is trained with

symbol-wise CE. How many output neurons does the receiver need? What should the initial loss be in nats and in bits? If training reaches a CE of 0.5 bits, what does that guarantee?

Answers

1. MSE: the mean, $c = 3$. MAE: the median, $c = 2$. With 60 the mean jumps to 21, the median stays 2: MSE is far more affected by the outlier.
2. $a = \sigma(-1) = 0.2689$. $y = 1$: $\ell = -\ln 0.2689 = 1.313$, $\frac{\partial \ell}{\partial z} = a - 1 = -0.731$. $y = 0$: $\ell = -\ln 0.7311 = 0.313$, $\frac{\partial \ell}{\partial z} = 0.269$.
3. $a = \sigma(5) = 0.9933$. BCE: $a - y = 0.9933$. MSE: $2(0.9933)(0.9933)(0.0067) = 0.0132$. BCE's is about 75 times larger.
4. $e^{\mathbf{z}} = (1.6487, 7.3891, 0.3679, 1)$, sum 10.4057; $\mathbf{p} = (0.1584, 0.7101, 0.0354, 0.0961)$. Loss $= -\ln 0.7101 = 0.342$. Gradient $\mathbf{p} - \mathbf{y} = (0.1584, -0.2899, 0.0354, 0.0961)$; sum $= 0$.
5. $m = 50$; LSE $= 50 + \ln(1 + e^{-100}) \approx 50$; $\ell = \text{LSE} - z_1 = 50 - (-50) = 100$. Naive in float32: $e^{50} = 5.2 \times 10^{21}$ is still representable, but $p_1 = e^{-50}/(e^{50} + e^{-50}) \approx 3.8 \times 10^{-44}$ is a denormal number with only a few significant bits, so $-\ln p_1$ comes out as 99.98 instead of 100. With logits ± 100, e^{100} overflows to `inf` and the result is `nan`.
6. $\ell(z, 1) = \max(z, 0) - z + \ln(1 + e^{-|z|}) = \max(-z, 0) + \ln(1 + e^{-|z|})$ (since $\max(z, 0) - z = \max(0, -z)$), and $\ell(-z, 0) = \max(-z, 0) + \ln(1 + e^{-|z|})$. Equal. It makes sense because predicting logit z for class 1 is the same as predicting $-z$ for class 0: $\sigma(-z) = 1 - \sigma(z)$.
7. True class: $0.9 + 0.01 = 0.91$; others: 0.01 each. The optimum has $\mathbf{p} = \tilde{\mathbf{y}}$, so the margin is $\ln(0.91/0.01) = \ln 91 = 4.51$.
8. $2^8 = 256$ outputs. Initial loss $\ln 256 = 5.545$ nats $= 8$ bits. CE $= 0.5$ bits guarantees $I(S; \mathbf{Y}) \geq 8 - 0.5 = 7.5$ bits per message.

Self-test quiz

1. Which noise assumption turns maximum likelihood into MSE?
   Additive Gaussian noise with constant variance.
2. What is $\partial \ell / \partial z$ for sigmoid + BCE, and for softmax + CE?
   $a - y$ and $\mathbf{p} - \mathbf{y}$.
3. Why is MSE on sigmoid outputs a bad classification loss?
   Its gradient carries the factor $a(1 - a)$, which vanishes when the neuron is confidently wrong.
4. Your 100-class classifier prints an initial loss of 4.6. Good or bad?
   Good: $\ln 100 = 4.605$.
5. Multi-label problem: softmax or sigmoids?
   Independent sigmoids with BCE per label.
6. What does a network trained with bit-wise BCE output at its logits, ideally?
   The bit LLRs $\ln \frac{P(b=1|\mathbf{y})}{P(b=0|\mathbf{y})}$.

Further reading

- I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016 — section 5.5 (maximum likelihood) and 6.2 (cost functions and output units).
- C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006 — chapter 4 on logistic regression and the canonical link.
- P. J. Huber, "Robust estimation of a location parameter", *Annals of Mathematical Statistics*, 35(1), 1964.
- C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens and Z. Wojna, "Rethinking the Inception architecture for computer vision", CVPR 2016 — label smoothing.
- T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár, "Focal loss for dense object detection", ICCV 2017.
- S. Cammerer, F. Ait Aoudia, S. Dörner, M. Stark, J. Hoydis and S. ten Brink, "Trainable communication systems: concepts and prototype", *IEEE Trans. Communications*, 2020 — bit-wise BCE, BMI and bit-metric decoding for learned systems.

Chapter 9

Gradient Descent and the Learning Rate

In the last few chapters you built a neural network's forward pass and chose a loss function that measures how wrong the network is. A loss on its own does not train anything, though. We also need a procedure that takes the current parameters and *changes them* so the loss goes down, and then repeats that millions of times. Nearly every network in this book, and nearly every network trained in industry, uses some form of **gradient descent** for this.

In this chapter you will learn what gradient descent does geometrically and why it works. You will see the single most important hyperparameter in deep learning, the **learning rate**, and derive exactly when it makes training converge, crawl, zig-zag or blow up. Then you will meet the practical forms used on real data (mini-batch and stochastic gradient descent), the noise they bring, and the schedules people use to change the learning rate during training. The chapter ends with a long wireless section showing that the LMS adaptive equalizer, which communication engineers have used since the 1960s, *is* stochastic gradient descent. Once you see that, everything in this chapter will also tell you how a modem adapts.

9.1 The foggy mountain

Intuition: walking down a mountain in thick fog

Imagine you are standing somewhere on a hilly landscape in fog so thick you can see only the ground under your boots. You want to reach the lowest valley. You cannot see the valley, and you have no map. What can you do?

You can feel the slope under your feet. If the ground tilts down towards the north-east, take a step north-east. Stop, feel the slope again, take another step. Repeat. As long as each step is short enough that the slope does not change much during the step, every step takes you lower.

That is gradient descent, completely. The landscape is the loss L plotted against the parameters. Your position is the current parameter vector θ. "Feeling the slope" means computing the gradient $\nabla L(\theta)$ (backpropagation, Chapter 10, does this for a neural network). "Taking a step" means changing θ. The *length* of each step is controlled by the learning rate η. Steps that are too short take forever. Steps that are too long carry you across the valley and up the far side, perhaps higher than where you started.

Two facts from Part I make this precise. First, the gradient $\nabla L(\theta)$ is the vector of partial derivatives $\left(\frac{\partial L}{\partial \theta_1}, \ldots, \frac{\partial L}{\partial \theta_P}\right)$, one per parameter. Second, it points in the direction in which L *increases* fastest, so its negative points in the direction of steepest decrease. Let us see why, because the same calculation also tells us how big a step is safe.

9.1.1 Why minus the gradient is the steepest way down

Stand at θ and consider a small move δ (a vector with the same number of entries as θ). A first-order Taylor expansion (Chapter 3) says the loss after the move is approximately

$$L(\theta + \delta) \approx L(\theta) + \nabla L(\theta)^{\mathsf{T}} \delta. \tag{9.1}$$

The change in loss is the dot product $\nabla L^{\mathsf{T}} \delta = \|\nabla L\| \, \|\delta\| \cos\phi$, where ϕ is the angle between the gradient and the move. If we fix the step length $\|\delta\|$ and are free to choose only its direction, the change is most negative when $\cos\phi = -1$, which means δ points exactly opposite to the gradient. So the steepest descent direction is $-\nabla L$. If we move along it with a step $\delta = -\eta \, \nabla L$ for some small positive number η, Eq. (9.1) gives

$$L(\theta - \eta \nabla L) \approx L(\theta) - \eta \, \|\nabla L(\theta)\|^2 . \tag{9.2}$$

Because $\|\nabla L\|^2 \geq 0$, the loss goes *down* by about $\eta \, \|\nabla L\|^2$, unless we are already at a point where the gradient is zero. The word "approximately" carries a warning: the Taylor expansion holds only for small moves. If η is too large, the curvature we ignored takes over and the loss can go up. Section 9.3 works out exactly where that happens.

Definition: gradient descent

Given a differentiable loss $L(\theta)$, a starting point θ_0 and a positive number η called the **learning rate** (or step size), **gradient descent** produces the sequence

$$\theta_{k+1} = \theta_k - \eta \, \nabla L(\theta_k), \qquad k = 0, 1, 2, \ldots$$

Each repetition is one **iteration** (one **step**, one **update**). We stop after a fixed number of steps, when the loss stops improving, or when the gradient becomes very small.

Key equation: the gradient descent update

$$\boxed{\theta \;\leftarrow\; \theta - \eta \, \nabla_\theta L(\theta)} \tag{9.3}$$

Read it as: *new parameters = old parameters minus (learning rate) times (slope)*. Every parameter moves against its own partial derivative. A parameter with a large positive derivative is decreased a lot; one with a negative derivative is increased.

Figure 9.1 shows the loop as a block diagram. Training a neural network means running this loop: forward pass, loss, gradient, update, and back to the forward pass with the new parameters.

9.2 A complete worked example in one dimension

Take the simplest possible loss: one parameter w and

$$L(w) = (w - 3)^2.$$

It is a parabola whose lowest point is at $w^\star = 3$, where $L = 0$. Its derivative is $L'(w) = 2(w - 3)$, so the update Eq. (9.3) reads

$$w_{k+1} = w_k - \eta \cdot 2(w_k - 3).$$

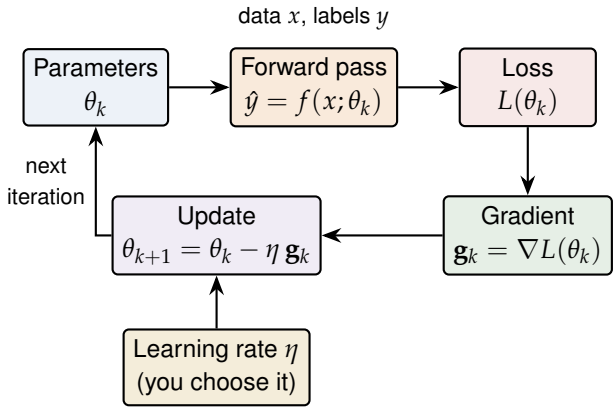


Figure 9.1: The gradient descent loop. The parameters produce predictions; the predictions produce a loss; backpropagation turns the loss into a gradient; the update rule moves the parameters a distance proportional to the learning rate η against the gradient, and the loop repeats. The only knob in the loop that you set by hand is η, which is why it matters so much.

Worked example: $w_0 = 0$, $\eta = 0.1$

Step 0. $w_0 = 0$. Slope $L'(0) = 2(0-3) = -6$. Negative slope means the loss decreases to the right, so we should increase w. Update: $w_1 = 0 - 0.1 \times (-6) = 0.6$.
Step 1. $L'(0.6) = 2(0.6-3) = -4.8$, so $w_2 = 0.6 - 0.1 \times (-4.8) = 0.6 + 0.48 = 1.08$.
Step 2. $L'(1.08) = 2(1.08-3) = -3.84$, so $w_3 = 1.08 + 0.384 = 1.464$.
The losses are $L(w_0) = 9$, $L(w_1) = 5.76$, $L(w_2) = 3.6864$, $L(w_3) = 2.3593$. Each step moves us towards 3, and each step is shorter than the one before because the slope gets gentler as we approach the bottom. Nobody told the algorithm to slow down; the shrinking gradient did it automatically.

Look at the distance to the minimum, $e_k = w_k - 3$: it is -3, -2.4, -1.92, -1.536. Each one is 0.8 times the previous one. That is no accident. Subtract 3 from both sides of the update:

$$w_{k+1} - 3 = w_k - 3 - 2\eta(w_k - 3)$$
$$e_{k+1} = (1 - 2\eta)\, e_k.$$

With $\eta = 0.1$ the **contraction factor** is $1 - 2(0.1) = 0.8$. After k steps, $e_k = 0.8^k e_0$, and the loss is $L_k = e_k^2 = 0.64^k \times 9$. To get within 0.01 of the minimum we need $3 \times 0.8^k < 0.01$, that is $k > \ln(0.01/3) / \ln 0.8 \approx 25.6$, so 26 steps.

Worked example: the same problem with $\eta = 1.1$: divergence

Now the factor is $1 - 2(1.1) = -1.2$. Starting again from $w_0 = 0$ ($e_0 = -3$):

$$w_1 = 0 - 1.1 \times (-6) = 6.6, \qquad e_1 = 3.6,$$
$$w_2 = 6.6 - 1.1 \times 2(3.6) = -1.32, \qquad e_2 = -4.32,$$
$$w_3 = -1.32 - 1.1 \times 2(-4.32) = 8.184, \qquad e_3 = 5.184.$$

The parameter jumps from one side of the valley to the other, landing higher each time: $|e_k| = 3 \times 1.2^k$ grows without bound. The loss goes 9, 12.96, 18.66, 26.87, ... This is what a "diverging" or "exploding" training run looks like: the loss climbs and eventually becomes `inf` or `nan`.

Figure 9.2 shows four learning rates on this parabola. Notice the special value $\eta = 0.5$: the factor is $1 - 2(0.5) = 0$, and the first step lands exactly on the minimum. With $\eta = 0.9$ the factor is -0.8: we overshoot every time but by less and less, so we still converge, in a zig-zag.

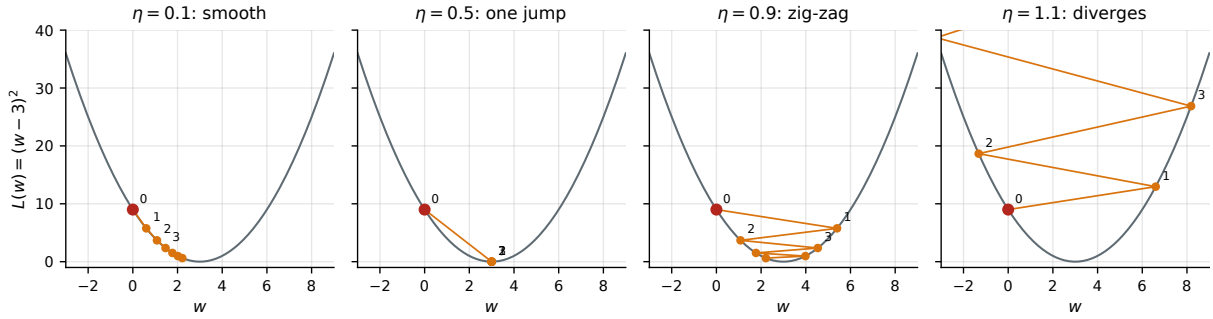


Figure 9.2: Six gradient-descent steps on $L(w) = (w-3)^2$ from $w_0 = 0$ (red dot), for four learning rates; the small numbers label the iterations. With $\eta = 0.1$ (factor 0.8) the steps shrink smoothly towards $w = 3$. With $\eta = 0.5$ (factor 0) one step lands on the minimum. With $\eta = 0.9$ (factor -0.8) every step overshoots but the overshoot shrinks. With $\eta = 1.1$ (factor -1.2) every step overshoots by more than it corrected, and the iterates fly off the plot. Script: `code/ch09/fig_gd_basics.py`.

9.3 When does gradient descent converge? The stability condition

Close to a minimum, any smooth loss looks like a parabola (a second-order Taylor expansion), so the parabola tells us almost everything about the learning rate. Write a general one-dimensional quadratic as

$$L(w) = \tfrac{c}{2}(w - w^\star)^2 + L^\star,$$

where $c > 0$ is the **curvature** (the second derivative $L''(w) = c$), $w^\star$ the minimizer and $L^\star$ the minimum value. Our example $(w-3)^2$ has $c = 2$, $w^\star = 3$, $L^\star = 0$. The derivative is $L'(w) = c(w - w^\star)$, so with $e_k = w_k - w^\star$:

$$w_{k+1} = w_k - \eta c(w_k - w^\star)$$
$$e_{k+1} = (1 - \eta c)\, e_k \quad \Longrightarrow \quad e_k = (1 - \eta c)^k e_0.$$

The error shrinks to zero if and only if the factor has magnitude below one:

$$|1 - \eta c| < 1 \iff -1 < 1 - \eta c < 1 \iff 0 < \eta c < 2.$$

Key equation: stability of gradient descent on a quadratic

$$0 < \eta < \frac{2}{c} \qquad \text{(and the fastest choice is } \eta = 1/c\text{, which reaches the minimum in one step).} \tag{9.4}$$

Every regime in Fig. 9.2 is now explained by the single number ηc:

ηc	factor $1 - \eta c$	behaviour
$0 < \eta c < 1$	between 0 and 1	monotone convergence (slow if ηc is tiny)
$\eta c = 1$	0	jump straight to the minimum
$1 < \eta c < 2$	between -1 and 0	oscillating convergence (zig-zag)
$\eta c = 2$	-1	bounces forever between two points
$\eta c > 2$	below -1	divergence

For $(w-3)^2$, $c=2$, so the limit is $\eta < 1$; $\eta = 1.0$ bounces between 0 and 6 forever, and $\eta = 1.1$ diverges.

Intuition: curvature sets the speed limit

Curvature measures how quickly the slope changes as you move. On a sharply curved loss, the slope you measured at the start of a step is badly wrong by the end of it, so you must take short steps. On a gently curved loss, the slope stays roughly constant, so long steps are safe. The learning rate is a speed limit, and the sharpest bend in the road sets it.

9.3.1 Many parameters: the condition number and the zig-zag

Real networks have millions of parameters, and the curvature is different in different directions. The simplest model of this is an elongated bowl with two parameters:

$$L(w_0, w_1) = \tfrac{1}{2}\left(w_0^2 + 10\, w_1^2\right).$$

The gradient is $\nabla L = (w_0,\ 10w_1)$, so the update separates into two independent one-dimensional problems:

$$w_0 \leftarrow (1-\eta)\, w_0, \qquad w_1 \leftarrow (1-10\eta)\, w_1.$$

The curvature is $c = 1$ along w_0 and $c = 10$ along w_1. We need *both* factors below one in magnitude, so Eq. (9.4) must hold for the steepest direction: $\eta < 2/10 = 0.2$. But the progress along the gentle direction w_0 is governed by the factor $1-\eta$, which is close to 1 when η is small. We are trapped: the steep direction forces a small step, and with a small step the gentle direction crawls.

Worked example: three learning rates on the elongated bowl

- $\eta = 0.02$: factors 0.98 (gentle) and 0.8 (steep). The steep coordinate dies quickly, but the gentle one shrinks by only 2% per step: starting from $w_0 = -9$ it is still $-9 \times 0.98^{40} \approx -4.0$ after 40 steps, and the loss is still 8.05.
- $\eta = 0.15$: factors 0.85 and -0.5. The steep coordinate flips sign every step (zig-zag across the valley) while shrinking by half; the gentle one now moves at a decent 15% per step. After 40 steps the loss is about 10^{-4}.
- $\eta = 0.21$: factors 0.79 and -1.1. The gentle direction would be happy, but the steep direction diverges: $|w_1|$ grows by 10% per step and the loss explodes.

What is the best fixed learning rate? We want the *slower* of the two factors to be as fast as possible, so we minimize $\max(|1-\eta\lambda_{\min}|, |1-\eta\lambda_{\max}|)$, where $\lambda_{\min} = 1$ and $\lambda_{\max} = 10$ are the two curvatures. As η grows, the first term falls and the second rises (once it has passed zero), so the maximum is smallest where they are equal, with the first positive and the second negative:

$$\begin{aligned} 1-\eta\lambda_{\min} &= -(1-\eta\lambda_{\max}) \\ 2 &= \eta(\lambda_{\min}+\lambda_{\max}) \quad \Longrightarrow \quad \eta_{\text{best}} = \frac{2}{\lambda_{\min}+\lambda_{\max}}. \end{aligned}$$

Substituting back, the factor achieved in both directions is

$$1 - \frac{2\lambda_{\min}}{\lambda_{\min}+\lambda_{\max}} = \frac{\lambda_{\max}-\lambda_{\min}}{\lambda_{\max}+\lambda_{\min}} = \frac{\kappa-1}{\kappa+1}, \qquad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}}.$$

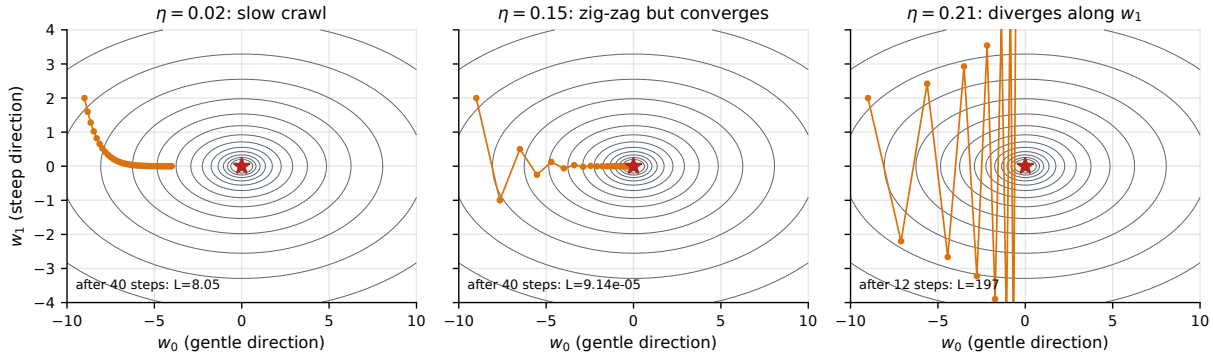


Figure 9.3: Gradient descent on the elongated bowl $L = \frac{1}{2}(w_0^2 + 10w_1^2)$, started at $(-9, 2)$; contours are lines of equal loss and the star is the minimum. Left: a small step kills the steep w_1 error quickly and then creeps along the flat floor of the valley. Middle: a larger step bounces across the valley (sign flips of w_1) but makes good progress along it. Right: a step just above $2/c_{\max} = 0.2$ is unstable in the steep direction, even though the gentle direction would tolerate it. Script: `code/ch09/fig_gd_basics.py`.

Definition: condition number

For a quadratic loss $L = \frac{1}{2}\theta^{\mathsf{T}}\mathbf{H}\theta$ with a symmetric positive-definite curvature matrix $\mathbf{H}$ (the *Hessian*, the matrix of second derivatives), the **condition number** is $\kappa = \lambda_{\max}/\lambda_{\min}$, the ratio of its largest to smallest eigenvalue. Even with the best fixed learning rate, gradient descent shrinks the error by only $(\kappa - 1)/(\kappa + 1)$ per step.

For our bowl, $\kappa = 10$, $\eta_{\text{best}} = 2/11 \approx 0.18$ and the factor is $9/11 \approx 0.818$; reducing the error a thousandfold takes $\ln(10^{-3})/\ln(0.818) \approx 34$ steps. With $\kappa = 1000$ the factor is $999/1001 \approx 0.998$, and the same reduction takes about 3450 steps. *Ill-conditioning*, meaning a big spread of curvatures, is the main reason plain gradient descent is slow. When $\mathbf{H}$ is not diagonal, the eigenvectors of $\mathbf{H}$ (Chapter 2) are the "natural axes" of the bowl. In those rotated coordinates the problem separates exactly as above, so the conclusion is unchanged. Momentum and Adam (Chapter 11) and normalization layers (Chapter 12) are, to a large extent, remedies for ill-conditioning.

Wireless connection: a control loop is gradient descent with a loop gain

You have tuned this trade-off many times without calling it gradient descent. A digital AGC loop updates its gain as $g \leftarrow g - \mu\,(|y|^2 - P_{\text{target}})$, and a first-order timing or frequency loop updates its estimate by μ times a detector output. Each is a descent step on some error, and the loop gain μ is a learning rate. A gain that is too high makes the loop ring or go unstable (factor below -1); a gain that is too low makes acquisition slow (factor near $+1$). The "loop bandwidth versus jitter" trade-off in PLL design is the same as the "learning rate versus noise" trade-off we meet in Section 9.5.

9.4 The shape of real loss landscapes

Figure 9.4 shows a two-parameter loss surface as a 3-D picture, with a gradient-descent path running down it. Real neural-network losses have millions of dimensions and cannot be drawn. Low-dimensional slices still give useful intuition, as long as we remember the ways in which high dimensions differ.

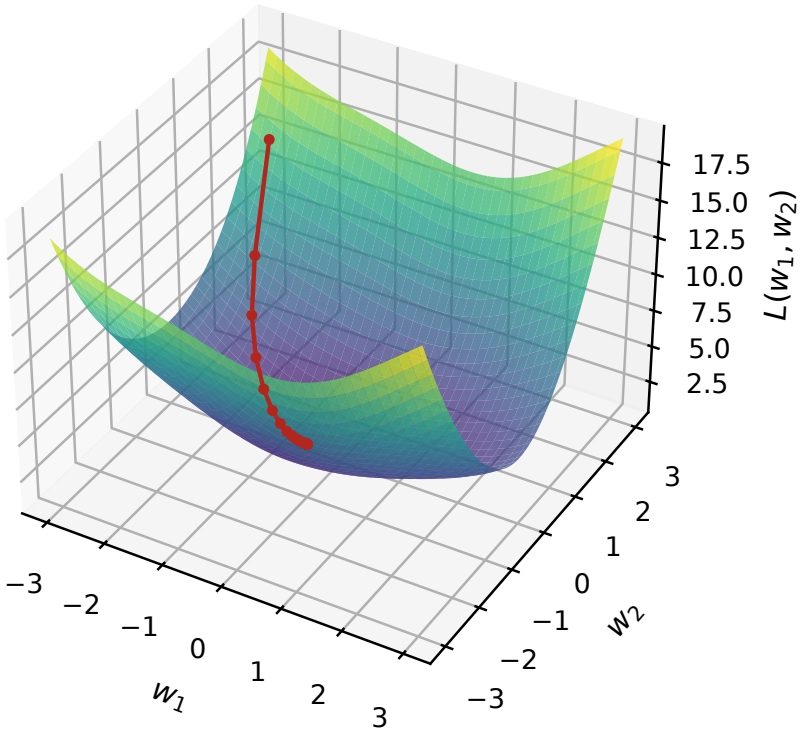


Figure 9.4: A two-parameter loss surface $L(w_1, w_2)$ drawn as a landscape, with 40 gradient-descent steps ($\eta = 0.12$) in red. The path starts high on the slope, turns towards the steepest descent direction, and curves into the valley as the local slope changes. Each red dot is one "feel the ground, take a step" iteration of the foggy-mountain walk. Script: `code/ch09/fig_gd_basics.py`.

Definition: convex function

A loss is **convex** if the straight line segment between any two points on its graph lies on or above the graph: $L(\alpha\theta_a + (1-\alpha)\theta_b) \leq \alpha L(\theta_a) + (1-\alpha)L(\theta_b)$ for all θ_a, θ_b and $0 \leq \alpha \leq 1$. A convex loss is bowl-shaped, so every point where the gradient is zero is a global minimum.

Linear regression with squared error and logistic regression with cross-entropy are convex. For them gradient descent with a suitable η always finds the best solution. A neural network with even one hidden layer is **non-convex**: swapping two hidden neurons (and their weights) gives a different parameter vector with exactly the same loss, so there are many equally good minima, and the landscape between them must have hills. A non-convex landscape can contain several kinds of terrain (Fig. 9.5):

- **Local minima**: points lower than everything nearby but not the lowest overall. A walker who only feels the local slope can get stuck there.
- **Saddle points**: points where the gradient is zero but the surface curves *up* in some directions and *down* in others, like a horse saddle or a mountain pass.
- **Plateaus**: large, nearly flat regions where the gradient is tiny, so steps are tiny and progress is slow even though we are far from any minimum.

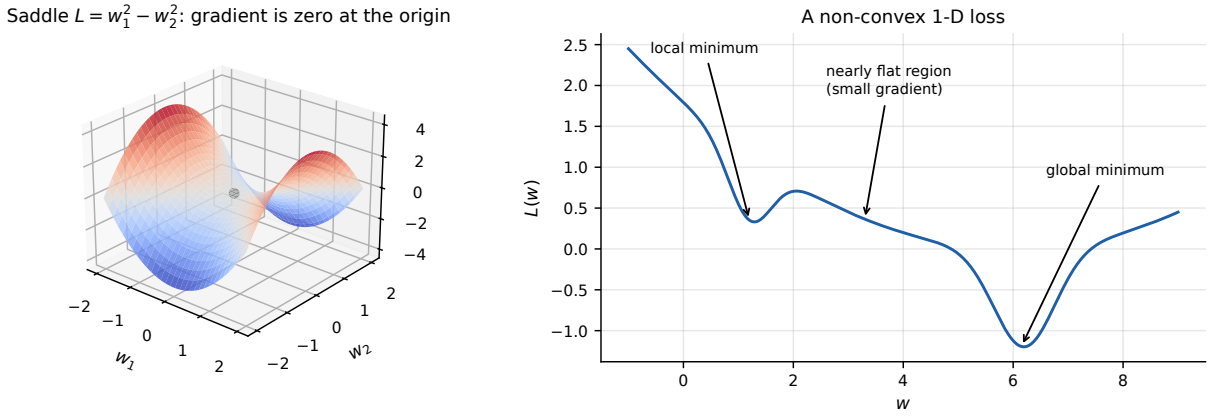


Figure 9.5: Left: the saddle $L = w_1^2 - w_2^2$. At the black dot the gradient is zero, yet it is not a minimum: the surface rises along w_1 and falls along w_2. Right: a one-dimensional non-convex loss with a local minimum (where a gradient-descent walker started on the left would stop), a nearly flat stretch where steps become small, and the global minimum. Script: `code/ch09/fig_gd_basics.py`.

Worked example: escaping a saddle

For $L = w_1^2 - w_2^2$ the gradient is $(2w_1, -2w_2)$, so gradient descent gives $w_1 \leftarrow (1-2\eta)w_1$ and $w_2 \leftarrow (1+2\eta)w_2$. The w_1 coordinate shrinks (it is a "valley" direction), while the w_2 coordinate is multiplied by a factor *larger* than one, so it grows. If we start exactly at $w_2 = 0$ we slide into the saddle and stay there. But with $\eta = 0.1$, a tiny offset $w_2 = 10^{-6}$ grows by 1.2 per step and reaches size 1 after $\ln(10^6)/\ln(1.2) \approx 76$ steps. During those 76 steps the loss barely changes: from outside, a saddle looks exactly like a plateau. This is one reason training curves sometimes stay flat for a while and then suddenly drop.

How bad are local minima in practice? Research on large networks (for example Dauphin et al., 2014, and Choromanska et al., 2015) found that in very high dimensions, most points with

zero gradient are saddles rather than local minima. For a point to be a local minimum, the surface must curve upward in *every one* of millions of directions, which is unlikely unless the loss is already low. The local minima that do exist in big networks tend to have loss values close to the global minimum. Saddles and plateaus, not bad local minima, are usually what slows training down, and the noise of stochastic gradient descent (next section) helps push the parameters off them. This is an empirical observation about typical networks, not a theorem. Small networks and badly designed problems can still get stuck.

9.5 Batch, mini-batch and stochastic gradient descent

For a dataset of N examples the training loss is an average of per-example losses:

$$L(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell_i(\theta), \qquad \nabla L(\theta) = \frac{1}{N}\sum_{i=1}^{N} \nabla \ell_i(\theta).$$

The exact gradient needs a forward and backward pass through *all* N examples. With $N = 60\,000$ images (the size of MNIST), that is 60 000 passes for a single step. It is far more efficient to *estimate* the gradient from a random subset.

Definition: batch, mini-batch and stochastic gradient descent

- **Batch (full-batch) GD**: every step uses all N examples. Exact gradient, expensive steps.
- **Mini-batch GD**: every step uses a random subset $\mathcal{B}$ of B examples (typically $B = 32$ to 1024) and the estimate $\hat{\mathbf{g}}_B = \frac{1}{B}\sum_{i\in\mathcal{B}} \nabla \ell_i(\theta)$.
- **Stochastic GD (SGD)**, strictly: $B = 1$, one example per step. In everyday usage "SGD" also means mini-batch GD, and PyTorch's `torch.optim.SGD` is used with any batch size.

An **epoch** is one full pass through the training set. An **iteration** is one parameter update, that is, one mini-batch.

Worked example: epochs and iterations

MNIST has $N = 60\,000$ training images. With $B = 64$, one epoch is $60\,000/64 = 937.5$ mini-batches, so 938 iterations: 937 full batches plus a last one with $60\,000 - 937 \times 64 = 32$ images. (PyTorch's `DataLoader` keeps the short last batch by default, or drops it with `drop_last=True`.) Ten epochs are therefore 9380 parameter updates. With full-batch GD, ten epochs would be only ten updates.

9.5.1 Gradient noise and its variance

The mini-batch gradient is a random variable, since it depends on which examples were drawn. Its expected value is the true gradient (it is *unbiased*). If each example in the batch is drawn uniformly at random from the dataset, then each $\nabla \ell_i$ has mean $\mathbf{g} = \nabla L$. Averaging gives

$$\mathbb{E}[\hat{\mathbf{g}}_B] = \frac{1}{B}\sum_{i\in\mathcal{B}} \mathbb{E}[\nabla \ell_i] = \frac{1}{B} \cdot B\,\mathbf{g} = \mathbf{g}.$$

How far is a single estimate from the truth? Let $\sigma^2 = \mathbb{E}\,\|\nabla \ell_i - \mathbf{g}\|^2$ be the spread of the per-example gradients. If the B examples are drawn independently, the variance of an average of B

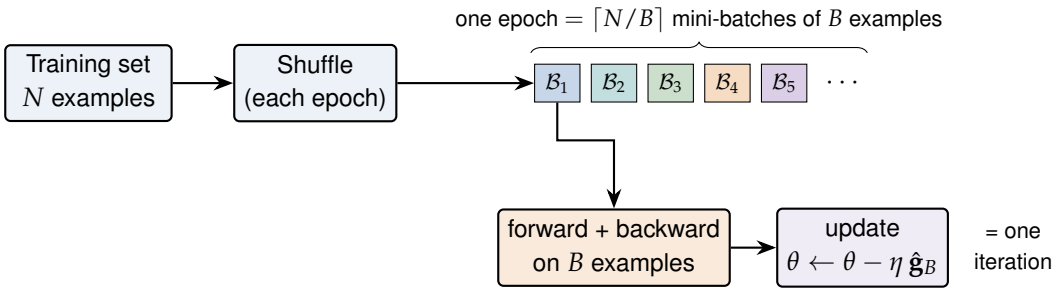


Figure 9.6: Mini-batch training. At the start of each epoch the training set is shuffled and cut into mini-batches. Each mini-batch gives one gradient estimate and one parameter update (an iteration). After all mini-batches have been used, one epoch is complete and the data are reshuffled.

independent terms is the variance of one term divided by B (the cross terms have zero mean):

$$\mathbb{E}\,\|\hat{\mathbf{g}}_B - \mathbf{g}\|^2 = \frac{1}{B^2}\sum_{i\in\mathcal{B}} \mathbb{E}\,\|\nabla \ell_i - \mathbf{g}\|^2 = \frac{\sigma^2}{B}. \tag{9.5}$$

So the *noise* in the gradient shrinks like $1/B$, and its typical size (standard deviation) like $1/\sqrt{B}$. Quadrupling the batch halves the noise amplitude but costs four times the computation. (Drawing without replacement from a finite dataset multiplies Eq. (9.5) by $(N-B)/(N-1)$, which is close to one unless B is a sizeable fraction of N.)

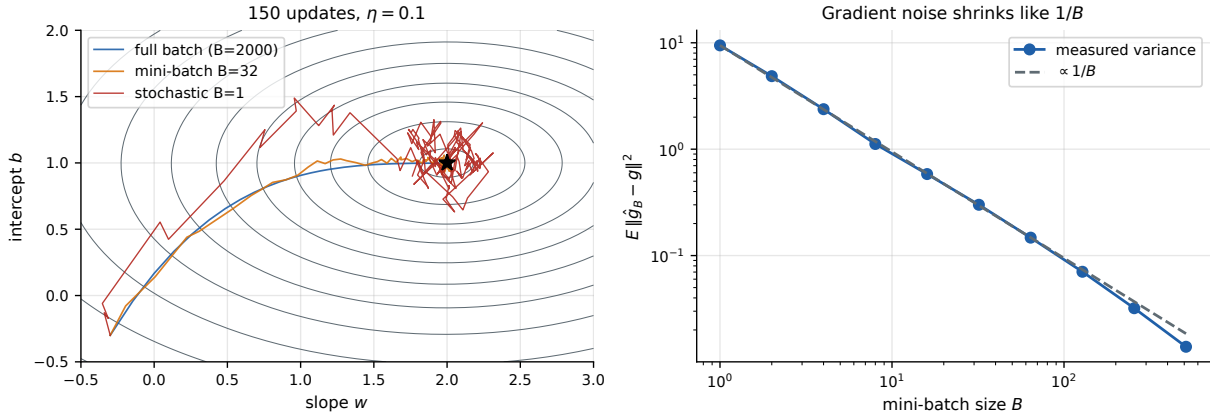


Figure 9.7: Left: 150 updates with $\eta = 0.1$ on a two-parameter linear-regression loss (contours) for full-batch, mini-batch ($B = 32$) and single-example ($B = 1$) gradients. The full-batch path is smooth. The $B = 32$ path is nearly as smooth at $1/62$ of the cost per step. The $B = 1$ path heads in the right direction but keeps jittering around the minimum (star) instead of settling. Right: measured variance of the mini-batch gradient at a fixed point (3000 draws per B). It follows the $1/B$ line of Eq. (9.5); at $B = 512$ it falls slightly below the line, by the finite-dataset factor $(2000 - 512)/1999 \approx 0.74$. Script: `code/ch09/fig_minibatch.py`.

The simulation in Fig. 9.7 confirms the $1/B$ law: the product $B \times$ variance stays at about 9.0 to 9.7 from $B = 1$ to $B = 128$. The left panel shows the practical consequence. With a constant learning rate, noisy gradients never let the parameters settle exactly at the minimum. They keep bouncing inside a "noise ball" whose size grows with η and shrinks with B. There are two ways to shrink the ball at the end of training: increase B, or, much more commonly, *decrease the learning rate* (Section 9.6). Classical stochastic-approximation theory (Robbins and Monro, 1951) shows that SGD converges if the step sizes satisfy $\sum_t \eta_t = \infty$ (large enough to travel any distance) and $\sum_t \eta_t^2 < \infty$ (shrinking fast enough to average the noise away), for example $\eta_t \propto 1/t$.

Intuition: why noise is not all bad

A noisy step is a slightly drunken walk downhill. It wastes some effort, but it can also shake the walker off a saddle point or out of a narrow, sharp valley. There is evidence (Keskar et al., 2017) that very large batches tend to find sharper minima that generalize slightly worse, and that the noise of small batches acts as a mild regularizer. In practice: choose the batch size to use your hardware efficiently (powers of two from 32 to 512 are typical), then tune the learning rate for that batch size. A common rule when changing the batch size is to scale η in proportion to B, combined with warm-up (Goyal et al., 2017).

Wireless connection: averaging pilots is the same $1/B$ law

When a 5G receiver estimates a channel coefficient from DMRS pilots, each pilot gives an unbiased but noisy estimate, and averaging B pilots (over subcarriers in a coherence bandwidth, or symbols in a coherence time) reduces the estimation-error variance by $1/B$. That is exactly Eq. (9.5). The engineering trade-off is the same as well: averaging more pilots reduces noise but costs resources (pilot overhead in the radio case, computation in the learning case), and averaging over too wide a window mixes in stale information (channel variation, or parameters that have moved on).

9.6 Learning-rate schedules

Early in training we want large steps to cross the landscape quickly. Late in training we want small steps to settle into a minimum and shrink the noise ball. A **learning-rate schedule** η_t changes the learning rate with the epoch (or iteration) t. The common ones, with initial rate η_0 and total length T, are:

$$\begin{aligned}
\text{Step decay:} \quad & \eta_t = \eta_0 \, \gamma^{\lfloor t/s \rfloor} \quad \text{(multiply by } \gamma\text{, e.g. 0.5 or 0.1, every } s \text{ epochs)}, \\
\text{Exponential:} \quad & \eta_t = \eta_0 \, \gamma^{t} \quad \text{(shrink by a constant factor every epoch)}, \\
\text{Cosine:} \quad & \eta_t = \eta_{\min} + \tfrac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T}\right), \\
\text{Linear warm-up:} \quad & \eta_t = \eta_0 \, \frac{t+1}{T_w} \quad \text{for } t < T_w, \text{ then any of the above.}
\end{aligned}$$

Floor brackets $\lfloor \cdot \rfloor$ mean "round down". The cosine schedule starts at η_0 ($\cos 0 = 1$), passes $\frac{1}{2}(\eta_0 + \eta_{\min})$ at the halfway point ($\cos(\pi/2) = 0$) and ends at $\eta_{\min}$ ($\cos \pi = -1$). It was popularized by Loshchilov and Hutter (2017) and is today's default for many models.

Worked example: evaluating the schedules

With $\eta_0 = 0.1$: step decay with $\gamma = 0.5$, $s = 30$ gives 0.1, 0.05, 0.025, 0.0125 at epochs $0, 30, 60, 90$. Exponential decay with $\gamma = 0.96$ gives $0.1 \times 0.96^{50} \approx 0.0130$ at epoch 50. Cosine with $\eta_{\min} = 0$, $T = 100$ gives exactly 0.05 at epoch 50.

Why warm-up? At initialization the parameters are random and the gradients can be large and erratic. Optimizers such as Adam (Chapter 11) also have poor statistics in the first few steps. A large learning rate at that moment can throw the network into a bad region it never recovers from. Ramping η up linearly over the first few hundred or thousand iterations avoids this, and it is essential for Transformers and large-batch training.

9.6.1 Finding a good learning rate: the range test

How do you choose η_0 in the first place? A cheap and effective method is the **learning-rate range test** (Smith, 2017). Start with a tiny learning rate and increase it exponentially every mini-batch (for example from 10^{-4} to 10 over a few hundred steps), recording a smoothed training loss. Plot the loss against η on a log axis. At first the loss barely moves (steps too small), then it falls quickly, then it bottoms out, and finally it explodes once η passes the stability limit. A good η_0 is in the steep-descent region, a few times smaller than the value at the minimum of the curve.

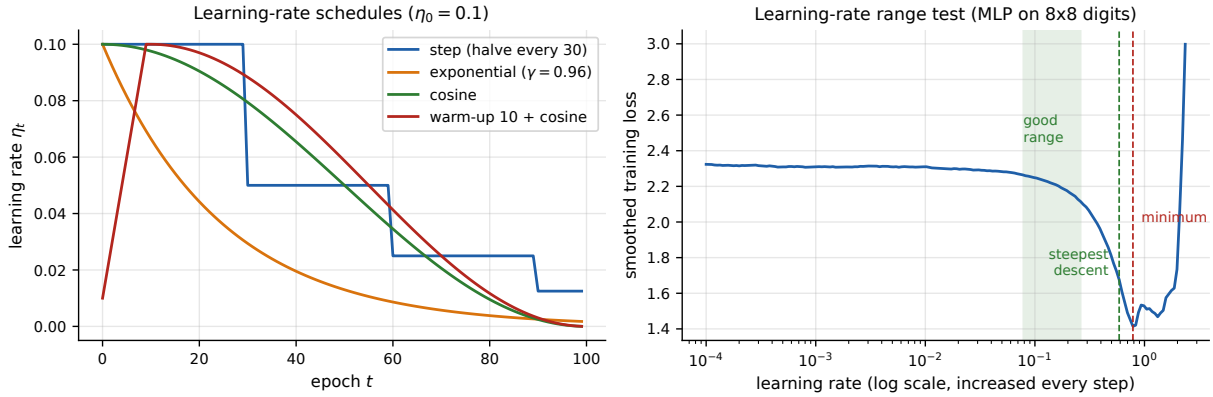


Figure 9.8: Left: four learning-rate schedules over 100 epochs from $\eta_0 = 0.1$. Step decay drops abruptly, exponential decay drops fastest early on, cosine stays high for longer and then glides down, and warm-up ramps up for 10 epochs before following a cosine. Right: a real range test on a 64–64–10 MLP trained on the bundled 8×8 digits dataset ($B = 64$, plain SGD). The loss is flat for $\eta < 10^{-2}$, falls steeply around 0.1–0.6, bottoms out near $\eta \approx 0.8$, and blows up just above it. The shaded band (one-tenth to one-third of the minimum position) is a sensible choice of η_0. Script: `code/ch09/fig_schedules.py`.

Listing 9.1: The core of a learning-rate range test in PyTorch (full script: `code/ch09/fig_schedules.py`).

```
lr_min, lr_max, n = 1e-4, 10.0, 200
for i in range(n):
    lr = lr_min * (lr_max / lr_min) ** (i / (n - 1))    # exponential sweep
    for g in opt.param_groups: g['lr'] = lr
    idx = torch.randint(0, len(X), (64,))                # random mini-batch
    loss = F.cross_entropy(model(X[idx]), y[idx])
    opt.zero_grad(); loss.backward(); opt.step()
    smooth = loss.item() if i == 0 else 0.9*smooth + 0.1*loss.item()
    lrs.append(lr); losses.append(smooth)
    if smooth > 2 * min(losses) and i > 20: break        # it exploded: stop
```

Pitfall: symptoms of a wrong learning rate

Too high: the loss jumps around, rises, or becomes `nan` within a few iterations. **Too low**: the loss decreases in a straight, slow line and is still falling steadily when you stop. **About right**: a fast initial drop that gradually levels off. Always plot the training loss per iteration on a log axis. When something is wrong with training, try dividing the learning rate by 3 or 10 before you change anything else.

9.7 Wireless highlight: the LMS equalizer is stochastic gradient descent

Everything in this chapter has a direct ancestor in adaptive signal processing. In 1960 Bernard Widrow and Ted Hoff introduced the **least-mean-squares (LMS)** algorithm for training adaptive linear elements. It went on to become the workhorse of adaptive equalizers, echo cancellers and interference cancellers in modems. LMS is, line for line, stochastic gradient descent with batch size one on a mean-squared-error loss. Deriving it that way connects every concept in this chapter to something you can measure on a radio.

9.7.1 The problem: undoing intersymbol interference

A BPSK transmitter sends symbols $s_k \in \{-1, +1\}$. A multipath channel with impulse response $\mathbf{h} = (h_0, h_1, h_2)$ smears each symbol over three sample times, and the receiver adds Gaussian noise n_k:

$$x_k = h_0 s_k + h_1 s_{k-1} + h_2 s_{k-2} + n_k.$$

We use $\mathbf{h} = (0.4,\ 1.0,\ 0.3)$. The main tap is $h_1 = 1$ (so the channel delays the signal by one sample), with a pre-cursor of 0.4 and a post-cursor of 0.3. Sample x_k therefore mixes the wanted symbol s_{k-1} with its two neighbours, which is **intersymbol interference** (ISI). In the worst case the interference adds up to $0.4 + 0.3 = 0.7$, so the two BPSK levels ± 1 are smeared into the ranges $[0.3, 1.7]$ and $[-1.7, -0.3]$, plus noise (Fig. 9.11, left).

A **linear transversal equalizer** is an FIR filter with M adjustable taps $\mathbf{w} = (w_0, \ldots, w_{M-1})$ applied to the last M received samples:

$$y_k = \mathbf{w}^{\mathsf{T}} \mathbf{x}_k, \qquad \mathbf{x}_k = (x_k,\ x_{k-1},\ \ldots,\ x_{k-M+1})^{\mathsf{T}}.$$

We want y_k to equal a transmitted symbol $d_k = s_{k-D}$ for some decision delay D. During a *training sequence* (a known preamble or pilot pattern) the receiver knows d_k, so it can compute the error $e_k = d_k - y_k$. Figure 9.9 shows the structure. Compare it with Fig. 9.1: it is a one-layer linear neural network with no activation function.

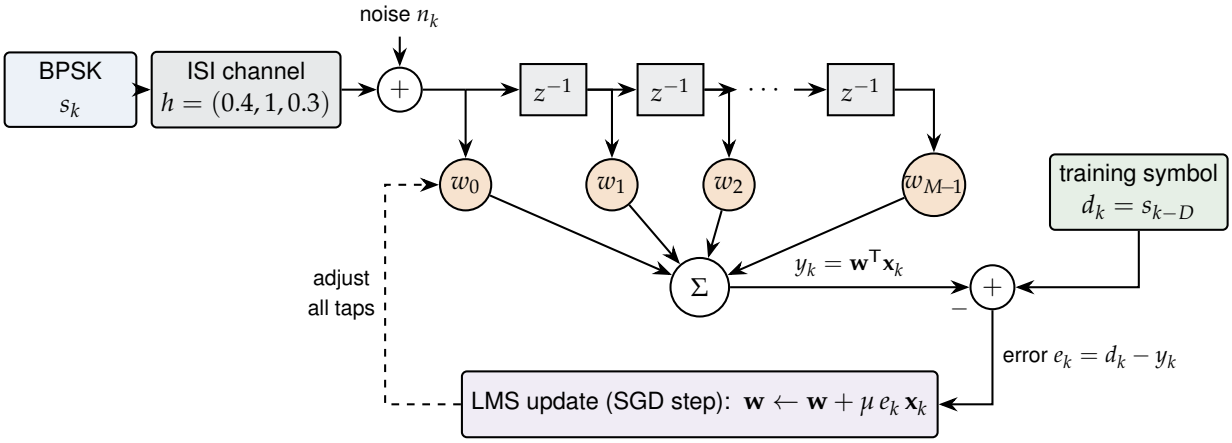


Figure 9.9: An LMS adaptive equalizer. The received samples pass down a tapped delay line; the equalizer output is a weighted sum $y_k = \mathbf{w}^{\mathsf{T}}\mathbf{x}_k$ (a single linear neuron). During training the known symbol d_k gives the error e_k, and every tap is nudged by μe_k times its own input sample. This is exactly one stochastic-gradient-descent step on the loss $\frac{1}{2}e_k^2$ with learning rate μ.

9.7.2 Deriving LMS as SGD

The natural loss is the **mean squared error** over the random symbols and noise,

$$J(\mathbf{w}) = \mathbb{E}\big[e_k^2\big] = \mathbb{E}\big[(d_k - \mathbf{w}^{\mathsf{T}}\mathbf{x}_k)^2\big].$$

Expanding the square and using linearity of expectation:

$$J(\mathbf{w}) = \mathbb{E}[d_k^2] - 2\,\mathbf{w}^{\mathsf{T}}\underbrace{\mathbb{E}[d_k\mathbf{x}_k]}_{\mathbf{p}} + \mathbf{w}^{\mathsf{T}}\underbrace{\mathbb{E}[\mathbf{x}_k\mathbf{x}_k^{\mathsf{T}}]}_{\mathbf{R}}\mathbf{w},$$

where $\mathbf{R}$ is the $M \times M$ input autocorrelation matrix and $\mathbf{p}$ the cross-correlation between the input and the desired symbol. This is a quadratic bowl in $\mathbf{w}$ with curvature matrix $2\mathbf{R}$, exactly the kind of loss analysed in Section 9.3.1. Its gradient is $\nabla J = -2\mathbf{p} + 2\mathbf{R}\mathbf{w}$. Setting it to zero gives the best possible linear equalizer, the **Wiener solution**:

$$\mathbf{R}\,\mathbf{w}^\star = \mathbf{p} \quad\Longrightarrow\quad \mathbf{w}^\star = \mathbf{R}^{-1}\mathbf{p}, \qquad J_{\min} = \mathbb{E}[d_k^2] - \mathbf{p}^{\mathsf{T}}\mathbf{w}^\star.$$

Full-batch gradient descent on J would be $\mathbf{w} \leftarrow \mathbf{w} - \eta(2\mathbf{R}\mathbf{w} - 2\mathbf{p})$. The trouble is that the receiver does not know $\mathbf{R}$ and $\mathbf{p}$, which are expectations over the unknown channel. Widrow and Hoff's idea was the SGD idea: replace the expectation by the single most recent sample. The per-sample loss is $\ell_k(\mathbf{w}) = \frac{1}{2}e_k^2$ (the $\frac{1}{2}$ just removes a factor of 2). With $e_k = d_k - \mathbf{w}^{\mathsf{T}}\mathbf{x}_k$ the chain rule gives

$$\nabla_{\mathbf{w}}\ell_k = e_k\,\frac{\partial e_k}{\partial \mathbf{w}} = e_k\,(-\mathbf{x}_k) = -e_k\mathbf{x}_k,$$

and one SGD step with learning rate μ is

Key equation: LMS = SGD on the squared error, batch size 1

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \mu\,\nabla\ell_k = \mathbf{w}_k + \mu\,e_k\,\mathbf{x}_k, \qquad e_k = d_k - \mathbf{w}_k^{\mathsf{T}}\mathbf{x}_k. \tag{9.6}$$

That is the whole LMS algorithm: $2M$ multiplications per symbol, no matrix inversion, no knowledge of the channel. The per-sample gradient is unbiased: $\mathbb{E}[-e_k\mathbf{x}_k] = \mathbf{R}\mathbf{w} - \mathbf{p} = \frac{1}{2}\nabla J$. So on average LMS follows the true gradient of the MSE, and the noise analysis of Section 9.5 applies.

9.7.3 What the theory of this chapter predicts

Taking expectations of Eq. (9.6) (and assuming, as the standard analysis does, that $\mathbf{w}_k$ is roughly independent of the current $\mathbf{x}_k$) gives

$$\mathbb{E}[\mathbf{w}_{k+1}] - \mathbf{w}^\star = (\mathbf{I} - \mu\mathbf{R})\,\big(\mathbb{E}[\mathbf{w}_k] - \mathbf{w}^\star\big).$$

This is the multi-dimensional version of $e_{k+1} = (1 - \eta c)e_k$, with the eigenvalues λ_i of $\mathbf{R}$ playing the role of the curvatures. Every lesson transfers directly:

- **Stability**: the mean converges only if $|1 - \mu\lambda_i| < 1$ for all i, that is $0 < \mu < 2/\lambda_{\max}$. This is Eq. (9.4) again. Because the noise also has to stay bounded, the practical (mean-square) limit is tighter. A widely used safe rule is $\mu < 2/\operatorname{tr}(\mathbf{R}) = 2/(MP_x)$, where P_x is the received power.
- **Speed and condition number**: the slowest mode decays like $(1 - \mu\lambda_{\min})^k$. A channel with deep spectral nulls makes $\mathbf{R}$ ill-conditioned (large *eigenvalue spread*), so LMS converges slowly. This is the zig-zag of Fig. 9.3.

- **Noise floor**: with a constant μ the taps keep jittering around $\mathbf{w}^\star$, so the steady-state MSE is above $J_{\min}$. The excess (the *misadjustment*) grows with μ. This is the SGD noise ball.

For our channel with an $M = 11$-tap equalizer, delay $D = 6$ and SNR $= 20\,\text{dB}$, the script computes $\mathbf{R}$ exactly: its eigenvalues range from 0.120 to 2.826 (eigenvalue spread 23.5), so the mean-stability limit is $2/\lambda_{\max} = 0.71$ and the safe rule gives $\mu < 2/\operatorname{tr}\mathbf{R} = 0.14$. The Wiener solution achieves $J_{\min} = 0.0315$ ($-15.0\,\text{dB}$).

Listing 9.2: LMS equalizer training loop (from `code/ch09/lms_equalizer.py`).

```
h = np.array([0.4, 1.0, 0.3]); M = 11; D = 6       # channel, taps, delay
s = rng.choice([-1.0, 1.0], n + M)                  # BPSK training symbols
x = np.convolve(s, h)[:n + M] + np.sqrt(sigma2) * rng.standard_normal(n + M)
w = np.zeros(M)
for k in range(n):
    xk = x[k + M - 1::-1][:M]          # regressor: last M received samples
    y  = w @ xk                        # equalizer output  (forward pass)
    e  = s[k + M - 1 - D] - y          # error vs. known symbol (loss = e^2/2)
    w  = w + mu * e * xk               # SGD step = LMS update
```

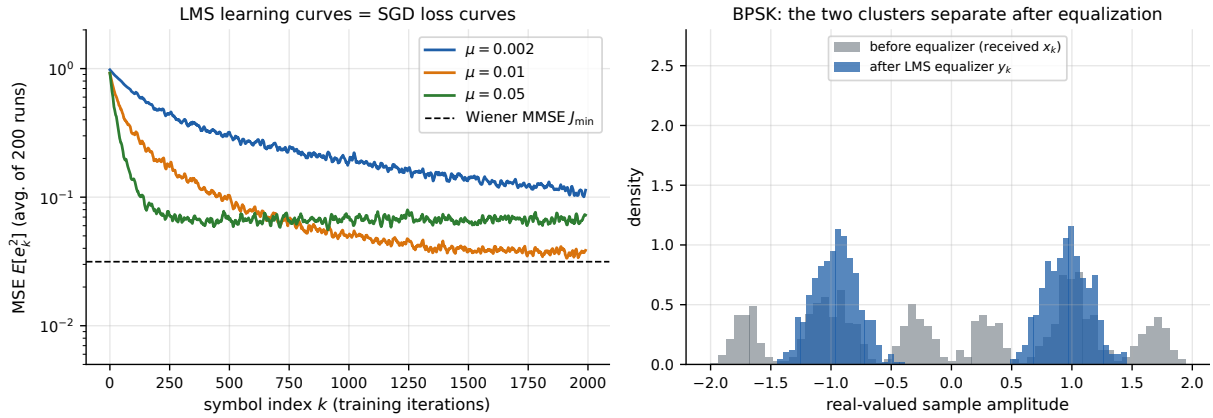


Figure 9.10: Left: LMS learning curves (MSE versus symbol index, averaged over 200 independent runs) for three step sizes, with the Wiener limit $J_{\min}$ (dashed). $\mu = 0.002$ is stable and quiet but still converging after 2000 symbols (MSE 0.118). $\mu = 0.05$ converges fastest (below 0.1 after about 140 symbols) but settles on a high noise floor (MSE 0.067). $\mu = 0.01$ is the compromise: it reaches 0.0375, close to $J_{\min} = 0.0315$, after about 1500 symbols. Right: histogram of received samples before equalization (several smeared sub-clusters at $\pm 1 \pm 0.4 \pm 0.3$ caused by ISI) and of the equalizer output after training (two tight clusters at ± 1). Script: `code/ch09/lms_equalizer.py`.

Figures 9.10 and 9.11 show the result. The five central taps learned after 3000 symbols with $\mu = 0.01$ are close to the Wiener taps:

$$\mathbf{w}_{\text{LMS}} : (0.242, -0.563, 1.307, -0.409, 0.117), \qquad \mathbf{w}^\star : (0.251, -0.578, 1.312, -0.424, 0.129).$$

They approximate the inverse of the channel: alternating signs that cancel the pre- and post-cursors. On 200 000 fresh symbols, a receiver that simply takes the sign of the main-tap sample makes 8.6×10^{-4} bit errors (the worst-case ISI patterns push some samples across zero), while the trained equalizer made no errors at all.

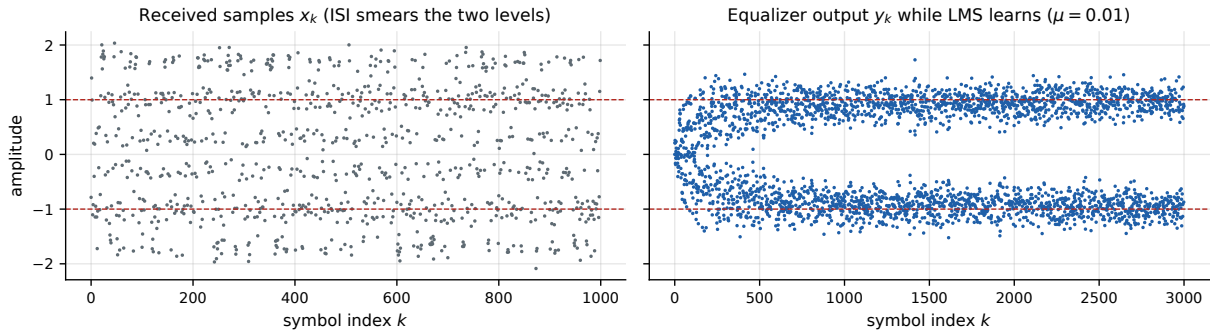


Figure 9.11: The BPSK "constellation" (the real-valued amplitude) as time runs left to right. Left: received samples; ISI spreads each level into several sub-levels, some of which come close to zero. Right: equalizer output while LMS trains with $\mu = 0.01$. All taps start at zero, so the output starts at zero, and within a few hundred symbols the samples condense onto the two ideal levels ± 1 (dashed). This is the same picture as a training loss falling.

Wireless connection: a dictionary between adaptive filtering and deep learning

Adaptive filtering	Deep learning
step size μ	learning rate η
training sequence / pilots	labelled training data
LMS (sample-by-sample)	SGD with $B = 1$
block LMS	mini-batch SGD
eigenvalue spread of $\mathbf{R}$	condition number of the Hessian
misadjustment	SGD noise floor
normalized LMS ($\mu / \|\mathbf{x}_k\|^2$)	adaptive step sizes (Adam, Chapter 11)
decision-directed mode	self-training on own predictions
RLS (whitens with $\mathbf{R}^{-1}$)	second-order / preconditioned methods

Modern 5G NR receivers mostly equalize in the frequency domain, using one complex tap per subcarrier computed from DMRS channel estimates, so the classical time-domain LMS equalizer is less visible than it was in single-carrier modems. LMS-type updates are still common in echo cancellation, adaptive interference cancellation and the adaptation of digital predistortion. More importantly, the neural receivers of Part V are trained with exactly the update in Eq. (9.6), generalized from one linear layer to many nonlinear layers by backpropagation (Chapter 10).

Pitfall: complex baseband

For complex signals (QAM, OFDM) the LMS update becomes $\mathbf{w} \leftarrow \mathbf{w} + \mu\, e_k\, \mathbf{x}_k^*$, with a complex conjugate, if the output is defined as $y_k = \mathbf{w}^{\mathsf{T}}\mathbf{x}_k$ (or $\mathbf{w} \leftarrow \mathbf{w} + \mu\, e_k^* \mathbf{x}_k$ for $y_k = \mathbf{w}^{\mathsf{H}}\mathbf{x}_k$). The conjugate comes from differentiating $|e_k|^2 = e_k e_k^*$ with respect to complex weights (Wirtinger calculus). Leaving it out is a classic bug: the algorithm then rotates the taps in the wrong direction and does not converge.

9.8 Putting it together: a checklist

1. Pick a batch size that suits your hardware (start with 32–256).
2. Run a learning-rate range test (a few hundred iterations). Choose η_0 in the steep-descent

region, around one-tenth to one-third of the loss minimum's position.

3. Train with warm-up (a few percent of the steps) followed by cosine or step decay.
4. Watch the training loss per iteration on a log scale. If it explodes, divide η by 3–10. If it is a slow straight line, multiply by 3.
5. If the loss stalls on a plateau, be patient for a while (it may be a saddle). If it stays stuck, look at the initialization and the optimizer (Chapter 11).

Chapter recap

- Gradient descent repeats $\theta \leftarrow \theta - \eta \nabla L(\theta)$. Minus the gradient is the steepest downhill direction, and for small η each step lowers the loss by about $\eta \, \|\nabla L\|^2$.
- On a quadratic with curvature c the error is multiplied by $1 - \eta c$ every step. Training converges iff $0 < \eta < 2/c$; $\eta = 1/c$ is optimal. For $(w-3)^2$ with $\eta = 0.1$ the factor is 0.8 ($0 \to 0.6 \to 1.08 \to 1.464$); with $\eta = 1.1$ it is -1.2 and training diverges.
- With many parameters the steepest direction limits η and the flattest direction limits speed. The condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ gives a best rate of $(\kappa - 1)/(\kappa + 1)$ per step and causes zig-zag paths.
- Neural losses are non-convex: local minima, saddles and plateaus. In high dimensions, saddles and plateaus are the more common obstacle.
- Mini-batch gradients are unbiased with noise variance σ^2/B. An epoch is a pass over the data: 60 000 examples with $B = 64$ make 938 iterations.
- Schedules (step, exponential, cosine, warm-up) take big steps early and small steps late. The range test finds a good η_0 in minutes.
- The LMS equalizer $\mathbf{w} \leftarrow \mathbf{w} + \mu e_k \mathbf{x}_k$ is SGD with $B = 1$ on the MSE. Its stability limit, eigenvalue-spread slowdown and misadjustment are this chapter's theory in radio form.

Exercises

1. For $L(w) = (w-3)^2$, $w_0 = 0$ and $\eta = 0.25$, compute w_1, w_2, w_3 and the contraction factor. How many steps until $|w_k - 3| < 0.01$?
2. For $L(w) = 5(w+1)^2$, what is the curvature c? Give the largest stable learning rate and the one-step learning rate.
3. On $L = \frac{1}{2}(w_0^2 + 100 w_1^2)$, find the largest stable η, the best fixed η and its contraction factor. Roughly how many steps does it take to reduce the error by 10^3?
4. A dataset has 50 000 examples and you use $B = 128$. How many iterations make one epoch? How many updates happen in 20 epochs? How large is the last mini-batch of each epoch?
5. If the gradient noise standard deviation at $B = 32$ is 0.4 (in some unit), what is it at $B = 128$ and $B = 512$? What does each cost per step relative to $B = 32$?
6. For $L = w_1^2 - w_2^2$ with $\eta = 0.05$, starting at $(1, 10^{-4})$: what are w_1 and w_2 after k steps? After how many steps does $|w_2|$ exceed 1?
7. An LMS equalizer has $M = 8$ taps and received power $P_x = 0.5$. Give the safe step-size

limit $2/(MP_x)$. If you double the received power without changing μ, what can happen, and how does normalized LMS avoid this?

8. (Conceptual) Explain in your own words why a constant learning rate with SGD cannot converge exactly to a minimum, while full-batch GD can. Suggest two remedies.

Answers

1. Factor $1 - 2(0.25) = 0.5$. $w_1 = 0 + 0.25 \times 6 = 1.5$; $w_2 = 1.5 + 0.25 \times 3 = 2.25$; $w_3 = 2.25 + 0.25 \times 1.5 = 2.625$ (errors $-3, -1.5, -0.75, -0.375$). We need $3 \times 0.5^k < 0.01$, so $k > \log_2 300 \approx 8.2$: 9 steps.

2. $L''(w) = 10$, so $c = 10$. Stable for $0 < \eta < 2/10 = 0.2$; one-step rate $\eta = 1/c = 0.1$. Check: $w_1 = w_0 - 0.1 \times 10(w_0 + 1) = -1$ for any w_0.

3. Curvatures 1 and 100. Stable iff $\eta < 2/100 = 0.02$. Best $\eta = 2/101 \approx 0.0198$, factor $99/101 \approx 0.980$. Steps: $\ln(10^{-3})/\ln(99/101) \approx 6.908/0.0200 \approx 345$.

4. $50\,000/128 = 390.625$, so 391 iterations per epoch: 390 full batches and a last batch of $50\,000 - 390 \times 128 = 80$ examples. 20 epochs make 7820 updates.

5. The standard deviation scales like $1/\sqrt{B}$: at $B = 128$ it is $0.4/\sqrt{4} = 0.2$; at $B = 512$ it is $0.4/\sqrt{16} = 0.1$. Compute per step is $4\times$ and $16\times$ that of $B = 32$.

6. $w_1 \leftarrow 0.9\, w_1$, $w_2 \leftarrow 1.1\, w_2$, so $w_1 = 0.9^k$ and $w_2 = 10^{-4} \times 1.1^k$. $|w_2| > 1$ needs $1.1^k > 10^4$, i.e. $k > 4 \ln 10 / \ln 1.1 = 9.21/0.0953 \approx 96.6$: 97 steps. Meanwhile $w_1 \approx 0.9^{97} \approx 4 \times 10^{-5}$, so for most of those steps the walker sits almost on the saddle.

7. $2/(8 \times 0.5) = 0.5$. Doubling P_x doubles every eigenvalue of $\mathbf{R}$ (since $\mathbf{R} = \mathbb{E}[\mathbf{x}\mathbf{x}^{\mathsf{T}}]$ scales with power), so the effective $\mu\lambda$ doubles and a previously stable μ may become unstable. Normalized LMS uses the step $\mu/(\epsilon + \|\mathbf{x}_k\|^2)$, which divides out the input power, making the behaviour independent of signal level. This is an early form of adaptive step size.

8. With SGD the gradient at the minimum is not zero for an individual mini-batch: different batches disagree, so the update keeps kicking the parameters around with steps of size about $\eta \times$(noise). The parameters wander in a ball whose size grows with η and shrinks with B. Full-batch GD uses the exact gradient, which really is zero at the minimum, so the steps vanish. Remedies: decay the learning rate (a schedule satisfying $\sum \eta_t = \infty$, $\sum \eta_t^2 < \infty$), increase the batch size late in training, or average the iterates.

Self-test quiz

1. In $\theta \leftarrow \theta - \eta \nabla L$, why the minus sign?
   The gradient points uphill (direction of fastest increase); we want to go downhill.

2. A loss has curvature $c = 4$. Which learning rates converge?
   $0 < \eta < 0.5$; $\eta = 0.25$ *reaches the minimum of the quadratic in one step.*

3. Your training loss becomes `nan` after 20 iterations. What is the first thing to try?
   Reduce the learning rate by a factor of 3–10 (and check for division by zero or log of zero).

4. What is the difference between an epoch and an iteration?
   An iteration is one parameter update on one mini-batch; an epoch is one pass over the whole training set

($\lceil N/B \rceil$ iterations).

5. Why does the zig-zag in an elongated valley happen?
   The learning rate is large relative to the steep direction's curvature (factor between -1 and 0, so it overshoots) while being small for the gentle direction; ill-conditioning, $\kappa \gg 1$.

6. In the LMS update $\mathbf{w} \leftarrow \mathbf{w} + \mu e_k \mathbf{x}_k$, what plays the role of the loss and of the batch size?
   The loss is $\frac{1}{2}e_k^2$ (instantaneous squared error); the batch size is 1 (one symbol per update).

Further reading

- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016, Ch. 4 (numerical optimization, condition number) and Ch. 8 (optimization for training deep models).
- L. Bottou, F. E. Curtis, J. Nocedal, "Optimization methods for large-scale machine learning," *SIAM Review*, 60(2), 2018. A thorough treatment of SGD theory.
- H. Robbins, S. Monro, "A stochastic approximation method," *Annals of Mathematical Statistics*, 22(3), 1951.
- L. N. Smith, "Cyclical learning rates for training neural networks," *IEEE WACV*, 2017 (the learning-rate range test).
- I. Loshchilov, F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," *ICLR*, 2017 (cosine schedules).
- P. Goyal et al., "Accurate, large minibatch SGD: training ImageNet in 1 hour," arXiv preprint 1706.02677, 2017 (linear scaling rule and warm-up).
- Y. Dauphin et al., "Identifying and attacking the saddle point problem in high-dimensional non-convex optimization," *NeurIPS*, 2014.
- B. Widrow, M. E. Hoff, "Adaptive switching circuits," *IRE WESCON Convention Record*, 1960 (the LMS algorithm).
- S. Haykin, *Adaptive Filter Theory*, 5th ed., Pearson, 2014. LMS, NLMS, RLS, misadjustment and eigenvalue spread in depth.

Chapter 10

Backpropagation, Derived and Computed by Hand

Gradient descent (Chapter 9) needs the gradient of the loss with respect to *every* parameter, and a modern network has millions or billions of them. **Backpropagation** is the algorithm that computes all of these partial derivatives exactly, in a single backward sweep through the network, for roughly twice the cost of the forward pass. Without it, deep learning would not exist in practice.

This chapter takes the mystery out of backpropagation. We start with a picture (a relay race run backwards) and the idea of a computational graph. Then we derive the four batch equations every framework implements, index by index, so that you can see where each transpose comes from. After that we push a complete two-layer network through one training step *by hand*, with every number checked in Python. You will watch one hidden unit die along the way, and we will discuss why. We then look at the softmax output, the difference between forward-mode and reverse-mode automatic differentiation, and a 59-line autograd engine that reproduces the hand example exactly. The chapter closes with the idea behind end-to-end learned radio links: backpropagating *through the channel*.

10.1 The relay race in reverse

Intuition: blame passed backwards along a relay team

Picture a relay team. Runner 1 hands the baton to runner 2, who hands it to runner 3, who crosses the line. The team finishes 2 seconds slower than the target. Whose fault is it, and by how much?

The coach starts at the finish line, which is the only place where the error is directly visible. She asks runner 3: "How much did the final time change per unit of your effort?" Runner 3 then passes the blame back to runner 2, scaled by how strongly runner 2's hand-off affected runner 3's leg. Runner 2 passes it on to runner 1 in the same way. Every runner receives a share of blame equal to *the blame of the next runner times their own local influence on that runner*. Nobody needs to understand the whole race; each runner only knows their own hand-off.

Backpropagation is exactly this. The forward pass carries the baton (the activations) from the input to the loss. The backward pass carries the blame (the gradients) from the loss back to the input. At each stage, the incoming blame is multiplied by a *local derivative* that the stage can compute by itself. That multiplication is the chain rule.

Recall the chain rule from Chapter 3: if L depends on a only through b, then $\frac{\mathrm{d}L}{\mathrm{d}a} = \frac{\mathrm{d}L}{\mathrm{d}b}\,\frac{\mathrm{d}b}{\mathrm{d}a}$. For a long chain $a \to b \to c \to L$:

$$\frac{\mathrm{d}L}{\mathrm{d}a} = \underbrace{\frac{\mathrm{d}L}{\mathrm{d}c}}_{\text{blame at } c} \cdot \underbrace{\frac{\mathrm{d}c}{\mathrm{d}b}}_{\text{local}} \cdot \underbrace{\frac{\mathrm{d}b}{\mathrm{d}a}}_{\text{local}}.$$

We could evaluate this product from left to right or from right to left. Backpropagation evaluates

it starting from the *loss end*, so that the partial product $\frac{dL}{dc}$, then $\frac{dL}{db}$, then $\frac{dL}{da}$ is reused for every parameter that feeds into c, b or a. This reuse is what makes it efficient (Section 10.6).

10.2 Computational graphs

Definition: computational graph

A **computational graph** is a directed graph whose nodes are values (inputs, parameters, intermediate results, the loss), and whose edges show which values were used to compute which. Every intermediate node is produced by a simple *operation* (add, multiply, exp, ReLU, matrix product, ...) whose local derivatives we know.

Worked example: a value used twice

Let $f(a,b) = (a+b)\,b$ at $a = 2$, $b = 3$. Introduce the intermediate $c = a + b$ so that $f = c \cdot b$.
Forward: $c = 2 + 3 = 5$, $f = 5 \times 3 = 15$.
Backward: start with $\frac{\partial f}{\partial f} = 1$.

- Multiply node $f = c \cdot b$: local derivatives $\frac{\partial f}{\partial c} = b = 3$ and $\frac{\partial f}{\partial b}\big|_{\text{direct}} = c = 5$.
- Add node $c = a + b$: local derivatives $\frac{\partial c}{\partial a} = 1$, $\frac{\partial c}{\partial b} = 1$. So $\frac{\partial f}{\partial a} = \frac{\partial f}{\partial c} \cdot 1 = 3$, and c sends another $3 \cdot 1 = 3$ to b.
- b received blame along *two* paths: 5 directly from the multiply and 3 via c. The total is the **sum**: $\frac{\partial f}{\partial b} = 5 + 3 = 8$.

Check by expanding: $f = ab + b^2$, so $\frac{\partial f}{\partial a} = b = 3$ and $\frac{\partial f}{\partial b} = a + 2b = 8$. Correct.

The rule "*when a value feeds several consumers, add up the gradients coming back from each*" is the multivariable chain rule. In code it shows up as `grad += ...` rather than `grad = ...`. Forgetting it is one of the most common bugs in hand-written backprop.

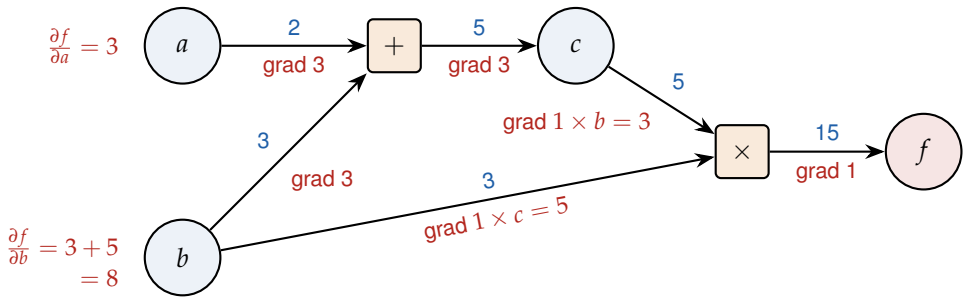


Figure 10.1: Computational graph of $f = (a+b)\,b$. Blue numbers above the edges are forward values; red numbers below the edges are the gradients $\partial f / \partial(\cdot)$ flowing backwards along them, each the downstream gradient multiplied by a local derivative. Because b feeds two operations, it receives two gradient contributions (5 and 3), which are added.

10.3 The backpropagation equations for a layer

Now the real thing. We use the book's batch-first convention (Chapter 1). A batch of N examples enters a dense layer as a matrix $\mathbf{A}_{\text{prev}} \in \mathbb{R}^{N \times n_{\text{in}}}$ (row n is example n). The layer computes

$$\mathbf{Z} = \mathbf{A}_{\text{prev}}\mathbf{W} + \mathbf{b}, \qquad \mathbf{A} = f(\mathbf{Z}), \tag{10.1}$$

with weights $\mathbf{W} \in \mathbb{R}^{n_{\text{in}} \times n_{\text{out}}}$ (rows indexed by inputs, columns by units), bias $\mathbf{b} \in \mathbb{R}^{1 \times n_{\text{out}}}$ added to every row, and an activation f applied element by element. In index form,

$$Z_{nj} = \sum_{i=1}^{n_{\text{in}}} A^{\text{prev}}_{ni} W_{ij} + b_j, \qquad A_{nj} = f(Z_{nj}).$$

The loss L is a single number, usually the *mean* over the batch. Throughout, the notation $d\mathbf{X}$ means "the matrix of partial derivatives $\frac{\partial L}{\partial X_{..}}$, with the same shape as $\mathbf{X}$". Assume the layer above has already given us $d\mathbf{A} = \frac{\partial L}{\partial \mathbf{A}}$, the blame arriving at this layer's output. We need four things: $d\mathbf{Z}$, $d\mathbf{W}$, $d\mathbf{b}$, and $d\mathbf{A}_{\text{prev}}$ (to pass further back).

1. Through the activation. A_{nj} depends only on Z_{nj} (the activation acts element-wise), so the chain rule has a single term:

$$dZ_{nj} = \frac{\partial L}{\partial Z_{nj}} = \frac{\partial L}{\partial A_{nj}} \frac{\partial A_{nj}}{\partial Z_{nj}} = dA_{nj}\, f'(Z_{nj}) \quad \Longrightarrow \quad d\mathbf{Z} = d\mathbf{A} \odot f'(\mathbf{Z}).$$

2. The weights. W_{ij} appears in Z_{nk} only when $k = j$, and then with coefficient A^{prev}_{ni}. It appears in *every* row n, though, because all examples share the same weights. Summing over every place it appears:

$$dW_{ij} = \sum_{n=1}^{N} \sum_{k} \frac{\partial L}{\partial Z_{nk}} \frac{\partial Z_{nk}}{\partial W_{ij}} = \sum_{n=1}^{N} dZ_{nj}\, A^{\text{prev}}_{ni} = \sum_{n} (\mathbf{A}^{\mathsf{T}}_{\text{prev}})_{in} (d\mathbf{Z})_{nj} \quad \Longrightarrow \quad d\mathbf{W} = \mathbf{A}^{\mathsf{T}}_{\text{prev}} d\mathbf{Z}.$$

3. The bias. b_j appears in Z_{nj} for every n, with coefficient 1:

$$db_j = \sum_{n=1}^{N} dZ_{nj} \quad \Longrightarrow \quad d\mathbf{b} = \text{column sums of } d\mathbf{Z}.$$

4. The input (to pass back). A^{prev}_{ni} appears in Z_{nj} for every unit j of the same example n, with coefficient W_{ij}:

$$dA^{\text{prev}}_{ni} = \sum_{j} dZ_{nj}\, W_{ij} = \sum_{j} (d\mathbf{Z})_{nj} (\mathbf{W}^{\mathsf{T}})_{ji} \quad \Longrightarrow \quad d\mathbf{A}_{\text{prev}} = d\mathbf{Z}\, \mathbf{W}^{\mathsf{T}}.$$

Key equation: backpropagation through one dense layer, batch form

$$\begin{aligned} d\mathbf{Z} &= d\mathbf{A} \odot f'(\mathbf{Z}) && \text{(element-wise)} \\ d\mathbf{W} &= \mathbf{A}^{\mathsf{T}}_{\text{prev}}\, d\mathbf{Z} && (n_{\text{in}} \times N)(N \times n_{\text{out}}) \\ d\mathbf{b} &= \textstyle\sum_n d\mathbf{Z}_{n,:} && \text{(sum over the batch)} \\ d\mathbf{A}_{\text{prev}} &= d\mathbf{Z}\, \mathbf{W}^{\mathsf{T}} && (N \times n_{\text{out}})(n_{\text{out}} \times n_{\text{in}}) \end{aligned} \tag{10.2}$$

At the output layer, for softmax + cross-entropy or sigmoid + binary cross-entropy with the loss averaged over the batch, the first line is replaced by the remarkably simple

$$d\mathbf{Z}_{\text{out}} = \frac{1}{N}(\mathbf{P} - \mathbf{Y}), \tag{10.3}$$

where $\mathbf{P}$ holds the predicted probabilities and $\mathbf{Y}$ the one-hot (or $0/1$) targets.

Intuition: what the equations say

$d\mathbf{W} = \mathbf{A}_{\text{prev}}^{\mathsf{T}} d\mathbf{Z}$: a weight's gradient is (its input) $\times$ (the blame at its output), summed over examples. A weight connecting an inactive input (zero) to anything gets no gradient, which is fair because it did not influence the result. $d\mathbf{A}_{\text{prev}} = d\mathbf{Z}\,\mathbf{W}^{\mathsf{T}}$: blame flows backwards along the same wires as the forward signal, weighted by the same weights. The backward pass is the forward pass run in reverse through the transposed matrices.

10.3.1 The shape trick

You never need to memorize where the transposes go. Each gradient has the same shape as the quantity it belongs to, and there is usually only one way to multiply the available matrices to get that shape. For example, $d\mathbf{W}$ must be $n_{\text{in}} \times n_{\text{out}}$. We have $\mathbf{A}_{\text{prev}}$ ($N \times n_{\text{in}}$) and $d\mathbf{Z}$ ($N \times n_{\text{out}}$). The only product that gives $n_{\text{in}} \times n_{\text{out}}$ is $\mathbf{A}_{\text{prev}}^{\mathsf{T}} d\mathbf{Z}$, which contracts over the batch dimension N, exactly the sum over examples we derived. Likewise $d\mathbf{A}_{\text{prev}}$ must be $N \times n_{\text{in}}$, and the only option is $d\mathbf{Z}\,\mathbf{W}^{\mathsf{T}}$.

Pitfall: mean versus sum

If the loss is the mean over the batch, the factor $1/N$ enters once, at the output ($d\mathbf{Z}_{\text{out}} = (\mathbf{P} - \mathbf{Y})/N$), and then $d\mathbf{W}$ and $d\mathbf{b}$ automatically become averages over the batch. Do *not* divide again in each layer. If you use a summed loss instead, drop the $1/N$, and remember that the effective learning rate is then N times larger.

10.3.2 The output layer: why $\mathbf{P} - \mathbf{Y}$

Sigmoid + binary cross-entropy. For one example, $p = \sigma(z)$ and $L = -[y \ln p + (1-y)\ln(1-p)]$. From Chapter 7, $\sigma'(z) = p(1-p)$. Then

$$\frac{\mathrm{d}L}{\mathrm{d}p} = -\frac{y}{p} + \frac{1-y}{1-p} = \frac{-y(1-p) + (1-y)p}{p(1-p)} = \frac{p-y}{p(1-p)},$$
$$\frac{\mathrm{d}L}{\mathrm{d}z} = \frac{\mathrm{d}L}{\mathrm{d}p}\,\frac{\mathrm{d}p}{\mathrm{d}z} = \frac{p-y}{p(1-p)} \cdot p(1-p) = p - y.$$

The awkward $p(1-p)$ factors cancel exactly. That is why these two functions are always used together.

Softmax + cross-entropy. For one example with K classes, $p_k = e^{z_k} / \sum_m e^{z_m}$ and $L = -\sum_k y_k \ln p_k$ with $\sum_k y_k = 1$. The softmax Jacobian (derived in Chapter 8) is

$$\frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j), \qquad \delta_{kj} = \begin{cases} 1 & k = j \\ 0 & k \neq j. \end{cases}$$

Every p_k depends on every z_j, so the chain rule sums over k:

$$\frac{\partial L}{\partial z_j} = \sum_k \frac{\partial L}{\partial p_k}\frac{\partial p_k}{\partial z_j} = \sum_k \left(-\frac{y_k}{p_k}\right) p_k(\delta_{kj} - p_j) = -\sum_k y_k\delta_{kj} + p_j\sum_k y_k = -y_j + p_j.$$

In vector form, $\frac{\partial L}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}$, and averaging over a batch of N rows gives Eq. (10.3).

Worked example: softmax gradient

Logits $\mathbf{z} = (2.0,\ 1.0,\ 0.1)$, true class 1 (so $\mathbf{y} = (1, 0, 0)$). Softmax: $\mathbf{p} = (0.6590,\ 0.2424,\ 0.0986)$. Loss $-\ln 0.6590 = 0.4170$. Gradient $\mathbf{p} - \mathbf{y} = (-0.3410,\ 0.2424,\ 0.0986)$. Gradient descent *raises* the correct logit (negative gradient) and lowers the two wrong ones, each in proportion to how much probability it wrongly received.

10.4 The complete hand example

This is the most important section of the chapter. Take a pencil and do every step yourself. The network (Fig. 10.2) has 2 inputs, 2 ReLU hidden units and 1 sigmoid output, trained with binary cross-entropy on one example.

Input and target	$\mathbf{x} = [1,\ 2],\ y = 1$
Hidden weights (rows = inputs, columns = hidden units)	$\mathbf{W}_1 = \begin{bmatrix} 0.1 & 0.3 \\ 0.2 & -0.1 \end{bmatrix},\ \mathbf{b}_1 = [0,\ 0]$
Output weights	$\mathbf{w}_2 = [0.6,\ -0.5]^\mathsf{T},\ b_2 = 0.1$
Activations, loss, learning rate	ReLU hidden, sigmoid output, BCE, $\eta = 0.5$

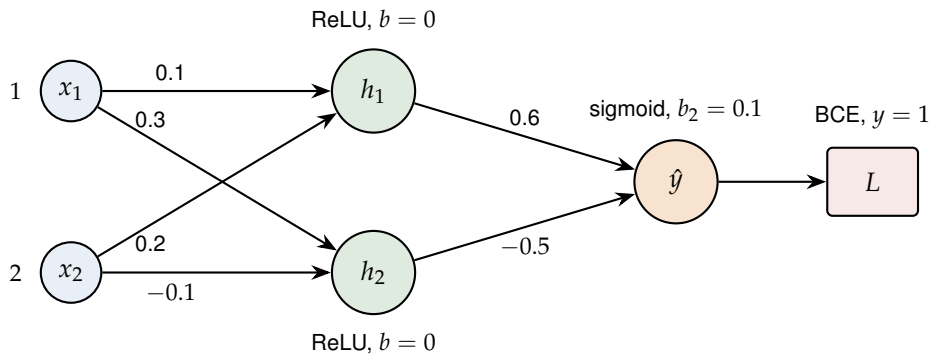


Figure 10.2: The 2–2–1 network of the hand example, with the initial weights on the arrows. The hidden units use ReLU, the output uses a sigmoid, and the loss is binary cross-entropy for the target $y = 1$.

10.4.1 Forward pass

Worked example: forward pass

Hidden pre-activations $\mathbf{z}_1 = \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1$ (row vector times matrix):

$$z_{1,1} = 1(0.1) + 2(0.2) + 0 = 0.5, \qquad z_{1,2} = 1(0.3) + 2(-0.1) + 0 = 0.1.$$

Hidden activations: both positive, so ReLU passes them unchanged: $\mathbf{h} = [0.5,\ 0.1]$.

Output pre-activation: $z_2 = \mathbf{h}\,\mathbf{w}_2 + b_2 = 0.5(0.6) + 0.1(-0.5) + 0.1 = 0.30 - 0.05 + 0.10 = 0.35$.
Prediction: $\hat{y} = \sigma(0.35) = 1/(1 + e^{-0.35}) = 1/(1 + 0.7047) = 0.5866$.
Loss: with $y = 1$, $L = -\ln \hat{y} = -\ln 0.5866 = 0.5334$.

10.4.2 Backward pass

Worked example: backward pass, one stage at a time

Output: sigmoid + BCE gives $\frac{\mathrm{d}L}{\mathrm{d}z_2} = \hat{y} - y = 0.5866 - 1 = -0.4134$. The sign is negative because increasing z_2 would *decrease* the loss: the network should be more confident in class 1.
Output weights: $z_2 = h_1 w_{2,1} + h_2 w_{2,2} + b_2$, so

$$\frac{\partial L}{\partial w_{2,1}} = h_1 \frac{\mathrm{d}L}{\mathrm{d}z_2} = 0.5(-0.4134) = -0.2067, \qquad \frac{\partial L}{\partial w_{2,2}} = 0.1(-0.4134) = -0.0413,$$
$$\frac{\partial L}{\partial b_2} = -0.4134.$$

In matrix form: $d\mathbf{w}_2 = \mathbf{h}^\top dz_2$, which is $\mathbf{A}_{\text{prev}}^\top d\mathbf{Z}$ with $N = 1$.
Blame on the hidden activations: $d\mathbf{h} = dz_2\,\mathbf{w}_2^\top$:

$$\frac{\partial L}{\partial h_1} = w_{2,1}\,dz_2 = 0.6(-0.4134) = -0.2480, \qquad \frac{\partial L}{\partial h_2} = -0.5(-0.4134) = +0.2067.$$

Increasing h_1 would help (negative gradient); increasing h_2 would hurt, because h_2 reaches the output through a negative weight.
Through the ReLU: $\mathrm{ReLU}'(z) = 1$ for $z > 0$. Both $z_{1,j} > 0$, so $d\mathbf{z}_1 = d\mathbf{h} \odot [1, 1] = [-0.2480,\ 0.2067]$.
Hidden weights: $d\mathbf{W}_1 = \mathbf{x}^\top d\mathbf{z}_1$ (an outer product):

$$d\mathbf{W}_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} -0.2480 & 0.2067 \end{bmatrix} = \begin{bmatrix} -0.2480 & 0.2067 \\ -0.4961 & 0.4134 \end{bmatrix}, \qquad d\mathbf{b}_1 = [-0.2480,\ 0.2067].$$

Row i of $d\mathbf{W}_1$ is input x_i times the hidden blame. The weights from $x_2 = 2$ get twice the gradient of those from $x_1 = 1$.

Table 10.1 collects every number. The script `code/ch10/hand_example.py` prints this table and also checks all nine gradients against central finite differences, $\big(L(\theta + \epsilon) - L(\theta - \epsilon)\big)/2\epsilon$ with $\epsilon = 10^{-6}$. The largest disagreement is 7×10^{-11}.

Figure 10.3 draws the same calculation as a computational graph, with forward values above the arrows and gradients below.

Table 10.1: Every quantity in the hand example (four decimals), as printed by `code/ch10/hand_example.py`.

Quantity	Formula	Value
$\mathbf{z}_1$	$\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1$	$[0.5,\ 0.1]$
$\mathbf{h}$	$\mathrm{ReLU}(\mathbf{z}_1)$	$[0.5,\ 0.1]$
z_2	$\mathbf{h}\mathbf{w}_2 + b_2$	0.35
$\hat{y}$	$\sigma(z_2)$	0.5866
L	$-\ln \hat{y}$	0.5334
dz_2	$\hat{y} - y$	-0.4134
$d\mathbf{w}_2$	$\mathbf{h}^\top dz_2$	$[-0.2067,\ -0.0413]$
db_2	dz_2	-0.4134
$d\mathbf{h}$	$dz_2\, \mathbf{w}_2^\top$	$[-0.2480,\ 0.2067]$
$d\mathbf{z}_1$	$d\mathbf{h} \odot \mathrm{ReLU}'(\mathbf{z}_1)$	$[-0.2480,\ 0.2067]$
$d\mathbf{W}_1$	$\mathbf{x}^\top d\mathbf{z}_1$	$\begin{bmatrix} -0.2480 & 0.2067 \\ -0.4961 & 0.4134 \end{bmatrix}$
$d\mathbf{b}_1$	$d\mathbf{z}_1$	$[-0.2480,\ 0.2067]$

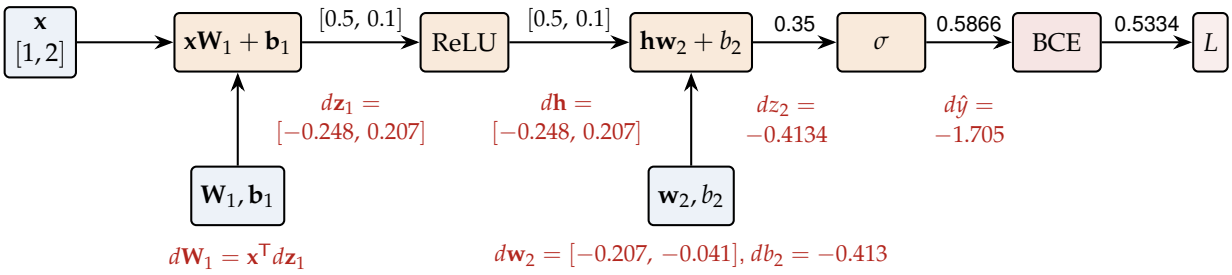


Figure 10.3: The hand example as a computational graph. Forward values (black, above the arrows) flow left to right; gradients (red, below) flow right to left. Each red number is the one to its right multiplied by a local derivative: $d\hat{y} = -1/\hat{y} = -1.705$; $dz_2 = d\hat{y} \cdot \hat{y}(1-\hat{y}) = -0.4134$; $d\mathbf{h} = dz_2 \mathbf{w}_2^\top$; $d\mathbf{z}_1 = d\mathbf{h} \odot \mathrm{ReLU}'$.

10.4.3 The update, and a hidden unit dies

Worked example: one gradient-descent step with $\eta = 0.5$

New parameters $\theta \leftarrow \theta - 0.5\, d\theta$:

$$\mathbf{W}_1 = \begin{bmatrix} 0.1 & 0.3 \\ 0.2 & -0.1 \end{bmatrix} - 0.5 \begin{bmatrix} -0.2480 & 0.2067 \\ -0.4961 & 0.4134 \end{bmatrix} = \begin{bmatrix} 0.2240 & 0.1967 \\ 0.4480 & -0.3067 \end{bmatrix},$$

$$\mathbf{b}_1 = [0.1240,\ -0.1033], \qquad \mathbf{w}_2 = [0.7033,\ -0.4793]^\mathsf{T}, \qquad b_2 = 0.1 + 0.2067 = 0.3067.$$

New forward pass: $z_{1,1} = 0.2240 + 2(0.4480) + 0.1240 = 1.2441$ and $z_{1,2} = 0.1967 + 2(-0.3067) - 0.1033 = -0.5201$. ReLU gives $\mathbf{h} = [1.2441,\ 0]$. Then $z_2 = 1.2441(0.7033) + 0 + 0.3067 = 1.1817$, $\hat{y} = \sigma(1.1817) = 0.7653$, and $L = -\ln 0.7653 = 0.2675$.
The loss halved, from 0.5334 to 0.268, in one step. But hidden unit 2 now has a negative pre-activation, $z_{1,2} = -0.5201$.

Look at what happens on the *next* backward pass. $\mathrm{ReLU}'(-0.5201) = 0$, so $dz_{1,2} = 0$, so every gradient for unit 2's incoming weights and bias is exactly zero. Nothing will ever move them again, and unit 2 stays off for this input forever. We trained on only one example, so unit 2 is now permanently **dead**. Further steps still reduce the loss (0.139, 0.080, 0.053, ...), but only through unit 1. In Fig. 10.4 (left) unit 2's pre-activation stays frozen at -0.5201.

Why did it die? The gradient correctly said that unit 2 was hurting (it feeds the output through a negative weight), so the update pushed its pre-activation down. With $\eta = 0.5$ the push was large enough to overshoot past zero, into the region where ReLU has no slope and no way back.

Definition: dying ReLU

A ReLU unit is **dead** when its pre-activation is negative for every input in the training set. Its output is then always zero and its gradient always zero, so gradient descent can never revive it. Dead units waste capacity.

Pitfall: preventing dead ReLUs

- Use a smaller learning rate, or warm-up (Chapter 9). Large early updates are the usual culprit, as the right panel of Fig. 10.4 shows.
- Use **Leaky ReLU** ($f(z) = \max(\alpha z, z)$ with $\alpha \approx 0.01$) or GELU/ELU (Chapter 7). Their slope is never exactly zero, so a unit can always recover.
- Use proper initialization (He initialization, Chapter 11) and normalization layers (Chapter 12) so that pre-activations start centred around zero.
- Monitor it: count the fraction of units whose activation is zero for an entire batch. A few percent is normal; a large fraction means trouble.

10.5 A whole network: the backward pass as an algorithm

For an ℓ-layer network, Eq. (10.2) is simply applied layer by layer from the top down. The forward pass must *cache* every $\mathbf{A}^{[l]}$ and $\mathbf{Z}^{[l]}$, because the backward pass needs them. That cache is why training uses much more memory than inference.

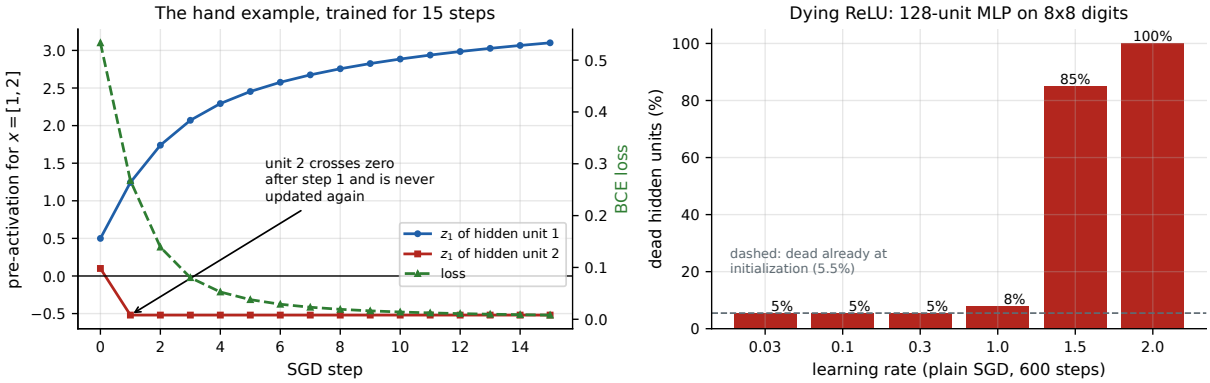


Figure 10.4: Left: the hand example trained for 15 steps. The loss (green, right axis) falls steadily; hidden unit 1's pre-activation (blue) grows, while unit 2's (red) jumps below zero after the first step and never moves again. Right: fraction of dead hidden units (units that output zero for all 1797 images) in a 64–128–10 ReLU network trained for 600 SGD steps on the 8×8 digits. The dashed line marks the 5.5% of units that are already dead at initialization (the digit pixels are all non-negative, so a unit whose weights are mostly negative never fires). Up to $\eta = 1$ training barely kills more units; at $\eta = 1.5$ it kills 85%, and at $\eta = 2$ all of them (training accuracy collapses to 10%, chance level). Script: `code/ch10/fig_hand_dynamics.py`.

Listing 10.1: Backpropagation for a dense ReLU network with a softmax output (NumPy, batch-first).

```
def forward(X, params):
    cache = [(None, X)]                          # (Z, A) per layer; A0 = X
    A = X
    for l, (W, b) in enumerate(params):
        Z = A @ W + b
        A = softmax(Z) if l == len(params) - 1 else np.maximum(Z, 0)
        cache.append((Z, A))
    return A, cache

def backward(P, Y, params, cache):
    N = Y.shape[0]
    dZ = (P - Y) / N                             # softmax + CE, mean over batch
    grads = [None] * len(params)
    for l in reversed(range(len(params))):
        W, b = params[l]
        A_prev = cache[l][1]
        grads[l] = (A_prev.T @ dZ, dZ.sum(axis=0))    # dW, db
        if l > 0:
            dA = dZ @ W.T                         # pass blame down
            dZ = dA * (cache[l][0] > 0)           # through ReLU of layer l
    return grads
```

Every deep learning framework does exactly this, generalized to arbitrary graphs: convolutions, attention, normalization and so on. Each operation only needs to know how to compute its own local backward step. Chapter 13 builds a complete NumPy network on this code and verifies it with gradient checking.

10.6 Forward mode versus reverse mode

The chain rule for a chain of operations $\mathbf{x} \to \mathbf{u} \to \mathbf{v} \to L$ is a product of Jacobian matrices:

$$\frac{\partial L}{\partial \mathbf{x}} = \underbrace{\frac{\partial L}{\partial \mathbf{v}}}_{1\times n_v} \underbrace{\frac{\partial \mathbf{v}}{\partial \mathbf{u}}}_{n_v\times n_u} \underbrace{\frac{\partial \mathbf{u}}{\partial \mathbf{x}}}_{n_u\times n_x}.$$

Matrix multiplication is associative, so the product can be evaluated in either order.

- **Forward mode** goes from right to left: start from the input side with a direction $\dot{\mathbf{x}}$ and push "how much does every intermediate change if $\mathbf{x}$ moves along $\dot{\mathbf{x}}$" forward through the graph, alongside the values. Each sweep gives the derivative of all outputs with respect to *one* input direction (a Jacobian-vector product, JVP). To get the gradient with respect to P parameters you need P sweeps.
- **Reverse mode** goes from left to right: start from the output with $\frac{\partial L}{\partial L} = 1$ and push the row vector of blame backwards. Each sweep gives the derivative of *one* scalar output with respect to all inputs at once (a vector-Jacobian product, VJP). Backpropagation is reverse mode.

Worked example: forward mode on $f = (a + b)\, b$

To get $\frac{\partial f}{\partial a}$, seed $\dot{a} = 1$, $\dot{b} = 0$ and propagate derivatives forward alongside the values: $\dot{c} = \dot{a} + \dot{b} = 1$, $\dot{f} = \dot{c}\, b + c\, \dot{b} = 1(3) + 5(0) = 3$. To get $\frac{\partial f}{\partial b}$ we need a *second* sweep with $\dot{a} = 0$, $\dot{b} = 1$: $\dot{c} = 1$, $\dot{f} = 1(3) + 5(1) = 8$. Two inputs need two forward sweeps, whereas the reverse sweep of Fig. 10.1 produced both numbers at once.

A loss is always one number, and a network has millions of parameters, so reverse mode wins by a factor of roughly P. (Forward mode wins in the opposite situation, when there are few inputs and many outputs, as in sensitivity analysis of a simulator with a handful of knobs.) The price of reverse mode is memory, since all intermediate values must be stored for the backward sweep.

10.6.1 How expensive is backprop?

For a dense layer, the forward pass is one matrix product, $\mathbf{A}_{\text{prev}}\mathbf{W}$, costing about $2Nn_{\text{in}}n_{\text{out}}$ floating-point operations. The backward pass needs *two* products of the same size, $\mathbf{A}_{\text{prev}}^{\mathsf{T}} d\mathbf{Z}$ and $d\mathbf{Z}\,\mathbf{W}^{\mathsf{T}}$. So the backward pass costs about twice the forward pass, and a full training step (forward + backward) about three times a forward pass. Element-wise operations (activations) are cheap in comparison. This rule of thumb, "backward $\approx 2\times$ forward", holds for any graph of matrix products. Reverse-mode differentiation never costs more than a small constant multiple of evaluating the function, a classical result of automatic differentiation theory. Figure 10.5 (left) confirms it: the measured ratio is 2.75. For a 5-layer net of width w the forward pass costs $\propto 4w^2$ and the backward pass $\propto 2 \times 4w^2 - w^2$, the $-w^2$ being the skipped input gradient, giving $(4 + 7)/4 = 2.75$.

10.7 An autograd engine in 59 lines

Frameworks like PyTorch do not have the four layer equations hard-coded. They record a computational graph while your code runs and then apply the chain rule node by node. We can

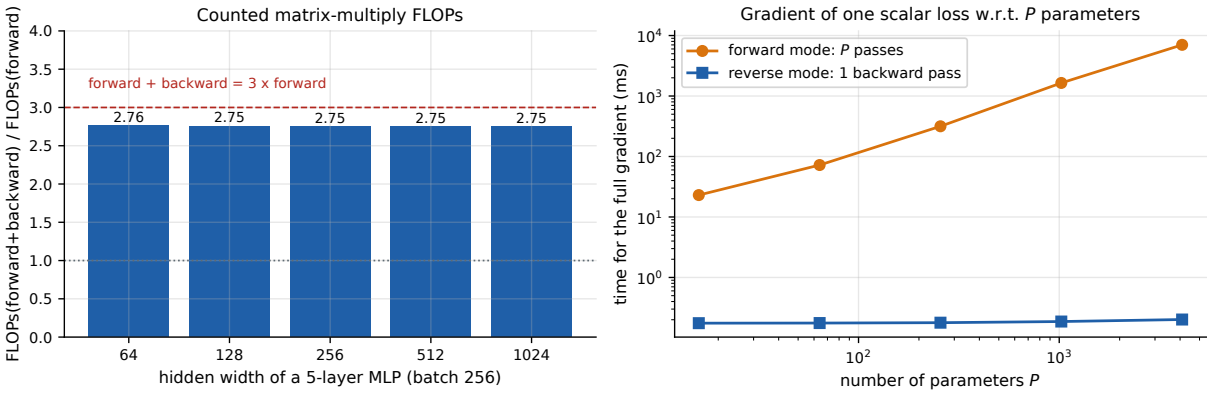


Figure 10.5: Left: matrix-multiply FLOPs of the backward pass versus the forward pass for 5-layer MLPs of increasing width, counted with PyTorch's `FlopCounterMode`. Forward + backward costs 2.75× the forward pass at every width. The theoretical value is 3×, and it comes out slightly lower here because the first layer does not need the gradient with respect to the input data. Right: wall-clock time to obtain the full gradient of a scalar loss with respect to P parameters. Forward mode (PyTorch's `forward_ad`, one JVP per parameter) grows linearly with P; reverse mode (one `backward()`) stays flat. At $P = 4096$ the gap is more than four orders of magnitude. Script: `code/ch10/fig_cost_modes.py`.

build a complete working version for scalars in a few dozen lines. The design follows Andrej Karpathy's educational *micrograd*.

Each `Value` stores a number (`data`), the gradient of the final output with respect to it (`grad`), the values it was computed from (`_parents`), and a small function `_backward` that knows the local derivatives of the operation that produced it. Calling `L.backward()` sorts the graph so that every node comes after its parents (a *topological sort*), sets $\frac{\partial L}{\partial L} = 1$, and calls each node's `_backward` in reverse order. Each node therefore receives its complete gradient before passing blame on to its own parents.

Listing 10.2: A scalar reverse-mode autograd engine (`code/ch10/micrograd.py`).

```
"""Chapter 10: a tiny scalar autograd engine (reverse-mode automatic
    differentiation)."""
import math

class Value:
    def __init__(self, data, parents=(), op=''):
        self.data, self.grad = float(data), 0.0
        self._parents, self._op = parents, op
        self._backward = lambda: None          # how to push my grad to my parents

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), '+')
        def _backward():                       # d(a+b)/da = 1, d(a+b)/db = 1
            self.grad += out.grad
            other.grad += out.grad
        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), '*')
        def _backward():                       # d(ab)/da = b, d(ab)/db = a
```

```
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

    def _unary(self, f, dfdx, op):              # helper for one-input functions
        out = Value(f(self.data), (self,), op)
        def _backward():
            self.grad += dfdx(self.data, out.data) * out.grad
        out._backward = _backward
        return out

    def exp(self):  return self._unary(math.exp, lambda x, y: y, 'exp')
    def log(self):  return self._unary(math.log, lambda x, y: 1 / x, 'log')
    def tanh(self): return self._unary(math.tanh, lambda x, y: 1 - y * y, 'tanh')
    def relu(self): return self._unary(lambda x: max(x, 0.0), lambda x, y: float(x
     > 0), 'relu')
    def inv(self):  return self._unary(lambda x: 1 / x, lambda x, y: -y * y, 'inv'
    )

    __radd__ = __add__
    __rmul__ = __mul__
    def __neg__(self):        return self * -1
    def __sub__(self, other): return self + (-other)
    def __rsub__(self, other): return (-self) + other
    def __truediv__(self, other): return self * (other if isinstance(other, Value)
     else Value(other)).inv()
    def __rtruediv__(self, other): return self.inv() * other

    def backward(self):
        order, seen = [], set()                 # topological sort: parents before
    children
        def visit(v):
            if v not in seen:
                seen.add(v)
                for p in v._parents: visit(p)
                order.append(v)
        visit(self)
        self.grad = 1.0                         # dL/dL = 1
        for v in reversed(order):               # walk from the output back to the
    inputs
            v._backward()
```

Notice the += in every `_backward`: that is the "add gradients from every consumer" rule of Fig. 10.1. The `_unary` helper handles one-input functions given their derivative; for tanh, the derivative is written in terms of the output y as $1 - y^2$. Now we rebuild the hand example from these atoms. Even the sigmoid is assembled from `exp`, addition and division, so the engine has never heard of a sigmoid or a neural network:

Listing 10.3: Reproducing the hand example with the engine (`code/ch10/run_micrograd.py`).

```
from micrograd import Value
x = [Value(1.0), Value(2.0)]; y = 1.0
W1 = [[Value(0.1), Value(0.3)], [Value(0.2), Value(-0.1)]]   # W1[i][j]: input i
    -> hidden j
b1 = [Value(0.0), Value(0.0)]
W2 = [Value(0.6), Value(-0.5)]; b2 = Value(0.1)

z1 = [x[0] * W1[0][j] + x[1] * W1[1][j] + b1[j] for j in range(2)]
```

```
h = [z.relu() for z in z1]
z2 = h[0] * W2[0] + h[1] * W2[1] + b2
yhat = 1 / (1 + (-z2).exp())                          # sigmoid built from exp and
    division
L = -(y * yhat.log() + (1 - y) * (1 - yhat).log())   # binary cross-entropy
L.backward()

print(f"yhat = {yhat.data:.4f}   L = {L.data:.4f}")
print("dW2 =", [round(w.grad, 4) for w in W2], "  db2 =", round(b2.grad, 4))
print("dW1 =", [[round(W1[i][j].grad, 4) for j in range(2)] for i in range(2)])
print("db1 =", [round(b.grad, 4) for b in b1])
```

Its output matches Table 10.1 digit for digit:

```
yhat = 0.5866   L = 0.5334
dW2 = [-0.2067, -0.0413]   db2 = -0.4134
dW1 = [[-0.248, 0.2067], [-0.4961, 0.4134]]
db1 = [-0.248, 0.2067]
```

PyTorch's `autograd` works on exactly the same principle, but on tensors instead of scalars. Every tensor operation records a node with a backward function, and `loss.backward()` runs the reverse sweep. Each tensor operation's backward function implements its own version of Eq. (10.2), so it is far faster than our scalar engine.

Wireless connection: automatic differentiation for DSP blocks

Because autograd only needs local derivatives, any signal-processing block made of differentiable operations can sit inside a trained model and have its parameters learned: FIR filters, FFTs, phase rotations $e^{j\phi}$, matched filters, a digital predistorter's polynomial, even an OFDM modulator. This is *differentiable signal processing*: keep the structure that radio engineering has proven (OFDM, matched filtering), and let gradient descent tune what is hard to model (non-linear PA coefficients, an equalizer for unknown hardware). PyTorch supports complex tensors and differentiates them with the same Wirtinger convention mentioned in the LMS pitfall of Chapter 9.

10.8 Wireless: backpropagating through the channel

In a classical radio link, the transmitter and receiver are designed separately, block by block. The deep learning view (O'Shea and Hoydis, 2017; Chapter 21) treats the entire link as one big computational graph: a transmitter neural network maps a message to channel symbols, the channel corrupts them, and a receiver network outputs a probability for each possible message. Training the whole chain end to end with cross-entropy requires the gradient of the loss with respect to the *transmitter's* weights, and that gradient has to pass backwards *through the channel*.

For an AWGN channel, $\mathbf{y} = \mathbf{x} + \mathbf{n}$, where $\mathbf{n}$ is a fresh random sample on every forward pass. In the backward pass the sampled $\mathbf{n}$ is treated as a constant that happened to be added. The Jacobian of the addition is the identity, $\frac{\partial y_i}{\partial x_j} = \delta_{ij}$, so

$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}}\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}}\mathbf{I} = \frac{\partial L}{\partial \mathbf{y}}. \tag{10.4}$$

The gradient passes *straight through* the noise, exactly like the add node in Fig. 10.1. Two other channel models are just as easy:

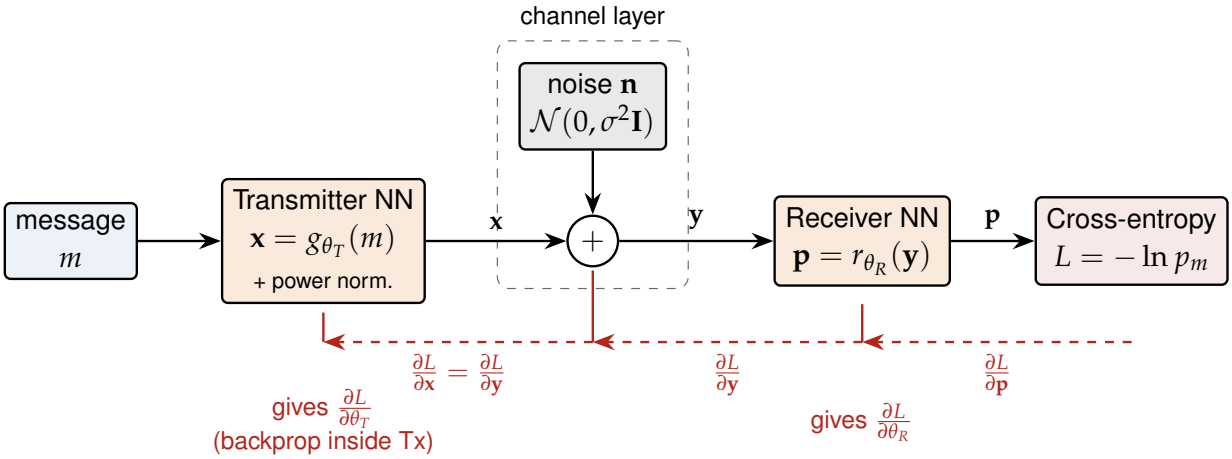


Figure 10.6: An end-to-end learned link as one computational graph. Forward (black): message → transmitter network → symbols $\mathbf{x}$ → additive noise → receiver network → loss. Backward (red, dashed): the gradient reaches the channel output $\mathbf{y}$, passes through the addition unchanged ($\partial\mathbf{y}/\partial\mathbf{x} = \mathbf{I}$), and continues into the transmitter. The noise is just a number that was added; it receives a gradient but has no parameters to update.

- **Flat fading**, $\mathbf{y} = h\mathbf{x} + \mathbf{n}$ with a known real gain h: $\frac{\partial L}{\partial \mathbf{x}} = h\,\frac{\partial L}{\partial \mathbf{y}}$. (For complex baseband, h^* appears via the Wirtinger convention.)
- **Multipath**, $\mathbf{y} = \mathbf{h} * \mathbf{x} + \mathbf{n}$ (convolution, a linear map $\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}$): $\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}}\,\mathbf{H}$, in the row-vector convention. This is the $d\mathbf{A}_{\text{prev}} = d\mathbf{Z}\,\mathbf{W}^{\mathsf{T}}$ rule with the transposes arranged to match, and in signal-processing terms it is correlation with the channel response, a matched filter run backwards.

Averaged over many noise samples, these gradients point in the direction that reduces the *expected* loss over the channel's randomness. This is the same idea as the mini-batch gradient of Chapter 9 and as the "reparameterization trick" we meet with variational autoencoders. The power-normalization step at the end of the transmitter (scaling so that the average symbol energy is 1) is also differentiable, so backpropagation automatically learns constellations that respect the power constraint.

Wireless connection: when the channel is not differentiable

Equation (10.4) assumes we have a differentiable *model* of the channel inside the computer. Over the air we do not: the real channel, the DAC/ADC quantization and the power amplifier sit between transmitter and receiver, and we cannot run `backward()` through the atmosphere. Part V shows the remedies: train on a channel model and fine-tune the receiver on real data; learn a differentiable surrogate of the channel (for example with a GAN); or estimate the transmitter gradient without a channel model using reinforcement-learning-style perturbations (model-free end-to-end learning). Hard quantizers have zero derivative almost everywhere, and are commonly handled with the *straight-through estimator*: use the quantizer in the forward pass, but pretend its derivative is 1 in the backward pass. These techniques all exist to bring back the simple identity in Eq. (10.4).

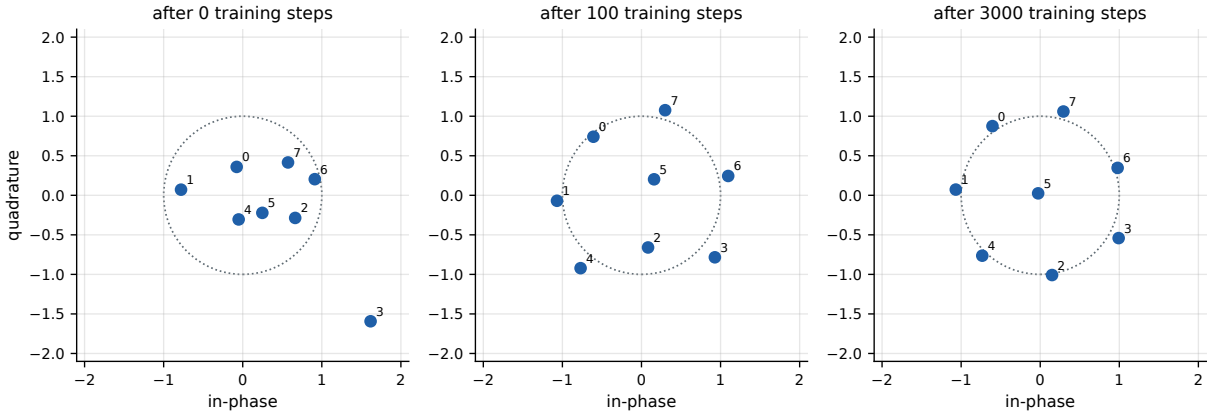


Figure 10.7: Our small-scale demonstration of end-to-end training through an AWGN channel layer: $M = 8$ messages, two real dimensions (one complex symbol), average energy normalized to 1 (dotted circle), SNR $= 12$ dB, a 2–32–8 receiver MLP, Adam with learning rate 0.01, 3000 batches of 256. The transmitter starts with random points (left) and, driven only by gradients that passed backwards through $\mathbf{y} = \mathbf{x} + \mathbf{n}$, spreads them apart (middle). It ends with one point at the centre and seven on a ring (right), a well-known good 8-point packing. Its minimum distance is 0.888 versus 0.765 for 8-PSK, and its measured symbol error rate is 1.1% versus 3.1% for 8-PSK with an optimal detector at the same SNR (200 000 symbols each). Script: `code/ch10/e2e_channel.py`.

Chapter recap

- Backpropagation is the chain rule applied from the loss backwards. Each node multiplies the incoming gradient by its local derivative; gradients arriving at a node from several consumers are added.
- Dense layer, batch-first: $d\mathbf{Z} = d\mathbf{A} \odot f'(\mathbf{Z})$, $d\mathbf{W} = \mathbf{A}_{\text{prev}}^{\mathsf{T}} d\mathbf{Z}$, $d\mathbf{b} =$ column sums, $d\mathbf{A}_{\text{prev}} = d\mathbf{Z}\,\mathbf{W}^{\mathsf{T}}$. At a softmax/sigmoid + cross-entropy output, $d\mathbf{Z} = (\mathbf{P} - \mathbf{Y})/N$. Shapes tell you where the transposes go.
- Hand example: $\hat{y} = 0.5866$, $L = 0.5334$, $dz_2 = -0.4134$, $d\mathbf{w}_2 = [-0.2067, -0.0413]$, $d\mathbf{h} = [-0.2480, 0.2067]$, $d\mathbf{W}_1 = \mathbf{x}^{\mathsf{T}} d\mathbf{z}_1$. After one step with $\eta = 0.5$ the loss is 0.268, and hidden unit 2 dies ($z = -0.5201$, zero gradient forever).
- Reverse mode computes the gradient of one scalar with respect to all P parameters in one sweep; forward mode needs P sweeps. The backward pass costs about $2\times$ the forward pass, at the price of storing activations.
- An autograd engine is just values plus local backward functions plus a topological sort; 59 lines reproduce the hand example.
- A differentiable channel layer passes gradients through: for $\mathbf{y} = \mathbf{x} + \mathbf{n}$, $\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}}$. This is the foundation of end-to-end learned transceivers.

Exercises

1. For $f(a, b, c) = (a\,b + c)^2$ at $a = 1$, $b = -2$, $c = 3$, draw the graph, do the forward pass, and compute $\frac{\partial f}{\partial a}$, $\frac{\partial f}{\partial b}$, $\frac{\partial f}{\partial c}$ by backpropagation.
2. A dense layer has $\mathbf{A}_{\text{prev}} \in \mathbb{R}^{32\times 100}$ and 50 units. Give the shapes of $\mathbf{W}$, $\mathbf{b}$, $\mathbf{Z}$, $d\mathbf{W}$, $d\mathbf{b}$ and $d\mathbf{A}_{\text{prev}}$, and write each gradient as a matrix expression.

3. Redo the hand example with target $y = 0$ (everything else unchanged). Compute L, dz_2, $d\mathbf{w}_2$, $d\mathbf{h}$ and $d\mathbf{W}_1$.
4. In the hand example, suppose the hidden activation were tanh instead of ReLU. Compute $\mathbf{h}$, $\hat{y}$ and $d\mathbf{z}_1$ (use $\tanh'(z) = 1 - \tanh^2 z$).
5. Softmax with logits $(1, 1, 1)$ and true class 3: give $\mathbf{p}$, L and $\frac{\partial L}{\partial \mathbf{z}}$. Explain why the gradient has this pattern.
6. Show that for a batch of N identical copies of one example with a mean loss, $d\mathbf{W}$ equals the single-example gradient. What changes with a summed loss?
7. (Conceptual) Why does reverse mode need more memory than forward mode? Name one technique that trades extra computation for less memory.
8. For the flat-fading channel $y = hx + n$ with $h = 0.5$ and $\frac{\partial L}{\partial y} = -2$, compute $\frac{\partial L}{\partial x}$ and $\frac{\partial L}{\partial h}$ (as if h were a learnable parameter). What does $\frac{\partial L}{\partial h}$ depend on?

Answers

1. Let $u = ab = -2$, $v = u + c = 1$, $f = v^2 = 1$. Backward: $\frac{\partial f}{\partial v} = 2v = 2$; $\frac{\partial f}{\partial c} = 2 \cdot 1 = 2$; $\frac{\partial f}{\partial u} = 2$; $\frac{\partial f}{\partial a} = 2b = -4$; $\frac{\partial f}{\partial b} = 2a = 2$.
2. $\mathbf{W}$: 100×50; $\mathbf{b}$: 1×50; $\mathbf{Z}$: 32×50; $d\mathbf{W} = \mathbf{A}_{\text{prev}}^{\mathsf{T}} d\mathbf{Z}$: 100×50; $d\mathbf{b} = \sum_n d\mathbf{Z}_{n,:}$: 1×50; $d\mathbf{A}_{\text{prev}} = d\mathbf{Z}\,\mathbf{W}^{\mathsf{T}}$: 32×100.
3. The forward pass is unchanged ($\hat{y} = 0.5866$), so $L = -\ln(1 - 0.5866) = 0.8834$. $dz_2 = \hat{y} - 0 = 0.5866$. $d\mathbf{w}_2 = [0.5, 0.1] \times 0.5866 = [0.2933,\ 0.0587]$. $d\mathbf{h} = 0.5866\,[0.6, -0.5] = [0.3520,\ -0.2933]$. ReLU passes both, so $d\mathbf{W}_1 = \begin{bmatrix} 0.3520 & -0.2933 \\ 0.7039 & -0.5866 \end{bmatrix}$. Every sign flips relative to $y = 1$, as it should.
4. $\mathbf{h} = \tanh([0.5, 0.1]) = [0.4621,\ 0.0997]$. $z_2 = 0.4621(0.6) + 0.0997(-0.5) + 0.1 = 0.3274$, $\hat{y} = \sigma(0.3274) = 0.5811$, $dz_2 = -0.4189$. $d\mathbf{h} = -0.4189[0.6, -0.5] = [-0.2513,\ 0.2094]$. $\tanh' = [1 - 0.4621^2,\ 1 - 0.0997^2] = [0.7864,\ 0.9901]$. So $d\mathbf{z}_1 = [-0.1976,\ 0.2074]$.
5. $\mathbf{p} = (1/3, 1/3, 1/3)$, $L = \ln 3 = 1.0986$, $\frac{\partial L}{\partial \mathbf{z}} = (1/3,\ 1/3,\ -2/3)$. The gradient sums to zero (softmax is unchanged by adding a constant to all logits), pushes the true logit up and the others down equally.
6. With identical rows, $d\mathbf{Z} = \frac{1}{N}(\mathbf{P} - \mathbf{Y})$ has N identical rows $\frac{1}{N}(\mathbf{p} - \mathbf{y})$, and $d\mathbf{W} = \sum_n \mathbf{a}^{\mathsf{T}} \frac{1}{N}(\mathbf{p} - \mathbf{y}) = \mathbf{a}^{\mathsf{T}}(\mathbf{p} - \mathbf{y})$, the single-example gradient. With a summed loss it is N times larger.
7. Reverse mode must store every intermediate activation from the forward pass until the backward sweep reaches it, whereas forward mode can discard values as soon as they have been used. *Gradient checkpointing* (rematerialization) stores only some activations and recomputes the others during the backward pass.
8. $\frac{\partial L}{\partial x} = h\,\frac{\partial L}{\partial y} = 0.5(-2) = -1$. $\frac{\partial L}{\partial h} = x\,\frac{\partial L}{\partial y} = -2x$: it depends on the transmitted symbol, just as a weight's gradient depends on its input ($d\mathbf{W} = \mathbf{A}_{\text{prev}}^{\mathsf{T}} d\mathbf{Z}$).

Self-test quiz

1. A value v feeds three later operations. How do you get $\frac{\partial L}{\partial v}$?
   Add the three gradient contributions coming back from the three consumers.
2. What is $d\mathbf{Z}$ at a softmax + cross-entropy output with a mean loss over N examples?
   $(\mathbf{P} - \mathbf{Y})/N$.
3. Which matrix product gives $d\mathbf{W}$, and why does it sum over the batch?
   $\mathbf{A}_{prev}^{\top} d\mathbf{Z}$; *the weights are shared by all examples, so their gradient collects contributions from every row.*
4. Why can a dead ReLU unit never recover under gradient descent?
   Its pre-activation is negative for all inputs, so $\mathrm{ReLU}' = 0$ *and all gradients of its incoming weights and bias are exactly zero.*
5. Why do frameworks use reverse mode rather than forward mode for training?
   One scalar loss and millions of parameters: reverse mode gives all partial derivatives in one backward sweep (about $2\times$ *forward cost); forward mode would need one sweep per parameter.*
6. For the AWGN channel $\mathbf{y} = \mathbf{x} + \mathbf{n}$, what is $\frac{\partial L}{\partial \mathbf{x}}$?
   $\frac{\partial L}{\partial \mathbf{y}}$: *addition has identity Jacobian, so the gradient passes straight through the noise.*

Further reading

- D. E. Rumelhart, G. E. Hinton, R. J. Williams, "Learning representations by back-propagating errors," *Nature*, 323, 533–536, 1986.
- A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind, "Automatic differentiation in machine learning: a survey," *Journal of Machine Learning Research*, 18(153), 2018.
- A. Griewank, A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed., SIAM, 2008.
- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016, Section 6.5 (back-propagation and other differentiation algorithms).
- A. Karpathy, *micrograd* (a tiny scalar autograd engine, open source on GitHub) and the accompanying lecture "The spelled-out intro to neural networks and backpropagation."
- T. O'Shea, J. Hoydis, "An introduction to deep learning for the physical layer," *IEEE Transactions on Cognitive Communications and Networking*, 3(4), 2017.

Chapter 11

Making Training Work: Gradients, Initialization and Optimizers

You now know how to compute gradients (Chapter 10) and how to use them to step downhill (Chapter 9). In principle that is enough to train any network. In practice, a deep network trained with naive choices often does not train at all. The loss sits flat, or explodes to `nan`, or creeps down so slowly that you give up. For about twenty years (roughly 1986–2006) this was the main reason deep networks were considered impractical.

This chapter is about the handful of ideas that fixed it. First we look at **vanishing and exploding gradients**: why the gradient reaching the early layers of a deep network is a long product of factors, and why such products shrink or blow up exponentially. Then **initialization**: why all-zero weights fail, and how a short variance calculation gives the Xavier and He rules that keep signals the right size through any depth. Then **optimizers**: momentum, Nesterov, AdaGrad, RMSProp, Adam and AdamW, each with its equations, its intuition and a worked number. Finally, **gradient clipping**, **residual connections** and **warm-up**. Two wireless boxes connect these to RNN channel predictors and to training on fixed-point DSP hardware.

11.1 Vanishing and exploding gradients

Intuition: whispering down a long line

Think of the children's game in which a message is whispered down a line of people. Suppose each person passes on the message at 25% of the loudness they heard it. After 10 people the message is $0.25^{10} \approx 10^{-6}$ as loud as at the start, and nobody at the far end hears anything. If instead each person shouts 10% louder than they heard, after 50 people the message is $1.1^{50} \approx 117$ times louder, which is deafening and distorted. Only if each person passes the message on at almost exactly the same volume does it survive a long line.
In a deep network, the "message" travelling backwards is the gradient, and each layer multiplies it by a factor. Backpropagation through ℓ layers multiplies ℓ such factors together, so unless each factor is close to one, the gradient at the early layers either vanishes or explodes.

11.1.1 The product of Jacobians

Write a deep network as a chain $\mathbf{h}^{[l]} = f\big(\mathbf{h}^{[l-1]}\mathbf{W}^{[l]} + \mathbf{b}^{[l]}\big)$ for $l = 1, \dots, \ell$ (one example, row vectors). By the chain rule, the gradient of the loss with respect to an early hidden layer is

$$\frac{\partial L}{\partial \mathbf{h}^{[k]}} = \frac{\partial L}{\partial \mathbf{h}^{[\ell]}} \frac{\partial \mathbf{h}^{[\ell]}}{\partial \mathbf{h}^{[\ell-1]}} \frac{\partial \mathbf{h}^{[\ell-1]}}{\partial \mathbf{h}^{[\ell-2]}} \cdots \frac{\partial \mathbf{h}^{[k+1]}}{\partial \mathbf{h}^{[k]}}. \tag{11.1}$$

Each factor is a *layer Jacobian*. With $\mathbf{z}^{[l]} = \mathbf{h}^{[l-1]}\mathbf{W}^{[l]} + \mathbf{b}^{[l]}$, it is the matrix of partial derivatives $\partial h_j^{[l]} / \partial h_i^{[l-1]} = W_{ij}^{[l]} f'(z_j^{[l]})$. In the row-vector convention, multiplying a gradient by it means

$$\frac{\partial L}{\partial \mathbf{h}^{[l-1]}} = \Big(\frac{\partial L}{\partial \mathbf{h}^{[l]}} \odot f'(\mathbf{z}^{[l]})\Big)\mathbf{W}^{[l]\mathsf{T}},$$

which is exactly the $d\mathbf{A}_{\text{prev}} = (d\mathbf{A} \odot f'(\mathbf{Z}))\mathbf{W}^{\mathsf{T}}$ rule of Chapter 10. Figure 11.1 pictures the repeated multiplication.

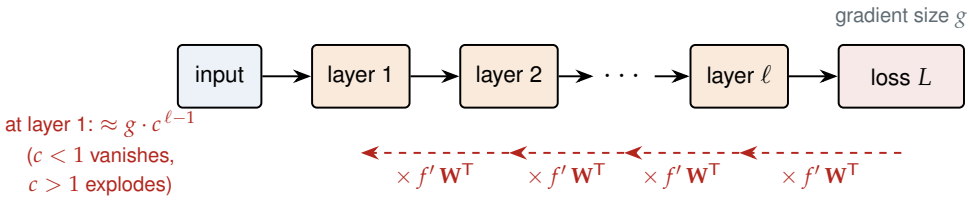


Figure 11.1: Why depth makes gradients fragile. The forward pass (black) runs left to right. On the way back (red), each layer multiplies the gradient by its activation derivative and its transposed weight matrix. If each multiplication scales the gradient by a typical factor c, the gradient arriving at layer 1 is about $c^{\ell-1}$ times the one at the top: $0.25^{10} \approx 10^{-6}$ for sigmoid layers, $1.1^{50} \approx 117$ for slightly too-large weights.

Each step backwards multiplies by f' (element-wise) and by $\mathbf{W}^{\mathsf{T}}$. The size of the gradient after $\ell - k$ steps depends on how much each of these factors stretches or shrinks vectors.

The activation factor. The sigmoid's derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ is at most 0.25, reached at $z = 0$ (Chapter 7). So even in the best case, passing back through ten sigmoid layers multiplies the gradient by at most

$$0.25^{10} \approx 9.5 \times 10^{-7} \approx 10^{-6}$$

from the activations alone, and by $0.25^{20} \approx 10^{-12}$ through twenty. Unless the weights compensate by being large (which brings other problems), the early layers receive essentially no gradient and do not learn. ReLU, by contrast, has derivative exactly 1 for every active unit. This is the main reason ReLU replaced sigmoid in hidden layers.

The weight factor: an eigenvalue view. Suppose, for simplicity, that the activation is linear and every layer has the same weight matrix $\mathbf{W}$. Then Eq. (11.1) becomes $\frac{\partial L}{\partial \mathbf{h}^{[k]}} = \frac{\partial L}{\partial \mathbf{h}^{[\ell]}} \left(\mathbf{W}^{\mathsf{T}}\right)^{\ell-k}$. If $\mathbf{W}^{\mathsf{T}}$ has eigenvectors $\mathbf{u}_i$ with eigenvalues λ_i (Chapter 2), any gradient can be written as a combination $\sum_i c_i \mathbf{u}_i$, and repeated multiplication gives $\sum_i c_i \lambda_i^{\ell-k} \mathbf{u}_i$. The components with $|\lambda_i| < 1$ decay exponentially and those with $|\lambda_i| > 1$ grow exponentially. Only a matrix whose eigenvalues all have magnitude close to 1 transmits gradients over many layers. The largest $|\lambda_i|$ is called the **spectral radius** ρ. After 50 layers, $\rho = 0.9$ gives $0.9^{50} \approx 0.005$ and $\rho = 1.1$ gives $1.1^{50} \approx 117$. Real networks have different matrices in each layer and nonlinear activations, but the lesson is the same: *the gradient is a product, and products of many numbers away from one either vanish or explode.*

Definition: vanishing and exploding gradients

Vanishing gradients: the gradient norm decreases exponentially as it travels back through layers (or time steps), so early layers learn extremely slowly or not at all. **Exploding gradients**: the gradient norm grows exponentially, producing huge, unstable updates and often `inf/nan`.

11.1.2 An experiment: sigmoid versus ReLU

Figure 11.2 shows the gradient norm in each layer of a 10-layer, 64-unit MLP at initialization, evaluated on a batch of the bundled 8×8 digits. With sigmoid activations (and the Xavier initialization described below) the gradient reaching layer 1 is about 10^{-5} times that of layer 10:

the measured ratio is 8.6×10^{-6}. Each layer back loses a factor of about 4, close to the 0.25 bound. With ReLU and He initialization, every layer receives a gradient of similar size (ratio 0.69).

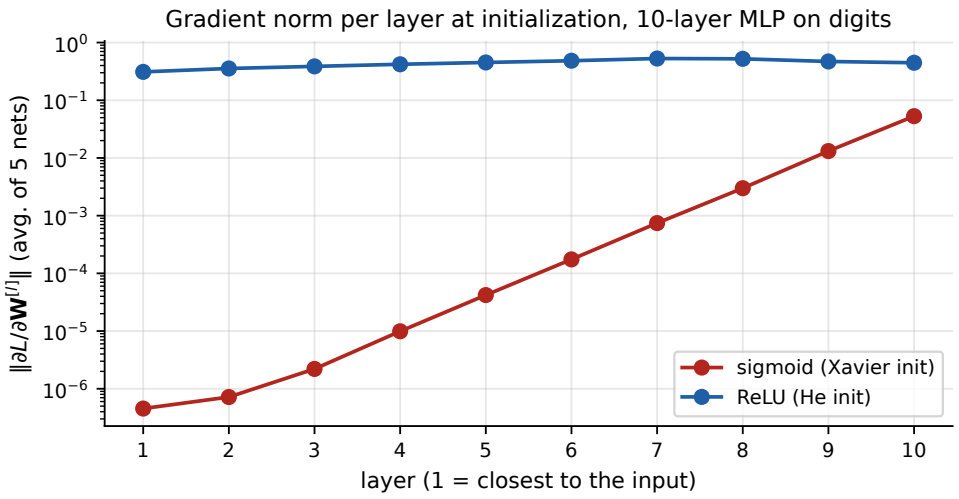


Figure 11.2: Norm of the weight gradient in each layer of a 10-layer MLP at initialization (average of 5 random networks, batch of 256 digit images). With sigmoid hidden units the gradient shrinks by roughly $4\times$ per layer going backwards, so layer 1 receives about 10^{-5} of what layer 10 receives: the early layers are effectively frozen. With ReLU and He initialization the gradient is nearly the same size in every layer. Script: `code/ch11/fig_gradnorms_init.py`.

11.2 Initialization

11.2.1 Why not start at zero? The symmetry problem

The most obvious initialization, all weights zero, does not work. Consider a hidden layer whose units all have identical incoming weights (zero, or any other shared value). Then:

1. In the forward pass, every hidden unit computes the same pre-activation and the same output.
2. In the backward pass, each unit's gradient is (its outgoing weight) $\times$ (the blame from above) $\times f'$. If the outgoing weights are also identical, every unit receives the same gradient.
3. So the update changes every unit's weights by the same amount, and they remain identical.

By induction, the units stay clones forever, and a layer of 512 units behaves like a layer of one unit. (With all weights exactly zero and ReLU, it is worse: all hidden outputs are zero, so the outgoing weights receive zero gradient, and the incoming weights receive zero gradient because the outgoing weights are zero. Only the output bias learns.) **Random** initialization *breaks the symmetry*, giving each unit a different starting point so that they learn different features. Biases, on the other hand, can safely start at zero, because the random weights already make the units different.

Random, but how large? Too small and the signal (and the gradient) shrinks layer after layer; too large and it explodes. The right scale comes from a simple variance calculation.

11.2.2 Variance propagation

Consider one unit in a layer with n_{in} inputs:

$$z = \sum_{i=1}^{n_{\text{in}}} w_i x_i.$$

Assume (i) the weights w_i are independent with mean 0 and variance $\operatorname{Var}(w)$, (ii) the inputs x_i are independent of the weights and of each other, with mean 0 and variance $\operatorname{Var}(x)$. We want $\operatorname{Var}(z)$.

Step 1: one term. For independent w and x with zero mean, $\mathbb{E}[wx] = \mathbb{E}[w]\mathbb{E}[x] = 0$, and

$$\operatorname{Var}(wx) = \mathbb{E}[w^2x^2] - (\mathbb{E}[wx])^2 = \mathbb{E}[w^2]\,\mathbb{E}[x^2] - 0 = \operatorname{Var}(w)\operatorname{Var}(x),$$

using $\mathbb{E}[w^2] = \operatorname{Var}(w)$ when the mean is zero (and likewise for x).

Step 2: the sum. The n_{in} terms are independent, so their variances add:

Key equation: variance of a pre-activation

$$\operatorname{Var}(z) = n_{\text{in}} \operatorname{Var}(w) \operatorname{Var}(x). \tag{11.2}$$

To keep the signal the same size from layer to layer we want $\operatorname{Var}(z) = \operatorname{Var}(x)$, which requires $n_{\text{in}} \operatorname{Var}(w) = 1$, that is, $\operatorname{Var}(w) = 1/n_{\text{in}}$. This is sometimes called *LeCun initialization*, and it suits activations that behave like the identity near zero, such as tanh.

The backward pass. The same argument applied to $d\mathbf{A}_{\text{prev}} = d\mathbf{Z}\,\mathbf{W}^{\mathsf{T}}$ (each input-gradient is a sum of n_{out} terms $W_{ij}\,dZ_j$) gives $\operatorname{Var}(dx) = n_{\text{out}} \operatorname{Var}(w) \operatorname{Var}(dz)$. Keeping the gradient size constant therefore wants $\operatorname{Var}(w) = 1/n_{\text{out}}$. Unless $n_{\text{in}} = n_{\text{out}}$, we cannot satisfy both.

11.2.3 Xavier (Glorot) initialization

Glorot and Bengio (2010) proposed a compromise: use the average fan $\frac{1}{2}(n_{\text{in}} + n_{\text{out}})$ in place of n_{in} or n_{out}:

Key equation: Xavier / Glorot initialization

$$\operatorname{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}. \tag{11.3}$$

Normal version: $w \sim \mathcal{N}\left(0,\ \frac{2}{n_{\text{in}}+n_{\text{out}}}\right)$. Uniform version: $w \sim U[-a, a]$ with $a = \sqrt{\dfrac{6}{n_{\text{in}} + n_{\text{out}}}}$.

The uniform bound comes from the variance of a uniform distribution on $[-a, a]$, which is $a^2/3$ (integrate w^2 over the interval and divide by its length $2a$: $\frac{1}{2a}\int_{-a}^{a} w^2\,dw = \frac{a^2}{3}$). Setting $a^2/3 = 2/(n_{\text{in}} + n_{\text{out}})$ gives $a = \sqrt{6/(n_{\text{in}} + n_{\text{out}})}$. Xavier is the right choice for tanh and sigmoid-like activations, and for linear layers.

11.2.4 He (Kaiming) initialization

ReLU sets half of its inputs to zero, which changes the calculation. If z is symmetric around zero, then $a = \operatorname{ReLU}(z)$ is zero half of the time and equal to z the other half, so

$$\mathbb{E}[a^2] = \tfrac{1}{2}\mathbb{E}[z^2] = \tfrac{1}{2}\operatorname{Var}(z).$$

In the next layer, the input x is this ReLU output. Its mean is no longer zero, but the derivation of Eq. (11.2) only needs zero-mean *weights* and the second moment of x: $\mathrm{Var}(wx) = \mathbb{E}[w^2]\mathbb{E}[x^2]$. So

$$\mathrm{Var}(z^{[l]}) = n_{\text{in}}\,\mathrm{Var}(w)\,\mathbb{E}\left[(a^{[l-1]})^2\right] = n_{\text{in}}\,\mathrm{Var}(w)\cdot\tfrac{1}{2}\,\mathrm{Var}(z^{[l-1]}).$$

To keep $\mathrm{Var}(z)$ constant through the layers we need $\frac{1}{2}n_{\text{in}}\,\mathrm{Var}(w) = 1$. He et al. (2015) therefore proposed:

Key equation: He / Kaiming initialization for ReLU

$$\mathrm{Var}(w) = \frac{2}{n_{\text{in}}}, \qquad w \sim \mathcal{N}\left(0,\ \tfrac{2}{n_{\text{in}}}\right) \ \text{ or } \ U\left[-\sqrt{6/n_{\text{in}}},\ \sqrt{6/n_{\text{in}}}\right]. \tag{11.4}$$

The factor 2 compensates for ReLU discarding half of the signal.

Worked example: a $784 \to 128$ layer (an MNIST input layer)

- Xavier: $\mathrm{Var}(w) = 2/(784+128) = 2/912 = 0.002193$, so std $= \sqrt{0.002193} = 0.0468$; uniform bound $\sqrt{6/912} = 0.0811$.
- He: $\mathrm{Var}(w) = 2/784 = 0.002551$, so std $= 0.0505$; uniform bound $\sqrt{6/784} = 0.0875$.
- LeCun: std $= \sqrt{1/784} = 0.0357$.

The numbers look similar, but the difference compounds over depth. With He initialization, the pre-activation variance of a ReLU layer is preserved exactly (factor $\frac{1}{2}\times 784\times\frac{2}{784} = 1$ per layer). With Xavier it is multiplied by $\frac{1}{2}\times 784\times 0.002193 = 0.86$ per layer here, and for a square layer by exactly 0.5, which shrinks the variance about 1000-fold (2^{-10}) over ten square layers.

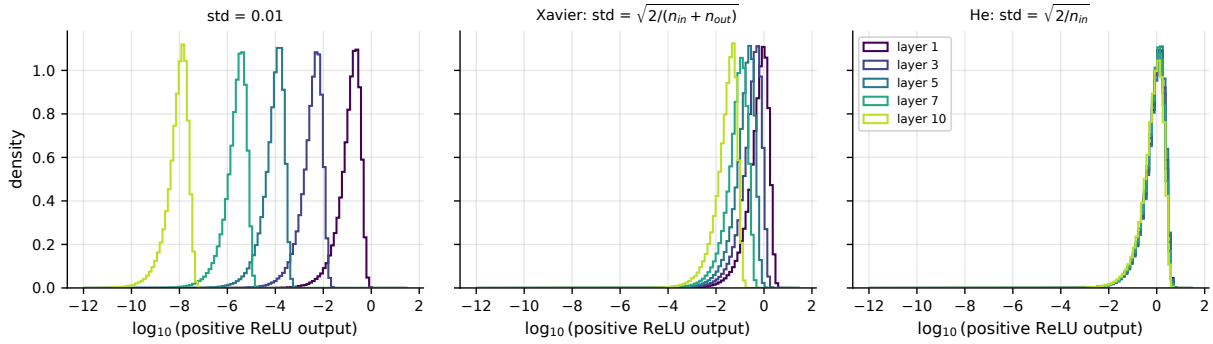


Figure 11.3: Distribution of the positive ReLU outputs, on a log scale, at layers 1, 3, 5, 7 and 10 of a 10-layer network of 512×512 ReLU layers fed with unit-variance inputs. Left: a "small random" init (std 0.01) shrinks the activations by about 6.5× per layer ($\sqrt{512}\times 0.01/\sqrt{2}$), so by layer 10 they are around 10^{-8}: the signal has vanished. Middle: Xavier (Var $= 2/1024$) loses a factor $\sqrt{2}$ in standard deviation per layer, about 20× over ten layers. Right: He (Var $= 2/512$) keeps the distribution essentially unchanged at every depth. Script: `code/ch11/fig_gradnorms_init.py`.

Pitfall: framework defaults

PyTorch's `nn.Linear` initializes weights from $U[-1/\sqrt{n_{\text{in}}}, 1/\sqrt{n_{\text{in}}}]$, which has variance $1/(3n_{\text{in}})$. That is smaller than both LeCun and He. It works for shallow nets and for nets with normalization layers, but for a deep plain ReLU MLP

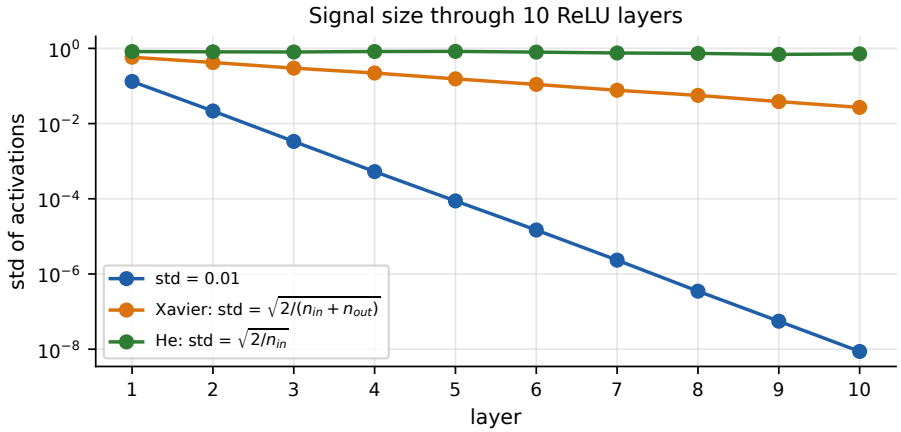


Figure 11.4: Standard deviation of the activations versus depth for the three initializations of Fig. 11.3. Measured values at layer 10: 8.7×10^{-9} (std 0.01), 0.027 (Xavier) and 0.71 (He). The straight lines on this log axis are the exponential decay predicted by Eq. (11.2).

call `nn.init.kaiming_normal_(layer.weight, nonlinearity='relu')` explicitly. Normalization layers (Chapter 12) and residual connections (Section 11.5) make networks much less sensitive to the exact initialization, which is one reason they are so popular.

11.3 Optimizers

Plain gradient descent has two weaknesses that we saw in Chapter 9. It zig-zags in ravines where curvatures differ (ill-conditioning), and it uses one learning rate for all parameters even though some parameters see large gradients and others tiny ones. Modern optimizers fix these with two ideas: *momentum* (average the gradients over time) and *adaptive step sizes* (scale each parameter's step by the typical size of its gradient). Figure 11.5 shows how they are related.

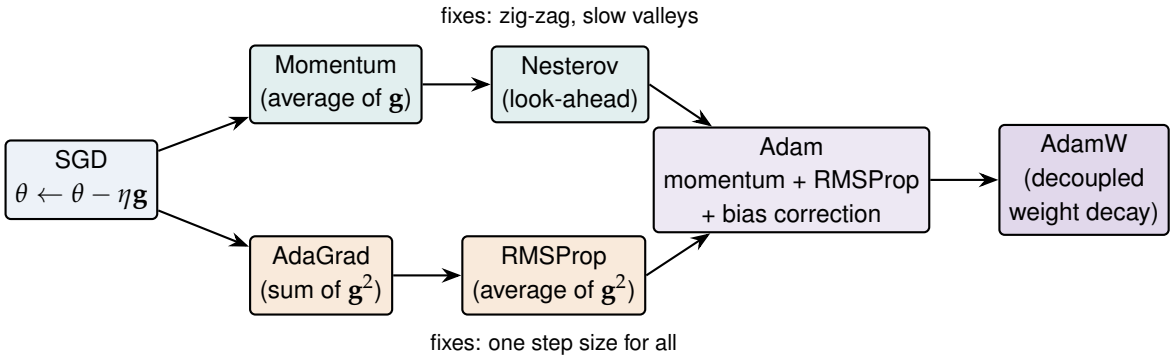


Figure 11.5: The family tree of first-order optimizers. The upper branch accumulates the gradient itself (momentum), which damps oscillations and speeds up travel along consistent directions. The lower branch accumulates the squared gradient per parameter, giving each parameter its own step size. Adam combines both, and AdamW fixes how Adam handles weight decay.

Throughout, $\mathbf{g}_t = \nabla L(\theta_{t-1})$ is the (mini-batch) gradient at step t, and all operations on vectors (squares, square roots, divisions) are element-wise.

11.3.1 Momentum: the heavy ball

Intuition: a heavy ball rolling in a valley

Plain gradient descent is a walker with no inertia: at every step it goes exactly where the local slope points. In a narrow ravine the slope points mostly across the ravine, so the walker bounces from wall to wall. Now replace the walker with a heavy ball. The ball builds up speed along directions where the slope is consistent (along the ravine floor), while its momentum averages out the left–right pushes across the ravine, which cancel each other. It reaches the bottom faster and with less zig-zag.

Definition: SGD with momentum

Keep a *velocity* vector $\mathbf{v}$ (initially $\mathbf{0}$) and a momentum coefficient $\mu \in [0, 1)$, typically 0.9:

$$\mathbf{v}_t = \mu\, \mathbf{v}_{t-1} + \mathbf{g}_t, \qquad \theta_t = \theta_{t-1} - \eta\, \mathbf{v}_t.$$

Unrolling the recursion shows what the velocity is:

$$\mathbf{v}_t = \mathbf{g}_t + \mu \mathbf{g}_{t-1} + \mu^2 \mathbf{g}_{t-2} + \cdots = \sum_{k=0}^{t-1} \mu^k \mathbf{g}_{t-k},$$

an *exponentially weighted sum* of past gradients, where a gradient from k steps ago carries weight μ^k. Two consequences follow:

- **Consistent directions accelerate.** If the gradient is a constant $\mathbf{g}$, then $\mathbf{v}_t = \mathbf{g}(1 + \mu + \cdots + \mu^{t-1}) = \mathbf{g}\,\frac{1-\mu^t}{1-\mu} \to \frac{\mathbf{g}}{1-\mu}$, a geometric series. With $\mu = 0.9$ the ball eventually moves $10\times$ faster than plain SGD with the same η (after 10 steps it has reached $1 - 0.9^{10} = 65\%$ of that terminal speed). With $\mu = 0.99$ the factor is 100.
- **Oscillating directions cancel.** If the gradient alternates in sign, $+g, -g, +g, \ldots$, the weighted sum is $g(1 - \mu + \mu^2 - \cdots) \to g/(1+\mu)$, which is about $g/2$ for $\mu = 0.9$. Across-ravine oscillations are damped while along-ravine progress is amplified twentyfold.

The *memory* of momentum is about $1/(1-\mu)$ steps: 10 steps for $\mu = 0.9$. For an ill-conditioned quadratic, a well-tuned momentum improves the convergence factor from $(\kappa - 1)/(\kappa + 1)$ to about $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ (Polyak, 1964). For $\kappa = 25$ that is $2/3$ instead of 0.92, roughly 17 instead of 86 steps per thousandfold error reduction.

11.3.2 Nesterov momentum: look before you leap

A heavy ball overshoots: by the time it notices that the valley floor curves upward, it is already moving too fast. Nesterov's accelerated gradient (1983; introduced to deep learning by Sutskever et al., 2013) evaluates the gradient not at the current position but at the position the momentum is *about to carry us to*:

$$\mathbf{v}_t = \mu\, \mathbf{v}_{t-1} + \nabla L\big(\theta_{t-1} - \eta\mu\, \mathbf{v}_{t-1}\big), \qquad \theta_t = \theta_{t-1} - \eta\, \mathbf{v}_t.$$

If the look-ahead point is already past the valley floor, its gradient points back, and the correction is applied one step earlier than with classical momentum. In practice Nesterov is a small, nearly free improvement (`torch.optim.SGD(..., momentum=0.9, nesterov=True)`). PyTorch implements an algebraically equivalent rearrangement that needs only the gradient at the current point.

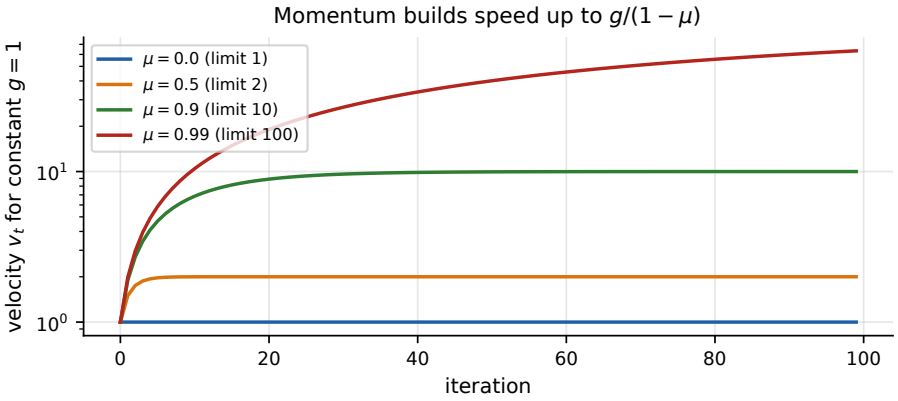


Figure 11.6: Velocity of momentum for a constant gradient $g = 1$. The velocity grows towards the terminal value $g/(1-\mu)$: 1, 2, 10 and 100 for $\mu = 0,\ 0.5,\ 0.9,\ 0.99$. Larger μ means a higher top speed and a longer memory, but also a longer time to build up (and to slow down when the slope changes). Script: `code/ch11/fig_optimizers.py`.

11.3.3 AdaGrad: per-parameter step sizes

Different parameters see very different gradient sizes. Weights attached to rare input features get small or infrequent gradients, while biases and common-feature weights get large ones. AdaGrad (Duchi, Hazan and Singer, 2011) gives each parameter its own effective learning rate, inversely proportional to the root of the sum of its past squared gradients:

$$\mathbf{s}_t = \mathbf{s}_{t-1} + \mathbf{g}_t^2, \qquad \theta_t = \theta_{t-1} - \eta\, \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t} + \epsilon},$$

with a tiny ϵ (e.g. 10^{-8}) to avoid division by zero. A parameter with consistently large gradients has a large $\mathbf{s}$ and small steps; a rarely updated one keeps a large step. Dividing by $\sqrt{\mathbf{s}}$ also makes the step roughly independent of the gradient's scale. If a gradient component is constant at g, then $\sqrt{s_t} = |g|\,\sqrt{t}$ and the step is $\eta\ \mathrm{sign}(g)/\sqrt{t}$. AdaGrad's weakness is that $\mathbf{s}$ only grows, so every learning rate decays like $1/\sqrt{t}$. That suits convex problems, but in deep networks training often stalls too early.

11.3.4 RMSProp: forget the distant past

RMSProp (proposed by Hinton in a 2012 lecture course) replaces AdaGrad's sum with an *exponential moving average*, so old gradients are forgotten:

$$\mathbf{s}_t = \beta\, \mathbf{s}_{t-1} + (1-\beta)\, \mathbf{g}_t^2, \qquad \theta_t = \theta_{t-1} - \eta\, \frac{\mathbf{g}_t}{\sqrt{\mathbf{s}_t} + \epsilon},$$

with $\beta \approx 0.9$–0.99. Now $\sqrt{\mathbf{s}_t}$ estimates the recent root-mean-square (RMS) gradient of each parameter, hence the name. In an elongated bowl, the steep direction has large gradients and is divided by a large RMS; the flat direction has small gradients and is divided by a small RMS. The effective curvatures are equalized, a cheap form of *preconditioning* that attacks the condition-number problem of Chapter 9 directly.

11.3.5 Adam: momentum plus RMSProp, with bias correction

Adam (Kingma and Ba, 2015) combines both ideas. It keeps an exponential average of the gradient (first moment, like momentum) and of the squared gradient (second moment, like RMSProp):

Key equation: Adam

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\,\mathbf{g}_t, & \hat{\mathbf{m}}_t &= \frac{\mathbf{m}_t}{1-\beta_1^t}, \\ \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\,\mathbf{g}_t^2, & \hat{\mathbf{v}}_t &= \frac{\mathbf{v}_t}{1-\beta_2^t}, \\ \theta_t &= \theta_{t-1} - \eta\,\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t}+\epsilon}. \end{aligned} \tag{11.5}$$

Defaults: $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, with $\mathbf{m}_0 = \mathbf{v}_0 = \mathbf{0}$.

Why bias correction? Both averages start at zero, so in early steps they are biased towards zero. With $\mathbf{m}_0 = 0$, the first average is $\mathbf{m}_1 = (1-\beta_1)\mathbf{g}_1 = 0.1\,\mathbf{g}_1$, ten times too small. In general, if the gradients have a constant expected value $\mathbb{E}[\mathbf{g}]$,

$$\mathbb{E}[\mathbf{m}_t] = (1-\beta_1)\sum_{k=1}^{t}\beta_1^{t-k}\mathbb{E}[\mathbf{g}] = (1-\beta_1)\frac{1-\beta_1^t}{1-\beta_1}\mathbb{E}[\mathbf{g}] = (1-\beta_1^t)\,\mathbb{E}[\mathbf{g}],$$

so dividing by $1-\beta_1^t$ removes the bias exactly. The same holds for $\mathbf{v}_t$ with β_2. Since $\beta_2 = 0.999$ is so close to 1, the uncorrected $\mathbf{v}$ would be about $1000\times$ too small for the first few steps, making $\sqrt{\mathbf{v}}$ about $30\times$ too small and the first steps about $30\times$ too large. After a few thousand steps, $\beta^t \approx 0$ and the corrections no longer matter.

Worked example: the first Adam step

One parameter, first gradient $g_1 = 2$, default hyperparameters.

$$\begin{aligned} m_1 &= 0.9(0) + 0.1(2) = 0.2, & \hat{m}_1 &= 0.2/(1-0.9) = 2, \\ v_1 &= 0.999(0) + 0.001(2^2) = 0.004, & \hat{v}_1 &= 0.004/(1-0.999) = 4, \\ \Delta\theta &= -0.001 \times \frac{2}{\sqrt{4}+10^{-8}} = -0.001. \end{aligned}$$

The first step has size exactly $\eta = 0.001$, whatever the size of g_1: with $g_1 = 2000$ it would be the same. Without bias correction it would have been $0.001 \times 0.2/\sqrt{0.004} = 0.00316$, more than three times too large. If the second gradient is again 2, the step is again 0.001. For a consistent gradient, Adam moves each parameter by about η per step. The learning rate in Adam is roughly *the maximum distance each parameter moves per step*, a very interpretable quantity.

The ratio $\hat{\mathbf{m}}/\sqrt{\hat{\mathbf{v}}}$ is like a signal-to-noise ratio for each parameter. When the recent gradients agree, $|\hat{m}| \approx \sqrt{\hat{v}}$ and the step is about η. When they are noisy and disagree in sign, $\hat{m}$ averages towards zero while $\sqrt{\hat{v}}$ stays large, so the step shrinks automatically.

11.3.6 AdamW: decoupled weight decay

Weight decay shrinks weights towards zero at every step to fight overfitting (Chapter 12). With plain SGD it is equivalent to adding an L2 penalty $\frac{\lambda}{2}\|\theta\|^2$ to the loss: the gradient gains a term $\lambda\theta$, and the update becomes $\theta \leftarrow \theta - \eta(\mathbf{g} + \lambda\theta) = (1-\eta\lambda)\theta - \eta\mathbf{g}$. With Adam the two are *not* equivalent. If you add $\lambda\theta$ to the gradient, it passes through the division by $\sqrt{\hat{\mathbf{v}}}$, so parameters with large gradients are barely regularized and those with small gradients are regularized strongly.

That is not what was intended. Loshchilov and Hutter (2019) proposed to *decouple* the decay from the adaptive step:

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} + \lambda\, \theta_{t-1} \right),$$

where $\mathbf{m}$ and $\mathbf{v}$ are computed from the loss gradient only. This is **AdamW** (`torch.optim.AdamW`, typical $\lambda = 0.01$–0.1). It usually generalizes better than Adam with an L2 term and is the default optimizer for Transformers.

11.3.7 Comparing the optimizers

Figure 11.7 runs all six optimizers for 100 steps on an elongated bowl $L = \frac{1}{2}(w_0^2 + 25 w_1^2)$ (condition number 25), each with a reasonably tuned learning rate. Plain SGD must keep $\eta < 2/25$; with $\eta = 0.075$ it zig-zags across the steep direction while creeping along the gentle one. Momentum (same $\mu = 0.9$, smaller η) overshoots in big swings because the ball carries too much speed. Nesterov's look-ahead damps those swings and is the fastest of the momentum family, reaching a loss of 1.6×10^{-3} after 25 steps against 0.86 for SGD. The adaptive methods head almost straight for the minimum, because dividing by the RMS gradient equalizes the two directions. RMSProp with a constant η ends up dithering near the minimum, since its normalized step never shrinks. Adam's momentum smooths this, but with $\eta = 0.5$ it spirals around before settling.

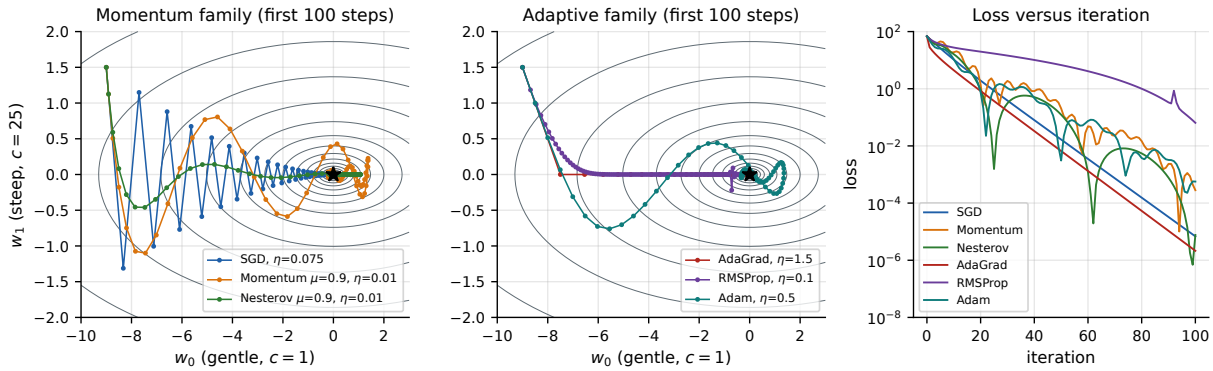


Figure 11.7: Six optimizers on the ravine $L = \frac{1}{2}(w_0^2 + 25 w_1^2)$, from $(-9, 1.5)$, 100 steps each (dots mark the iterates). Left: SGD zig-zags; momentum oscillates with large amplitude; Nesterov anticipates and damps the oscillation. Middle: AdaGrad, RMSProp and Adam rescale each coordinate by its gradient RMS, so they first move diagonally towards the floor and then straight along it; RMSProp keeps jittering near the minimum because its step does not decay. Right: loss versus iteration. The ranking depends on the problem and on the tuning. No optimizer wins everywhere, which is why practitioners keep a few in their toolbox. Script: `code/ch11/fig_optimizers.py`.

Pitfall: which optimizer should I use?

A pragmatic default: **AdamW** with $\eta = 10^{-3}$ (smaller, about 10^{-4}–3×10^{-4}, for Transformers and fine-tuning), weight decay 0.01, warm-up plus cosine decay. Adam is forgiving and works across architectures with little tuning. **SGD with Nesterov momentum 0.9** and a carefully tuned schedule is still competitive, and sometimes generalizes slightly better, for convolutional image classifiers. When switching optimizers, *re-tune the learning rate*: Adam's 10^{-3} and SGD's 0.1 are not comparable numbers.

11.4 Gradient clipping

Even with good initialization, an occasional batch can produce a huge gradient: a cliff in the loss landscape, a rare outlier, or a long product of Jacobians in a recurrent net. One such step can throw the parameters far away and undo hours of training. **Gradient clipping** caps the step.

Definition: clipping by global norm

Compute the norm of the full gradient vector (all parameters concatenated), $\|\mathbf{g}\|$. If it exceeds a threshold c, rescale:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|}\right).$$

The direction is preserved; only the length is capped at c.

Worked example: clipping

$\mathbf{g} = (3, 4)$, $\|\mathbf{g}\| = 5$, threshold $c = 1$: $\mathbf{g} \leftarrow (3, 4) \times \frac{1}{5} = (0.6, 0.8)$, with norm 1 and the same direction. A gradient with norm 0.5 would be left untouched.

In PyTorch it is one line between `loss.backward()` and `opt.step()`:

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

The function returns the norm before clipping, which is worth logging: frequent clipping is a symptom that the learning rate is too high. Clipping each element separately is also possible (PyTorch: `clip_grad_value_`), but it changes the gradient's direction and is less common.

Wireless connection: exploding gradients in RNN channel predictors

A recurrent network that predicts a fading channel from its past (CSI prediction, Chapter 28) applies the same weight matrix at every time step: $\mathbf{h}_t = \tanh(\mathbf{h}_{t-1}\mathbf{W} + \mathbf{x}_t\mathbf{U} + \mathbf{b})$. Backpropagation through time therefore multiplies by the same Jacobian once per step, exactly the eigenvalue picture of Section 11.1. Figure 11.9 (middle) shows a linear recurrence with a random 32×32 matrix scaled to spectral radius 0.9, 1.0 and 1.1: after 50 steps the gradient has shrunk to 1.5×10^{-3}, stayed around 0.25, or grown to 38. Predicting a Doppler-faded channel several slots ahead needs memory over many samples, so the gradient must travel far. Practical recipes are: use LSTM or GRU cells, whose gating creates near-identity paths (Chapter 16); clip the gradient norm, typically at 1–5; truncate backpropagation through time to a window comparable to the channel coherence time; and initialize recurrent matrices to be orthogonal (all eigenvalues of magnitude 1).

11.5 Residual connections: gradient highways

Initialization keeps the gradient the right size at the start of training, but as the weights change, a very deep plain network can drift back into vanishing or exploding territory. He et al. (2016) found a structural cure that made networks with over 100 layers trainable: the **residual connection** (skip connection). Instead of having each block compute a new representation $\mathbf{h} \leftarrow F(\mathbf{h})$, let it compute a *correction* that is added to its input:

$$\mathbf{h}^{[l]} = \mathbf{h}^{[l-1]} + F\big(\mathbf{h}^{[l-1]}\big).$$

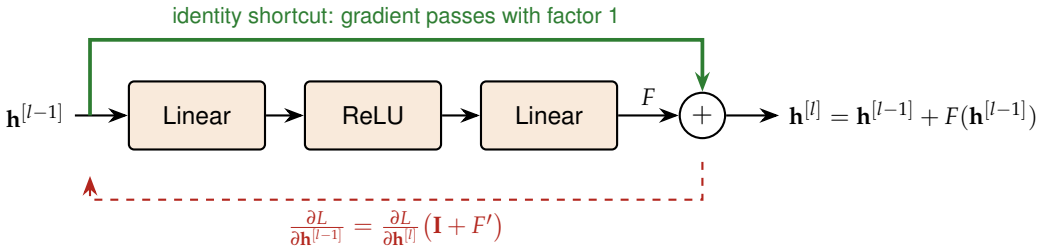


Figure 11.8: A residual block. The input skips around the learned branch F (green) and is added to its output. In the backward pass (red) the gradient flows through the identity shortcut unchanged, plus a contribution through F.

Why it helps. Differentiate the block. The Jacobian of a sum is the sum of the Jacobians, and the Jacobian of the identity map is the identity matrix:

Key equation: gradient through a residual block

$$\frac{\partial \mathbf{h}^{[l]}}{\partial \mathbf{h}^{[l-1]}} = \frac{\partial}{\partial \mathbf{h}^{[l-1]}}\left(\mathbf{h}^{[l-1]} + F(\mathbf{h}^{[l-1]})\right) = \mathbf{I} + \frac{\partial F}{\partial \mathbf{h}^{[l-1]}} = \mathbf{I} + F'. \tag{11.6}$$

In the product of Jacobians, Eq. (11.1), each factor is now $\mathbf{I} + F_l'$ rather than F_l'. Expanding the product over many blocks gives

$$\prod_l (\mathbf{I} + F_l') = \mathbf{I} + \sum_l F_l' + \sum_{l<m} F_l' F_m' + \cdots$$

It always contains the term $\mathbf{I}$: a direct path from the loss to every layer along which the gradient is not multiplied by anything. Even if every F_l' is small, the gradient reaching layer 1 is at least as large as the one at the top. The shortcuts act as *gradient highways*. A second benefit is easy optimization: if a block is not useful, it only needs $F \approx 0$ to become the identity, rather than having to learn the identity map through weight matrices.

Figure 11.9 (left) compares a 30-layer tanh MLP with and without shortcuts, both with PyTorch's default initialization. In the plain net, the gradient at layer 1 is 7×10^{-7} of that at layer 30. In the residual net, every layer receives a gradient of the same size (the ratio is 1.15).

11.6 Learning-rate warm-up, revisited

Chapter 9 introduced warm-up as a schedule. We can now say more precisely why it matters, especially for adaptive optimizers.

- **Unreliable second moments.** In Adam, $\hat{\mathbf{v}}_t$ is an average over only t gradients in the first steps. Bias correction fixes its mean but not its variance, so some parameters get a small $\sqrt{\hat{v}}$ by chance, and their steps are too large. Liu et al. (2020) analysed this and showed that a warm-up has the effect of variance reduction.
- **Large initial gradients.** At initialization the loss is high and the gradients can be large and inconsistent. Steps that are too big at this stage can kill ReLU units (Chapter 10) or push normalization statistics into bad regions.
- **Large batches.** When the learning rate is scaled up with the batch size (Chapter 9), the full rate is only safe after the network has moved away from its chaotic initial state (Goyal et al., 2017).

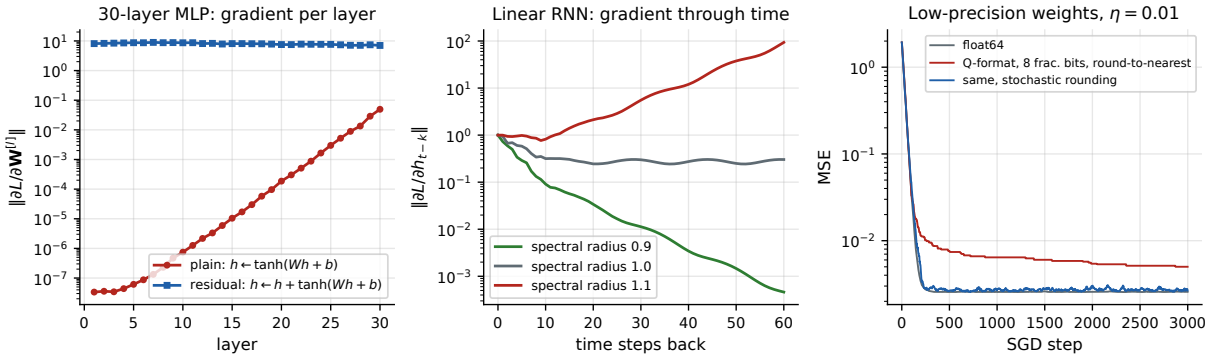


Figure 11.9: Left: weight-gradient norm per layer in a 30-layer, 64-unit tanh network without (red) and with (blue) residual shortcuts, average of 5 random initializations. The plain net's gradient decays exponentially towards the input; the residual net's is flat. Middle: gradient norm propagated back through time in a linear recurrence $\mathbf{h}_t = \mathbf{h}_{t-1}\mathbf{W}$, for random $\mathbf{W}$ with spectral radius 0.9, 1.0 and 1.1. Right: SGD on a linear-regression problem with weights stored in fixed point with 8 fractional bits (LSB $= 2^{-8} \approx 0.0039$). Round-to-nearest stalls at twice the float MSE because small updates are rounded away; stochastic rounding tracks the float curve. Script: `code/ch11/fig_residual_rnn_fixed.py`.

A linear warm-up over the first 1–5% of training steps (a few hundred to a few thousand iterations) is cheap insurance. In PyTorch it can be built with `torch.optim.lr_scheduler.LinearLR` chained with a cosine scheduler via `SequentialLR`.

Listing 11.1: A robust PyTorch training step combining this chapter's tools.

```
model = MyNet()                                    # He init inside for ReLU layers
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)
warm = torch.optim.lr_scheduler.LinearLR(opt, start_factor=0.01, total_iters=500)
cos  = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=total_steps - 500)
sched = torch.optim.lr_scheduler.SequentialLR(opt, [warm, cos], milestones=[500])
for x, y in loader:
    loss = loss_fn(model(x), y)
    opt.zero_grad()
    loss.backward()
    gnorm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    opt.step(); sched.step()                       # log loss and gnorm!
```

11.7 Training on limited-precision hardware

Everything so far assumed 32-bit floating point. Many deployments, and some on-device training, use 16-bit floats (FP16/BF16) or fixed-point integers. The training dynamics then change in specific ways.

Wireless connection: fixed-point and low-precision training for embedded DSP

On a fixed-point DSP (a CEVA or TI core inside a baseband SoC, for instance), a weight in Q-format with F fractional bits can only change in multiples of the LSB 2^{-F}. An SGD update ηg smaller than half an LSB is *rounded to zero*: late in training, when gradients become small, learning silently stops. That is the stall in Fig. 11.9 (right): with $F = 8$ and $\eta = 0.01$ the typical update near the optimum is about 0.003, below the LSB of 0.0039, and the MSE stops at 0.0051

instead of 0.0026. Remedies:

- **Stochastic rounding**: round up with probability equal to the fractional remainder. The rounding error then has zero mean, so tiny updates survive on average. In the figure it recovers the float result (0.0027). Gupta et al. (2015) showed that 16-bit fixed-point training works with stochastic rounding.
- **Master weights**: keep a high-precision (32-bit) copy of the weights for the update and quantize only for the forward and backward passes. This is how mixed-precision training on GPUs works.
- **Loss scaling**: in FP16, small gradients underflow to zero. Multiply the loss by a large factor (e.g. 2^{10}) before backprop and divide the gradients back afterwards (`torch.amp.GradScaler`).
- **Mind the dynamic range**: Adam's $\sqrt{\hat{v}} + \epsilon$ division and softmax exponentials need wide range. BF16 (8 exponent bits) is far safer than FP16 (5 exponent bits) for this reason.
- **Saturating arithmetic and clipping**: on a DSP, overflow wraps around unless saturation is enabled. Gradient clipping and normalization keep values inside the representable range.

More often the network is trained in float on a server and only *inference* runs in fixed point, via quantization (post-training, or quantization-aware training that simulates rounding in the forward pass and uses the straight-through estimator in the backward pass). Even then, the initialization and normalization ideas of this chapter matter: well-scaled activations use the limited bit width efficiently.

Chapter recap

- Backprop multiplies one Jacobian per layer (or time step). Products of many factors vanish or explode: $0.25^{10} \approx 10^{-6}$ for sigmoid derivatives, and ρ^{ℓ} for a weight matrix with spectral radius ρ.
- Zero (or identical) initialization leaves all units in a layer as clones forever. Random weights break the symmetry; biases can start at zero.
- $\mathrm{Var}(z) = n_{\text{in}} \mathrm{Var}(w) \mathrm{Var}(x)$. Xavier: $\mathrm{Var}(w) = 2/(n_{\text{in}} + n_{\text{out}})$, uniform bound $\sqrt{6/(n_{\text{in}} + n_{\text{out}})}$, for tanh/sigmoid/linear. He: $2/n_{\text{in}}$ for ReLU. For $784 \to 128$ the standard deviations are 0.0468 (Xavier) and 0.0505 (He).
- Momentum averages gradients ($\mathbf{v} \to \mathbf{g}/(1-\mu)$ for constant gradients) and damps oscillations; Nesterov looks ahead. AdaGrad and RMSProp scale each parameter's step by its RMS gradient. Adam does both with bias correction (first step $= \eta$ exactly). AdamW decouples weight decay.
- Clip the global gradient norm to survive rare huge gradients. Residual blocks have Jacobian $\mathbf{I} + F'$, so gradients have an identity highway to every layer. Warm-up protects the first, fragile steps.
- Low-precision training: updates smaller than an LSB vanish under round-to-nearest. Use stochastic rounding, high-precision master weights and loss scaling.

Exercises

1. A 20-layer network uses tanh, and at initialization the typical $\left|\tanh'(z)\right|$ is 0.6. Ignoring the weights, by what factor is the gradient at layer 1 smaller than at layer 20? What if ReLU were used and all units were active?
2. Compute the Xavier standard deviation and uniform bound, and the He standard deviation, for a $256 \to 64$ layer.
3. Using $\mathrm{Var}(z) = n_{\text{in}} \mathrm{Var}(w) \mathrm{Var}(x)$, show that a ReLU network of square $n \times n$ layers initialized with Xavier loses half of its pre-activation variance per layer. After 10 layers, by what factor has the standard deviation shrunk?
4. Momentum with $\mu = 0.9$, constant gradient $g = 0.5$, $\eta = 0.01$. What are the velocity and step after 1, 2 and 3 iterations? What is the terminal step size?
5. Adam, default hyperparameters. The first gradient is $g_1 = 2$ and the second is $g_2 = -2$. Compute m_2, v_2, $\hat{m}_2$, $\hat{v}_2$ and the second step.
6. AdaGrad with $\eta = 1$ on a parameter whose gradient is always $g = 3$. What is the step at iterations 1, 4 and 100? Why does this become a problem for long training runs?
7. A gradient vector $(6, -8, 0)$ is clipped to a global norm of 5. What is the clipped vector? What would per-element clipping to $[-5, 5]$ give, and why is that different?
8. (Conceptual) For the residual block $\mathbf{h}^{[l]} = \mathbf{h}^{[l-1]} + F(\mathbf{h}^{[l-1]})$, show that even if $F' = 0$ for every block, the gradient reaches layer 1 unchanged. What happens to a plain network if one layer's Jacobian is zero?

Answers

1. $0.6^{19} \approx 6.1 \times 10^{-5}$ (19 activation derivatives between layer 20 and layer 1). With ReLU and all units active, the factor is $1^{19} = 1$, so only the weights determine the scaling.
2. Xavier: Var $= 2/(256 + 64) = 2/320 = 0.00625$, std 0.0791, uniform bound $\sqrt{6/320} = 0.137$. He: Var $= 2/256 = 0.00781$, std 0.0884.
3. With Xavier and $n_{\text{in}} = n_{\text{out}} = n$, $\mathrm{Var}(w) = 2/(2n) = 1/n$. ReLU halves the second moment: $\mathrm{Var}(z^{[l]}) = n \cdot \frac{1}{n} \cdot \frac{1}{2} \mathrm{Var}(z^{[l-1]}) = \frac{1}{2} \mathrm{Var}(z^{[l-1]})$. After 10 layers, the variance factor is $2^{-10} \approx 10^{-3}$ and the std factor is $2^{-5} = 1/32$. This matches the factor of about 20–30 seen in Fig. 11.4.
4. $v_1 = 0.5$, step 0.005; $v_2 = 0.9(0.5) + 0.5 = 0.95$, step 0.0095; $v_3 = 0.9(0.95) + 0.5 = 1.355$, step 0.01355. Terminal velocity $0.5/(1 - 0.9) = 5$, terminal step 0.05, ten times plain SGD's 0.005.
5. $m_1 = 0.2$, $v_1 = 0.004$. $m_2 = 0.9(0.2) + 0.1(-2) = -0.02$; $v_2 = 0.999(0.004) + 0.001(4) = 0.007996$. $\hat{m}_2 = -0.02/(1 - 0.81) = -0.1053$; $\hat{v}_2 = 0.007996/(1 - 0.998001) = 4.000$. Step $= -0.001 \times (-0.1053)/2 = +5.3 \times 10^{-5}$. The sign flip almost cancelled the momentum, so Adam takes a tiny step: the per-parameter signal-to-noise ratio is low.
6. $s_t = 9t$, step $= \eta g/\sqrt{s_t} = 3/(3\sqrt{t}) = 1/\sqrt{t}$: 1, 0.5, 0.1. The effective learning rate decays like $1/\sqrt{t}$ regardless of whether we are near a minimum. After 10^6 steps it is 10^{-3} of the initial value and training effectively stops. RMSProp and Adam fix this by forgetting old gradients.

7. Norm $= \sqrt{36+64} = 10$; clipped vector $= (6,-8,0) \times 5/10 = (3,-4,0)$. Per-element clipping gives $(5,-5,0)$, norm 7.07, pointing in a different direction (45 degrees instead of the original angle). Norm clipping preserves the descent direction; element clipping does not.
8. With $F' = 0$, each block's Jacobian is $\mathbf{I} + 0 = \mathbf{I}$, and the product over all blocks is $\mathbf{I}$, so $\frac{\partial L}{\partial \mathbf{h}^{[0]}} = \frac{\partial L}{\partial \mathbf{h}^{[\ell]}}$ exactly. In a plain network the Jacobians multiply, so a single zero Jacobian (e.g. a layer of all-dead ReLUs) makes the whole product zero, and no gradient reaches any layer below it.

Self-test quiz

1. Why do sigmoid hidden layers cause vanishing gradients?
   $\sigma' \leq 0.25$, and backprop multiplies one such factor per layer: $0.25^{10} \approx 10^{-6}$.
2. What goes wrong if all weights of a hidden layer start equal?
   All units compute the same output and receive the same gradient, so they stay identical forever (no symmetry breaking).
3. Which initialization suits ReLU, and where does its factor 2 come from?
   He, $\mathrm{Var}(w) = 2/n_{in}$; ReLU zeroes half of its inputs, halving the second moment.
4. In Adam, what is the size of the very first step for a single parameter, and why?
   Exactly η (up to ϵ): after bias correction $\hat{m}_1 = g_1$ and $\sqrt{\hat{v}_1} = |g_1|$.
5. What does AdamW change relative to Adam with an L2 penalty?
   It applies weight decay directly to the weights, outside the adaptive division by $\sqrt{\hat{v}}$.
6. Why do residual connections prevent vanishing gradients?
   Each block's Jacobian is $\mathbf{I} + F'$, so the product over blocks always contains an identity path from the loss to every layer.

Further reading

- X. Glorot, Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," *AISTATS*, 2010.
- K. He, X. Zhang, S. Ren, J. Sun, "Delving deep into rectifiers: surpassing human-level performance on ImageNet classification," *ICCV*, 2015 (He initialization).
- K. He, X. Zhang, S. Ren, J. Sun, "Deep residual learning for image recognition," *CVPR*, 2016.
- Y. Bengio, P. Simard, P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Networks*, 5(2), 1994.
- R. Pascanu, T. Mikolov, Y. Bengio, "On the difficulty of training recurrent neural networks," *ICML*, 2013 (gradient clipping).
- I. Sutskever, J. Martens, G. Dahl, G. Hinton, "On the importance of initialization and momentum in deep learning," *ICML*, 2013.
- J. Duchi, E. Hazan, Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," *JMLR*, 12, 2011 (AdaGrad).
- D. P. Kingma, J. Ba, "Adam: a method for stochastic optimization," *ICLR*, 2015.

- I. Loshchilov, F. Hutter, “Decoupled weight decay regularization,” *ICLR*, 2019 (AdamW).
- L. Liu et al., “On the variance of the adaptive learning rate and beyond,” *ICLR*, 2020 (why warm-up helps Adam).
- S. Gupta, A. Agrawal, K. Gopalakrishnan, P. Narayanan, “Deep learning with limited numerical precision,” *ICML*, 2015 (stochastic rounding).

Chapter 12

Generalization: Overfitting, Regularization, Normalization and Evaluation

A network that scores perfectly on its training data but fails on new data is useless. In Chapter 5 we met the central goal of machine learning, *generalization*: doing well on data the model has never seen. With the tools of the last three chapters you can drive the training loss of almost any network to nearly zero. This chapter is about making that success carry over to new data, and about *measuring* honestly whether it does.

We start by watching overfitting happen, with polynomials of degree 1, 4 and 15, and by deriving the **bias–variance decomposition** that explains it. We then mention the modern *double descent* phenomenon, which complicates the classical story. Next comes the regularization toolbox: L2 and L1 penalties (with their Bayesian interpretation as priors), dropout, early stopping and data augmentation, including augmentations specific to radio signals. Then normalization layers (BatchNorm and LayerNorm), with a fully worked example. Finally, evaluation: how to split data without fooling yourself, how to tune hyperparameters, and which metrics to report, from precision and recall to ROC curves, calibration, BER and the detection-versus-false-alarm trade-off of spectrum sensing.

12.1 Underfitting and overfitting

Intuition: the student who memorizes past exam papers

Two students prepare for an exam using the same 20 past questions. The first barely studies and learns one crude rule of thumb. He does poorly on the practice questions and poorly on the real exam: he *underfits*. The second memorizes all 20 answers word for word, including the typos in the answer key. She scores 100% on the practice questions, but on the real exam the questions are slightly different and she is lost: she *overfits*. A third student learns the underlying principles. Her practice score is good but not perfect, and her exam score is almost as good. That third student is what we want our model to be.

Figure 12.1 makes this concrete. Twenty noisy samples are drawn from $y = \sin(\pi x) + \varepsilon$ with noise standard deviation $\sigma = 0.25$, and we fit polynomials by least squares (Chapter 5).

- **Degree 1** (a straight line) cannot bend to follow the sine. Training MSE 0.215, test MSE 0.320: both are high, so the model **underfits** (it has high bias).
- **Degree 4** follows the sine closely. Training MSE 0.029, test MSE 0.071: both are close to the noise floor $\sigma^2 = 0.0625$.
- **Degree 15** wiggles through nearly every training point. Training MSE 0.002, far *below* the noise floor, so it has fitted the noise itself. Test MSE 52.4: it **overfits** catastrophically, oscillating wildly between and beyond the training points.

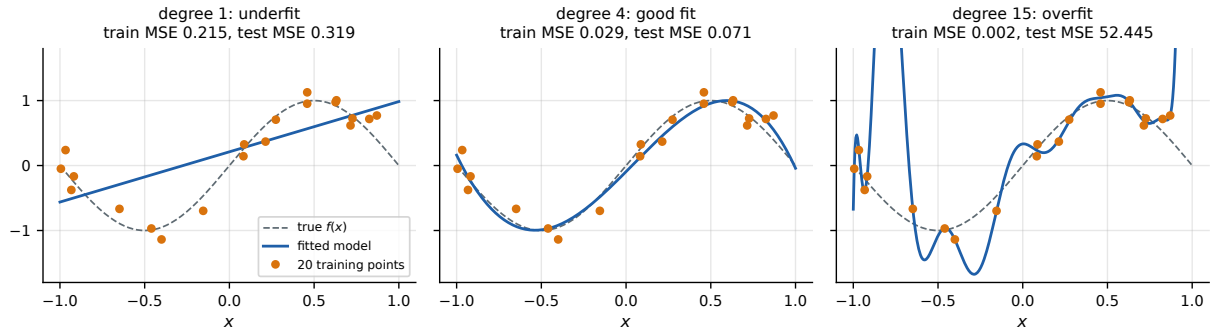


Figure 12.1: Polynomials of degree 1, 4 and 15 fitted to the same 20 noisy samples of $\sin(\pi x)$ (dashed). The line is too rigid (underfit). Degree 4 captures the shape. Degree 15 threads through the training points but swings wildly elsewhere (overfit). The training MSE always falls as the degree rises; the test MSE (on 2000 fresh points) does not. Script: `code/ch12/fig_overfit_biasvar.py`.

Definition: underfitting, overfitting, generalization gap

The **generalization gap** is the test error minus the training error. A model **underfits** when its training error is itself high: it lacks the capacity, or the training, to capture the pattern. It **overfits** when its training error is low but the gap is large: it has learned details of the particular training sample (noise, coincidences) that do not hold for new data.

Figure 12.2 (left) repeats the experiment for every degree from 0 to 15, averaged over 200 random training sets. Training error falls monotonically with capacity, while test error follows the classic **U-shape**: it falls while the model gains the flexibility it needs, then rises once the extra flexibility is spent on fitting noise. The right panel shows **learning curves**, the errors as a function of training-set size. For the rigid degree-1 model, train and test errors meet quickly at a high value: more data does not help, a sign of underfitting. For the flexible degree-12 model the gap is enormous with little data and shrinks as data grows: more data *does* help, a sign of overfitting. Learning curves are one of the most useful diagnostics you can plot for your own models.

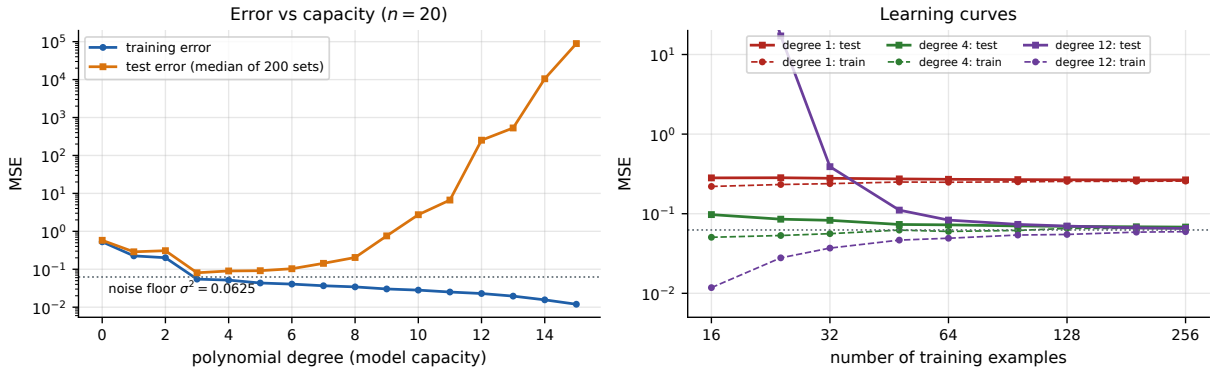


Figure 12.2: Left: training and test MSE versus polynomial degree with $n = 20$ points (training error averaged and test error the median over 200 random training sets). The test curve is U-shaped, with its minimum around degree 3–4; the dotted line is the irreducible noise σ^2. Right: learning curves for degrees 1, 4 and 12. Underfitting models (degree 1) converge quickly to a high error; overfitting models (degree 12) have a large train–test gap that closes as data grows. Script: `code/ch12/fig_overfit_biasvar.py`.

12.2 The bias–variance decomposition

Why is the test curve U-shaped? Think of the training set as random: if we had collected a different set of 20 points, we would have fitted a different model. A model's error on a new point then has three sources, and we can separate them exactly for squared error.

Set-up. Data are generated as $y = f(x) + \varepsilon$, where f is the true function and ε is noise with $\mathbb{E}[\varepsilon] = 0$ and $\mathrm{Var}(\varepsilon) = \sigma^2$, independent of everything else. We train on a random dataset $\mathcal{D}$ and obtain a prediction function $\hat{f}_{\mathcal{D}}$. Fix a test input x and write $\hat{f} = \hat{f}_{\mathcal{D}}(x)$ (random, because $\mathcal{D}$ is), $\bar{f} = \mathbb{E}_{\mathcal{D}}[\hat{f}]$ (the average prediction over all possible training sets), and $f = f(x)$. The expected test error at x, averaged over training sets and test noise, is $\mathbb{E}\big[(y - \hat{f})^2\big]$.

Step 1: separate the noise. Write $y - \hat{f} = (f - \hat{f}) + \varepsilon$ and expand:

$$\mathbb{E}\big[(y - \hat{f})^2\big] = \mathbb{E}\big[(f - \hat{f})^2\big] + 2\,\mathbb{E}\big[(f - \hat{f})\varepsilon\big] + \mathbb{E}[\varepsilon^2].$$

The test noise ε is independent of $\hat{f}$ (which depends only on the training data) and has zero mean, so the middle term is $2\,\mathbb{E}[f - \hat{f}]\,\mathbb{E}[\varepsilon] = 0$. The last term is σ^2.

Step 2: split the model error around its average. Add and subtract $\bar{f}$: $f - \hat{f} = (f - \bar{f}) + (\bar{f} - \hat{f})$. Then

$$\mathbb{E}\big[(f - \hat{f})^2\big] = (f - \bar{f})^2 + 2(f - \bar{f})\,\mathbb{E}[\bar{f} - \hat{f}] + \mathbb{E}\big[(\bar{f} - \hat{f})^2\big].$$

Here $f - \bar{f}$ is a constant (not random), and $\mathbb{E}[\bar{f} - \hat{f}] = \bar{f} - \bar{f} = 0$ by the definition of $\bar{f}$, so the middle term vanishes again.

Key equation: bias–variance decomposition of squared error

$$\mathbb{E}\big[(y - \hat{f}(x))^2\big] = \underbrace{\big(f(x) - \bar{f}(x)\big)^2}_{\text{bias}^2} + \underbrace{\mathbb{E}\big[(\hat{f}(x) - \bar{f}(x))^2\big]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible noise}}. \tag{12.1}$$

- **Bias**^{2}: how far the *average* model is from the truth. It is large when the model family is too rigid to represent f, whatever data it sees.
- **Variance**: how much the model changes from one training set to another. It is large when the model is so flexible that it follows the random details of each sample.
- **Noise** σ^2: no model can predict the random part of y. It is the floor in Fig. 12.2.

Intuition: darts

Think of each trained model as a dart thrown at the bullseye $f(x)$. Bias is how far the *centre* of your cluster of darts is from the bullseye. Variance is how *spread out* the cluster is. A rigid model throws a tight cluster in the wrong place; a flexible model throws a cluster centred on the bullseye but scattered all over the board. Total error is (centre offset)^2 + spread.

We can *measure* each term by simulation: train 500 models on 500 independent training sets, average their predictions to get $\bar{f}$, and compute both terms on a grid of test points. Figure 12.3 (left) shows the result with $n = 30$ points per set. Bias^2 falls from 0.205 (degree 1) to below 10^{-3} (degrees 3–5), while variance rises from 0.021 to 0.48 (degree 7) and 46 (degree 9). Their sum plus σ^2 is lowest at degree 3, where the total is 0.081.

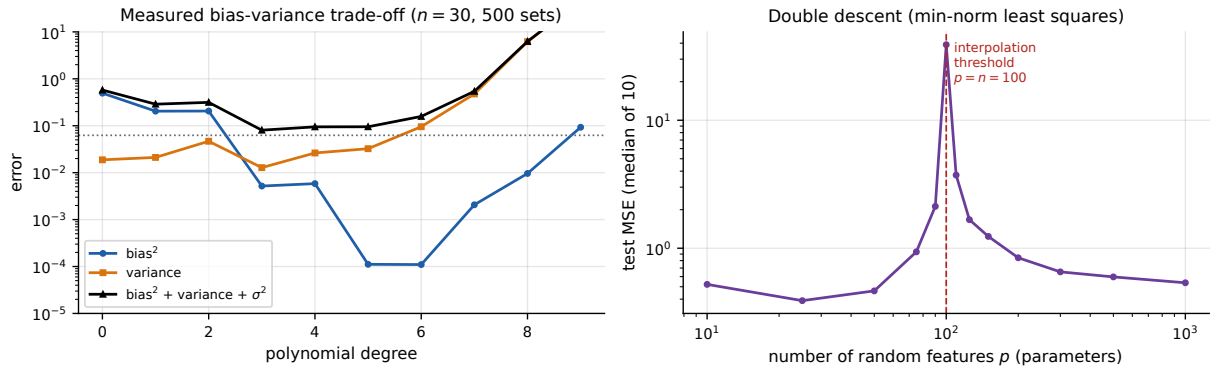


Figure 12.3: Left: bias^2, variance and their sum plus noise, measured over 500 independent training sets of 30 points each. Increasing capacity trades bias for variance, giving the U-shaped total. Right: double descent. Test MSE of minimum-norm least squares on p random ReLU features of a 10-dimensional input with $n = 100$ training points. The error rises as p approaches n, peaks sharply at the interpolation threshold $p = n$ (median MSE 63), and then *falls again* as p grows further, reaching 0.33 at $p = 1000$, close to the best under-parameterized model (0.26 at $p = 25$) and still decreasing. Script: `code/ch12/fig_overfit_biasvar.py`.

12.2.1 An honest caveat: double descent

The U-curve is not the whole story for modern networks, which often have far more parameters than training examples and still generalize well. Belkin et al. (2019) and Nakkiran et al. (2020) documented **double descent**: as capacity increases, test error first follows the U, peaks around the *interpolation threshold* (the capacity at which the model can just fit the training data exactly), and then decreases again in the heavily over-parameterized regime. Figure 12.3 (right) reproduces this with a simple random-features model.

The intuition: right at the threshold there is exactly one way to fit the data, and it is usually a wild one, so the variance blows up. Far beyond it there are infinitely many perfect fits. The training algorithm (here the minimum-norm solution; for networks, gradient descent from a small random initialization) picks a smooth, low-norm one, an *implicit regularization*. Double descent does not make bias–variance wrong. It shows that "number of parameters" is a poor measure of effective capacity. Practical consequences: a larger network is often better if trained with suitable regularization and enough data, and the classical warning "never use more parameters than data points" does not apply to deep networks. You still need the validation set to tell you where you are.

12.3 Regularization

Definition: regularization

Regularization is any change to the learning procedure meant to reduce test error, possibly at the expense of training error, usually by discouraging overly complex solutions.

Figure 12.4 maps the toolbox we cover.

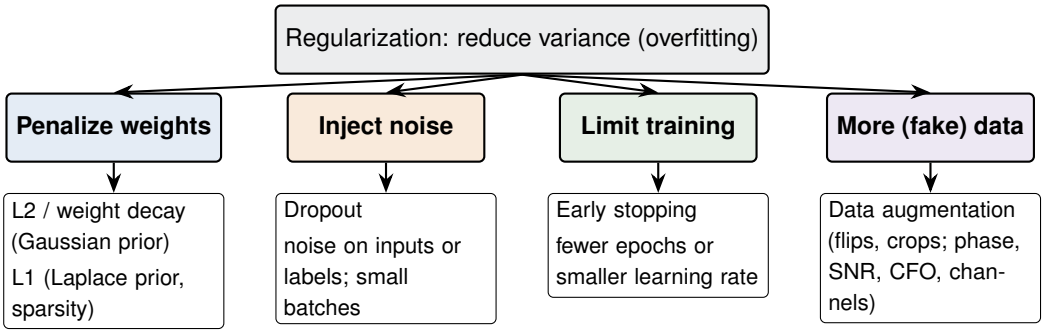


Figure 12.4: The regularization toolbox. Every technique trades a little training-set performance for lower variance, by constraining the weights, adding randomness, stopping before memorization sets in, or enlarging the effective dataset. Normalization layers (Section 12.4) also have a mild regularizing effect.

12.3.1 L2 regularization and weight decay

Add a penalty on the size of the weights to the training loss:

$$J(\mathbf{w}) = L(\mathbf{w}) + \frac{\lambda}{2}\,\|\mathbf{w}\|_2^2 = L(\mathbf{w}) + \frac{\lambda}{2}\sum_i w_i^2. \tag{12.2}$$

The strength $\lambda \geq 0$ is a hyperparameter. Biases are usually not penalized. The gradient gains a term $\lambda\mathbf{w}$, so a gradient-descent step becomes

$$\mathbf{w} \leftarrow \mathbf{w} - \eta\big(\nabla L + \lambda\mathbf{w}\big) = (1-\eta\lambda)\,\mathbf{w} - \eta\nabla L.$$

Before each ordinary step, every weight is multiplied by $1-\eta\lambda$, slightly less than one: it *decays* towards zero. With $\eta = 0.1$ and $\lambda = 0.01$ the factor is 0.999 per step. Weights the data do not actively support slowly fade away. (For adaptive optimizers, use the decoupled version, AdamW, Chapter 11.)

Why small weights generalize: the Bayesian view. Regularization has a clean probabilistic meaning. Suppose the targets are $y_i = f(\mathbf{x}_i;\mathbf{w}) + \varepsilon_i$ with Gaussian noise $\varepsilon_i \sim \mathcal{N}(0,\sigma^2)$, and suppose that *before* seeing data we believe each weight is probably small, as expressed by a Gaussian **prior** $w_j \sim \mathcal{N}(0,\tau^2)$. Bayes' rule (Chapter 4) gives the **posterior**

$$p(\mathbf{w}\mid\mathcal{D}) \propto p(\mathcal{D}\mid\mathbf{w})\,p(\mathbf{w}) = \prod_i \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(y_i - f(\mathbf{x}_i;\mathbf{w}))^2}{2\sigma^2}} \prod_j \frac{1}{\sqrt{2\pi\tau^2}} e^{-\frac{w_j^2}{2\tau^2}}.$$

The **maximum a posteriori** (MAP) estimate maximizes this, or equivalently minimizes its negative logarithm. Taking $-\ln$ turns products into sums and removes the exponentials; the constants from the normalizing factors do not depend on $\mathbf{w}$ and can be dropped:

$$-\ln p(\mathbf{w}\mid\mathcal{D}) = \frac{1}{2\sigma^2}\sum_i \big(y_i - f(\mathbf{x}_i;\mathbf{w})\big)^2 + \frac{1}{2\tau^2}\sum_j w_j^2 + \text{const.}$$

Multiplying by $2\sigma^2/N$ (which does not change the minimizer) gives exactly Eq. (12.2) with the mean squared error as L and

$$\lambda = \frac{\sigma^2}{N\tau^2}.$$

Key equation: L2 regularization = Gaussian prior

Minimizing loss $+ \frac{\lambda}{2}\|\mathbf{w}\|^2$ is MAP estimation with a zero-mean Gaussian prior on the weights. A strong prior belief in small weights (small τ^2) or noisy data (large σ^2) means more regularization; more data (large N) means less.

For example, with $\sigma^2 = 1$, $\tau^2 = 10$ and $N = 1$, $\lambda = 0.1$. With $N = 1000$ examples the same beliefs give $\lambda = 10^{-4}$: the data overwhelm the prior.

12.3.2 L1 regularization and sparsity

Replace the squared norm with the sum of absolute values:

$$J(\mathbf{w}) = L(\mathbf{w}) + \lambda \|\mathbf{w}\|_1 = L(\mathbf{w}) + \lambda \sum_i |w_i| .$$

Repeating the MAP argument with a **Laplace prior** $p(w_j) \propto e^{-|w_j|/b}$, which has a sharp peak at zero and heavier tails than a Gaussian, produces this penalty. Its key property is **sparsity**: L1 drives many weights to *exactly* zero, whereas L2 only makes them small.

Why exactly zero? The soft threshold. Take the simplest case, a single weight with loss $\frac{1}{2}(w-a)^2$, where a is the unregularized optimum. With L2, $J = \frac{1}{2}(w-a)^2 + \frac{\lambda}{2}w^2$. Setting $J' = w - a + \lambda w = 0$ gives $w = a/(1+\lambda)$, so every weight shrinks by the same factor and none becomes exactly zero. With L1, $J = \frac{1}{2}(w-a)^2 + \lambda|w|$. For $w > 0$, setting $J' = w - a + \lambda = 0$ gives $w = a - \lambda$, valid only if $a > \lambda$. For $w < 0$, $J' = w - a - \lambda = 0$ gives $w = a + \lambda$, valid only if $a < -\lambda$. If $|a| \le \lambda$, neither works, and the minimum is at the kink $w = 0$. Altogether:

$$w^\star = S_\lambda(a) = \operatorname{sign}(a)\ \max\big(\,|a| - \lambda,\ 0\big), \tag{12.3}$$

the **soft-threshold** operator. It sets small weights to exactly zero and shrinks large ones by λ. For example, with $\lambda = 0.3$, $(1.0,\ 0.2,\ -0.5,\ -0.1) \mapsto (0.7,\ 0,\ -0.2,\ 0)$.

Figure 12.5 shows the geometry. Minimizing loss + penalty is equivalent to minimizing the loss inside a ball $\|\mathbf{w}\| \le t$. The L2 ball is round, so the loss contours usually touch it at a point where both coordinates are non-zero. The L1 ball is a diamond with corners on the axes, and the contours tend to touch it first at a corner, where some coordinates are exactly zero.

Wireless connection: sparsity in wireless: compressed sensing and pruning

Soft thresholding is the core step of ISTA/LASSO solvers used for sparse channel estimation, in which a wideband or mmWave channel has only a few dominant paths in the delay or angle domain. Unfolding those iterations into network layers ("LISTA", learned ISTA) is a standard deep-learning-for-PHY technique. L1 penalties on weights or whole channels also give *pruned* networks with many exact zeros, which matters when a neural receiver must fit the memory and MAC budget of a baseband DSP.

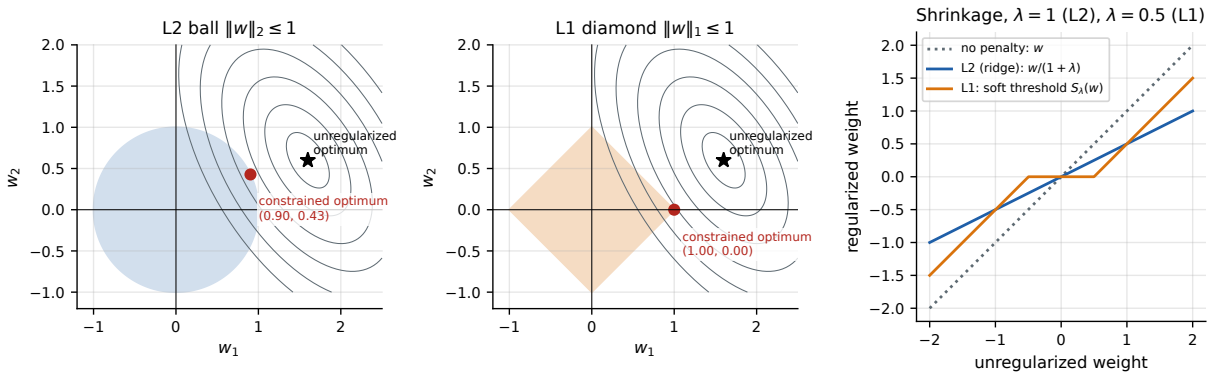


Figure 12.5: Left and middle: contours of a quadratic loss whose unconstrained optimum (star) lies outside the constraint set. With the L2 ball the constrained optimum is $(0.90,\ 0.43)$, so both weights survive, shrunk. With the L1 diamond it is $(1.00,\ 0.00)$, a corner: w_2 is exactly zero. Right: the resulting shrinkage rules for one weight, uniform scaling $w/(1+\lambda)$ for L2 versus the soft threshold $S_\lambda(w)$ of Eq. (12.3) for L1, which is flat (exactly zero) for $|w| \le \lambda$. Script: `code/ch12/fig_regularization.py`.

12.3.3 Dropout

Intuition: a team that cannot rely on any one member

Imagine a team in which, every day, a random half of the members stay home. The team cannot depend on any single star; every member must be useful on their own, and knowledge must be spread redundantly. Dropout does this to neurons: during training, each hidden unit is switched off at random, so no unit can rely on a particular partner being present. The network is forced to learn redundant, robust features instead of fragile co-adaptations.

Definition: inverted dropout

During training, for each hidden activation a_j draw an independent mask $m_j \sim \text{Bernoulli}(1-p)$ ($m_j = 1$ with probability $1-p$) and output

$$\tilde{a}_j = \frac{m_j\, a_j}{1-p}.$$

p is the **drop probability** (typically 0.1–0.5). At test time dropout is switched off: $\tilde{a}_j = a_j$.

Why divide by $1-p$? So that the expected activation is unchanged:

$$\mathbb{E}[\tilde{a}_j] = \frac{a_j}{1-p}\,\mathbb{E}[m_j] = \frac{a_j}{1-p}(1-p) = a_j.$$

The next layer therefore sees inputs of the same average size during training and testing, and no rescaling is needed at test time. (The original formulation scaled the weights by $1-p$ at test time instead; "inverted" dropout moves the scaling into training.) For example, with $p = 0.5$, activations $(2, 4, 6, 8)$ and mask $(1, 0, 1, 0)$, the output is $(4, 0, 12, 0)$.

The ensemble view. Each mask defines a different "thinned" sub-network, and a network with n droppable units has 2^n of them, all sharing weights. Training with dropout trains a huge ensemble of these sub-networks, each on a few mini-batches. Using the full network with expected activations at test time approximates averaging the ensemble's predictions (Srivastava

et al., 2014). Averaging many models reduces variance, which is the core of dropout's regularizing effect. Keeping dropout on at test time and averaging several stochastic forward passes (*Monte Carlo dropout*) gives a rough estimate of predictive uncertainty.

Pitfall: train versus eval mode

Dropout (and BatchNorm, below) behaves differently in training and evaluation. In PyTorch, call `model.train()` before training and `model.eval()` before validation or inference. Forgetting `eval()` gives noisy, pessimistic validation scores; forgetting `train()` after validation silently disables regularization.

12.3.4 Early stopping

During long training, validation loss typically falls, reaches a minimum, and then rises as the network starts memorizing (Fig. 12.6, left). **Early stopping** monitors the validation loss after each epoch, saves the model whenever it improves, and stops after the loss has not improved for a set number of epochs (the *patience*). The saved best model is the one we keep. Early stopping is free, needs no change to the model, and limits how far the weights can travel from their small initial values. For linear models with quadratic loss it can be shown to act much like L2 regularization, with the number of steps playing the role of $1/\lambda$.

Figure 12.6 shows a 64–256–256–10 MLP trained with AdamW on only 200 digit images, 20% of whose labels we deliberately corrupted to make memorization tempting. Without regularization the training loss goes to 0.002 (the network memorizes even the wrong labels). The validation loss bottoms out at 0.52 at epoch 34 (accuracy 87.5%) and then climbs to 1.19, while validation accuracy drops to 73.1%. Early stopping at epoch 34 would have kept the 87.5%. Strong weight decay ($\lambda = 5$ in AdamW) keeps the final validation loss at 0.64 (accuracy 85.9%), and dropout with $p = 0.5$ reaches the lowest validation loss of all (0.44 at epoch 127), though it too drifts upward with very long training.

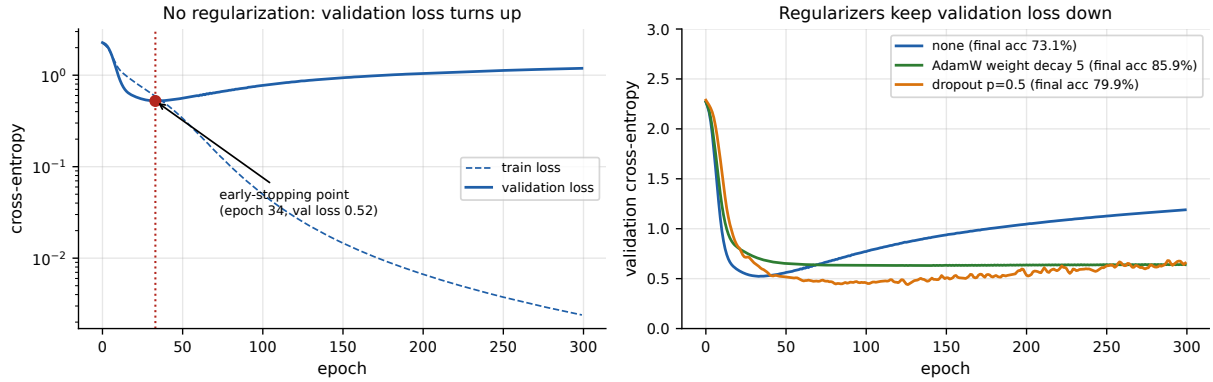


Figure 12.6: A deliberately overfitting setting: 200 training digits with 20% wrong labels, 1597 clean validation digits. Left: without regularization the training loss (dashed) keeps falling while the validation loss turns upward after epoch 34; early stopping would return the model at the red dot. Right: validation loss with no regularization, strong decoupled weight decay, and dropout ($p = 0.5$). Both regularizers prevent most of the late-training damage; final validation accuracies are in the legend. Script: `code/ch12/fig_regularization.py`.

12.3.5 Data augmentation

The most reliable cure for overfitting is more data. When collecting it is expensive, we can create new training examples by applying transformations that change the input but not the label. These encode our prior knowledge of which variations the model should ignore.

Images: random crops and translations, horizontal flips (not for digits, where a flipped 2 is not a 2), small rotations and scalings, colour and brightness jitter, added noise, cutout, and mixup (blending two images and their labels).

Wireless signals: the physics of the channel tells us exactly which transformations preserve the label (the modulation type, the transmitted bits after appropriate relabelling, or the presence of a signal), as Fig. 12.7 shows.

- **Random phase rotation** $r[n] \to r[n]e^{j\phi}$, $\phi \sim U[0, 2\pi)$: an unsynchronized receiver sees an arbitrary carrier phase. For symmetric constellations, rotations by multiples of 90° (QAM) or $360^\circ/M$ (PSK) also map the constellation to itself.
- **SNR variation**: add extra noise to reach a random target SNR, so that one clean recording yields training data at many SNRs.
- **Carrier frequency offset (CFO)**: multiply by $e^{j2\pi\Delta f\, n}$ with a random small Δf, which makes the constellation rotate over time.
- **Time shift and timing offset**: circularly shift or crop the burst; resample with a small fractional delay.
- **Channel augmentation**: convolve with randomly drawn multipath channels (e.g. from 3GPP TDL/CDL models) or apply Rayleigh/Rician fading gains, so that the model sees many propagation conditions.
- **Hardware impairments**: IQ imbalance, DC offset, phase noise and PA non-linearity drawn at random.

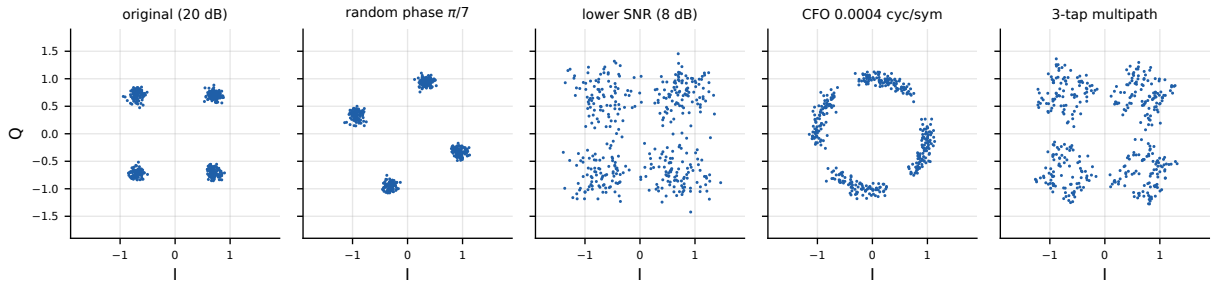


Figure 12.7: Label-preserving augmentations of one QPSK burst (400 symbols): the original at 20 dB SNR; a random phase rotation; extra noise down to 8 dB; a small carrier frequency offset (the constellation rotates during the burst and smears into arcs); and a 3-tap multipath channel (each point becomes a cluster from intersymbol interference). A modulation classifier trained on such variations learns to ignore them. Script: `code/ch12/fig_metrics.py`.

Wireless connection: augmentation versus the true test distribution

Augmentation is powerful but can mislead you if the augmented distribution does not match deployment. A modulation classifier trained only on simulated AWGN with random phase may still fail on real captures with phase noise and multipath. Two habits help. Hold out *real, un-augmented* recordings as the test set. And report results per impairment, not just on

average, so that you can see which conditions the model has actually learned. Part VI (the modulation-classification labs) does this systematically for AWGN, Rayleigh and IIR-type channels.

12.4 Normalization layers

Intuition: re-centring the inputs of every layer

In Chapter 5 we standardized input features (zero mean, unit variance) because gradient descent works much better on well-scaled inputs: the loss bowl is rounder, with a smaller condition number (Chapter 9). But the inputs of layer 5 are the outputs of layer 4, and their scale drifts during training as the earlier weights change. Normalization layers re-standardize the inputs of every layer on the fly, so each layer always sees well-scaled numbers.

12.4.1 Batch normalization

Batch normalization (Ioffe and Szegedy, 2015) normalizes each feature (each unit, or each channel in a CNN) using the statistics of the current mini-batch.

Key equation: BatchNorm, training mode

For one feature with values $x_1, \dots, x_B$ in a mini-batch of size B:

$$\begin{aligned} \mu_{\mathcal{B}} &= \frac{1}{B}\sum_{i=1}^{B} x_i, & \sigma_{\mathcal{B}}^2 &= \frac{1}{B}\sum_{i=1}^{B}(x_i-\mu_{\mathcal{B}})^2, \\ \hat{x}_i &= \frac{x_i-\mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2+\epsilon}}, & y_i &= \gamma\,\hat{x}_i+\beta. \end{aligned} \tag{12.4}$$

ϵ (e.g. 10^{-5}) avoids division by zero. γ (scale) and β (shift) are *learned* parameters, one pair per feature, initialized to $\gamma = 1$, $\beta = 0$.

Why the learnable γ and β? Forcing every feature to have exactly zero mean and unit variance could limit what the network can represent. A sigmoid unit, for instance, would always operate in its near-linear middle region. With γ and β, the network can choose any mean and scale, including undoing the normalization ($\gamma = \sigma_{\mathcal{B}}$, $\beta = \mu_{\mathcal{B}}$) if that helps. What it gains is that the scale is now controlled by two explicit parameters instead of by a delicate interplay of all the weights below.

Worked example: BatchNorm on a batch of four

One feature takes the values $\mathbf{x} = [1, 2, 3, 6]$ in a mini-batch of $B = 4$.
Mean: $\mu_{\mathcal{B}} = (1+2+3+6)/4 = 12/4 = 3$.
Variance (divide by B, not $B-1$): deviations $-2, -1, 0, 3$; squares $4, 1, 0, 9$; $\sigma_{\mathcal{B}}^2 = 14/4 = 3.5$.
Normalize: $\sqrt{3.5 + 10^{-5}} = 1.8708$, so

$$\hat{\mathbf{x}} = \left[\tfrac{-2}{1.8708}, \tfrac{-1}{1.8708}, \tfrac{0}{1.8708}, \tfrac{3}{1.8708}\right] = [-1.069,\ -0.535,\ 0,\ 1.604].$$

Check: the mean is 0 and the variance is $(1.143 + 0.286 + 0 + 2.571)/4 = 1.000$ (to rounding).
Scale and shift: with learned $\gamma = 2$, $\beta = 0.5$, $\mathbf{y} = 2\hat{\mathbf{x}} + 0.5 = [-1.638,\ -0.569,\ 0.5,\ 3.707]$.

Train versus evaluation statistics. At test time we may predict a single example, so batch statistics are unavailable, and the prediction for one example should not depend on which other examples happen to share its batch. During training, BatchNorm therefore keeps **running averages** of the mean and variance:

$$\mu_{\text{run}} \leftarrow (1-\alpha)\,\mu_{\text{run}} + \alpha\,\mu_{\mathcal{B}}, \qquad \sigma^2_{\text{run}} \leftarrow (1-\alpha)\,\sigma^2_{\text{run}} + \alpha\,\sigma^2_{\mathcal{B}},$$

with momentum $\alpha = 0.1$ in PyTorch, which also uses the unbiased batch variance (divided by $B-1$) for the running estimate. In `eval()` mode these fixed running statistics replace $\mu_{\mathcal{B}}$ and $\sigma^2_{\mathcal{B}}$, so BatchNorm becomes a fixed affine map. Starting from $\mu_{\text{run}} = 0$, $\sigma^2_{\text{run}} = 1$, the batch above updates them to $0.9(0) + 0.1(3) = 0.3$ and $0.9(1) + 0.1(14/3) = 1.367$.

Why it helps. BatchNorm allows much larger learning rates, makes training far less sensitive to initialization, and often regularizes slightly, because the batch statistics add noise that depends on which examples share the batch. The original paper attributed the benefit to reducing "internal covariate shift". Later analysis (Santurkar et al., 2018) argued that the main effect is a smoother loss landscape with better-behaved gradients. Either way, it was the key that made very deep CNNs easy to train.

Pitfall: BatchNorm with small or unusual batches

With very small batches (2–8, common for large inputs on limited memory), batch statistics are noisy and BatchNorm hurts. It also couples examples within a batch, which is awkward for recurrent networks and for sequence models with variable lengths. And if the test data come from a different distribution (a new SNR range, a new radio front-end), the running statistics are wrong for them. Re-estimating them on unlabelled target data is a simple form of domain adaptation.

12.4.2 Layer normalization

Layer normalization (Ba, Kiros and Hinton, 2016) normalizes across the *features of one example* instead of across the batch for one feature. For an example with features $x_1, \dots, x_d$:

$$\mu = \frac{1}{d}\sum_{j=1}^{d} x_j, \qquad \sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(x_j-\mu)^2, \qquad y_j = \gamma_j \frac{x_j - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_j.$$

It is independent of the batch, behaves identically in training and evaluation, and works for any batch size and sequence length. That is why it is the normalization of choice in Transformers and RNNs. If the vector $[1, 2, 3, 6]$ above were the four features of *one* example, LayerNorm would produce the same $\hat{\mathbf{x}}$; the difference lies only in which axis the statistics are taken over (Fig. 12.8).

12.5 Splitting data honestly

Definition: training, validation and test sets

- **Training set**: used to fit the parameters (weights) by gradient descent.
- **Validation set** (development set): used to choose hyperparameters (architecture, learning rate, regularization, epoch for early stopping) and to compare models.

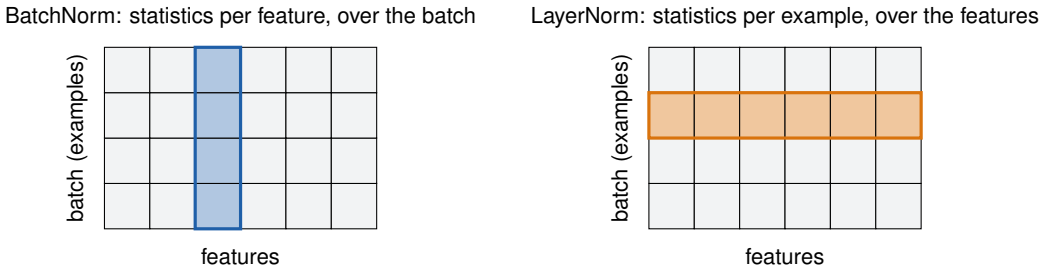


Figure 12.8: Which numbers are pooled to compute the mean and variance. BatchNorm (left, blue) normalizes each feature column across the examples of the mini-batch. LayerNorm (right, orange) normalizes each example's row across its features. CNN variants (GroupNorm, InstanceNorm) pool over other combinations of channels and spatial positions.

- **Test set**: used *once*, at the very end, to report the expected performance on new data. It must not influence any decision.

Typical splits are 70/15/15 or 80/10/10. With millions of examples, 98/1/1 can be enough.

Why three sets and not two? Every time you look at a score and make a decision based on it, you fit a little to the data that produced it. If you tried 50 hyperparameter settings and kept the best validation score, that score is optimistically biased. It is the maximum of 50 noisy estimates. The untouched test set gives an unbiased final number.

***k*-fold cross-validation.** When data are scarce, setting aside a validation set wastes precious examples and gives a noisy estimate. In k-fold cross-validation, split the training data into k equal *folds* (typically $k = 5$ or 10). Train k models, each on $k - 1$ folds, validating on the remaining one, and average the k validation scores (Fig. 12.9). Every example is used for validation exactly once. The spread of the k scores also gives an idea of the uncertainty. The cost is k trainings, so this is common for small models and rare for large networks.

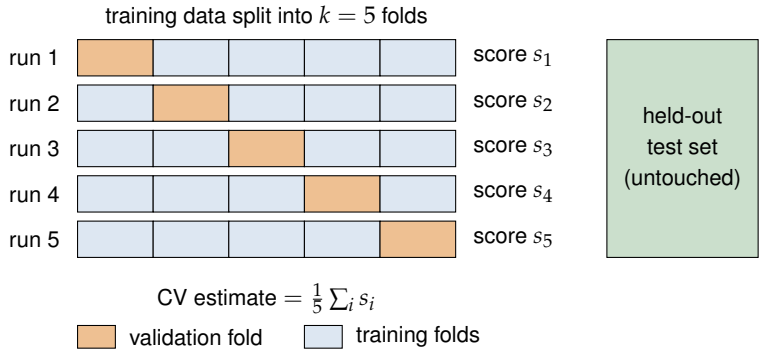


Figure 12.9: 5-fold cross-validation. Each run trains on four folds (blue) and validates on the fifth (orange); the five scores are averaged. The test set (green) stays outside the whole procedure.

Stratification. With imbalanced classes, a random split may leave a fold with very few examples of a rare class. *Stratified* splitting samples each class separately so that every split keeps the overall class proportions (`sklearn.model_selection.StratifiedKFold`). In wireless datasets, stratify by SNR as well as by class, so that every split covers the full SNR range.

Pitfall: data leakage

Leakage is any route by which information about the validation or test data reaches training. It produces wonderful scores that collapse in deployment. Common forms:

- **Preprocessing on all data**: computing normalization statistics, PCA or feature selection on the full dataset before splitting. Fit them on the training set only, then apply them to the others.
- **Duplicates and near-duplicates**: the same image or burst in both train and test sets. Overlapping windows cut from one long I/Q recording are near-duplicates. If windows are split at random, the test set contains slices of the very same transmissions the model trained on.
- **Temporal leakage**: random splits of time series let the model "see the future". For channel prediction, split by time (train on earlier, test on later) or by recording session.
- **Group leakage**: samples from the same transmitter device, user, location or measurement campaign on both sides of the split. If the goal is to generalize to new devices or sites, split by device or site (`GroupKFold`).
- **Tuning on the test set**: looking at test scores repeatedly while developing. Then the test set has silently become a validation set.

12.6 Hyperparameter tuning

Hyperparameters, such as the learning rate, weight decay, dropout rate, width, depth and batch size, are chosen by training models with different settings and comparing validation scores. Three rules make this efficient.

1. Search scale parameters on a log scale. The difference between learning rates 10^{-4} and 10^{-3} matters as much as that between 10^{-2} and 10^{-1}. Sample the *exponent* uniformly: $\eta = 10^{u}$ with $u \sim U[-5, -1]$. The same goes for weight decay and Adam's ϵ. Dropout rates and momentum are fine on a linear scale (or as $1 - \mu$ on a log scale for momentum close to 1).

2. Prefer random search to grid search. Bergstra and Bengio (2012) observed that usually only a few hyperparameters matter much, but we do not know in advance which ones. A 3×3 grid spends 9 trials but tries only *3 distinct values* of each hyperparameter. If only one of them matters, six trials are wasted. Nine random trials try *9 distinct values* of every hyperparameter (Fig. 12.10).

3. Go coarse to fine, and stop bad runs early. Start with a wide random search of short runs, narrow the ranges around the best results, then train the most promising settings fully. Methods such as successive halving and Hyperband stop unpromising runs early, and Bayesian optimization (e.g. the Optuna library) chooses the next trial based on previous results. Tune the learning rate first: it is almost always the most important hyperparameter (Chapter 9).

12.7 Classification metrics

12.7.1 The confusion matrix and the scores built from it

For a binary problem (positive = the class we want to detect: a signal present, a fault, an intruder), every prediction falls into one of four cells:

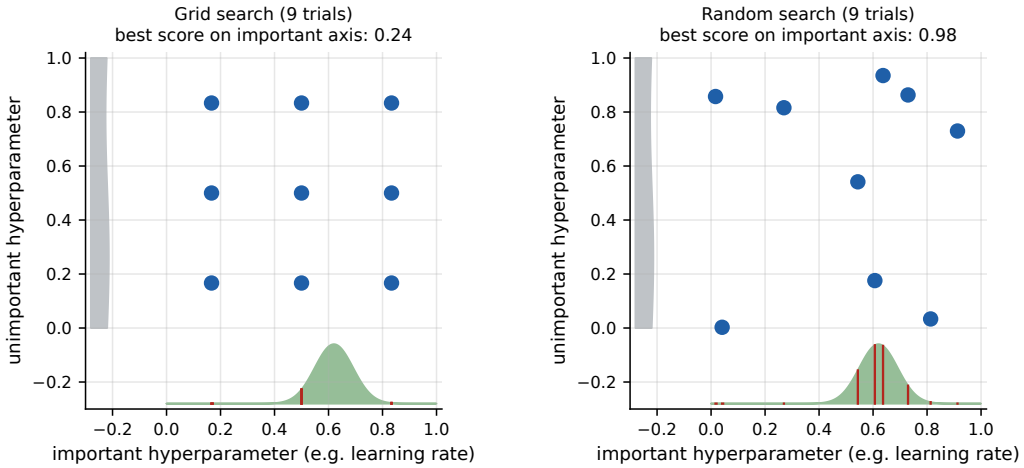


Figure 12.10: Why random search beats grid search when only one hyperparameter matters. The green curve below each panel is the validation score as a function of the important hyperparameter, with a narrow peak; the grey band at the left shows that the other hyperparameter barely matters. A 3×3 grid (left) tests only three values of the important parameter and misses the peak (best score 0.24). Nine random trials (right) test nine distinct values and land near the peak (0.98). Script: `code/ch12/fig_metrics.py`.

	predicted positive	**predicted negative**
actually positive	TP (true positive)	FN (false negative, a miss)
actually negative	FP (false positive, a false alarm)	TN (true negative)

Definition: accuracy, precision, recall, F1

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{precision} = \frac{TP}{TP + FP},$$
$$\text{recall (sensitivity, TPR)} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Precision asks: *of the alarms I raised, how many were real?* Recall asks: *of the real positives, how many did I catch?* F_1 is their harmonic mean. Also useful: specificity $= TN/(TN + FP)$, and the false positive rate FPR $= FP/(FP + TN) = 1 - \text{specificity}$.

Worked example: computing the four scores

A detector is tested on 100 cases with $TP = 40$, $FP = 10$, $FN = 20$, $TN = 30$.

$$\text{accuracy} = \frac{40 + 30}{100} = 0.70, \qquad \text{precision} = \frac{40}{40 + 10} = 0.80,$$
$$\text{recall} = \frac{40}{40 + 20} = 0.667, \qquad F_1 = \frac{2(0.80)(0.667)}{0.80 + 0.667} = \frac{1.067}{1.467} = 0.727.$$

Also, specificity $= 30/40 = 0.75$ and FPR $= 0.25$. The detector is fairly trustworthy when it fires (80%), but it misses one positive in three.

Pitfall: accuracy on imbalanced data

If 99% of the examples are negative, a model that always says "negative" has 99% accuracy and zero recall. With imbalanced classes, report precision, recall, F_1 (or balanced accuracy, the average recall over classes) and the full confusion matrix. For K classes the confusion matrix is $K \times K$, and per-class precision and recall can be averaged equally over classes (*macro* average) or weighted by class frequency (*weighted* average).

12.7.2 Thresholds, ROC and PR curves

Most classifiers output a score or probability $\hat{p}$, and we predict "positive" when $\hat{p} \geq t$. The threshold t trades errors against each other: lowering t catches more positives (higher recall) at the price of more false alarms. The **ROC curve** (receiver operating characteristic, a term from radar) plots TPR against FPR as t sweeps from 1 to 0. The **area under the curve** (AUC) summarizes it in one number: 1.0 is perfect and 0.5 is random guessing. AUC also has a direct interpretation: it is the probability that a randomly chosen positive gets a higher score than a randomly chosen negative.

When positives are rare, the ROC curve can look excellent while precision is poor, because FPR divides by the large number of negatives. The **precision–recall curve** then shows the trade-off more honestly. Its baseline is the positive-class prevalence, not 0.5, and its area is called the *average precision* (AP).

Calibration. A model is **calibrated** if, among all cases where it predicts probability 0.8, about 80% are actually positive. Calibration matters whenever probabilities are used downstream: soft information passed to a channel decoder, risk-based decisions, or combining models. A *reliability diagram* bins the predictions and plots the observed frequency against the mean predicted probability. The *expected calibration error* (ECE) is the bin-weighted average gap. Modern deep networks trained with cross-entropy are often *over-confident*. A simple fix is *temperature scaling*: divide the logits by a scalar $T > 1$ fitted on the validation set (Guo et al., 2017).

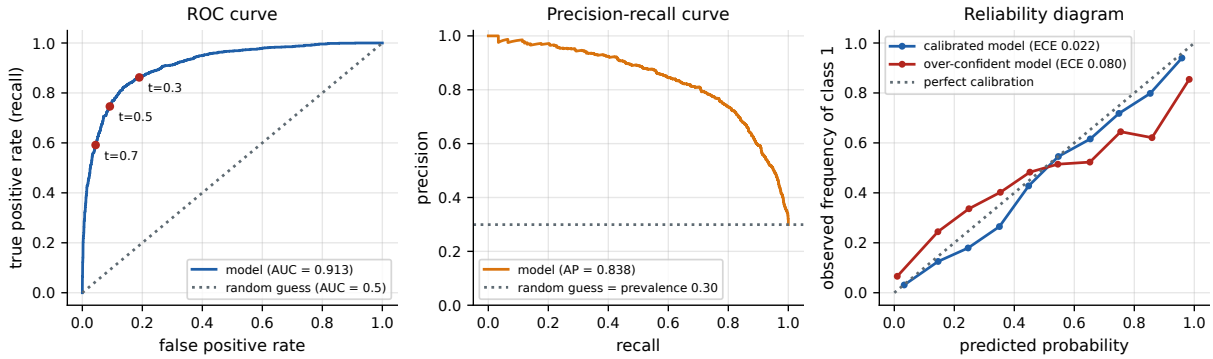


Figure 12.11: Threshold-free views of a binary classifier (4000 synthetic cases, 30% positive). Left: ROC curve, AUC $= 0.914$; the dots mark thresholds $t = 0.3, 0.5, 0.7$, and moving along the curve trades recall for false alarms. Middle: precision–recall curve, AP $= 0.839$; the dotted line is the prevalence baseline. Right: reliability diagram for the true-posterior (calibrated) model (ECE 0.022) and an over-confident version with sharpened logits (ECE 0.081). Both models have *identical* ROC and PR curves, because sharpening preserves the ranking, so calibration has to be checked separately. Script: `code/ch12/fig_metrics.py`.

12.8 Regression metrics

For real-valued targets with predictions $\hat{y}_i$:

- **MSE** $= \frac{1}{N}\sum(y_i - \hat{y}_i)^2$ and **RMSE** $= \sqrt{\text{MSE}}$ (in the units of y). Both are sensitive to outliers because errors are squared.
- **MAE** $= \frac{1}{N}\sum|y_i - \hat{y}_i|$, which is more robust to outliers.
- $R^2 = 1 - \text{MSE}/\operatorname{Var}(y)$: the fraction of the target's variance explained. $R^2 = 0$ means no better than predicting the mean; it can be negative.
- **NMSE** $= \sum\|\mathbf{y}_i - \hat{\mathbf{y}}_i\|^2 / \sum\|\mathbf{y}_i\|^2$, usually quoted in dB. This is the standard metric for channel estimation and CSI prediction (Chapter 28): -20 dB NMSE means the error energy is 1% of the channel energy.

12.9 Wireless metrics

A neural receiver is ultimately judged by communication metrics, not by its cross-entropy.

Bit and block error rates. The **BER** is the fraction of bits decoded wrongly, and the **BLER** is the fraction of transport blocks (codewords) that fail their CRC. 5G link adaptation typically targets a BLER of 10% for the initial transmission, and that is the operating point at which receiver gains are usually quoted. Both are plotted against SNR or E_b/N_0 on a logarithmic error axis (Fig. 12.12, left), and improvements are read *horizontally* as dB gain at a fixed error rate. Two statistical cautions apply. To estimate an error rate P reliably you need on the order of $100/P$ trials (about 100 errors). And the test data must be generated with *fresh* random bits, noise and channels, never reused from training.

Accuracy versus SNR. For classification tasks (modulation recognition, signal detection), a single accuracy averaged over all SNRs hides what matters. Classifiers are near chance at very low SNR and near perfect at high SNR, so the average depends mostly on which SNR range was included. Always plot accuracy per SNR (Fig. 12.12, middle). Two illustrative classifiers can have nearly the same average (54.7% versus 56.8%), yet one is much better at low SNR (33% versus 20% at -6 dB) and the other at high SNR (95.5% versus 89% at 10 dB). Which is better depends on where your system operates.

Detection and false alarm. Spectrum sensing, preamble detection, PRACH detection and intrusion or rogue-transmitter detection are binary hypothesis tests: $\mathcal{H}_0$ (noise only) versus $\mathcal{H}_1$ (signal present). The radio-engineering names for the metrics of Section 12.7 are

$$P_D = \Pr(\text{decide } \mathcal{H}_1 \mid \mathcal{H}_1) = \text{recall (TPR)}, \qquad P_{FA} = \Pr(\text{decide } \mathcal{H}_1 \mid \mathcal{H}_0) = \text{FPR},$$

and the ROC curve, which plots P_D against P_{FA}, originated in exactly this setting. The usual design rule (Neyman–Pearson) fixes the false-alarm rate, for example $P_{FA} = 0.1$ for spectrum sensing in IEEE 802.22 or a much smaller value for PRACH. One then maximizes P_D at that operating point, so a learned detector should be compared at the same P_{FA}, not by accuracy.

Worked example: the energy detector's ROC

The classical energy detector sums the energy of N complex samples, $T = \sum_{n=1}^{N}|r[n]|^2$, and compares it with a threshold γ. With unit-variance complex Gaussian noise, $2T$ follows a chi-squared distribution with $2N$ degrees of freedom under $\mathcal{H}_0$. Under $\mathcal{H}_1$, with a Gaussian-like

signal at SNR s, $2T/(1+s)$ follows the same distribution. So

$$P_{FA} = \Pr\left(\chi^2_{2N} > 2\gamma\right), \qquad P_D = \Pr\left(\chi^2_{2N} > 2\gamma/(1+s)\right).$$

For $N = 32$ and $P_{FA} = 0.1$, $\gamma = 39.4$; at $s = -5\,\text{dB}$ this gives $P_D = 0.622$, and a Monte Carlo check with 20 000 trials gives 0.621. At $-10\,\text{dB}$, P_D drops to 0.24; at 0 dB it is 0.993. A neural spectrum sensor earns its place only if its ROC lies above these curves under the same conditions.

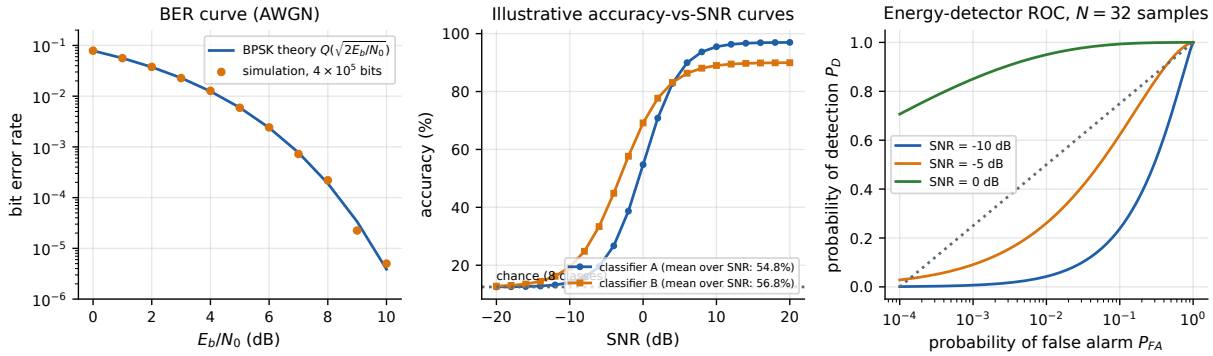


Figure 12.12: Wireless evaluation views. Left: BER of BPSK over AWGN, theory and a 4×10^5-bit simulation; they agree down to about 10^{-4}, below which too few errors are observed (at 10 dB, theory 3.9×10^{-6} predicts about 2 errors). Middle: two *illustrative* accuracy-versus-SNR curves with nearly equal averages but different strengths. Right: ROC of an energy detector with $N = 32$ samples at three SNRs (theory; the Monte Carlo values at $P_{FA} = 0.1$ match to within 0.001). Script: `code/ch12/fig_metrics.py`.

Wireless connection: a reporting checklist for learned PHY blocks

When you evaluate a neural receiver, detector or classifier, report: (1) the classical baseline under identical conditions (LMMSE or LS channel estimation, the energy or matched-filter detector, an ML or cumulant-based classifier); (2) curves versus SNR, not averages; (3) the operating-point metric that matters (BLER at 10%, P_D at a fixed P_{FA}, NMSE in dB); (4) generalization to channel models, Doppler and SNR ranges *not* seen in training; (5) confidence intervals or the number of error events; and (6) complexity (parameters, MACs, latency), because a 0.3 dB gain that costs $100\times$ the computation of an LMMSE receiver may not be worth it on a DSP.

Chapter recap

- Underfitting: high training error (high bias). Overfitting: low training error, large gap (high variance). Polynomials of degree 1/4/15 on 20 points give test MSE 0.32/0.07/52.
- Expected squared error = bias^2 + variance + σ^2. Capacity trades bias for variance. Double descent: past the interpolation threshold, test error can fall again.
- L2 = Gaussian prior (MAP), with update $(1-\eta\lambda)\mathbf{w} - \eta\nabla L$. L1 = Laplace prior, soft threshold, exact zeros. Dropout: mask and divide by $1-p$ so the expectation is preserved; an implicit ensemble. Early stopping and data augmentation (images: flips and crops; radio: phase, SNR, CFO, delay, channels).

- BatchNorm: normalize each feature with batch statistics, then $\gamma\hat{x} + \beta$; running statistics at evaluation. $[1, 2, 3, 6] \to$ mean 3, variance 3.5, $\hat{x} \approx [-1.069, -0.535, 0, 1.604]$. LayerNorm normalizes over the features of each example.
- Train/validation/test discipline, k-fold CV, stratification, and avoiding leakage (preprocessing, duplicates, time, groups). Random search on log scales.
- Metrics: from TP=40, FP=10, FN=20, TN=30, accuracy 0.70, precision 0.80, recall 0.667, F_1 0.727. ROC/AUC, PR/AP, calibration. Wireless: BER/BLER vs SNR, accuracy vs SNR, P_D at fixed P_{FA}.

Exercises

1. A model has training MSE 0.01 and validation MSE 0.40; another has 0.35 and 0.38. Which is underfitting and which overfitting? Suggest two remedies for each.
2. At a test point, 5 models trained on different datasets predict 1.8, 2.2, 2.0, 2.4, 1.6, the true $f(x) = 2.5$ and $\sigma^2 = 0.1$. Estimate bias^2, variance and the expected squared error.
3. With $\eta = 0.05$ and $\lambda = 10^{-3}$, what factor multiplies each weight per step from weight decay alone? After 10 000 steps with zero loss gradient, what fraction of a weight remains?
4. Apply the soft threshold with $\lambda = 0.4$ to $(0.9, -0.3, 0.4, -1.5)$, and the L2 shrinkage $w/(1 + \lambda)$ with $\lambda = 0.4$ to the same vector.
5. Inverted dropout with $p = 0.2$ on activations $(5, 10, 0, 15, 20)$ with mask $(1, 1, 0, 0, 1)$: what is the output? Show that the expected output equals the input.
6. BatchNorm on the batch $[2, 4, 4, 6]$ with $\gamma = 1$, $\beta = 0$: compute $\mu_{\mathcal{B}}$, $\sigma^2_{\mathcal{B}}$ and $\hat{\mathbf{x}}$.
7. A preamble detector is evaluated on 1000 slots, 100 containing a preamble. It raises 120 alarms, 90 of them correct. Fill in the confusion matrix and compute accuracy, precision, recall, F_1, P_D and P_{FA}.
8. (Conceptual) You cut 1-ms windows with 50% overlap from 20 long I/Q recordings and split the windows at random into train and test sets, obtaining 99% test accuracy. Why might this be misleading, and how should you split instead?

Answers

1. The first overfits (tiny training error, huge gap): add regularization (weight decay, dropout, early stopping), add or augment data, or reduce capacity. The second underfits (training error itself high): increase capacity (wider or deeper), train longer or with a better learning rate, reduce regularization, or add better features.
2. Mean prediction $\bar{f} = 10/5 = 2.0$; bias$^2 = (2.5 - 2.0)^2 = 0.25$. Variance $= \frac{1}{5}(0.04 + 0.04 + 0 + 0.16 + 0.16) = 0.08$. Expected error $= 0.25 + 0.08 + 0.1 = 0.43$.
3. $1 - \eta\lambda = 1 - 5 \times 10^{-5} = 0.99995$. After 10^4 steps: $0.99995^{10^4} \approx e^{-0.5} \approx 0.61$.
4. Soft threshold: $(0.5, 0, 0, -1.1)$; the 0.4 sits exactly at the threshold and becomes 0. L2: $(0.643, -0.214, 0.286, -1.071)$; nothing becomes zero.
5. Scale $1/(1 - 0.2) = 1.25$: output $(6.25, 12.5, 0, 0, 25)$. For each unit, $\mathbb{E}[\tilde{a}] =$

$1.25\,a\ \Pr(m=1) = 1.25\,a\,(0.8) = a$.

6. $\mu = 4$; deviations $-2, 0, 0, 2$; $\sigma^2 = 8/4 = 2$; $\hat{\mathbf{x}} = (-2, 0, 0, 2)/\sqrt{2} = (-1.414,\ 0,\ 0,\ 1.414)$.
7. $TP = 90$, $FP = 30$, $FN = 10$, $TN = 870$. Accuracy $= 960/1000 = 0.96$; precision $= 90/120 = 0.75$; recall $= P_D = 0.90$; $F_1 = 2(0.75)(0.9)/1.65 = 0.818$; $P_{FA} = 30/900 = 0.033$. The high accuracy is mostly due to the 900 easy negatives.
8. Overlapping windows from the same recording share half their samples, and all windows from one recording share the transmitter, channel, hardware and interference conditions. Random splitting puts near-duplicates of training windows into the test set (leakage), so the score measures memorization of 20 recordings, not generalization. Split by recording: put whole recordings, or better whole devices and sessions, in the test set, e.g. 16 recordings for training and 4 for testing, or use `GroupKFold` with the recording as the group.

Self-test quiz

1. Training error is low, validation error high, and the gap shrinks as you add data. What is the diagnosis?
   Overfitting (high variance); more data, augmentation or regularization will help.
2. Name the three terms of the bias–variance decomposition.
   Bias^2 (average model vs truth), variance (spread over training sets), irreducible noise σ^2.
3. Which prior corresponds to L2 regularization, and which to L1?
   Zero-mean Gaussian for L2; Laplace for L1 (which yields sparsity).
4. Why does inverted dropout divide by $1 - p$?
   So that the expected activation during training equals the activation used at test time.
5. What does BatchNorm use at evaluation time instead of batch statistics?
   Running (exponential moving) averages of the mean and variance collected during training.
6. A spectrum sensor must keep $P_{FA} \le 0.01$. How should two detectors be compared?
   By their P_D at $P_{FA} = 0.01$ (a point on the ROC curve), across the SNRs of interest, not by accuracy.

Further reading

- T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009, Ch. 3 and 7 (ridge, lasso, bias–variance, cross-validation).
- M. Belkin, D. Hsu, S. Ma, S. Mandal, "Reconciling modern machine-learning practice and the classical bias–variance trade-off," *PNAS*, 116(32), 2019.
- P. Nakkiran et al., "Deep double descent: where bigger models and more data hurt," *ICLR*, 2020.
- N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *JMLR*, 15, 2014.
- S. Ioffe, C. Szegedy, "Batch normalization: accelerating deep network training by reducing internal covariate shift," *ICML*, 2015.
- J. L. Ba, J. R. Kiros, G. E. Hinton, "Layer normalization," arXiv preprint 1607.06450, 2016.

- S. Santurkar, D. Tsipras, A. Ilyas, A. Madry, “How does batch normalization help optimization?” *NeurIPS*, 2018.
- J. Bergstra, Y. Bengio, “Random search for hyper-parameter optimization,” *JMLR*, 13, 2012.
- C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, “On calibration of modern neural networks,” *ICML*, 2017.
- S. M. Kay, *Fundamentals of Statistical Signal Processing, Vol. II: Detection Theory*, Prentice Hall, 1998 (Neyman–Pearson, ROC, energy detection).

Part III

Building Networks Yourself

Chapter 13

Lab: A Neural Network from Scratch in NumPy

Up to now you have built a neural network on paper: the forward pass in Chapter 7, the loss in Chapter 8, gradient descent in Chapter 9 and backpropagation in Chapter 10. In this lab you turn all of it into about 140 lines of plain NumPy that train a real classifier. Nothing is hidden: there is no framework, no automatic differentiation, no magic. Every line corresponds to an equation you have already derived, and we will point at that equation each time.

By the end of the chapter you will have:

- a reusable file `code/ch13/nn_numpy.py` implementing a multilayer perceptron (MLP) with any number of ReLU hidden layers, He initialisation, a numerically stable softmax cross-entropy output, L2 regularisation, mini-batch stochastic gradient descent and a prediction function;
- a **numerical gradient check** — the single most important debugging tool in this book — that proves your backward pass is correct to about ten significant digits;
- a network that separates three interleaved spirals that no straight line can separate;
- a set of controlled experiments (remove the nonlinearity, use a huge learning rate, break the backward pass on purpose, change the width) whose outcomes you will learn to predict;
- a wireless mini-lab in which the same code learns a 16-QAM detector that beats the textbook nearest-point detector when the receiver suffers from residual phase noise.

Why bother, when PyTorch (Chapter 14) does all of this in ten lines? Because when a PyTorch model misbehaves — and it will — you need to know what is happening underneath. Engineers who once wrote a backward pass by hand debug faster, choose learning rates more sensibly and recognise symptoms ("the loss is stuck at 1.0986") instantly. This is the same reason a radio engineer writes one FFT by hand before trusting a library.

13.1 The plan: data in, gradients out

Intuition: a factory with a quality inspector

Think of the network as a factory line. Raw material (a point (x_1, x_2)) enters on the left and passes through stations (layers). Each station multiplies by a matrix, adds a bias and squashes negative numbers to zero. At the end an inspector (the loss) compares the product with the order (the label) and writes a complaint. The complaint is then sent back up the line, station by station (backpropagation), and every station adjusts its machine a little (gradient descent) so that next time the complaint is smaller. A gradient check is an *external auditor* who verifies, by trying tiny adjustments one at a time, that the complaints sent back up the line are correct.

Figure 13.1 shows the whole program as a data-flow diagram. The forward pass stores

("caches") every intermediate result, because the backward pass needs them. Keep this picture in front of you while reading the code.

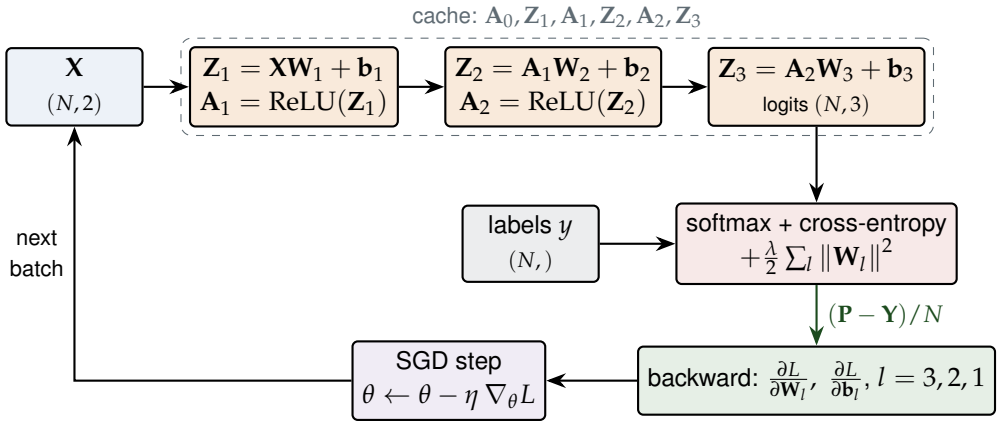


Figure 13.1: Data flow of the NumPy network for the spiral problem (two hidden layers). Solid arrows on top: the forward pass, which also fills the cache (dashed box). The loss produces the starting gradient $(\mathbf{P} - \mathbf{Y})/N$; the backward pass (green) reads the cached activations to produce one gradient per parameter; the SGD step updates the parameters and the loop repeats with the next mini-batch.

The file is organised into a handful of small functions. Figure 13.2 shows who calls whom. Each function is tested separately and each has a one-line mathematical meaning.

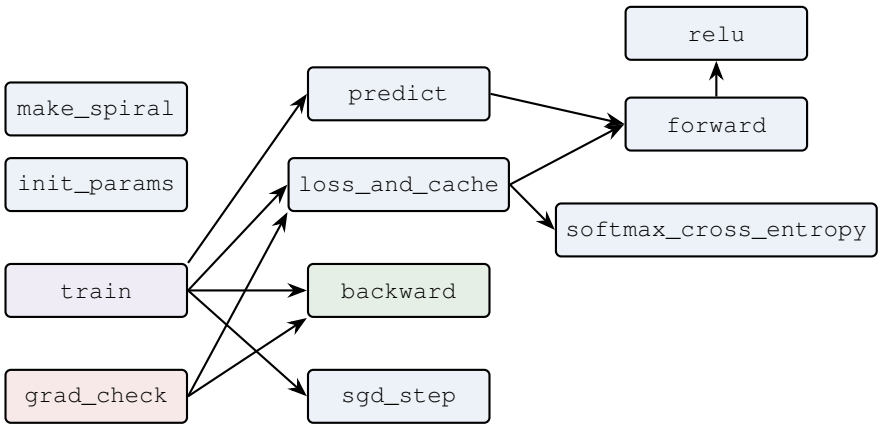


Figure 13.2: Call graph of `nn_numpy.py`. `train` repeatedly calls `loss_and_cache` (forward pass plus loss), `backward` and `sgd_step`. The auditor `grad_check` calls the same loss function many times and compares with `backward`. Data and parameters are created by `make_spiral` and `init_params`.

Conventions. We use the *batch-first* convention of Chapter 1: a mini-batch of N examples with d features is a matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$, one example per row. A layer with d_{in} inputs and d_{out} outputs has $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$, and computes $\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{b}$, where adding the row vector $\mathbf{b}$ to every row is NumPy *broadcasting*. Parameters live in a Python dictionary with keys `"W1"`, `"b1"`, `"W2"`, ... so that a network of any depth is just a longer dictionary.

13.2 The data: three interleaved spirals

A good test problem must be impossible for a linear model yet small enough to plot. The classic choice is the three-arm spiral popularised by Andrej Karpathy's Stanford CS231n notes.

Listing 13.1: `make_spiral`: three noisy spiral arms.

```
9  def make_spiral(n_per_class=100, n_classes=3, noise=0.2, seed=0):
10     rng = np.random.default_rng(seed)
11     N = n_per_class * n_classes
12     X = np.zeros((N, 2))
13     y = np.zeros(N, dtype=int)
14     for c in range(n_classes):
15         idx = np.arange(n_per_class * c, n_per_class * (c + 1))
16         r = np.linspace(0.0, 1.0, n_per_class)                  # radius
17         t = (np.linspace(c * 4, (c + 1) * 4, n_per_class)       # angle
18              + rng.normal(0.0, noise, n_per_class))
19         X[idx] = np.c_[r * np.sin(t), r * np.cos(t)]
20         y[idx] = c
21     return X, y
```

Line by line:

- **Line 10** creates a random-number generator with a fixed seed, so every run produces exactly the same data (reproducibility).
- **Lines 11–13** allocate the outputs: $N = 300$ points with two coordinates, and an integer label for each.
- **Line 15**: class c owns rows $100c$ to $100c + 99$.
- **Line 16**: the radius grows linearly from 0 to 1 along the arm: $r_i = i/99$ for $i = 0, \ldots, 99$.
- **Lines 17–18**: the angle sweeps 4 radians (about 229°), and each class starts where the previous one ended: class 0 covers $[0, 4]$, class 1 $[4, 8]$, class 2 $[8, 12]$ radians. Gaussian noise with standard deviation 0.2 rad makes the arms fuzzy, so the classes slightly overlap.
- **Line 19** converts polar to Cartesian coordinates, $x_1 = r \sin t$, $x_2 = r \cos t$; `np.c_` glues the two columns side by side.

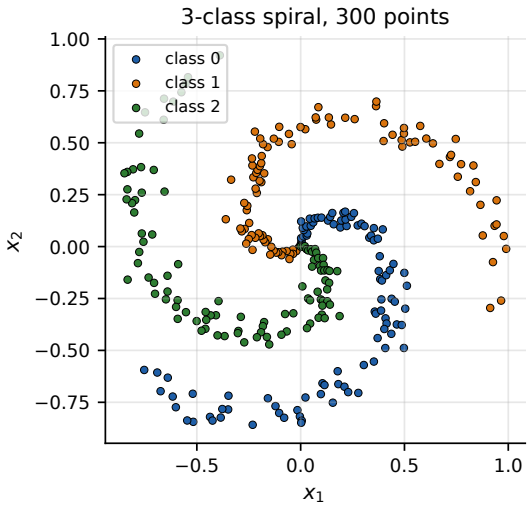


Figure 13.3: The training set: 100 points per class. Follow one colour from the centre outwards: each arm winds around the origin, so any straight line through the plane cuts every arm several times. Note also that the first point of every class is *exactly* the origin $(0, 0)$ (radius zero) — this innocent detail will matter in the gradient check.

13.3 Parameters: He initialisation

Listing 13.2: `init_params`: one weight matrix and one bias vector per layer.

```
25 def init_params(layer_sizes, seed=0):
26     """He initialisation: W ~ N(0, 2/fan_in), b = 0."""
27     rng = np.random.default_rng(seed)
28     params = {}
29     for l in range(1, len(layer_sizes)):
30         fan_in, fan_out = layer_sizes[l - 1], layer_sizes[l]
31         params[f"W{l}"] = rng.standard_normal((fan_in, fan_out)) * np.sqrt(2.0 /
       fan_in)
32         params[f"b{l}"] = np.zeros(fan_out)
33     return params
```

The argument `layer_sizes` is a list such as `[2, 100, 100, 3]`: 2 inputs, two hidden layers of 100 neurons and 3 outputs (one score per class). The loop of line 29 visits $l = 1, 2, 3$. Line 31 draws every weight from a standard normal distribution and scales it by $\sqrt{2/\text{fan_in}}$; line 32 sets biases to zero. The network therefore has

$$\underbrace{2 \cdot 100 + 100}_{\text{layer 1}} + \underbrace{100 \cdot 100 + 100}_{\text{layer 2}} + \underbrace{100 \cdot 3 + 3}_{\text{layer 3}} = 300 + 10\,100 + 303 = 10\,703 \text{ parameters.}$$

Definition: He (Kaiming) initialisation

For a layer followed by ReLU, draw each weight independently as $W_{ij} \sim \mathcal{N}(0,\ 2/d_{\text{in}})$, where d_{in} ("fan-in") is the number of inputs to the layer, and set the biases to zero.

Why the factor 2? Take one pre-activation $z = \sum_{i=1}^{d_{\text{in}}} a_i W_i$ (bias zero). If the weights are independent of the inputs, have mean zero and variance σ_W^2, then

$$\text{Var}(z) = \sum_{i=1}^{d_{\text{in}}} \text{Var}(a_i W_i) = \sum_{i=1}^{d_{\text{in}}} \mathbb{E}[a_i^2]\,\mathbb{E}[W_i^2] = d_{\text{in}}\,\sigma_W^2\,\mathbb{E}[a^2].$$

(The middle step uses $\text{Var}(aW) = \mathbb{E}[a^2W^2] - (\mathbb{E}[aW])^2 = \mathbb{E}[a^2]\mathbb{E}[W^2] - 0$ for independent a, W with $\mathbb{E}[W] = 0$.) If the previous layer's pre-activations were symmetric around zero with variance v, then $a = \text{ReLU}(z_{\text{prev}})$ is zero half of the time and equal to z_{prev} the other half, so $\mathbb{E}[a^2] = \frac{1}{2}v$. Hence $\text{Var}(z) = d_{\text{in}}\sigma_W^2 \cdot \frac{1}{2}v$. To keep the variance the same from layer to layer ($\text{Var}(z) = v$) we need $d_{\text{in}}\sigma_W^2/2 = 1$, i.e. $\sigma_W^2 = 2/d_{\text{in}}$. The ReLU throws away half of the signal energy, and the factor 2 puts it back. For our first layer $d_{\text{in}} = 2$ gives standard deviation 1; for the second layer $d_{\text{in}} = 100$ gives $\sqrt{0.02} = 0.141$.

Pitfall: all-zero weights

Initialising all weights to zero looks tidy but is fatal: every hidden neuron computes the same thing, receives the same gradient and stays identical forever (the "symmetry" is never broken). Random initialisation is not optional.

13.4 The forward pass

Listing 13.3: `relu` and `forward`.

```
37 def relu(Z):
38     return np.maximum(0.0, Z)
39
40
41 def forward(params, X, act=relu):
42     L = len(params) // 2                        # number of layers
43     cache = {"A0": X}
44     A = X
45     for l in range(1, L + 1):
46         Z = A @ params[f"W{l}"] + params[f"b{l}"]
47         A = act(Z) if l < L else Z              # last layer: raw scores
48         cache[f"Z{l}"], cache[f"A{l}"] = Z, A
49     return A, cache                             # A = logits, shape (N, K)
```

- **Line 38**: $\mathrm{ReLU}(z) = \max(0, z)$ applied element by element to a whole matrix at once.
- **Line 42**: the dictionary holds two entries per layer, so the number of layers is half its length. This is what makes the code depth-agnostic.
- **Line 43**: we cache the input as $\mathbf{A}_0 = \mathbf{X}$; the gradient of $\mathbf{W}_1$ needs it.
- **Line 46** is the layer equation of Chapter 7: $\mathbf{Z}_l = \mathbf{A}_{l-1}\mathbf{W}_l + \mathbf{b}_l$. Shapes: $(N, d_{l-1}) \times (d_{l-1}, d_l) \to (N, d_l)$, and $\mathbf{b}_l$ of shape $(d_l,)$ is broadcast to every row.
- **Line 47**: hidden layers apply the activation; the last layer does *not*. Its raw outputs are the **logits** (unnormalised class scores). The softmax lives inside the loss, for numerical reasons explained next.
- **Line 48** stores $\mathbf{Z}_l$ (needed for the ReLU derivative) and $\mathbf{A}_l$ (needed for the next weight gradient).

The parameter `act` lets us swap ReLU for another function; we will use it to remove the nonlinearity in Section 13.10.

13.5 The loss: stable softmax cross-entropy

Recall from Chapter 8 that for logits $\mathbf{z} \in \mathbb{R}^K$ the softmax probabilities and cross-entropy loss of an example with true class y are

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \ell = -\ln p_y = -z_y + \ln \sum_{j=1}^{K} e^{z_j}. \tag{13.1}$$

The last term, $\ln \sum_j e^{z_j}$, is called **log-sum-exp**.

13.5.1 Why we need a stability trick

A computer stores a `float64` up to about 1.8×10^{308}. Since $e^{710} > 10^{308}$, a logit of 710 makes e^z overflow to `inf`, and then `inf/inf` = `nan`. Logits of several hundred do occur in badly-behaved training runs (we will see one with $\eta = 5$). The fix is a one-line identity: for any constant m,

$$\ln \sum_j e^{z_j} = \ln \left(e^m \sum_j e^{z_j - m} \right) = m + \ln \sum_j e^{z_j - m}.$$

Choosing $m = \max_j z_j$ makes every shifted logit $z_j - m \le 0$, so every exponential lies in $(0, 1]$ and at least one equals 1: no overflow is possible, and the sum is at least 1, so no $\ln 0$ either. The log-probabilities become

$$\ln p_k = z_k - \ln \sum_j e^{z_j} = (z_k - m) - \ln \sum_j e^{z_j - m}. \tag{13.2}$$

The shift cancels: subtracting the same number from every logit does not change the softmax.

Listing 13.4: Loss functions: stable softmax cross-entropy and L2 regularisation.

```
53 def softmax_cross_entropy(logits, y):
54     N = logits.shape[0]
55     shifted = logits - logits.max(axis=1, keepdims=True)    # stability
56     log_sum_exp = np.log(np.exp(shifted).sum(axis=1, keepdims=True))
57     log_probs = shifted - log_sum_exp                       # log softmax
58     loss = -log_probs[np.arange(N), y].mean()
59     P = np.exp(log_probs)                                   # probabilities
60     Y = np.zeros_like(P)
61     Y[np.arange(N), y] = 1.0                                # one-hot labels
62     dlogits = (P - Y) / N
63     return loss, dlogits
64
65
66 def loss_and_cache(params, X, y, l2, act=relu):
67     logits, cache = forward(params, X, act)
68     data_loss, dlogits = softmax_cross_entropy(logits, y)
69     L = len(params) // 2
70     reg = 0.5 * l2 * sum(np.sum(params[f"W{l}"] ** 2) for l in range(1, L + 1))
71     return data_loss + reg, dlogits, cache
```

- **Line 55**: subtract each row's maximum (`keepdims=True` keeps the shape $(N, 1)$ so broadcasting subtracts it from the whole row).
- **Line 56**: log-sum-exp of the shifted logits, shape $(N, 1)$.
- **Line 57**: Eq. (13.2), the log-probabilities of all classes, shape (N, K).
- **Line 58**: "fancy indexing" `log_probs[np.arange(N), y]` picks, in row i, the column y_i — the log-probability of the correct class. The loss is minus their average: $L_{\text{data}} = -\frac{1}{N}\sum_i \ln p_{i,y_i}$.
- **Lines 59–61**: probabilities $\mathbf{P}$ and the one-hot label matrix $\mathbf{Y}$ ($Y_{ik} = 1$ if $k = y_i$, else 0).
- **Line 62**: the gradient with respect to the logits, derived below.

Worked example: log-sum-exp by hand

Take one example with logits $\mathbf{z} = (2,\ 1,\ 0.1)$ and true class $y = 0$.

1. Maximum $m = 2$; shifted logits $(0,\ -1,\ -1.9)$.
2. Exponentials: $(1,\ 0.3679,\ 0.1496)$; their sum is 1.5174; $\ln 1.5174 = 0.4170$.
3. Log-probabilities: $(0 - 0.4170,\ -1 - 0.4170,\ -1.9 - 0.4170) = (-0.4170,\ -1.4170,\ -2.3170)$.
4. Probabilities: $\mathbf{P} = (0.6590,\ 0.2424,\ 0.0986)$, which sum to 1.
5. Loss: $\ell = -\ln p_0 = 0.4170$.

6. Gradient ($N = 1$): $\mathbf{P} - \mathbf{Y} = (0.6590 - 1,\ 0.2424,\ 0.0986) = (-0.3410,\ 0.2424,\ 0.0986)$.

The gradient pushes the correct logit up (negative gradient, so gradient *descent* increases it) and the two wrong logits down, each in proportion to how much probability it wrongly holds. Now try $\mathbf{z} = (1000,\ 999)$: the naive formula computes $e^{1000} =$ `inf`; the shifted one computes $e^{0} = 1$, $e^{-1} = 0.368$ and returns $p = (0.731,\ 0.269)$ without complaint.

13.5.2 The gradient $(\mathbf{P} - \mathbf{Y})/N$

From Eq. (13.1), $\ell = -z_y + \ln \sum_j e^{z_j}$. Differentiate with respect to one logit z_k. The first term gives -1 if $k = y$ and 0 otherwise, which is $-Y_k$. For the second, the chain rule ($\frac{d}{du} \ln u = 1/u$) gives

$$\frac{\partial}{\partial z_k} \ln \sum_j e^{z_j} = \frac{1}{\sum_j e^{z_j}} \cdot e^{z_k} = p_k.$$

So $\partial \ell / \partial z_k = p_k - Y_k$. The batch loss is the *average* of N such terms, so each example's gradient is divided by N:

Key equation: gradient of mean softmax cross-entropy

$$\frac{\partial L_{\text{data}}}{\partial \mathbf{Z}_L} = \frac{1}{N}(\mathbf{P} - \mathbf{Y}) \qquad \text{(shape } N \times K\text{)}.$$

This is the same result as in Chapters 8 and 10; notice how clean it is — no exponentials, no division by small probabilities. It is the main reason softmax and cross-entropy are always used as a pair.

13.5.3 L2 regularisation

Line 70 adds the penalty $R = \frac{\lambda}{2} \sum_l \|\mathbf{W}_l\|^2 = \frac{\lambda}{2} \sum_l \sum_{i,j} W_{l,ij}^2$ (biases are not penalised; they do not make the function wiggly). The total loss is $L = L_{\text{data}} + R$. Since $\partial(\frac{\lambda}{2} W^2)/\partial W = \lambda W$, the penalty adds $\lambda \mathbf{W}_l$ to each weight gradient, which pulls every weight a little towards zero at every step ("weight decay"). The factor $\frac{1}{2}$ is there only to make that derivative tidy.

13.6 The backward pass

Listing 13.5: `backward`: the four equations of Chapter 10 in a loop.

```
75 def backward(params, cache, dlogits, l2, relu_mask=True):
76     L = len(params) // 2
77     grads = {}
78     dZ = dlogits                                          # dLoss/dZ_L
79     for l in range(L, 0, -1):
80         A_prev = cache[f"A{l-1}"]
81         grads[f"W{l}"] = A_prev.T @ dZ + l2 * params[f"W{l}"]
82         grads[f"b{l}"] = dZ.sum(axis=0)
83         if l > 1:
84             dA = dZ @ params[f"W{l}"].T                   # dLoss/dA_{l-1}
85             dZ = dA * (cache[f"Z{l-1}"] > 0) if relu_mask else dA
86     return grads
```

This short loop implements the batch backpropagation equations of Chapter 10. Write $\delta_l = \partial L / \partial \mathbf{Z}_l$ (shape $N \times d_l$; called `dZ` in the code). Then for $l = L, L-1, \dots, 1$:

Key equation: backward pass of an MLP, batch-first

$$\frac{\partial L}{\partial \mathbf{W}_l} = \mathbf{A}_{l-1}^{\top} \delta_l + \lambda \mathbf{W}_l, \qquad \frac{\partial L}{\partial \mathbf{b}_l} = \sum_{i=1}^{N} (\delta_l)_{i,:}, \tag{13.3}$$

$$\frac{\partial L}{\partial \mathbf{A}_{l-1}} = \delta_l \, \mathbf{W}_l^{\top}, \qquad \delta_{l-1} = \frac{\partial L}{\partial \mathbf{A}_{l-1}} \odot \mathbb{1}[\mathbf{Z}_{l-1} > 0]. \tag{13.4}$$

- **Line 78**: start with $\delta_L = (\mathbf{P} - \mathbf{Y})/N$ from the loss.
- **Line 79**: walk the layers backwards, $L, L-1, \dots, 1$.
- **Line 81**: weight gradient. Shapes check: $(d_{l-1}, N) \times (N, d_l) = (d_{l-1}, d_l)$, the shape of $\mathbf{W}_l$. The matrix product sums over the batch automatically: entry (j,k) is $\sum_i A_{l-1,ij}\, \delta_{l,ik}$, "input j times error k, added over examples". Then the L2 term.
- **Line 82**: bias gradient. A bias is added to every row, so its gradient is the sum of the errors over all rows.
- **Line 84**: send the error back through the weights (a transpose appears because the forward pass multiplied on the right by $\mathbf{W}_l$).
- **Line 85**: the ReLU derivative is 1 where the pre-activation was positive and 0 elsewhere; `(Z > 0)` is a Boolean mask that NumPy treats as 1/0 in the multiplication. The `relu_mask` flag exists only so that we can break this line deliberately later.

Each layer costs two matrix products backward (lines 81 and 84) versus one forward, which is why a training step costs roughly three forward passes.

Pitfall: shape bugs that do not crash

`A_prev.T @ dZ` and `dZ.T @ A_prev` both run; the second gives the transpose of the right answer and crashes only if the matrix is not square. For a square layer (100×100) it silently trains wrongly. Always write the shape of each factor next to a gradient formula, and always run the gradient check.

13.7 Training: mini-batch SGD and prediction

Listing 13.6: `sgd_step`, `predict` and the training loop.

```
90  def sgd_step(params, grads, lr):
91      for k in params:
92          params[k] -= lr * grads[k]
93
94
95  def predict(params, X, act=relu):
96      logits, _ = forward(params, X, act)
97      return logits.argmax(axis=1)
98
99
100 def train(params, X, y, lr=0.5, l2=1e-3, batch_size=64, epochs=1000,
```

```
101           seed=0, act=relu, backward_fn=backward, log_every=100):
102     rng = np.random.default_rng(seed)
103     N = X.shape[0]
104     history = {"loss": [], "acc": []}
105     for epoch in range(epochs):
106         perm = rng.permutation(N)                          # reshuffle
107         for start in range(0, N, batch_size):
108             idx = perm[start:start + batch_size]
109             loss, dlogits, cache = loss_and_cache(params, X[idx], y[idx], l2, act)
110             grads = backward_fn(params, cache, dlogits, l2)
111             sgd_step(params, grads, lr)
112         full_loss = loss_and_cache(params, X, y, l2, act)[0]
113         acc = np.mean(predict(params, X, act) == y)
114         history["loss"].append(full_loss)
115         history["acc"].append(acc)
116         if log_every and epoch % log_every == 0:
117             print(f"epoch {epoch:4d}  loss {full_loss:.4f}  acc {acc:.3f}")
118         if not np.isfinite(full_loss):
119             print(f"diverged at epoch {epoch}")
120             break
121     return history
```

- **Lines 90–92**: the gradient-descent update of Chapter 9, $\theta \leftarrow \theta - \eta\, \partial L/\partial\theta$, applied to every array in place (`-=` modifies the array inside the dictionary rather than creating a new one).
- **Lines 95–97**: prediction is the class with the largest logit. We do not need the softmax: it is monotonic, so the largest logit is also the largest probability.
- **Line 106**: at the start of every epoch the examples are shuffled, so each mini-batch is a fresh random sample. (Without shuffling, our spiral data is sorted by class and every early batch would contain a single class.)
- **Lines 107–108**: slice the shuffled indices into batches of 64 (the last batch of an epoch is smaller: $300 = 4 \times 64 + 44$).
- **Lines 109–111**: the core of training — forward and loss, backward, update. One epoch is 5 such steps.
- **Lines 112–115**: once per epoch, measure loss and accuracy on the full training set for the history plot.
- **Lines 118–120**: if the loss becomes `inf` or `nan`, stop instead of wasting time.

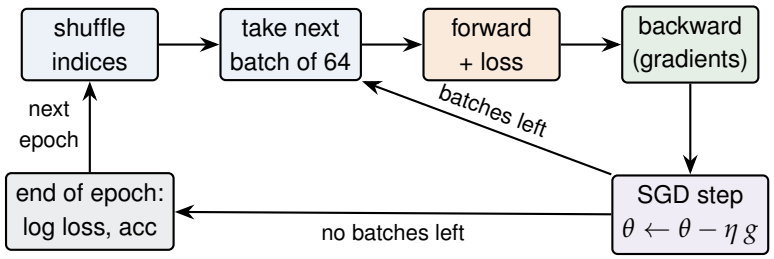


Figure 13.4: The mini-batch training loop of `train`: an outer loop over epochs (reshuffle) and an inner loop over mini-batches (forward, backward, update).

13.8 The gradient check: auditing backprop

Your backward pass is a long chain of hand-derived formulas. One wrong transpose, a missing $1/N$ or a forgotten mask, and the network still trains — just worse — and you may never notice. The gradient check compares the analytic gradient against a gradient obtained by brute force from the definition of a derivative.

Intuition: wiggle one knob and watch the meter

To measure how sensitive the loss is to one weight, nudge that weight up by a tiny ε, read the loss, nudge it down by ε, read again; the change in loss divided by the change in weight is the slope. It is slow (two full forward passes per parameter) but it relies on nothing except the forward pass, which is easy to get right.

13.8.1 Why the centred difference

Expand the loss around the current value w with a Taylor series (Chapter 3):

$$L(w+\varepsilon) = L(w) + \varepsilon L'(w) + \tfrac{\varepsilon^2}{2}L''(w) + \tfrac{\varepsilon^3}{6}L'''(w) + \dots$$
$$L(w-\varepsilon) = L(w) - \varepsilon L'(w) + \tfrac{\varepsilon^2}{2}L''(w) - \tfrac{\varepsilon^3}{6}L'''(w) + \dots$$

The forward difference has an error proportional to ε: $\frac{L(w+\varepsilon)-L(w)}{\varepsilon} = L'(w) + \frac{\varepsilon}{2}L''(w) + \dots$. Subtracting the two expansions, the even terms cancel:

Key equation: centred difference

$$\frac{L(w+\varepsilon) - L(w-\varepsilon)}{2\varepsilon} = L'(w) + \frac{\varepsilon^2}{6}L'''(w) + \dots$$

The error is now proportional to ε^2. With $\varepsilon = 10^{-5}$ that is about 10^{-10} instead of 10^{-5}.

Worked example: forward versus centred difference

Let $f(w) = w^3$ at $w = 2$, so the true slope is $3w^2 = 12$. With $\varepsilon = 10^{-5}$: forward difference $= (2.00001^3 - 8)/10^{-5} = 12.00006$ (error $6 \times 10^{-5} = 3w\varepsilon$, as predicted by $\frac{\varepsilon}{2}f'' = \frac{\varepsilon}{2}\cdot 6w$); centred difference $= 12.0000000002$ (error $2 \times 10^{-10} \approx \varepsilon^2 f'''/6 = \varepsilon^2$).

Why float64. The numerator $L(w+\varepsilon) - L(w-\varepsilon)$ is the difference of two numbers that agree in their first five digits. A `float32` carries only about 7 significant digits, so after the subtraction only 2 meaningful digits are left. A `float64` carries about 16, leaving about 11. All our NumPy arrays are `float64` by default; frameworks default to `float32`, which is why `torch.autograd.gradcheck` in Chapter 14 insists on double precision.

13.8.2 Relative error

Gradients of different parameters have very different sizes, so we compare in relative terms. For each parameter array (e.g. all of $\mathbf{W}_2$) with analytic gradient $\mathbf{a}$ and numerical gradient $\mathbf{n}$ (both flattened),

$$\text{rel. error} = \frac{\|\mathbf{a} - \mathbf{n}\|}{\|\mathbf{a}\| + \|\mathbf{n}\|}. \tag{13.5}$$

A value of 1 means the two are unrelated; around 10^{-7} or smaller (in float64, with smooth functions) means they agree to many digits.

Listing 13.7: `grad_check`: centred differences for every single parameter.

```
125 def grad_check(params, X, y, l2, eps=1e-5, backward_fn=backward, verbose=True):
126     loss, dlogits, cache = loss_and_cache(params, X, y, l2)
127     grads = backward_fn(params, cache, dlogits, l2)        # analytic
128     errors = {}
129     for name, P in params.items():
130         num = np.zeros_like(P)
131         for i in np.ndindex(P.shape):                      # every entry
132             old = P[i]
133             P[i] = old + eps
134             loss_plus = loss_and_cache(params, X, y, l2)[0]
135             P[i] = old - eps
136             loss_minus = loss_and_cache(params, X, y, l2)[0]
137             P[i] = old                                     # restore!
138             num[i] = (loss_plus - loss_minus) / (2 * eps)
139         a = grads[name]
140         rel = np.linalg.norm(a - num) / max(np.linalg.norm(a) + np.linalg.norm(num
    ), 1e-12)
141         errors[name] = rel
142         if verbose:
143             print(f"{name:3s} shape {str(P.shape):9s} relative error {rel:.2e}")
144     return errors
```

- **Lines 126–127**: the analytic gradients from your backward pass (any backward function can be passed in — we will pass a broken one).
- **Line 131**: `np.ndindex` iterates over every index tuple of an array of any shape, e.g. $(0,0), (0,1), \ldots$.
- **Lines 132–137**: nudge the entry up, evaluate the loss, nudge down, evaluate, and *restore* the original value. Forgetting line 137 would corrupt the network and every later comparison.
- **Line 138**: the centred difference.
- **Line 140**: Eq. (13.5); the `max(...,1e-12)` avoids dividing by zero when both gradients are exactly zero.

Cost: $2 \times$ (number of parameters) forward passes. For the full 10 703-parameter network that is fine but slow; we check a smaller $[2, 10, 10, 3]$ network on 10 points (183 parameters). Shape-generic code that passes the check on a small network is correct for the large one.

13.8.3 Results, and the ReLU kink

Read Table 13.1 column by column.

Left column. Every parameter passes with errors around 10^{-10}. That is the centred-difference truncation error and float64 round-off, nothing more. Your backward pass is correct.

Middle column: a false alarm caused by a kink. Our first batch was `X[::30]`, whose first point is `X[0]`$= (0,0)$, the start of class 0's arm. At that point $\mathbf{Z}_1 = \mathbf{0} \cdot \mathbf{W}_1 + \mathbf{b}_1 = \mathbf{b}_1$, and at initialisation $\mathbf{b}_1 = \mathbf{0}$. So all ten first-layer neurons sit *exactly* on the kink of the ReLU, where the function $\max(0, z)$ has no derivative: the slope is 0 on the left and 1 on the right. When the gradient checker nudges $b_{1,j}$ to $+\varepsilon$, neuron j switches on; at $-\varepsilon$ it stays off. The centred difference

Param.	shape	correct backward (10 generic points)	correct backward, batch with origin	broken backward (no ReLU mask)
$\mathbf{W}_1$	$(2, 10)$	9.6×10^{-11}	1.7×10^{-10}	4.2×10^{-1}
$\mathbf{b}_1$	$(10,)$	3.3×10^{-11}	$\mathbf{3.0 \times 10^{-2}}$	3.5×10^{-1}
$\mathbf{W}_2$	$(10, 10)$	7.8×10^{-11}	9.1×10^{-11}	2.9×10^{-1}
$\mathbf{b}_2$	$(10,)$	1.9×10^{-11}	$\mathbf{6.4 \times 10^{-2}}$	3.1×10^{-1}
$\mathbf{W}_3$	$(10, 3)$	5.6×10^{-11}	6.7×10^{-11}	5.6×10^{-11}
$\mathbf{b}_3$	$(3,)$	5.6×10^{-11}	3.2×10^{-11}	5.6×10^{-11}

Table 13.1: Relative errors printed by `grad_check` (script `run_spiral.py`, $\lambda = 10^{-3}$, $\varepsilon = 10^{-5}$). Left: a correct backward pass agrees with the numerical gradient to about ten digits, far better than the 10^{-7} target. Middle: the same correct code on a batch that contains the point $(0, 0)$ — the biases look wrong, but the code is fine (see text). Right: a backward pass with the ReLU mask deleted fails loudly, but only in the layers below the bug.

thus measures the *average* of the left and right slopes, $\frac{1}{2}$, while our code (`Z > 0` is False at $Z = 0$) uses slope 0. The same happens in layer 2, because $\mathbf{A}_1 = \mathbf{0}$ at the origin makes $\mathbf{Z}_2 = \mathbf{b}_2 = \mathbf{0}$ too. Why are the weights unaffected? Nudging $W_{1,ij}$ changes Z_1 by $\varepsilon \times x_i = \varepsilon \times 0$ at the origin: it cannot cross the kink. The lesson:

Pitfall: gradient checks and ReLU kinks

A numerical gradient is meaningless when a pre-activation lies within ε of 0, because the finite difference straddles the kink. In real data this happens rarely and produces an occasional larger error (say 10^{-4}); with special inputs (exact zeros, zero biases) it can hit many units at once. When a check fails only for biases or only occasionally, test on random generic inputs, or use a smooth activation (e.g. tanh) just for the check, before hunting for a bug.

Right column: a real bug. The broken backward pass forgets the ReLU mask in line 85 (it treats ReLU as the identity on the way back). The output layer $\mathbf{W}_3, \mathbf{b}_3$ is still correct, because its gradient is computed before the mask is ever used; everything below fails with errors around 0.3–0.4. The check therefore not only detects the bug but *localises* it: the error starts at the first layer whose gradient passes through the faulty line. This is how you use gradient checking in practice.

13.9 Training the spiral network

13.9.1 First sanity check: the initial loss

Before training, compute the loss of the freshly initialised network. If it predicts nothing in particular, each of the $K = 3$ classes gets probability about $\frac{1}{3}$, and the cross-entropy is $-\ln \frac{1}{3} = \ln 3 = 1.0986$. Our script prints:

```
initial loss 1.2350  vs ln(3) = 1.0986
initial loss with small last layer 1.0990
```

The value 1.235 is slightly above $\ln 3$ because with He initialisation the logits are not tiny: their standard deviation s is about 0.52, and random logits make the network confidently wrong on some points. (A second-order expansion of log-sum-exp gives $\mathbb{E}[\ell] \approx \ln K + \frac{s^2}{2}(1 - \frac{1}{K}) = 1.0986 + 0.09 \approx 1.19$, the right ballpark.) Scaling the last layer by 0.01 shrinks the logits and gives

1.0990, essentially $\ln 3$. Both are healthy. An initial loss of 5 or 50 would mean the logits are huge (a scaling bug); an initial loss far below $\ln 3$ before any training would mean label leakage.

Intuition: the magic number 1.0986

Memorise $\ln K$ for your problem: $\ln 2 = 0.693$, $\ln 3 = 1.099$, $\ln 10 = 2.303$. If the training loss sits at $\ln K$ and never moves, the network has learned nothing at all — it outputs uniform probabilities. You will meet this number again in the learning-rate experiment.

13.9.2 The loss curve

We now train the $[2, 100, 100, 3]$ network with $\eta = 0.5$, $\lambda = 10^{-3}$, batch 64 for 1000 epochs (5000 SGD steps, about ten seconds on a laptop):

```
X, y = nn.make_spiral(100, 3, 0.2, seed=0)
p = nn.init_params([2, 100, 100, 3], seed=1)
hist = nn.train(p, X, y, lr=0.5, l2=1e-3, batch_size=64, epochs=1000, seed=0)
```

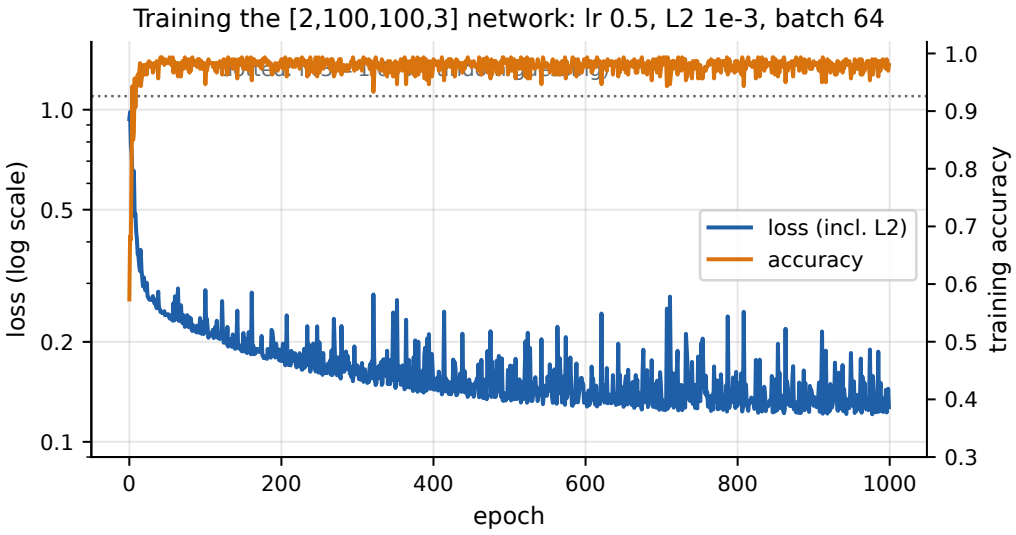


Figure 13.5: Loss (blue, log scale, including the L2 term) and training accuracy (orange) after each epoch. The loss falls quickly from about 1 to 0.43 after 10 epochs, then slowly to 0.13 after 1000. Accuracy jumps above 94 % within 10 epochs and settles at 97–98 %. The jagged curve is the signature of stochastic mini-batches with a fairly large learning rate; the dotted line marks $\ln 3$.

Figure 13.5 shows the result. The final training accuracy is 98.0 %. The few remaining errors are points that the angular noise has pushed into a neighbouring arm; a perfect score would mean memorising noise.

13.9.3 Decision boundaries: the MLP versus a linear classifier

To draw a decision boundary we classify every point of a 300×300 grid covering the plane and colour it by the predicted class. As a baseline we train the *same* code with `layer_sizes=[2, 3]`: no hidden layer, so the model is a linear softmax classifier (multinomial logistic regression).

A linear classifier assigns each class a score $z_k = \mathbf{x}^{\mathsf{T}}\mathbf{w}_k + b_k$, and the boundary between two classes is where two scores are equal — a straight line. Three straight lines meeting at a point cut the plane into three wedges, and no choice of wedges can follow a spiral; 54 % is about the

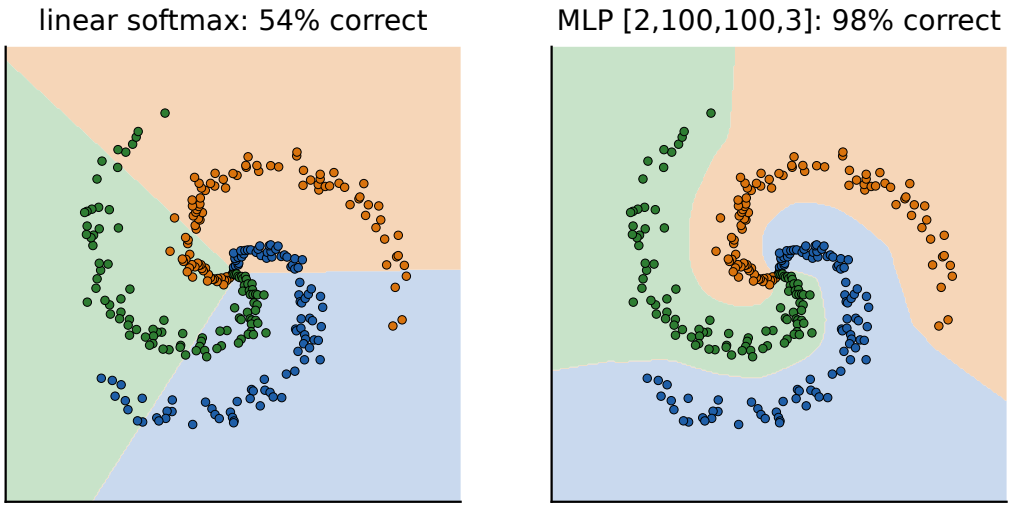


Figure 13.6: Left: the best a linear softmax classifier can do — three wedge-shaped regions separated by straight lines, 54 % accuracy (chance is 33 %). Right: the two-hidden-layer MLP bends the boundaries to follow the arms, 98 %. Both models were trained with exactly the same code, loss and optimiser; only the hidden layers differ.

best possible. The MLP's hidden ReLUs each draw a line and fold the plane along it (Chapter 7); the second layer combines these folds into curved, piecewise-linear regions. Zoom into the right-hand panel and you can see that the curved boundary is in fact made of many short straight segments.

13.10 Experiments: predict, then run

Each experiment below changes exactly one thing. Before reading the result, write down what you expect. All figures come from `code/ch13/run_spiral.py`.

13.10.1 Experiment 1: remove the ReLU

Replace the activation by the identity, `act = lambda Z: Z`, and use a backward pass without the mask (the identity's derivative is 1). The network still has two hidden layers of 100 units, so is it still powerful? No: without the nonlinearity

$$\mathbf{Z}_3 = \big((\mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2\big)\mathbf{W}_3 + \mathbf{b}_3 = \mathbf{X}\underbrace{(\mathbf{W}_1\mathbf{W}_2\mathbf{W}_3)}_{\mathbf{W}_{\text{eff}}\ (2\times 3)} + \mathbf{b}_{\text{eff}},$$

a product of matrices is just another matrix, and the "deep" network collapses to a linear classifier with 10 703 parameters doing the job of 9. (We used $\eta = 0.05$ here: the linear network has no dead units to limit the size of its outputs, and the larger rate is unstable.) The left panel of Fig. 13.7 confirms it: straight boundaries and 54 % accuracy, exactly the linear result.

13.10.2 Experiment 2: learning rate 5

In Chapter 9 you saw that gradient descent diverges when the step is too large. With $\eta = 5$ (ten times our value) the run printed:

```
epoch    0  loss 14.3816  acc 0.333
epoch   10  loss 8.3185  acc 0.333
epoch   50  loss 2.2782  acc 0.333
```

Within the first five SGD steps the mini-batch loss jumped from 1.5 to 55: each step overshot so badly that some logits became large, the next gradient was larger still, and the weights blew up. Then something characteristic of ReLU networks happened: the huge updates pushed almost every hidden unit to a state where its pre-activation is negative for *every* input. After 60 epochs, 99 of the 100 first-layer units and all 100 second-layer units were **dead** (always zero output, hence always zero gradient). The network outputs the same class for every input (accuracy 33 %), and its data loss is stuck at $1.11 \approx \ln 3$. The total loss in the printout still falls, but only because the L2 term slowly shrinks the giant weights — a trap for anyone who watches only the loss value. With $\eta = 20$ or more the loss reaches 10^8 and beyond in the first epoch. The right panel of Fig. 13.7 compares $\eta = 0.5$ (smooth descent), $\eta = 2$ (still learning but very noisy) and $\eta = 5$ (broken).

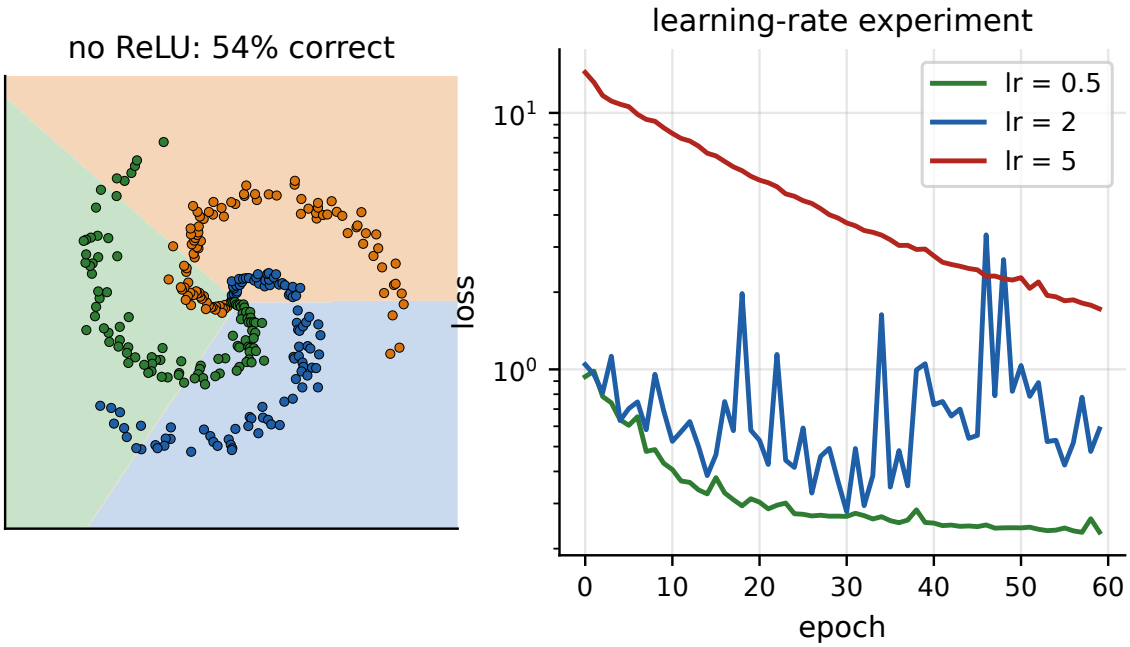


Figure 13.7: Left: the network with ReLU replaced by the identity draws only straight boundaries (54 %), exactly like the linear classifier — depth without nonlinearity is useless. Right: loss curves for three learning rates over 60 epochs. $\eta = 0.5$ descends steadily; $\eta = 2$ makes progress but jumps around; $\eta = 5$ starts at 14 after the first epoch, has killed nearly every ReLU, and the slow decline is only the L2 penalty shrinking the exploded weights.

Pitfall: divergence does not always look like NaN

In a ReLU network a too-large learning rate often produces *dead* units rather than infinities. Symptoms: accuracy exactly $1/K$, data loss at $\ln K$, all predictions the same class, gradients of early layers exactly zero. Cure: restart with a smaller learning rate (and, in Chapter 14, an adaptive optimiser such as Adam).

13.10.3 Experiment 3: a broken backward pass

We already saw it in Table 13.1: deleting the ReLU mask gives relative errors of 0.3–0.4 in layers 1 and 2. What happens if you train with that broken gradient anyway? Figure 13.8 shows the answer. For hundreds of epochs nothing dramatic happens: the loss wanders between about 0.8 and 4, and the accuracy between 49 and 84 % — clearly better than chance, so a hurried engineer might conclude "it trains, just not very well; let me tune the learning rate". Meanwhile the wrong gradients steadily inflate the weights (the update for a dead unit should be zero, but the broken code keeps pushing it), and at epoch 753 the logits overflow and the loss becomes `nan`. That is exactly why bugs in hand-written backward passes are so dangerous: training "sort of works" and the failure, when it comes, appears far from its cause. The gradient check gives a crisp yes/no answer in seconds.

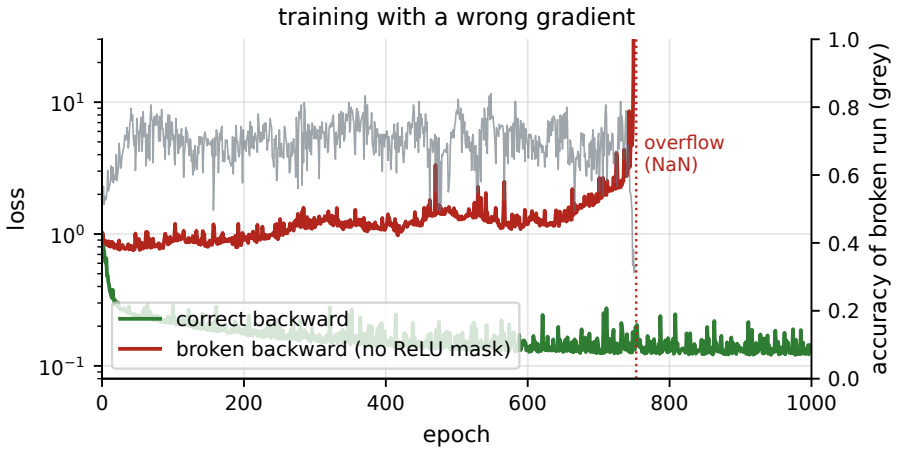


Figure 13.8: Training the spiral network with a broken backward pass (ReLU mask missing, red loss; grey: its accuracy on the right axis) versus the correct one (green). The broken run neither fails immediately nor learns properly; it wanders at a loss of order 1 with erratic accuracy (peak 84 %), then overflows to NaN at epoch 753 (dotted line).

13.10.4 Experiment 4: hidden width

Finally, keep two hidden layers but change their width to 5, 20 and 100 neurons. Each ReLU contributes one "fold line", so a narrow network can build only a coarse polygonal boundary. Beyond some width the training accuracy saturates at the level set by the label noise. On this problem 20 units suffice; more width costs computation without benefit, though in larger problems extra width, combined with regularisation, often helps optimisation.

13.11 Wireless mini-lab: a learned 16-QAM detector

The spiral is a toy. Here is the same 140-line network solving a problem every receiver solves billions of times per second: deciding which constellation point was sent.

Wireless connection: why straight-line detectors stop being optimal

In pure AWGN the maximum-likelihood (ML) detector for equally likely symbols is the *nearest-point* detector: pick the constellation point closest to the received sample. Its decision regions are bounded by straight lines halfway between neighbours (the familiar square grid

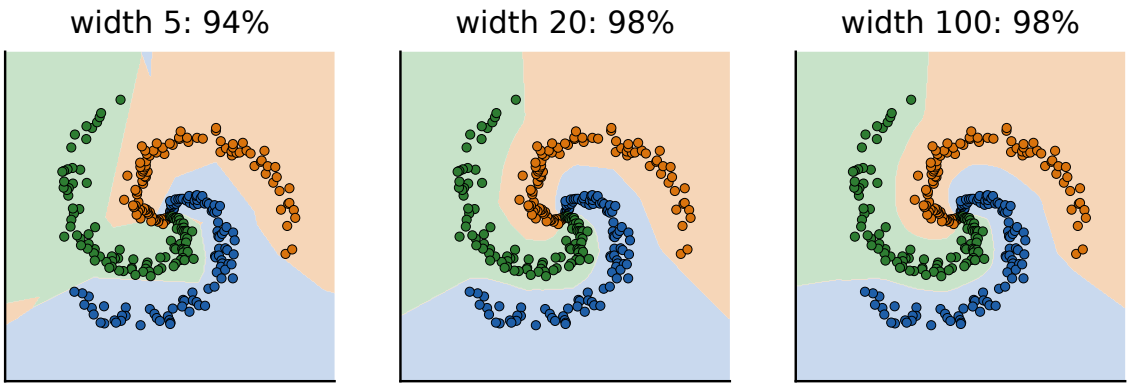


Figure 13.9: Decision regions for hidden widths 5, 20 and 100 (two hidden layers, 1000 epochs, same settings). Width 5 already reaches 94 % but its boundaries are made of a few long straight pieces and miss the tips of the arms. Width 20 (98 %) and width 100 (98 %) follow the arms smoothly; going from 20 to 100 buys little on this problem.

for 16-QAM), and every receiver implements them with a few comparisons. But a real receiver has impairments left over after synchronisation: residual carrier-frequency offset, oscillator phase noise, IQ imbalance, PA compression. A residual phase error ϕ rotates the received point around the origin, $r = s\,e^{j\phi} + w$, so the noise cloud of each point becomes a short *arc*, longest for the outer points (an arc of radius $|s|$ and angle ϕ has length $|s|\phi$). The optimal boundaries are then curved, and a detector that learns them from data can beat the textbook rule.

13.11.1 System model

We transmit 16-QAM with levels $\{-3,-1,1,3\}/\sqrt{10}$ on each axis, so the average symbol energy is $E_s = \frac{2\cdot(1+9)/2}{10} = 1$. The received sample is

$$r = s\,e^{j\phi} + w, \qquad \phi \sim \mathcal{N}(0,\sigma_\phi^2), \quad w \sim \mathcal{CN}(0,N_0), \quad \mathrm{SNR} = E_s/N_0 = 1/N_0, \tag{13.6}$$

with residual phase-noise standard deviation $\sigma_\phi = 0.15$ rad (8.6°), drawn independently for every symbol (a simple model of fast residual phase jitter). The corner points have $|s| = \sqrt{18/10} = 1.34$, so their tangential spread $|s|\sigma_\phi \approx 0.20$ is not small compared with the half-spacing between levels, $1/\sqrt{10} = 0.32$.

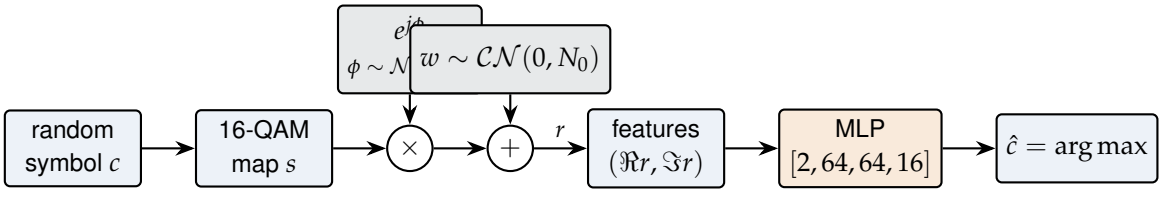


Figure 13.10: The 16-QAM detection mini-lab. The channel applies a random phase rotation and AWGN; the NumPy MLP sees only the two real numbers $(\Re r, \Im r)$ and outputs 16 logits, one per constellation point. Its training label is the transmitted symbol index c.

13.11.2 Three detectors

1. **Nearest point**: $\hat{c} = \arg\min_c |r - s_c|$ — optimal without phase noise, mismatched here.

2. **The NumPy MLP**: input $(\Re r, \Im r)$, two hidden layers of 64 ReLUs, 16 softmax outputs. It is a 16-class version of the spiral problem: points in the plane, labelled by which constellation point produced them. We train it on 32 000 received samples whose SNR is drawn uniformly between 14 and 28 dB, so one network must work at every SNR; $\eta = 0.3$, $\lambda = 10^{-5}$, batch 128, 40 epochs.

3. **Exact ML** (a benchmark that knows σ_ϕ and the SNR): the likelihood of symbol s_c averages the Gaussian noise density over the unknown phase,

$$p(r \mid s_c) = \int \frac{e^{-\phi^2/(2\sigma_\phi^2)}}{\sqrt{2\pi}\sigma_\phi} \, \frac{1}{\pi N_0} e^{-|r - s_c e^{j\phi}|^2/N_0} \, d\phi,$$

   which we evaluate numerically on 41 phase values between $\pm 4\sigma_\phi$, then pick the largest.

The core of the experiment, using exactly the functions of this chapter:

```
lab_tr = rng.integers(0, 16, 32000)              # transmitted symbol indices
snr_tr = rng.uniform(14, 28, 32000)              # a different SNR for each sample
X_tr = to_features(channel(lab_tr, snr_tr))      # (32000, 2): real, imag
params = nn.init_params([2, 64, 64, 16], seed=0)
hist = nn.train(params, X_tr, lab_tr, lr=0.3, l2=1e-5, batch_size=128, epochs=40)
r = channel(lab, snr)                            # fresh test symbols
ser_mlp = np.mean(nn.predict(params, to_features(r)) != lab)
```

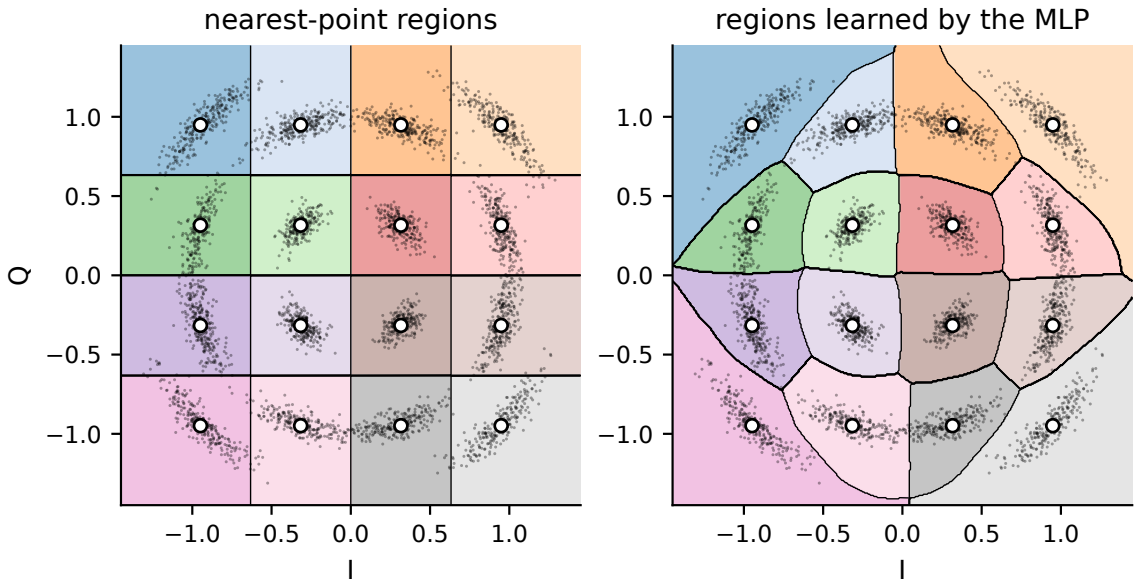


Figure 13.11: Received samples at 24 dB (black dots) with the decision regions of the two practical detectors (white circles: the 16 constellation points). Left: nearest-point regions — a rigid square grid. Notice how the clouds of the outer points are bent into arcs that poke across the straight boundaries. Right: the regions learned by the MLP. The boundaries between outer points are rotated and bent to follow the arcs, the outer regions look like sectors of a ring, and the inner four points get a "pinwheel" shape. Nobody told the network about phase noise; it inferred the geometry from labelled examples.

Figures 13.11 and 13.12 show the outcome. At 24 dB the SERs are 2.98×10^{-2} (nearest point), 1.23×10^{-2} (MLP) and 1.03×10^{-2} (exact ML). Three observations are worth making.

First, the floor. As SNR $\to \infty$ the nearest-point detector still errs whenever the rotation alone moves an outer point across a straight boundary; no amount of transmit power cures that. The

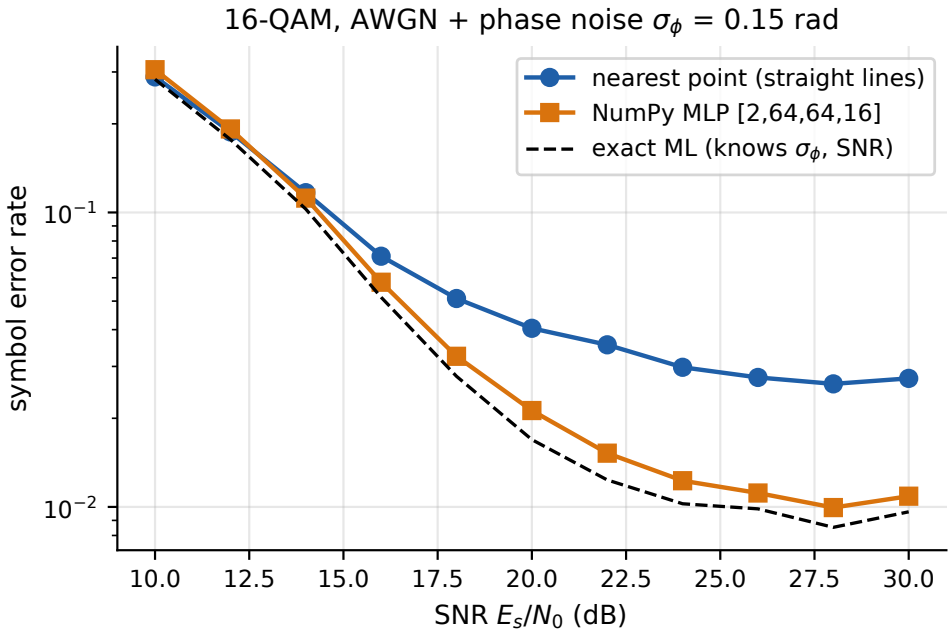


Figure 13.12: Symbol error rate versus SNR (40 000 test symbols per point). The nearest-point detector hits an *error floor* near 2.7×10^{-2}: above about 20 dB the noise is negligible and the errors are caused by phase rotation alone. The MLP reaches about 1.0×10^{-2}, within roughly 15 % of the exact ML detector, without knowing σ_ϕ or the SNR. Below 14 dB (outside its training range) the MLP is slightly worse than nearest point.

MLP and the ML detector rotate the boundaries with the arcs, and their floor is lower (it is not zero, because adjacent outer arcs still overlap slightly at $\sigma_\phi = 0.15$).

Second, generalisation across SNR. The network was never given the SNR, and was trained on a mixture of 14–28 dB. Between those limits it tracks the ML curve closely. Below 14 dB it is slightly worse than even the nearest-point detector, because at low SNR the optimal regions become more like the AWGN squares and the network never saw such data. Training range matters — a theme that will return in Part V.

Third, honesty about the baseline. A real receiver would not use nearest-point detection in the presence of strong residual phase noise; it would track the phase with a pilot-aided or decision-directed loop and *then* slice. The point of this lab is not that an MLP beats a well-designed receiver, but that a network trained from labelled data discovers curved, near-ML decision regions automatically, with no model of the impairment. That is the central promise of the data-driven receivers in Part V.

Wireless connection: where this idea goes next

Exactly this "classify the received sample" view underlies learned demappers that output bit log-likelihood ratios instead of hard symbols (replace the 16-way softmax by 4 sigmoid outputs, one per bit), neural receivers that take a whole resource grid as input, and the end-to-end autoencoder of Part V, in which the transmitter's constellation is learned too. In a gNB, the same reasoning suggests where learned detectors can pay off: wherever the impairment is hard to model analytically (PA nonlinearity, phase noise at mmWave, interference) but easy to record.

13.12 From this lab to frameworks

Everything PyTorch does in Chapter 14 is a generalisation of what you just wrote:

This chapter	PyTorch equivalent
`init_params` (He)	`nn.Linear` (with `nn.init.kaiming_normal_` if wanted)
`forward`	`model(X)` / `nn.Module.forward`
`softmax_cross_entropy`	`nn.CrossEntropyLoss` (also stable log-sum-exp)
`backward` (hand-derived)	`loss.backward()` (automatic differentiation)
`sgd_step`, L2	`torch.optim.SGD(..., weight_decay=...)`
`grad_check`	`torch.autograd.gradcheck`

The one thing frameworks remove is the hand-written backward pass. You have now written one, checked it, and seen a bug caught; from here on you may trust automatic differentiation with a clear conscience.

Chapter recap

- An MLP in NumPy is four small functions: `forward` ($\mathbf{Z}_l = \mathbf{A}_{l-1}\mathbf{W}_l + \mathbf{b}_l$, ReLU), a stable softmax cross-entropy (subtract the row maximum; gradient $(\mathbf{P} - \mathbf{Y})/N$), `backward` ($\mathbf{A}_{l-1}^\top \delta_l$, column sums, $\delta_l \mathbf{W}_l^\top$, ReLU mask) and an SGD update. L2 adds $\lambda \mathbf{W}$ to each weight gradient.
- He initialisation, $\mathcal{N}(0, 2/d_{\text{in}})$, keeps activation variance constant through ReLU layers.
- Always check the initial loss against $\ln K$, and always gradient-check a new backward pass with centred differences in float64. Expect relative errors $\lesssim 10^{-7}$; ReLU kinks (pre-activations near zero) can cause false alarms, real bugs give errors of order 10^{-1} and are localised to the layers below the bug.
- On spirals: linear softmax 54 %, MLP 98 %. Removing ReLU collapses the network to linear; $\eta = 5$ kills the ReLUs (loss at $\ln 3$); width 5 gives coarse boundaries, 20–100 smooth ones.
- Under residual phase noise the optimal 16-QAM regions are curved; the same NumPy MLP learns them and removes most of the nearest-point detector's error floor, coming close to exact ML.

Exercises

1. How many parameters does a $[2, 50, 50, 50, 4]$ network have? How many forward passes does a full gradient check of it need?
2. For logits $\mathbf{z} = (3,\ 0,\ -1)$ and true class 1, compute by hand (with the stability trick) the probabilities, the loss and $\mathbf{P} - \mathbf{Y}$.
3. A layer has $d_{\text{in}} = 256$. What standard deviation does He initialisation use? If you mistakenly used standard deviation 1, by roughly what factor would the variance of the pre-activations grow per layer?
4. Train the spiral network with the broken backward pass (ReLU mask removed) but with $\eta = 0.1$ instead of 0.5. Does it still blow up? Does it ever reach 98 %? Explain why tuning the learning rate cannot fix a wrong gradient.
5. Your gradient check prints 3×10^{-1} for $\mathbf{W}_1, \mathbf{b}_1$ and 10^{-10} for everything else in a three-

layer network. Where would you look for the bug?

6. Explain why the centred difference with $\varepsilon = 10^{-5}$ in float32 gives poor gradient checks even for perfectly correct code. Estimate how many significant digits survive.
7. In the 16-QAM lab, what would happen to the advantage of the MLP if σ_ϕ were set to 0? To 0.3 rad? Predict, then run `run_qam.py` with the new value.
8. Modify `softmax_cross_entropy` to return also the predicted probabilities, and add a function `accuracy(params, X, y)`. Why is `predict` allowed to skip the softmax?

Answers

1. Layer by layer: $2 \cdot 50 + 50 = 150$; $50 \cdot 50 + 50 = 2550$ (twice); $50 \cdot 4 + 4 = 204$. Total $150 + 2550 + 2550 + 204 = 5454$ parameters. A full check needs two forward passes per parameter: 10 908 forward passes (plus one forward/backward for the analytic gradient).
2. Shift by $m = 3$: $(0, -3, -4)$. Exponentials $(1,\ 0.0498,\ 0.0183)$, sum 1.0681, $\ln 1.0681 = 0.0659$. Log-probabilities $(-0.0659,\ -3.0659,\ -4.0659)$; probabilities $(0.9362,\ 0.0466,\ 0.0171)$. Loss $= -\ln p_1 = 3.0659$ (large: the network is confident in the wrong class). $\mathbf{P} - \mathbf{Y} = (0.9362,\ -0.9534,\ 0.0171)$.
3. $\sqrt{2/256} = 0.0884$. Using 1 instead multiplies σ_W^2 by $1/0.0078 = 128$: the pre-activation variance grows by a factor of about 128 per layer, so after a few layers the activations overflow or the softmax saturates (initial loss far above $\ln K$).
4. A smaller step slows down whatever the broken update does. In our run with $\eta = 0.1$ there is no overflow within 1000 epochs, but the accuracy peaks at 86 % and ends at 75 % (loss 0.70), far from the 98 % of the correct code. The broken "gradient" is not the derivative of any loss we are minimising (it is the gradient of a different, linear network evaluated with ReLU activations), so a stationary point of the broken update is not a minimum of our loss, and no learning rate makes the two coincide. The cure is to fix the code, which the gradient check pinpoints.
5. The output layers pass, so everything from the loss down to $\boldsymbol{\delta}_2$ and $\partial L/\partial \mathbf{W}_2$ is correct. The bug is in what is computed after that: the propagation $\boldsymbol{\delta}_2 \mathbf{W}_2^{\mathsf{T}}$, the ReLU mask of layer 1 ($\mathbf{Z}_1 > 0$), or the formulas for $\partial L/\partial \mathbf{W}_1 = \mathbf{X}^{\mathsf{T}} \boldsymbol{\delta}_1$ and $\partial L/\partial \mathbf{b}_1$.
6. float32 has about 7 significant digits. $L(w \pm \varepsilon)$ differ in the 5th–6th significant digit (the change is $\approx \varepsilon L'$, i.e. about 10^{-5} relative), so after subtraction only 1–2 digits remain; the relative error of the numerical gradient is 10^{-2}–10^{-1}. Choosing $\varepsilon = 10^{-3}$ helps a little (more digits survive, but truncation error grows to $\sim 10^{-6}$); the real fix is float64.
7. With $\sigma_\phi = 0$ the channel is pure AWGN, nearest point is exactly ML, and the MLP can at best match it (in practice it is very slightly worse). With $\sigma_\phi = 0.3$ the arcs of the outer points overlap heavily; all detectors get a much higher floor, but the gap between nearest point and the learned/ML detectors grows, because the straight boundaries are even more mismatched.
8. Add `return loss, dlogits, P;` then `accuracy = np.mean(predict(params, X) == y)`. The softmax $p_k = e^{z_k} / \sum_j e^{z_j}$ is strictly increasing in z_k with the same denominator for every k, so $\arg\max_k p_k = \arg\max_k z_k$.

Self-test quiz

1. A freshly initialised 10-class network has a training loss of 2.30. Is something wrong?
   No: $\ln 10 = 2.303$*, the loss of uniform predictions. That is exactly what we expect.*
2. Why does `forward` not apply a softmax on the last layer?
   The softmax is computed inside the loss with the log-sum-exp trick, which is numerically stable and gives the simple gradient $(\mathbf{P} - \mathbf{Y})/N$*; prediction only needs the argmax of the logits.*
3. What is the gradient of $\frac{\lambda}{2}\|\mathbf{W}\|^2$ with respect to $\mathbf{W}$?
   $\lambda\mathbf{W}$.
4. The gradient check of a correct ReLU network gives 5×10^{-2} for one bias vector only. Name a harmless explanation.
   Some pre-activations lie within ε *of zero (on the ReLU kink), e.g. an input exactly at the origin with zero biases; the finite difference then averages the left and right slopes.*
5. Removing all activation functions from a 5-layer MLP gives what kind of model?
   A linear (affine) model: the product of the weight matrices is a single matrix.
6. Why can a learned 16-QAM detector beat nearest-point detection under phase noise?
   Phase noise bends the noise clouds into arcs, so the optimal (ML) boundaries are curved; nearest point uses straight boundaries and suffers an error floor, while the MLP learns curved ones.

Further reading

- A. Karpathy et al., *CS231n: Convolutional Neural Networks for Visual Recognition*, Stanford course notes, "Neural Networks Case Study" and "Gradient checks" sections (the source of the spiral example).
- K. He, X. Zhang, S. Ren, J. Sun, "Delving deep into rectifiers: surpassing human-level performance on ImageNet classification," *Proc. IEEE ICCV*, 2015 (He initialisation).
- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016, Chapter 6 (MLPs and back-propagation) and Section 4.1 (overflow, underflow and the softmax).
- M. Nielsen, *Neural Networks and Deep Learning*, Determination Press, 2015 (free online), Chapters 1–2: a network written from scratch in NumPy.
- T. O'Shea and J. Hoydis, "An introduction to deep learning for the physical layer," *IEEE Trans. Cognitive Communications and Networking*, vol. 3, no. 4, 2017 (learned transmitters and receivers; Part V of this book).

Chapter 14

Lab: PyTorch and an End-to-End Project

In Chapter 13 you wrote every line of a neural network yourself, including the backward pass, and proved it correct with a gradient check. That was the hard way, and it paid off: you know exactly what a framework must do. Now we let a framework do it. **PyTorch** gives you three things: *tensors* (NumPy arrays that can live on a GPU), *autograd* (automatic, exact backpropagation through any code you write) and a library of building blocks (layers, losses, optimisers, data loaders, saving and export).

This chapter has three parts. First we learn PyTorch's core ideas one at a time, with tiny examples whose answers you can check by hand, and we rebuild the spiral network of Chapter 13 to show that PyTorch reproduces our NumPy numbers to twelve digits. Second, we carry out a complete **end-to-end project**: a handwritten-digit classifier, from raw data to a saved model that classifies a photo of your own handwriting, with all the engineering that real projects need — validation split, early stopping, checkpoints, a test set touched once, export to ONNX. Third, a wireless section builds a PyTorch `Dataset` for complex baseband IQ frames with on-the-fly noise augmentation, the data pipeline you will reuse in Parts V and VI.

14.1 Tensors: NumPy arrays with superpowers

Definition: tensor

A **tensor** is an n-dimensional array of numbers of a single type. Each tensor has a `shape` (size along each dimension), a `dtype` (number format, e.g. `float32`) and a `device` (where its memory lives: `cpu`, `cuda` for an NVIDIA GPU, `mps` for Apple silicon).

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
print(x.shape, x.dtype, x.device)   # torch.Size([2, 2]) torch.float32 cpu
print(x @ x)                        # matrix product: [[7, 10], [15, 22]]
print(x * x)                        # element-wise:   [[1, 4], [9, 16]]
n = torch.zeros(3, dtype=torch.int64)
a = x.numpy()                       # a NumPy VIEW of the same memory
a[0, 0] = 100.0                     # ... so this also changes x[0, 0]
```

Everything you know from NumPy carries over: indexing, slicing, broadcasting, `@` for matrix products, `sum(dim=...)` instead of `sum(axis=...)`. Check by hand: $\left(\begin{smallmatrix}1&2\\3&4\end{smallmatrix}\right)^2$ has top-left entry $1\cdot1+2\cdot3=7$.

dtype. PyTorch's default floating type is `float32` (NumPy's is `float64`). Single precision is plenty for training and twice as fast and compact, but it is *not* enough for numerical gradient checks (Section 14.6). Class labels must be integers (`int64`, called `torch.long`). Mixing types is a classic error: `torch.tensor(np_array)` on a NumPy `float64` array gives a `float64` tensor, and multiplying it by `float32` weights raises "expected scalar type Float but found Double". Fix it with `.float()` or `dtype=torch.float32`.

device. A computation runs on the device where its tensors live, and all tensors in one operation must be on the same device. The standard idiom picks the best device available:

```
device = ("cuda" if torch.cuda.is_available() else
          "mps" if torch.backends.mps.is_available() else "cpu")
model = model.to(device)              # moves all parameters
xb, yb = xb.to(device), yb.to(device) # move each batch inside the loop
```

`model.to(device)` moves the parameters in place; for a tensor, `.to(device)` returns a *new* tensor, so you must assign the result. The sharing trick with `.numpy()` works only on the CPU; for a GPU tensor use `t.cpu().numpy()`, and for a tensor that requires gradients first call `.detach()`.

14.2 Autograd: backpropagation for free

Intuition: a tape recorder

When a tensor has `requires_grad=True`, PyTorch records every operation performed on it, building the computational graph of Chapter 10 on the fly. Calling `.backward()` on a scalar result plays the tape in reverse, applying the chain rule at each recorded node, and deposits $\partial(\text{result})/\partial(\text{leaf})$ in the `.grad` field of every leaf tensor that asked for it. You write only the forward pass; the backward pass of Chapter 13 is generated for you.

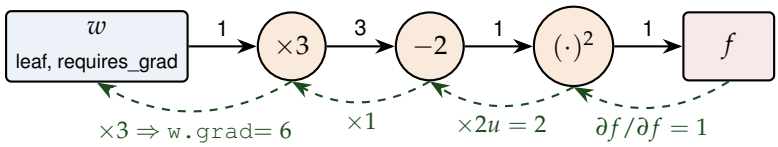


Figure 14.1: The graph recorded for $f = (3w-2)^2$ at $w = 1$ (forward values on the solid arrows; $u = 3w - 2 = 1$). `f.backward()` walks the dashed green arrows from right to left, multiplying local derivatives (chain rule), and stores the result in `w.grad`.

```
w = torch.tensor(1.0, requires_grad=True)
f = (3 * w - 2) ** 2
f.backward()
print(w.grad)           # tensor(6.)    by hand: 2*(3*1-2)*3 = 6
```

14.2.1 The gradient accumulates!

This is the single most important fact about autograd, and the source of the most common PyTorch bug. `.backward()` *adds* the new gradient to whatever is already stored in `.grad`; it does not overwrite it. Our script (`code/ch14/torch_basics.py`) continues:

```
f = (3 * w - 2) ** 2;  f.backward()
print(w.grad)          # tensor(12.)   -- 6 + 6, NOT 6
w.grad.zero_()         # reset (in place: trailing underscore)
f = (3 * w - 2) ** 2;  f.backward()
print(w.grad)          # tensor(6.)
```

Why would anyone design it this way? Because accumulation is exactly what the chain rule needs when a parameter is used in several places (the gradient is the *sum* of contributions from

each use, Chapter 10), and it lets you split a batch that does not fit in memory into pieces and add their gradients ("gradient accumulation"). The price is that in a training loop you must clear the gradients before every backward pass, with `optimizer.zero_grad()`. Forget it, and each step uses the sum of all gradients since the start of training — the effective learning rate grows without bound and training goes haywire.

Note also that each `backward()` frees the recorded graph to save memory; that is why we recompute `f` before calling `backward()` again.

Worked example: autograd on one neuron, checked by hand

Inputs $\mathbf{x} = (1, -2)$, weights $\mathbf{w} = (0.5, -0.25)$, bias $b = 0.1$, target 2, loss $L = (\mathrm{ReLU}(z) - 2)^2$ with $z = \mathbf{w} \cdot \mathbf{x} + b$. By hand: $z = 0.5 \cdot 1 + (-0.25)(-2) + 0.1 = 1.1 > 0$, so $\mathrm{ReLU}(z) = 1.1$ and $L = (1.1 - 2)^2 = 0.81$. Backward: $\partial L/\partial z = 2(1.1 - 2) \cdot 1 = -1.8$ (the ReLU passes the gradient because $z > 0$); $\partial L/\partial \mathbf{w} = -1.8\,\mathbf{x} = (-1.8,\ 3.6)$; $\partial L/\partial b = -1.8$.

```
xv = torch.tensor([1.0, -2.0])
wv = torch.tensor([0.5, -0.25], requires_grad=True)
b  = torch.tensor(0.1, requires_grad=True)
loss = (torch.relu(wv @ xv + b) - 2.0) ** 2
loss.backward()
print(loss.item(), wv.grad, b.grad)   # 0.81  tensor([-1.8, 3.6])  tensor(-1.8)
```

(The script prints 0.80999994 and −1.7999999: float32 round-off.)

14.2.2 Switching autograd off

During evaluation we do not need gradients, and recording the graph wastes memory and time. The context manager `with torch.no_grad():` turns recording off for everything inside it. Similarly, `t.detach()` returns a copy of a tensor cut off from the graph, and `t.item()` converts a one-element tensor to a Python number. Use `loss.item()` when logging — appending the tensor `loss` itself to a list keeps every iteration's graph alive and slowly exhausts memory.

14.3 Modules, losses and optimisers

14.3.1 `nn.Module` and `nn.Sequential`

A **module** is an object that holds parameters and defines a `forward` method. `nn.Linear(d_in, d_out)` is a module holding a weight of shape $(d_{\text{out}}, d_{\text{in}})$ and a bias of shape (d_{out}) and computing $\mathbf{X}\mathbf{W}^{\mathsf{T}} + \mathbf{b}$. Note the transpose: PyTorch stores weights as (out, in), the transpose of our Chapter 13 convention. Modules nest; `nn.Sequential` chains modules so that the output of one is the input of the next:

```
import torch.nn as nn
model = nn.Sequential(nn.Linear(2, 100), nn.ReLU(),
                      nn.Linear(100, 100), nn.ReLU(),
                      nn.Linear(100, 3))          # logits, no softmax
logits = model(X)                                 # calls forward()
print(sum(p.numel() for p in model.parameters())) # 10703, as in Ch. 13
```

For anything that is not a straight chain (skip connections, two inputs, custom logic), subclass `nn.Module` yourself:

```python
class MLP(nn.Module):
    def __init__(self, sizes=(2, 100, 100, 3)):
        super().__init__()                          # always call this first
        self.layers = nn.ModuleList(
            [nn.Linear(a, b) for a, b in zip(sizes[:-1], sizes[1:])])
    def forward(self, x):
        for layer in self.layers[:-1]:
            x = torch.relu(layer(x))
        return self.layers[-1](x)                   # logits
```

Assigning a module (or an `nn.Parameter`) as an attribute of the model registers it, so that `model.parameters()`, `model.to(device)` and `model.state_dict()` find it automatically. (A plain Python list of layers is *not* registered — hence `nn.ModuleList`.)

14.3.2 Loss functions

`nn.CrossEntropyLoss()` takes *logits* of shape (N, K) and integer labels of shape (N), and computes the mean softmax cross-entropy with the log-sum-exp trick — exactly what our function `softmax_cross_entropy` did. Do not put a softmax layer in the model before it — the loss would apply softmax twice and training would stall. Two other common losses are `nn.MSELoss`, for regression, and `nn.BCEWithLogitsLoss`, for binary or multi-label outputs (it includes the sigmoid).

14.3.3 Optimisers

An optimiser holds references to the parameters and implements an update rule from Chapter 9 and Part II:

```python
opt = torch.optim.SGD(model.parameters(), lr=0.5, weight_decay=1e-3)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
```

The call `opt.step()` applies one update using the current gradient `p.grad` of every parameter `p`; `opt.zero_grad()` clears them. For SGD, `weight_decay`=λ adds $\lambda \mathbf{W}$ to the gradient — our L2 term — but note that it also decays the biases unless you put them in a separate parameter group.

14.4 The canonical training loop

Every PyTorch training script, from this chapter's tiny MLP to a billion-parameter Transformer, contains the same five lines:

Key equation: the five lines of PyTorch training

```python
opt.zero_grad()                 # 1. clear old gradients
logits = model(xb)              # 2. forward pass
loss = loss_fn(logits, yb)      # 3. loss
loss.backward()                 # 4. backward pass: fills p.grad
opt.step()                      # 5. update: p -= lr * p.grad (for SGD)
```

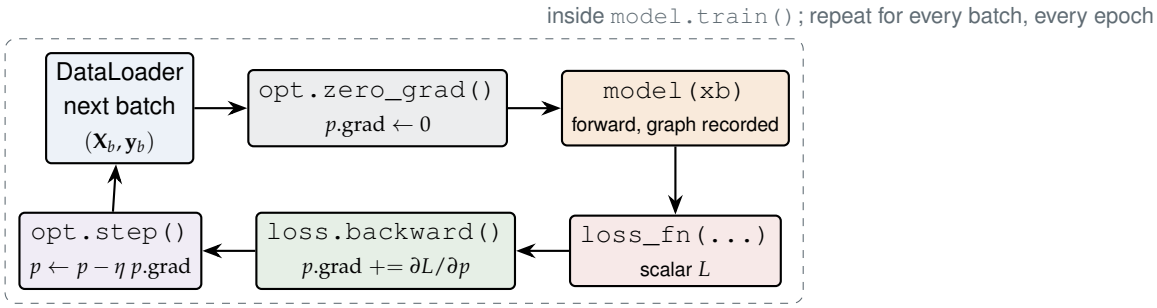


Figure 14.2: The canonical PyTorch training loop. Note += in the backward step: gradients accumulate, which is why `zero_grad()` must come before every backward pass.

14.4.1 `train()` and `eval()` modes

Some layers behave differently during training and evaluation. **Dropout** zeroes a random fraction p of activations during training (and scales the survivors by $1/(1-p)$ so the expected value is unchanged) but must do nothing at evaluation time; **batch normalisation** uses batch statistics during training and stored running averages during evaluation. `model.train()` and `model.eval()` switch every such layer in the model. They do *not* switch autograd off — that is the job of `torch.no_grad()` — so evaluation code needs both:

```
model.eval()
with torch.no_grad():
    logits = model(x_val)
model.train()          # remember to switch back before training again
```

14.5 The spiral network, again: PyTorch versus NumPy

To convince ourselves that PyTorch does exactly what we did by hand, we build the $[2, 100, 100, 3]$ network in PyTorch, copy into it the *same* He-initialised weights that `nn_numpy.init_params` produced (transposed, because PyTorch stores $\mathbf{W}^{\mathsf{T}}$), switch to float64 with `.double()`, and train both implementations on identical mini-batches.

Listing 14.1: Building the PyTorch twin of the Chapter 13 network (`spiral_torch.py`).

```
model = tnn.Sequential(
    tnn.Linear(2, 100), tnn.ReLU(),
    tnn.Linear(100, 100), tnn.ReLU(),
    tnn.Linear(100, 3),
).double()

# copy the NumPy He-initialised weights into the PyTorch layers
p_np = npnet.init_params([2, 100, 100, 3], seed=1)
linears = [m for m in model if isinstance(m, tnn.Linear)]
with torch.no_grad():
    for l, lin in enumerate(linears, start=1):
        lin.weight.copy_(torch.tensor(p_np[f"W{l}"].T))    # PyTorch stores (out, in)
        lin.bias.copy_(torch.tensor(p_np[f"b{l}"]))
```

Line 31 is the key: `copy_` writes into the existing parameter (in place) and it must happen inside `torch.no_grad()` (line 29), otherwise autograd would record the copy as part of the

model. Inside the training loop, the PyTorch step is the five canonical lines (plus the L2 term added to the loss), and the NumPy step uses the functions of Chapter 13. The output:

```
initial loss  NumPy 1.424037275129  PyTorch 1.424037275129
layer 1: max |dW diff| 6.9e-18   max |db diff| 2.3e-17
layer 2: max |dW diff| 2.8e-17   max |db diff| 4.2e-17
layer 3: max |dW diff| 5.6e-17   max |db diff| 1.6e-16
after 300 epochs: loss NumPy 0.1660171894  PyTorch 0.1660171894  max diff 7.7e-12
```

(The initial loss now includes the L2 term, $1.235 + 0.189$.) At initialisation the gradients agree to 10^{-16}, the limit of float64 arithmetic: PyTorch's automatic differentiation and our hand-derived backward pass compute the same numbers, only summed in a slightly different order. After 1500 SGD steps the two loss curves still agree to 10^{-11} (Fig. 14.3). Your NumPy network was right, and PyTorch's autograd is doing nothing mysterious.

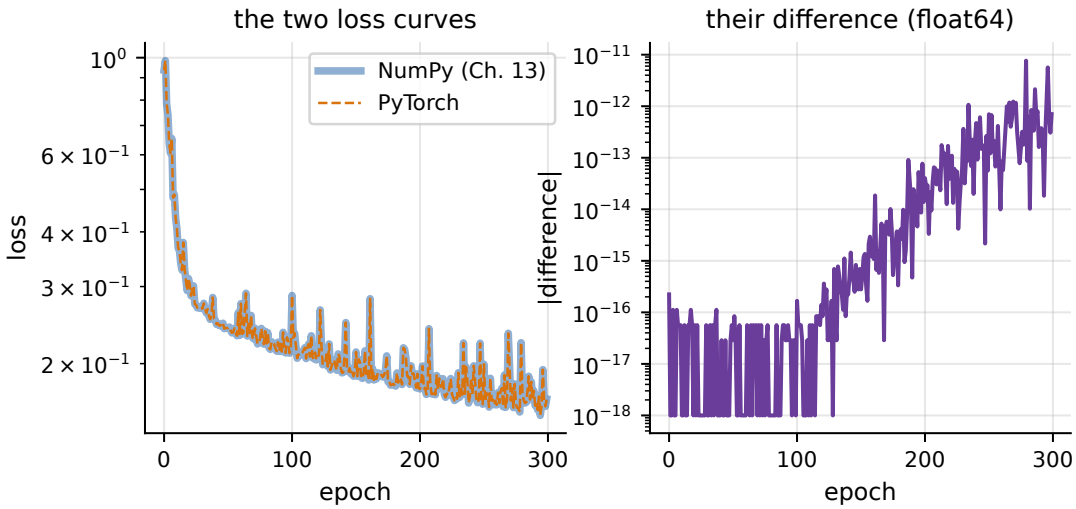


Figure 14.3: Left: the loss curves of the NumPy network (thick blue) and its PyTorch twin (thin dashed orange), trained on the same batches from the same initial weights, lie exactly on top of each other. Right: their absolute difference starts at 10^{-16} (float64 round-off) and grows slowly, as tiny round-off differences are amplified by 1500 SGD steps, reaching only about 10^{-11}.

14.6 Gradient checking in PyTorch

Autograd is reliable for built-in operations, but when you write your own differentiable function (a custom `torch.autograd.Function` with a hand-written `backward`, e.g. for a channel model or a quantiser), you must verify it exactly as in Chapter 13. PyTorch provides the auditor:

```
def tiny_mlp(X, W1, b1, W2, b2):
    H = torch.relu(X @ W1 + b1)
    return nn.functional.cross_entropy(H @ W2 + b2, torch.tensor([0, 1, 2, 1]))

args = [torch.randn(4, 2, dtype=torch.float64)] + \
       [torch.randn(*s, dtype=torch.float64, requires_grad=True)
        for s in [(2, 5), (5,), (5, 3), (3,)]]
print(torch.autograd.gradcheck(tiny_mlp, args, eps=1e-6, atol=1e-4))    # True
```

`gradcheck` perturbs every input that requires gradients by $\pm$`eps`, forms centred differences and compares them with autograd's answer, raising an error on mismatch. Run the same check with

float32 inputs and it fails with "Jacobian mismatch" (plus a warning that inputs are not double precision) even though nothing is wrong — the same digit-cancellation effect we discussed in Chapter 13. Always gradient-check in float64.

14.7 End-to-end project: a handwritten-digit classifier

We now build a complete project. The classic dataset is **MNIST**: 70 000 greyscale images of handwritten digits, 28×28 pixels, split into 60 000 training and 10 000 test images. Below is the full MNIST script for you to run on your own computer (it downloads about 12 MB). Because the machine that produced this book cannot download data, we then run the *identical* pipeline on scikit-learn's bundled `load_digits` dataset — 1797 images of 8×8 pixels — and show those real results. Only the data-loading lines and the input size differ.

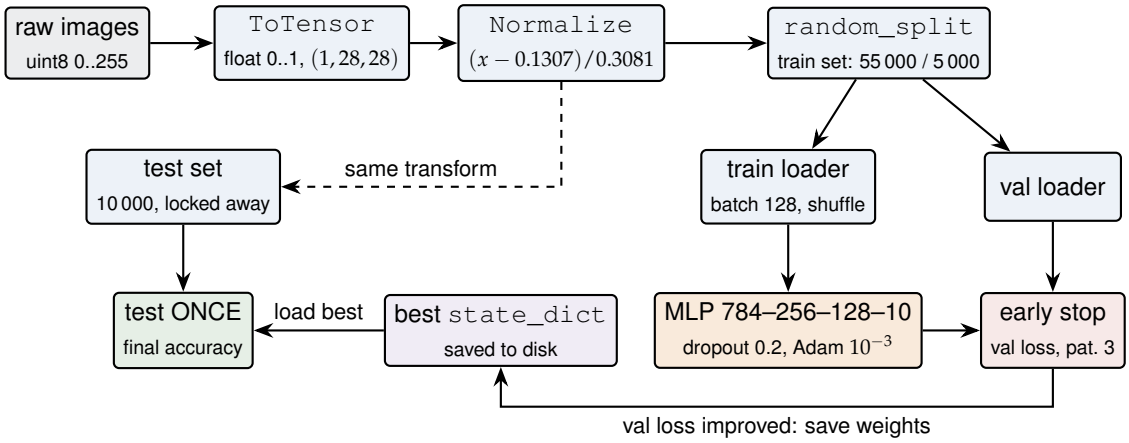


Figure 14.4: The end-to-end data and training pipeline. The test set receives the same transform as the training data but is never used for any decision; only the final, best model (chosen on the validation set) is evaluated on it, once.

14.7.1 The MNIST script

Listing 14.2: `mnist_mlp.py`, part 1: data and model.

```
import copy, torch, torch.nn as nn
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms

torch.manual_seed(0)
device = ("cuda" if torch.cuda.is_available() else
          "mps" if torch.backends.mps.is_available() else "cpu")

# ---------------- 1. data
tf = transforms.Compose([transforms.ToTensor(),                    # [0,255] ->
    [0,1]
                         transforms.Normalize((0.1307,), (0.3081,))])
full = datasets.MNIST("data", train=True, download=True, transform=tf)
test = datasets.MNIST("data", train=False, download=True, transform=tf)
train_set, val_set = random_split(full, [55000, 5000],
                                  generator=torch.Generator().manual_seed(0))
train_dl = DataLoader(train_set, batch_size=128, shuffle=True)
val_dl = DataLoader(val_set, batch_size=1000)
test_dl = DataLoader(test, batch_size=1000)
```

```
21
22 # ---------------- 2. model
23 model = nn.Sequential(
24     nn.Flatten(),                                  # (B,1,28,28) -> (B,784)
25     nn.Linear(784, 256), nn.ReLU(), nn.Dropout(0.2),
26     nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.2),
27     nn.Linear(128, 10)).to(device)                 # logits
28 loss_fn = nn.CrossEntropyLoss()
29 opt = torch.optim.Adam(model.parameters(), lr=1e-3)
```

- **Lines 8–9** choose the device (Section 14.1).
- **Line 12**: `ToTensor` converts a PIL image with pixels 0–255 into a float tensor of shape $(1, 28, 28)$ (channel, height, width) with values in $[0, 1]$.
- **Line 13**: `Normalize((0.1307,),(0.3081,))` computes $(x - \mu)/\sigma$ with $\mu = 0.1307$ and $\sigma = 0.3081$, the mean and standard deviation of all MNIST training pixels. Centred, unit-variance inputs suit He initialisation and make optimisation better conditioned (Chapter 9). The tuples have one entry because there is one channel.
- **Lines 14–15**: the standard training and test sets; the transform is applied lazily each time an image is read.
- **Lines 16–17**: `random_split` carves 5000 images out of the training set for validation; the seeded generator makes the split reproducible.
- **Lines 18–20**: `DataLoader`s deliver batches. Only the training loader shuffles; evaluation order does not matter, and large batches (1000) make evaluation fast.
- **Lines 23–27**: the model. `nn.Flatten` turns $(B, 1, 28, 28)$ into $(B, 784)$. Two hidden layers (256 and 128 ReLUs), each followed by `Dropout(0.2)`, and a 10-logit output. Parameters: $784 \cdot 256 + 256 + 256 \cdot 128 + 128 + 128 \cdot 10 + 10 = 235\,146$.
- **Lines 28–29**: cross-entropy on logits and the Adam optimiser with the usual learning rate 10^{-3}.

Listing 14.3: `mnist_mlp.py`, part 2: evaluation, early stopping, saving, testing.

```
31 def evaluate(dl):
32     model.eval()
33     total, correct, loss_sum = 0, 0, 0.0
34     with torch.no_grad():
35         for xb, yb in dl:
36             xb, yb = xb.to(device), yb.to(device)
37             logits = model(xb)
38             loss_sum += loss_fn(logits, yb).item() * len(yb)
39             correct += (logits.argmax(1) == yb).sum().item()
40             total += len(yb)
41     return loss_sum / total, correct / total
42
43 # ---------------- 3. training with early stopping
44 best_val, best_state, patience, bad = float("inf"), None, 3, 0
45 for epoch in range(50):
46     model.train()
47     for xb, yb in train_dl:
48         xb, yb = xb.to(device), yb.to(device)
49         opt.zero_grad()
50         loss = loss_fn(model(xb), yb)
```

```
51          loss.backward()
52          opt.step()
53      val_loss, val_acc = evaluate(val_dl)
54      print(f"epoch {epoch:2d}  val loss {val_loss:.4f}  val acc {val_acc:.4f}")
55      if val_loss < best_val:
56          best_val, bad = val_loss, 0
57          best_state = copy.deepcopy(model.state_dict())
58      else:
59          bad += 1
60          if bad >= patience:
61              print("early stopping"); break
62
63  # ---------------- 4. save, reload, test ONCE
64  torch.save(best_state, "mnist_mlp.pt")
65  model.load_state_dict(torch.load("mnist_mlp.pt", weights_only=True))
66  test_loss, test_acc = evaluate(test_dl)
67  print(f"TEST accuracy {test_acc:.4f}")
```

- **Lines 31–41**: `evaluate` switches to eval mode (dropout off) and disables autograd, then accumulates the *sum* of losses (`loss_fn` returns a batch mean, so we multiply by the batch size) and the number of correct predictions. Dividing at the end gives exact averages even if the last batch is smaller.
- **Lines 45–52**: one epoch of the canonical loop. Line 46 matters: `evaluate` left the model in eval mode, so we switch dropout back on at the start of each epoch.
- **Lines 55–61**: **early stopping**. Whenever the validation loss improves we remember a deep copy of the weights (`state_dict()` returns references to the live tensors, which keep changing, hence `copy.deepcopy`). If it fails to improve for 3 consecutive epochs (the *patience*), we stop.
- **Line 64** saves the best weights; **line 65** loads them back with `weights_only=True` (Section 14.8).
- **Line 66**: the test set is used exactly once, at the very end. If you looked at test accuracy after every change and kept the best, the test set would silently become a second validation set and your final number would be optimistically biased.

Intuition: three data sets, three jobs

The *training* set is the textbook the student studies. The *validation* set is the mock exam used to decide when to stop studying and which study method works best. The *test* set is the real exam, sat once. A student who saw the real exam paper in advance gets a high mark that says nothing about what they know.

14.7.2 Running the same pipeline on `load_digits`

Our offline version, `code/ch14/digits_mlp.py`, differs only in the first block:

Listing 14.4: `digits_mlp.py`: the data block replaced for the offline 8×8 digits.

```
19  digits = load_digits()
20  X_all = torch.tensor(digits.images, dtype=torch.float32) / 16.0   # -> [0,1]
21  y_all = torch.tensor(digits.target, dtype=torch.int64)
22  g = torch.Generator().manual_seed(0)
23  perm = torch.randperm(len(X_all), generator=g)
```

```
test_idx, rest_idx = perm[:360], perm[360:]                          # 20 % held out
mean = X_all[rest_idx].mean().item(); std = X_all[rest_idx].std().item()
print(f"pixel mean {mean:.4f}  std {std:.4f}  (MNIST: 0.1307, 0.3081)")
X_norm = ((X_all - mean) / std).unsqueeze(1)                         # (N,1,8,8)
full = TensorDataset(X_norm[rest_idx], y_all[rest_idx])
test = TensorDataset(X_norm[test_idx], y_all[test_idx])
n_val = 287
train_set, val_set = random_split(full, [len(full) - n_val, n_val],
                                  generator=torch.Generator().manual_seed(0))
print(f"train {len(train_set)}  val {len(val_set)}  test {len(test)}")
train_dl = DataLoader(train_set, batch_size=64, shuffle=True)
val_dl = DataLoader(val_set, batch_size=1000)
test_dl = DataLoader(test, batch_size=1000)
```

The images have values 0–16, so line 20 divides by 16. There is no official test split, so lines 22–24 hold out a random 20 % (360 images) as the test set *before* computing anything else; line 25 computes the normalisation statistics on the remaining images only (computing them on all data would leak information from the test set). The printed statistics are $\mu = 0.3056$, $\sigma = 0.3761$ — different from MNIST's, because these digits fill their small frame. Lines 30–32 split the rest into 1150 training and 287 validation images. Each training batch has shape $(64, 1, 8, 8)$, dtype `float32`, with labels $(64,)$ `int64`. The model is the same MLP with 64 inputs instead of 784 (50 826 parameters); everything from `evaluate` onwards is character-for-character the MNIST code.

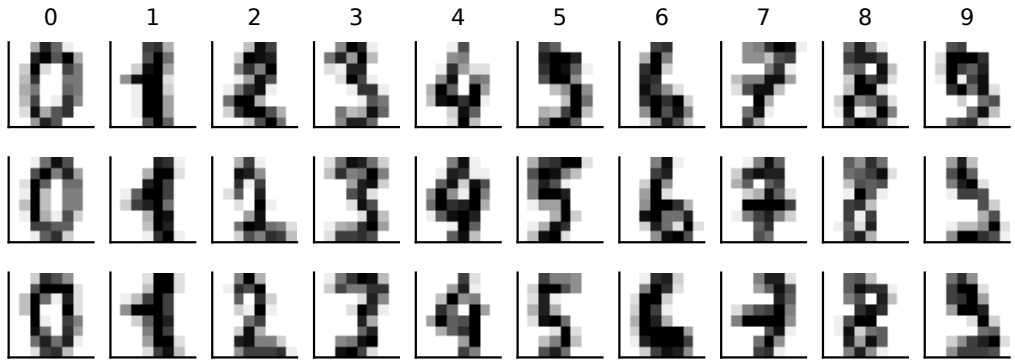


Figure 14.5: Three examples of each class from `load_digits`. At 8×8 pixels the digits are coarse — a 4 and a 9, or a 3 and a 5, can differ by only a few pixels — yet an MLP classifies them well.

14.7.3 Results

Training took 17 seconds on one CPU core. The log (abridged):

```
epoch  0  train loss 1.9706  val loss 1.4383  val acc 0.8502
epoch  5  train loss 0.1716  val loss 0.1186  val acc 0.9721
epoch 20  train loss 0.0398  val loss 0.0409  val acc 0.9861
epoch 23  train loss 0.0328  val loss 0.0502  val acc 0.9861
early stopping at epoch 23; best epoch 20
TEST loss 0.0620  TEST accuracy 0.9833
```

The final **test accuracy is 98.3 %** (354 of 360). On full MNIST, the same 784–256–128–10 network with dropout typically reaches roughly 98 % test accuracy after a few epochs; run `mnist_mlp.py` and see what you get. Figures 14.6 to 14.8 show how to look at a classifier: curves to diagnose training, the confusion matrix to find which classes are confused, and the misclassified examples

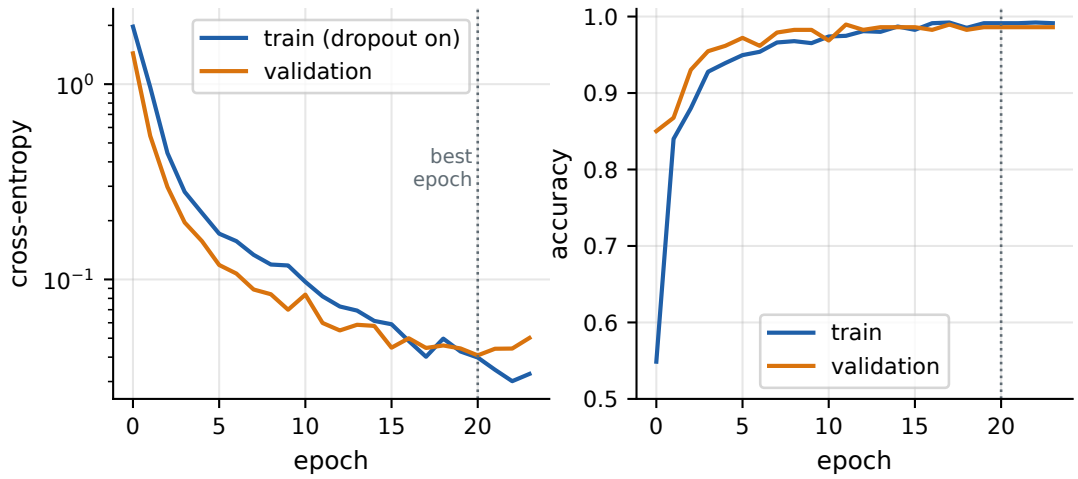


Figure 14.6: Learning curves. Left: cross-entropy (log scale) on the training set (averaged during the epoch, dropout on) and on the validation set. Right: accuracy. The validation loss reaches its minimum at epoch 20 (dotted line) and rises for the next three epochs, triggering early stopping with patience 3; the weights from epoch 20 are restored. Early on the validation loss is *below* the training loss: the training figure is measured with dropout active and averaged over an epoch in which the network was still improving.

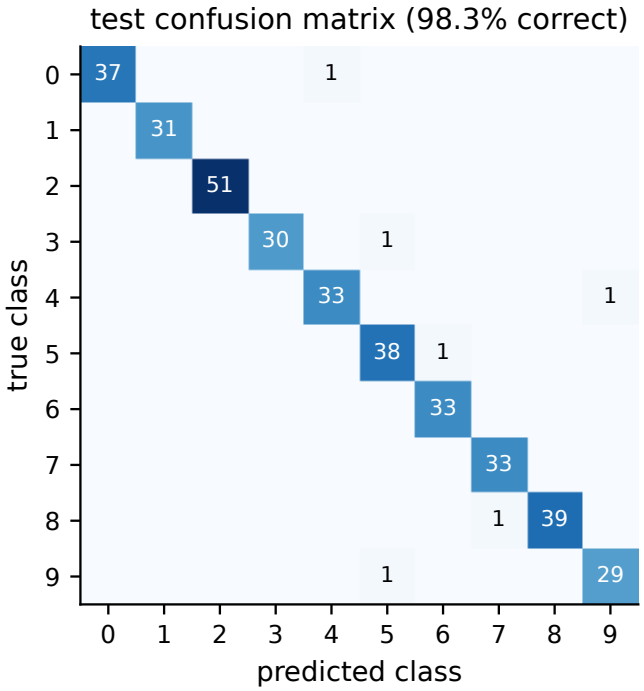


Figure 14.7: Confusion matrix on the 360 held-out test images: row = true digit, column = predicted digit. Almost all mass is on the diagonal (354 correct, 98.3 %). The six errors are scattered: $0 \to 4$, $3 \to 5$, $4 \to 9$, $5 \to 6$, $8 \to 7$, $9 \to 5$ — pairs that share strokes at low resolution.

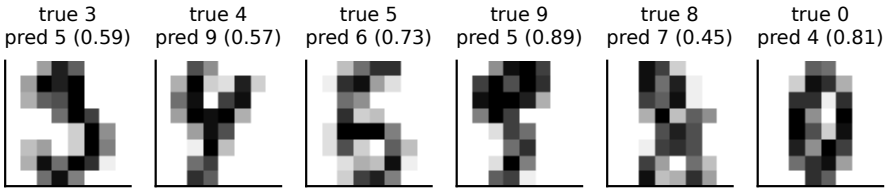


Figure 14.8: All six misclassified test images, with the true label, the prediction and the softmax probability of the prediction. Several are genuinely ambiguous at 8×8 (the "4" predicted as 9, the "9" predicted as 5). Note that the network is often only moderately confident (0.45–0.89) when wrong — the probabilities carry useful information.

to understand *why*. Always look at your errors; they often reveal label noise, preprocessing bugs or a class that needs more data.

14.7.4 Predicting your own handwriting

The real test of a model is data it has never seen *from a different source*. Write a digit on paper, photograph it and classify it:

Listing 14.5: `predict_image.py`: preprocessing a photo exactly like MNIST.

```
11 def predict_image(path, weights="mnist_mlp.pt"):
12     model = build_model()
13     model.load_state_dict(torch.load(weights, weights_only=True))
14     model.eval()                                        # dropout OFF
15     img = Image.open(path).convert("L")                 # 1. greyscale
16     img = ImageOps.invert(img)                          # 2. MNIST = white ink on
       black
17     img = img.point(lambda p: 255 if p > 50 else 0)     #    remove paper texture
18     img = img.crop(img.getbbox())                       # 3. tight box around the ink
19     w, h = img.size; s = 20 / max(w, h)                 # 4. longest side -> 20 px
20     img = img.resize((max(1, round(w * s)), max(1, round(h * s))), Image.LANCZOS)
21     canvas = Image.new("L", (28, 28), 0)                # 5. paste centred in 28x28
22     canvas.paste(img, ((28 - img.size[0]) // 2, (28 - img.size[1]) // 2))
23     x = torch.tensor(np.array(canvas), dtype=torch.float32) / 255.0
24     x = (x - 0.1307) / 0.3081                           # 6. SAME normalisation
25     with torch.no_grad():
26         probs = torch.softmax(model(x[None, None]), dim=1)[0]   # (1,1,28,28)
27     return probs.argmax().item(), probs
```

The network was trained on images with very specific properties, and your photo has none of them. Every line of preprocessing undoes one difference:

1. **Line 15**: convert to a single greyscale channel (a phone photo has three colour channels).
2. **Line 16**: **invert**. MNIST digits are white ink (≈ 255) on a black background (0); your photo is dark ink on white paper. Without inversion the network sees a white page with a black digit — something it has never seen.
3. **Line 17**: threshold away the grey texture and shadows of the paper, which MNIST does not have (its background is exactly 0).
4. **Lines 18–22**: **size and position**. MNIST digits were scaled so that they fit a 20×20 box and were then centred in the 28×28 frame (by centre of mass). We crop to the ink, scale the longest side to 20 pixels, keeping the aspect ratio, and paste the result in the middle. A digit

that fills the whole photo or sits in a corner looks nothing like the training data to an MLP, which has no built-in notion of translation.

5. **Lines 23–24**: **scale to** $[0,1]$ **and normalise with the training statistics** (0.1307, 0.3081) — never with statistics computed from your own image.
6. **Line 26**: add batch and channel dimensions, $(28,28) \to (1,1,28,28)$, and remember `model.eval()` (line 14) so dropout is off.

How much do these mistakes matter? We applied them to the trained digits model on its own test set (`preprocess_pitfalls.py`); Fig. 14.9 shows the result.

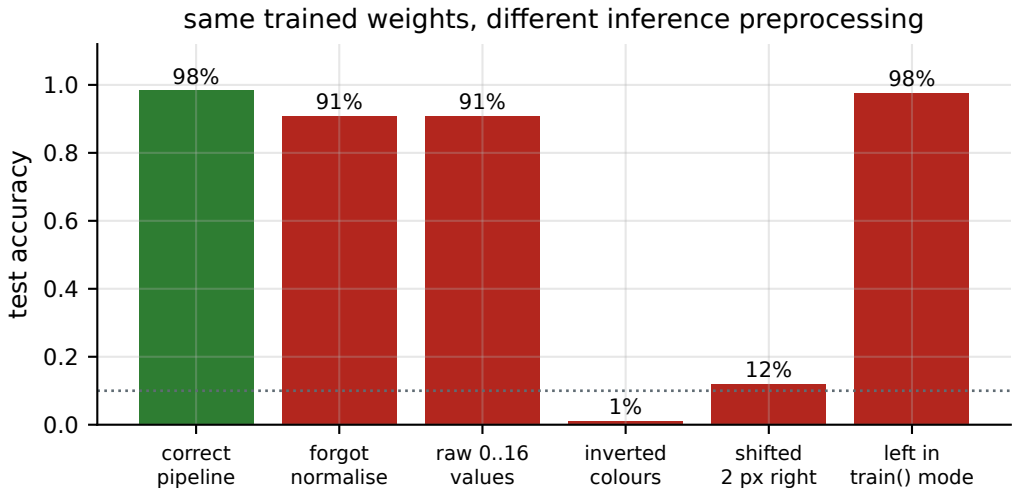


Figure 14.9: The same trained weights evaluated on the same test images with different inference preprocessing (dotted line: chance, 10 %). Forgetting the normalisation, or feeding raw 0–16 values, costs 7 points here (91 %); inverting the colours destroys the classifier (1 %, worse than chance); shifting the digits by just 2 of the 8 pixels gives 12 %. Leaving the model in `train()` mode (dropout active) costs little for this model (97.6 % on average) but makes predictions random from run to run.

Pitfall: training/serving skew

The most common reason a model that scored 98 % "does not work" on real inputs is that inference preprocessing differs from training preprocessing. Write one preprocessing function, use it in both places, and save the normalisation constants together with the weights (we store them in the checkpoint below).

14.8 Saving, loading and deploying

14.8.1 `state_dict` versus the whole model

A model's `state_dict()` is an ordered dictionary mapping parameter names (`"1.weight"`, `"1.bias"`, ...) to tensors. Saving it stores only numbers; to load, you rebuild the model from your code and pour the numbers in:

```
torch.save(model.state_dict(), "digits_mlp.pt")
model = build_model()                                  # same architecture
model.load_state_dict(torch.load("digits_mlp.pt", weights_only=True))
model.eval()
```

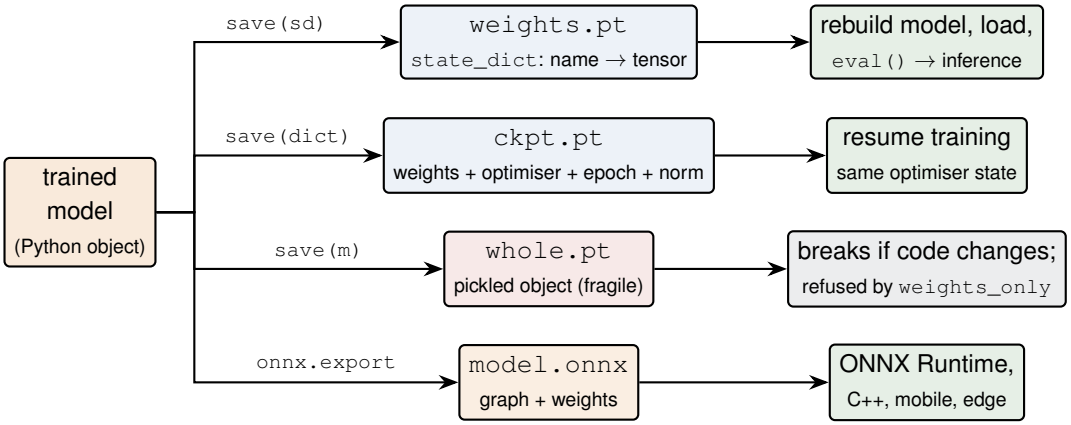


Figure 14.10: Ways to persist a model. The `state_dict` (top; `sd = model.state_dict()`) is the recommended format for weights; a checkpoint dictionary adds everything needed to resume training; pickling the whole model object ties the file to your exact code; ONNX export produces a framework-independent file for deployment.

The alternative, `torch.save(model, path)`, pickles the entire Python object. It looks convenient but the file stores references to your classes by module path, so renaming a file or class breaks loading, and unpickling can execute arbitrary code from an untrusted file.

Pitfall: PyTorch 2.6 and `weights_only`

Since PyTorch 2.6, `torch.load` defaults to `weights_only=True`: it will only unpickle tensors, primitive types and containers of them, which is safe to use on files from the internet. Loading a whole pickled model then fails with an `UnpicklingError` (our script demonstrates this). Use state dicts; if you must load an old whole-model file *that you trust*, pass `weights_only=False` explicitly.

14.8.2 Checkpoints for resuming training

Long training runs crash, get pre-empted or need to be continued. A checkpoint is a dictionary containing everything required to resume as if nothing happened:

```
ckpt = {"epoch": best_epoch, "model": best_state,
        "optimizer": opt.state_dict(),          # Adam's moment estimates!
        "val_loss": best_val, "norm": (mean, std)}
torch.save(ckpt, "digits_ckpt.pt")
ck = torch.load("digits_ckpt.pt", weights_only=True)   # plain types: allowed
model.load_state_dict(ck["model"]); opt.load_state_dict(ck["optimizer"])
start_epoch = ck["epoch"] + 1
```

The optimiser state matters: Adam keeps running averages of every gradient and squared gradient; restarting it from zero causes a burst of badly-scaled updates. We also store the normalisation constants so that inference code cannot use the wrong ones.

14.8.3 An inference function

A deployed model is wrapped in a function that takes raw input and returns an answer, hiding all details: `predict_image` above is an example. Good inference functions (i) load weights once, not per call; (ii) call `model.eval()` and use `torch.no_grad()` (or the faster

`torch.inference_mode()`); (iii) apply exactly the training preprocessing; (iv) accept a batch, since one call on 100 inputs is much faster than 100 calls; and (v) return probabilities as well as the decision, so the caller can reject low-confidence inputs.

14.8.4 Exporting to ONNX

ONNX (Open Neural Network Exchange) is a framework-independent file format describing a computation graph and its weights. Exporting lets you run the model without Python or PyTorch — in C++, on a phone, in a browser, or on embedded accelerators whose toolchains import ONNX.

Listing 14.6: `export_onnx.py`: export and run with ONNX Runtime.

```
11 model.eval()                                          # export the inference graph
12
13 dummy = torch.randn(1, 1, 8, 8)                       # example input: fixes
       shapes
14 onnx_path = os.path.join(HERE, "digits_mlp.onnx")
15 torch.onnx.export(model, (dummy,), onnx_path,
16                    input_names=["image"], output_names=["logits"],
17                    dynamic_shapes=({0: torch.export.Dim("batch")},),   # any batch
       size
18                    external_data=False)               # weights inside the one file
19 print("exported", os.path.basename(onnx_path), os.path.getsize(onnx_path), "bytes"
       )
20
21 import onnxruntime as ort
22 sess = ort.InferenceSession(onnx_path, providers=["CPUExecutionProvider"])
23 x = torch.randn(5, 1, 8, 8)
24 out_onnx = sess.run(["logits"], {"image": x.numpy()})[0]
25 with torch.no_grad():
26     out_torch = model(x).numpy()
27 print("ONNX Runtime output shape", out_onnx.shape,
28       " max |diff| vs PyTorch %.1e" % np.abs(out_onnx - out_torch).max())
```

The exporter runs the model on an example input (line 13) and records the operations. The argument `dynamic_shapes` declares that dimension 0 (the batch) may vary; without it the exported graph would accept batches of exactly one image. Older PyTorch versions use instead the argument `dynamic_axes`, written as `{"image": {0: "batch"}}`; recent versions export via `torch.export` by default and recommend `dynamic_shapes`. `external_data=False` keeps the weights inside the single `.onnx` file. Lines 21–24 load the file in **ONNX Runtime**, a separate high-performance inference engine, and run it on NumPy input. Our script prints:

```
exported digits_mlp.onnx 210796 bytes
ONNX Runtime output shape (5, 10)  max |diff| vs PyTorch 9.5e-07
```

The 210 kB file holds the 50 826 float32 weights ($50\,826 \times 4 = 203\,304$ bytes) plus the graph description, and its outputs match PyTorch to float32 round-off.

Wireless connection: from PyTorch to a base station

A learned receiver block (a demapper, a channel estimator) is trained in PyTorch on a GPU workstation but must run on very different hardware: an x86 server in a virtualised RAN, a GPU-accelerated L1 such as NVIDIA Aerial, or a DSP/FPGA in a small cell. ONNX is a common hand-off point: inference engines such as ONNX Runtime and TensorRT import

it, and many embedded toolchains do too. For a fixed-point DSP like the CEVA cores in a small-cell SoC, the exported graph is typically further quantised to int8/int16 — a topic that makes the simple, ReLU-based MLPs of this chapter attractive, since they need only multiply-accumulates and a comparison.

14.9 Reproducibility, devices and speed

Seeds. Random numbers appear in initialisation, data shuffling, dropout and data splits. To make a run repeatable, seed every generator at the start:

```
import random, numpy as np, torch
random.seed(0); np.random.seed(0); torch.manual_seed(0)  # also seeds CUDA
torch.use_deterministic_algorithms(True)   # optional: error on non-deterministic ops
```

Pass explicit `torch.Generator().manual_seed(...)` objects to `random_split` and, if needed, to `DataLoader`. Even with all seeds fixed, results can differ slightly across PyTorch versions, hardware and thread counts (floating-point addition is not associative, and parallel sums are added in varying orders). Report results averaged over several seeds rather than a single lucky run.

Devices. Write device-agnostic code: choose `device` once, move the model once, move every batch inside the loop. On a GPU, larger batches and `DataLoader(..., num_workers=4, pin_memory=True)` keep the device busy. Apple's `mps` backend supports most operations; if one is missing, setting the environment variable `PYTORCH_ENABLE_MPS_FALLBACK=1` runs it on the CPU. On a small CPU-only machine, `torch.set_num_threads(n)` controls parallelism (all scripts in this book use 1 thread so that several can run side by side).

Mixed precision. On modern GPUs, computing in 16-bit floating point roughly doubles speed and halves memory. PyTorch's automatic mixed precision runs suitable operations in float16 or bfloat16 while keeping the master weights in float32:

```
scaler = torch.amp.GradScaler("cuda")       # rescales the loss to avoid fp16 underflow
for xb, yb in train_dl:
    opt.zero_grad()
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        loss = loss_fn(model(xb), yb)
    scaler.scale(loss).backward()
    scaler.step(opt); scaler.update()
```

With `dtype=torch.bfloat16` (wider exponent range) the scaler is usually unnecessary.

14.10 Common PyTorch bugs

14.11 Wireless: a Dataset for complex IQ frames

Radio signals are complex-valued: each baseband sample is $r[n] = I[n] + jQ[n]$. Standard PyTorch layers expect real tensors, so the near-universal convention in deep learning for wireless

Bug	Symptom	Fix
missing `opt.zero_grad()`	loss erratic, explodes; grows worse with time	call it before every `backward()`
softmax before `CrossEntropyLoss`	training very slow, loss plateaus	feed raw logits
forgot `model.eval()`	noisy, slightly worse validation; predictions change between calls	`eval()` for evaluation, `train()` before training
evaluation without `no_grad`	memory grows, slow evaluation	wrap in `torch.no_grad()`
logging `loss` not `loss.item()`	memory leak over epochs	log Python numbers
labels as float or one-hot	dtype/shape error in the loss	`int64` class indices, shape $(N,)$
`float64` data, `float32` model	"expected Float but found Double"	`.float()` on inputs
tensors on different devices	"expected all tensors to be on the same device"	`.to(device)` for model and every batch
`x.to(device)` not assigned	same error	`x = x.to(device)`
`state_dict()` saved without copying	"best" model equals last model	`copy.deepcopy(model.state_dict())`
normalisation stats from all data	optimistic test score	compute on training data only
different preprocessing at inference	great test score, poor real-world results	one shared preprocessing function
validation set not shuffled out of training	validation too optimistic	disjoint splits (`random_split`)

Table 14.1: Frequent PyTorch mistakes, how they show up, and the cure. Most of them do not raise an error.

— used, for example, by the RadioML modulation-classification datasets — is to store a frame of L complex samples as a real tensor of shape $(2, L)$: row 0 holds the in-phase parts, row 1 the quadrature parts. A batch then has shape $(B, 2, L)$, exactly the (batch, channels, length) layout that 1-D convolutions (Chapter 15) expect.

Definition: `Dataset` and `DataLoader`

A PyTorch `Dataset` is any object with two methods: `__len__()` (how many examples) and `__getitem__(i)` (return example i). A `DataLoader` wraps a dataset, draws indices (shuffled if requested), calls `__getitem__` for each, and stacks the results into batch tensors, optionally using several worker processes.

The key design choice for wireless data is **on-the-fly augmentation**: we store only the clean frames and add fresh noise every time a frame is requested. Each epoch then sees a different noise realisation at a different SNR, so a modest number of stored frames behaves like an unlimited dataset, and the network cannot memorise particular noise samples.

Listing 14.7: `iq_dataset.py`: an IQ-frame dataset with random-SNR AWGN augmentation.

```
19 class IQFrames(Dataset):
20     """Clean frames are generated once and stored; noise is added in __getitem__,
21     so every epoch sees a fresh noise realisation at a fresh random SNR."""
22
23     def __init__(self, n_frames, frame_len=128, snr_range=(0.0, 20.0), seed=0):
24         rng = np.random.default_rng(seed)
25         self.names = list(MODS)
26         self.labels = rng.integers(0, len(self.names), n_frames)
27         clean = np.empty((n_frames, frame_len), dtype=complex)
28         for i, c in enumerate(self.labels):
29             const = MODS[self.names[c]]
30             clean[i] = const[rng.integers(0, len(const), frame_len)]
31         # store as real tensor (N, 2, L): channel 0 = I, channel 1 = Q
32         self.clean = torch.tensor(np.stack([clean.real, clean.imag], axis=1),
    dtype=torch.float32)
33         self.snr_range = snr_range
34
35     def __len__(self):
36         return len(self.labels)
37
38     def __getitem__(self, i):
39         x = self.clean[i]                                    # (2, L), Es = 1
40         snr_db = torch.empty(1).uniform_(*self.snr_range)
41         n0 = 10 ** (-snr_db / 10)                            # noise power per
    complex sample
42         noise = torch.randn_like(x) * torch.sqrt(n0 / 2)     # N0/2 on I and on Q
43         return x + noise, int(self.labels[i]), snr_db.item()
```

- **Lines 26–30**: pick a random modulation (BPSK, QPSK or 16-QAM, all with unit average energy) for each frame and fill it with random symbols.
- **Line 32**: convert (N, L) complex to $(N, 2, L)$ real `float32`, once, at construction.
- **Lines 38–43**: `__getitem__` draws $\mathrm{SNR}_{\mathrm{dB}} \sim \mathcal{U}(0, 20)$, converts it to the noise power $N_0 = 10^{-\mathrm{SNR}_{\mathrm{dB}}/10}$ (since $E_s = 1$), and adds real Gaussian noise of variance $N_0/2$ to each of I and Q. This is $w \sim \mathcal{CN}(0, N_0)$: total complex noise power $N_0/2 + N_0/2 = N_0$, so E_s/N_0 is the requested SNR. The method returns the noisy frame, the label and the SNR (useful later

for plotting accuracy against SNR).

Worked example: noise standard deviation for a given SNR

At SNR $= 10$ dB: $N_0 = 10^{-1} = 0.1$, so each of I and Q gets noise of variance 0.05 and standard deviation $\sqrt{0.05} = 0.224$. For QPSK points at $(\pm 0.707, \pm 0.707)$, the distance to the decision boundary (the axes) is $0.707 = 3.2$ standard deviations, so the clouds in the middle panel of Fig. 14.11 are well separated. At 3 dB, $N_0 = 0.501$ and the standard deviation is 0.50: only 1.4 standard deviations, and the clouds merge.

The demonstration in the same script prints the shapes:

```
one item: (2, 128) torch.float32 label 2 16QAM snr 9.9 dB
same index again -> new SNR 14.5 dB, max |x - x2| = 0.832
batch: (32, 2, 128) torch.float32 (32,) torch.int64 (32,) torch.float64
measured minus nominal SNR: mean +0.01 dB, std 0.39 dB
```

Asking for item 0 twice returns two different noisy versions of the same clean frame. The `DataLoader` stacks 32 items into a $(32, 2, 128)$ batch, 32 integer labels and 32 SNRs (Python floats become `float64`). The last line checks the noise generator: measuring the SNR of 2000 noisy copies of a QPSK frame gives on average exactly the nominal value, with a spread of 0.39 dB because 128 samples is a short frame (Exercise 6).

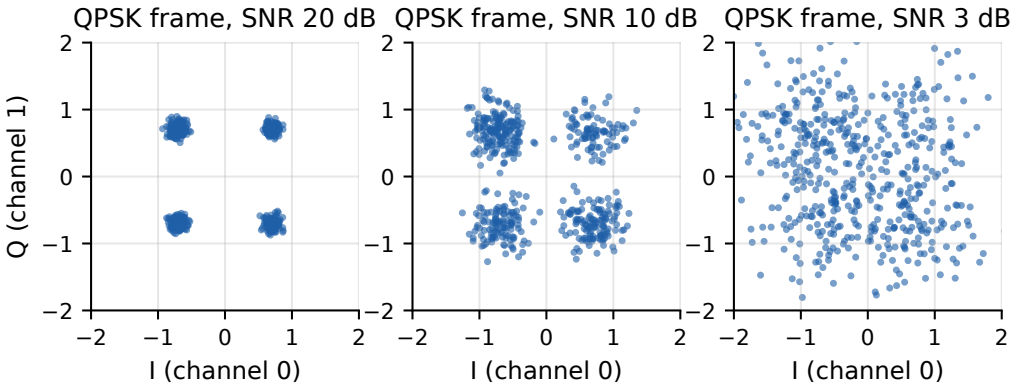


Figure 14.11: Four augmented draws of the same stored QPSK frame (128 symbols each), plotted with channel 0 (I) on the horizontal and channel 1 (Q) on the vertical axis, at SNR 20, 10 and 3 dB. The stored data never change; the noise is generated anew on every access.

Wireless connection: what else belongs in `__getitem__`

AWGN is the simplest augmentation. The same place is where you would apply a random carrier-frequency offset ($r[n]e^{j2\pi\Delta f n/f_s}$), a random phase, a random fractional timing offset, a multipath channel drawn from a tapped-delay-line model such as 3GPP TDL-A/B/C, or IQ imbalance — each with fresh random parameters per access. Training over such randomised impairments ("domain randomisation") is what makes a learned modulation classifier or receiver robust to the conditions of a real capture; Part VI does exactly this. Two practical points: keep the generation in PyTorch or NumPy operations on whole frames (Python loops over samples are slow), and when using `num_workers>0` give each worker its own random seed (PyTorch seeds its own generator per worker automatically, but NumPy generators created in `__init__` are copied identically into every worker).

Chapter recap

- Tensors are NumPy-like arrays with a `dtype` (default `float32`; labels `int64`) and a `device`; all operands must share a device.
- Autograd records operations on tensors with `requires_grad=True`; `backward()` *adds* gradients into `.grad`, hence `zero_grad()`. Turn recording off with `torch.no_grad()`; switch dropout/batch-norm behaviour with `train()`/`eval()`.
- The canonical loop: `zero_grad` → forward → loss → `backward` → `step`. A PyTorch twin of the Chapter 13 network reproduces its gradients to 10^{-16}; `torch.autograd.gradcheck` verifies custom gradients (in float64).
- End-to-end project: normalise with training statistics, split train/validation/test, early-stop on validation loss, save the best `state_dict`, test once. On 8×8 digits: 98.3 % test accuracy. At inference, match the training preprocessing exactly (inversion, size, centring, normalisation).
- Save weights as `state_dict` and load with `weights_only=True` (the default since PyTorch 2.6); checkpoints include optimiser state; ONNX exports the model for other runtimes. Seed everything for reproducibility.
- IQ frames are stored as real $(2, L)$ tensors; adding random-SNR noise inside `__getitem__` is cheap, unlimited data augmentation.

Exercises

1. Compute by hand the gradient of $f = w_1^2 w_2 + \sin(w_2)$ at $(w_1, w_2) = (2, 0)$, then verify with autograd. What does `w.grad` contain if you call `backward()` three times (recomputing f each time) without zeroing?
2. Count the parameters of the MNIST MLP 784–256–128–10. How many bytes does its `state_dict` occupy in float32? In float16?
3. A student's training loss decreases for 50 steps, then explodes. Their loop is `for xb,yb in dl: loss=loss_fn(model(xb),yb); loss.backward(); opt.step()`. What is wrong, and why does it work at first?
4. Why does the validation loss in Fig. 14.6 start below the training loss? Give two reasons.
5. You trained on MNIST and your photographed digit "7" is classified as "1" with 95 % confidence. List, in order, the preprocessing checks you would make.
6. The measured SNR of a 128-sample QPSK frame with nominal SNR ρ has a standard deviation of about 0.39 dB. Explain where this spread comes from and how it scales with the frame length L.
7. Modify `IQFrames` so that `__getitem__` also applies a uniformly random phase rotation $e^{j\theta}$, $\theta \sim \mathcal{U}(0, 2\pi)$, using only real arithmetic on the $(2, L)$ tensor.
8. Why must the normalisation statistics be computed on the training split only? Construct a scenario in which computing them on all data changes the test accuracy noticeably.

Answers

1. $\partial f/\partial w_1 = 2w_1w_2 = 0$; $\partial f/\partial w_2 = w_1^2 + \cos w_2 = 4 + 1 = 5$. Code: `w=torch.tensor([2.,0.],requires_grad=True); f=w[0]**2*w[1]+torch.sin(w[1]); f.backward()` gives `tensor([0., 5.])`. After three backward passes without zeroing: `tensor([0., 15.])`.

2. $784 \cdot 256 + 256 = 200\,960$; $256 \cdot 128 + 128 = 32\,896$; $128 \cdot 10 + 10 = 1290$; total 235 146 parameters. float32: 940 584 bytes (≈ 0.94 MB) plus small metadata; float16: 470 292 bytes.

3. `opt.zero_grad()` is missing, so `.grad` holds the sum of all gradients so far. After k steps the update is roughly k times too large (for consistent gradient directions). Early on k is small and the effective step is reasonable, so the loss falls; as k grows the steps overshoot and the loss explodes.

4. (i) The training loss is measured with dropout active, which deliberately damages the network; the validation loss uses the full network (`eval()` mode). (ii) The training figure is an average over the epoch, during which the network was improving, while validation is measured at the end of the epoch with the improved weights.

5. Grey-scale; colours inverted (white ink on black); background exactly black (threshold); digit cropped and scaled to about 20×20 inside 28×28, centred; values scaled to $[0, 1]$ then normalised with $0.1307/0.3081$; tensor shape $(1, 1, 28, 28)$; `model.eval()`. Finally display the preprocessed 28×28 image and compare it by eye with MNIST samples — also check the 7 is not written with a crossbar style absent from MNIST.

6. Measured SNR uses the realised noise energy $\sum_n |w[n]|^2$ over L samples, which is random: it is $N_0/2$ times a chi-square variable with $2L$ degrees of freedom, with relative standard deviation $\sqrt{2/(2L)} = 1/\sqrt{L}$. For $L = 128$ that is 0.088, i.e. $10\log_{10}(1 \pm 0.088) \approx \pm 0.38$ dB — matching the measured 0.39 dB. The spread shrinks as $1/\sqrt{L}$: a 4 times longer frame halves it.

7. With $c = \cos\theta$, $s = \sin\theta$: $(I + jQ)(c + js) = (cI - sQ) + j(sI + cQ)$. In code: `th=torch.rand(1)*2*math.pi; c,s=torch.cos(th),torch.sin(th); x=torch.stack([c*x[0]-s*x[1], s*x[0]+c*x[1]])`, applied before adding noise.

8. The test set must simulate unseen future data; any statistic computed from it leaks information into training. Scenario: the test images come from a different scanner and are brighter. Using statistics from all data shifts the normalisation towards the test distribution, partially correcting the shift and inflating the reported accuracy relative to what a deployed model (normalised with training statistics) would achieve.

Self-test quiz

1. What are the five lines of the canonical PyTorch training step, in order?
   `opt.zero_grad()`, `logits = model(xb)`, `loss = loss_fn(logits, yb)`, `loss.backward()`, `opt.step()`.

2. Does `model.eval()` stop gradient computation?
   No. It changes the behaviour of layers such as dropout and batch norm; `torch.no_grad()` stops the recording of gradients.

3. Why does `torch.autograd.gradcheck` require float64 inputs?
   Centred differences subtract nearly equal loss values; float32 keeps too few significant digits, so the numerical gradient is too inaccurate to compare.
4. What does `weights_only=True` protect against?
   Unpickling arbitrary Python objects, which can execute code; it only loads tensors and basic containers. It is the default from PyTorch 2.6.
5. Why store IQ data as a $(2, L)$ real tensor rather than complex?
   Standard layers (linear, convolution) operate on real tensors; the two rows (I and Q) are treated as two input channels.
6. What is the advantage of adding noise in `__getitem__` rather than once when the dataset is created?
   Every access produces a fresh noise realisation and SNR, giving effectively unlimited, non-memorisable training data from a fixed set of clean frames.

Further reading

- PyTorch documentation, `https://pytorch.org/docs`: the tutorials "Learn the Basics" and "Saving and loading models", the note "Autograd mechanics", and the reference pages for `torch.onnx` and `torch.amp`.
- A. Paszke et al., "PyTorch: an imperative style, high-performance deep learning library," *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, 1998 (the origin of MNIST).
- D. P. Kingma and J. Ba, "Adam: a method for stochastic optimization," *ICLR*, 2015.
- N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *JMLR*, vol. 15, 2014.
- T. J. O'Shea, J. Corgan, T. C. Clancy, "Convolutional radio modulation recognition networks," *Proc. EANN*, 2016 (IQ frames as $2 \times L$ inputs; the RadioML data).
- ONNX Runtime documentation, `https://onnxruntime.ai`.

Part IV

Deep Architectures and Reinforcement Learning

Chapter 15

Convolutional Neural Networks

So far every network in this book has been a *multilayer perceptron* (MLP): each neuron is connected to every input. That is fine for a handful of features, but it is a poor fit for *images* and *signals*, where the inputs are arranged on a grid (pixels in 2-D, samples in time) and the same pattern can appear anywhere. In this chapter you will learn the *convolutional layer*, the idea that made modern computer vision possible and that is now everywhere in physical-layer deep learning. You already know it under another name: a convolutional layer is a *bank of FIR filters whose taps are learned by gradient descent*. We will build it from the 1-D sliding dot product up to multi-channel 2-D layers, derive its backpropagation completely, implement it in NumPy and verify it with a gradient check, train real CNNs on images and on signals, and finish with the classic architectures (LeNet-5, VGG, ResNet, Inception) and the IQ input representation used for modulation recognition in Chapter 22.

15.1 Why not just use an MLP?

Take a tiny 28×28 grey-scale image: 784 inputs. A first MLP layer with 1000 neurons already has $784 \times 1000 + 1000 \approx 785{,}000$ weights. A modest 224×224 colour image has 150,528 inputs, so the same layer would need 150 million weights. Worse than the size is what those weights *mean*:

- **No notion of neighbourhood.** An MLP treats pixel $(3, 5)$ and pixel $(3, 6)$ as unrelated inputs. If you shuffled all pixels with a fixed permutation, the MLP would learn equally well — it ignores the fact that nearby pixels (or nearby time samples) belong together.
- **No reuse.** A vertical edge in the top-left corner and a vertical edge in the bottom-right corner are the *same* feature, but an MLP must learn a separate set of weights for each location. It needs many more examples to see the pattern in every position.
- **No shift tolerance.** Move a digit two pixels to the right and the MLP sees a completely different input vector. We will measure this in Section 15.8.1: an MLP that is 97% accurate on centred digits drops below 50% when the digits are shifted by one pixel.

Intuition: a torch scanning a wall

Imagine searching a large dark wall for a small symbol using a torch with a narrow beam. You do not need a different eye for every spot of the wall: you use *the same eye* and slide the beam across, asking at every position "does what I see look like the symbol?". A convolutional layer does exactly this. A small set of weights (the *kernel* or *filter*, the torch's "template") is slid across the input, and at every position we compute a single similarity score. The resulting map of scores tells us *where* the pattern is.

Two structural assumptions, called *inductive biases*, are built in: **locality** (each output depends only on a small neighbourhood of the input) and **weight sharing** (the same weights are used at every position). Together they give *translation equivariance*: shift the input, and the output map shifts by the same amount. These are exactly the properties of a linear time-invariant (LTI) filter — which is why signal-processing engineers feel at home with CNNs.

15.2 1-D convolution: the sliding dot product

15.2.1 The operation

Let $x[0], x[1], \ldots, x[N-1]$ be a signal of N samples and $k[0], \ldots, k[K-1]$ a kernel of K weights (K is small, e.g. 3, 5 or 7). Place the kernel over samples $x[i], \ldots, x[i+K-1]$, multiply element by element, and add:

Definition: 1-D "convolution" as used in deep learning

$$y[i] \;=\; \sum_{m=0}^{K-1} k[m]\, x[i+m] \;+\; b, \qquad i = 0, 1, \ldots, N-K. \tag{15.1}$$

The kernel k and the scalar bias b are the layer's learnable parameters. Each output $y[i]$ is the *dot product* between the kernel and one window of the input. Without padding there are $N-K+1$ valid positions, so the output is shorter than the input.

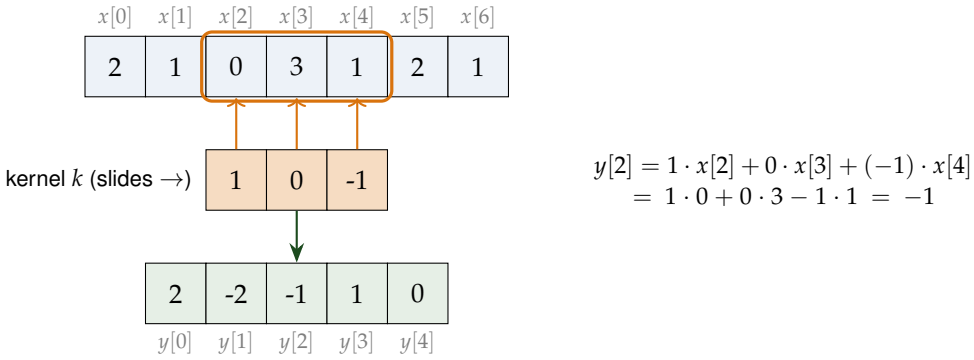


Figure 15.1: 1-D convolution (deep-learning convention) as a sliding dot product. The orange kernel $[1, 0, -1]$ sits over three input samples; their weighted sum is one output. Sliding the kernel one step at a time produces the five outputs in green. Kernel weights are the same at every position (*weight sharing*).

Worked example: sliding the kernel by hand

Input $x = [2, 1, 0, 3, 1, 2, 1]$ ($N = 7$), kernel $k = [1, 0, -1]$ ($K = 3$), $b = 0$. There are $7-3+1 = 5$ outputs, and each is $y[i] = 1 \cdot x[i] + 0 \cdot x[i+1] + (-1) \cdot x[i+2]$:

$$y[0] = 2+0-0 = 2, \qquad y[1] = 1+0-3 = -2, \qquad y[2] = 0+0-1 = -1,$$
$$y[3] = 3+0-2 = 1, \qquad y[4] = 1+0-1 = 0.$$

So $y = [2, -2, -1, 1, 0]$, exactly the green row of Fig. 15.1. This kernel computes $x[i] - x[i+2]$: a crude *negative derivative*. It is large where the signal falls and negative where it rises.

15.2.2 The engineer's view: an FIR filter

Recall that a causal FIR filter with impulse response $h[0..K-1]$ computes the *true convolution*

$$(x * h)[n] = \sum_{m=0}^{K-1} h[m]\, x[n-m]. \tag{15.2}$$

Compare with (15.1): the deep-learning layer uses $x[i+m]$ (the kernel runs *forwards* along the signal), the FIR filter uses $x[n-m]$ (the impulse response is *time-reversed* before sliding). The deep-learning operation is therefore mathematically a *cross-correlation*, written $x \star k$:

$$(x \star k)[i] = \sum_m k[m]\, x[i+m] = (x * \tilde{k})[i+K-1], \qquad \tilde{k}[m] = k[K-1-m]. \tag{15.3}$$

In words: a deep-learning "convolution" with kernel k is a true convolution with the flipped kernel $\tilde{k}$, up to a shift of the output index.

Pitfall: convolution vs. cross-correlation

Every deep-learning library (PyTorch `Conv1d/Conv2d`, TensorFlow, JAX) computes cross-correlation but calls it convolution. For *learning* this makes no difference: the network simply learns the flipped kernel. But it matters when you (i) copy a known filter into a layer (a matched filter must be *conjugated and reversed* in DSP, but only *conjugated* in a DL layer — cross-correlation already *is* matched filtering), or (ii) compare learned kernels with impulse responses. Figure 15.2 shows the difference for an asymmetric kernel.

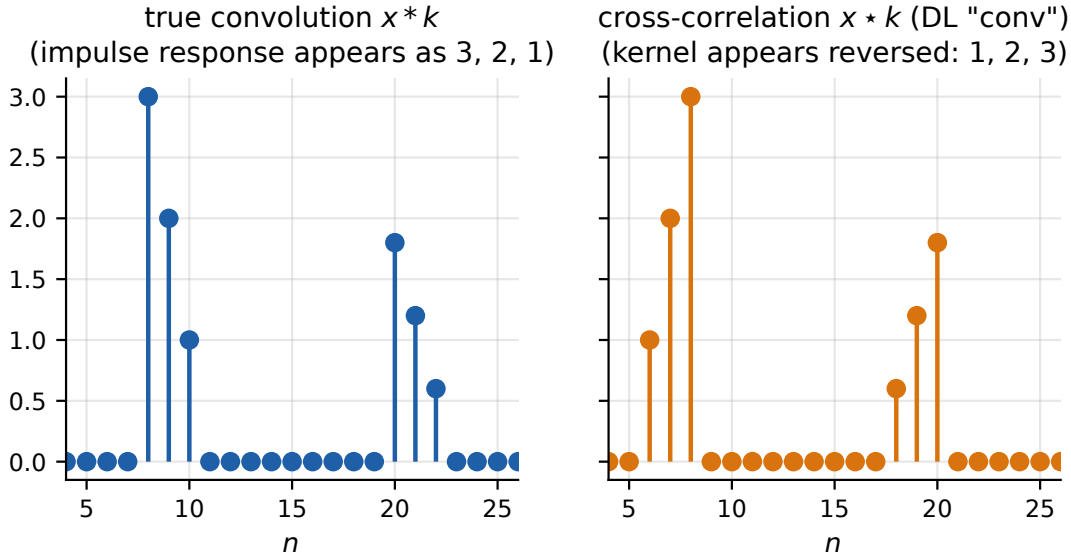


Figure 15.2: The same input (two impulses, at $n = 8$ and $n = 20$ with amplitudes 1 and 0.6) filtered with the asymmetric kernel $[3, 2, 1]$. Left: true convolution — every impulse is replaced by a copy of the impulse response $3, 2, 1$ running forward in time. Right: the cross-correlation computed by deep-learning layers — the copy appears *reversed* $(1, 2, 3)$ and *before* the impulse. Look at the order of the stem heights.

Because the layer is an FIR filter, all your filter intuition carries over: a kernel of all-equal positive taps is a moving average (low-pass); a kernel $[-1, 0, 1]$ is a discrete derivative (high-pass, an edge detector); a windowed sinusoid is a band-pass filter. Figure 15.3 shows two hand-designed kernels in action. A CNN *learns* such kernels from data.

15.3 2-D convolution and multiple channels

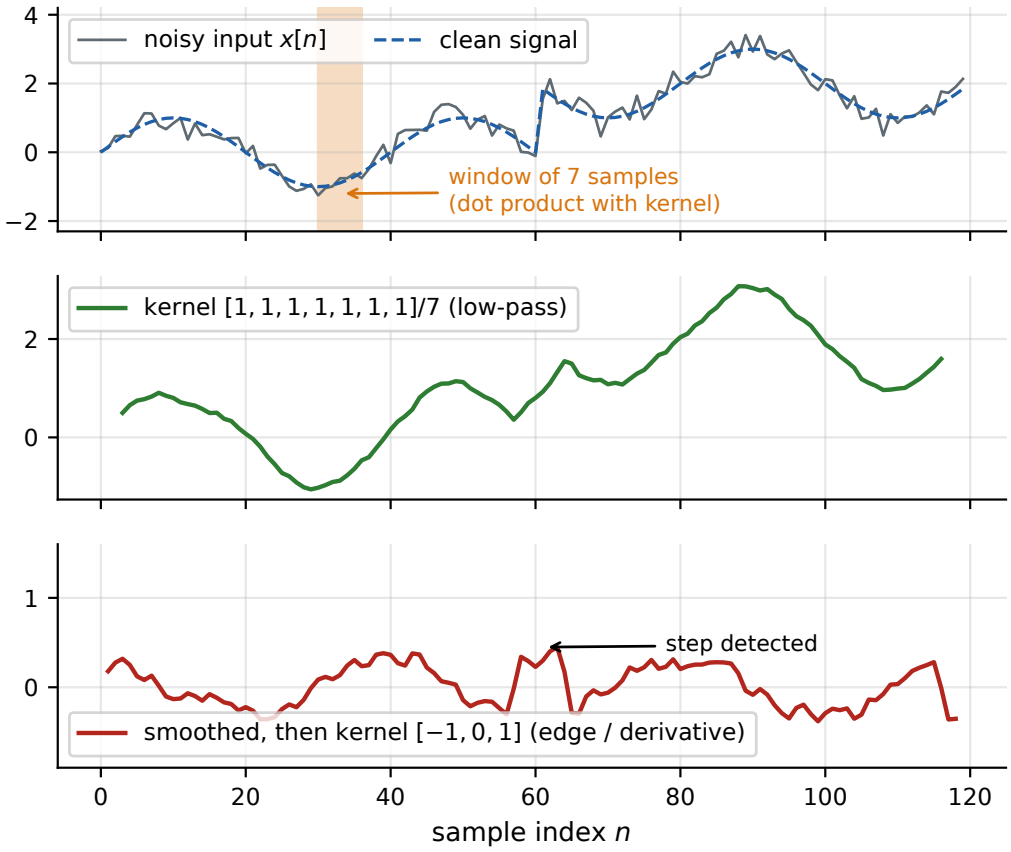


Figure 15.3: A 1-D convolutional kernel is an FIR filter. Top: a noisy signal (slow sine plus a step at $n = 60$); the shaded band is one 7-sample window. Middle: the moving-average kernel removes most of the noise. Bottom: after smoothing, the difference kernel $[-1, 0, 1]$ responds strongly at the step — an edge detector. Note how each output is shorter than its input ("valid" positions only). Script: `code/ch15/conv1d_demo.py`.

15.3.1 Single channel

For an image X of size $H \times W$ and a $k \times k$ kernel K, the kernel now slides in two directions:

$$Y[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} K[m, n]\, X[i + m,\, j + n] + b. \tag{15.4}$$

The output Y is called a *feature map* (or activation map). Each $Y[i, j]$ is the dot product between the kernel and the $k \times k$ patch whose top-left corner is (i, j).

Worked example: a vertical-edge detector on a 4×4 image

Let

$$X = \begin{bmatrix} 1 & 2 & 1 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}, \qquad K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}, \qquad b = 0.$$

K adds the left column of a patch and subtracts the right column: it responds to a *vertical* change of brightness. The output has $(4 - 3 + 1) \times (4 - 3 + 1) = 2 \times 2$ entries. For every entry, write "(left column sum) $-$ (right column sum)" of the 3×3 patch:

- $Y[0, 0]$: rows 0–2, columns 0–2. Left column $1 + 1 + 0 = 2$; right column $1 + 1 + 0 = 2$. $Y[0, 0] = 2 - 2 = 0$.
- $Y[0, 1]$: rows 0–2, columns 1–3. Left column $2 + 1 + 1 = 4$; right column $0 + 1 + 1 = 2$. $Y[0, 1] = 4 - 2 = 2$.
- $Y[1, 0]$: rows 1–3, columns 0–2. Left column $1 + 0 + 0 = 1$; right column $1 + 0 + 1 = 2$. $Y[1, 0] = 1 - 2 = -1$.
- $Y[1, 1]$: rows 1–3, columns 1–3. Left column $1 + 1 + 0 = 2$; right column $1 + 1 + 1 = 3$. $Y[1, 1] = 2 - 3 = -1$.

$$Y = \begin{bmatrix} 0 & 2 \\ -1 & -1 \end{bmatrix}.$$

Positive values mean "brighter on the left", negative "brighter on the right". (A *true* 2-D convolution flips the kernel in both directions and would give $-Y$; verified in `code/ch15/conv_numpy.py`.)

15.3.2 Multi-channel input and output

Real inputs have several *channels*: an RGB image has 3, a hidden layer may have 64. A convolutional layer maps C_{in} input channels to C_{out} output channels. Each output channel has its own kernel, and that kernel is a small 3-D block that spans *all* input channels:

Definition: multi-channel convolutional layer

Input $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$, weights $K \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$, bias $b \in \mathbb{R}^{C_{\text{out}}}$. Output channel o is

$$Y[o, i, j] = b[o] + \sum_{c=0}^{C_{\text{in}}-1} \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} K[o, c, m, n]\, X[c,\, i + m,\, j + n]. \tag{15.5}$$

So one output value mixes a $k \times k$ neighbourhood of *every* input channel. The weight tensor shape $(C_{\text{out}}, C_{\text{in}}, k, k)$ is exactly PyTorch's `conv.weight.shape`.

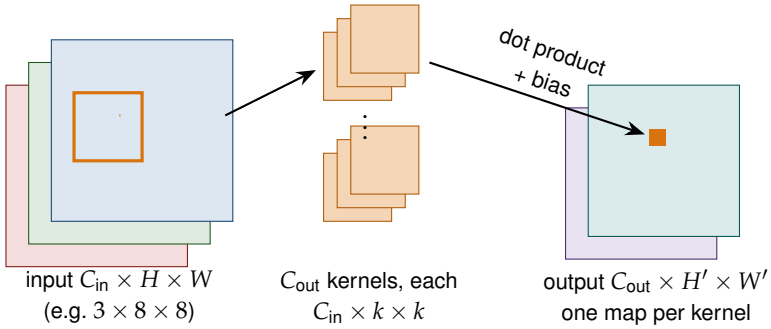


Figure 15.4: A multi-channel convolutional layer. Each kernel (orange) is a $C_{in} \times k \times k$ block; placed over a patch of the input volume, it produces *one number* of *one* output map. C_{out} kernels produce C_{out} maps, stacked into the output volume, which becomes the input of the next layer.

15.3.3 Counting parameters

Each output channel has $C_{in} \cdot k \cdot k$ weights plus one bias, so

Key equation: parameters of a convolutional layer

$$\#\text{params} = C_{out}\left(C_{in}\,k^2 + 1\right). \tag{15.6}$$

It does *not* depend on the image size $H \times W$ — the same kernels are reused at every position.

Worked example: parameter counts

- `Conv2d(1, 32, 3)`: $32\,(1 \cdot 9 + 1) = 32 \cdot 10 = 320$.
- `Conv2d(16, 32, 5)`: $32\,(16 \cdot 25 + 1) = 32 \cdot 401 = 12{,}832$.
- `Conv2d(32, 64, 3)`: $64\,(32 \cdot 9 + 1) = 64 \cdot 289 = 18{,}496$.

Compare: a fully connected layer from a $1 \times 28 \times 28$ image to $32 \times 26 \times 26$ outputs would need $784 \cdot 21{,}632 \approx 17$ million weights. The convolution uses 320.

15.4 Padding, stride and dilation

Three knobs control *where* the kernel is placed.

Padding p adds p rows/columns of zeros around the border. Without it every layer shrinks the map by $k-1$ and border pixels are seen by fewer kernel positions. With $p = (k-1)/2$ ("same" padding, for odd k) the size is preserved.

Stride s moves the kernel s positions at a time instead of one. A stride of 2 halves the map size: it is a *downsampler* (in DSP terms, filtering followed by decimation by s).

Dilation d spreads the kernel taps d positions apart ("*à trous*", with holes). A 3-tap kernel with $d = 2$ reads $x[i], x[i+2], x[i+4]$: it covers $d(k-1)+1 = 5$ samples with only 3 weights.

Key equation: output size

For input length n (per spatial dimension), kernel k, padding p, stride s, dilation d:

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - d(k-1) - 1}{s} \right\rfloor + 1 \overset{d=1}{=} \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1. \tag{15.7}$$

Derivation. After padding the input has length $n + 2p$. The (dilated) kernel spans $k_{\text{eff}} = d(k-1)+1$ samples, so its first position starts at index 0 and its last possible start is $n + 2p - k_{\text{eff}}$. Starts occur at $0, s, 2s, \ldots$; the number of multiples of s in $[0,\ n + 2p - k_{\text{eff}}]$ is $\lfloor (n + 2p - k_{\text{eff}})/s \rfloor + 1$. Substituting k_{eff} gives (15.7).

Worked example: output sizes

- $n = 28,\ k = 3,\ p = 0,\ s = 1$: $\lfloor 25/1 \rfloor + 1 = 26$.
- $n = 28,\ k = 3,\ p = 1,\ s = 1$: $\lfloor 27/1 \rfloor + 1 = 28$ ("same").
- $n = 32,\ k = 5,\ p = 2,\ s = 2$: $\lfloor (32 + 4 - 5)/2 \rfloor + 1 = \lfloor 15.5 \rfloor + 1 = 16$.
- $n = 64,\ k = 3,\ p = 1,\ s = 2$: $\lfloor (64 + 2 - 3)/2 \rfloor + 1 = \lfloor 31.5 \rfloor + 1 = 32$.

15.5 Pooling

A *pooling* layer summarises each small window of a feature map by a single number, with *no learnable parameters*. It acts on each channel separately.

- **Max pooling** (2×2, stride 2): keep the largest value in each window. "Is the feature present somewhere in this area?"
- **Average pooling**: keep the mean. A low-pass filter followed by decimation.
- **Global pooling**: one window covering the *whole* map, giving one number per channel. Global average pooling (GAP) is used at the end of most modern CNNs instead of a large flatten + dense layer; global *max* pooling answers "did the feature occur anywhere?", ideal for detecting a burst at an unknown time.

Worked example: max and average pooling, and their backward pass

$$A = \begin{bmatrix} 1 & 3 & 2 & 0 \\ 4 & 2 & 1 & 1 \\ 0 & 1 & 5 & 2 \\ 3 & 2 & 1 & 3 \end{bmatrix} \xrightarrow{2\times 2\ \max} \begin{bmatrix} 4 & 2 \\ 3 & 5 \end{bmatrix}, \qquad \xrightarrow{2\times 2\ \text{avg}} \begin{bmatrix} 2.5 & 1 \\ 1.5 & 2.75 \end{bmatrix}.$$

(Top-left window: $\{1, 3, 4, 2\}$, max 4, mean $10/4 = 2.5$. Bottom-right: $\{5, 2, 1, 3\}$, max 5, mean 2.75.)

Backward. Suppose the upstream gradient is $\partial L/\partial(\text{pooled}) = \begin{bmatrix} g_1 & g_2 \\ g_3 & g_4 \end{bmatrix}$. For max pooling, the output equals one input of the window, so its derivative is 1 for that input (the *argmax*)

and 0 for the others:

$$\frac{\partial L}{\partial A}\Big|_{\max} = \begin{bmatrix} 0 & 0 & g_2 & 0 \\ g_1 & 0 & 0 & 0 \\ 0 & 0 & g_4 & 0 \\ g_3 & 0 & 0 & 0 \end{bmatrix}.$$

(The top-right window $\{2,0,1,1\}$ has its max at the top-left of the window, value 2.) For average pooling, each output is $\frac{1}{4}\sum$ of its window, so every input receives a quarter: $\partial L/\partial A_{ij} = g/4$ for all four cells of the window.

Pitfall: ties in max pooling

If two values in a window are equal, the gradient is routed to just one of them (frameworks pick the first). This is fine in practice, but it is why a naive gradient check can fail exactly at ties — use random real-valued test inputs.

15.6 Receptive field

The *receptive field* of a unit is the region of the *original input* that can influence it. A unit in the first 3×3 layer sees 3×3 pixels. A unit in the second 3×3 layer sees a 3×3 patch of the first layer, each element of which sees 3×3 pixels; together they cover 5×5 pixels (Fig. 15.5).

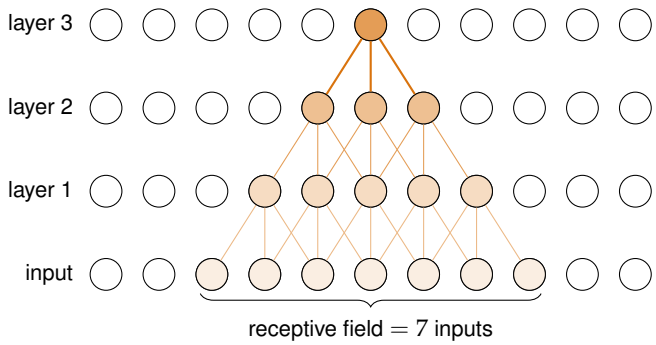


Figure 15.5: Receptive-field growth for three stacked kernels of size 3 (stride 1). The top unit sees 3 units of layer 2, which together see 5 units of layer 1, which see 7 input samples. Each extra layer adds $k - 1 = 2$.

Key equation: receptive-field recursion

With $r_0 = 1$ and "jump" (spacing of layer-l units measured in input samples) $J_0 = 1$:

$$r_l = r_{l-1} + (k_l - 1)\, d_l\, J_{l-1}, \qquad J_l = J_{l-1}\, s_l. \tag{15.8}$$

Why? Layer l looks at k_l units of layer $l-1$ spaced d_l units apart; adjacent units of layer $l-1$ are J_{l-1} input samples apart. So, beyond the single unit's own field r_{l-1}, the kernel adds $(k_l - 1)d_l J_{l-1}$ more samples. Strides multiply the jump, which is why downsampling makes the receptive field grow fast. Pooling layers obey the same rule (2×2 pooling with stride 2: $k = 2$, $s = 2$).

Worked example: receptive field

Two 3×3 convs (stride 1): $r = 1 + 2 + 2 = 5$. Three: 7. Conv3 (s=1) $\rightarrow$ maxpool2 (s=2) $\rightarrow$ conv3: $r_1 = 3$, $r_2 = 3 + 1 \cdot 1 = 4$ with $J_2 = 2$, $r_3 = 4 + 2 \cdot 2 = 8$.

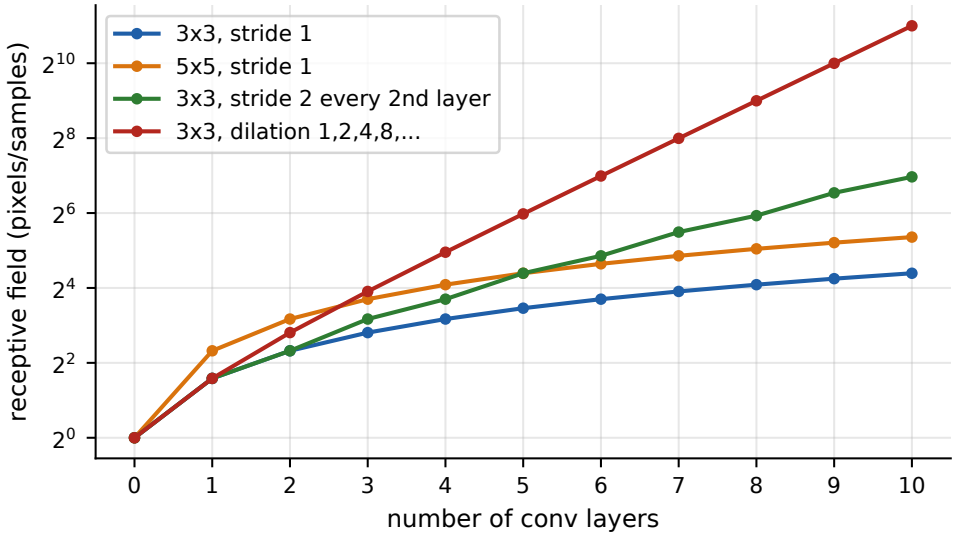


Figure 15.6: Receptive field versus depth (log scale). Stride-1 stacks grow *linearly* ($+k-1$ per layer); inserting stride 2 every other layer, or doubling the dilation each layer (as in WaveNet-style 1-D models), grows it *exponentially*. Script: `code/ch15/receptive_field.py`.

15.7 Backpropagation through a convolution

We now derive the gradients completely. You only need the chain rule of Part I and one observation: a convolution is a linear layer whose weight matrix is sparse and has repeated (shared) entries. Shared weights receive the *sum* of the gradients from every place they are used.

15.7.1 1-D first

Take $y[i] = \sum_m k[m]x[i+m] + b$ and suppose backprop has delivered $\delta[i] = \partial L/\partial y[i]$ for every output (we write δ for dY). Every output depends on $k[m]$, so by the multivariate chain rule

$$\frac{\partial L}{\partial k[m]} = \sum_i \frac{\partial L}{\partial y[i]}\,\frac{\partial y[i]}{\partial k[m]} = \sum_i \delta[i]\,x[i+m].$$

$$\frac{\partial L}{\partial b} = \sum_i \delta[i] \cdot 1 = \sum_i \delta[i].$$

The first line is *the cross-correlation of the input with the upstream gradient*: slide δ over x. For the input, $x[p]$ appears in $y[i]$ whenever $p = i + m$, i.e. $m = p - i$, with $0 \le m \le K-1$:

$$\frac{\partial L}{\partial x[p]} = \sum_i \delta[i]\,\frac{\partial y[i]}{\partial x[p]} = \sum_i \delta[i]\,k[p-i],$$

where terms with $p - i$ outside $0..K-1$ are zero. This is a *true, "full" convolution* of δ with k (formula (15.2)). Equivalently: pad δ with $K-1$ zeros on both sides and cross-correlate it with the *flipped* kernel.

Worked example: 1-D backward pass by hand

$x = [1,2,3,4]$, $k = [1,-1]$, so $y = [x_0 - x_1,\ x_1 - x_2,\ x_2 - x_3] = [-1,-1,-1]$. Let $\delta = [1,0,2]$.

$$\begin{aligned}
\partial L/\partial k[0] &= \delta_0 x_0 + \delta_1 x_1 + \delta_2 x_2 = 1 + 0 + 6 = 7,\\
\partial L/\partial k[1] &= \delta_0 x_1 + \delta_1 x_2 + \delta_2 x_3 = 2 + 0 + 8 = 10,\\
\partial L/\partial x &= [\delta_0 k_0,\ \delta_0 k_1 + \delta_1 k_0,\ \delta_1 k_1 + \delta_2 k_0,\ \delta_2 k_1] = [1,\ -1,\ 2,\ -2].
\end{aligned}$$

Check directly: L changes as $\sum_i \delta_i y_i = 1(x_0 - x_1) + 0 + 2(x_2 - x_3)$, whose derivative w.r.t. x is indeed $[1,-1,2,-2]$.

15.7.2 2-D, multi-channel

Exactly the same steps applied to (15.5), with $\delta[o,i,j] = \partial L/\partial Y[o,i,j]$, give:

Key equation: gradients of a convolutional layer

$$\frac{\partial L}{\partial K[o,c,m,n]} = \sum_{i,j} \delta[o,i,j]\, X[c,i+m,j+n] \quad \Longleftrightarrow \quad dK_{o,c} = X_c \star \delta_o \tag{15.9}$$

$$\frac{\partial L}{\partial b[o]} = \sum_{i,j} \delta[o,i,j] \tag{15.10}$$

$$\frac{\partial L}{\partial X[c,p,q]} = \sum_{o}\sum_{i,j} \delta[o,i,j]\, K[o,c,p-i,q-j] \quad \Longleftrightarrow \quad dX_c = \sum_{o} \mathrm{pad}(\delta_o) \star \mathrm{rot}_{180}(K_{o,c}) \tag{15.11}$$

Read them in words: **(i)** the weight gradient is the input correlated with the output gradient (each tap accumulates "input value $\times$ error" over every position it was used); **(ii)** the bias gradient is the sum of the output gradient over positions; **(iii)** the input gradient is the output gradient, zero-padded by $k-1$, correlated with the kernel rotated by 180° — i.e. a full true convolution. Each input pixel collects error from every output it helped to produce, and from every output channel. Note the beautiful duality: *the backward pass of a convolution is itself a convolution*, so the same fast code is used forwards and backwards. (For stride $s > 1$ one first inserts $s-1$ zeros between the entries of δ; this is the "transposed convolution" used by decoder networks that upsample, such as the convolutional autoencoders later in Part IV.)

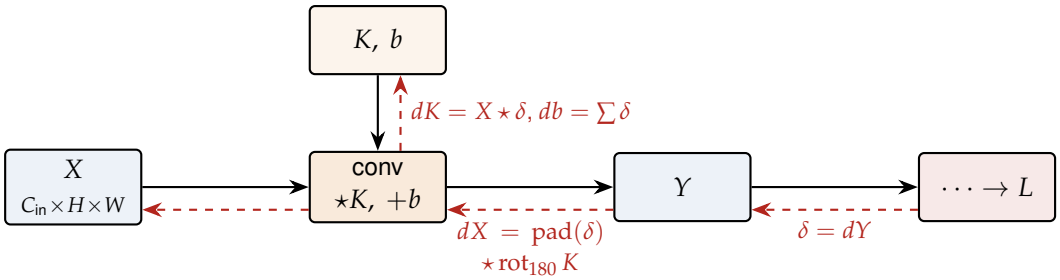


Figure 15.7: Computational graph of a convolutional layer. Solid: forward pass. Dashed red: backward pass. Both backward quantities are themselves correlations/convolutions.

15.7.3 NumPy implementation and gradient check

The listing below is a direct transcription of (15.5) and (15.9)–(15.11), written with plain loops for clarity (real libraries use matrix tricks such as `im2col` or FFTs, but compute the same numbers).

```
# ---------- forward / backward of a conv layer (stride 1, no padding) ----------
def conv2d_forward(X, K, b):
    """X: (C_in, H, W)  K: (C_out, C_in, k, k)  b: (C_out,)  ->  Y: (C_out, H-k+1, W-k+1)
    Deep-learning 'convolution' = cross-correlation (no kernel flip)."""
    C_out, C_in, k, _ = K.shape
    H, W = X.shape[1:]
    Ho, Wo = H - k + 1, W - k + 1
    Y = np.zeros((C_out, Ho, Wo))
    for o in range(C_out):
        for i in range(Ho):
            for j in range(Wo):
                Y[o, i, j] = np.sum(X[:, i:i+k, j:j+k] * K[o]) + b[o]
    return Y

def corr2d_valid(A, B):
    """single-channel 'valid' cross-correlation of A with B."""
    k1, k2 = B.shape
    Ho, Wo = A.shape[0] - k1 + 1, A.shape[1] - k2 + 1
    return np.array([[np.sum(A[i:i+k1, j:j+k2] * B) for j in range(Wo)] for i in range(Ho)])

def conv2d_backward(X, K, dY):
    """Returns dX, dK, db given upstream gradient dY = dL/dY."""
    C_out, C_in, k, _ = K.shape
    dK = np.zeros_like(K); dX = np.zeros_like(X)
    db = dY.sum(axis=(1, 2))                                  # bias touches every output
    for o in range(C_out):
        for c in range(C_in):
            dK[o, c] = corr2d_valid(X[c], dY[o])              # dK = X  (corr)  dY
            dYpad = np.pad(dY[o], k - 1)                      # 'full' padding
            dX[c] += corr2d_valid(dYpad, np.rot90(K[o, c], 2))  # full conv of dY with K
    return dX, dK, db
```

To test it we build a small pipeline *conv* $\to$ *ReLU* $\to$ 2×2 *max-pool* $\to$ *weighted sum* (so all three backward rules are exercised), compute analytic gradients, and compare them with central finite differences $\big(L(\theta+\varepsilon) - L(\theta-\varepsilon)\big)/2\varepsilon$ exactly as in Chapter 13. The script prints:

```
relative error dK: 3.60e-10
relative error db: 5.89e-11
relative error dX: 1.87e-10
```

Relative errors around 10^{-10} (anything below 10^{-7} is excellent in double precision) confirm that the derivation is right.

15.8 Training CNNs on real data

15.8.1 Digits: CNN versus MLP

Scikit-learn ships 1797 handwritten digits as 8×8 images (no download needed). We train, on 70% of them, a small CNN

$$\underbrace{\text{conv}(1{\to}8, 3{\times}3)}_{80} \to \text{ReLU} \to \underbrace{\text{conv}(8{\to}16, 3{\times}3)}_{1168} \to \text{ReLU} \to \text{pool}_2 \to \underbrace{\text{dense}(256{\to}10)}_{2570}$$

(3818 parameters) and an MLP $64 \to 40 \to 10$ (3010 parameters) with the same optimiser (Adam, learning rate 3×10^{-3}, 40 epochs).

```python
class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, 3, padding=1)     # 8x8 -> 8x8,  8 maps
        self.conv2 = nn.Conv2d(8, 16, 3, padding=1)    # 8x8 -> 8x8, 16 maps
        self.fc = nn.Linear(16 * 4 * 4, 10)            # after one 2x2 max-pool
    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.max_pool2d(F.relu(self.conv2(x)), 2)     # -> (16, 4, 4)
        return self.fc(x.flatten(1))
```

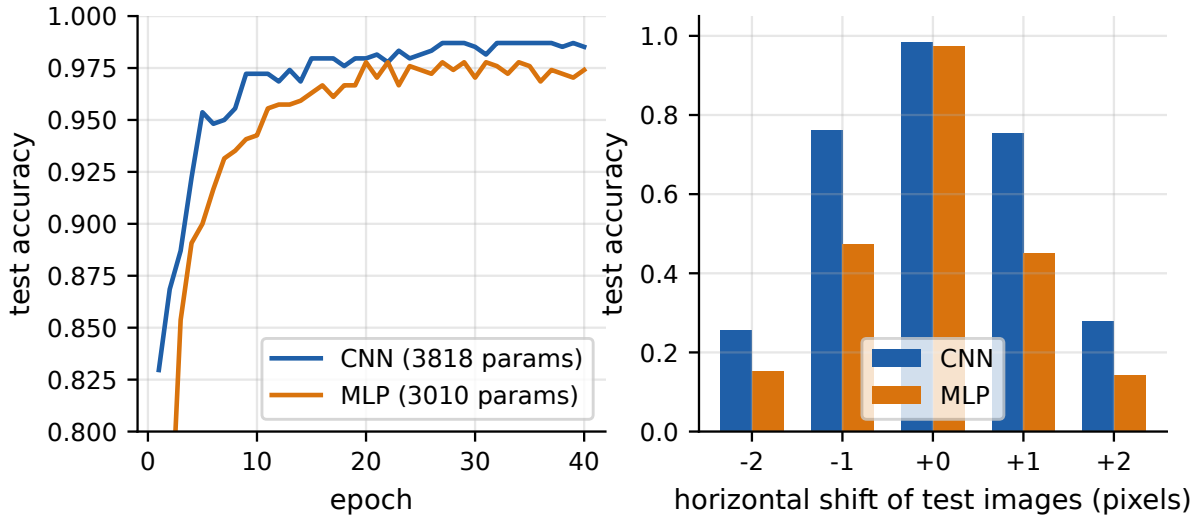


Figure 15.8: Left: test accuracy during training. With a similar parameter budget the CNN reaches 98.5% against the MLP's 97.4% on this run. Right: accuracy when the *test* images are shifted horizontally (neither model saw shifted images in training). At ± 1 pixel the CNN keeps about 76% while the MLP falls to about 46%. On 8×8 images one pixel is 12% of the width, so both degrade — but weight sharing makes the CNN far more tolerant. Script: `code/ch15/train_digits_cnn.py`.

What did the network learn? Figure 15.9 shows the eight first-layer kernels and the feature maps they produce for one test digit. Several kernels are clearly oriented edge detectors (positive on one side, negative on the other); their maps light up along the strokes of matching orientation.

15.8.2 A 1-D CNN on signals

Now a task closer to the reader's daily work. Each example is 128 samples of Gaussian noise, and in three of the four classes a 24-sample Hann-windowed burst is added at a *random* position with random sign: a slow tone (0.06 cycles/sample), a fast tone (0.25) or a chirp. The fourth class

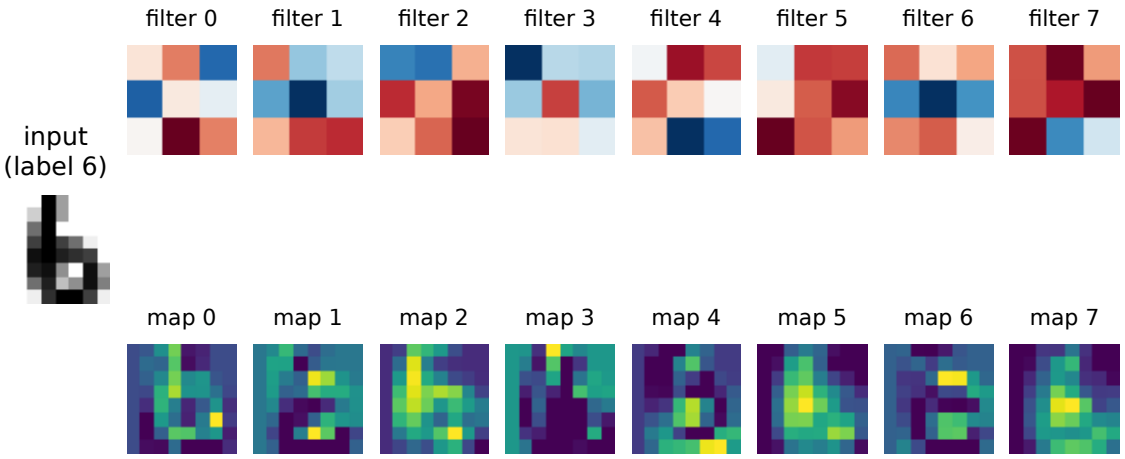


Figure 15.9: Top row: the eight learned 3×3 kernels of `conv1` (red positive, blue negative). Bottom row: the ReLU feature maps for the input digit on the left. Look for kernels with a clear red/blue split: these are edge detectors, and their maps highlight strokes of one orientation.

is noise only (Fig. 15.10). A CNN with two 1-D conv layers (8 kernels of 15 taps, then 16 kernels of 7 taps), a max-pool, and *global max pooling over time* followed by a $16 \to 4$ linear layer has only 1108 parameters.

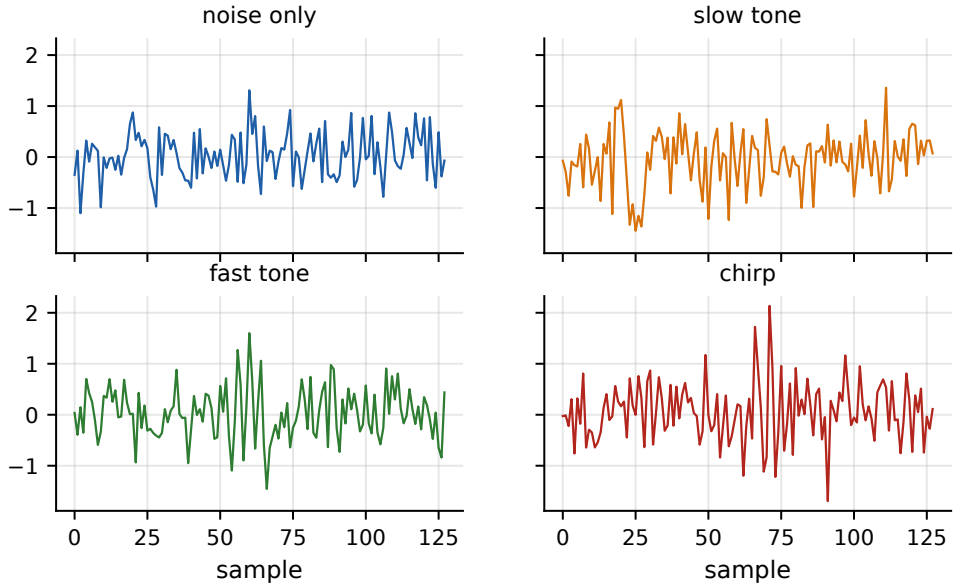


Figure 15.10: One test example per class of the synthetic burst-detection task. The bursts are hard to see by eye at this noise level and appear at random positions — a natural job for shift-equivariant convolutions followed by global max pooling ("was the pattern anywhere?").

After 15 epochs it classifies 95.7% of 3200 test signals correctly; most errors are weak bursts confused with noise. More interesting are the learned kernels (Fig. 15.11): their frequency responses are *band-pass filters* peaking near the two tone frequencies. Nobody told the network about Fourier analysis — gradient descent discovered that matched band-pass filtering is the right front end.

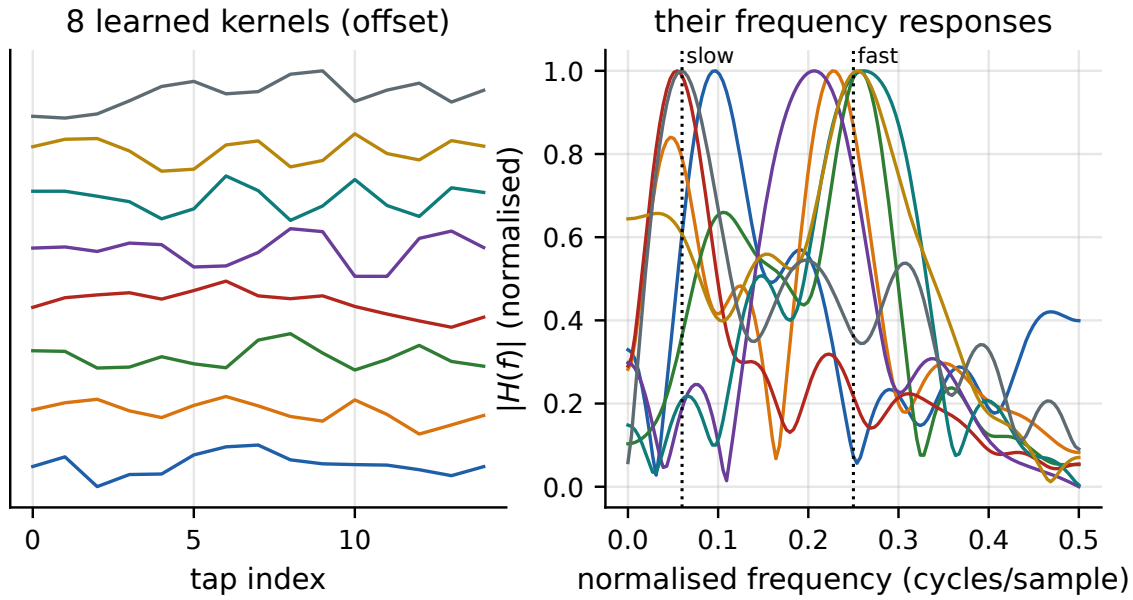


Figure 15.11: Left: the 8 learned 15-tap kernels of the first layer. Right: magnitude of their frequency responses $|H(f)|$. Dotted lines mark the slow and fast tone frequencies: several kernels have become band-pass filters tuned near them. Script: `code/ch15/train_signal_cnn1d.py`.

Wireless connection: a CNN is a learned matched-filter bank

The optimum detector for a known waveform $s[n]$ in white Gaussian noise is the matched filter: correlate the received signal with s^* and threshold the peak. A conv layer followed by global max pooling computes *exactly* "correlate with each kernel, take the peak". So the first layer of a CNN-based detector or classifier is a bank of matched filters whose templates are *learned*: if the true waveforms are known it can approximate them; if they are distorted by an unknown channel, it learns templates that work on average over the distortions. The deeper layers then combine filter outputs nonlinearly — something a classical filter bank does not do. This is the mental model to carry into modulation recognition (Chapter 22), spectrum sensing and preamble detection.

15.8.3 The classic MNIST network

The listing below is the standard MNIST CNN, with the same architecture as the official PyTorch examples repository. Follow the shapes: $1 \times 28 \times 28 \xrightarrow{\text{conv } 3\times3} 32 \times 26 \times 26 \xrightarrow{\text{conv } 3\times3} 64 \times 24 \times 24 \xrightarrow{\text{maxpool } 2} 64 \times 12 \times 12$, and $64 \cdot 12 \cdot 12 = 9216$ features enter the first dense layer. It has 1,199,882 parameters, 98% of them in `fc1` — a reminder that dense layers, not convolutions, dominate the parameter count.

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)      # 28x28x1  -> 26x26x32
        self.conv2 = nn.Conv2d(32, 64, 3, 1)     # 26x26x32 -> 24x24x64
        self.dropout1 = nn.Dropout(0.25)
        self.dropout2 = nn.Dropout(0.5)
        self.fc1 = nn.Linear(9216, 128)          # 12*12*64 = 9216 after 2x2 max-
    pool
```

```
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = F.max_pool2d(x, 2)                  # 24x24x64 -> 12x12x64
        x = self.dropout1(x)
        x = torch.flatten(x, 1)                 # -> 9216 features
        x = F.relu(self.fc1(x))
        x = self.dropout2(x)
        return self.fc2(x)                      # logits for 10 digits
```

Trained for a few epochs (Adadelta, learning rate 1.0), this network *typically* reaches about 99% test accuracy on MNIST. Treat that as a typical figure, not a guarantee: the exact value depends on seed, epochs and optimiser settings. (MNIST must be downloaded, so the book's figures use the offline digits above; the full training function is in `code/ch15/mnist_cnn_listing.py`.)

15.9 Efficient convolutions: 1×1 and depthwise-separable

1×1 convolution. With $k = 1$, (15.5) becomes $Y[o,i,j] = b[o] + \sum_c K[o,c]X[c,i,j]$: at every pixel, the channel vector is multiplied by the same $C_{\text{out}} \times C_{\text{in}}$ matrix. It mixes channels but not space — a dense layer applied per pixel. It is used to cheaply change the number of channels ("bottlenecks").

Depthwise-separable convolution. A standard conv mixes space and channels at the same time, costing $C_{\text{out}}C_{\text{in}}k^2$ weights. Factor it into (i) a *depthwise* conv: one $k \times k$ kernel per input channel, applied to that channel alone ($C_{\text{in}}k^2$ weights), then (ii) a *pointwise* 1×1 conv mixing channels ($C_{\text{in}}C_{\text{out}}$ weights). The weight ratio is

$$\frac{C_{\text{in}}k^2 + C_{\text{in}}C_{\text{out}}}{C_{\text{out}}C_{\text{in}}k^2} = \frac{1}{C_{\text{out}}} + \frac{1}{k^2}.$$

Example: $C_{\text{in}} = 64$, $C_{\text{out}} = 128$, $k = 3$: standard 73,728 weights; separable $576 + 8192 = 8768$, a ratio of $1/128 + 1/9 \approx 0.119$ — about $8.4\times$ fewer weights and multiplications. MobileNet-type networks, often used on embedded and edge receivers, are built from this block.

15.10 Classic architectures as block diagrams

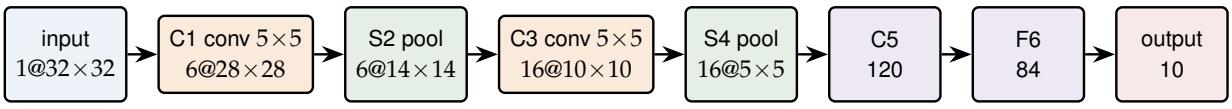


Figure 15.12: LeNet-5 (LeCun et al., 1998), the template of every CNN since: alternate convolution (feature extraction) and subsampling (pooling), then fully connected layers. The original used tanh-like activations and average-type subsampling; modern re-implementations use ReLU and max pooling.

Why does the residual connection help? If $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$ then by the chain rule $\partial\mathbf{y}/\partial\mathbf{x} = \partial F/\partial\mathbf{x} + \mathbf{I}$. The identity term means the gradient arriving at $\mathbf{y}$ reaches $\mathbf{x}$ *undiminished*, whatever F does; stacking many blocks no longer multiplies many small Jacobians together. You will see the same "additive path" idea in the LSTM cell state of Chapter 16.

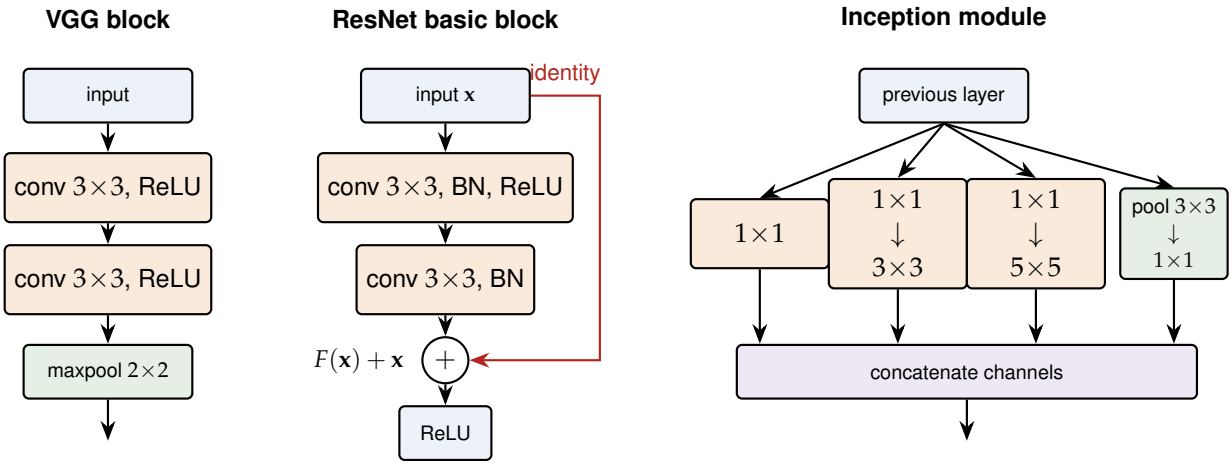


Figure 15.13: Three famous building blocks. **VGG**: only 3×3 convs; two stacked 3×3 layers have the receptive field of one 5×5 ($18C^2$ instead of $25C^2$ weights, plus an extra nonlinearity). **ResNet**: the block learns a *residual* $F(\mathbf{x})$ added to an identity shortcut, so gradients flow straight through the "+" and networks of 100+ layers train. **Inception**: parallel branches with different kernel sizes look at several scales at once; 1×1 convs reduce channels before the expensive branches.

15.11 CNNs on IQ samples

A complex baseband signal $r[n] = I[n] + jQ[n]$ cannot be fed directly to a real-valued network. The standard representation, introduced for radio modulation recognition, stacks the real and imaginary parts as a $2 \times N$ *real image*: row 0 is $I[0..N-1]$, row 1 is $Q[0..N-1]$. Equivalently it is a 1-D signal with 2 channels.

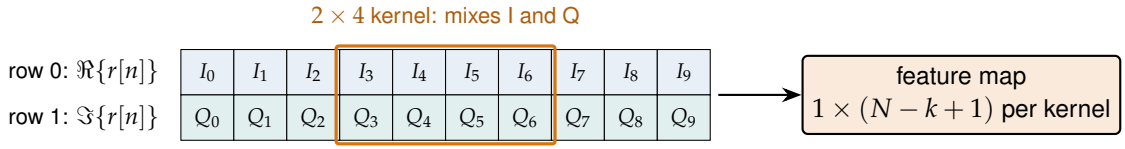


Figure 15.14: The $2 \times N$ IQ input. A first-layer kernel of size $2 \times k$ spans both rows, so each output is $\sum_m a_m I[n+m] + \sum_m c_m Q[n+m]$ — a real linear combination of I and Q over k samples.

Wireless connection: why a $2 \times k$ first kernel mixes I and Q

With a kernel $\begin{bmatrix} a_0 & \cdots & a_{k-1} \\ c_0 & \cdots & c_{k-1} \end{bmatrix}$ the output is $y[n] = \sum_m \big(a_m I[n+m] + c_m Q[n+m]\big)$. Choose $a_m = \Re\{s_m\}$ and $c_m = \Im\{s_m\}$ for a complex template s: then $y[n] = \Re\{\sum_m s_m^* \, r[n+m]\}$, the real part of a complex matched filter (cross-correlation with s). A second kernel with $a_m = -\Im\{s_m\}$, $c_m = \Re\{s_m\}$ produces the imaginary part. So *a pair of real $2 \times k$ kernels implements one complex FIR filter*, and a bank of them can learn phase-sensitive templates (pulse shapes, symbol transitions, frequency offsets). A $1 \times k$ kernel, by contrast, filters I and Q *separately* with the same real taps and can never rotate the constellation. Rule of thumb from the modulation-recognition literature: use $2 \times k$ (or 1-D conv with 2 input channels) in the first layer, then $1 \times k$ afterwards. We return to this in Chapter 22.

Wireless connection: other physical-layer uses

2-D CNNs treat a channel estimate $H[\text{subcarrier}, \text{OFDM symbol}]$ as an image (real and imaginary parts as 2 channels) to denoise or interpolate pilot-based estimates; spectrograms are processed as images for spectrum sensing and interference classification; and 1-D dilated CNNs model power-amplifier memory effects for digital predistortion. In each case the same two biases — locality and weight sharing — match the physics: channels are correlated over neighbouring subcarriers and symbols, and a waveform feature means the same thing wherever it occurs.

Chapter recap

- A convolutional layer slides a small kernel over the input and takes dot products: *locality* + *weight sharing* $\Rightarrow$ translation equivariance and very few parameters, $C_{\text{out}}(C_{\text{in}}k^2+1)$.
- DL "convolution" is cross-correlation (no flip). In DSP terms it is an FIR filter bank with learned taps.
- Output size $\lfloor (n+2p-k)/s \rfloor + 1$; padding preserves size, stride downsamples, dilation enlarges the receptive field cheaply. Receptive field: $r_l = r_{l-1} + (k_l - 1)d_l J_{l-1}$.
- Pooling summarises windows; max-pool routes the gradient to the argmax, average-pool spreads it equally.
- Backprop: $dK = X \star \delta$, $db = \sum \delta$, $dX = \text{pad}(\delta) \star \text{rot}_{180} K$ — the backward pass is itself a convolution. Verified by gradient check ($\sim 10^{-10}$).
- 1×1 convs mix channels; depthwise-separable convs cut cost by $\approx 1/C_{\text{out}} + 1/k^2$. Architectures: LeNet, VGG, ResNet (identity shortcut), Inception (multi-scale).
- For IQ data use a $2 \times N$ input; a $2 \times k$ first kernel pair is a complex FIR / matched filter. A CNN is a learned matched-filter bank.

Exercises

1. Compute the 1-D DL convolution of $x = [1, 3, 2, 0, 1]$ with $k = [2, -1]$ ($b = 0$). Then compute the *true* full convolution $x * k$. How are they related?
2. A layer `Conv2d(3, 64, 7, stride=2, padding=3)` receives a $3 \times 224 \times 224$ image. Give the output shape and the number of parameters.
3. Compute the receptive field of: conv 3×3 (s=1) $\rightarrow$ conv 3×3 (s=2) $\rightarrow$ conv 3×3 (s=1) $\rightarrow$ conv 3×3 (s=1).
4. For $X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 1 \end{bmatrix}$ and $K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ compute $Y = X \star K$ (valid). With upstream gradient $\delta = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ compute dK and dX using (15.9) and (15.11).
5. Explain why replacing one 5×5 conv layer by two stacked 3×3 layers (same channels C, no bias) keeps the receptive field, and compute the saving in weights.
6. How many weights does a depthwise-separable layer with $C_{\text{in}} = C_{\text{out}} = 256$, $k = 3$ need, compared with a standard conv? (Ignore biases.)

7. (Wireless) You want a first layer that implements 16 independent *complex* FIR filters of 8 taps on IQ input. How many real 2×8 kernels and how many real weights (no bias) do you need? Why can a `Conv2d(1,16,(1,8))` layer not do this?

Answers

1. DL: $y[i] = 2x[i] - x[i+1]$: $[2-3,\ 6-2,\ 4-0,\ 0-1] = [-1,4,4,-1]$. True full convolution: $(x * k)[n] = 2x[n] - x[n-1]$ for $n = 0..5$: $[2,\ 6-1,\ 4-3,\ 0-2,\ 2-0,\ -1] = [2,5,1,-2,2,-1]$. The DL result equals the valid part of the true convolution with the *flipped* kernel $[-1,2]$: $(x * [-1,2])[n] = -x[n] + 2x[n-1]$ gives at $n = 1..4$: $-3+2 = -1$, $-2+6 = 4$, $0+4 = 4$, $-1+0 = -1$. Same.

2. $\lfloor (224+6-7)/2 \rfloor + 1 = \lfloor 111.5 \rfloor + 1 = 112$: output $64 \times 112 \times 112$. Parameters $64(3 \cdot 49 + 1) = 64 \cdot 148 = 9472$.

3. $r_1 = 3$, $J_1 = 1$; $r_2 = 3 + 2 \cdot 1 = 5$, $J_2 = 2$; $r_3 = 5 + 2 \cdot 2 = 9$; $r_4 = 9 + 2 \cdot 2 = 13$.

4. $Y[i,j] = X[i,j] - X[i+1,j+1]$: $Y = \begin{bmatrix} 1-1 & 2-3 \\ 0-1 & 1-1 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}$. $dK[m,n] = \sum_{i,j} \delta[i,j] X[i+m, j+n] = X[m,n] + X[m+1,n+1]$: $dK = \begin{bmatrix} 1+1 & 2+3 \\ 0+1 & 1+1 \end{bmatrix} = \begin{bmatrix} 2 & 5 \\ 1 & 2 \end{bmatrix}$. $dX[p,q] = \sum_{i,j} \delta[i,j] K[p-i, q-j]$: only $\delta[0,0]$ and $\delta[1,1]$ are nonzero, so $dX = K$ placed at $(0,0)$ plus K placed at $(1,1)$: $dX = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1+1 & 0 \\ 0 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -1 \end{bmatrix}$.

5. Receptive field $1+2+2 = 5$, same as $1+4$. Weights: $25C^2$ versus $2 \cdot 9C^2 = 18C^2$, a 28% saving, plus one extra nonlinearity (more expressive).

6. Standard: $256 \cdot 256 \cdot 9 = 589{,}824$. Separable: $256 \cdot 9 + 256 \cdot 256 = 2304 + 65{,}536 = 67{,}840$, ratio ≈ 0.115 $(1/256 + 1/9)$.

7. Each complex filter needs a real-part and an imaginary-part kernel: 32 kernels of 2×8, i.e. $32 \cdot 16 = 512$ real weights (equivalently 16 complex filters $\times$ 8 complex taps $\times$ 2 reals $\times$ 2 output parts). A 1×8 kernel slides along each row separately: it applies the same real taps to I and to Q and never combines them, so it cannot implement $\Re\{s^* r\}$, which needs $\Re s \cdot I + \Im s \cdot Q$.

Self-test quiz

1. What two inductive biases does a convolutional layer build in?
   Locality and weight sharing (giving translation equivariance).

2. `Conv2d(1,32,3)` has how many parameters?
   $32(1 \cdot 9 + 1) = 320$.

3. Output size for $n = 32$, $k = 5$, $p = 2$, $s = 2$?
   $\lfloor 31/2 \rfloor + 1 = 16$.

4. During backprop through 2×2 max pooling, which inputs receive gradient?
   Only the argmax of each window; the others get zero.

5. Which operation gives $\partial L / \partial X$ for a conv layer?

Full convolution of δ with the kernel, i.e. zero-padded δ correlated with the 180°-rotated kernel.

6. Why use a $2 \times k$ first kernel on a $2 \times N$ IQ input?
   It combines I and Q, so a pair of such kernels can implement a complex (phase-sensitive) FIR/matched filter.

Further reading

- Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, 86(11), 1998 (LeNet-5).
- K. Simonyan, A. Zisserman, "Very deep convolutional networks for large-scale image recognition," ICLR 2015 (VGG).
- K. He, X. Zhang, S. Ren, J. Sun, "Deep residual learning for image recognition," CVPR 2016 (ResNet).
- C. Szegedy et al., "Going deeper with convolutions," CVPR 2015 (Inception/GoogLeNet).
- A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017 (depthwise-separable convs).
- V. Dumoulin, F. Visin, "A guide to convolution arithmetic for deep learning," arXiv preprint 1603.07285, 2016 (output sizes, padding, stride, transposed convs).
- T. J. O'Shea, J. Corgan, T. C. Clancy, "Convolutional radio modulation recognition networks," EANN 2016 (CNNs on $2 \times N$ IQ; see Chapter 22).
- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016, Chapter 9.

Chapter 16

Recurrent Networks: RNN, LSTM and GRU

A received radio signal, a sentence, a stock price, a stream of channel estimates: all are *sequences*, where the meaning of the current sample depends on what came before. The networks of earlier chapters take a fixed-size input and have no memory. In this chapter you will learn *recurrent neural networks* (RNNs), which read a sequence one step at a time and carry a *hidden state* — a running summary of the past — from step to step. You will derive *backpropagation through time* by hand, see precisely why plain RNNs forget (the vanishing-gradient problem, explained through the spectral radius of a matrix), and learn the two cures that made sequence learning practical: the *LSTM* and the *GRU*. We will measure the difference experimentally, and finish with a wireless experiment that prepares Chapter 27: predicting a Rayleigh fading channel several samples ahead with an LSTM, compared with the classical linear (Wiener) predictor.

16.1 Sequence data and why we need memory

A sequence is an ordered list $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T$ where each $\mathbf{x}_t \in \mathbb{R}^d$ is a vector of d features at time t. Examples: $d = 2$ for IQ samples (I_t, Q_t); $d = 1$ for a scalar sensor; $d = 10{,}000$ for one-hot encoded words. Three properties make sequences different from the images of Chapter 15:

- **Variable length.** One sentence has 5 words, the next 50. A model with a fixed input size cannot handle both.
- **Order matters.** "dog bites man" $\neq$ "man bites dog". A model that averages its inputs throws this away.
- **Long-range dependencies.** The verb at the end of a German sentence agrees with a subject at the start; a decoder must remember the state of a convolutional code over many symbols.

A 1-D CNN handles order and position, but only within its receptive field. A recurrent network instead keeps a *state* that can, in principle, remember arbitrarily far back.

Intuition: reading with a notepad

Imagine reading a long book through a slot that shows only one word at a time. You keep a small notepad: after each word you update your notes ("the detective suspects the butler; it is raining"). You never re-read old pages; everything you know about the past lives in the notepad. An RNN's hidden state $\mathbf{h}_t$ is that notepad, and the *same* rule for updating the notes is used at every word. Learning means learning *what to write down*.

16.2 The vanilla RNN

Definition: Elman / vanilla RNN

With hidden size h, input size d and output size q:

$$\mathbf{a}_t = \mathbf{W}_{xh}\,\mathbf{x}_t + \mathbf{W}_{hh}\,\mathbf{h}_{t-1} + \mathbf{b}_h, \tag{16.1}$$

$$\mathbf{h}_t = \tanh(\mathbf{a}_t), \tag{16.2}$$

$$\hat{\mathbf{y}}_t = \mathbf{W}_{hy}\,\mathbf{h}_t + \mathbf{b}_y. \tag{16.3}$$

$\mathbf{W}_{xh} \in \mathbb{R}^{h\times d}$ maps input to hidden, $\mathbf{W}_{hh} \in \mathbb{R}^{h\times h}$ maps the previous state to the new one (the *recurrent* weights), $\mathbf{W}_{hy} \in \mathbb{R}^{q\times h}$ reads out. The initial state $\mathbf{h}_0$ is usually $\mathbf{0}$. **The same weights are used at every time step** — weight sharing across time, just as a CNN shares weights across space.

We use column vectors here because it makes the time recursion easiest to read; in code each $\mathbf{x}_t$ is a row of a batch, exactly as before. The tanh squashes the state into $(-1, 1)$ so it cannot blow up in the forward pass. Parameters: $hd + h^2 + h$ for the recurrent part plus $qh + q$ for the read-out — *independent of the sequence length* T.

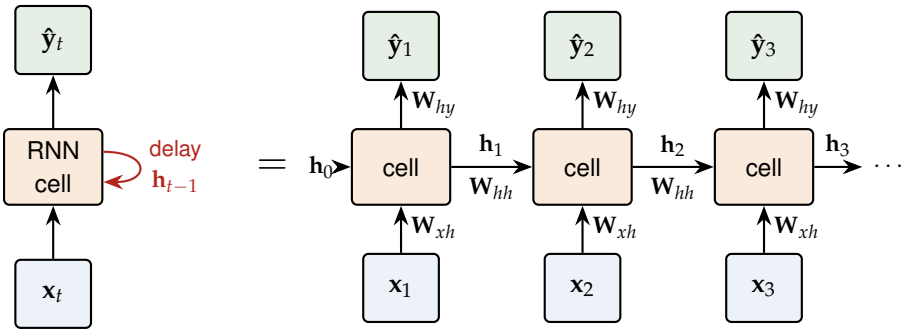


Figure 16.1: Left: the RNN as a loop — the state is fed back through a one-step delay (an IIR structure, in DSP terms). Right: the same network *unrolled* in time. Unrolled, it is just a very deep feed-forward network with one layer per time step, in which all layers share the weights $\mathbf{W}_{xh}$, $\mathbf{W}_{hh}$, $\mathbf{W}_{hy}$.

Worked example: two steps of a scalar RNN

Take $d = h = 1$, $W_{xh} = 0.5$, $W_{hh} = 0.8$, $b_h = 0$, $h_0 = 0$, inputs $x = [1, 1]$.

$$t = 1: \quad a_1 = 0.5 \cdot 1 + 0.8 \cdot 0 = 0.5, \qquad h_1 = \tanh(0.5) = 0.462,$$

$$t = 2: \quad a_2 = 0.5 \cdot 1 + 0.8 \cdot 0.462 = 0.870, \qquad h_2 = \tanh(0.870) = 0.701.$$

The same input gave a larger state the second time, because h_2 also contains a (decayed) copy of h_1: the network "remembers" that it has seen a 1 before.

Wireless connection: an RNN is a nonlinear IIR filter

Drop the tanh and take scalars: $h_t = w\,h_{t-1} + u\,x_t$. This is a first-order IIR filter (a leaky integrator) with pole at $z = w$; its impulse response is $u\,w^t$. It is stable if $|w| < 1$, and its memory decays like w^t. A vanilla RNN is a *vector, nonlinear* version: the poles become the eigenvalues of $\mathbf{W}_{hh}$, and the tanh adds saturation. Everything you know about IIR stability

foreshadows the gradient problems below: a filter with poles inside the unit circle forgets exponentially, one with poles outside it explodes.

16.2.1 Sequence tasks: many-to-one, many-to-many, sequence-to-sequence

How we use the outputs depends on the task (Fig. 16.2):

- **Many-to-one**: read the whole sequence, use only the last state $\mathbf{h}_T$ (classification of a burst; modulation recognition).
- **Many-to-many (aligned)**: one output per input (per-symbol detection, equalisation, tagging each word).
- **Sequence-to-sequence**: an *encoder* RNN compresses the input sequence into its final state; a *decoder* RNN, initialised with that state, generates an output sequence of possibly different length (translation; multi-step forecasting).

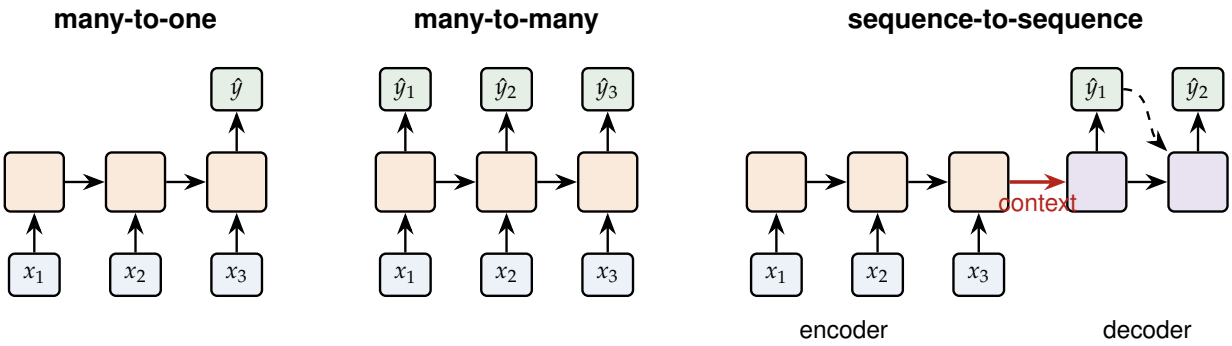


Figure 16.2: Three ways to use a recurrent network. In sequence-to-sequence models, the encoder's final state (red) is the "context" that initialises the decoder; the decoder often feeds its previous output back as its next input (dashed).

16.3 Backpropagation through time (BPTT)

Because the unrolled RNN is a feed-forward network, we train it with ordinary backpropagation applied to the unrolled graph. The only new feature is that the *same* weights appear in every layer.

16.3.1 Gradient for shared weights

Let the total loss be a sum of per-step losses, $L = \sum_{t=1}^{T} L_t(\hat{\mathbf{y}}_t, \mathbf{y}_t)$ (for many-to-one only L_T is nonzero). Imagine giving every time step its own private copy $\mathbf{W}_{hh}^{(t)}$ of the recurrent matrix, all set equal. A small change Δ to the shared $\mathbf{W}_{hh}$ changes *every* copy by Δ, so by the multivariate chain rule

$$\frac{\partial L}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \frac{\partial L}{\partial \mathbf{W}_{hh}^{(t)}}. \tag{16.4}$$

The gradient of a shared weight is the sum of the gradients of its copies — the same rule we used for CNN kernels shared over positions. Each copy's gradient is the familiar "local error times layer input" of Chapter 9.

16.3.2 The backward recursion

Define the error signal at the pre-activation, $\delta_t = \partial L / \partial \mathbf{a}_t$ (a column vector). Two paths from $\mathbf{h}_t$ lead to the loss: through the output $\hat{\mathbf{y}}_t$ (now), and through $\mathbf{a}_{t+1}$ (the future). So

$$\frac{\partial L}{\partial \mathbf{h}_t} = \underbrace{\mathbf{W}_{hy}^{\mathsf{T}} \frac{\partial L_t}{\partial \hat{\mathbf{y}}_t}}_{\text{from the output at } t} + \underbrace{\mathbf{W}_{hh}^{\mathsf{T}} \delta_{t+1}}_{\text{from the future}}, \qquad (\delta_{T+1} = \mathbf{0}) \tag{16.5}$$

$$\delta_t = \frac{\partial L}{\partial \mathbf{h}_t} \odot \left(1 - \mathbf{h}_t^2\right), \tag{16.6}$$

using $\tanh'(a) = 1 - \tanh^2(a)$. We run (16.5)–(16.6) *backwards* from $t = T$ to $t = 1$, and accumulate, by (16.4),

Key equation: BPTT weight gradients

$$\frac{\partial L}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \delta_t \, \mathbf{h}_{t-1}^{\mathsf{T}}, \qquad \frac{\partial L}{\partial \mathbf{W}_{xh}} = \sum_{t=1}^{T} \delta_t \, \mathbf{x}_t^{\mathsf{T}}, \qquad \frac{\partial L}{\partial \mathbf{b}_h} = \sum_{t=1}^{T} \delta_t. \tag{16.7}$$

Worked example: BPTT by hand on the two-step RNN

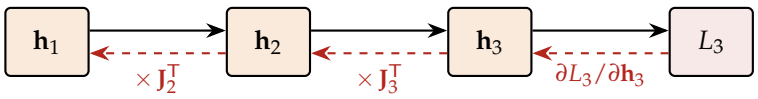
Continue the example above with a target $y = 1$ on the *final* state only: $L = \frac{1}{2}(h_2 - 1)^2$ (read-out $\hat{y} = h_2$).

$$\begin{aligned}
\partial L / \partial h_2 &= h_2 - 1 = 0.701 - 1 = -0.2988, \\
\delta_2 &= -0.2988 \cdot (1 - 0.701^2) = -0.2988 \cdot 0.5083 = -0.1519, \\
\partial L / \partial h_1 &= W_{hh}\, \delta_2 = 0.8 \cdot (-0.1519) = -0.1215 \quad \text{(no output loss at } t = 1\text{)}, \\
\delta_1 &= -0.1215 \cdot (1 - 0.462^2) = -0.1215 \cdot 0.7864 = -0.0955.
\end{aligned}$$

Sum over time, (16.7):

$$\begin{aligned}
\partial L / \partial W_{hh} &= \delta_2 h_1 + \delta_1 h_0 = (-0.1519)(0.462) + 0 = -0.0702, \\
\partial L / \partial W_{xh} &= \delta_2 x_2 + \delta_1 x_1 = -0.1519 - 0.0955 = -0.2474.
\end{aligned}$$

Both gradients are negative, so gradient descent *increases* both weights, making h_2 larger and closer to the target 1. PyTorch's autograd gives the same -0.0702 and -0.2474 (checked in `code/ch16/rnn_gradients.py`).



$$\frac{\partial L_3}{\partial \mathbf{h}_1} = \mathbf{J}_2^{\mathsf{T}} \mathbf{J}_3^{\mathsf{T}} \frac{\partial L_3}{\partial \mathbf{h}_3}, \qquad \mathbf{J}_t = \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \operatorname{diag}(1 - \mathbf{h}_t^2)\, \mathbf{W}_{hh}$$

Figure 16.3: The backward pass in time multiplies the error by one Jacobian per step. After k steps the error has been multiplied by k matrices: it shrinks or grows geometrically.

16.4 Vanishing and exploding gradients

16.4.1 The Jacobian product

How does the loss at step T depend on the state k steps earlier? Unrolling (16.5) (ignoring the output terms) gives

$$\frac{\partial L_T}{\partial \mathbf{h}_{T-k}} = \Big(\prod_{j=T-k+1}^{T} \mathbf{J}_j^{\mathsf{T}} \Big) \frac{\partial L_T}{\partial \mathbf{h}_T}, \qquad \mathbf{J}_j = \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \operatorname{diag}\big(1 - \mathbf{h}_j^2\big)\, \mathbf{W}_{hh}. \tag{16.8}$$

(The Jacobian follows from (16.1)–(16.2): $\partial h_{j,r} / \partial h_{j-1,s} = (1 - h_{j,r}^2)\, W_{hh,rs}$.) This product of k matrices is the heart of the problem.

16.4.2 The spectral-radius argument

Take the simplest case: a *linear* RNN (no tanh, so the diagonal factor is $\mathbf{I}$). Then the product is $(\mathbf{W}_{hh}^{\mathsf{T}})^k$. Suppose $\mathbf{W}_{hh}$ can be diagonalised, $\mathbf{W}_{hh}^{\mathsf{T}} = \mathbf{V} \boldsymbol{\Lambda} \mathbf{V}^{-1}$, with eigenvalues $\lambda_1, \dots, \lambda_h$. Then

$$(\mathbf{W}_{hh}^{\mathsf{T}})^k = \mathbf{V} \boldsymbol{\Lambda}^k \mathbf{V}^{-1} = \mathbf{V}\, \operatorname{diag}(\lambda_1^k, \dots, \lambda_h^k)\, \mathbf{V}^{-1}.$$

Write the incoming gradient in the eigenvector basis, $\mathbf{g} = \sum_i c_i \mathbf{v}_i$. Then $(\mathbf{W}_{hh}^{\mathsf{T}})^k \mathbf{g} = \sum_i c_i \lambda_i^k \mathbf{v}_i$. For large k the term with the largest $|\lambda_i|$ — the *spectral radius* $\rho(\mathbf{W}_{hh}) = \max_i |\lambda_i|$ — dominates:

Key equation: vanishing / exploding

$$\left\| \frac{\partial L_T}{\partial \mathbf{h}_{T-k}} \right\| \sim \rho^k \quad \Longrightarrow \quad \begin{cases} \rho < 1: & \text{gradient vanishes exponentially (forgets)} \\ \rho > 1: & \text{gradient explodes exponentially} \end{cases} \tag{16.9}$$

With the tanh, each step also multiplies by $\operatorname{diag}(1 - h^2)$, whose entries are ≤ 1 (and much smaller when units saturate). This can only make vanishing *worse*; a precise bound is $\|\mathbf{J}_j\| \le \|\mathbf{W}_{hh}\|$, so if the largest singular value of $\mathbf{W}_{hh}$ is below 1, gradients must vanish. Figure 16.4 shows the geometric behaviour numerically.

Intuition: a whisper passed down a long line

A message whispered down a line of 50 people: if each person passes on 80% of the loudness, the last one hears $0.8^{50} \approx 10^{-5}$ of it — nothing. If each person shouts 20% louder, the end of the line is deafening. Gradient descent needs the error signal from step T to reach step $T - 50$ at a usable strength; a vanilla RNN can only do that if every link has gain almost exactly 1, which is a knife-edge.

Consequence: a vanilla RNN learns short-range patterns easily, but the gradient that would teach it a dependency 50 steps long is buried under the gradients from short-range patterns. It is not that the RNN *cannot represent* long memory — it is that gradient descent *cannot find* it.

16.4.3 Practical remedies: truncated BPTT and gradient clipping

Truncated BPTT. For very long streams (a continuous radio recording) we cannot unroll millions of steps. We cut the stream into chunks of τ steps (say 100), run forward through each chunk carrying the hidden state over from the previous chunk, but back-propagate only within the chunk (in PyTorch: `h = h.detach()` between chunks). Memory and compute become $O(\tau)$; the price is that dependencies longer than τ get no direct gradient.

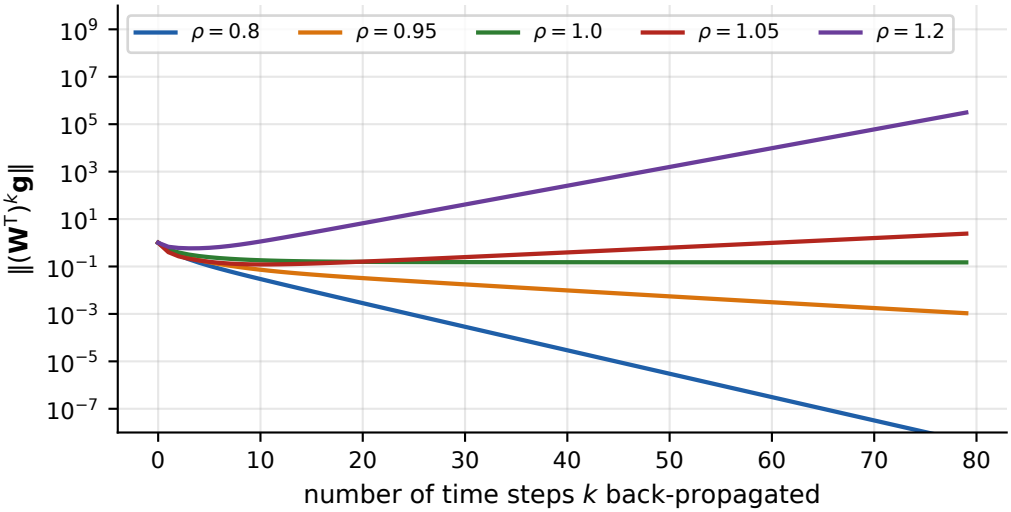


Figure 16.4: Norm of a gradient vector after being back-propagated k steps through a linear RNN, $(\mathbf{W}^\mathsf{T})^k\mathbf{g}$, for a fixed random symmetric $\mathbf{W}$ rescaled to spectral radius ρ. On this log scale the curves are straight lines with slope $\log\rho$: $\rho = 0.8$ loses more than six orders of magnitude in 60 steps, while $\rho = 1.2$ gains more than three (after a short initial dip while the dominant eigen-direction takes over). Only $\rho \approx 1$ keeps the signal. Script: `code/ch16/rnn_gradients.py`.

Gradient clipping fixes *exploding* gradients. If the global gradient norm $\|\mathbf{g}\|$ exceeds a threshold c, rescale it:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\Big(1,\ \frac{c}{\|\mathbf{g}\|}\Big). \tag{16.10}$$

The direction is kept; only the step length is limited. Example: $\mathbf{g} = (3, 4)$, $\|\mathbf{g}\| = 5$, $c = 1 \Rightarrow \mathbf{g} \leftarrow (0.6, 0.8)$. In PyTorch: `torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)` between `backward()` and `step()`. Clipping does nothing for *vanishing* gradients — that needs a new architecture.

16.5 Long Short-Term Memory (LSTM)

The LSTM (Hochreiter and Schmidhuber, 1997; the forget gate was added by Gers, Schmidhuber and Cummins, 2000) adds a second state, the *cell state* $\mathbf{c}_t$, which is updated *additively* and controlled by *gates*.

Intuition: a conveyor belt with three valves

Picture the cell state as a conveyor belt running through time. Items on the belt travel along untouched unless someone acts. At each step three valves — each a number between 0 and 1 computed from the current input and previous output — decide: how much of what is on the belt to *keep* (forget gate f), how much new material to *put on* (input gate i), and how much of the belt to *show* to the outside world (output gate o). Because the belt is only scaled and added to (never pushed through a squashing nonlinearity), information — and gradient — can ride along it for hundreds of steps.

Definition: LSTM cell

With σ the logistic sigmoid, $\odot$ element-wise product, and $[\mathbf{h}_{t-1}; \mathbf{x}_t]$ the stacked vector of size $h + d$:

$$\mathbf{f}_t = \sigma\big(\mathbf{W}_f[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f\big) \qquad \text{forget gate} \tag{16.11}$$

$$\mathbf{i}_t = \sigma\big(\mathbf{W}_i[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i\big) \qquad \text{input gate} \tag{16.12}$$

$$\mathbf{g}_t = \tanh\big(\mathbf{W}_g[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_g\big) \qquad \text{candidate values} \tag{16.13}$$

$$\mathbf{o}_t = \sigma\big(\mathbf{W}_o[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o\big) \qquad \text{output gate} \tag{16.14}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t \qquad \text{cell state (the belt)} \tag{16.15}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \qquad \text{hidden state / output} \tag{16.16}$$

Each $\mathbf{W}_\bullet \in \mathbb{R}^{h \times (h+d)}$. Gates use σ because their job is to be a soft switch in $(0, 1)$; the candidate uses tanh so it can add positive or negative values.

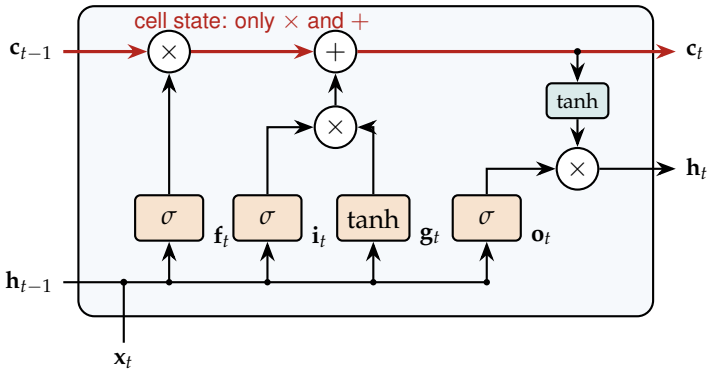


Figure 16.5: The LSTM cell. The red line at the top is the cell state $\mathbf{c}$ (the conveyor belt): it is multiplied by the forget gate and has the gated candidate added, nothing else. The hidden state $\mathbf{h}_t$ is a gated, squashed view of the belt. All four orange blocks read the same input $[\mathbf{h}_{t-1}; \mathbf{x}_t]$.

Worked example: one LSTM step with scalars

$d = h = 1$, $x_t = 1$, $h_{t-1} = 0$, $c_{t-1} = 0.5$. Choose weights so that the pre-activations are 0.5 (forget), 1.0 (input), 0.8 (candidate), 0 (output). Then

$$f_t = \sigma(0.5) = 0.622, \quad i_t = \sigma(1.0) = 0.731, \quad g_t = \tanh(0.8) = 0.664, \quad o_t = \sigma(0) = 0.5,$$

$$c_t = f_t c_{t-1} + i_t g_t = 0.622 \cdot 0.5 + 0.731 \cdot 0.664 = 0.311 + 0.485 = 0.797,$$
$$h_t = o_t \tanh(c_t) = 0.5 \cdot \tanh(0.797) = 0.5 \cdot 0.662 = 0.331.$$

About 62% of the old memory was kept and new content was written on top.

16.5.1 Why the gradient survives: $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \mathbf{f}_t$

Differentiate (16.15) with respect to $\mathbf{c}_{t-1}$, holding the gates fixed (the direct path along the belt):

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}}\Big|_{\text{direct}} = \operatorname{diag}(\mathbf{f}_t), \qquad \text{so} \qquad \frac{\partial \mathbf{c}_T}{\partial \mathbf{c}_{T-k}}\Big|_{\text{direct}} = \operatorname{diag}\Big(\prod_{j=T-k+1}^{T} \mathbf{f}_j\Big). \tag{16.17}$$

Compare with (16.8). In the vanilla RNN the gradient is multiplied by a full matrix $\mathbf{W}_{hh}$ and a $\tanh'$ factor at every step — fixed by the weights, the same for every input. In the LSTM it is multiplied by the forget gate, a number the network *chooses at each step from the data*. If the network has learned that something must be remembered, it sets $f \approx 1$ and both the information and its gradient pass through unchanged, however many steps. (There are also indirect paths through the gates and $\mathbf{h}$, but the direct path is what guarantees a non-vanishing route.) This is the same principle as the ResNet shortcut of Chapter 15: an additive path with gain close to 1.

Pitfall: initialise the forget bias high

With the default initialisation the forget-gate pre-activation is around 0, so $f \approx \sigma(0) = 0.5$ at the start of training, and $0.5^{50} \approx 10^{-15}$: the LSTM begins life as forgetful as a vanilla RNN. Initialising the forget-gate bias to 1 ($f \approx 0.73$) or higher is a standard trick (Gers et al. 2000; Jozefowicz et al. 2015). In our memory experiment below a bias of 3 ($f \approx 0.95$) was necessary for the 60-step task. In PyTorch the gate order inside `bias_ih_l0` is (input, forget, cell, output), so the forget slice is `[h:2*h]`.

16.5.2 Counting LSTM parameters

Four gate blocks, each with an $h \times (h+d)$ matrix and an h-vector bias:

Key equation: LSTM parameters

$$\#\text{params} = 4\big(h(h+d)+h\big). \tag{16.18}$$

For $d = 10$, $h = 20$: $4(20 \cdot 30 + 20) = 4 \cdot 620 = 2480$. PyTorch's `nn.LSTM(10, 20)` reports 2560, because it keeps *two* bias vectors per gate (`bias_ih` and `bias_hh`, which are simply added): $2480 + 4 \cdot 20 = 2560$. For comparison, a vanilla RNN with the same sizes has $20 \cdot 30 + 20 = 620$ (PyTorch: 640), so an LSTM is $4\times$ larger and $\approx 4\times$ slower per step.

16.6 The Gated Recurrent Unit (GRU)

The GRU (Cho et al., 2014) merges the cell and hidden states and uses two gates instead of three:

Definition: GRU cell

$$\mathbf{z}_t = \sigma\big(\mathbf{W}_z[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_z\big) \qquad \text{update gate} \tag{16.19}$$
$$\mathbf{r}_t = \sigma\big(\mathbf{W}_r[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_r\big) \qquad \text{reset gate} \tag{16.20}$$
$$\tilde{\mathbf{h}}_t = \tanh\big(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_h\big) \qquad \text{candidate} \tag{16.21}$$
$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1} \qquad \text{interpolate old and new} \tag{16.22}$$

The update gate $\mathbf{z}$ plays the role of both forget gate ($\mathbf{z}$) and input gate ($1 - \mathbf{z}$): the new state is a convex mix of the old state and a candidate. When $z \approx 1$, $\mathbf{h}_t \approx \mathbf{h}_{t-1}$ and $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1} \approx \mathbf{I}$ — the same additive "gradient highway". The reset gate lets the candidate ignore the past when a new segment starts. (Conventions differ: PyTorch writes $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{n}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1}$ as here, and applies $\mathbf{r}_t$ after the matrix product; some papers swap z and $1 - z$.) Parameters: $3(h(h+d)+h)$; PyTorch `nn.GRU(10,20)` has 1920. GRUs are 25% cheaper than LSTMs and in

many benchmarks perform similarly; there is no universal winner.

16.7 Bidirectional and stacked RNNs

Stacked (deep) RNNs. The hidden sequence $\mathbf{h}_1^{(1)}, \dots, \mathbf{h}_T^{(1)}$ of one RNN layer becomes the input sequence of a second layer, and so on (`num_layers=2` in PyTorch). Lower layers capture local patterns, higher layers slower structure — like stacking conv layers.

Bidirectional RNNs. When the whole sequence is available before we must decide (offline decoding, classifying a recorded burst), run one RNN forwards and another backwards and concatenate their states, $\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$. Each output then sees both the past and the future (Fig. 16.6). A bidirectional RNN is *non-causal*, so it cannot be used for real-time prediction — the same distinction as between a causal and a zero-phase filter.

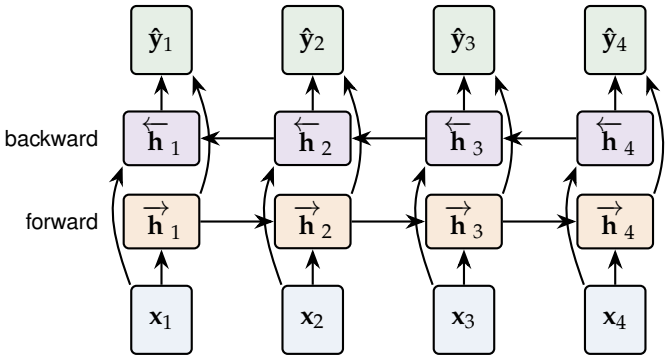


Figure 16.6: A bidirectional RNN. The forward chain (orange) reads left to right, the backward chain (purple) right to left; each output combines both, so $\hat{\mathbf{y}}_2$ depends on the whole sequence.

16.8 Experiments: watching memory fail and succeed

16.8.1 Gradient magnitude versus time lag (at initialisation)

We feed random sequences of length 100 to a freshly initialised PyTorch `nn.RNN` and `nn.LSTM` (hidden size 32, LSTM forget bias 1), put a loss on the *final* state only, and measure $\|\partial L/\partial \mathbf{x}_t\|$ for every t — how strongly an input at lag $T-t$ can influence learning. Figure 16.7 shows the result averaged over 20 sequences. This is an *illustration at initialisation*, not a statement about trained networks: training changes the weights and hence the curves.

16.8.2 A long-range memory task

Each training sequence begins with one of four "marker" symbols (one-hot over 8 symbols); the following $T-1$ steps are random "distractor" symbols. After the last step the network must say which marker it saw at $t=1$. Chance is 25%. All three models have hidden size 32, a linear read-out on the final state (many-to-one), Adam with learning rate 3×10^{-3}, batch 64, gradient clipping at norm 1, and the gate-bias trick (forget/update bias 3) for the LSTM and GRU.

```
def batch(B, T, g):
    y = torch.randint(0, NCLS, (B,), generator=g)
    s = torch.randint(NCLS, NSYM, (B, T), generator=g)       # random distractors 4..7 at every step
```

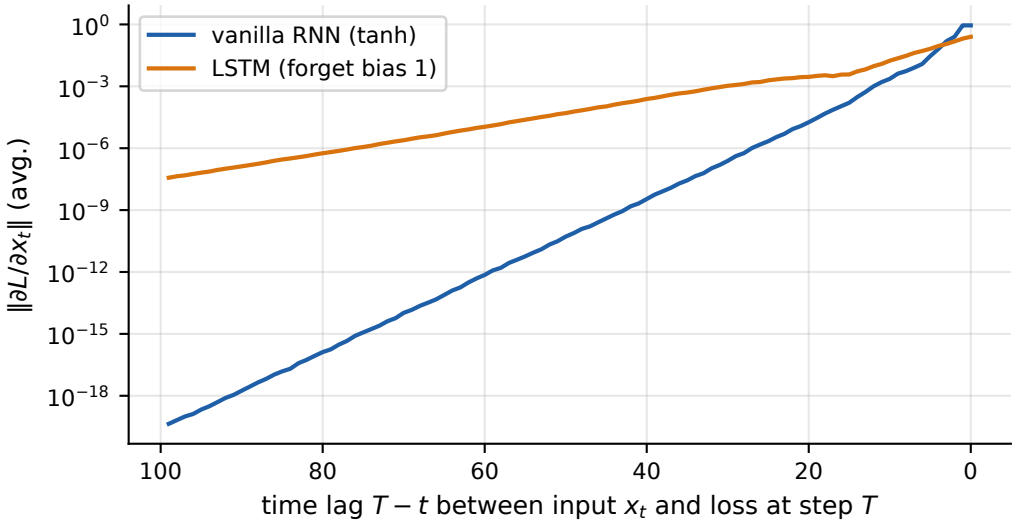


Figure 16.7: Average gradient norm reaching input $\mathbf{x}_t$ from a loss at the last step, versus time lag (log scale), for untrained networks. Both decay geometrically, but the vanilla RNN loses about ten orders of magnitude over 50 steps while the LSTM loses less than four. Gradient descent can therefore still "hear" distant inputs through an LSTM.

```
    s[:, 0] = y
    return F.one_hot(s, NSYM).float(), y

class Model(nn.Module):
    def __init__(self, kind):
        super().__init__()
        self.rnn = {'RNN': nn.RNN, 'LSTM': nn.LSTM, 'GRU': nn.GRU}[kind](NSYM, H,
    batch_first=True)
        self.out = nn.Linear(H, NCLS)
        # gate-bias trick: start in "remember" mode. PyTorch gate order: LSTM (i,
    f, g, o); GRU (r, z, n)
        if kind == 'LSTM':                                  # forget gate f ~
    sigmoid(3) = 0.95
            with torch.no_grad(): self.rnn.bias_ih_l0[H:2*H].fill_(FORGET_BIAS)
        if kind == 'GRU':                                   # update gate z ~
    0.95 keeps h_{t-1}
            with torch.no_grad(): self.rnn.bias_ih_l0[H:2*H].fill_(FORGET_BIAS)
    def forward(self, x):
        o, _ = self.rnn(x); return self.out(o[:, -1])
```

Pitfall: what the experiment does and does not show

The vanilla RNN *could* represent the solution (it only has to hold one of four values steady), and with enough tuning, longer training or orthogonal initialisation it sometimes finds it. The experiment shows that, with identical training effort, gated architectures find long-range solutions far more easily. While preparing this run we also saw that with a forget bias of only 1 the LSTM *also* stayed at chance at $T = 40$ — the gates must start wide open.

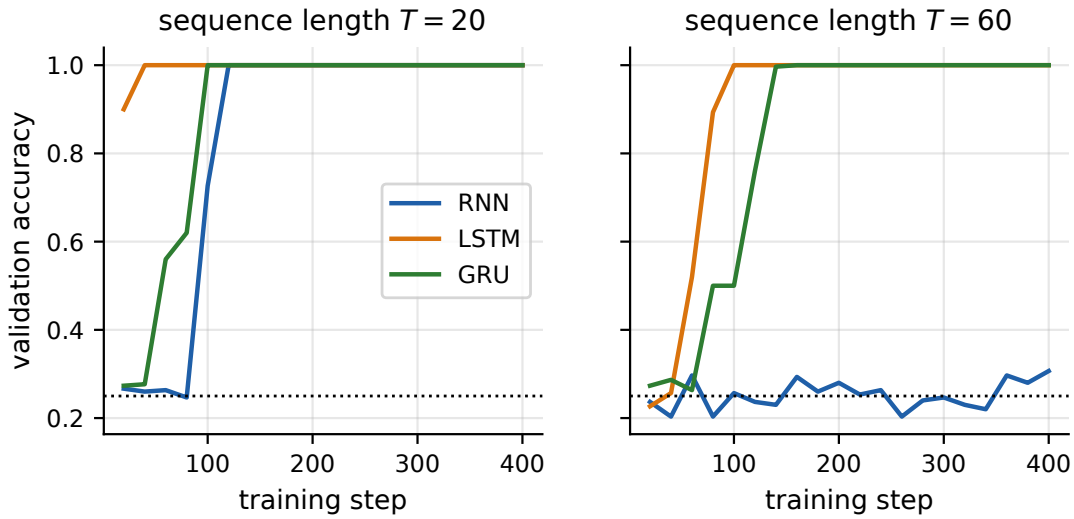


Figure 16.8: Validation accuracy on the memory task. With $T = 20$ all three models solve it. With $T = 60$ the vanilla RNN stays at chance level (dotted line, 25%) for the whole run, while the LSTM and GRU reach 100% within about 150 steps. Our small single-seed run; script `code/ch16/memory_task.py`.

16.9 The CLDNN: convolution + LSTM + dense

CNNs extract local features shift-invariantly; LSTMs model how features evolve over time. Combining them gives the *CLDNN* (Convolutional, Long short-term memory, Deep Neural Network), introduced by Sainath et al. (2015) for speech recognition and later applied to radio modulation recognition, where we will meet it again in Chapter 22.

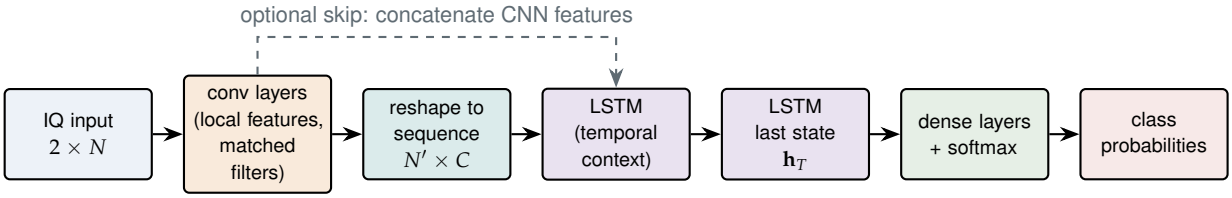


Figure 16.9: CLDNN block diagram. Convolutions turn the raw signal into a sequence of feature vectors (one per time position, C channels); LSTMs integrate them over time; dense layers classify from the final state. Sainath et al. also fed the CNN input to the LSTM through a skip connection (dashed).

In PyTorch, the only subtlety is the reshape: a `Conv1d` output of shape (batch, C, N') must be permuted to (batch, N', C) before an `LSTM(..., batch_first=True)`:

```
class CLDNN(nn.Module):
    def __init__(self, n_classes, C=32, H=64):
        super().__init__()
        self.conv = nn.Sequential(nn.Conv1d(2, C, 7, padding=3), nn.ReLU(),
                                  nn.Conv1d(C, C, 5, padding=2), nn.ReLU())
        self.lstm = nn.LSTM(C, H, batch_first=True)
        self.fc = nn.Sequential(nn.Linear(H, 64), nn.ReLU(), nn.Linear(64,
    n_classes))
    def forward(self, x):                    # x: (B, 2, N)  I and Q as channels
        f = self.conv(x).permute(0, 2, 1)    # (B, N, C)  sequence of feature
    vectors
        o, (hT, cT) = self.lstm(f)
        return self.fc(hT[-1])               # many-to-one: use the final state
```

16.10 Wireless experiment: predicting a fading channel

16.10.1 The channel model

A narrowband mobile channel with many scatterers around a moving receiver is a complex Gaussian process $h[t]$ (Rayleigh fading). Clarke's model says that if waves arrive uniformly from all directions, the normalised autocorrelation is

$$R_h[k] = \mathbb{E}\big[h[t+k]h^*[t]\big] = J_0(2\pi f_D T_s k), \tag{16.23}$$

where J_0 is the zeroth-order Bessel function of the first kind, $f_D = v f_c / c$ the maximum Doppler shift and T_s the sampling interval. The product $f_D T_s$ (normalised Doppler) is the only parameter that matters: larger means faster fading. We simulate it with the sum-of-sinusoids method (Jakes' idea, in the randomised form of Zheng and Xiao):

$$h[t] = \frac{1}{\sqrt{M}} \sum_{m=1}^{M} \Big[\cos\big(2\pi f_D T_s t \cos\alpha_m + \phi_m\big) + j\cos\big(2\pi f_D T_s t \sin\alpha_m + \psi_m\big)\Big], \tag{16.24}$$

with $M = 16$ paths, $\alpha_m = (2\pi m - \pi + \theta)/(4M)$ and random phases θ, ϕ_m, ψ_m. Each sinusoid is one Doppler-shifted path; their sum is approximately Gaussian, and its autocorrelation matches (16.23) (Fig. 16.10).

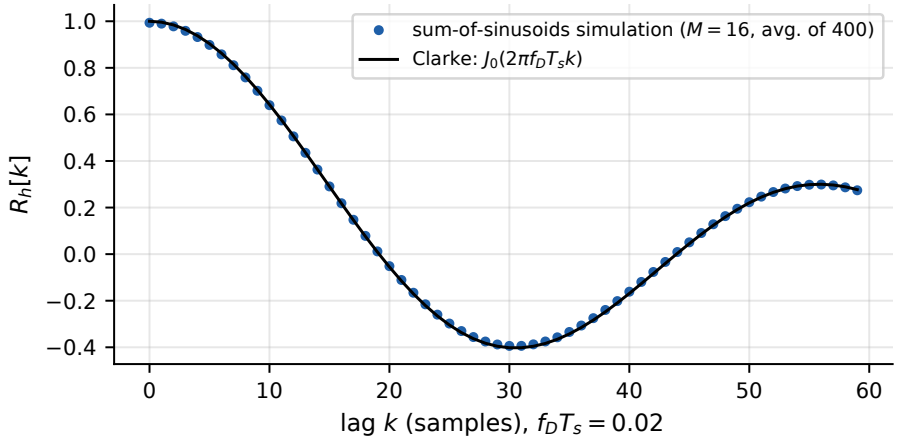


Figure 16.10: Check of the channel simulator: empirical autocorrelation of the sum-of-sinusoids generator (dots) against Clarke's J_0 curve (line) at $f_D T_s = 0.02$. The first zero near lag 19 is the coherence time; beyond it the channel is essentially unpredictable by simple extrapolation.

16.10.2 The prediction problem

From the last $P = 16$ *noisy* observations $y[t-P], \dots, y[t-1]$ with $y = h + n$, $n \sim \mathcal{CN}(0, \sigma^2)$ at 25 dB SNR, predict the clean future $h[t], \dots, h[t+K-1]$ for $K = 8$. Each realisation draws $f_D T_s$ uniformly in $[0.01, 0.04]$, so the predictor must cope with a range of speeds. We train on 400 realisations of 200 samples (12,000 windows) and test on 150 fresh realisations (4,500 windows). Metric: normalised MSE per horizon k,

$$\mathrm{NMSE}(k) = \frac{\mathbb{E}|\hat{h}[t+k-1] - h[t+k-1]|^2}{\mathbb{E}|h|^2}.$$

Three predictors.

1. *Hold*: $\hat{h}[t+k-1] = y[t-1]$ (assume the channel is frozen — what happens when CSI is simply outdated).
2. *Linear Wiener / AR predictor*: $\hat{h}[t+k-1] = \sum_{p=1}^{P} w_{k,p}\, y[t-p]$ with complex weights, one set per horizon, fitted by least squares on the training windows. This is the classical linear MMSE predictor (equivalent to the Wiener solution $\mathbf{w}_k = \mathbf{R}_{yy}^{-1}\mathbf{r}_{hy,k}$ estimated from data), averaged over the Doppler range.
3. *LSTM*: a 2-layer LSTM (hidden 48) reads the 16 past samples as a sequence of $(\Re y, \Im y)$ pairs; a linear head on the last state outputs all $2K$ real numbers at once. Trained with MSE, Adam, clipping.

```
def to_t(X): return torch.tensor(np.stack([X.real, X.imag], -1), dtype=torch.
    float32)   # (N, P, 2)
class Predictor(nn.Module):
    def __init__(self, H=48):
        super().__init__()
        self.lstm = nn.LSTM(2, H, num_layers=2, batch_first=True)
        self.head = nn.Linear(H, 2*K)
    def forward(self, x):
        o, _ = self.lstm(x); return self.head(o[:, -1]).view(-1, K, 2)
```

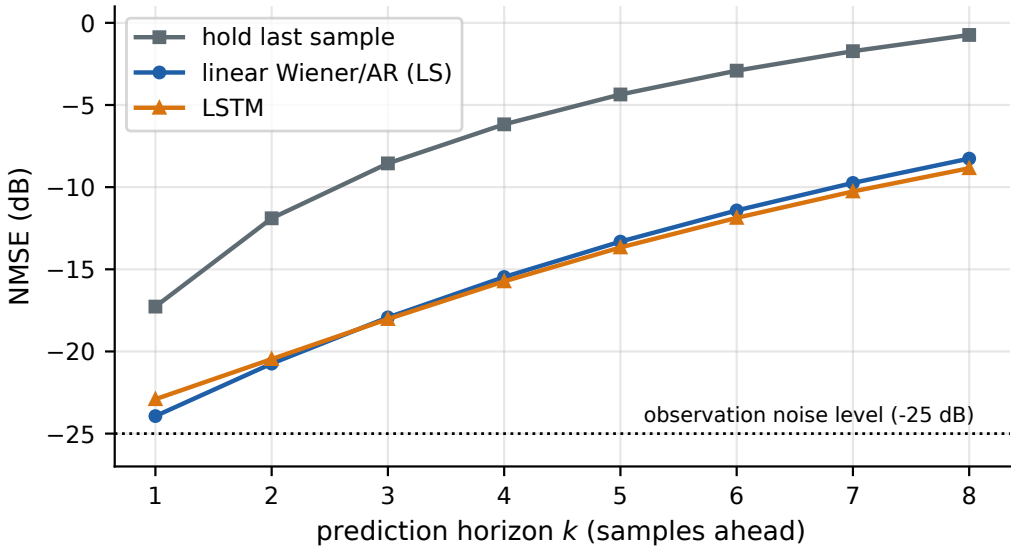


Figure 16.11: NMSE versus prediction horizon (our small-scale experiment). Holding the last sample is useless beyond 2–3 samples. The linear predictor and the LSTM are close: the linear one is about 1 dB better one step ahead, the LSTM about 0.5 dB better at the longest horizons. Script: `code/ch16/channel_prediction.py`.

16.10.3 Interpreting the result

The honest conclusion from Fig. 16.11 is that *for this channel model the LSTM roughly matches the linear predictor, it does not crush it*. That is exactly what theory predicts. The simulated channel is a Gaussian process; for Gaussian processes the MMSE predictor *is linear*, so a well-trained linear predictor is already near-optimal for any *single* Doppler value. The LSTM's small edge at long horizons (and, in our run, at low Doppler: -11.0 dB versus -9.2 dB at $k = 8$) comes from adapting to the speed: the linear predictor uses one set of weights averaged over all $f_D T_s$, while

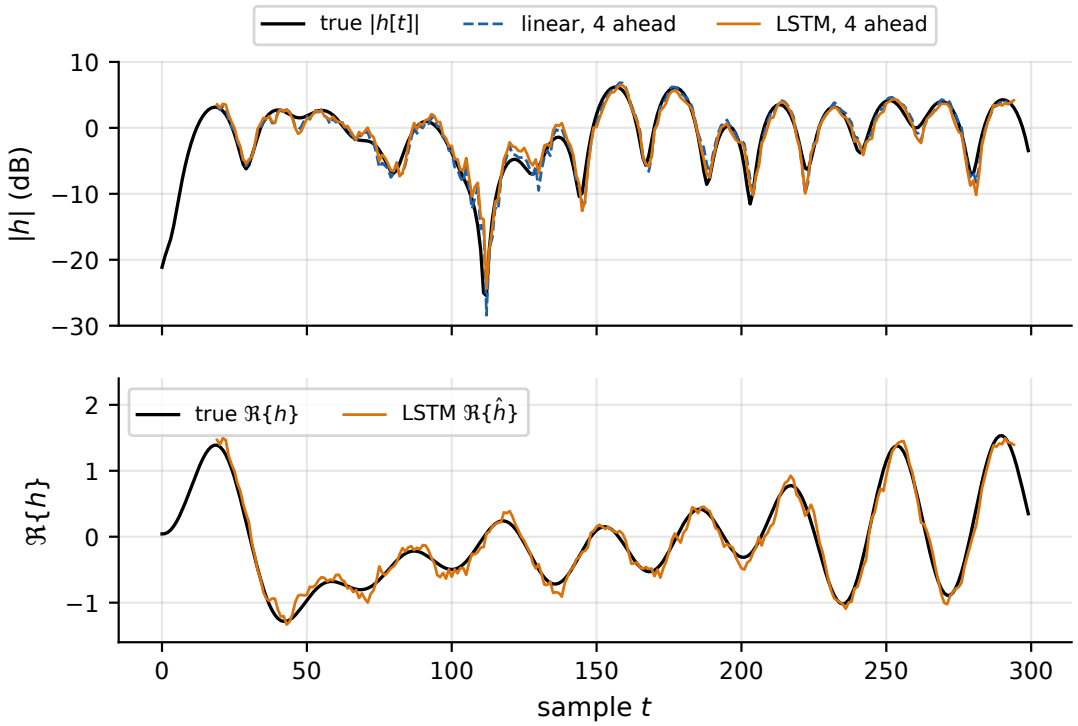


Figure 16.12: Four-step-ahead prediction on a fresh realisation ($f_D T_s = 0.03$). Top: envelope in dB; both predictors track the fades but overshoot in the deepest notches. Bottom: real part of the true channel and the LSTM prediction.

the LSTM can implicitly estimate the Doppler from the input and adjust. Its small deficit one step ahead reflects finite training and capacity.

Wireless connection: when learned predictors are worth it

Deep predictors earn their keep when the channel departs from the stationary Gaussian picture: non-isotropic scattering, a few dominant paths whose Doppler drifts, mixed speeds, measured (non-synthetic) CSI, or when many antennas/subcarriers can be predicted jointly. Even then, the linear Wiener/AR predictor is the baseline every paper must beat. Chapter 27 studies a research paper on CSI prediction with recurrent networks; keep this experiment's lesson in mind when you read its results.

Wireless connection: outdated CSI in 5G

In a TDD massive-MIMO cell the base station measures uplink SRS and uses it to precode the downlink a few slots later. At 30 km/h and 3.5 GHz, $f_D \approx 97$ Hz; with a 5 ms SRS period $f_D T_s \approx 0.49$ — far beyond the coherence time, so the precoder works with stale CSI. Prediction (classical Kalman/AR or learned) is how systems claw back the lost beamforming gain; 3GPP Release 18 studied AI/ML-based CSI prediction as one of its physical-layer use cases.

Chapter recap

- An RNN updates a hidden state $\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b})$ with the same weights at every step; unrolled, it is a deep network with shared layers. Worked

example: $h_1 = 0.462$, $h_2 = 0.701$.

- BPTT = backprop on the unrolled graph; gradients of shared weights are *summed over time*: $\partial L/\partial \mathbf{W}_{hh} = \sum_t \delta_t \mathbf{h}_{t-1}^{\mathsf{T}}$.
- The backward signal is multiplied by one Jacobian $\mathrm{diag}(1-\mathbf{h}^2)\mathbf{W}_{hh}$ per step $\Rightarrow$ it scales like ρ^k: vanishing ($\rho < 1$) or exploding ($\rho > 1$). Clipping fixes exploding; truncated BPTT bounds cost.
- LSTM: gated, additive cell state; $\partial \mathbf{c}_t/\partial \mathbf{c}_{t-1} = \mathrm{diag}(\mathbf{f}_t)$ is chosen by the data, so memory and gradients can persist. Params $4(h(h+d)+h)$ (2480 for d=10, h=20; PyTorch 2560). Initialise the forget bias high.
- GRU: two gates, interpolates old and candidate state; $\approx 25\%$ cheaper. Bidirectional = non-causal; stacked = deeper.
- Experiments: at $T = 60$ only gated RNNs learned the memory task. CLDNN = conv + LSTM + dense (Chapter 22).
- Fading-channel prediction: LSTM $\approx$ linear Wiener predictor on Gaussian Clarke channels (as theory predicts), with a small edge from Doppler adaptation; both far better than holding stale CSI.

Exercises

1. For the scalar RNN with $W_{xh} = 0.5$, $W_{hh} = 0.8$, $h_0 = 0$, compute h_3 for $x = [1, 1, 0]$. Then compute h_3 for $x = [0, 0, 0]$ with $h_0 = 0.9$. Which property of the RNN does the second case illustrate?
2. Show, step by step, that with no tanh and $L = h_T$ the scalar RNN $h_t = wh_{t-1} + ux_t$ gives $\partial L/\partial x_{T-k} = u\, w^k$. For $w = 0.9$, how many steps until the gradient is below 1% of its value at lag 0?
3. A vector gradient $\mathbf{g} = (6, 8)$ is clipped with threshold $c = 5$. What is the clipped vector? What if $c = 20$?
4. Compute the number of parameters of (a) `nn.LSTM(64,128)` and (b) `nn.GRU(64,128)` in PyTorch (two bias vectors per gate). (c) What would a single-bias implementation of (a) have?
5. In the LSTM, suppose $f_t = 0.98$ at every step on the direct path. By what factor is $\partial c_T/\partial c_{T-100}$ multiplied? Compare with a vanilla RNN whose per-step gain is 0.9.
6. Explain why a bidirectional LSTM is appropriate for classifying a recorded 1-ms burst but not for predicting the channel one slot ahead.
7. (Wireless) Using (16.23), the first zero of $J_0(x)$ is at $x \approx 2.405$. At $f_D T_s = 0.02$, at which lag does the autocorrelation first reach zero? Relate this to Fig. 16.10.

Answers

1. $h_1 = 0.462$, $h_2 = 0.701$, $a_3 = 0.5 \cdot 0 + 0.8 \cdot 0.701 = 0.561$, $h_3 = \tanh(0.561) = 0.509$. Second case: $h_1 = \tanh(0.72) = 0.617$, $h_2 = \tanh(0.493) = 0.457$, $h_3 = \tanh(0.365) = 0.350$. With no input the state decays towards zero: the RNN's memory fades (here

roughly by 0.75 per step).

2. $h_T = wh_{T-1} + ux_T = w(wh_{T-2} + ux_{T-1}) + ux_T = \cdots = \sum_{k\geq 0} w^k u\, x_{T-k} + w^T h_0$. Differentiating w.r.t. x_{T-k} leaves uw^k. $0.9^k < 0.01 \Leftrightarrow k > \ln 0.01 / \ln 0.9 = 43.7$, so 44 steps.
3. $\|\mathbf{g}\| = 10 > 5$: $\mathbf{g} \leftarrow (6,8)\cdot 5/10 = (3,4)$. With $c = 20$ nothing changes.
4. (a) $4(128\cdot 64 + 128\cdot 128 + 2\cdot 128) = 4(8192 + 16384 + 256) = 4\cdot 24832 = 99{,}328$. (b) $3\cdot 24832 = 74{,}496$. (c) $99{,}328 - 4\cdot 128 = 98{,}816$.
5. $0.98^{100} = e^{100\ln 0.98} = e^{-2.02} \approx 0.133$: still 13% of the gradient. $0.9^{100} \approx 2.7\times 10^{-5}$.
6. A recorded burst is available in full before classification, so using future samples is allowed and helps. Channel prediction must be causal: at decision time the future samples do not exist, so the backward chain has nothing to read.
7. $2\pi\cdot 0.02\cdot k = 2.405 \Rightarrow k = 2.405/(0.1257) \approx 19.1$. The figure's curve crosses zero just after lag 19: beyond about 19 samples the channel is uncorrelated with its present value.

Self-test quiz

1. Why is the gradient of $\mathbf{W}_{hh}$ a sum over time steps?
   Because the same matrix is used at every step; a shared weight's gradient is the sum of the gradients of all its copies.
2. What quantity of $\mathbf{W}_{hh}$ decides whether gradients vanish or explode in a linear RNN?
   Its spectral radius ρ (largest eigenvalue magnitude): the gradient scales like ρ^k.
3. What does gradient clipping fix, and what does it not fix?
   It limits exploding gradients; it does nothing for vanishing gradients.
4. In an LSTM, what is $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1}$ along the direct path?
   $\mathrm{diag}(\mathbf{f}_t)$, *the forget gate — data-dependent and close to 1 when memory is needed.*
5. How many parameters does `nn.LSTM(10,20)` have, and why not 2480?
   2560; *PyTorch uses two bias vectors per gate, adding* $4\cdot 20 = 80$.
6. Why did the LSTM only roughly match the linear predictor on the Clarke channel?
   The channel is a Gaussian process, for which the MMSE predictor is linear; the LSTM's gain comes mainly from adapting to the Doppler.

Further reading

- J. L. Elman, "Finding structure in time," *Cognitive Science*, 14(2), 1990.
- Y. Bengio, P. Simard, P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Trans. Neural Networks*, 5(2), 1994.
- R. Pascanu, T. Mikolov, Y. Bengio, "On the difficulty of training recurrent neural networks," ICML 2013 (spectral-radius analysis, gradient clipping).
- S. Hochreiter, J. Schmidhuber, "Long short-term memory," *Neural Computation*, 9(8), 1997; F. A. Gers, J. Schmidhuber, F. Cummins, "Learning to forget: continual prediction with LSTM," *Neural Computation*, 12(10), 2000.

- K. Cho et al., “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” EMNLP 2014 (GRU, seq2seq).
- R. Jozefowicz, W. Zaremba, I. Sutskever, “An empirical exploration of recurrent network architectures,” ICML 2015 (forget-gate bias).
- T. N. Sainath, O. Vinyals, A. Senior, H. Sak, “Convolutional, long short-term memory, fully connected deep neural networks,” ICASSP 2015 (CLDNN).
- R. H. Clarke, “A statistical theory of mobile-radio reception,” *Bell Syst. Tech. J.*, 47(6), 1968; Y. R. Zheng, C. Xiao, “Simulation models with correct statistical properties for Rayleigh fading channels,” *IEEE Trans. Commun.*, 51(6), 2003.

Chapter 17

Attention and Transformers

In Chapter 16 a recurrent network read a sequence one element at a time and squeezed everything it had seen into a single hidden-state vector. That works, but it has two weaknesses: information from far back has to survive many multiplications to reach the present, and the computation cannot be parallelised across time because step t waits for step $t-1$. In 2017 Vaswani and colleagues showed that you can throw the recurrence away entirely. Their *Transformer* lets every element of a sequence look directly at every other element and decide, with a learned and differentiable "relevance score", how much of each one to take. That single mechanism — *attention* — is now the engine inside large language models, image models, speech recognisers and, increasingly, physical-layer receivers.

In this chapter you will learn:

- what queries, keys and values are, using a library-search picture, and how scaled dot-product attention turns them into an output;
- why the scores are divided by $\sqrt{d_k}$ (we derive the variance argument);
- causal masking, multi-head attention, and positional encodings (sinusoidal and rotary);
- how a complete Transformer block is built from attention, a feed-forward network, residual connections and LayerNorm;
- encoders, decoders, cross-attention, the quadratic cost, and Vision Transformers;
- how to train a tiny Transformer in PyTorch and read what its attention heads have learned;
- how the OFDM resource grid becomes a sequence of tokens for channel estimation, CSI feedback and signal classification, and where 3GPP stands on AI/ML in the air interface.

17.1 The idea of attention

17.1.1 A library search

Intuition: queries, keys and values in a library

You walk into a library with a question in your head — say, "how does an LDPC decoder work?". That question is your **query**. Every book on the shelves has a label on its spine: a few words that advertise what it is about. Those labels are the **keys**. The contents of each book — the pages you will actually read — are the **values**.

You compare your query with every key. A book labelled "Channel coding" matches well; "Gardening in the desert" matches badly. Instead of taking only the single best book, you take a little from *every* book, in proportion to how well its label matched: you read mostly the coding book, skim a signal-processing book, and practically ignore the gardening one. What you walk out with is a *weighted blend of the values*, with weights decided by *query–key similarity*.

Attention is exactly this, done with vectors: similarity is a dot product, "in proportion to how

well it matched" is a softmax, and "a little from every book" is a weighted sum.

Notice two things that make this powerful. First, the weights are not fixed: a different query produces a different blend of the same books. Second, the key (what a book *advertises*) and the value (what it *contains*) are separate. A token can advertise "I am a pilot symbol on subcarrier 12" and deliver "here is the channel estimate I measured".

17.1.2 From words to vectors

In a neural network each element of the sequence — a word, a digit, a patch of an image, one resource element of an OFDM grid — is first turned into a vector of d_{model} numbers, its *embedding*. For a discrete symbol (a digit, a word) the embedding is simply a learned lookup table: row i of a trainable matrix $\mathbf{E}$ is the vector for symbol i, which is the same as multiplying a one-hot vector by $\mathbf{E}$. For continuous inputs (I/Q samples, image patches) it is a linear layer. Stack the n vectors as the rows of a matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{model}}}$. From $\mathbf{X}$ we make three new matrices by multiplying with three learned weight matrices:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \qquad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \qquad \mathbf{V} = \mathbf{X}\mathbf{W}_V, \tag{17.1}$$

with $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. Row i of $\mathbf{Q}$ is the query of token i ("what am I looking for?"), row j of $\mathbf{K}$ is the key of token j ("what do I advertise?"), and row j of $\mathbf{V}$ is its value ("what do I hand over if chosen?"). When the queries, keys and values all come from the same sequence $\mathbf{X}$ we call it **self-attention**. The network learns $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ by backpropagation, so it learns *what to look for* and *what to advertise*.

17.2 Scaled dot-product attention

Definition: scaled dot-product attention

Given queries $\mathbf{Q} \in \mathbb{R}^{n_q \times d_k}$, keys $\mathbf{K} \in \mathbb{R}^{n_k \times d_k}$ and values $\mathbf{V} \in \mathbb{R}^{n_k \times d_v}$,

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\mathbf{V}, \tag{17.2}$$

where the softmax is applied separately to each row. The $n_q \times n_k$ matrix $\mathbf{A} = \text{softmax}(\mathbf{Q}\mathbf{K}^{\mathsf{T}}/\sqrt{d_k})$ is the **attention matrix**: entry A_{ij} is how much query i attends to key j; every row is non-negative and sums to one.

Read (17.2) from the inside out:

1. $\mathbf{S} = \mathbf{Q}\mathbf{K}^{\mathsf{T}}$ has entries $S_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j = \sum_{m=1}^{d_k} q_{im} k_{jm}$: the dot product (Chapter 2) measures how aligned query i is with key j.
2. Divide by $\sqrt{d_k}$ to keep the scores in a sensible range (Section 17.2.2).
3. Softmax along each row: $A_{ij} = e^{S_{ij}/\sqrt{d_k}} / \sum_l e^{S_{il}/\sqrt{d_k}}$ turns scores into weights that are positive and sum to one.
4. Multiply by $\mathbf{V}$: output row i is $\mathbf{o}_i = \sum_j A_{ij}\mathbf{v}_j$, a weighted average of the value vectors.

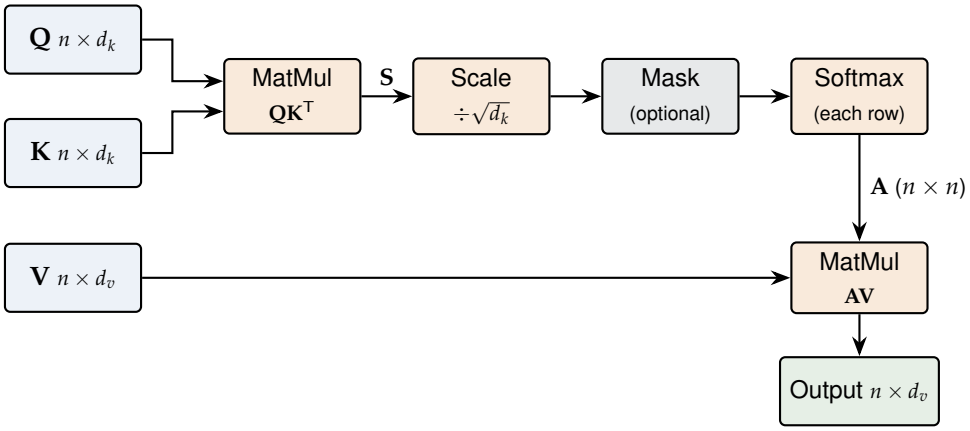


Figure 17.1: The computational graph of scaled dot-product attention, Eq. (17.2). Every box is differentiable, so gradients flow back into $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ and from there into $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$. The optional mask is used for causal (decoder) attention, Section 17.2.3. Note the $n \times n$ attention matrix in the middle: it is the source of the quadratic cost discussed in Section 17.7.

Worked example: attention by hand

Take $n = 3$ tokens with $d_k = d_v = 2$ and, to keep things simple, $\mathbf{K} = \mathbf{Q}$:

$$\mathbf{Q} = \mathbf{K} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{V} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}.$$

Step 1: scores. $\mathbf{S} = \mathbf{QK}^\mathsf{T}$. Row 1 is query $\mathbf{q}_1 = (1, 0)$ dotted with each key: $(1,0)\cdot(1,0) = 1$, $(1,0)\cdot(0,1) = 0$, $(1,0)\cdot(1,1) = 1$. Doing all rows,

$$\mathbf{S} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

Step 2: scale. $\sqrt{d_k} = \sqrt{2} \approx 1.4142$, so row 1 becomes $(0.7071,\ 0,\ 0.7071)$.
Step 3: softmax of row 1. $e^{0.7071} = 2.0281$, $e^0 = 1$, $e^{0.7071} = 2.0281$; the sum is 5.0562. Dividing,

$$\mathbf{a}_1 = \left(\tfrac{2.0281}{5.0562},\ \tfrac{1}{5.0562},\ \tfrac{2.0281}{5.0562}\right) = (0.401,\ 0.198,\ 0.401).$$

Token 1 attends equally to itself and to token 3 (both keys contain its "direction" $(1,0)$) and half as much to token 2.
Step 4: weighted values.

$$\begin{aligned} \mathbf{o}_1 &= 0.401\,(1,2) + 0.198\,(3,4) + 0.401\,(5,6) \\ &= (0.401 + 0.593 + 2.006,\ \ 0.802 + 0.791 + 2.407) = (3.00,\ 4.00). \end{aligned}$$

Repeating for the other rows gives weights $(0.198, 0.401, 0.401)$ and $(0.248, 0.248, 0.503)$ and outputs $(3.41, 4.41)$ and $(3.51, 4.51)$. Token 3, whose query $(1,1)$ aligns best with its own key $(1,1)$ (score 2), puts half its attention on itself. All numbers were checked with the script `code/ch17/attention_basics.py`.

17.2.1 Attention in a dozen lines of code

The function below implements (17.2) including the optional causal mask. It works for any number of leading "batch" dimensions because `@` and `transpose(-2,-1)` act on the last two axes.

```
def attention(Q, K, V, causal=False):
    d_k = Q.shape[-1]
    scores = Q @ K.transpose(-2, -1) / d_k ** 0.5          # (..., n_q, n_k)
    if causal:
        n = scores.shape[-1]
        mask = torch.triu(torch.ones(n, n, dtype=torch.bool), diagonal=1)
        scores = scores.masked_fill(mask, float('-inf'))    # forbid looking ahead
    weights = torch.softmax(scores, dim=-1)                 # each row sums to 1
    return weights @ V, weights
```

PyTorch ships a fused, memory-efficient version,

`torch.nn.functional.scaled_dot_product_attention`.

Comparing ours with it on random tensors of shape $4 \times 8 \times 10 \times 16$ (batch, heads, tokens, d_k) gives a maximum absolute difference of 3.6×10^{-7} without the mask and 2.4×10^{-7} with it — floating-point round-off, so our understanding is correct.

17.2.2 Why divide by $\sqrt{d_k}$?

Suppose, as is roughly true at initialisation, that the components of a query $\mathbf{q}$ and a key $\mathbf{k}$ are independent random numbers with mean 0 and variance 1. The score is

$$s = \mathbf{q} \cdot \mathbf{k} = \sum_{m=1}^{d_k} q_m k_m .$$

Mean of one term. By independence, $\mathbb{E}[q_m k_m] = \mathbb{E}[q_m]\,\mathbb{E}[k_m] = 0 \cdot 0 = 0$.

Variance of one term. $\operatorname{Var}(q_m k_m) = \mathbb{E}[(q_m k_m)^2] - (\mathbb{E}[q_m k_m])^2 = \mathbb{E}[q_m^2]\,\mathbb{E}[k_m^2] - 0$. Since the mean is zero, $\mathbb{E}[q_m^2] = \operatorname{Var}(q_m) = 1$, and likewise for k_m. So $\operatorname{Var}(q_m k_m) = 1$.

Variance of the sum. The d_k terms are independent, and variances of independent variables add (Chapter 4):

$$\operatorname{Var}(s) = \sum_{m=1}^{d_k} \operatorname{Var}(q_m k_m) = d_k, \qquad \operatorname{std}(s) = \sqrt{d_k}.$$

The fix. Dividing by $\sqrt{d_k}$ gives $\operatorname{Var}(s/\sqrt{d_k}) = d_k/d_k = 1$ (because $\operatorname{Var}(cX) = c^2 \operatorname{Var}(X)$), *whatever* the head dimension.

Key equation: the scaling rule

$$\operatorname{Var}(\mathbf{q} \cdot \mathbf{k}) = d_k \quad \Longrightarrow \quad \operatorname{Var}\left(\frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}}\right) = 1.$$

Why does it matter? With $d_k = 512$ the raw scores have a standard deviation of about 22.6. Feeding numbers like $+40$ and -30 into a softmax gives a weight of essentially 1 on the largest and 0 elsewhere: the softmax *saturates*. Its gradient (Chapter 6) is $\partial a_i / \partial s_j = a_i(\delta_{ij} - a_j)$, which is nearly zero when all a's are 0 or 1, so learning stalls. Figure 17.2 shows both effects measured by simulation.

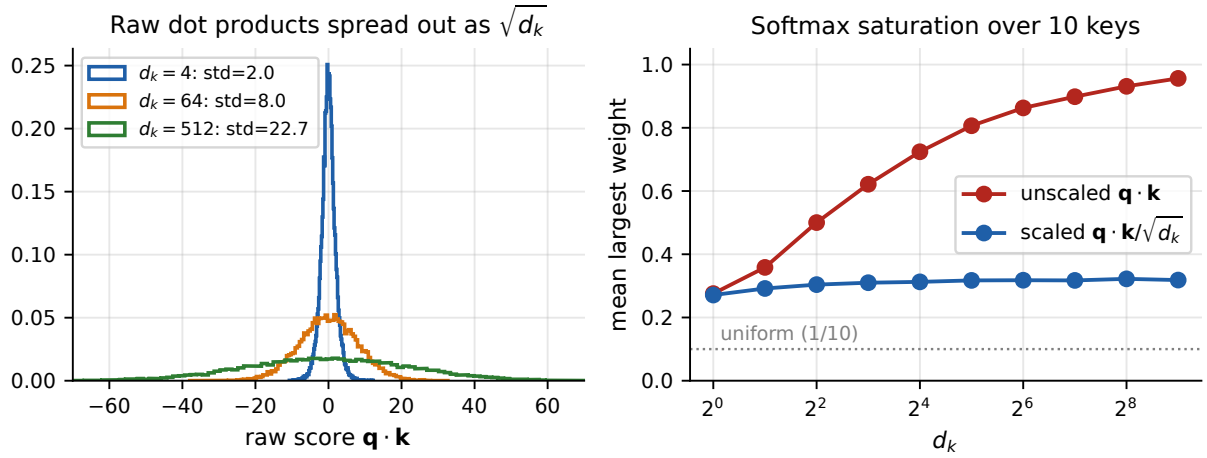


Figure 17.2: Left: histograms of raw dot products $\mathbf{q} \cdot \mathbf{k}$ of random unit-variance vectors. The measured standard deviation is 2.0, 8.0 and 22.7 for $d_k = 4, 64, 512$ — exactly $\sqrt{d_k}$. Right: the average largest softmax weight over 10 keys. Without scaling (red) the softmax approaches a hard "winner takes all" (weight $\to 1$) as d_k grows, which kills its gradients; with scaling (blue) the attention stays equally soft for every d_k.

17.2.3 Causal masking

When a model *generates* a sequence one token at a time (a language model writing text, or a decoder producing bits), token i must not see tokens $i+1, i+2, \ldots$ — they do not exist yet at generation time, and letting the model peek during training would teach it to cheat. We enforce this by setting the scores above the diagonal to $-\infty$ *before* the softmax. Because $e^{-\infty} = 0$, those keys get exactly zero weight and the remaining weights still sum to one:

$$S_{ij} \leftarrow \begin{cases} S_{ij} & j \le i \\ -\infty & j > i \end{cases} \qquad \Longrightarrow \qquad \mathbf{A} = \begin{bmatrix} * & 0 & 0 & \cdots \\ * & * & 0 & \cdots \\ * & * & * & \cdots \\ \vdots & & & \ddots \end{bmatrix}.$$

Worked example: the worked example with a causal mask

Row 1 may only see key 1, so $\mathbf{a}_1 = (1,0,0)$ and $\mathbf{o}_1 = \mathbf{v}_1 = (1,2)$. Row 2 sees keys 1 and 2 with scaled scores $(0, 0.7071)$: $e^0 = 1$, $e^{0.7071} = 2.028$, so $\mathbf{a}_2 = (0.330, 0.670, 0)$ and $\mathbf{o}_2 = 0.330(1,2) + 0.670(3,4) = (2.34, 3.34)$. Row 3 sees everything and is unchanged: $(3.51, 4.51)$.

17.3 Multi-head attention

Intuition: several readers at once

One attention operation produces one weighted average per token, so it can follow only one "kind of relationship" at a time. In a sentence you may want to know simultaneously *which noun this verb belongs to*, *which word came just before*, and *whether we are inside a quotation*. Multi-head attention sends several independent readers into the library, each with its own way of phrasing queries and reading labels, and then staples their notes together.

Definition: multi-head attention

With h heads, each head $i = 1, \dots, h$ has its own projections $\mathbf{W}_Q^{(i)}, \mathbf{W}_K^{(i)} \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $\mathbf{W}_V^{(i)} \in \mathbb{R}^{d_{\text{model}} \times d_v}$:

$$\text{head}_i = \text{Attention}(\mathbf{X}\mathbf{W}_Q^{(i)},\ \mathbf{X}\mathbf{W}_K^{(i)},\ \mathbf{X}\mathbf{W}_V^{(i)}),$$

$$\text{MHA}(\mathbf{X}) = \left[\text{head}_1\ \text{head}_2\ \cdots\ \text{head}_h\right]\mathbf{W}_O, \qquad \mathbf{W}_O \in \mathbb{R}^{hd_v \times d_{\text{model}}}. \tag{17.3}$$

The brackets mean concatenation side by side, giving an $n \times hd_v$ matrix, which $\mathbf{W}_O$ maps back to width d_{model}.

The usual choice is $d_k = d_v = d_{\text{model}}/h$. In the original Transformer of Vaswani et al. (2017), $d_{\text{model}} = 512$, $h = 8$ and therefore $d_k = d_v = 64$. With this choice the total cost is about the same as one big head of width 512: we have not added computation, we have *split* it into eight independent subspaces. Counting weights (ignoring biases): $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ for all heads together are three 512×512 matrices and $\mathbf{W}_O$ is a fourth, so one multi-head attention layer has $4 \times 512^2 = 1{,}048{,}576$ weights, independent of the sequence length n.

In code, all heads are computed at once: one big $d_{\text{model}} \times 3d_{\text{model}}$ matrix produces $\mathbf{Q}, \mathbf{K}, \mathbf{V}$, which are then *reshaped* from (B, n, d_{model}) to (B, h, n, d_k) so that the head index behaves like a batch index (lines 17–23 of the listing in Section 17.9).

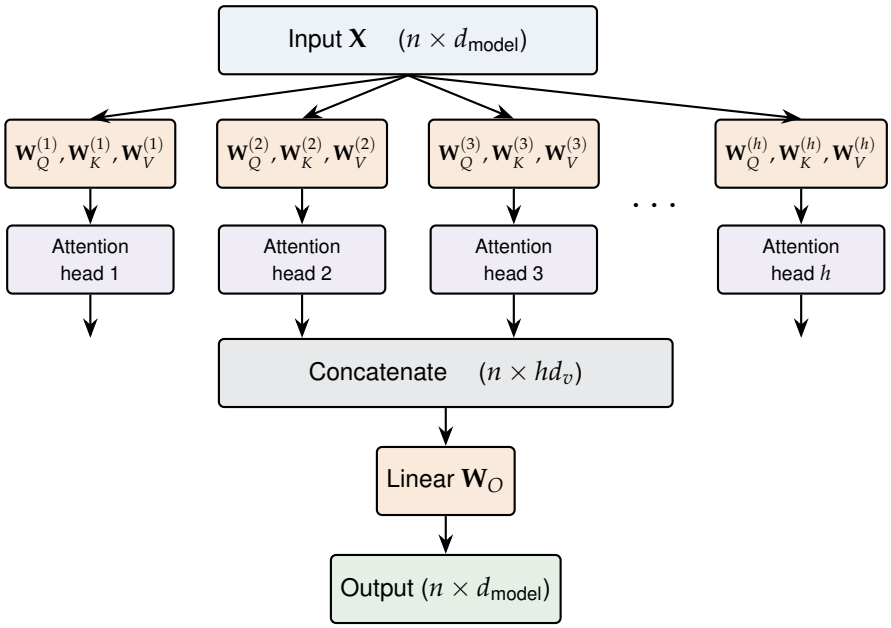


Figure 17.3: Multi-head attention, Eq. (17.3). Each head projects the same input into its own small subspace ($d_k = d_{\text{model}}/h$), runs scaled dot-product attention there, and the h results are concatenated and mixed by $\mathbf{W}_O$. Different heads are free to learn different relationships; we will see this happen in Figure 17.10.

17.4 Where am I? Positional encodings

Look again at (17.2). If you shuffle the rows of $\mathbf{X}$, the rows of $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ shuffle the same way, the attention matrix is permuted on both sides, and the output rows come out shuffled in exactly the same order. Attention is **permutation-equivariant**: it treats the input as a *set*, not a sequence. "Dog bites man" and "man bites dog" would look identical. We must inject the position of each

token.

17.4.1 Sinusoidal encodings

Vaswani et al. add to each token embedding a fixed vector $\mathbf{pe}(p) \in \mathbb{R}^{d_{\text{model}}}$ that depends only on its position $p = 0, 1, 2, \ldots$:

$$\text{PE}(p, 2i) = \sin\Big(\frac{p}{10000^{2i/d_{\text{model}}}}\Big), \qquad \text{PE}(p, 2i+1) = \cos\Big(\frac{p}{10000^{2i/d_{\text{model}}}}\Big), \tag{17.4}$$

for $i = 0, 1, \ldots, d_{\text{model}}/2 - 1$. Write $\omega_i = 10000^{-2i/d_{\text{model}}}$: dimension pair i is a sine/cosine pair rotating at angular frequency ω_i radians per position. Pair 0 has $\omega_0 = 1$ (period $2\pi \approx 6.3$ positions); the last pair has $\omega \approx 1/10000$ (period about 63,000 positions). It is like the hands of a clock — seconds, minutes, hours — or, for a radio engineer, a bank of oscillators at geometrically spaced frequencies: the fast ones resolve nearby positions, the slow ones tell far-apart positions apart.

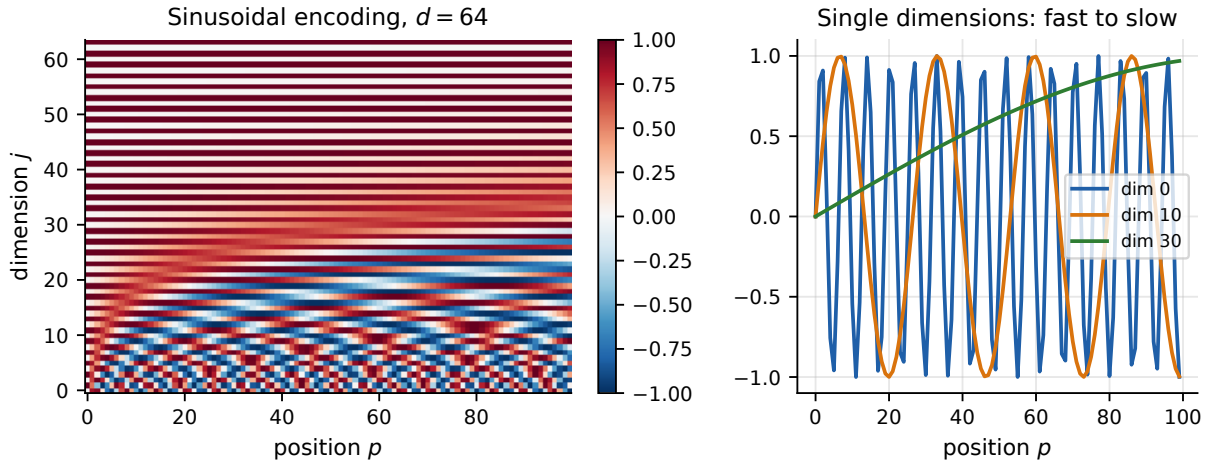


Figure 17.4: Sinusoidal positional encodings, Eq. (17.4), for 100 positions and $d_{\text{model}} = 64$. Left: each column is the vector added to the token at that position; low dimensions (bottom) oscillate fast, high dimensions slowly, so every column is a unique "barcode". Right: three individual dimensions — dimension 0 has period 2π, dimension 10 a period of about 26, dimension 30 barely starts its first cycle in 100 positions.

Why sines and cosines? Because a shift in position is a *rotation*. Using $\sin(a+b) = \sin a \cos b + \cos a \sin b$ and $\cos(a+b) = \cos a \cos b - \sin a \sin b$ with $a = \omega_i p$, $b = \omega_i k$:

$$\begin{bmatrix} \sin \omega_i (p+k) \\ \cos \omega_i (p+k) \end{bmatrix} = \begin{bmatrix} \cos \omega_i k & \sin \omega_i k \\ -\sin \omega_i k & \cos \omega_i k \end{bmatrix} \begin{bmatrix} \sin \omega_i p \\ \cos \omega_i p \end{bmatrix}.$$

The matrix depends only on the offset k, not on p, so "k positions later" is the same linear map everywhere — easy for a network to learn. Moreover the dot product of two encodings depends only on their distance:

$$\mathbf{pe}(p) \cdot \mathbf{pe}(p+k) = \sum_i \big[\sin \omega_i p \, \sin \omega_i (p+k) + \cos \omega_i p \, \cos \omega_i (p+k)\big] = \sum_i \cos(\omega_i k),$$

using $\cos(A - B) = \cos A \cos B + \sin A \sin B$. With $d_{\text{model}} = 64$ this equals 32 at $k = 0$ and falls to 30.9, 23.5 and 18.9 at $k = 1, 5, 20$ (checked in the script): nearby positions look similar, distant ones less so.

Many modern models simply *learn* a position vector per position (a trainable matrix, as in our lab below); this works well but cannot extrapolate beyond the longest training sequence.

17.4.2 Rotary position embedding (RoPE): the intuition

Rather than *adding* position to the token, RoPE (Su et al., 2021, used in many current language models) *rotates* each query and key. Split $\mathbf{q}$ into pairs of coordinates; treat each pair as a point in a plane (or a complex number, as you would an I/Q sample); rotate the pair at position m by angle $m\theta_i$. Do the same to keys at position n. For one pair, with $\mathbf{R}(\phi)$ the 2×2 rotation matrix,

$$\big(\mathbf{R}(m\theta)\mathbf{q}\big) \cdot \big(\mathbf{R}(n\theta)\mathbf{k}\big) = \mathbf{q}^{\mathsf{T}}\mathbf{R}(m\theta)^{\mathsf{T}}\mathbf{R}(n\theta)\mathbf{k} = \mathbf{q}^{\mathsf{T}}\mathbf{R}\big((n-m)\theta\big)\mathbf{k},$$

because $\mathbf{R}(\alpha)^{\mathsf{T}} = \mathbf{R}(-\alpha)$ and rotations compose by adding angles. The score depends on the *relative* position $n - m$ only. In complex notation this is $\mathrm{Re}\{q\, e^{jm\theta}\,(k\, e^{jn\theta})^*\} = \mathrm{Re}\{qk^* e^{-j(n-m)\theta}\}$ — a wireless engineer will recognise a phase rotation that cancels everything but the difference, just like a frequency offset seen through a correlator.

Wireless connection: position in time and frequency

On an OFDM resource grid a token has *two* positions: subcarrier index k and symbol index l. A channel $H[k, l]$ that is smooth in frequency and time is correlated with its neighbours according to the *offsets* Δk (set by the delay spread) and Δl (set by the Doppler spread) — exactly the relative-position structure that sinusoidal encodings and RoPE make easy to express. A two-dimensional encoding (half of the dimensions for k, half for l) lets attention learn "trust pilots that are close in frequency, and in time only if the user is slow".

17.5 The Transformer block

A Transformer is a stack of identical blocks. Each block has two sub-layers: multi-head self-attention, which *mixes information between tokens*, and a position-wise feed-forward network (FFN), which *processes each token on its own*. Each sub-layer is wrapped in a residual connection and a layer normalisation.

17.5.1 The ingredients

Residual connection (Chapter 15): the sub-layer computes a *correction* that is added to its input, $\mathbf{x} \leftarrow \mathbf{x} + F(\mathbf{x})$. The identity path lets gradients flow straight through dozens of blocks.

Layer normalisation (Chapter 12): for each token vector $\mathbf{x} \in \mathbb{R}^d$ separately, subtract its mean and divide by its standard deviation, then apply a learned scale γ and shift β:

$$\mu = \frac{1}{d}\sum_m x_m, \quad \sigma^2 = \frac{1}{d}\sum_m (x_m - \mu)^2, \quad \mathrm{LN}(\mathbf{x})_m = \gamma_m \frac{x_m - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_m.$$

Unlike batch normalisation, nothing depends on the other sequences in the batch or on the sequence length, which suits variable-length data. Example: $\mathbf{x} = (1, 2, 3, 6)$ has $\mu = 3$, $\sigma^2 = (4 + 1 + 0 + 9)/4 = 3.5$, so (with $\gamma = 1, \beta = 0$) $\mathrm{LN}(\mathbf{x}) = (-1.069, -0.535, 0, 1.604)$.

Feed-forward network: two linear layers with a nonlinearity, applied to each token independently with the same weights:

$$\mathrm{FFN}(\mathbf{x}) = \mathrm{GELU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2, \qquad \mathbf{W}_1 \in \mathbb{R}^{d \times 4d},\ \mathbf{W}_2 \in \mathbb{R}^{4d \times d}.$$

The hidden layer is four times wider than the model (2048 for $d_{\text{model}} = 512$ in the original paper). The FFN holds about two thirds of a block's weights: $2 \times 512 \times 2048 \approx 2.1$ million versus

1.05 million for attention. A useful mental model is that attention *gathers* information and the FFN *thinks about it*.

GELU (Gaussian error linear unit) is a smooth ReLU: $\mathrm{GELU}(x) = x\,\Phi(x)$, where Φ is the standard normal cumulative distribution. It lets a large positive input through almost unchanged ($\mathrm{GELU}(2) = 1.954$), blocks a large negative one ($\mathrm{GELU}(-2) = -0.046$), and bends smoothly in between ($\mathrm{GELU}(1) = 0.841$, $\mathrm{GELU}(-1) = -0.159$). The original Transformer used ReLU; GELU became standard with BERT and GPT.

17.5.2 Pre-norm versus post-norm

The 2017 paper placed LayerNorm *after* each residual addition ("post-norm"): $\mathbf{x} \leftarrow \mathrm{LN}(\mathbf{x} + F(\mathbf{x}))$. Most modern models put it *before* the sub-layer ("pre-norm"):

Key equation: a pre-norm Transformer block

$$\mathbf{x} \leftarrow \mathbf{x} + \mathrm{MHA}\left(\mathrm{LN}_1(\mathbf{x})\right)$$
$$\mathbf{x} \leftarrow \mathbf{x} + \mathrm{FFN}\left(\mathrm{LN}_2(\mathbf{x})\right)$$

In pre-norm the residual path is a clean highway from input to output with nothing in the way, which makes deep stacks train stably without a delicate learning-rate warm-up. A final LayerNorm is applied after the last block.

17.6 Encoders, decoders and cross-attention

Transformer blocks come in three flavours, distinguished by *who may attend to whom*.

- **Encoder** (e.g. BERT, ViT, our lab model): unmasked self-attention. Every token sees every other token. Use it when the whole input is available at once and you want a representation of it — classification, tagging, estimation.
- **Decoder-only** (e.g. GPT): causal self-attention. Token i sees only tokens $\leq i$. Trained to predict the next token, then generates by feeding each output back in.
- **Encoder–decoder** (the original translation model): the encoder reads the source sentence; each decoder block has causal self-attention over the target produced so far *plus* a **cross-attention** sub-layer in which the queries come from the decoder and the keys and values come from the encoder output: $\mathrm{Attention}(\mathbf{X}_{\mathrm{dec}}\mathbf{W}_Q,\ \mathbf{H}_{\mathrm{enc}}\mathbf{W}_K,\ \mathbf{H}_{\mathrm{enc}}\mathbf{W}_V)$. The decoder asks questions; the encoded source supplies the library.

Wireless connection: cross-attention as a receiver that consults its pilots

Think of channel estimation as an encoder–decoder problem. The "source" is the set of received pilot resource elements, each tagged with its (k, l) position; the encoder builds a representation of what the channel looks like at those pilots. The "target" queries are all the data resource elements, each carrying only its position. Cross-attention lets each data RE ask "which pilots are relevant to me?" and blend their channel estimates — a *learned, data-dependent interpolator*. Classical LMMSE interpolation does the same with fixed weights computed from assumed delay/Doppler statistics; attention computes its weights on the fly

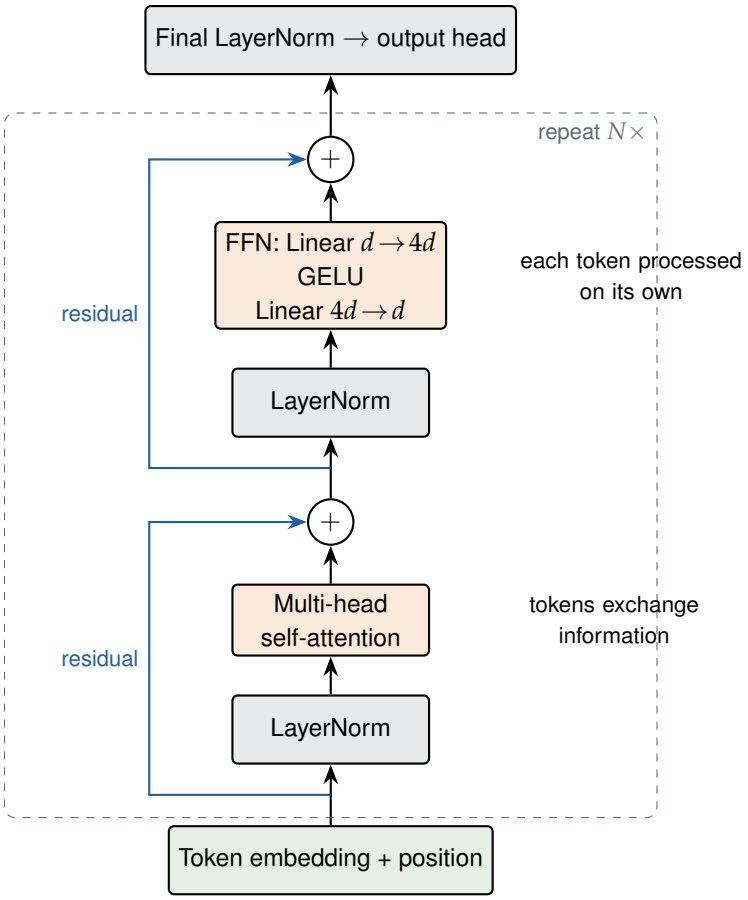


Figure 17.5: A pre-norm Transformer block. The blue residual paths carry the input straight through; each sub-layer only adds a correction computed from a normalised copy. The block is repeated N times (6 in the original encoder, 2 in our lab model, dozens in large language models), and a final LayerNorm feeds the task-specific output layer.

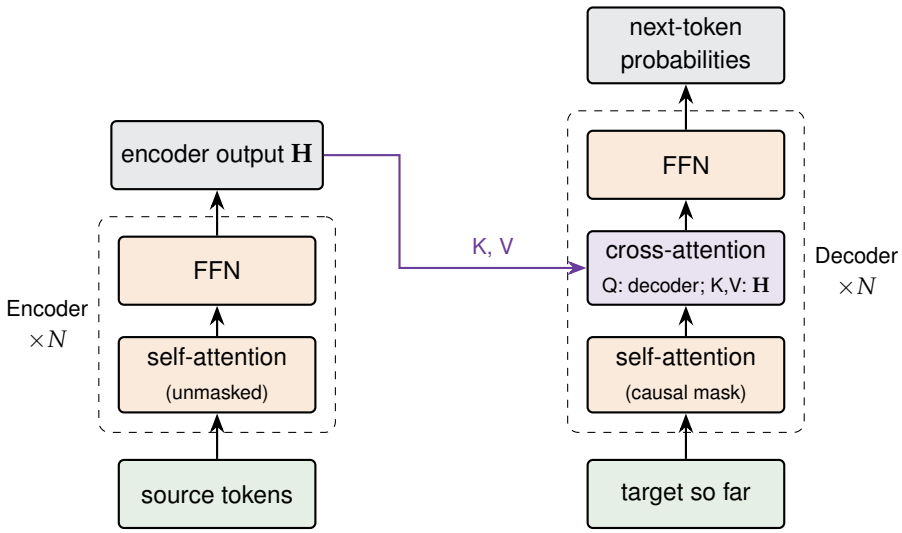


Figure 17.6: The encoder–decoder Transformer (residuals and LayerNorms omitted for clarity). The encoder sees the whole source; the decoder sees only the past of the target (causal mask) and consults the encoded source through cross-attention. Removing the left column gives a decoder-only model (GPT); removing the right column gives an encoder-only model (BERT, ViT).

from the received signal itself.

17.7 The cost of looking at everything

The attention matrix has $n \times n$ entries per head. Computing $\mathbf{QK}^{\mathsf{T}}$ takes about $n^2 d_k$ multiply–adds per head, so about $n^2 d_{\text{model}}$ over all heads, and the same again for $\mathbf{AV}$. The projections and the FFN cost about $n\, d_{\text{model}}^2$ each (linear in n). Counting floating-point operations (2 per multiply–add): attention $4n^2 d$, projections $8nd^2$, FFN $16nd^2$. Attention dominates when $4n^2 d > 24nd^2$, that is when $n > 6d$ — about 3000 tokens for $d = 512$ (Figure 17.7).

Memory is often the tighter limit. For $n = 4096$ tokens and 8 heads the attention matrices alone hold $8 \times 4096^2 \approx 134$ million numbers — 537 MB in 32-bit floats, for a single layer of a single example. Doubling n quadruples this. Fused kernels such as FlashAttention (the kind of implementation behind `scaled_dot_product_attention`) avoid ever storing $\mathbf{A}$ by computing the softmax in tiles, which removes the memory problem but not the n^2 arithmetic. Approximate "efficient attention" variants (local windows, low-rank, linear attention) trade some accuracy for linear cost.

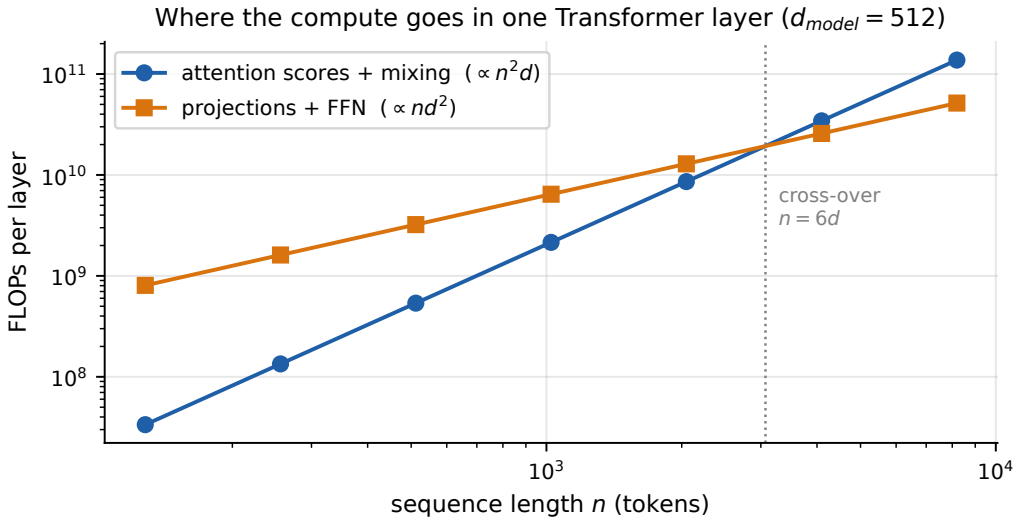


Figure 17.7: FLOPs per Transformer layer with $d_{\text{model}} = 512$. The projections and FFN grow linearly with sequence length n (slope 1 on this log–log plot); attention grows quadratically (slope 2) and takes over beyond $n = 6d_{\text{model}} \approx 3000$ tokens. This is why tokenisation — how many tokens you cut your input into — is a first-order design decision.

Pitfall: one token per resource element

A full 100 MHz NR carrier at 30 kHz subcarrier spacing has 273 resource blocks $\times$ 12 = 3276 subcarriers and 14 symbols per slot: 45,864 resource elements. One token per RE would need an attention matrix of 2.1×10^9 entries per head. Practical designs make each token a *group* of REs (one resource block per symbol gives $273 \times 14 = 3822$ tokens; one resource block across the slot gives 273), or attend along frequency and time separately (axial attention).

17.8 Vision Transformers

Dosovitskiy et al. (2021) asked: what if we treat an image as a sentence? Cut a 224×224 image into 16×16 patches — $14 \times 14 = 196$ of them — flatten each patch ($16 \cdot 16 \cdot 3 = 768$ numbers), map it linearly to d_{model}, add a learned position embedding, prepend one extra learnable "class" token, and run a standard Transformer encoder. The output vector of the class token goes to a classifier. Figure 17.8 shows the same tokenisation on one of our 8×8 digits.

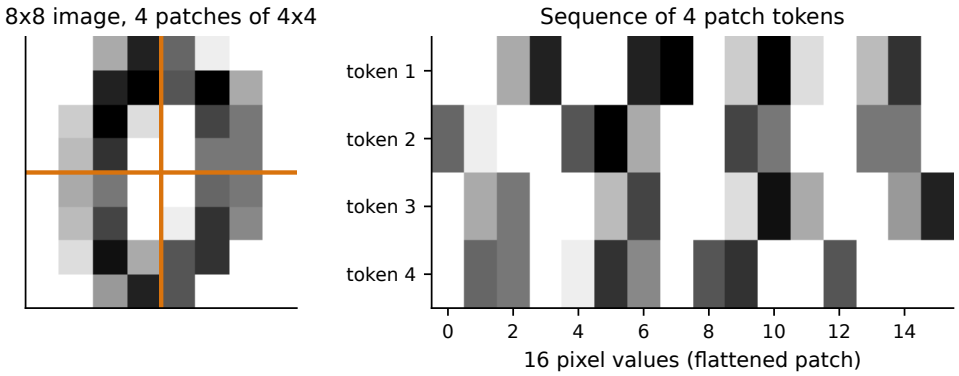


Figure 17.8: Vision-Transformer tokenisation on an 8×8 digit: four 4×4 patches, each flattened into a 16-number token. A linear layer then maps every token to d_{model} dimensions. The Transformer never sees a pixel grid — it learns spatial relationships purely through attention and position embeddings.

A ViT has much weaker built-in assumptions than a CNN (Chapter 15): no locality, no weight sharing across positions beyond the patch embedding. The paper found that, as a consequence, ViTs trained on mid-sized datasets underperform comparable CNNs but overtake them when pretrained on very large datasets. Lesson: *inductive bias substitutes for data*. This trade-off reappears constantly in wireless, where data can be cheap (simulation) but domain shift is real.

17.9 Lab: a tiny Transformer that reverses and sorts

Theory is best tested on a task where we know what the right answer is *and* what a sensible attention pattern would look like. We train an encoder on two toy problems with sequences of $n = 10$ random digits:

- **reverse**: input `3 7 1 9 4 4 0 8 2 6` → output `6 2 8 0 4 4 9 1 7 3`;
- **sort**: same input → `0 1 2 3 4 4 6 7 8 9`.

The model predicts all ten output digits in parallel (one 10-way softmax per position), trained with cross-entropy. Neither task can be solved token-by-token: position i of the output depends on other positions of the input, so information *must* travel through attention.

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.h, self.d_k = n_heads, d_model // n_heads
        self.W_qkv = nn.Linear(d_model, 3 * d_model)  # Q,K,V in one matrix
        self.W_o = nn.Linear(d_model, d_model)  # output projection
    def forward(self, x):                    # x: (B, n, d_model)
        B, n, d = x.shape
        q, k, v = self.W_qkv(x).split(d, dim=-1)
        # (B,n,d) -> (B,h,n,d_k): one d_k slice per head
```

```
        q, k, v = (t.view(B, n, self.h, self.d_k).transpose(1, 2)
                   for t in (q, k, v))
        scores = q @ k.transpose(-2, -1) / self.d_k ** 0.5  # (B,h,n,n)
        self.attn = torch.softmax(scores, dim=-1)  # keep for plots
        out = (self.attn @ v).transpose(1, 2).reshape(B, n, d)  # concat
        return self.W_o(out)
class Block(nn.Module):                    # pre-norm Transformer block
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.ln1, self.ln2 = nn.LayerNorm(d_model), nn.LayerNorm(d_model)
        self.mha = MultiHeadAttention(d_model, n_heads)
        self.ffn = nn.Sequential(nn.Linear(d_model, 4 * d_model), nn.GELU(),
                                 nn.Linear(4 * d_model, d_model))
    def forward(self, x):
        x = x + self.mha(self.ln1(x))    # residual around attention
        x = x + self.ffn(self.ln2(x))    # residual around FFN
        return x
```

The model has $d_{\text{model}} = 64$, $h = 4$ heads ($d_k = 16$), 2 pre-norm blocks, a learned position embedding and a linear output layer: 102,026 parameters in total. We train with AdamW (batch 128, one-cycle learning-rate schedule peaking at 2×10^{-3}) for 800 steps on *reverse* and 1200 on *sort*, drawing fresh random sequences every step, and validate on 1000 sequences never used for training.

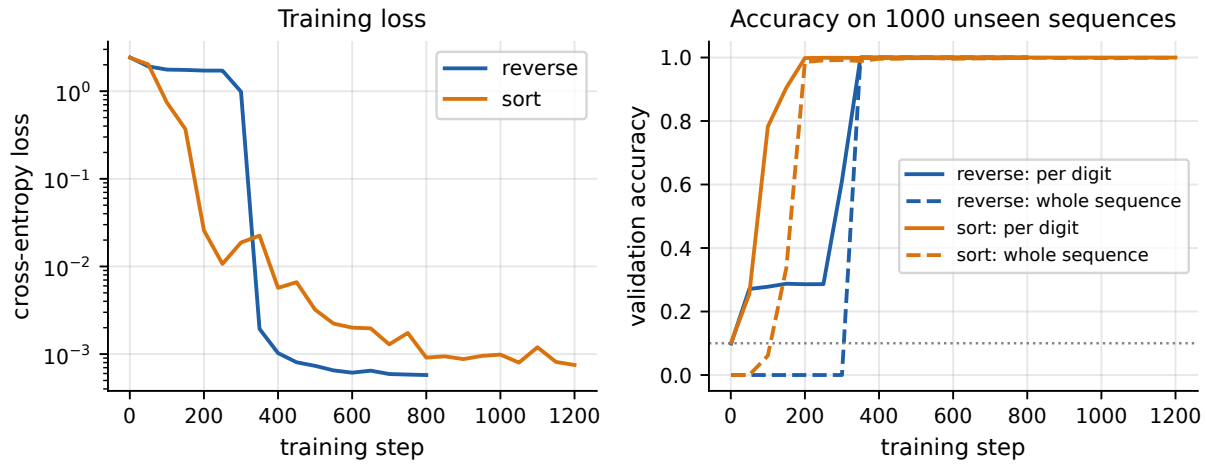


Figure 17.9: Learning curves of the tiny Transformer (our run). Left: training loss on a log scale. Right: validation accuracy per digit (solid) and for whole sequences (dashed); the dotted line is chance (0.1 per digit). *Reverse* sits on a plateau near 27% per-digit accuracy for 250 steps and then "clicks" within 100 steps to 100%; *sort* improves more gradually. Final: reverse 100% of sequences correct; sort 99.99% of digits and 99.9% of sequences.

The plateau-then-click shape of the *reverse* curve is typical of attention models: until the query and key projections line up so that position i finds position $9 - i$, the attention is nearly uniform and the best the network can do is guess common digits; once the alignment appears, it is self-reinforcing.

17.9.1 What did the heads learn?

Now the reward: we can *look inside*. Figures 17.10 and 17.11 show the attention matrices of the trained models for the input `3 7 1 9 4 4 0 8 2 6`.

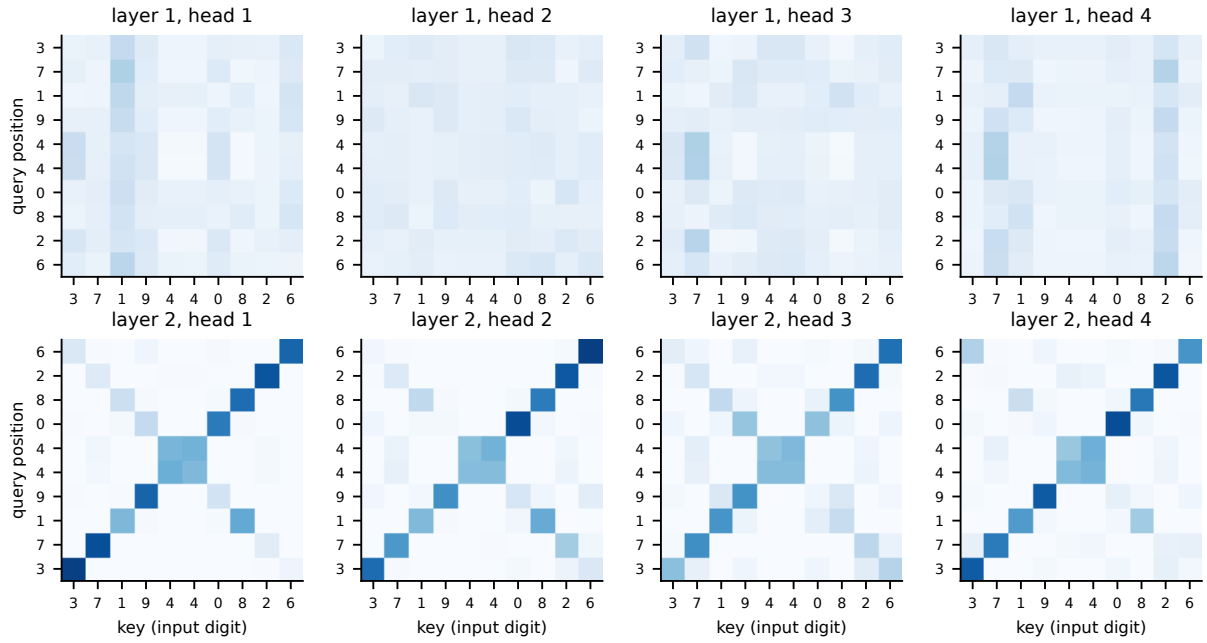


Figure 17.10: Attention weights of the trained *reverse* model (rows: query positions, labelled in layer 2 by the digit the model outputs there; columns: input digits). Layer 1 is diffuse. In layer 2 every head has learned the *anti-diagonal*: output position i reads input position $9 - i$. Note the two 4s in the middle: the model splits its attention between them, which is harmless because both carry the same digit.

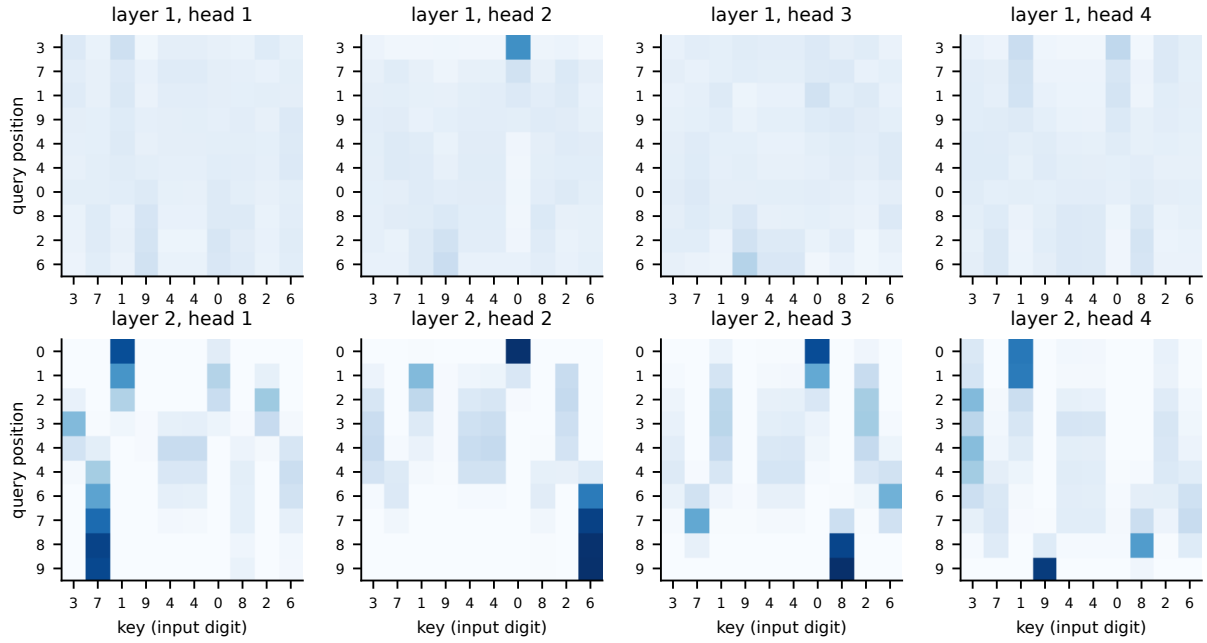


Figure 17.11: Attention weights of the trained *sort* model on the same input. In layer 2 the pattern is no longer tied to position but to *content*: the query that outputs 0 looks at the 0 (head 2) and 1 (heads 1, 4); the queries outputting 7, 8, 9 look at the 7, 8 and 9 in the input (heads 1, 3, 4), wherever they are. Different heads specialise in different parts of the digit range.

To check that these pictures are not cherry-picked, we averaged over 1000 random sequences (`code/ch17/analyse_heads.py`). For *reverse*, the fraction of attention placed on the mirror position $9 - i$ is 0.10–0.11 for every layer-1 head (exactly the uniform value $1/10$) but 0.61–0.72 for the four layer-2 heads. For *sort*, the fraction placed on input tokens whose digit equals the output digit is 0.42–0.49 in layer 2, against 0.19 if attention were uniform. So: layer 1 mostly prepares representations (position and content mixed together), and layer 2 does the routing — by *position* for reversal, by *content* for sorting. The same architecture discovered two qualitatively different algorithms.

Pitfall: attention maps are a clue, not an explanation

Attention weights tell you where information was *read from*, not what was done with it afterwards (the FFN, $\mathbf{W}_O$ and the residual stream all transform it). On a toy problem the maps line up beautifully with the algorithm; on real data treat them as hints to be confirmed by experiments such as masking a head and measuring the drop in accuracy.

17.10 Transformers in the physical layer

Wireless connection: the resource grid as a sequence of tokens

An OFDM slot is a two-dimensional grid of complex numbers $Y[k, l]$ (subcarrier k, symbol l). To feed it to a Transformer:

1. **Tokenise**: choose a unit — one RE, one resource block on one symbol, or one subcarrier across the whole slot. Stack the real and imaginary parts (and, if useful, known pilot values or a least-squares estimate) of that unit into a vector.
2. **Embed**: a linear layer maps each vector to d_{model}.
3. **Position**: add 2-D encodings of (k, l), or use RoPE on each axis.
4. **Encode**: N Transformer blocks; each RE can now borrow information from any pilot or data RE in the slot.
5. **Head**: per-token outputs (channel estimate $\hat{H}[k, l]$, or bit log-likelihood ratios) or a pooled output (a class label).

For *channel estimation*, the attention weights play the role of the interpolation filter between pilots; for *signal classification* (modulation type, interferer type, Chapter 22) the tokens are short chunks of I/Q samples and a class token is read out, exactly as in ViT.

17.10.1 CSI feedback compression

In FDD massive MIMO the gNB cannot measure the downlink channel; the UE must feed it back over a narrow uplink. CsiNet (Wen, Shih and Jin, 2018) treated the angular–delay channel matrix as an image and compressed it with a convolutional autoencoder — the UE runs the encoder, the gNB the decoder (Chapter 18 explains autoencoders). TransNet (Cui, Guo and Song, *IEEE Wireless Communications Letters*, 2022) replaced the convolutions with a two-layer Transformer encoder and decoder ("full attention"), arguing that attention captures long-range correlations across the angular–delay map that small convolution kernels miss. This UE-encoder/gNB-decoder split is precisely the "two-sided model" that has made standardisation difficult, as we see next.

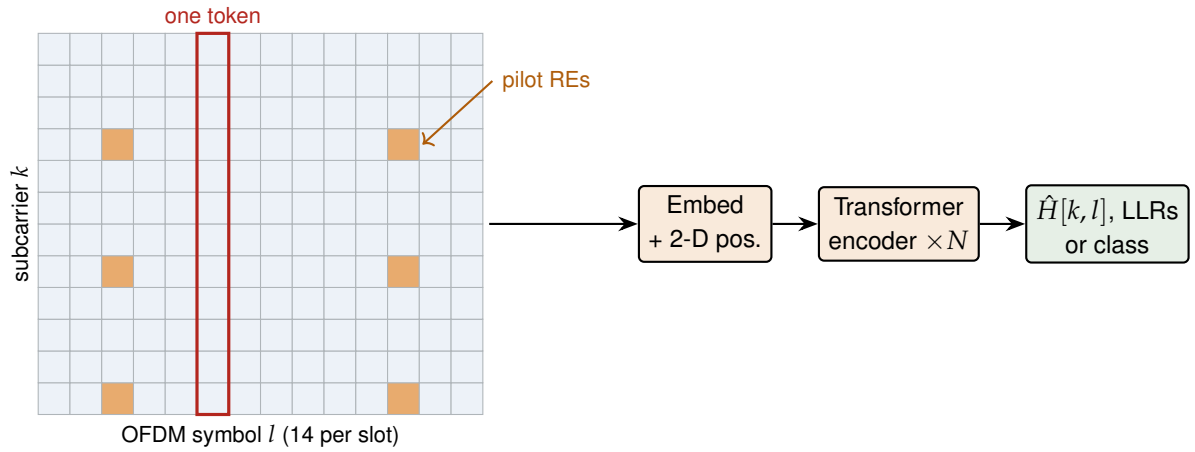


Figure 17.12: A Transformer over the OFDM resource grid. Here one token is one OFDM symbol's 12 subcarriers (red outline) — other choices are one RE or one resource block. Pilot REs (orange) carry known symbols; after the encoder, every token has attended to every pilot and can output its own channel estimate, soft bits, or contribute to a classification.

Wireless connection: what 3GPP has (and has not) standardised

Release 18 ran the first 3GPP *study* on AI/ML for the NR air interface, documented in TR 38.843 (v18.0.0, December 2023). It examined three use cases: **CSI feedback** — spatial–frequency compression with a *two-sided* model (encoder in the UE, decoder in the gNB) and time-domain CSI *prediction* with a one-sided model; **beam management** — predicting the best beams in the spatial and time domains from a subset of measurements to cut overhead and latency; and **positioning** — direct AI/ML positioning (fingerprinting) and AI/ML-assisted positioning, aimed at non-line-of-sight conditions. The study also defined life-cycle-management concepts (data collection, model monitoring, switching) and the distinction between one-sided and two-sided models. It did not standardise any particular network architecture.

For Release 19, 3GPP's own overview describes a RAN1 project specifying CSI prediction, beam management and positioning, while two-sided CSI compression continued as a *study*: the interoperability question — whose encoder works with whose decoder, and who trains them — is unresolved. So, honestly: Transformer-based CSI compressors are an active research topic and a candidate for 6G, not a feature of today's deployed 5G.

Chapter recap

- Attention = query–key similarity (dot product) → softmax weights → weighted average of values: $\text{softmax}(\mathbf{QK}^\mathsf{T}/\sqrt{d_k})\mathbf{V}$.
- Dot products of d_k-dimensional unit-variance vectors have variance d_k; dividing by $\sqrt{d_k}$ keeps the softmax from saturating.
- A causal mask ($-\infty$ above the diagonal) stops a token from seeing the future.
- Multi-head attention splits d_{model} into h subspaces ($512 = 8 \times 64$ in the original Transformer) so different heads learn different relationships.
- Attention is permutation-equivariant; positional encodings (sinusoidal, learned, or rotary) supply order. Sinusoids make a position shift a rotation; RoPE makes scores

depend on relative position.

- A pre-norm block: $\mathbf{x} + \mathrm{MHA}(\mathrm{LN}\,\mathbf{x})$, then $\mathbf{x} + \mathrm{FFN}(\mathrm{LN}\,\mathbf{x})$, FFN $=$ Linear($d \to 4d$), GELU, Linear($4d \to d$).
- Encoder (see all), decoder (causal), cross-attention (queries from one sequence, keys/-values from another). Cost and memory grow as n^2; tokenisation is a key design choice. ViT = Transformer on image patches.
- A 100k-parameter Transformer learned reversal (positional heads) and sorting (content heads); on the resource grid, attention acts as a learned pilot interpolator.

Exercises

1. **(Numeric)** Using the worked example's $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$, compute row 2 of the attention output by hand and confirm $(3.41, 4.41)$.
2. **(Numeric)** Repeat row 1 of the worked example *without* dividing by $\sqrt{d_k}$. How do the weights change? Then imagine all scores multiplied by 10: what happens to the weights, and why is that bad for learning?
3. **(Conceptual)** Show that if $\mathbf{P}$ is a permutation matrix, $\mathrm{Attention}(\mathbf{PQ}, \mathbf{PK}, \mathbf{PV}) = \mathbf{P}\,\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$. What does this imply about the need for positional encodings?
4. **(Numeric)** The query and key components have variance $\sigma^2 = 4$ instead of 1, and $d_k = 64$. What is the variance of $\mathbf{q} \cdot \mathbf{k}$? By what should you divide to get unit variance?
5. **(Numeric)** Count the weights (ignore biases) of one encoder block of the original Transformer: $d_{\text{model}} = 512$, $h = 8$, $d_{\text{ff}} = 2048$, plus two LayerNorms. How many blocks fit in 44 million weights?
6. **(Numeric)** A Transformer processes one NR slot of 273 resource blocks $\times$ 14 symbols, one token per resource block per symbol, with 4 heads. How many attention weights per layer? How many if tokens were single REs?
7. **(Conceptual)** In the *sort* lab, why do the layer-2 heads attend by content rather than position? Could a model that only had positional attention patterns sort?
8. **(Conceptual)** For channel estimation on the resource grid, would you choose encoder-only, decoder-only, or encoder–decoder? Justify, and say where a causal mask might nevertheless be useful.

Answers

1. Row 2 of $\mathbf{S}$ is $(0, 1, 1)$; scaled: $(0, 0.7071, 0.7071)$. Exponentials $1, 2.0281, 2.0281$, sum 5.0562, weights $(0.198, 0.401, 0.401)$. Output: $0.198(1, 2) + 0.401(3, 4) + 0.401(5, 6) = (0.198 + 1.203 + 2.006,\ 0.396 + 1.604 + 2.406) = (3.41, 4.41)$.
2. Unscaled scores $(1, 0, 1)$: $e^1 = 2.718$, $e^0 = 1$; sum 6.437; weights $(0.422, 0.155, 0.422)$ — sharper than $(0.401, 0.198, 0.401)$. With scores $(10, 0, 10)$: weights $\approx (0.5, 0.00005, 0.5)$; token 2 is completely ignored and the softmax Jacobian $a_i(\delta_{ij} - a_j)$ is almost zero for it, so gradients cannot change its weight: learning stalls.
3. $(\mathbf{PQ})(\mathbf{PK})^\mathsf{T} = \mathbf{PQK}^\mathsf{T}\mathbf{P}^\mathsf{T}$. The row-wise softmax commutes with permuting rows and with permuting columns (it just reorders the entries it normalises), so $\mathrm{softmax}(\mathbf{PSP}^\mathsf{T}) =$

$\mathbf{PAP}^\mathsf{T}$. Multiplying by $\mathbf{PV}$: $\mathbf{PAP}^\mathsf{T}\mathbf{PV} = \mathbf{PAV}$ since $\mathbf{P}^\mathsf{T}\mathbf{P} = \mathbf{I}$. The output is just the original output reordered, so attention alone cannot tell sequence order: positions must be injected.

4. Each term has variance $\mathbb{E}[q^2]\mathbb{E}[k^2] = 4 \cdot 4 = 16$; the sum of 64 terms has variance 1024, std 32. Divide by $32 = \sigma^2\sqrt{d_k}$. (In practice LayerNorm keeps inputs near unit scale, which is why $\sqrt{d_k}$ alone works.)

5. Attention: $4 \times 512^2 = 1{,}048{,}576$. FFN: $2 \times 512 \times 2048 = 2{,}097{,}152$. LayerNorms: $2 \times 2 \times 512 = 2048$. Total 3,147,776 $\approx$ 3.15M per block; $44/3.15 \approx 14$ blocks.

6. $n = 273 \times 14 = 3822$ tokens; $3822^2 \approx 1.46 \times 10^7$ weights per head, 5.8×10^7 for 4 heads. With single REs: $n = 45{,}864$, $n^2 \approx 2.1 \times 10^9$ per head, 8.4×10^9 for 4 heads — 144 times more (because $12^2 = 144$).

7. In sorting, the output at position i is "the $(i{+}1)$-th smallest digit", which can be anywhere in the input: its location depends on the values, not on i. So the query for output position i must find keys by *what they contain*. A purely positional pattern is fixed for all inputs and implements a fixed permutation (like reversal); sorting needs an input-dependent permutation, which content-based attention provides.

8. Encoder-only is natural: the whole slot (all pilots) is received before estimation, so every RE should see every pilot. An encoder–decoder with pilot tokens as the encoder input and data REs as decoder queries is a sensible alternative. A causal mask over *symbols* is useful if you want to estimate or predict the channel for symbol l using only symbols $\leq l$ — for low-latency processing or for channel *prediction* into future slots.

Self-test quiz

1. In attention, which quantity determines *how much* of each token is taken, and which determines *what* is taken?
2. What is the variance of $\mathbf{q} \cdot \mathbf{k}$ for unit-variance independent components in d_k dimensions?
3. How is causal masking implemented, and why does it keep each row a valid probability distribution?
4. In the original Transformer, $d_{\text{model}} = 512$ and $h = 8$. What is d_k?
5. Why do Transformers need positional information while RNNs and CNNs do not?
6. In cross-attention, which sequence supplies the queries and which the keys and values?

Answers: 1. The query–key scores (after softmax) set the weights; the values are what is blended. 2. d_k. 3. Set scores above the diagonal to $-\infty$ before the softmax; $e^{-\infty} = 0$ and the remaining weights are renormalised to sum to one. 4. $512/8 = 64$. 5. Self-attention is permutation-equivariant (it sees a set); RNNs process in order and CNN kernels are tied to relative positions. 6. Queries from the decoder (the sequence being produced); keys and values from the encoder output.

Further reading

- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, "Attention is all you need," *NeurIPS*, 2017. The original Transformer.
- D. Bahdanau, K. Cho, Y. Bengio, "Neural machine translation by jointly learning to align and translate," *ICLR*, 2015. The first attention mechanism, on top of an RNN.
- A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," *ICLR*, 2021. Vision Transformer.
- J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, Y. Liu, "RoFormer: Enhanced Transformer with rotary position embedding," arXiv:2104.09864, 2021.
- R. Xiong et al., "On layer normalization in the Transformer architecture," *ICML*, 2020. Pre-norm versus post-norm.
- C.-K. Wen, W.-T. Shih, S. Jin, "Deep learning for massive MIMO CSI feedback," *IEEE Wireless Communications Letters*, 2018 (CsiNet); Y. Cui, A. Guo, C. Song, "TransNet: Full attention network for CSI feedback in FDD massive MIMO system," *IEEE Wireless Communications Letters*, vol. 11, no. 5, 2022.
- 3GPP TR 38.843, "Study on Artificial Intelligence (AI)/Machine Learning (ML) for NR air interface," v18.0.0, 2023; X. Lin, "An overview of the 3GPP study on artificial intelligence for 5G New Radio," arXiv:2308.05315, 2023.

Chapter 18

Autoencoders, VAEs, GANs, Diffusion and Transfer Learning

Every network so far has been a *discriminative* model: give it an input $\mathbf{x}$ and it predicts a label y. This chapter is about networks that learn the data *itself*. An autoencoder learns to compress $\mathbf{x}$ into a few numbers and rebuild it; a variational autoencoder, a generative adversarial network (GAN) and a diffusion model learn the whole probability distribution $p(\mathbf{x})$ well enough to draw *new* samples from it. We finish with transfer learning: reusing what one network has learned to solve a new problem with little data.

For a communication engineer these ideas are not exotic. A transmitter–channel–receiver chain *is* an autoencoder: it encodes bits into a waveform, the channel corrupts it, and the receiver reconstructs the bits. Generative models can learn a channel from measurements when no equation describes it well, and transfer learning is how a receiver trained in simulation is made to work over the air.

You will learn:

- autoencoders: encoder, bottleneck, decoder; undercomplete and denoising variants; a 2-D latent map of handwritten digits;
- variational autoencoders: the latent-variable model, the ELBO derived step by step, the closed-form KL between Gaussians, and the reparameterisation trick;
- GANs: the minimax game, the optimal discriminator, the link to the Jensen–Shannon divergence, the non-saturating loss and mode collapse;
- diffusion models (DDPM): the forward noising process and its closed form, the noise-prediction loss, and sampling;
- transfer learning: feature extraction versus fine-tuning, demonstrated by a real experiment;
- the wireless uses of each.

18.1 Autoencoders

Intuition: describe a face in five words

Imagine you must describe a friend's face over a bad phone line using only five numbers, so that an artist at the other end can draw it. You would not waste numbers on individual pixels; you would choose the most informative properties — face width, hair length, skin tone, and so on. If the artist's drawing looks like your friend, your five numbers captured what matters. An autoencoder learns both halves of this game: the *encoder* learns which few numbers to send, and the *decoder* learns how to draw from them. Nobody tells it what the numbers should mean; it discovers them by trying to reconstruct.

Definition: autoencoder

An autoencoder is a pair of networks: an **encoder** $\mathbf{z} = f_\phi(\mathbf{x})$ mapping an input $\mathbf{x} \in \mathbb{R}^D$ to a **code** or **latent vector** $\mathbf{z} \in \mathbb{R}^d$, and a **decoder** $\hat{\mathbf{x}} = g_\theta(\mathbf{z})$ mapping it back. Both are trained together to minimise the reconstruction error, for example the mean squared error

$$\mathcal{L}(\phi, \theta) = \frac{1}{N} \sum_{n=1}^{N} \left\| \mathbf{x}_n - g_\theta(f_\phi(\mathbf{x}_n)) \right\|^2 . \tag{18.1}$$

If $d < D$ the autoencoder is **undercomplete**: the code is a **bottleneck** that forces compression.

No labels appear in (18.1): the input is its own target. This is *self-supervised* learning. Without the bottleneck the network could simply learn the identity function and learn nothing useful; the bottleneck forces it to find the structure in the data. A linear autoencoder with MSE loss learns the same subspace as principal component analysis (PCA, Chapter 2); nonlinear layers let it learn curved structure that PCA cannot.

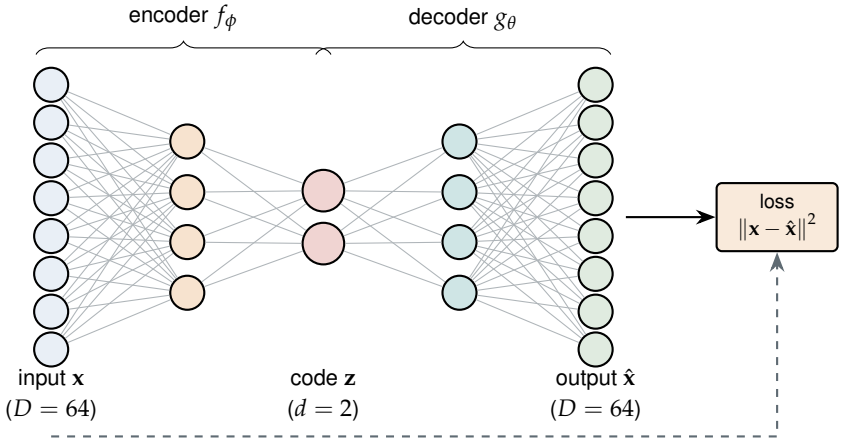


Figure 18.1: An undercomplete autoencoder. The 64 pixels of an 8×8 digit are squeezed through a 2-number bottleneck (red) and expanded back. The loss compares the output with the input itself (dashed path), so no labels are needed. In our script the hidden layers are wider (128 and 32 units) than drawn here.

18.1.1 A 2-D map of handwritten digits

We train an autoencoder with layers $64 \to 128 \to 32 \to 2 \to 32 \to 128 \to 64$ (ReLU inside, a sigmoid at the output because pixels are scaled to $[0, 1]$) on the 1797 8×8 digits bundled with scikit-learn, for 150 epochs with Adam. Because the code has two dimensions we can plot every digit as a point (Figure 18.2).

How much does the bottleneck cost? The mean squared reconstruction error per pixel (pixels in $[0, 1]$) was 0.0245 with $d = 2$, 0.0165 with $d = 4$, 0.0108 with $d = 8$ and 0.0091 with $d = 16$: diminishing returns, as the main "directions of variation" of the digits are captured first.

18.1.2 Denoising autoencoders

A different way to stop an autoencoder learning the identity: corrupt the input and ask for the *clean* version,

$$\mathcal{L} = \frac{1}{N} \sum_{n} \left\| \mathbf{x}_n - g_\theta\big(f_\phi(\tilde{\mathbf{x}}_n)\big) \right\|^2 , \qquad \tilde{\mathbf{x}}_n = \mathbf{x}_n + \boldsymbol{\epsilon}_n .$$

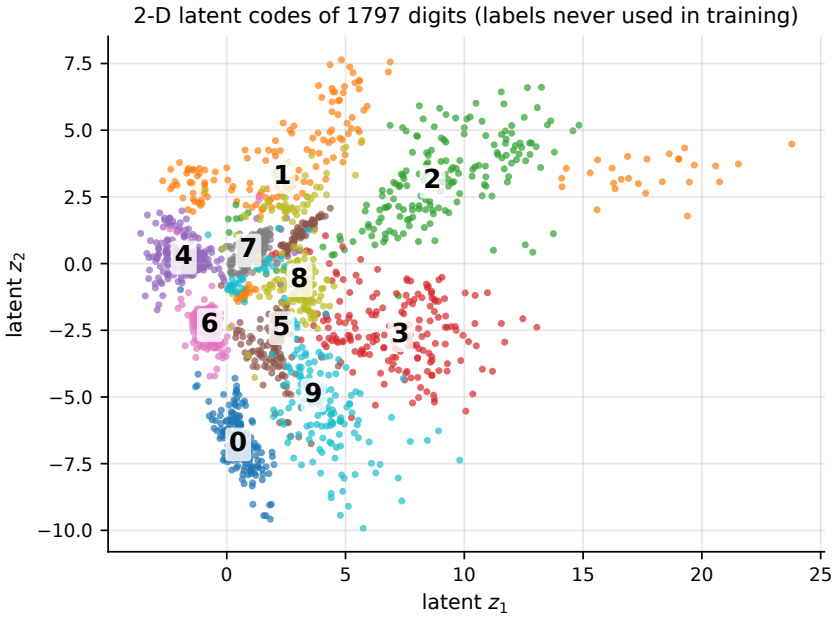


Figure 18.2: The 2-D codes $\mathbf{z} = f_\phi(\mathbf{x})$ of all 1797 digits, coloured by their true label *after* training — the autoencoder never saw a label. Digits of the same class gather together (0, 2, 3, 4, 6 form clear islands) because similar images are cheapest to reconstruct from similar codes. Visually similar classes (1, 7, 8, 9, 5) crowd in the middle. Notice also the empty regions and the irregular, unbounded shape: we will return to this when we try to *generate* from the latent space.

To succeed, the network must learn what digits look like so it can tell signal from noise. Figure 18.3 compares a plain and a denoising autoencoder (both $d = 16$) on inputs with Gaussian noise of standard deviation 0.4 (pixels then clipped to $[0, 1]$).

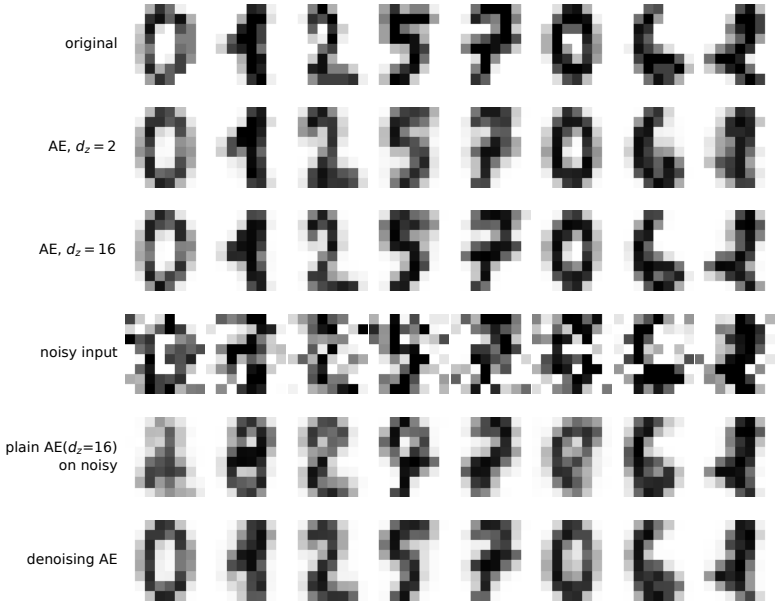


Figure 18.3: Rows 1–3: originals and their reconstructions with a 2-number and a 16-number code. With $d = 2$ the strokes are blurred toward a "typical" digit; with $d = 16$ they are nearly perfect. Rows 4–6: noisy inputs; the plain autoencoder, never trained on noise, produces smudges; the denoising autoencoder, trained to map noisy inputs to clean targets, restores recognisable digits.

Wireless connection: the communication system is an autoencoder

Put a transmitter, channel and receiver side by side with Figure 18.1. The transmitter maps a message $s \in \{1, \dots, M\}$ (one of $M = 2^k$ possible k-bit words) to n complex channel uses — an *encoder*. The channel adds noise and distortion — a fixed, non-trainable *noise layer*. The receiver maps the n noisy samples back to a probability for each of the M messages — a *decoder*. Train end-to-end with cross-entropy, and the network invents its own modulation and coding: the code $\mathbf{z}$ *is* the transmitted signal, and the bottleneck is the channel capacity. A normalisation layer enforces the transmit power constraint, just as the bottleneck enforces compression here. This is the idea of O'Shea and Hoydis (2017), which Chapter 21 develops in full. Note the twist: an ordinary autoencoder *compresses* ($n < D$ in real numbers), whereas the communication autoencoder adds *redundancy* so that the message survives noise; the denoising autoencoder is the closer analogy.

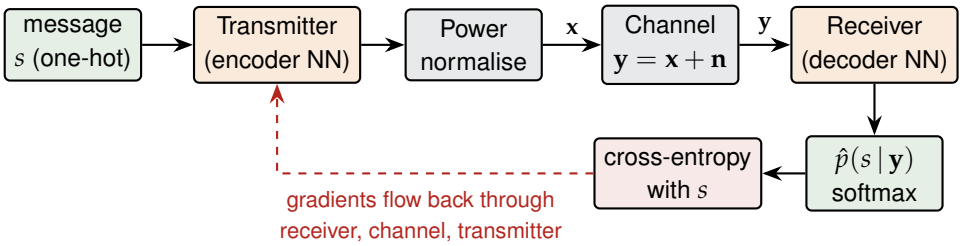


Figure 18.4: The physical layer as an autoencoder (preview of Chapter 21). The encoder is the transmitter, the noise layer is the channel and the decoder is the receiver. If the channel is differentiable (as AWGN is), the loss gradient flows all the way back into the transmitter, which learns a constellation adapted to the channel.

18.2 Variational autoencoders

Look again at Figure 18.2. Suppose we want to *generate* a new digit: pick a point $\mathbf{z}$ and decode it. Which point? The cloud has an odd shape, gaps, and long tails; a point in a gap decodes to garbage. A plain autoencoder gives no rule for choosing $\mathbf{z}$. The variational autoencoder (VAE, Kingma and Welling, 2014) fixes this by making the latent space follow a known, simple distribution — a standard Gaussian — so that we can sample from it.

18.2.1 The latent-variable model

Intuition: a recipe with a random seed

Imagine every digit is produced by a two-step process: first nature draws a few hidden "style" numbers $\mathbf{z}$ (slant, thickness, which digit) from a simple bell-shaped distribution; then a complicated but deterministic machine, the decoder, turns those numbers into pixels. If we could learn that machine, generating a new digit would be easy: draw $\mathbf{z}$ from the bell curve and run the machine.

Formally, the model is

$$\mathbf{z} \sim p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I}), \qquad \mathbf{x} \sim p_\theta(\mathbf{x} \mid \mathbf{z}),$$

where $p_\theta(\mathbf{x}|\mathbf{z})$ is defined by the decoder network. For our pixels in $[0,1]$ we take each pixel

to be an independent Bernoulli variable with probability $\hat{x}_m = \sigma(\text{decoder output}_m)$, so that $\log p_\theta(\mathbf{x}|\mathbf{z}) = \sum_m [x_m \log \hat{x}_m + (1 - x_m)\log(1 - \hat{x}_m)]$ — minus the binary cross-entropy (Chapter 8). We would like to fit θ by maximum likelihood, maximising $\log p_\theta(\mathbf{x})$ for the training digits, where

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, \mathrm{d}\mathbf{z}.$$

This integral is the problem: it averages over *all* possible codes, and for almost all of them $p_\theta(\mathbf{x}|\mathbf{z})$ is tiny. Sampling $\mathbf{z}$ at random would essentially never hit the few codes that explain a given digit.

18.2.2 The evidence lower bound, step by step

The fix is to bring in a second network, the **encoder** $q_\phi(\mathbf{z} \mid \mathbf{x})$, which for a given $\mathbf{x}$ proposes the codes that are likely to have produced it. We choose a Gaussian with a diagonal covariance whose mean and variances come from the encoder network:

$$q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}\big(\mathbf{z};\, \boldsymbol{\mu}_\phi(\mathbf{x}),\, \mathrm{diag}(\boldsymbol{\sigma}^2_\phi(\mathbf{x}))\big).$$

Now derive a quantity we *can* compute. Start from the log-likelihood of one data point and use the fact that $\int q_\phi(\mathbf{z}|\mathbf{x})\, \mathrm{d}\mathbf{z} = 1$:

$$\begin{aligned}
\log p_\theta(\mathbf{x}) &= \int q_\phi(\mathbf{z}|\mathbf{x})\, \log p_\theta(\mathbf{x})\, \mathrm{d}\mathbf{z} && \text{(multiply by 1)} \\
&= \mathbb{E}_q\left[\log \frac{p_\theta(\mathbf{x},\mathbf{z})}{p_\theta(\mathbf{z}|\mathbf{x})}\right] && \text{(Bayes: } p(\mathbf{x}) = p(\mathbf{x},\mathbf{z})/p(\mathbf{z}|\mathbf{x})\text{)} \\
&= \mathbb{E}_q\left[\log \frac{p_\theta(\mathbf{x},\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} \cdot \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p_\theta(\mathbf{z}|\mathbf{x})}\right] && \text{(multiply and divide by } q\text{)} \\
&= \underbrace{\mathbb{E}_q\left[\log \frac{p_\theta(\mathbf{x},\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})}\right]}_{\text{ELBO}} + \underbrace{\mathbb{E}_q\left[\log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p_\theta(\mathbf{z}|\mathbf{x})}\right]}_{\mathrm{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, p_\theta(\mathbf{z}|\mathbf{x})) \,\geq\, 0}.
\end{aligned}$$

Here $\mathbb{E}_q[\cdot]$ means the average over $\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})$. The second term is a Kullback–Leibler divergence (Chapter 4), which is never negative. Therefore the first term is a lower bound on the log-likelihood of the data ("evidence"), hence its name, the **evidence lower bound** (ELBO). The bound is tight when the encoder matches the true posterior exactly.

Now expand the ELBO using $p_\theta(\mathbf{x},\mathbf{z}) = p_\theta(\mathbf{x}|\mathbf{z})\, p(\mathbf{z})$:

$$\mathrm{ELBO} = \mathbb{E}_q\big[\log p_\theta(\mathbf{x}|\mathbf{z})\big] + \mathbb{E}_q\left[\log \frac{p(\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})}\right] = \mathbb{E}_q\big[\log p_\theta(\mathbf{x}|\mathbf{z})\big] - \mathrm{KL}\big(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, p(\mathbf{z})\big).$$

Key equation: the VAE objective

$$\log p_\theta(\mathbf{x}) \;\geq\; \mathcal{L}_{\mathrm{ELBO}} = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\big[\log p_\theta(\mathbf{x} \mid \mathbf{z})\big]}_{\text{reconstruction}} - \underbrace{\mathrm{KL}\big(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\big\|\, \mathcal{N}(\mathbf{0}, \mathbf{I})\big)}_{\text{regulariser}}. \tag{18.2}$$

Train encoder and decoder together by *maximising* the ELBO (minimising $-$ELBO) averaged over the data.

Both terms have a plain meaning. The *reconstruction* term says: codes drawn from the encoder should decode back to the input — this is the autoencoder loss. The *KL* term says: each digit's cloud of codes should look like the standard Gaussian — centred near zero, with spread near one. It pulls all the clouds into the same compact ball and stops the encoder from using tiny variances (which would make it a plain autoencoder again). The tension between the two produces a latent space that is *filled*, with no gaps, so that any $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ decodes to something sensible.

18.2.3 KL divergence between two Gaussians in closed form

For one latent dimension with $q = \mathcal{N}(\mu, \sigma^2)$ and $p = \mathcal{N}(0, 1)$:

$$\mathrm{KL}(q \,\|\, p) = \mathbb{E}_q\big[\log q(z) - \log p(z)\big].$$

The log-densities are $\log q(z) = -\frac{1}{2}\log(2\pi\sigma^2) - \frac{(z-\mu)^2}{2\sigma^2}$ and $\log p(z) = -\frac{1}{2}\log(2\pi) - \frac{z^2}{2}$. Subtract (the $\log 2\pi$ terms cancel):

$$\log q(z) - \log p(z) = -\tfrac{1}{2}\log\sigma^2 - \frac{(z-\mu)^2}{2\sigma^2} + \frac{z^2}{2}.$$

Take the average over $z \sim \mathcal{N}(\mu, \sigma^2)$, using two facts: $\mathbb{E}_q[(z-\mu)^2] = \sigma^2$ (definition of variance) and $\mathbb{E}_q[z^2] = \mathrm{Var}(z) + (\mathbb{E}z)^2 = \sigma^2 + \mu^2$:

$$\mathrm{KL} = -\tfrac{1}{2}\log\sigma^2 - \frac{\sigma^2}{2\sigma^2} + \frac{\sigma^2+\mu^2}{2} = \tfrac{1}{2}\big(\mu^2 + \sigma^2 - 1 - \log\sigma^2\big).$$

With independent dimensions the KL of the product is the sum over dimensions:

Key equation: KL from a diagonal Gaussian to the standard normal

$$\mathrm{KL}\big(\mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}\,\boldsymbol{\sigma}^2) \,\big\|\, \mathcal{N}(\mathbf{0}, \mathbf{I})\big) = \frac{1}{2}\sum_{j=1}^{d}\big(\mu_j^2 + \sigma_j^2 - 1 - \log\sigma_j^2\big). \tag{18.3}$$

Check: it is zero exactly when $\mu_j = 0$ and $\sigma_j = 1$ (since $\sigma^2 - 1 - \log\sigma^2 \geq 0$ with equality at $\sigma^2 = 1$). The μ^2 term pulls means toward the origin; the $\sigma^2 - \log\sigma^2$ term penalises variances that are too small *or* too large.

Worked example: one KL term by hand

The encoder outputs $\mu = 1$, $\sigma = 0.5$ for one dimension. Then $\sigma^2 = 0.25$, $\log\sigma^2 = -1.3863$, and $\mathrm{KL} = \frac{1}{2}(1 + 0.25 - 1 + 1.3863) = \frac{1}{2}(1.6363) = 0.818$ nats. Moving the mean to 0 would remove 0.5 of this; widening σ to 1 would remove the remaining 0.318.

18.2.4 The reparameterisation trick

To estimate the reconstruction term we draw $\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})$ and evaluate $\log p_\theta(\mathbf{x}|\mathbf{z})$. But sampling is not a differentiable operation: how does the gradient reach ϕ through a random draw? Rewrite the sample as a deterministic function of the parameters and an independent noise:

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{18.4}$$

This $\mathbf{z}$ has exactly the distribution $\mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}\,\boldsymbol{\sigma}^2)$ (shifting and scaling a standard Gaussian), but now the randomness sits in $\boldsymbol{\epsilon}$, which does not depend on ϕ. The derivatives are ordinary: $\partial z_j/\partial\mu_j = 1$ and $\partial z_j/\partial\sigma_j = \epsilon_j$. Example: $\mu = 1$, $\sigma = 0.5$, drawn $\epsilon = -0.4$ gives $z = 1 + 0.5(-0.4) = 0.8$; a small increase in μ moves z by the same amount, a small increase in σ moves it by -0.4 times that amount.

18.2.5 The VAE in code and its results

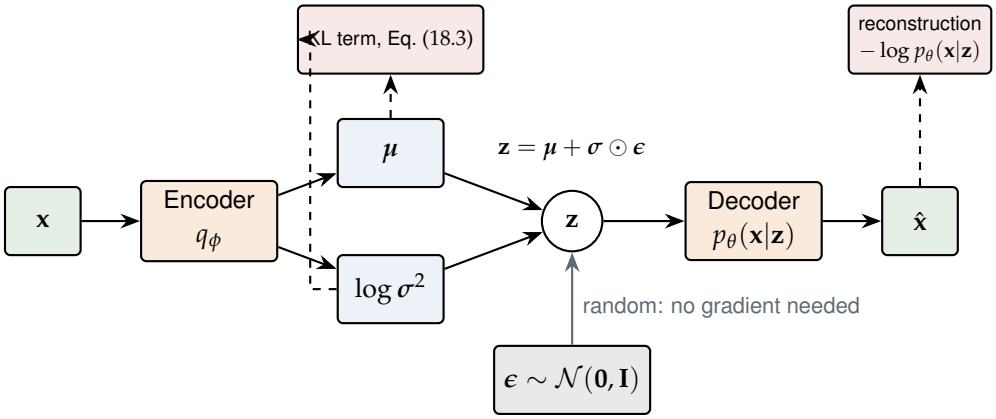


Figure 18.5: The VAE with the reparameterisation trick. The encoder outputs a mean and a log-variance for each latent dimension; the sample $\mathbf{z}$ is formed as $\boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$ with external noise, so backpropagation flows through $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ into the encoder. The loss is the reconstruction term plus the KL term.

```python
class VAE(nn.Module):
    def __init__(self, d_z=2):
        super().__init__()
        self.enc = nn.Sequential(nn.Linear(64, 128), nn.ReLU(),
                                 nn.Linear(128, 64), nn.ReLU())
        self.mu, self.logvar = nn.Linear(64, d_z), nn.Linear(64, d_z)
        self.dec = nn.Sequential(nn.Linear(d_z, 64), nn.ReLU(),
                                 nn.Linear(64, 128), nn.ReLU(),
                                 nn.Linear(128, 64))   # pixel logits
    def forward(self, x):
        h = self.enc(x); mu, logvar = self.mu(h), self.logvar(h)
        eps = torch.randn_like(mu)
        z = mu + torch.exp(0.5 * logvar) * eps   # reparameterization trick
        return self.dec(z), mu, logvar

def neg_elbo(logits, x, mu, logvar):
    # reconstruction: -E log p(x|z) (Bernoulli pixels)
    rec = F.binary_cross_entropy_with_logits(logits, x, reduction='sum')
    kl = 0.5 * torch.sum(mu ** 2 + logvar.exp() - 1 - logvar)  # KL(q||N(0,I))
    return (rec + kl) / x.shape[0], rec / x.shape[0], kl / x.shape[0]
```

Two practical details: the encoder outputs $\log \sigma^2$ rather than σ (any real number is a valid log-variance, so no constraint is needed), and one sample of $\boldsymbol{\epsilon}$ per data point per step is enough — the noise averages out over many steps. We trained with a 2-D latent space for 200 epochs. At the end of training the loss per digit was 23.8 nats, made of 20.5 nats of reconstruction error and 3.2 nats of KL — the 2-D code is "allowed" to carry only a few nats of information about each image, which is why reconstructions are less sharp than with the plain autoencoder.

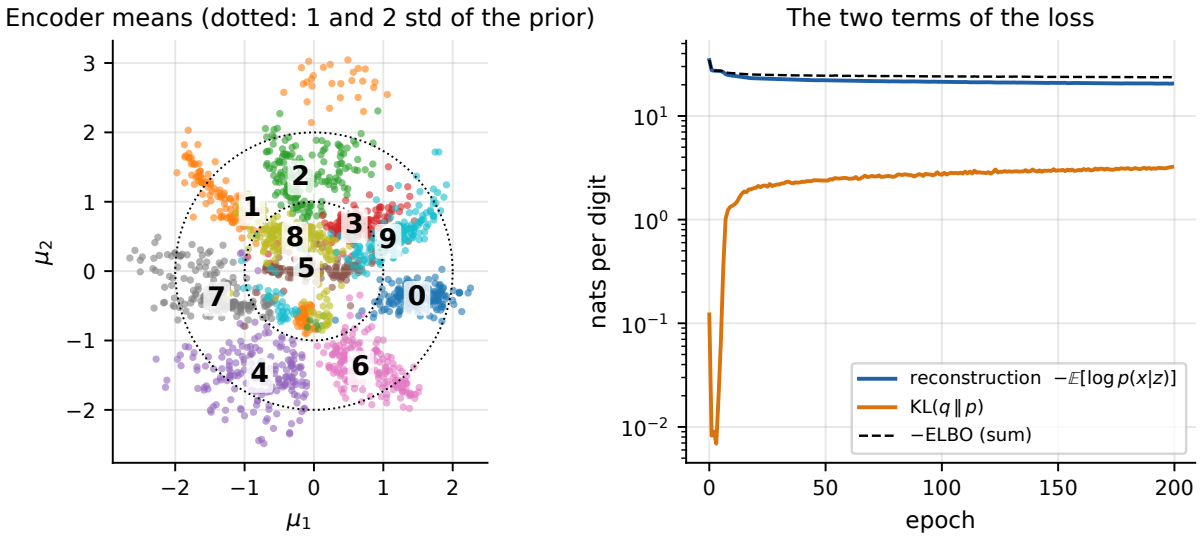


Figure 18.6: Left: encoder means $\mu_\phi(\mathbf{x})$ for all digits. Compared with the plain autoencoder (Figure 18.2) the codes now fill a compact disc of radius about 2–3 around the origin, matching the prior $\mathcal{N}(\mathbf{0}, \mathbf{I})$ (dotted circles at 1 and 2 standard deviations), and classes still cluster. Right: the two parts of the loss during training (log scale). The reconstruction term falls; the KL term *rises* at first as the encoder starts to use the latent space to carry information, then settles: the price, in nats, of the information stored in $\mathbf{z}$.

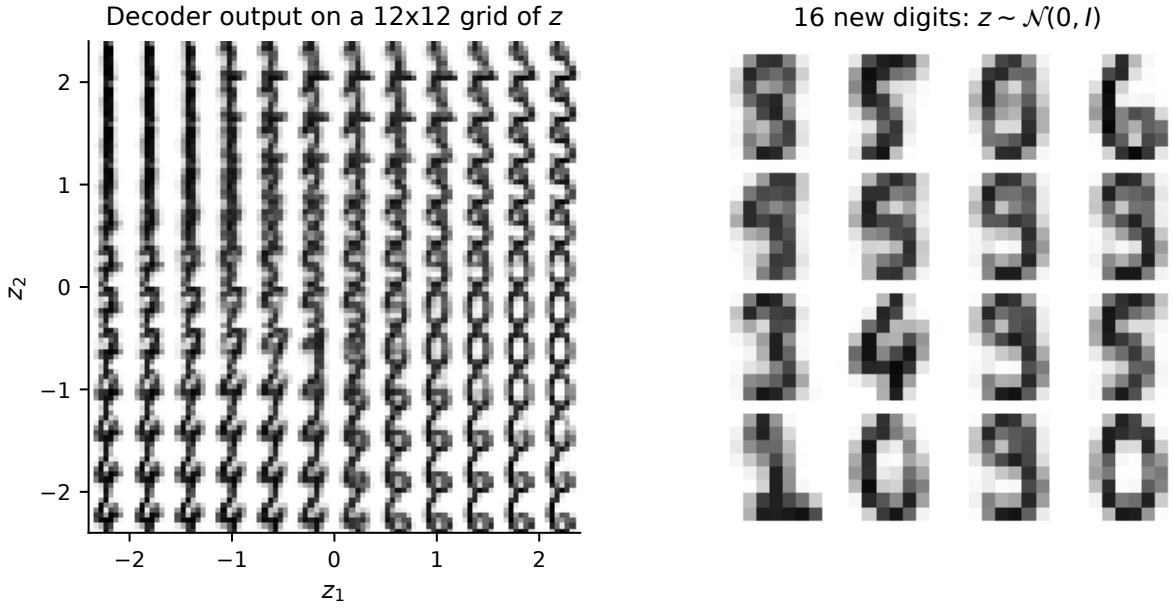


Figure 18.7: Generating with the trained VAE. Left: decoder outputs for a 12×12 grid of codes between -2.2 and 2.2; the digits morph smoothly into each other — there are no "holes" that decode to noise. Right: 16 brand-new digits made by drawing $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and decoding. They are recognisable but blurry, a known weakness of VAEs with simple pixel-wise likelihoods.

18.3 Generative adversarial networks

Intuition: the forger and the detective

A forger (the **generator** G) paints fake banknotes; a detective (the **discriminator** D) inspects notes and says "real" or "fake". The detective learns from her mistakes and gets sharper; the forger learns from the detective's verdicts — specifically, from *which direction* would have made his fake look more real — and gets better. If both keep improving, the forger eventually produces notes the detective cannot tell from real ones: she can do no better than guessing, 50/50.

Definition: GAN

A generator G_θ maps random noise $\mathbf{z} \sim p_z$ (e.g. $\mathcal{N}(\mathbf{0}, \mathbf{I})$) to a sample $G(\mathbf{z})$; the distribution of these samples is called p_g. A discriminator $D_\psi(\mathbf{x}) \in (0,1)$ outputs the probability that $\mathbf{x}$ is real. They play the **minimax game** (Goodfellow et al., 2014)

$$\min_G \max_D \; V(D,G) = \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}\big[\log D(\mathbf{x})\big] + \mathbb{E}_{\mathbf{z}\sim p_z}\big[\log\big(1 - D(G(\mathbf{z}))\big)\big]. \tag{18.5}$$

V is the (negative) binary cross-entropy of a classifier that labels real data 1 and fakes 0. The discriminator maximises it — it wants to classify well. The generator minimises it — it wants the classifier to fail. Unlike a VAE, a GAN never writes down $p_g(\mathbf{x})$; it only learns to *sample* from it.

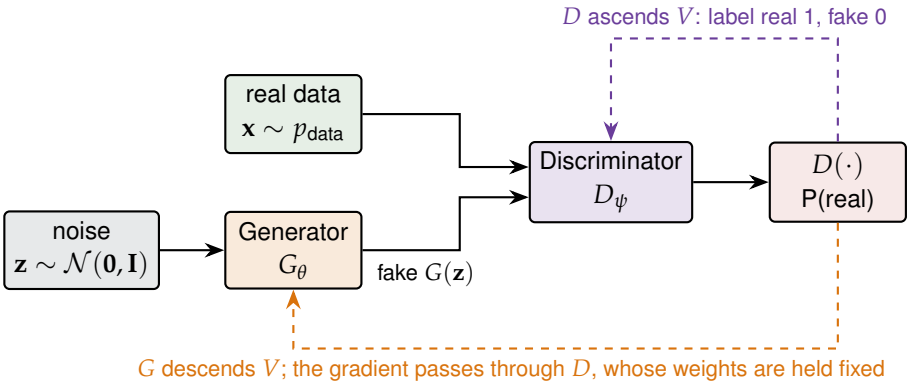


Figure 18.8: A GAN. The discriminator sees real samples and generator samples and is trained to tell them apart. The generator is trained through the discriminator: its gradient tells it how to move each fake point to look more real. The two updates alternate.

18.3.1 The optimal discriminator

Fix G, so p_g is fixed, and ask what D maximises V. Write the second expectation as an average over $\mathbf{x} \sim p_g$ (the samples $G(\mathbf{z})$) and combine both into one integral:

$$V(D,G) = \int \Big[p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) + p_g(\mathbf{x}) \log\big(1 - D(\mathbf{x})\big)\Big] \mathrm{d}\mathbf{x}.$$

We can maximise the integrand separately at every point $\mathbf{x}$. At a fixed $\mathbf{x}$ write $a = p_{\text{data}}(\mathbf{x})$, $b = p_g(\mathbf{x})$, $y = D(\mathbf{x}) \in (0,1)$; we maximise $h(y) = a \log y + b \log(1-y)$. Set the derivative to zero:

$$h'(y) = \frac{a}{y} - \frac{b}{1-y} = 0 \implies a(1-y) = by \implies y = \frac{a}{a+b}.$$

The second derivative $h''(y) = -a/y^2 - b/(1-y)^2 < 0$ confirms a maximum.

Key equation: optimal discriminator

$$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})}. \tag{18.6}$$

The best detective outputs the fraction of the probability at $\mathbf{x}$ that comes from real data. Example: where $p_{\text{data}} = 0.6$ and $p_g = 0.2$, $D^* = 0.6/0.8 = 0.75$. Where the generator is perfect, $p_g = p_{\text{data}}$ and $D^* = \frac{1}{2}$ everywhere.

18.3.2 What the generator minimises: the Jensen–Shannon divergence

Substitute D^* back into V:

$$V(D^*, G) = \mathbb{E}_{p_{\text{data}}}\left[\log \frac{p_{\text{data}}}{p_{\text{data}} + p_g}\right] + \mathbb{E}_{p_g}\left[\log \frac{p_g}{p_{\text{data}} + p_g}\right].$$

Introduce the mixture $m = \frac{1}{2}(p_{\text{data}} + p_g)$, so $p_{\text{data}} + p_g = 2m$. Then $\log \frac{p_{\text{data}}}{2m} = \log \frac{p_{\text{data}}}{m} - \log 2$, and similarly for the second term:

$$V(D^*, G) = \mathrm{KL}(p_{\text{data}} \,\|\, m) + \mathrm{KL}(p_g \,\|\, m) - 2\log 2 = 2\,\mathrm{JS}(p_{\text{data}} \,\|\, p_g) - 2\log 2,$$

where $\mathrm{JS}(p\|q) = \frac{1}{2}\mathrm{KL}(p\|m) + \frac{1}{2}\mathrm{KL}(q\|m)$ is the **Jensen–Shannon divergence**, a symmetric measure of the difference between two distributions that is zero only when they are equal. So, *if the discriminator is always optimal*, training the generator minimises the JS divergence between the generated and the real distribution, and the global optimum is $p_g = p_{\text{data}}$, where $V = -2\log 2 \approx -1.386$. In terms of the discriminator's cross-entropy loss ($-V$), equilibrium sits at $2\ln 2 \approx 1.386$ nats.

18.3.3 The non-saturating loss

Early in training the fakes are obviously fake, $D(G(\mathbf{z})) \approx 0$, and the generator's term $\log(1 - D(G(\mathbf{z})))$ is flat there: its derivative with respect to the discriminator's logit ℓ (where $D = \sigma(\ell)$) is $-\sigma(\ell) \approx 0$. The generator gets almost no gradient exactly when it needs it most. Goodfellow et al. therefore proposed in practice to train G to *maximise* $\log D(G(\mathbf{z}))$ instead, i.e. to minimise

$$\mathcal{L}_G = -\mathbb{E}_{\mathbf{z}}\big[\log D(G(\mathbf{z}))\big],$$

whose derivative with respect to ℓ is $-(1 - \sigma(\ell)) \approx -1$ when $D \approx 0$: a strong signal. Both objectives have the same fixed point; this "non-saturating" form simply amounts to labelling fakes as real when computing the generator's binary cross-entropy. At equilibrium ($D = \frac{1}{2}$) it equals $\ln 2 \approx 0.693$.

18.3.4 A tiny GAN on eight Gaussians

A classic test: the real data are a mixture of 8 Gaussians (standard deviation 0.05) placed evenly on a circle of radius 2. Generator and discriminator are both MLPs with three hidden layers of 128 LeakyReLU units; $\mathbf{z}$ is 2-D Gaussian noise. We alternate one discriminator step and one generator step (non-saturating loss), with Adam (learning rate 10^{-3}, $\beta_1 = 0.5$) and batches of 256, for 6000 steps.

```
    real = sample_8gauss(256); fake = G(torch.randn(256, 2))
    # --- discriminator: push D(real) -> 1, D(fake) -> 0 ---
    bce = F.binary_cross_entropy_with_logits
    d_loss = bce(D(real), torch.ones(256, 1)) + bce(D(fake.detach()), torch.zeros
    (256, 1))
    optD.zero_grad(); d_loss.backward(); optD.step()
    # --- generator, non-saturating: maximise log D(G(z)) ---
    g_loss = bce(D(fake), torch.ones(256, 1))
    optG.zero_grad(); g_loss.backward(); optG.step()
```

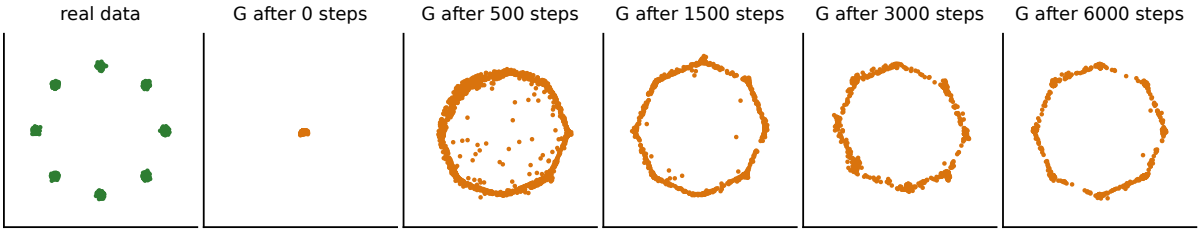


Figure 18.9: Our GAN run, with the same 1500 noise vectors decoded at each snapshot. At step 0 the untrained generator maps all noise to a small blob. By step 500 it has spread onto the circle; by 1500–6000 it concentrates mass on the eight modes, with thin "bridges" between them. Bridges are characteristic: G is a continuous function of a connected noise space, so it cannot map onto eight disjoint islands without passing through the gaps.

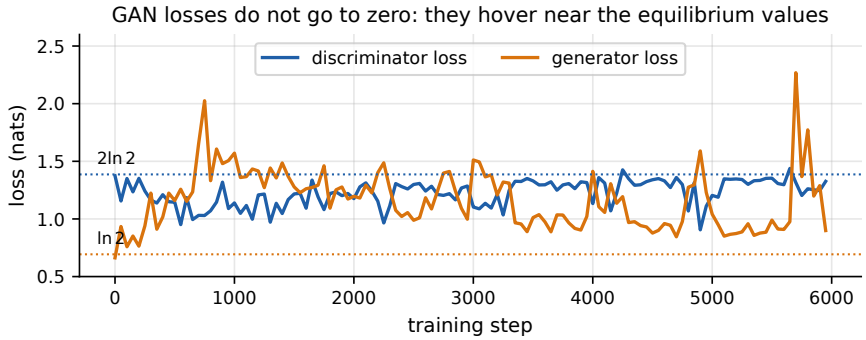


Figure 18.10: Discriminator and generator losses during the same run. Unlike supervised training, nothing converges to zero: the two players keep pushing against each other. The dotted lines mark the theoretical equilibrium values derived above, $2\ln 2$ for the discriminator and $\ln 2$ for the non-saturating generator loss. GAN losses are therefore a poor guide to sample quality — always look at the samples.

How well did it cover the data? Of the 1500 samples at step 6000, 99.9% fell within 0.3 of the circle of radius 2, and every one of the eight modes received samples (between 130 and 239 of them; a perfect generator would give about 188 each). This run covered all the modes; with a different seed, learning rate or a narrower network the same code can collapse onto a few of them, which is worth trying yourself. The final average losses were 1.31 for D and 1.08 for G, close to but not at the equilibrium values: the discriminator still finds a weak edge, mostly on the bridges.

Pitfall: mode collapse

Nothing in (18.5) forces the generator to cover *all* of the data. If it finds one output that currently fools the discriminator, it can map most of its noise there; the discriminator then learns to reject that region, the generator jumps to another mode, and so on. The symptom

is low diversity: a face GAN that draws the same face, or an 8-Gaussian GAN that covers only 3 modes. Remedies include minibatch features that let D see diversity, alternative losses (Wasserstein GAN with gradient penalty), spectral normalisation, and simply tuning learning rates. Always measure coverage — here we counted samples per mode — not just the losses.

Wireless connection: learning channel models with GANs

Some channels have no convenient equation: a power amplifier with memory, a mmWave link with blockage, an industrial site full of metal. If you have measurements of input–output pairs, a *conditional* GAN can learn $p(\mathbf{y} \mid \mathbf{x})$: the generator takes the transmitted signal $\mathbf{x}$ and noise $\mathbf{z}$ and outputs a realistic received $\mathbf{y}$; the discriminator sees $(\mathbf{x}, \mathbf{y})$ pairs, real or generated. O'Shea, Roy and West (2019) used this idea to approximate channel responses, and Ye, Li and Juang (2018) used a conditional GAN as a *differentiable* surrogate channel so that an end-to-end autoencoder (Figure 18.4) could be trained even when the real channel provides no gradients. Diffusion models are now used for the same purpose and for generating synthetic CSI to augment small measurement campaigns. The risk is the same as mode collapse: a surrogate channel that misses rare deep fades will produce a receiver that has never seen them.

18.4 Diffusion models

Intuition: un-mixing ink

Drop ink into water and watch it spread until the glass is uniformly grey. That process is easy, and it destroys the shape of the drop step by small step. Now imagine a film of it played backwards: the grey slowly gathers into a crisp drop. A diffusion model learns to play that film backwards. During training we add a little noise to images many times over until nothing but noise is left — easy, no learning needed. Then a network learns to undo *one small step* of noising. To generate, start from pure noise and apply the learned un-noising step again and again.

18.4.1 The forward process

The denoising diffusion probabilistic model (DDPM, Ho, Jain and Abbeel, 2020) fixes a **noise schedule** $\beta_1, \dots, \beta_T$, small numbers between 0 and 1, and defines a Markov chain that starts at a data point $\mathbf{x}_0$ and gradually corrupts it:

$$\mathbf{x}_t = \sqrt{1-\beta_t}\,\mathbf{x}_{t-1} + \sqrt{\beta_t}\,\boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad t = 1, \dots, T. \tag{18.7}$$

Each step shrinks the signal slightly (factor $\sqrt{1-\beta_t}$) and adds a little fresh noise (variance β_t). The shrinking is chosen so that the total variance stays at 1: if $\mathrm{Var}(\mathbf{x}_{t-1}) = 1$ then $\mathrm{Var}(\mathbf{x}_t) = (1-\beta_t) + \beta_t = 1$. Define

$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \alpha_1 \alpha_2 \cdots \alpha_t.$$

18.4.2 Jumping to any step: the closed form

To train we need $\mathbf{x}_t$ at random t, and running t steps of (18.7) each time would be slow. Fortunately, there is a shortcut. Unroll two steps:

$$\begin{aligned}\mathbf{x}_t &= \sqrt{\alpha_t}\,\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_t \\ &= \sqrt{\alpha_t}\big(\sqrt{\alpha_{t-1}}\,\mathbf{x}_{t-2} + \sqrt{1-\alpha_{t-1}}\,\boldsymbol{\epsilon}_{t-1}\big) + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_t \\ &= \sqrt{\alpha_t\alpha_{t-1}}\,\mathbf{x}_{t-2} + \underbrace{\sqrt{\alpha_t(1-\alpha_{t-1})}\,\boldsymbol{\epsilon}_{t-1} + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_t}_{\text{sum of two independent zero-mean Gaussians}}.\end{aligned}$$

A sum of independent zero-mean Gaussians is a zero-mean Gaussian whose variance is the sum of the variances:

$$\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t) = \alpha_t - \alpha_t\alpha_{t-1} + 1 - \alpha_t = 1 - \alpha_t\alpha_{t-1}.$$

So $\mathbf{x}_t = \sqrt{\alpha_t\alpha_{t-1}}\,\mathbf{x}_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\,\bar{\boldsymbol{\epsilon}}$ with a single $\bar{\boldsymbol{\epsilon}} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. The pattern is clear: after each further step the product of α's grows by one factor. By induction, all the way back to $\mathbf{x}_0$:

Key equation: forward process in closed form

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{18.8}$$

A wireless engineer reads this immediately: $\mathbf{x}_t$ is the clean signal received at a signal-to-noise ratio $\mathrm{SNR}_t = \bar{\alpha}_t/(1-\bar{\alpha}_t)$, which falls from very high at $t = 1$ to nearly zero at $t = T$. Diffusion is a sweep across SNR.

Worked example: the schedule in numbers

We use $T = 100$ steps with β_t rising linearly from 10^{-4} to 0.05 (Ho et al. used $T = 1000$ and β from 10^{-4} to 0.02). Then $\bar{\alpha}_1 = 0.9999$, $\bar{\alpha}_{10} = 0.977$, $\bar{\alpha}_{50} = 0.534$ and $\bar{\alpha}_{100} = 0.078$. At the last step the signal amplitude factor is $\sqrt{0.078} = 0.28$ and the noise factor $\sqrt{1-0.078} = 0.96$: an SNR of $0.078/0.922 = 0.085$, about -10.7 dB — almost pure noise.

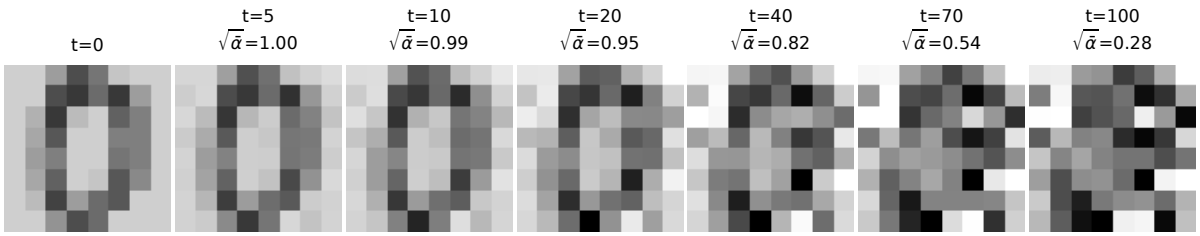


Figure 18.11: The forward process (18.8) applied to an 8×8 digit 0 (pixels scaled to $[-1, 1]$), using the same noise $\boldsymbol{\epsilon}$ for every panel so you can follow it. The digit is still obvious at $t = 20$ ($\sqrt{\bar{\alpha}} = 0.95$), hard to see at $t = 70$, and essentially gone at $t = 100$. No learning is involved in this direction.

18.4.3 Learning to reverse: the noise-prediction loss

The reverse step $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is modelled as a Gaussian whose mean is computed by a network. Ho et al. showed that the variational bound (the same ELBO idea as for the VAE, applied to the

whole chain) simplifies, after reparameterising, to a strikingly simple recipe: train a network $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ to *predict the noise* that was added.

Key equation: DDPM training loss ("simple" form)

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0,\, t\sim\mathcal{U}\{1..T\},\, \boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\Big[\big\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\big(\sqrt{\bar\alpha_t}\,\mathbf{x}_0 + \sqrt{1-\bar\alpha_t}\,\boldsymbol{\epsilon},\; t\big)\big\|^2\Big]. \tag{18.9}$$

In words: take a clean example, pick a random step t, noise it in one jump with (18.8), and ask the network which noise you used. Predicting $\boldsymbol{\epsilon}$ is equivalent to predicting $\mathbf{x}_0$, since $\mathbf{x}_0 = (\mathbf{x}_t - \sqrt{1-\bar\alpha_t}\,\boldsymbol{\epsilon})/\sqrt{\bar\alpha_t}$; it is a denoising autoencoder trained at every SNR at once, told the SNR through t.

18.4.4 Sampling

Start from $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0},\mathbf{I})$ and for $t = T, T-1, \dots, 1$ compute

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar\alpha_t}}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sqrt{\beta_t}\,\mathbf{w}, \qquad \mathbf{w}\sim\mathcal{N}(\mathbf{0},\mathbf{I}), \tag{18.10}$$

with no added noise at the final step ($t = 1$). The bracket removes the predicted noise contribution of step t and rescales the signal; the last term re-injects a little fresh randomness, which keeps samples diverse (it is the variance of the reverse step; Ho et al. found $\sigma_t^2 = \beta_t$ works well). The cost is T network evaluations per sample, which is why faster samplers are an active research area.

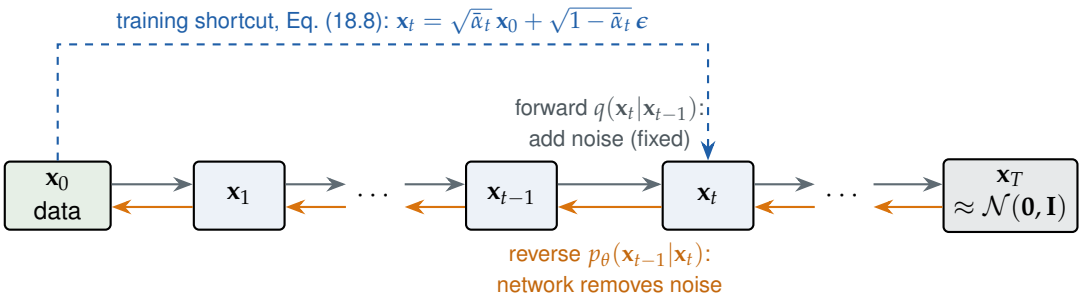


Figure 18.12: Diffusion as two Markov chains. Grey (top): the fixed forward process adds noise step by step. Orange (bottom): the learned reverse process removes it, one step at a time, using the noise-prediction network in Eq. (18.10). Blue dashed: during training we never run the chain; Eq. (18.8) jumps from $\mathbf{x}_0$ to any $\mathbf{x}_t$ directly.

18.4.5 A tiny diffusion model on eight Gaussians

We train an MLP (three hidden layers of 128 SiLU units) whose input is the 2-D point $\mathbf{x}_t$ concatenated with a 16-number sinusoidal embedding of t (the same idea as the positional encoding of Chapter 17). Training uses (18.9) with batches of 512 for 6000 steps.

```
losses = []
for step in range(6000):
    x0b = sample_8gauss(512)
    t = torch.randint(1, T + 1, (512,))
    e = torch.randn_like(x0b)
    ab = abar[t - 1][:, None]
    xt = ab.sqrt() * x0b + (1 - ab).sqrt() * e    # jump to step t (closed form)
```

```
    loss = ((model(xt, t) - e) ** 2).mean()      # predict the added noise
    opt.zero_grad(); loss.backward(); opt.step(); sched.step()
```

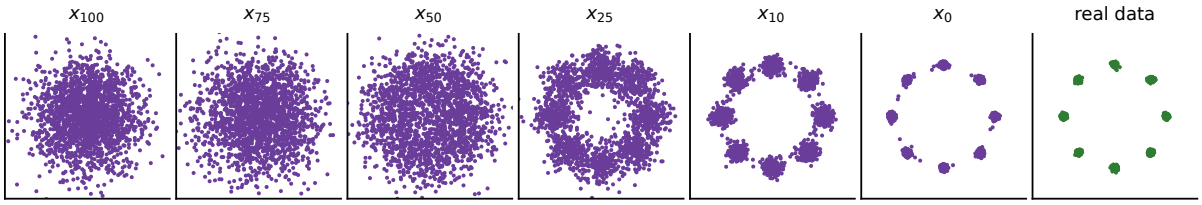


Figure 18.13: Sampling from our trained diffusion model: 2000 points start as Gaussian noise $\mathbf{x}_{100}$ and are denoised with Eq. (18.10). Little happens until about $t = 50$; by $t = 25$ a ring has formed; by $t = 10$ eight clusters have separated; at $t = 0$ the samples sit on the eight modes, compared with real data on the right. All eight modes are populated — diffusion models are much less prone to mode collapse than GANs, because every training example contributes to the loss at every noise level.

Quantitatively, all 2000 generated samples lay within 0.3 of the radius-2 circle and the eight modes received between 223 and 274 samples each (ideal: 250). The diffusion model needed 100 network evaluations per sample, the GAN only one — the price of its stability and coverage.



Figure 18.14: Training loss of the diffusion model. It does not go to zero and should not: at large t the noise dominates and is fundamentally unpredictable from $\mathbf{x}_t$ alone in the parts of the space where several modes overlap, so the irreducible error is substantial. The loss is a useful health check, but sample quality is judged by looking at samples.

	VAE	**GAN**	**Diffusion**
Training signal	ELBO (likelihood bound)	adversarial game	noise-prediction MSE
Training stability	stable	delicate	stable
Sample sharpness	often blurry	sharp	sharp
Mode coverage	good	risk of collapse	good
Sampling cost	1 network pass	1 network pass	T passes (10s–1000)
Latent space	yes, low-dim, smooth	yes (noise $\mathbf{z}$)	same size as data

Table 18.1: The three generative families at a glance.

18.5 Transfer learning

Intuition: you do not learn to drive a truck from zero

Someone who drives a car learns to drive a van in an afternoon: steering, braking, reading the road all carry over; only the size and the mirrors are new. Neural networks are similar. The early layers of an image network learn edges, corners and textures that are useful for almost any image task; only the last layers are specific to "cat versus dog". Transfer learning reuses the general layers and retrains the specific ones, so a new task needs far less data.

Definition: transfer learning

A model is first **pretrained** on a source task A with plenty of data; its weights are then reused for a target task B. Two main strategies:

- **Feature extraction**: freeze the pretrained body (all layers except the output layer), replace the output layer by a fresh one for task B, and train only that new layer. Cheap, robust with very little data, but cannot adapt the features.
- **Fine-tuning**: start from the pretrained weights, replace the output layer, and train *all* layers on task B, typically with a smaller learning rate so the useful features are adjusted rather than destroyed.

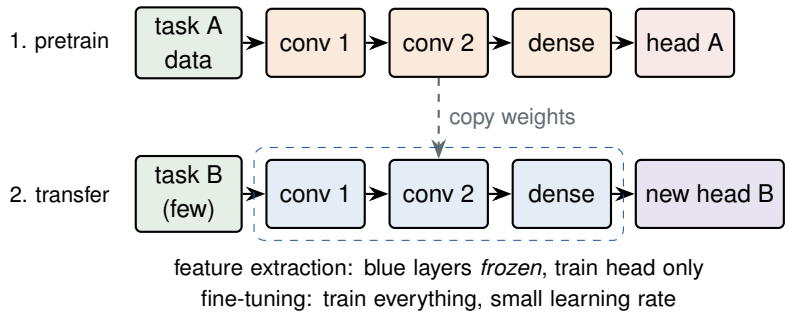


Figure 18.15: Transfer learning. A network trained on a large task A donates its body (the feature layers) to task B; only the output head is replaced. Whether the body is frozen or fine-tuned is the key choice.

18.5.1 The standard recipe with torchvision

With real images the usual starting point is a network pretrained on ImageNet (1.28 million images, 1000 classes). In PyTorch the `torchvision` package provides such models. The code below shows both strategies for a ResNet-18 adapted to a 5-class problem. (It downloads the pretrained weights on first use, so we print it for reference; our experiment below is fully offline.)

```
import torch, torch.nn as nn
from torchvision.models import resnet18, ResNet18_Weights

# pretrained on ImageNet; then a new head: 512 -> 5 classes
model = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
model.fc = nn.Linear(model.fc.in_features, 5)

# (a) feature extraction: freeze everything except the new head
```

```python
for name, p in model.named_parameters():
    p.requires_grad = name.startswith("fc.")
opt = torch.optim.Adam(model.fc.parameters(), lr=1e-3)

# (b) fine-tuning: train all layers, smaller lr for the pretrained body
for p in model.parameters():
    p.requires_grad = True
body = [p for n, p in model.named_parameters() if not n.startswith("fc.")]
opt = torch.optim.Adam([{"params": body, "lr": 1e-4},
                        {"params": model.fc.parameters(), "lr": 1e-3}])
# Inputs must be normalised as in pretraining:
# transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
```

Pitfall: preprocessing and BatchNorm

A pretrained network expects inputs prepared exactly as during pretraining (same size, same normalisation). And if you freeze a body that contains batch normalisation, also keep it in evaluation mode (`model.eval()` for those layers), otherwise its running statistics drift toward your small new dataset even though the weights are "frozen".

18.5.2 Experiment 1: when transfer does *not* help

To see transfer learning at work we need two related tasks. A first, natural-sounding idea: split the scikit-learn digits into **task A**, classifying digits 0–4 (901 images, used for pretraining), and **task B**, classifying digits 5–9, where we pretend labels are scarce. We hold out 400 task-B images for testing and give each method only n labelled examples per class, $n \in \{1, 2, 5, 10, 20, 50\}$. The network is a small CNN: two 3×3 convolutions (16 and 32 channels), max-pooling, a 64-unit dense layer, and a 5-way head. We compare training from scratch, feature extraction (new head, frozen body) and fine-tuning (new head, all layers, learning rate 3×10^{-4} instead of 10^{-3}), averaging over 5 random draws of the training set.

```python
        # (1) train from scratch
        torch.manual_seed(s)
        m = fit(SmallCNN(), Xtr, ytr, ep, 1e-3)
        res['scratch'][si, ni] = acc(m, Xte, yte)
        # (2) feature extraction: new head, frozen body
        torch.manual_seed(s)
        m = copy.deepcopy(pre); m.head = nn.Linear(64, 5)
        for p in m.body.parameters(): p.requires_grad = False
        m = fit(m, Xtr, ytr, ep, 1e-3, params=m.head.parameters())
        res['feature'][si, ni] = acc(m, Xte, yte)
        # (3) fine-tuning: new head, all layers, smaller learning rate
        torch.manual_seed(s)
        m = copy.deepcopy(pre); m.head = nn.Linear(64, 5)
        m = fit(m, Xtr, ytr, ep, 3e-4)
        res['finetune'][si, ni] = acc(m, Xte, yte)
```

This is not the textbook picture, and it is worth understanding rather than hiding. With 1 example per class, scratch reached 0.73, fine-tuning 0.69 and feature extraction only 0.49; with 50 per class, 0.97, 0.96 and 0.71. A diagnosis (`code/ch18/transfer_probe.py`, with the same architecture pretrained the same way): of the 64 units of the pretrained dense layer, 27 are *never active* on digits 5–9 — the network learned features that discriminate 0–4 and discarded everything else. A simple linear classifier trained on the pretrained *convolutional* features did

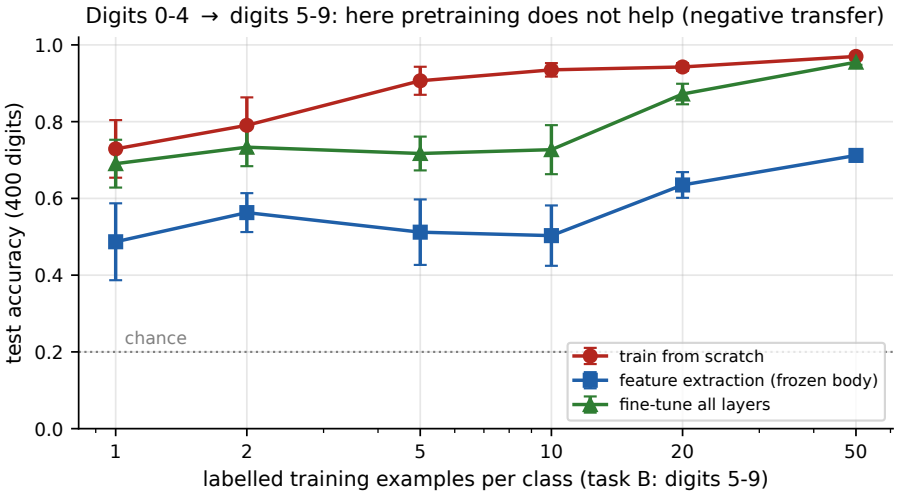


Figure 18.16: Experiment 1 (our run): test accuracy on digits 5–9 against the number of labelled examples per class (mean ± one standard deviation over 5 random training sets). Training from scratch is best at *every* size; feature extraction from the 0–4 network is worst; fine-tuning recovers only with 20–50 examples per class. Pretraining on a different set of classes did not help here.

about as well as one on raw pixels or on the features of a *randomly initialised* network (around 0.70 at 1 example per class, 0.89 at 5). Two lessons follow. First, a network trained on only five classes and 900 images learns narrow features; ImageNet pretraining works because 1000 classes and a million images force *general* features. Second, the later a layer, the more task-specific it is (Yosinski et al., 2014), so freezing the whole body is the riskiest choice when the tasks differ. When transfer makes results worse it is called **negative transfer**.

18.5.3 Experiment 2: same task, different conditions (a sim-to-real analogue)

The situation that matters most in wireless is different: the *task* stays the same (demodulate, estimate, classify) but the *conditions* change — from simulation to hardware, from one channel to another. We mimic this with digits. The source ("simulation") is 1000 clean digits of all ten classes. The target ("over the air") is the other 797 digits passed through a "channel": a 3×3 blur, a random gain between 0.6 and 1.0, and additive Gaussian noise of standard deviation 0.15 (Figure 18.17). Now the classes are the same, so the pretrained 10-way head is kept, and we compare: training from scratch on the few impaired examples, retraining only the head, and fine-tuning all layers, over 3 random draws.

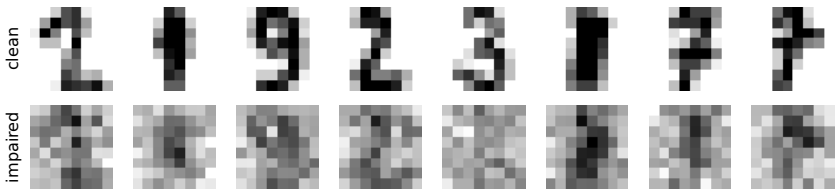


Figure 18.17: Experiment 2 data: clean source digits (top) and the same digits after our simulated "channel" (bottom): blur, random attenuation and additive noise.

With 1 impaired example per class, training from scratch reached 0.38, retraining the head 0.66 and fine-tuning all layers 0.69; with 20 per class, 0.71, 0.74 and 0.77. Fine-tuning is best throughout, and its advantage over scratch is largest exactly where data are scarcest — a gap of

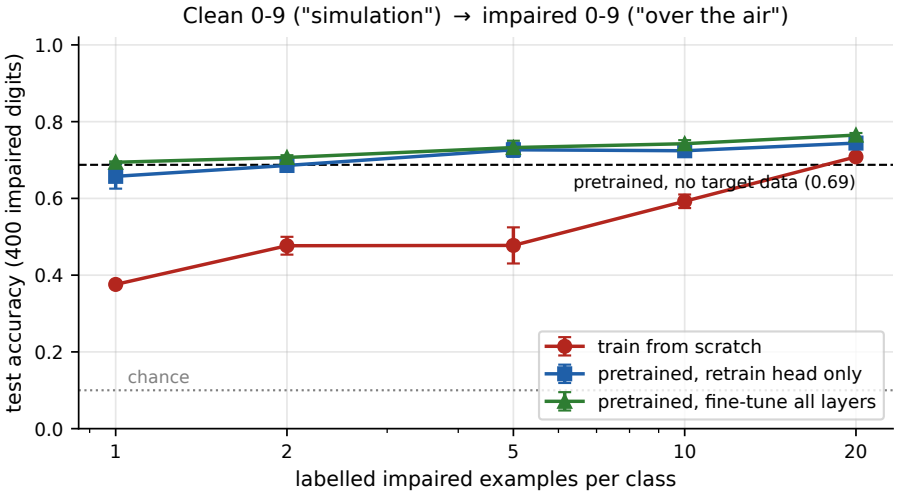


Figure 18.18: Experiment 2 (our run): accuracy on 400 impaired test digits. The network trained on clean digits already scores 0.69 on impaired digits without seeing any of them (dashed line). A little target data lifts both transfer methods above it; training from scratch needs about 20 examples per class just to reach the same level. Here transfer is a clear win.

31 percentage points at one example per class, shrinking to 6 at twenty. This is the regime transfer learning is for.

Wireless connection: transfer across SNR, channels, and from simulation to over the air

Transfer learning is everywhere in wireless deep learning because the *target* conditions are never exactly the training conditions:

- **Across SNR.** A demodulator or decoder trained at one SNR degrades at another. Training over a range of SNRs is the first remedy; fine-tuning briefly at the operating SNR is the second. Diffusion training, which literally trains one network at every noise level with the level as an input, is a useful pattern to borrow: feed the estimated SNR to the network.
- **Across channels.** A CSI-feedback autoencoder or channel estimator trained on one 3GPP channel model (say CDL-A, indoor) and deployed in another (CDL-D, urban) loses accuracy. Fine-tuning on a few hundred samples from the new environment typically recovers most of it; freezing early layers and retraining later ones is common.
- **Simulation to over the air.** The cheapest data come from a simulator, but real hardware adds phase noise, IQ imbalance, PA nonlinearity and timing offsets that the simulator idealises. The standard workflow is to pretrain on large amounts of simulated data and fine-tune on a smaller over-the-air capture — exactly our Experiment 2, with "clean digits" replaced by "simulated frames" and "impaired digits" by "measured frames". Dörner, Cammerer, Hoydis and ten Brink (2018) did this for the autoencoder of Figure 18.4: trained on a channel model, then the receiver was fine-tuned on real measurements from software-defined radios. Chapter 24 returns to it.

In the 3GPP Release 18 study on AI/ML for the air interface (Chapter 17), the ability of a model to *generalise* across scenarios, and how to monitor and update it when it does not, was one of the central evaluation questions.

Chapter recap

- An autoencoder learns $\mathbf{x} \to \mathbf{z} \to \hat{\mathbf{x}}$ by minimising reconstruction error; a bottleneck ($d < D$) or input noise (denoising) forces it to learn structure. A 2-D code of digits clusters by class without labels.
- The communication chain is an autoencoder: transmitter = encoder, channel = noise layer, receiver = decoder (Chapter 21).
- A VAE makes the latent space a standard Gaussian. It maximises the ELBO $= \mathbb{E}_q[\log p_\theta(\mathbf{x}|\mathbf{z})] - \mathrm{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \| \mathcal{N}(\mathbf{0}, \mathbf{I}))$, with $\mathrm{KL} = \frac{1}{2}\sum(\mu^2 + \sigma^2 - 1 - \log\sigma^2)$, trained through the sample $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$.
- A GAN plays $\min_G \max_D \mathbb{E}[\log D(\mathbf{x})] + \mathbb{E}[\log(1 - D(G(\mathbf{z})))]$. The optimal discriminator is $D^* = p_{\text{data}}/(p_{\text{data}} + p_g)$, and then G minimises $2\,\mathrm{JS} - 2\log 2$. Use the non-saturating generator loss; watch for mode collapse.
- DDPM: forward $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}$ (an SNR sweep); train $\boldsymbol{\epsilon}_\theta$ to predict the noise; sample by T small denoising steps from pure noise.
- Transfer learning: reuse a pretrained body, then retrain the head (feature extraction) or everything with a small learning rate (fine-tuning). It pays off when source features are general and target data are scarce — in our same-task, shifted-conditions experiment fine-tuning beat scratch by 31 points at one example per class — but narrow pretraining on different classes can give negative transfer.

Exercises

1. **(Numeric)** An encoder outputs $\boldsymbol{\mu} = (0.5, -1)$ and $\log\boldsymbol{\sigma}^2 = (0, -2)$. Compute the KL term (18.3). Which dimension is "paying" more and why?
2. **(Numeric)** With $\mu = 2$, $\sigma = 0.1$ and noise draw $\epsilon = 1.5$, what is the reparameterised z? What are $\partial z/\partial\mu$ and $\partial z/\partial\sigma$?
3. **(Conceptual)** Why does a plain autoencoder make a poor generator, and which term in the VAE loss fixes this? What happens if you multiply the KL term by a very large weight?
4. **(Numeric)** At some $\mathbf{x}$, $p_{\text{data}}(\mathbf{x}) = 0.1$ and $p_g(\mathbf{x}) = 0.4$. What is $D^*(\mathbf{x})$? What should the generator do there?
5. **(Derivation)** Show that $\mathrm{JS}(p\|q) \le \log 2$, with equality when p and q do not overlap. Use this to explain why the original GAN loss gives the generator little useful gradient when real and fake distributions are far apart.
6. **(Numeric)** For a DDPM with constant $\beta_t = 0.02$, compute $\bar{\alpha}_t$ for $t = 10$ and $t = 100$ and the corresponding SNR in dB.
7. **(Conceptual)** In Experiment 1, feature extraction was the worst method, while in Experiment 2 retraining only the head was nearly as good as fine-tuning. Explain the difference. What would you change in Experiment 1 to make pretraining more useful?
8. **(Conceptual, wireless)** You have 10^6 simulated and 2000 measured received frames for training a neural demodulator. Outline a transfer-learning plan, including what you would freeze and how you would decide whether it worked.

Answers

1. Dimension 1: $\mu^2 = 0.25$, $\sigma^2 = e^0 = 1$, $\log\sigma^2 = 0$: $\frac{1}{2}(0.25 + 1 - 1 - 0) = 0.125$. Dimension 2: $\mu^2 = 1$, $\sigma^2 = e^{-2} = 0.1353$, $\log\sigma^2 = -2$: $\frac{1}{2}(1 + 0.1353 - 1 + 2) = 1.568$. Total 1.693 nats. Dimension 2 pays more: its mean is further from 0 and its variance is much smaller than 1, i.e. it stores more precise information about $\mathbf{x}$.
2. $z = 2 + 0.1 \times 1.5 = 2.15$; $\partial z/\partial\mu = 1$, $\partial z/\partial\sigma = \epsilon = 1.5$.
3. Its latent codes occupy an irregular region with gaps; there is no known distribution to sample $\mathbf{z}$ from, and points in gaps decode to nonsense. The KL term pulls every $q_\phi(\mathbf{z}|\mathbf{x})$ toward $\mathcal{N}(\mathbf{0}, \mathbf{I})$ so the codes fill a known ball. With a huge KL weight the encoder sets $q = \mathcal{N}(\mathbf{0}, \mathbf{I})$ for every input (KL $= 0$), $\mathbf{z}$ carries no information, and the decoder outputs the average digit ("posterior collapse").
4. $D^* = 0.1/(0.1 + 0.4) = 0.2$. The generator puts too much mass here (4 times the data density): it should move samples away from this region toward regions where $D^* > \frac{1}{2}$.
5. With $m = \frac{1}{2}(p + q)$, $p \le 2m$ so $\log(p/m) \le \log 2$ and $\mathrm{KL}(p\|m) = \mathbb{E}_p[\log(p/m)] \le \log 2$; likewise $\mathrm{KL}(q\|m) \le \log 2$, so JS $\le \log 2$. If p and q have disjoint supports then wherever $p > 0$, $m = p/2$ and $\log(p/m) = \log 2$ exactly, so JS $= \log 2$. When the fakes are far from the data, JS is stuck at its maximum $\log 2$ no matter how far: moving fakes closer (but still not overlapping) does not change the loss, so there is no gradient to follow. (This motivated the non-saturating loss and the Wasserstein GAN.)
6. $\alpha = 0.98$; $\bar{\alpha}_{10} = 0.98^{10} = 0.817$, SNR $= 0.817/0.183 = 4.47$, i.e. 6.5 dB. $\bar{\alpha}_{100} = 0.98^{100} = 0.133$, SNR $= 0.133/0.867 = 0.153$, i.e. -8.1 dB.
7. In Experiment 1 the frozen body had learned features that separate 0–4 only (27 of its 64 dense units were never active on 5–9), so a head on top of them had little to work with; the tasks share pixels but not the features the narrow pretraining kept. In Experiment 2 the task is identical and only the input statistics change, so the pretrained features (strokes, loops) remain the right ones and even the pretrained head is already good (0.69 with no target data); a little target data merely adapts it. To help Experiment 1: pretrain on a broader task (more classes, more data, or a self-supervised task such as the autoencoder of Section 18.1), freeze only the early convolutional layers, and fine-tune the rest.
8. Pretrain the full network on the 10^6 simulated frames (with a range of SNRs and impairments — phase noise, CFO, IQ imbalance — to cover the real hardware). Split the 2000 measured frames into fine-tuning (e.g. 1500) and test (500) sets, never mixing them. Try (a) the simulated model as-is, (b) retraining only the last layers, (c) fine-tuning all layers with a learning rate about 10 times smaller, with early stopping on a validation subset. Judge by BER/BLER versus SNR on the held-out measured frames, compared with a classical receiver and with a network trained only on the 2000 measured frames.

Self-test quiz

1. What prevents an undercomplete autoencoder from learning the identity function?
2. Write the two terms of the ELBO and say what each encourages.
3. Why is the reparameterisation trick needed?

4. What is the optimal discriminator in a GAN, and what value does it take when the generator is perfect?
5. In DDPM, what does the network predict, and what is the closed-form expression for $\mathbf{x}_t$?
6. Name one situation in which transfer learning can make results *worse* than training from scratch.

Answers: 1. The bottleneck: the code has fewer dimensions than the input, so information must be compressed. 2. $\mathbb{E}_q[\log p_\theta(\mathbf{x}|\mathbf{z})]$ *encourages accurate reconstruction;* $-\mathrm{KL}(q_\phi(\mathbf{z}|\mathbf{x})\|\mathcal{N}(\mathbf{0},\mathbf{I}))$ *keeps the codes close to the prior so the latent space can be sampled. 3. Sampling* $\mathbf{z}$ *is not differentiable with respect to* ϕ*; writing* $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$ *moves the randomness into* $\boldsymbol{\epsilon}$ *so gradients flow through* $\boldsymbol{\mu}, \boldsymbol{\sigma}$*. 4.* $D^* = p_{data}/(p_{data} + p_g)$*; it equals* $\frac{1}{2}$ *everywhere. 5. The noise* $\boldsymbol{\epsilon}$ *added to* $\mathbf{x}_0$*;* $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$*. 6. When the source features are too narrow or unrelated to the target task (negative transfer), e.g. a body pretrained on a few different classes, especially if it is frozen.*

Further reading

- D. P. Kingma, M. Welling, "Auto-encoding variational Bayes," *ICLR*, 2014. The VAE and the reparameterisation trick.
- P. Vincent, H. Larochelle, Y. Bengio, P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," *ICML*, 2008.
- I. Goodfellow et al., "Generative adversarial nets," *NeurIPS*, 2014. The minimax game, D^* and the JS result.
- M. Arjovsky, S. Chintala, L. Bottou, "Wasserstein generative adversarial networks," *ICML*, 2017.
- J. Ho, A. Jain, P. Abbeel, "Denoising diffusion probabilistic models," *NeurIPS*, 2020.
- J. Yosinski, J. Clune, Y. Bengio, H. Lipson, "How transferable are features in deep neural networks?" *NeurIPS*, 2014.
- T. O'Shea, J. Hoydis, "An introduction to deep learning for the physical layer," *IEEE Trans. Cognitive Communications and Networking*, 2017; S. Dörner, S. Cammerer, J. Hoydis, S. ten Brink, "Deep learning based communication over the air," *IEEE J. Selected Topics in Signal Processing*, 2018.
- H. Ye, G. Y. Li, B.-H. F. Juang, D. Sivanesan, "Channel agnostic end-to-end learning based communication systems with conditional GAN," *IEEE Globecom Workshops*, 2018; T. J. O'Shea, T. Roy, N. West, "Approximating the void: Learning stochastic channel models from observation with variational generative adversarial networks," *ICNC*, 2019.

Chapter 19

Reinforcement Learning: From Bandits to Deep Q-Networks and Policy Gradients

Every network so far in this book learned from a teacher: someone handed it an input together with the correct answer, and the loss measured the distance between the two. Many of the most interesting problems in radio do not come with such a teacher. Which of 64 beams should the base station try next? On which of 16 channels should a cognitive radio transmit when it can only listen to one at a time? How much power should a handset spend when the channel keeps changing? Nobody can label the "correct" action, because whether an action was good is only revealed *afterwards*, by the success or failure of what followed. *Reinforcement learning* (RL) is the mathematics of learning from such consequences.

In this chapter you will build RL from nothing: the agent–environment loop; the multi-armed bandit, which is RL without state; Markov decision processes and the Bellman equations, derived line by line and solved by hand on a small gridworld; model-free learning by Monte Carlo, TD(0), Q-learning and SARSA; deep Q-networks (DQN) with experience replay and a target network; partially observable problems with belief states, the exact setting of dynamic multichannel access; and finally policy gradients, derived from the log-derivative trick, including the Gaussian exploration policy that later lets us train a radio transmitter *without* a channel model. Six experiments, all small enough to run on a laptop, make every idea visible. This chapter is the foundation for three paper chapters in Part V: deep RL for dynamic multichannel access, online learning in BitTorrent, and model-free end-to-end learning of communication systems.

19.1 Learning from consequences

Intuition: training a dog

You cannot explain to a dog in words what "sit" means. Instead you wait until it happens to sit, and you give it a treat. Very quickly the dog notices that the treat is more likely after sitting, and it starts to sit more often. Notice the three ingredients. (i) The dog *acts* first and is judged afterwards; there is no demonstration of the correct move. (ii) The treat may come some time after the useful action, so the dog must work out which of its recent actions earned it (the *credit assignment* problem). (iii) If the dog only ever repeated the first trick that earned a treat, it would never discover a better one; it has to *try things* (exploration). Reinforcement learning is exactly this, written in mathematics.

The learner is called the **agent**; everything it interacts with is the **environment**. Time runs in discrete steps $t = 0, 1, 2, \ldots$ (for a radio, these are naturally time slots). At each step the agent observes a **state** S_t, chooses an **action** A_t, and the environment responds with a numerical **reward** R_{t+1} and a new state S_{t+1}. Figure 19.1 shows the loop. The agent's goal is to choose actions so that the *sum of future rewards* is as large as possible.

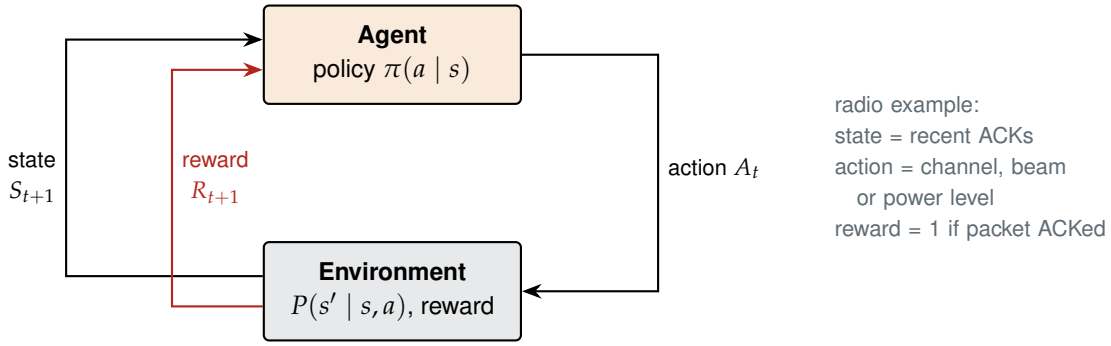


Figure 19.1: The agent–environment loop. At every step the agent sees a state, picks an action using its policy, and the environment returns a reward and the next state. Nothing tells the agent which action would have been best: it only sees the reward of the action it actually took.

19.1.1 How this differs from supervised learning

It helps to put the two side by side.

- **No labels, only evaluations.** In supervised learning the loss says *what* the right answer was. In RL the reward says only *how good* the chosen action turned out to be. A reward of 0.3 does not tell you whether another action would have given 0.9.
- **Delayed reward.** A move in chess, or a choice to spend battery power now, may only pay off many steps later. The learner must propagate credit backwards in time.
- **The data depend on the learner.** The agent's own choices decide which situations it sees next, so the training data are neither fixed nor independent. A bad early policy can trap the agent in uninformative regions.
- **Exploration versus exploitation.** To earn reward the agent should use what it already knows (exploit); to learn it must try actions whose value is uncertain (explore). Balancing the two is the central tension of the whole subject, and it appears in its purest form in bandits.

19.2 Multi-armed bandits: RL without state

Intuition: a row of slot machines

Imagine a casino with K slot machines ("one-armed bandits"). Each pays out with its own unknown probability. You have T coins. Every coin spent on a poor machine is a coin not spent on the best one — but you cannot know which one is best without spending coins on several of them. There is no state here: pulling a lever does not change the machines. The only question is whom to trust, and for how long.

Definition: stochastic K-armed bandit

There are K actions (*arms*). Pulling arm a returns a random reward R drawn from an unknown distribution with mean μ_a. Let $\mu^* = \max_a \mu_a$ and let $\Delta_a = \mu^* - \mu_a \geq 0$ be the *gap* of arm a. If

the agent pulls arms $A_1, \dots, A_T$, its (pseudo-)**regret** after T pulls is

$$\mathcal{R}_T = \sum_{t=1}^{T} (\mu^* - \mu_{A_t}) = \sum_a \Delta_a \, \mathbb{E}[N_a(T)], \tag{19.1}$$

where $N_a(T)$ counts how often arm a was pulled. (The second form takes expectations: each pull of arm a "loses" Δ_a on average.) Regret is the reward you gave away by not knowing the best arm in advance.

The second form of (19.1) is illuminating: to have small regret you must pull bad arms rarely, but you cannot know they are bad without pulling them. A policy that stops exploring too early may lock onto a bad arm and suffer regret that grows *linearly* in T. The best possible behaviour is regret that grows like $\log T$: Lai and Robbins showed in 1985 that no reasonable policy can do better than logarithmic growth, so "$\log T$ regret" is the gold standard.

Wireless connection: beam and channel selection as a bandit

A 5G gNB serving a user with K candidate SSB beams, or a cognitive radio choosing among K channels, faces a bandit if the channel statistics are stationary: each beam/channel a delivers the packet with an unknown probability μ_a; the reward is the HARQ ACK (1) or NACK (0). Regret counts the packets lost by using a sub-optimal beam while learning. Our experiment below uses exactly this model with $K = 8$ beams.

All bandit algorithms keep an estimate of every arm's mean. After arm a has been pulled n_a times with rewards $r_1, \dots, r_{n_a}$, the sample mean is $\hat{\mu}_a = \frac{1}{n_a} \sum_i r_i$. You can update it incrementally, without storing all rewards: since $n\hat{\mu}_n = (n-1)\hat{\mu}_{n-1} + r_n$,

$$\hat{\mu}_n = \hat{\mu}_{n-1} + \frac{1}{n} \left(r_n - \hat{\mu}_{n-1} \right). \tag{19.2}$$

Read this as "new estimate = old estimate + step size $\times$ (target $-$ old estimate)". Almost every update rule in this chapter has exactly this shape.

19.2.1 ε-greedy

With probability $1 - \varepsilon$ pull the arm with the largest $\hat{\mu}_a$ (exploit); with probability ε pull a uniformly random arm (explore). It is simple and robust, but notice the flaw: it keeps exploring at rate ε forever, even after the best arm is obvious, so each step costs about $\varepsilon\bar{\Delta}$ where $\bar{\Delta}$ is the average gap. Its regret therefore grows *linearly*. Decaying ε_t (for example $\varepsilon_t \propto 1/t$) fixes this in theory but requires tuning.

19.2.2 UCB1: optimism in the face of uncertainty

Intuition: the benefit of the doubt

When choosing a restaurant, you might give a place you visited only once "the benefit of the doubt" because one bad meal is weak evidence. UCB does this systematically: it scores each arm by an *optimistic* estimate — the highest mean that is still plausible given the data. Arms tried rarely have wide uncertainty and therefore large optimistic scores, so they get tried; once tried often, their score shrinks to their true mean.

UCB1 (Auer, Cesa-Bianchi and Fischer, 2002) first plays each arm once and then, at time t, chooses

Key equation: UCB1

$$A_t = \arg\max_a \left[\hat{\mu}_a + \sqrt{\frac{2 \ln t}{n_a}} \right]. \tag{19.3}$$

The bonus comes from Hoeffding's inequality: for rewards in $[0,1]$, the probability that the true mean exceeds $\hat{\mu}_a + c$ is at most $e^{-2n_a c^2}$. Choosing $c = \sqrt{2 \ln t / n_a}$ makes this probability $e^{-4 \ln t} = t^{-4}$, which shrinks so fast that the optimistic score is almost never below the true mean. The $\ln t$ in the numerator slowly inflates every bonus, so even an arm that looked bad is eventually re-checked; the n_a in the denominator deflates the bonus of well-explored arms.

The bound. Auer et al. proved (their Theorem 1) that for rewards in $[0,1]$ the expected regret of UCB1 after T pulls satisfies

$$\mathcal{R}_T \;\le\; 8 \sum_{a:\Delta_a>0} \frac{\ln T}{\Delta_a} + \left(1 + \frac{\pi^2}{3}\right) \sum_a \Delta_a. \tag{19.4}$$

Two things to notice: regret grows only like $\ln T$, and arms with *small* gaps cost the most, because they are hardest to distinguish from the best arm.

Worked example: one UCB1 decision

After $t = 10$ pulls, arm A has 4 successes in 6 pulls and arm B has 2 successes in 4 pulls. Then $\ln 10 = 2.303$ and

$$\text{A: } \hat{\mu}_A = 0.667, \quad \sqrt{2(2.303)/6} = 0.876, \quad \text{score} = 1.543;$$
$$\text{B: } \hat{\mu}_B = 0.500, \quad \sqrt{2(2.303)/4} = 1.073, \quad \text{score} = 1.573.$$

UCB1 pulls B, even though its sample mean is lower: four pulls are too few to rule out that B is actually the better arm.

19.2.3 Thompson sampling: act on a random belief

Thompson sampling is Bayesian. For Bernoulli rewards (ACK/NACK) we keep, for each arm, a probability distribution over its unknown success probability μ_a. The convenient choice is the **Beta distribution** $\text{Beta}(\alpha, \beta)$ on $[0,1]$, with density proportional to $\mu^{\alpha-1}(1-\mu)^{\beta-1}$ and mean $\alpha/(\alpha+\beta)$. Start from $\text{Beta}(1,1)$ (uniform: "no idea"). After observing reward $r \in \{0,1\}$, Bayes' rule multiplies the prior by the likelihood $\mu^r(1-\mu)^{1-r}$:

$$p(\mu \mid r) \;\propto\; \mu^{\alpha-1}(1-\mu)^{\beta-1} \cdot \mu^r(1-\mu)^{1-r} = \mu^{(\alpha+r)-1}(1-\mu)^{(\beta+1-r)-1},$$

which is again a Beta: $\text{Beta}(\alpha + r, \beta + 1 - r)$. So the posterior is just "α = 1+successes, β = 1+failures". The algorithm: at each step, *draw one sample* $\tilde{\mu}_a \sim \text{Beta}(\alpha_a, \beta_a)$ for every arm, pull the arm with the largest sample, update its counts. An uncertain arm has a wide posterior and so sometimes produces a large sample and gets explored; a confidently bad arm almost never does. Exploration fades automatically, at exactly the rate the evidence justifies.

Worked example: Beta posterior

An arm has given ACK, ACK, NACK, ACK. Starting from $\mathrm{Beta}(1,1)$ the posterior is $\mathrm{Beta}(1+3,1+1) = \mathrm{Beta}(4,2)$, with mean $4/6 = 0.667$ and standard deviation $\sqrt{\alpha\beta/((\alpha+\beta)^2(\alpha+\beta+1))} = \sqrt{8/252} = 0.178$: still very uncertain after four packets.

19.2.4 Experiment: regret curves for beam selection

Figure 19.2 compares the three algorithms on 8 beams with ACK probabilities

$$\mu = (0.30,\ 0.45,\ 0.50,\ 0.55,\ \mathbf{0.70},\ 0.60,\ 0.40,\ 0.35),$$

averaged over 200 independent runs of 30 000 slots (`code/ch19/bandits.py`). The core of the script is short:

```python
def run(policy, rng, eps=0.1):
    n = np.zeros((RUNS, K)); s = np.zeros((RUNS, K))   # pulls / successes per arm
    regret = np.zeros(T); rows = np.arange(RUNS)
    for t in range(T):
        if t < K and policy != 'ts':
            a = np.full(RUNS, t)                       # play every arm once
        elif policy == 'eps':
            explore = rng.random(RUNS) < eps
            a = np.where(explore, rng.integers(K, size=RUNS), np.argmax(s / n,
    axis=1))
        elif policy == 'ucb':
            a = np.argmax(s / n + np.sqrt(2 * np.log(t) / n), axis=1)
        elif policy == 'ts':
            a = np.argmax(rng.beta(1 + s, 1 + n - s), axis=1)  # one posterior
    sample per arm
        r = rng.random(RUNS) < MU[a]                   # Bernoulli reward (ACK
     = 1)
        n[rows, a] += 1; s[rows, a] += r
        regret[t] = np.mean(MU.max() - MU[a])          # expected (pseudo-)
    regret
    return np.cumsum(regret)
```

After 30 000 slots the regrets were 724 (ε-greedy), 519 (UCB1) and 90 (Thompson). Two lessons. First, UCB1 starts *worse* than ε-greedy: its bonus is deliberately conservative, so it explores heavily early on, but it wins in the long run because its exploration dies out. Second, the UCB1 bound (19.4) evaluates to about 2 800 here — true but loose; bounds guarantee the shape ($\log T$), not the constant.

19.2.5 Contextual bandits

Often the best arm depends on a visible *context* $\mathbf{x}_t$: the best beam depends on the user's position, the best modulation and coding scheme on the reported CQI. A **contextual bandit** observes $\mathbf{x}_t$, picks A_t, and gets reward with mean $\mu_a(\mathbf{x}_t)$. It is still a bandit, because the action does not influence the next context. A popular algorithm, LinUCB, models $\mu_a(\mathbf{x}) = \boldsymbol{\theta}_a^{\mathsf{T}}\mathbf{x}$, estimates $\boldsymbol{\theta}_a$ by ridge regression, $\hat{\boldsymbol{\theta}}_a = \mathbf{A}_a^{-1}\mathbf{b}_a$ with $\mathbf{A}_a = \lambda\mathbf{I} + \sum \mathbf{x}\mathbf{x}^{\mathsf{T}}$ and $\mathbf{b}_a = \sum r\mathbf{x}$ over the times arm a was played, and adds the bonus $c\sqrt{\mathbf{x}^{\mathsf{T}}\mathbf{A}_a^{-1}\mathbf{x}}$, the width of the confidence region in the direction of the current context. Replace the linear model by a neural network and you have a "neural bandit". When actions *do* change future contexts, we need the full machinery of the next section.

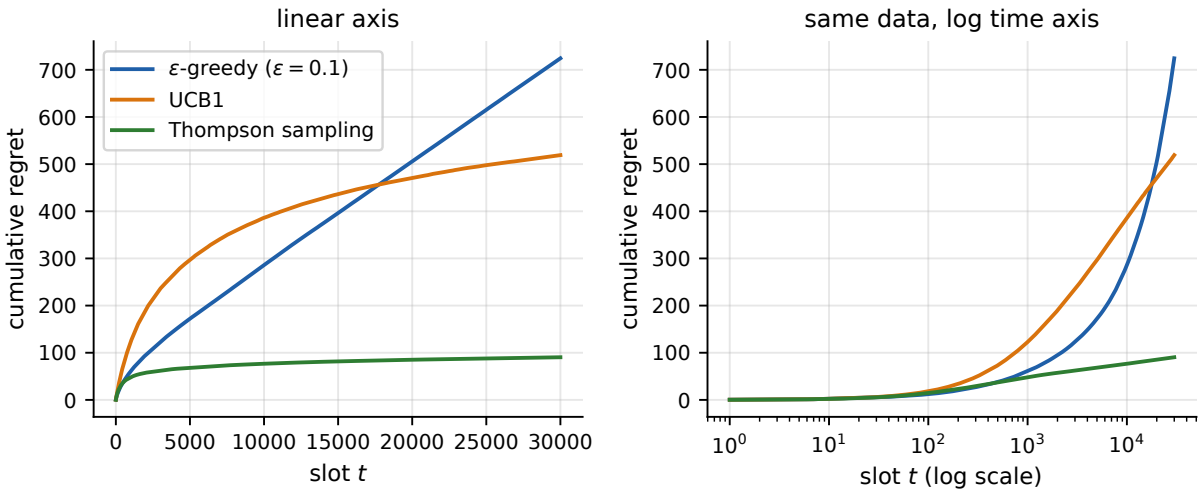


Figure 19.2: Cumulative regret of three bandit algorithms on an 8-beam selection task (mean of 200 runs). Left: ε-greedy grows linearly forever (slope $\approx \varepsilon\bar{\Delta} = 0.022$ packets lost per slot), UCB1 bends over, Thompson sampling is lowest. Right: the same data on a logarithmic time axis — a curve that becomes a straight line here grows like $\log t$; Thompson sampling and UCB1 approach that behaviour while ε-greedy curves upwards.

Wireless connection: rate adaptation and link adaptation

Classical outer-loop link adaptation nudges an SINR offset up after an ACK and down after a NACK so that the BLER converges to a target such as 10%. Viewed as a contextual bandit, the context is the reported CQI and recent HARQ history, the arms are MCS indices, and the reward is the delivered bits (TBS $\times$ ACK). Bandit-based rate adaptation (for example Thompson sampling over MCS indices, exploiting that success probability decreases with MCS) learns the best MCS per context without a hand-tuned offset step.

19.3 Markov decision processes

Bandits ignore the fact that actions change the world. In a maze, a step to the left changes where you are, and that changes what you can do next. Markov decision processes (MDPs) add this state.

Definition: Markov decision process

An MDP is a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

- $\mathcal{S}$: the set of states; $\mathcal{A}$: the set of actions;
- $P(s' \mid s, a)$: the probability of moving to state s' when taking action a in state s;
- $R(s, a, s')$: the expected reward received on that transition;
- $\gamma \in [0, 1)$: the *discount factor*.

The word *Markov* means the next state depends only on the current state and action, not on the earlier history: $P(S_{t+1} \mid S_t, A_t, S_{t-1}, A_{t-1}, \dots) = P(S_{t+1} \mid S_t, A_t)$. The state must therefore contain everything relevant about the past.

19.3.1 Return and discounting

The **return** from time t is the discounted sum of future rewards

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \tag{19.5}$$

Why discount? (i) Mathematically, if rewards are bounded by $R_{\max}$ then $|G_t| \le R_{\max} \sum_k \gamma^k = R_{\max}/(1-\gamma)$, which is finite even for tasks that never end (a radio link runs forever). (ii) Practically, γ sets an *effective horizon* of about $1/(1-\gamma)$ steps: with $\gamma = 0.9$ the agent mostly cares about the next 10 steps, with $\gamma = 0.99$ about 100. (iii) A reward now is more certain than a reward later. The single most useful identity in RL is the recursion obtained by pulling the first term out of (19.5):

$$G_t = R_{t+1} + \gamma \left(R_{t+2} + \gamma R_{t+3} + \dots \right) = R_{t+1} + \gamma G_{t+1}. \tag{19.6}$$

Worked example: a return by hand

With $\gamma = 0.9$ and rewards $R_1 = 0, R_2 = 0, R_3 = 1$ followed by the end of the episode, $G_0 = 0 + 0.9 \cdot 0 + 0.81 \cdot 1 = 0.81$. Using the recursion: $G_2 = 1$, $G_1 = 0 + 0.9 \cdot 1 = 0.9$, $G_0 = 0 + 0.9 \cdot 0.9 = 0.81$. Same answer, computed backwards.

19.3.2 Policies and value functions

A **policy** $\pi(a \mid s)$ is the probability of choosing action a in state s (a deterministic policy puts all its probability on one action, written $a = \pi(s)$). Its quality is measured by two value functions:

$$V^\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right] \qquad \text{(state-value: how good is it to be in } s\text{?)} \tag{19.7}$$

$$Q^\pi(s,a) = \mathbb{E}_\pi \left[G_t \mid S_t = s, A_t = a \right] \qquad \text{(action-value: how good is doing } a \text{ in } s\text{?)} \tag{19.8}$$

The subscript π says that all future actions are drawn from π. The two are linked: averaging Q^π over the policy's own action choice gives V^π,

$$V^\pi(s) = \sum_a \pi(a \mid s)\, Q^\pi(s,a). \tag{19.9}$$

19.3.3 The Bellman expectation equation, derived

Start from the definition of Q^π and use the recursion (19.6):

$$\begin{aligned} Q^\pi(s,a) &= \mathbb{E}_\pi \left[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a \right] \\ &= \sum_{s'} P(s' \mid s,a) \Big(R(s,a,s') + \gamma\, \mathbb{E}_\pi \left[G_{t+1} \mid S_{t+1} = s' \right] \Big) \\ &= \sum_{s'} P(s' \mid s,a) \Big(R(s,a,s') + \gamma\, V^\pi(s') \Big). \end{aligned}$$

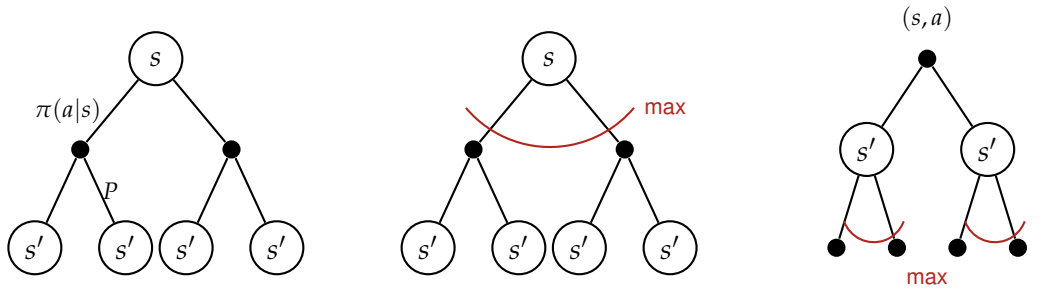
In the second line we averaged over where we land, s'; given s', the Markov property says the future return no longer depends on how we got there, so its expectation is exactly $V^\pi(s')$. Substituting this into (19.9):

Key equation: Bellman expectation equations

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^{\pi}(s') \right], \tag{19.10}$$

$$Q^{\pi}(s, a) = \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma \sum_{a'} \pi(a' \mid s') Q^{\pi}(s', a') \right]. \tag{19.11}$$

In words: *the value of where I am = the reward I expect on the next step + γ times the value of where I expect to land.* For a finite MDP, (19.10) is a system of $|\mathcal{S}|$ linear equations in the $|\mathcal{S}|$ unknowns $V^{\pi}(s)$. We can solve it directly, or iteratively by using the right-hand side as an update rule (*iterative policy evaluation*), which converges because the update is a contraction: it shrinks the distance between any two value estimates by a factor γ every sweep.



(a) V^{π}: average over actions *and* next states

(b) V^{*}: *maximise* over actions, average over next states

(c) Q^{*}: average over s', then max over a'

Figure 19.3: Backup diagrams. Open circles are states, black dots are actions. A value is computed by "backing up" the values of the leaves to the root: averaging where the environment or the policy chooses, maximising where an optimal agent chooses. Diagram (c) is the picture behind Q-learning.

19.3.4 Optimality and the Bellman optimality equation

A policy π^{*} is *optimal* if $V^{\pi^{*}}(s) \geq V^{\pi}(s)$ for every state and every policy. Its value functions are written V^{*} and Q^{*}. The optimal agent, in state s, picks the action with the largest $Q^{*}(s, a)$ and then continues optimally, so $V^{*}(s) = \max_{a} Q^{*}(s, a)$ — the average over π in (19.9) becomes a maximum. Making the same replacement in the derivation above:

Key equation: Bellman optimality equations

$$V^{*}(s) = \max_{a} \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^{*}(s') \right], \tag{19.12}$$

$$Q^{*}(s, a) = \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma \max_{a'} Q^{*}(s', a') \right]. \tag{19.13}$$

Once Q^{*} is known, acting optimally is trivial: $\pi^{*}(s) = \arg\max_{a} Q^{*}(s, a)$. No look-ahead is needed, because Q^{*} already summarises the entire future. That is why learning Q^{*} is so attractive, and it is what Q-learning and DQN do. Unlike (19.10), equation (19.12) is *nonlinear* (because of the max), so we solve it iteratively.

19.3.5 Dynamic programming: value iteration and policy iteration

When P and R are known, two classic algorithms find V^*:

- **Value iteration.** Start with $V_0(s) = 0$ and repeatedly apply the right-hand side of (19.12): $V_{k+1}(s) = \max_a \sum_{s'} P(s' \mid s,a)[R + \gamma V_k(s')]$. Each sweep propagates value information one step further from the rewards. Because this update is a γ-contraction, $V_k \to V^*$ from any start.
- **Policy iteration.** Alternate two steps: (1) *evaluate* the current policy, solving (19.10) for V^π; (2) *improve* it greedily, $\pi'(s) = \arg\max_a \sum_{s'} P[R + \gamma V^\pi(s')]$. The *policy improvement theorem* guarantees $V^{\pi'} \geq V^\pi$ everywhere; since there are finitely many deterministic policies, the loop terminates at π^*, usually after very few iterations.

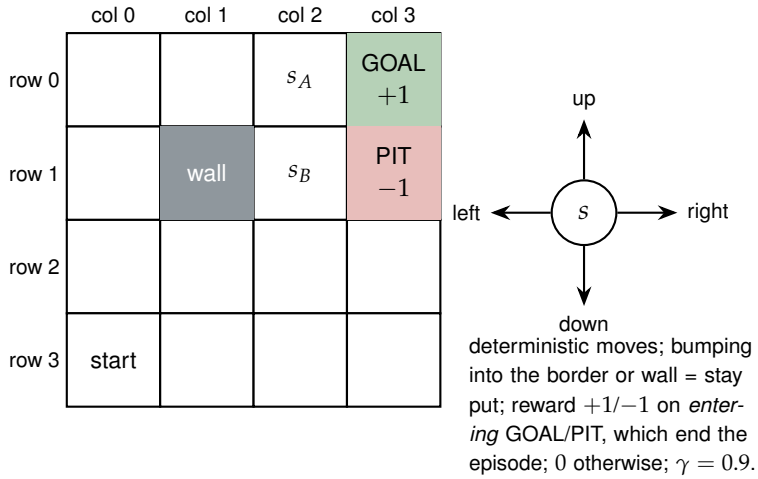


Figure 19.4: The 4×4 gridworld used throughout the chapter. Cells are named (row, col). The agent starts at $(3,0)$; GOAL $(0,3)$ and PIT $(1,3)$ are terminal. Cell $s_A = (0,2)$ is next to the goal, cell $s_B = (1,2)$ is next to both.

Worked example: value iteration by hand on the gridworld

Use the gridworld of Fig. 19.4. Terminal states have value 0 (nothing more can happen). Start with $V_0 \equiv 0$.

Sweep 1. In $s_A = (0,2)$: moving right enters GOAL, giving $1 + 0.9 \cdot 0 = 1$; every other move gives $0 + 0.9 \cdot 0 = 0$. So $V_1(s_A) = 1$. In $s_B = (1,2)$: right enters PIT, -1; up goes to s_A, $0 + 0.9\,V_0(s_A) = 0$; so $V_1(s_B) = \max(-1, 0, 0, 0) = 0$. Every other non-terminal cell also gets 0.

Sweep 2. $V_2(s_B) = \max\big(\underbrace{0 + 0.9 \cdot 1}_{\text{up}}, \underbrace{-1}_{\text{right}}, \dots\big) = 0.9$, and $V_2(0,1) = 0 + 0.9\,V_1(s_A) = 0.9$ (moving right). $V_2(s_A) = 1$ stays.

Sweep 3. $V_3(0,0) = 0.9 \cdot V_2(0,1) = 0.81$ and $V_3(2,2) = 0.9 \cdot V_2(s_B) = 0.81$.

The pattern is clear: value spreads outwards from the goal, one cell per sweep, shrinking by a factor 0.9 per step. A cell from which the goal is k moves away has $V^* = \gamma^{k-1}$, because the $+1$ arrives as reward R_k and is discounted by γ^{k-1}. From the start, the shortest path $(3,0) \to (2,0) \to (1,0) \to (0,0) \to (0,1) \to (0,2) \to$GOAL has six moves, so $V^*(3,0) = \gamma^5 = 0.59049$. (Count carefully: it is easy to be off by one here.) The script confirms $V^*(3,0) = 0.5905$, reached at sweep 6.

Worked example: policy evaluation by hand: one equation of the random policy

Under the uniform random policy ($\pi(a \mid s) = 1/4$), the script finds $V^\pi(s_A) = 0.2698$, $V^\pi(0,1) = 0.1329$ and $V^\pi(s_B) = -0.3146$. Check (19.10) at s_A: up bumps the border (stay in s_A), right enters GOAL, down goes to s_B, left goes to $(0,1)$:

$$\begin{aligned} V^\pi(s_A) &= \tfrac{1}{4}\big[(0 + 0.9 \cdot 0.2698) + (1 + 0) + (0 + 0.9 \cdot (-0.3146)) + (0 + 0.9 \cdot 0.1329)\big] \\ &= \tfrac{1}{4}(1.0793) = 0.2698. \checkmark \end{aligned}$$

The equation is satisfied: the numbers are self-consistent, which is all the Bellman equation demands.

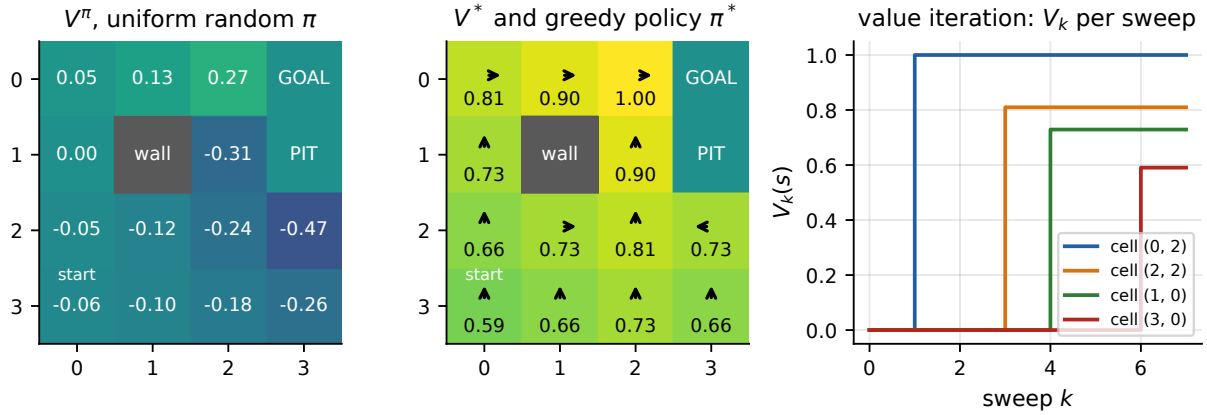


Figure 19.5: Left: value of the uniform random policy — cells near the pit are negative because a random walker often falls in. Middle: optimal values V^* from value iteration, with the greedy policy as arrows; values are exact powers of 0.9 (e.g. $0.59 = 0.9^5$ at the start), and the policy steers around the wall and away from the pit. Right: V_k for four cells during value iteration; each cell "switches on" at the sweep in which the goal's value reaches it, and the whole grid is exact after 6 sweeps.

Policy iteration on the same grid, started from the uniform random policy (whose value at the start is -0.063), already reaches the optimal value at the start, $V(3,0) = 0.5905$, after a *single* improvement step; three further evaluate–improve rounds only adjust the policy elsewhere (and between equally good actions) until it stops changing. Few iterations is typical of policy iteration.

Pitfall: the reward on terminal transitions

Get the terminal convention right in code: when s' is terminal, the target is just r, never $r + \gamma V(s')$ with a stale non-zero $V(s')$. Forgetting this is the most common bug in hand-written RL code, and it silently inflates all values near the end of episodes.

19.4 Model-free learning: Monte Carlo, TD, Q-learning, SARSA

Dynamic programming needs P and R. A radio rarely knows the transition probabilities of its interference environment. *Model-free* methods learn values directly from experienced transitions (s, a, r, s').

Intuition: estimating a commute time

You want to know how long your drive home takes. *Monte Carlo*: drive all the way and write down the total; after many trips, average. *Temporal difference*: at the half-way junction you already know how long the first half took and you have an *estimate* of the rest; if this sum differs from your morning prediction, you update the prediction *now*, without waiting to arrive. TD learns a guess from a guess — this is called *bootstrapping*.

19.4.1 Monte Carlo versus TD(0)

Both methods use the incremental-mean shape (19.2) with a constant step size $\alpha \in (0, 1]$. **Monte Carlo** waits for the end of the episode and moves $V(S_t)$ towards the actual return:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[G_t - V(S_t) \right]. \tag{19.14}$$

TD(0) replaces the unknown G_t by the recursion (19.6) with the current estimate standing in for G_{t+1}:

Key equation: TD(0) update

$$V(S_t) \leftarrow V(S_t) + \alpha \big[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD target}} - V(S_t) \big], \qquad \delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t). \tag{19.15}$$

δ_t is the *TD error*: the surprise between what we predicted and what one step of experience suggests. The trade-off:

- MC targets are *unbiased* (their mean is the true V^π) but have *high variance*: G_t adds up the randomness of a whole episode.
- TD targets are *biased* while V is wrong, but have *low variance* (one random reward and one transition), they can be used online after every step, and they work for continuing tasks that never end.

19.4.2 Q-learning, derived

To *control* (find good actions) without a model we learn Q instead of V, because choosing $\arg\max_a Q(s, a)$ needs no knowledge of P. The Bellman optimality equation (19.13) says that $Q^*(s, a)$ equals the expectation of the random quantity $R + \gamma \max_{a'} Q^*(S', a')$ over the next transition. Each experienced transition (s, a, r, s') gives one *sample* of that quantity. To estimate a mean from samples we use the incremental rule (19.2) with step α, plugging in our current estimate Q for the unknown Q^*:

Key equation: Q-learning (Watkins, 1989)

$$Q(s, a) \leftarrow Q(s, a) + \alpha \Big[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \Big]. \tag{19.16}$$

Watkins and Dayan (1992) proved that tabular Q-learning converges to Q^* with probability 1 provided every state–action pair keeps being visited and the step sizes decay suitably ($\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$). Crucially, the max in the target means Q-learning learns the value of the *optimal* policy whatever exploratory policy generated the data: it is **off-policy**.

Worked example: one Q-learning update with numbers

Let $\alpha = 0.1$, $\gamma = 0.9$. The agent is in s with current estimate $Q(s, \text{right}) = 0.5$. It moves right, gets $r = 0$, and lands in s', where its current estimates are $Q(s', \cdot) = (0.2, 0.8, 0.1, 0.4)$.

$$\begin{aligned} \text{TD target} &= r + \gamma \max_{a'} Q(s', a') = 0 + 0.9 \times 0.8 = 0.72, \\ \text{TD error} &= 0.72 - 0.5 = 0.22, \\ Q(s, \text{right}) &\leftarrow 0.5 + 0.1 \times 0.22 = 0.522. \end{aligned}$$

The estimate moved 10% of the way towards the one-step look-ahead.

19.4.3 SARSA: the on-policy cousin

SARSA uses the quintuple (S, A, R, S', A') and replaces the max by the action A' the agent *actually* takes next:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma Q(s', a') - Q(s, a) \right]. \tag{19.17}$$

This is a sample of the Bellman *expectation* equation (19.11) for the current (exploring) policy, so SARSA learns the value of the policy it follows, exploration included — it is **on-policy**. In a world with cliffs or pits, SARSA learns that "wandering randomly 10% of the time next to the pit is dangerous" and prefers safer paths; Q-learning learns the optimal path assuming perfect behaviour later.

19.4.4 Exploration schedules

The usual behaviour policy is ε-greedy on Q. A constant ε keeps learning about everything but wastes reward forever (as in the bandit); a schedule such as $\varepsilon_e = \max(\varepsilon_{\min}, 0.98^e)$ (episode e) or a linear ramp from 1 to 0.02 explores widely at first and exploits later. Alternatives are *Boltzmann* (softmax) exploration, $\pi(a \mid s) \propto e^{Q(s,a)/\tau}$ with temperature τ, and *optimistic initialisation* (start all Q high so that untried actions look attractive), a cousin of UCB.

19.4.5 Experiment: tabular Q-learning on the gridworld

The heart of `code/ch19/gridworld.py` is a dozen lines; note that lines 62 and 63 are the only difference between Q-learning and SARSA:

```
    Q = {s: np.zeros(4) for s in STATES}; returns = np.zeros(episodes)
    def act(s, eps): return rng.integers(4) if rng.random() < eps else int(np.
    argmax(Q[s] + 1e-9 * rng.random(4)))
    for e in range(episodes):
        eps = eps_sched(e); s = START; a = act(s, eps); G, disc = 0.0, 1.0
        for t in range(max_steps):
            s2, r = step(s, a); G += disc * r; disc *= GAMMA
            a2 = act(s2, eps)
            if s2 in TERMINAL: target = r
            elif method == 'q': target = r + GAMMA * Q[s2].max()          # off-
    policy
            else: target = r + GAMMA * Q[s2][a2]                          # SARSA:
     on-policy
            Q[s][a] += alpha * (target - Q[s][a])
            s, a = s2, a2
            if s in TERMINAL: break
```

```
        returns[e] = G
    return Q, returns
```

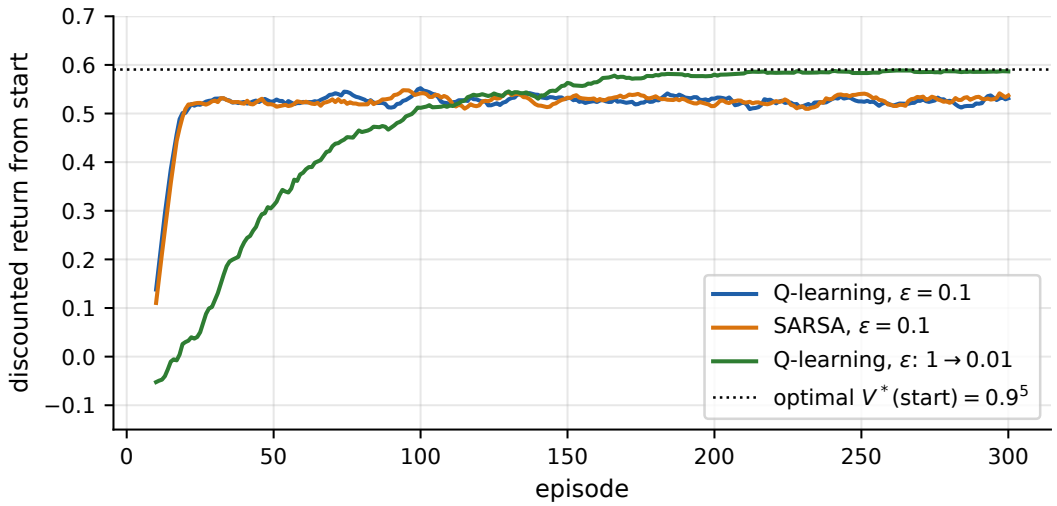


Figure 19.6: Learning curves on the gridworld (discounted return of each training episode, mean of 50 runs, 10-episode moving average; $\alpha = 0.1$). With constant $\varepsilon = 0.1$ both Q-learning and SARSA learn within about 20 episodes but plateau below the optimum 0.59 because they keep taking random actions. With a decaying schedule, Q-learning explores longer at first (slower start) but ends at the optimum.

The script also reads out the learned values. After 300 episodes, Q-learning's $\max_a Q(\text{start}, a)$ equals 0.590 — the true V^* — even though its *behaviour* was ε-greedy; this is off-policy learning at work. SARSA's estimate is 0.484: lower, because it correctly prices in its own future random steps near the pit. Both answers are right; they answer different questions.

19.5 Function approximation and Deep Q-Networks

A table with one entry per state is hopeless when the state is a 84×84 image, or the last 8 sensing results on 16 channels (3^{128} possible histories). We replace the table by a neural network $Q(s, a; \theta)$ with weights θ that takes the state and outputs one Q-value per action. Similar states now share information: the network *generalises*.

19.5.1 From the Q-learning rule to a loss function

We want $Q(s, a; \theta)$ to satisfy the Bellman optimality equation. For a transition (s, a, r, s') define the target

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-), \tag{19.18}$$

where θ^- is a (frozen) copy of the weights, explained below. Treat y as if it were a supervised label and minimise the squared error

Key equation: DQN loss (Mnih et al., 2015)

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\big(y - Q(s, a; \theta)\big)^2\Big], \qquad \nabla_\theta L = -2\,\mathbb{E}\Big[\big(y - Q(s, a; \theta)\big)\nabla_\theta Q(s, a; \theta)\Big]. \tag{19.19}$$

The gradient treats y as a constant (we do not differentiate through θ^-). A gradient step $\theta \leftarrow \theta - \eta \nabla_\theta L$ pushes $Q(s,a;\theta)$ towards y by an amount proportional to the TD error. For a table (one weight per entry, $\nabla_\theta Q$ = indicator of the entry) it reduces *exactly* to the Q-learning rule (19.16) with $\alpha = 2\eta$. Mnih et al. clipped the error term to $[-1,1]$, which is equivalent to using the Huber ("smooth L1") loss instead of the square; we do the same.

19.5.2 Why naive deep Q-learning diverges, and the two fixes

Feeding consecutive transitions into SGD with $\theta^- = \theta$ often blows up. Two reasons, and two remedies:

1. **Correlated samples.** Consecutive transitions are nearly identical (the same corridor, the same channel state). SGD assumes roughly independent samples; a long streak of similar ones drags the network to over-fit the current region and forget the rest. *Fix: experience replay.* Store every transition in a buffer $\mathcal{D}$ of the last N transitions and train on minibatches sampled *uniformly at random* from it. This breaks correlations, and every transition is reused many times (data efficiency), which matters when interactions are expensive, as they are on air.

2. **A moving target.** With $\theta^- = \theta$, every update to $Q(s,a)$ also changes $Q(s',\cdot)$ (neighbouring states share weights), so the target y moves with the very prediction chasing it — like a dog chasing its own tail. Errors feed on themselves through the max. *Fix: target network.* Keep a separate copy θ^- that is updated only every C steps ($\theta^- \leftarrow \theta$). Between updates the regression problem is fixed and stable.

In the Nature paper, the agent received the last four 84×84 grey-scale frames of an Atari game, used a convolutional network with one output per joystick action, a replay memory of one million transitions and a target network refreshed every 10 000 updates, and reached a level comparable to a professional human tester on a large fraction of the 49 games tested, with the same architecture and hyper-parameters for all games.

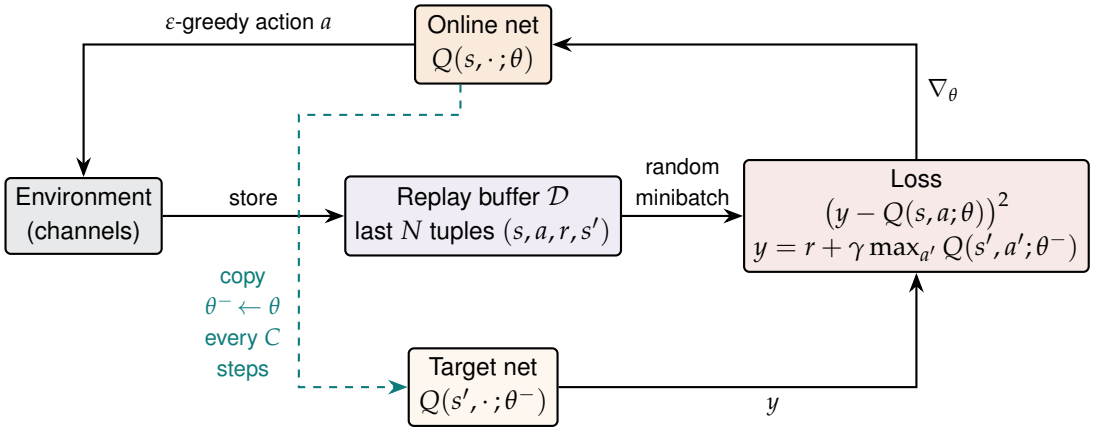


Figure 19.7: The DQN training loop. The online network acts in the environment; transitions go into the replay buffer; random minibatches are drawn from the buffer, targets are computed with the slowly updated target network, and the online network takes a gradient step. Every C steps the target network is overwritten with the online weights.

19.5.3 Double DQN

The max in (19.18) has a systematic upward bias. If the estimates $Q(s', a')$ are noisy but unbiased, the maximum of several noisy numbers is on average larger than the maximum of their means: $\mathbb{E}[\max_a X_a] \geq \max_a \mathbb{E}[X_a]$ (Jensen's inequality, since max is convex). The same network both *selects* the best-looking action and *evaluates* it, so whichever action got lucky noise is picked and its lucky value is used. Double DQN (van Hasselt, Guez and Silver, 2016) decouples the two: the online network selects, the target network evaluates,

$$y^{\text{Double}} = r + \gamma\, Q\Big(s',\ \arg\max_{a'} Q(s', a'; \theta);\ \theta^-\Big). \tag{19.20}$$

Because the noise in the two networks is partly independent, a lucky selection is less likely to come with a lucky evaluation. The change is one line of code.

19.5.4 Experiment: DQN for dynamic multichannel access

We now write our own environment — no external RL library. There are $N = 4$ independent channels, each a two-state Gilbert–Elliott Markov chain (good $= 1$, bad $= 0$) with transition probabilities $p_{11} = P(\text{good} \to \text{good})$ and $p_{01} = P(\text{bad} \to \text{good})$:

$$p_{11} = (0.90,\ 0.80,\ 0.95,\ 0.70), \qquad p_{01} = (0.20,\ 0.10,\ 0.10,\ 0.30).$$

Each slot the user picks one channel, transmits, and observes only that channel's state (ACK if good). Reward $= 1$ for a success. The agent does *not* know the p's. Its input is the history of the last $M = 8$ slots, each encoded as a 4-vector holding $+1$ (sensed good), -1 (sensed bad) or 0 (not sensed) in the channel's position: a 32-dimensional state. The network is a $32 \to 32 \to 32 \to 4$ MLP with ReLU; replay buffer 5000, minibatch 32, $\gamma = 0.9$, Adam with learning rate 10^{-3}, target network copied every 100 slots, ε ramped linearly from 1 to 0.02 over 2500 slots. The learning step:

```
        if n_buf >= 500 and t % 2 == 0:                                    #
    learn from a random minibatch
            idx = rng.integers(n_buf, size=batch)
            s_b, a_b, r_b, s2_b = map(torch.from_numpy, (S[idx], A[idx], R[idx],
    S2[idx]))
            with torch.no_grad():
                if double:   # online net chooses a', target net evaluates it
                    a2 = q(s2_b).argmax(1, keepdim=True); q_next = q_tgt(s2_b).
    gather(1, a2).squeeze(1)
                else:        # vanilla DQN: target net both chooses and evaluates
                    q_next = q_tgt(s2_b).max(1).values
                y = r_b + gamma * q_next                          # continuing task:
    no terminal states
            q_sa = q(s_b).gather(1, a_b.unsqueeze(1)).squeeze(1)
            loss = nn.functional.smooth_l1_loss(q_sa, y)
            opt.zero_grad(); loss.backward(); opt.step()
            qlog[t] = q_sa.mean().item()                        # how big does the
    net think Q is?
        if t % target_every == 0: q_tgt.load_state_dict(q.state_dict())
```

To judge the result we need yardsticks, computed by the same script over 20 000 slots: a *random* channel (0.534 success rate), the *best fixed* channel (0.663), the *myopic belief policy* of Section 19.6, which knows the p's (0.847), and a *genie* who sees all channel states of the previous slot (0.899; no causal policy can beat it).

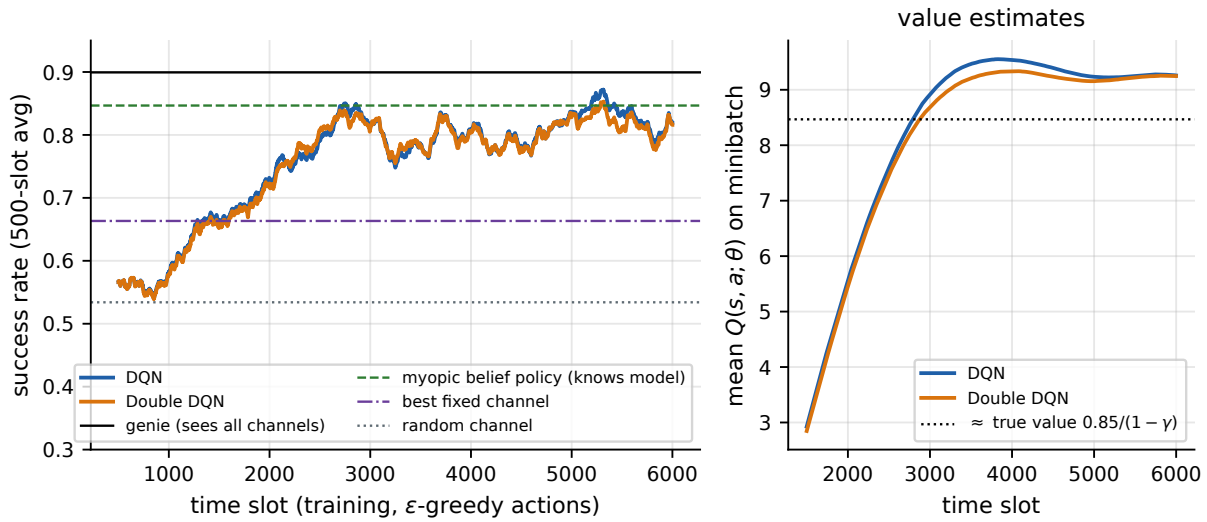


Figure 19.8: Our DQN on a 4-channel Gilbert–Elliott access task (mean of 2 seeds; 500-slot moving average of the training reward, which includes ε-exploration). Left: within a few thousand slots both DQN and Double DQN climb from random-channel performance past the best fixed channel to the level of the model-based myopic policy — without knowing any transition probability. Right: the mean predicted Q on training minibatches. Both settle around 9.2, above the value $\approx 0.85/(1-\gamma) = 8.5$ that the achieved success rate supports: the max-induced overestimation. With only 4 actions the Double-DQN correction is small here.

Evaluated greedily (no exploration) on fresh channel realisations, the final networks achieved success rates of 0.832 and 0.795 (DQN, two seeds) and 0.837 and 0.839 (Double DQN), against 0.847 for the myopic policy that knows the model. The network has learned, purely from ACKs and without any transition probability, a policy almost as good as the model-based myopic rule; inspecting its choices shows the same basic habit of staying on a channel while it keeps working and switching after a failure. Be honest about what the right panel shows: in this small problem the overestimation is mild and Double DQN barely changes it; van Hasselt et al. report large benefits on Atari, where there are up to 18 actions and long horizons. That is typical of RL: a fix that matters enormously at scale may be invisible in a toy.

Pitfall: RL is noisy — use several seeds

The two DQN seeds above differ by almost four percentage points. RL results depend strongly on the random seed, because early luck shapes which data the agent collects. Always report several seeds (and ideally a spread), and never conclude that one variant is better from a single run.

Wireless connection: dynamic spectrum access

In opportunistic spectrum access a secondary user senses a few of many licensed channels per slot and transmits where the primary user is idle. Primary activity is bursty and correlated in time, which is exactly the Gilbert–Elliott model above. The later paper chapter on deep RL for dynamic multichannel access uses this formulation at a larger scale (more channels, correlated channel dynamics) and compares DQN with the myopic and Whittle-index policies introduced in the next section. Our small experiment is the core of that work.

19.6 Partial observability: POMDPs and belief states

In the channel-access task the agent sees only the channel it used, so the observation is *not* the state: the true state is the vector of all channel conditions. When the Markov property fails for observations, we have a *partially observable* MDP.

Definition: POMDP and belief

A POMDP adds to an MDP a set of observations $\mathcal{O}$ and an observation model $O(o \mid s', a)$, the probability of seeing o after action a leads to state s'. The **belief** $b_t(s) = P(S_t = s \mid o_1, a_1, \ldots, o_t, a_t)$ is the posterior distribution of the hidden state given the whole history. The belief is a *sufficient statistic*: an optimal policy needs nothing else from the past, and the belief itself evolves as a Markov process. A POMDP is therefore an MDP whose state is the belief.

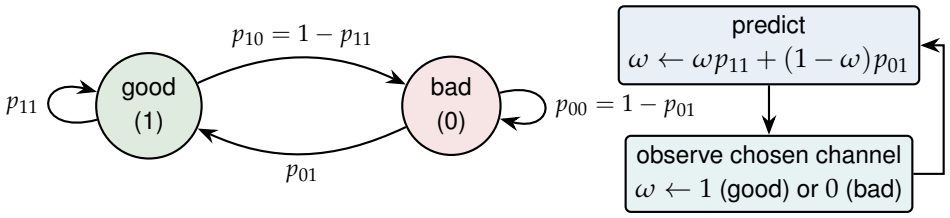


Figure 19.9: Left: a Gilbert–Elliott channel is a two-state Markov chain. Right: the belief filter for one channel. Every slot the belief is propagated through the chain; if the channel is sensed, the belief collapses to the observed state. Unsensed channels drift towards the stationary probability.

19.6.1 Belief update for a Gilbert–Elliott channel, derived

Let $\omega_t = P(\text{good at } t \mid \text{history})$ for one channel. **Prediction.** By the law of total probability over the state at time t,

$$\begin{aligned}\omega_{t+1}^{-} &= P(\text{good}_{t+1} \mid \text{good}_t)\,\omega_t + P(\text{good}_{t+1} \mid \text{bad}_t)\,(1-\omega_t) = p_{11}\omega_t + p_{01}(1-\omega_t) \\ &= p_{01} + (p_{11} - p_{01})\,\omega_t.\end{aligned}$$

Correction. If we sense the channel and get observation o with likelihoods $P(o \mid \text{good})$ and $P(o \mid \text{bad})$, Bayes' rule gives

$$\omega_{t+1} = \frac{P(o \mid \text{good})\,\omega_{t+1}^{-}}{P(o \mid \text{good})\,\omega_{t+1}^{-} + P(o \mid \text{bad})\,(1-\omega_{t+1}^{-})}.$$

With perfect sensing ($P(\text{ACK} \mid \text{good}) = 1$, $P(\text{ACK} \mid \text{bad}) = 0$) this collapses to $\omega_{t+1} = 1$ after an ACK and 0 after a NACK. With sensing errors (missed detection, false alarm) the general formula is used. **Drift.** An unsensed channel follows the linear recursion $\omega \leftarrow p_{01} + (p_{11} - p_{01})\omega$, whose fixed point $\omega_\infty = p_{01} + (p_{11} - p_{01})\omega_\infty$ gives

$$\omega_\infty = \frac{p_{01}}{1 - p_{11} + p_{01}} = \frac{p_{01}}{p_{01} + p_{10}},$$

the stationary probability of the good state. Writing $\omega_k - \omega_\infty = (p_{11} - p_{01})^k(\omega_0 - \omega_\infty)$, the approach is geometric with ratio $p_{11} - p_{01}$.

Worked example: belief after a NACK

Take $p_{11} = 0.9$, $p_{01} = 0.2$, so $p_{11} - p_{01} = 0.7$ and $\omega_\infty = 0.2/0.3 = 0.667$. After a NACK, $\omega_0 = 0$, and if we stop sensing: $\omega_1 = 0.2$, $\omega_2 = 0.2 + 0.7(0.2) = 0.34$, $\omega_3 = 0.2 + 0.7(0.34) = 0.438$, $\omega_4 = 0.507$ — matching $\omega_k = 0.667(1 - 0.7^k)$. A channel that just failed becomes worth retrying only after a few slots.

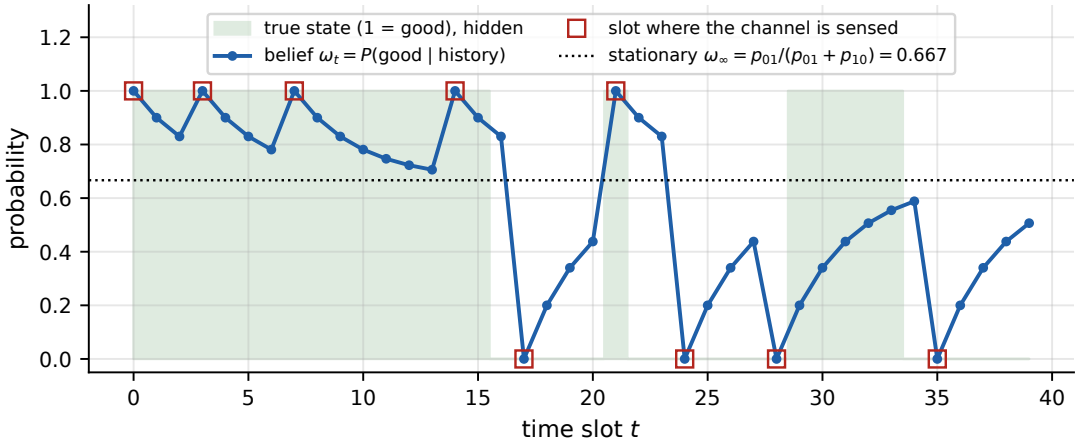


Figure 19.10: Belief of one Gilbert–Elliott channel ($p_{11} = 0.9$, $p_{01} = 0.2$) that is sensed only in the marked slots. Between observations the belief decays (after "good") or recovers (after "bad") geometrically towards the stationary value 0.667; at each observation it jumps to the truth. The shaded area is the hidden true state.

19.6.2 The myopic policy and the Whittle index

With N channels and one sensing action per slot, the expected immediate reward of choosing channel i is its predicted belief ω_i^-. The **myopic** (greedy) policy picks $\arg\max_i \omega_i^-$: it maximises the immediate expected reward and ignores the information value of sensing. For statistically identical, positively correlated channels ($p_{11} \geq p_{01}$) it has a remarkably simple structure (stay on a good channel; after a failure move to the channel unobserved for longest), and it has been proven optimal under such conditions (Zhao, Krishnamachari and Liu, 2008; Ahmad et al., 2009). For non-identical channels, like ours, it is generally sub-optimal.

The general problem is a **restless bandit**: every arm (channel) evolves whether played or not. Whittle (1988) relaxed the hard constraint "sense exactly one channel per slot" to a constraint on average, priced by a Lagrange multiplier that acts like a *subsidy* m paid for *not* sensing. For each channel separately, the **Whittle index** of a belief ω is the subsidy that makes sensing and not sensing equally attractive; the policy then senses the channel with the largest index. When a problem is *indexable*, this decomposes a joint problem over N channels into N single-channel problems. For Gilbert–Elliott channels, Liu and Zhao (2010) showed indexability and derived the index in closed form; it coincides with the myopic policy for identical channels and generally improves on it for non-identical ones. Myopic and Whittle are the classical baselines against which DQN-based channel access is judged.

19.6.3 Learning without a model: history as input

The belief update needs p_{11}, p_{01}. A DQN has a simpler option: feed it the *last M actions and observations*, as we did. If M is long enough the history contains (approximately) the information in the belief — the time since each channel was last sensed and what it showed — and the network learns the belief map and the policy at once. A recurrent network (LSTM/GRU, from the chapter on recurrent networks) can replace the fixed window and keep a learned belief in its hidden state.

19.7 Policy gradients

Value-based methods learn Q and act greedily. That is awkward when actions are *continuous*: the transmit power, a phase shift, or the $2n$ real numbers a transmitter sends over the air. Taking $\max_a$ over a continuum is itself an optimisation problem. Policy-gradient methods instead parametrise the policy directly, $\pi_\theta(a \mid s)$, and climb the gradient of expected return.

Intuition: do more of what worked

Act randomly around your current habit. When a random variation earns more than you expected, shift your habit slightly towards it; when it earns less, shift away. You never need to know *why* the outcome was good — the environment can be a black box — you only need to know which direction makes the variation you just tried more likely. This is the whole of REINFORCE.

19.7.1 The log-derivative trick and the policy gradient theorem

Let a trajectory be $\tau = (s_0, a_0, r_1, s_1, a_1, \ldots, s_T)$ and $R(\tau) = \sum_t \gamma^t r_{t+1}$ its return. Under policy π_θ its probability is

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t)\, P(s_{t+1} \mid s_t, a_t).$$

The objective is $J(\theta) = \mathbb{E}_{\tau\sim p_\theta}[R(\tau)] = \sum_\tau p_\theta(\tau) R(\tau)$ (an integral for continuous spaces). Differentiate:

$$\begin{aligned}\nabla_\theta J &= \sum_\tau \nabla_\theta p_\theta(\tau)\, R(\tau) = \sum_\tau p_\theta(\tau)\, \frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)}\, R(\tau) = \sum_\tau p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, R(\tau) \\ &= \mathbb{E}_{\tau\sim p_\theta}\left[\nabla_\theta \log p_\theta(\tau)\, R(\tau)\right].\end{aligned}$$

The middle step is the **log-derivative** (likelihood-ratio) trick, $\nabla p = p\, \nabla \log p$, which follows from the chain rule $\nabla \log p = \nabla p / p$. Its power: a gradient of an expectation becomes an expectation of something we can *sample*. Now take the log of $p_\theta(\tau)$ — products become sums:

$$\log p_\theta(\tau) = \log \rho_0(s_0) + \sum_t \log \pi_\theta(a_t \mid s_t) + \sum_t \log P(s_{t+1} \mid s_t, a_t).$$

Only the middle sum depends on θ; the initial distribution and *the environment dynamics drop out of the gradient*. We never need a model of P — or of the channel:

$$\nabla_\theta J = \mathbb{E}_\tau\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, R(\tau)\right]. \tag{19.21}$$

Causality. An action at time t cannot affect rewards received before t; those terms average to zero (by the baseline argument below), so $R(\tau)$ can be replaced by the reward-to-go $G_t = \sum_{k\ge t} \gamma^{k-t} r_{k+1}$

(up to a factor γ^t usually dropped in practice). This gives the form behind the **policy gradient theorem** (Sutton et al., 2000):

Key equation: policy gradient

$$\nabla_\theta J = \mathbb{E}_\pi\Big[\sum_t \nabla_\theta \log \pi_\theta(A_t \mid S_t)\,\big(G_t - b(S_t)\big)\Big], \tag{19.22}$$

where $b(s)$ is any baseline that does not depend on the action, justified next.

19.7.2 REINFORCE and baselines

REINFORCE (Williams, 1992) estimates (19.22) by sampling: run the policy, compute G_t, and step $\theta \leftarrow \theta + \eta \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)(G_t - b)$. In PyTorch we write the *surrogate loss* $-\sum_t \log \pi_\theta(a_t \mid s_t)\,(G_t - b)$ with $G_t - b$ detached, and let autograd do the rest.

Why a baseline is allowed. For any $b(s)$ that does not depend on a,

$$\mathbb{E}_{a\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, b(s)\right] = b(s) \sum_a \pi_\theta(a \mid s) \frac{\nabla_\theta \pi_\theta(a \mid s)}{\pi_\theta(a \mid s)} = b(s)\, \nabla_\theta \underbrace{\sum_a \pi_\theta(a \mid s)}_{=1} = 0.$$

Subtracting it leaves the expected gradient unchanged. **Why it helps.** If all rewards are positive (say between 10 and 11), raw REINFORCE increases the probability of *every* sampled action, just some more than others; the useful signal is a small difference between large noisy numbers. Subtracting $b(s) \approx V^\pi(s)$ turns the weight into an *advantage*: positive if the action did better than usual, negative if worse. The variance drops dramatically. A good choice is a learned $b_\phi(s)$, trained by regression of G_t on s.

19.7.3 Gaussian policies for continuous actions

For a real-valued action, let the network output a mean $\mu_\theta(s)$ and sample $a = \mu_\theta(s) + w$, $w \sim \mathcal{N}(0, \sigma^2)$. The log-density is

$$\log \pi_\theta(a \mid s) = -\frac{(a - \mu_\theta(s))^2}{2\sigma^2} - \log\left(\sigma\sqrt{2\pi}\right).$$

Differentiate with respect to the mean (the last term is constant):

$$\frac{\partial}{\partial \mu} \log \pi = \frac{a - \mu}{\sigma^2} = \frac{w}{\sigma^2}, \qquad \nabla_\theta \log \pi_\theta(a \mid s) = \frac{w}{\sigma^2}\, \nabla_\theta \mu_\theta(s). \tag{19.23}$$

If σ is learned too, $\frac{\partial}{\partial \sigma} \log \pi = \frac{(a-\mu)^2}{\sigma^3} - \frac{1}{\sigma} = \frac{w^2 - \sigma^2}{\sigma^3}$: widen the exploration when large perturbations paid off. For a vector action $\mathbf{a} = \boldsymbol{\mu}_\theta(s) + \mathbf{w}$, $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$, the components are independent, the log-density is a sum, and $\nabla_{\boldsymbol{\mu}} \log \pi = \mathbf{w}/\sigma^2$.

Intuition: reading equation (19.23)

The REINFORCE step for the mean is $\Delta\mu \propto (G - b)\, w/\sigma^2$: *move the mean in the direction of the perturbation you just tried, scaled by how much better than expected it turned out.* Averaged over many perturbations this estimates the slope of the expected reward — a stochastic finite-difference gradient that never needs the derivative of the environment.

Wireless connection: model-free transmitter training

In end-to-end learning (autoencoder transmitter–channel–receiver), backpropagation needs the derivative of the channel, which is unknown on real hardware. The model-free approach treats the transmitter as a Gaussian policy: it sends the perturbed symbols $\mathbf{x}_p = \mathbf{x} + \mathbf{w}$, $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$, where $\mathbf{x} = f_\theta(m)$ is the transmitter output. The receiver computes a per-example loss ℓ and feeds it back as a (negative) reward. With (19.23), the transmitter gradient is estimated as

$$\nabla_\theta J \approx -\frac{1}{B} \sum_{i=1}^{B} (\ell_i - b)\, \frac{1}{\sigma^2} \left(\nabla_\theta \mathbf{x}_i\right)^{\mathsf{T}} \mathbf{w}_i,$$

a REINFORCE step in which the "environment" is the real channel plus the receiver. This is exactly the method studied in the Part V paper chapter on model-free end-to-end learning; the next experiment is its one-dimensional rehearsal.

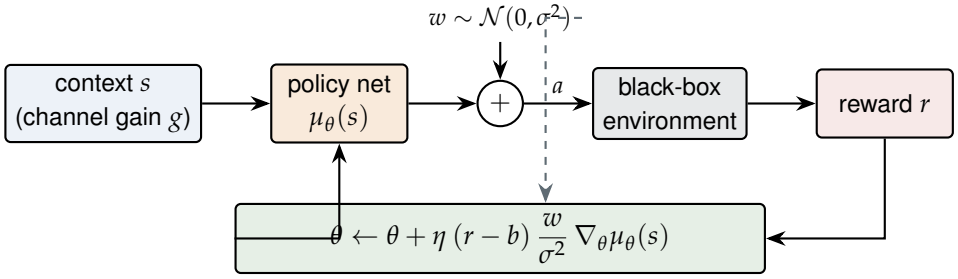


Figure 19.11: REINFORCE with a Gaussian policy. Exploration noise w is added to the network's mean action; only the scalar reward comes back. The update multiplies the (baseline-corrected) reward by the noise that was used, so no gradient ever flows through the environment.

Worked example: one REINFORCE step for power control

Let the reward be $r = \log_2(1 + g\,a) - 0.5\,a$ (rate minus an energy price), $g = 1$, current mean $\mu = 1.0$, $\sigma = 0.4$, baseline $b = 0.5$. Sample $w = 0.3$, so $a = 1.3$ and

$$r = \log_2(2.3) - 0.65 = 1.2016 - 0.65 = 0.5516, \qquad r - b = 0.0516, \qquad \frac{w}{\sigma^2} = \frac{0.3}{0.16} = 1.875.$$

The estimated gradient with respect to μ is $0.0516 \times 1.875 = 0.0968 > 0$: increase the power. Correct — setting $\mathrm{d}r/\mathrm{d}a = \frac{g}{(1+ga)\ln 2} - 0.5 = 0$ gives $a^* = \frac{1}{0.5 \ln 2} - \frac{1}{g} = 2.885 - 1 = 1.885$ for $g = 1$.

19.7.4 Experiment: REINFORCE learns water-filling

In `code/ch19/reinforce_power.py` the context is a channel gain g drawn uniformly from $[0.25, 3]$, the action is the power $a = \mu_\theta(g) + w$ with $\sigma = 0.4$, and the reward is $\log_2(1 + g \max(a, 0)) - 0.5 \max(a, 0)$. The optimal policy, from the derivative above, is the water-filling rule $p^*(g) = \max\left(0, \frac{1}{0.5 \ln 2} - \frac{1}{g}\right)$ — but the agent is never told the reward formula. The policy mean μ_θ is a $1 \to 32 \to 1$ tanh network; the optional baseline $b_\phi(g)$ is a second small network fitted to the rewards. Each iteration uses 64 samples.

```
    for it in range(ITERS):
        g = sample_g(BATCH)
        m = mu(g)
```

```
        w = SIGMA * torch.randn_like(m)                    # Gaussian exploration
        a = (m + w).detach()                               # the action actually
    sent
        r = reward(g, a)                                   # environment: no
    gradient through it
        logp = -((a - m) ** 2) / (2 * SIGMA ** 2)          # log pi(a|g) up to a
    constant
        b = vb(g) if baseline else torch.zeros_like(r)
        adv = (r - b).detach()
        loss_pi = -(adv * logp).mean()                     # REINFORCE surrogate
        loss_b = ((r - b) ** 2).mean() if baseline else 0.0
        opt.zero_grad(); (loss_pi + loss_b).backward()
        gvar.append(mu[2].bias.grad.item())
        opt.step()
```

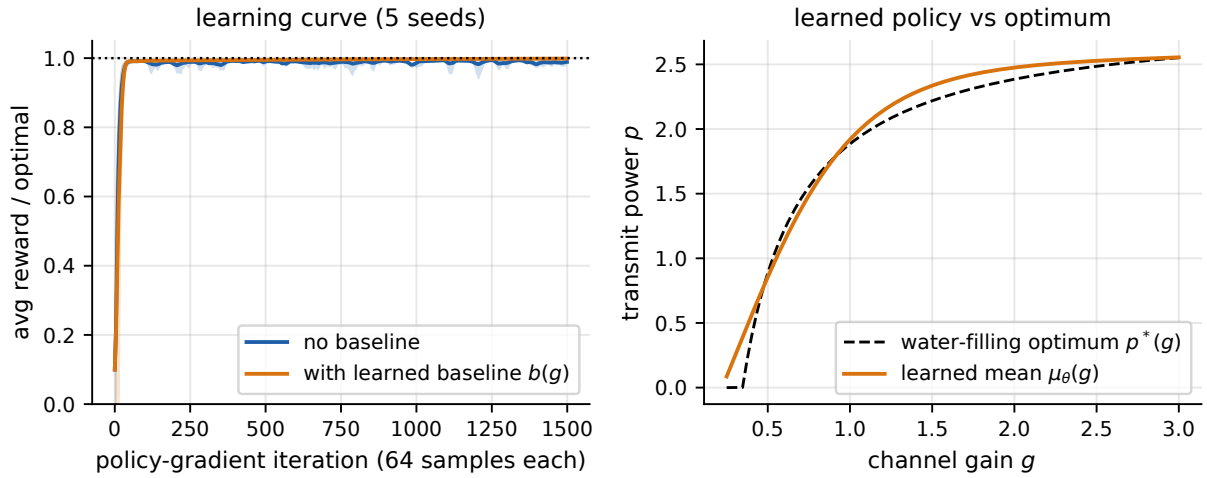


Figure 19.12: REINFORCE with a Gaussian exploration policy on continuous power control (5 seeds; shading = min/max). Left: average reward of the deterministic mean policy divided by the optimum. Both variants reach 95% within about 30 iterations; with the learned baseline the curve is smoother and ends higher (0.999 vs 0.989). Right: the learned mean $\mu_\theta(g)$ lies almost on the water-filling curve, learned from rewards alone.

The script also measures the variance of one gradient component (the output bias of μ_θ) across iterations: 1.2×10^{-1} without baseline and 2.9×10^{-4} with it — a reduction by a factor of about 400 for the same expected gradient. Here the problem is easy enough that both converge; in harder problems (long horizons, many parameters) that variance difference decides whether learning works at all.

19.7.5 Actor–critic: the best of both worlds

REINFORCE waits for the whole return G_t (Monte Carlo, high variance). **Actor–critic** methods replace $G_t - b(S_t)$ by a TD estimate of the advantage. A *critic* V_ϕ is trained by TD(0) (19.15), and its TD error

$$\delta_t = r_{t+1} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

is an estimate of the advantage of the action just taken: in expectation, $\mathbb{E}[\delta_t \mid s_t, a_t] = Q^\pi(s_t, a_t) - V^\pi(s_t)$. The *actor* then steps $\theta \leftarrow \theta + \eta\, \delta_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$ after every transition. Modern algorithms — A2C/A3C, PPO (which limits how far each update can move the policy), DDPG and SAC for continuous control — are refinements of this actor–critic loop.

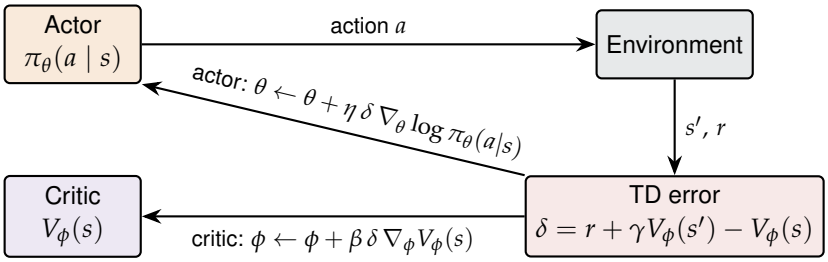


Figure 19.13: Actor–critic. The critic learns state values by TD; its TD error tells the actor whether the last action was better ($\delta > 0$) or worse ($\delta < 0$) than expected, and the actor adjusts its policy accordingly.

19.8 A map of RL methods and their wireless uses

Table 19.1: The methods of this chapter at a glance.

Method	Model?	Actions	Typical wireless use
Bandits (UCB, Thompson)	no	discrete	beam/channel/MCS selection, stationary
Value/policy iteration	yes	discrete	planning when P is known (queues, Markov channels)
Q-learning, SARSA	no	discrete, small $\mathcal{S}$	small state spaces, e.g. power levels with few states
DQN, Double DQN	no	discrete, large $\mathcal{S}$	dynamic spectrum access, handover, scheduling
POMDP + belief	yes	discrete	sensing with known Markov models; myopic/Whittle
REINFORCE, actor–critic	no	continuous or discrete	power control, beamforming, model-free transmitters

Wireless connection: power control and beam management

Power control: each link chooses its transmit power to maximise its own (or the network's) rate minus energy cost; interference couples the links, so the environment is non-stationary from each agent's viewpoint. Multi-agent DQN (discretised power levels) and actor–critic methods (continuous power) have both been used. *Beam management*: in mmWave, sweeping all beams costs overhead; treating beam pairs as arms of a (contextual) bandit, with position or sub-6 GHz measurements as context, cuts the number of beams tested, while an MDP formulation can also decide *when* to re-sweep.

Wireless connection: BitTorrent and peer-to-peer resource allocation

In BitTorrent each peer uploads to only a few neighbours at a time (it "unchokes" them) and preferentially rewards those who upload back (tit-for-tat). Periodically it unchokes a random peer — the "optimistic unchoke". In our language, each neighbour is an arm whose reward is the download rate it reciprocates; tit-for-tat is exploitation and the optimistic unchoke is ε-greedy exploration. Because other peers also adapt, the rewards are non-stationary, which motivates online-learning and MDP formulations (discounted or sliding-window estimates) as studied in the later chapter on online learning in BitTorrent.

Pitfall: reward hacking

An agent maximises *exactly* the reward you write, not the one you meant. A rate-only reward may teach a power-control agent to blast maximum power and destroy its neighbours; a throughput reward measured only on successful packets may teach a scheduler to stop transmitting difficult packets. Design rewards that include the costs you care about, and always inspect the learned behaviour, not just the reward curve.

Chapter recap

- RL learns from rewards, not labels: rewards are evaluative, often delayed, and the data depend on the agent's own actions, which forces a trade-off between exploration and exploitation.
- Bandits are RL without state. Regret measures the cost of learning; ε-greedy has linear regret, UCB1 has $O(\log T)$ regret with the bound (19.4), and Thompson sampling (Beta posteriors for ACK/NACK) is usually best in practice.
- An MDP is $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$. The return obeys $G_t = R_{t+1} + \gamma G_{t+1}$, which gives the Bellman expectation equations for V^π, Q^π and, replacing averages over actions by a max, the Bellman optimality equations for V^*, Q^*. Value and policy iteration solve them when P is known.
- Without a model, TD(0) and Q-learning update estimates towards one-step targets: $Q \leftarrow Q + \alpha[r + \gamma \max Q(s', \cdot) - Q]$. Q-learning is off-policy; SARSA is on-policy.
- DQN replaces the table by a network trained on $(y - Q(s, a; \theta))^2$ with $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$; experience replay decorrelates samples and a target network stabilises the target. Double DQN reduces the max overestimation.
- With partial observations, the belief is the right state. For a Gilbert–Elliott channel it evolves as $\omega \leftarrow p_{01} + (p_{11} - p_{01})\omega$ between observations; the myopic and Whittle-index policies are model-based baselines, and a DQN fed with an observation history learns without the model.
- Policy gradients use $\nabla J = \mathbb{E}[\nabla \log \pi \, (G - b)]$; the dynamics drop out. A baseline cuts variance without bias. For a Gaussian policy $\nabla_\mu \log \pi = w/\sigma^2$ — the basis of model-free transmitter training. Actor–critic uses the TD error as the advantage.

Exercises

1. **(Bandit regret.)** Three channels have ACK probabilities 0.9, 0.8, 0.5. A policy pulls them 800, 150 and 50 times in 1000 slots. Compute the regret. What would the regret of pure ε-greedy with $\varepsilon = 0.1$ be, approximately, after 10^5 slots, assuming it has identified the best arm?
2. **(UCB1 by hand.)** At $t = 20$, arm 1 has $\hat{\mu} = 0.60$ from $n = 15$ pulls and arm 2 has $\hat{\mu} = 0.40$ from $n = 5$ pulls. Which arm does UCB1 pull? How many pulls of arm 2 (with the same mean estimate and t) would make UCB1 prefer arm 1?
3. **(Returns and discounting.)** An agent receives reward 1 in every slot forever. Compute G_0 for $\gamma = 0.9$ and $\gamma = 0.99$. A channel-access agent gets reward 1 in a fraction 0.85 of slots on average; what is its expected return for $\gamma = 0.9$? Relate your answer to the

right panel of Fig. 19.8.

4. **(Bellman equation.)** In the gridworld, verify $V^*(2,3) = 0.729$ from the Bellman optimality equation using the values in Fig. 19.5. Which action achieves the max? Why is "up" not optimal there?

5. **(Q-learning versus SARSA.)** In state s with $Q(s,a) = 0.4$, the agent takes a, receives $r = -0.1$ and lands in s' with $Q(s',\cdot) = (0.6, 0.2, 0.3)$. Its ε-greedy behaviour then selects the third action. With $\alpha = 0.5$, $\gamma = 0.9$, compute the new $Q(s,a)$ under Q-learning and under SARSA.

6. **(Belief update with sensing errors.)** A channel has $p_{11} = 0.8$, $p_{01} = 0.3$, and the current belief is $\omega = 0.5$. (a) Predict ω^- for the next slot. (b) The sensor reports "idle/good" with probability 0.95 if the channel is good and 0.1 if it is bad. It reports "good". Compute the posterior. (c) Compute ω_∞.

7. **(Gaussian policy gradient.)** For $a = \mu + w$, $w \sim \mathcal{N}(0, \sigma^2)$ and a reward $r(a) = -(a-2)^2$, show that $\mathbb{E}[r(a)\, w/\sigma^2] = \frac{\mathrm{d}}{\mathrm{d}\mu}\mathbb{E}[r(a)]$ by computing both sides in closed form. What happens to the variance of the estimator $r\, w/\sigma^2$ as $\sigma \to 0$, and how does a baseline $b = \mathbb{E}[r]$ help?

8. **(Conceptual: why not supervised learning?)** A colleague proposes training a channel-selection network by supervised learning, using as label "the channel that turned out to be good in the next slot". Explain what information this requires that the agent does not have, and when the proposal would work (hint: compare with the genie in Fig. 19.8).

Answers

1. Regret $= \sum_a \Delta_a N_a = 0 \cdot 800 + 0.1 \cdot 150 + 0.4 \cdot 50 = 15 + 20 = 35$ lost ACKs. ε-greedy explores in 10% of slots, choosing uniformly among 3 arms, so per slot it loses $0.1 \cdot \frac{1}{3}(0 + 0.1 + 0.4) = 0.0167$; after 10^5 slots that is about 1 667 — linear growth.

2. $\ln 20 = 2.996$. Arm 1: $0.60 + \sqrt{2(2.996)/15} = 0.60 + 0.632 = 1.232$. Arm 2: $0.40 + \sqrt{2(2.996)/5} = 0.40 + 1.095 = 1.495$. UCB1 pulls arm 2. It prefers arm 1 when $0.40 + \sqrt{5.99/n_2} < 1.232$, i.e. $\sqrt{5.99/n_2} < 0.832$, $n_2 > 5.99/0.692 = 8.66$: from $n_2 = 9$ pulls onward (keeping $t = 20$).

3. Geometric series: $G_0 = 1/(1-\gamma) = 10$ for $\gamma = 0.9$ and 100 for $\gamma = 0.99$. Expected return $\approx 0.85/(1-0.9) = 8.5$. In Fig. 19.8 the networks predict about 9.2, so they overestimate by roughly 8% — the max-bias of Q-learning with function approximation (part of the gap may also come from states that are genuinely better than average).

4. From $(2,3)$: up enters PIT (-1); left goes to $(2,2)$ with $0 + 0.9 \cdot 0.81 = 0.729$; down goes to $(3,3)$ with $0.9 \cdot 0.6561 = 0.590$; right bumps the border and stays: $0.9 \cdot 0.729 = 0.656$. Max $= 0.729$ by going left. "Up" would end the episode in the pit with reward -1.

5. Q-learning target: $-0.1 + 0.9 \cdot \max(0.6, 0.2, 0.3) = -0.1 + 0.54 = 0.44$; new $Q = 0.4 + 0.5(0.44 - 0.4) = 0.42$. SARSA target: $-0.1 + 0.9 \cdot 0.3 = 0.17$; new $Q = 0.4 + 0.5(0.17 - 0.4) = 0.285$. SARSA is lower because it accounts for the sub-optimal action the exploring policy actually took.

6. (a) $\omega^- = 0.3 + (0.8 - 0.3)(0.5) = 0.55$. (b) $\omega = \frac{0.95 \cdot 0.55}{0.95 \cdot 0.55 + 0.1 \cdot 0.45} = \frac{0.5225}{0.5675} = 0.921$. A "good" report is strong but not conclusive evidence. (c) $\omega_\infty = 0.3/(0.3 + 0.2) = 0.6$.

7. $\mathbb{E}[r] = -\mathbb{E}[(\mu - 2 + w)^2] = -(\mu-2)^2 - \sigma^2$, so $\frac{\mathrm{d}}{\mathrm{d}\mu}\mathbb{E}[r] = -2(\mu-2)$. Other side: $\mathbb{E}[r\,w]/\sigma^2 = -\mathbb{E}[((\mu-2)^2 + 2(\mu-2)w + w^2)w]/\sigma^2 = -[0 + 2(\mu-2)\sigma^2 + \mathbb{E}w^3]/\sigma^2 = -2(\mu-2)$, using $\mathbb{E}w = \mathbb{E}w^3 = 0$, $\mathbb{E}w^2 = \sigma^2$. Equal. The estimator's variance contains the term $\mathbb{E}[r^2 w^2]/\sigma^4 \approx r(\mu)^2/\sigma^2$, which blows up as $\sigma \to 0$ unless $r(\mu) \approx 0$; subtracting $b \approx \mathbb{E}[r] \approx r(\mu)$ removes this dominant term, leaving a variance of order $(r'(\mu))^2$ that stays bounded. This is why baselines are essential when rewards are far from zero.
8. The label "which channel will be good next slot" requires knowing the states of *all* channels, but the agent only observes the one it used. It would work only if all channels could be observed after the fact (wideband sensing) — then the problem becomes supervised prediction and its best achievable performance is that of the genie. With single-channel feedback, the agent never sees counterfactual outcomes, and must explore: that is exactly what makes it an RL problem.

Self-test quiz

1. Which algorithm keeps exploring at a fixed rate forever and therefore has linear regret?
   Answer: ε-greedy with constant ε.
2. Write the Bellman optimality equation for Q^* in one line.
   Answer: $Q^*(s,a) = \sum_{s'} P(s' \mid s,a)[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')]$.
3. What problem does the target network in DQN solve, and what problem does experience replay solve?
   Answer: the target network stops the regression target from moving with every update (instability); replay breaks the correlation between consecutive samples and reuses data.
4. Is Q-learning on-policy or off-policy? Why?
   Answer: off-policy; its target uses $\max_{a'} Q(s',a')$, the value of the greedy policy, regardless of the action the behaviour policy takes next.
5. For a Gilbert–Elliott channel that has not been sensed, to what value does the belief converge?
   Answer: the stationary probability $\omega_\infty = p_{01}/(p_{01} + 1 - p_{11})$.
6. For $a = \mu_\theta(s) + w$, $w \sim \mathcal{N}(0, \sigma^2)$, what is $\nabla_\theta \log \pi_\theta(a \mid s)$, and why does the environment's derivative never appear in REINFORCE?
   Answer: $(w/\sigma^2)\nabla_\theta \mu_\theta(s)$; the transition probabilities do not depend on θ, so their log-terms vanish when differentiating $\log p_\theta(\tau)$.

Further reading

- R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018. The standard text; Chapters 2–6 cover bandits, MDPs, dynamic programming, Monte Carlo and TD; Chapter 13 policy gradients. Freely available from the authors' website.
- V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015. The DQN paper: experience replay and target network.
- R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, pp. 229–256, 1992. REINFORCE and baselines.

- P. Auer, N. Cesa-Bianchi and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, pp. 235–256, 2002. UCB1 and its regret bound.
- C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992.
- H. van Hasselt, A. Guez and D. Silver, "Deep reinforcement learning with double Q-learning," AAAI 2016.
- R. S. Sutton, D. McAllester, S. Singh and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," NeurIPS 1999 (published 2000).
- P. Whittle, "Restless bandits: activity allocation in a changing world," *Journal of Applied Probability*, vol. 25A, 1988; K. Liu and Q. Zhao, "Indexability of restless bandit problems and optimality of Whittle index for dynamic multichannel access," *IEEE Trans. Information Theory*, 2010.
- T. Lattimore and C. Szepesvári, *Bandit Algorithms*, Cambridge University Press, 2020, for everything about bandits, including Thompson sampling and contextual bandits.

Part V

Deep Learning for Wireless Communications

Chapter 20

A Wireless Communications Primer for Deep Learning

This chapter opens Part V. Every paper chapter that follows takes one block of a radio link — the modulator, the channel, the detector, the decoder, the channel estimator, the medium-access controller — and replaces it with (or helps it with) a neural network. To understand what those networks learn, why they are trained the way they are, and how to judge whether they work, you need the language of wireless communications: complex baseband signals, constellations, noise, fading, equalization, OFDM, MIMO, channel codes, and multiple access. We build all of it here from first principles, with full derivations and figures produced by real simulations (all scripts are in `code/ch20/`).

You will also meet the exact signal types and channels used in the Kaggle-style labs of Part VI: the eleven RadioML-style modulations (including the analog ones), additive white Gaussian noise (AWGN), Rayleigh multipath, and IIR-type channels with long, resonant impulse responses. Throughout, orange *wireless connection* boxes are replaced by a second kind of bridge: we keep pointing out which classical receiver task is secretly a classification problem, which is a regression problem, and which is a sequential decision problem — because that is exactly how the later chapters turn them into deep learning problems.

Intuition: the one-paragraph version of this chapter

A transmitter turns bits into a few complex numbers per microsecond (*symbols*), smooths them into a waveform, and shifts that waveform up to a radio frequency. The air scales, delays, rotates and smears the waveform and adds noise. The receiver shifts it back down, undoes as much of the damage as it can (synchronization, equalization), decides which symbols were sent (detection), and uses the deliberate redundancy added by a channel code to correct the remaining mistakes (decoding). Every one of these steps is a function from noisy numbers to a decision — and a neural network is a very good function approximator.

20.1 Radio waves and the communication system

20.1.1 Carrier, frequency, wavelength and bandwidth

A radio transmitter drives an alternating current into an antenna; the antenna launches an electromagnetic wave that travels at the speed of light $c \approx 3 \times 10^8\,\mathrm{m/s}$. A pure tone of frequency f (cycles per second, hertz) repeats in space every

$$\lambda = \frac{c}{f} \qquad \text{(wavelength, metres)}. \tag{20.1}$$

At WiFi's 2.4 GHz, $\lambda = 3 \times 10^8 / 2.4 \times 10^9 = 0.125\,\mathrm{m} = 12.5\,\mathrm{cm}$; at the 5G mid-band carrier 3.5 GHz, $\lambda = 8.57\,\mathrm{cm}$; at 28 GHz (millimetre wave), $\lambda = 1.07\,\mathrm{cm}$. The wavelength matters because antenna sizes, the spacing of antenna elements in an array (typically $\lambda/2$), and the distance over which fading changes are all measured in wavelengths.

A pure tone carries no information: it is perfectly predictable. To send information we *modulate* the tone — we change its amplitude and phase (and hence, instantaneously, its frequency) in a way that depends on the data. The tone is then called the **carrier**, its frequency f_c the **carrier frequency**. The faster we change the amplitude and phase, the wider the range of frequencies the signal occupies around f_c. That range is the **bandwidth** B (in Hz). A fundamental rule we will make precise later: *you can send at most about B independent complex numbers per second in a bandwidth of B hertz.* A 20 MHz WiFi channel therefore carries roughly 20 million complex symbols per second (minus overheads); a 100 MHz 5G carrier, five times more.

Definition: carrier, bandwidth, symbol rate

The **carrier frequency** f_c is the centre frequency of the radio signal. The **bandwidth** B is the width of the frequency band the signal occupies. The **symbol rate** $R_s = 1/T$ is the number of complex symbols sent per second; T is the symbol period. With ideal pulses $R_s = B$; with practical pulses of roll-off β (Section 20.3.4) $B = (1+\beta)R_s$.

20.1.2 The end-to-end block diagram

Figure 20.1 is the most important picture of Part V. The top two rows (read clockwise) are the classical design, in which each block is derived by hand from a mathematical model and optimised separately. The bottom row shows the "learned" alternatives that the coming chapters study.

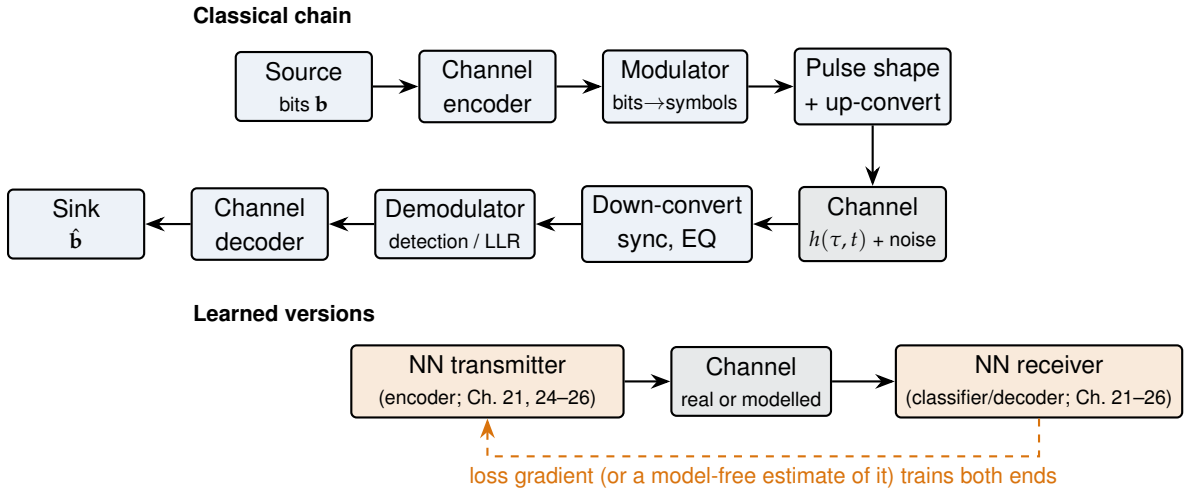


Figure 20.1: The communication system. **Top:** the classical chain (transmitter left to right, receiver right to left) of separately designed blocks. **Bottom:** the deep-learning view in which the transmitter and receiver are neural networks trained jointly to minimise a loss (e.g. cross-entropy on the transmitted message), through a known differentiable channel model or, when none exists, with model-free gradient estimates. Individual blocks (detector, decoder, channel estimator) can also be replaced one at a time.

Let us name each block once; the rest of the chapter opens them one by one.

- **Source.** Produces information bits $\mathbf{b} \in \{0,1\}^k$ (after source compression such as audio or video coding, which we do not study).
- **Channel encoder.** Adds structured redundancy: maps k bits to $n > k$ coded bits (code rate $R = k/n$) so that errors can be corrected (Section 20.9).
- **Modulator.** Maps groups of coded bits to complex symbols from a finite alphabet, the

constellation (Section 20.3).

- **Pulse shaping and up-conversion.** Turns the symbol sequence into a smooth, band-limited waveform and moves it to the carrier (Section 20.2).
- **Channel.** Everything between the antennas: attenuation, multipath echoes, Doppler, interference, thermal noise, and the hardware imperfections of both radios (Sections 20.4 to 20.6).
- **Receiver front end.** Down-conversion, filtering, sampling, synchronization and equalization (Section 20.7).
- **Demodulator/detector.** Decides which symbol was sent, or better, outputs a soft confidence (log-likelihood ratio) for each bit.
- **Channel decoder.** Uses the redundancy to recover $\hat{\mathbf{b}}$; success is measured by the bit error rate (BER) and block error rate (BLER).

Wireless connection: why replace blocks with networks at all?

Each classical block is optimal *only for the model it was derived from* (e.g. AWGN, linear channel, perfect synchronization) and only in isolation. Real links have nonlinear amplifiers, hardware impairments, interference and channels that no tidy formula describes. A network trained end to end on data can absorb such effects and can optimise the whole chain jointly — at the price of needing training data and of being harder to analyse. The paper chapters are, in essence, a careful study of when that trade is worth it.

20.2 Passband and complex baseband

20.2.1 Why complex numbers?

Intuition: two knobs on one carrier

A sinusoid at a fixed frequency f_c has exactly two adjustable properties: its *amplitude* and its *phase*. Equivalently, it is a mixture of a cosine and a sine at f_c with two adjustable weights. Two real numbers per instant — that is a complex number. A complex number $x = I + jQ = Ae^{j\phi}$ is simply bookkeeping for "how much cosine (I, in-phase) and how much negative sine (Q, quadrature)" or, in polar form, "what amplitude A and what phase ϕ". Picture it as an arrow (a **phasor**) of length A at angle ϕ that rotates at f_c revolutions per second; the real signal is its shadow on the horizontal axis.

Recall Euler's formula $e^{j\theta} = \cos\theta + j\sin\theta$ (Part I). Write the slowly varying **complex baseband** (or *complex envelope*) signal as

$$x(t) = I(t) + jQ(t) = A(t)\, e^{j\phi(t)},$$

where $I(t)$ and $Q(t)$ are real, change at the symbol rate (millions of times per second), and are much slower than the carrier (billions of cycles per second).

Key equation: passband from baseband

$$s(t) = \mathrm{Re}\left\{x(t)\, e^{j2\pi f_c t}\right\} = I(t)\cos(2\pi f_c t) - Q(t)\sin(2\pi f_c t) = A(t)\cos\big(2\pi f_c t + \phi(t)\big). \tag{20.2}$$

Derivation. Multiply out using Euler's formula:

$$\begin{aligned} x(t)e^{j2\pi f_c t} &= \big(I + jQ\big)\big(\cos(2\pi f_c t) + j\sin(2\pi f_c t)\big) \\ &= \underbrace{I\cos(2\pi f_c t) - Q\sin(2\pi f_c t)}_{\text{real part}} + j\underbrace{\big(I\sin(2\pi f_c t) + Q\cos(2\pi f_c t)\big)}_{\text{imaginary part}}, \end{aligned}$$

using $j \cdot j = -1$ in the $jQ \cdot j\sin$ term. Taking the real part gives the middle expression of (20.2). For the polar form, $xe^{j2\pi f_c t} = Ae^{j(2\pi f_c t + \phi)}$ whose real part is $A\cos(2\pi f_c t + \phi)$. □

So the physical (real) radio signal is completely determined by the complex baseband signal $x(t)$ and the known carrier f_c. All of digital signal processing — and all of the neural networks in Part V — work on samples of $x(t)$: the famous "I/Q samples". When a modulation-classification network takes an input tensor of shape 2×128, the two rows are I and Q of 128 consecutive complex baseband samples.

Figure 20.2 shows four QPSK symbols at phases $45°, 135°, 225°, 315°$. The top panel is the baseband pair $(I(t), Q(t))$; the middle panel the two weighted carriers; the bottom panel their sum, the passband signal. Look at the symbol boundaries: the amplitude never changes, but the carrier's phase jumps by 90°. The information is in that phase.

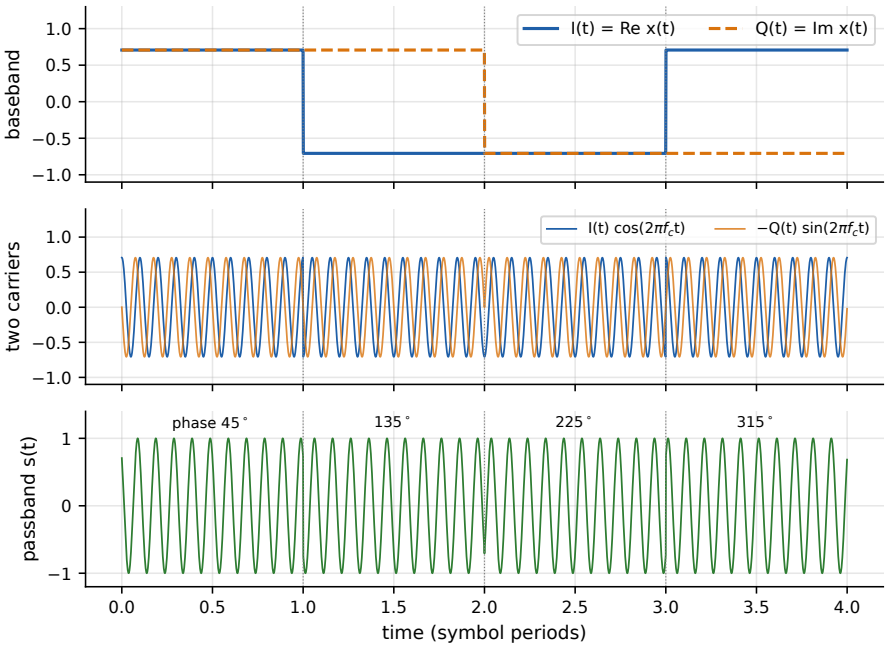


Figure 20.2: From complex baseband to passband (our simulation, $f_c = 10$ cycles per symbol for visibility; real carriers have thousands). **Top:** I and Q of four QPSK symbols. **Middle:** $I\cos$ and $-Q\sin$. **Bottom:** the transmitted $s(t)$ of (20.2). The envelope is constant; each new symbol shifts the carrier phase by 90°.

20.2.2 Up- and down-conversion

Figure 20.3 shows the hardware that implements (20.2) (the *IQ modulator*) and its inverse (the *IQ demodulator*). The receiver recovers I by multiplying by $2\cos(2\pi f_c t)$ and low-pass filtering:

$$\begin{aligned} 2s(t)\cos(2\pi f_c t) &= 2I\cos^2(2\pi f_c t) - 2Q\sin(2\pi f_c t)\cos(2\pi f_c t) \\ &= I\big(1+\cos(4\pi f_c t)\big) - Q\sin(4\pi f_c t) \\ &\xrightarrow{\text{low-pass}} I(t), \end{aligned}$$

using $2\cos^2\theta = 1+\cos 2\theta$ and $2\sin\theta\cos\theta = \sin 2\theta$. The terms at $2f_c$ are removed by the low-pass filter. Similarly, $-2s(t)\sin(2\pi f_c t) = -I\sin(4\pi f_c t) + Q\big(1-\cos(4\pi f_c t)\big) \to Q(t)$, using $2\sin^2\theta = 1-\cos 2\theta$. In complex notation the receiver computes $x(t) = \mathrm{LPF}\{2s(t)e^{-j2\pi f_c t}\}$.

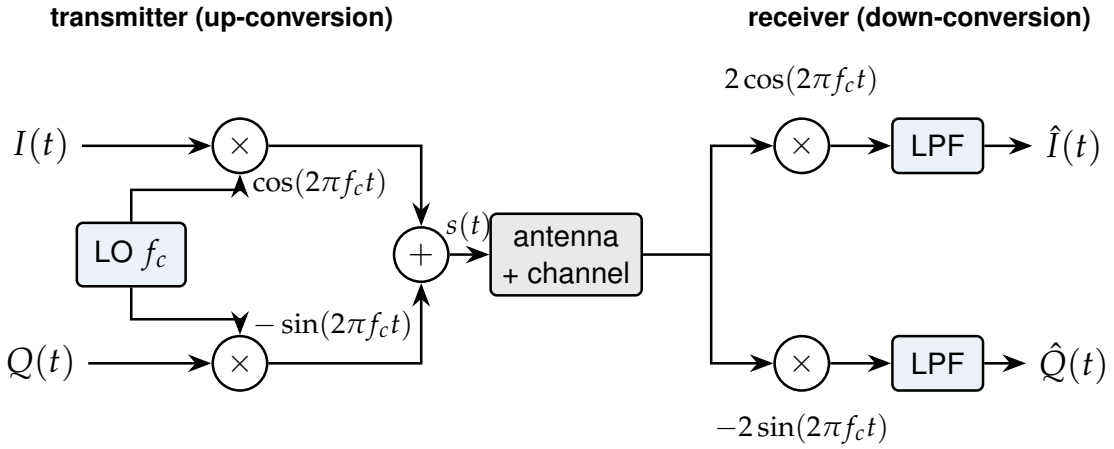


Figure 20.3: IQ modulator (left) and IQ demodulator (right). The local oscillator (LO) supplies the cosine and the 90°-shifted sine. If the receiver LO differs slightly in frequency or phase from the transmitter's, the recovered $\hat{I} + j\hat{Q}$ is rotated — the carrier frequency offset and phase errors of Section 20.6.

20.2.3 Sampling and Nyquist

A real signal whose spectrum lies within $[-W, W]$ is completely determined by samples taken at rate $f_s \geq 2W$ (the Nyquist–Shannon sampling theorem); sampling slower causes *aliasing*, where high frequencies fold onto low ones. A complex baseband signal occupying $[-B/2, B/2]$ has a total width B and needs $f_s \geq B$ complex samples per second — that is two real streams (I and Q) at B each, the same total of $2B$ real numbers per second as a real signal of bandwidth B. In practice receivers *oversample* (e.g. 2, 4 or 8 samples per symbol) to make filtering and timing recovery easy; RadioML-style datasets use 8 samples per symbol.

Worked example: sizes and rates

A 5G NR carrier of 100 MHz at 30 kHz subcarrier spacing is sampled at 122.88 Msample/s (4096-point FFT $\times$ 30 kHz). Each complex sample is two 16-bit integers, so the raw I/Q stream is $122.88 \times 10^6 \times 32 \approx 3.9$ Gbit/s per antenna — the reason front-haul interfaces compress I/Q data, and the reason a neural receiver must be cheap per sample.

20.3 Digital modulation

20.3.1 From bits to symbols

A modulator with an alphabet of M points maps each group of $k = \log_2 M$ bits to one complex symbol. The set of points, drawn in the complex (I–Q) plane, is the **constellation**. Figure 20.4 shows the most common ones, generated by our script with unit average energy and their bit labels.

Definition: common constellations

- **BPSK** (binary phase-shift keying), $M = 2$: $x \in \{-1, +1\}$.
- **QPSK**, $M = 4$: $x \in \frac{1}{\sqrt{2}}\{\pm 1 \pm j\}$ — two independent BPSKs on I and Q.
- **M-PSK**: $x_m = e^{j(2\pi m/M + \theta_0)}$, $m = 0, \ldots, M-1$ — points equally spaced on a circle.
- **M-PAM** (pulse-amplitude modulation): real levels $\{\pm 1, \pm 3, \ldots, \pm(M-1)\}$ scaled.
- **M-QAM** (quadrature amplitude modulation), $M = 4^m$: a $\sqrt{M} \times \sqrt{M}$ grid, i.e. $\sqrt{M}$-PAM on I and on Q.

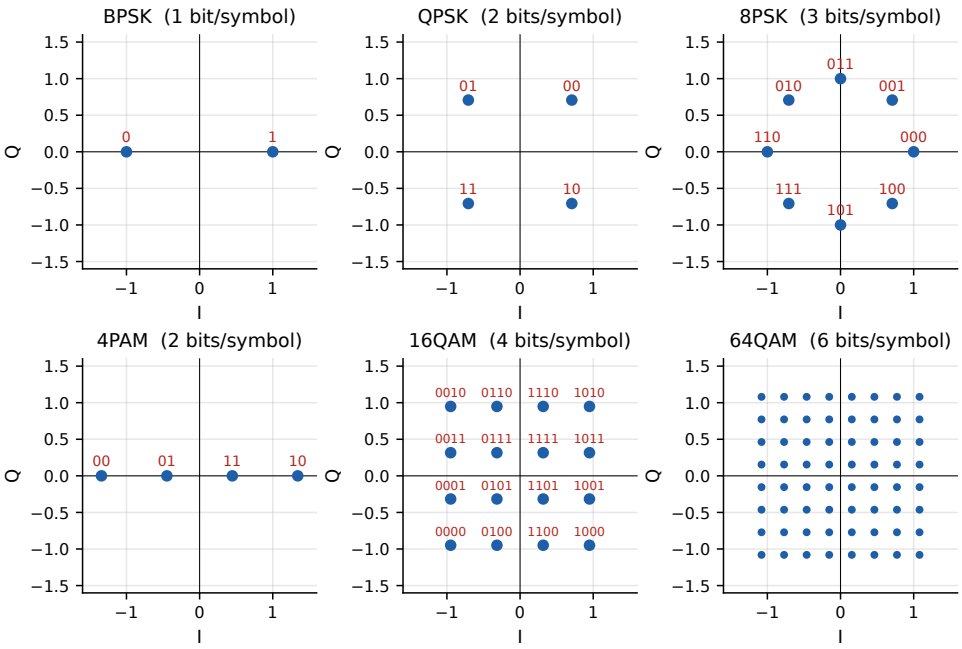


Figure 20.4: Constellations with unit average symbol energy and Gray bit labels (red). Notice that every pair of horizontally or vertically adjacent 16-QAM points differs in exactly one bit, and that adjacent 8-PSK points also differ in one bit. Higher order means more bits per symbol but points that are closer together, hence more vulnerable to noise.

20.3.2 Gray coding

When noise causes a detection error, the detector almost always picks a *neighbouring* point. **Gray coding** labels points so that neighbours differ in a single bit; then a symbol error usually costs one bit error, not several. For a $\sqrt{M}$-PAM axis the Gray label of level index i is $i \oplus (i \gg 1)$ (bitwise XOR with itself shifted right): $0, 1, 2, 3 \to 00, 01, 11, 10$. A square QAM label concatenates the

Gray labels of the I and Q indices. With Gray coding the bit error rate is well approximated by $\mathrm{BER} \approx \mathrm{SER}/k$ at moderate-to-high SNR, where SER is the symbol error rate.

20.3.3 Average energy and normalization

Different constellations must be compared at equal *average* energy. The average symbol energy (with equally likely symbols) is

$$E_s = \mathbb{E}\big[|x|^2\big] = \frac{1}{M}\sum_{m=1}^{M} |x_m|^2 .$$

Worked example: 16-QAM has $E_s = 10$ before normalization

Using levels $\{\pm 1, \pm 3\}$ on both axes, the 16 points are $a + jb$ with $a, b \in \{-3, -1, 1, 3\}$. Then $\mathbb{E}[a^2] = \frac{1}{4}(9 + 1 + 1 + 9) = 5$ and similarly $\mathbb{E}[b^2] = 5$, so $E_s = \mathbb{E}[a^2] + \mathbb{E}[b^2] = 10$. To obtain unit energy we divide every point by $\sqrt{10}$: the corner point $3 + 3j$ becomes $0.949 + 0.949j$ and the inner point $1 + j$ becomes $0.316 + 0.316j$. The same calculation for 64-QAM with levels $\{\pm 1, \pm 3, \pm 5, \pm 7\}$ gives $\mathbb{E}[a^2] = \frac{1}{4}(1 + 9 + 25 + 49) = 21$, so $E_s = 42$. In general, square M-QAM with odd-integer levels has $E_s = \frac{2}{3}(M - 1)$: check $\frac{2}{3} \cdot 15 = 10$, $\frac{2}{3} \cdot 63 = 42$.

The energy per *bit* is $E_b = E_s/k$. This normalization is what an autoencoder's "power normalization layer" enforces (Chapter 21): without it, a network would simply learn to shout, spreading its points arbitrarily far apart.

20.3.4 Pulse shaping, ISI and eye diagrams

The symbols a_n are numbers; to transmit them we need a continuous waveform. We send

$$x(t) = \sum_{n} a_n \, p(t - nT), \tag{20.3}$$

a train of copies of a pulse $p(t)$, one every symbol period T, each scaled by its symbol. A rectangular pulse would be simple but its spectrum (sinc-shaped) decays slowly and spills into neighbouring channels. A narrow spectrum requires a long, smooth pulse — but then pulses overlap in time. Overlap is harmless *if* at every sampling instant $t = kT$ all other pulses pass through zero:

$$p(kT) = \begin{cases} 1, & k = 0 \\ 0, & k \neq 0 \end{cases} \qquad \text{(Nyquist's zero-ISI criterion).} \tag{20.4}$$

If this fails, the sample at $t = kT$ contains pieces of neighbouring symbols: **inter-symbol interference** (ISI).

Key equation: raised-cosine pulse

With roll-off factor $0 \le \beta \le 1$,

$$p(t) = \operatorname{sinc}\!\left(\frac{t}{T}\right)\frac{\cos(\pi\beta t/T)}{1 - (2\beta t/T)^2},$$

$$P(f) = \begin{cases} T, & |f| \le \frac{1-\beta}{2T} \\ \frac{T}{2}\left[1 + \cos\!\left(\frac{\pi T}{\beta}\left(|f| - \frac{1-\beta}{2T}\right)\right)\right], & \frac{1-\beta}{2T} < |f| \le \frac{1+\beta}{2T} \\ 0, & \text{otherwise,} \end{cases} \tag{20.5}$$

with $\mathrm{sinc}(u) = \sin(\pi u)/(\pi u)$. The two-sided bandwidth is $(1+\beta)/T$.

Why it works: $\mathrm{sinc}(t/T)$ is zero at every nonzero integer multiple of T, so the product is too, satisfying (20.4). The extra factor makes the pulse decay as $1/t^3$ instead of $1/t$, so small timing errors cause little ISI. In the frequency domain, $P(f)$ is flat in the middle and rolls off with a half-cosine; $\beta = 0$ is the ideal brick-wall filter of width $1/T$ and $\beta = 1$ doubles the bandwidth (Fig. 20.5).

In practice the filter is split evenly between the radios: the transmitter uses a **root-raised-cosine** (RRC) filter with $|H_{\mathrm{RRC}}(f)| = \sqrt{P(f)}$ and the receiver uses the same RRC as its *matched filter*; their cascade is the raised cosine. The matched filter also maximizes the SNR at the sampling instant (a classical result), which is why the receiver's filter equals the transmit pulse.

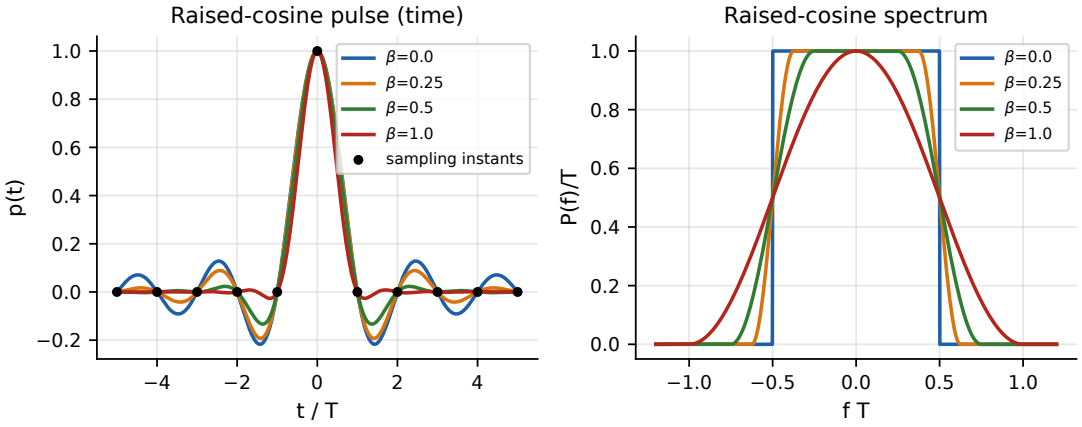


Figure 20.5: Raised-cosine pulses and spectra for four roll-offs. **Left:** every pulse passes through zero at all nonzero integer multiples of T (black dots), so neighbours do not interfere at the sampling instants; larger β means faster-decaying tails. **Right:** larger β costs bandwidth. 5G and WiFi use OFDM instead (Section 20.7.2), but single-carrier links and RadioML signals use RRC filters with $\beta \approx 0.2$–0.35.

An **eye diagram** overlays many two-symbol-long segments of the received waveform. Where all the traces pinch to a few points (the open "eye"), sampling is clean; a closing eye reveals ISI or noise. Figure 20.6 shows a clean RRC/RRC link, the same link with an echo of amplitude 0.5 one symbol later (ISI: each level splits into several), and the clean link with noise.

20.3.5 Frequency modulation and analog modulations (the RadioML zoo)

Modulation-recognition datasets such as RadioML 2016.10a contain eleven classes: eight digital (BPSK, QPSK, 8PSK, PAM4, QAM16, QAM64, CPFSK, GFSK) and three analog (AM-DSB, AM-SSB, WBFM). The linear digital ones you know. The others:

- **FSK** (frequency-shift keying) sends bit $b_n \in \{\pm 1\}$ as a tone at $f_c + b_n \Delta f$. In complex baseband, $x(t) = e^{j\theta(t)}$ with instantaneous frequency $\frac{1}{2\pi}\dot{\theta}(t) = b_n \Delta f$. It has *constant envelope* ($|x| = 1$), so it tolerates nonlinear amplifiers — a key feature for low-cost radios.
- **CPFSK** (continuous-phase FSK) keeps the phase continuous across symbols by integrating the frequency: $\theta(t) = 2\pi h \sum_n b_n\, q(t - nT)$, with modulation index $h = 2\Delta f\, T$ and q the integral of the frequency pulse. $h = 1/2$ is *minimum-shift keying* (MSK). Continuity removes spectral splatter.
- **GFSK** smooths the frequency pulse with a Gaussian filter of bandwidth–time product BT

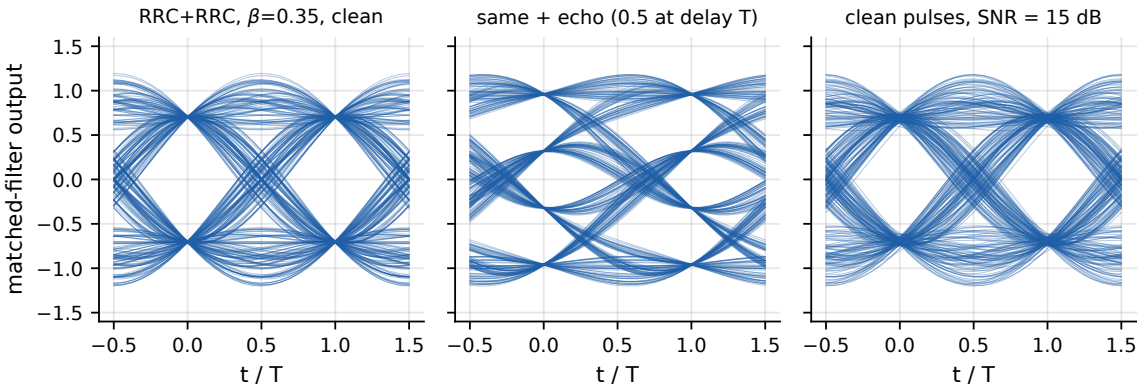


Figure 20.6: Eye diagrams of BPSK with RRC transmit and receive filters ($\beta = 0.35$, our simulation). **Left:** the eye is wide open at $t = 0$ and $t = T$. **Middle:** a one-symbol echo of half amplitude splits the two levels into four ($\pm 1 \pm 0.5$), shrinking the vertical opening — the ISI an equalizer must remove. **Right:** AWGN at 15 dB blurs the traces but leaves the eye open.

(Bluetooth uses $BT = 0.5$, GSM's GMSK $BT = 0.3$), narrowing the spectrum further at the cost of some ISI.

- **AM-DSB** (double-sideband amplitude modulation): $x(t) = 1 + \mu\, m(t)$ for an audio message $m(t)$ and modulation depth μ; the spectrum has a carrier spike plus mirror-image sidebands, and $Q \approx 0$.
- **AM-SSB** (single sideband) keeps only one sideband: $x(t) = m(t) + j\hat{m}(t)$, where $\hat{m}$ is the Hilbert transform of m (a 90° phase shift of every frequency component). This halves the bandwidth.
- **WBFM** (wideband FM, broadcast radio): $x(t) = e^{j2\pi k_f \int m(\tau)\, d\tau}$, frequency deviation up to ± 75 kHz.

Figure 20.7 shows six of these from our generator, in the time and frequency domains. Look at what a classifier can exploit: AM-DSB has a flat Q and a carrier spike; SSB is one-sided in frequency; the FSK family has constant envelope (the I and Q traces are sinusoid-like, never shrinking); QPSK's envelope fluctuates with the RRC pulses; WBFM is broad and flat.

Wireless connection: ML bridge: modulation recognition = classification

Modulation recognition maps a window of N complex samples (a $2 \times N$ real tensor) to one of C labels. It is a textbook supervised classification problem: softmax output, cross-entropy loss. What makes it hard is that the same class looks very different under different SNR, frequency offset, phase, timing and multipath — nuisance variables a good classifier must become invariant to.

20.4 The AWGN channel and optimal detection

20.4.1 Noise

Electrons in every resistor and amplifier jiggle thermally, producing a random voltage. Summing the effects of an astronomical number of independent electrons, the central limit theorem (Part I) makes the noise Gaussian. Its power spectral density is flat ("white") over any band of interest: N_0

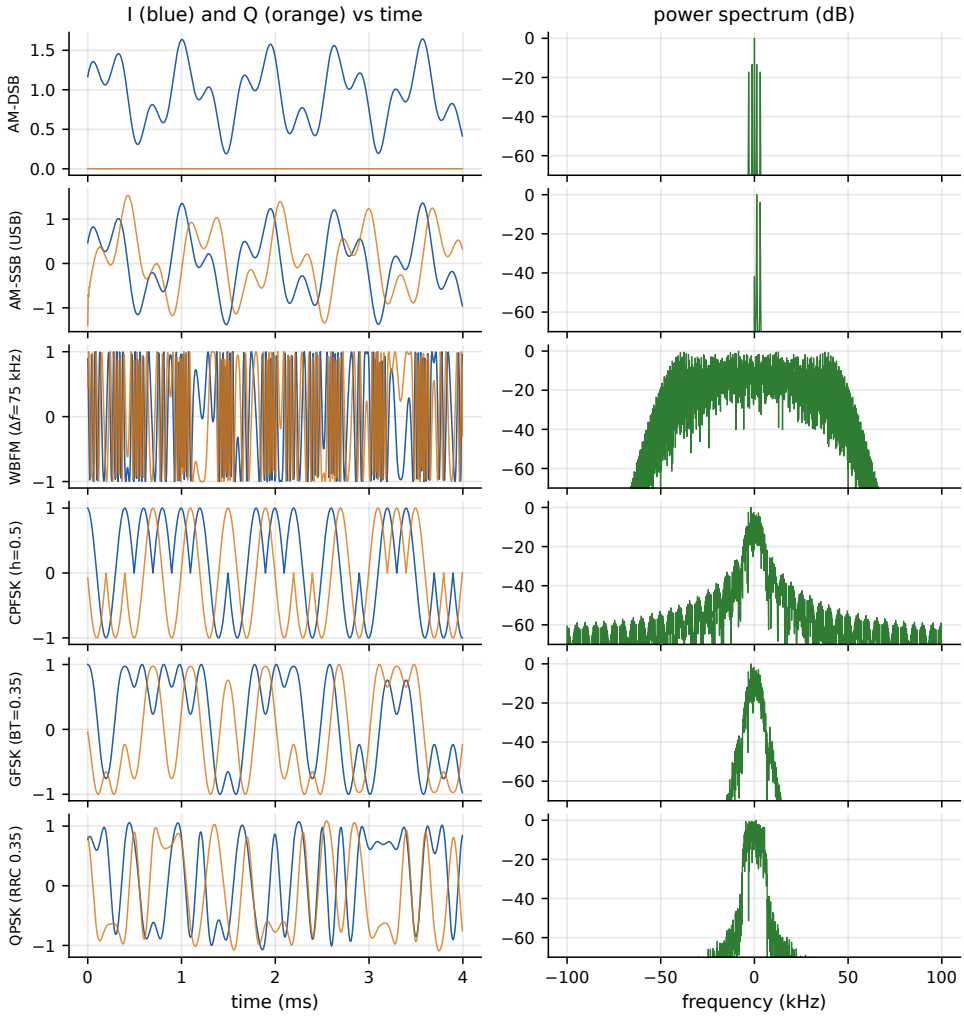


Figure 20.7: Six signal types of the RadioML family, generated by `fig_modulation.py` at 200 ksample/s (no noise). **Left:** I (blue) and Q (orange) over 4 ms. **Right:** power spectra. These visible differences — envelope constancy, spectral symmetry, carrier spikes, bandwidth — are what hand-crafted features (cumulants, Part VI) and CNN classifiers (Chapter 22) learn to detect.

watts per hertz, with $N_0 = k_B T_0 \approx 4 \times 10^{-21}$ W/Hz at room temperature, i.e. -174 dBm/Hz. Over a bandwidth B and with a receiver noise figure NF (dB) the noise power is $-174 + 10\log_{10} B + \text{NF}$ dBm; for 20 MHz and NF $= 7$ dB, $-174 + 73 + 7 = -94$ dBm.

Definition: AWGN channel and circular complex Gaussian noise

After matched filtering and sampling once per symbol, the AWGN channel is

$$y = x + n, \qquad n \sim \mathcal{CN}(0, N_0), \tag{20.6}$$

meaning $n = n_I + j n_Q$ with n_I, n_Q independent, each $\mathcal{N}(0, N_0/2)$. Its density is

$$p(n) = \frac{1}{\pi N_0} \exp\left(-\frac{|n|^2}{N_0}\right).$$

The density follows from multiplying the two real Gaussians: $\frac{1}{\sqrt{\pi N_0}} e^{-n_I^2/N_0} \cdot \frac{1}{\sqrt{\pi N_0}} e^{-n_Q^2/N_0}$, since a variance of $N_0/2$ gives $\frac{1}{\sqrt{2\pi N_0/2}} e^{-n_I^2/(2\cdot N_0/2)}$. "Circularly symmetric" means the density depends only on $|n|$: noise has no preferred direction, so rotating the constellation does not change detection performance.

20.4.2 SNR, E_s/N_0 and E_b/N_0

With symbols of energy E_s and noise $\mathcal{CN}(0, N_0)$, the per-symbol signal-to-noise ratio is $\text{SNR} = E_s/N_0$. Because modulations carry different numbers of bits per symbol, fair comparisons use the energy per information bit: $E_b = E_s/(kR)$ for k bits/symbol and code rate R ($R = 1$ uncoded). In decibels ($x_{\text{dB}} = 10\log_{10} x$):

$$\frac{E_s}{N_0}\Big|_{\text{dB}} = \frac{E_b}{N_0}\Big|_{\text{dB}} + 10\log_{10}(kR). \tag{20.7}$$

Worked example: converting SNR measures

At $E_b/N_0 = 10$ dB uncoded: QPSK ($k = 2$) has $E_s/N_0 = 10 + 3.01 = 13.01$ dB; 16-QAM ($k = 4$) has $10 + 6.02 = 16.02$ dB. With a rate-1/2 code on 16-QAM, $kR = 2$, so $E_s/N_0 = 13.01$ dB. Conversely, a RadioML sample labelled "SNR $= 0$ dB" with 8 samples per symbol has per-*sample* SNR of 0 dB; per symbol, after a matched filter that collects the 8 samples' energy, E_s/N_0 is up to $10\log_{10} 8 = 9$ dB higher. Always ask "SNR per what?"

20.4.3 Maximum-likelihood and MAP detection

Given y, which symbol was sent? Bayes' rule gives the posterior

$$P(x_m \mid y) = \frac{p(y \mid x_m) P(x_m)}{p(y)}.$$

The **MAP** (maximum a posteriori) detector picks the x_m with the largest posterior; it minimises the probability of symbol error (any other rule picks a less probable symbol at least sometimes). Since $p(y)$ does not depend on m,

$$\hat{x}_{\text{MAP}} = \arg\max_m \; p(y \mid x_m) P(x_m).$$

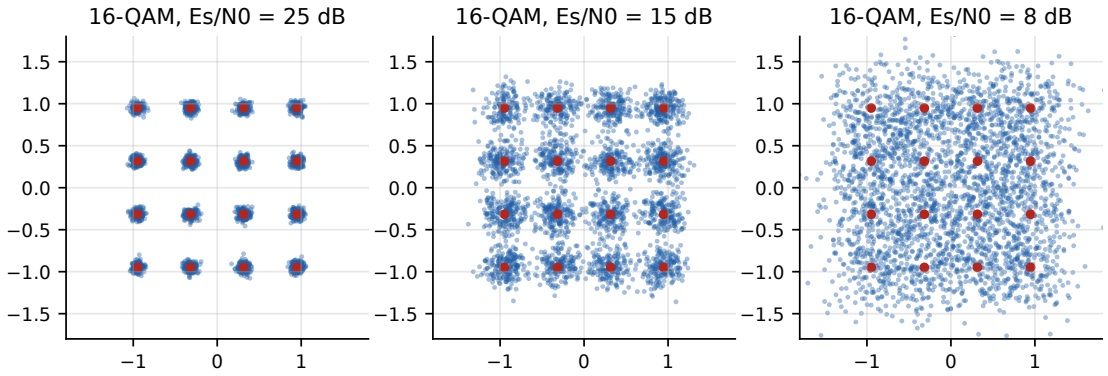


Figure 20.8: Received 16-QAM symbols (blue) around the transmitted points (red) at three values of E_s/N_0. Each point becomes a circular Gaussian cloud whose radius scales with $\sqrt{N_0}$. At 8 dB the clouds overlap heavily and many symbols cross into a neighbour's region.

For equally likely symbols $P(x_m) = 1/M$ is constant, leaving the **ML** (maximum likelihood) rule $\hat{x}_{\mathrm{ML}} = \arg\max_m p(y \mid x_m)$. In AWGN, $p(y \mid x_m) = p_n(y - x_m) = \frac{1}{\pi N_0} \exp(-|y - x_m|^2 / N_0)$. The logarithm is increasing, so maximizing the density is the same as maximizing its log, $-\ln(\pi N_0) - |y - x_m|^2 / N_0$, which is the same as *minimizing* $|y - x_m|^2$:

Key equation: ML detection in AWGN = minimum distance

$$\hat{x}_{\mathrm{ML}} = \arg\min_m |y - x_m|^2, \qquad \hat{x}_{\mathrm{MAP}} = \arg\min_m \left[|y - x_m|^2 - N_0 \ln P(x_m) \right]. \tag{20.8}$$

The decision regions of the minimum-distance rule are the **Voronoi cells** of the constellation: for QPSK, the four quadrants; for 16-QAM, a grid of squares; for M-PSK, pie slices.

Wireless connection: ML bridge: detection = classification

The MAP detector computes $P(x_m \mid y)$ for every class m and takes the argmax. A neural network with a softmax output trained with cross-entropy is trained to output exactly these posteriors (cross-entropy is minimized when the outputs equal the true posteriors). In AWGN the network can at best rediscover the Voronoi cells; under nonlinear distortion or non-Gaussian interference, where no closed form exists, it can do better than the minimum-distance rule.

20.4.4 Error probabilities

Define the Gaussian tail function $Q(u) = P(Z > u) = \int_u^\infty \frac{1}{\sqrt{2\pi}} e^{-z^2/2} dz$ for $Z \sim \mathcal{N}(0,1)$. A Gaussian with standard deviation σ exceeds $a > 0$ with probability $Q(a/\sigma)$.

BPSK. Send $x = \pm\sqrt{E_b}$ (one bit per symbol, so $E_s = E_b$). Only the real part matters; the real noise has variance $\sigma^2 = N_0/2$. Suppose $+\sqrt{E_b}$ was sent; an error occurs if $y = \sqrt{E_b} + n_I < 0$, i.e. $n_I < -\sqrt{E_b}$, which by symmetry has probability

$$P_b = Q\left(\frac{\sqrt{E_b}}{\sqrt{N_0/2}}\right) = Q\left(\sqrt{\frac{2E_b}{N_0}}\right). \tag{20.9}$$

QPSK. With $x = \sqrt{E_s/2}(\pm 1 \pm j)$, the I and Q bits are independent BPSKs, each with amplitude $\sqrt{E_s/2} = \sqrt{E_b}$ (since $E_s = 2E_b$) and noise variance $N_0/2$. So each bit has error probability

$Q(\sqrt{2E_b/N_0})$ — *exactly the same BER as BPSK* at the same E_b/N_0, while carrying twice the bits in the same bandwidth. That is why QPSK is everywhere.

M-PSK. Adjacent points on a circle of radius $\sqrt{E_s}$ separated by angle $2\pi/M$ are $d = 2\sqrt{E_s}\sin(\pi/M)$ apart. The probability of crossing the boundary halfway to one neighbour is $Q\big(\frac{d/2}{\sqrt{N_0/2}}\big) = Q\big(\sqrt{2E_s/N_0}\sin(\pi/M)\big)$; there are two nearest neighbours, so

$$P_s \approx 2Q\left(\sqrt{\frac{2E_s}{N_0}}\sin\frac{\pi}{M}\right), \qquad P_b \approx \frac{P_s}{k}\ \text{(Gray)}.$$

Square M-QAM. On each axis we have $\sqrt{M}$-PAM with levels $\pm d, \pm 3d, \ldots$ Average energy $E_s = \frac{2}{3}(M-1)d^2$ (from the example above, scaled by d^2), so $d^2 = \frac{3E_s}{2(M-1)}$. An inner level has two neighbours, an outer level one; averaging, the per-axis symbol error is $2(1-\frac{1}{\sqrt{M}})Q\big(d/\sqrt{N_0/2}\big)$. Two axes and Gray coding then give

$$P_b \approx \frac{4}{k}\left(1-\frac{1}{\sqrt{M}}\right)Q\left(\sqrt{\frac{3k}{M-1}\frac{E_b}{N_0}}\right), \tag{20.10}$$

where we used $d^2/(N_0/2) = \frac{3E_s}{(M-1)N_0} = \frac{3kE_b}{(M-1)N_0}$.

Worked example: BER numbers

BPSK at $E_b/N_0 = 9.6$ dB: $2E_b/N_0 = 2\times 10^{0.96} = 18.24$, $\sqrt{18.24} = 4.27$, $Q(4.27) \approx 9.7\times 10^{-6}$ — the classical "10^{-5} at 9.6 dB". 16-QAM at $E_b/N_0 = 10$ dB: $k=4$, $M=16$, the argument is $\sqrt{12\times 10/15} = \sqrt{8} = 2.83$, and $P_b \approx \frac{4}{4}\cdot\frac{3}{4}\cdot Q(2.83) = 0.75\times 2.34\times 10^{-3} = 1.75\times 10^{-3}$.

Figure 20.9 compares these formulas with a Monte-Carlo simulation of 6×10^5 bits per point. The markers lie on the lines: the theory is right, and our simulator is right. Note the roughly 4 dB penalty per doubling of constellation order beyond QPSK.

The simulation loop is short enough to print in full; the same five lines are the core of every BER curve in Part V.

```python
import numpy as np
from commpy20 import constellation, bits_to_symbols, detect_bits, awgn
rng = np.random.default_rng(0)
pts, lab = constellation("16QAM")          # unit-energy, Gray-labelled
for ebno_db in [4, 8, 12]:
    bits = rng.integers(0, 2, 400_000)
    x = bits_to_symbols(bits, pts, lab)                  # 4 bits -> 1 symbol
    y, N0 = awgn(x, ebno_db + 10*np.log10(4), rng)       # Es/N0 = Eb/N0 + 6 dB
    ber = np.mean(detect_bits(y, pts, lab) != bits)      # minimum distance
    print(ebno_db, ber)
```

20.5 Real channels: path loss, shadowing and fading

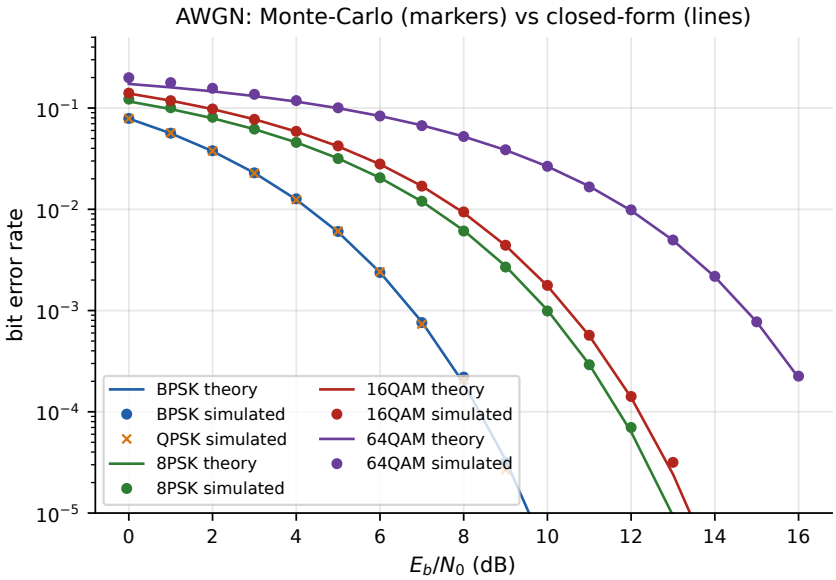


Figure 20.9: BER versus E_b/N_0 in AWGN. Lines: closed forms (20.9)–(20.10). Markers: our Monte-Carlo simulation with minimum-distance detection. BPSK and QPSK coincide (crosses on the blue line). Every curve falls "like a waterfall" — exponentially in SNR — which is the signature of the Gaussian tail $Q(u) \le \frac{1}{2}e^{-u^2/2}$.

20.5.1 Large-scale effects: path loss and shadowing

Intuition: spreading energy

An antenna radiating P_t watts equally in all directions spreads it over a sphere of area $4\pi d^2$ at distance d. A receive antenna collects the power falling on its *effective area*, which for an isotropic antenna is $\lambda^2/(4\pi)$. Received power therefore falls as $1/d^2$ — and, at fixed antenna gain, also as λ^2, i.e. as $1/f^2$: higher carriers "lose" more, the motivation for the large antenna arrays of millimetre-wave 5G.

Multiplying power density $P_tG_t/(4\pi d^2)$ by effective area $G_r\lambda^2/(4\pi)$ gives **Friis' free-space equation**

$$P_r = P_tG_tG_r\left(\frac{\lambda}{4\pi d}\right)^2, \qquad \mathrm{FSPL}_{\mathrm{dB}} = 20\log_{10}\frac{4\pi d}{\lambda}. \tag{20.11}$$

At 3.5 GHz ($\lambda = 0.0857$ m), $\mathrm{FSPL}(1\,\mathrm{m}) = 43.3$ dB and $\mathrm{FSPL}(100\,\mathrm{m}) = 83.3$ dB: 20 dB per decade of distance. Real environments with ground reflection and obstacles lose faster; the empirical **log-distance model** with **shadowing** is

$$\mathrm{PL}(d)_{\mathrm{dB}} = \mathrm{PL}(d_0)_{\mathrm{dB}} + 10\,n\log_{10}\frac{d}{d_0} + X_\sigma, \qquad X_\sigma \sim \mathcal{N}(0,\sigma^2), \tag{20.12}$$

with path-loss exponent $n \approx 2$ (free space) to 4 (dense urban, indoor through walls) and shadowing standard deviation $\sigma \approx 4$–10 dB caused by buildings and bodies (Fig. 20.10). Shadowing is *log-normal*: Gaussian in dB.

20.5.2 Small-scale effects: multipath

The signal reaches the receiver along many paths — direct, reflected from walls, scattered from cars — each with its own amplitude a_i, delay τ_i and Doppler shift. The baseband channel impulse

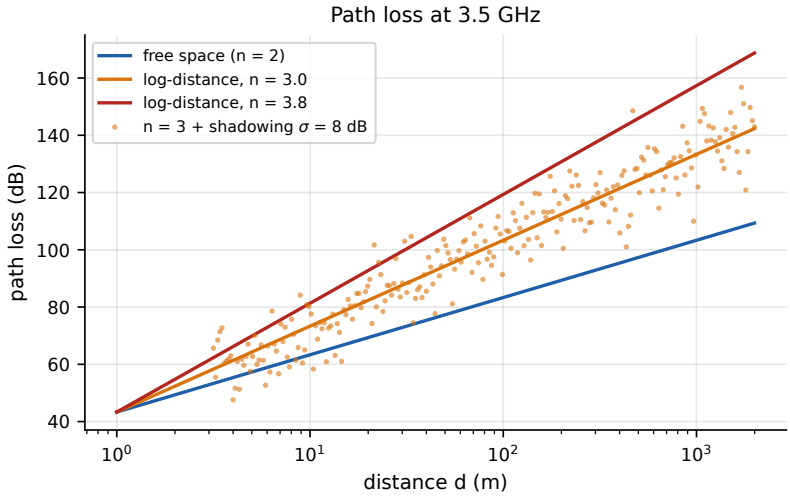


Figure 20.10: Path loss at 3.5 GHz: Friis free space, log-distance with exponents 3 and 3.8, and random log-normal shadowing around the $n = 3$ line. RSS-based localization (Chapter 29) inverts exactly this relationship to turn received power into distance — and shadowing's scatter is why it is hard.

response is

$$h(\tau, t) = \sum_{i=1}^{N} a_i \, e^{-j2\pi f_c \tau_i(t)} \, \delta\big(\tau - \tau_i(t)\big). \tag{20.13}$$

The phase factor $e^{-j2\pi f_c \tau_i}$ is crucial: a change in path length of just $\lambda/2$ (4 cm at 3.5 GHz) changes a path's phase by 180°. Paths that added constructively can cancel a few centimetres away. This is **fading**.

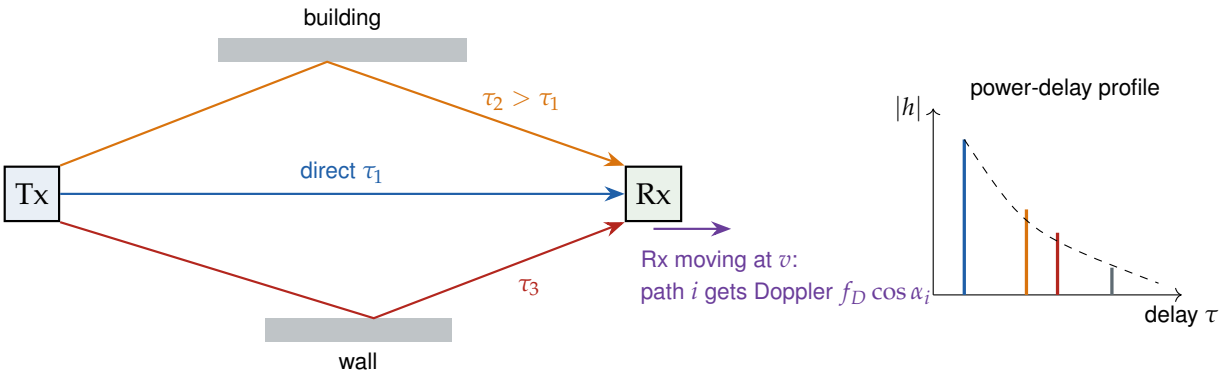


Figure 20.11: Multipath propagation. Each path has its own delay, amplitude and phase; if the receiver moves, each path's phase rotates at its own Doppler frequency. The *power-delay profile* (right) summarises average power versus delay; its spread determines frequency selectivity.

Delay spread and coherence bandwidth

If the delays τ_i differ by much less than a symbol period, all echoes arrive "at once" and the channel just multiplies each symbol by one complex number h: **flat fading**. If they differ by a symbol or more, echoes of earlier symbols land on later ones: ISI, or **frequency-selective fading**. The **rms delay spread** σ_τ is the standard deviation of delay weighted by path power. The frequency response $H(f) = \sum_i a_i e^{-j2\pi(f_c+f)\tau_i}$ changes appreciably over frequency differences of order $1/\sigma_\tau$; the **coherence bandwidth** is commonly approximated as $B_c \approx 1/(5\sigma_\tau)$ (frequencies

this far apart are about 50% correlated). Indoor $\sigma_\tau \approx 50\,\text{ns}$ gives $B_c \approx 4\,\text{MHz}$; urban macro $\sigma_\tau \approx 1\,\mu\text{s}$ gives $B_c \approx 200\,\text{kHz}$. A signal with bandwidth $B \ll B_c$ sees flat fading; $B \gg B_c$ sees frequency selectivity.

Doppler and coherence time

A receiver moving at speed v sees a path arriving at angle α_i to its direction of motion with Doppler shift $f_D \cos\alpha_i$, where $f_D = v/\lambda$ is the **maximum Doppler frequency**. At 120 km/h (33.3 m/s) and 3.5 GHz, $f_D = 33.3/0.0857 = 389\,\text{Hz}$. The channel therefore changes on a time scale of the **coherence time** $T_c \approx 0.423/f_D \approx 1.1\,\text{ms}$ — comparable to a 5G slot. This is why CSI goes stale and why CSI *prediction* (Chapter 27) is valuable.

Clarke/Jakes model. Suppose many paths arrive with angles uniformly distributed over $[0, 2\pi)$ and equal power. A path at angle α has Doppler $f = f_D \cos\alpha$. The power arriving with Doppler in $[f, f+df]$ is proportional to the probability of angles mapping there. Since α is uniform with density $\frac{1}{2\pi}$ and both α and $-\alpha$ map to the same f, the density of f is $p(f) = 2 \cdot \frac{1}{2\pi}\left|\frac{d\alpha}{df}\right|$. From $f = f_D\cos\alpha$, $|df/d\alpha| = f_D|\sin\alpha| = f_D\sqrt{1-(f/f_D)^2}$. Hence

$$S(f) = \frac{1}{\pi f_D \sqrt{1-(f/f_D)^2}}, \qquad |f| < f_D, \tag{20.14}$$

the U-shaped **Jakes spectrum**, with infinite peaks at $\pm f_D$ (paths arriving head-on or from behind). Its inverse Fourier transform, the autocorrelation of $h(t)$, is $J_0(2\pi f_D \Delta t)$, a Bessel function whose first zero at $\Delta t \approx 0.38/f_D$ explains the size of T_c. Figure 20.12 shows a simulated fading trace and its spectrum.

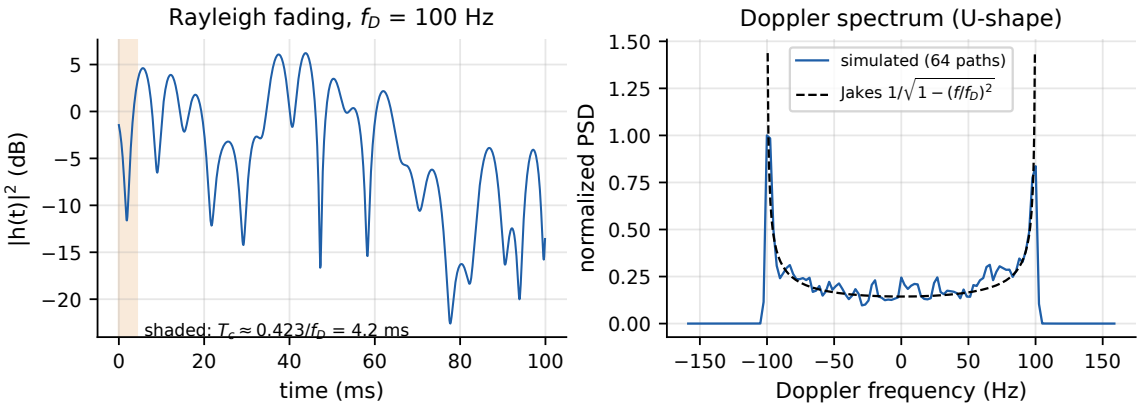


Figure 20.12: A sum of 64 Doppler-shifted paths with uniform angles (our simulation, $f_D = 100\,\text{Hz}$). **Left:** received power over 100 ms: deep fades of 20 dB or more occur several times; the shaded band is one coherence time. **Right:** averaged power spectrum versus the Jakes formula (20.14). The U shape is the fingerprint of mobile fading.

20.5.3 Rayleigh and Rician fading from the central limit theorem

Consider flat fading at one instant: $h = \sum_{i=1}^{N} a_i e^{j\phi_i}$ with phases $\phi_i = -2\pi f_c \tau_i$ that, because $f_c\tau_i$ spans many cycles, are effectively independent and uniform on $[0, 2\pi)$. Each term has zero mean (a uniform phase averages $e^{j\phi}$ to zero) and, by symmetry, its real and imaginary parts are uncorrelated with equal variance $a_i^2/2$. By the central limit theorem, for large N,

$$h \approx \mathcal{CN}(0, \Omega), \qquad \Omega = \sum_i a_i^2 = \mathbb{E}\,|h|^2 .$$

Key equation: Rayleigh envelope and uniform phase

Write $h = X + jY = re^{j\theta}$ with $X, Y \sim \mathcal{N}(0, \Omega/2)$ independent. The joint density is $p(x,y) = \frac{1}{\pi\Omega} e^{-(x^2+y^2)/\Omega}$. Change to polar coordinates: $x = r\cos\theta$, $y = r\sin\theta$, area element $dx\,dy = r\,dr\,d\theta$, so

$$p(r,\theta) = \frac{r}{\pi\Omega} e^{-r^2/\Omega} = \underbrace{\frac{2r}{\Omega} e^{-r^2/\Omega}}_{p(r)\text{: Rayleigh}} \cdot \underbrace{\frac{1}{2\pi}}_{p(\theta)\text{: uniform}}, \qquad r \geq 0,\ \theta \in [0, 2\pi). \tag{20.15}$$

The envelope $r = |h|$ is **Rayleigh**, the phase is uniform and independent of it, and the power $\gamma = r^2$ is **exponential**: $p(\gamma) = \frac{1}{\Omega} e^{-\gamma/\Omega}$.

The last claim: $P(r^2 \leq g) = P(r \leq \sqrt{g}) = \int_0^{\sqrt{g}} \frac{2r}{\Omega} e^{-r^2/\Omega} dr = 1 - e^{-g/\Omega}$, whose derivative is $\frac{1}{\Omega} e^{-g/\Omega}$. An exponential power means deep fades are common: $P(\gamma < \Omega/10) = 1 - e^{-0.1} \approx 9.5\%$ — one in ten moments sees a 10 dB fade; $P(\gamma < \Omega/100) \approx 1\%$.

If one path is strong and stable (line of sight) with amplitude μ, $h = \mu + \mathcal{CN}(0, \sigma^2)$ and $|h|$ is **Rician** with K-factor $K = \mu^2/\sigma^2$ (LOS power over scattered power):

$$p(r) = \frac{2(K+1)r}{\Omega} \exp\left(-K - \frac{(K+1)r^2}{\Omega}\right) I_0\left(2r\sqrt{\frac{K(K+1)}{\Omega}}\right),$$

where I_0 is the modified Bessel function of the first kind. $K = 0$ recovers Rayleigh; $K \to \infty$ is no fading. Figure 20.13 confirms the CLT argument numerically: with only two paths the envelope is far from Rayleigh, but by five it is close, and fifty paths match it perfectly.

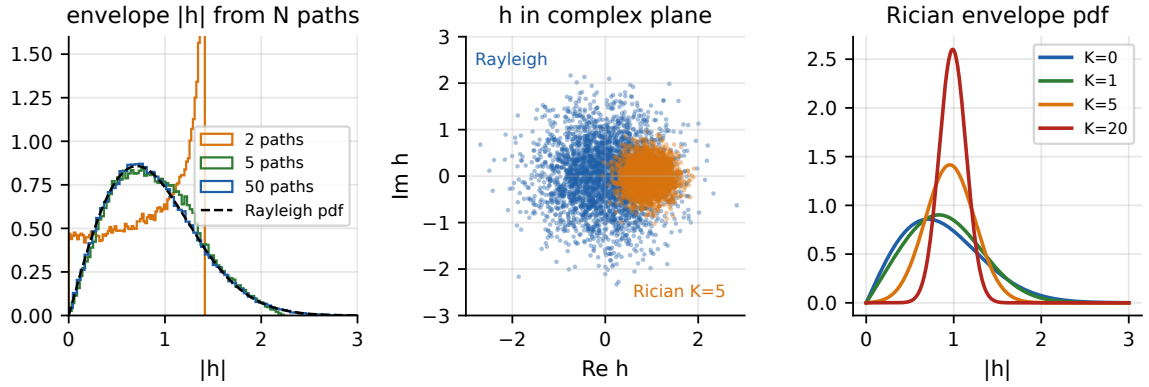


Figure 20.13: Rayleigh and Rician fading emerge from sums of random-phase paths. **Left:** histogram of $|h|$ for 2, 5 and 50 equal-power paths versus the Rayleigh pdf (2 paths: a peak near $|h| = \sqrt{2}$ where the two phasors align). **Middle:** realizations of h: Rayleigh is a circular Gaussian cloud at the origin; Rician with $K = 5$ is a cloud displaced by the LOS component. **Right:** Rician envelope pdfs; larger K concentrates the envelope near 1 (less fading).

20.5.4 BER in Rayleigh fading and the idea of diversity

With flat fading $y = hx + n$ and perfect knowledge of h at the receiver, BPSK sees an instantaneous SNR $\gamma = |h|^2 E_b/N_0$ and error probability $Q(\sqrt{2\gamma})$. Averaging over the exponential distribution

of γ with mean $\bar{\gamma}$:

$$\bar{P}_b = \int_0^\infty Q(\sqrt{2\gamma})\frac{1}{\bar{\gamma}}e^{-\gamma/\bar{\gamma}}d\gamma = \Big[-Q(\sqrt{2\gamma})e^{-\gamma/\bar{\gamma}}\Big]_0^\infty + \int_0^\infty e^{-\gamma/\bar{\gamma}}\,\frac{d}{d\gamma}Q(\sqrt{2\gamma})\,d\gamma$$
$$= \frac{1}{2} - \int_0^\infty e^{-\gamma/\bar{\gamma}}\frac{1}{\sqrt{2\pi}}e^{-\gamma}\frac{1}{\sqrt{2\gamma}}d\gamma = \frac{1}{2} - \frac{1}{2\sqrt{\pi}}\int_0^\infty \gamma^{-1/2}e^{-\gamma(1+1/\bar{\gamma})}d\gamma.$$

We integrated by parts and used $\frac{d}{d\gamma}Q(\sqrt{2\gamma}) = -\frac{1}{\sqrt{2\pi}}e^{-\gamma}\cdot\frac{1}{\sqrt{2\gamma}}$ (chain rule: $Q'(u) = -\frac{1}{\sqrt{2\pi}}e^{-u^2/2}$ and $\frac{d}{d\gamma}\sqrt{2\gamma} = \frac{1}{\sqrt{2\gamma}}$). The remaining integral is $\int_0^\infty \gamma^{-1/2}e^{-c\gamma}d\gamma = \sqrt{\pi/c}$ with $c = 1 + 1/\bar{\gamma}$. Therefore

Key equation: BPSK in Rayleigh fading

$$\bar{P}_b = \frac{1}{2}\left(1 - \sqrt{\frac{\bar{\gamma}}{1+\bar{\gamma}}}\right) \approx \frac{1}{4\bar{\gamma}} \quad (\bar{\gamma} \gg 1). \tag{20.16}$$

The approximation uses $\sqrt{\bar{\gamma}/(1+\bar{\gamma})} = (1+1/\bar{\gamma})^{-1/2} \approx 1 - \frac{1}{2\bar{\gamma}}$.

Worked example: the price of fading

At $\bar{\gamma} = 20\,\text{dB} = 100$: exact $\bar{P}_b = 0.5(1 - \sqrt{100/101}) = 2.48 \times 10^{-3}$; approximation $1/400 = 2.5 \times 10^{-3}$. In AWGN the same BER needs only 6.0 dB. Fading costs 14 dB here, and at 10^{-5} the gap exceeds 30 dB.

Why so bad? The BER falls only as $1/\bar{\gamma}$ — ten times more power buys ten times fewer errors — instead of exponentially. Errors are dominated by the deep fades, whose probability is $P(\gamma < \epsilon) \approx \epsilon/\bar{\gamma}$. The cure is **diversity**: send the information over L independently fading paths (antennas, frequencies, time slots) so that all must fade at once for an error. With maximal-ratio combining (MRC) of L independent branches, $\bar{P}_b \propto \bar{\gamma}^{-L}$; the exponent L is the **diversity order**, visible as the slope on a log–log plot (Fig. 20.14). OFDM with coding gains frequency diversity, MIMO gains spatial diversity, interleaving plus coding gains time diversity.

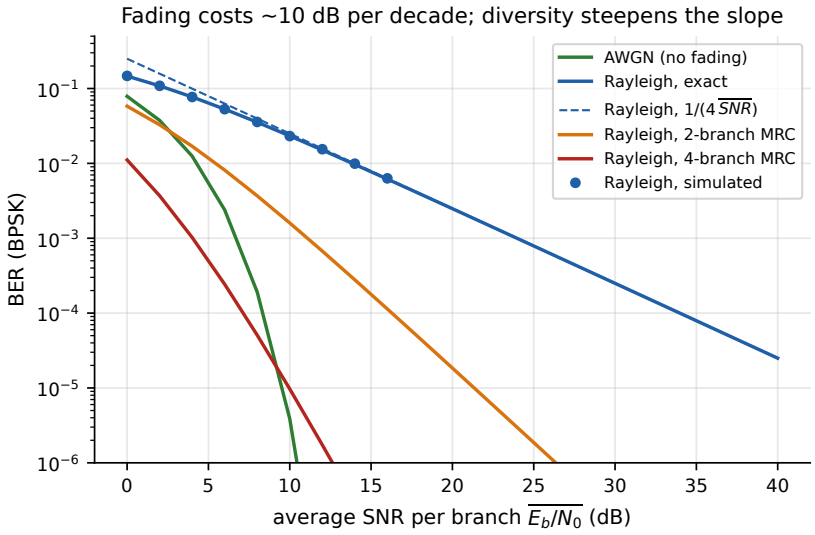


Figure 20.14: BPSK BER in Rayleigh fading. The exact curve (20.16) (blue) matches our simulation (dots) and the $1/(4\bar{\gamma})$ asymptote: one decade per 10 dB. AWGN (green) falls exponentially. Two- and four-branch MRC receive diversity steepen the slope to two and four decades per 10 dB.

20.5.5 Frequency-selective channels as FIR and IIR filters

Sampling (20.13) at the symbol rate turns the channel into a discrete-time filter. The **tapped-delay-line** (TDL) model is an FIR filter:

$$y[n] = \sum_{\ell=0}^{L-1} h_\ell \, x[n-\ell] + w[n], \qquad h_\ell \sim \mathcal{CN}(0, P_\ell), \tag{20.17}$$

where P_ℓ is the power-delay profile; a common choice is exponential, $P_\ell \propto e^{-\ell/\tau_{\text{rms}}}$. Each tap fades independently (Rayleigh), and the frequency response $H(f) = \sum_\ell h_\ell e^{-j2\pi f \ell}$ has notches wherever taps cancel. 3GPP's TDL-A/B/C and CDL models are exactly this, with standardized delays and powers.

Some labs use an **IIR-type channel**: a pole–zero filter

$$y[n] = \sum_{k=0}^{q} b_k x[n-k] - \sum_{k=1}^{p} a_k y[n-k] + w[n], \qquad H(z) = \frac{\sum_k b_k z^{-k}}{1 + \sum_k a_k z^{-k}}. \tag{20.18}$$

The feedback terms make the impulse response infinitely long. Each pole p_i (root of the denominator) contributes a term $\propto p_i^n$, which decays only as $|p_i|^n$; a pole of magnitude 0.92 still has 1% amplitude after $\ln 0.01 / \ln 0.92 \approx 55$ samples. Poles close to the unit circle create sharp **resonances** (peaks) in $|H(f)|$ at frequency $\angle p_i / (2\pi)$; zeros create notches. Such a channel models long reverberant echoes (e.g. in enclosed metallic environments) compactly, and it is harder for a classifier because ISI extends over far more symbols than a short FIR window. Figure 20.16 compares the two models.

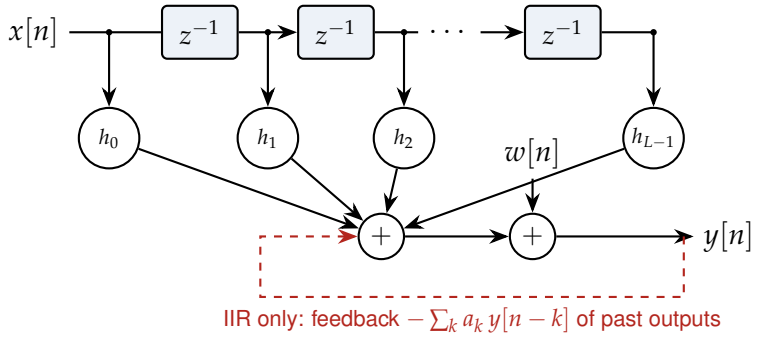


Figure 20.15: The FIR tapped-delay-line channel (20.17): delayed copies of the input, each weighted by a random complex gain h_ℓ, are summed. An IIR channel (20.18) also feeds back weighted past outputs, which makes the echo tail infinitely long. Note that this structure is exactly a 1-D convolution — the operation a CNN layer performs.

```python
import numpy as np
from scipy.signal import lfilter
rng = np.random.default_rng(1)
def rayleigh_fir(x, L=8, tau=2.0):            # exponential-PDP TDL channel
    p = np.exp(-np.arange(L)/tau); p /= p.sum()
    h = np.sqrt(p/2)*(rng.standard_normal(L) + 1j*rng.standard_normal(L))
    return np.convolve(x, h)[:len(x)]
def iir_channel(x, poles=(0.92*np.exp(0.6j), 0.85*np.exp(-1.9j)), zeros=(0.3-0.2j,)):
    a, b = np.poly(poles), np.poly(zeros)     # H(z) = B(z)/A(z)
    return lfilter(b, a, x)
```

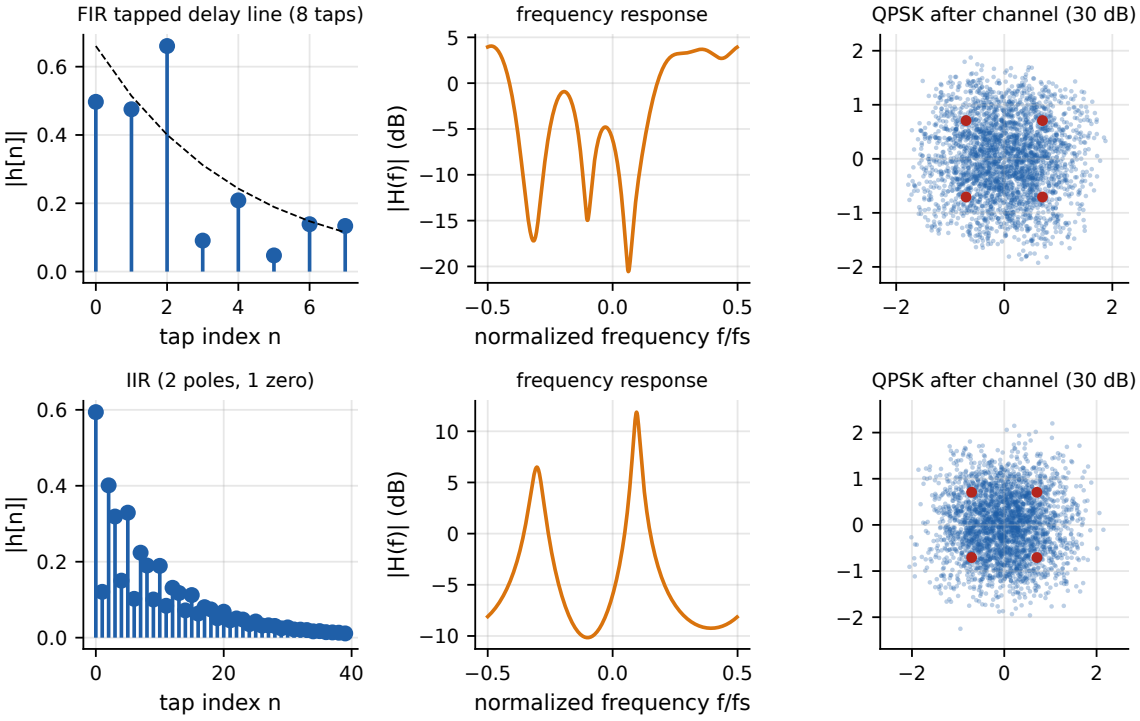


Figure 20.16: **Top:** an 8-tap Rayleigh FIR channel with exponential power-delay profile (dashed envelope): its frequency response has deep notches, and QPSK at 30 dB SNR is smeared into a blob by ISI. **Bottom:** an IIR channel with two poles ($|p| = 0.92, 0.85$) and one zero: the impulse response rings for tens of samples, the frequency response shows two resonant peaks, and the constellation is again destroyed. Without equalization, neither constellation is detectable — which is why modulation classifiers trained only on AWGN fail on these channels (Part VI).

20.6 Hardware impairments

Even with a perfect channel, two imperfect radios distort the signal. Each impairment leaves a characteristic fingerprint on the constellation (Fig. 20.17); learning to see them is a skill every PHY engineer has and every neural receiver must acquire.

- **Carrier frequency offset (CFO).** The receiver's oscillator runs at $f_c + \Delta f$. After down-conversion, $y[n] = x[n]e^{j2\pi\Delta f\, nT} + w[n]$: the constellation *rotates* steadily, by $360^\circ \Delta f\, T$ per symbol. A 1 ppm crystal error at 3.5 GHz is 3.5 kHz, i.e. a full turn every 0.29 ms. Over a window, points smear into rings.
- **Phase noise.** The oscillator phase jitters randomly: $y[n] = x[n]e^{j\varphi[n]} + w[n]$. Points spread *tangentially* into arcs; outer points, with larger radius, spread more in distance. Phase noise grows with carrier frequency and limits high-order QAM at millimetre wave.
- **Timing offset.** Sampling at $nT + \epsilon$ instead of nT violates the zero-ISI condition (20.4): each sample picks up pieces of neighbours, so points blur in all directions.
- **IQ imbalance.** The I and Q branches of Fig. 20.3 have slightly different gains ($1 \pm \epsilon$) or are not exactly 90° apart (phase error ϕ). The square grid becomes a stretched, sheared parallelogram. Mathematically $y = \mu x + \nu x^*$: a mirror-image "ghost" x^* leaks in.
- **Power-amplifier (PA) nonlinearity.** Amplifiers saturate. AM/AM compression (Rapp model: $g(r) = r/(1 + (r/A_{\text{sat}})^{2p})^{1/(2p)}$) pulls the outer points inwards; AM/PM conversion rotates high-amplitude points more than low ones. Constant-envelope signals (PSK, FSK) suffer least — the deep reason FSK-family signals survive in cheap radios.

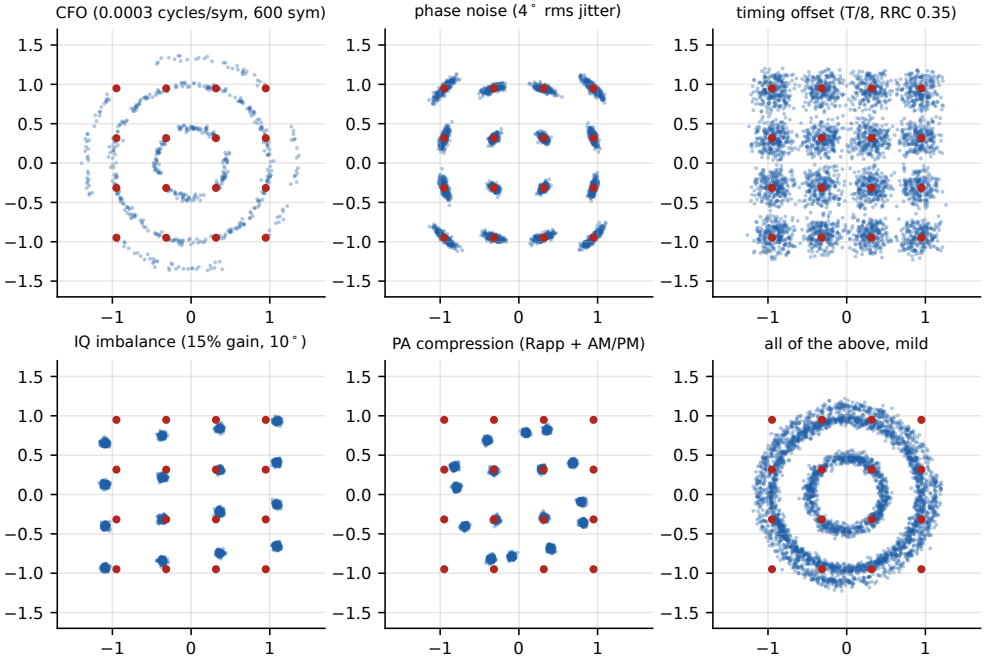


Figure 20.17: Fingerprints of hardware impairments on 16-QAM at 30 dB SNR (our simulation; red: ideal points). CFO turns points into rings; phase noise into arcs; timing offset into isotropic blur; IQ imbalance into a skewed grid; PA saturation compresses and twists the outer ring. Real signals carry several at once (bottom right). Synchronization algorithms estimate and remove CFO, phase and timing; IQ and PA effects need calibration or digital pre-distortion — or a receiver that learns them.

Wireless connection: ML bridge: impairments as nuisance variables and data augmentation

For a modulation classifier, CFO, phase and timing offsets are *nuisance parameters*: they change the input but not the label. Two strategies, both used later: (1) *augment* training data with random impairments so the network learns invariance (RadioML-style generators apply random CFO, timing drift and multipath); (2) build the invariance into the architecture or the input representation (e.g. amplitude/phase or higher-order statistics, which are rotation-invariant).

20.7 Equalization, OFDM and channel estimation

20.7.1 Linear equalizers: ZF, MMSE and LMS

After a frequency-selective channel, $y[n] = \sum_\ell h_\ell x[n-\ell] + w[n]$. An **equalizer** is a filter $\mathbf{w}$ (length L_w) applied to the window $\mathbf{y}_n = [y[n], y[n-1], \ldots, y[n-L_w+1]]^\mathsf{T}$ to estimate a delayed symbol: $\hat{x}[n-D] = \mathbf{w}^\mathsf{T}\mathbf{y}_n$.

- **Zero forcing (ZF)** chooses $\mathbf{w}$ so that the combined channel–equalizer response is a pure delay, ignoring noise. In the frequency domain $W(f) = 1/H(f)$. Where $H(f)$ has a deep notch, $1/H(f)$ is huge and the noise there is amplified enormously — ZF's weakness.
- **MMSE (Wiener)** minimises $\mathbb{E}\,|x[n-D] - \mathbf{w}^\mathsf{T}\mathbf{y}_n|^2$. Setting the gradient to zero (exactly as for linear regression in Part II) gives the normal equations

$$\mathbf{w}_{\text{MMSE}} = \mathbf{R}_{yy}^{-1}\mathbf{r}_{yx}, \qquad \mathbf{R}_{yy} = \mathbb{E}[\mathbf{y}_n^*\mathbf{y}_n^\mathsf{T}],\ \mathbf{r}_{yx} = \mathbb{E}[\mathbf{y}_n^* x[n-D]]. \tag{20.19}$$

  In the frequency domain $W(f) = H^*(f)/(|H(f)|^2 + N_0/E_s)$: like ZF when the channel is strong, but it backs off when $|H(f)|^2$ falls below the noise level.
- **LMS** (least mean squares) finds $\mathbf{w}_{\text{MMSE}}$ without knowing the statistics, by stochastic gradient descent on the instantaneous squared error $|e[n]|^2$, $e[n] = x[n-D] - \mathbf{w}^\mathsf{T}\mathbf{y}_n$ (using known training symbols):

$$\mathbf{w} \leftarrow \mathbf{w} + \mu\, e[n]\, \mathbf{y}_n^*. \tag{20.20}$$

  This is *exactly* the SGD update of a single linear neuron with squared loss (Chapter 9). The LMS equalizer, invented by Widrow and Hoff in 1960, is arguably the first neural network deployed at scale: every dial-up modem had one.

Figure 20.18 shows an 11-tap equalizer on a three-tap channel at 18 dB: the received QPSK clouds are smeared by ISI; after MMSE equalization they are tight again; the LMS learning curve descends to the Wiener MSE in about 2000 symbols. On this mild channel ZF is only slightly worse than MMSE; on a channel with a deep notch the gap would be large.

20.7.2 OFDM: turning a hard channel into many easy ones

Intuition: many slow lanes instead of one fast lane

A long ISI channel is hard because each symbol is short compared with the echoes. OFDM (orthogonal frequency-division multiplexing) sends N symbols *in parallel*, each on its own narrow subcarrier, so that each symbol lasts N times longer. Each subcarrier's band is narrower than the coherence bandwidth, so each sees flat fading — one complex gain. A

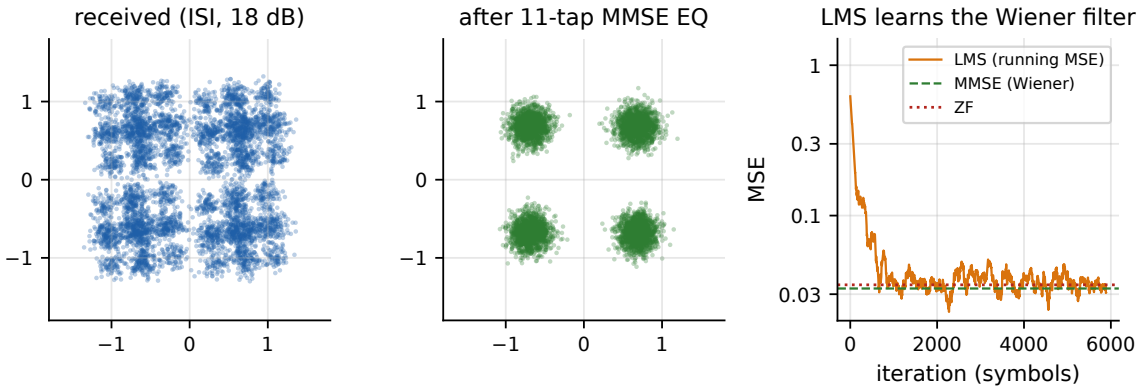


Figure 20.18: Linear equalization of the channel $h \propto [0.3,\ 1,\ 0.3e^{j0.3}]$ (our simulation). **Left:** received QPSK with ISI. **Middle:** after an 11-tap MMSE equalizer. **Right:** LMS ($\mu = 0.01$) running mean-squared error converges to the MMSE (Wiener) level computed from (20.19); ZF's MSE is slightly higher.

short guard interval (the cyclic prefix) absorbs the echoes. Equalization collapses to one division per subcarrier.

Transmitter. Take N frequency-domain symbols $X[0], \ldots, X[N-1]$ and compute the inverse DFT

$$x[n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[k] e^{j2\pi kn/N}, \qquad n = 0, \ldots, N-1.$$

Each $X[k]$ modulates a complex exponential at frequency k/N cycles per sample. Then prepend a **cyclic prefix** (CP): copy the last N_{cp} samples to the front, $[x[N-N_{\text{cp}}], \ldots, x[N-1], x[0], \ldots, x[N-1]]$.

Why the CP works (derivation). The channel performs *linear* convolution with $h_0, \ldots, h_{L-1}$. Assume $N_{\text{cp}} \geq L-1$. The receiver discards the first N_{cp} samples and keeps $r[n]$ for $n = 0, \ldots, N-1$:

$$r[n] = \sum_{\ell=0}^{L-1} h_\ell \, \tilde{x}[n-\ell] + w[n],$$

where $\tilde{x}$ is the transmitted sequence with prefix. For $n - \ell < 0$ (reaching back into the prefix), the prefix makes $\tilde{x}[n-\ell] = x[n-\ell+N]$; for $n - \ell \geq 0$, $\tilde{x}[n-\ell] = x[n-\ell]$. Both cases are $x[(n-\ell) \bmod N]$. Because $\ell \leq L-1 \leq N_{\text{cp}}$, we never reach before the start of the prefix, so the previous OFDM symbol does not leak in. Hence

$$r[n] = \sum_{\ell=0}^{L-1} h_\ell \, x[(n-\ell) \bmod N] + w[n] = (h \circledast x)[n] + w[n],$$

a **circular convolution**. The DFT turns circular convolution into multiplication (the convolution theorem for the DFT). Taking the DFT, $R[k] = \frac{1}{\sqrt{N}} \sum_n r[n] e^{-j2\pi kn/N}$:

Key equation: OFDM: one complex gain per subcarrier

$$R[k] = H[k]\, X[k] + W[k], \qquad H[k] = \sum_{\ell=0}^{L-1} h_\ell e^{-j2\pi k\ell/N}, \qquad W[k] \sim \mathcal{CN}(0, N_0), \tag{20.21}$$

so the **one-tap equalizer** $\hat{X}[k] = R[k]/H[k]$ (ZF) or $H^*[k]R[k]/(|H[k]|^2 + N_0)$ (MMSE) re-

moves the channel.

For completeness, here is the convolution-theorem step. Substitute $r = h \circledast x$ into the DFT and write $(m)_N$ for m mod N:

$$\frac{1}{\sqrt{N}} \sum_n \sum_\ell h_\ell \, x[(n-\ell)_N] \, e^{-j2\pi kn/N} = \sum_\ell h_\ell e^{-j2\pi k\ell/N} \cdot \frac{1}{\sqrt{N}} \sum_n x[(n-\ell)_N] \, e^{-j2\pi k(n-\ell)/N}$$
$$= H[k] \, X[k],$$

because the inner sum over a full period is unchanged by the circular shift. The price of the CP is overhead: $N_{\text{cp}}/(N + N_{\text{cp}})$ of time and energy (e.g. $16/80 = 20\%$ for $N = 64$, $N_{\text{cp}} = 16$; about 7% in 5G NR with normal CP).

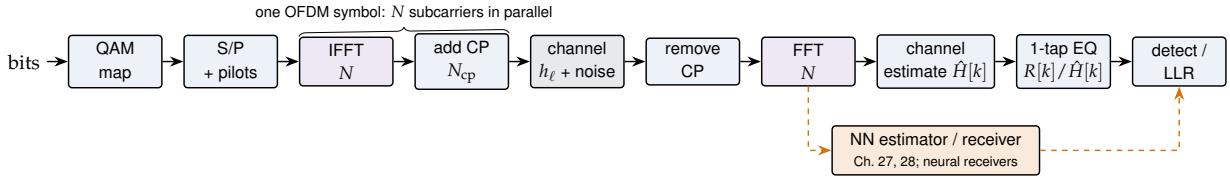


Figure 20.19: An OFDM link. The IFFT and FFT (purple) and the cyclic prefix convert a frequency-selective channel into N parallel flat channels (20.21). Pilots inserted on known subcarriers allow the channel to be estimated. The dashed orange path shows where neural channel estimators and neural receivers plug in.

20.7.3 Pilots and channel estimation: LS and LMMSE

The receiver needs $H[k]$. The transmitter inserts known **pilot** (reference) symbols X_p on a subset $\mathcal{P}$ of subcarriers (and OFDM symbols).

Least squares (LS). From (20.21), at a pilot, $\hat{H}_{\text{LS}}[k] = R[k]/X_p[k] = H[k] + W[k]/X_p[k]$: unbiased, with error variance $N_0/\left|X_p\right|^2$. Between pilots one interpolates (linearly or otherwise).

Linear MMSE (LMMSE). Exploit the fact that $H[k]$ at nearby subcarriers are correlated (within the coherence bandwidth). Let $\mathbf{h}$ be all channel values, $\hat{\mathbf{h}}_p$ the LS estimates at pilots, $\mathbf{R}_{hh_p} = \mathbb{E}[\mathbf{h}\mathbf{h}_p^{\mathsf{H}}]$ and $\mathbf{R}_{h_ph_p} = \mathbb{E}[\mathbf{h}_p\mathbf{h}_p^{\mathsf{H}}]$. The linear estimator minimizing mean squared error is (the same normal-equation derivation as (20.19))

$$\hat{\mathbf{h}}_{\text{LMMSE}} = \mathbf{R}_{hh_p}\left(\mathbf{R}_{h_ph_p} + \tfrac{N_0}{E_p}\mathbf{I}\right)^{-1}\hat{\mathbf{h}}_p. \tag{20.22}$$

For a TDL channel with power-delay profile P_ℓ, $\mathbb{E}[H[k]H^*[k']] = \sum_\ell P_\ell e^{-j2\pi(k-k')\ell/N}$, so the correlation matrices follow directly from the PDP. LMMSE both denoises and interpolates; its cost is needing the second-order statistics and a matrix inverse.

Wireless connection: ML bridge: channel estimation = regression

Channel estimation maps noisy pilot observations to a complex vector $\hat{\mathbf{h}}$ — a regression problem with mean-squared-error loss. LMMSE is the best *linear* regressor; a neural network can beat it when the channel's distribution is not Gaussian or its statistics are unknown, by learning the structure of real channels from data (treating the time–frequency grid as an image is a popular trick). CSI *prediction* (Chapter 27) is regression in time: forecast $\mathbf{h}$ at future slots from past ones — a sequence model's job.

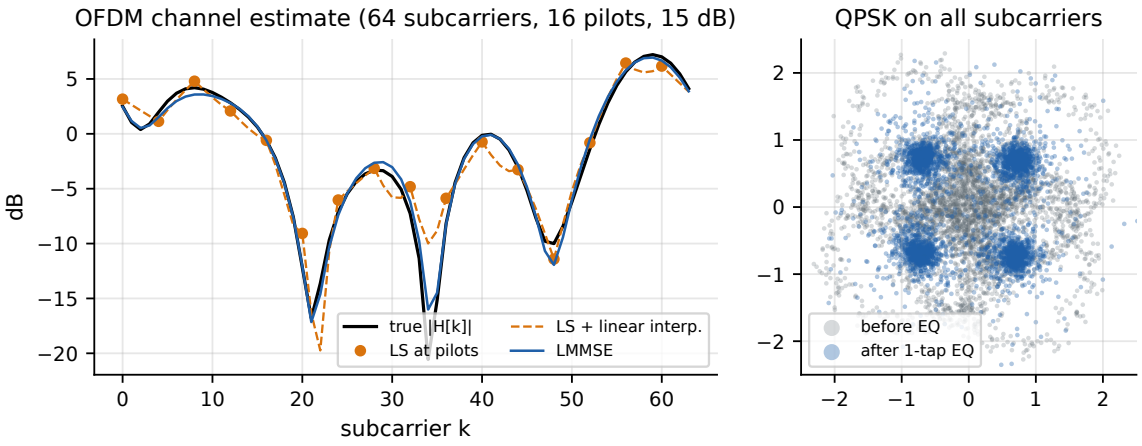


Figure 20.20: OFDM channel estimation (our simulation: 64 subcarriers, 8-tap exponential-PDP Rayleigh channel, comb pilots every 4th subcarrier, 15 dB). **Left:** LS estimates at pilots are noisy and linear interpolation misses the notch near $k = 50$; LMMSE (using the PDP) tracks the true $|H[k]|$ closely (MSE 0.032 for LS vs 0.0067 for LMMSE in this run). **Right:** QPSK before (gray) and after (blue) one-tap equalization with the LMMSE estimate; the residual spread comes from subcarriers in deep fades.

20.7.4 What CSI and RSS mean in practice

Channel state information (CSI) is any knowledge of the channel: at its richest, the complex gain $H[k]$ for every subcarrier k and every transmit/receive antenna pair, i.e. a tensor of shape $N_r \times N_t \times N_{\text{sc}}$. **Received signal strength (RSS)**, reported by almost every radio as the RSSI (indicator, in dBm), is a single number: total received power averaged over the band and over a packet. RSS is coarse — it hides multipath structure and fluctuates with fading — but universally available; CSI is rich but requires PHY-level access.

WiFi CSI became widely available to researchers through tools that expose the estimates computed by commercial cards. The best known, the Linux 802.11n CSI Tool for the Intel 5300 card, reports CSI for **30 subcarrier groups** per antenna pair (grouping $N_g = 2$ in 20 MHz, $N_g = 4$ in 40 MHz), i.e. a complex matrix of size $N_{\text{tx}} \times N_{\text{rx}} \times 30$ per packet. Fingerprinting localization (Chapter 29) learns the mapping from these CSI (or RSS) vectors to position.

20.8 MIMO: multiple antennas

20.8.1 The model

With N_t transmit and N_r receive antennas and flat fading (or one OFDM subcarrier),

$$\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}, \qquad \mathbf{y}, \mathbf{n} \in \mathbb{C}^{N_r},\ \mathbf{x} \in \mathbb{C}^{N_t},\ \mathbf{H} \in \mathbb{C}^{N_r \times N_t},\ \mathbf{n} \sim \mathcal{CN}(\mathbf{0}, N_0\mathbf{I}). \tag{20.23}$$

Entry H_{ij} is the complex gain from transmit antenna j to receive antenna i; with rich scattering and antennas at least $\lambda/2$ apart, the entries are roughly independent $\mathcal{CN}(0,1)$. Total transmit power is fixed: $\mathbb{E}\,\|\mathbf{x}\|^2 = E_s$.

Multiple antennas offer two different gifts:

- **Diversity:** send the *same* information over many independently fading paths so that a deep fade on one is compensated by others. This improves reliability (the BER slope).
- **Spatial multiplexing:** send *different* streams from different antennas and separate them at

the receiver using the different spatial signatures (columns of $\mathbf{H}$). This multiplies the data rate by up to $\min(N_t, N_r)$ with no extra bandwidth or power.

One cannot have the maximum of both at once; the *diversity–multiplexing trade-off* (Zheng and Tse) quantifies the compromise.

20.8.2 The Alamouti code: transmit diversity with two antennas

Intuition: sending a symbol and its mirror

With two transmit antennas and one receive antenna, simply transmitting the same symbol from both gives $(h_1 + h_2)s$, which can still fade (the two gains can cancel). Alamouti's trick sends two symbols over two time slots in a pattern that lets the receiver separate them perfectly with simple linear combining, and each symbol ends up seeing $|h_1|^2 + |h_2|^2$ — both paths' energy. Diversity order 2 at full rate.

Encoding. In slot 1, antenna 1 sends s_1 and antenna 2 sends s_2; in slot 2, antenna 1 sends $-s_2^*$ and antenna 2 sends s_1^* (each with power halved so the total is unchanged). Assuming the channel is constant over both slots,

$$\begin{aligned} y_1 &= h_1 s_1 + h_2 s_2 + n_1, \\ y_2 &= -h_1 s_2^* + h_2 s_1^* + n_2. \end{aligned}$$

Combining. Conjugate the second equation: $y_2^* = -h_1^* s_2 + h_2^* s_1 + n_2^*$. Stack $[y_1,\ y_2^*]^{\mathsf{T}} = \mathbf{H}_A [s_1, s_2]^{\mathsf{T}} + [n_1, n_2^*]^{\mathsf{T}}$ with

$$\mathbf{H}_A = \begin{bmatrix} h_1 & h_2 \\ h_2^* & -h_1^* \end{bmatrix}, \qquad \mathbf{H}_A^{\mathsf{H}} \mathbf{H}_A = \begin{bmatrix} h_1^* & h_2 \\ h_2^* & -h_1 \end{bmatrix} \begin{bmatrix} h_1 & h_2 \\ h_2^* & -h_1^* \end{bmatrix} = (|h_1|^2 + |h_2|^2)\mathbf{I}.$$

(Check the off-diagonal entry: $h_1^* h_2 + h_2(-h_1^*) = 0$.) The columns are orthogonal, so multiplying by $\mathbf{H}_A^{\mathsf{H}}$ decouples the symbols:

$$\begin{aligned} z_1 &= h_1^* y_1 + h_2 y_2^* = (|h_1|^2 + |h_2|^2) s_1 + h_1^* n_1 + h_2 n_2^*, \\ z_2 &= h_2^* y_1 - h_1 y_2^* = (|h_1|^2 + |h_2|^2) s_2 + h_2^* n_1 - h_1 n_2^*. \end{aligned}$$

Each z_i is a scaled copy of one symbol in noise of variance $(|h_1|^2 + |h_2|^2)N_0$, so the effective SNR is $(|h_1|^2 + |h_2|^2)E_s/(2N_0)$ — the sum of two independent exponentials, hence diversity order 2 (with a 3 dB power penalty versus two-antenna receive MRC, because the transmitter must split its power without knowing the channel). Our script `verify_numbers.py` confirms numerically that $z_i/(|h_1|^2 + |h_2|^2)$ returns s_i exactly in the absence of noise.

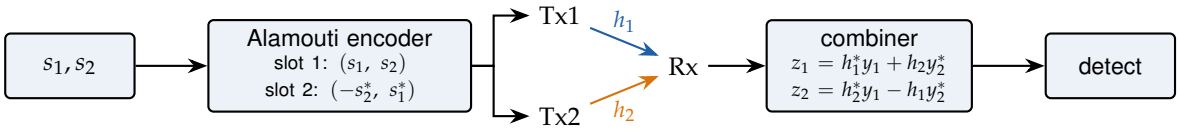


Figure 20.21: Alamouti space–time block code: two symbols over two antennas and two slots. Orthogonality of the code matrix lets a linear combiner separate s_1 and s_2, each enjoying the energy of both paths. MIMO autoencoders (Chapter 26) are tested on whether they can learn schemes of comparable quality.

20.8.3 MIMO capacity and SVD precoding

The **capacity** of a channel is the highest rate (bits per channel use) at which information can be sent with arbitrarily small error probability. For the scalar AWGN channel with SNR ρ it is Shannon's $C = \log_2(1+\rho)$ bit/s/Hz. For MIMO with the transmitter splitting power equally across antennas (no CSI at the transmitter),

Key equation: MIMO capacity

$$C = \log_2 \det\left(\mathbf{I}_{N_r} + \frac{\rho}{N_t}\mathbf{H}\mathbf{H}^{\mathsf{H}}\right) = \sum_{i=1}^{r} \log_2\left(1 + \frac{\rho}{N_t}\sigma_i^2\right) \text{ bit/s/Hz}, \tag{20.24}$$

where σ_i are the singular values of $\mathbf{H}$ and $r = \operatorname{rank}\mathbf{H} \le \min(N_t, N_r)$.

The second form explains everything. Take the singular value decomposition (Part I) $\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathsf{H}}$, with unitary $\mathbf{U}, \mathbf{V}$ and diagonal $\mathbf{\Sigma}$. If the transmitter knows $\mathbf{H}$, it **precodes** $\mathbf{x} = \mathbf{V}\tilde{\mathbf{x}}$ and the receiver computes $\tilde{\mathbf{y}} = \mathbf{U}^{\mathsf{H}}\mathbf{y}$:

$$\tilde{\mathbf{y}} = \mathbf{U}^{\mathsf{H}}(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathsf{H}}\mathbf{V}\tilde{\mathbf{x}} + \mathbf{n}) = \mathbf{\Sigma}\tilde{\mathbf{x}} + \tilde{\mathbf{n}}, \qquad \tilde{\mathbf{n}} = \mathbf{U}^{\mathsf{H}}\mathbf{n} \sim \mathcal{CN}(\mathbf{0}, N_0\mathbf{I}),$$

since $\mathbf{U}^{\mathsf{H}}\mathbf{U} = \mathbf{V}^{\mathsf{H}}\mathbf{V} = \mathbf{I}$ and a unitary rotation of white noise is still white. Component by component, $\tilde{y}_i = \sigma_i \tilde{x}_i + \tilde{n}_i$: the MIMO channel has become r *parallel, non-interfering scalar channels* (**eigen-channels**) with gains σ_i. Each carries $\log_2(1 + P_i\sigma_i^2/N_0)$ bits; with equal power $P_i = E_s/N_t$ this gives (20.24). (Optimal power allocation across eigen-channels is *water-filling*: more power to stronger eigen-channels.) The determinant form follows because $\det(\mathbf{I} + c\mathbf{H}\mathbf{H}^{\mathsf{H}}) = \det(\mathbf{I} + c\mathbf{U}\mathbf{\Sigma}\mathbf{\Sigma}^{\mathsf{H}}\mathbf{U}^{\mathsf{H}}) = \prod_i (1 + c\sigma_i^2)$.

Worked example: a 2×2 channel by hand

Let $\mathbf{H} = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$ (real, symmetric). Its eigenvalues are 1 ± 0.5, so its singular values are $\sigma_1 = 1.5$, $\sigma_2 = 0.5$. At $\rho = 10$ (10 dB) with $N_t = 2$: $C = \log_2(1 + 5 \cdot 2.25) + \log_2(1 + 5 \cdot 0.25) = \log_2(12.25) + \log_2(2.25) = 3.615 + 1.170 = 4.785$ bit/s/Hz. A single-antenna link at the same SNR gets $\log_2 11 = 3.46$. The determinant formula gives the same 4.785 (checked in `verify_numbers.py`).

At high SNR each eigen-channel adds $\log_2\rho$, so capacity grows like $\min(N_t, N_r)\log_2\rho$: $\min(N_t, N_r)$ extra bits per second per hertz for every doubling of SNR (3 dB). Figure 20.22 shows this from a Monte-Carlo average over Rayleigh channels: between 20 and 30 dB the 4×4 curve rises 4.2 times faster than the 1×1.

Wireless connection: ML bridge: CSI feedback compression

In FDD systems the base station cannot measure the downlink $\mathbf{H}$; the user must measure and feed it back. A full $\mathbf{H}$ per subcarrier is far too many numbers. Compressing it is an autoencoder problem: the UE's encoder squeezes CSI into a short code, the base station's decoder reconstructs it. This is precisely the "CSI feedback enhancement" use case standardized by 3GPP (Section 20.11).

20.9 Channel coding

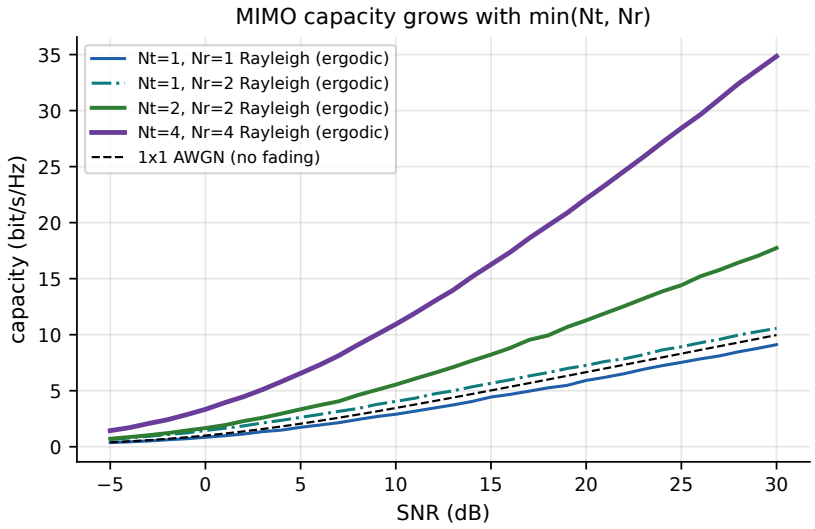


Figure 20.22: Ergodic capacity of i.i.d. Rayleigh MIMO channels from (20.24), averaged over 1500 channel draws per point. At 20 dB: $1\times1 \approx 5.9$, $1\times2 \approx 7.3$, $2\times2 \approx 11.3$, $4\times4 \approx 22$ bit/s/Hz. Receive diversity (1×2) shifts the curve; multiplexing (2×2, 4×4) multiplies its slope. Note that 1×1 Rayleigh lies slightly below AWGN: fading costs capacity too, though far less than it costs uncoded BER.

20.9.1 Why redundancy helps

Intuition: spelling over a bad phone line

On a noisy line you spell "B as in Bravo": you add redundancy so the listener can correct a mis-heard letter. A channel code adds carefully structured redundant bits so that the set of valid transmitted words (*codewords*) are far apart from one another. Noise moves the received word a little; the decoder snaps it back to the nearest valid codeword. It works because among all 2^n possible words only 2^k are codewords, so they can be placed far apart.

An (n, k) **block code** maps k information bits to n coded bits; the **code rate** is $R = k/n$. The **Hamming distance** between two binary words is the number of positions in which they differ, and the **minimum distance** $d_{\min}$ of a code is the smallest distance between distinct codewords. A code with $d_{\min}$ can correct any $t = \lfloor (d_{\min} - 1)/2 \rfloor$ bit errors (balls of radius t around codewords do not overlap).

20.9.2 Hamming(7,4): a complete worked example

The Hamming(7,4) code has $k = 4$, $n = 7$, $R = 4/7$, $d_{\min} = 3$, so it corrects one error. It is *linear*: codewords are $\mathbf{c} = \mathbf{mG} \pmod 2$ for a 4×7 generator matrix $\mathbf{G}$, and $\mathbf{c}$ is a codeword iff $\mathbf{Hc}^{\mathsf{T}} = \mathbf{0} \pmod 2$ for the 3×7 parity-check matrix $\mathbf{H}$. In systematic form,

$$\mathbf{G} = \left[\begin{array}{cccc|ccc} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{array}\right] = [\mathbf{I}_4 \,|\, \mathbf{P}], \qquad \mathbf{H} = \left[\begin{array}{cccc|ccc} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{array}\right] = [\mathbf{P}^{\mathsf{T}} \,|\, \mathbf{I}_3].$$

Since $\mathbf{GH}^{\mathsf{T}} = \mathbf{P} + \mathbf{P} = \mathbf{0} \pmod 2$, every codeword passes the parity check. The seven columns of $\mathbf{H}$ are the seven distinct nonzero 3-bit patterns — the property that makes single-error correction work.

Worked example: encode, corrupt, syndrome-decode

Encode $\mathbf{m} = [1\,0\,1\,1]$: $\mathbf{c} = \mathbf{mG}$ adds rows 1, 3 and 4 of $\mathbf{G}$ modulo 2: $[1000110] + [0010011] + [0001111] = [1\,0\,1\,1\,0\,1\,0]$. The first four bits are the message (systematic); the parity bits are $p_1 = m_1 + m_2 + m_4 = 0$, $p_2 = m_1 + m_3 + m_4 = 1$, $p_3 = m_2 + m_3 + m_4 = 0 \pmod 2$.
Corrupt bit 3: $\mathbf{r} = [1\,0\,0\,1\,0\,1\,0]$.
Syndrome $\mathbf{s} = \mathbf{Hr}^{\mathsf{T}}$, one row of $\mathbf{H}$ at a time (multiply position by position, add modulo 2), with $\mathbf{r} = [1\,0\,0\,1\,0\,1\,0]$: row 1 $[1\,1\,0\,1\,1\,0\,0]$ gives $1+0+0+1+0+0+0 = 2 \equiv 0$; row 2 $[1\,0\,1\,1\,0\,1\,0]$ gives $1+0+0+1+0+1+0 = 3 \equiv 1$; row 3 $[0\,1\,1\,1\,0\,0\,1]$ gives $0+0+0+1+0+0+0 = 1$. So $\mathbf{s} = [0\,1\,1]^{\mathsf{T}}$.
Correct: $\mathbf{s}$ equals column 3 of $\mathbf{H}$, $[0\,1\,1]^{\mathsf{T}}$, so bit 3 is in error. Flip it to recover $\mathbf{c}$, and read the message from the first four bits. (Why: $\mathbf{r} = \mathbf{c} + \mathbf{e}$ gives $\mathbf{Hr}^{\mathsf{T}} = \mathbf{He}^{\mathsf{T}}$, which for a single error in position i is column i of $\mathbf{H}$.)

20.9.3 Hard versus soft decisions and the LLR

A **hard-decision** decoder first slices each received value to a bit and then decodes bits, as above. This throws information away: a received value of $+0.02$ and one of $+1.3$ both become "0", although the first is almost a coin toss. A **soft-decision** decoder uses the reliability of each bit, expressed as a log-likelihood ratio.

Definition: log-likelihood ratio

For a coded bit c and observation y, $\mathrm{LLR}(c) = \ln \dfrac{P(c=0 \mid y)}{P(c=1 \mid y)}$. Its sign is the hard decision (positive $\Rightarrow$ 0); its magnitude is the confidence.

LLR for BPSK in AWGN (derivation). Map $c = 0 \mapsto x = +1$, $c = 1 \mapsto x = -1$; $y = x + n$, $n \sim \mathcal{N}(0, \sigma^2)$, equal priors. Then

$$\mathrm{LLR} = \ln \frac{p(y \mid x=+1)}{p(y \mid x=-1)} = \ln \frac{e^{-(y-1)^2/(2\sigma^2)}}{e^{-(y+1)^2/(2\sigma^2)}} = \frac{(y+1)^2 - (y-1)^2}{2\sigma^2} = \frac{4y}{2\sigma^2},$$

Key equation: BPSK LLR

$$\mathrm{LLR}(y) = \frac{2y}{\sigma^2}. \tag{20.25}$$

The LLR is just the received value scaled by the noise level. Example: $y = 0.4$ at $\sigma^2 = 0.5$ gives $\mathrm{LLR} = 1.6$, i.e. $P(c=0 \mid y) = 1/(1+e^{-1.6}) = 0.83$. Note the sigmoid: $P(c=0 \mid y) = \sigma_{\mathrm{sig}}(\mathrm{LLR})$, so *a logistic-regression output is an LLR in disguise*.

ML block decoding. With soft values $\mathbf{y}$ for a block, the ML decoder picks the codeword whose BPSK image $\mathbf{s}(\mathbf{c}) \in \{\pm 1\}^n$ is closest to $\mathbf{y}$: $\hat{\mathbf{c}} = \arg\min_{\mathbf{c}} \|\mathbf{y} - \mathbf{s}(\mathbf{c})\|^2 = \arg\max_{\mathbf{c}} \mathbf{y}^{\mathsf{T}}\mathbf{s}(\mathbf{c})$ (since $\|\mathbf{s}\|^2 = n$ is the same for all codewords). For Hamming(7,4) this is a search over $2^4 = 16$ correlations — trivial. For a code with $k = 100$, 2^{100} codewords make brute force impossible, which is why practical codes are designed for efficient approximate decoders (belief propagation for LDPC, successive cancellation for polar).

Wireless connection: ML bridge: decoding = classification over 2^k codewords

The ML decoder is a 2^k-class classifier from $\mathbf{y} \in \mathbb{R}^n$ to the transmitted message. A neural decoder with a 2^k-way softmax (or k sigmoid outputs, one per bit) learns this mapping; Chapter 23 studies how well it can — and finds the exponential number of classes to be the central difficulty (the "curse of dimensionality" of decoding).

Figure 20.23 shows our simulation of block error rate for uncoded 4-bit blocks, Hamming(7,4) with hard syndrome decoding, and Hamming(7,4) with soft ML decoding, all at equal energy per *information* bit (each coded bit gets $E_s = RE_b$). At BLER $= 10^{-4}$ we read about 9.1, 8.5 and 7.1 dB respectively: a **coding gain** of ≈ 0.7 dB with hard decisions and ≈ 2 dB with soft decisions. Soft decoding gains roughly 2 dB over hard decoding in general — one reason every modern receiver passes LLRs, not bits.

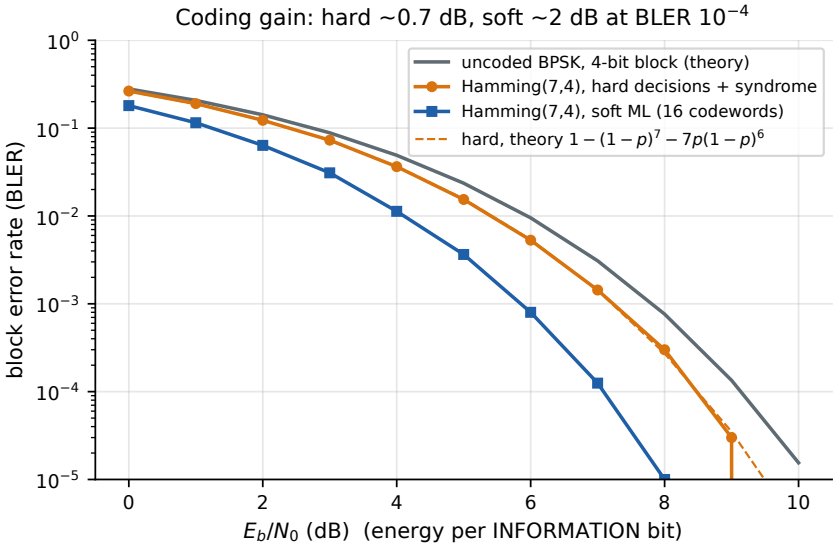


Figure 20.23: BLER of uncoded BPSK (4-bit blocks), Hamming(7,4) with hard syndrome decoding (simulation and theory: a block fails when ≥ 2 of 7 bits flip), and Hamming(7,4) with soft ML decoding (correlate with all 16 codewords). The horizontal gap at a target BLER is the coding gain. A neural decoder for this code (Chapters 21 and 23) should approach the soft-ML curve.

20.9.4 The Shannon limit

Shannon (1948) proved that over AWGN with bandwidth B, reliable communication is possible at any rate below $C = B\log_2(1+\mathrm{SNR})$ and impossible above it. Write the spectral efficiency $\eta = R_b/B$ (bit/s/Hz) and $\mathrm{SNR} = E_bR_b/(N_0B) = \eta E_b/N_0$. Requiring $\eta \leq \log_2(1+\eta E_b/N_0)$ gives

$$\frac{E_b}{N_0} \geq \frac{2^\eta - 1}{\eta} \xrightarrow{\eta \to 0} \ln 2 = -1.59\,\mathrm{dB}. \tag{20.26}$$

At $\eta = 1$ the limit is 0 dB; at $\eta = 2$, 1.76 dB; at $\eta = 4$, 5.74 dB. Uncoded BPSK needs 9.6 dB for 10^{-5} BER, so there is up to 11 dB of potential coding gain. Turbo codes (1993), LDPC codes (Gallager 1962, rediscovered in the 1990s) and polar codes (Arıkan 2009) come within a fraction of a dB of the limit for long blocks.

20.9.5 Polar codes and LDPC codes in brief

Polar codes. Start from the 2×2 kernel $\mathbf{F} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$: two input bits (u_1, u_2) are sent as $(x_1, x_2) = (u_1 \oplus u_2,\ u_2)$. Over two uses of a channel W, a successive decoder first decodes u_1 from both outputs — it must cope with u_2 unknown, so u_1 sees a *worse* channel W^- — and then decodes u_2 knowing u_1, seeing two independent looks at u_2, a *better* channel W^+. For the binary erasure channel with erasure probability ϵ, the erasure probabilities become exactly $2\epsilon - \epsilon^2$ (worse) and ϵ^2 (better). Applying the kernel recursively, $\mathbf{F}^{\otimes n}$ for $N = 2^n$, the N synthesized bit-channels **polarize**: almost all become either nearly perfect or nearly useless (Fig. 20.24). The code puts information on the K best bit-channels and fixes the rest to known values (**frozen bits**, usually 0). Arıkan proved that the fraction of good channels tends to the capacity, so polar codes achieve capacity with successive-cancellation decoding of complexity $O(N \log N)$; in practice a list decoder with a CRC (CA-SCL) is used.

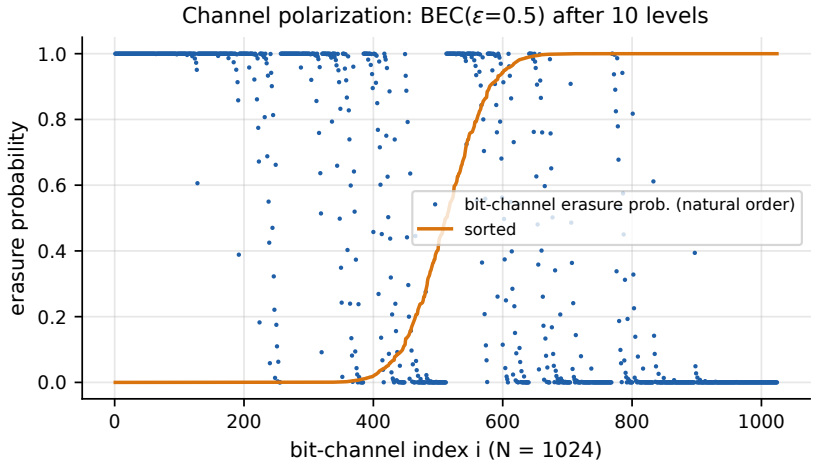


Figure 20.24: Channel polarization for $N = 1024$ starting from an erasure channel with $\epsilon = 0.5$ (capacity 0.5), computed with the exact recursion $\epsilon \mapsto (2\epsilon - \epsilon^2,\ \epsilon^2)$. Sorted (orange), 37% of bit-channels have erasure probability below 0.01 and 37% above 0.99; the middle shrinks as N grows. Information bits go on the good channels, frozen bits on the bad.

LDPC codes are linear codes whose parity-check matrix $\mathbf{H}$ is large and *sparse*. Drawn as a bipartite *Tanner graph* (variable nodes for bits, check nodes for parity equations), they are decoded by **belief propagation**: nodes iteratively exchange LLR messages along the edges. Neural belief propagation — "unrolling" the iterations into network layers with trainable weights on the edges — is a well-known deep-learning-for-decoding idea.

20.10 Medium access, spectrum and networks

20.10.1 Multiple access

Many users share one spectrum. The classical options divide a resource:

- **FDMA**: each user gets its own frequency band (1G analog cellular).
- **TDMA**: users take turns in time slots on a shared band (GSM: 8 slots per carrier).
- **CDMA**: all users transmit simultaneously on the full band, each spreading its symbols with a distinct code; the receiver correlates with the desired code (3G).

- **OFDMA**: the OFDM time–frequency grid is divided into resource blocks assigned to different users (4G LTE, 5G NR, WiFi 6).

Random access protocols (ALOHA, CSMA/CA in WiFi: "listen before talk") let users contend without central scheduling, at the cost of collisions.

20.10.2 Spectrum sensing and dynamic spectrum access

Much licensed spectrum is idle much of the time in a given place. In **dynamic spectrum access** (cognitive radio), secondary users opportunistically use channels left idle by primary users. This needs **spectrum sensing** — deciding whether a channel is busy — whose simplest form is energy detection: declare "busy" if $\frac{1}{N}\sum_n |y[n]|^2 > \eta$, a binary hypothesis test (busy vs. idle) with a threshold chosen for a target false-alarm probability. Modulation recognition generalizes sensing: not just "is someone there?" but "what kind of signal?"

The occupancy of each channel over time is commonly modelled by a two-state **Gilbert–Elliott Markov chain**: in each slot a channel is *good* (idle, 0) or *bad* (busy, 1), and it switches with probabilities $p_{01} = P(\text{bad next} \mid \text{good now})$ and $p_{10} = P(\text{good next} \mid \text{bad now})$. The transition matrix is

$$\mathbf{P} = \begin{bmatrix} 1-p_{01} & p_{01} \\ p_{10} & 1-p_{10} \end{bmatrix}, \qquad \boldsymbol{\pi} = \left[\frac{p_{10}}{p_{01}+p_{10}}, \frac{p_{01}}{p_{01}+p_{10}}\right].$$

The stationary distribution $\boldsymbol{\pi}$ solves $\boldsymbol{\pi}\mathbf{P} = \boldsymbol{\pi}$: balance of flow, $\pi_0 p_{01} = \pi_1 p_{10}$, plus $\pi_0 + \pi_1 = 1$. For $p_{01} = 0.05$, $p_{10} = 0.2$, a channel is busy $0.05/0.25 = 20\%$ of the time, in bursts of mean length $1/p_{10} = 5$ slots. Because the state persists, the past predicts the future — this memory is what a learning agent exploits.

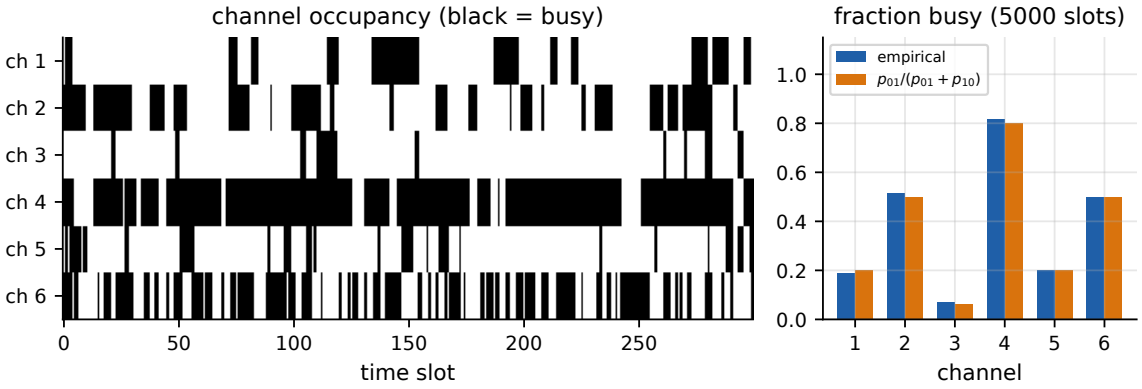


Figure 20.25: Six channels evolving as independent Gilbert–Elliott chains with different (p_{01}, p_{10}) (our simulation). **Left:** 300 slots of occupancy; bursty channels (ch 4) differ visibly from rapidly switching ones (ch 6). **Right:** over 5000 slots the empirical busy fraction matches $p_{01}/(p_{01}+p_{10})$. A user who can sense only one channel per slot must learn which to try — the dynamic multichannel access problem solved with deep Q-learning in Chapter 30.

Wireless connection: ML bridge: access = sequential decision making

Choosing a channel each slot based on past observations is a partially observable Markov decision process: state = (unknown) occupancy of all channels, action = which channel to use, reward = success or collision. Deep reinforcement learning (Part IV) learns a policy from experience without knowing p_{01}, p_{10}; for i.i.d. channels it reduces to a multi-armed bandit.

20.10.3 Peer-to-peer networks

In client–server systems all users download from one server, whose upload capacity becomes the bottleneck. In **peer-to-peer** (P2P) systems every participant both downloads and uploads. In **BitTorrent**, a file is split into pieces; peers that hold the whole file are *seeds*, those still downloading are *leechers*, and the set of peers sharing one file is a *swarm*. A *tracker* (or a distributed hash table) introduces peers to each other. Each peer uploads to a few neighbours that reciprocate best ("tit-for-tat" choking) plus one random "optimistic unchoke" to discover better partners, and requests the rarest pieces first. Every decision — which peers to unchoke, which pieces to request, how to split upload bandwidth — is made online, with partial and changing information about others' behaviour: an online-learning problem, studied in Chapter 31. Wireless enters through capacity-limited, fluctuating links between peers.

20.11 5G NR context for the practitioner

- **Numerology.** NR uses OFDM with subcarrier spacing $\Delta f = 15 \cdot 2^{\mu}$ kHz, $\mu = 0, 1, 2, 3, 4$ (15, 30, 60, 120, 240 kHz; larger values were added for higher bands in later releases). A slot has 14 OFDM symbols (normal CP), so slot duration is $1/2^{\mu}$ ms. At $\mu = 0$ the useful symbol length is $1/15\,\text{kHz} = 66.7\,\mu$s and the normal CP is about $4.7\,\mu$s. Larger spacing gives shorter symbols, which tolerate more Doppler and phase noise (millimetre wave) but less delay spread.
- **Channel coding.** LDPC codes (two base graphs, for large and small blocks) protect the data channels (PDSCH/PUSCH); polar codes protect control information (DCI on PDCCH, UCI on PUCCH/PUSCH) and the broadcast channel. Data uses HARQ with incremental redundancy.
- **Modulation.** QPSK up to 256-QAM (1024-QAM in the downlink in later releases), plus $\pi/2$-BPSK in the uplink for low peak-to-average power.
- **CSI acquisition.** The gNB transmits CSI reference signals (CSI-RS); the UE estimates the channel and returns a **CSI report**: CQI (channel quality indicator — which modulation and coding scheme it can support), PMI (precoding matrix indicator — the index of the best precoder in a codebook; Type I and the higher-resolution Type II codebooks) and RI (rank indicator — how many MIMO layers). This codebook feedback is the classical counterpart of learned CSI compression.
- **SSB and beam management.** The synchronization signal block (PSS, SSS and the broadcast channel PBCH, spanning 4 OFDM symbols $\times$ 240 subcarriers) is transmitted periodically in a burst of beams pointing in different directions (*beam sweeping*). The UE measures each beam's received power (SS-RSRP), reports the best, and the link is refined with narrower CSI-RS beams. At millimetre wave, with up to 64 SSB beams, exhaustive sweeping is costly — hence AI-based beam prediction.

20.11.1 AI/ML for the NR air interface in 3GPP

3GPP studied AI/ML for the NR air interface in a Release 18 study item, documented in TR 38.843. It selected three representative use cases:

1. **CSI feedback enhancement**: spatial–frequency compression of CSI with a *two-sided* model (encoder at the UE, decoder at the gNB) and time-domain CSI *prediction* with a one-sided

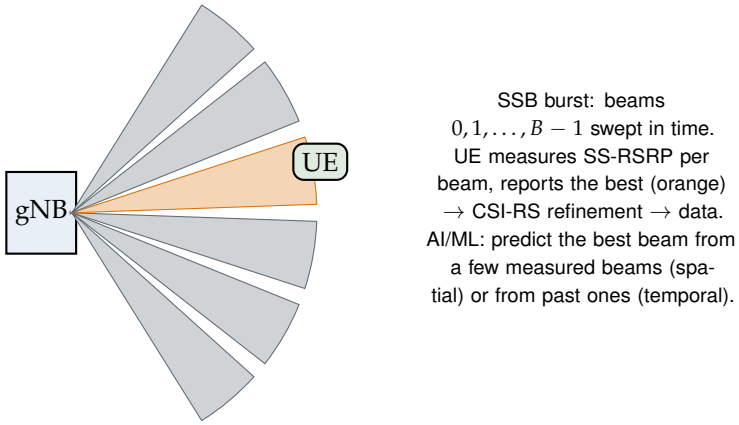


Figure 20.26: SSB beam sweeping. Each beam is a separate SSB transmission; the UE reports the strongest. AI/ML beam management (3GPP Rel-18 study, Rel-19 specification) reduces measurement overhead by predicting the best beam from a subset of measurements.

model;

2. **beam management**: spatial-domain and temporal downlink beam prediction;
3. **positioning accuracy enhancement**: direct AI/ML positioning (CSI/channel impulse response → position) and AI/ML-assisted positioning (the model outputs intermediate measurements such as line-of-sight probability or timing).

The study also defined a general framework: model life-cycle management (data collection, training, monitoring, activation/switching/fallback), the distinction between one-sided and two-sided models, and levels of UE–network collaboration. In Release 19 the normative work item (NR_AIML_air) specified support for one-sided models in **beam management, positioning and CSI prediction**, together with a generic life-cycle-management framework; two-sided CSI compression and UE-side training-data collection were deferred to Release 20, which is also where 3GPP's 6G studies begin to consider an AI-native air interface. These use cases map directly onto chapters in this Part: CSI compression (Chapter 26), CSI prediction (Chapter 27), channel-feature and beam prediction (Chapter 28), and positioning (Chapter 29).

20.12 Where each later chapter plugs in

Table 20.1 is a map for the rest of Part V and the labs of Part VI: for each, which block of Fig. 20.1 is learned, what the ML problem type is, and which sections of this chapter you need.

Chapter recap

- A radio signal is $s(t) = \mathrm{Re}\{x(t)e^{j2\pi f_c t}\}$; all processing (and every neural network in Part V) works on complex baseband I/Q samples of $x(t)$. About B complex symbols per second fit in B hertz.
- Modulation maps $k = \log_2 M$ bits to one of M constellation points (PSK, PAM, QAM; FSK and analog AM/FM in RadioML). Normalize to unit average energy (16-QAM with $\pm 1, \pm 3$ has $E_s = 10$); Gray-label so a symbol error costs one bit; shape pulses with (root-)raised cosines to avoid ISI.
- AWGN: $y = x + n$, $n \sim \mathcal{CN}(0, N_0)$. ML detection is minimum distance; MAP is classifi-

Table 20.1: Where each later chapter plugs into the communication system.

Chapter topic	Block(s) learned	ML problem	Prereq. §
Physical layer as an autoencoder	transmitter + receiver	joint classification of M messages through a differentiable channel layer	20.3, 20.4, 20.9
Modulation recognition	signal classifier (sensing)	CNN classification of I/Q windows across SNR	20.3, 20.6
Neural channel decoding	decoder	classification over 2^k codewords / per-bit sigmoids from LLRs	20.9
Model-free end-to-end learning	transmitter without channel model	policy-gradient (RL) training of the encoder, supervised receiver	20.5, 20.6
Learning with noisy feedback	transmitter training over a feedback link	gradient estimation under noisy rewards	20.4
MIMO autoencoders	precoder + detector, CSI encoder/decoder	multi-antenna autoencoder; CSI compression	20.8
CSI prediction	channel estimator in time	sequence regression (RNN/LSTM)	20.5, 20.7
Channel-feature prediction	link adaptation, beam choice	regression of SNR/delay/AoA/beam features	20.5, 20.11
WiFi CSI/RSS localization	positioning function	regression or classification of position from CSI/RSS fingerprints	20.5, 20.7
Dynamic multichannel access	MAC (channel selection)	deep Q-learning in a POMDP	20.10
BitTorrent online learning	peer/rate selection	online learning, bandits	20.10
Labs: modulation classification (AWGN, Rayleigh, IIR)	signal classifier	feature-based and CNN classifiers; robustness to channels	20.3–20.5.5

cation. BPSK and QPSK have $P_b = Q(\sqrt{2E_b/N_0})$; $E_s/N_0 = E_b/N_0 + 10\log_{10}(kR)$ dB.

- Real channels: path loss $(1/d^n)$, log-normal shadowing, multipath fading. Sums of random-phase paths give Rayleigh (or Rician) fading; BER then falls only as $1/(4\bar{\gamma})$; diversity of order L restores a slope of $\bar{\gamma}^{-L}$. Frequency selectivity is an FIR (or IIR) filter; Doppler sets the coherence time.
- Impairments (CFO, phase noise, timing, IQ imbalance, PA nonlinearity) have characteristic constellation fingerprints.
- Equalizers: ZF, MMSE (Wiener), LMS (= SGD on a linear neuron). OFDM with a cyclic prefix turns convolution into per-subcarrier multiplication; pilots enable LS/LMMSE channel estimation (= regression).
- MIMO: $\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}$; Alamouti gives diversity 2; SVD splits $\mathbf{H}$ into eigen-channels; capacity grows as $\min(N_t, N_r)\log_2$ SNR.
- Coding: rate k/n, $d_{\min}$, Hamming(7,4) syndrome decoding, LLR $= 2y/\sigma^2$, soft decoding gains ≈ 2 dB, Shannon limit -1.59 dB; polar codes (control) and LDPC (data) in 5G.
- Access: TDMA/FDMA/CDMA/OFDMA, sensing, Gilbert–Elliott channels (RL), P2P networks (online learning). 3GPP Rel-18 studied AI/ML for CSI feedback, beam management and positioning; Rel-19 specified beam management, positioning and CSI prediction.

Exercises

1. **(Wavelength and Doppler.)** A train at 300 km/h uses a 2.6 GHz carrier. Compute λ, f_D and the coherence time $T_c \approx 0.423/f_D$. How many 0.5 ms slots fit in T_c?
2. **(Energy normalization.)** Compute E_s for 8-PAM with levels $\{\pm 1, \pm 3, \pm 5, \pm 7\}$ and the scaling that makes it unit energy. Check against $(M^2-1)/3$.
3. **(SNR bookkeeping.)** A 64-QAM link with a rate-3/4 code operates at $E_s/N_0 = 20$ dB. What is E_b/N_0 in dB?
4. **(Baseband.)** Show that if the receiver's oscillator has a phase error θ, i.e. it computes LPF$\{2s(t)e^{-j(2\pi f_c t+\theta)}\}$, the recovered baseband is $x(t)e^{-j\theta}$. What does this do to a QPSK constellation if $\theta = 45°$?
5. **(BER.)** Using (20.9) and (20.16), compute BPSK BER at 10 dB in AWGN and in Rayleigh fading. By what factor does fading increase the BER?
6. **(OFDM CP.)** An OFDM system has $N = 1024$ subcarriers at 15 kHz spacing. The channel's maximum excess delay is 3 μs. What is the minimum CP length in samples (sample rate $N\Delta f$), and what fraction of time is overhead?
7. **(Hamming.)** Encode $\mathbf{m} = [0\,1\,1\,0]$ with the $\mathbf{G}$ of Section 20.9. Flip bit 6, compute the syndrome and show that it identifies the error.
8. **(LLR.)** For BPSK in AWGN with $\sigma^2 = 0.25$, compute the LLRs and $P(c = 0 \mid y)$ for $y = -0.1$ and $y = 0.9$. Which bit is more reliable?
9. **(MIMO.)** For $\mathbf{H} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$, find the singular values and the capacity at $\rho = 10$ with equal power. Why is this channel no better than a single stream?

10. **(Markov channel; conceptual.)** A channel has $p_{01} = 0.1$, $p_{10} = 0.3$. What fraction of slots is it idle? If it was idle in the last slot, what is the probability it is idle now? Explain why a policy that remembers the last observation beats one that chooses channels at random.

Answers

1. $\lambda = 3 \times 10^8 / 2.6 \times 10^9 = 0.1154\,\text{m}$. $v = 300/3.6 = 83.3\,\text{m/s}$, so $f_D = 83.3/0.1154 = 722\,\text{Hz}$. $T_c = 0.423/722 = 0.586\,\text{ms}$: barely more than one 0.5 ms slot. The CSI measured in one slot is nearly stale in the next — a strong case for CSI prediction.

2. $\mathbb{E}[a^2] = \frac{2(1+9+25+49)}{8} = 21$, so $E_s = 21$ and the points are divided by $\sqrt{21} = 4.58$. The formula $(M^2-1)/3 = (64-1)/3 = 21$ agrees.

3. $kR = 6 \times 0.75 = 4.5$, $10\log_{10} 4.5 = 6.53\,\text{dB}$, so $E_b/N_0 = 20 - 6.53 = 13.47\,\text{dB}$.

4. $2s(t) = xe^{j2\pi f_c t} + x^* e^{-j2\pi f_c t}$ (since $\text{Re}\{z\} = \frac{z+z^*}{2}$). Multiplying by $e^{-j(2\pi f_c t+\theta)}$ gives $xe^{-j\theta} + x^* e^{-j(4\pi f_c t+\theta)}$; the low-pass filter removes the second term, leaving $x(t)e^{-j\theta}$. A 45° error rotates QPSK points from the diagonals onto the axes, exactly onto the decision boundaries: the BER becomes about $1/2$ for one of the two bits per symbol. This is why phase must be estimated (or a network must be trained with random phase).

5. AWGN: $Q(\sqrt{20}) = Q(4.47) = 3.9 \times 10^{-6}$. Rayleigh: $\frac{1}{2}(1 - \sqrt{10/11}) = 0.0233$. Fading increases BER by a factor of about 6000.

6. The sample rate is $1024 \times 15\,\text{kHz} = 15.36\,\text{MHz}$, so $3\,\mu\text{s} = 46.08$ samples: at least 47 samples of CP (NR uses 72 at this rate for the normal CP). Overhead $47/(1024+47) = 4.4\%$ (with 72: 6.6%).

7. $\mathbf{c}$ = row 2 + row 3 of $\mathbf{G} = [0100101] + [0010011] = [0110110]$. Flip bit 6: $\mathbf{r} = [0110100]$. $\mathbf{Hr}^\mathsf{T}$: row 1 $[1101100] \cdot \mathbf{r} = 0+1+0+0+1 = 0$; row 2 $[1011010] \cdot \mathbf{r} = 0+0+1+0+0+0 = 1$; row 3 $[0111001] \cdot \mathbf{r} = 1+1+0+0 = 0$. So $\mathbf{s} = [010]^\mathsf{T}$, which is column 6 of $\mathbf{H}$: bit 6 is wrong.

8. $\text{LLR} = 2y/\sigma^2 = 8y$. For $y = -0.1$: $\text{LLR} = -0.8$, $P(c=0 \mid y) = 1/(1+e^{0.8}) = 0.31$. For $y = 0.9$: $\text{LLR} = 7.2$, $P(c=0 \mid y) = 0.9993$. The second bit is far more reliable; a soft decoder will trust it and preferentially flip the first.

9. $\mathbf{HH}^\mathsf{T} = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}$ with eigenvalues 4 and 0, so $\sigma_1 = 2$, $\sigma_2 = 0$. $C = \log_2(1 + \frac{10}{2} \cdot 4) = \log_2 21 = 4.39\,\text{bit/s/Hz}$. Rank 1: both antennas see identical channels (e.g. a keyhole or perfectly correlated antennas), so only one eigen-channel exists. It still has array gain ($\sigma_1^2 = 4$ versus 1) but no multiplexing gain.

10. Idle fraction $\pi_0 = p_{10}/(p_{01}+p_{10}) = 0.3/0.4 = 0.75$. $P(\text{idle now} \mid \text{idle before}) = 1 - p_{01} = 0.9 > 0.75$. Because the state persists, the last observation is informative: a policy that stays on a channel it just found idle succeeds 90% of the time versus 75% for random choice. Exploiting this memory across many channels, when only one can be sensed at a time, is the essence of the DQN access problem.

Self-test quiz

1. Why are wireless baseband signals complex-valued?
   Answer: a carrier at fixed f_c has two degrees of freedom, amplitude and phase (equivalently cosine and sine weights I, Q); a complex number records both.
2. In AWGN with equally likely symbols, what does the ML detector compute?
   Answer: the constellation point nearest to y in Euclidean distance (minimum-distance rule; Voronoi decision regions).
3. Why does BPSK BER in Rayleigh fading decay only as $1/\bar{\gamma}$?
   Answer: errors are dominated by deep fades, whose probability $P(|h|^2 < \epsilon) \approx \epsilon/\bar{\gamma}$ falls only linearly with average SNR.
4. What does the cyclic prefix do in OFDM?
   Answer: if at least as long as the channel memory, it makes the linear channel convolution look circular, so after the FFT each subcarrier sees one complex gain $H[k]$ and no inter-symbol interference.
5. What is the LLR of a BPSK bit in AWGN, and what ML object is it related to?
   Answer: $2y/\sigma^2$; passing it through a sigmoid gives $P(c = 0 \mid y)$, i.e. it is the logit of a binary classifier.
6. Which AI/ML use cases did 3GPP Rel-18 study, and which did Rel-19 specify?
   Answer: Rel-18: CSI feedback enhancement (compression and prediction), beam management, positioning accuracy. Rel-19: beam management, positioning and CSI prediction (one-sided models); two-sided CSI compression deferred to Rel-20.

Further reading

- J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed., McGraw-Hill, 2008 — the classical reference for modulation, detection, equalization and coding.
- D. Tse and P. Viswanath, *Fundamentals of Wireless Communication*, Cambridge University Press, 2005 — fading, diversity, MIMO and capacity explained with deep intuition.
- A. Goldsmith, *Wireless Communications*, Cambridge University Press, 2005 — path loss, shadowing, fading statistics, adaptive modulation and multicarrier systems.
- B. Sklar, *Digital Communications: Fundamentals and Applications*, 2nd ed., Prentice Hall, 2001 — a gentle, very readable introduction with link budgets and coding.
- S. M. Alamouti, "A simple transmit diversity technique for wireless communications," *IEEE J. Sel. Areas Commun.*, 16(8), 1998; E. Arıkan, "Channel polarization," *IEEE Trans. Inf. Theory*, 55(7), 2009.
- 3GPP TR 38.843, *Study on Artificial Intelligence (AI)/Machine Learning (ML) for NR air interface* (Release 18).

Chapter 21

An Introduction to Deep Learning for the Physical Layer

For seventy years, communication engineers have built transmitters and receivers as a chain of separately designed blocks: source coding, channel coding, modulation, pulse shaping, synchronisation, equalisation, demodulation, decoding. Each block is optimal for a mathematical model of its own small problem. In 2017 Tim O'Shea and Jakob Hoydis asked a disarmingly simple question: *what if we throw away the blocks, write the whole transmitter and the whole receiver as neural networks, put a channel model between them, and train everything together to minimise the probability of a wrong message?* The answer was a paper that started the field of *end-to-end learned communications*.

In this chapter you will learn exactly how that works. We will build the "communication system as an autoencoder" with its (n, k) notation, derive its power normalisation and its noise layer from E_b/N_0, and see why the categorical cross-entropy is the right loss. We then go through the paper's results (learned codes that match a Hamming code with maximum-likelihood decoding, learned constellations, multi-user interference channels, radio transformer networks, and CNN-based modulation classification), and finish with *our own* reproduction on a laptop CPU, a practitioner's view of what this means for 5G NR and 6G, and the open problems the paper listed.

Paper at a glance

Paper. T. O'Shea and J. Hoydis, "An Introduction to Deep Learning for the Physical Layer", *IEEE Transactions on Cognitive Communications and Networking*, vol. 3, no. 4, pp. 563–575, Dec. 2017. arXiv:1702.00832 (v1 Feb. 2017, v2 Jul. 2017).
Problem. Can a complete physical layer (transmitter, channel, receiver) be learned from data instead of designed block by block, and where else can deep learning help the PHY?
Key idea. Treat transmitter and receiver as the encoder and decoder of an *autoencoder* whose "bottleneck" is a noisy channel layer. Train both jointly with stochastic gradient descent to recover a message out of $M = 2^k$ from n channel uses.
Main results (as reported). A $(7, 4)$ autoencoder trained at $E_b/N_0 = 7\,\text{dB}$ reaches the block error rate of the Hamming $(7, 4)$ code with maximum-likelihood decoding; a $(2, 2)$ autoencoder matches uncoded BPSK and an $(8, 8)$ autoencoder beats it; learned constellations resemble QPSK, 16-PSK or a hexagonal 16-point grid; in a two-user interference channel the learned scheme beats time-sharing by about 0.7 dB $(4, 4)$ and about 1 dB $(4, 8)$ at BLER 10^{-3}; radio transformer networks speed up training and improve BLER in Rayleigh multipath; a CNN on raw IQ samples beats a boosted-tree classifier on expert features by about 4 dB at low-to-medium SNR in modulation classification.

21.1 The problem and why the classical approach can fall short

Intuition: the relay race and the dance couple

A classical transceiver is a relay race. Each runner (coder, modulator, equaliser, decoder) is trained alone and only needs to hand the baton over cleanly. If every leg is run optimally, the race is excellent — *provided* the track looks like the one they trained on. An autoencoder is a dance couple. Transmitter and receiver practise *together*, so each learns the habits of the other and of the floor (the channel). They may invent steps that no textbook teaches, but only for the floor they practised on.

The block-wise approach is not "wrong". Shannon's separation theorem tells us that, for long blocks and stationary channels, separating source and channel coding loses nothing; information theory gives us capacity-approaching codes (LDPC, polar), and decades of engineering give us excellent synchronisers and equalisers. O'Shea and Hoydis point out three places where the classical recipe can fall short:

1. **Models are imperfect.** Each block is optimal for a tractable model (linear, Gaussian, stationary). Real hardware has nonlinear amplifiers, IQ imbalance, phase noise and quantisation. A learned system can, in principle, optimise for whatever the true channel is.
2. **Block-wise optimality is not global optimality.** A sequence of individually optimal blocks need not be jointly optimal, especially for *short* blocks where the asymptotic arguments behind separation do not apply. Joint coding and modulation for short packets is exactly the regime of control signalling and IoT.
3. **Speed and parallelism.** A trained neural network is a fixed sequence of matrix products, which runs efficiently on massively parallel hardware with predictable latency, while iterative algorithms may not.

21.2 Background you need

This chapter builds on Part II (dense layers, softmax, cross-entropy, Adam), Chapter 18 style autoencoders (Part IV) and the wireless primer of Chapter 20 (BPSK, AWGN, E_b/N_0, BER/BLER, Hamming codes). Here is a compact refresher of the three wireless facts we use constantly.

Messages, blocks and rates. A transmitter wants to send one of M equally likely messages $s \in \mathcal{M} = \{1, \dots, M\}$. Since $M = 2^k$, each message carries k bits. It does so with n *channel uses* (here: n complex baseband samples). The **rate** is

$$R = \frac{k}{n} \quad \text{bits per channel use.} \tag{21.1}$$

The AWGN channel. The receiver observes $\mathbf{y} = \mathbf{x} + \mathbf{w}$, where $\mathbf{w}$ is white Gaussian noise. With complex samples, each real and imaginary part of w_i is Gaussian with variance β; equivalently $N_0 = 2\beta$ is the noise power per complex sample.

E_b/N_0**.** To compare systems with different rates fairly we measure the received *energy per information bit* E_b relative to N_0. If the average energy per complex symbol is E_s, then $E_b = E_s/R$ because one channel use carries R bits.

Definition: block error rate

The *block error rate* (BLER) is the probability that the decoded message differs from the transmitted one, $P_e = \Pr(\hat{s} \neq s)$, estimated by counting errors over many simulated blocks. For one-shot systems like ours the BLER is the natural metric: the receiver outputs a whole message, not individual bits.

21.3 System model: communication as an autoencoder

21.3.1 The three components

The paper writes the transmitter as a function $f : \mathcal{M} \to \mathbb{R}^{2n}$ (or $\mathbb{C}^n$), the channel as a conditional probability density $p(\mathbf{y} \mid \mathbf{x})$, and the receiver as $g : \mathbb{C}^n \to \mathcal{M}$:

$$s \xrightarrow{f} \mathbf{x} = f(s) \xrightarrow{p(\mathbf{y}|\mathbf{x})} \mathbf{y} \xrightarrow{g} \hat{s}. \tag{21.2}$$

Both f and g are neural networks; the channel is a layer *without trainable parameters*. An (n, k) autoencoder is the system that sends one of $M = 2^k$ messages using n channel uses. Note that the paper uses complex notation, but the network itself handles $2n$ real numbers (the I and Q parts stacked), because the deep-learning libraries of 2017 had no complex layers.

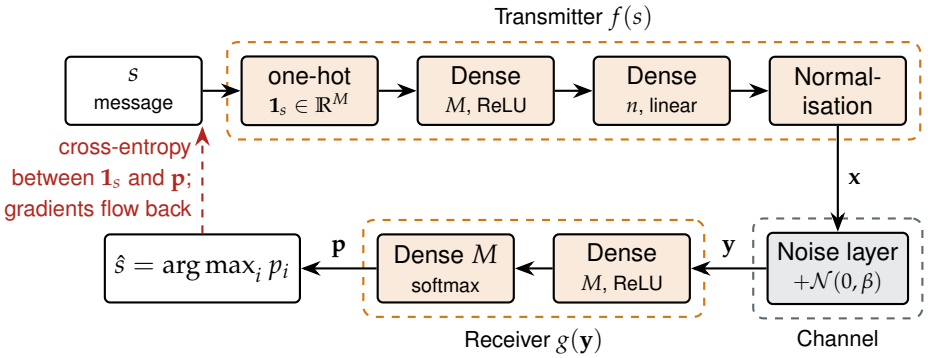


Figure 21.1: The (n, k) autoencoder of O'Shea and Hoydis drawn as a block diagram. Orange blocks are trainable; the gray channel block has no parameters but is differentiable, so the gradient of the loss (red dashed path) reaches the transmitter. Everything between the one-hot vector and the softmax is one ordinary feed-forward network with a noise layer in the middle.

21.3.2 The layer-by-layer architecture

Table 21.1 lists the layout given in the paper (its Table IV). The message enters as a *one-hot vector* $\mathbf{1}_s \in \mathbb{R}^M$: all zeros except a one at position s. The first dense layer with ReLU and the second dense layer with linear activation map it to n real numbers (in our code: n real numbers for the real-valued (n, k) examples and $2n$ for complex ones). The receiver is a dense layer with M ReLU units and a final dense layer with M outputs followed by softmax.

Worked example: counting parameters

Add up the four dense layers for general M and n: $(M^2 + M) + (Mn + n) + (nM + M) + (M^2 + M) = 2M^2 + 2Mn + 3M + n$. Rewrite this as $(2M + 1)(M + n) + 2M = 2M^2 + 2Mn + M + n + 2M$, which is the same thing, and is exactly the formula given in the paper. For

Table 21.1: Autoencoder layout reported in the paper, with the number of trainable parameters per layer (weights + biases) computed for the $(7,4)$ case ($M = 16$, $n = 7$).

Part	Layer	Output dim.	Params, $(7,4)$
Transmitter	Input (one-hot)	$M = 16$	0
	Dense + ReLU	$M = 16$	$16 \cdot 16 + 16 = 272$
	Dense + linear	$n = 7$	$16 \cdot 7 + 7 = 119$
	Normalisation	$n = 7$	0
Channel	Noise	$n = 7$	0
Receiver	Dense + ReLU	$M = 16$	$7 \cdot 16 + 16 = 128$
	Dense + softmax	$M = 16$	$16 \cdot 16 + 16 = 272$
	Total		**791**

$(2,2)$: $M = 4$, $(9)(6) + 8 = 62$. For $(7,4)$: $(33)(23) + 32 = 791$. For $(8,8)$: $M = 256$, $(513)(264) + 512 = 135{,}944$. These are the three numbers the paper reports. Note how the count grows like $M^2 = 4^k$: this is the first hint of the scalability problem we return to in Section 21.7.

The paper also remarks that a single transmit layer could represent the same mapping (a one-hot vector times a weight matrix simply selects a column), but in their experiments SGD found better solutions with two transmit layers. This is a recurring lesson in deep learning: over-parameterisation often makes optimisation easier.

21.3.3 The normalisation layer: energy or average power

Without a constraint the transmitter would "cheat": scaling $\mathbf{x}$ up makes the noise relatively smaller and the loss lower. The paper discusses two constraints, implemented by a normalisation layer at the end of the transmitter.

Definition: two transmit constraints

Energy constraint (per message): $\|\mathbf{x}\|_2^2 \le n$, enforced for every message by

$$\mathbf{x} = \sqrt{n}\,\frac{\mathbf{z}}{\|\mathbf{z}\|_2}, \tag{21.3}$$

where $\mathbf{z}$ is the output of the last dense layer. Every codeword then lies on a sphere of radius $\sqrt{n}$. **Average power constraint**: $\mathbb{E}[|x_i|^2] \le 1$ for all i, enforced over a batch of B messages by

$$\mathbf{x}^{(b)} = \sqrt{n}\,\frac{\mathbf{z}^{(b)}}{\sqrt{\frac{1}{B}\sum_{b'=1}^{B}\left\|\mathbf{z}^{(b')}\right\|_2^2}}. \tag{21.4}$$

Individual codewords may now have more or less energy than n, as long as the average is n.

Intuition: sphere versus ball

With the energy constraint all M points must sit on the surface of a sphere: a ring of radius $\sqrt{2}$ when $n = 2$. That is PSK territory. The average power constraint lets points move inside and outside the ring as long as the average energy stays the same. With 16 points, packing

some of them inside the ring is more efficient than squeezing 16 points onto a circle: this is why 16-QAM beats 16-PSK, and why the paper's learned (2,4) constellation under average power looks like a hexagonal grid.

21.3.4 The noise layer and its variance from E_b/N_0

The channel is an additive noise layer: $\mathbf{y} = \mathbf{x} + \mathbf{w}$ with each real component of $\mathbf{w}$ drawn from $\mathcal{N}(0, \beta)$. The paper sets

Key equation: noise variance of the channel layer

$$\beta = \left(2R\,\frac{E_b}{N_0}\right)^{-1}, \qquad R = \frac{k}{n}. \tag{21.5}$$

Derivation. Use the real-valued view, where each of the n channel uses is one real number with unit average energy (the energy constraint gives $\|\mathbf{x}\|^2 = n$, so energy per real channel use $E_s = 1$). One real channel use carries $R = k/n$ bits, so $E_b = E_s/R = 1/R$. Real white noise with two-sided spectral density $N_0/2$ has variance $\beta = N_0/2$ per real sample. Hence

$$\frac{E_b}{N_0} = \frac{1/R}{2\beta} \quad \Longrightarrow \quad \beta = \frac{1}{2R\,E_b/N_0}.$$

This is exactly the BPSK convention: with $R = 1$ and amplitude ± 1 the bit error probability is $Q(1/\sqrt{\beta}) = Q\big(\sqrt{2E_b/N_0}\big)$, the textbook BPSK formula. In the $(7,4)$ autoencoder each codeword has 7 real components, so its BLER can be compared directly to the Hamming $(7,4)$ code sent with BPSK, which also uses 7 real channel uses for 4 bits.

Worked example: noise level at the training point

The paper trains at $E_b/N_0 = 7\,\text{dB}$. For $(7,4)$: $R = 4/7$, $E_b/N_0 = 10^{0.7} = 5.012$, so $\beta = 1/(2 \cdot 0.5714 \cdot 5.012) = 0.1746$ and the noise standard deviation is $\sqrt{\beta} = 0.418$. Each codeword component has average magnitude 1, so the noise is large but not overwhelming — exactly the regime where the network has to discover redundancy to survive.

21.3.5 One-hot messages and the categorical cross-entropy

The receiver ends with a softmax, so its output $\mathbf{p} \in (0,1)^M$ is a probability vector with $\sum_i p_i = 1$, interpreted as $p_i \approx \Pr(s = i \mid \mathbf{y})$. The loss is the categorical cross-entropy between the one-hot label and $\mathbf{p}$:

$$\mathcal{L} = -\sum_{i=1}^{M} [\mathbf{1}_s]_i \log p_i = -\log p_s. \tag{21.6}$$

Averaging over a mini-batch gives the training objective. Why is this the right loss? Because the receiver that minimises the expected cross-entropy is the one whose softmax equals the true posterior $\Pr(s \mid \mathbf{y})$, and choosing $\hat{s} = \arg\max_i p_i$ from the true posterior is the maximum a-posteriori (MAP) decision, which minimises the BLER. So minimising cross-entropy pushes the receiver towards the MAP detector, and pushes the transmitter towards codewords that the MAP detector rarely confuses.

Worked example: one training sample by hand

Take $M = 4$ and $s = 3$. Suppose the receiver outputs logits $(0.2, -1.0, 2.1, 0.4)$. The softmax is $e^{z_i} / \sum_j e^{z_j}$: the exponentials are $(1.221, 0.368, 8.166, 1.492)$, which sum to 11.247, so $\mathbf{p} = (0.109, 0.033, 0.726, 0.133)$. The loss is $-\log 0.726 = 0.320$. The gradient with respect to the logits is $\mathbf{p} - \mathbf{1}_s = (0.109, 0.033, -0.274, 0.133)$ (Chapter 8): push the correct logit up, the others down. Backpropagation then carries this gradient through the receiver, *through the noise layer* (whose derivative is simply $\partial \mathbf{y} / \partial \mathbf{x} = \mathbf{I}$), through the normalisation, and into the transmitter weights.

Pitfall: the channel must be differentiable

The gradient reaches the transmitter only because we know the channel in closed form and can differentiate $\mathbf{y} = \mathbf{x} + \mathbf{w}$. For a real radio channel and real hardware we cannot. The paper names this as an open challenge; it later led to model-free training with reinforcement learning, and to learning a GAN model of the channel (see Section 21.7 and Chapter 24).

21.4 Training procedure as reported

The paper's recipe is short:

- Training data is free: draw random messages s, and the noise layer draws fresh noise every time. There is no fixed data set and no overfitting in the classical sense.
- Training is done at a *fixed* $E_b / N_0 = 7$ dB with Adam at learning rate 0.001 and the categorical cross-entropy loss.
- The authors observed that increasing the batch size during training improves accuracy.
- The trained system is then evaluated over a whole range of E_b / N_0 by Monte-Carlo simulation of the BLER.

The paper does not report the number of training iterations or batch sizes used, so we choose our own in Section 21.6. In the discussion section the authors warn that the choice of training SNR matters: training at low SNR, for instance, did not allow the network to discover structure that matters at high SNR.

21.5 Results reported in the paper

21.5.1 Learned codes versus Hamming and uncoded BPSK

The headline experiment compares a $(7, 4)$ autoencoder with BPSK transmission of 4 bits (uncoded, "$(4, 4)$"), and with the Hamming $(7, 4)$ code using either hard-decision decoding or maximum-likelihood decoding (MLD). The paper reports that the autoencoder learned, without any prior knowledge, an encoder and decoder that together achieve the BLER of the Hamming $(7, 4)$ code with MLD, over the plotted range -4 to 8 dB. A second plot shows that the $(2, 2)$ autoencoder achieves the same BLER as uncoded BPSK, while the $(8, 8)$ autoencoder *outperforms* uncoded BPSK over the full range: with 8 channel uses for 8 bits it has learned a joint coding-and-modulation scheme with a coding gain.

Intuition: why ML decoding matters for the comparison

A hard-decision Hamming decoder first slices each received number to a bit and then corrects at most one bit error. It throws away the reliability information in $|y_i|$. The ML decoder compares $\mathbf{y}$ with all 16 codewords in Euclidean distance and picks the closest one; for equal-energy codewords and Gaussian noise this is the optimal (minimum BLER) decision. The autoencoder's receiver is a soft decoder by construction, so the fair benchmark is MLD.

Baselines in closed form. In uncoded BPSK a bit is in error with probability $p = Q(\sqrt{2E_b/N_0})$, and a 4-bit block is wrong if any bit is wrong: $P_e = 1 - (1-p)^4$. With the Hamming code at rate $4/7$, each coded bit is wrong with probability $p_c = Q\big(\sqrt{(8/7)\,E_b/N_0}\big)$; hard decision fails when 2 or more of the 7 bits are wrong:

$$P_e^{\mathrm{HD}} = 1 - (1-p_c)^7 - 7p_c(1-p_c)^6. \tag{21.7}$$

MLD has no simple closed form; we simulate it. At high SNR its BLER is dominated by the minimum distance: Hamming codewords differ in at least 3 positions, i.e. squared Euclidean distance $4 \cdot 3 = 12$ between BPSK codewords. Compared with uncoded BPSK (squared distance 4 per bit, but energy per bit 1 instead of $7/4$), the asymptotic gain is $10\log_{10}(R\,d_{\min}) = 10\log_{10}(\frac{4}{7} \cdot 3) = 2.34\,\mathrm{dB}$.

Worked example: the baselines at 7 dB

$E_b/N_0 = 5.012$. Uncoded: $p = Q(\sqrt{10.02}) = Q(3.166) = 7.73 \times 10^{-4}$, so $P_e = 1 - (1-p)^4 = 3.09 \times 10^{-3}$. Hamming HD: $p_c = Q(\sqrt{5.73}) = Q(2.393) = 8.35 \times 10^{-3}$, so $P_e^{\mathrm{HD}} = 1 - (0.99165)^7 - 7(0.00835)(0.99165)^6 = 1.42 \times 10^{-3}$. Hard-decision Hamming gains only about a factor of two in BLER over uncoded transmission at this SNR; soft (ML) decoding gains about a factor of 28 (our simulation in Section 21.6 gives 1.1×10^{-4}).

21.5.2 Learned constellations

For $n = 2$ the transmit vectors can be drawn directly as points in a plane. The paper shows four panels: (a) $(2,2)$ with the energy constraint converges to QPSK with an arbitrary rotation; (b) a system the text calls $(4,2)$ and the caption calls $(2,4)$, which produces a rotated 16-PSK constellation under the energy constraint; (c) $(2,4)$ with the average power constraint gives a mixed pentagonal/hexagonal grid whose BLER the authors report as indistinguishable from 16-QAM; (d) for $(7,4)$, where the codewords live in 7 dimensions, a two-dimensional t-SNE embedding of the *received noisy* vectors $\mathbf{y}$, which shows 16 separate clusters.

Definition: t-SNE

t-distributed stochastic neighbour embedding is a visualisation method that places high-dimensional points in 2-D so that points which are close in the original space stay close in the picture. Distances between far-apart clusters in a t-SNE plot carry little meaning; only the neighbourhood structure does.

21.5.3 Multiple transmitters and receivers: the interference channel

The paper extends the idea to two transmitter–receiver pairs that share the channel. Transmitter 1 sends s_1 and transmitter 2 sends s_2; each receiver sees both signals:

$$\mathbf{y}_1 = \mathbf{x}_1 + \mathbf{x}_2 + \mathbf{n}_1, \qquad \mathbf{y}_2 = \mathbf{x}_2 + \mathbf{x}_1 + \mathbf{n}_2, \qquad \mathbf{n}_i \sim \mathcal{CN}(0, \beta \mathbf{I}_n). \tag{21.8}$$

This is the simplest interference channel. There are now two autoencoders and two losses $\tilde{L}_1, \tilde{L}_2$ (the cross-entropies of each receiver). They must be trained jointly because each transmitter is interference for the other receiver.

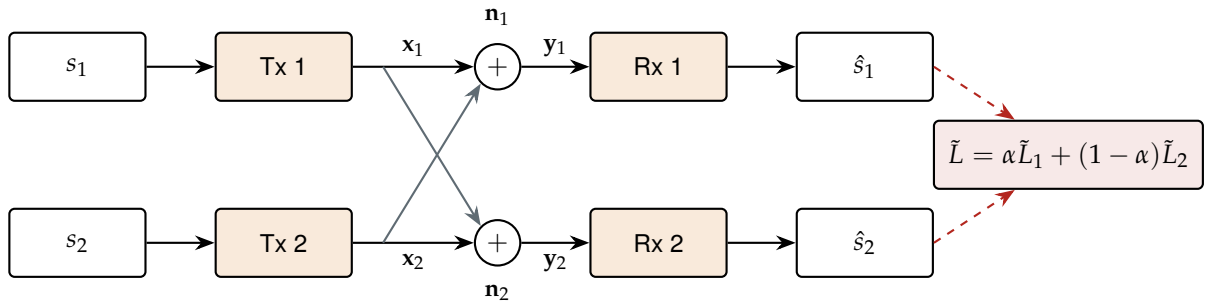


Figure 21.2: The two-user interference channel trained as two coupled autoencoders. Each transmitter's signal reaches both receivers (gray arrows). A single weighted loss couples the two links; the weight α is adapted during training so that neither link is sacrificed.

The weighted loss. A natural objective is $\tilde{L} = \alpha\tilde{L}_1 + (1-\alpha)\tilde{L}_2$ with $\alpha \in [0,1]$. The paper observes that the obvious $\alpha = 0.5$ does *not* give equal performance and generally leads to unfair, suboptimal solutions. Instead α is updated dynamically at each mini-batch t:

Key equation: dynamic loss weight for the interference channel

$$\alpha_{t+1} = \frac{\tilde{L}_1(\theta_t)}{\tilde{L}_1(\theta_t) + \tilde{L}_2(\theta_t)}, \qquad t > 0, \qquad \alpha_0 = 0.5. \tag{21.9}$$

If link 1 is currently doing worse (larger $\tilde{L}_1$), its weight increases, so the next gradient step favours it. At equilibrium the two losses are roughly equal, i.e. the solution is fair.

Worked example: one weight update

Suppose at step t the losses are $\tilde{L}_1 = 0.30$ and $\tilde{L}_2 = 0.10$. Then $\alpha_{t+1} = 0.30/0.40 = 0.75$: three quarters of the gradient now comes from link 1. If after some steps $\tilde{L}_1 = \tilde{L}_2 = 0.15$, then $\alpha = 0.5$ and both links are served equally.

The paper uses $(n,k) \in \{(1,1),(2,2),(4,4),(4,8)\}$ with an average power constraint, and compares with time-sharing (TS) baselines that use $2^{2k/n}$-QAM: each transmitter sends on every other channel use. It reports identical BLER for $(1,1)$ and $(2,2)$ and gains of around 0.7 dB for $(4,4)$ and 1 dB for $(4,8)$ at a BLER of 10^{-3}. For $(1,1)$ the transmitters learned BPSK-like constellations in orthogonal directions (which is equivalent to time sharing in I/Q); for $(2,2)$ the constellations were no longer orthogonal and the authors interpret them as a form of superposition coding.

21.5.4 Radio transformer networks

An important idea in the paper is to let the receiver use *expert knowledge* without hard-coding it. Many channel effects have a known parametric form: a carrier phase offset multiplies every sample by $e^{j\varphi}$; a time offset shifts the signal; a multipath channel convolves it with a short filter. A *radio transformer network* (RTN) adds three pieces to the receiver:

1. a learned **parameter estimator** $g_\omega : \mathbb{R}^n \to \mathbb{R}^p$ that looks at $\mathbf{y}$ and outputs a parameter vector $\boldsymbol{\omega}$;
2. a fixed, differentiable **parametric transform** $t : \mathbb{R}^n \times \mathbb{R}^p \to \mathbb{R}^{n'}$ that applies the known correction $\bar{\mathbf{y}} = t(\boldsymbol{\omega}, \mathbf{y})$;
3. a learned **discriminative network** $g : \mathbb{R}^{n'} \to \mathcal{M}$ that decodes the canonicalised signal $\bar{\mathbf{y}}$.

All three are trained end-to-end with the same cross-entropy loss. The estimator is *never told* the true phase or channel; it learns whatever parameters make the decoder's job easiest. This is the communications version of spatial transformer networks in computer vision, which learn to rotate and crop an image before classifying it.

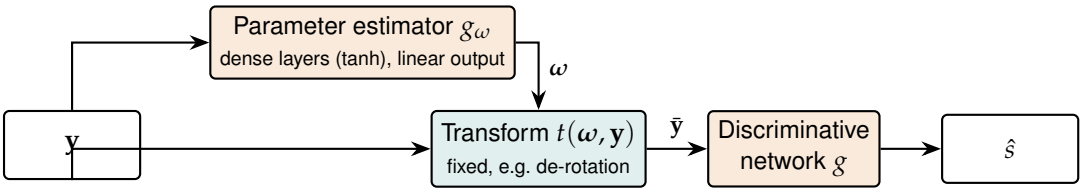


Figure 21.3: A radio transformer network inside the receiver. A small network estimates channel parameters $\boldsymbol{\omega}$ from $\mathbf{y}$, a fixed differentiable transform uses them to undo the channel effect, and the decoder works on the corrected signal $\bar{\mathbf{y}}$. Expert knowledge enters through the *form* of t, not through labelled parameters.

For a phase offset φ, with $\mathbf{y}_c = e^{j\varphi}\tilde{\mathbf{y}}_c$, the estimator outputs $\hat{\varphi}$ and the transform de-rotates:

$$\bar{\mathbf{y}} = t(\hat{\varphi}, \mathbf{y}) = \left[\cos\hat{\varphi}\,\Re\{\mathbf{y}_c\} + \sin\hat{\varphi}\,\Im\{\mathbf{y}_c\},\ \cos\hat{\varphi}\,\Im\{\mathbf{y}_c\} - \sin\hat{\varphi}\,\Re\{\mathbf{y}_c\}\right]^{\mathsf{T}}, \tag{21.10}$$

which is just the real-valued way of writing $e^{-j\hat{\varphi}}\mathbf{y}_c$ (check: $(a+jb)(\cos\hat{\varphi} - j\sin\hat{\varphi}) = (a\cos\hat{\varphi} + b\sin\hat{\varphi}) + j(b\cos\hat{\varphi} - a\sin\hat{\varphi})$). The paper describes this estimator as two dense layers with tanh activations followed by a linear output layer.

The paper's main RTN experiment uses a multipath channel with $L = 3$ i.i.d. Rayleigh taps $\mathbf{h}_c \sim \mathcal{CN}(0, L^{-1}\mathbf{I}_L)$, so that $y_{c,i} = \sum_{\ell=1}^{L} h_{c,\ell} x_{c,i-\ell+1} + n_{c,i}$, and an $(8,4)$ autoencoder. Here the estimator predicts a complex vector $\boldsymbol{\omega}_c$ (represented by $2L$ real numbers), and the transform computes the complex convolution of $\mathbf{y}_c$ with $\boldsymbol{\omega}_c$ — a learned equaliser filter. The reported result: the plain $(8,4)$ autoencoder struggles to match differential BPSK with maximum-likelihood sequence estimation plus a Hamming $(7,4)$ code, while the autoencoder with RTN outperforms that baseline; the RTN version also trains faster and, according to the paper, consistently beats the plain autoencoder independently of the chosen hyper-parameters.

21.5.5 CNNs for modulation classification

The last application is not end-to-end communication but *signal classification*: given 128 complex baseband samples, which of ten modulations (analog and digital: AM, FM, PSK, QAM, ...) produced them? The paper uses a dataset of 1.2 million such sequences at 20 SNRs from -20 to

Table 21.2: The modulation-classification CNN as listed in the paper (324,330 trainable parameters in total).

Layer	Output dimensions
Input (I and Q rows)	2×128
Conv, 128 filters of size 2×8, ReLU	128×121
Max pool, size 2, stride 2	128×60
Conv, 64 filters of size 1×16, ReLU	64×45
Max pool, size 2, stride 2	64×22
Flatten	1408
Dense 128, ReLU $\rightarrow$ Dense 64, ReLU $\rightarrow$ Dense 32, ReLU	32
Dense 10, softmax	10

18 dB, generated with harsh channel effects (multipath fading, sample-rate and centre-frequency offset), and referenced as RadioML 2016.10b. The CNN (the paper's Table V) is:

Worked example: checking the shapes and the parameter count

A 2×8 kernel slid along 128 samples with no padding gives $128 - 8 + 1 = 121$ positions and collapses the two rows: output 128×121. Pooling by 2 gives $\lfloor 121/2 \rfloor = 60$. A length-16 kernel gives $60 - 16 + 1 = 45$; pooling gives 22; flattening gives $64 \cdot 22 = 1408$. Parameters: conv1 $128 \cdot (2 \cdot 8) + 128 = 2176$; conv2 $64 \cdot (128 \cdot 16) + 64 = 131{,}136$; dense $1408 \cdot 128 + 128 = 180{,}352$; $128 \cdot 64 + 64 = 8256$; $64 \cdot 32 + 32 = 2080$; $32 \cdot 10 + 10 = 330$. Sum: 324,330 — exactly the paper's figure.

The comparison is with gradient-boosted trees (and a single tree) trained on a mix of 16 analog and cumulant expert features. The paper reports that the CNN outperforms the boosted feature classifier by around 4 dB at low-to-medium SNR, with almost identical performance at high SNR; the single tree is about 6 dB worse than the CNN at medium SNR and 3.5% worse at high SNR. At 10 dB the CNN's main confusions are QAM16 versus QAM64 and wideband FM versus AM-DSB. Chapter 22 treats this application in depth.

21.6 Our small-scale reproduction

All code is in `code/ch21/`. Everything runs on one CPU thread; each script finishes in a few minutes. We reproduce (i) the $(7,4)$ and $(2,2)$ BLER curves, (ii) learned constellations for several (n,k) under both constraints, (iii) a t-SNE picture of the $(7,4)$ received signals, and (iv) a tiny RTN experiment with a random phase offset.

21.6.1 The model in PyTorch

```
class Autoencoder(nn.Module):
    def __init__(self, n, k, norm="energy"):
        super().__init__()
        self.n, self.k, self.M, self.R, self.norm = n, k, 2 ** k, k / n, norm
        M = self.M
        self.tx = nn.Sequential(nn.Linear(M, M), nn.ReLU(), nn.Linear(M, n))
        self.rx = nn.Sequential(nn.Linear(n, M), nn.ReLU(), nn.Linear(M, M))

    def encode(self, s):
```

```
        """s: LongTensor of message indices -> normalised transmit vectors x (batch, n)."""
        x = self.tx(nn.functional.one_hot(s, self.M).float())
        if self.norm == "energy":           # every codeword has ||x||^2 = n
            x = np.sqrt(self.n) * x / x.norm(dim=1, keepdim=True)
        else:                               # average power: E||x||^2 = n over the batch
            x = np.sqrt(self.n) * x / x.pow(2).sum(1).mean().sqrt()
        return x

    def constellation(self):
        """All M transmit vectors (average-power normalisation done over all M points)."""
        with torch.no_grad():
            return self.encode(torch.arange(self.M)).numpy()

    def forward(self, s, sigma):
        x = self.encode(s)
        y = x + sigma * torch.randn_like(x)
        return self.rx(y)

def train_ae(model, ebno_db=7.0, steps=4000, batch=1000, lr=1e-3, seed=0, log_every=0, cosine=False):
    torch.manual_seed(seed)
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    sch = torch.optim.lr_scheduler.CosineAnnealingLR(opt, steps) if cosine else None
    sigma = noise_std(ebno_db, model.R)
    lossf = nn.CrossEntropyLoss()
    hist = []
    for it in range(steps):
        s = torch.randint(0, model.M, (batch,))
        loss = lossf(model(s, sigma), s)
        opt.zero_grad(); loss.backward(); opt.step()
        if sch: sch.step()
        if log_every and it % log_every == 0:
            hist.append((it, loss.item()))
    return hist
```

The normalisation is applied inside `encode`; the average-power version normalises over the batch, exactly as in Eq. (21.4). For evaluation we freeze the constellation (all M codewords) and send random messages through the AWGN channel.

21.6.2 Two training recipes

We trained the $(7,4)$ autoencoder with two recipes, each 4000 Adam steps with batches of 1000 random messages, followed by 500 steps with batches of 5000 at learning rate 10^{-4} (the paper's "larger batches help" observation):

- **Recipe A (paper-like):** train at $E_b/N_0 = 7\,\text{dB}$ with constant learning rate 10^{-3}.
- **Recipe B (ours):** train at $E_b/N_0 = 3\,\text{dB}$ with learning rate 10^{-2} decayed to zero by a cosine schedule, then the large-batch phase at 7 dB.

The paper probably trained for much longer than our 4500 steps; with our short budget recipe A stopped early in a local optimum, so we added recipe B to show what a better-tuned short training achieves.

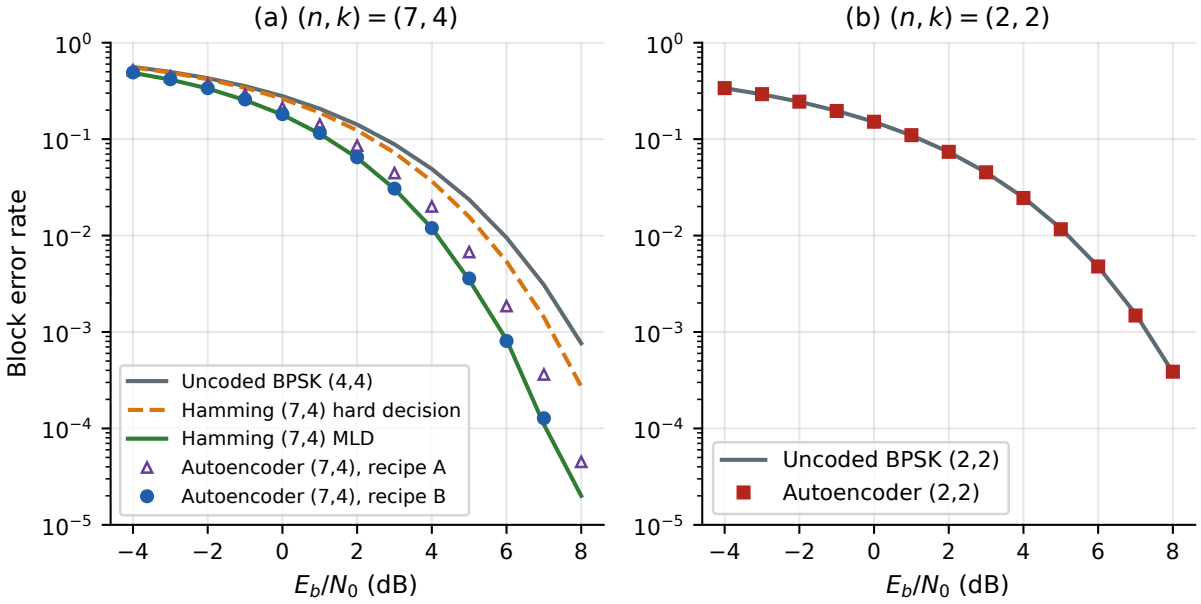


Figure 21.4: Our reproduction (AWGN, 400,000 Monte-Carlo blocks per point). (a) $(7, 4)$: the autoencoder trained with recipe B (blue dots) lies on top of the Hamming $(7, 4)$ code with ML decoding (green line), about 2 dB better than uncoded BPSK at BLER 10^{-3}, and clearly better than hard-decision Hamming decoding (orange dashed). Recipe A (purple triangles, short constant-rate training at 7 dB) is about 0.5 dB worse. (b) $(2, 2)$: the autoencoder (red squares) coincides with uncoded BPSK (gray), as in the paper.

What we found. Figure 21.4 shows the result. With recipe B the learned $(7, 4)$ system matches Hamming MLD within Monte-Carlo accuracy over the whole range -4 to 8 dB; at 6 dB, for example, we measured 8.1×10^{-4} versus 8.3×10^{-4} for MLD, at 4 dB 1.20×10^{-2} versus 1.19×10^{-2}. That is the paper's main claim, reproduced. The learned code's minimum Euclidean distance was 3.30, close to (but not equal to) the Hamming code's $\sqrt{12} = 3.46$: the network did not rediscover the Hamming code itself, but a different set of 16 points on the 7-dimensional sphere with practically the same error rate. Recipe A gave a smaller minimum distance (3.09) and lost about half a dB — a direct illustration of the paper's warning that training hyper-parameters and training SNR matter. The $(2, 2)$ autoencoder exactly matches uncoded BPSK, as reported.

Pitfall: don't train once and conclude

Our first attempt (recipe A) suggested the autoencoder was 0.5–0.7 dB worse than MLD. Separating the problem showed why: decoding the learned constellation with an *ideal* ML detector gave nearly the same BLER as the neural receiver, so the receiver was fine; it was the *transmitter* (the constellation) that had not converged. Whenever an end-to-end system underperforms, test the transmitter and the receiver separately against ideal counterparts.

21.6.3 Learned constellations

Figure 21.5 reproduces the qualitative findings of the paper's constellation figure and extends them to $k = 6$. A useful number to check the pictures: with average energy 2 per 2-D symbol, 16-PSK has minimum distance $2\sqrt{2}\sin(\pi/16) = 0.552$ and 16-QAM has 0.894. Our energy-constrained $(2, 4)$ points reached 0.543 (almost 16-PSK) and our average-power $(2, 4)$ points reached 0.830, close to 16-QAM; the paper reports its hexagonal (2,4) solution as having BLER

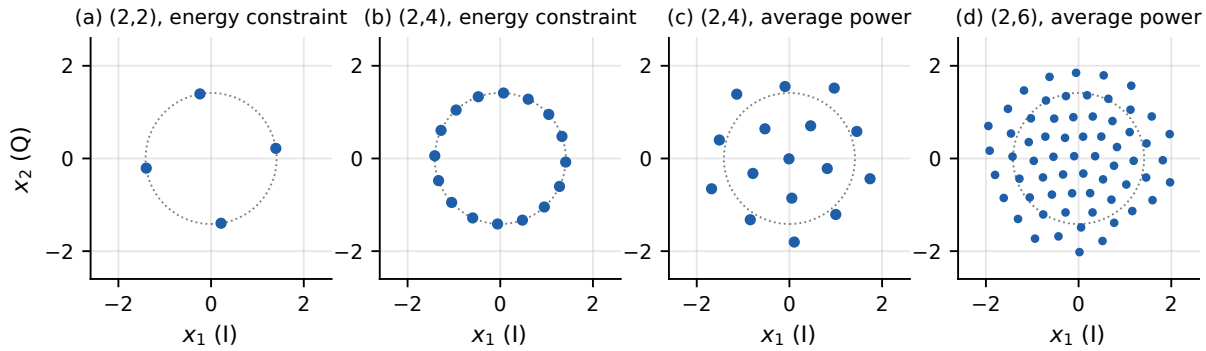


Figure 21.5: Constellations learned by our $n = 2$ autoencoders (dotted circle: radius $\sqrt{2}$, the energy-constraint sphere). (a) $(2, 2)$ converges to a rotated QPSK. (b) $(2, 4)$ with the energy constraint places 16 points on the circle: a rotated, slightly irregular 16-PSK. (c) $(2, 4)$ with the average power constraint fills the disc with a centre point and two rings, a hexagonal-like packing. (d) $(2, 6)$, 64 points under average power: a roughly hexagonal lattice inside a disc.

indistinguishable from 16-QAM. The (2,6) solution is the network's answer to "design 64-QAM": a hexagonal packing with a round boundary, which is what lattice theory says is denser than the square grid.

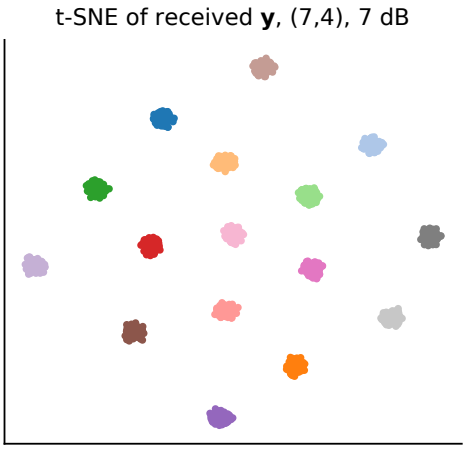


Figure 21.6: t-SNE embedding of 960 received vectors $\mathbf{y} \in \mathbb{R}^7$ of our $(7, 4)$ autoencoder at 7 dB (60 per message, one colour per message). The 16 messages form 16 well separated clusters, which is why the receiver almost never confuses them at this SNR. Compare with the paper's Fig. 4d.

21.6.4 A tiny RTN: learning to undo a random phase

The paper's RTN experiment uses multipath fading. We built the simplest possible RTN problem to see the mechanism: $k = 4$ bits over $n = 4$ *complex* channel uses, a channel that multiplies the whole block by $e^{j\varphi}$ with φ uniform in $[0, 2\pi)$, and complex AWGN. The plain autoencoder's receiver sees the rotated block. The RTN receiver has a small estimator (two tanh layers and a linear output, as in the paper) followed by a de-rotation and the same decoder. We made one change from the paper's phase example: our estimator outputs a complex number $w = a + jb$ and the transform multiplies by $\bar{w}/\,|w| = e^{-j\hat{\varphi}}$. Predicting the angle $\hat{\varphi}$ directly fails, because a smooth network cannot produce the jump from 2π to 0; predicting a point on the unit circle has no such discontinuity (the paper's multipath RTN also predicts complex filter taps).

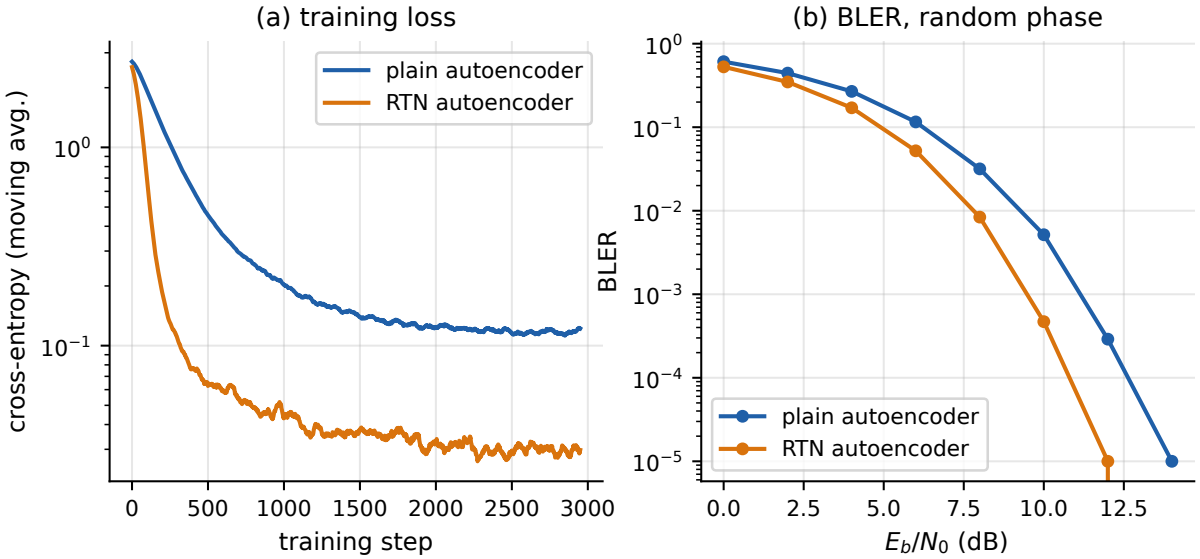


Figure 21.7: Our RTN experiment (random phase in $[0, 2\pi)$, $(n, k) = (4, 4)$ complex, trained at 8 dB for 3000 steps). (a) Training loss of the plain autoencoder and of the autoencoder with RTN. (b) BLER versus E_b/N_0. Both receivers use one hidden layer of $M = 16$ units, as in the paper. The RTN receiver trains much faster and reaches a lower BLER.

What we found. With the receiver sized as in the paper (one hidden layer of $M = 16$ ReLU units), the RTN helps in exactly the two ways the paper reports for its multipath experiment (Fig. 21.7). Training converges much faster and to a lower loss (about 0.03 versus 0.12 after 3000 steps), and the BLER is clearly lower: at 10 dB we measured 4.7×10^{-4} with the RTN versus 5.2×10^{-3} without, roughly a 1.5–2 dB gain at BLER 10^{-3}. The plain autoencoder has to learn a code whose 16 codewords stay distinguishable under *every* rotation and a receiver that recognises them at any angle; the RTN receiver gets a de-rotated signal and can spend its small capacity on decoding.

An honest caveat: in a first run we gave both receivers four times more hidden units ($4M = 64$; run `python rtn_phase.py 64`). The plain autoencoder then reached the same BLER as the RTN version (it had enough capacity to learn rotation invariance by itself), and the RTN's only advantage was a smoother start of training. So, in our toy problem, the RTN's benefit is largest when the receiver is small — which is the regime that matters on embedded hardware. We also found that the learned $\hat{\varphi} = \arg w$ is *not* the true phase: the estimator only needs a phase that maps every received block to a canonical orientation *up to the code's own symmetries*, and nothing forces it to agree with φ. This is typical of RTNs: the parameters are useful, not necessarily physical.

21.6.5 Comparison with the paper

Item	Paper (as reported)	Our reproduction
$(7,4)$ AE vs Hamming MLD	same BLER	same BLER (recipe B); 0.5 dB worse with a short paper-like recipe
$(2,2)$ AE vs uncoded BPSK	same BLER	same BLER
$(2,2)$ constellation	rotated QPSK	rotated QPSK
$(2,4)$ energy / avg. power	16-PSK / hexagonal, ≈16-QAM BLER	16-PSK / hexagonal-like, $d_{\min}$ close to 16-QAM
$(7,4)$ t-SNE	16 clusters	16 clusters
RTN	faster training, better BLER in multipath	faster training, 1.5–2 dB better (random phase, small receiver); no BLER gain with a 4× wider receiver

21.7 Limitations and follow-up work

The paper's own discussion section lists open challenges, which we paraphrase:

- **Data sets.** Unlike vision (MNIST, ImageNet), communications had no standard benchmark data sets for comparing learned methods.
- **Data representation and training SNR.** How to represent complex signals for a network, and at which SNR to train, are open; training at low SNR did not reveal structure needed at high SNR.
- **Complex-valued networks.** Communications is full of phase rotations, conjugates and magnitudes; libraries lacked complex-valued layers.
- **Scalability.** The biggest challenge: the one-hot representation needs $M = 2^k$ inputs and outputs. Already for $k = 100$ bits, $M = 2^{100}$ and training is prohibitive.
- **System identification for real channels.** Backpropagation needs a differentiable channel model; for real hardware and channels this is a major challenge.
- **Beyond communications.** The same tools could serve learning tasks such as through-wall tracking and gesture recognition from radio signals.

Several of these were attacked soon after. The "no channel model" problem was addressed by training the transmitter with reinforcement learning (a policy-gradient estimate of the gradient through an unknown channel; see Chapter 24) by Aoudia and Hoydis, by learning a generative model of the channel, and by over-the-air experiments with software-defined radios (Dörner et al.). The scalability problem is usually sidestepped by *not* learning the code: keep a classical channel code (LDPC, polar) and learn only the constellation ("geometric shaping") or the demapper, which outputs bit-wise log-likelihood ratios and is trained with a binary cross-entropy instead of the categorical one. Chapter 23 studies the other half of the problem: learning only the *decoder* for a given code, where the 2^k curse reappears.

Pitfall: the training channel is the deployment channel

An autoencoder is optimal only for the channel model it was trained on. If it was trained in AWGN at 7 dB and deployed in a fading channel with carrier offset, nothing guarantees graceful degradation. Classical systems degrade predictably because their assumptions are explicit; learned systems' assumptions are implicit in the training data.

21.8 From paper to practice in 5G NR and 6G

Wireless connection: where the autoencoder idea stands in standards

5G NR (3GPP Releases 15–17) has no learned transmitter or receiver: modulation (QPSK to 256-QAM, with 1024-QAM added in Release 17 for some cases), LDPC codes for data and polar codes for control are fully specified. In Release 18, 3GPP ran a *study* on AI/ML for the NR air interface (TR 38.843) with three representative use cases: CSI feedback (including compression with a "two-sided" model, i.e. an encoder at the UE and a decoder at the gNB — an autoencoder in exactly this chapter's sense), beam management, and positioning. Release 19 moved parts of this into normative work. A learned *constellation and waveform*, as in this paper, is discussed as a 6G research topic ("AI-native air interface"), not as a 5G feature.

For a PHY engineer the practical value of the paper is less "replace the modem" and more a set of tools:

- **Learned receivers and demappers.** A neural receiver that outputs LLRs for the existing LDPC decoder can absorb impairments (phase noise, non-linearity, imperfect channel estimates) without changing the standard. This is where most industrial prototypes sit.
- **Two-sided models need interoperability.** When the encoder and decoder are made by different vendors (UE and gNB), the joint training of this chapter is not possible in practice. That is why the 3GPP study spent much effort on how two-sided models are trained and tested.
- **Short-block joint coding.** For tiny payloads (a few bits of control information) the autoencoder can match or beat hand-designed short codes, as the $(8, 8)$ result suggests; but standards already use carefully optimised short codes (e.g. Reed–Muller-based coding for very small UCI payloads in NR).
- **Geometric shaping.** Learned constellations under a power constraint approach the hexagonal packing gain; with a fixed outer code this is a realistic 6G direction, but receivers must then handle non-Gray, non-square constellations.
- **RTN thinking.** Putting known structure (rotations, convolutions, FFTs) inside a learned receiver is now standard practice; it reduces the data and model size needed and makes behaviour more interpretable.

Wireless connection: a T2200-class gNB perspective

On a DSP-based baseband (fixed-point, small memory) the autoencoder's receiver is just two small matrix products for $(7, 4)$ — 791 parameters fit easily in on-chip memory. The obstacle is not compute but the 2^k output layer: a 1000-bit transport block can never be a single softmax. Learned components therefore enter at the symbol level (demapping, equalisation, channel estimation) where dimensions are small and fixed.

Chapter recap

- An (n, k) autoencoder sends one of $M = 2^k$ messages over n channel uses; rate $R = k/n$. Transmitter: one-hot $\to$ Dense(M, ReLU) $\to$ Dense(n) $\to$ normalisation. Receiver: Dense(M, ReLU) $\to$ Dense(M) $\to$ softmax.

- Normalisation enforces $\|\mathbf{x}\|^2 = n$ per message (energy) or on average (power). The channel is a noise layer with $\beta = (2R\,E_b/N_0)^{-1}$.
- Categorical cross-entropy drives the receiver towards the MAP detector and the transmitter towards well separated codewords.
- Reported: $(7,4)$ AE = Hamming $(7,4)$ MLD; $(2,2)$ AE = BPSK; $(8,8)$ AE beats BPSK; learned constellations QPSK, 16-PSK, hexagonal; interference channel gains of 0.7–1 dB with a dynamically weighted loss; RTN helps in multipath; CNN classifier beats expert features by about 4 dB at low/medium SNR.
- Our CPU reproduction confirms the $(7,4)$ and $(2,2)$ results and the constellations, and shows that training recipe and training SNR matter.
- Open problems: scalability in k, differentiable channel models, complex-valued networks, data sets, choice of training SNR.

Exercises

1. Compute the number of trainable parameters of a $(4,8)$ and a $(16,8)$ autoencoder with the paper's architecture. Which part dominates?
2. Compute the noise standard deviation $\sqrt{\beta}$ for a $(2,2)$ autoencoder at $E_b/N_0 = 4$ dB, and the bit error probability of uncoded BPSK at the same E_b/N_0.
3. For 16-PSK and square 16-QAM with the same average energy $E_s = 2$, compute the minimum distance. How many dB of extra energy does 16-PSK need to have the same minimum distance as 16-QAM?
4. Explain in your own words why, for equally likely messages, the receiver that minimises the expected categorical cross-entropy also gives the minimum-BLER decision.
5. In the interference channel, the losses at step t are $\tilde{L}_1 = 0.05$ and $\tilde{L}_2 = 0.20$. Compute α_{t+1} and explain which link the next update favours. What would happen with a fixed $\alpha = 0.5$ if one link is intrinsically harder?
6. Verify that Eq. (21.10) with $\hat{\varphi} = \varphi$ exactly undoes the rotation $\mathbf{y}_c = e^{j\varphi}\tilde{\mathbf{y}}_c$.
7. Why does the average-power constraint give a lower BLER than the energy constraint for $(2,4)$ but not for $(2,2)$?
8. (Coding) Modify `ae_bler.py` to train an $(8,8)$ autoencoder (256 messages) and compare it with uncoded BPSK. How long does it take compared with $(7,4)$, and why?

Answers

1. Use $(2M+1)(M+n)+2M$ with $M = 256$. $(4,8)$: $513 \cdot 260 + 512 = 133{,}892$. $(16,8)$: $513 \cdot 272 + 512 = 140{,}048$. The two $M \times M$ layers ($2(M^2+M) = 131{,}584$) dominate; n hardly matters.
2. $R = 1$, $E_b/N_0 = 10^{0.4} = 2.512$, $\beta = 1/(2 \cdot 2.512) = 0.199$, $\sqrt{\beta} = 0.446$. BPSK: $p = Q(\sqrt{2 \cdot 2.512}) = Q(2.241) = 0.0125$.
3. 16-PSK: points on a circle of radius $\sqrt{2}$, $d = 2\sqrt{2}\sin(\pi/16) = 0.552$. 16-QAM: levels $\pm 1, \pm 3$ scaled by c with $E_s = c^2 \cdot 10 = 2$, so $c = 0.447$ and $d = 2c = 0.894$. Ratio of

distances $0.894/0.552 = 1.62$; energy ratio $1.62^2 = 2.62$, i.e. about 4.2 dB.

4. The expected cross-entropy $\mathbb{E}[-\log p_s(\mathbf{y})]$, for each fixed $\mathbf{y}$, equals $-\sum_i \Pr(s = i|\mathbf{y}) \log p_i(\mathbf{y})$. This is minimised over probability vectors $\mathbf{p}$ by $\mathbf{p} = \Pr(\cdot|\mathbf{y})$ (Gibbs' inequality: cross-entropy $\geq$ entropy with equality iff the distributions are equal). A perfectly trained receiver therefore outputs the posterior, and $\arg\max_i p_i$ is the MAP decision, which minimises $\Pr(\hat{s} \neq s)$.

5. $\alpha_{t+1} = 0.05/0.25 = 0.2$, so link 2 gets weight 0.8: the update favours the worse link 2. With fixed $\alpha = 0.5$ the optimiser would minimise the sum and could happily make the easy link nearly perfect while leaving the hard link poor (the paper reports such unfair solutions).

6. Write $\tilde{y} = a + jb$, $y = e^{j\varphi}\tilde{y} = (a\cos\varphi - b\sin\varphi) + j(a\sin\varphi + b\cos\varphi)$. Insert into the first component: $\cos\varphi(a\cos\varphi - b\sin\varphi) + \sin\varphi(a\sin\varphi + b\cos\varphi) = a$. Second: $\cos\varphi(a\sin\varphi + b\cos\varphi) - \sin\varphi(a\cos\varphi - b\sin\varphi) = b$. So $\bar{y} = \tilde{y}$.

7. With 4 points on a circle (QPSK) the points are already as far apart as possible for their energy; moving one inside the circle would bring it closer to others. With 16 points, the circle is crowded; moving some inside and others slightly outside (keeping average energy) increases the minimum distance, as in 16-QAM or hexagonal packings.

8. Change the loop to `(8, 8, 'B')`. With $M = 256$ the network has 135,944 parameters and the softmax has 256 outputs, so each step costs roughly $(256/16)^2 \approx 250$ times more in the $M \times M$ layers, and many more steps are needed because there are 256 classes to separate. You should see a gain over uncoded BPSK at all E_b/N_0, as in the paper.

Self-test quiz

1. What does $(n,k) = (7,4)$ mean, and what is M? *16 messages (4 bits) over 7 channel uses; $M = 16$.*
2. Which layer has no trainable parameters yet must be differentiable? *The channel (noise) layer.*
3. With the energy constraint, where do all codewords lie? *On a sphere of radius $\sqrt{n}$.*
4. What is the paper's training E_b/N_0, optimiser and learning rate? *7 dB, Adam, 0.001.*
5. Which benchmark does the $(7,4)$ autoencoder match? *Hamming (7,4) with ML decoding.*
6. Why does an RTN help the receiver? *It applies a known differentiable correction (e.g. de-rotation) with learned parameters, so the decoder sees a canonical signal.*

Further reading

- T. O'Shea and J. Hoydis, "An Introduction to Deep Learning for the Physical Layer," *IEEE Trans. Cognitive Communications and Networking*, vol. 3, no. 4, pp. 563–575, 2017 (arXiv:1702.00832). The paper of this chapter.
- S. Dörner, S. Cammerer, J. Hoydis and S. ten Brink, "Deep Learning-Based Communication over the Air," *IEEE J. Selected Topics in Signal Processing*, vol. 12, no. 1, pp. 132–143, 2018. The

first over-the-air autoencoder.

- F. Ait Aoudia and J. Hoydis, "Model-free Training of End-to-end Communication Systems," *IEEE J. Selected Areas in Communications*, 2019 (arXiv:1812.05929). Training without a channel model (Chapter 24).
- T. Gruber, S. Cammerer, J. Hoydis and S. ten Brink, "On Deep Learning-Based Channel Decoding," CISS 2017 (Chapter 23).
- N. E. West and T. J. O'Shea, "Deep Architectures for Modulation Recognition," IEEE DySPAN 2017 (Chapter 22).
- X. Lin, "An Overview of the 3GPP Study on Artificial Intelligence for 5G New Radio," arXiv:2308.05315, 2023. A readable summary of TR 38.843.

Chapter 22

Deep Architectures for Modulation Recognition

A spectrum monitor, a cognitive radio or an electronic-warfare receiver often sees a signal it knows nothing about. The first question it must answer is: *what kind of modulation is this?* BPSK or QPSK, 16-QAM or 64-QAM, FM or AM? This is *automatic modulation recognition* (also called modulation classification). It was one of the first radio problems where deep learning clearly beat decades of hand-crafted features, and it became the "MNIST of radio machine learning".

This chapter is built around the paper by Nathan West and Tim O'Shea, *Deep Architectures for Modulation Recognition* (IEEE DySPAN 2017). The authors took the convolutional network of an earlier paper and asked whether the famous deep architectures of computer vision and speech — deeper CNNs, residual networks, Inception modules, and the convolutional-LSTM network CLDNN — would do better on radio signals. Their answer was surprising: *depth hardly helped*; what helped was adding a recurrent (LSTM) layer. You will learn the problem, the classical approaches, the RadioML data set, every architecture as a block diagram, the findings, and then train your own CNN and CLDNN on a synthetic data set that runs on a CPU.

Paper at a glance

Paper. N. E. West and T. J. O'Shea, "Deep Architectures for Modulation Recognition", *IEEE International Symposium on Dynamic Spectrum Access Networks (DySPAN)*, Baltimore, March 2017. arXiv:1703.09197.
Problem. Classify the modulation of a short burst of raw complex baseband samples (128 IQ samples, 11 classes) over a wide SNR range, without hand-crafted features.
Key idea. Systematically tune and compare deep architectures from vision and speech — CNNs of different depth, filter count and filter length, residual networks, Inception networks, and a CLDNN (convolutional layers followed by an LSTM and dense layers, with a bypass connection) — on the RadioML 2016.10a data set.
Main results (as reported). With 50 filters per layer, filter lengths of 7–12 taps all gave about 61% accuracy averaged over all SNRs; adding convolutional layers gave little to no improvement; a 9-layer ResNet reached similar loss and accuracy as the tuned CNN but in fewer epochs; Inception modules gave no improvement; the CLDNN consistently outperformed the other architectures for SNRs above -8 dB. The main remaining confusions are among analog modulations and among higher-order QAMs. Conclusion: radio modulation recognition is not limited by network depth; future work should improve learned synchronisation and equalisation.

22.1 The problem and why classical methods fall short

Intuition: recognising a language without understanding it

You can often tell whether someone on the radio speaks Spanish or Japanese without understanding a word: the rhythm, the sounds, the melody give it away. Modulation recognition is the same: the receiver does not know the symbol timing, the carrier frequency, the phase or the pulse shape, and cannot demodulate. It must recognise the "accent" of the waveform — how its amplitude and phase move — from a few hundred samples buried in noise.

Classical approaches fall into two families.

Likelihood-based classification. If the receiver knew the channel exactly (noise level, phase, timing, carrier offset), it could compute, for each candidate modulation m, the likelihood $p(\mathbf{r} \mid m)$ of the received samples $\mathbf{r}$ by averaging over all possible transmitted symbol sequences, and pick the largest. This is optimal in theory. In practice the unknown parameters must be estimated or averaged over, the computation is heavy, and the method is fragile when the assumed model (e.g. perfect synchronisation) is wrong.

Feature-based classification. The receiver computes a handful of expert statistics, then applies a simple classifier (decision tree, SVM). Classic features include the variance of the instantaneous amplitude, phase and frequency, spectral peaks, and *higher-order cumulants*. For a zero-mean complex signal x with unit power, two widely used fourth-order cumulants are

$$C_{40} = \mathbb{E}[x^4] - 3\,\mathbb{E}[x^2]^2, \qquad C_{42} = \mathbb{E}[|x|^4] - \left|\mathbb{E}[x^2]\right|^2 - 2\,\mathbb{E}[|x|^2]^2. \tag{22.1}$$

Worked example: cumulants of ideal constellations

For BPSK, $x = \pm 1$: $\mathbb{E}[x^2] = 1$, $\mathbb{E}[x^4] = 1$, $\mathbb{E}[|x|^4] = 1$, so $C_{40} = 1 - 3 = -2$ and $C_{42} = 1 - 1 - 2 = -2$. For QPSK with points $e^{j(\pi/4 + k\pi/2)}$: $x^2 = \pm j$ averages to 0, $x^4 = -1$, so $C_{40} = -1$ and $C_{42} = 1 - 0 - 2 = -1$. For 8-PSK: x^4 takes the values ± 1 equally, so $C_{40} = 0$, while $C_{42} = -1$. A Python check (`code/ch22`) for unit-power square QAM gives $C_{42} = -0.68$ for 16-QAM and -0.62 for 64-QAM. These numbers tell the story of this chapter in advance: PSK orders are well separated by C_{40}, but 16-QAM and 64-QAM differ by only 0.06 — and the variance of a cumulant estimated from a short, noisy burst is much larger than that.

Features work well at high SNR and with clean synchronisation, but they were designed for idealised models. Channel effects (multipath, frequency offset, timing drift) and short observation windows hurt them, and each new modulation needs new expert work. O'Shea, Corgan and Clancy had shown in 2016 that a small CNN trained directly on raw IQ samples beat feature-based classifiers at low SNR. West and O'Shea's paper asks the natural next question: *can bigger, more modern architectures do even better?*

22.2 Background you need

This chapter uses convolutional layers (Chapter 15), LSTMs (Chapter 16), residual connections, dropout, softmax with cross-entropy, and Adam (Part II). From the wireless primer (Chapter 20) we need complex baseband, pulse shaping, and the impairments below.

IQ samples as a 2×128 image. A complex baseband burst $r[0], \ldots, r[127]$ is stored as two real rows: the in-phase part $\Re\{r[t]\}$ and the quadrature part $\Im\{r[t]\}$. A CNN can treat this as a tiny 2×128 "image" and slide filters along time; a filter of size 1×8 looks at 8 consecutive samples of one row, a filter of size 2×8 looks at both rows together.

The impairments. A real receiver sees $r[t] = e^{j(2\pi f_o t + \theta)} (h * s)[t - \tau] + w[t]$: an unknown carrier frequency offset f_o and phase θ (rotation of the constellation, which spins over time), a channel impulse response h (multipath, frequency-selective fading), a timing offset τ and a sample-rate offset (the clock drifts), and noise w. A classifier must be *invariant* to most of these.

Definition: SNR in modulation recognition

In these data sets the SNR is the ratio of the average signal power to the noise power per sample, in dB. Because the signal is oversampled (here 8 samples per symbol), a given sample SNR corresponds to a higher symbol energy-to-noise ratio E_s/N_0 (roughly by the oversampling factor). Accuracy-versus-SNR curves are therefore not directly comparable with BER curves in E_b/N_0.

22.3 The RadioML 2016.10a data set

The paper evaluates everything on **RadioML 2016.10a**, a synthetic data set generated with GNU Radio by O'Shea and West (GNU Radio Conference, 2016) and published by DeepSig. The paper describes the task as identifying the modulation of a 128-sample complex time-domain vector among 11 classes, with channel effects consisting of a power delay profile, frequency-selective fading, local-oscillator offset, and additive white Gaussian noise. From the dataset's documentation and the earlier CNN paper we can add:

- **Classes (11):** 8 digital — BPSK, QPSK, 8PSK, QAM16, QAM64, BFSK/GFSK, CPFSK, PAM4 — and 3 analog — WBFM, AM-DSB, AM-SSB. (The 2016 CNN paper lists BFSK, the dataset files label the class GFSK.)
- **SNR:** 20 values from -20 dB to $+18$ dB in 2 dB steps.
- **Size:** 1000 examples for each (modulation, SNR) pair, i.e. $11 \times 20 \times 1000 = 220{,}000$ examples, each a 2×128 array.
- **Sources:** random (whitened) bits for the digital classes; real voice audio for the analog classes, which matters for the confusions we discuss later.

Pitfall: a dataset is part of the result

RadioML 2016.10a is a *simulation* with particular choices (8 samples per symbol for the digital classes, a particular fading model, short 128-sample windows). Later work found quirks in it, for example examples whose labelled SNR does not match their measured SNR, and the silence periods in the analog audio. Accuracy numbers reported on it say how well a network fits *this* generator, not how it will perform on a real receiver. The paper does not state its train/test split, batch size or number of epochs; we therefore compare our results with the paper qualitatively only.

22.4 The architectures

22.4.1 The baseline CNN and its tuning

The paper starts from a network similar to "CNN2" of O'Shea and Corgan (2016), chosen because that work had shown a large improvement over expert features. In the 2016 paper CNN2 had two convolutional layers (256 filters of size 1×3, then 80 filters of size 2×3), a dense layer of 256 units, an 11-way softmax, ReLU activations and dropout, and its authors reported a test accuracy of roughly 87.4% (the per-SNR curves in that paper are the safer reference, because accuracy depends so strongly on SNR). West and O'Shea describe their baseline as two convolutional layers, one dense hidden layer and a softmax classifier, with ReLU and 50% dropout in the hidden layers, and then tune it:

1. **Number of filters.** With 1×3 filters, they swept 20 to 90 filters per layer in steps of 10. Accuracy was very similar over a wide window from about 30 to 70 filters, so they fixed 50 filters per layer.
2. **Filter length.** With 50 filters, they swept the number of taps from 3 to 12. Short filters (3–6 taps) were clearly worse; 7–12 taps all gave about 61% accuracy over the whole data set (all SNRs), with statistically insignificant differences. They continued with 1×8 filters.
3. **Depth.** With 50 filters of length 8, varying the number of convolutional layers showed little to no improvement in accuracy.

Intuition: why 8 taps

In RadioML the digital signals have 8 samples per symbol. A filter of 7–12 taps spans about one symbol: long enough to see one pulse and the transition to the next, like a matched filter. A 3-tap filter sees less than half a symbol and cannot "see" a pulse shape. Notice how a hyper-parameter search rediscovered a piece of radio engineering.

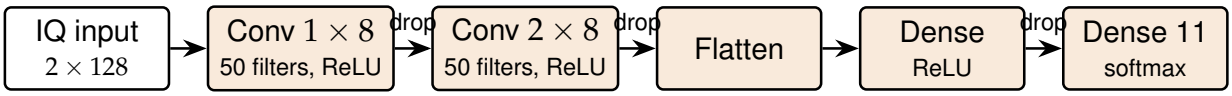


Figure 22.1: The tuned baseline CNN of West and O'Shea: two convolutional layers with 50 filters of length 8, one dense hidden layer and a softmax over the 11 classes, with 50% dropout. We draw the second layer as 2×8 (spanning I and Q) as in CNN2; the paper describes filter lengths but not every kernel shape, and does not state the dense width.

22.4.2 Residual networks

Residual networks (He et al., 2015) add *shortcut* connections: the output of a block is $\mathbf{h}_{\ell+2} = F(\mathbf{h}_\ell) + \mathbf{h}_\ell$, where F is two convolutional layers. Because the gradient can flow along the identity path, very deep networks become trainable.

The paper reports that the 9-layer residual network reaches similar training loss, validation loss and accuracy as the hyper-parameter-optimised CNN, but learns in fewer epochs. Depth via shortcuts speeds up training but does not raise the ceiling.

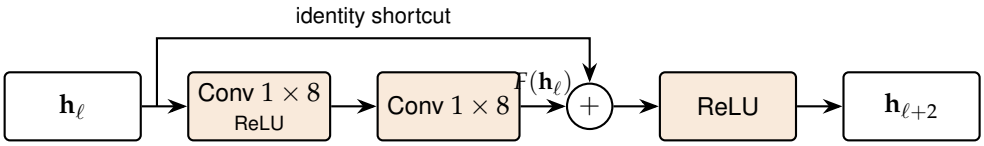


Figure 22.2: A residual block: the input skips two layers and is added to their output. The paper compares a 9-layer residual network (and 5–9 layer variants) with the tuned CNN.

22.4.3 Inception modules

An Inception module (Szegedy et al., 2014) runs several convolutions with different filter sizes *in parallel* on the same input and concatenates their outputs, so the next layer can choose the scale it needs. West and O'Shea's modules for RF data have three branches of 50 filters each: 1×1, 1×3 and 1×8, the longer ones preceded by 1×1 "bottleneck" convolutions. They tried networks with 1 to 4 such modules and report no improvement over the tuned CNN.

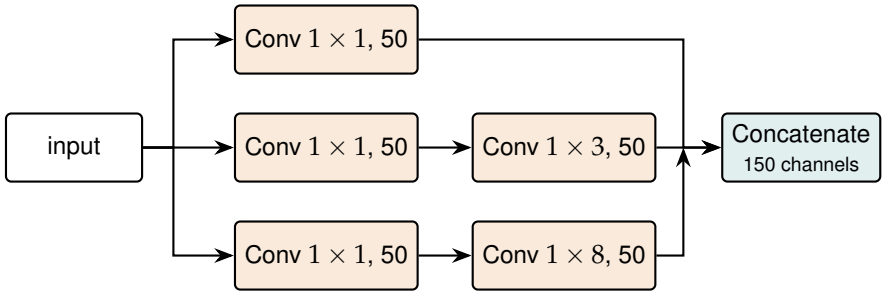


Figure 22.3: An Inception module for RF data as described by West and O'Shea: parallel branches of 50 filters with lengths 1, 3 and 8 (the longer branches preceded by 1×1 convolutions), concatenated along the channel axis.

22.4.4 The CLDNN

The CLDNN (Convolutional, Long short-term memory, fully connected Deep Neural Network) was introduced by Sainath et al. for speech recognition: convolutional layers extract local features, an LSTM models how they evolve over time, and dense layers classify. West and O'Shea's CLDNN for RF uses 1×8 convolutions with 50 filters, followed by LSTM layers and a final dense classification layer, plus one important detail: a *bypass connection*. In their Fig. 8, the output of the first 1×8 convolution is concatenated with the output of the subsequent convolution stack before it enters the LSTM, so the LSTM sees both low-level and higher-level features. They report that this forward connection gives better accuracy and more stable gradient descent. A variant with pooling (intended as a convolutional matched-filter detector) did not help.

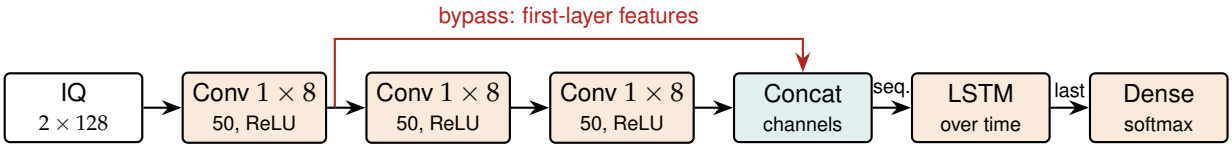


Figure 22.4: The CLDNN for RF data as described by West and O'Shea: a stack of 1×8 convolutions, a bypass that concatenates the first convolution's output with the stack's output, LSTM layers that read the feature sequence over time, and a dense softmax classifier. The paper does not give LSTM or dense sizes in the text.

Intuition: why an LSTM helps where depth does not

A CNN stacks local pattern detectors; deeper stacks see longer windows and more abstract combinations. Images are hierarchical (edges → textures → parts → objects), so depth pays. A modulated signal is not very hierarchical: it is a sequence of symbols drawn from an alphabet. Telling 16-QAM from 64-QAM is about *statistics accumulated over time* (how many distinct amplitude levels appear, how often), not about deeper shapes. An LSTM is built to accumulate evidence along a sequence, so it adds exactly the missing ingredient.

22.5 Training setup as reported

The paper trained all networks with Keras on the Theano backend, on an Nvidia GTX 1070 GPU, using the Adam optimiser (chosen to reduce the importance of the learning-rate hyper-parameter) and the categorical cross-entropy loss over the 11 classes, with dropout as regulariser. The reported metric is top-1 classification accuracy, both averaged over all SNRs ("all-SNR accuracy") and per SNR. The paper does not report learning rate, batch size, number of epochs or the train/test split. For reference, the earlier CNN2 paper reported about 96,000 training and 64,000 test/validation examples, batch size 1024, and epochs of roughly 15 seconds on a Titan X GPU.

22.6 Results reported in the paper

We summarise the findings without reproducing the paper's figures.

- **Filters per layer** (paper Fig. 3): a broad plateau from about 30 to 70 filters; differences are most visible at higher SNR.
- **Filter length** (Fig. 4): 7–12 taps all at about 61% all-SNR accuracy; 3–6 taps clearly worse.
- **Depth** (Fig. 5): 2 to 6 convolutional layers give little to no improvement. The authors interpret this as radio modulation recognition not being limited by depth.
- **ResNet** (Fig. 6): similar loss and accuracy as the tuned CNN, reached in fewer epochs.
- **Inception**: no improvement over the tuned CNN.
- **CLDNN** (Fig. 9): consistently better than all other architectures for SNRs above -8 dB.
- **Confusions** (Fig. 10, all-SNR confusion matrix of the CLDNN): two main areas, among the analog modulations and among the higher-order QAMs. The authors expect the QAM confusions to improve with better synchronisation and less channel impairment, and consider the analog confusions hard to fix.
- **Filter visualisation** (Figs. 11–12): first-layer filters look like shaped low-pass filters, some frequency-selective filters and DC blockers; inputs synthesised by gradient ascent to excite particular units resemble crude PSK and FM/FSK waveforms.

Why are the analog classes confused? The earlier CNN2 paper gives the clue: the analog sources are real voice recordings, and voice has *silences*. During a silence a WBFM signal and an AM-DSB signal both reduce to an (almost) unmodulated carrier, so a 128-sample window taken during a pause contains no information about the modulation. That paper reported WBFM being misclassified as AM-DSB at high SNR for this reason, and 8PSK being confused with QPSK

because a short sequence of 8PSK symbols can happen to lie only on the QPSK subset of points. No architecture can resolve an example that carries no information.

Wireless connection: a classifier is a detector bank

The likelihood approach is a bank of detectors, one per hypothesis. A CNN's first layer is literally a bank of FIR filters (the paper's filter visualisation shows low-pass, band-selective and DC-blocking shapes), followed by nonlinear energy-like statistics. The network learns the filter bank and the decision rule jointly from data. The LSTM then performs the role of a long integration window, like a non-coherent accumulator in an acquisition receiver.

22.7 Our small-scale reproduction

We cannot download RadioML (and its 220,000 examples would be too slow on our shared CPU anyway), so we wrote a small generator, `code/ch22/radioml_lite.py`, that imitates its format: 128 complex samples at 8 samples per symbol, stored as 2×128.

22.7.1 Our synthetic data set

- **8 digital classes:** BPSK, QPSK, 8PSK, PAM4, QAM16, QAM64 (all with root-raised-cosine pulses, roll-off 0.35, span 8 symbols), and GFSK (Gaussian frequency pulse, $BT = 0.5$) and CPFSK (rectangular frequency pulse), both with modulation index 0.5.
- **Impairments:** random carrier phase uniform in $[0, 2\pi)$, a small carrier frequency offset uniform in $\pm 5 \times 10^{-4}$ cycles per sample (up to $\pm 23^\circ$ of extra rotation across the window), a random timing offset (the window starts at a random sample within one symbol), and AWGN at a sample SNR between -10 and 18 dB in 2 dB steps (15 values). No multipath and no sample-rate offset — our channel is easier than RadioML's.
- **Size:** 200 examples per (class, SNR) for training (24,000 total) and 80 for testing (9,600).

22.7.2 The two networks

Our networks are deliberately small so that each trains in about two to three minutes of CPU time. Table 22.1 lists them layer by layer.

Worked example: the LSTM parameter count

An LSTM with input size I and hidden size H has four gates, each with an input weight matrix $H \times I$, a recurrent matrix $H \times H$ and (in PyTorch) two bias vectors of length H. Total $4(HI + H^2 + 2H)$. With $I = 64$ and $H = 48$: $4(3072 + 2304 + 96) = 4 \cdot 5472 = 21{,}888$, matching the table. Note that the CLDNN has less than a third of the CNN's parameters: the CNN spends almost 90% of its parameters on the dense layer after flattening.

```
class BaselineCNN(nn.Module):
    """Two conv layers (1x8, then 2x8 across I/Q), one dense layer, softmax.
    Shaped like the tuned
    baseline of West & O'Shea (8-tap filters), scaled down to 32 filters for our
    CPU budget."""
    def __init__(self, nf=32, dense=64, p=0.3):
        super().__init__()
```

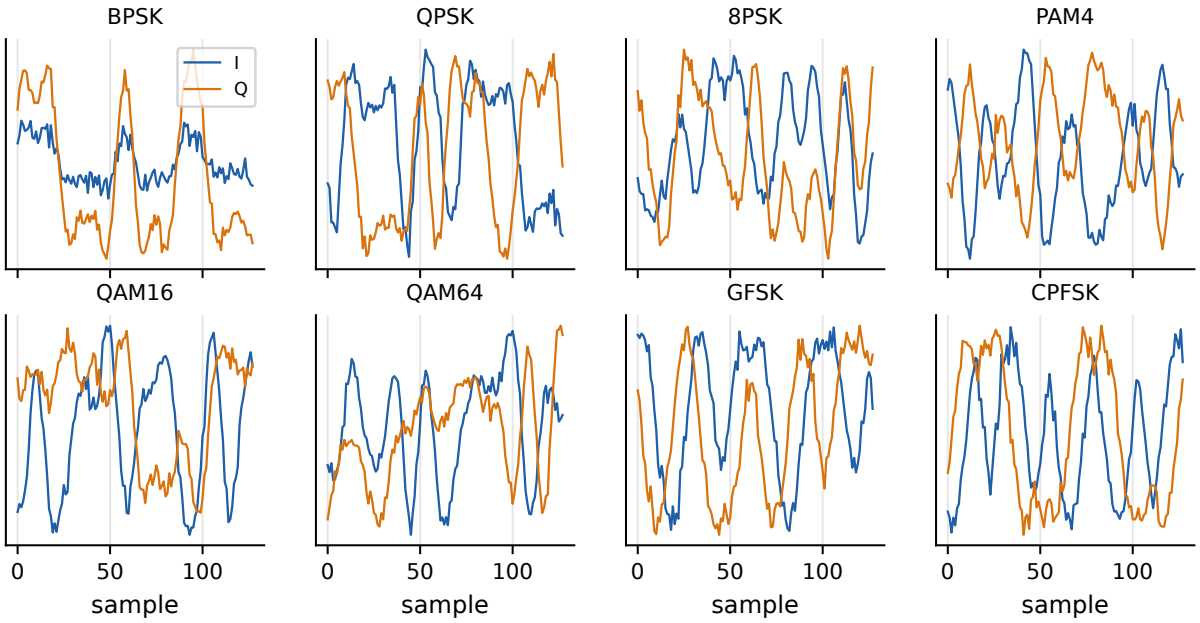


Figure 22.5: One example of each of our 8 classes at 18 dB, I (blue) and Q (orange) against sample index. The linear modulations look like band-limited noise; GFSK and CPFSK are constant-envelope sinusoids whose frequency switches every 8 samples. This is what the network sees.

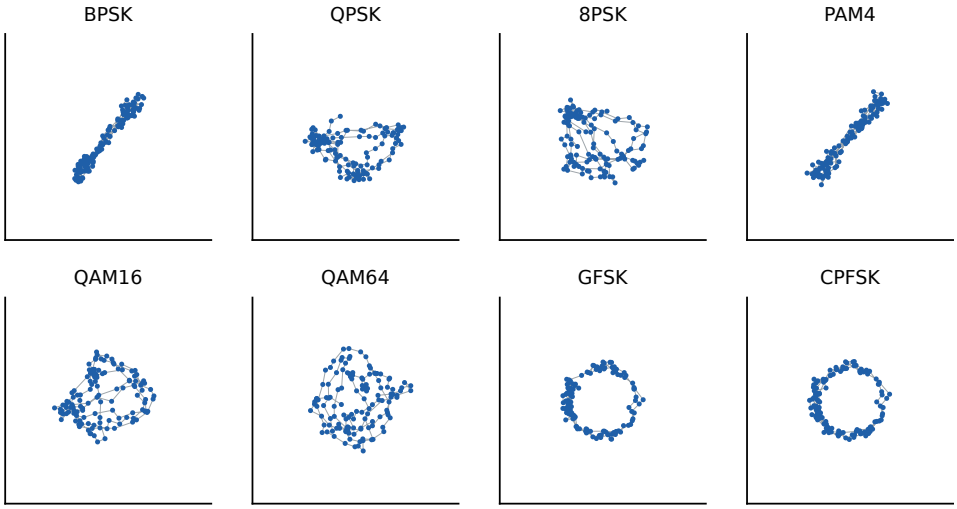


Figure 22.6: The same kind of examples plotted in the IQ plane (dots are samples, gray lines connect consecutive samples). BPSK and PAM4 lie on a rotated line; QPSK and 8PSK form rings with structure; QAM16 and QAM64 look like similar fuzzy clouds after pulse shaping and random rotation; the FSK classes trace a circle. Telling QAM16 from QAM64 from 16 symbols is hard even by eye.

Table 22.1: Our two networks (PyTorch, `code/ch22/train_amc.py`). Shapes are channels × height × time for the CNN and channels × time for the CLDNN, for one example.

Layer	Output shape	Params
Baseline CNN		
Input	$1 \times 2 \times 128$	0
Conv 1×8, 32 filters, pad 4, ReLU, dropout 0.3	$32 \times 2 \times 129$	288
Conv 2×8, 32 filters, ReLU	$32 \times 1 \times 122$	16,416
Max-pool 1×2, dropout, flatten	1952	0
Dense 64, ReLU, dropout	64	124,992
Dense 8 (softmax in the loss)	8	520
Total		**142,216**
CLDNN		
Input	2×128	0
Conv1 length 7, 32 filters, ReLU	32×128	480
Conv2, Conv3 length 7, 32 filters, ReLU	32×128	$2 \times 7{,}200$
Concat(Conv1, Conv3) (bypass)	64×128	0
Max-pool 4 (sequence for the LSTM)	64×32	0
LSTM, 48 hidden units, last output	48	21,888
Dropout, Dense 64 ReLU, Dense 8	8	3,656
Total		**40,424**

```
        self.f = nn.Sequential(
            nn.Conv2d(1, nf, (1, 8), padding=(0, 4)), nn.ReLU(), nn.Dropout(p),  # (nf, 2, 129)
            nn.Conv2d(nf, nf, (2, 8)), nn.ReLU(), nn.MaxPool2d((1, 2)),          # (nf, 1, 61)
            nn.Dropout(p), nn.Flatten())
        self.c = nn.Sequential(nn.Linear(nf * 61, dense), nn.ReLU(), nn.Dropout(p), nn.Linear(dense, NC))
    def forward(self, x):                          # x: (B, 2, 128)
        return self.c(self.f(x.unsqueeze(1)))

class CLDNN(nn.Module):
    """Conv -> 2 more conv (7 taps) ; concat(first conv output, third conv output) -> LSTM -> dense.
    The bypass follows West & O'Shea's description; filter, LSTM and dense sizes are our choice."""
    def __init__(self, nf=32, hid=48, p=0.3):
        super().__init__()
        self.c1 = nn.Sequential(nn.Conv1d(2, nf, 7, padding=3), nn.ReLU())
        self.c23 = nn.Sequential(nn.Conv1d(nf, nf, 7, padding=3), nn.ReLU(),
                                 nn.Conv1d(nf, nf, 7, padding=3), nn.ReLU())
        self.pool = nn.MaxPool1d(4)                # 128 -> 32 time steps for the LSTM (speed)
        self.lstm = nn.LSTM(2 * nf, hid, batch_first=True)
        self.head = nn.Sequential(nn.Dropout(p), nn.Linear(hid, 64), nn.ReLU(), nn.Linear(64, NC))
    def forward(self, x):
        a = self.c1(x)
        b = self.c23(a)
        h = self.pool(torch.cat([a, b], 1)).transpose(1, 2)    # (B, T=32, 2nf)
        out, _ = self.lstm(h)
        return self.head(out[:, -1])
```

Both networks are trained with Adam (learning rate 2×10^{-3}, cosine decay), batch size 256, 10 epochs, cross-entropy loss, on all SNRs mixed together, exactly as one trains on RadioML.

22.7.3 Results

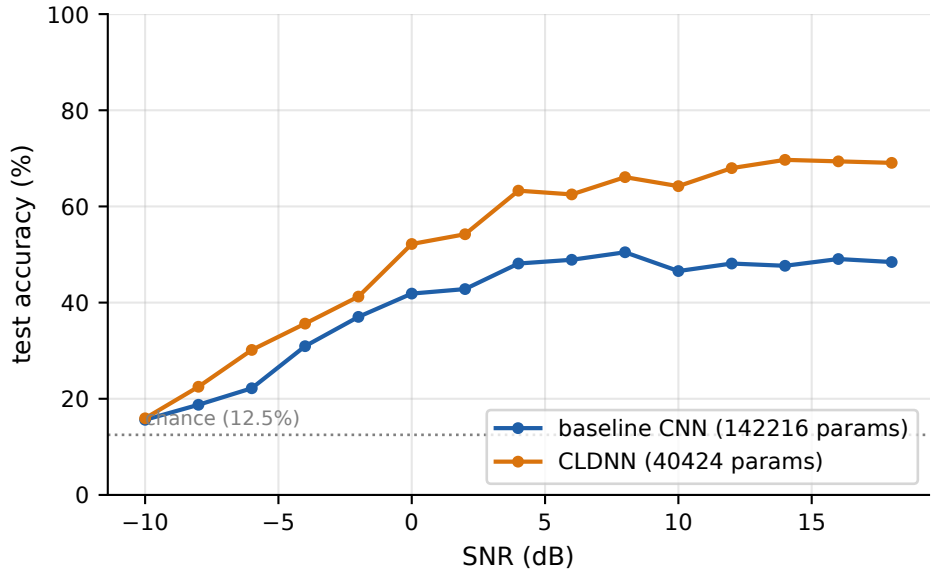


Figure 22.7: Our reproduction: test accuracy versus SNR on the synthetic 8-class data (80 test examples per class and SNR). The CLDNN (orange) is better than the baseline CNN (blue) at every SNR above −10 dB, and the gap grows with SNR (about 69% versus 48% at high SNR), although the CLDNN has less than a third of the parameters. Both curves saturate well below 100%: some classes stay confused even without noise.

Accuracy versus SNR. Figure 22.7 is our analogue of the paper's Fig. 9. Averaged over all SNRs (−10 to 18 dB), the CNN reached 39.8% and the CLDNN 52.3%; averaged over SNRs of 10 dB and above, 48.0% and 68.1%. The ranking is the one the paper reports: the CLDNN wins over the whole range except the lowest SNR, where both are near chance (12.5%). The *size* of our gap is much larger than in the paper, and we do not claim it is representative: our CNN is small (32 filters), trained for only 10 epochs, and Fig. 22.8 shows its training had not converged, while the CLDNN learned faster. Its training loss (1.36 after 10 epochs versus 1.10 for the CLDNN) shows that the CNN is *under*fitting, not overfitting — it lacks the right inductive bias, which is the paper's message in a stronger form.

Confusions. Figure 22.9 shows where the errors come from. At 18 dB, 49% of QAM64 examples are labelled QAM16 and 20% of QAM16 examples are labelled QAM64; 30% of 8PSK examples are labelled QPSK and a third go to QAM16. This is the higher-order-QAM confusion the paper reports, and the 8PSK-versus-QPSK confusion of the CNN2 paper, appearing in our synthetic data for the same reasons: 16 symbols with random rotation, timing and a slowly spinning carrier are simply not enough to count amplitude rings or resolve 8 phases, as the cumulant example earlier predicted. GFSK versus CPFSK (38% of GFSK labelled CPFSK) is our analogue of the analog-class confusion: the two differ only by the smoothing of the frequency pulse, a subtle feature at this window length. The paper's other main confusion (WBFM versus AM-DSB) cannot appear because we generated no analog classes.

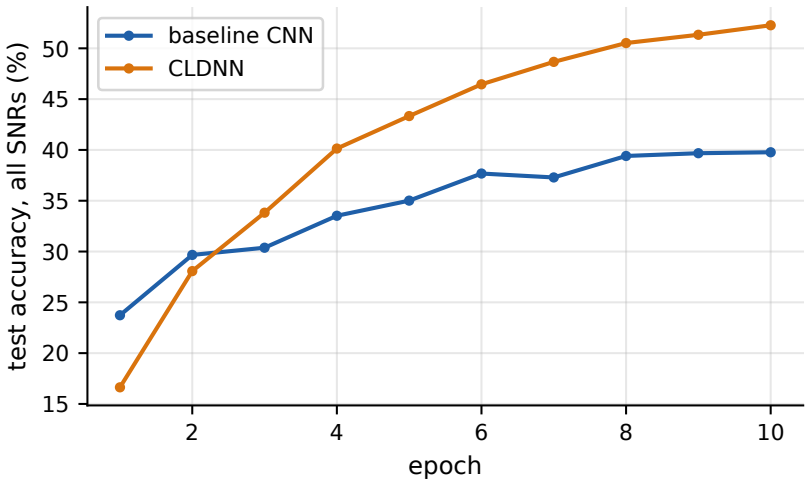


Figure 22.8: Test accuracy (all SNRs) after each training epoch. The CLDNN learns faster and keeps improving; the CNN flattens out near 40%.

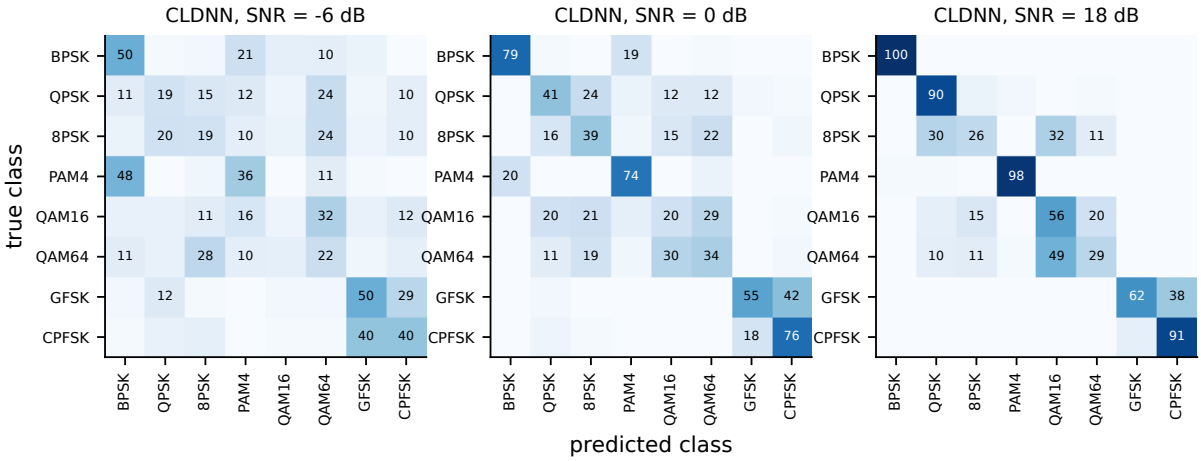


Figure 22.9: Confusion matrices of our CLDNN at -6, 0 and 18 dB (rows: true class, columns: predicted class, numbers in %, entries below 10% not printed). At 18 dB, BPSK, PAM4, QPSK and CPFSK are almost perfect, while QAM16/QAM64 confuse each other heavily, 8PSK leaks into QPSK and the QAMs, and GFSK is often called CPFSK. At low SNR everything blurs into a few groups (real-valued BPSK/PAM4, the PSK/QAM cloud, the FSK pair).

What we can and cannot conclude. Our experiment supports the qualitative claims: (i) a CNN-only classifier on short IQ windows saturates well below perfect accuracy, (ii) adding an LSTM that integrates features over time helps, and (iii) the remaining errors are concentrated in classes that differ only in fine amplitude/phase statistics. It does *not* test the depth, ResNet and Inception findings (we trained only two models), and our absolute accuracies are not comparable with RadioML numbers because the data, number of classes and channel differ.

22.8 Limitations and follow-up work

- **Synthetic data.** All conclusions are on one synthetic data set with short windows. The authors themselves note that more complex radio environments — varying temporal behaviour, multi-modulation protocols — are needed. O'Shea, Roy and Clancy later released the larger RadioML 2018.01A (24 classes, 1024-sample frames, over-the-air and synthetic) and reported that deep residual networks *did* help on longer frames.
- **Fixed sample rate and bandwidth.** Every example has the same oversampling; the authors suggest learning bandwidth normalisation or multi-bandwidth features.
- **Synchronisation and equalisation.** The paper's main recommendation: architectures that *learn to undo* the channel — for example radio transformer networks (Chapter 21) — rather than simply deeper feature extractors.
- **Closed set.** The classifier must pick one of 11 classes even for a signal it has never seen; open-set recognition, out-of-distribution detection and calibrated confidence are later research topics.
- **Hyper-parameter-heavy conclusions.** Statements like "depth does not help" hold for the tuned settings, data and window length studied; with other data (longer frames, more classes) later work found that depth can help.

Follow-up work has since explored complex-valued networks, transformers on IQ sequences, signal-to-image transforms (constellation diagrams, spectrograms), and semi-supervised learning to reduce labelled data. Part VI of this book returns to modulation classification as a set of hands-on labs with cumulant features and deep classifiers under AWGN, Rayleigh multipath and other channels.

22.9 From paper to practice in 5G NR and 6G

Wireless connection: who needs blind modulation recognition?

Inside a 5G NR link, the UE never needs to guess the modulation: the gNB signals the modulation and coding scheme (MCS index) in downlink control information (DCI), and the MCS tables of TS 38.214 map it to QPSK, 16-, 64-, 256- or (from Release 17) 1024-QAM. Blind modulation recognition matters *outside* the cooperative link: spectrum monitoring and regulation, dynamic spectrum sharing and coexistence (e.g. detecting incumbents in shared bands), interference hunting, signals intelligence, and detection of rogue or fake base stations, where a passive sensor must characterise signals it was not invited to decode.

Practical lessons for a PHY engineer:

- **Window length matters more than depth.** 128 samples at 8 samples/symbol is only 16 symbols. Distinguishing 16-QAM from 64-QAM reliably needs hundreds of symbols; a sensor that can integrate longer (or combine several windows, which is what the LSTM does internally) wins.
- **Do the cheap synchronisation first.** Coarse CFO estimation (e.g. from the M-th power spectrum), symbol-rate estimation and resampling to a fixed samples-per-symbol make the network's task much easier, as the paper's conclusion suggests.
- **For NR waveforms, classify structure, not constellations.** An NR downlink is OFDM with a known numerology; identifying SSB/PSS/SSS, subcarrier spacing and bandwidth is usually more useful than per-RE constellation classification, which varies per PDSCH allocation.
- **Embedded deployment.** Our CLDNN has 40k parameters; a CNN like CNN2 a few hundred thousand. Both fit in fixed-point on a DSP or FPGA, but the LSTM's sequential recurrence limits parallelism; a 1-D CNN with a global pooling layer is often the practical compromise.
- **6G sensing.** 6G discussions include integrated sensing and communication and AI-native spectrum management, where automatic recognition of waveforms and emitters is a building block.

Chapter recap

- Modulation recognition: classify a short burst of raw IQ samples into one of several modulations, under unknown phase, frequency, timing, channel and noise.
- Classical: likelihood methods (optimal but model-fragile and heavy) and feature methods (cumulants etc. plus a simple classifier).
- RadioML 2016.10a: 11 classes (8 digital, 3 analog), 128 IQ samples, SNR -20 to 18 dB, 220,000 examples generated with GNU Radio channel models.
- West and O'Shea: tuned CNN (50 filters, 7–12 taps, about 61% all-SNR accuracy); more conv layers, ResNet and Inception do not raise accuracy (ResNet trains in fewer epochs); CLDNN with a bypass is best above -8 dB.
- Remaining errors: higher-order QAM (needs better synchronisation/equalisation, longer windows) and analog classes (silence carries no information).
- Our synthetic reproduction reproduces the qualitative ranking (CLDNN above CNN) and the QAM16/QAM64 confusion.

Exercises

1. Compute C_{40} and C_{42} for unit-power 4-PAM (levels $\pm 1, \pm 3$ scaled to unit power).
2. A CNN layer has 50 filters of size 1×8 on a one-channel 2×128 input with no padding. What is its output shape and how many parameters does it have? Repeat for 50 filters of size 2×8 applied to that output.
3. Show that rotating a constellation by a fixed phase θ leaves C_{42} unchanged but multiplies C_{40} by $e^{j4\theta}$. What does this imply for using $|C_{40}|$ as a feature?
4. Why does an 8-tap filter make sense for RadioML's digital classes? What filter length

would you choose if the data had 4 samples per symbol?

5. The paper reports that a ResNet reaches the same accuracy as the CNN in fewer epochs. Explain why shortcut connections speed up training even when they do not improve the final accuracy.
6. Give two reasons why WBFM and AM-DSB examples can be impossible to distinguish in a 128-sample window.
7. In our CLDNN, what is the effect of removing the bypass (feeding only Conv3 to the LSTM)? Predict, then try it in `train_amc.py`.

Answers

1. Unit power: levels $\pm 1, \pm 3$ have mean square 5, so scale by $1/\sqrt{5}$. Real signal, so $\mathbb{E}[x^2] = \mathbb{E}[|x|^2] = 1$ and $\mathbb{E}[x^4] = \mathbb{E}[|x|^4] = (1 + 81)/2/25 = 1.64$. $C_{40} = 1.64 - 3 = -1.36$; $C_{42} = 1.64 - 1 - 2 = -1.36$ (matches our Python check).
2. First layer: height 2 (a 1×8 kernel keeps both rows), width $128 - 8 + 1 = 121$: output $50 \times 2 \times 121$; parameters $50 \cdot 8 + 50 = 450$. Second layer: kernel covers 50 channels $\times 2 \times 8$, output $50 \times 1 \times 114$; parameters $50 \cdot (50 \cdot 2 \cdot 8) + 50 = 40{,}050$.
3. With $x' = e^{j\theta}x$: $|x'| = |x|$ and $\left|\mathbb{E}[x'^2]\right| = \left|\mathbb{E}[x^2]\right|$, so C_{42} is unchanged. $\mathbb{E}[x'^4] = e^{j4\theta}\mathbb{E}[x^4]$ and $\mathbb{E}[x'^2]^2 = e^{j4\theta}\mathbb{E}[x^2]^2$, so $C'_{40} = e^{j4\theta}C_{40}$. With an unknown carrier phase, use $|C_{40}|$, which is rotation-invariant.
4. One symbol spans 8 samples, so an 8-tap filter covers one pulse (like a matched filter) and one symbol transition. With 4 samples per symbol, about 4–6 taps would play the same role (and longer filters would span several symbols).
5. The identity path gives every layer a direct gradient signal and makes each block start close to the identity, so early training is not slowed by vanishing or badly conditioned gradients through many layers. If the extra depth adds no useful features (the task is not hierarchical), the final accuracy stays the same while convergence is faster.
6. (i) During speech pauses both are an almost pure carrier, so the window contains no modulation. (ii) With low audio frequencies, 128 samples cover only a tiny fraction of an audio cycle, so the slow amplitude (AM) or frequency (FM) variation is barely visible and both look like a slowly varying tone.
7. Expect slightly lower accuracy and slower, noisier training, as West and O'Shea report that the bypass improves accuracy and stabilises gradient descent: without it, the LSTM sees only features after three nonlinear layers and loses the fine-grained first-layer information. (Results on our small data may be within noise; repeat with several seeds before concluding.)

Self-test quiz

1. How many classes, samples per example and SNR values does RadioML 2016.10a have? *11 classes, 128 complex samples, 20 SNRs (−20 to 18 dB).*
2. What filter length range gave the best (about equal) accuracy for the tuned CNN? *7–12 taps (about 61% all-SNR).*

3. Did adding convolutional layers help? *Little to no improvement.*
4. What advantage did the ResNet show? *Same accuracy, fewer training epochs.*
5. Which architecture was best, and above which SNR? *The CLDNN, above -8 dB.*
6. Name the two main confusion groups. *Analog modulations (e.g. WBFM/AM-DSB) and higher-order QAMs (QAM16/QAM64).*

Further reading

- N. E. West and T. J. O'Shea, "Deep Architectures for Modulation Recognition," *IEEE DySPAN*, 2017 (arXiv:1703.09197). The paper of this chapter.
- T. J. O'Shea, J. Corgan and T. C. Clancy, "Convolutional Radio Modulation Recognition Networks," arXiv:1602.04105, 2016 (Engineering Applications of Neural Networks, 2016). The CNN2 baseline.
- T. O'Shea and N. West, "Radio Machine Learning Dataset Generation with GNU Radio," *Proc. GNU Radio Conference*, vol. 1, no. 1, 2016. How RadioML 2016 was generated.
- T. N. Sainath, O. Vinyals, A. Senior and H. Sak, "Convolutional, Long Short-Term Memory, Fully Connected Deep Neural Networks," *ICASSP*, pp. 4580–4584, 2015. The original CLDNN.
- T. J. O'Shea, T. Roy and T. C. Clancy, "Over-the-Air Deep Learning Based Radio Signal Classification," *IEEE J. Selected Topics in Signal Processing*, vol. 12, no. 1, 2018. RadioML 2018 and deep ResNets on longer frames.
- T. O'Shea and J. Hoydis, "An Introduction to Deep Learning for the Physical Layer," 2017 (Chapter 21), Section III-D.

Chapter 23

On Deep Learning-Based Channel Decoding

Every modem ends with a channel decoder: the block that takes noisy soft values from the demodulator and recovers the information bits by exploiting the redundancy added by the channel code. Decoders for LDPC, turbo and polar codes are among the most carefully engineered algorithms in communications. Could a neural network simply *learn* to decode, by being shown noisy codewords and the bits that produced them? And if it can, has it learned a decoding *algorithm*, or merely memorised a table?

This chapter studies the paper by Tobias Gruber, Sebastian Cammerer, Jakob Hoydis and Stephan ten Brink, *On Deep Learning-Based Channel Decoding* (CISS 2017). With a small fully connected network, they reached maximum a-posteriori (MAP) bit error rate for short codes of length 16 — and discovered something deeper: a network trained on a *structured* code (a polar code) can decode codewords it never saw during training, while a network trained on a *random* code cannot. You will learn why decoding is a classification problem with 2^k classes, how to build the polar and random codes and the exhaustive MAP decoder, the network and its training, the normalised validation error, and you will reproduce both the MAP comparison and the generalisation experiment on a CPU.

Paper at a glance

Paper. T. Gruber, S. Cammerer, J. Hoydis and S. ten Brink, "On Deep Learning-Based Channel Decoding", *Proc. 51st Annual Conference on Information Sciences and Systems (CISS)*, Baltimore, March 2017. arXiv:1701.07738.

Problem. Can a deep neural network perform one-shot decoding of short block codes (random and polar) and reach MAP performance, and does it learn a generalisable decoding rule?

Key idea. Train a fully connected network (e.g. three hidden layers 128-64-32, ReLU, sigmoid outputs) on noisy BPSK codewords generated on the fly by non-trainable modulation and noise layers, and measure its BER relative to MAP over a range of SNRs with a new metric, the normalised validation error (NVE).

Main results (as reported). For $N = 16$, $k = 8$ codes, MAP BER is achievable for both polar and random codes; structured (polar) codes are easier to learn; the network generalises to codewords never seen in training for polar codes but not for random codes; the required training grows exponentially with k (the number of classes is 2^k), which limits the approach to short codes.

23.1 The problem and why it is hard

Intuition: a dictionary or a grammar?

Imagine learning a language with 256 words. You can memorise all 256 (a dictionary), or you can learn the rules that generate them (a grammar). With 256 words both work. With 2^{50} words only the grammar can possibly work. A neural decoder faces exactly this choice. A random code is a list of arbitrary words: there is no grammar to learn. A polar code is generated by a simple recursive rule: a learner that discovers (part of) that rule can handle words it has never seen.

23.1.1 Decoding is classification over 2^k classes

A binary block code maps k information bits $\mathbf{u} \in \{0,1\}^k$ to N code bits $\mathbf{x} \in \{0,1\}^N$, with rate $r = k/N$. There are exactly 2^k codewords. After transmission the receiver observes a noisy vector $\mathbf{y} \in \mathbb{R}^N$ and must decide which of the 2^k codewords was sent: *decoding is a classification problem with 2^k classes*. A neural network can do classification, so in principle it can decode.

The catch is the **curse of dimensionality**. To learn a classifier you must see enough examples of every class. With $k = 8$ there are 256 classes — easy. A modest code of length $N = 100$ and rate $1/2$ has $2^{50} \approx 10^{15}$ codewords, far too many to ever show to a network, let alone with many noise realisations each. A neural decoder can only work for practical codes if it learns a rule that *generalises* from the codewords it has seen to those it has not. Whether that happens is the central question of the paper.

23.1.2 What classical decoders do

The optimal decoders are easy to write and impossible to compute for long codes:

- **Block-MAP (= ML for equiprobable messages):** choose the codeword with the largest likelihood; for BPSK in AWGN, the one with the largest correlation $\langle \mathbf{y}, \mathbf{s} \rangle$ with its BPSK image $\mathbf{s}$. It minimises the block error rate.
- **Bit-MAP:** for each information bit, choose the value with the larger posterior probability, summing over all codewords. It minimises the bit error rate. The paper's benchmark is MAP BER.

Both need a sum or a maximum over 2^k codewords. Practical decoders (belief propagation for LDPC, successive cancellation and list decoding for polar codes) exploit code structure to run in roughly $O(N \log N)$ or $O(N)$ operations per iteration, at some loss compared with MAP. Before 2017, attempts to train neural networks as decoders (since the 1990s) had been limited by the hardware and training methods of the time; the paper revisits the idea with modern deep learning tools.

23.2 Background you need

From Chapter 20: BPSK, AWGN, E_b/N_0, log-likelihood ratios, linear block codes and generator matrices. From Part II: dense layers, ReLU, sigmoid, binary cross-entropy, Adam.

Definition: polar code

Let $\mathbf{F} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$ and $\mathbf{G}_N = \mathbf{F}^{\otimes n}$, the n-fold Kronecker power, with $N = 2^n$. A polar code of length N and dimension k places the k information bits on a chosen set $\mathcal{A}$ of k positions of a length-N vector $\mathbf{v}$ (the *information set*), sets the other $N - k$ positions (the *frozen* bits) to 0, and transmits $\mathbf{x} = \mathbf{v}\,\mathbf{G}_N$ (modulo 2). The information set is chosen as the k most reliable "synthetic channels" created by the recursive transform.

The Kronecker power is built recursively: $\mathbf{G}_2 = \mathbf{F}$, and $\mathbf{G}_{2N} = \begin{bmatrix} \mathbf{G}_N & \mathbf{0} \\ \mathbf{G}_N & \mathbf{G}_N \end{bmatrix}$. The rows of $\mathbf{G}_{16}$ have weights 1, 2, 4, 8 or 16 (a row's weight is 2^w, where w is the number of ones in the binary expansion of its index, counting from 0). Freezing the low-weight rows is what gives the code a good minimum distance.

Polarisation in one step. Combine two copies of a binary erasure-like channel with "badness" (Bhattacharyya parameter) z. One step of the transform $[v_1, v_2] \mapsto [v_1 \oplus v_2, v_2]$ creates a worse channel for v_1 (it must be decoded seeing both outputs but not knowing v_2) and a better one for v_2 (decoded knowing v_1). The badness evolves as

$$z^- = 2z - z^2 \quad \text{(worse)}, \qquad z^+ = z^2 \quad \text{(better)}. \tag{23.1}$$

For the erasure channel these are exact; for AWGN they are standard bounds and a common construction heuristic. Starting from the BPSK/AWGN value $z_0 = e^{-E_s/N_0} = e^{-r\,E_b/N_0}$ and applying the recursion n times gives a badness for each of the N positions; we freeze the $N - k$ worst.

Worked example: the frozen set of our (16,8) polar code

At a design $E_b/N_0 = 1$ dB, $E_s/N_0 = 0.5 \cdot 10^{0.1} = 0.629$ and $z_0 = e^{-0.629} = 0.533$. Four steps of Eq. (23.1) give, for positions $0, \ldots, 15$, the values

i	0	1	2	3	4	5	6	7
z_i	1.000	0.995	0.991	0.823	0.977	0.721	0.608	0.140
i	8	9	10	11	12	13	14	15
z_i	0.931	0.543	0.419	0.056	0.286	0.024	0.013	0.000

(rounded; the scheme is implemented in `polar_info_set` below). The 8 smallest are at positions $\mathcal{A} = \{7, 9, 10, 11, 12, 13, 14, 15\}$; the frozen set is $\{0, 1, 2, 3, 4, 5, 6, 8\}$. The rows of $\mathbf{G}_{16}$ in $\mathcal{A}$ have weights $8, 4, 4, 8, 4, 8, 8, 16$, so the minimum distance is 4. (5G NR instead reads the information set off the fixed reliability sequence of TS 38.212, Table 5.3.1.2-1; restricted to indices below 16, our reading of that table selects $\{6, 7, 10, 11, 12, 13, 14, 15\}$ — position 6 instead of 9, both rows of weight 4 — but check the table yourself. The paper does not state its frozen set; any such choice gives $d_{\min} = 4$ here.)

Worked example: encoding one message

Take $\mathbf{u} = (1, 0, 1, 1, 0, 0, 1, 0)$. Place it on $\mathcal{A}$: $\mathbf{v}$ has ones at positions 7, 10, 11, 14 and zeros elsewhere. Then $\mathbf{x} = \mathbf{v}\mathbf{G}_{16}$ is the modulo-2 sum of rows 7, 10, 11 and 14 of $\mathbf{G}_{16}$, which gives $\mathbf{x} = (0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0)$ (checked in Python). BPSK maps $0 \to +1$, $1 \to -1$.

Definition: random code, as in the paper

A random (N, k) code is a set of 2^k codewords picked at random from $\{0,1\}^N$, subject to a minimum Hamming distance between codewords (the paper picks random codewords "with a Hamming distance larger than two"). The mapping from messages to codewords is an arbitrary table. Such a code is generally *not linear*: the sum of two codewords is usually not a codeword, so there is no generator matrix and no algebraic structure to exploit.

23.3 System model

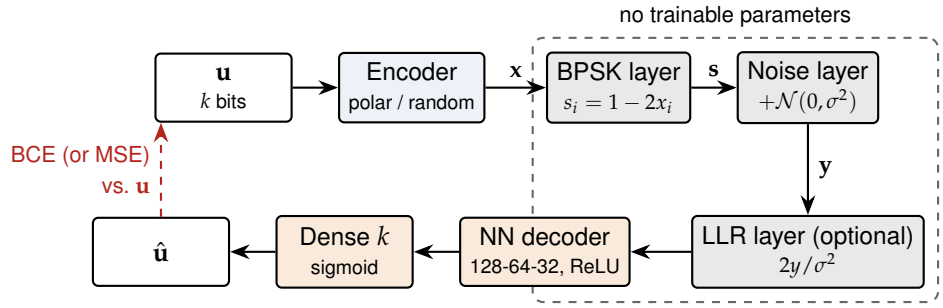


Figure 23.1: The deep-learning decoding setup of Gruber et al. (after their Fig. 1). The encoder is fixed; BPSK modulation, noise and (optionally) the LLR computation are implemented as extra network layers without trainable parameters, so every training pass sees fresh noise. Only the decoder is trained.

Channel. The code bits are BPSK-modulated, $s_i = 1 - 2x_i \in \{+1, -1\}$, and sent over an AWGN channel:

$$y_i = s_i + w_i, \qquad w_i \sim \mathcal{N}(0, \sigma^2), \qquad i = 1, \dots, N. \tag{23.2}$$

Each code bit has energy $E_s = 1$ and each information bit $E_b = E_s / r = N/k$, so

$$\sigma^2 = \frac{N_0}{2} = \frac{1}{2\, r\, E_b / N_0}. \tag{23.3}$$

Decoder input. The network can be fed either the raw channel values $\mathbf{y}$ or the log-likelihood ratios

$$\mathrm{LLR}(y_i) = \ln \frac{P(x_i = 0 \mid y_i)}{P(x_i = 1 \mid y_i)} = \frac{2}{\sigma^2}\, y_i, \tag{23.4}$$

which requires knowing σ^2. Since the LLR is just a scaled version of y_i, the information is the same; only the scale differs.

Worked example: LLRs at the polar training point

At $E_b/N_0 = 1$ dB and $r = 1/2$: $\sigma^2 = 1/(2 \cdot 0.5 \cdot 1.259) = 0.794$, $\sigma = 0.891$. Received values $y = (0.8, -1.1, 0.3)$ give LLRs $2y/\sigma^2 = (2.01, -2.77, 0.76)$: the first two bits are fairly reliable (a 0 and a 1), the third is uncertain.

Output. The last layer has k sigmoid units; output j is the estimated probability that $u_j = 1$, and $\hat{u}_j = 1$ if it exceeds 0.5. Note the design choice: *not* a softmax over 2^k classes (as in Chapter 21), but k independent bit estimates. The output layer stays small even though the problem is a 2^k-way classification.

The MAP benchmark. For bit j, the bit-MAP decision compares

$$P(u_j = b \mid \mathbf{y}) \propto \sum_{\mathbf{u}:\, u_j = b} \exp\left(\frac{\langle \mathbf{y}, \mathbf{s}(\mathbf{u})\rangle}{\sigma^2}\right), \qquad b \in \{0,1\}, \tag{23.5}$$

where $\mathbf{s}(\mathbf{u})$ is the BPSK image of the codeword of $\mathbf{u}$. This follows from Bayes' rule with equiprobable messages: $p(\mathbf{y} \mid \mathbf{s}) \propto \exp(-\|\mathbf{y}-\mathbf{s}\|^2/2\sigma^2)$, and $\|\mathbf{y}-\mathbf{s}\|^2 = \|\mathbf{y}\|^2 - 2\langle \mathbf{y}, \mathbf{s}\rangle + N$, where only the middle term depends on the codeword. For $k = 8$ this is a sum over 256 codewords — trivial on a computer, and the reason $N = 16$, $k = 8$ is a perfect laboratory.

Worked example: bit-MAP decoding of a tiny polar code by hand

Take the $(4,2)$ polar code with information set $\{2,3\}$ of $\mathbf{G}_4 = \mathbf{F}^{\otimes 2}$. Its four codewords for $\mathbf{u} = (u_1,u_2) = (0,0),(0,1),(1,0),(1,1)$ are 0000, 1111, 1010, 0101, with BPSK images $(+1,+1,+1,+1)$, $(-1,-1,-1,-1)$, $(-1,+1,-1,+1)$, $(+1,-1,+1,-1)$. Receive $\mathbf{y} = (0.9,-0.2,0.4,-1.1)$ with $\sigma^2 = 0.5$. The correlations $\langle \mathbf{y}, \mathbf{s}\rangle$ are 0, 0, -2.6, $+2.6$, so the metrics $\langle \mathbf{y}, \mathbf{s}\rangle/\sigma^2$ are $0, 0, -5.2, 5.2$. For u_1: codewords with $u_1 = 1$ are the last two, $\ln(e^{-5.2} + e^{5.2}) = 5.200$; with $u_1 = 0$, $\ln(e^0 + e^0) = 0.693$. Hence $P(u_1 = 1 \mid \mathbf{y}) = 1/(1 + e^{0.693 - 5.200}) = 0.989$. For u_2: codewords 2 and 4 give $\ln(e^0 + e^{5.2}) = 5.206$, codewords 1 and 3 give $\ln(e^0 + e^{-5.2}) = 0.006$, so $P(u_2 = 1 \mid \mathbf{y}) = 0.995$. The MAP decision is $\hat{\mathbf{u}} = (1,1)$, i.e. codeword 0101 — the one with the largest correlation, as block-ML would also choose. These are exactly the posterior probabilities a perfectly trained sigmoid output would produce.

Intuition: what a practical polar decoder does instead

Successive-cancellation (SC) decoding walks through $v_0, v_1, \ldots, v_{N-1}$ in order. Frozen positions are known to be 0; for an information position it computes an LLR using the channel values *and all previous decisions*, through the same butterfly recursion that built $\mathbf{G}_N$ ("f" and "g" operations on pairs of LLRs). Its cost is $O(N \log N)$ instead of 2^k. The price: an early wrong decision cannot be undone, which is why NR receivers use *list* SC decoding with a CRC. A neural network that learns to decode a polar code must, implicitly, learn something like these pairwise combinations.

23.4 The neural decoder and its training

23.4.1 Architecture

The paper denotes a decoder with three hidden layers of 128, 64 and 32 nodes as "128-64-32". Each hidden layer uses ReLU and the output layer uses sigmoid. Larger networks (256-128-64, 512-256-128, and 1024-512-256 in the scaling experiment) are also studied.

Table 23.1: The 128-64-32 decoder for $N = 16$, $k = 8$, with parameter counts (weights + biases) computed by us.

Layer	Input	Output	Parameters
Dense, ReLU	16	128	$16 \cdot 128 + 128 = 2176$
Dense, ReLU	128	64	$128 \cdot 64 + 64 = 8256$
Dense, ReLU	64	32	$64 \cdot 32 + 32 = 2080$
Dense, sigmoid	32	8	$32 \cdot 8 + 8 = 264$
Total			**12,776**

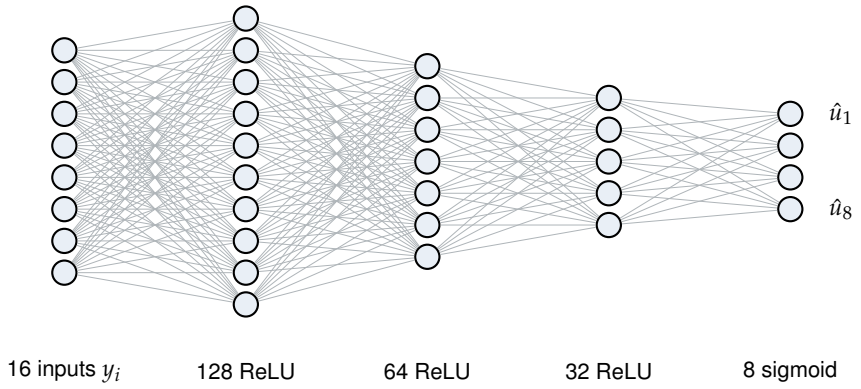


Figure 23.2: The 128-64-32 fully connected decoder (drawn with fewer nodes per layer). It maps 16 channel values to 8 bit probabilities in one shot — no iterations, no knowledge of the code except what it learns from examples.

23.4.2 Training data, loss and optimiser

The training set is the set of all 2^k codewords (or, in the generalisation experiment, a subset). An *epoch* is one pass over the training set; because the noise layer draws a new realisation every time, the network never sees the same noisy input twice even though there are only 256 codewords. The paper trains with Adam, compares mean squared error (MSE) and binary cross-entropy (BCE) losses and LLR versus channel-value inputs, and implements everything in Keras with the Theano backend. Training lengths are given as numbers of epochs M_{ep}, from 2^{10} up to 2^{18} in the main experiments (and up to 2^{22} in places). The paper does not report the learning rate or the batch size.

23.4.3 Choosing the training SNR and the NVE

At which SNR should we train? Training at very high SNR shows the network almost noiseless codewords, so it never learns what noise looks like; training at very low SNR buries the code structure in noise. To compare training SNRs, the paper defines the **normalised validation error**:

Key equation: normalised validation error (NVE)

$$\mathrm{NVE}(\rho_t) = \frac{1}{S}\sum_{s=1}^{S} \frac{\mathrm{BER}_{\mathrm{NND}}(\rho_t, \rho_{v,s})}{\mathrm{BER}_{\mathrm{MAP}}(\rho_{v,s})}. \tag{23.6}$$

Here ρ_t is the training E_b/N_0, $\rho_{v,s}$ are S validation SNRs, $\mathrm{BER}_{\mathrm{NND}}(\rho_t, \rho_{v,s})$ is the BER of the neural decoder trained at ρ_t and tested at $\rho_{v,s}$, and $\mathrm{BER}_{\mathrm{MAP}}$ is the MAP BER. NVE $= 1$ means

MAP performance at every validation point; NVE $= 2$ means, on average, twice the MAP BER. The paper uses $S = 20$ validation points between 0 and 5 dB with 20,000 validation examples per point.

Worked example: computing an NVE

Suppose three validation points give $\mathrm{BER}_{\mathrm{NND}} = (0.10, 0.020, 0.0012)$ and $\mathrm{BER}_{\mathrm{MAP}} = (0.08, 0.015, 0.0008)$. The ratios are 1.25, 1.33 and 1.50, so $\mathrm{NVE} = (1.25 + 1.33 + 1.50)/3 = 1.36$. Notice that the high-SNR point, where BERs are tiny, counts as much as the others: the NVE is a *relative* measure and penalises a decoder that falls behind MAP at high SNR.

The paper's NVE-versus-training-SNR curves (for a 128-64-32 network with $M_{ep} = 2^{16}$) have a minimum at an intermediate SNR; from then on it uses a training E_b/N_0 of **1 dB for polar codes and 4 dB for random codes**.

23.5 Results reported in the paper

- **Number of epochs** (paper Fig. 3): for $N = 16$, $r = 1/2$ and the 128-64-32 network, the BER improves steadily with M_{ep} from 2^{10} to 2^{18}; for the polar code it gets close to MAP, while the random code needs more training to approach MAP. Structured codes are easier to learn.
- **Input and loss** (Fig. 4): LLR inputs speed up learning for few epochs, but the advantage disappears for large M_{ep}; with enough training it does not matter whether LLRs or channel values are used and whether MSE or BCE is the loss.
- **Network size** (Fig. 5): MAP performance can be achieved, and — somewhat surprisingly — the larger the network, the fewer training epochs are needed.
- **Scaling with** k (Fig. 6): for a 1024-512-256 network with $M_{ep} = 2^{16}$ and codes of length 16, 32 and 64 with various rates, the NVE depends mainly on k, not on N, and grows exponentially with k. The number of classes 2^k, not the block length, is what makes learning hard.
- **Generalisation** (Fig. 7): the network is trained on a subset $\mathcal{X}_p$ containing p% of the codewords and tested on all codewords and on the unseen ones. For the polar code the network decodes codewords that were never seen during training; for the random code it cannot decode unseen codewords at all. A per-codeword analysis (Fig. 8, at $E_b/N_0 = 4.16$ dB for the codewords left out with $p = 80\%$) supports this.

Intuition: why structure enables generalisation

A polar code is *linear*: the codeword of $\mathbf{u} \oplus \mathbf{u}'$ is the sum of the codewords of $\mathbf{u}$ and $\mathbf{u}'$, and each information bit is a fixed parity pattern over the code bits. A network that learns "bit u_j is determined by these combinations of received values" has learned a rule valid for all codewords, seen or not. In a random code, the bits of an unseen message are an arbitrary label attached to an arbitrary point; nothing in the 230 training points says anything about the other 26.

23.6 Our small-scale reproduction

The code is in `code/ch23/`. The file `codes.py` builds the two codes and the MAP decoders; `nn_decoder.py` trains the neural decoders and makes the figures.

23.6.1 Our codes

- **Polar (16,8):** $\mathbf{G}_{16} = \mathbf{F}^{\otimes 4}$, information set $\{7, 9, 10, 11, 12, 13, 14, 15\}$ from the Bhattacharyya recursion at 1 dB (see the worked example). It is linear with $d_{\min} = 4$ and a distance spectrum (from any codeword) of 28 neighbours at distance 4, 198 at 8, 28 at 12 and 1 at 16.
- **Random (16,8):** 256 codewords drawn uniformly, keeping a candidate only if it is at distance ≥ 3 from all previously kept ones. Result: $d_{\min} = 3$ with on average 2.3 neighbours at distance 3 and 7.1 at distance 4 per codeword. It is not linear.

The random code thus has a *smaller* minimum distance but far fewer close neighbours than the polar code, so the two codes have similar MAP performance at moderate SNR (MAP BER 4.2×10^{-2} for polar and 4.5×10^{-2} for random at 2 dB in our simulation).

```
def polar_generator(n=4):
    F = np.array([[1, 0], [1, 1]])
    G = np.array([[1]])
    for _ in range(n):
        G = np.kron(G, F)
    return G

def polar_info_set(N=16, K=8, design_ebno_db=1.0):
    # recursion in the index order used by x = u G with G = F^{kron n}
    z = np.array([np.exp(-0.5 * 10 ** (design_ebno_db / 10))])
    while len(z) < N:
        new = np.empty(2 * len(z))
        new[0::2] = 2 * z - z ** 2      # index 2i   : worse ('minus') channel
        new[1::2] = z ** 2              # index 2i+1 : better ('plus') channel
        z = new
    # the first split applied to the raw channel ends up as the most significant bit of the index,
    # which is exactly the u-index order of x = u F^{kron n}  (u = [u_first_half, u_second_half])
    zu = z
    info = np.sort(np.argsort(zu)[:K])
    return info, zu

def polar_codebook(design_ebno_db=1.0):
    G = polar_generator()
    info, zu = polar_info_set(N, K, design_ebno_db)
    U = np.zeros((2 ** K, N), int); U[:, info] = MSGS
    return U @ G % 2, info, zu

def random_codebook(seed=0, dmin=3):
    rng = np.random.default_rng(seed)
    cw = []
    while len(cw) < 2 ** K:
        c = rng.integers(0, 2, N)
        if all((c != d).sum() >= dmin for d in cw):
            cw.append(c)
    return np.array(cw)
```

```
def map_bits(y, C, ebno_db):
    """Bit-wise MAP decision for the 8 information bits by exhaustive search over the 256 codewords.
    P(u_j = b | y) is proportional to sum over codewords with u_j = b of exp(<y, x>/sigma^2)."""
    s2 = sigma_from_ebno(ebno_db) ** 2
    X = 1 - 2 * C
    out = np.empty((len(y), K), int)
    for a in range(0, len(y), 10000):                    # chunks keep memory small
        metric = y[a:a + 10000] @ X.T / s2               # (B, 256) log-likelihood up to a constant
        for j in range(K):
            l1 = logsumexp(metric[:, MSGS[:, j] == 1], axis=1)
            l0 = logsumexp(metric[:, MSGS[:, j] == 0], axis=1)
            out[a:a + 10000, j] = (l1 > l0).astype(int)
    return out

def ml_block(y, C):
    """Block-wise ML (= MAP for equiprobable messages): the codeword with the largest correlation."""
    return np.argmax(y @ (1 - 2 * C).T, axis=1)
```

23.6.2 Our decoder and training

```
def make_net(sizes=(128, 64, 32)):
    layers, d = [], N
    for h in sizes:
        layers += [nn.Linear(d, h), nn.ReLU()]; d = h
    layers += [nn.Linear(d, K)]                  # sigmoid is folded into BCEWithLogitsLoss
    return nn.Sequential(*layers)

def train(C, idx_train, ebno_train, epochs, seed=0, reps=None, lr=1e-2, sizes=(128, 64, 32)):
    """Each optimiser step uses `reps` noisy copies of every training codeword (= reps epochs).
    By default reps = epochs / 2048, i.e. always 2048 Adam steps with growing batches."""
    torch.manual_seed(seed)
    reps = reps or max(1, epochs // 2048)
    net = make_net(sizes); opt = torch.optim.Adam(net.parameters(), lr=lr)
    steps = epochs // reps
    sch = torch.optim.lr_scheduler.CosineAnnealingLR(opt, steps)
    X = torch.tensor(1 - 2 * C[idx_train], dtype=torch.float32).repeat(reps, 1)
    U = torch.tensor(MSGS[idx_train], dtype=torch.float32).repeat(reps, 1)
    sig = sigma_from_ebno(ebno_train); lossf = nn.BCEWithLogitsLoss()
    for it in range(steps):
        y = X + sig * torch.randn_like(X)
        loss = lossf(net(y), U)
        opt.zero_grad(); loss.backward(); opt.step(); sch.step()
    return net
```

We follow the paper where it is specific: 128-64-32 with ReLU, sigmoid outputs (inside `BCEWithLogitsLoss`), BCE loss, Adam, raw channel values as input, fresh noise every pass, training E_b/N_0 of 1 dB (polar) and 4 dB (random). Our choices: learning rate 10^{-2} with cosine decay, and every training run uses 2048 Adam steps whose batch contains $M_{ep}/2048$ noisy copies

of each training codeword, so that M_{ep} epochs fit in about a minute of CPU time. The largest budget we use is $M_{ep} = 2^{16}$, a quarter of the paper's 2^{18}.

23.6.3 Result 1: BER versus MAP

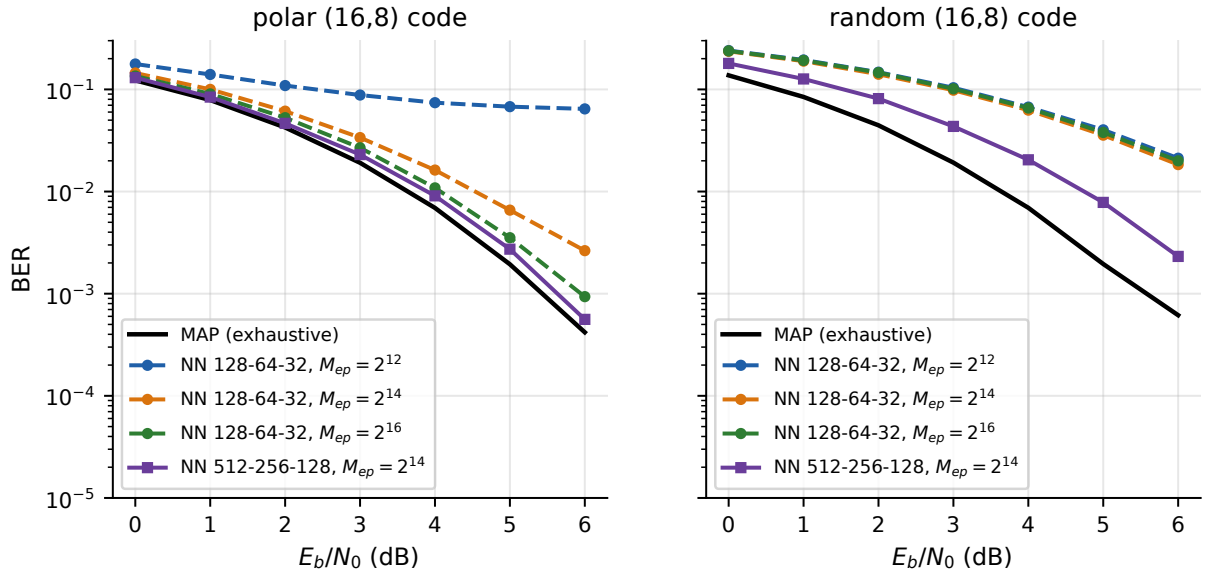


Figure 23.3: Our reproduction of the BER comparison (60,000 test blocks per point for the networks, 40,000 for MAP). Left: polar (16,8). With more training epochs the 128-64-32 decoder (dashed) approaches the exhaustive MAP decoder (black); the larger 512-256-128 network (purple) gets within 6–40% of MAP BER with only 2^{14} epochs. Right: random (16,8). The 128-64-32 network stalls far from MAP regardless of epochs within our budget; the larger network does much better but its BER remains 1.3–4 times the MAP BER.

Figure 23.3 is our analogue of the paper's Fig. 3 (with Fig. 5's network-size effect added). The numbers tell the story:

- **Polar code, 128-64-32.** At 4 dB the BER is 7.4×10^{-2} after 2^{12} epochs, 1.6×10^{-2} after 2^{14} and 1.09×10^{-2} after 2^{16}, against 6.9×10^{-3} for MAP. The gap to MAP shrinks steadily (ratio 2.35 $\rightarrow$ 1.57 at 4 dB from 2^{14} to 2^{16} epochs), consistent with the paper's report that close-to-MAP performance needs about 2^{18} epochs — four times our largest budget.
- **Random code, 128-64-32.** The BER at 4 dB stays near 6.5×10^{-2}, almost ten times the MAP value, and does *not* improve from 2^{12} to 2^{16} epochs in our training setup. The structured code is learned much faster, as the paper reports.
- **Bigger network.** The 512-256-128 network trained for only 2^{14} epochs is the best decoder for both codes (polar: ratio to MAP 1.06–1.41 over 0–6 dB; random: 1.3–4). This matches the paper's "somewhat surprising" observation that larger networks need fewer epochs.

We did not reach MAP exactly for either code; the paper did, with 2^{18} epochs on a GPU. Our results reproduce the *trends* (more epochs and bigger nets approach MAP; polar is easier than random) but not the endpoint.

Pitfall: a stalled curve is not a law of nature

The flat 128-64-32 random-code curves do not prove that this network *cannot* learn the random code: with our fixed 2048 optimiser steps, more epochs only means larger batches, and the optimisation itself may be the bottleneck. A different learning-rate schedule or many more steps might move it. When you reproduce a paper on a budget, separate "this did not work" from "this cannot work".

23.6.4 Result 2: the training SNR and the NVE

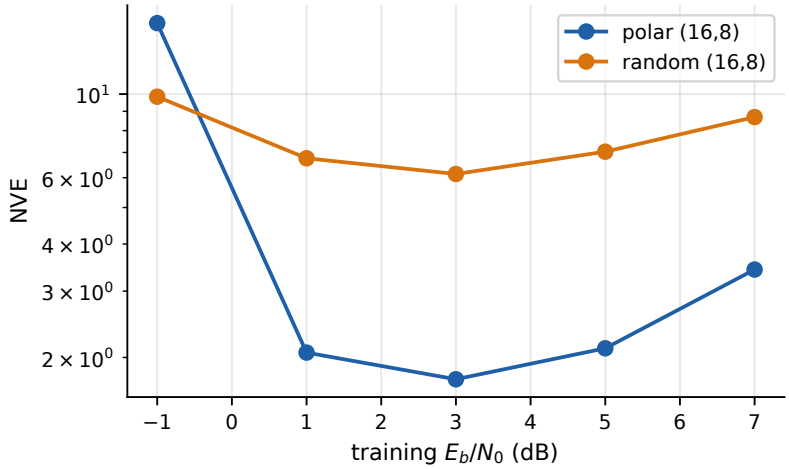


Figure 23.4: Normalised validation error of a 128-64-32 decoder trained for 2^{13} epochs at different training E_b/N_0 (validated at $0, 1, \ldots, 5$ dB, 40,000 blocks each). Both curves are U-shaped: training in too much noise or too little noise hurts. The polar code reaches a much lower NVE than the random code.

Figure 23.4 repeats the idea of the paper's Fig. 2 with a smaller budget (2^{13} epochs, $S = 6$ validation points). We find the same U shape: for the polar code the NVE is 15.4 when training at -1 dB, 2.06 at 1 dB, 1.75 at 3 dB, 2.12 at 5 dB and 3.42 at 7 dB. For the random code it ranges from 9.8 down to 6.1 (at 3 dB) and back up to 8.7. Our optimum for the polar code lies between 1 and 3 dB, close to the paper's 1 dB; for the random code our curve is flat between 1 and 5 dB, consistent with the paper's choice of 4 dB but too flat to pin the optimum down. Note how much easier the polar code is for the same budget: its best NVE is 3.5 times lower.

23.6.5 Result 3: generalisation to unseen codewords

For each code and each $p \in \{50\%, 70\%, 90\%\}$ we chose a random subset $\mathcal{X}_p$ of $p \cdot 256$ messages, trained a 128-64-32 decoder only on them (2^{15} epochs), and measured the BLER separately on the seen codewords and on the $(1-p) \cdot 256$ unseen ones.

Figure 23.5 is the clearest result of this chapter and reproduces the paper's main finding. At 4 dB the polar decoder decodes unseen codewords with BLER 0.73, 0.36 and 0.16 for $p = 50\%$, 70%, 90% — far better than the 255/256 error probability of guessing, and steadily falling as SNR rises. For the random code the unseen-codeword BLER is 0.996 for all three values of p: essentially every unseen codeword is decoded as some *seen* codeword. Interestingly, for the random code the BLER on the seen codewords gets *worse* as p grows ($0.08 \to 0.17 \to 0.25$ at 4 dB), simply because there are more codewords to memorise with the same network and budget; for

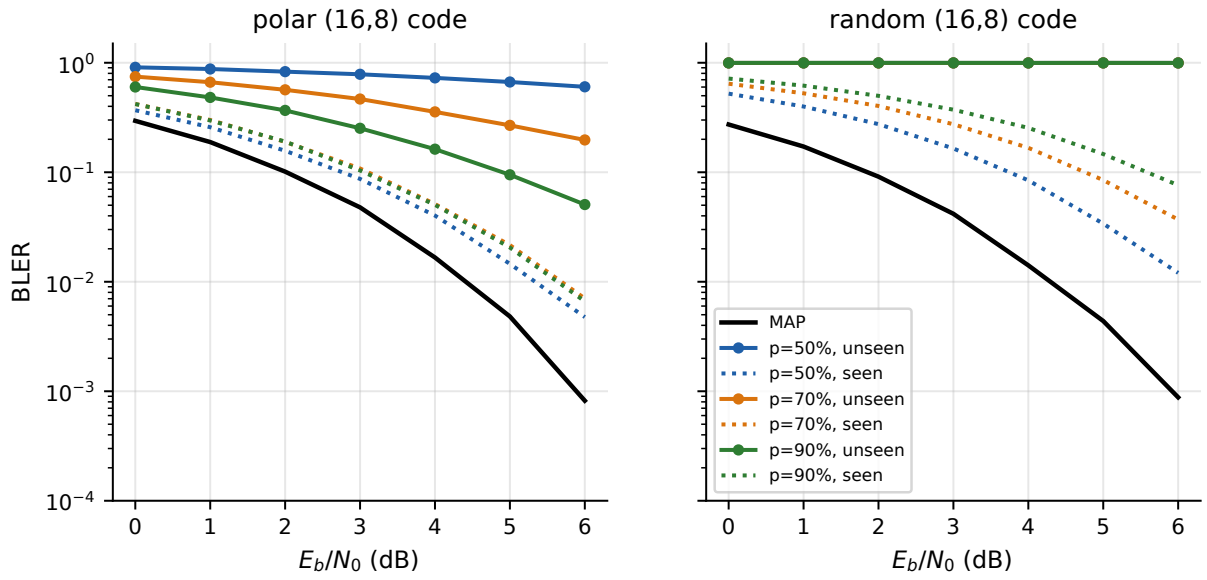


Figure 23.5: Generalisation to unseen codewords (our analogue of the paper's Fig. 7). Solid lines: BLER on codewords never seen in training; dotted: BLER on the training codewords; black: MAP BLER over all codewords. Polar (left): the unseen-codeword BLER falls with SNR and improves strongly as p grows — the decoder has learned something general. Random (right): the unseen-codeword BLER stays at about 1 for every p and SNR — the decoder can only reproduce what it has memorised.

the polar code it stays near 0.05.

Wireless connection: a decoder that never saw a codeword can still decode it

The polar decoder trained on 90% of the codewords decodes the remaining 10% with BLER 0.16 at 4 dB, about ten times worse than MAP but nowhere near random guessing. Since polar codewords are XOR combinations of 8 basis rows, a network that has learned how each information bit depends on the received values can apply that dependence to a new combination. This is the "grammar" of the opening intuition, and it is exactly what a practical decoder must have: no receiver will ever see all 2^k codewords of a real code during training.

23.7 Limitations and follow-up work

- **Exponential training complexity.** The paper itself concludes that the approach is limited to short codes (it studies $N \leq 64$) because training grows exponentially with k. A one-shot MLP decoder for a 5G LDPC code with k in the thousands is out of the question.
- **Generalisation is partial.** Even for polar codes, unseen codewords are decoded worse than seen ones (in the paper and in our runs); the network learns some, not all, of the structure.
- **No algorithmic prior.** The MLP knows nothing about parity checks or the polar recursion; it must infer them from examples.

The research that followed mostly *injects the algorithm into the network*. Nachmani, Be'ery and Burshtein "unfolded" belief propagation into a network with one layer per iteration and learned weights on its edges, keeping the BP structure and therefore its scalability, and improved BP for short BCH codes. Cammerer, Gruber, Hoydis and ten Brink partitioned the polar encoding graph into small sub-blocks, each decoded by a small neural network, and connected them with

belief-propagation stages, which reduces the training complexity. Other lines learn the scaling or offset parameters of min-sum decoders, or use neural networks to guide list decoders. All of them answer the paper's question the same way: learn *inside* a structured algorithm, not instead of it.

23.8 From paper to practice in 5G NR and 6G

Wireless connection: the codes of 5G NR

5G NR uses quasi-cyclic LDPC codes (two base graphs) for the data channels (PDSCH/-PUSCH) and CA-polar codes for control information (DCI on PDCCH, UCI on PUCCH/-PUSCH, and the PBCH), as specified in TS 38.212. Polar code construction in NR uses a single nested reliability sequence of length 1024 from which the information set for any (N, K) is read off — the same idea as our Bhattacharyya ranking, but fixed in the standard. Very small UCI payloads use repetition, simplex or Reed–Muller-based block codes instead of polar codes. Nothing in the standard is a neural decoder; the decoder implementation, however, is left to the vendor.

What this means for a baseband engineer:

- **One-shot MLP decoders are not a product.** At $k = 8$ the exhaustive MAP decoder costs 256×16 multiply-accumulates plus a log-sum-exp — cheaper and better than training a network. For NR payloads (PDCCH carries DCI of a few tens of bits, with a CRC; PDSCH up to thousands) the 2^k wall rules out generic learned decoders.
- **Learned components inside classical decoders are realistic.** Learned normalisation/offset factors in layered min-sum LDPC decoders, learned weights in unrolled BP, or a small network choosing list size or early termination are cheap to implement in fixed point and can be trained offline, then frozen.
- **Short-block regime.** For very short control payloads (URLLC, IoT), near-MAP decoding of short codes matters, and neural or ML-aided decoders remain an active research topic for 6G.
- **Robustness.** A neural decoder trained in AWGN at one SNR is only guaranteed for that setting; classical decoders work with any correctly scaled LLR. Learned demappers that output well-calibrated LLRs for a classical decoder are usually the better engineering split.

Chapter recap

- Decoding a (N, k) code is classification over 2^k codewords; MAP decoding is a sum/-maximum over all of them, feasible for $k = 8$, impossible for practical k.
- Gruber et al. train a 128-64-32 MLP (ReLU, sigmoid outputs) on BPSK/AWGN codewords generated on the fly, with BCE or MSE loss and Adam; training E_b/N_0 1 dB (polar), 4 dB (random), chosen via the NVE.
- Reported: MAP BER is reachable for $N = 16$, $k = 8$; polar codes are learned faster; LLR vs. channel input and MSE vs. BCE do not matter with enough training; bigger nets need fewer epochs; difficulty grows exponentially with k.
- Key finding: the network generalises to unseen codewords for structured (polar) codes but not for random codes.

- Our CPU reproduction confirms the generalisation finding clearly, and shows the slow approach to MAP with a limited training budget.

Exercises

1. Build $\mathbf{G}_4 = \mathbf{F}^{\otimes 2}$ by hand and list all codewords of the $(4, 2)$ polar code with information set $\{1, 3\}$ (positions counted from 0). What is its minimum distance?
2. Starting from $z_0 = 0.5$, apply the recursion Eq. (23.1) twice to get the four badness values of a length-4 polar code. Which two positions would you freeze for $k = 2$?
3. Compute σ for $r = 1/2$ at $E_b/N_0 = 4\,\text{dB}$ and the LLR of a received value $y = -0.3$.
4. How many parameters does a 1024-512-256 decoder have for $N = 64$ and $k = 32$? How many output classes does the underlying classification problem have?
5. Explain why the NVE weights a factor-2 BER loss at 5 dB as much as one at 0 dB. Give one advantage and one drawback of this.
6. A linear code decoder must output $\mathbf{u}$ for any codeword. Show that for a linear code, if a decoder works correctly for codewords $\mathbf{c}_1$ and $\mathbf{c}_2$ under the rule "u_j = a fixed XOR of code bits", it also works for $\mathbf{c}_1 \oplus \mathbf{c}_2$. Why does nothing similar hold for a random code?
7. (Coding) Modify `nn_decoder.py` to use LLR inputs and MSE loss, train for 2^{12} epochs, and compare with channel values and BCE. Do you see the paper's "early advantage that disappears"?

Answers

1. $\mathbf{G}_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$ (rows 0 to 3). With $\mathbf{u} = (u_1, u_2)$ placed on positions 1 and 3, the codeword is $u_1 \cdot \text{row}_1 \oplus u_2 \cdot \text{row}_3$: $(0,0) \to 0000$, $(1,0) \to 1100$, $(0,1) \to 1111$, $(1,1) \to 1100 \oplus 1111 = 0011$. The nonzero codewords have weights 2, 4, 2, so the minimum distance is 2.
2. Step 1: $z^- = 2(0.5) - 0.25 = 0.75$, $z^+ = 0.25$. Step 2 (applied to each, ordered as in our code): positions $0, 1$ come from 0.75: $2(0.75) - 0.5625 = 0.9375$ and 0.5625; positions $2, 3$ from 0.25: 0.4375 and 0.0625. Freeze positions 0 and 1 (badness 0.94 and 0.56); information set $\{2, 3\}$.
3. $\sigma^2 = 1/(2 \cdot 0.5 \cdot 10^{0.4}) = 1/2.512 = 0.398$, $\sigma = 0.631$. LLR $= 2(-0.3)/0.398 = -1.51$, i.e. moderately confident that the bit is 1.
4. $64 \cdot 1024 + 1024 = 66{,}560$; $1024 \cdot 512 + 512 = 524{,}800$; $512 \cdot 256 + 256 = 131{,}328$; $256 \cdot 32 + 32 = 8224$. Total 730,912 parameters. The classification problem has $2^{32} \approx 4.3 \times 10^9$ classes.
5. The NVE averages BER *ratios*, so each SNR point contributes equally regardless of its absolute BER. Advantage: a decoder cannot hide poor high-SNR behaviour (error floors) behind good low-SNR numbers. Drawback: high-SNR BER estimates are noisy (few errors with 20,000 examples), so the NVE inherits that variance; the choice of SNR

range also changes the value.

6. If $u_j = \bigoplus_{i \in S_j} c_i$ for codewords $\mathbf{c}_1$ and $\mathbf{c}_2$ with messages $\mathbf{u}_1$, $\mathbf{u}_2$, then for $\mathbf{c}_1 \oplus \mathbf{c}_2$ (the codeword of $\mathbf{u}_1 \oplus \mathbf{u}_2$ by linearity) the same XOR gives $u_{1,j} \oplus u_{2,j}$, which is correct. So a rule learned on a set of codewords extends to all their XOR combinations — a few well-chosen codewords determine the whole code. In a random code the codeword of $\mathbf{u}_1 \oplus \mathbf{u}_2$ is unrelated to those of $\mathbf{u}_1$ and $\mathbf{u}_2$, so no rule extends.
7. With LLR inputs the input scale is about $2/\sigma^2 \approx 2.5$ times larger at 1 dB, which can speed up early training with a fixed learning rate; MSE gives smaller gradients for confident errors than BCE. After enough epochs both should converge to similar BER, as the paper reports. (Results depend on the learning rate; try two seeds.)

Self-test quiz

1. How many classes does decoding an $(N, k) = (16, 8)$ code involve? $2^8 = 256$ *codewords.*
2. What does "128-64-32" denote? *Three hidden layers with 128, 64 and 32 ReLU units.*
3. Which training E_b/N_0 did the paper choose for polar and for random codes? *1 dB and 4 dB.*
4. What does NVE $= 1$ mean? *The decoder matches MAP BER at all validation SNRs.*
5. What determines the training difficulty: N or k? *k (the number of classes 2^k).*
6. Which code allows generalisation to unseen codewords, and why? *The polar code: it is structured (linear), so a learned rule applies to all codewords.*

Further reading

- T. Gruber, S. Cammerer, J. Hoydis and S. ten Brink, "On Deep Learning-Based Channel Decoding," *Proc. CISS*, Baltimore, 2017 (arXiv:1701.07738). The paper of this chapter.
- E. Arıkan, "Channel Polarization: A Method for Constructing Capacity-Achieving Codes for Symmetric Binary-Input Memoryless Channels," *IEEE Trans. Information Theory*, vol. 55, no. 7, pp. 3051–3073, 2009. The origin of polar codes.
- E. Nachmani, Y. Be'ery and D. Burshtein, "Learning to Decode Linear Codes Using Deep Learning," *Allerton Conference*, 2016; and E. Nachmani et al., "Deep Learning Methods for Improved Decoding of Linear Codes," *IEEE J. Selected Topics in Signal Processing*, vol. 12, no. 1, pp. 119–131, 2018. Unfolded, weighted belief propagation.
- S. Cammerer, T. Gruber, J. Hoydis and S. ten Brink, "Scaling Deep Learning-Based Decoding of Polar Codes via Partitioning," *IEEE GLOBECOM*, 2017 (arXiv:1702.06901).
- 3GPP TS 38.212, "NR; Multiplexing and channel coding." LDPC and polar coding in 5G NR.
- T. O'Shea and J. Hoydis, "An Introduction to Deep Learning for the Physical Layer," 2017 (Chapter 21).

Chapter 24

End-to-End Learning of Communications Systems Without a Channel Model

In Chapter 21 we built a whole communication link — transmitter, channel and receiver — as one neural network and trained it by backpropagation. That worked beautifully in simulation, but it relied on a fact that is false in a real radio: that the channel is a known, differentiable mathematical function through which gradients can flow. A real channel is a box of amplifiers, oscillators, antennas and propagation that you can *use*, but whose derivative you can never *ask for*.

This chapter studies the paper that showed how to train the transmitter anyway. Its trick is to stop treating the transmitter as a function to be differentiated and start treating it as a *reinforcement-learning agent* that tries slightly different signals, hears back how well each one did, and nudges itself towards the better ones. The receiver, meanwhile, keeps learning by ordinary supervised learning. You will see why this works (it is the log-derivative trick of Chapter 19 in a new suit), derive the gradient estimate line by line, check it by hand on a toy example, and then run our own small reproduction on AWGN and Rayleigh block-fading channels.

Paper at a glance

Paper. F. Ait Aoudia and J. Hoydis, "End-to-End Learning of Communications Systems Without a Channel Model", *52nd Asilomar Conference on Signals, Systems, and Computers*, 2018, pp. 298–303 (arXiv:1804.02276). Both authors were at Nokia Bell Labs, Paris-Saclay.

Journal version. F. Ait Aoudia and J. Hoydis, "Model-Free Training of End-to-End Communication Systems", *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 11, pp. 2503–2516, Nov. 2019 (arXiv:1812.05929).

Problem. Autoencoder-based end-to-end learning needs the gradient of the channel, which does not exist for a real, black-box channel.

Key idea. Alternate two phases: train the receiver by supervised learning (it never needs the channel gradient), and train the transmitter by reinforcement learning, perturbing its output with Gaussian noise and using the receiver's per-example losses, fed back over a reliable link, as the (negative) reward.

Main result. On AWGN and Rayleigh block-fading channels ($M = 256$ messages, $N = 4$ channel uses), the alternating method reaches error rates similar to fully supervised training with a known channel model. On AWGN it converges more slowly at first; on Rayleigh block fading the paper reports faster convergence with less variance.

24.1 The problem: you cannot differentiate the air

24.1.1 Recap: the autoencoder link

Recall the set-up of Chapter 21. A message m is drawn from $\mathbb{M} = \{1, \dots, M\}$. The transmitter is a neural network with parameters $\boldsymbol{\theta}_T$ that maps m to N complex channel symbols,

$$\mathbf{x} = f^{(T)}_{\boldsymbol{\theta}_T}(m) \in \mathbb{C}^N, \tag{24.1}$$

normalised so that the average energy per complex symbol is one. The channel produces an output $\mathbf{y}$ drawn from a conditional distribution $P(\mathbf{y} \mid \mathbf{x})$. The receiver network with parameters $\boldsymbol{\theta}_R$ turns $\mathbf{y}$ into a probability vector over the M messages,

$$\mathbf{p} = f^{(R)}_{\boldsymbol{\theta}_R}(\mathbf{y}), \qquad p_i \geq 0, \quad \textstyle\sum_{i=1}^{M} p_i = 1. \tag{24.2}$$

The loss for one example is the cross-entropy between the true message and the prediction,

$$l = -\log \left[f^{(R)}_{\boldsymbol{\theta}_R}(\mathbf{y}) \right]_m, \tag{24.3}$$

i.e. minus the log of the probability that the receiver gives to the message that was actually sent.

24.1.2 Where the chain rule breaks

To train the transmitter by gradient descent we need $\nabla_{\boldsymbol{\theta}_T} l$. Write the forward pass as a chain $m \to \mathbf{x} \to \mathbf{y} \to \mathbf{p} \to l$. The chain rule (Chapter 3) gives

$$\frac{\partial l}{\partial \boldsymbol{\theta}_T} = \underbrace{\frac{\partial l}{\partial \mathbf{y}}}_{\text{receiver: known}} \underbrace{\frac{\partial \mathbf{y}}{\partial \mathbf{x}}}_{\text{channel: ???}} \underbrace{\frac{\partial \mathbf{x}}{\partial \boldsymbol{\theta}_T}}_{\text{transmitter: known}}. \tag{24.4}$$

The first and last factors are inside our own networks, so autograd computes them. The middle factor is the Jacobian of the channel: how the received samples would change if we wiggled each transmitted sample. For a simulated AWGN channel, $\mathbf{y} = \mathbf{x} + \mathbf{n}$ and the Jacobian is the identity — trivial. For a real radio it would include the non-linear power amplifier, the local-oscillator phase noise, the I/Q imbalance, the sampling-clock offset, the multipath, and so on. The paper puts it plainly: in an actual system the channel is a black box whose inputs and outputs can be observed but whose instantaneous transfer function, and therefore its gradient, is unknown.

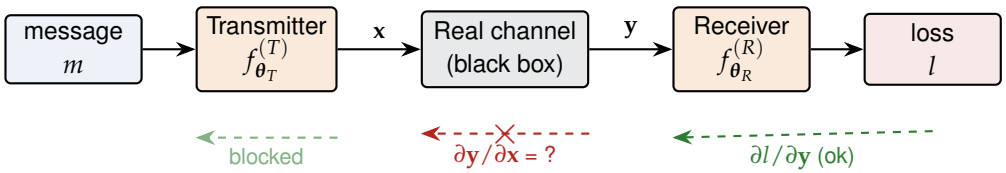


Figure 24.1: Why the classic autoencoder cannot be trained over the air. Forward arrows (top) are fine: we can always send $\mathbf{x}$ and record $\mathbf{y}$. The backward pass (dashed, bottom) reaches the receiver input but stops at the channel, because the Jacobian $\partial \mathbf{y} / \partial \mathbf{x}$ of a real channel is unknown. Without it, no gradient reaches the transmitter parameters $\boldsymbol{\theta}_T$.

Intuition: tuning a guitar you cannot see

Imagine tuning a guitar string with your eyes closed while a friend across the room shouts how out-of-tune the note sounds. You do not know the physics that turns peg rotation into pitch error (that is the "channel gradient"). But you can twist the peg a tiny random amount,

pluck, and hear the friend's score. If the score got better, twist further in that direction; if worse, go the other way. Repeat many times and the string converges to the right pitch. That is exactly how the transmitter will learn: *random small perturbations, a scalar score fed back, and a correlation between the two.*

24.1.3 Earlier work-arounds

Before this paper, two routes were known. One is to train the full autoencoder on a simulated channel model and then fine-tune only the receiver on real measurements; this was demonstrated over the air with software-defined radios by Dörner et al. (cited in the paper). The transmitter is then stuck with whatever it learned for the model, so any mismatch between model and reality costs performance. A second route is to learn a *model* of the channel first (for example with a generative adversarial network, Chapter 18) and backpropagate through the learned model. That requires a large, accurate generative model and still yields only an approximate gradient. The paper's route needs no channel model at all.

24.2 Background: gradients of averages you cannot differentiate

24.2.1 The log-derivative trick, once more

In Chapter 19 we met the policy-gradient theorem. Here is the one-line version we need. Let a be a random "action" drawn from a distribution $\pi_{\boldsymbol{\psi}}(a)$ with parameters $\boldsymbol{\psi}$, and let $L(a)$ be any function of the action — possibly a black box we can only evaluate. We want the gradient of the expected value

$$J(\boldsymbol{\psi}) = \mathbb{E}_{a\sim\pi_{\boldsymbol{\psi}}}\big[L(a)\big] = \int L(a)\,\pi_{\boldsymbol{\psi}}(a)\,\mathrm{d}a. \tag{24.5}$$

Differentiate under the integral. Only π depends on $\boldsymbol{\psi}$:

$$\begin{aligned}
\nabla_{\boldsymbol{\psi}} J &= \int L(a)\,\nabla_{\boldsymbol{\psi}}\pi_{\boldsymbol{\psi}}(a)\,\mathrm{d}a && (L \text{ does not depend on } \boldsymbol{\psi})\\
&= \int L(a)\,\pi_{\boldsymbol{\psi}}(a)\,\frac{\nabla_{\boldsymbol{\psi}}\pi_{\boldsymbol{\psi}}(a)}{\pi_{\boldsymbol{\psi}}(a)}\,\mathrm{d}a && (\text{multiply and divide by } \pi)\\
&= \int L(a)\,\pi_{\boldsymbol{\psi}}(a)\,\nabla_{\boldsymbol{\psi}}\log\pi_{\boldsymbol{\psi}}(a)\,\mathrm{d}a && (\text{since } \nabla\log u = \nabla u/u)\\
&= \mathbb{E}_{a\sim\pi_{\boldsymbol{\psi}}}\big[L(a)\,\nabla_{\boldsymbol{\psi}}\log\pi_{\boldsymbol{\psi}}(a)\big].
\end{aligned}$$

Key equation: score-function (REINFORCE) gradient

$$\nabla_{\boldsymbol{\psi}}\,\mathbb{E}_{a\sim\pi_{\boldsymbol{\psi}}}\big[L(a)\big] = \mathbb{E}_{a\sim\pi_{\boldsymbol{\psi}}}\big[L(a)\,\nabla_{\boldsymbol{\psi}}\log\pi_{\boldsymbol{\psi}}(a)\big] \approx \frac{1}{B}\sum_{i=1}^{B} L(a^{(i)})\,\nabla_{\boldsymbol{\psi}}\log\pi_{\boldsymbol{\psi}}(a^{(i)}). \tag{24.6}$$

Look at what the right-hand side requires: samples $a^{(i)}$ from our own policy, the *values* $L(a^{(i)})$, and the gradient of the log-density of our own policy. *Nothing* about how L depends on a. That is precisely the situation of the transmitter: it controls the distribution of what it sends, it can be told the losses, but it cannot differentiate the channel.

Pitfall: reward or loss?

RL texts maximise a reward; here we minimise a loss. They are the same thing with a sign flip: the loss l plays the role of a negative reward, and we take a gradient *descent* step on J. If you accidentally ascend, the transmitter learns to make the receiver as wrong as possible — a surprisingly effective way to build a jammer.

24.2.2 A one-step bandit

In the language of Chapter 19, the transmitter faces a *contextual bandit*: the state (context) is the message m, the action is the transmitted vector, and the episode ends after one step with the cost l. There is no sequence of states, no discount factor, no bootstrapping — which is why the plain REINFORCE estimator works well here, whereas it struggles in long-horizon tasks.

24.3 System model and the stochastic transmitter

24.3.1 Channels used in the paper

The paper evaluates two channels.

- **AWGN:** $\mathbf{y} = \mathbf{x} + \mathbf{n}$ with $\mathbf{n} \sim \mathcal{CN}(\mathbf{0}, \sigma^2\mathbf{I})$; since the energy per complex symbol is one, $\mathrm{SNR} = 1/\sigma^2$.
- **Rayleigh block fading (RBF):** one complex gain h multiplies the whole block, $\mathbf{y} = h\mathbf{x} + \mathbf{n}$, with $h \sim \mathcal{CN}(0,1)$ drawn independently for each message and unknown to transmitter and receiver.

The paper's RBF receiver contains a small sub-network that estimates h and divides it out (Section 24.5), and the journal version writes the RBF model in real-valued form. Crucially, *the training algorithm itself never uses these equations*: they are only used to simulate the black box.

24.3.2 Turning the transmitter into a policy

REINFORCE needs a *random* action whose density we know. A neural transmitter is deterministic: message in, vector out. The paper makes it stochastic by adding Gaussian exploration noise, but in an energy-preserving way:

Key equation: exploration policy of the transmitter

$$\mathbf{x}_p = \sqrt{1-\sigma_\pi^2}\,\mathbf{x} + \mathbf{w}, \qquad \mathbf{w} \sim \mathcal{CN}(\mathbf{0}, \sigma_\pi^2 \mathbf{I}_N), \qquad \sigma_\pi \in (0,1). \tag{24.7}$$

Why the factor $\sqrt{1-\sigma_\pi^2}$? Because $\mathbf{x}$ and $\mathbf{w}$ are independent and zero-mean noise adds its power: the average energy per symbol of $\mathbf{x}_p$ is $(1-\sigma_\pi^2)\cdot 1 + \sigma_\pi^2 = 1$. The perturbed signal therefore satisfies the same power constraint as the clean one, so exploration does not secretly buy extra transmit power. In the conference paper σ_π^2 is fixed (the paper states it is not learned) and the reported value is $\sigma_\pi^2 = 0.02$.

Equation (24.7) says that, given the message, $\mathbf{x}_p$ is circularly-symmetric complex Gaussian

with mean $\sqrt{1-\sigma_\pi^2}\, f_{\theta_T}^{(T)}(m)$ and covariance $\sigma_\pi^2 \mathbf{I}$. Its density (the policy) is

$$\pi(\mathbf{x}_p \mid m) = \frac{1}{(\pi\sigma_\pi^2)^N} \exp\Big(- \frac{\big\| \mathbf{x}_p - \sqrt{1-\sigma_\pi^2}\, f_{\theta_T}^{(T)}(m) \big\|_2^2}{\sigma_\pi^2} \Big). \tag{24.8}$$

(The π in $(\pi\sigma_\pi^2)^N$ is the number 3.14159; the π on the left is the policy — standard but unlucky notation.) This is just N independent copies of the complex Gaussian density of Chapter 4: each complex dimension has variance σ_π^2, i.e. $\sigma_\pi^2/2$ in each of its real and imaginary parts.

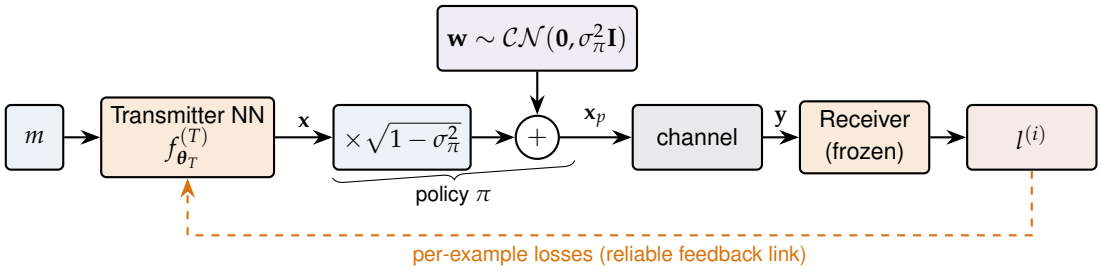


Figure 24.2: The transmitter as an RL agent. During transmitter training its deterministic output $\mathbf{x}$ is shrunk by $\sqrt{1-\sigma_\pi^2}$ and perturbed by Gaussian noise $\mathbf{w}$, giving the action $\mathbf{x}_p$ with the same average power. The receiver is frozen; it scores every example with its cross-entropy loss and these scalars are sent back. Note what is *not* sent back: no gradients, no channel estimate, no received samples.

24.3.3 The policy-gradient estimate, step by step

The transmitter's objective is the average loss over messages, exploration noise and channel:

$$J(\theta_T) = \mathbb{E}_m\, \mathbb{E}_{\mathbf{x}_p \sim \pi(\cdot|m)}\, \mathbb{E}_{\mathbf{y} \sim P(\cdot|\mathbf{x}_p)} \big[l(\mathbf{y}, m) \big] = \mathbb{E}_m\, \mathbb{E}_{\mathbf{x}_p \sim \pi(\cdot|m)} \big[\mathcal{L}(m, \mathbf{x}_p) \big], \tag{24.9}$$

where $\mathcal{L}(m, \mathbf{x}_p) = \mathbb{E}[l \mid m, \mathbf{x}_p]$ is the expected loss once message and action are fixed (the paper writes $\mathcal{L}(s,a)$ in RL notation). Applying (24.6) for each m:

$$\nabla_{\theta_T} J = \mathbb{E}_m\, \mathbb{E}_{\mathbf{x}_p} \big[\mathcal{L}(m, \mathbf{x}_p)\, \nabla_{\theta_T} \log \pi(\mathbf{x}_p \mid m) \big]. \tag{24.10}$$

Now the score $\nabla \log \pi$. Take the log of (24.8). Write $a = \sqrt{1-\sigma_\pi^2}$ and $\mathbf{f} = f_{\theta_T}^{(T)}(m)$:

$$\log \pi(\mathbf{x}_p \mid m) = -N \log(\pi\sigma_\pi^2) - \frac{1}{\sigma_\pi^2} \big\| \mathbf{x}_p - a\mathbf{f} \big\|_2^2 .$$

The first term is a constant. For the second, treat each complex number as two reals, so $\mathbf{f}$ and $\mathbf{x}_p$ become vectors in $\mathbb{R}^{2N}$ and $\big\|\mathbf{x}_p - a\mathbf{f}\big\|^2 = \sum_{k=1}^{2N} (x_{p,k} - a f_k)^2$. Differentiating one term with respect to f_k:

$$\frac{\partial}{\partial f_k} \Big[- \frac{(x_{p,k} - a f_k)^2}{\sigma_\pi^2} \Big] = - \frac{2(x_{p,k} - a f_k) \cdot (-a)}{\sigma_\pi^2} = \frac{2a}{\sigma_\pi^2} (x_{p,k} - a f_k).$$

So $\nabla_{\mathbf{f}} \log \pi = \frac{2a}{\sigma_\pi^2}(\mathbf{x}_p - a\mathbf{f})$, and one more application of the chain rule through the transmitter network (whose Jacobian $\nabla_{\theta_T}\mathbf{f}$ autograd knows) gives

Key equation: score of the Gaussian transmitter policy

$$\nabla_{\theta_T} \log \pi(\mathbf{x}_p \mid m) = \frac{2\sqrt{1-\sigma_\pi^2}}{\sigma_\pi^2} \big(\nabla_{\theta_T} f_{\theta_T}^{(T)}(m) \big)^{\mathsf{T}} \big(\mathbf{x}_p - \sqrt{1-\sigma_\pi^2}\, f_{\theta_T}^{(T)}(m) \big). \tag{24.11}$$

This is exactly the expression in the paper. Notice that $\mathbf{x}_p - a\mathbf{f} = \mathbf{w}$, the exploration noise that was actually drawn. Finally replace the expectations in (24.10) by a mini-batch of B_T examples, and the unknown $\mathcal{L}(m, \mathbf{x}_p)$ by the single observed loss $l^{(i)}$ (an unbiased sample of it):

Key equation: transmitter gradient estimate used in training

$$\nabla_{\theta_T} \widetilde{J} = \frac{1}{B_T} \sum_{i=1}^{B_T} l^{(i)} \, \nabla_{\theta_T} \log \pi\big(\mathbf{x}_p^{(i)} \mid m^{(i)}\big), \qquad l^{(i)} = -\log \big[f_{\theta_R}^{(R)}(\mathbf{y}^{(i)})\big]_{m^{(i)}}. \tag{24.12}$$

Intuition: reading equation (24.12)

Each term is "(how bad this example was) $\times$ (the direction in which this example's noise pushed the signal)". If the noise $\mathbf{w}^{(i)}$ happened to push the signal in a direction that produced a *large* loss, the term points along $\mathbf{w}^{(i)}$, and the descent step moves the codeword *away* from that direction. If another sample's noise produced a small loss, its term is small and the codeword moves away from it less. On average the codeword drifts towards the directions that lower the loss — exactly a finite-difference gradient, but estimated from random probes and a scalar score.

Worked example: policy gradient by hand on a 1-D toy

Take one real "symbol" $f = \theta = 0.5$, exploration variance $\sigma_\pi^2 = 0.02$ (so $a = \sqrt{0.98} = 0.98995$ and the clean, shrunk symbol is $a\theta = 0.49497$), and pretend the black-box loss is $L(v) = (v-1)^2$. For a single real dimension the noise has variance $\sigma_\pi^2/2 = 0.01$ and the score is $\frac{2a}{\sigma_\pi^2} w$ times $\partial f/\partial\theta = 1$.

Draw two samples, $w = +0.1$ and $w = -0.1$:

- $w = +0.1$: $x_p = 0.59497$, loss $L = 0.16405$, score $= \frac{2(0.98995)}{0.02}(0.1) = 9.8995$, product $= 1.62397$.
- $w = -0.1$: $x_p = 0.39497$, loss $L = 0.36606$, score $= -9.8995$, product $= -3.62377$.

Average: $(1.62397 - 3.62377)/2 = -0.99990$. The exact gradient of $\mathbb{E}[L(a\theta + w)]$ is $2a(a\theta - 1) = 2(0.98995)(0.49497 - 1) = -0.99990$. The estimator nailed it (with a symmetric pair and a quadratic loss it is exact). Notice the individual products are large and of opposite signs: the estimate is the small difference of two big numbers. That is variance, the price of not knowing the channel.

Variance reduction. Subtract the batch-mean loss $b = (0.16405 + 0.36606)/2 = 0.26505$ from each loss before multiplying: $(0.16405 - 0.26505)(9.8995) = -0.99990$ and $(0.36606 - 0.26505)(-9.8995) = -0.99990$. Each single sample is now exact. Subtracting a baseline does not change the expectation (because $\mathbb{E}[\nabla \log \pi] = 0$), but it removes the huge "common-mode" part of the loss. (The paper does not use a baseline; we use it as an optional improvement in our experiments.)

24.3.4 Why only the transmitter needs RL

The receiver sees $\mathbf{y}$ directly and computes its own loss (24.3); $\nabla_{\theta_R} l$ requires only the receiver's own Jacobian. So the receiver is trained by plain supervised learning on pairs $(\mathbf{y}^{(i)}, m^{(i)})$ — it just needs to know which messages were sent, which in practice means a training sequence known to both ends (like a long pilot). The journal version notes that no perturbation is applied during receiver training, since it is not needed to compute the receiver gradient.

24.4 The alternating training algorithm

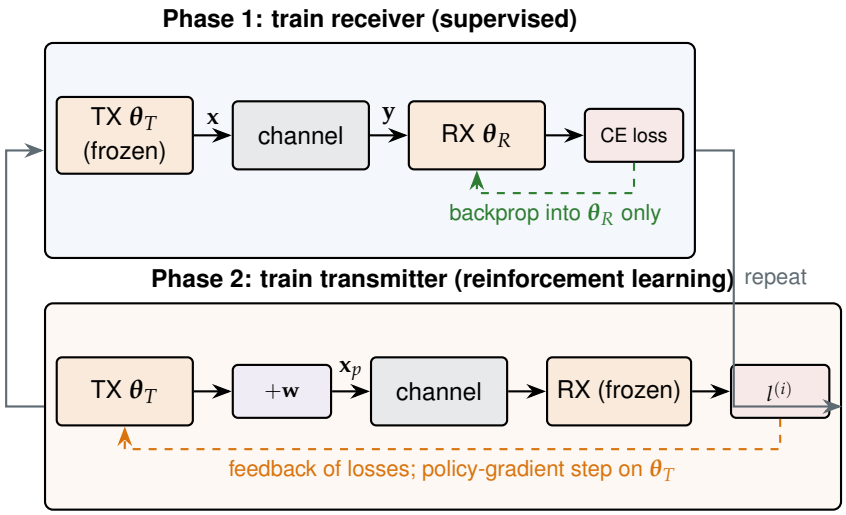


Figure 24.3: The two phases of one training iteration. Phase 1 is ordinary supervised learning of the receiver with the transmitter frozen and unperturbed. Phase 2 freezes the receiver, perturbs the transmitter output, and updates the transmitter with the policy-gradient estimate built from the fed-back per-example losses. The channel is only ever *used*, never differentiated.

The paper's Algorithm 1 is a simple outer loop:

1. **Repeat until a stop criterion is met:**
2. TRAINRECEIVER: draw B_R random messages, transmit $\mathbf{x} = f^{(T)}(m)$ (no exploration), record $\mathbf{y}$, take an SGD (Adam) step on the mean cross-entropy w.r.t. $\boldsymbol{\theta}_R$.
3. TRAINTRANSMITTER: draw B_T messages, compute $\mathbf{x}$, sample $\mathbf{x}_p$ from (24.7), transmit, let the receiver compute the per-example losses $l^{(i)}$ and feed them back over a reliable link, take an SGD (Adam) step with (24.12).

The journal version reports performing ten gradient steps on the receiver followed by ten on the transmitter in each iteration. Batch sizes and learning rates are not given in the conference paper.

Wireless connection: what the feedback link has to carry

Per transmitter step the receiver must return B_T real numbers. With B_T in the hundreds or thousands this is a modest control-plane load, but it is *not* free and it has to be reliable: the paper explicitly lists the need for an additional reliable feedback channel during training as its main limitation. Chapter 25 removes the reliability assumption. Compare this with 5G NR, where the uplink already carries scalar quality reports (CQI) and HARQ ACK/NACKs — coarse versions of exactly this kind of feedback.

24.5 Architectures reported in the paper

The paper uses deliberately small networks (Table 24.1). The transmitter is an embedding of size $M \times M$ followed by an ELU, a dense layer with $2N$ linear outputs, a real-to-complex conversion and an energy normalisation layer that enforces unit average energy per symbol. The AWGN receiver converts the N complex samples to $2N$ reals, then uses one dense hidden layer of M ReLU units and a softmax over M outputs.

Block	Layer (as described in the paper)	Output
Transmitter	embedding $M \times M$, ELU	M
	dense, linear	$2N$ reals
	real $\to$ complex; normalisation (unit avg. energy/symbol)	N complex
Receiver (AWGN)	complex $\to$ real	$2N$
	dense, M units, ReLU	M
	dense, M units, softmax	M probabilities
Receiver (RBF)	complex $\to$ real; two dense layers estimating $\hat{h}$	2 reals ($\hat{h}$)
	divide received samples by $\hat{h}$	$2N$
	then the same layers as the AWGN receiver	M probabilities

Table 24.1: Transmitter and receiver architectures of the paper ($M = 256$, $N = 4$ in the experiments). Exact widths of the two channel-estimation layers are not specified in the conference text.

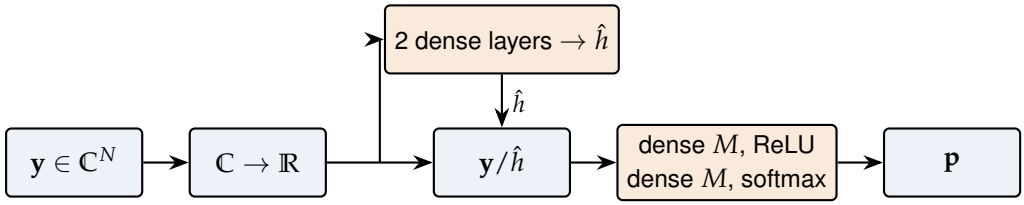


Figure 24.4: Receiver for Rayleigh block fading, drawn after the paper's description: a small network estimates the complex gain $\hat{h}$ from the received block, the block is divided by $\hat{h}$ (a learned one-tap equaliser, in the spirit of the "radio transformer network" of Chapter 21), and the AWGN-style classifier decides.

24.6 Results reported in the paper

Set-up. $M = 256$ messages (8 bits) over $N = 4$ complex channel uses, i.e. 2 bits per channel use. Training at 10 dB on AWGN and 20 dB on RBF, Adam optimiser, $\sigma_\pi^2 = 0.02$. The comparison is against the fully supervised autoencoder of Chapter 21, which backpropagates through the known channel model.

Convergence (paper Fig. 5). Over the first 500 training iterations on AWGN, the supervised method converges faster, which is expected since it has the true gradient; the paper notes that after roughly 400 iterations there is no significant difference. On RBF the picture reverses: the paper reports that the alternating method converges faster and with significantly less variance, and attributes this to the random channel gain having less impact on the per-example losses that drive the transmitter than on the backpropagated gradients.

Final performance (paper Fig. 6). Error rate versus SNR after convergence is similar for the two training methods on both AWGN and RBF: the alternating method, without any channel model, matches the fully supervised approach.

Journal version. The JSAC paper extends the study considerably. It proves (its Theorem 1) that, under some conditions, the gradient of the relaxed (perturbed) loss tends to the true gradient as the perturbation standard deviation goes to zero, while also noting that a smaller perturbation increases the estimator variance, so that larger batches are needed. Its simulations use $\sigma = 0.15$ (standard deviation) on AWGN and RBF; it compares against QPSK and against "Agrell", a

256-point subset of the E_8 lattice obtained by numerical sphere-packing optimisation. On AWGN, both model-free and model-aware training beat QPSK but do not reach Agrell; on RBF, the learned systems outperform both QPSK and Agrell. Model-aware training converges faster in the first iterations (curves averaged over 200 seeds), but the difference fades. A noisy-feedback experiment shows no BLER increase for feedback SNRs above about 6 dB. The journal paper also demonstrates the method over the air and over coaxial cable with USRP B210 radios at 2.3 GHz combined with OFDM, outperforming QPSK in both settings, and applies it to image transmission and a fibre-optic channel model.

24.7 Our small-scale reproduction

We now implement both training methods ourselves. To fit the CPU budget of this book (each script under four minutes on one core), we shrink the problem to $M = 16$ messages (4 bits) over $N = 2$ complex channel uses — the same rate of 2 bits per channel use as the paper, but a much smaller codebook. Everything else follows the paper: the transmitter is dense(M, ELU) $\to$ dense($2N$, linear) $\to$ energy normalisation (a dense layer on a one-hot input is the same thing as an embedding); the receiver is dense(64, ReLU) $\to$ dense(M); exploration uses (24.7) with $\sigma_\pi^2 = 0.02$; losses are fed back perfectly. One "iteration" is one receiver step followed by one transmitter step, each on a batch of 256 messages, with Adam (learning rates 10^{-2} for the receiver and the supervised system, $3 \cdot 10^{-3}$ for the RL transmitter). All code is in `code/ch24/`.

Listing 24.1: The two phases and the supervised baseline (`code/ch24/e2e.py`). The transmitter step builds a "surrogate" whose autograd gradient equals the estimate (24.12): the losses are computed under `no_grad`, so they act as constants multiplying $\nabla \log \pi$.

```
def rx_step(tx, rx, opt_r, M, B, snr_db, kind):
    """Receiver step: supervised cross-entropy, no perturbation."""
    m = torch.randint(0, M, (B,))
    with torch.no_grad():
        y = channel(tx(m), snr_db, kind)
    loss = F.cross_entropy(rx(y), m)
    opt_r.zero_grad(); loss.backward(); opt_r.step()
    return loss.item()


def tx_step_pg(tx, rx, opt_t, M, B, snr_db, kind, sig2=0.02, baseline=False):
    """Transmitter step as an RL agent.
    Action x_p = sqrt(1-sig2) x + w, w ~ CN(0, sig2 I).
    Reward = minus the per-example losses fed back."""
    m = torch.randint(0, M, (B,))
    x = tx(m)                                   # depends on theta_T
    a = math.sqrt(1 - sig2)
    with torch.no_grad():
        xp = a * x + torch.randn_like(x) * math.sqrt(sig2 / 2)  # action
        y = channel(xp, snr_db, kind)           # black box
        l = F.cross_entropy(rx(y), m, reduction="none")  # fed back
        if baseline:
            l = l - l.mean()
    # log pi(xp|m) = -||xp - a x||^2 / sig2 + const
    logpi = -((xp - a * x) ** 2).sum(1) / sig2
    surrogate = (l * logpi).mean()              # its gradient = PG estimate
    opt_t.zero_grad(); surrogate.backward(); opt_t.step()
    return l.mean().item()


```

```python
def joint_step(tx, rx, opt, M, B, snr_db, kind):
    """Supervised baseline: backprop through a KNOWN channel."""
    m = torch.randint(0, M, (B,))
    loss = F.cross_entropy(rx(channel(tx(m), snr_db, kind)), m)
    opt.zero_grad(); loss.backward(); opt.step()
    return loss.item()
```

Pitfall: the surrogate-loss trick

PyTorch cannot evaluate (24.12) directly, so we write a scalar whose gradient *is* (24.12): `(l * logpi).mean()` with `l` detached. Two classic bugs: (i) forgetting to detach `l` (then autograd backpropagates through the simulated channel and you are secretly doing supervised training — your "model-free" result is a lie); (ii) detaching `x` inside `logpi` (then the gradient is zero). We put the channel and receiver inside `torch.no_grad()` to make (i) impossible.

24.7.1 AWGN: training curves and error rates

Figure 24.5 shows the BLER at the training SNR of 7 dB during training. As in the paper, the supervised system descends faster at the very start — it receives the exact gradient through the known channel. The model-free system needs a few hundred more iterations and then tracks the supervised one closely, though in our 1500-iteration budget a small gap remains.

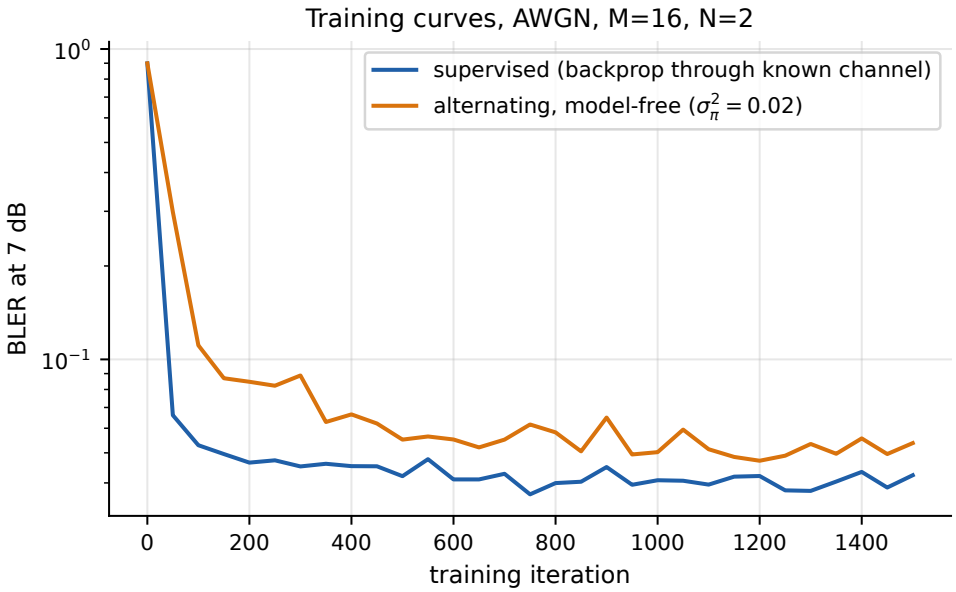


Figure 24.5: Our reproduction, AWGN, $M = 16$, $N = 2$: BLER at 7 dB during training (evaluated on 20 000 fresh messages every 50 iterations). Supervised training (blue) has the true gradient and converges almost immediately; alternating model-free training (orange) starts more slowly and then follows a similar curve, slightly above it.

Numbers. At 10 dB we measured (100 000 test messages per point) BLER $2.29 \cdot 10^{-3}$ (supervised), $4.46 \cdot 10^{-3}$ (alternating) and $3.13 \cdot 10^{-3}$ (QPSK, exact); at 12 dB, $1.7 \cdot 10^{-4}$, $4.1 \cdot 10^{-4}$ and $1.4 \cdot 10^{-4}$. So the model-free system pays roughly half a decibel in our short run. The paper, training a larger system for longer, reports no significant final difference; our gap is a budget effect rather than a contradiction — the training curve in Figure 24.5 is still drifting down at the end.

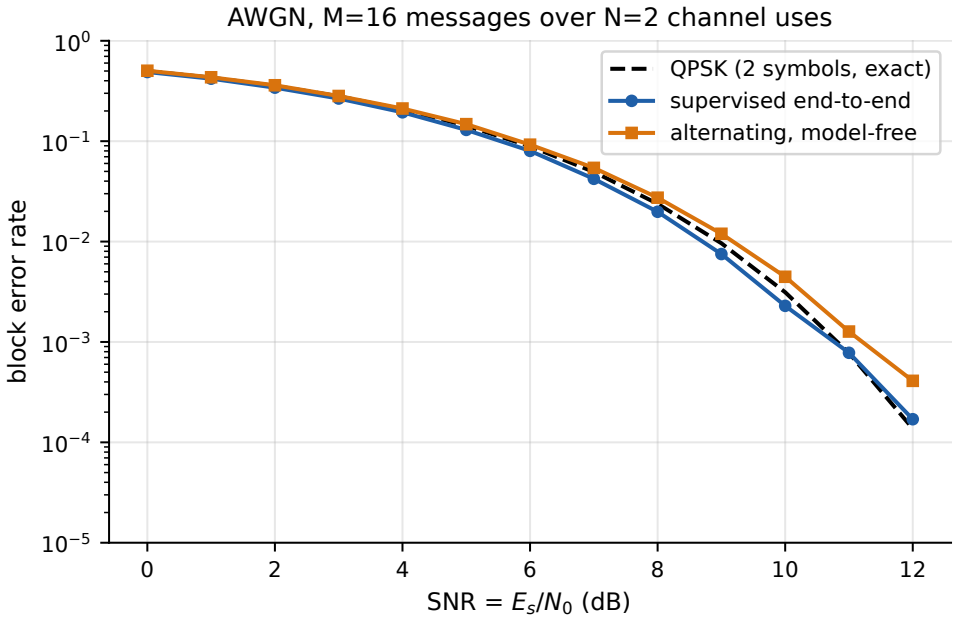


Figure 24.6: Our reproduction: BLER vs SNR after 1500 iterations. Dashed: the exact BLER of sending the 4 bits as two Gray-mapped QPSK symbols. The supervised autoencoder sits on the QPSK curve (at this small size, two QPSK symbols are already an excellent 16-point code in four real dimensions); the model-free system is within a fraction of a dB of it and reaches $4.5 \cdot 10^{-3}$ at 10 dB versus $2.3 \cdot 10^{-3}$ for supervised training.

24.7.2 Learned constellations

To see what the transmitter learns, we repeat the training with $N = 1$ (one complex symbol, $M = 16$, trained at 15 dB) so that the codebook is a picture in the plane (Figure 24.7). Both methods arrive at roughly hexagonal packings — concentric rings with a point near the centre, the shape that sphere-packing theory predicts beats the square 16-QAM grid at equal average power. The symbol error rates at 15 dB were $1.87 \cdot 10^{-2}$ (supervised) and $2.04 \cdot 10^{-2}$ (model-free); for reference, 16-QAM has an exact SER of $1.78 \cdot 10^{-2}$ there, so neither learned constellation has quite reached a well-designed one after 1500 short iterations.

24.7.3 Rayleigh block fading

We next switch the black box to Rayleigh block fading, with one $h \sim \mathcal{CN}(0, 1)$ per message, and use the estimate-and-divide receiver of Figure 24.4 (estimator: dense(32, ReLU) $\to$ dense(2); classifier as before), training at 20 dB as in the paper. Neither transmitter nor receiver is told h; the receiver must infer it from the shape of the received block itself.

Two observations. First, in agreement with the paper, the model-free method reaches the same performance as supervised training on this channel; in our run it never fell behind after the first 200 iterations. Second, both learned systems slightly beat a classical pilot-plus-16-QAM design of the same rate: the autoencoder learns a codebook whose shape lets the receiver estimate the fading gain *and* the data jointly, without spending a whole channel use on a pilot. The genie curve is far better, but it is cheating: it assumes perfect channel knowledge for free. (We did not reproduce the paper's convergence-speed advantage for alternating training on RBF; with our small network the two training curves are too close to tell apart.)

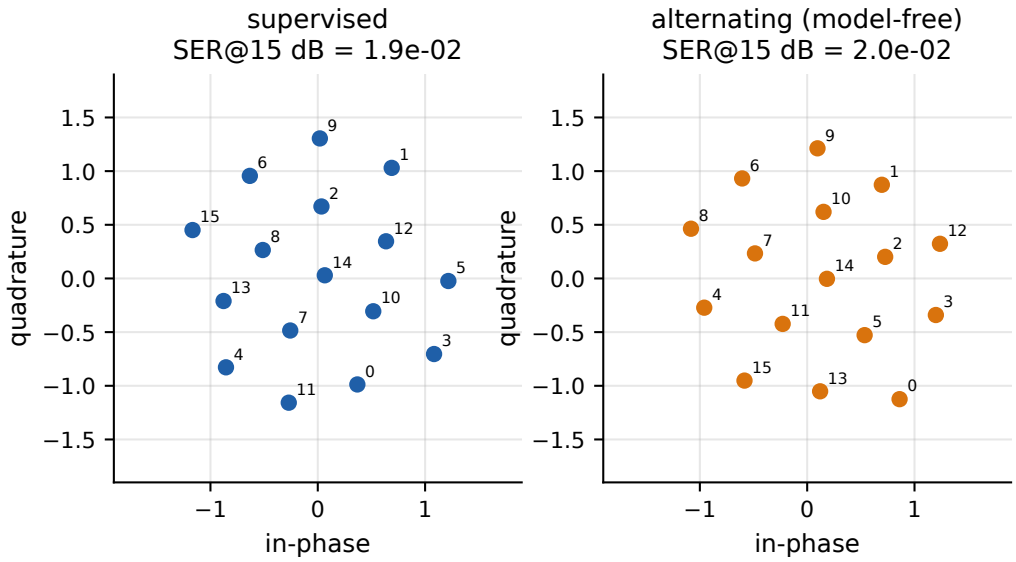


Figure 24.7: Our reproduction: 16-point constellations ($N = 1$) learned by supervised training (left) and by the model-free alternating method (right) at 15 dB. Numbers are message indices. Both are ring-like, roughly hexagonal packings; the model-free transmitter discovered this shape without ever computing a channel gradient — only from noisy probes and scalar losses.

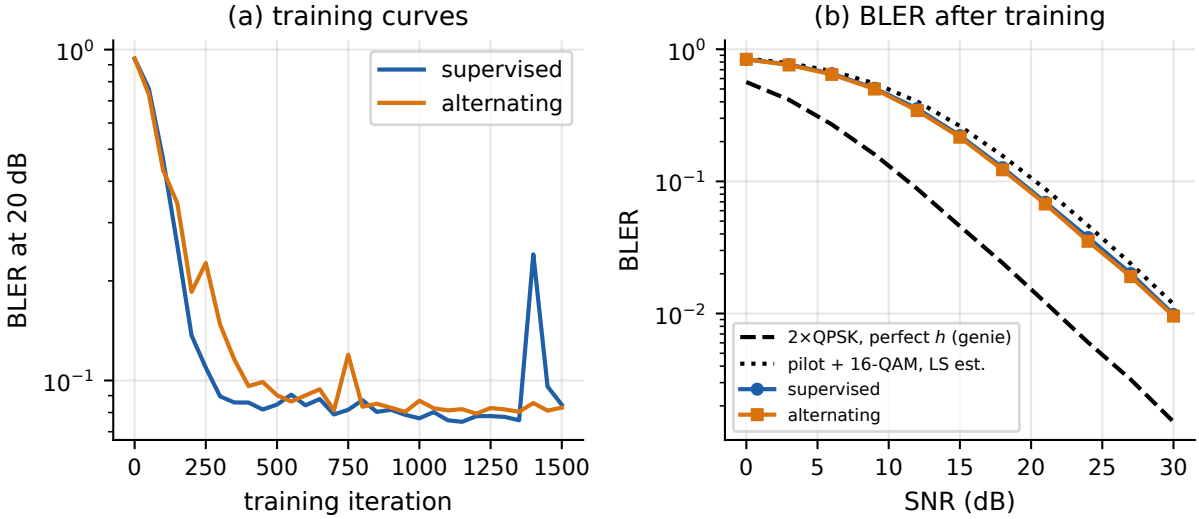


Figure 24.8: Our reproduction, Rayleigh block fading, $M = 16$, $N = 2$. (a) BLER at 20 dB during training: both methods follow similar curves; the supervised run shows a short instability near iteration 1400. (b) BLER vs SNR: supervised and model-free systems are indistinguishable ($9.8 \cdot 10^{-3}$ vs $9.6 \cdot 10^{-3}$ at 30 dB). Dotted: a practical non-learned scheme at the same rate (one pilot, then one 16-QAM symbol, least-squares channel estimate), slightly worse ($1.2 \cdot 10^{-2}$ at 30 dB). Dashed: two QPSK symbols with a *genie* that tells the receiver h exactly — an unreachable lower reference.

24.7.4 How much should the transmitter explore?

The exploration variance σ_π^2 trades off two errors. Small σ_π^2: the probes are tiny, so the gradient is that of the true loss (low bias), but each loss difference is buried under the channel-noise fluctuations of l (high variance — recall the factor $1/\sigma_\pi^2$ in (24.11)). Large σ_π^2: big, informative probes, but the transmitter is optimising a smoothed, perturbed loss and wastes a fraction σ_π^2 of its power on noise during training.

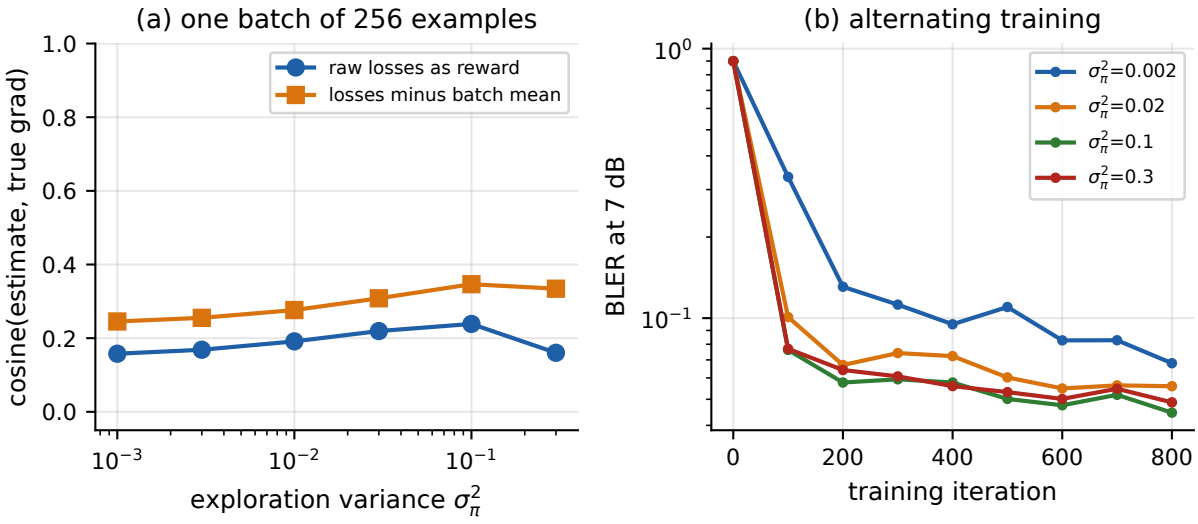


Figure 24.9: Our study of the exploration variance (AWGN, $M = 16$, $N = 2$, 7 dB). (a) Quality of a single-batch (256 examples) policy-gradient estimate, measured as its cosine similarity with the true gradient (which we can compute here because our simulated channel is differentiable), averaged over 10 batches. Subtracting the batch-mean loss (a baseline) helps at every σ_π^2. (b) Alternating training with four values of σ_π^2: very small exploration (0.002) learns clearly more slowly.

Figure 24.9(a) is sobering: even the best single-batch estimate has a cosine similarity of only about 0.2–0.35 with the true gradient. Training works anyway because Adam averages many such noisy estimates over iterations (Chapter 10). The quality improves slowly with σ_π^2 up to about 0.1 and then drops again as the bias of large probes sets in. Panel (b) confirms the effect on learning: $\sigma_\pi^2 = 0.002$ is clearly slower (BLER $6.8 \cdot 10^{-2}$ after 800 iterations) than 0.02 ($5.6 \cdot 10^{-2}$), 0.1 ($4.5 \cdot 10^{-2}$) or 0.3 ($4.9 \cdot 10^{-2}$). The precise optimum depends on the problem, batch size and SNR; the paper's choice of 0.02 is a safe middle value. Remember that exploration is only used during transmitter training — at deployment the transmitter sends the clean $\mathbf{x}$.

Wireless connection: exploration costs real SNR on a real link

Over the air, $\mathbf{w}$ is generated by the transmitter's own DSP and radiated. With $\sigma_\pi^2 = 0.02$ the clean part of the signal is $10\log_{10}(0.98/0.02) \approx 16.9$ dB above the exploration noise, so at a link SNR well below 17 dB the exploration noise is a small extra impairment. At high SNR (say 30 dB, typical for 256-QAM in 5G NR), it would dominate the thermal noise during training, and the transmitter would be optimising for a noisier link than the real one. This is one practical reason to make σ_π small, or to anneal it.

24.8 Limitations and follow-ups

- **Reliable feedback.** The method needs a reliable link to return per-example losses during training — the paper's own stated limitation. Chapter 25 studies what happens when that feedback is noisy, and how to learn the feedback link too.
- **Sample efficiency.** The score-function estimator has high variance; the transmitter needs many more channel uses than a backprop-trained one would, and the gap grows with the number of transmitter outputs. The journal version compares against SPSA (simultaneous perturbation stochastic approximation), finding that SPSA's variance grows much faster with the number of parameters.
- **Block-based messages.** Like Chapter 21, the system maps one of M messages to one block; $M = 256$ is far from the thousands of bits of a real transport block. Scaling requires coded bit-wise designs (e.g. learned constellations under a standard channel code).
- **Alternatives.** Later work trains a generative model of the channel ("GAN-based channel models") and backpropagates through it, or combines both ideas; others remove the feedback entirely by having the receiver learn alone around a fixed, robust transmitter.

24.9 From paper to practice in 5G NR/6G

- **Where the transmitter lives.** In the downlink, the gNB is the transmitter and the UE computes losses. The losses would have to travel in the uplink (e.g. multiplexed on PUSCH), during a dedicated training phase with a known pseudo-random message sequence playing the role of labels — conceptually like a very long, structured reference signal.
- **What can be learned.** Realistic first targets are not whole PHYs but parameterised pieces: a learned constellation (geometric shaping) used with the standard LDPC code, or the pilot pattern, both of which 6G studies on "AI-native air interfaces" have examined. The paper's algorithm applies unchanged if the "transmitter" is just the constellation points.
- **Hardware impairments.** The real selling point is training through components no model captures well: a strongly driven PA, low-resolution DACs/ADCs in sub-THz radios, or phase noise at FR2/FR3 carriers. The model-free gradient sees them all.
- **Signalling and standardisation.** A learned transmitter must be known to the receiver at inference time. In a standardised system, either the codebook is exchanged by signalling after training (like a new MCS table) or both ends run identical, synchronised training — issues 3GPP's AI/ML study items for the air interface (Rel-18 onwards) have only begun to address.

24.10 Recap, exercises and quiz

Chapter recap

- End-to-end autoencoder training needs $\partial \mathbf{y}/\partial \mathbf{x}$, which a real channel does not provide. The receiver does not need it; the transmitter does.
- The paper makes the transmitter a stochastic policy, $\mathbf{x}_p = \sqrt{1-\sigma_\pi^2}\,\mathbf{x} + \mathbf{w}$ with $\mathbf{w} \sim \mathcal{CN}(\mathbf{0}, \sigma_\pi^2 \mathbf{I})$, which keeps the average power at one.

- The log-derivative trick turns the unknown gradient into an average of (per-example loss) $\times$ (score), with the score $\frac{2\sqrt{1-\sigma_\pi^2}}{\sigma_\pi^2}(\nabla f)^{\mathsf{T}}(\mathbf{x}_p - \sqrt{1-\sigma_\pi^2} f)$.
- Training alternates: supervised receiver steps, then RL transmitter steps using losses fed back over a reliable link.
- The paper reports performance similar to supervised training on AWGN and Rayleigh block fading; our small reproduction ($M = 16$, $N = 2$) agrees on RBF and comes within about 0.5 dB on AWGN in a short run.
- The exploration variance trades bias against variance; subtracting a baseline reduces variance for free.

Exercises

1. **(Power.)** Show that $\mathbb{E}\left\|\mathbf{x}_p\right\|^2 / N = 1$ when $\mathbb{E}\left\|\mathbf{x}\right\|^2 / N = 1$ and $\mathbf{w} \sim \mathcal{CN}(\mathbf{0}, \sigma_\pi^2 \mathbf{I})$ is independent of $\mathbf{x}$. What would the average energy be if we used $\mathbf{x}_p = \mathbf{x} + \mathbf{w}$ with $\sigma_\pi^2 = 0.02$, in dB above one?
2. **(Score by hand.)** For $N = 1$, $\sigma_\pi^2 = 0.1$, $f = 0.6 + 0.2j$ and a sampled $x_p = 0.7 + 0.1j$, compute $\nabla_{\mathbf{f}} \log \pi$ as a 2-vector (real, imaginary).
3. **(Zero-mean score.)** Prove that $\mathbb{E}_{a\sim\pi_{\boldsymbol{\psi}}}[\nabla_{\boldsymbol{\psi}} \log \pi_{\boldsymbol{\psi}}(a)] = \mathbf{0}$, and conclude that subtracting any constant b from the losses leaves the expectation of (24.12) unchanged.
4. **(Toy variance.)** In the worked example, repeat the calculation with $w = \pm 0.2$ instead of ± 0.1. Is the two-sample estimate still exact? Why?
5. **(Which phase needs what?)** List what the transmitter and receiver must each know or receive in (a) supervised end-to-end training and (b) alternating training. Which of these items could be obtained in a real deployment?
6. **(Exploration budget.)** If the link runs at 25 dB SNR during training, what SNR does the receiver effectively see when $\sigma_\pi^2 = 0.02$ (treat exploration noise as extra Gaussian noise)? And with $\sigma_\pi^2 = 0.002$?
7. **(Code.)** Modify `tx_step_pg` so that the losses are fed back with a one-bit quantiser ("above or below the batch median"). Predict, then test, whether training still works.

Answers

1. $\mathbb{E}\left\|\mathbf{x}_p\right\|^2 = (1-\sigma_\pi^2)\mathbb{E}\left\|\mathbf{x}\right\|^2 + 2\sqrt{1-\sigma_\pi^2}\,\mathrm{Re}\,\mathbb{E}[\mathbf{x}^{\mathsf{H}}\mathbf{w}] + \mathbb{E}\left\|\mathbf{w}\right\|^2 = (1-\sigma_\pi^2)N + 0 + N\sigma_\pi^2 = N$. The cross term vanishes because $\mathbf{w}$ is zero-mean and independent of $\mathbf{x}$. Without the scaling: energy $1 + \sigma_\pi^2 = 1.02$, i.e. $10\log_{10} 1.02 = 0.086$ dB above one — small, but a systematic cheat.
2. $a = \sqrt{0.9} = 0.94868$; $af = 0.56921 + 0.18974j$; $x_p - af = 0.13079 - 0.08974j$; multiply by $2a/\sigma_\pi^2 = 18.974$: $\nabla_{\mathbf{f}} \log \pi = (2.4816, -1.7027)$.
3. $\mathbb{E}[\nabla \log \pi] = \int \pi \frac{\nabla \pi}{\pi}\,\mathrm{d}a = \nabla \int \pi\,\mathrm{d}a = \nabla 1 = \mathbf{0}$. Hence $\mathbb{E}[(l-b)\nabla \log \pi] = \mathbb{E}[l \nabla \log \pi] - b\,\mathbb{E}[\nabla \log \pi] = \mathbb{E}[l \nabla \log \pi]$.
4. $x_p = 0.69497$ and 0.29497; losses 0.09304 and 0.49706; scores $\pm 2(0.98995)(0.2)/0.02 = \pm 19.799$; products 1.8421 and -9.8413; average -3.9996 — four times the true gradient

-0.9999. So the two-sample estimate is *not* exact here. Reason: with $c = a\theta$, a symmetric pair gives $\frac{2a}{\sigma_\pi^2} \cdot \frac{L(c+w)-L(c-w)}{2}\, w = \frac{2a}{\sigma_\pi^2}\, 2(c-1)\, w^2$, which equals the true gradient $2a(c-1)$ only if $w^2 = \sigma_\pi^2/2$, the policy's variance per real dimension. The worked example happened to use exactly $w^2 = 0.01 = \sigma_\pi^2/2$; $w = \pm 0.2$ has four times that. Averaged over $w \sim \mathcal{N}(0, \sigma_\pi^2/2)$, $\mathbb{E}[w^2] = \sigma_\pi^2/2$ and the estimator is unbiased; individual samples are not.

5. (a) Supervised: TX needs $\partial l/\partial \mathbf{x}$, i.e. receiver gradients *and* the channel Jacobian; RX needs labels m. Only the labels are realistic. (b) Alternating: RX needs labels m (a known training sequence); TX needs its own messages, its own sampled noise $\mathbf{w}$, and the scalar losses fed back. All are obtainable; the feedback link costs uplink capacity.

6. Exploration noise power relative to signal: the clean signal has power 0.98, the receiver noise $10^{-2.5} = 0.00316$, the exploration noise 0.02 (before the channel; same gain as the signal on AWGN). Effective SNR $= 0.98/(0.02 + 0.00316) = 42.3$, i.e. 16.3 dB. With $\sigma_\pi^2 = 0.002$: $0.998/(0.002 + 0.00316) = 193$, i.e. 22.9 dB. Exploration matters a lot at high SNR.

7. Replace `l` by `(l > l.median()).float() - 0.5`. The estimate is now the score weighted by a sign; its expectation is no longer exactly proportional to the gradient, but it still correlates positively with it (bad probes are pushed away, good ones pulled towards), so training usually still works, more slowly. This is the idea behind "rank-based" evolution strategies.

Self-test quiz

1. Which factor in $\partial l/\partial \boldsymbol{\theta}_T$ is unavailable for a real channel?
   Answer: the channel Jacobian $\partial \mathbf{y}/\partial \mathbf{x}$.

2. Why does the receiver not need reinforcement learning?
   Answer: its loss depends on its parameters only through its own network; it sees $\mathbf{y}$ and the label directly, so ordinary backprop works.

3. What is the purpose of the factor $\sqrt{1-\sigma_\pi^2}$ in $\mathbf{x}_p$?
   Answer: to keep the average transmit energy equal to one after adding exploration noise.

4. What exactly is fed back from receiver to transmitter during transmitter training?
   Answer: the per-example cross-entropy losses (one real number per transmitted example), over a reliable link.

5. Does the paper report that the model-free method ends up worse than supervised training?
   Answer: no — similar final error rates on AWGN and RBF; slower initial convergence on AWGN, faster with less variance on RBF.

6. Name one cost of choosing σ_π^2 too small and one of choosing it too large.
   Answer: too small $\Rightarrow$ high-variance gradient estimates, slow learning; too large $\Rightarrow$ biased (smoothed) objective and wasted training power.

Further reading

- F. Ait Aoudia and J. Hoydis, "End-to-End Learning of Communications Systems Without a Channel Model", Asilomar 2018, pp. 298–303; arXiv:1804.02276.
- F. Ait Aoudia and J. Hoydis, "Model-Free Training of End-to-End Communication Systems", *IEEE JSAC*, 37(11):2503–2516, 2019; arXiv:1812.05929. Theorem on the $\sigma \to 0$ limit, hardware experiments, noisy feedback.
- S. Dörner, S. Cammerer, J. Hoydis and S. ten Brink, "Deep Learning Based Communication Over the Air", *IEEE J. Sel. Topics Signal Process.*, 2018. Autoencoder on SDRs with receiver fine-tuning.
- T. O'Shea and J. Hoydis, "An Introduction to Deep Learning for the Physical Layer", *IEEE Trans. Cogn. Commun. Netw.*, 2017 (Chapter 21).
- R. J. Williams, "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning", *Machine Learning*, 8:229–256, 1992 (REINFORCE).

Chapter 25

Deep Reinforcement Learning Autoencoder with Noisy Feedback

Chapter 24 showed how to train a transmitter without knowing the channel: perturb what you send, let the receiver score each example, and feed those scores back. The whole construction rested on one convenient assumption — that the scores arrive at the transmitter *exactly*. In a real radio, the scores have to travel back over the air, through the same kind of noisy, fading channel we are trying to learn to communicate over. This chapter studies the paper that asked what happens then, and that answered with a neat idea: the feedback link can itself be *learned*, with the very same algorithm, so that two radios bootstrap each other.

You will learn how noise on the fed-back losses enters the policy-gradient estimate, why it leaves the estimate unbiased but inflates its variance, why there is a threshold below which feedback noise simply does not matter, and how a pair of devices can train a forward link and a feedback link together. As always, we reproduce the key effects ourselves on a CPU.

Paper at a glance

Paper. M. Goutay, F. Ait Aoudia and J. Hoydis, "Deep Reinforcement Learning Autoencoder with Noisy Feedback", *International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt)*, 2019, pp. 1–6 (arXiv:1810.05419). Nokia Bell Labs, Paris-Saclay.

Problem. The model-free alternating training of Chapter 24 assumes a perfectly reliable link that returns per-example losses to the transmitter.

Key idea. (i) Model the feedback errors as additive zero-mean Gaussian noise on the losses and analyse its effect: the gradient estimator stays unbiased, while its variance gains an extra term proportional to the noise variance. (ii) Replace the reliable link by a *learned feedback system* that transmits real numbers, trained with the same alternating RL algorithm, so that two devices train each other's links.

Main result. Up to a loss-noise variance (MSE) of about 10^{-2}, training with noisy feedback achieves the same BLER as with perfect feedback. Communication systems trained through the learned feedback system match those trained with perfect feedback, on AWGN and Rayleigh block fading, given a sufficiently high (and realistic) training SNR.

25.1 The problem: the feedback travels through a channel too

Intuition: a coach shouting across a noisy stadium

In Chapter 24 the transmitter was an athlete trying slightly different moves while a coach shouted a score after each one. Now the stadium is noisy: sometimes the athlete hears "7" when the coach said "5". If the mishearing is random — as likely too high as too low — then over many attempts the athlete still learns which moves the coach prefers; it just takes longer, because some lessons are wrong. Only when the noise is so loud that the scores are mostly

fiction does learning collapse. This chapter puts numbers on "longer" and "so loud".

Recall the transmitter update of Chapter 24. The transmitter sends a perturbed signal $\mathbf{x}_p^{(i)} = \sqrt{1-\sigma_c^2}\, f_{\theta_T}^{(T)}(m^{(i)}) + \mathbf{w}^{(i)}$, $\mathbf{w}^{(i)} \sim \mathcal{CN}(\mathbf{0}, \sigma_c^2 \mathbf{I})$, and (we follow this paper's notation: subscript c for the communication system, σ_c^2 for its exploration variance and S_c for its batch size) updates its parameters with

$$\nabla_{\theta_T} J = \frac{1}{S_c} \sum_{i=1}^{S_c} l^{(i)}\, D^{(i)}, \qquad D^{(i)} \triangleq \nabla_{\theta_T} \log \pi_{\theta_T}\big(\mathbf{x}_p^{(i)} \mid m^{(i)}\big). \tag{25.1}$$

$D^{(i)}$, the *score*, is computed entirely at the transmitter (it knows the message, its own network and the noise it drew). The losses $l^{(i)}$ are computed at the receiver. They must cross from receiver to transmitter, and that crossing is the topic of this chapter.

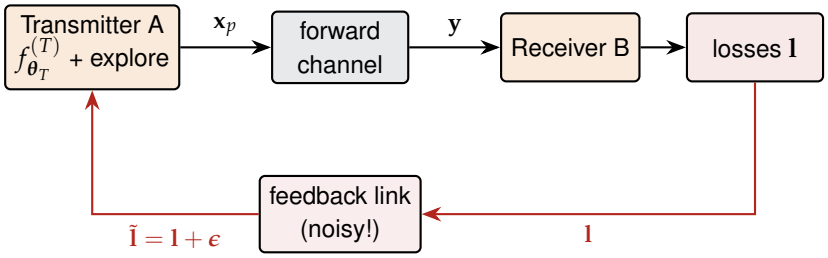


Figure 25.1: Training a communication system with a feedback link. Device A's transmitter explores; device B's receiver scores each example with its cross-entropy loss; the vector of losses $\mathbf{l}$ crosses a feedback link and arrives as $\tilde{\mathbf{l}}$. Chapter 24 assumed $\tilde{\mathbf{l}} = \mathbf{l}$. Here $\tilde{\mathbf{l}}$ is corrupted.

25.2 Background: what noise does to a stochastic gradient

Before touching the paper's equations, two facts from probability (Chapter 4) do most of the work.

Definition: unbiased estimator

A random vector $\hat{\mathbf{g}}$ is an *unbiased* estimator of $\mathbf{g}$ if $\mathbb{E}[\hat{\mathbf{g}}] = \mathbf{g}$. Stochastic gradient descent with an unbiased gradient estimate converges (with a suitable learning rate) to the same place as exact gradient descent would; noise only slows it down.

Definition: cumulative element-wise variance

For a random vector $\hat{\mathbf{g}} \in \mathbb{R}^P$, the paper measures the spread by the sum of the variances of its components, $V = \sum_{k=1}^{P} \mathrm{Var}(\hat{g}_k) = \mathbb{E}\,\|\hat{\mathbf{g}} - \mathbb{E}\hat{\mathbf{g}}\|_2^2$. It is the "total noise power" of the gradient estimate.

Fact 1 (zero-mean noise does not bias a product with an independent factor). If ϵ is independent of D and $\mathbb{E}[\epsilon] = 0$, then $\mathbb{E}[(l+\epsilon)D] = \mathbb{E}[lD] + \mathbb{E}[\epsilon]\mathbb{E}[D] = \mathbb{E}[lD]$.

Fact 2 (independent noise adds variance). The variance of a sum of independent, zero-mean-difference terms is the sum of their variances; and the variance of an average of S i.i.d. terms is $1/S$ times the variance of one.

25.3 System model: losses corrupted by Gaussian noise

The paper first abstracts the feedback link into a single number, its *mean squared error*. The losses that reach the transmitter are

Key equation: noisy loss feedback model

$$\tilde{\mathbf{l}} = \mathbf{l} + \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}\left(\mathbf{0}, \sigma_l^2 \mathbf{I}_{S_c}\right), \tag{25.2}$$

so that $\sigma_l^2 = \mathbb{E}[(\tilde{l} - l)^2]$ is exactly the MSE between sent and received losses. The noise is independent of everything else. The transmitter, not knowing the difference, uses $\tilde{l}^{(i)}$ in place of $l^{(i)}$ in (25.1):

$$\nabla_{\theta_T} \tilde{J} = \frac{1}{S_c} \sum_{i=1}^{S_c} \tilde{l}^{(i)} D^{(i)} = \nabla_{\theta_T} J + \frac{1}{S_c} \sum_{i=1}^{S_c} \epsilon^{(i)} D^{(i)}. \tag{25.3}$$

The second form makes the damage explicit: an extra random vector, a noise-weighted sum of scores, is added to the clean estimate.

A convenient way to read σ_l^2 is as a feedback SNR. The journal version of Chapter 24's paper defines $\mathrm{SNR}_{fb} = \mathbb{E}[l^2]/\sigma_\epsilon^2$ for exactly this noise model. In our reproduction a typical trained system has $\mathbb{E}[l^2] \approx 0.87$, so $\sigma_l^2 = 10^{-2}$ corresponds to $\mathrm{SNR}_{fb} \approx 19$ dB and $\sigma_l^2 = 1$ to about -0.6 dB (Figure 25.2).

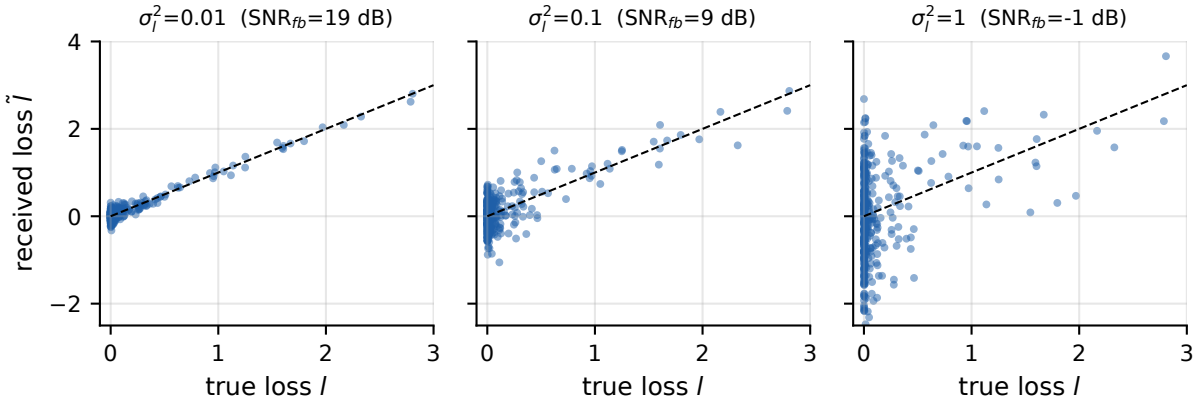


Figure 25.2: Our reproduction: how feedback noise corrupts the reward signal. Each dot is one transmitted example; horizontal axis is the loss the receiver computed, vertical axis the loss the transmitter received under (25.2). Most true losses are close to zero (the receiver is usually right); a few are large (errors). At $\sigma_l^2 = 0.01$ the received values hug the diagonal; at $\sigma_l^2 = 1$ the small losses are completely buried and only the large ones remain recognisable — yet those large ones are exactly the informative "this probe was bad" messages.

25.4 Analysis: unbiased, but noisier

25.4.1 No bias

Take the expectation of (25.3). By Fact 1, since each $\epsilon^{(i)}$ is zero-mean and independent of $D^{(i)}$,

$$\mathbb{E}\left[\nabla_{\theta_T} \tilde{J}\right] = \mathbb{E}\left[\nabla_{\theta_T} J\right] + \frac{1}{S_c} \sum_i \mathbb{E}[\epsilon^{(i)}]\, \mathbb{E}[D^{(i)}] = \mathbb{E}\left[\nabla_{\theta_T} J\right]. \tag{25.4}$$

This is the paper's first observation: with zero-mean feedback errors, the loss-function gradient estimator is not biased. On average the transmitter still walks downhill.

25.4.2 Extra variance

Now the spread. Drop the example index and write one term as $\tilde{l}D = lD + \epsilon D$. Its cumulative variance is

$$\begin{aligned}\mathbb{E}\left\|\tilde{l}D - \mathbb{E}[\tilde{l}D]\right\|^2 &= \mathbb{E}\left\|(lD - \mathbb{E}[lD]) + \epsilon D\right\|^2 \qquad (\text{since } \mathbb{E}[\tilde{l}D] = \mathbb{E}[lD])\\ &= \mathbb{E}\left\|lD - \mathbb{E}[lD]\right\|^2 + 2\,\mathbb{E}[\epsilon]\,\mathbb{E}\left[(lD - \mathbb{E}[lD])^{\mathsf{T}} D\right] + \mathbb{E}[\epsilon^2]\,\mathbb{E}\left\|D\right\|^2\\ &= \mathbb{E}\left\|lD - \mathbb{E}[lD]\right\|^2 + \sigma_l^2\,\mathbb{E}\left\|D\right\|^2,\end{aligned}$$

where the second line uses the independence of ϵ from l and D, and the third uses $\mathbb{E}[\epsilon] = 0$ and $\mathbb{E}[\epsilon^2] = \sigma_l^2$. Averaging S_c i.i.d. terms divides by S_c (Fact 2), which gives the paper's decomposition:

Key equation: variance of the gradient estimate with noisy feedback

$$V = \frac{1}{S_c}\Big(\underbrace{\mathbb{E}\left\|lD - \mathbb{E}[lD]\right\|_2^2}_{A\text{: noise-free variance}} + \underbrace{\sigma_l^2\,\mathbb{E}\left\|D\right\|_2^2}_{B\text{: feedback noise}}\Big). \tag{25.5}$$

25.4.3 What is $\mathbb{E}\left\|D\right\|^2$?

From Chapter 24, eq. (24.11), $D = \frac{2\sqrt{1-\sigma_c^2}}{\sigma_c^2}\mathbf{G}^{\mathsf{T}}\mathbf{w}$, where $\mathbf{G} = \nabla_{\boldsymbol{\theta}_T} f_{\boldsymbol{\theta}_T}^{(T)}(m)$ is the transmitter's Jacobian ($2N \times P$ in real form) and $\mathbf{w}$ is the drawn exploration noise, with variance $\sigma_c^2/2$ in each of its $2N$ real components. Then

$$\mathbb{E}\left\|D\right\|^2 = \frac{4(1-\sigma_c^2)}{\sigma_c^4}\,\mathbb{E}\left[\mathbf{w}^{\mathsf{T}}\mathbf{G}\mathbf{G}^{\mathsf{T}}\mathbf{w}\right] = \frac{4(1-\sigma_c^2)}{\sigma_c^4}\cdot\frac{\sigma_c^2}{2}\,\mathbb{E}\left\|\mathbf{G}\right\|_F^2,$$

using $\mathbb{E}[\mathbf{w}^{\mathsf{T}}\mathbf{A}\mathbf{w}] = \frac{\sigma_c^2}{2}\operatorname{tr}(\mathbf{A})$ and $\operatorname{tr}(\mathbf{G}\mathbf{G}^{\mathsf{T}}) = \left\|\mathbf{G}\right\|_F^2$. So in our derivation $B = \frac{2(1-\sigma_c^2)\sigma_l^2}{\sigma_c^2}\mathbb{E}\left\|\mathbf{G}\right\|_F^2$. The paper prints term B as $\frac{4\sigma_l^2}{\sigma_c^4}\mathbb{E}\left\|\nabla_{\boldsymbol{\theta}_T} f_{\boldsymbol{\theta}_T}^{\mathsf{T}}(m)\right\|_F^2$, i.e. with a different power of σ_c; we could not reconcile the constant from the short derivation available to us (it may come from a different convention for the noise or for D). It does not change any conclusion: **term B grows linearly with the feedback noise variance σ_l^2, and grows as the exploration variance σ_c^2 shrinks.** Small exploration amplifies feedback noise, because the transmitter divides by σ_c^2 to turn small probes into a gradient.

25.4.4 The threshold behaviour

Equation (25.5) predicts two regimes. While $B \ll A$, the estimate's variance is dominated by the inherent randomness of the probes and of the forward channel, and feedback noise is invisible. When σ_l^2 grows past the point where $B \approx A$, V starts rising linearly with σ_l^2. The paper's numerical evaluation (its Fig. 2, with $M = 256$, $N_c = 4$, $\sigma_c^2 = 0.02$, $S_c = 10^5$) shows exactly this: V is not significantly affected up to some value of σ_l^2, beyond which it increases linearly. It also notes that V decreases over the course of training, because training lowers the losses and hence their spread.

Intuition: why small losses make feedback noise easy to ignore

Term A is the variance of lD. Early in training many losses are large and wildly different, so A is huge and a little feedback noise is nothing by comparison. Late in training most losses are near zero; A shrinks, and the same feedback noise becomes relatively more important. Feedback quality matters most at the end, when the system is fine-tuning.

Worked example: variance bookkeeping with numbers

Suppose for one parameter the clean per-example products lD have variance $A_1 = 40$ and $\mathbb{E}[D^2] = 9$. With $S_c = 256$ examples: perfect feedback gives $V_1 = 40/256 = 0.156$. With $\sigma_l^2 = 0.01$: $V_1 = (40 + 0.09)/256 = 0.157$ — a 0.2% increase, invisible. With $\sigma_l^2 = 1$: $(40 + 9)/256 = 0.191$, +23%. With $\sigma_l^2 = 10$: $(40 + 90)/256 = 0.508$, more than three times the original. To restore the original variance at $\sigma_l^2 = 10$ you would need $S_c = 256 \times 130/40 = 832$ examples per step. *Feedback noise can always be traded for batch size* — i.e. for more channel uses.

25.5 The proposed approach: learn the feedback link too

Knowing that a feedback MSE of about 10^{-2} is good enough, the paper's second contribution asks: can we build such a link without assuming one exists? Its answer: two devices, A and B, each have a transmitter and a receiver. Transmitter A → receiver B is the *communication system* we want to train. Transmitter B → receiver A is a *feedback system* whose job is to carry real numbers. Train the feedback system with the same alternating algorithm, then use it to carry the losses.

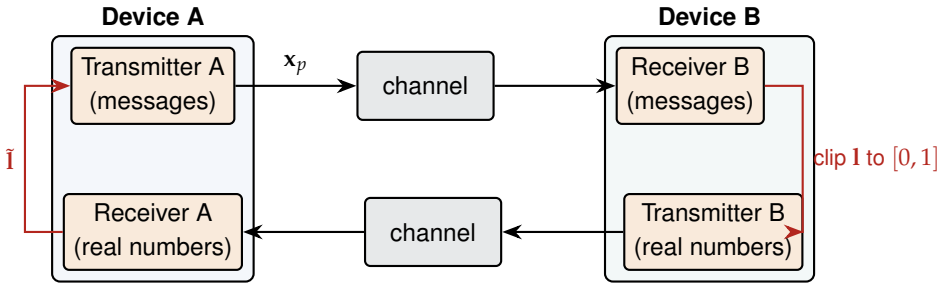


Figure 25.3: Training the communication system A→B using a learned feedback system B→A. Receiver B computes the per-example losses, clips them to $[0, 1]$, and transmitter B sends them as real numbers; receiver A recovers $\tilde{\mathbf{l}}$, which drives the policy-gradient update of transmitter A. The roles swap when training the feedback system.

25.5.1 The feedback system: transmitting real numbers

The feedback system maps a real number r to N_f complex channel symbols and back. Its receiver is trained by supervised learning to minimise the MSE between r and its estimate $\hat{r}$. Its transmitter is trained by RL with the same Gaussian exploration, $\mathbf{X}_p = \sqrt{1 - \sigma_f^2}\,\mathbf{X} + \mathbf{W}$ with $\mathbf{W}$ circular complex Gaussian of variance σ_f^2, where now the per-example "losses" are squared errors $(\hat{r} - r)^2$. Those squared errors need to travel back to transmitter B — over the communication system A→B! The two systems are therefore trained in turn (the paper's Algorithm 3): an inner loop alternating TRAINTRANSMITTERA/TRAINRECEIVERB, then an inner loop alternating TRAINTRANSMITTERB/TRAINRECEIVERA, repeated until a stop criterion. During training the

numbers r are drawn uniformly in $[0,1]$; to match these conditions, the per-example losses of the communication system are clipped to $[0,1]$ before being sent.

Block	Layers (as reported)	Output
Feedback transmitter	dense $10N_f$, ELU; dense $2N_f$, linear; energy normalisation	N_f complex
Feedback receiver (AWGN)	dense $10N_f$, ReLU; dense 1, linear	$\hat{r}$
Feedback receiver (RBF)	radio-transformer-network receiver; second-to-last layer $10N_f$ units; last layer 1 unit, linear	$\hat{r}$
Communication system	same architecture as in Chapter 24's paper	

Table 25.1: Architectures of the paper. Hyperparameters reported: $M = 256$; $N_c = N_f = 4$ (AWGN) or 5 (RBF); batch sizes $S_c = S_f = 10^5$; $\sigma_c^2 = \sigma_f^2 = 0.02$; Adam; training at 10 dB (AWGN) and 20 dB (RBF); $r \sim \mathcal{U}(0,1)$.

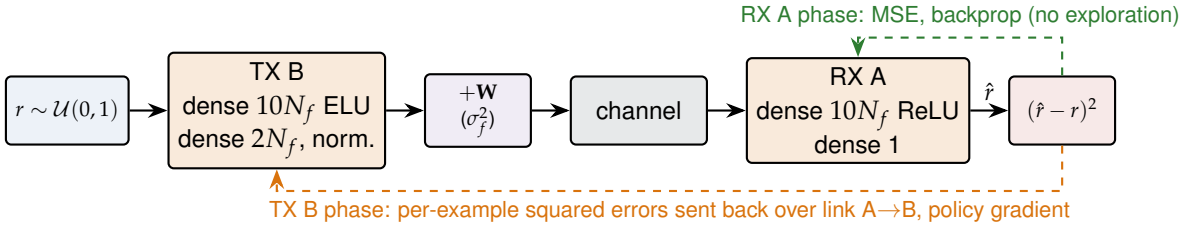


Figure 25.4: Training the feedback system (paper's Fig. 5 idea, redrawn). It is the same alternating algorithm as in Chapter 24, with a regression loss: receiver A minimises the MSE between r and $\hat{r}$; transmitter B explores with Gaussian perturbations and is updated from per-example squared errors. Those errors travel over the communication system A→B — the two links take turns being each other's feedback.

25.5.2 The analog baseline

To judge the learned feedback system the paper compares it with the simplest possible way to send a real number: put it, centred and scaled to unit power, on both I and Q of every channel use, repeated over N_f uses,

$$\mathbf{x} = \frac{r - \mathbb{E}[r] + j\,(r - \mathbb{E}[r])}{\sqrt{2\,\mathrm{var}(r)}}\,\mathbf{1}_{N_f}, \tag{25.6}$$

and average the received copies (on RBF the paper uses pilot-based equalisation for the analog scheme). For AWGN this is easy to analyse. Each of the $2N_f$ real copies of the scaled value carries noise of variance $\sigma^2/2$ (with $\sigma^2 = 1/\mathrm{SNR}$); averaging divides that by $2N_f$; undoing the scaling multiplies it by $2\,\mathrm{var}(r)$. Hence

$$\mathrm{MSE}_{\text{analog}} = 2\,\mathrm{var}(r) \cdot \frac{\sigma^2/2}{2N_f} = \frac{\mathrm{var}(r)}{2N_f\,\mathrm{SNR}}. \tag{25.7}$$

Worked example: how good is the analog feedback?

With $r \sim \mathcal{U}(0,1)$, $\mathrm{var}(r) = 1/12$; with $N_f = 4$: MSE $= \frac{1}{96\,\mathrm{SNR}}$. At 10 dB: $1/960 = 1.04 \cdot 10^{-3}$; at 0 dB: $1.04 \cdot 10^{-2}$; at -10 dB: 0.104. So, on AWGN, analog repetition already crosses the paper's 10^{-2} threshold at about 0 dB — consistent with the paper's finding that above 0 dB either system can replace a perfect feedback link. (Our simulation in Section 25.7 measured $1.04 \cdot 10^{-3}$, $1.04 \cdot 10^{-2}$ and 0.104.)

25.6 Results reported in the paper

Effect of noise on training (Section III). On AWGN at 10 dB with $M = 256$, $N_c = 4$, $\sigma_c^2 = 0.02$, $S_c = 10^5$: the variance V is flat for small σ_l^2 and then rises linearly (paper Fig. 2). The BLER of systems trained with noisy feedback (paper Fig. 3) equals that with perfect feedback up to $\sigma_l^2 \approx 10^{-2}$, and increases quickly beyond.

The feedback system (paper Fig. 6). On AWGN, the learned feedback system and the analog scheme achieve similar MSE, and above 0 dB both have MSE below 10^{-2}, so either could replace a perfect feedback link. On RBF, the learned feedback system outperforms analog transmission, with a maximum gain of about 3 dB around an MSE of 10^{-2}; at 10 dB it provides an MSE of 10^{-2}.

The full system (paper Fig. 7). Communication systems trained using the learned feedback system achieve the same BLER as those trained with perfect feedback. On AWGN the learned system outperforms QPSK but has slightly higher BLER than the Agrell sphere-packing constellation; on RBF it outperforms both QPSK and Agrell.

25.7 Our small-scale reproduction

Our code (`code/ch25/`) is self-contained and re-implements the alternating training at a smaller scale: $M = 16$ messages over $N_c = 2$ complex channel uses on AWGN, trained at 7 dB, $\sigma_c^2 = 0.02$, batch 256, Adam (learning rates 10^{-2} receiver, $3 \cdot 10^{-3}$ transmitter). With $S_c = 256$ instead of the paper's 10^5 our gradient estimates are far noisier, so the threshold will sit at a different σ_l^2 — but the shape of the effects should be the same. The feedback path is a single function:

Listing 25.1: Feedback models (`code/ch25/rlfb.py`). `gauss` is the paper's abstract noise model; `analog` implements (25.6) over N_f AWGN channel uses; `learned` uses a trained feedback transmitter/receiver pair.

```
@torch.no_grad()
def send_back(l, kind, var_l=0.0, Nf=4, fb_snr_db=10.0, link=None):
    """What transmitter A receives when receiver B feeds back l."""
    if kind == "perfect":
        return l
    if kind == "gauss":
        return l + torch.randn_like(l) * math.sqrt(var_l)
    lc = l.clamp(0, 1)
    if kind == "analog":
        # paper eq. (6), r ~ U(0,1): same value on I and Q of all Nf uses
        mu, var = 0.5, 1 / 12
        amp = (lc - mu) / math.sqrt(2 * var)
        x = torch.cat([amp[:, None].repeat(1, Nf), amp[:, None].repeat(1, Nf)], 1)
        y = awgn(x, fb_snr_db)
        return y.mean(1) * math.sqrt(2 * var) + mu   # average 2Nf copies
    if kind == "learned":
        ftx, frx = link
        return frx(awgn(ftx(lc), fb_snr_db))
    raise ValueError(kind)
```

The training loop is Chapter 24's, with one change in the transmitter step: the losses pass through `send_back` before they multiply the score.

Listing 25.2: Alternating training with a feedback model (`code/ch25/rlfb.py`). Line by line it is the algorithm of Figure 25.1: a supervised receiver step, then a transmitter step whose reward signal is whatever survived the feedback link.

```
def train_alternating(M, N, snr_db, iters, fb="perfect", B=256, sig2=0.02, s=1,
                      eval_every=0, clip=False, **fbargs):
    seed(s)
    tx, rx = Tx(M, N), Rx(M, N)
    ot = torch.optim.Adam(tx.parameters(), 3e-3)
    orr = torch.optim.Adam(rx.parameters(), 1e-2)
    a = math.sqrt(1 - sig2)
    curve = []
    for it in range(iters + 1):
        if eval_every and it % eval_every == 0:
            curve.append(bler(tx, rx, M, snr_db, n=20000))
        if it == iters:
            break
        # receiver step: supervised, local (no feedback needed)
        m = torch.randint(0, M, (B,))
        with torch.no_grad():
            y = awgn(tx(m), snr_db)
        loss = F.cross_entropy(rx(y), m)
        orr.zero_grad(); loss.backward(); orr.step()
        # transmitter step: per-example losses travel back over the feedback link
        m = torch.randint(0, M, (B,))
        x = tx(m)
        with torch.no_grad():
            xp = a * x + torch.randn_like(x) * math.sqrt(sig2 / 2)
            l = F.cross_entropy(rx(awgn(xp, snr_db)), m, reduction="none")
            if clip:
                l = l.clamp(0, 1)
            lt = send_back(l, fb, **fbargs)
        logpi = -((xp - a * x) ** 2).sum(1) / sig2
        sur = (lt * logpi).mean()
        ot.zero_grad(); sur.backward(); ot.step()
    return tx, rx, curve
```

25.7.1 Experiment 1: Gaussian loss noise

Figure 25.5(a) estimates V directly: we freeze a partially trained system, draw 40 independent batches, compute the policy-gradient estimate for each, and sum the per-component variances. It reproduces the paper's threshold shape: V is flat (42.6) up to $\sigma_l^2 = 10^{-3}$, barely moves at 10^{-2} (43.1) and 10^{-1} (48.0), then grows linearly: 98.6 at $\sigma_l^2 = 1$ and 610 at 10. Panel (b) trains a fresh system for 700 iterations at each noise level. Up to $\sigma_l^2 = 0.1$ the final BLER is the same as with perfect feedback (all within 4.8–$5.0 \cdot 10^{-2}$; the small differences are run-to-run noise), while at $\sigma_l^2 = 1$ it degrades to $6.3 \cdot 10^{-2}$.

Why is our knee at a larger σ_l^2 than the paper's? Term A in (25.5) measures how noisy the estimate already is without feedback errors. Our small system, small batch and short training keep A relatively large, so it takes more feedback noise to matter. Also note that our BLER panel uses one seed per point, so differences below about 5% relative are not meaningful.

25.7.2 Experiment 2: losses sent over a noisy channel

Next we make the feedback physical. The receiver clips its losses to $[0, 1]$ and sends them with the analog scheme (25.6) over $N_f = 4$ AWGN channel uses at a feedback SNR of $+10$, 0, -10

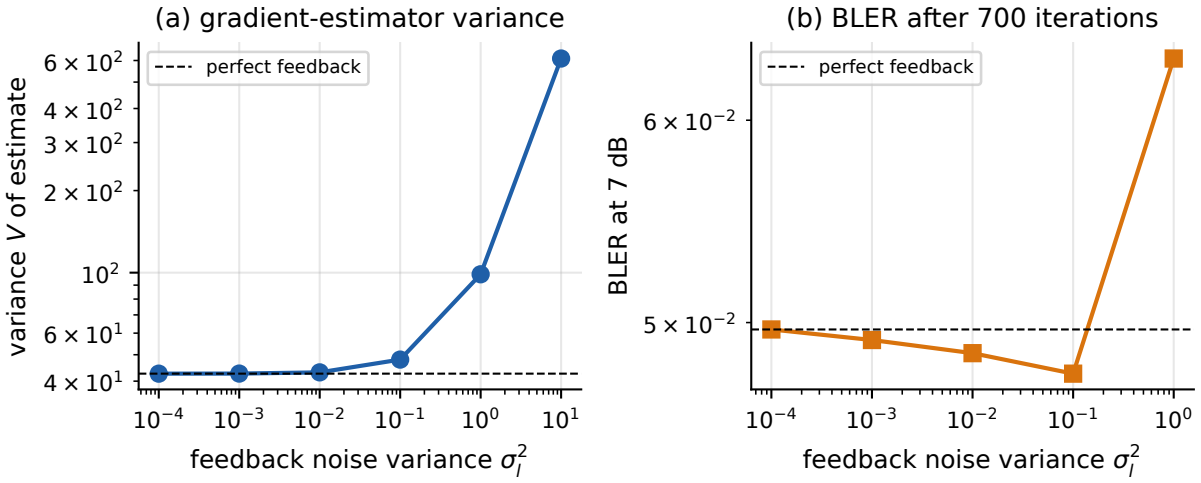


Figure 25.5: Our reproduction of the paper's Section III experiment. (a) Cumulative variance V of the transmitter gradient estimate (40 batches of 256) vs feedback noise variance: flat, then linear — the two regimes of (25.5). (b) BLER at 7 dB after 700 training iterations with noisy losses: no penalty up to $\sigma_l^2 \approx 0.1$, clear degradation at $\sigma_l^2 = 1$. The paper finds the knee at $\sigma_l^2 \approx 10^{-2}$ with its larger system ($M = 256$); the position of the knee depends on term A, i.e. on the system, batch size and training stage.

or -20 dB. Figure 25.6 shows the result. The measured feedback MSEs are $1.0 \cdot 10^{-3}$, $1.0 \cdot 10^{-2}$, 0.10 and 1.04, matching (25.7). For $+10$ and 0 dB the trained systems are indistinguishable from perfect feedback over the whole SNR range. At -10 dB (MSE 0.1) there is a small loss (BLER $4 \cdot 10^{-4}$ vs $3 \cdot 10^{-4}$ at 12 dB). At -20 dB (MSE about 1) the loss is clear: about 1 dB, and visibly slower training.

Pitfall: clipping is not free

Clipping to $[0, 1]$ throws away information: a loss of 3 (a confident wrong decision) and a loss of 1 both arrive as 1. Early in training many losses exceed one, so the clipped feedback under-weights the worst probes. In our runs this did not hurt (compare the perfect-feedback BLERs of Figures 25.5 and 25.6), but with a different loss scale you may need to rescale rather than clip, or use the receiver's error indicator instead of its cross-entropy.

25.7.3 Experiment 3: a learned feedback link

Finally we train a feedback system for real numbers, with the paper's layer sizes (Table 25.1), $N_f = 4$, using alternating training at a feedback SNR of 0 dB (to save time, its own squared errors are assumed to reach transmitter B; in the paper they would travel over the A→B link). We then use it to carry the clipped losses.

The learned feedback link reaches an MSE of $9.7 \cdot 10^{-3}$ at 0 dB (analog: $1.05 \cdot 10^{-2}$) and $7.2 \cdot 10^{-2}$ at -10 dB (analog: 0.105), so on AWGN the two are very close, as the paper reports. Above the training SNR our learned link has an error floor of about $1.7 \cdot 10^{-3}$, whereas analog repetition keeps improving — a reminder that a network trained at one operating point need not generalise to others. Panel (b) shows why the two are so similar on AWGN: the learned transmitter maps r almost linearly onto a fixed direction, which is what analog modulation does; for a Gaussian channel and a uniform source, little more is possible with four channel uses. Used as the feedback path at 0 dB, the learned link gives a forward-link BLER close to perfect feedback

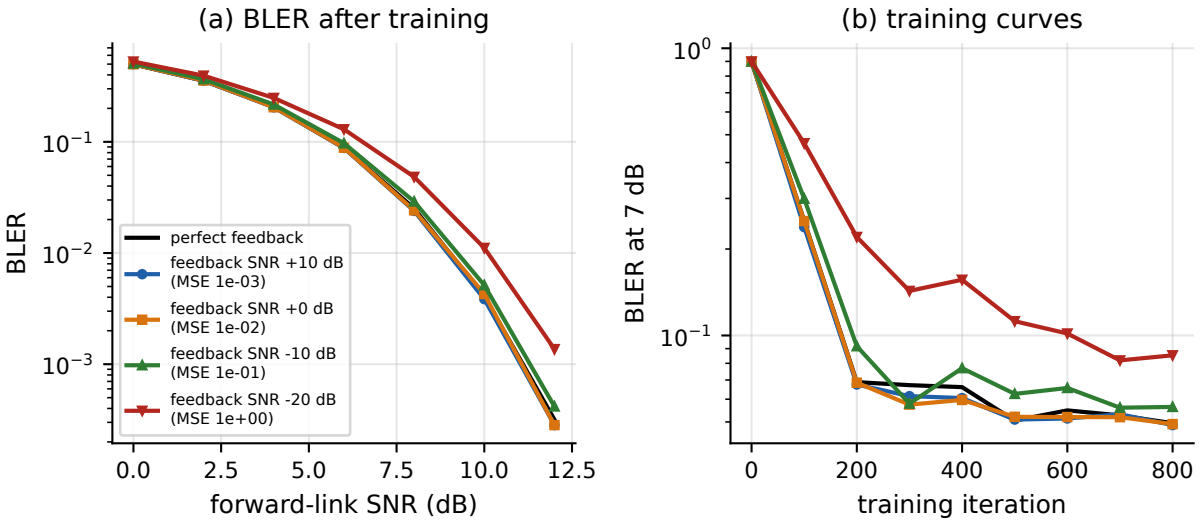


Figure 25.6: Our reproduction: alternating training with losses clipped to $[0, 1]$ and fed back by analog repetition over $N_f = 4$ AWGN channel uses. (a) BLER vs forward SNR after 800 iterations, for perfect feedback and four feedback SNRs (feedback MSE in brackets). (b) Training curves (BLER at 7 dB). Down to a feedback SNR of 0 dB the curves are on top of the perfect-feedback curve; only at -20 dB, where the feedback MSE is about 1, does training slow down and the final BLER rise.

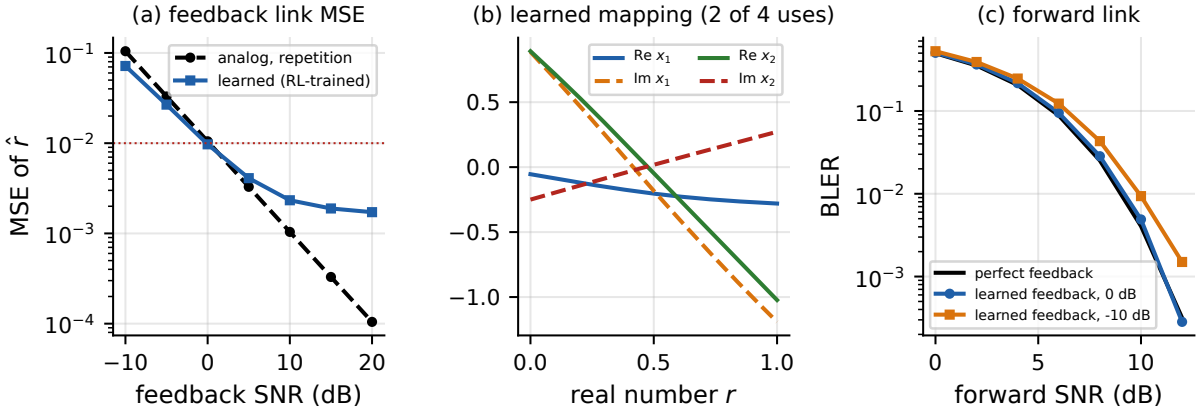


Figure 25.7: Our reproduction with a learned feedback system ($N_f = 4$, trained at 0 dB). (a) Feedback MSE vs SNR for the learned link and analog repetition on AWGN; the dotted line marks 10^{-2}. Around the training SNR (0 dB) both are at about 10^{-2}; the learned link is slightly better below 0 dB and saturates above it (an error floor from training at a single SNR with a small network). (b) Two of the eight real outputs of the learned feedback transmitter as functions of r: nearly linear, i.e. it rediscovered analog transmission with a learned direction in $\mathbb{R}^8$. (c) Forward-link BLER with perfect feedback and with learned feedback at 0 dB and -10 dB.

($3 \cdot 10^{-4}$ at 12 dB for both; slightly higher at intermediate SNRs, e.g. $4.9 \cdot 10^{-3}$ vs $4.1 \cdot 10^{-3}$ at 10 dB); at -10 dB the penalty grows to roughly 1 dB. The paper's stronger result on RBF, where its learned link beats analog by up to about 3 dB, we did not attempt to reproduce.

Wireless connection: a feedback SNR of 0 dB is generous — or is it?

Our "0 dB" means the feedback samples arrive at unit SNR, but each loss uses $N_f = 4$ complex channel uses. For a batch of 256 examples that is 1024 complex symbols of feedback per transmitter step — about one OFDM symbol on a 100 MHz NR carrier with 30 kHz spacing (273 PRBs $\times$ 12 subcarriers $=$ 3276 resource elements). Cheap per step, but training needs thousands of steps, and the paper's batches of 10^5 examples would need about 30 OFDM symbols of feedback per step. A practical design would quantise and compress the losses, or feed back only every few steps.

25.8 Limitations and follow-ups

- **Gaussian, independent errors.** The analysis assumes zero-mean noise independent of the losses. Real feedback errors may be biased (e.g. clipping, quantisation, or a feedback receiver with a systematic error) or bursty (fading). A biased feedback signal biases the gradient — unlike Gaussian noise, it is not fixed by bigger batches.
- **Bootstrapping.** The two links train each other; at the very start both are random and the feedback is meaningless. The paper's alternating loops work in simulation; robust start-up procedures over real hardware are not studied.
- **Overhead.** Every training example costs N_f feedback channel uses; the paper's batches (10^5) make this significant. Variance-reduction (baselines), loss quantisation and less frequent feedback are natural follow-ups.
- **Beyond.** Later work removed the feedback entirely, e.g. training only the receiver around a fixed or slowly adapted transmitter, or using differentiable *learned* channel models, at the cost of other assumptions.

25.9 From paper to practice in 5G NR/6G

- **Feedback already exists.** NR has many scalar feedback channels: CQI, RSRP/SINR reports, HARQ-ACK. They are heavily quantised (4-bit CQI, 1-bit ACK) and protected by coding — i.e. "reliable but coarse". The analysis here suggests coarse-but-unbiased feedback is acceptable if the batch is large enough; a 1-bit ACK/NACK is essentially a quantised block-error loss.
- **Two-sided AI models.** 3GPP's Rel-18/19 study of AI/ML for the air interface distinguishes one-sided from two-sided models (e.g. CSI compression with an encoder at the UE and a decoder at the gNB). Joint training of two-sided models across vendors faces exactly the issue studied here: how to exchange training signals between the two sides over a real link.
- **Analog vs digital feedback.** The learned feedback link is an analog (uncoded) scheme. In a standard, the losses would more likely be quantised and sent on PUCCH/PUSCH with standard coding; then the relevant "MSE" is the quantisation error plus residual decoding errors, and the threshold $\sigma_l^2 \approx 10^{-2}$ translates into a required number of bits per loss (for a

uniform quantiser on $[0,1]$, MSE $= \Delta^2/12$ with step $\Delta = 2^{-b}$, so 2 bits already give $5.2 \cdot 10^{-3}$ and 3 bits $1.3 \cdot 10^{-3}$).

25.10 Recap, exercises and quiz

Chapter recap

- Model-free alternating training needs the per-example losses at the transmitter; in practice they arrive through a noisy channel.
- With additive zero-mean noise $\tilde{l} = l + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma_l^2)$, the policy-gradient estimate stays unbiased; its variance becomes $V = \frac{1}{S_c}(A + \sigma_l^2 \mathbb{E}\,\|D\|^2)$.
- Hence a threshold: feedback noise is harmless while $\sigma_l^2 \mathbb{E}\,\|D\|^2 \ll A$. The paper finds no BLER loss up to an MSE of about 10^{-2}.
- Small exploration variance amplifies feedback noise ($\|D\|$ scales like $1/\sigma_c$); larger batches compensate for any feedback noise.
- The feedback link can be a learned real-number transmission system, trained with the same alternating RL algorithm; the paper reports performance identical to perfect feedback at realistic training SNRs, and on RBF a learned link beating analog repetition by up to about 3 dB.
- Our reproduction ($M = 16$, $N_c = 2$) shows the flat-then-linear variance, no loss down to a feedback SNR of 0 dB with analog repetition, and a learned link that rediscovers near-analog transmission on AWGN.

Exercises

1. **(Bias.)** Suppose the feedback adds noise with mean $\mu \neq 0$: $\tilde{l} = l + \epsilon$, $\mathbb{E}[\epsilon] = \mu$. Show that the gradient estimator is still unbiased. (Hint: Chapter 24, Exercise 3.) What if the error is *multiplicative*, $\tilde{l} = (1+\delta)l$ with $\mathbb{E}[\delta] = 0$?
2. **(Correlated error.)** Now let $\tilde{l} = l + \beta(l - \bar{l})$ for a constant β (the feedback receiver systematically exaggerates deviations from the mean $\bar{l}$). What happens to the expected gradient?
3. **(Variance arithmetic.)** With $A = 50$, $\mathbb{E}\,\|D\|^2 = 200$ and $S_c = 1000$, find V for $\sigma_l^2 \in \{0, 10^{-3}, 10^{-2}, 10^{-1}\}$. At which σ_l^2 does feedback noise double V?
4. **(Analog MSE.)** Using (25.7), what feedback SNR do you need to reach MSE $= 10^{-2}$ with $N_f = 1$, 4 and 16? How many dB does each doubling of N_f buy?
5. **(Quantised feedback.)** Losses clipped to $[0,1]$ are quantised with b bits by a uniform quantiser (with dither, so the error is uniform and independent). For which b is the MSE below 10^{-2}? Below 10^{-3}? Compare the channel uses needed with the analog scheme at 0 dB.
6. **(Exploration vs feedback noise.)** Using $\mathbb{E}\,\|D\|^2 \propto 1/\sigma_c^2$, by what factor does term B change if σ_c^2 is reduced from 0.02 to 0.005? What does that imply for the tolerable feedback MSE?
7. **(Code.)** In `train_alternating`, subtract the batch mean of $\tilde{l}$ before multiplying by

the score. Does this reduce term A, term B, or both? Test it at $\sigma_l^2 = 1$.

Answers

1. $\mathbb{E}[\tilde{l}D] = \mathbb{E}[lD] + \mathbb{E}[\epsilon]\mathbb{E}[D] = \mathbb{E}[lD] + \mu \cdot \mathbf{0}$, because the score has zero mean (Chapter 24, Exercise 3). A constant offset is just a baseline. Multiplicative: $\mathbb{E}[(1+\delta)lD] = \mathbb{E}[1+\delta]\mathbb{E}[lD] = \mathbb{E}[lD]$ if δ is independent of l and D — still unbiased (but the variance grows with l^2).
2. $\mathbb{E}[\tilde{l}D] = (1+\beta)\mathbb{E}[lD] - \beta\bar{l}\,\mathbb{E}[D] = (1+\beta)\mathbb{E}[lD]$. The expected gradient is scaled by $(1+\beta)$: the direction is right, only the effective learning rate changes. (If $\beta < -1$ the sign flips and the transmitter would ascend the loss.) Biases that are not of this simple form can rotate the gradient.
3. $V = (50 + 200\sigma_l^2)/1000$: 0.050, 0.0502, 0.052, 0.070. Doubling when $200\sigma_l^2 = 50$, i.e. $\sigma_l^2 = 0.25$.
4. $\mathrm{SNR} = \mathrm{var}(r)/(2N_f\,\mathrm{MSE}) = (1/12)/(2N_f \cdot 0.01) = 4.17/N_f$: $N_f = 1$: 4.17 (6.2 dB); $N_f = 4$: 1.04 (0.2 dB); $N_f = 16$: 0.26 (-5.8 dB). Each doubling of N_f buys 3 dB.
5. $\mathrm{MSE} = \Delta^2/12$ with $\Delta = 2^{-b}$: $b = 2$: $5.2 \cdot 10^{-3} < 10^{-2}$; $b = 1$: $2.1 \cdot 10^{-2}$ (too high). For 10^{-3}: $b = 4$ gives $3.3 \cdot 10^{-4}$, $b = 3$ gives $1.3 \cdot 10^{-3}$ (just too high), so $b = 4$. Two bits need at least one QPSK channel use plus coding overhead and some SNR margin; the analog scheme at 0 dB needs four channel uses for MSE 10^{-2}. Digital quantised feedback is competitive.
6. $B \propto \sigma_l^2/\sigma_c^2$ (with our constants; $1/\sigma_c^4$ times σ_c^2 from $\mathbb{E}\,\|\mathbf{w}\|^2$): reducing σ_c^2 by a factor 4 multiplies B by 4, so the tolerable σ_l^2 drops by about 4 (if A stays similar).
7. Subtracting the batch mean of $\tilde{l}$ removes the common-mode part of both l and ϵ; it mostly reduces A (the large "all losses are similar" component) and leaves the per-example noise part of B nearly unchanged (the batch mean of S_c independent noises has variance only σ_l^2/S_c). Expect faster training overall, but the threshold on σ_l^2 moves *down*, because B becomes relatively more important.

Self-test quiz

1. What exactly is fed back from receiver B to transmitter A?
   Answer: the per-example losses (cross-entropy) of the transmitted batch, clipped to $[0, 1]$ when the learned feedback system is used.
2. Why does zero-mean Gaussian noise on the losses not bias the gradient estimate?
   Answer: it is independent of the score and has zero mean, so its contribution averages to zero.
3. Which term of V depends on the feedback noise and how?
   Answer: term $B = \sigma_l^2\mathbb{E}\,\|D\|^2$, linear in σ_l^2.
4. Up to what feedback MSE does the paper observe no BLER loss?
   Answer: about 10^{-2}.
5. How is the feedback system trained?
   Answer: with the same alternating algorithm — receiver by MSE regression, transmitter by policy gradient — alternating with the training of the communication system so that each link carries the other's

losses.

6. On which channel does the paper report the learned feedback link beating analog transmission, and by how much?
   Answer: on Rayleigh block fading, up to about 3 dB around an MSE of 10^{-2}.

Further reading

- M. Goutay, F. Ait Aoudia and J. Hoydis, "Deep Reinforcement Learning Autoencoder with Noisy Feedback", WiOpt 2019, pp. 1–6; arXiv:1810.05419.
- F. Ait Aoudia and J. Hoydis, "Model-Free Training of End-to-End Communication Systems", *IEEE JSAC*, 37(11):2503–2516, 2019. Section on training with a noisy feedback link (feedback SNR).
- F. Ait Aoudia and J. Hoydis, "End-to-End Learning of Communications Systems Without a Channel Model", Asilomar 2018 (Chapter 24).
- E. Greensmith, P. Bartlett and J. Baxter, "Variance Reduction Techniques for Gradient Estimates in Reinforcement Learning", *JMLR*, 5:1471–1530, 2004.

Chapter 26

Deep Learning Based MIMO Communications

Every modern cellular link uses several antennas at each end. Multiple-input multiple-output (MIMO) processing is what turns those antennas into reliability (diversity), into extra data streams (multiplexing) or into focused energy (beamforming). Decades of information theory and clever algebra — the Alamouti code, singular-value decomposition (SVD) precoding, linear and maximum-likelihood detectors — tell us how to do each of these optimally under clean assumptions.

This chapter studies one of the first papers to ask whether an autoencoder, the end-to-end learned link of Chapter 21, can *discover* MIMO schemes by itself: with no channel knowledge at the transmitter (open loop) and with channel knowledge fed back (closed loop), possibly quantised. We first rebuild the MIMO toolbox from first principles, then present the paper's architectures and reported results, and finally run our own 2×2 experiments, comparing against Alamouti and SVD precoding with exact maximum-likelihood detection.

Paper at a glance

Paper. T. J. O'Shea, T. Erpek and T. C. Clancy, "Deep Learning Based MIMO Communications", arXiv:1707.07980, 2017. Virginia Tech (Bradley Dept. of ECE) and DeepSig. A closely related conference paper by the same authors is "Physical layer deep learning of encodings for the MIMO fading channel", Allerton Conference on Communication, Control, and Computing, 2017.

Problem. Can an end-to-end autoencoder learn transmit/receive schemes for the Rayleigh-fading MIMO channel that compete with expert designs, both without and with channel state information (CSI) at the transmitter?

Key idea. Insert a random MIMO channel layer (random $\mathbf{H}$, complex matrix multiplication, power normalisation, AWGN) between a neural encoder and decoder. For closed loop, concatenate $\mathbf{H}$ (or a ν-bit quantised version of it, obtained with a Lloyd quantiser) to the encoder input.

Main result. 2×1 open loop: the autoencoder's BER exceeds the Alamouti code's above about 15 dB SNR (Alamouti slightly better at low SNR). 2×2 closed loop with perfect CSI: the autoencoder outperforms an SVD-precoding baseline over the SNR range tested. With quantised CSI, learned schemes remain competitive and degrade more gracefully than the baseline as the number of CSI bits shrinks.

26.1 The problem and the intuition

Intuition: a choir instead of a soloist

A single antenna is a soloist: if the room happens to absorb her note at your seat (a fade), you hear nothing. Two antennas are a duet. If they sing *the same tune in a clever interleaved way* (Alamouti), you are unlikely to lose both at once — that is *diversity*. If they sing *two different tunes* and you have two ears that hear them mixed differently, you can separate them — that is *multiplexing*. If the singers know where you sit (CSI at the transmitter), they can all aim at you — that is *beamforming*. The autoencoder is a choir that has never had a singing lesson; it is told only "make the listener understand the message" and must find out which of these tricks, or which mixture of them, works.

The appeal is obvious: classical MIMO schemes are each optimal for some idealised criterion (e.g. diversity order, or capacity with Gaussian inputs), but a real system has finite constellations, a fixed number of channel uses per block, a particular detector and imperfect CSI. A learned scheme optimises the actual end-to-end block error rate under all of these constraints at once.

26.2 Background: MIMO in one sitting

26.2.1 The flat-fading MIMO channel

With N_t transmit and N_r receive antennas, one channel use maps the transmitted vector $\mathbf{x} \in \mathbb{C}^{N_t}$ to

Key equation: MIMO channel

$$\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}, \qquad \mathbf{H} \in \mathbb{C}^{N_r \times N_t}, \quad \mathbf{n} \sim \mathcal{CN}(\mathbf{0}, \sigma^2 \mathbf{I}_{N_r}). \tag{26.1}$$

Entry h_{ij} is the complex gain from transmit antenna j to receive antenna i. In *Rayleigh* fading (Chapter 20), the h_{ij} are i.i.d. $\mathcal{CN}(0,1)$: rich scattering, no line of sight. In *block* fading, $\mathbf{H}$ stays constant over a block of T channel uses and is redrawn independently for the next block; stacking the T uses as columns gives

$$\mathbf{Y} = \mathbf{H}\mathbf{X} + \mathbf{W}, \qquad \mathbf{X} \in \mathbb{C}^{N_t \times T}, \ \mathbf{Y}, \mathbf{W} \in \mathbb{C}^{N_r \times T}. \tag{26.2}$$

We normalise the transmit energy to one per channel use, summed over antennas: $\mathbb{E}\,\|\mathbf{X}\|_F^2 = T$. Then each receive antenna sees average signal power $\mathbb{E}|\sum_j h_{ij} x_j|^2 = \sum_j \mathbb{E}|x_j|^2 = 1$, and $\mathrm{SNR} = 1/\sigma^2$ is the average SNR per receive antenna.

26.2.2 Diversity: the Alamouti code

Fading is deadly for a single antenna because when $|h|$ is small the SNR collapses, and at high SNR the error probability of uncoded transmission over Rayleigh fading falls only as $1/\mathrm{SNR}$. With two independent paths, both must fade at the same time, which makes errors fall like $1/\mathrm{SNR}^2$: *diversity order* 2.

The Alamouti scheme gets this diversity with two transmit antennas and no CSI at the transmitter. Over two channel uses it sends two symbols s_1, s_2:

$$\mathbf{X} = \frac{1}{\sqrt{2}} \begin{bmatrix} s_1 & -s_2^* \\ s_2 & s_1^* \end{bmatrix} \begin{matrix} \leftarrow \text{antenna 1} \\ \leftarrow \text{antenna 2} \end{matrix} \tag{26.3}$$

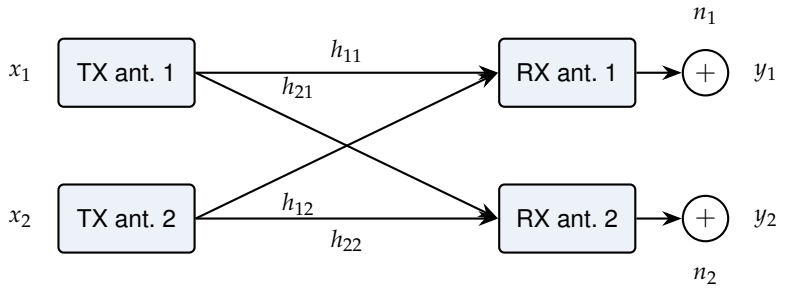


Figure 26.1: The 2×2 MIMO channel: every receive antenna hears a different complex mixture of both transmit antennas, plus its own noise; $y_i = h_{i1}x_1 + h_{i2}x_2 + n_i$.

(columns are time slots; the $1/\sqrt{2}$ keeps the total power at one per channel use). With one receive antenna and gains h_1, h_2:

$$r_1 = \tfrac{1}{\sqrt{2}}(h_1 s_1 + h_2 s_2) + n_1, \qquad r_2 = \tfrac{1}{\sqrt{2}}(-h_1 s_2^* + h_2 s_1^*) + n_2.$$

Combine $z_1 = h_1^* r_1 + h_2 r_2^*$ and $z_2 = h_2^* r_1 - h_1 r_2^*$. Expanding z_1, the s_2 terms cancel: $\tfrac{1}{\sqrt{2}}(h_1^* h_2 s_2 - h_2 h_1^* s_2) = 0$, leaving

$$z_1 = \tfrac{1}{\sqrt{2}}(|h_1|^2 + |h_2|^2)\, s_1 + \tilde{n}_1, \qquad z_2 = \tfrac{1}{\sqrt{2}}(|h_1|^2 + |h_2|^2)\, s_2 + \tilde{n}_2, \tag{26.4}$$

with $\tilde{n}_i \sim \mathcal{CN}(0, (|h_1|^2 + |h_2|^2)\sigma^2)$. Each symbol sees the *sum* of the two channel powers: diversity 2, with simple symbol-by-symbol detection. With two receive antennas, the combiner sums over both, giving diversity 4.

Worked example: Alamouti by hand

Let $h_1 = 0.8 + 0.6j$, $h_2 = -0.5 + 0.5j$, $s_1 = (1+j)/\sqrt{2}$, $s_2 = (1-j)/\sqrt{2}$ (QPSK), no noise. Then $r_1 = \tfrac{1}{\sqrt{2}}(h_1 s_1 + h_2 s_2) = 0.1 + 1.2j$ and $r_2 = \tfrac{1}{\sqrt{2}}(-h_1 s_2^* + h_2 s_1^*) = -0.1 - 0.2j$. Combining: $z_1 = h_1^* r_1 + h_2 r_2^* = 0.75 + 0.75j$ and $z_2 = h_2^* r_1 - h_1 r_2^* = 0.75 - 0.75j$. With $|h_1|^2 + |h_2|^2 = 1 + 0.5 = 1.5$, the prediction is $\tfrac{1.5}{\sqrt{2}} s_1 = 0.75 + 0.75j$ and $\tfrac{1.5}{\sqrt{2}} s_2 = 0.75 - 0.75j$. Exact.

26.2.3 Multiplexing and SVD precoding

If the transmitter knows $\mathbf{H}$, it can do better than diversity. Write the singular-value decomposition (Chapter 2)

$$\mathbf{H} = \mathbf{U}\mathbf{\Lambda}\mathbf{V}^{\mathsf{H}}, \tag{26.5}$$

with unitary $\mathbf{U}$ ($N_r \times N_r$), $\mathbf{V}$ ($N_t \times N_t$) and diagonal $\mathbf{\Lambda}$ holding the singular values $\lambda_1 \geq \lambda_2 \geq 0$. Precode $\mathbf{x} = \mathbf{V}\mathbf{s}$ and post-process $\tilde{\mathbf{y}} = \mathbf{U}^{\mathsf{H}}\mathbf{y}$:

$$\tilde{\mathbf{y}} = \mathbf{U}^{\mathsf{H}}(\mathbf{U}\mathbf{\Lambda}\mathbf{V}^{\mathsf{H}}\mathbf{V}\mathbf{s} + \mathbf{n}) = \mathbf{\Lambda}\mathbf{s} + \mathbf{U}^{\mathsf{H}}\mathbf{n}. \tag{26.6}$$

Because $\mathbf{U}$ is unitary, $\mathbf{U}^{\mathsf{H}}\mathbf{n}$ is still white Gaussian noise. The MIMO channel has become $\min(N_t, N_r)$ parallel, non-interfering scalar channels ("eigenmodes") with gains λ_k: $\tilde{y}_k = \lambda_k s_k + \tilde{n}_k$. One can then send two independent streams (*spatial multiplexing*), or put all power on the strongest eigenmode (*beamforming*), or distribute power and rate by water-filling.

Worked example: eigenmode gains

For $\mathbf{H} = \begin{bmatrix} 1 & 0.5 \\ 0.2j & 0.8 \end{bmatrix}$, numerical SVD gives $\lambda_1 = 1.223$, $\lambda_2 = 0.659$, so the two eigenmode power gains are $\lambda_1^2 = 1.495$ and $\lambda_2^2 = 0.435$ (their sum 1.93 equals $\|\mathbf{H}\|_F^2 = 1 + 0.25 + 0.04 + 0.64$). With equal power on two streams, the weak stream is $10\log_{10}(1.495/0.435) = 5.4$ dB worse than the strong one; at a fixed rate it is often better to drop the weak mode and use a denser constellation on the strong one. A Rayleigh 2×2 channel frequently has a very weak second mode — remember this when we look at the learned schemes.

26.2.4 Maximum-likelihood detection

For any finite codebook $\{\mathbf{X}_1, \ldots, \mathbf{X}_M\}$ known to the receiver, together with $\mathbf{H}$, the optimal (minimum block-error) detector under Gaussian noise picks

$$\hat{m} = \arg\min_{m} \; \|\mathbf{Y} - \mathbf{H}\mathbf{X}_m\|_F^2 . \tag{26.7}$$

For $M = 16$ this is just 16 distance computations, so in all our baselines we use exact ML. This also lets us separate two questions about a learned system: *is the learned code good?* (decode it with ML) and *is the learned decoder good?* (compare its decisions to ML's).

Wireless connection: open loop vs closed loop in 5G NR

NR supports both. Open-loop transmit diversity (e.g. SFBC, the frequency-domain Alamouti, in LTE) and NR's "transparent" precoder cycling serve high-mobility users for whom CSI is stale. Closed-loop operation uses CSI-RS and PMI/RI/CQI reports from the UE, chosen from Type I or Type II codebooks — exactly "quantised CSI feedback" in this paper's sense, with a few to a few hundred bits. In TDD, channel reciprocity lets the gNB estimate $\mathbf{H}$ from uplink SRS directly, approximating perfect CSIT.

26.3 System model of the paper

The paper considers single-user MIMO over Rayleigh fading with $\mathbf{H}$ of i.i.d. zero-mean, unit-variance circularly symmetric complex Gaussian entries, AWGN with $\mathbb{E}[\mathbf{n}\mathbf{n}^{\mathsf{H}}] = \sigma^2\mathbf{I}$, and varies the noise level to sweep SNR. Two configurations are studied:

- **Spatial diversity, 2×1, open loop**: no CSI at the transmitter. The autoencoder sends 4 bits over two consecutive complex time slots — the same density as Alamouti with QPSK (two 2-bit symbols over two slots). Baseline: Alamouti with QPSK.
- **Spatial multiplexing, 2×2, closed loop**: CSI at the transmitter, either perfect or quantised to ν bits. Baseline: SVD-based precoding with QPSK on each stream (4 bits per channel use) and a ZF or MMSE equaliser; the baseline simulation uses 100 subframes of 1000 symbols, with the channel constant over a subframe (coherence time equal to the subframe).

Results are reported as bit error rate (BER) versus SNR. The paper does not tabulate the exact network sizes, training SNR or training length; it mentions batch normalisation between layers, Adam, categorical cross-entropy, and a Keras/TensorFlow implementation, and notes that its results were obtained without significant hyperparameter tuning or long training runs.

26.4 Architectures

26.4.1 Open loop

The open-loop network (paper's Fig. 2) is the autoencoder of Chapter 21 with a MIMO channel inside. The paper writes it as a composition of layers,

$$f(\mathbf{s},\boldsymbol{\theta}) = \mathrm{dec}\Big(\mathrm{awg}\big(\mathrm{norm}(\mathrm{mul}(\mathrm{enc}(\mathbf{s}),\mathrm{rnd}())),\sigma\big)\Big), \tag{26.8}$$

where enc is the learned encoder (one-hot $\mathbf{s} \mapsto \mathbf{x}$, N_t streams of n samples), rnd draws a random $N_r \times N_t$ channel, mul does the complex matrix multiplication $\mathbf{HX}$, norm normalises the average power, awg adds Gaussian noise, and dec is the learned decoder ending in a softmax over the 2^k messages. Because the channel layers are differentiable simulations, the whole thing is trained by ordinary backpropagation (contrast Chapter 24).

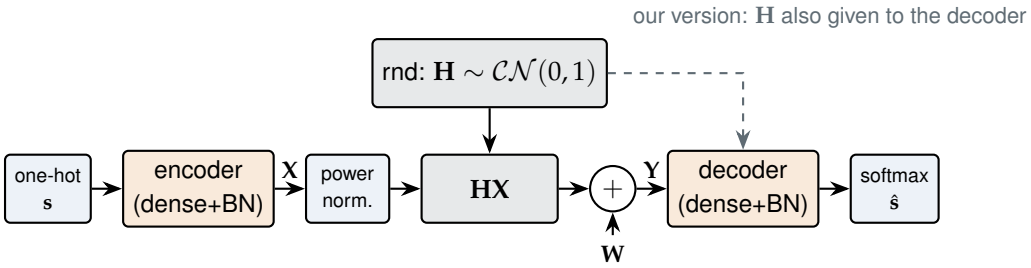


Figure 26.2: Open-loop MIMO autoencoder, drawn after the paper's layer list (enc, rnd, mul, norm, awg, dec). The transmitter never sees $\mathbf{H}$. The dashed arrow is a choice we make in our reproduction (receiver has perfect CSI, as a coherent receiver with pilots would); the paper's open-loop description does not state that $\mathbf{H}$ is input to the decoder.

26.4.2 Closed loop

For the closed-loop case (paper's Fig. 3), the channel realisation is drawn first and *concatenated to the message before encoding*, so the encoder learns a channel-dependent mapping $(\mathbf{s},\mathbf{H}) \mapsto \mathbf{x}$. The paper writes

$$f(\mathbf{s},\boldsymbol{\theta}) = \Big(\lambda\mathbf{H}.\,\mathrm{dec}\big(\mathrm{awg}(\mathrm{norm}(\mathrm{mul}(\mathrm{enc}(\mathbf{s},\mathbf{H}),\mathbf{H})),\sigma)\big)\Big)\big(\mathrm{rnd}()\big), \tag{26.9}$$

i.e. one random $\mathbf{H}$ is shared by the encoder input and the channel multiplication. For limited feedback (paper's Fig. 4), the real-valued $\mathbf{H}$ is mapped to one of 2^ν quantisation cells by a Lloyd (k-means) quantiser designed in advance, and the cell index $\mathbf{H}_\nu$ — known to both ends — is what the encoder receives; the paper evaluates $\nu = 1, 2, 4, 8$ bits. For deployment (paper's Fig. 5), the receiver would estimate $\hat{\mathbf{H}}$ (by classical estimation or a neural regression), quantise it and feed back $\mathbf{H}_\nu$; the paper notes that $\mathbf{H}_\nu$ can be assumed equal in both directions.

26.5 Results reported in the paper

2×1 diversity (paper Fig. 7). The autoencoder's BER is better than the Alamouti code's when the SNR is above approximately 15 dB, while Alamouti is slightly better at low SNR. The learned constellations show a superposition-coding-like scheme with uneven power distribution between the two antennas.

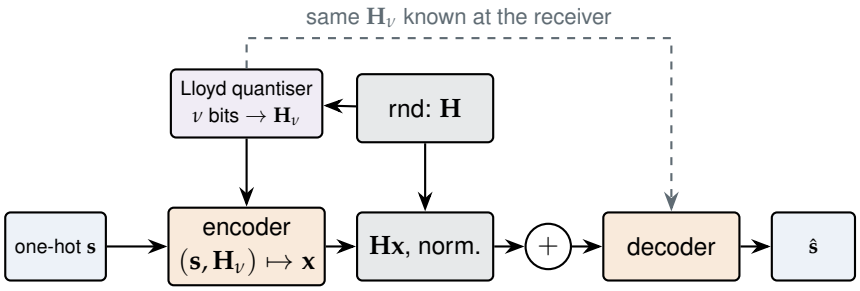


Figure 26.3: Closed-loop MIMO autoencoder with ν-bit CSI feedback, after the paper's description. One random $\mathbf{H}$ drives the channel; its quantised index $\mathbf{H}_\nu$ is concatenated to the message at the encoder (and is known at the receiver). With perfect CSI, $\mathbf{H}$ itself replaces $\mathbf{H}_\nu$.

2×2 perfect CSI (paper Fig. 10). The paper describes extremely promising results relative to the SVD baseline: the autoencoder outperforms it over the SNR range evaluated. The received constellations look like constant-amplitude phase encodings.

2×2 quantised CSI (paper Figs. 14–15). The baseline degrades as CSI is represented with fewer bits. The autoencoder shows, for some quantised cases (2, 4 and 8 bits), *better* performance than its own perfect-CSI version — an effect the authors attribute to learning; for 1-bit CSI it learns a multi-level scheme with constant-modulus encodings at two power levels, and for 2-bit CSI an irregular, 16-QAM-like multi-level scheme. Their best autoencoder (paper Fig. 20) reaches BER below about 10^{-5} before 20 dB, a significant improvement over the baseline across the whole range.

Pitfall: read baselines carefully

A learned scheme "beating" a baseline says as much about the baseline as about the learned scheme. The paper's SVD baseline uses equal-power QPSK on both eigenmodes with a linear (ZF/MMSE) equaliser. At 4 bits per channel use over Rayleigh 2×2, the weak eigenmode often has very low gain, and putting half the bits on it is far from the best fixed-rate classical design. In our reproduction we therefore add a second, stronger classical baseline: all 4 bits as 16-QAM on the dominant eigenmode (beamforming), with ML detection.

26.6 Our small-scale reproduction

We study a 2×2 Rayleigh block-fading channel (i.i.d. $\mathcal{CN}(0,1)$ entries) in two settings. The code is in `code/ch26/` (`mimo.py` with the shared pieces; one script per experiment). All networks are trained by backpropagation through the simulated channel, with Adam, cross-entropy, a step learning-rate schedule and SNR drawn uniformly in a 6 dB window around the training SNR. Baselines and the learned codes are evaluated with exact ML detection (26.7), and the learned systems also with their own neural decoders.

The closed-loop transmitter is the only unusual piece: it has to output a whole *codebook* for each channel realisation so that the power constraint can be applied per realisation.

Listing 26.1: Closed-loop transmitter (`code/ch26/mimo.py`): for every $\mathbf{H}$ in the batch it evaluates the network on all 16 messages and normalises that codebook to unit average energy, then the message picks its codeword.

```
class CLTx(nn.Module):
```

	(a) open loop	(b) closed loop
Message	$M = 16$ (4 bits)	$M = 16$ (4 bits)
Channel uses	$T = 2$, $\mathbf{H}$ constant over both	$T = 1$
Rate	2 bits/channel use	4 bits/channel use
Transmitter	one-hot $\to$ dense 64 (ELU) $\to$ dense 8 $\to$ $\mathbf{X} \in \mathbb{C}^{2\times 2}$, codebook normalised to $\mathbb{E}\,\|\mathbf{X}\|_F^2 = 2$	$[\text{one-hot}, \mathbf{H}]$ (24 reals) $\to$ 2$\times$dense 128 (ReLU) $\to$ dense 4 $\to$ $\mathbf{x} \in \mathbb{C}^2$; normalised per $\mathbf{H}$ over the 16 codewords
Receiver	$[\mathbf{Y}, \mathbf{H}]$ (16 reals) $\to$ 2$\times$dense 256 (ReLU) $\to$ 16	$[\mathbf{y}, \mathbf{H}]$ (12 reals) $\to$ 2$\times$dense 256 (ReLU) $\to$ 16
Training	2000 it., batch 512, lr $2 \cdot 10^{-3}$, 9–15 dB	1400 it., batch 128, lr $2 \cdot 10^{-3}$, 12–18 dB
Baselines	Alamouti+QPSK; spatial mux. BPSK per antenna per slot; both ML	SVD $\mathbf{x} = \mathbf{V}\mathbf{s}$ with 2$\times$QPSK; dominant eigenmode with 16-QAM; both ML

Table 26.1: Our reproduction settings. Receivers know $\mathbf{H}$ (coherent detection) in all cases.

```
    """[one-hot m, H] -> x (NT complex, one channel use). For each H the M
    codewords are
    normalised to unit average energy (power constraint per channel realisation).
    """
    def __init__(self, M, h=128):
        super().__init__()
        self.M = M
        self.net = mlp(M + 2 * NR * NT, h, 2 * NT)

    def codebooks(self, H):
        B = H.shape[0]
        hr = c2r(H)[:, None, :].expand(B, self.M, -1)
        oh = torch.eye(self.M)[None].expand(B, -1, -1)
        X = self.net(torch.cat([oh, hr], 2).reshape(B * self.M, -1))
        X = r2c(X, (NT,)).reshape(B, self.M, NT)
        e = (X.abs() ** 2).sum(2).mean(1, keepdim=True)              # (B,1)
        return X / torch.sqrt(e[..., None])                           # (B, M, NT
    )
```

26.6.1 (a) Open loop: learned code vs Alamouti

Figure 26.4 shows the block error rate of all 4 bits. The learned system clearly beats the naive spatial-multiplexing BPSK design at the same rate (e.g. at 15 dB, $1.7 \cdot 10^{-3}$ for the neural decoder and $5.2 \cdot 10^{-4}$ for ML decoding of the learned code, versus $3.2 \cdot 10^{-3}$). But it does *not* reach Alamouti: at 12 dB, Alamouti+QPSK with ML has BLER $1.1 \cdot 10^{-3}$, the learned code with ML $4.4 \cdot 10^{-3}$, and the learned code with its neural decoder $6.6 \cdot 10^{-3}$. Its slope at high SNR is also shallower than Alamouti's.

Why? The diversity of a space-time code is governed by the *rank and determinant criterion*: for every pair of codewords, the difference matrix $\mathbf{D} = \mathbf{X}_i - \mathbf{X}_j$ should have full rank, and the minimum of $\det(\mathbf{D}\mathbf{D}^{\mathsf{H}})$ over pairs governs the coding gain. For Alamouti with QPSK we computed a minimum determinant of 1.0; for our learned code it is 0.019 — some pairs of codewords are nearly rank-deficient, so a fade that aligns with their difference causes errors, and the error curve's slope approaches that of a lower diversity order. Cross-entropy training at 9–15 dB with a finite batch rarely "sees" such deep, rare fades and has little incentive to fix them. Figure 26.5 shows the learned signal set: like the paper's, it is irregular and spreads power unequally across antennas (we measured 37%/63%).

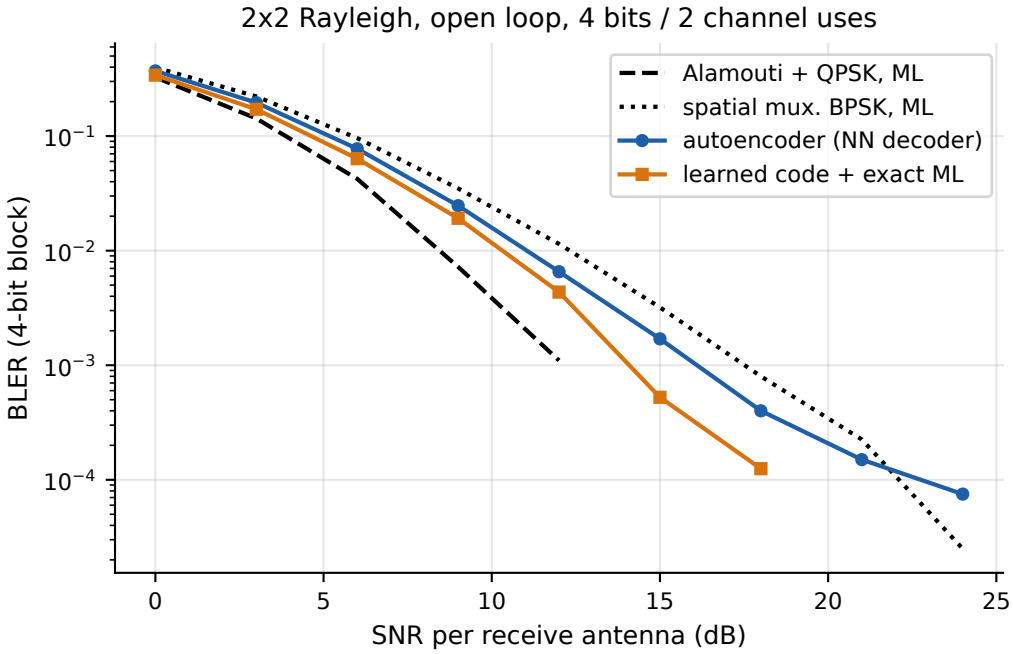


Figure 26.4: Our reproduction (a): 2×2 Rayleigh, no CSI at the transmitter, 4 bits over 2 channel uses. Dashed: Alamouti+QPSK with ML; dotted: BPSK on each antenna in each slot with ML. Circles: the autoencoder with its own neural decoder; squares: the same learned code decoded by exact ML. Missing points at high SNR had zero errors in 40 000 trials. The learned code beats the naive multiplexing design but stays clearly behind Alamouti.

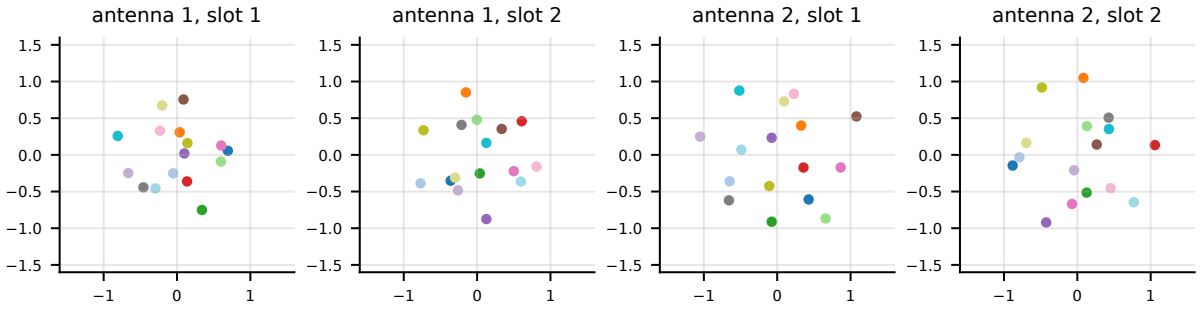


Figure 26.5: Our reproduction (a): the learned open-loop code. Each panel shows the 16 complex values sent from one antenna in one time slot (colours identify messages, consistent across panels). Unlike Alamouti, the learned code uses irregular, multi-level points and unequal antenna powers — qualitatively what the paper describes as superposition-like coding with uneven power distribution.

This is a genuine difference from the paper, which reports the autoencoder overtaking Alamouti above about 15 dB for 2×1. Plausible reasons: their architecture and training differ from ours and are not fully specified; their curves are BER rather than BLER (bit mapping matters); and our network was trained for under four minutes. What our experiment does confirm is the paper's qualitative picture of the learned signal set, and that a learned code easily beats a naive design — while a carefully designed orthogonal code remains a strong opponent at high SNR.

26.6.2 (b) Closed loop: learned precoding vs SVD

With perfect CSI at the transmitter (Figure 26.6), the learned code with ML detection is far better than SVD precoding with two equal-power QPSK streams: at 15 dB, $5.9 \cdot 10^{-3}$ versus $9.5 \cdot 10^{-2}$, and the gap widens with SNR. This reproduces the paper's headline result. However, the stronger classical baseline — 16-QAM on the dominant eigenmode — is better still ($2.6 \cdot 10^{-3}$ at 15 dB, and no errors in 14 000 trials from 21 dB), so in our run the autoencoder found a good scheme but not the best classical one. Also note the large gap between the neural decoder and ML decoding of the same learned code at high SNR (e.g. $2.9 \cdot 10^{-2}$ vs $5.9 \cdot 10^{-3}$ at 15 dB): our small, briefly trained decoder must learn to equalise every possible $\mathbf{H}$, and it flattens out at high SNR. A longer training run or a decoder with built-in structure would close part of this gap.

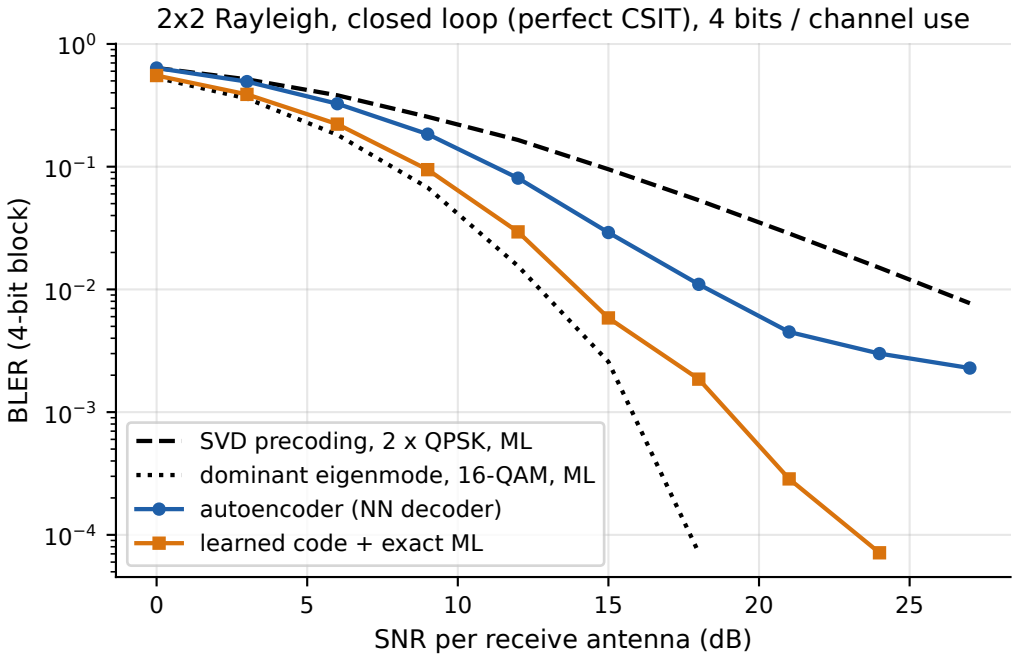


Figure 26.6: Our reproduction (b): 2×2 Rayleigh, perfect CSI at the transmitter, 4 bits per channel use. Dashed: SVD precoding with two equal-power QPSK streams (the paper-style baseline), ML. Dotted: dominant-eigenmode beamforming with 16-QAM, ML. The learned precoder (squares, ML-decoded) beats the equal-power SVD baseline by a wide margin, but not the beamforming baseline; the neural decoder (circles) loses a lot at high SNR.

What did the autoencoder learn? Project each learned codeword onto the eigen-directions, $\mathbf{V}^{\mathsf{H}}\mathbf{x}$, to see how much energy goes into each eigenmode (Figure 26.7). Averaged over 1000 channel draws, 80% of the energy goes to the dominant mode. And the split adapts to the channel: when the second singular value is tiny ($\lambda_2/\lambda_1 = 0.08$) the network puts 98% of its energy into mode 1 and arranges the 16 points as a 2-D constellation there — it has rediscovered beamforming; when the modes are more balanced ($\lambda_2/\lambda_1 = 0.59$) it uses 88% / 12% and spreads

the points over both. This is the behaviour water-filling and "rank adaptation" would suggest, learned from nothing but the end-to-end loss.

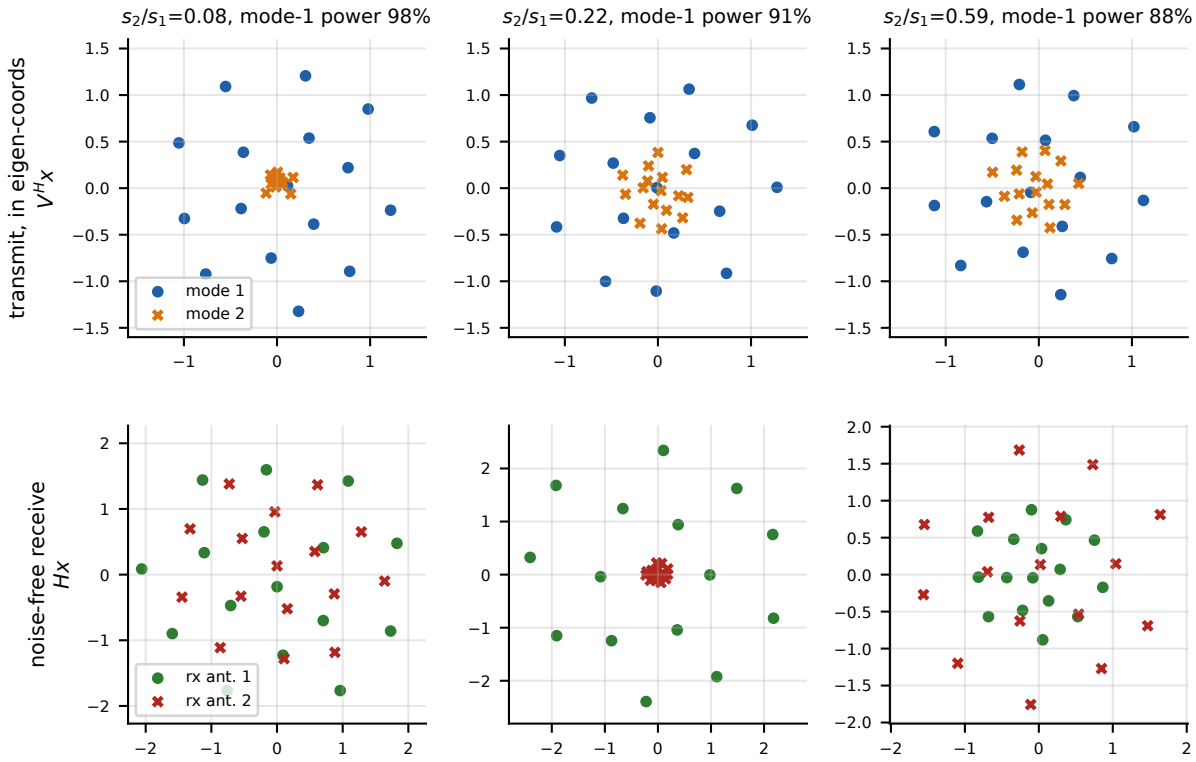


Figure 26.7: Our reproduction (b): learned closed-loop signal sets for three channel realisations (columns), ordered by λ_2/λ_1. Top: transmitted codewords in eigen-coordinates $\mathbf{V}^{\mathrm{H}}\mathbf{x}$ (dots: mode 1, crosses: mode 2) and the fraction of power on mode 1. Bottom: noise-free received points $\mathbf{Hx}$ at the two receive antennas. When the second mode is weak the network collapses onto the strong mode (beamforming); otherwise it uses both.

Wireless connection: rank adaptation, learned

NR's UE reports a Rank Indicator (RI) precisely because transmitting two layers over a channel with a weak second eigenmode wastes power. The learned closed-loop autoencoder makes this decision continuously, per channel realisation, and even "half-uses" the second mode — something a discrete RI cannot express. In practice, the gNB's link adaptation (rank, PMI and MCS selection) already approximates this behaviour with tables; an AI-based alternative would have to deal with quantised CSI and with estimation errors, which the paper's quantised-CSI experiments begin to explore.

26.7 Limitations and follow-ups

- **Tiny scale.** 2 antennas, 4 bits per message. The number of messages grows as 2^k, so one-hot autoencoders do not scale to realistic rates; later work uses bit-wise (binary cross-entropy) outputs and combines learned constellations with standard channel codes.
- **Perfect CSI at the receiver, i.i.d. Rayleigh.** No channel estimation error, no spatial correlation, no frequency selectivity. The paper's own future-work list includes learned channel estimation, larger arrays (e.g. 8×4, massive MIMO), multi-tap delay spread, OFDM, and

multi-user MIMO.

- **Baselines.** As our experiments show, conclusions depend strongly on which classical scheme is chosen; fair comparisons should include ML detection and rank/power adaptation.
- **Analysis.** The authors acknowledge that learned schemes make rigorous analytic performance bounds difficult; diversity order, for instance, is not guaranteed by training (our open-loop code's small minimum determinant illustrates this).

26.8 From paper to practice in 5G NR/6G

- **CSI feedback compression.** The most mature descendant of this paper is not end-to-end MIMO transmission but learned CSI compression (an encoder at the UE, a decoder at the gNB, e.g. CsiNet and successors), studied in 3GPP's Rel-18 AI/ML study for the air interface as a two-sided model. The paper's quantised-CSI experiments are an early form of this idea.
- **Codebooks.** NR Type I/II codebooks are hand-designed quantisers of the dominant eigenvectors. A learned precoder conditioned on a ν-bit index, as in the paper, is a learned codebook — attractive for irregular arrays and for correlated channels where uniform codebooks waste bits.
- **Receivers first.** Learned MIMO *detectors* (e.g. unfolded/deep-unrolled detectors) are easier to deploy than learned transmitters, because they need no standard change; the open-loop decoder in this chapter is a toy version.
- **6G.** Proposals for AI-native air interfaces revisit end-to-end learning with learned constellations and pilot-less transmission; the closed-loop results here (learning beamforming and rank adaptation on its own) are a useful sanity check for such systems.

26.9 Recap, exercises and quiz

Chapter recap

- MIMO: $\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}$. Without CSIT, codes like Alamouti give diversity; with CSIT, SVD precoding $\mathbf{x} = \mathbf{V}\mathbf{s}$ turns the channel into parallel eigenmodes.
- The paper builds autoencoders with a random-MIMO channel layer: open loop (encoder sees only the message) and closed loop (encoder also sees $\mathbf{H}$ or a ν-bit Lloyd-quantised $\mathbf{H}_\nu$).
- Reported: 2×1 open loop beats Alamouti above about 15 dB; 2×2 closed loop beats an SVD baseline; quantised-CSI schemes degrade gracefully.
- Our 2×2 reproduction: open-loop learned code beats naive multiplexing but not Alamouti (small minimum determinant); closed-loop learned precoder beats equal-power SVD by a wide margin, learns beamforming/rank adaptation, but not 16-QAM beamforming.
- Lessons: choose strong baselines with ML detection; separate code quality (ML-decode the learned code) from decoder quality.

Exercises

1. **(Alamouti noise.)** Show that $\tilde{n}_1 = h_1^* n_1 + h_2 n_2^*$ has variance $(|h_1|^2 + |h_2|^2)\sigma^2$ and hence that the post-combining SNR of s_1 is $\frac{(|h_1|^2+|h_2|^2)}{2\sigma^2}$ for unit-energy s_1.
2. **(SVD.)** Verify that $\mathbf{U}^{\mathsf{H}}\mathbf{n}$ is $\mathcal{CN}(\mathbf{0}, \sigma^2\mathbf{I})$ if $\mathbf{n}$ is and $\mathbf{U}$ is unitary.
3. **(Which baseline?)** For $\lambda_1^2 = 3$, $\lambda_2^2 = 0.1$ and SNR $= 10$ dB, compute the per-stream SNRs for equal-power two-stream SVD (power $1/2$ each) and for single-stream beamforming. Using the rule of thumb that 16-QAM needs about 6 dB more SNR than QPSK for the same SER, which design would you choose at 4 bits/channel use?
4. **(Rank criterion.)** For two Alamouti codewords $\mathbf{X}_i, \mathbf{X}_j$, show that $\mathbf{D}\mathbf{D}^{\mathsf{H}} = \frac{1}{2}(|\Delta_1|^2 + |\Delta_2|^2)\mathbf{I}$ where $\Delta_k = s_k^{(i)} - s_k^{(j)}$. Why does this guarantee full diversity?
5. **(Quantised CSI.)** With ν bits of CSI and a Lloyd quantiser on the 8 real numbers of a 2×2 $\mathbf{H}$, how many cells are there for $\nu = 1, 2, 4, 8$? Why might the paper's autoencoder perform *better* with 2–8-bit CSI than with perfect CSI (think about what the network must learn)?
6. **(Per-H normalisation.)** Our closed-loop transmitter normalises the codebook separately for each $\mathbf{H}$. What would change if we normalised only on average over the batch? Which constraint corresponds to a real power amplifier?
7. **(Code.)** Train the open-loop autoencoder with an added loss term that penalises small $\det(\mathbf{D}\mathbf{D}^{\mathsf{H}})$ over codeword pairs. Predict the effect on the high-SNR slope.

Answers

1. n_1, n_2 independent $\mathcal{CN}(0, \sigma^2)$ and n_2^* has the same distribution as n_2. $\mathrm{Var}(\tilde{n}_1) = |h_1|^2\sigma^2 + |h_2|^2\sigma^2$. Signal part $\frac{1}{\sqrt{2}}(|h_1|^2 + |h_2|^2)s_1$ has power $\frac{1}{2}(|h_1|^2 + |h_2|^2)^2$. Ratio: $\frac{(|h_1|^2+|h_2|^2)}{2\sigma^2}$, i.e. the combined channel power of both paths, divided by 2 because the transmit power is split between the two antennas.
2. $\mathbb{E}[\mathbf{U}^{\mathsf{H}}\mathbf{n}\mathbf{n}^{\mathsf{H}}\mathbf{U}] = \mathbf{U}^{\mathsf{H}}\sigma^2\mathbf{I}\mathbf{U} = \sigma^2\mathbf{U}^{\mathsf{H}}\mathbf{U} = \sigma^2\mathbf{I}$; a linear map of a circular Gaussian vector is circular Gaussian.
3. SNR $= 10$. Two streams: $3 \cdot 0.5 \cdot 10 = 15$ (11.8 dB) and $0.1 \cdot 0.5 \cdot 10 = 0.5$ (-3 dB) — the weak QPSK stream is hopeless. Beamforming: $3 \cdot 10 = 30$ (14.8 dB) for one 16-QAM stream, equivalent to QPSK at about 8.8 dB, comfortably better than -3 dB. Choose beamforming.
4. $\mathbf{D} = \frac{1}{\sqrt{2}}\begin{bmatrix}\Delta_1 & -\Delta_2^* \\ \Delta_2 & \Delta_1^*\end{bmatrix}$. Multiply: diagonal entries $\frac{1}{2}(|\Delta_1|^2 + |\Delta_2|^2)$, off-diagonal $\frac{1}{2}(\Delta_1\Delta_2^* - \Delta_2^*\Delta_1) = 0$. For distinct codewords at least one $\Delta_k \neq 0$, so $\mathbf{D}\mathbf{D}^{\mathsf{H}}$ is a positive multiple of $\mathbf{I}$ — full rank for every pair, which by the rank criterion gives the maximum diversity order $N_t N_r$.
5. 2, 4, 16, 256 cells. With perfect CSI the encoder must learn a continuous mapping from 8 real numbers to good codebooks; with quantised CSI it only has to learn 2^ν discrete codebooks (a much easier problem for a small network with finite training), and the quantiser itself acts as a well-designed partition. Imperfect optimisation in the perfect-CSI case can therefore lose more than the quantisation does.

6. Batch-average normalisation lets the transmitter use more power for some channels and less for others (power allocation across time), which can improve average error rates but is not allowed by a per-symbol-block peak/average PA constraint. Per-$\mathbf{H}$ normalisation is the stricter, more realistic constraint for a single block.
7. The penalty pushes codeword pairs towards full-rank differences; expect a steeper high-SNR slope (closer to Alamouti's diversity), possibly at a small cost at low SNR.

Self-test quiz

1. What is the difference between the paper's open-loop and closed-loop autoencoders?
   Answer: in closed loop the encoder input is the message concatenated with $\mathbf{H}$ *(or its quantised index* $\mathbf{H}_\nu$*); in open loop it is the message alone.*
2. What does SVD precoding turn the MIMO channel into?
   Answer: parallel non-interfering scalar channels with gains equal to the singular values.
3. How is CSI quantised in the paper?
   Answer: a ν*-bit Lloyd quantiser on the real-valued* $\mathbf{H}$*, with the quantisation values known at both ends;* $\nu = 1, 2, 4, 8$*.*
4. What does the paper report for 2×1 open loop vs Alamouti?
   Answer: the autoencoder is better above about 15 dB; Alamouti is slightly better at low SNR.
5. Why did our open-loop learned code fall behind Alamouti at high SNR?
   Answer: some codeword pairs have nearly rank-deficient differences (small minimum determinant), reducing the effective diversity.
6. What did the closed-loop autoencoder learn when $\lambda_2 \ll \lambda_1$?
   Answer: to put almost all energy on the dominant eigenmode (beamforming).

Further reading

- T. J. O'Shea, T. Erpek and T. C. Clancy, "Deep Learning Based MIMO Communications", arXiv:1707.07980, 2017.
- T. J. O'Shea, T. Erpek and T. C. Clancy, "Physical layer deep learning of encodings for the MIMO fading channel", Allerton Conference on Communication, Control, and Computing, 2017.
- S. M. Alamouti, "A Simple Transmit Diversity Technique for Wireless Communications", *IEEE J. Sel. Areas Commun.*, 16(8):1451–1458, 1998.
- V. Tarokh, N. Seshadri and A. R. Calderbank, "Space-Time Codes for High Data Rate Wireless Communication: Performance Criterion and Code Construction", *IEEE Trans. Inf. Theory*, 44(2):744–765, 1998 (rank and determinant criteria).
- D. Tse and P. Viswanath, *Fundamentals of Wireless Communication*, Cambridge University Press, 2005, Chapters 3 and 7.
- C.-K. Wen, W.-T. Shih and S. Jin, "Deep Learning for Massive MIMO CSI Feedback", *IEEE Wireless Commun. Lett.*, 7(5):748–751, 2018 (CsiNet).

Chapter 27

Channel State Information Prediction for 5G: A Deep Learning Approach

A base station can only adapt to the channel it *knows*. It chooses the modulation and coding scheme, the precoder, the beam and the scheduled user from *channel state information* (CSI), and that CSI is always a little old: it was measured on a reference signal some milliseconds ago, quantised, reported and processed before anyone uses it. When the user moves or the weather changes, yesterday's (or even the last slot's) CSI is a poor guide to the next transmission. This chapter studies a paper that treats the problem as *forecasting*: learn from historical data how CSI evolves, and predict the value that will hold when it is actually used.

You will learn: why CSI goes stale and how fast (with the Jakes correlation from Chapter 20); why "use the last value" and linear autoregression are the baselines to beat; how a convolutional network (Chapter 15) and an LSTM (Chapter 16) can be stacked into a predictor that also uses side information such as weather; and why the paper trains in two steps — offline on a large history, then online on fresh measurements. We then build our own small version, train it on a synthetic channel with Doppler fading, shadowing and weather effects, and test the offline-then-online idea under a change of environment.

Paper at a glance

Title: Channel State Information Prediction for 5G Wireless Communications: A Deep Learning Approach.
Authors: Changqing Luo, Jinlong Ji, Qianlong Wang, Xuhui Chen, Pan Li (Case Western Reserve University at the time).
Venue: IEEE Transactions on Network Science and Engineering, vol. 7, no. 1, pp. 227–236, Jan.–Mar. 2020 (published online in 2018, DOI 10.1109/TNSE.2018.2848960).
Problem: classical CSI estimation is computationally heavy for 5G systems that use massive MIMO, OFDM and millimetre waves; can CSI instead be *predicted* quickly from historical data?
Key idea: a scheme called **OCEAN** (online CSI prediction). A data sample combines several features that affect the CSI of a radio link with the CSI value itself; a learning framework integrating a **CNN** and an **LSTM** maps history to future CSI; an **offline–online two-step training** mechanism makes predictions more stable in practice.
Main result (as stated in the abstract): experiments in four scenarios (two outdoor, two indoor) show fast prediction and an *average difference ratio* (ADR) between predicted and measured CSI of about 2.650–3.457 %.

Pitfall: what we could and could not verify

We verified the bibliographic data and the abstract of the paper. We could *not* access the full text, so the exact layer sizes, the precise definition of ADR, the measurement hardware and the size of the data set are not reported in this chapter. Where we describe the input features (frequency band, location, time, weather, temperature, humidity) and the CSI-as-

image representation, we rely on the abstract together with a follow-up paper that explicitly adopts OCEAN's data model (Pecorella, Fantacci and Picano, *Sensors* 2020). Everything in the "reproduction" section is our own design, inspired by the paper, not a copy of it.

27.1 The problem: CSI is always late

Intuition: driving with a photograph of the road

Imagine driving while looking only at a photograph of the road taken two seconds ago. On an empty straight motorway you will be fine; in a busy town you will crash. CSI is that photograph. If the channel changes slowly compared with the delay between measurement and use, old CSI is fine. If it changes quickly — a fast user, a sudden rain shower, a bus passing between you and the cell — the transmitter is steering by a stale picture. *Prediction* is like extrapolating where the road is going from the last few photographs, plus what you know about the traffic and the weather.

Figure 27.1 shows the life of one CSI value in 5G NR. The gNB transmits a CSI reference signal (CSI-RS) in slot n. The UE estimates the channel, computes a report (CQI, PMI, RI) and sends it on PUCCH or PUSCH a few slots later. The gNB decodes the report, schedules, and transmits data in a later slot. The total *CSI age* τ is the sum of the reporting delay, the processing time and the waiting time until the next reference signal. With periodic CSI-RS every 5–20 ms, ages of 5–20 ms are perfectly normal.

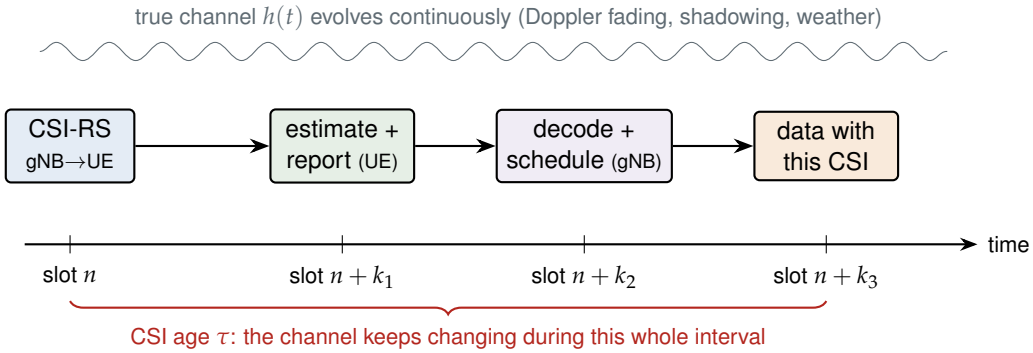


Figure 27.1: Why CSI is outdated. The value measured at slot n is used only at slot $n+k_3$. A predictor running at the gNB (or at the UE) tries to report or use $\hat{h}(t+\tau)$ instead of $h(t)$.

27.1.1 How fast does the channel forget?

From Chapter 20 recall the Clarke/Jakes model: with many paths arriving from uniformly distributed angles, the normalised complex channel gain $h(t)$ (with $\mathbb{E}\,|h|^2 = 1$) has autocorrelation

$$\rho(\tau) \;=\; \mathbb{E}\big[h(t+\tau)\,h^*(t)\big] \;=\; J_0(2\pi f_D \tau), \qquad f_D = \frac{v}{\lambda} = \frac{v f_c}{c}, \tag{27.1}$$

where J_0 is the Bessel function of the first kind and order zero, v the speed, f_c the carrier and c the speed of light. The right panel of Fig. 27.3 plots J_0 for several Doppler frequencies: at $f_D = 10$ Hz the correlation is still above 0.9 after 7 ms, while at $f_D = 50$ Hz it has dropped to zero after less than 8 ms.

Key equation: the price of stale CSI

Using the old value $h(t)$ as the estimate of $h(t+\tau)$ ("persistence") gives mean-squared error

$$\mathbb{E}\,|h(t+\tau)-h(t)|^2 = 2\big(1-\rho(\tau)\big), \tag{27.2}$$

while the best *linear* predictor that uses only $h(t)$, namely $\hat{h} = \rho(\tau)\,h(t)$, gives

$$\mathbb{E}\,|h(t+\tau)-\rho h(t)|^2 = 1-\rho(\tau)^2. \tag{27.3}$$

Derivation. Expand the square, using $\mathbb{E}\,|h(t)|^2 = \mathbb{E}\,|h(t+\tau)|^2 = 1$ and $\mathbb{E}[h(t+\tau)h^*(t)] = \rho$ (real for the Jakes model):

$$\begin{aligned}\mathbb{E}\,|h(t+\tau)-a\,h(t)|^2 &= \mathbb{E}\,|h(t+\tau)|^2 - a\,\mathbb{E}[h(t+\tau)h^*(t)] - a\,\mathbb{E}[h^*(t+\tau)h(t)] + a^2\mathbb{E}\,|h(t)|^2\\ &= 1-2a\rho+a^2.\end{aligned}$$

With $a=1$ (persistence) this is $2-2\rho$, which is (27.2). Treating it as a parabola in a, the derivative $-2\rho+2a$ vanishes at $a=\rho$, and substituting gives $1-2\rho^2+\rho^2 = 1-\rho^2$, which is (27.3). Note that persistence error *exceeds 1* as soon as $\rho < 1/2$: beyond that point, "use the last value" is worse than simply predicting the long-term average (zero for h), whose error is exactly 1.

Worked example: how stale is 5 ms?

Take $f_D = 20\,\text{Hz}$ and $\tau = 5\,\text{ms}$. Then $2\pi f_D\tau = 0.628$ and $\rho = J_0(0.628) = 0.9037$. Persistence error $= 2(1-0.9037) = 0.193$ ($-7.2\,\text{dB}$ relative to the channel power); the one-tap linear predictor gives $1-0.9037^2 = 0.183$ ($-7.4\,\text{dB}$). Double the age to $\tau = 10\,\text{ms}$: $\rho = 0.6425$, persistence 0.715 ($-1.5\,\text{dB}$) and the linear predictor 0.587 ($-2.3\,\text{dB}$). Now raise the Doppler to $f_D = 50\,\text{Hz}$ at $\tau = 5\,\text{ms}$: $\rho = 0.472$, persistence 1.056 — worse than knowing nothing — while the linear predictor still achieves 0.777. *Lesson:* the useful prediction horizon is a fraction of the coherence time, and even a trivial predictor beats "use the last value" once the CSI is old.

With only one past sample the gain from prediction is small. With a *history* of samples, a predictor can exploit the shape of the Doppler spectrum (a sum of sinusoids is very predictable over short horizons), and the slower components — shadowing, weather, time of day — are predictable over much longer horizons. That is the opportunity OCEAN targets.

27.2 Background: from autoregression to CNN+LSTM

27.2.1 Persistence and autoregressive predictors

Definition: autoregressive (AR) predictor

An AR(p) predictor forecasts a scalar series x_t as a weighted sum of its last p values, $\hat{x}_{t+1} = \sum_{i=1}^{p} a_i\,x_{t+1-i} + b$. The weights are found by least squares on historical data. For a horizon of k steps one can either iterate the one-step predictor or fit a separate set of weights $a_i^{(k)}$ for each horizon ("direct" prediction, which we use).

Least squares chooses $\mathbf{a}$ to minimise $\sum_t (x_{t+k} - \mathbf{a}^{\mathsf{T}}\mathbf{z}_t)^2$, where $\mathbf{z}_t = [x_t, \ldots, x_{t-p+1}, 1]^{\mathsf{T}}$. Stacking all training rows $\mathbf{z}_t^{\mathsf{T}}$ into a matrix $\mathbf{Z}$ and the targets into $\mathbf{y}$, setting the gradient $-2\mathbf{Z}^{\mathsf{T}}(\mathbf{y}-\mathbf{Z}\mathbf{a})$ to zero gives the normal equations $\mathbf{Z}^{\mathsf{T}}\mathbf{Z}\,\mathbf{a} = \mathbf{Z}^{\mathsf{T}}\mathbf{y}$, i.e. $\mathbf{a} = (\mathbf{Z}^{\mathsf{T}}\mathbf{Z})^{-1}\mathbf{Z}^{\mathsf{T}}\mathbf{y}$ — exactly the linear regression of Chapter 5.

Worked example: fitting AR(1) by hand

Five measurements of a (centred) channel-quality series: $x = [1.0, 0.8, 0.7, 0.5, 0.45]$. For AR(1) without bias, $a = \sum_t x_t x_{t-1} / \sum_t x_{t-1}^2$. Numerator: $0.8 \cdot 1.0 + 0.7 \cdot 0.8 + 0.5 \cdot 0.7 + 0.45 \cdot 0.5 = 0.8 + 0.56 + 0.35 + 0.225 = 1.935$. Denominator: $1.0^2 + 0.8^2 + 0.7^2 + 0.5^2 = 1 + 0.64 + 0.49 + 0.25 = 2.38$. So $a = 0.813$. One-step forecast: $\hat{x}_6 = 0.813 \cdot 0.45 = 0.366$; two-step (iterating): $0.813^2 \cdot 0.45 = 0.297$. The AR model has learned "the series decays by about 19 % per step" — a linear rule. It cannot learn that a rain shower starting *now* will drop the SNR by several dB; that needs side information and a nonlinear model.

27.2.2 What the LSTM and the CNN add

An LSTM (Chapter 16) is a nonlinear, *stateful* generalisation of the AR predictor: it reads the history one step at a time, keeps a memory cell $\mathbf{c}_t$ and a hidden state $\mathbf{h}_t$, and its gates decide what to remember and what to forget. With input $\mathbf{x}_t$:

$$\begin{aligned} \mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1};\mathbf{x}_t] + \mathbf{b}_f), & \mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1};\mathbf{x}_t] + \mathbf{b}_i), & \mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1};\mathbf{x}_t] + \mathbf{b}_o), \\ \tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c[\mathbf{h}_{t-1};\mathbf{x}_t] + \mathbf{b}_c), & \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, & \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \end{aligned}$$

After the last history step, a linear layer maps $\mathbf{h}_t$ to the forecast. Because the gates are learned, the LSTM can, for example, ignore old samples when a regime change (weather) occurs and rely on long memory otherwise.

A CNN (Chapter 15) contributes *local pattern extraction*. If CSI is measured on several frequency bands, the history forms a two-dimensional array (time $\times$ frequency) — an image. Neighbouring sub-bands are correlated (within the coherence bandwidth), and a fade typically moves smoothly across time and frequency. A small convolution kernel sliding over this image learns such local patterns with very few weights, and its outputs form a compact feature sequence for the LSTM.

Wireless connection: prediction is regression in time

In the language of Chapter 20, channel estimation is a regression from pilots to $\hat{\mathbf{h}}$; CSI prediction is the same regression shifted in time: from *past* pilots (and context) to *future* $\mathbf{h}$. The loss is the mean-squared error, the target is a real-valued vector, and the input is a sequence — the textbook use case for recurrent and convolutional networks.

27.3 System model of OCEAN

27.3.1 What is predicted and from what

The abstract states that the authors first identify several important features affecting the CSI of a radio link, and that a data sample consists of the information of these features together with the CSI. The follow-up work by Pecorella *et al.*, which adopts this data model and cites OCEAN for it, lists the factors as: the **frequency band** f_n out of a set $\{f_1, \ldots, f_N\}$ of 5G bands; the **location** l_m, expressed as a sub-region of the cell; the **time slot** τ_p; and the **weather**, quantised into nine levels (sunny, lightly cloudy, cloudy, lightly rainy, medium rainy, heavily rainy, light snow, medium snow, heavy snow); humidity and temperature are also named as factors. One sample is then a tuple

$$\text{STATE} = \big(f_n,\ l_m,\ \tau_p,\ w_r,\ \ldots,\ \text{CSI}\big), \tag{27.4}$$

and the history of such samples is arranged as a two-dimensional "pixel image" that the CNN can process. In our notation, write $c_n(t)$ for the CSI on band n at time t for a given link, $\mathbf{c}(t) = [c_1(t), \dots, c_N(t)]^{\mathsf{T}}$, and $\mathbf{u}(t)$ for the context vector (location, temperature, humidity, weather). The prediction problem is

$$\hat{\mathbf{c}}(t+k) = g_\theta\Big(\mathbf{c}(t-W+1), \dots, \mathbf{c}(t);\ \mathbf{u}(t-W+1), \dots, \mathbf{u}(t+k)\Big), \qquad k = 1, \dots, H, \tag{27.5}$$

where W is the history length, H the horizon and g_θ the network with parameters θ. Including *future* context $\mathbf{u}(t+k)$ is a modelling choice we make in our reproduction: weather and temperature are known from sensors or forecasts, and location can be extrapolated from the trajectory.

Pitfall: "CSI" means different things in different papers

In MIMO-OFDM theory CSI is the complex matrix $\mathbf{H}[k]$ for every subcarrier. OCEAN, by contrast, predicts a scalar *channel quality value per link and band* from context and history; a later paper that compares against OCEAN describes the baselines as using estimated SNR from pilots. Because we could not read the full text, we treat the predicted quantity as a received-quality value such as SNR in dB, and our reproduction predicts exactly that. Keep this distinction in mind when you compare with the 3GPP "CSI prediction" of Section 27.7, which predicts the full channel.

27.3.2 The learning framework

Figure 27.2 is our block diagram of the OCEAN pipeline as described in the abstract: a CNN extracts features from the CSI/feature image, an LSTM models the temporal evolution, and an output layer produces the predicted CSI. Training happens in two steps.

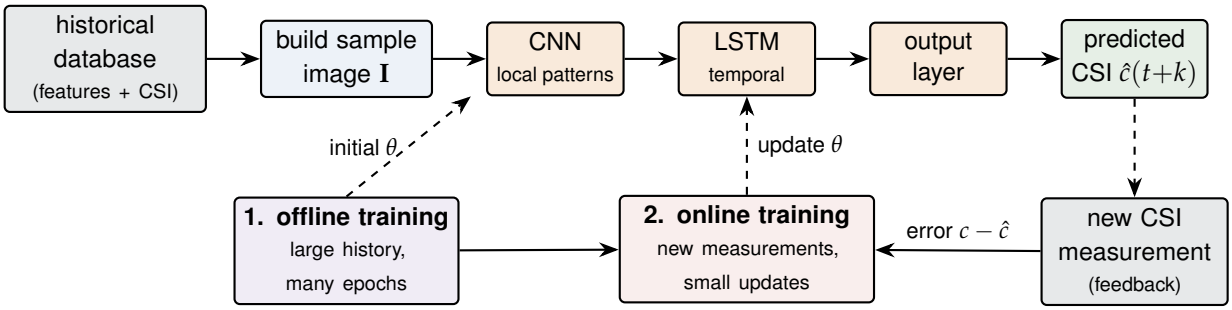


Figure 27.2: The OCEAN idea as a block diagram (our drawing, based on the abstract). Historical samples (features plus CSI) are arranged as an image, passed through a CNN and an LSTM, and mapped to the predicted CSI. The parameters θ are first trained offline on a large history; during operation, newly measured CSI values are compared with the predictions and used for online updates, which keeps the predictor matched to current conditions.

27.3.3 Offline–online two-step training

Offline training minimises the mean-squared error over a large historical data set $\mathcal{D}_{\text{off}}$:

$$\theta_0 = \arg\min_\theta \ \frac{1}{|\mathcal{D}_{\text{off}}|} \sum_{(\text{history}, \mathbf{c}_{\text{fut}}) \in \mathcal{D}_{\text{off}}} \ \frac{1}{HN} \sum_{k=1}^{H} \left\| \hat{\mathbf{c}}(t+k) - \mathbf{c}(t+k) \right\|^2 . \tag{27.6}$$

This is ordinary supervised learning with Adam or SGD. The online step starts from θ_0 and, as each new measurement arrives, takes a few gradient steps on recent data only:

$$\theta_{j+1} = \theta_j - \eta_{\text{on}} \nabla_\theta \frac{1}{|\mathcal{B}_j|} \sum_{\mathcal{B}_j} \|\hat{\mathbf{c}} - \mathbf{c}\|^2 , \tag{27.7}$$

where $\mathcal{B}_j$ is a small batch drawn from the most recent measurements and η_{on} a small learning rate. Why two steps? The offline phase captures the general structure (how SNR responds to weather, how fast fading decorrelates), which needs a lot of data and many epochs. The online phase tracks what the history did not contain: a new environment, a seasonal shift, a re-tilted antenna. Starting the online phase from θ_0 rather than from random weights is *transfer learning* (Chapter 19): it needs far fewer fresh samples. The abstract describes the benefit as making predictions more stable in practical systems.

27.3.4 How the paper measures accuracy

The abstract reports the *average difference ratio* (ADR) between predicted and measured CSI, in percent. We could not read its exact definition; a natural reading is the average relative error

$$\text{ADR} = \frac{100\,\%}{M} \sum_{i=1}^{M} \frac{|\hat{c}_i - c_i|}{|c_i|}. \tag{27.8}$$

Be careful when comparing relative errors across papers: the value depends strongly on the units (dB or linear) and on the mean level of c. A 1 dB error on a 20 dB SNR is a 5 % "difference ratio" in dB but a 26 % error in linear power. In our reproduction we therefore report the *root-mean-square error in dB*, the unit that matters for link adaptation (one CQI step corresponds to roughly 1–2 dB of SNR).

Worked example: three error metrics on the same predictions

Measured SNRs $[11, 12, 7, 12]$ dB, predicted $[10, 12, 9, 11]$ dB. Errors: $-1, 0, +2, -1$. MAE $= (1 + 0 + 2 + 1)/4 = 1.0$ dB. RMSE $= \sqrt{(1 + 0 + 4 + 1)/4} = \sqrt{1.5} = 1.22$ dB (RMSE punishes the single 2 dB miss more). Relative error in dB: $(1/11 + 0 + 2/7 + 1/12)/4 = (0.091 + 0 + 0.286 + 0.083)/4 = 11.5\,\%$. Same predictions, three very different-looking numbers.

27.4 Results reported in the paper

From the abstract, which is all we could verify:

- Four case studies were considered: **two outdoor and two indoor** scenarios, with extensive experiments in each.
- OCEAN obtains the predicted CSI values **very quickly** — the motivation was precisely that classical estimation is too slow and complex for 5G.
- Prediction is **highly accurate**: the ADR between predicted and measured CSI is between **2.650 % and 3.457 %** across the scenarios.

Independent follow-up work gives a second view. Pecorella, Fantacci and Picano (*Sensors* 2020) adopt OCEAN's data model and replace the recurrent part by a deep echo-state network, reporting better accuracy than the CNN-based approach on their own data. Another group, proposing a

"blind" CSI predictor for vehicle-to-infrastructure millimetre-wave links (AIMCOM2 workshop, 2020), re-implemented OCEAN as a baseline and reported an average NMSE of 1.76×10^{-2} for OCEAN against 5.92×10^{-2} for a plain LSTM and 9.14×10^{-3} for their own method, in their setting. These numbers are not comparable to the original ADR, but they confirm the qualitative picture: CNN+LSTM beats a plain LSTM, and the field moved on quickly.

27.5 Our small-scale reproduction

We cannot reproduce the authors' measurements, so we build a synthetic world in which the same questions can be asked honestly: *Does a learned predictor beat persistence and AR? Does side information help? Does offline-then-online training help when the environment changes?* All code is in `code/ch27/`; every number below comes from running it.

27.5.1 The synthetic channel

Each "link" is one user observed for $T = 1200$ samples spaced $T_s = 1\,\text{ms}$ on $N = 8$ sub-bands spaced 0.5 MHz. The measured SNR in dB is the sum of five components:

$$\text{SNR}_n(t) = \underbrace{\mu}_{\text{distance}} + \underbrace{s(t)}_{\text{shadowing}} + \underbrace{o(t)}_{\text{context}} + \underbrace{10\log_{10}|h_n(t)|^2}_{\text{fast fading}} + \underbrace{e_n(t)}_{\text{meas. noise}} . \tag{27.9}$$

- $\mu \sim \mathcal{U}(8, 24)$ dB is a per-link mean (path loss).
- $s(t)$ is shadowing: a first-order AR process in dB, $s(t) = a\,s(t-1) + \varepsilon(t)$ with $a = e^{-1/400}$ (correlation time 400 ms) and a stationary standard deviation of 3 dB.
- $o(t) = -1.2\,w(t) - 0.04\,(\text{hum}(t) - 50) + 0.05\,(\text{temp}(t) - 20)$ is the context effect: each weather level costs 1.2 dB, with weather $w \in \{0, \dots, 8\}$ changing abruptly 2–5 times per link (rain starts or stops), and humidity and temperature drifting slowly.
- $h_n(t) = \sum_{p=1}^{24} \sqrt{P_p}\, e^{j(2\pi f_D \cos\alpha_p t - 2\pi f_n \tau_p + \phi_p)}$ is a 24-path Clarke model with uniform angles α_p, an exponential power-delay profile with 0.3 μs mean delay, and Doppler $f_D \sim \mathcal{U}(10, 30)$ Hz. The delays make the sub-bands differently faded; the Doppler makes them change in time.
- $e_n(t) \sim \mathcal{N}(0, 0.5^2)$ dB is estimation noise.

Figure 27.3 shows one link. The "image" view in the top panel is exactly the representation the CNN sees: fades appear as dark streaks that are correlated across neighbouring sub-bands.

27.5.2 Training data and predictors

We generate 160 training links and 40 test links (fresh random seeds) and cut sliding windows: $W = 24$ ms of history, $H = 20$ future steps (1–20 ms), stride 16. This gives 11 680 training and 2 920 test windows. SNR is normalised as $(\text{SNR} - 15)/8$; context features are scaled to roughly unit range. We compare four predictors:

1. **Persistence**: $\hat{\mathbf{c}}(t+k) = \mathbf{c}(t)$ for all k.
2. **AR(8)**: per sub-band, the last 8 values plus a bias; one least-squares weight vector per horizon (direct prediction), fitted on all training windows.
3. **LSTM**: SNR history only, 48 hidden units, a linear head that outputs all $H \times N = 160$ values, added to the last measured value (a *residual* formulation: the network predicts the change).

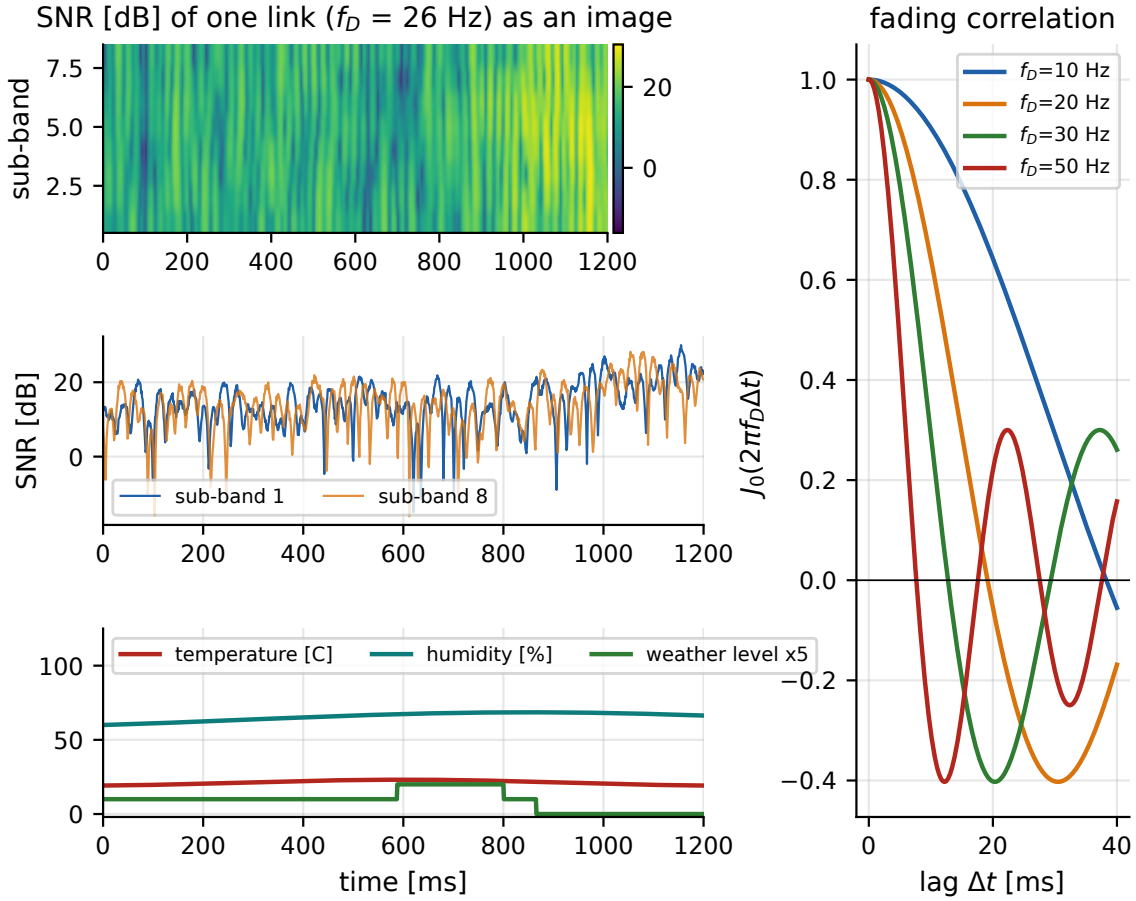


Figure 27.3: One synthetic link (our data). **Top left:** SNR per sub-band and time as an image; dark streaks are fades, partly shared by neighbouring sub-bands. **Middle:** two sub-bands: fast Rayleigh fades of 10–20 dB ride on slow shadowing; note the step when the weather level changes. **Bottom:** the context features (temperature, humidity, weather level). **Right:** the Jakes correlation $J_0(2\pi f_D \Delta t)$ for f_D = 10–50 Hz; fast fading becomes unpredictable from its own past after a few ms at the higher Doppler values.

4. **CNN+LSTM with context** (our OCEAN-style model, Fig. 27.4 and Table 27.1): the 24×8 history image passes through two 3×3 convolutions; the sub-band axis is pooled to two values, giving 16 features per time step; these are concatenated with the raw SNRs and the 3 context features and fed to an LSTM; the head also receives the context of the 20 future steps.

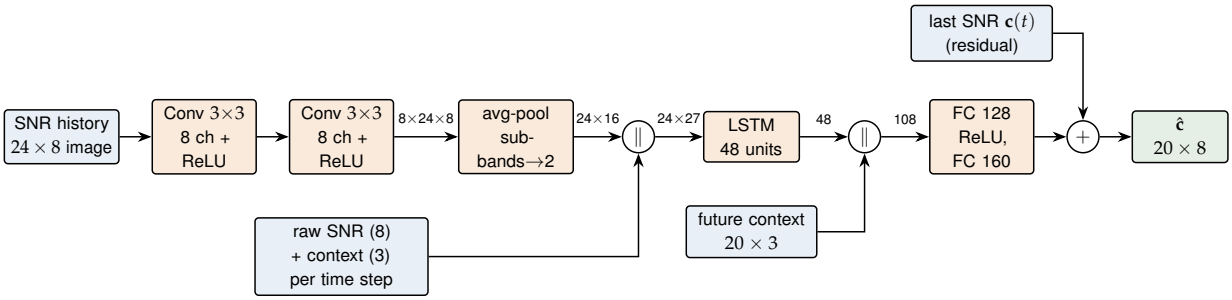


Figure 27.4: Our OCEAN-style CNN+LSTM predictor (50 040 parameters). The CNN turns the time–frequency image into a short feature vector per time step; the LSTM summarises the 24-step sequence; the head combines that summary with the known future context and predicts the *change* relative to the last measurement for 20 steps and 8 sub-bands. $\|$ denotes concatenation.

Table 27.1: Layers and parameter counts of our two neural predictors (verified with PyTorch). For an LSTM with input size n and h hidden units, PyTorch stores $4h(n+h)$ weights and two bias vectors of length $4h$: e.g. $4 \cdot 48 \cdot (8+48) + 8 \cdot 48 = 11\,136$.

Model	Layer	Output shape	Parameters
LSTM	LSTM (input 8, hidden 48)	48	11 136
	Linear $48 \to 160$	20×8	7 840
	total		**18 976**
CNN+LSTM	Conv2d $1 \to 8$, 3×3	$8 \times 24 \times 8$	80
	Conv2d $8 \to 8$, 3×3	$8 \times 24 \times 8$	584
	AdaptiveAvgPool to $(24, 2)$	24×16	0
	LSTM (input $16{+}8{+}3 = 27$, hidden 48)	48	14 784
	Linear $(48+60) \to 128$ + ReLU	128	13 952
	Linear $128 \to 160$	20×8	20 640
	total		**50 040**

Both networks are trained offline for 6 epochs with Adam (learning rate 2×10^{-3}, cosine decay, batch 256) on the MSE loss (27.6); the whole script, including data generation, runs in about three minutes on one CPU thread. The core of the model is short:

```
class CNNLSTMPred(nn.Module):
    """OCEAN-style: a CNN extracts features across the F sub-bands at every time step,
    an LSTM models their evolution together with the context features,
    and the head also sees the (known) future context."""
    def __init__(self, H, ch=8, hidden=48):
        super().__init__()
        self.H = H
        # the W x F history is treated as a one-channel image (time x sub-band);
        # 3x3 convolutions mix neighbouring times and sub-bands, then the
        # sub-band axis is pooled to 2 values, leaving a feature sequence over time
        self.cnn = nn.Sequential(
```

```
            nn.Conv2d(1, ch, 3, padding=1), nn.ReLU(),
            nn.Conv2d(ch, ch, 3, padding=1), nn.ReLU(),
            nn.AdaptiveAvgPool2d((None, 2)))        # -> [B, ch, W, 2]
        self.lstm = nn.LSTM(2 * ch + F + NCTX, hidden, batch_first=True)
        self.head = nn.Sequential(nn.Linear(hidden + H * NCTX, 128), nn.ReLU(),
                                  nn.Linear(128, H * F))

    def forward(self, x, c_future):
        B, W, _ = x.shape
        snr = x[:, :, :F]
        feat = self.cnn(snr.unsqueeze(1))            # [B, ch, W, 2]
        feat = feat.permute(0, 2, 1, 3).reshape(B, W, -1)
        z = torch.cat([feat, snr, x[:, :, F:]], -1)  # CNN features + raw SNR +
context
        _, (h, _) = self.lstm(z)
        last = snr[:, -1].unsqueeze(1)
        out = self.head(torch.cat([h[-1], c_future], -1)).view(-1, self.H, F)
        return last + out
```

27.5.3 Offline results: error versus horizon

Figure 27.5 is the main result. All four methods start at about 1.4 dB RMSE at a 1 ms horizon — there the error is dominated by the 0.5 dB measurement noise and by the residual fading change in one millisecond. As the horizon grows, persistence degrades fastest (7.65 dB at 20 ms), AR(8) reaches 6.65 dB, the LSTM 6.15 dB and the CNN+LSTM 6.05 dB. Averaged over all 20 horizons the RMSEs are 5.97 (persistence), 5.32 (AR), 4.93 (LSTM) and 4.85 dB (CNN+LSTM).

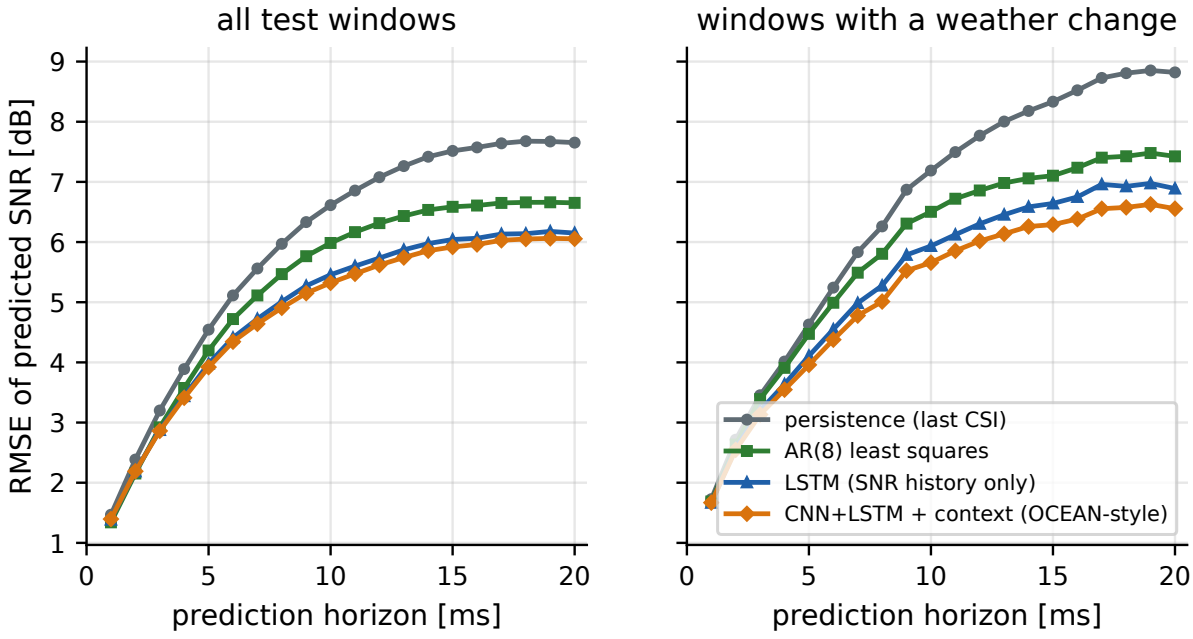


Figure 27.5: Our reproduction: RMSE of the predicted SNR versus horizon on 40 unseen test links. **Left:** all 2 920 test windows. The learned predictors beat persistence by about 1.5 dB at 20 ms; the CNN+LSTM with context is best, but only slightly better than the history-only LSTM. **Right:** the 121 windows in which the weather level changes during the forecast. Here persistence and AR suffer most (8.8 and 7.4 dB at 20 ms) and the context-aware CNN+LSTM has a clearer advantage (6.55 dB versus 6.89 dB for the LSTM).

Two observations deserve emphasis. First, the curves flatten after about 10 ms: once the fast fading has decorrelated (Fig. 27.3, right), no method can predict the next fade, and the best any predictor can do is forecast the slowly varying *mean* — shadowing plus context. The remaining 6 dB error is essentially the standard deviation of Rayleigh fading in dB (about 5.6 dB) plus noise. This is the same ceiling that (27.3) predicts: when $\rho \to 0$ the error tends to the full variance of the fast component. Second, the value of context shows up mainly where context *changes*. In ordinary windows the weather is constant and the LSTM can infer its effect from the recent SNR level; only when rain starts or stops inside the forecast window does knowing the future weather help, and there the CNN+LSTM gains about 0.3–0.4 dB over the history-only LSTM at long horizons.

Figure 27.6 shows forecasts on a fresh link. Each orange segment is one 20-step forecast issued at the start of the segment. The predictor follows the slow trend and the beginning of each fade, but it cannot foresee fades that start more than a few milliseconds ahead. Around $t = 480$ ms the weather level steps up and the forecasts immediately shift downwards, while persistence (dotted) keeps the old level.

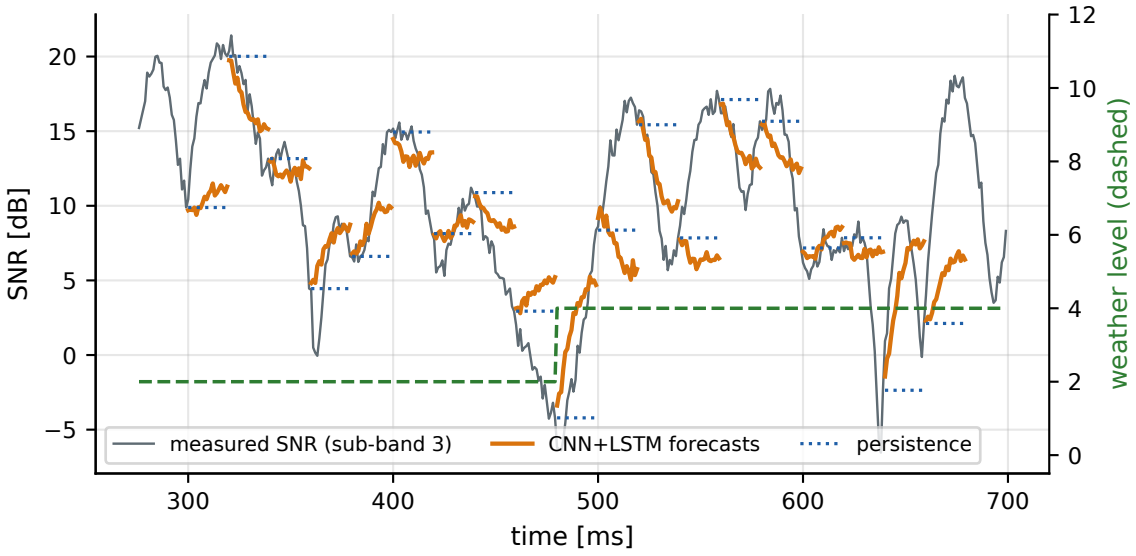


Figure 27.6: Our reproduction: 20-ms forecasts by the offline CNN+LSTM on an unseen link (one sub-band shown). Gray: measured SNR. Orange: forecasts, one per 20 ms. Blue dotted: persistence. Green dashed: weather level (right axis). Short-term continuation of fades and the response to the weather step are learned; deep fades several ms ahead are not predictable.

27.5.4 Offline then online: adapting to a new environment

To test the two-step idea we deploy the offline model in *environment B*, where users move faster (Doppler 35–55 Hz instead of 10–30 Hz), each weather level costs 2.0 dB instead of 1.2 dB, humidity hurts twice as much and there is an extra 2 dB loss. Measurements from 24 new links arrive as a stream of 1 752 windows. After each block of 64 new windows we take 4 Adam steps on mini-batches of 64 drawn from the most recent 256 windows (a small sliding memory), and we evaluate on 16 separate environment-B test links. We compare: the frozen offline model; the same model with online fine-tuning (learning rate 5×10^{-4}); a model of the same architecture trained online from scratch (learning rate 2×10^{-3}); and persistence.

Figure 27.7 confirms the logic of OCEAN's two steps. After only 64 new windows, the fine-tuned model is at 6.04 dB while the scratch model is still at persistence level (7.17 dB); after 1 024 windows the numbers are 5.86 versus 6.61 dB; at the end of the stream 5.70 versus 6.22 dB.

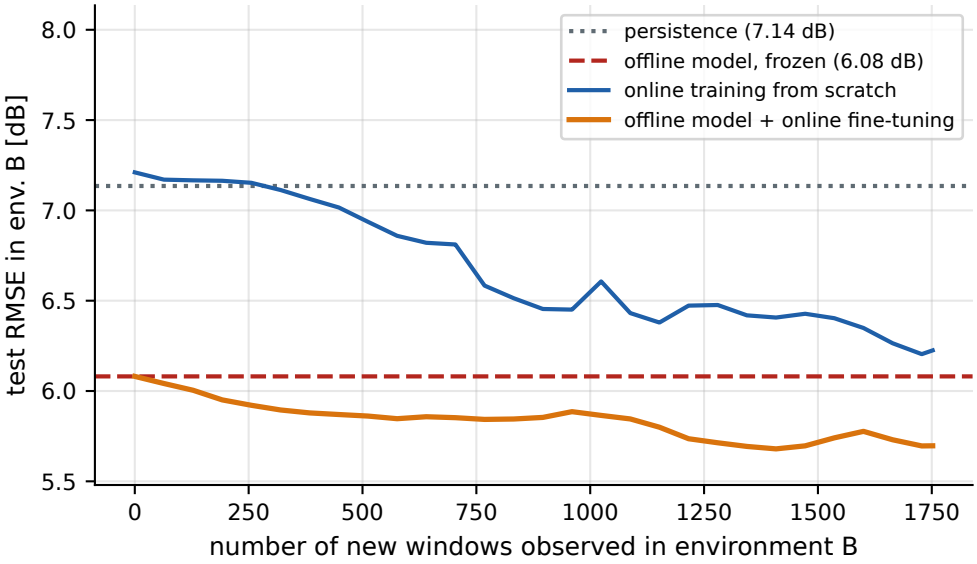


Figure 27.7: Our reproduction of the offline–online idea. In the new environment, the frozen offline model (red dashed, 6.08 dB) is already far better than persistence (7.14 dB). Online fine-tuning (orange) improves it to about 5.7 dB within a few hundred windows. A model trained online from scratch (blue) needs the whole stream to approach the frozen offline model and never catches the fine-tuned one.

Offline training supplies general knowledge (how fading and weather shape the SNR), and online training corrects what is specific to the new place. The fine-tuning curve is slightly noisy because every update uses only recent data; a smaller learning rate gives a smoother but slower curve — the classic stability–plasticity trade-off.

Pitfall: honest comparison with the paper

Our numbers are *not* comparable with the paper's 2.65–3.46 % ADR. Our channel contains strong Rayleigh fading at millisecond resolution, which no predictor can foresee beyond the coherence time; the paper's measured CSI values and their sampling interval are unknown to us and may be considerably smoother. What our experiment does reproduce qualitatively is the paper's story: a CNN+LSTM that uses history and context beats simple baselines, and offline training followed by online updates gives accurate predictions quickly in a changing environment.

Pitfall: leakage when you build the windows

Windows cut from the same link overlap in time. If you shuffle them randomly into training and test sets, the test set contains almost the same samples as the training set and the error looks far too good. Always split by *link* (or by day, or by location), as we did, and in the online experiment evaluate on links that never appear in the stream.

27.6 Limitations and follow-ups

- **Scalar CSI.** Predicting a quality value per band is enough for link adaptation and scheduling, but beamforming and MU-MIMO need the full complex channel. Later work predicts **H** itself with LSTMs, GRUs, transformers or graph networks, typically on 3GPP channel models.

- **Context availability.** Weather and temperature are cheap to obtain; location is not always known at the gNB, and the gain from context is concentrated in rare events (weather changes in our experiment).
- **Generalisation.** A model trained in one cell may not transfer to another; online fine-tuning helps, but needs labelled data (actual measurements) and a policy for when to update and how to avoid forgetting.
- **Fundamental horizon.** Fast fading is predictable only over a fraction of the coherence time; beyond it any predictor converges to the slow mean (Fig. 27.5). Gains must be judged against this ceiling, not against zero error.
- **Alternatives.** Follow-ups replaced the LSTM by echo-state networks (Pecorella *et al.*, 2020) or predicted CSI blindly from received signals rather than historical CSI (the AIMCOM2 2020 work mentioned above). Classical Wiener/Kalman predictors with a Doppler model remain strong baselines whenever the Jakes model is accurate.

27.7 From paper to practice in 5G NR and 6G

CSI aging in numbers. At $f_c = 3.5\,\text{GHz}$ a pedestrian at 3 km/h has $f_D \approx 9.7\,\text{Hz}$ and J_0 at 5 ms is 0.98: CSI that is 5 ms old is almost perfect. A car at 30 km/h has $f_D \approx 97\,\text{Hz}$ and the correlation at 5 ms is already -0.28: the reported CSI says almost nothing about the fast fading at the moment of transmission. This is why MU-MIMO gains measured in the lab disappear on moving users and why CSI prediction is attractive.

CSI-RS periodicity and reporting. NR configures periodic, semi-persistent or aperiodic CSI-RS and CSI reports by RRC. Periodic resources have periodicities from 4 to 640 slots; at 30 kHz subcarrier spacing (0.5 ms slots) a period of 10 slots means a fresh measurement every 5 ms. Denser CSI-RS reduce aging but cost resource elements and UE power; prediction is the alternative that trades computation for overhead. In the simplest deployment, a gNB-side predictor of OCEAN's type — forecasting CQI or SNR from the history of reports plus context — needs no standard change at all: it can sit inside the scheduler's link adaptation, next to the usual outer-loop offset driven by HARQ ACK/NACK.

3GPP Release 18 and 19. Release 18 contained the first 3GPP study on AI/ML for the NR air interface (TR 38.843), which examined three representative use cases: CSI feedback enhancement, beam management and positioning. For CSI feedback, two sub-use cases were studied: spatial-frequency CSI compression with a two-sided model, and *time-domain CSI prediction with a UE-side model* — the UE observes past CSI-RS and forecasts the channel for a future time instance. Release 18 MIMO work in parallel enhanced CSI reporting for high and medium UE velocities. In Release 19, RAN1 continued with normative work that, according to 3GPP's Release 19 overview, covers CSI prediction, beam management and positioning, while CSI compression remained under study. Note the difference from OCEAN: the 3GPP use case predicts the *channel* (for precoder reporting) at the UE, from CSI-RS history, under a common life-cycle framework (data collection, model monitoring, fallback to legacy CSI), whereas OCEAN predicts a link-quality value from history and context.

6G. Proposals for AI-native air interfaces treat prediction as a building block: predicting CSI, beams and blockage from sensing data (camera, radar, positioning) and from integrated sensing and communication. The OCEAN lesson carries over: use every cheap predictor of the channel you have, train offline on large data, and adapt online.

Wireless connection: a practical recipe for a gNB-side CQI predictor

(1) Log, per UE, the CQI/SNR reports with timestamps, band and a coarse position (beam index or timing advance) plus cell-level context (weather, load). (2) Train offline per cell family with a split by day. (3) Deploy with a fallback: use the prediction only when its recent error (tracked online) is below that of persistence. (4) Fine-tune online on recent reports with a small learning rate. This mirrors Fig. 27.2 and is exactly the life-cycle logic later formalised by 3GPP.

Chapter recap

- CSI is always outdated by the time it is used; with the Jakes model, persistence costs $2(1 - J_0(2\pi f_D \tau))$ and the best one-tap linear predictor $1 - J_0^2$.
- OCEAN (Luo *et al.*, IEEE TNSE 2020) predicts CSI from historical data using a sample that combines influencing features (band, location, time, weather and related context) with CSI, a CNN+LSTM framework, and offline–online two-step training; the abstract reports 2.650–3.457 % average difference ratio in two outdoor and two indoor scenarios.
- In our reproduction, learned predictors beat persistence by about 1.5 dB at 20 ms; context mainly helps when context changes; and fine-tuning an offline model in a new environment reaches good accuracy far faster than training from scratch.
- Fast fading is predictable only within a fraction of the coherence time; beyond that only the slow mean (shadowing, context) can be forecast.
- In NR, CSI-RS periodicity and reporting delay set the CSI age; 3GPP studied UE-side CSI prediction in Release 18 and continued AI/ML normative work in Release 19.

Exercises

1. **(numeric)** A UE moves at 60 km/h at $f_c = 3.5$ GHz. Compute f_D, and the persistence and one-tap linear prediction errors (27.2)–(27.3) for $\tau = 2$ ms. (Use $J_0(x) \approx 1 - x^2/4 + x^4/64$.)
2. **(numeric)** For what value of ρ does persistence give exactly the same MSE as predicting zero? What CSI age does this correspond to at $f_D = 20$ Hz? (The first solution of $J_0(x) = 0.5$ is $x \approx 1.52$.)
3. **(numeric)** Fit AR(1) with bias, $\hat{x}_{t+1} = ax_t + b$, to $x = [2, 4, 3, 5, 4]$ by least squares (solve the 2×2 normal equations).
4. **(conceptual)** Why does our residual formulation (predicting the change relative to the last measurement) help training? What would happen at a 1 ms horizon without it?
5. **(conceptual)** In Fig. 27.5, the CNN+LSTM gains most on windows with a weather change. Explain why context gives little benefit on the other windows, and design a data set in which context would help everywhere.
6. **(numeric)** Verify the parameter count of the first LSTM in Table 27.1 and compute the count for the same LSTM with 64 hidden units.
7. **(conceptual)** Name two risks of online fine-tuning in a live network and one safeguard for each.

8. **(coding)** Change `online_finetune.py` so that the stream is *not* shuffled (windows arrive link by link). Predict and then observe what happens to the fine-tuning curve.

Answers

1. $v = 16.67\,\text{m/s}$, $\lambda = c/f_c = 0.0857\,\text{m}$, $f_D = 194.4\,\text{Hz}$. $x = 2\pi f_D \tau = 2\pi \cdot 194.4 \cdot 0.002 = 2.443$. Series: $1 - x^2/4 + x^4/64 = 1 - 1.492 + 0.557 = 0.065$; the exact value is $J_0(2.443) = -0.02$ (the truncated series is rough for $x > 2$). Either way $\rho \approx 0$. Persistence: $2(1-\rho) \approx 2.0$ — twice as bad as predicting zero. Linear predictor: $1 - \rho^2 \approx 1.0$, i.e. no information left. At 60 km/h, 2 ms-old CSI is useless for the fast fading.

2. $2(1-\rho) = 1 \Rightarrow \rho = 0.5$. $J_0(2\pi f_D \tau) = 0.5$ at $2\pi f_D \tau \approx 1.52$, so $\tau = 1.52/(2\pi \cdot 20) = 12.1\,\text{ms}$.

3. Pairs (x_t, x_{t+1}): $(2,4), (4,3), (3,5), (5,4)$. Sums: $\sum x_t = 14$, $\sum x_t^2 = 4 + 16 + 9 + 25 = 54$, $\sum x_{t+1} = 16$, $\sum x_t x_{t+1} = 8 + 12 + 15 + 20 = 55$, $n = 4$. Normal equations: $54a + 14b = 55$, $14a + 4b = 16$. From the second, $b = 4 - 3.5a$; substituting, $54a + 56 - 49a = 55 \Rightarrow 5a = -1 \Rightarrow a = -0.2$, $b = 4.7$. The fitted model oscillates around $4.7/1.2 = 3.92$, matching the zig-zag data.

4. The future SNR is close to the last value for short horizons, so the target change is small and centred near zero; the network only has to learn corrections and starts out as a persistence predictor (head outputs are near zero at initialisation). Without it, the network must reproduce the absolute level of the last value through all its layers; early in training the 1 ms prediction would be much worse than persistence and convergence slower.

5. Within a window where weather is constant, its effect is already contained in the recent SNR level, which the LSTM sees; the context adds no new information. Context helps only when it changes inside the horizon (rain onset), or when the history is short or missing. A data set where context helps everywhere: predict for a *new* link with no history (cold start), or use long horizons (hours) where weather and time of day dominate.

6. $n = 8$, $h = 48$: $4 \cdot 48 \cdot (8 + 48) = 10\,752$ weights plus $2 \cdot 4 \cdot 48 = 384$ biases $= 11\,136$. With $h = 64$: $4 \cdot 64 \cdot 72 = 18\,432$ plus $512 = 18\,944$.

7. (a) Corrupted or unrepresentative data (a faulty sensor, an outage) can drag the model away — safeguard: monitor the error and roll back to the offline weights when it exceeds persistence. (b) Catastrophic forgetting of rare situations that do not appear in recent data (snow in summer) — safeguard: replay a small buffer of historical data during online updates, or limit the learning rate/number of steps.

8. Without shuffling, consecutive updates see only one link at a time, so the model chases that link's particular mean and Doppler and the test error becomes noisy and may temporarily increase; the improvement over the frozen model shrinks. A larger replay memory (spanning several links) restores the benefit.

Self-test quiz

1. What does OCEAN stand for, conceptually, and which two network types does it integrate?
   An online CSI prediction scheme; it integrates a CNN and an LSTM.
2. What are the two steps of OCEAN's training mechanism, and why two?
   Offline training on a large historical data set, then online training with new measurements; the first learns general structure, the second keeps predictions stable and matched to current conditions.
3. Under the Jakes model, at what correlation does persistence become worse than predicting the mean?
   When $\rho < 0.5$, since its MSE $2(1-\rho)$ then exceeds 1.
4. Why do all predictors' errors flatten at long horizons in our reproduction?
   Fast fading has decorrelated; only the slow mean (shadowing and context) remains predictable, so the error approaches the fading variance.
5. What accuracy does the paper's abstract report, and in how many scenarios?
   An average difference ratio of 2.650–3.457 % in four scenarios (two outdoor, two indoor).
6. How does 3GPP's Release-18 "CSI prediction" differ from OCEAN?
   It is a UE-side model predicting the future channel from past CSI-RS for CSI reporting, within a standard AI/ML life-cycle framework; OCEAN predicts a link-quality CSI value from history and context features.

Further reading

- C. Luo, J. Ji, Q. Wang, X. Chen, P. Li, "Channel State Information Prediction for 5G Wireless Communications: A Deep Learning Approach," *IEEE Trans. Network Science and Engineering*, vol. 7, no. 1, pp. 227–236, 2020.
- T. Pecorella, R. Fantacci, B. Picano, "Improving CSI Prediction Accuracy with Deep Echo State Networks in 5G Networks," *Sensors*, vol. 20, no. 22, 6475, 2020.
- X. Lin, "An Overview of the 3GPP Study on Artificial Intelligence for 5G New Radio," arXiv:2308.05315, 2023.
- 3GPP TR 38.843, "Study on Artificial Intelligence (AI)/Machine Learning (ML) for NR Air Interface," Release 18.
- S. Hochreiter, J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, 1997 — the LSTM of Chapter 16.

Chapter 28

Predicting Wireless Channel Features using Neural Networks

A millimetre-wave base station with a hundred antennas can point a pencil beam at a user — but only if it knows where to point, and only if the user points its own (smaller) array back. Finding the right pair of beams by trying them all costs time and reference signals, and at 28 GHz the channel changes quickly enough that the search must be repeated often. The paper in this chapter asks a simple, data-driven question: the base station can *see* some features of the channel (received power, delays and arrival directions of the paths); can a neural network trained on many examples *infer* features it cannot see — in particular the directions in which the user's own array should transmit?

The same idea appears throughout modern wireless systems under different names: predicting the downlink from the uplink in FDD, predicting beams from position, building radio maps from sparse measurements, or "channel charting". This chapter teaches all of it through one concrete paper. You will learn what "channel features" are and how to compute them from a multipath channel; why features observed at one end of a link are statistically linked to features at the other end; how the paper encodes the base station's view as a vector over angular bins and the user's view as a beam-indicator vector; how a small multilayer perceptron (Chapter 7) is trained to map one to the other; and how to evaluate it with normalised errors and ROC curves. We then build our own geometric street-canyon simulator, repeat the paper's experiment, and extend it to the related task of predicting channel features from user location.

Paper at a glance

Title: Predicting Wireless Channel Features using Neural Networks.
Authors: Shiva Navabi (University of Southern California), Chenwei Wang, Ozgun Y. Bursalioglu, Haralabos Papadopoulos (DOCOMO Innovations, Palo Alto).
Venue: IEEE International Conference on Communications (ICC) 2018, Kansas City; preprint arXiv:1802.00107 (Feb. 2018).
Problem: at a large-array base station (BS), estimate channel features that are observable only at the user side — the angles of departure (AoD) of the dominant paths at the user — from features the BS observes (received signal strength (RSS), multipath delays, angles of arrival (AoA)).
Key idea: represent the BS observation as a vector over the BS's DFT (virtual-angle) bins, holding normalised RSS or delay in each occupied bin; represent the target as an indicator vector over the user's DFT bins; learn the map with a two-hidden-layer neural network trained on ray-tracing data.
Data: ray-tracing channels at 28 GHz for Shinjuku Square, Tokyo (150 m × 120 m, 13 BSs, 13 609 user spots on a 1-m grid), calibrated against measurements.
Main result: BS-side and user-side features are strongly correlated (normalised mutual information close to 0.97 at high BS resolution); all neural predictors beat the sample-average baseline; RSS is a more reliable predictor than delay; RSS-based and combined schemes detect

the dominant user beams with probability at least 0.9 at a false-alarm probability of 0.1.

28.1 The problem: features you cannot see

Intuition: hearing an echo and guessing where the speaker faces

Stand in a courtyard with your eyes closed while a friend speaks. You hear the direct sound and a few echoes from the walls; you can tell *from which directions* they reach you and how loud each is. Could you guess in which direction your friend's mouth is pointing? Not by pure geometry alone — but if you had stood in that courtyard a thousand times with your eyes open and learned the pattern "when the loud echo comes from the left wall, the speaker is facing north-west", you could make an excellent guess. The base station is in the same position: it hears the user's pilot via a few paths, and it wants to know the directions in which the paths *left* the user. The building geometry links the two; a neural network learns the link from data.

28.1.1 Why the user-side angle matters at millimetre wave

At 28 GHz the wavelength is $\lambda = c/f_c = 10.7\,\text{mm}$. An array of N elements with half-wavelength spacing has an aperture of $N\lambda/2$: 100 elements span 53.6 cm and 10 elements 5.4 cm — the "roughly 53 cm and 4.8 cm" apertures the paper quotes for its BS and user arrays. Such arrays form narrow beams, and narrow beams are essential because free-space loss at 28 GHz is large: at 50 m, $20\log_{10}(4\pi d/\lambda) = 95.4\,\text{dB}$. Both ends must beamform to close the link, which means both ends must know the directions of the good paths.

Beam training (in 5G NR: SSB sweeps followed by CSI-RS beam refinement) finds these directions by trial. With N_{BS} beams at the BS and N_{MS} at the user, an exhaustive search tests $N_{\text{BS}}N_{\text{MS}}$ pairs, and the paper stresses that hierarchical searches need receiver feedback, which is itself overhead. If the BS could *predict* which user beams will be good, it could tell the user to test only those — or skip the user-side search altogether.

28.1.2 What the BS can and cannot observe

Figure 28.1 sets up the geometry. The user transmits an uplink pilot. The channel consists of L paths; path l has complex amplitude α_l, delay τ_l, AoA θ_l at the BS and AoD ϕ_l at the user. With its large array the BS can resolve the paths in angle and delay, and so observes $\{r_l, \tau_l, \theta_l\}$ where $r_l = |\alpha_l|^2$ is the received power of path l — all subject to its finite resolution. The angles ϕ_l, however, describe how the paths *left* the user, and they are invisible at the BS. The paper's question, in its own framing: can the BS extract features that are only available at the user side, based on what it observes?

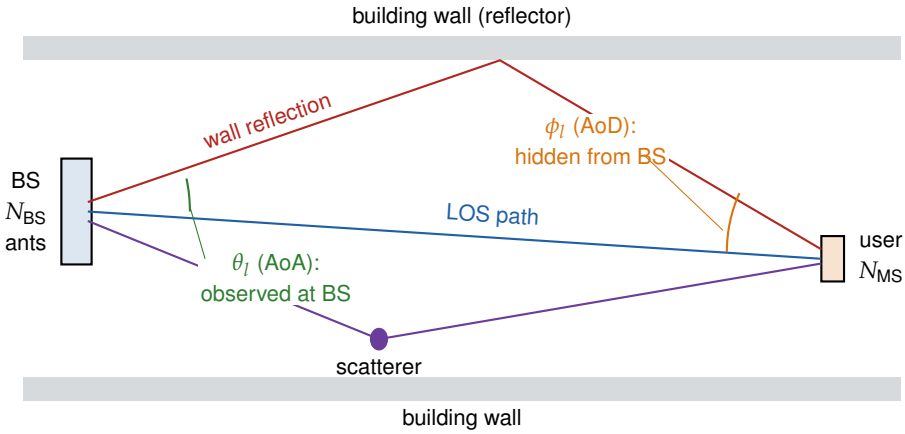


Figure 28.1: The prediction problem. Each path has a power r_l, a delay τ_l, an angle of arrival θ_l at the BS and an angle of departure ϕ_l at the user. The BS measures $\{r_l, \tau_l, \theta_l\}$ from the uplink pilot; ϕ_l is seen only at the user. The fixed geometry of walls and scatterers ties the two sides together.

28.2 Background: channel features

Definition: channel features

A *channel feature* is any compact statistic computed from the multipath channel $\{\alpha_l, \tau_l, \theta_l, \phi_l\}_{l=1}^{L}$ that is useful for system design. Common examples: total received power $P = \sum_l |\alpha_l|^2$; mean delay and RMS delay spread; RMS angle spread at either end; the set of dominant angular bins (beams); the number of significant paths; LOS/NLOS state.

For powers $p_l = |\alpha_l|^2$ and delays τ_l, the power-weighted mean delay and the RMS delay spread are

$$\bar{\tau} = \frac{\sum_l p_l \tau_l}{\sum_l p_l}, \qquad \sigma_\tau = \sqrt{\frac{\sum_l p_l (\tau_l - \bar{\tau})^2}{\sum_l p_l}}, \tag{28.1}$$

and the RMS angle spread is defined the same way with angles (taking care to wrap differences into $(-\pi, \pi]$). The delay spread sets the coherence bandwidth ($B_c \approx 1/(5\sigma_\tau)$ is a common rule of thumb) and hence the cyclic-prefix and pilot-spacing requirements; the angle spread tells how many beams are needed to capture most of the energy.

Worked example: features of a three-path channel

Three paths with powers $p = [1, 0.5, 0.25]$ (linear, relative) and delays $\tau = [100, 150, 250]$ ns. Total power $P = 1.75$ ($+2.43$ dB relative to the first path). Mean delay: $\bar{\tau} = (100 + 75 + 62.5)/1.75 = 237.5/1.75 = 135.7$ ns. Squared deviations: $(100 - 135.7)^2 = 1276$, $(150 - 135.7)^2 = 204$, $(250 - 135.7)^2 = 13\,061$; weighted sum $1276 + 0.5 \cdot 204 + 0.25 \cdot 13\,061 = 1276 + 102 + 3265 = 4643$. Delay spread $\sigma_\tau = \sqrt{4643/1.75} = \sqrt{2653} = 51.5$ ns. Note how the weak but late third path dominates the spread.

28.2.1 Virtual angles and DFT beams

A uniform linear array (ULA) of N elements with spacing $\lambda/2$ responds to a plane wave from angle θ (measured from broadside) with the steering vector

$$\mathbf{a}_N(\theta) = \left[1,\ e^{j\pi \sin\theta},\ e^{j2\pi \sin\theta},\ \ldots,\ e^{j(N-1)\pi \sin\theta}\right]^{\mathsf{T}}. \tag{28.2}$$

The *virtual angle* $u = \sin\theta \in [-1, 1]$ enters linearly in the phase. The N columns of the DFT matrix are steering vectors at the equally spaced virtual angles $u_k = -1 + (2k+1)/N$, $k = 0, \ldots, N-1$, and they are orthogonal. A DFT codebook therefore splits the virtual-angle axis into N bins of width $2/N$; a path with virtual angle u falls into bin

$$k(\theta) = \left\lfloor \frac{(\sin\theta + 1)}{2} N \right\rfloor. \tag{28.3}$$

A large array has many narrow bins (it can resolve paths that are close in angle); a small array has a few wide bins. With $N = 10$ a path at $\theta = 30°$ ($u = 0.5$) falls into bin $\lfloor 0.75 \cdot 10 \rfloor = 7$; with $N = 100$ a path at $-20°$ ($u = -0.342$) falls into bin 32.

Wireless connection: DFT bins are the beams of 5G NR

NR Type-I codebooks and SSB beam sweeps are, to a first approximation, DFT beams (often oversampled). "Which user-side DFT bins hold a strong path" is therefore exactly "which UE beams are worth testing in beam refinement". Predicting it is predicting a beam-management decision.

28.3 System model and problem formulation

28.3.1 Channel model

The paper works in two dimensions (azimuth only). With a ULA of N_{BS} elements at the BS and N_{MS} at the user, both with half-wavelength spacing, the narrowband uplink channel matrix is the sum of the L paths,

$$\mathbf{H} = \sum_{l=1}^{L} \alpha_l \, e^{-j2\pi f \tau_l} \, \mathbf{a}_{N_{\mathrm{BS}}}(\theta_l) \, \mathbf{a}_{N_{\mathrm{MS}}}(\phi_l)^{\mathsf{H}}. \tag{28.4}$$

In the DFT (beamspace) domain, $\mathbf{F}_{\mathrm{BS}}^{\mathsf{H}} \mathbf{H} \mathbf{F}_{\mathrm{MS}}$ is approximately sparse: path l shows up near row $k(\theta_l)$ and column $k(\phi_l)$. The rows are what the BS sees; the columns are what the user sees.

28.3.2 Feature representation

The heart of the paper is how both sides are turned into fixed-length vectors that a neural network can digest (Fig. 28.2).

BS input $\mathbf{x} \in \mathbb{R}^{N_{\mathrm{BS}}}$. Each path is assigned to its BS bin $k(\theta_l)$. For the RSS representation, an occupied bin holds the normalised RSS $(r - r_{\min})/(r_{\max} - r_{\min})$, where $r_{\min}, r_{\max}$ are the smallest and largest path RSS in the data set, and empty bins hold zero. For the delay representation, an occupied bin holds a normalised delay, and empty bins hold the value 1, which the paper interprets as a fictitious, very late (non-existent) path. The two can be concatenated into a $2N_{\mathrm{BS}}$ vector.

User output $\mathbf{y} \in \{0, 1\}^{N_{\mathrm{MS}}}$. Each path is assigned to its user bin $k(\phi_l)$, and the output is an *indication vector*: entry k is 1 if some path occupies user bin k, 0 otherwise. The paper's toy example uses $N_{\mathrm{BS}} = 6$ and $N_{\mathrm{MS}} = 4$: three paths occupy BS bins 2, 4 and 5; two of them share user bin 2 and the third is in user bin 4, so $\mathbf{y} = [0, 1, 0, 1]^{\mathsf{T}}$. The task becomes predicting beam indices.

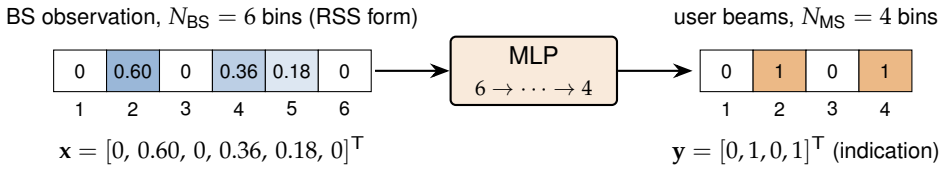


Figure 28.2: Feature representation (our drawing of the paper's toy example). Three paths occupy BS bins 2, 4, 5; their RSS values, normalised to $[0, 1]$ (example values -60, -72, -81 dBm with $r_{\min} = -90$, $r_{\max} = -40$ dBm), fill those bins. At the user, two paths share bin 2 and one is in bin 4, giving the indicator $[0, 1, 0, 1]$. The network maps the left vector to the right one.

Worked example: normalising RSS

With $r_{\min} = -90$ dBm and $r_{\max} = -40$ dBm in the data set, three paths received at -60, -72 and -81 dBm become $(-60 + 90)/50 = 0.60$, $(-72 + 90)/50 = 0.36$ and $(-81 + 90)/50 = 0.18$. The strongest path of the whole data set would map to 1 and a bin with no path to 0 — so "zero" means "nothing here", not "weak". This is one reason the RSS representation works well: magnitude and occupancy are encoded in one number.

28.3.3 Are the two sides related? The correlation analysis

Before training anything, the paper asks whether $\mathbf{x}$ carries information about $\mathbf{y}$ at all, using the information measures of Chapter 4. With X the BS features and Y the user features (both discretised),

$$I(Y; X) = H(Y) - H(Y \mid X), \qquad 0 \le \frac{I(Y; X)}{H(Y)} \le 1. \tag{28.5}$$

$H(Y)$ is the uncertainty about the user beams before looking at the BS; $H(Y \mid X)$ is what remains after looking; the normalised mutual information is the fraction of uncertainty removed (0 for independence, 1 for a deterministic relation). The paper reports that as the BS codebook size N_{BS} grows, $H(Y \mid X)$ decreases, $I(Y; X)$ grows, and the ratio approaches about 0.97 — with fine angular resolution, the BS view almost determines the user's beams. Based on this analysis the paper chooses $N_{BS} = 100$ and $N_{MS} = 10$.

Intuition: why resolution creates information

With 6 wide BS bins, many different user positions produce the same coarse picture ("something strong roughly ahead"), and the network cannot tell them apart. With 100 narrow bins, the exact pattern of arrival angles is almost a fingerprint of the user's location in the square — and once location is pinned down, the geometry fixes the departure angles. Resolution turns the BS observation into an implicit position estimate.

28.4 The neural network and its training

28.4.1 Architecture

The network is a plain multilayer perceptron (Fig. 28.3 and Table 28.1): input of 100 (RSS or delay) or 200 (both) nodes, a hidden layer of 50 sigmoid neurons, a hidden layer of 20 sigmoid neurons, and 10 linear output neurons, one per user DFT bin. Each output is a real number that should be close to 1 for a dominant beam and close to 0 otherwise.

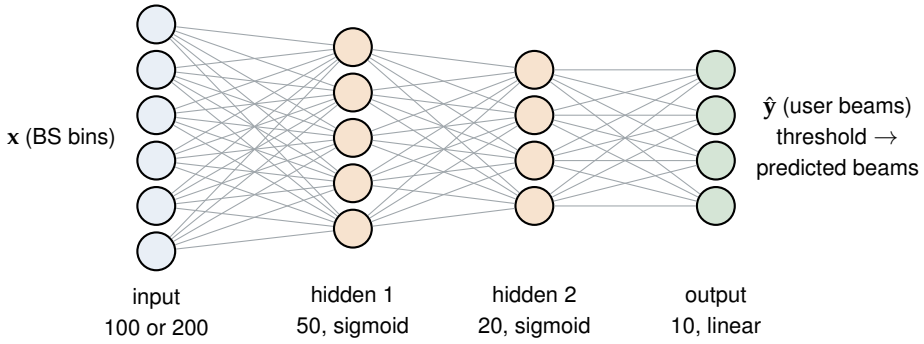


Figure 28.3: The paper's network (layer sizes as stated in the paper; only a few neurons drawn). Hidden layers use sigmoid activations; the output layer is linear because the task is posed as regression onto the 0/1 indicator vector with a mean-squared-error loss.

Table 28.1: Layer sizes of the paper's network and the resulting parameter counts (computed by us: weights plus biases).

Layer	Size	Activation	Parameters (100-input / 200-input)
Input	100 or 200	–	–
Hidden 1	50	sigmoid	$100 \cdot 50 + 50 = 5\,050$ / $10\,050$
Hidden 2	20	sigmoid	$50 \cdot 20 + 20 = 1\,020$
Output	10	linear	$20 \cdot 10 + 10 = 210$
Total			**6 280 / 11 280**

28.4.2 Loss and training

With training pairs $(\mathbf{x}_t^{(n)}, \mathbf{y}_t^{(n)})$, $n = 1, \dots, N_t$, the network $h_{\mathbf{W},\mathbf{b}}$ is trained on the regularised squared error (the paper's cost function):

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{N_t} \sum_{n=1}^{N_t} \left\| h_{\mathbf{W},\mathbf{b}}(\mathbf{x}_t^{(n)}) - \mathbf{y}_t^{(n)} \right\|^2 + \frac{\lambda}{2} \sum_{l} \left\| \mathbf{W}^{[l,l-1]} \right\|_F^2, \tag{28.6}$$

where $\mathbf{W}^{[l,l-1]}$ is the weight matrix between layers $l-1$ and l and $\|\cdot\|_F$ the Frobenius norm. The second term is L2 weight decay (Chapter 11). The paper uses backpropagation with batch gradient descent; it does not state the learning rate, number of iterations or λ.

28.4.3 Four schemes

The paper compares four ways of feeding the BS observations:

- NN_r: input is the normalised RSS vector (100 entries).
- NN_τ: input is the normalised delay vector (100 entries).
- $\mathrm{NN}_{\mathrm{seq}}$: a sequential architecture in which the outputs of the RSS and delay networks are fed to another network.
- $\mathrm{NN}_{r,\tau}$: the RSS and delay vectors concatenated into a 200-entry input.

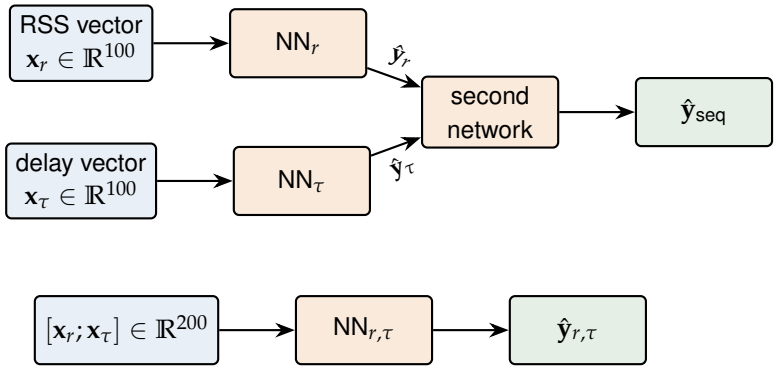


Figure 28.4: The two ways of combining RSS and delay. Top: $\mathrm{NN}_{\mathrm{seq}}$ stacks a second network on the outputs of the single-feature networks (a form of ensembling/stacking). Bottom: $\mathrm{NN}_{r,\tau}$ concatenates the inputs (early fusion).

28.4.4 Evaluation measures

The paper uses two normalised squared errors. With the sample average of the training outputs $\bar{\mathbf{y}}_t$, define the training-output variance $S_t = \frac{1}{N_t - 1} \sum_n \left\| \mathbf{y}_t^{(n)} - \bar{\mathbf{y}}_t \right\|^2$, and on the test (validation) set of N_v samples

$$\eta_v = \frac{1}{N_v} \sum_n \left\| \mathbf{y}_v^{(n)} - \bar{\mathbf{y}}_t \right\|^2, \quad \eta_{\mathrm{NN}} = \frac{1}{N_v} \sum_n \left\| \mathbf{y}_v^{(n)} - h(\mathbf{x}_v^{(n)}) \right\|^2, \quad \rho_v = \frac{\eta_v}{S_t}, \quad \rho_{\mathrm{NN}} = \frac{\eta_{\mathrm{NN}}}{S_t}. \tag{28.7}$$

ρ_v measures how well a "predictor" that ignores the input and always outputs the training average does on the test set; it should be close to 1 when training and test sets are statistically similar. ρ_{NN} is the network's error on the same scale: the lower, the better, and $1 - \rho_{\mathrm{NN}}$ is like the coefficient of determination R^2 of Chapter 5.

The second measure treats beam prediction as detection. Thresholding each output entry at a level γ declares a bin "dominant" or not. A dominant bin correctly declared is a *detection*; a non-dominant bin declared dominant is a *false alarm*. Sweeping γ traces the receiver operating characteristic (ROC) curve of detection probability P_D against false-alarm probability P_F.

Worked example: one ROC point by hand

Suppose $\mathbf{y} = [0, 1, 0, 1]$ and the network outputs $\hat{\mathbf{y}} = [0.1, 0.8, 0.45, 0.35]$. With threshold $\gamma = 0.4$ we declare bins 2 and 3: bin 2 is a detection, bin 4 is missed, bin 3 is a false alarm, bin 1 is a correct rejection. Thus $P_D = 1/2$ and $P_F = 1/2$. With $\gamma = 0.3$ we declare bins 2, 3, 4: $P_D = 2/2 = 1$ and $P_F = 1/2$. Lowering the threshold moves along the ROC towards the upper right.

28.5 Results reported in the paper

Data. Channels were generated by a ray tracer (the paper attributes the data to NTT DOCOMO research labs) for Shinjuku Square in Tokyo at 28 GHz: a 150 m $\times$ 120 m area with 13 BSs and 13 609 user spots on a 1-m grid, calibrated against measurements. The experiments use BS number 8, in the centre of the area. For the plotted curves the data were split at random into 10 % training and 90 % test samples; a footnote states that a 95 %/5 % split was used in the experiments to obtain better results.

Correlation. $H(Y \mid X)$ decreases and $I(Y; X)/H(Y)$ increases with N_{BS}, reaching about 0.97.

Normalised errors. The paper's table reports $\rho_v = 1.0002$ for all schemes (training and test sets are statistically similar) and

$$\rho_{\mathrm{NN}} = 0.5735\ (\mathrm{NN}_r), \quad 0.6579\ (\mathrm{NN}_\tau), \quad 0.5246\ (\mathrm{NN}_{\mathrm{seq}}), \quad 0.5597\ (\mathrm{NN}_{r,\tau}).$$

All networks roughly halve the error of the sample average; delay alone is the weakest input; combining the two (sequentially or by concatenation) helps.

ROC. At $P_F \leq 0.1$, NN_r, $\mathrm{NN}_{\mathrm{seq}}$ and $\mathrm{NN}_{r,\tau}$ detect the target beams with probability $P_D \geq 0.9$; NN_τ's curve lies beneath the other three; all four outperform the sample average. The paper concludes that RSS at the BS is a more reliable predictor of the AoD than delay, and that combining features improves accuracy.

Pitfall: what the paper does not claim

The paper does not report a beam-selection algorithm, link-level throughput gains, results on measured (non-ray-traced) data, or generalisation to other BSs or cities; its conclusion names real-measurement validation, other architectures and beam-selection algorithms as future work. It also does not discuss FDD explicitly — the inference is "BS-side features → user-side features", which is the same *kind* of problem as uplink-to-downlink inference, but that link is our interpretation.

28.6 Our small-scale reproduction

We do not have the DOCOMO ray-tracing data, so we write our own simple geometric channel simulator and repeat the experiment on it. We then use the same simulator for a second, closely related task the paper motivates: predicting channel features (power, delay spread, angle spread) directly from the user's position. All code is in `code/ch28/`.

28.6.1 A street-canyon ray tracer in 150 lines

Our scene (Fig. 28.5) is a 2-D street 30 m wide and 120 m long between two building walls. The BS (100-element ULA, broadside along the street) stands at the street end. Ten point scatterers (lamp posts, signs) and six rectangular blockers (trucks, kiosks) are placed at random. For each user position on a 0.5-m grid we trace:

- the **LOS** path, unless a blocker cuts it; if blocked, a weak diffracted component 20 dB below free space is kept (a crude stand-in for diffraction);
- **single wall reflections** via the image method: reflect the user in the wall line $y = \pm 15$ m, intersect the BS–image line with the wall to find the bounce point; amplitude reflection coefficient 0.5 (-6 dB);
- **double wall reflections** (wall–opposite wall), by reflecting twice;
- **single-bounce scattering** off each point scatterer that both ends can see, with a random scattering coefficient.

Each path is checked segment by segment against the blockers, and its amplitude follows free-space spreading $\lambda/(4\pi d)$ times the bounce losses. From the path list we compute exactly the quantities of Section 28.3: AoA at the BS, AoD at the user, delays and powers. After removing grid points inside blockers we have 12 831 users (close to the paper's 13 609), with on average 7.6 paths per user; 36 % of users have their LOS blocked.

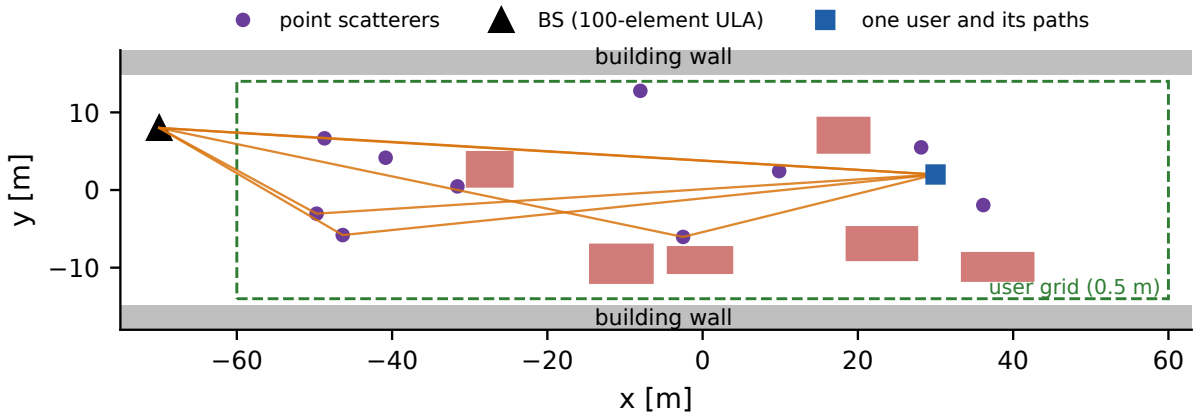


Figure 28.5: Our synthetic scene (top view). Gray: building walls (reflectors). Red: blockers. Purple: point scatterers. The BS (triangle) sits at the street end; the dashed box is the user grid (0.5-m spacing). Orange lines: the five paths traced for one user (LOS, two wall reflections and scatterer bounces).

The data-set script then builds the paper's representation: for each user a 100-bin BS vector (normalised RSS, and normalised delay with 1 in empty bins) and a 10-bin user indicator vector. We define a user bin as dominant if it holds a path within 15 dB of the user's strongest path; on average 3.2 of the 10 bins are dominant and 7.4 of the 100 BS bins are occupied.

28.6.2 Predicting user-side AoD bins

We train the paper's architecture ($100 \to 50 \to 20 \to 10$ with sigmoid hidden units and linear outputs, or 200 inputs for $\mathrm{NN}_{r,\tau}$) on the loss (28.6) with $\lambda = 10^{-4}$, using full-batch Adam for 3 000 steps (the paper used batch gradient descent; Adam simply converges faster). For $\mathrm{NN}_{\mathrm{seq}}$ the 20 outputs of NN_r and NN_τ feed a $20 \to 20 \to 10$ network. Like the paper's figures, we train on a random 10 % of users (1 283) and test on the other 90 % (11 548).

```
def mlp(n_in):
    return nn.Sequential(nn.Linear(n_in, 50), nn.Sigmoid(), nn.Linear(50, 20), nn.Sigmoid(), nn.Linear(20, 10))

def fit(net, X, Yt, lam=1e-4, steps=3000, lr=1e-2):
    """Full-batch training (the paper used batch gradient descent; we use full-batch Adam)."""
    X, Yt = torch.tensor(X), torch.tensor(Yt)
    opt = torch.optim.Adam(net.parameters(), lr=lr)
    for s in range(steps):
        out = net(X)
        l2 = sum((m.weight ** 2).sum() for m in net if isinstance(m, nn.Linear))
        loss = ((out - Yt) ** 2).sum(1).mean() + lam / 2 * l2
        opt.zero_grad(); loss.backward(); opt.step()
    return net
```

Table 28.2 and Fig. 28.6 reproduce the paper's qualitative findings almost point by point: $\rho_v \approx 1$; every network beats the sample average by a wide margin; delay alone is the weakest input; RSS is better; the combinations are best; and at $P_F = 0.1$ the RSS, sequential and combined schemes detect dominant beams with $P_D \approx 0.95$ while delay-only reaches about 0.91. Our ρ_{NN} values are lower (better) than the paper's because our world is much simpler — one street, a few dozen reflectors, no foliage or diffraction detail — so the BS view pins down the geometry more

Table 28.2: Our reproduction versus the paper (normalised test errors, lower is better; P_D at $P_F = 0.1$ read from the ROC curves). Different data (our street canyon versus ray-traced Shinjuku Square), same architecture and the same 10 %/90 % split.

	ρ_{NN}		P_D at $P_F = 0.1$	
Scheme	paper	ours	paper	ours
sample average (ρ_v)	1.0002	0.989	below all NNs	0.31
NN_r (RSS)	0.5735	0.281	≥ 0.9	0.95
NN_τ (delay)	0.6579	0.356	lowest NN curve	0.91
$\mathrm{NN}_{\mathrm{seq}}$	0.5246	0.255	≥ 0.9	0.95
$\mathrm{NN}_{r,\tau}$ (both)	0.5597	0.264	≥ 0.9	0.95

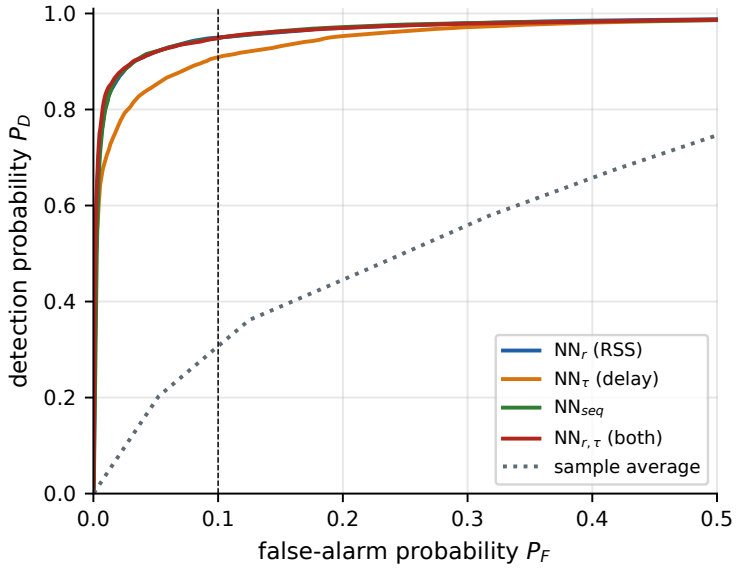


Figure 28.6: Our reproduction: ROC curves for predicting the dominant user-side AoD bins from BS observations (12 831 users, 10 % training). Dashed line: $P_F = 0.1$. As in the paper, the RSS-based, sequential and concatenated schemes lie on top of each other and clearly above the delay-only scheme; all are far above the sample-average baseline.

completely.

Figure 28.7 repeats the paper's resolution argument with a network instead of mutual information. Going from 5 to 10 BS bins lowers ρ_{NN} from 0.37 to 0.25; beyond that, the curve is flat (0.25–0.28 for 25, 50 and 100 bins, with small non-monotonic wiggles from training randomness). In our simple street most users are distinguished already by a 10-bin angular profile, and with only 1 283 training users the larger, sparser 100-bin input is not easier to learn. The paper's richer urban scene keeps gaining information up to its largest codebooks; this is a nice example of a qualitative result that depends on the environment's complexity.

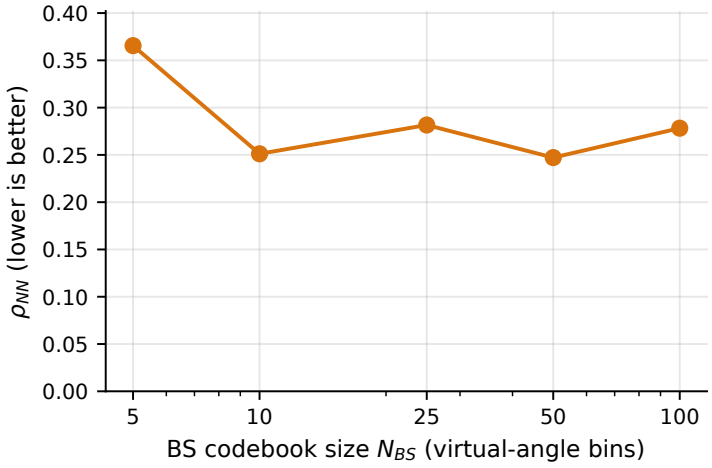


Figure 28.7: Our reproduction: normalised AoD-prediction error ρ_{NN} of the RSS network versus BS codebook size. Resolution helps strongly up to about 10 bins and then saturates in our simple scene.

28.6.3 Channel features from location: radio maps

If BS-side features work because they implicitly encode position, then position itself should predict channel features. This is the "radio map" problem: given a set of measured locations, predict received power, delay spread and angle spread anywhere. We train an MLP that takes the user position (x, y), passes it through a random Fourier-feature encoding

$$\mathbf{z}(\mathbf{p}) = \left[\, x/60,\; y/15,\; \sin(\mathbf{B}^{\mathsf{T}}\mathbf{p}),\; \cos(\mathbf{B}^{\mathsf{T}}\mathbf{p}) \,\right]$$

with 64 random frequencies on a length scale of about 4 m (this helps an MLP represent sharp shadow edges; a raw two-number input gives overly smooth maps), then two hidden layers of 128 ReLU units, and three outputs. As a baseline we use distance-weighted k-nearest-neighbour interpolation ($k = 4$) — the classic way to build a radio map from drive-test points.

Figures 28.8 to 28.10 summarise the radio-map experiment (3 000 held-out test locations). With 1 500 training points (0.46 per m^2) the MLP's mean absolute error is 1.08 dB for received power, 1.45 ns for delay spread and 1.19° for angle spread, against 1.17 dB, 1.47 ns and 1.23° for kNN; with 5 000 points both reach about 0.7 dB. With only 200 points kNN is clearly better (2.04 versus 2.72 dB): the MLP needs enough data to learn the map, while nearest-neighbour copying is a strong prior when data are scarce. The error CDFs show the character of each method: kNN has the smaller median (0.11 versus 0.30 dB for power — in smooth regions a close neighbour is almost exact) but the heavier tail (90th percentile 4.3 versus 3.1 dB — at shadow edges it copies the wrong side). The honest summary is that *data density, not the model, is the main driver*; a neural network earns its place when it can share structure across the map (and, in the paper, across inputs such as the BS angular profile) rather than by beating interpolation on a dense grid.

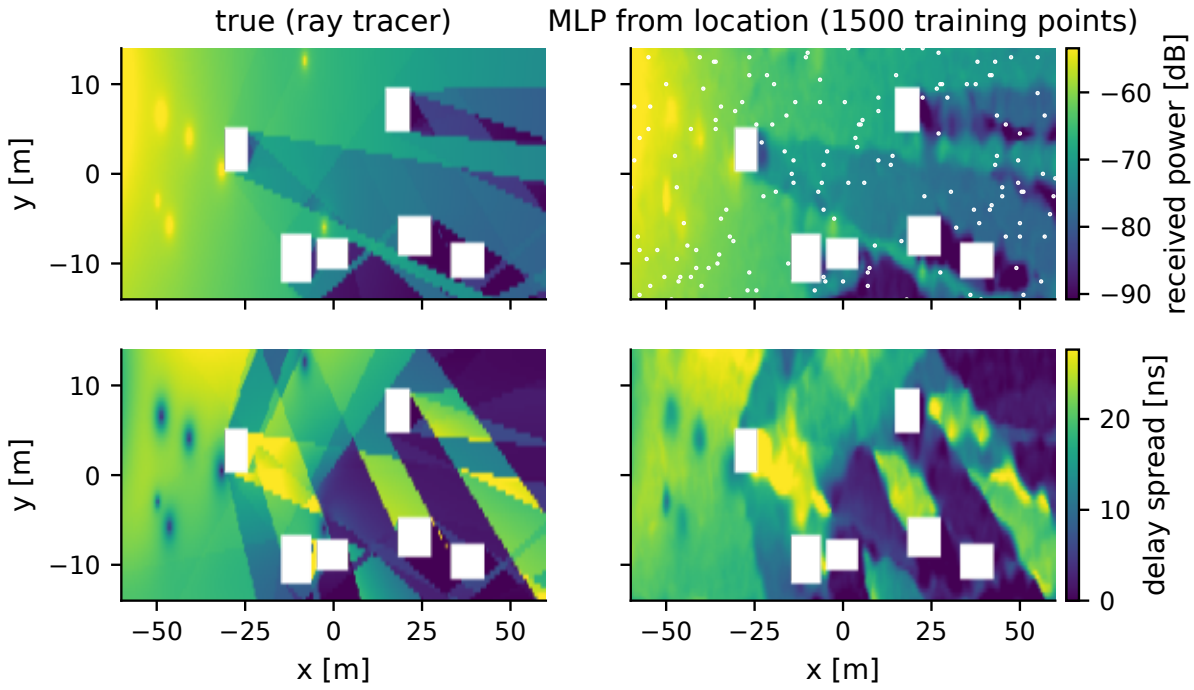


Figure 28.8: Our reproduction: true (left) and MLP-predicted (right) maps of received power (top) and RMS delay spread (bottom), from 1 500 training locations (0.46 points per m^2; a subset shown as white dots). White rectangles are blockers. The MLP recovers the shadow regions behind blockers and the stripes created by wall reflections; errors concentrate on sharp shadow boundaries.

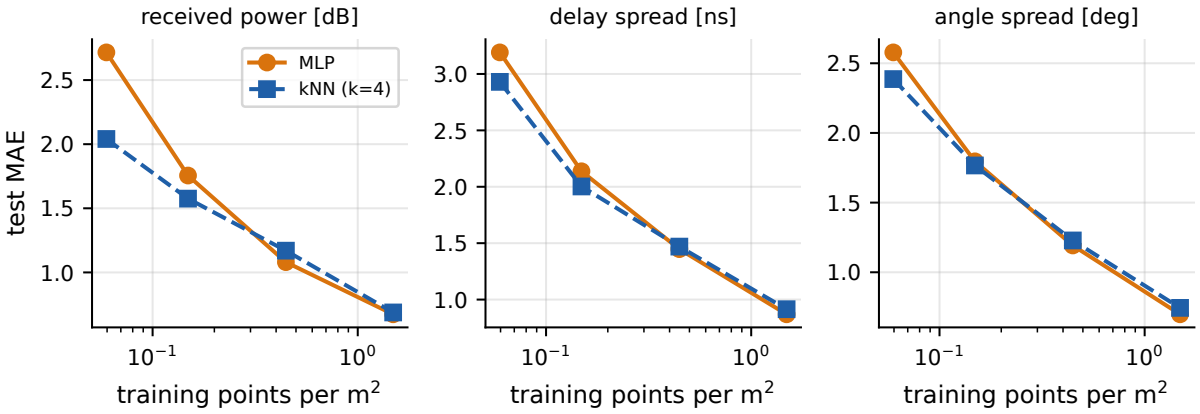


Figure 28.9: Our reproduction: test mean absolute error versus training-set density for the three features. Both methods improve steadily with density; kNN is better with very sparse data, and the MLP matches or slightly beats it from about 0.5 points per m^2.

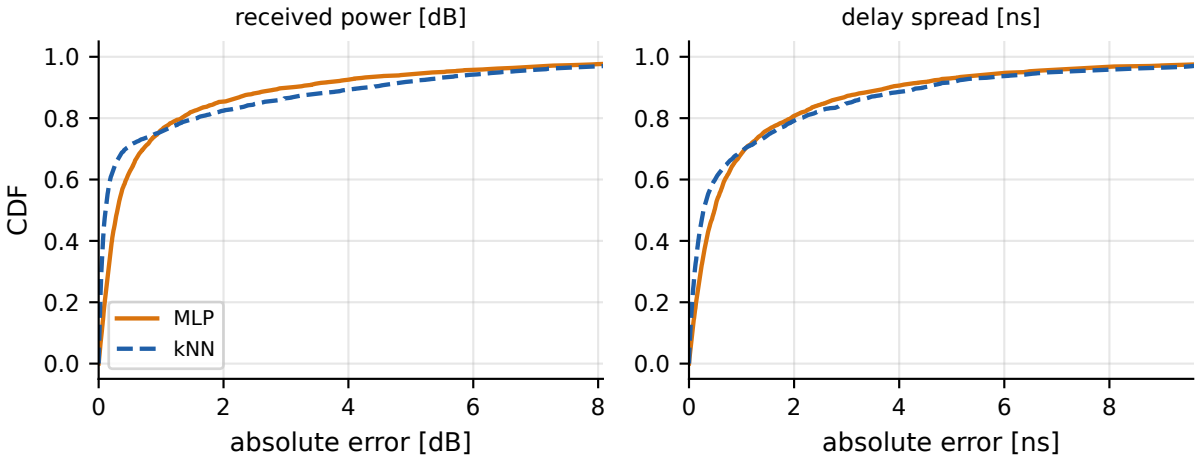


Figure 28.10: Our reproduction: CDF of absolute errors (1 500 training points, 3 000 test points). kNN is more often almost exact (it copies a nearby neighbour in smooth regions) but has a heavier tail at shadow edges; the MLP has a smaller tail.

Pitfall: random splits flatter location-based predictors

With a random split, every test point has training neighbours at a distance of about $1/\sqrt{\text{density}}$; at the paper's 10 %/90 % split of a 1-m grid that is about 3 m. A model evaluated this way is an *interpolator*. Deploying it in a new street, or after a new building is erected, is *extrapolation*, which neither the paper nor our experiment tests. A fair evaluation for deployment holds out whole regions or whole BSs.

28.7 Limitations and follow-ups

- **Simulation only.** Training and testing on ray-tracing data from one BS; the authors themselves list measured data as necessary future work.
- **Site specificity.** The network learns one environment; a new site needs new data. Transfer learning and generative channel models (neural radiance fields for radio, "digital twins") are follow-up directions.
- **Regression to indicators.** A linear output with MSE on 0/1 targets works, but a sigmoid output with binary cross-entropy (Chapter 8) is the principled multi-label choice and gives calibrated probabilities for beam ranking.
- **Beyond beam indices.** The natural next step is predicting a ranked list of beam *pairs* with their expected gains, which feeds directly into beam management. Later work (e.g. on position-aided and sub-6 GHz-aided mmWave beam selection, and the DeepMIMO ray-tracing data sets) pursued exactly this.

28.8 From paper to practice in 5G NR and 6G

Beam management. In NR, initial access uses SSB beam sweeping and the UE reports the best SSB; refinement uses CSI-RS (P2 at the gNB, P3 at the UE with repeated CSI-RS). Every predicted beam that need not be swept saves reference-signal resources and latency. 3GPP's Release-18 study on AI/ML for the air interface (TR 38.843) examined exactly this as the *beam management*

use case, with two sub-cases: spatial-domain beam prediction (predict the best beams from measurements of a subset of beams) and temporal beam prediction (predict future best beams from past measurements). Release 19 moved beam management, positioning and CSI prediction into normative work. The paper's idea — infer unmeasured beam qualities from measured channel features — is the ancestor of the spatial-domain sub-case.

FDD downlink from uplink. In FDD the uplink and downlink use different carriers, so the downlink channel is not the transpose of the uplink one. Large-scale features — path angles, delays and average powers — are, however, nearly frequency independent, because they are set by geometry. Predicting downlink features (or even downlink CSI) from uplink measurements is therefore an instance of the same "observed features → unobserved features" learning problem.

Radio maps and positioning. Radio maps of received power support coverage planning, handover optimisation and interference coordination; maps of delay and angle features are fingerprints for positioning (Chapter 29). The 3GPP study also examined AI/ML positioning, including direct fingerprinting from measured channel impulse responses or power-delay profiles.

6G. Proposals for 6G emphasise site-specific learning: digital twins built from ray tracing, sensing-aided beam prediction (camera, radar, GPS), and integrated sensing and communication. This paper is an early, clean demonstration of the principle that makes them work: the environment couples everything, so features observed at one place, band or end of the link predict features elsewhere.

Wireless connection: a minimal deployment recipe

(1) At the gNB, log uplink SRS-based angle/power profiles and the UE's reported best beams during normal operation (this gives labelled data for free). (2) Train a small network per cell. (3) Use its top-k predicted UE beams to configure a reduced P3 sweep, and fall back to the full sweep when the predicted beam's measured RSRP disappoints. (4) Retrain periodically.

Chapter recap

- Channel features are compact statistics of the multipath channel: received power, delay spread, angle spread, dominant beam indices.
- Navabi, Wang, Bursalioglu and Papadopoulos (ICC 2018) predict the user-side dominant AoD bins from BS-side observations, with RSS/delay vectors over 100 BS DFT bins, a $100 \to 50 \to 20 \to 10$ sigmoid MLP, and ray-traced 28-GHz data from Shinjuku Square.
- The features are strongly correlated (normalised MI ≈ 0.97 at high BS resolution); all networks beat the sample average; RSS beats delay; combining both helps; $P_D \geq 0.9$ at $P_F = 0.1$ for the RSS-based schemes.
- Our street-canyon reproduction shows the same ordering of schemes and a benefit of BS resolution that saturates early in our simple scene; MLPs and kNN can also build radio maps of power and delay spread from location, with accuracy governed mainly by measurement density.
- In NR this idea underlies AI/ML beam management (spatial-domain beam prediction), FDD uplink-to-downlink inference and radio-map-based positioning.

Exercises

1. **(numeric)** Compute the aperture of a 64-element and an 8-element half-wavelength ULA at 39 GHz.
2. **(numeric)** Four paths with powers $[1, 0.3, 0.3, 0.1]$ and delays $[0, 40, 60, 200]$ ns: compute the mean delay and RMS delay spread.
3. **(numeric)** In which bin of a 16-bin DFT codebook does a path at $\theta = -45^\circ$ fall? And at $\theta = +45^\circ$?
4. **(numeric)** Verify the parameter count of $\mathrm{NN}_{r,\tau}$ in Table 28.1.
5. **(conceptual)** Why does the delay representation put 1 (not 0) in empty bins? What would go wrong with 0?
6. **(numeric)** A network's outputs for a user are $[0.9, 0.2, 0.6, 0.05, 0.4]$ and the truth is $[1, 0, 1, 0, 0]$. Compute P_D and P_F at thresholds 0.5 and 0.3.
7. **(conceptual)** Explain why ρ_v is slightly above 1 in the paper (1.0002) and what a value of, say, 1.5 would tell you about the data split.
8. **(coding)** In `train_aod.py`, replace the linear output and MSE by a sigmoid output with binary cross-entropy. Do the ROC curves change? Why might they hardly change?

Answers

1. $\lambda = 3 \times 10^8 / 39 \times 10^9 = 7.69$ mm. 64 elements: $64 \cdot \lambda/2 = 24.6$ cm; 8 elements: 3.08 cm.
2. $P = 1.7$. $\bar{\tau} = (0 + 12 + 18 + 20)/1.7 = 50/1.7 = 29.4$ ns. Deviations squared: 864.4, 112.4, 936.4, 29 084; weighted: $864.4 + 33.7 + 280.9 + 2908.4 = 4087.4$. $\sigma_\tau = \sqrt{4087.4/1.7} = \sqrt{2404} = 49.0$ ns.
3. $\theta = -45^\circ$: $u = -0.707$, $(u+1)/2 \cdot 16 = 2.34 \Rightarrow$ bin 2 (counting from 0). $\theta = +45^\circ$: $u = 0.707$, $13.66 \Rightarrow$ bin 13.
4. $200 \cdot 50 + 50 = 10\,050$; $50 \cdot 20 + 20 = 1\,020$; $20 \cdot 10 + 10 = 210$; total 11 280.
5. Delay is "small = early = important". A 0 would mean "a path arriving at the minimum delay", i.e. the most important kind of path, in every empty bin — the network could not distinguish "no path" from "the LOS path". A 1 means "as late as possible", the least important, which is a natural encoding of absence.
6. Positives: bins 1 and 3; negatives: 2, 4, 5. Threshold 0.5: declared $\{1, 3\}$: $P_D = 2/2 = 1$, $P_F = 0/3 = 0$. Threshold 0.3: declared $\{1, 3, 5\}$: $P_D = 1$, $P_F = 1/3$.
7. $\rho_v = \eta_v / S_t$ compares the test-set spread around the *training* mean with the training-set spread. If the sets are drawn from the same distribution, the ratio is about 1 (slightly above because $\bar{\mathbf{y}}_t$ is not the test mean and because of the $N_t - 1$ normalisation). A value of 1.5 would indicate that test outputs are distributed differently — e.g. the test users come from a different region of the square — so the training mean is a poor reference and the networks would also be extrapolating.
8. Typically the ROC changes very little: the ROC depends only on the *ranking* of output scores across bins and users, and both losses push dominant bins' scores up and the others down. BCE gives better-calibrated probabilities (useful for choosing a threshold or the number of beams to test) and may train faster.

Self-test quiz

1. Which feature does the paper predict, and from which observations?
   The user-side dominant angle-of-departure bins, from BS-side RSS and/or delays arranged over the BS's angular (DFT) bins.
2. What data set and what network does the paper use?
   Ray-traced 28-GHz channels for Shinjuku Square, Tokyo (13 BSs, 13 609 user spots, 1-m grid); an MLP with 50 and 20 sigmoid hidden units and 10 linear outputs.
3. What does $\rho_{\mathrm{NN}} = 0.5$ mean?
   The network's test MSE is half the variance of the outputs, i.e. half the error of always predicting the training average.
4. Which input was the weakest predictor in the paper, and which combination was best by ρ_{NN}?
   Delay only (NN_{τ}, 0.6579) was weakest; the sequential scheme (NN_{seq}, 0.5246) was best.
5. Why does a larger BS array help predict user-side angles?
   Finer angular bins make the BS observation nearly unique to the user's position, and position (through the fixed geometry) determines the departure angles; the normalised mutual information grows towards 0.97.
6. Name the 3GPP use case that descends from this idea.
   AI/ML beam management, in particular spatial-domain beam prediction (Release-18 study, Release-19 normative work).

Further reading

- S. Navabi, C. Wang, O. Y. Bursalioglu, H. Papadopoulos, "Predicting Wireless Channel Features using Neural Networks," in *Proc. IEEE ICC*, 2018; arXiv:1802.00107.
- A. Alkhateeb, "DeepMIMO: A Generic Deep Learning Dataset for Millimeter Wave and Massive MIMO Applications," in *Proc. ITA*, 2019 — open ray-tracing data for exactly this kind of experiment.
- X. Lin, "An Overview of the 3GPP Study on Artificial Intelligence for 5G New Radio," arXiv:2308.05315, 2023.
- 3GPP TR 38.843, "Study on Artificial Intelligence (AI)/Machine Learning (ML) for NR Air Interface," Release 18.

Chapter 29

An Improved WiFi Indoor Localization Method Combining CSI and RSS

Satellite navigation tells your phone where it is to within a few metres — outdoors. Step into a shopping centre, a hospital or a factory hall and the satellite signals, already weaker than the thermal noise when they reach the ground, are attenuated by roofs and walls until the receiver loses lock. Indoors, the radio signals that *are* everywhere come from Wi-Fi access points. This chapter shows how to turn those signals into a position, using *fingerprinting*: survey the building once, record what the radio environment "looks like" at many known points, and later locate a device by finding the recorded fingerprint that best matches what it currently measures.

Two kinds of fingerprint are available from commodity Wi-Fi hardware. The **received signal strength** (RSS) is one number per access point, available on every phone, but it fluctuates and is coarse. The **channel state information** (CSI) is a complex channel gain per OFDM subcarrier (and antenna), much richer but harder to obtain and full of hardware-induced phase errors. The paper of this chapter proposes to combine them. You will learn the physics of both measurements, how to clean CSI phase, the offline/online structure of fingerprinting, distance metrics and k-nearest-neighbour matchers, probabilistic and deep-learning matchers, and how fusion works. Our reproduction then reveals something every engineer working on CSI fingerprinting should know: CSI is a superb fingerprint *at the surveyed spot* and a poor one a few centimetres away.

Paper at a glance

Title: An Improved WiFi Indoor Localization Method Combining Channel State Information and Received Signal Strength.
Authors: Linsheng Zhao, Hongpeng Wang, Pengpeng Li, Jingtai Liu (Nankai University).
Venue: Proc. 36th Chinese Control Conference (CCC), Dalian, China, July 2017, pp. 8964–8969; DOI 10.23919/ChiCC.2017.8028783.
Problem: RSS fingerprints are coarse and unstable indoors; CSI fingerprints are rich but noisy; can combining them give a more robust fingerprint?
Key idea (as described by a later paper that cites it): use both RSS and CSI as the fingerprint; pre-process in the offline and online phases — time-domain filtering of CSI to reduce noise and multipath effects, and dividing the channel into four sub-channels whose subcarrier responses are averaged to reduce redundancy — and estimate the location with an improved weighted k-nearest-neighbour (WkNN) algorithm.
Main result (as reported by the same citing paper): an average error distance of 1.79 m.

Pitfall: what we could and could not verify

We verified the title, the four authors, the venue, the pages and the DOI (via the publisher's metadata and the authors' laboratory page). We could *not* access the paper's text or abstract. The description of the method in the box above comes from the related-work section of Sanchez-Rodriguez *et al.*, "Fusion of Channel State Information and Received Signal Strength for Indoor Localization Using a Single Access Point" (*Remote Sensing*, 2020), which cites this

paper explicitly; we report it as their summary, not as a quotation of the original. We do not know the experimental environment, the number of access points or reference points, the exact filter, or the precise form of the "improved" WkNN. This chapter therefore teaches the *method family* — CSI+RSS fingerprint fusion with filtering, sub-channel averaging and WkNN — with verified related work (Horus, FIFS, DeepFi, CiFi) as anchors, and never invents details of the original.

29.1 Indoor positioning: the landscape

Intuition: recognising a room by its sound

Clap your hands in a bathroom, a church and a carpeted bedroom: each answers with a different echo. With your eyes closed you could tell which room you are in — not by computing distances, but by *recognising* a pattern you have heard before. Fingerprinting localization works the same way. We do not try to model how radio waves bounce around a building; we simply record the pattern at every spot once, and later recognise it.

There are two broad families of radio positioning:

- **Ranging / geometric methods** measure distances or angles to known anchors and solve geometry: time of arrival (ToA) or round-trip time (Wi-Fi fine timing measurement, 5G NR multi-RTT), time difference of arrival (TDoA, NR DL-TDoA), angle of arrival (AoA). With d_i the distance to anchor i at $\mathbf{a}_i$, the position $\mathbf{p}$ solves $\|\mathbf{p} - \mathbf{a}_i\| = d_i$ in a least-squares sense. They need accurate timing and line of sight; indoors, a blocked direct path makes the measured distance too long, a bias no amount of averaging removes.
- **Fingerprinting** learns the map from position to measurement *empirically*. Multipath, walls and furniture — the enemies of ranging — become features that make each location distinctive. The price is a site survey, and the map must be maintained when the environment changes.

Figure 29.1 shows the two phases of fingerprinting. In the **offline phase** a surveyor visits N reference points (RPs) at known coordinates $\mathbf{p}_j$ and records many packets at each; their processed summary $\mathbf{f}_j$ is stored in a database $\{(\mathbf{f}_j, \mathbf{p}_j)\}_{j=1}^{N}$. In the **online phase** a device measures a few packets, computes its fingerprint $\mathbf{f}$ the same way, and a *matcher* estimates $\hat{\mathbf{p}}$ from the database.

29.2 RSS: one number per access point

29.2.1 The log-distance model

The received power from an access point at distance d is modelled, in dB, as

$$P_r(d) = P_t - PL(d_0) - 10\,n \log_{10} \frac{d}{d_0} - \sum_{w} L_w + X_\sigma, \tag{29.1}$$

where P_t is the transmit power (dBm), $PL(d_0)$ the path loss at a reference distance d_0 (often 1 m; about 47 dB at 5 GHz in free space), n the path-loss exponent (2 in free space, typically 2.5–4 indoors), L_w the penetration loss of each wall crossed, and X_σ a shadowing term, Gaussian in dB with standard deviation σ (Chapter 20). For fingerprinting, the crucial distinction is between two kinds of randomness in X_σ:

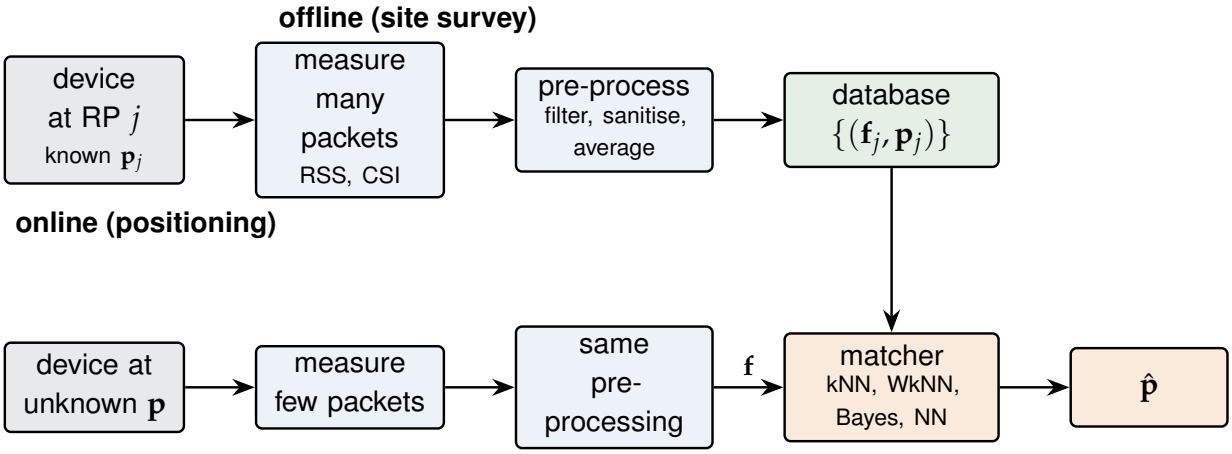


Figure 29.1: Fingerprinting localization. The offline survey builds a database of processed fingerprints at known reference points; online, the same processing is applied to a short measurement and a matcher compares it with the database. The pre-processing must be identical in both phases.

- **location-dependent** shadowing, which is the same each time you stand at a spot — it is *part of the fingerprint*, and the very thing that makes nearby spots distinguishable;
- **time-varying** fluctuation from people walking, doors, the device's orientation and automatic gain control — this is *noise* for fingerprinting.

Worked example: how much does RSS tell us about distance?

With $P_t = 20\,\text{dBm}$, $PL(1\,\text{m}) = 40\,\text{dB}$ and $n = 3$: at 1 m, $P_r = 20 - 40 = -20\,\text{dBm}$; at 10 m, $-20 - 30 = -50\,\text{dBm}$; at 20 m, $-20 - 30\log_{10} 20 = -59.0\,\text{dBm}$. Doubling the distance from 10 to 20 m changes RSS by only 9 dB. If the packet-to-packet fluctuation has a 2 dB standard deviation, one RSS reading at 10 m is consistent (within $\pm 2\sigma$) with $P_r \in [-54, -46]\,\text{dBm}$, i.e. distances from $10^{(-4/30)} \cdot 10 = 7.4\,\text{m}$ to $10^{(4/30)} \cdot 10 = 13.6\,\text{m}$. Range from RSS is crude — which is why fingerprinting does not try to compute range.

29.2.2 Why RSS fluctuates

An RSS value is the power summed over the whole channel bandwidth. The many multipath components add with random phases, so as a person walks by, the sum varies. Averaging over the band smooths the deep frequency-selective fades, but the remaining fluctuation is typically a few dB. Commodity drivers also report RSS in integer dBm. FIFS (Xiao *et al.*, 2012) summarised the problem: RSS is a coarse value measuring the received power of the whole channel, and it fluctuates over time in multipath-rich indoor environments. Panel (c) of Fig. 29.2 shows our simulated RSS: about 2 dB of packet-to-packet variation per access point.

29.3 CSI: a channel gain per subcarrier

29.3.1 What a Wi-Fi card reports

An 802.11n/ac receiver estimates the channel for equalisation on every OFDM subcarrier (Chapter 20). Some chipsets expose these estimates. The widely used Intel 5300 tool reports H for 30 subcarrier groups across a 20 MHz channel, for each transmit–receive antenna pair. In an ideal

world, for L paths with complex gains g_l and delays τ_l, the CSI at subcarrier frequency f_i would be

$$H(f_i) = \sum_{l=1}^{L} g_l \, e^{-j2\pi f_i \tau_l}. \tag{29.2}$$

Its *amplitude* $|H(f_i)|$ across subcarriers has a frequency-selective pattern of peaks and notches set by the delays; its *phase* carries the delays (and hence geometry). Both depend on the exact position, which is what makes CSI such a detailed fingerprint.

29.3.2 Measured phase and its sanitisation

The phase that commodity hardware reports is corrupted. Following the model used in the CSI-phase literature (e.g. PhaseFi/CiFi by Wang, Gao and Mao), the measured phase on subcarrier i is

$$\hat{\phi}_i = \phi_i - 2\pi \frac{m_i}{N} \Delta t + \beta + Z, \tag{29.3}$$

where ϕ_i is the true phase, m_i the subcarrier index (from -28 to 28 for the 30 Intel 5300 groups), N the FFT size (64), Δt the timing offset at the receiver (packet-detection delay plus sampling-frequency offset), β an unknown phase offset (carrier-frequency offset and PLL phase), and Z noise. Both Δt and β change from packet to packet, so the raw phase is useless as a fingerprint (gray curves in Fig. 29.2(a)).

The error term is *linear* in m_i, so we can remove the best line. A simple and widely used choice: let

$$a = \frac{\hat{\phi}_n - \hat{\phi}_1}{m_n - m_1}, \qquad b = \frac{1}{n} \sum_{i=1}^{n} \hat{\phi}_i, \qquad \tilde{\phi}_i = \hat{\phi}_i - a\, m_i - b, \tag{29.4}$$

where $\hat{\phi}$ is first *unwrapped* along the subcarriers. The slope a absorbs $-2\pi\Delta t / N$ (plus whatever linear part the true phase had) and b absorbs β (because the subcarrier indices are nearly symmetric around zero, $\sum m_i \approx 0$, the mean of $a\, m_i$ is nearly zero). What survives, $\tilde{\phi}_i$, is the *nonlinear* part of the true phase — the wiggles caused by multipath — which is stable across packets (orange in Fig. 29.2(a)).

Worked example: sanitising three subcarriers

Unwrapped phases $\hat{\phi} = [-3.0,\ 0.5,\ 4.2]$ rad at indices $m = [-28,\ 0,\ 28]$. Slope $a = (4.2 - (-3.0))/56 = 0.1286$; offset $b = (-3.0 + 0.5 + 4.2)/3 = 0.567$. Sanitised: $-3.0 + 0.1286 \cdot 28 - 0.567 = 0.033$; $0.5 - 0 - 0.567 = -0.067$; $4.2 - 0.1286 \cdot 28 - 0.567 = 0.033$. The large linear ramp has gone; only a small curvature (the middle subcarrier sits 0.1 rad below the line) remains. That curvature is the location information.

29.3.3 Amplitude and the AGC

Commodity receivers apply automatic gain control, so the overall scale of $|H|$ varies from packet to packet. A robust amplitude feature is the *shape* $20\log_{10}|H(f_i)| - \frac{1}{n}\sum_k 20\log_{10}|H(f_k)|$, which removes the common gain. Notice the division of labour: the *level* of the signal is what RSS measures; the *shape* across frequency is what CSI adds. This is the most basic argument for fusion: the two carry complementary information.

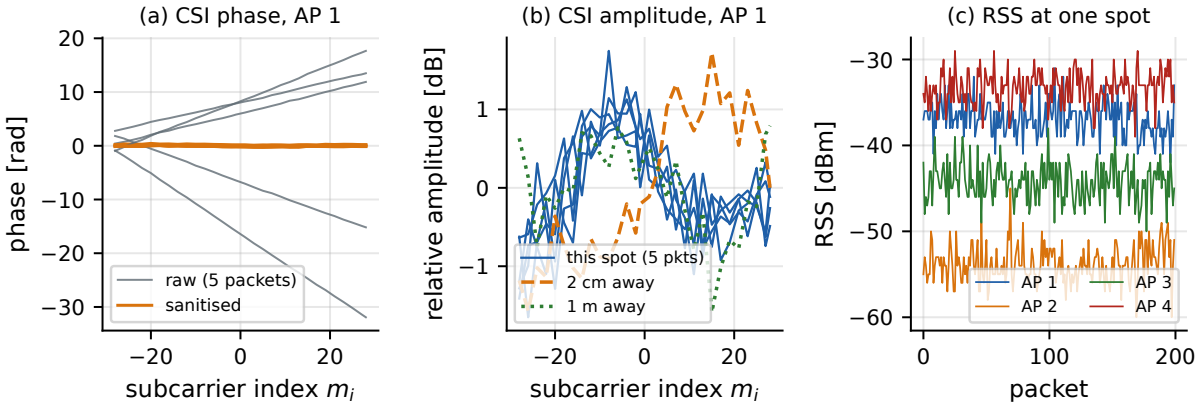


Figure 29.2: Our simulator's measurements at one spot. **(a)** Raw unwrapped CSI phase of five packets (gray) has random offsets and slopes; after linear-fit sanitisation (orange) the five packets coincide. **(b)** CSI amplitude across subcarriers (normalised to zero mean in dB): five packets at the same spot (blue) agree closely; 2 cm away (orange dashed) the pattern is already different; 1 m away (green dotted) it is unrelated. **(c)** RSS of four APs over 200 packets: about 2 dB of fluctuation, quantised to 1 dB.

29.3.4 How fast does CSI change with position?

Figure 29.2(b) shows the catch. The multipath sum (29.2) depends on the path phases $2\pi f \tau_l$; moving by half a wavelength ($\lambda/2 = 2.8$ cm at 5.32 GHz) changes some path lengths by up to $\lambda/2$, i.e. a phase change of up to π. Small-scale fading therefore decorrelates within a few centimetres (Chapter 20). A CSI fingerprint recorded at a reference point is extremely specific to *that* point. This property is a double-edged sword, as our reproduction will show.

29.4 Matching: from nearest neighbours to deep networks

29.4.1 Distances and kNN

Definition: kNN and WkNN localization

Given an online fingerprint $\mathbf{f}$ and database $\{(\mathbf{f}_j, \mathbf{p}_j)\}$, compute distances $D_j = \|\mathbf{f} - \mathbf{f}_j\|$ (usually Euclidean, possibly after scaling each feature). Let $\mathcal{N}_k$ be the k RPs with the smallest D_j. Then

$$\hat{\mathbf{p}}_{k\text{NN}} = \frac{1}{k}\sum_{j\in\mathcal{N}_k}\mathbf{p}_j, \qquad \hat{\mathbf{p}}_{\text{WkNN}} = \frac{\sum_{j\in\mathcal{N}_k} w_j \mathbf{p}_j}{\sum_{j\in\mathcal{N}_k} w_j}, \quad w_j = \frac{1}{D_j + \epsilon}. \tag{29.5}$$

$k = 1$ snaps to the best RP; larger k averages, which interpolates between RPs and reduces the impact of a single wrong match, but biases estimates towards the centre of the neighbours. Weighting by inverse distance lets the best match dominate. Variants (the "improved WkNN" of many papers, possibly including our paper) adapt k, discard neighbours whose distance is much larger than the best, or use other weight functions.

Worked example: kNN and WkNN by hand

Two APs, three RPs on a line: $\mathbf{p}_1 = (0, 0)$ with fingerprint $(-40, -60)$ dBm, $\mathbf{p}_2 = (2, 0)$ with $(-50, -55)$, $\mathbf{p}_3 = (4, 0)$ with $(-60, -45)$. Online measurement $(-48, -57)$. Distances:

$D_1 = \sqrt{8^2+3^2} = 8.544$, $D_2 = \sqrt{2^2+2^2} = 2.828$, $D_3 = \sqrt{12^2+12^2} = 16.97$. With $k = 2$ the neighbours are RP2 and RP1. kNN: $\hat{\mathbf{p}} = ((2+0)/2, 0) = (1, 0)$. WkNN: $w_2 = 1/2.828 = 0.354$, $w_1 = 1/8.544 = 0.117$; $\hat{x} = (0.354 \cdot 2 + 0.117 \cdot 0)/(0.354 + 0.117) = 1.50$, so $\hat{\mathbf{p}} = (1.50, 0)$ — pulled towards the much better match.

29.4.2 Scaling and fusing heterogeneous features

When RSS (4 numbers in dBm) and CSI (hundreds of numbers in dB and radians) are concatenated, the Euclidean distance is dominated by whichever block has more entries and larger values. A principled fusion therefore scales each block. Writing $\mathbf{f} = [\mathbf{f}^{\text{RSS}}; \mathbf{f}^{\text{CSI}}]$, a weighted distance is

$$D_j^2 = w_R \left\| \mathbf{f}^{\text{RSS}} - \mathbf{f}_j^{\text{RSS}} \right\|^2 + w_C \left\| \mathbf{f}^{\text{CSI}} - \mathbf{f}_j^{\text{CSI}} \right\|^2, \tag{29.6}$$

with weights chosen so that each block contributes equally on average (our choice: divide each block by its overall standard deviation and by the square root of its dimension), or tuned on validation data, or set from each block's reliability ("confidence"). This is *feature-level fusion*. The alternative, *decision-level fusion*, runs separate matchers on RSS and CSI and combines their position estimates or likelihoods — the confidence-degree fusion of Zhang *et al.* (CSPS 2019) is an example.

29.4.3 Reducing CSI: filtering and sub-channels

Two pre-processing steps reported for our paper are standard in the family:

- **Time-domain filtering.** Averaging (or median-filtering) the CSI over packets reduces noise and transient multipath from moving people. A related trick transforms $H(f)$ to the delay domain by an inverse FFT, keeps only the first few taps (the early, strong paths) and transforms back, removing late, unstable components.
- **Sub-channel averaging.** Within the coherence bandwidth ($B_c \approx 1/(5\sigma_\tau)$; for a 50 ns delay spread, about 4 MHz) neighbouring subcarriers are strongly correlated, so 30 values carry far fewer than 30 independent numbers. Splitting the band into four sub-channels and averaging the responses within each reduces the CSI fingerprint to four values per antenna pair — less redundancy, less noise, less storage, faster matching.

29.4.4 Probabilistic and deep-learning matchers

Probabilistic. Horus (Youssef and Agrawala, MobiSys 2005) models, at each RP, the distribution of RSS from each AP, and picks the RP with maximum posterior probability $P(\mathbf{p}_j \mid \mathbf{f}) \propto P(\mathbf{f} \mid \mathbf{p}_j) P(\mathbf{p}_j)$, assuming independent APs: $P(\mathbf{f} \mid \mathbf{p}_j) = \prod_a P(f_a \mid \mathbf{p}_j)$. FIFS (Xiao, Wu, Yi, Ni, ICCCN 2012) brought CSI into fingerprinting, combining frequency diversity (subcarriers) and spatial diversity (antennas) with a coherence-bandwidth-enhanced probability algorithm, and outperformed Horus in two indoor scenarios.

Worked example: a Bayesian matcher by hand

Two RPs, two APs, Gaussian RSS with $\sigma = 2$ dB at every RP. RP1 has means $(-49, -60)$ dBm, RP2 has $(-53, -63)$. We measure $(-50, -62)$. Assuming independent APs, the log-likelihoods (up to a common constant) are $-\frac{1}{2\sigma^2}\sum_a (f_a - \mu_{ja})^2$: for RP1, $-(1^2 + 2^2)/8 =$

-0.625; for RP2, $-(3^2+1^2)/8 = -1.25$. With equal priors the posterior of RP1 is $e^{-0.625}/(e^{-0.625}+e^{-1.25}) = 0.651$. A probabilistic matcher can return the MAP point (RP1) or the posterior-weighted mean $0.651\,\mathbf{p}_1 + 0.349\,\mathbf{p}_2$ — the probabilistic cousin of WkNN. With a common σ the ranking is identical to Euclidean nearest neighbour; the Bayesian view pays off when each RP and AP has its own learned variance.

Deep learning. DeepFi (Wang, Gao, Mao, Pandey, IEEE Trans. Veh. Technol. 2017) uses 90 CSI amplitudes (30 subcarriers $\times$ 3 antennas) from one AP; for each location it trains a deep network with four hidden layers (greedy layer-wise pre-training with restricted Boltzmann machines) and stores the *weights* as the fingerprint; online, a probabilistic method based on radial basis functions combines the reconstruction quality across locations. It reports mean errors of about 0.95 m in a living room and 1.8 m in a computer laboratory, against 1.2 and 2.3 m for FIFS; the RSS-based Horus and maximum-likelihood schemes had errors of 2.6 and 2.8 m in the laboratory. CiFi (Wang, Wang, Mao, IEEE Trans. Network Sci. Eng. 2020) estimates angles of arrival from phase differences between antennas, arranges them as 60×60 images and trains a CNN (Chapter 15), reporting mean errors of 1.79 m in a laboratory and 2.39 m in a corridor. ConFi (Chen *et al.*, IEEE Access 2017) also organises CSI into images for a CNN. The simplest neural matcher, which we use below, is an MLP regressor that maps the fingerprint directly to (x, y).

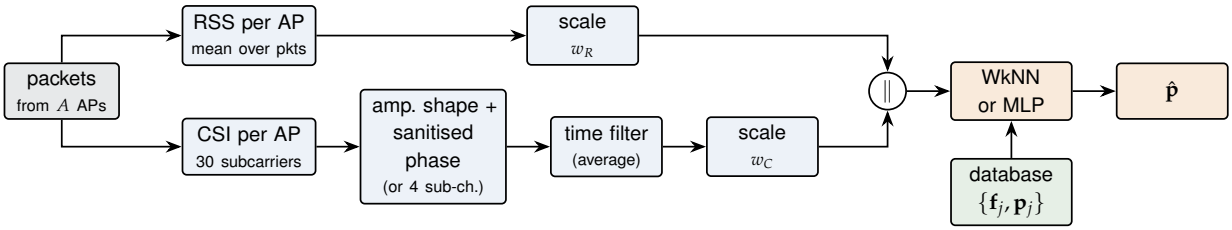


Figure 29.3: The CSI+RSS fusion pipeline as we implement it (our drawing of the method family; the original paper's exact processing could not be verified). RSS and CSI are processed separately, each block is scaled so that both contribute comparably to the distance, and the concatenated fingerprint is matched against the database.

29.5 Our small-scale reproduction

29.5.1 The simulated building

We simulate a 20 m $\times$ 12 m room with an interior wall (5 dB loss) at $x = 10$ m, pierced by a 2-m door, and four access points near the corners (Fig. 29.4). For every AP–device pair we build the multipath channel (29.2) at 5.32 GHz from the direct path, the four first-order wall reflections (image method, reflection coefficient 0.45) and 12 fixed point scatterers (furniture), each path with free-space $1/d$ amplitude, plus spatially smooth shadowing ($\sigma = 3$ dB, correlation scale about 2.5 m). From the same channel we derive:

- **RSS**: the band power in dBm, plus 2 dB per-packet fluctuation, rounded to 1 dB;
- **CSI**: H on the 30 Intel 5300 subcarrier groups, with per-packet AGC gain error (1.5 dB), small gain and phase dynamics of the reflected paths (people moving), 25 dB SNR noise, and a random phase offset β and timing slope per packet as in (29.3).

The offline survey uses 240 RPs on a 1-m grid with 20 packets each; the online device uses a burst of 3 packets. Fingerprints are: **RSS** (4 values); **CSI** (amplitude shape and sanitised

phase, $2 \times 30 \times 4 = 240$ values); **fused** CSI+RSS; and **sub-channel CSI + RSS** (amplitude shape averaged over 4 sub-channels, $4 \times 4 = 16$ values, plus RSS) — the reduction described for our paper. Matchers: 1-NN, kNN and WkNN ($k = 3$), and an MLP (64–64 ReLU, outputs (x, y), trained on the 240 database fingerprints with Gaussian noise augmentation).

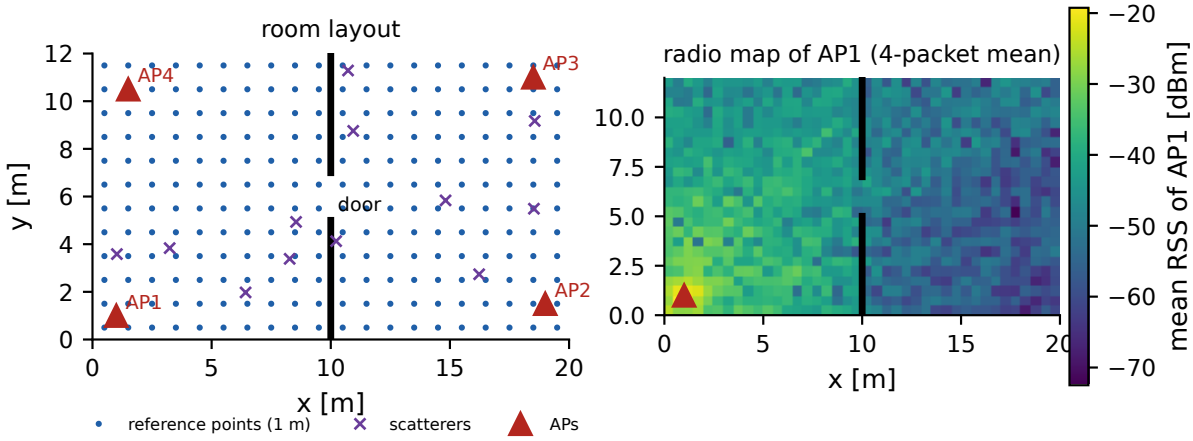


Figure 29.4: Our simulated building. **Left:** 240 reference points on a 1-m grid, four APs, the interior wall with a door, and the scatterers. **Right:** the mean RSS from AP1 over the room — the radio map RSS fingerprinting relies on: a smooth decay with distance, a step at the wall, and shadowing blotches a few metres across.

```python
def features(rss, csi):
    """RSS: mean over packets [A]. CSI amplitude shape: 20log10|H| minus its mean over
    subcarriers (removes the per-packet AGC gain), averaged over packets [A*30].
    Phase: linear-fit sanitised phase averaged over packets [A*30].
    Sub-channel CSI: mean of the amplitude shape over 4 groups of 7-8 subcarriers [A*4]."""
    amp = 20 * np.log10(np.abs(csi) + 1e-9)
    shape = amp - amp.mean(-1, keepdims=True)
    ph = sanitize_phase(csi)
    sub = np.stack([g.mean(-1) for g in np.array_split(shape, 4, axis=-1)], -1)
    return rss.mean(0), shape.mean(0).ravel(), ph.mean(0).ravel(), sub.mean(0).ravel()
```

```python
def sanitize_phase(csi):
    """Linear-fit removal: unwrap phase over subcarriers, subtract the line through the
    first and last subcarrier (slope a and offset b as in the CSI-phase literature)."""
    ph = np.unwrap(np.angle(csi), axis=-1)
    m = SUB
    a = (ph[..., -1] - ph[..., 0]) / (m[-1] - m[0])
    b = ph.mean(-1)
    return ph - a[..., None] * m - b[..., None]
```

29.5.2 Two test regimes

We evaluate in two ways, because the literature quietly uses both:

(A) **At surveyed points.** The device is placed back on the RPs (within about 2 mm) and new

packets are recorded. Many CSI fingerprinting papers evaluate this way: training and test positions coincide, and only time differs.

(B) **Off-grid.** 400 positions drawn uniformly at random in the room — where a real user would be.

Table 29.1: Our reproduction: mean localization error in metres (median in brackets). Regime A: test at the surveyed points; regime B: random off-grid points. Every number comes from `code/ch29/fingerprint_exp.py`.

Fingerprint	1-NN	WkNN ($k = 3$)	MLP
(A) at surveyed points			
RSS only	1.05 (0.00)	1.76 (1.40)	2.14 (1.96)
CSI only	1.71 (0.00)	4.16 (3.72)	4.95 (3.89)
CSI + RSS (fused)	**0.42** (0.00)	2.10 (1.82)	3.01 (2.70)
sub-channel CSI + RSS	**0.17** (0.00)	1.65 (1.35)	1.49 (1.36)
(B) random off-grid points			
RSS only	3.53 (2.89)	2.96 (2.55)	**2.74** (2.43)
CSI only	7.57 (6.40)	6.10 (5.42)	6.51 (5.70)
CSI + RSS (fused)	4.04 (3.52)	3.16 (2.69)	4.22 (3.89)
sub-channel CSI + RSS	3.80 (3.23)	3.14 (2.78)	3.27 (2.93)

29.5.3 What the numbers say

Regime A: fusion wins clearly. At the surveyed points, 1-NN with the fused fingerprint finds the exact reference point 90 % of the time (mean error 0.42 m), and with the sub-channel fingerprint 95 % of the time (0.17 m); RSS alone manages 73 % (1.05 m), and CSI alone 76 % (1.71 m — its mistakes are far away, because CSI alone has no notion of coarse distance). The two sources fail on *different* RPs, so the combination fixes most errors of each — the classic case for fusion. Sub-channel averaging helps further: it removes noise and redundancy and gives the 4 RSS values a fair share of the distance. This mirrors the claims of the CSI+RSS literature, including the 1.79 m average error reported for our paper, which is comparable to what DeepFi, FIFS and CiFi report on their own data.

Regime B: CSI becomes a liability. Off the grid, CSI alone is barely better than guessing (6.1 m mean with WkNN in a 20×12 m room), and adding CSI to RSS slightly *hurts* WkNN (3.16 versus 2.96 m). The reason is Fig. 29.6: the CSI pattern decorrelates within about $\lambda/2 = 2.8$ cm. A test point 10 cm from the nearest RP has a CSI fingerprint unrelated to that RP's, so the CSI part of the distance is random and drowns the useful RSS part. RSS, by contrast, varies smoothly over metres (Fig. 29.4) and interpolates gracefully between RPs.

Matchers. WkNN beats plain kNN slightly everywhere, as expected. The MLP is the best off-grid RSS matcher (2.74 m) — it learns a smooth inverse of the radio map — and the best WkNN-free matcher for the compact sub-channel fingerprint at the RPs, but it struggles with the 244-dimensional fused fingerprint from only 240 training examples: deep matchers need far more data than a 1-m survey of one room provides, which is why DeepFi and CiFi collect hundreds or thousands of packets per location.

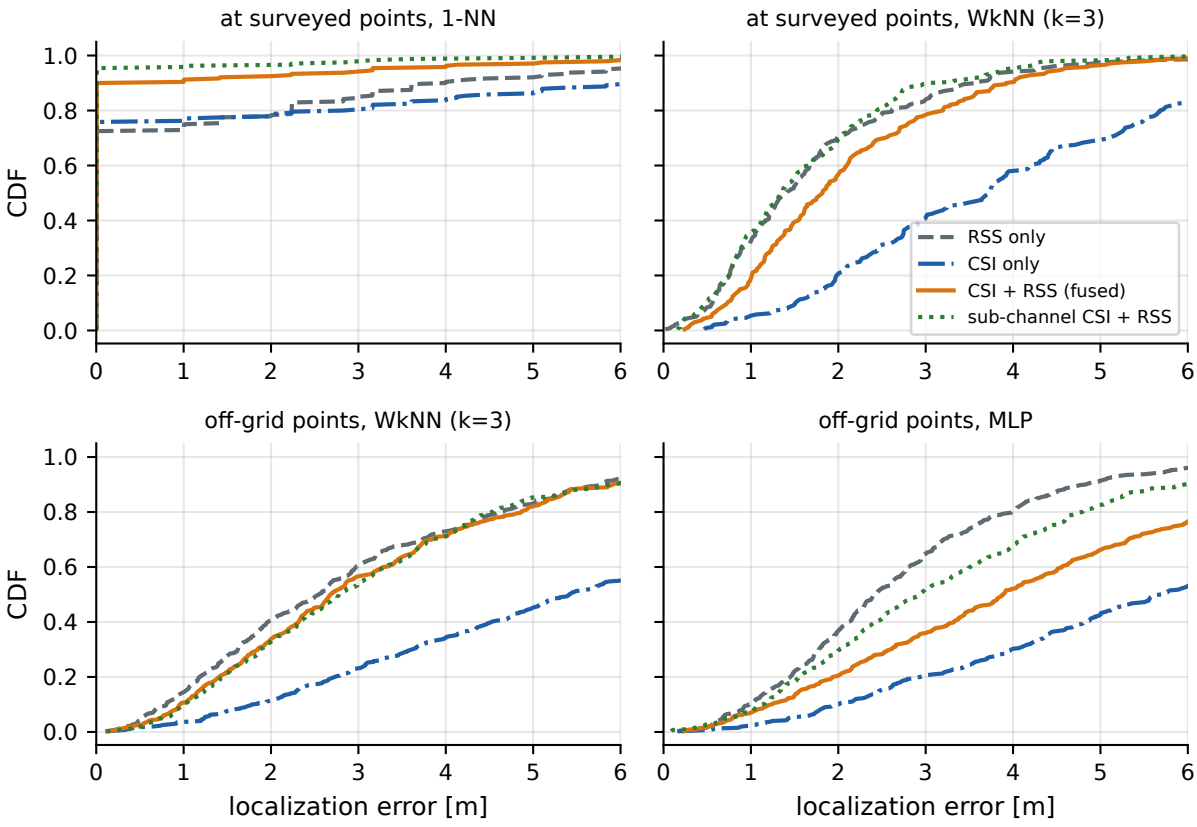


Figure 29.5: Our reproduction: CDFs of localization error. **Top:** test at surveyed points. With 1-NN, the fused fingerprints identify the exact RP for 90 % (full CSI+RSS) and 95 % (sub-channel CSI+RSS) of tests, against 73 % for RSS alone. **Bottom:** random off-grid points. CSI alone fails; RSS alone is as good as or better than the fused fingerprints.

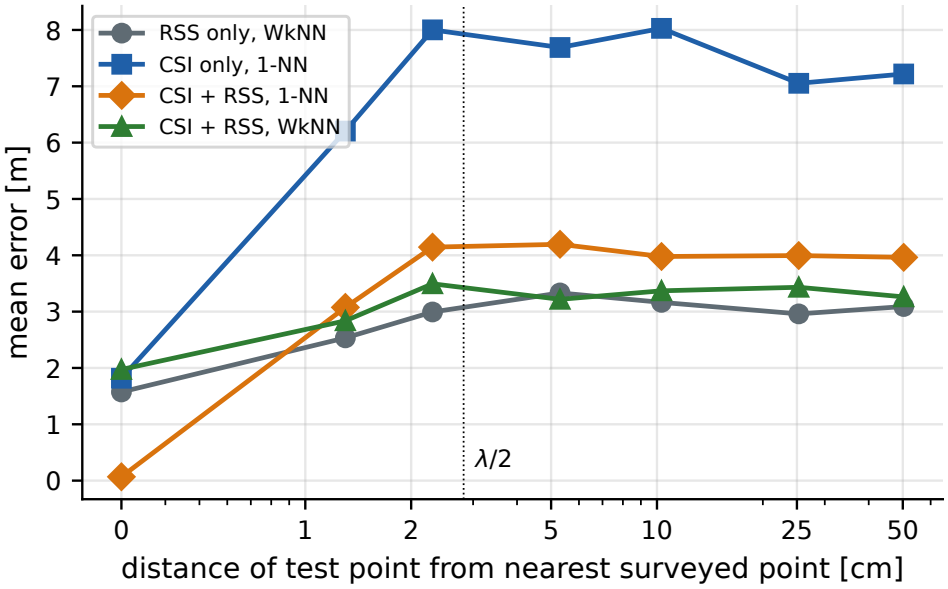


Figure 29.6: Our reproduction: mean error versus the test point's distance from the nearest surveyed point (120 RPs, random directions). Fused CSI+RSS with 1-NN is nearly perfect at 0 cm but degrades within a centimetre or two; past half a wavelength (dotted line) CSI no longer helps and RSS-based WkNN is the best choice.

Pitfall: evaluating CSI fingerprinting honestly

If test packets are recorded at the training spots, CSI fingerprinting looks spectacular. The relevant question for a product is how it performs at arbitrary positions, after furniture moves, with a different phone, a day later. When you read (or write) a CSI localization paper, check: are test locations distinct from training locations? How far apart? Was the environment changed between survey and test? Our experiment shows the answer can flip the conclusion.

Wireless connection: how the field responded

Later work makes CSI robust to small displacements by using features that vary slowly with position: angles of arrival from multiple antennas (CiFi), delay-domain profiles of the first taps, statistics over many packets or over a small area, and data collected along trajectories rather than at points. All of these trade the hyper-local detail of raw CSI for smoothness — moving it towards what RSS already offers, but with more dimensions.

29.6 Limitations and follow-ups

- **Survey cost and aging.** Every fingerprinting method needs a survey, and the database ages as furniture, people and APs change. Crowdsourcing, simultaneous localization and mapping, and transfer learning reduce the cost.
- **Device heterogeneity.** Different phones report RSS with offsets of several dB; CSI is available only on specific chipsets or with modified firmware. Calibration and offset-invariant features (differences between APs) help.
- **Single room.** Our reproduction and many papers use one room or one floor; building-scale systems need floor detection and hierarchical matching.
- **Ranging is back.** Wi-Fi fine timing measurement (802.11mc round-trip time) and ultra-wideband now give metre or sub-metre ranging on many devices; fingerprinting increasingly serves to handle NLOS bias rather than replace ranging.

29.7 From paper to practice in 5G NR and 6G

NR positioning. 5G NR (Release 16 onwards) specifies DL-TDoA, UL-TDoA, multi-RTT, DL-AoD and UL-AoA positioning with dedicated positioning reference signals, and Release 17 tightened accuracy targets for industrial IoT. These are ranging methods and suffer from NLOS bias indoors, exactly the problem fingerprinting sidesteps.

AI/ML positioning in 3GPP. The Release-18 study on AI/ML for the NR air interface (TR 38.843) examined positioning as one of its three use cases, with two approaches: *direct AI/ML positioning*, in which a model maps channel observations such as the channel impulse response or power-delay profile directly to the UE location — fingerprinting with a neural network — and *AI/ML-assisted positioning*, in which a model produces intermediate measurements (e.g. LOS/NLOS identification or corrected timing) for a classical position solver. The study used an indoor-factory scenario with dense clutter and heavy NLOS, where geometric methods struggle. Release 19 continued with normative work on AI/ML positioning. Our chapter's lessons transfer directly: the CIR is a CSI fingerprint, its fine structure decorrelates over a wavelength, and evaluation must use test positions different from the training positions.

Fusion in cellular. The 5G counterpart of CSI+RSS fusion is combining RSRP (the RSS analogue, per beam and per cell) with richer channel measurements (CIR, angle and timing). Beam-level RSRP from many SSB beams is itself a powerful fingerprint, already reported in measurement reports and minimisation-of-drive-tests (MDT) data.

6G. Integrated sensing and communication and "radio digital twins" aim to make every link a localization sensor. The same trade-off reappears: fine-grained channel features give centimetre precision where the model is accurate and fail sharply where it is not; coarse features generalise.

Chapter recap

- Indoor localization by fingerprinting: survey reference points offline, match online measurements to the database; no propagation model needed.
- RSS: one number per AP, smooth in space (good for interpolation), but fluctuating by a few dB and coarse. CSI: complex gain per subcarrier, rich, but the phase needs sanitisation (remove the linear term $a\,m_i + b$) and the fine structure decorrelates within about $\lambda/2$.
- Matchers: kNN, WkNN (inverse-distance weights), probabilistic (Horus, FIFS), deep (DeepFi, CiFi, ConFi, MLP regressors).
- Zhao, Wang, Li and Liu (CCC 2017) combine CSI and RSS, with time-domain filtering, four sub-channels and an improved WkNN; a citing paper reports their average error as 1.79 m. We could not verify further details.
- Our reproduction: at surveyed points, fusion (especially with sub-channel averaging) is far better than either source alone; at arbitrary positions, CSI's spatial decorrelation makes RSS-based matching as good or better — evaluation protocol matters.

Exercises

1. **(numeric)** With $P_t = 15\,\text{dBm}$, $PL(1\,\text{m}) = 46\,\text{dB}$, $n = 3.5$ and one 4-dB wall, compute the RSS at 8 m.
2. **(numeric)** A device measures $(-52, -61, -70)$ dBm from three APs. RP fingerprints: A $(-50, -60, -72)$ at $(1, 1)$; B $(-55, -58, -69)$ at $(3, 1)$; C $(-60, -65, -66)$ at $(5, 1)$. Estimate the position with 1-NN, 2-NN and 2-WkNN.
3. **(numeric)** Sanitise the unwrapped phases $[2.0,\ 2.9,\ 4.4,\ 5.1]$ at indices $m = [-21, -7, 7, 21]$ with (29.4).
4. **(conceptual)** Why must the slope removal in phase sanitisation be done *after* unwrapping? What happens otherwise?
5. **(numeric)** At 2.4 GHz, what is $\lambda/2$? Would you expect CSI fingerprints at 2.4 GHz to tolerate larger displacements than at 5 GHz? By what factor?
6. **(conceptual)** In (29.6), what goes wrong if you simply concatenate 4 RSS values (in dBm) with 240 unscaled CSI values?
7. **(conceptual)** Propose a test protocol for a CSI+RSS system that a product manager could trust.
8. **(coding)** In `fingerprint_exp.py`, change the RP spacing to 0.5 m and then to 2 m. How do the off-grid errors of RSS-only WkNN and fused WkNN change? Explain.

Answers

1. $P_r = 15 - 46 - 35\log_{10} 8 - 4 = 15 - 46 - 31.6 - 4 = -66.6\,\text{dBm}$.
2. Distances: $D_A = \sqrt{2^2+1^2+2^2} = 3.0$; $D_B = \sqrt{3^2+3^2+1^2} = 4.36$; $D_C = \sqrt{8^2+4^2+4^2} = 9.80$. 1-NN: $(1,1)$. 2-NN: mean of A and B $= (2,1)$. 2-WkNN: $w_A = 1/3.0 = 0.333$, $w_B = 1/4.36 = 0.229$; $\hat{x} = (0.333 \cdot 1 + 0.229 \cdot 3)/0.562 = 1.82$; $\hat{\mathbf{p}} = (1.82, 1)$.
3. $a = (5.1 - 2.0)/(21 - (-21)) = 3.1/42 = 0.0738$; $b = (2.0 + 2.9 + 4.4 + 5.1)/4 = 3.6$. Sanitised: $2.0 + 0.0738 \cdot 21 - 3.6 = -0.050$; $2.9 + 0.0738 \cdot 7 - 3.6 = -0.183$; $4.4 - 0.0738 \cdot 7 - 3.6 = 0.283$; $5.1 - 0.0738 \cdot 21 - 3.6 = -0.050$. (They sum to zero, as they must since $\sum m_i = 0$.)
4. The reported phase is wrapped into $(-\pi, \pi]$. A linear ramp of several radians appears as a sawtooth; a line through the first and last wrapped values is meaningless, and the residual would contain jumps of 2π. Unwrapping first restores the continuous ramp so that a line can remove it.
5. $\lambda = 3 \times 10^8 / 2.4 \times 10^9 = 12.5\,\text{cm}$, so $\lambda/2 = 6.25\,\text{cm}$, versus 2.8 cm at 5.32 GHz: about 2.2 times more tolerant — still only centimetres.
6. The CSI block dominates the Euclidean distance (240 terms versus 4), so the RSS information is effectively ignored; and because the units differ (dB of shape, radians, dBm), the relative weighting is arbitrary. The fused fingerprint would perform like CSI alone.
7. Survey on day 1; test on later days at positions not on the survey grid (random or along walking paths), with several device models and normal occupancy; report mean, median and 90th-percentile error, and compare against an RSS-only baseline on exactly the same test data.
8. When we ran this (10 survey packets per RP, 300 off-grid test points), the off-grid mean errors were: 0.5-m spacing, RSS 2.89 m and fused 3.06 m; 1-m spacing, 2.96 and 3.28 m; 2-m spacing, 3.01 and 3.26 m. Surprisingly little changes. The error is not limited by the grid spacing but by the *ambiguity* of four noisy RSS values: points metres apart can have similar RSS vectors (shadowing, symmetric geometry), so denser surveys barely help. CSI never helps off-grid, at any practical spacing, because even 0.5 m is about 18 half-wavelengths. To do better you need more APs, more stable features (e.g. angle or delay information) or temporal tracking.

Self-test quiz

1. Why does GPS fail indoors, and what are the two families of radio positioning?
   Satellite signals are too weak after building attenuation; ranging/geometric methods (ToA, TDoA, AoA, RTT) and fingerprinting.
2. What two error terms does linear-fit phase sanitisation remove?
   The timing-offset term linear in the subcarrier index (slope) and the constant phase offset (intercept).
3. In WkNN, why use inverse-distance weights?
   So that better-matching reference points contribute more to the position estimate than poorer ones.
4. What pre-processing steps and matcher are reported for Zhao *et al.* (CCC 2017), and by

whom?
Time-domain filtering of CSI, averaging over four sub-channels, and an improved WkNN, with an average error of 1.79 m — as summarised by Sanchez-Rodriguez et al. (Remote Sensing, 2020).

5. Why does CSI help at surveyed points but not at random positions in our reproduction?
Its fine structure decorrelates within about half a wavelength, so it identifies an exact surveyed spot but carries no information about spots a few centimetres away.

6. What is "direct AI/ML positioning" in the 3GPP Release-18 study?
A model maps channel observations such as the CIR or power-delay profile directly to the UE position — fingerprinting with a neural network.

Further reading

- L. Zhao, H. Wang, P. Li, J. Liu, "An improved WiFi indoor localization method combining channel state information and received signal strength," in *Proc. 36th Chinese Control Conference (CCC)*, 2017, pp. 8964–8969.
- D. Sánchez-Rodríguez *et al.*, "Fusion of Channel State Information and Received Signal Strength for Indoor Localization Using a Single Access Point," *Remote Sensing*, vol. 12, no. 12, 1995, 2020.
- J. Xiao, K. Wu, Y. Yi, L. M. Ni, "FIFS: Fine-Grained Indoor Fingerprinting System," in *Proc. ICCCN*, 2012.
- X. Wang, L. Gao, S. Mao, S. Pandey, "CSI-Based Fingerprinting for Indoor Localization: A Deep Learning Approach," *IEEE Trans. Vehicular Technology*, vol. 66, no. 1, pp. 763–776, 2017.
- X. Wang, X. Wang, S. Mao, "Deep Convolutional Neural Networks for Indoor Localization with CSI Images," *IEEE Trans. Network Science and Engineering*, vol. 7, no. 1, 2020.
- M. Youssef, A. Agrawala, "The Horus WLAN Location Determination System," in *Proc. ACM MobiSys*, 2005, pp. 205–218.
- S.-X. Zhang, X.-Y. Fan, X.-Y. Luo, "Wireless Indoor Positioning Algorithm Based on RSS and CSI Feature Fusion," in *Proc. CSPS 2019* (Springer LNEE, 2020).

Chapter 30

Deep Reinforcement Learning for Dynamic Multichannel Access

A radio that may use any one of N channels, but can only *listen* to one of them at a time, faces a very human problem: where do I look next? If the channels were independent coin flips, the answer would be "anywhere". But real channels have memory — an interferer that is on now will probably still be on in a millisecond, a Wi-Fi access point hops in a pattern, a neighbour's traffic on one channel is often mirrored by silence on another. A clever radio should exploit that structure. The difficulty is that it does not know the structure in advance, and it never sees the whole picture: in every slot it observes one channel out of N.

In this chapter we study the paper that showed, in a clean and reproducible way, that a Deep Q-Network (DQN, Chapter 19) can learn such a sensing policy from raw experience. You will learn how the problem is written as a *partially observable Markov decision process* (POMDP), why the exact solution blows up exponentially, which classic policies (myopic and Whittle index) are optimal in special cases and why they fail when channels are correlated, how the authors designed the DQN (its input, network and hyperparameters), what they report, and — most importantly — you will build the whole thing yourself: an environment with hidden structure, a DQN with experience replay and a target network, and the baselines, all trained on a laptop CPU.

Paper at a glance: Wang, Liu, Gomes, Krishnamachari (2018)

Paper. S. Wang, H. Liu, P. H. Gomes and B. Krishnamachari, "Deep Reinforcement Learning for Dynamic Multichannel Access in Wireless Networks", *IEEE Transactions on Cognitive Communications and Networking*, vol. 4, no. 2, pp. 257–265, June 2018. Preprint arXiv:1802.06958. The journal paper states that it extends the authors' ICNC 2017 conference paper of the same title.

Problem. One user, N channels; each channel is good or bad in each slot, and the N channels jointly follow an *unknown* Markov chain with 2^N states (channels may be correlated). Each slot the user picks one channel, transmits, and sees only that channel's state: reward $+1$ if good, -1 if bad.

Key idea. Treat the problem as a POMDP but do not compute beliefs. Feed a DQN the recent history of (chosen channel, observed result) and let it learn Q-values for the N channels by trial and error, with experience replay.

Main results (as reported). For $N = 16$ channels with a hidden fixed switching pattern, the DQN matches the optimal policy derived in the paper (which knows the pattern) and clearly beats a Whittle-index heuristic. With perfectly correlated channels it comes very close to the myopic policy that knows the full model. On a real 802.15.4 trace it obtains accumulated discounted reward 0.9473 versus 0.7673 (Whittle heuristic) and -2.1697 (random). An *adaptive* DQN that re-trains when its reward drops follows a changing environment.

30.1 The problem and the intuition

Intuition: sixteen doors, one light

Picture a corridor with 16 doors. Behind exactly one door a light is on, and every second the light jumps, with probability 0.9, to the *next* door of a secret sequence (say $3 \to 11 \to 6 \to \dots$); otherwise it stays. You may open one door per second: $+1$ point if the light is behind it, -1 if not. At first you are blind and open doors at random, scoring about -0.875 per try. But after a few hits you notice "whenever door 3 was lit, door 11 was lit next". Once you have learned the whole secret sequence you can follow the light: after a hit on door 3, open door 11; after a miss on door 11, the light must still be on door 3, and it will jump next — so open door 11 again. You now score $+1$ about 90% of the time. Nobody gave you the sequence: you inferred it from your own history of door openings. That is exactly what the DQN in this paper does.

The example already contains every ingredient of the paper: (i) a hidden *state* (which door is lit) that evolves as a Markov chain; (ii) a *partial observation* (you only see one door); (iii) *correlation* between channels (if door 3 is lit, all the others are dark — channels are strongly dependent); (iv) a policy that should use *history*, not just the last observation.

Wireless connection: where this happens

Cognitive radio and dynamic spectrum access: a secondary user looks for idle primary-user channels and can sense only a few at a time. *Wireless sensor networks* (the paper's motivation): IEEE 802.15.4 radios have 16 channels in the 2.4 GHz band that overlap Wi-Fi channels; Wi-Fi traffic makes several adjacent 802.15.4 channels bad together, i.e. correlated. *5G NR-U and Wi-Fi*: devices must pick a 20 MHz unlicensed sub-band, perform listen-before-talk on it, and only then learn whether it was free. In all three, "sense one, learn one" is the observation model.

30.2 Background you need

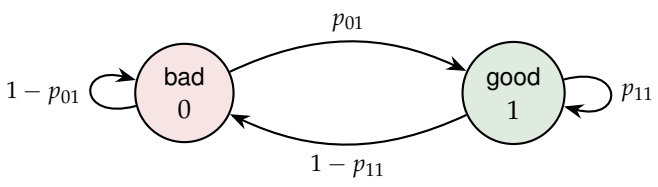
30.2.1 A two-state channel: the Gilbert–Elliott model

Definition: Gilbert–Elliott channel

A channel whose quality in slot t is $S(t) \in \{0, 1\}$ (1 = good, 0 = bad), where $S(t)$ is a two-state Markov chain with transition probabilities

$$p_{11} = \Pr[S(t{+}1) = 1 \mid S(t) = 1], \qquad p_{01} = \Pr[S(t{+}1) = 1 \mid S(t) = 0].$$

The chain is *positively correlated* if $p_{11} \geq p_{01}$ (good tends to be followed by good) and *negatively correlated* if $p_{11} < p_{01}$.



If we do not observe the channel, our knowledge is a single number, the *belief* $\omega(t) = \Pr[S(t) = 1 \mid \text{everything seen so far}]$. One slot later, by the law of total probability,

$$\omega(t{+}1) = \omega(t)\, p_{11} + \big(1 - \omega(t)\big)\, p_{01}. \tag{30.1}$$

If we *do* observe it, the belief collapses to 1 or 0 and then is propagated: next slot it is p_{11} (we saw good) or p_{01} (we saw bad).

Worked example: belief of one channel

Take $p_{11} = 0.8$, $p_{01} = 0.2$ and current belief $\omega = 0.6$. Unobserved, the next belief is $0.6 \times 0.8 + 0.4 \times 0.2 = 0.48 + 0.08 = 0.56$. If instead we sense it and find it good, the next belief is $p_{11} = 0.8$; if bad, $p_{01} = 0.2$. Keep propagating an unobserved channel and the belief drifts to the stationary value $\omega^\star$ solving $\omega^\star = \omega^\star p_{11} + (1 - \omega^\star)p_{01}$, i.e. $\omega^\star = p_{01}/(1 - p_{11} + p_{01}) = 0.2/0.4 = 0.5$.

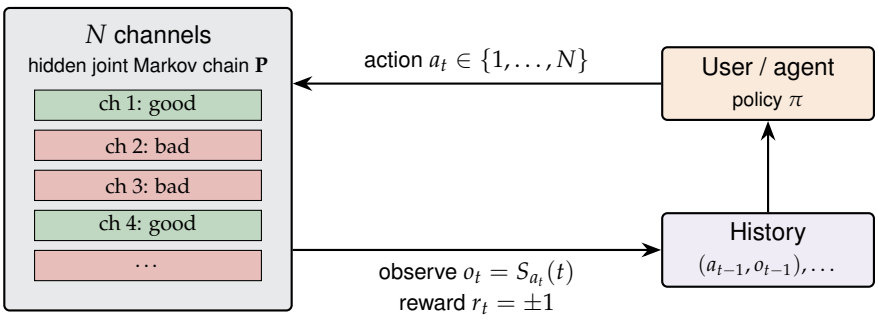
30.2.2 Many channels: the joint Markov chain

With N channels the system state is the vector $\mathbf{s}(t) = [S_1(t), \ldots, S_N(t)] \in \{0,1\}^N$, one of 2^N possibilities. The paper allows these to be *correlated*: the whole vector follows one Markov chain with a $2^N \times 2^N$ transition matrix $\mathbf{P}$, $P_{ij} = \Pr[\mathbf{s}(t{+}1) = \mathbf{s}_j \mid \mathbf{s}(t) = \mathbf{s}_i]$. Independent Gilbert–Elliott channels are the special case where $\mathbf{P}$ factorises into a product of N two-state chains.

30.2.3 From MDP to POMDP

Recall from Chapter 19: an MDP has states, actions, transition probabilities and rewards, and the agent *sees* the state. In a POMDP the agent only receives an *observation* that depends on the hidden state. The standard trick is to make the *belief* — the probability distribution over hidden states given the history — the new state. The belief is a sufficient statistic: an optimal policy can be written as a function of it.

30.3 System model



Time is slotted, $t = 1, 2, \ldots$. At the start of slot t:

1. the user chooses one channel $a_t \in \{1, \ldots, N\}$ (this is the *action*);
2. it transmits there and learns the channel state $o_t = S_{a_t}(t) \in \{0, 1\}$ (the *observation*); the other $N - 1$ channels remain unseen;

3. it receives the reward

$$r_t = \begin{cases} +1 & \text{if } S_{a_t}(t) = 1 \text{ (transmission succeeds)}, \\ -1 & \text{if } S_{a_t}(t) = 0 \text{ (transmission fails)}. \end{cases} \tag{30.2}$$

The goal is a policy π that maps the history to actions and maximises the expected discounted return $\mathbb{E}\left[\sum_{t\geq 1} \gamma^{t-1} r_t\right]$, with discount factor $0 < \gamma < 1$.

30.3.1 The POMDP and its belief

The belief is a vector $\Omega(t) = [\omega_1(t), \ldots, \omega_{2^N}(t)]$, with $\omega_i(t)$ the probability that the system is in joint state $\mathbf{s}_i$ given the history. After acting on channel a and observing o, Bayes' rule and one Markov step give

Key equation: belief update

$$\omega_j(t+1) = \sum_{i=1}^{2^N} \frac{\omega_i(t)\, \mathbb{1}\left[(\mathbf{s}_i)_a = o\right]}{\sum_k \omega_k(t)\, \mathbb{1}\left[(\mathbf{s}_k)_a = o\right]}\, P_{ij}. \tag{30.3}$$

Read it right to left: (1) zero out every joint state that contradicts what we just saw (the indicator); (2) renormalise so the remaining probabilities sum to one (the denominator); (3) push the result one step through the Markov chain (the sum over i with P_{ij}).

The value function of the belief MDP satisfies the Bellman equation

$$V(\Omega) = \max_a \Big\{ \underbrace{\textstyle\sum_i \omega_i \big(2(\mathbf{s}_i)_a - 1\big)}_{\text{expected reward}} + \gamma \sum_{o\in\{0,1\}} \Pr[o \mid \Omega, a]\; V\big(\Omega'(\Omega, a, o)\big) \Big\},$$

where $2(\mathbf{s}_i)_a - 1$ is $+1$ for a good channel and -1 for a bad one.

30.3.2 Why this explodes

The belief lives in a *continuous* simplex of dimension $2^N - 1$. For $N = 16$ channels that is $2^{16} = 65{,}536$ probabilities to track, a $65{,}536 \times 65{,}536$ transition matrix to know, and a value function over a 65,535-dimensional continuous space. The paper recalls that solving general POMDPs is PSPACE-hard. A tempting alternative — tabular Q-learning on the raw history of the last M (action, observation) pairs — also explodes: each slot has $2N$ possible (action, observation) pairs, so there are $(2N)^M$ histories; the paper notes this exponential growth. Figure 30.1 makes the scale concrete.

30.4 Classic baselines: myopic and Whittle index

Before deep RL, two structured policies were (and still are) the reference points.

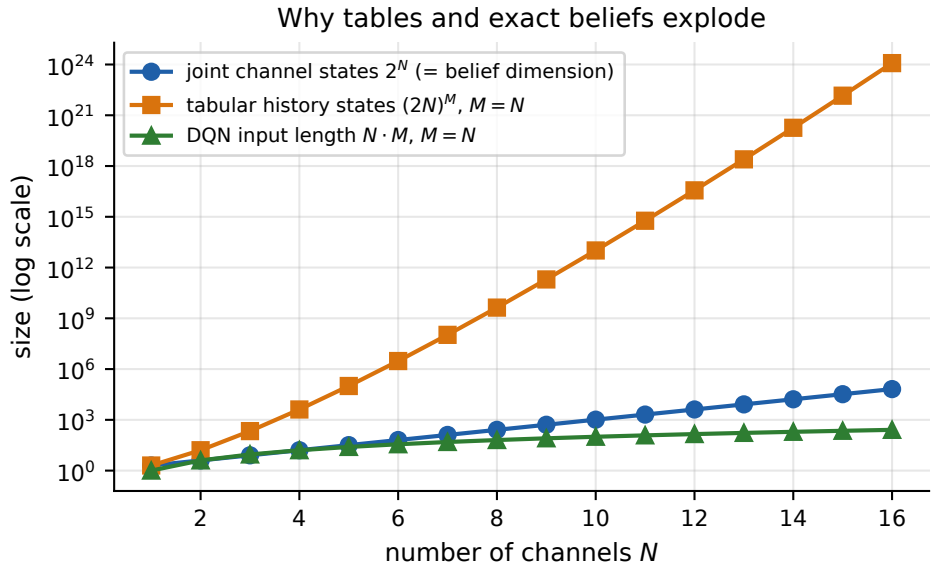


Figure 30.1: How fast the different representations grow with the number of channels N (log scale). The exact belief has 2^N entries; a table indexed by the last $M = N$ (action, observation) pairs has $(2N)^N$ rows ($\approx 10^{24}$ for $N = 16$). The DQN input used in this chapter (one length-N vector per remembered slot) only grows as $N \cdot M = N^2$: 256 numbers for $N = 16$. The network then has to *generalise* across histories instead of enumerating them.

30.4.1 The myopic policy

Definition: myopic policy

Choose the channel with the largest probability of being good *in this slot*: $a_t = \arg\max_a \Pr[S_a(t) = 1 \mid \text{history}] = \arg\max_a \sum_i \omega_i(t)(\mathbf{s}_i)_a$. It maximises the immediate expected reward and ignores the information value of sensing.

For *independent, identical* Gilbert–Elliott channels, Zhao, Krishnamachari and Liu (2008) and Ahmad et al. (2009) proved that the myopic policy is optimal for any number of channels when the channels are positively correlated ($p_{11} \geq p_{01}$), and for two or three channels in the negatively correlated case; the paper under study restates these results. In the positively correlated case the myopic policy even has a simple round-robin structure: stay on a channel while it is good; when it turns bad, move to the channel that was visited longest ago (its belief has relaxed furthest towards the stationary value, which is the highest belief among unvisited channels).

Worked example: one myopic step

Two independent identical channels, $p_{11} = 0.8$, $p_{01} = 0.2$. Last slot we sensed channel 1 and found it bad; channel 2 has not been seen for a long time, so $\omega_2 \approx \omega^\star = 0.5$. Then $\omega_1 = p_{01} = 0.2$ and ω_2 stays 0.5 (it is already stationary). Myopic picks channel 2: expected reward $0.5 \cdot (+1) + 0.5 \cdot (-1) = 0$ against $0.2 - 0.8 = -0.6$ for channel 1.

The myopic policy needs the model (**P**) and the exact belief (30.3), which for correlated channels is the 2^N-dimensional object we cannot afford. In our reproduction we can afford it only because we *design* environments whose hidden state is small.

30.4.2 The Whittle index

When channels are *independent* but not identical, the problem is a *restless multi-armed bandit*: each channel is an arm whose state evolves whether or not you pull it (unlike the bandits of Chapter 19, which freeze when not played). Whittle's idea is to give a "subsidy" m for leaving an arm passive and to define the index of an arm in belief state ω as the subsidy that makes you indifferent between sensing it and not. Then: each slot, sense the arm with the largest index. Liu and Zhao (2010) showed that the Gilbert–Elliott channel is indexable, derived the index in closed form, and proved it optimal under the same conditions as above (identical positively correlated channels, or two/three negatively correlated channels). For non-identical channels it is a strong heuristic.

Pitfall: independence is baked in

Both baselines assume independent channels. The paper notes that the Whittle index cannot be applied directly to correlated channels, so it uses a *heuristic*: compute, from the joint matrix $\mathbf{P}$ and Bayes' rule, a marginal two-state chain for each channel, ignore the correlation, and apply the Whittle index. Throwing away the correlation is exactly what makes this heuristic weak in the patterned scenarios below — if channel 3 being good tells you channel 11 will be good next, a per-channel model cannot see it.

30.5 The method: a DQN that reads the history

30.5.1 State: the recent history

The DQN never computes a belief. Its input is the history of the last M slots. Each slot is encoded as a length-N vector: the entry of the chosen channel is $+1$ if it was good, -1 if it was bad, and all other entries are 0. The paper uses $M = N$; the DQN state is the combination of these M vectors.

Worked example: encoding, $N = 4$, $M = 3$

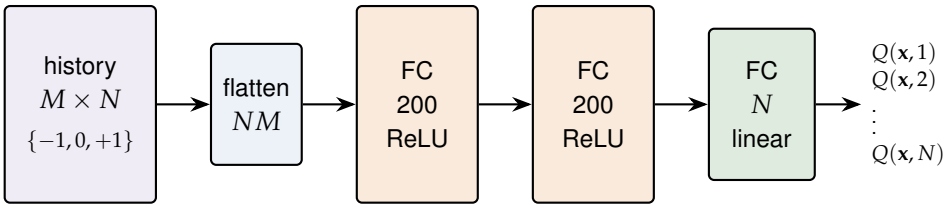
Suppose in the last three slots we chose channel 2 (good), channel 2 (bad), channel 4 (good), most recent first. Then

$$\mathbf{x}_t = [\underbrace{0,\ +1,\ 0,\ 0}_{t-1} \mid \underbrace{0,\ -1,\ 0,\ 0}_{t-2} \mid \underbrace{0,\ 0,\ 0,\ +1}_{t-3}] \in \mathbb{R}^{12}.$$

Both the action (position of the non-zero) and the observation (its sign) are packed into one vector. For $N = M = 16$ the input has 256 entries.

Why is a finite window enough? The belief (30.3) is a function of the entire history, but when the chain mixes, old observations matter less and less. A window of M slots is an approximation; making M large keeps more information at the price of a bigger input.

30.5.2 Network



Sec. VII of the paper: 2 hidden layers $\times$ 200
Sec. VIII: 3 hidden layers $\times$ 50

The Q-network is a plain fully connected network with ReLU activations. The output is a vector of length N: entry i estimates the discounted return of choosing channel i now and acting well afterwards. In the fixed-pattern experiments (the paper's Section VII) the network has two hidden layers of 200 neurons; for the more complex correlated and real-trace scenarios (Section VIII) the authors re-tuned it to three hidden layers of 50 neurons with a smaller learning rate, noting it trains faster.

30.5.3 Training as reported

Training is standard deep Q-learning with experience replay (Chapter 19): every transition $(\mathbf{x}_t, a_t, r_t, \mathbf{x}_{t+1})$ is stored in a replay memory; minibatches are drawn uniformly at random; the target uses a copy of the network with older weights θ^-:

Key equation: DQN target and loss

$$y_t = r_t + \gamma \max_{a'} Q(\mathbf{x}_{t+1}, a'; \theta^-), \qquad L(\theta) = \mathbb{E}_{(\mathbf{x},a,r,\mathbf{x}')\sim\mathcal{D}}\Big[\big(y - Q(\mathbf{x}, a; \theta)\big)^2\Big]. \tag{30.4}$$

Actions are chosen ϵ-greedily. Table 30.1 lists the hyperparameters the paper reports.

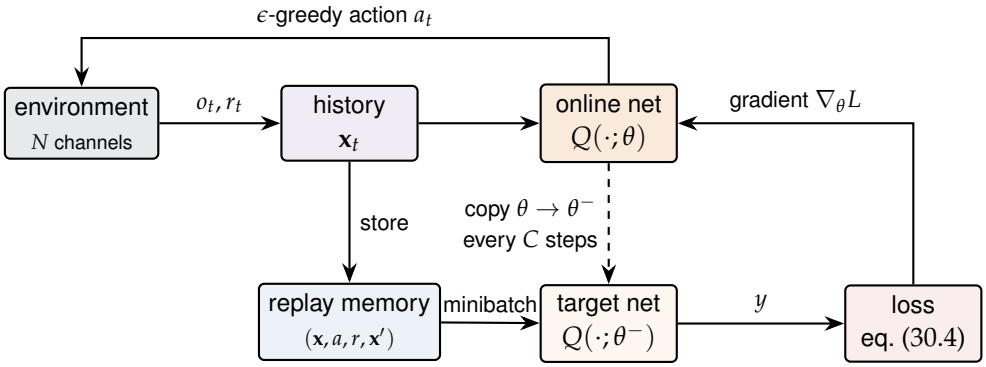
Table 30.1: DQN hyperparameters as reported by the paper (its Table I and Section VIII), next to the ones we use in our small-scale reproduction. The paper does not state a target-network update interval or replay memory size in the parts we could check.

Parameter	Paper	Our reproduction
History length M	$M = N = 16$	$M = N = 16$
Hidden layers	2×200 (Sec. VII); 3×50 (Sec. VIII)	2×128
Activation	ReLU	ReLU
Optimizer	Adam	Adam
Learning rate	10^{-4} (Sec. VII); 10^{-5} (Sec. VIII)	2×10^{-3}
Minibatch	32	64
Discount γ	0.9	0.9
ϵ-greedy	$\epsilon = 0.1$	$\epsilon = 0.1$
Training iterations	10^6	5000 steps $\times$ 32 parallel users
Target network	yes (older weights)	copied every 200 updates

Worked example: one TD target by hand

Suppose the user transmitted on a good channel ($r = +1$), and the target network estimates $\max_{a'} Q(\mathbf{x}', a'; \theta^-) = 7.2$. With $\gamma = 0.9$ the target is $y = 1 + 0.9 \times 7.2 = 7.48$. If the online

network currently outputs $Q(\mathbf{x}, a; \theta) = 6.9$, the squared error is $(7.48 - 6.9)^2 = 0.3364$, and the gradient step will push $Q(\mathbf{x}, a)$ up. Sanity check of the scale: a policy that earns 0.8 per slot forever has value $0.8/(1 - 0.9) = 8$, so Q-values around 7–8 are what we expect for a good policy.



30.5.4 Known patterns: the optimal policy the DQN is compared with

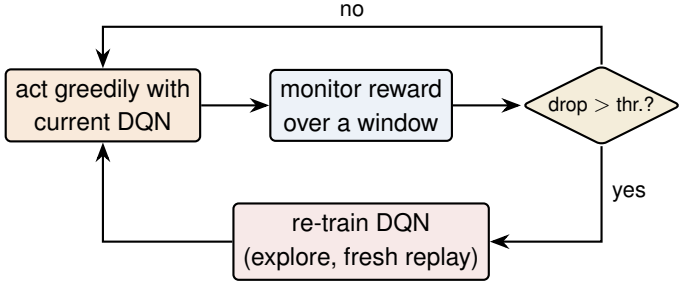
For the fixed-pattern experiments the paper derives the optimal policy when the pattern is *known* (its Theorem 1). The channels are divided into subsets that are activated (made good) one after the other in a fixed order; at each slot, with probability p the next subset is activated, otherwise the current one stays. The optimal rule: if $p \geq 0.5$, after a success move to the next subset, after a failure stay on the channel you just tried (the activation did not move, so it will probably move now); if $p < 0.5$, after a success stay, after a failure move to the next subset.

Worked example: the optimal reward is $2\max(p, 1-p) - 1$

Take one good channel and $p = 0.9$. After a success on channel c we pick the next channel c^+; it is good with probability p. After a failure on c^+ we know the activation stayed at c, so it will jump to c^+ with probability p in this slot: we pick c^+ again, success probability p. So in steady state every slot succeeds with probability $p = 0.9$, giving average reward $0.9 \cdot (+1) + 0.1 \cdot (-1) = 0.8$. For $p < 0.5$ the same argument gives $1 - p$. In general $\bar{r}^\star = 2\max(p, 1-p) - 1$. A random policy with one good channel out of 16 earns $\frac{1}{16} - \frac{15}{16} = -0.875$.

30.5.5 The adaptive DQN

Real environments change (a new Wi-Fi network appears, a primary user changes its schedule). The paper's Algorithm 3 adds a simple outer loop: keep evaluating the accumulated reward of the current policy; if it drops by more than a user-chosen threshold, re-train the DQN.



30.6 Results as reported

All experiments in the paper use $N = 16$ channels, "a typical multichannel WSN".

Single good channel, round-robin switching (Sec. VII-B). The paper evaluates five switching probabilities, $p \in \{0.3, 0.5, 0.7, 0.9, 0.95\}$, and plots the average discounted reward. It reports that the DQN reaches the same performance as the optimal policy in all five cases and is significantly better than the Whittle-index heuristic. (We do not quote numbers read off the plot.)

Arbitrary switching order (Sec. VII-C). The good channel follows an arbitrary (randomly generated) order; eight such patterns are tested with $p = 0.9$. Again the DQN is reported to reach the optimal performance in every case and to clearly beat the Whittle heuristic.

Multiple good channels (Sec. VII-D). The 16 channels are divided evenly into subsets of 1, 2, 4 or 8 channels that become good together. The DQN is reported to always reach the optimal performance, and its training time decreases as the number of good channels grows.

Perfectly correlated channels (Sec. VIII-A). Only two or three channels evolve independently; every other channel is an exact copy ($\rho = 1$) or exact inverse ($\rho = -1$) of one of them, in six simulated cases. Here the myopic policy, implemented with full knowledge of the model, is the best policy the paper can compute; the DQN, with no knowledge, is reported to come very close to it, and well above the random and Whittle-heuristic policies.

Real data trace (Sec. VIII-B). The authors recorded, on the Tutornet testbed (TelosB nodes, IEEE 802.15.4 radios) in an office with 8 Wi-Fi access points, about 17 hours of success/failure measurements on all 16 channels, and kept the 8 worse channels to create a hard scenario. Figure 30.2 plots the numbers in their Table II.

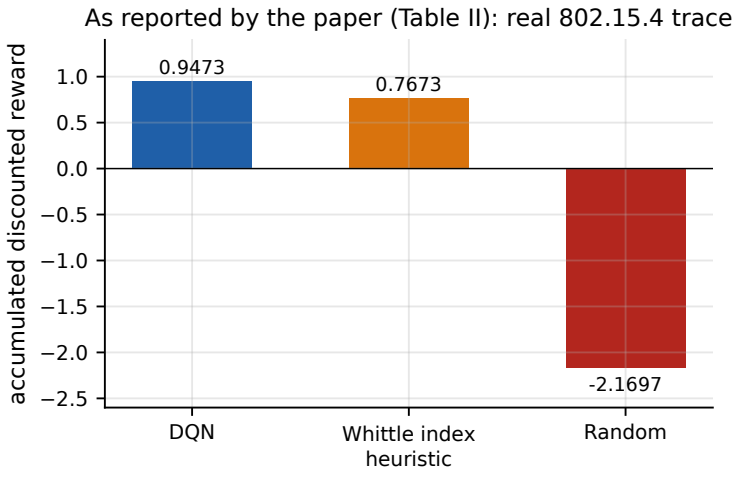


Figure 30.2: Numbers *as reported* by the paper (Table II) on the real 802.15.4 trace: accumulated discounted reward of DQN, the Whittle-index heuristic, and random channel selection. Re-plotted by us from the table; the DQN is about 23% higher than the heuristic.

Adaptive DQN (Sec. IX). A 16-channel system starts with 8 good channels per slot and then switches to a single good channel per slot. The paper's figure shows the DQN's reward collapse after the change, the drop being detected and re-learning starting around iteration 11, and the optimal policy being found again around iteration 70, while the Whittle heuristic remains unsatisfying because it cannot use the correlation. The threshold is left to the user.

30.7 Our small-scale reproduction

We now rebuild the core experiments on one CPU thread. All code is in `code/ch30/`: `mcenv.py` (environments, baselines, DQN) and one script per figure; each runs in about 2–4 minutes.

30.7.1 Environments and baselines

- **Hidden round-robin** (`RoundRobinEnv`): $N = 16$, exactly one good channel; with probability p the good channel moves to the next one in a *secret random permutation* of the channels (so it is the paper's "arbitrary switching" case; neighbouring channel numbers carry no hint).
- **Correlated** (`CorrelatedEnv`): three hidden, mutually independent Gilbert–Elliott channels with (p_{11}, p_{01}) equal to $(0.9, 0.2)$, $(0.8, 0.1)$ and $(0.6, 0.3)$. Each of the 16 channels is a copy or an inverse of one of them, in a shuffled layout — our version of the paper's perfectly correlated scenario.
- **Baselines**: *random*; the *exact belief* policy (for round-robin it tracks the posterior over the position of the good channel; with the myopic choice this reproduces the paper's optimal Theorem 1 rule; for the correlated case it is the paper's *myopic* policy over the $2^3 = 8$ hidden joint states, knowing **P**); an *independent-channel heuristic* that, like the paper's Whittle heuristic, models each channel by its own marginal two-state chain and ignores correlation, but picks the highest belief rather than the Whittle index; and a *genie* that sees all channels.

Listing 30.1: The hidden round-robin environment (from `mcenv.py`).

```
class RoundRobinEnv:
    def __init__(self, N=16, p=0.9, order=None, seed=0):
        self.N, self.p = N, p
        self.rng = np.random.default_rng(seed)
        self.order = np.arange(N) if order is None else np.asarray(order)
        self.pos = 0                                  # index into self.order
    def good(self):                                   # true state vector (0/1)
        s = np.zeros(self.N, int); s[self.order[self.pos]] = 1; return s
    def step(self, a):
        g = self.good()                               # state of the current slot
        r = 1.0 if g[a] == 1 else -1.0
        if self.rng.random() < self.p:                # hidden dynamics for the next
     slot
            self.pos = (self.pos + 1) % self.N
        return r, g
```

30.7.2 The DQN

Our agent follows the paper's design: history input with $M = N$, $\pm 1/0$ encoding, fully connected ReLU network, Adam, $\gamma = 0.9$, $\epsilon = 0.1$, experience replay and a target network. To fit in minutes

instead of hours we make two changes: we run 32 independent copies of the environment in parallel (all feeding the same replay memory, one gradient step per synchronous slot) and we use a larger learning rate. This trades sample efficiency for wall-clock time; it does not change what is being learned.

Listing 30.2: DQN agent: epsilon-greedy acting, replay storage, and the target-network update of eq. (30.4).

```
class DQNAgent:
    def __init__(self, N, M, hidden=(200, 200), lr=1e-3, gamma=0.9, eps=0.1,
                 mem=30_000, batch=64, target_every=200, seed=0):
        torch.manual_seed(seed); self.rng = np.random.default_rng(seed)
        self.N, self.M, self.gamma, self.eps, self.batch = N, M, gamma, eps, batch
        self.q = make_qnet(N, M, hidden); self.qt = make_qnet(N, M, hidden)
        self.qt.load_state_dict(self.q.state_dict())
        self.opt = torch.optim.Adam(self.q.parameters(), lr=lr, foreach=True)
        self.mem = mem; self.S = np.zeros((mem, N * M), np.float32)
        self.A = np.zeros(mem, np.int64); self.Rw = np.zeros(mem, np.float32)
        self.S2 = np.zeros((mem, N * M), np.float32); self.n = 0
        self.target_every, self.steps = target_every, 0
    def act(self, s, greedy=False):                       # s: (B, N*M)
        with torch.no_grad(): a = self.q(torch.from_numpy(s)).argmax(1).numpy()
        if not greedy:
            explore = self.rng.random(len(a)) < self.eps
            a = np.where(explore, self.rng.integers(0, self.N, len(a)), a)
        return a
    def store(self, s, a, r, s2):                         # store a batch of
    transitions
        idx = (self.n + np.arange(len(a))) % self.mem
        self.S[idx], self.A[idx], self.Rw[idx], self.S2[idx] = s, a, r, s2
        self.n += len(a)
    def learn(self):
        if self.n < 2000: return None
        idx = self.rng.integers(0, min(self.n, self.mem), self.batch)
        s, a = torch.from_numpy(self.S[idx]), torch.from_numpy(self.A[idx])
        r, s2 = torch.from_numpy(self.Rw[idx]), torch.from_numpy(self.S2[idx])
        with torch.no_grad():                             # target from the frozen
    network
            y = r + self.gamma * self.qt(s2).max(1).values
        qsa = self.q(s).gather(1, a[:, None]).squeeze(1)
        loss = nn.functional.mse_loss(qsa, y)
        self.opt.zero_grad(); loss.backward(); self.opt.step()
        self.steps += 1
        if self.steps % self.target_every == 0: self.qt.load_state_dict(self.q.
    state_dict())
        return loss.item()
```

30.7.3 Result 1: learning a hidden switching pattern

Figure 30.3 is the analogue of the paper's fixed-pattern result. The greedy DQN reaches 0.776 per slot, i.e. about 97% of the optimal 0.796 measured for the belief policy (the theoretical value is exactly 0.8). The paper, with 10^6 training iterations, reports that the DQN *matches* the optimum; our 5000-step budget leaves a small gap, consistent with that picture.

Figure 30.4 opens the black box. Reading the cross–dot pairs, you can see the network has learned two things: the secret successor of every channel (16 facts) and the "after a miss, try again" rule. Neither was programmed. For comparison, Figure 30.5 shows the exact belief the optimal

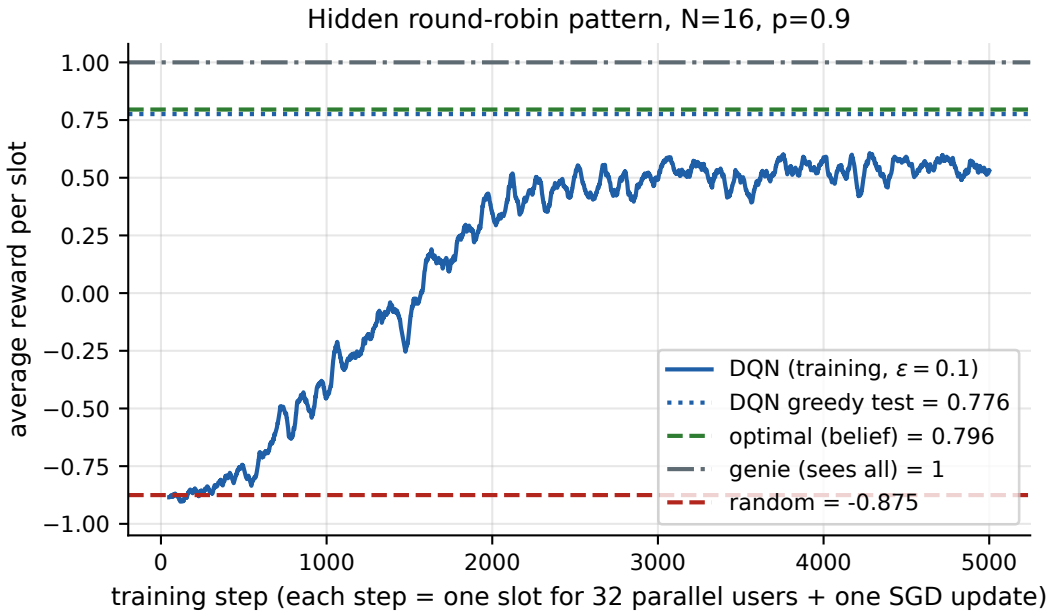


Figure 30.3: Our reproduction, hidden round-robin with $N = 16$ and $p = 0.9$. Blue: training reward (moving average, includes $\epsilon = 0.1$ exploration). The DQN starts at the random level (-0.875), discovers the pattern after about 1500–3000 steps, and its *greedy* test reward (dotted, 0.776) approaches the optimal policy that knows the secret order (dashed, 0.796; theory 0.8). The training curve sits lower because 10% of actions are random.



Figure 30.4: What the trained DQN actually does. The vertical axis is the position of a channel in the *hidden* order, so the good channel (thick grey) climbs a staircase. Green dots: DQN choices that hit the good channel; red crosses: misses. The misses occur precisely when the good channel did *not* move ($1 - p = 10\%$ of slots); right after a miss the DQN tries the same channel again — exactly the optimal rule of Theorem 1, discovered without being told.

policy maintains on a smaller example.

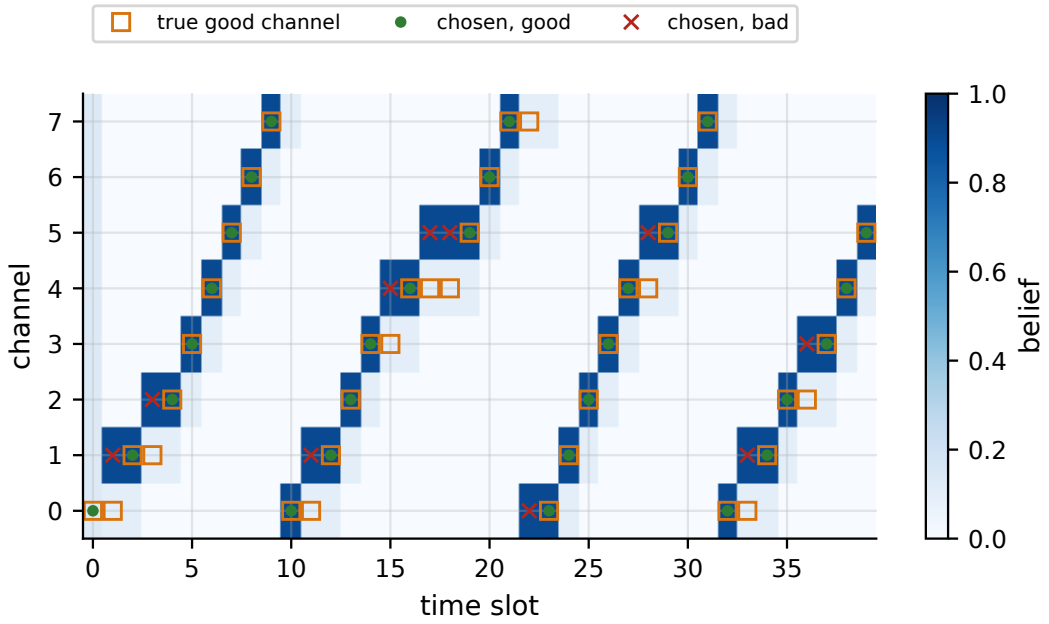


Figure 30.5: The exact Bayesian belief (colour) of the optimal policy for an 8-channel round-robin pattern with $p = 0.9$. At the start the belief is uniform and the policy searches; after the first hit the belief concentrates on one or two channels and the policy follows the staircase. The DQN has to encode this same information implicitly in its hidden layers.

Figure 30.6 is where our small budget shows. With the *same* 5000-step budget, the DQN nearly matches the optimum when the pattern is nearly deterministic ($p = 0.95$: 0.893 vs. 0.897; $p = 0.9$: 0.776 vs. 0.796), but it is far from optimal when the pattern is noisy ($p = 0.7$: -0.17 vs. 0.40; $p = 0.3$: -0.09 vs. 0.40), although still far above random (-0.875). The reason is the signal-to-noise ratio of the experience: with $p = 0.7$ the "successor" relation between two channels is confirmed only 70% of the time, and the optimal rule ("after a miss, retry") pays off only after the network has already learned every successor. The paper, with 10^6 training iterations, reports that the DQN reaches the optimum for all five values of p it tests; we therefore read our noisy-p points as *under-trained*, not as a contradiction. Re-running `psweep_one.py` with more steps or a smaller learning rate is the natural next experiment.

30.7.4 Result 2: correlated channels

Figure 30.7 reproduces the qualitative ordering the paper reports for its perfectly correlated scenario: random $\ll$ independent heuristic $<$ DQN $\lesssim$ myopic-with-full-knowledge. The DQN closes about 60% of the gap between the heuristic and the full-knowledge myopic policy. Why does it beat the heuristic? Because a single observation tells it about *several* channels at once: if channel 5 is good, its copies are good and its inverses are bad. The heuristic updates only the channel it sensed.

30.7.5 Result 3: adapting to a changing environment

Figure 30.8 shows the same three phases the paper reports: good performance, collapse after the change, detection and re-learning. The numbers: average reward 0.71 before the change, -0.92 in the 50 slots before detection, 0.64 over the last 300 slots.

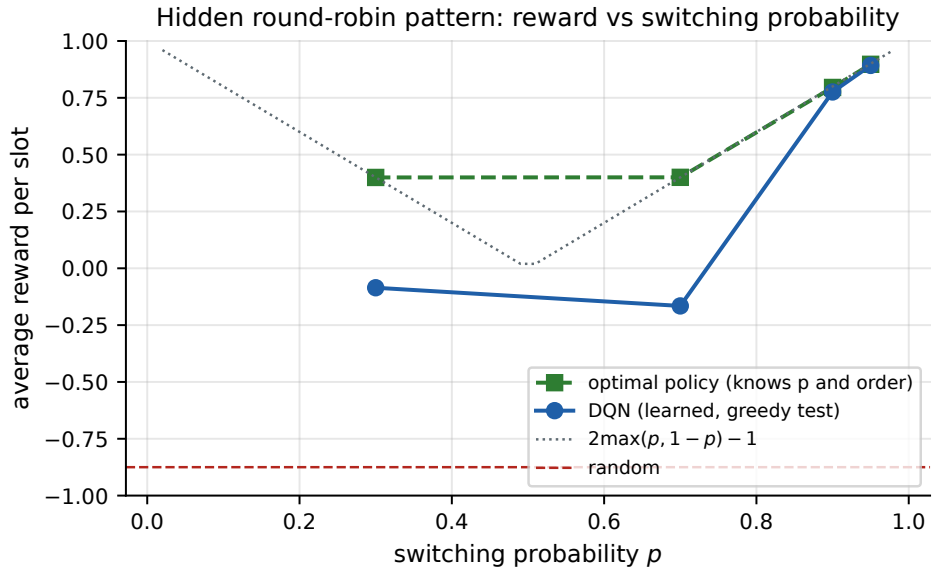


Figure 30.6: Our reproduction of the switching-probability sweep: average reward of the optimal policy and of the greedy DQN (each trained 5000 steps $\times$ 32 users) for $p \in \{0.3, 0.7, 0.9, 0.95\}$. The dotted curve is the steady-state optimum $2\max(p, 1-p) - 1$. With this small budget the DQN is near-optimal for the nearly deterministic patterns ($p \geq 0.9$) but clearly under-trained for the noisy ones ($p = 0.3, 0.7$), although far above random.

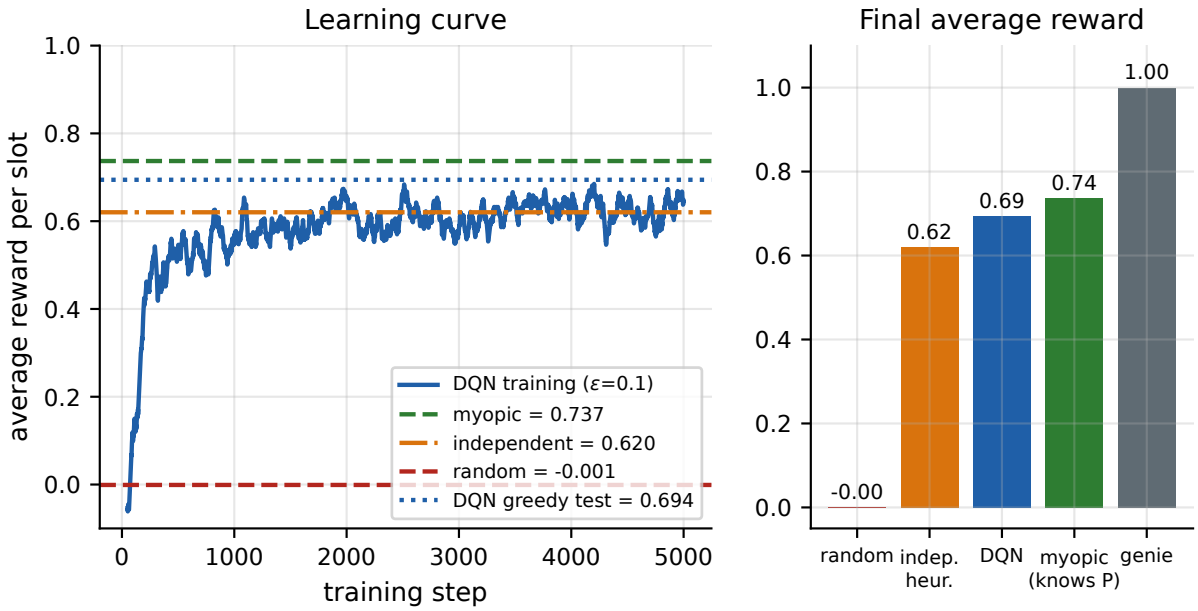


Figure 30.7: Our reproduction, 16 channels driven by 3 hidden Gilbert–Elliott chains (copies and inverses). Left: DQN training reward vs. the baselines. Right: final greedy reward. Random earns 0 (each channel is good half the time). The independent-channel heuristic reaches 0.62; the DQN, which knows nothing about the model, reaches 0.69; the myopic policy, which knows $\mathbf{P}$ and tracks the exact joint belief, reaches 0.74. The genie, seeing all channels, almost always finds a good one.

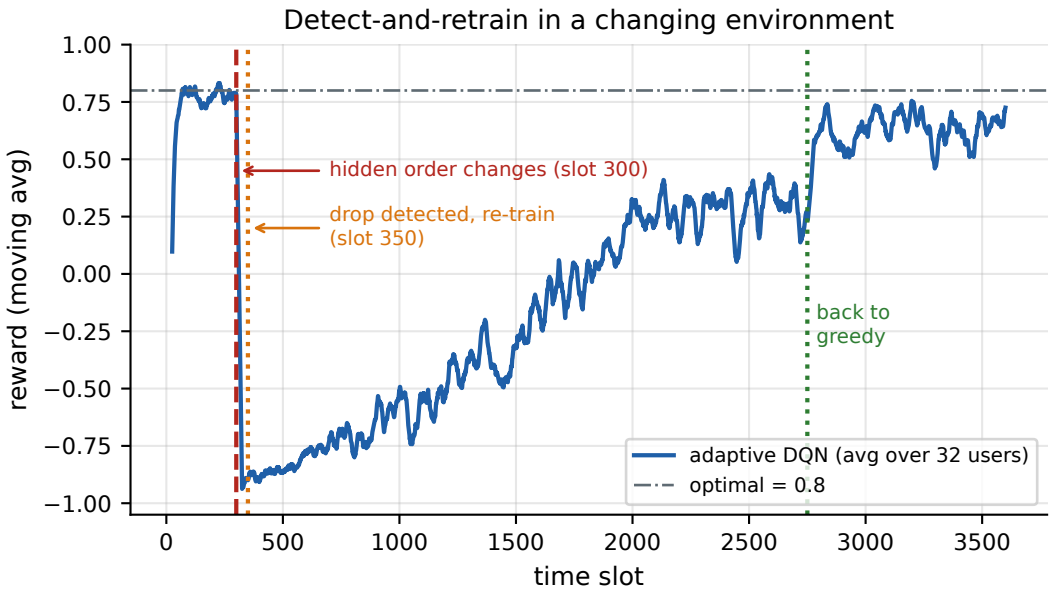


Figure 30.8: Our version of the adaptive DQN. The agent starts with the network trained in Figure 30.3. At slot 300 the secret order silently changes. The reward collapses to about -0.9, the 50-slot monitor detects the drop at slot 350, the agent re-trains online with a fresh replay memory (exploration keeps the curve noisy), and it recovers to a reward of about 0.64 (still exploring) before returning to greedy action.

Table 30.2: Summary of our reproduction (average reward per slot, greedy test; random and model-based baselines over 50,000 slots). Compare only qualitatively with the paper, which uses discounted rewards, different scenarios and 10^6 training iterations.

Scenario	Random	Indep. heur.	DQN	Optimal / myopic	Genie
Round-robin, $p = 0.9$	-0.875	—	0.776	0.796	1.0
Correlated (3 hidden chains)	0.00	0.62	0.69	0.74	1.0

Pitfall: what our reproduction does not show

We did not implement the Whittle index itself (only an independent-channel stand-in), we did not have the paper's real Tutornet trace, and our DQN is trained for far fewer steps. Our hyperparameters were chosen to make training fast, not tuned to match the paper. The agreement is qualitative: the *ordering* of methods and the *shape* of the learning and adaptation curves.

30.8 Limitations and follow-up work

- **Single user.** The paper studies one user; it discusses the multi-user case only as future work and notes that centralised approaches become computationally expensive. With several learners, each user's environment includes the others' policies and becomes non-stationary.
- **Fixed window.** The history window M is a hand-chosen approximation of the belief. Recurrent networks (Deep Recurrent Q-Networks, Hausknecht and Stone 2015) remove this choice by learning a memory; later multi-user spectrum-access work used LSTMs for exactly this reason.
- **Sample cost.** 10^6 iterations is cheap in simulation but slow on air. Our reproduction already hints at the gap: a hidden pattern of 16 facts took thousands of (parallel) slots to learn.
- **Detect-and-retrain is crude.** Algorithm 3 throws away what was learned and needs a threshold. Continual-learning or meta-learning approaches try to adapt faster.
- **One channel, one bit.** Real systems can sense several channels, get rates rather than good/bad, and care about delay or energy as well.

A representative follow-up is Naparstek and Cohen, "Deep multi-user reinforcement learning for distributed dynamic spectrum access", IEEE Transactions on Wireless Communications (2019), which uses an LSTM-based deep Q-network so that several users learn distributed access policies.

30.9 In practice: cognitive radio, NR-U and Wi-Fi channel selection

Wireless connection: 5G NR-U and Wi-Fi

In unlicensed bands (5 GHz, and 6 GHz where permitted), 5G NR-U and Wi-Fi share 20 MHz channels under listen-before-talk: before transmitting, a node performs a clear-channel assessment and only proceeds if the channel is idle. Each assessment is exactly the paper's observation: one bit about one channel, after the choice. Traffic of a neighbouring access point is bursty (positively correlated in time) and spills across adjacent channels (correlated across channels). A scheduler that learns, for example, "when sub-band 3 is busy, sub-band 4 is usually busy too, and sub-band 1 frees up within a few milliseconds" is learning exactly the correlation that the DQN exploited above.

Wireless connection: a deployment recipe

1. **Define the slot and the bit**: e.g. one LBT attempt per slot; "good" = clear assessment and acknowledged transmission.
2. **Log first**: record the per-channel outcome trace (as the authors did on Tutornet) and

train offline on replayed traces before going on air.

3. **Start from the cheap baselines**: a myopic policy on per-channel two-state estimates is a strong, explainable baseline; deploy the DQN only if it beats it on held-out traces.
4. **Keep the network small**: the input is $N \cdot M$ numbers and the output N Q-values; a few thousand weights fit easily into a DSP or a microcontroller.
5. **Monitor and re-train**: implement the adaptive loop with a conservative threshold, and fall back to the baseline while re-training.

Wireless connection: cognitive radio

For a secondary user in licensed spectrum the reward should penalise collisions with primary users more than a missed opportunity — a simple change to (30.2) (e.g. -5 for a collision) that the DQN accepts without any change to the algorithm. That flexibility, not just accuracy, is a key practical advantage over index policies, whose optimality proofs depend on the exact reward structure.

30.10 Recap, exercises and quiz

Chapter recap

- Dynamic multichannel access: N channels, each good/bad, joint hidden Markov chain; sense one per slot; reward $+1/-1$.
- It is a POMDP; the exact belief has 2^N entries and tabular history-based Q-learning has $(2N)^M$ states — both explode.
- Myopic and Whittle-index policies are optimal for independent identical positively correlated channels (and for 2–3 negatively correlated channels) but ignore cross-channel correlation.
- The paper's DQN reads the last $M = N$ (action, observation) pairs as $\pm 1/0$ vectors, uses fully connected ReLU layers (2×200 or 3×50), Adam, $\gamma = 0.9$, $\epsilon = 0.1$, replay and a target network.
- Reported: matches the optimal policy on fixed patterns; close to full-knowledge myopic on correlated channels; 0.9473 vs 0.7673 (Whittle heuristic) on a real trace; an adaptive re-training loop tracks changes.
- Our CPU reproduction: 0.776 vs optimal 0.796 on a hidden 16-channel pattern; 0.69 between the independent heuristic (0.62) and full-knowledge myopic (0.74) on correlated channels; detect-and-retrain recovers after a change.

Exercises

1. **(Belief)** A Gilbert–Elliott channel has $p_{11} = 0.7$, $p_{01} = 0.1$. (a) What is its stationary probability of being good? (b) It was observed good two slots ago and not since; what is the belief now?
2. **(Optimal reward)** For the single-good-channel round-robin with $p = 0.3$, state the

optimal rule and compute its average reward. What does random earn with $N = 8$?

3. **(Encoding)** With $N = 3$, $M = 2$, write the DQN input after: slot $t - 2$ channel 3 bad, slot $t - 1$ channel 1 good.
4. **(Sizes)** For $N = 8$ and $M = 8$, compute (a) the number of joint channel states, (b) the number of tabular history states, (c) the DQN input length, (d) the number of weights and biases of a 2×200 ReLU network with N outputs.
5. **(TD target)** A transition has $r = -1$, $\gamma = 0.9$ and target-network values $[2.0, 5.5, 4.1]$ at $\mathbf{x}'$. The online network gives $Q(\mathbf{x}, a) = 3.0$. Compute y and the squared error.
6. **(Conceptual)** Explain in your own words why the independent-channel heuristic cannot exploit the round-robin pattern, while the DQN can.
7. **(Conceptual)** Why does the training curve in Figure 30.3 plateau below the greedy test value? Estimate the plateau from the greedy value and $\epsilon = 0.1$.
8. **(Design)** Propose a reward for a cognitive-radio secondary user where collisions with the primary cost five times more than a successful transmission earns. How would the optimal behaviour change?

Answers

1. (a) $\omega^\star = p_{01}/(1 - p_{11} + p_{01}) = 0.1/(0.3 + 0.1) = 0.25$. (b) One slot after seeing good: $p_{11} = 0.7$. Next slot, unobserved: $0.7 \times 0.7 + 0.3 \times 0.1 = 0.49 + 0.03 = 0.52$.
2. With $p < 0.5$: after a success stay on the same channel; after a failure move to the next channel in the order. Each slot succeeds with probability $1 - p = 0.7$, so $\bar{r} = 0.7 - 0.3 = 0.4$. Random with $N = 8$: $\frac{1}{8} - \frac{7}{8} = -0.75$.
3. Most recent first: $[\,1, 0, 0 \mid 0, 0, -1\,]$.
4. (a) $2^8 = 256$. (b) $(2 \cdot 8)^8 = 16^8 = 4{,}294{,}967{,}296$. (c) $N \cdot M = 64$. (d) Layer 1: $64 \times 200 + 200 = 13{,}000$; layer 2: $200 \times 200 + 200 = 40{,}200$; output: $200 \times 8 + 8 = 1{,}608$; total 54,808.
5. $y = -1 + 0.9 \times \max(2.0, 5.5, 4.1) = -1 + 4.95 = 3.95$; squared error $(3.95 - 3.0)^2 = 0.9025$.
6. The heuristic models each channel separately: a channel's marginal probability of being good is about $1/16$ whatever happened elsewhere, and the knowledge "channel 3 was just good, so channel 11 is next" lives in the *joint* chain, which the heuristic discards. The DQN sees the joint history (which channel was good when) and its hidden layers can represent the successor relation.
7. During training 10% of actions are random, and a random action hits the single good channel only $1/16$ of the time. Roughly, the plateau is $0.9 \times 0.78 + 0.1 \times (-0.875) \approx 0.70 - 0.09 = 0.61$, a bit above what we see (≈ 0.55), because a random action also corrupts the next slot's history and decision.
8. For example $r = +1$ for success, -5 for collision with the primary, and 0 for choosing to stay silent (if that action is added). The user becomes more cautious: it transmits only when the probability of a free channel exceeds $5/6$ (where $p \cdot 1 - (1 - p) \cdot 5 > 0$), otherwise it prefers to wait and sense.

Self-test quiz

1. What does the user observe in each slot?
   Only the state (good/bad) of the one channel it chose.
2. Why is the exact POMDP solution impractical for $N = 16$?
   The belief has $2^{16} = 65{,}536$ entries and lives in a continuous space; POMDPs are PSPACE-hard in general.
3. When is the myopic policy optimal?
   For independent identical Gilbert–Elliott channels that are positively correlated ($p_{11} \geq p_{01}$), any N; and for 2–3 negatively correlated channels.
4. What is the DQN's input in the paper?
   The last $M = N$ slots, each a length-N vector with $+1/-1$ at the chosen channel (good/bad) and 0 elsewhere.
5. Name two hyperparameters the paper reports.
   Any of: $\gamma = 0.9$, $\epsilon = 0.1$, minibatch 32, Adam, ReLU, learning rate 10^{-4} (Sec. VII) or 10^{-5} (Sec. VIII), 10^6 iterations.
6. What triggers re-training in the adaptive DQN?
   A drop of the monitored accumulated reward by more than a user-set threshold.

Further reading

- S. Wang, H. Liu, P. H. Gomes, B. Krishnamachari, "Deep reinforcement learning for dynamic multichannel access in wireless networks", *IEEE Trans. Cognitive Communications and Networking*, 4(2):257–265, 2018 (arXiv:1802.06958). The authors published their channel environments and trace at `github.com/ANRGUSC/MultichannelDQN-channelModel`.
- Q. Zhao, B. Krishnamachari, K. Liu, "On myopic sensing for multi-channel opportunistic access: structure, optimality, and performance", *IEEE Trans. Wireless Communications*, 2008.
- S. H. A. Ahmad, M. Liu, T. Javidi, Q. Zhao, B. Krishnamachari, "Optimality of myopic sensing in multichannel opportunistic access", *IEEE Trans. Information Theory*, 2009.
- K. Liu, Q. Zhao, "Indexability of restless bandit problems and optimality of Whittle index for dynamic multichannel access", *IEEE Trans. Information Theory*, 2010.
- V. Mnih et al., "Human-level control through deep reinforcement learning", *Nature*, 2015 — the DQN recipe (replay + target network).
- O. Naparstek, K. Cohen, "Deep multi-user reinforcement learning for distributed dynamic spectrum access", *IEEE Trans. Wireless Communications*, 2019.

Chapter 31

Online Learning in BitTorrent Systems

This last paper chapter moves from the radio link to a whole network of selfish users. BitTorrent, the most successful peer-to-peer (P2P) file-sharing protocol, splits a file into pieces and lets every downloader upload the pieces it already has to others. Whether that works depends on one decision each peer makes every ten seconds: *to whom do I upload?* BitTorrent answers with a simple rule, "tit-for-tat". The paper we study replaces that rule with a learning agent that models how every neighbour reciprocates and plans for the long term, using exactly the MDP machinery of Chapter 19.

You will learn BitTorrent from zero (swarm, tracker, seeds and leechers, pieces, rarest-first, choking and unchoking, tit-for-tat, optimistic unchoking, free-riding), then the paper's formulation (a repeated stochastic game in which each peer solves a small MDP learned from observations), what it reports from a 100-node PlanetLab experiment, and you will build a small swarm simulator to test the idea yourself — with honest results that only partly agree with the paper. We finish with why the same "learn who reciprocates" idea matters for edge caching, device-to-device (D2D) content sharing and wireless resource allocation.

Paper at a glance: Izhak-Ratzin, Park, van der Schaar (2012)

Paper. R. Izhak-Ratzin, H. Park and M. van der Schaar, "Online Learning in BitTorrent Systems", *IEEE Transactions on Parallel and Distributed Systems*, vol. 23, no. 12, pp. 2280–2288, 2012 (DOI 10.1109/TPDS.2012.90). A shorter version, "Reinforcement learning in BitTorrent systems", appeared at IEEE INFOCOM 2011.

Problem. BitTorrent's peer selection (tit-for-tat plus random optimistic unchoking) is myopic and noisy: it reacts only to the last 20 seconds, keeps changing partners, and still donates bandwidth to free-riders.

Key idea. Model the peers' interactions as a repeated stochastic game. Each peer observes how its neighbours reciprocate, estimates their (discretised) states, rewards and transition probabilities, and computes a policy that maximises its long-term discounted download rate; the policy decides whom to unchoke.

Main results (as reported). Implemented in a real BitTorrent client and tested on PlanetLab: high-capacity peers download up to 33% faster, peer-selection fluctuations drop by 57%, and uploads to free-riders drop by 64% compared with regular BitTorrent.

31.1 BitTorrent from zero

Intuition: a potluck dinner

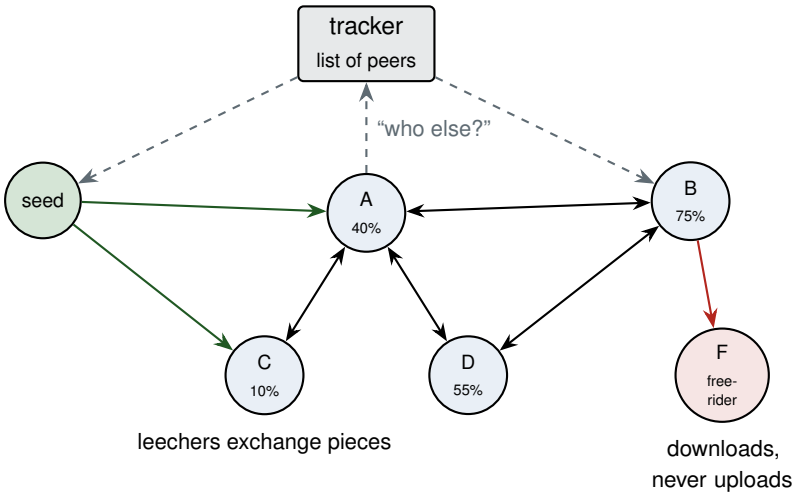
Fifty people want the same 400-page book. One person (the *seed*) owns a copy but has a slow photocopier. Instead of everybody queueing at the seed, the book is cut into 400 pages. Each person photocopies a few pages from whoever has them and immediately lets others copy what they hold. Soon everyone is both a reader and a copier, and the total copying capacity

grows with the number of people. The only problem: copying for others costs you time, and some people only take. BitTorrent's rules exist to make taking without giving unattractive.

31.1.1 The cast

Definition: swarm, seeds, leechers, tracker

A *swarm* is the set of peers sharing one file (described by a small `.torrent` metadata file). A *seed* has the complete file and only uploads. A *leecher* is still downloading; it uploads the pieces it already has. The *tracker* is a server that keeps a list of peers in the swarm; a joining peer asks it for a random subset of peers (its *peer set* or neighbours) and connects to them directly. (Modern clients can also find peers without a tracker, via a distributed hash table.)



31.1.2 Pieces and rarest-first

The file is split into pieces (typically 256 kB to a few MB), each with a SHA-1 hash in the metadata so that corrupted data can be detected. Peers tell each other which pieces they hold (a *bitfield*, then *have* messages). Which missing piece should a leecher request next?

Definition: rarest-first

Request the piece that is held by the *fewest* peers in your neighbourhood.

Why? If a rare piece is held only by the seed and the seed leaves, nobody can ever finish. Spreading rare pieces first keeps every piece available, and it makes each leecher *interesting* to others (it holds pieces they lack).

Worked example: rarest-first by counting

A leecher lacks pieces 4, 7 and 9. Among its 10 neighbours, piece 4 is held by 6, piece 7 by 2 and piece 9 by 5. Rarest-first asks for piece 7 first (availability 2), then 9, then 4.

31.1.3 Choking, unchoking and tit-for-tat

A peer could upload to all its neighbours at once, but splitting bandwidth over dozens of TCP connections is inefficient. Instead each peer uploads to only a few at a time.

Definition: choke / unchoke

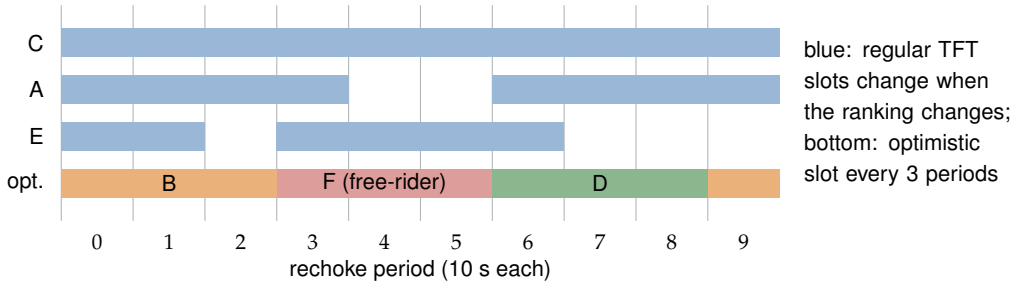
Peer j *unchokes* peer i when it agrees to upload to i; it *chokes* i when it refuses. BitTorrent keeps about four peers unchoked and re-decides every 10 seconds (a *rechoke period*).

Definition: tit-for-tat and optimistic unchoke

Tit-for-tat (TFT): the regular unchoke slots (typically three) go to the interested neighbours that uploaded *to me* at the highest rates over roughly the last 20 seconds. *Optimistic unchoke:* one extra slot goes to a randomly chosen neighbour, re-drawn every 30 seconds, so that new peers can get started and better partners can be discovered.

Worked example: one rechoke decision

Over the last 20 s peer j received from its interested neighbours: A 3 pieces, B 1, C 5, D 0, E 2. TFT unchokes the three best reciprocators: C, A, E. The optimistic slot goes to B or D at random. If D is a newcomer with nothing yet to give, the optimistic slot is its only chance to obtain its first pieces.



TFT is the famous strategy from Axelrod's repeated Prisoner's Dilemma tournaments: cooperate with those who cooperate with you. In BitTorrent it gives fast uploaders fast downloads, because fast uploaders tend to unchoke each other.

31.1.4 Free-riding and the limits of TFT

Definition: free-rider

A peer that downloads but uploads nothing (or very little).

Under TFT a free-rider never earns a regular slot, but it still benefits from *every* peer's optimistic unchoke and from seeds, which upload to anyone. Worse, TFT has weaknesses that measurement and modelling papers pointed out: its 20-second memory is short and noisy, so partners change often; and strategic clients (for example BitTyrant, Piatek et al. 2007) can exploit it. The paper's authors argue that peers should instead *learn* each neighbour's reciprocation behaviour and plan for the long run.

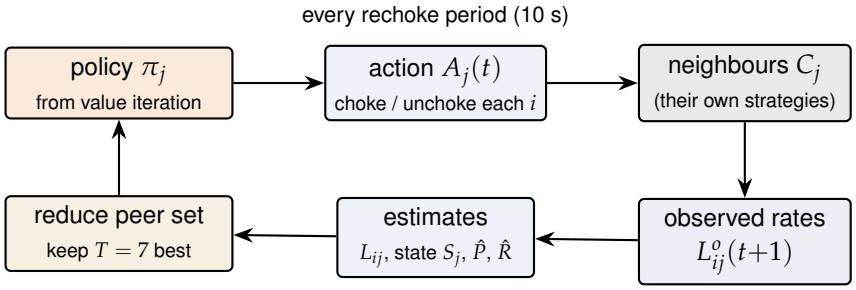
Wireless connection: the same structure in radio networks

Replace "peer" by "device", "pieces" by "content chunks" and "upload slot" by "a transmission opportunity on a D2D link" and you have cooperative content distribution in a cellular network. A phone that relays a video chunk to a neighbour spends battery and airtime — exactly the cost that tempts free-riding. Incentive and partner-selection mechanisms designed for BitTorrent are therefore directly relevant to D2D and edge-caching systems (Section 31.6).

31.2 The paper's formulation

31.2.1 A repeated stochastic game

The paper models the swarm as a stochastic game $\langle \mathcal{I}, \mathcal{S}, \mathcal{A}, P, R \rangle$: $\mathcal{I}$ is the set of peers, $\mathcal{S}$ and $\mathcal{A}$ are the joint state and action spaces, P the transition probabilities and R the rewards. A peer cannot control or even see the whole game, so each peer j treats its *associated peers* C_j (the neighbours it interacts with) as part of its environment and solves its own MDP.



31.2.2 States, actions, rewards

Reciprocation estimate. Peer j keeps, for each neighbour i, an estimate L_{ij} of the rate at which i uploads to j, updated after each period with the newly observed rate L^o_{ij} as a weighted average:

$$L_{ij}(t+1) = \gamma_j \, L^o_{ij}(t+1) + (1 - \gamma_j) \, L_{ij}(t), \qquad \gamma_j = 0.5 \text{ in the experiments.} \tag{31.1}$$

For a neighbour it has no history with, j is *optimistic*: it assumes the neighbour will reciprocate with high probability and at a high rate.

State. The state of peer j is the tuple of discretised reciprocation levels of its neighbours: $s_{ij} = 1$ if $L_{ij} > \theta_j$ and 0 otherwise, where θ_j is a predetermined threshold of peer j:

$$S_j(t) = \big(s_{ij}(t)\big)_{i \in C_j} \in \{0,1\}^{|C_j|}.$$

Action. $a_{ji}(t) \in \{0,1\}$: 0 if j chokes i, 1 if it unchokes i; the upload bandwidth is divided equally among the unchoked peers.

Reward. The reward of peer j in a state is the sum of the estimated download rates from its associated peers,

$$R_j\big(S_j(t)\big) = \sum_{i \in C_j} L_{ij}(t). \tag{31.2}$$

Objective. The policy maximises the cumulative discounted expected reward from the current period t_c on,

Key equation: long-term reciprocation

$$R_j^f\big(S_j(t_c)\big) = \mathbb{E}\Big[\sum_{t=t_c+1}^{\infty} \gamma^{\,t-(t_c+1)}\, R_j\big(S_j(t)\big)\Big], \qquad \gamma = 0.8 \text{ in the experiments.} \tag{31.3}$$

31.2.3 Learning the model and planning

The transition probabilities $\Pr[s_{ij}(t{+}1) \mid s_{ij}(t), a_{ji}(t)]$ are estimated from *empirical frequencies*: each period, j stores the triplet $(s_{ij}(t), a_{ji}(t), s_{ij}(t{+}1))$ for each neighbour and counts. With the estimated model in hand, the policy can be obtained by standard dynamic programming — the paper names value iteration and policy iteration as suitable methods. This is *model-based* reinforcement learning: learn $\hat{P}, \hat{R}$ from experience, then plan.

Intuition: why plan instead of react?

TFT asks: "who gave me the most in the last 20 seconds?" The learning peer asks: "if I unchoke Alice now, how likely is she to keep reciprocating next period, and the one after?" A peer that gave little in one period because it was itself starved may still be a reliable long-term partner; a peer that gave a lot once and then left is not. The discount factor $\gamma = 0.8$ means a period five steps ahead still counts with weight $0.8^5 \approx 0.33$.

Worked example: value iteration for one neighbour

Consider a single neighbour with state $s \in \{0,1\}$ (not reciprocating / reciprocating) and action $a \in \{0,1\}$ (choke / unchoke). Suppose the learned model is

$$\Pr[s'{=}1 \mid s,a]: \ (1,1) \to 0.9,\ (0,1) \to 0.6,\ (1,0) \to 0.3,\ (0,0) \to 0.05,$$

and the expected pieces received per period are $R(0,0) = 0$, $R(0,1) = 0.2$, $R(1,0) = 1$, $R(1,1) = 2$. Value iteration starts from $V = 0$; after one sweep $Q = R$, so $V = (0.2, 2)$. Second sweep, for example

$$Q(1,1) = 2 + 0.8\,\big(0.1 \times 0.2 + 0.9 \times 2\big) = 2 + 0.8 \times 1.82 = 3.456.$$

Iterating to convergence gives $Q(1,1) \approx 9.05$, $Q(1,0) \approx 6.92$, $Q(0,1) \approx 6.68$, $Q(0,0) \approx 5.44$: unchoking is better in both states, and even a currently silent neighbour ($s = 0$) is worth unchoking because unchoking makes it likely to start reciprocating.

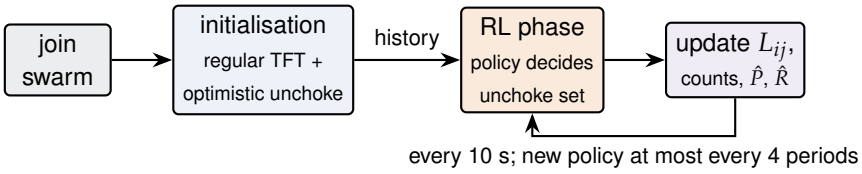
31.2.4 Making it tractable

With $|C_j|$ neighbours there are $2^{|C_j|}$ states and $2^{|C_j|}$ joint actions. The paper therefore reduces the peer set: j iteratively removes the neighbours with the smallest L_{ij}, keeping $T = 7$ (it reports that larger reduced sets make policy computation significantly slower).

31.2.5 The protocol

The mechanism replaces only BitTorrent's peer selection; piece selection (rarest-first) is unchanged.

1. **Initialisation phase:** use regular BitTorrent (TFT plus optimistic unchoking) to collect reciprocation history.
2. **RL phase:** every 10 seconds (as in BitTorrent) choose the unchoked set from the policy; a computed policy is kept for up to three additional rechoke periods; at least four peers are unchoked, more if the upload capacity is not saturated. Random decisions are used only in the initialisation phase or to collect history for newly joined peers.



31.3 Results as reported

Setup. The mechanism was implemented in the Enhanced CTorrent client (version 3.2), which can run in RL-enhanced or regular mode. Unless otherwise specified, experiments ran on 104 PlanetLab nodes: 100 leechers and 4 seeds with a combined capacity of 128 KB/s, serving a 99 MB file, all leechers starting together (a flash crowd). Upload capacities followed an empirical distribution of BitTorrent leechers (from measurements covering more than 300,000 IPs), scaled down by 1/20 to fit PlanetLab. Download capacity was not limited.

One learning peer in a regular swarm. Over seven trials, a single RL-enhanced leecher improved its median download time by 12–27%, for both high- and low-capacity leechers — the incentive for an individual to adopt the protocol.

Everyone learns. When all leechers (50 in this experiment) used the RL-enhanced mechanism, high-capacity leechers improved their median completion time by up to 33%. The mechanism increased collaboration among the top-capacity peers, and the paper reports that high-capacity leechers could raise their download rates as much as their upload rates. The price is paid by low-capacity leechers, whose download rates decreased by at most 36%; the paper notes that all slowed-down peers still downloaded faster than they uploaded. In other words the system became *fairer* in the sense of "you get what you give".

Stability. Comparing consecutive rechoke decisions, regular BitTorrent changed on average 2.1 peers per measurement, RL-enhanced 0.9 — the 57% reduction in fluctuations.

Free-riders. In networks with 50 contributing leechers and a varying number of free-riders, free-riders needed 8–20% more time (median) in the RL-enhanced swarm. Regular BitTorrent leechers uploaded about 2.8–3.7 times more to free-riders; for example, with 15 free-riders they gave 4.5% of their upload capacity to free-riders, versus 1.6% with RL — the 64% reduction.

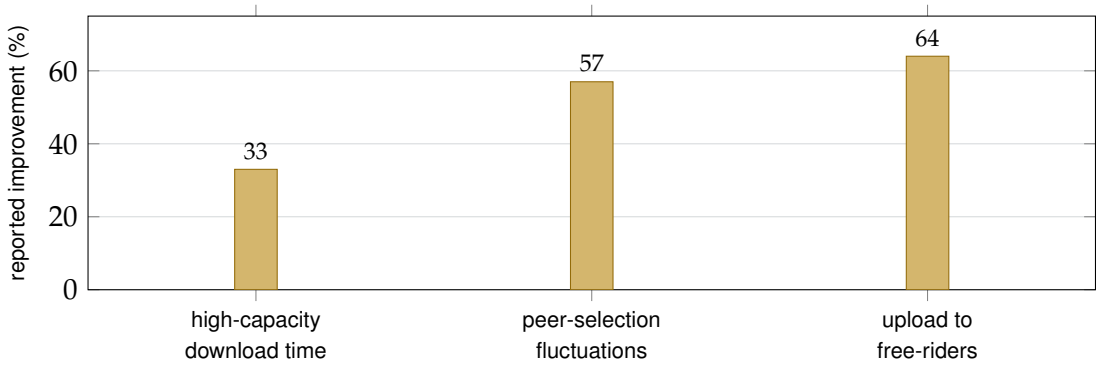


Figure 31.1: The paper's three headline numbers *as reported* (PlanetLab, RL-enhanced vs. regular BitTorrent): up to 33% shorter median completion time for high-capacity peers, 57% fewer peer-selection changes, 64% less upload to free-riders. Re-plotted by us from the text.

31.4 Our small-scale reproduction

We built a small, piece-level, round-based swarm simulator (`code/ch31/btsim.py`). It is far simpler than PlanetLab and the real client, so the goal is to understand the mechanism and see which effects survive, not to reproduce the paper's numbers.

31.4.1 The simulator

- A 200-piece file; 2 seeds of 4 pieces/round each; 24 leechers in four upload-capacity classes of 1, 2, 4 and 8 pieces per round (6 peers each); optionally 6 free-riders (capacity 0). Every peer knows every other (full mesh); download capacity is unlimited.
- One *round* = one rechoke period. Each uploader unchokes 4 peers and splits its capacity equally among them; fractional pieces accumulate per link. The uploader sends the rarest piece (in the swarm) that the receiver lacks. Leechers leave on completion; seeds unchoke random interested leechers.
- **TFT peers**: 3 regular slots to the peers that sent them most in the last 2 rounds, plus 1 optimistic slot re-drawn every 3 rounds.
- **Learning peers** (our simplified, *factored* version of the paper's mechanism): TFT for the first 6 rounds (initialisation); then, for each neighbour j, a two-state MDP with $s_j = \mathbb{1}[L_j > 0.1]$ and L_j updated by (31.1) with weight 0.5; transition counts with a uniform prior; rewards $\hat{R}(s,a)$ = recency-weighted average of pieces received from j after state s and action a, with a weak optimistic prior; value iteration with $\gamma = 0.8$; peer-set reduction to the $T = 7$ neighbours with largest L_j; unchoke the 3 with the largest $Q_j(s_j, 1)$, plus one slot to explore a peer never tried (or the 4th best); keep a decision for 3 rounds.

Pitfall: a factored approximation

The paper's MDP is over the *joint* state of up to 7 neighbours, with joint actions. Ours treats each neighbour independently (a two-state MDP per neighbour) and ranks neighbours by their value. This keeps the code short and the computation trivial, but it ignores interactions (e.g. unchoking one more peer reduces the rate given to the others).

In the listing below, `C[j,s,a,s']` are the transition counts, `Rsum/Rn` the reward estimates $\hat{R}_j(s,a)$, `L` the smoothed rates of (31.1), and `values()` runs value iteration on all per-neighbour

MDPs at once (with `einsum` doing the sum over next states).

Listing 31.1: The learning peer's per-neighbour model and value iteration (`btsim.py`).

```
    THETA = 0.1
    def __init__(self, n, u_i, lmax):
        self.L = np.full(n, float(lmax))           # optimistic initial estimate
        self.C = np.ones((n, 2, 2, 2))
        self.Rn = np.full((n, 2, 2), PRIOR_W)      # weak prior: PRIOR_W pseudo-
    observations
        self.Rsum = np.zeros((n, 2, 2))
        self.Rsum[:, :, 1] = lmax * PRIOR_W        # optimism for unseen peers
        self.prev_s = (self.L > self.THETA).astype(int)
        self.prev_a = np.zeros(n, int)
        self.hold, self.choice = 0, np.array([], int)

    def observe(self, recv):                       # recv[j] = pieces received
    from j this round
        idx = np.arange(len(recv))
        # recency-weighted mean (step 1/n, but never below R_STEP): tracks a
    changing swarm
        n_ = self.Rn[idx, self.prev_s, self.prev_a] + 1
        step = np.maximum(1.0 / n_, R_STEP)
        Rm = self.Rsum[idx, self.prev_s, self.prev_a] / self.Rn[idx, self.prev_s,
    self.prev_a]
        self.Rsum[idx, self.prev_s, self.prev_a] = (Rm + step * (recv - Rm)) * n_
        self.Rn[idx, self.prev_s, self.prev_a] = n_
        self.L = EMA * recv + (1 - EMA) * self.L   # paper: weighted average,
    weight 0.5
        s = (self.L > self.THETA).astype(int)
        self.C[idx, self.prev_s, self.prev_a, s] += 1
        self.prev_s = s

    def values(self):
        P = self.C / self.C.sum(-1, keepdims=True)          # P[j, s, a, s']
        R = self.Rsum / self.Rn                             # R[j, s, a]
        V = np.zeros((len(R), 2))
        for _ in range(30):                                 # value iteration
            Q = R + GAMMA * np.einsum('jsat,jt->jsa', P, V)
            V = Q.max(-1)
        return Q[np.arange(len(R)), self.prev_s, 1]         # Q_j(s_j, unchoke)
```

31.4.2 Result 1: the whole swarm learns

Figures 31.3 and 31.4 show our first honest disagreement. The paper reports that when everyone learns, high-capacity peers get up to 33% faster while low-capacity peers slow down. In our simulator the fastest class is unchanged and the *slowest* class speeds up. The mean completion time improves slightly (62.8 vs. 65.2 rounds). Our share ratios (uploaded/downloaded) by class are 0.43, 0.75, 0.94, 1.20 for TFT and 0.38, 0.70, 1.07, 1.26 for learning: the fast peers carry even more of the load.

Why the difference? In our simulator a fast peer's partners are chosen by long-term value, and every leecher that has *ever* reciprocated retains a decent value; the optimistic prior and the exploration slot also keep spreading bandwidth to slow peers. The paper's fast peers, with real TCP dynamics, a 99 MB file, joint-state planning and a seed bandwidth that is the real bottleneck, form tighter clusters. The stability effect, by contrast, reproduces clearly: the learning swarm changes 0.69 unchoke partners per peer per round versus 1.43 for TFT, a 52% reduction (paper:

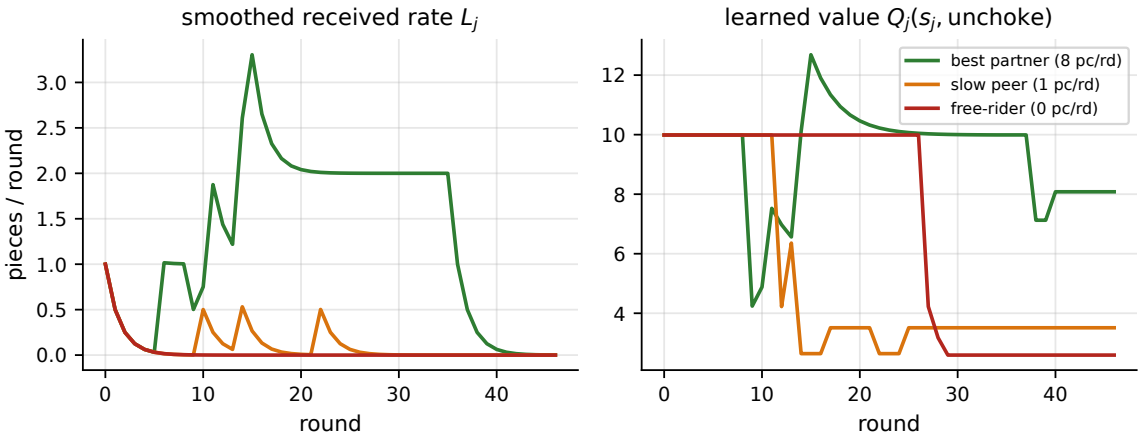


Figure 31.2: Inside one learning peer (capacity 8) in a TFT swarm with 6 free-riders. Left: the smoothed rate L_j received from three neighbours. Right: the learned value of unchoking them. All start at the optimistic prior (10). Once tried, the free-rider's value collapses (it never gives anything) and the slow peer's value drops to a low level; the best partner (another fast peer) keeps a high value while it reciprocates, and its value decays only after it finishes and stops uploading.

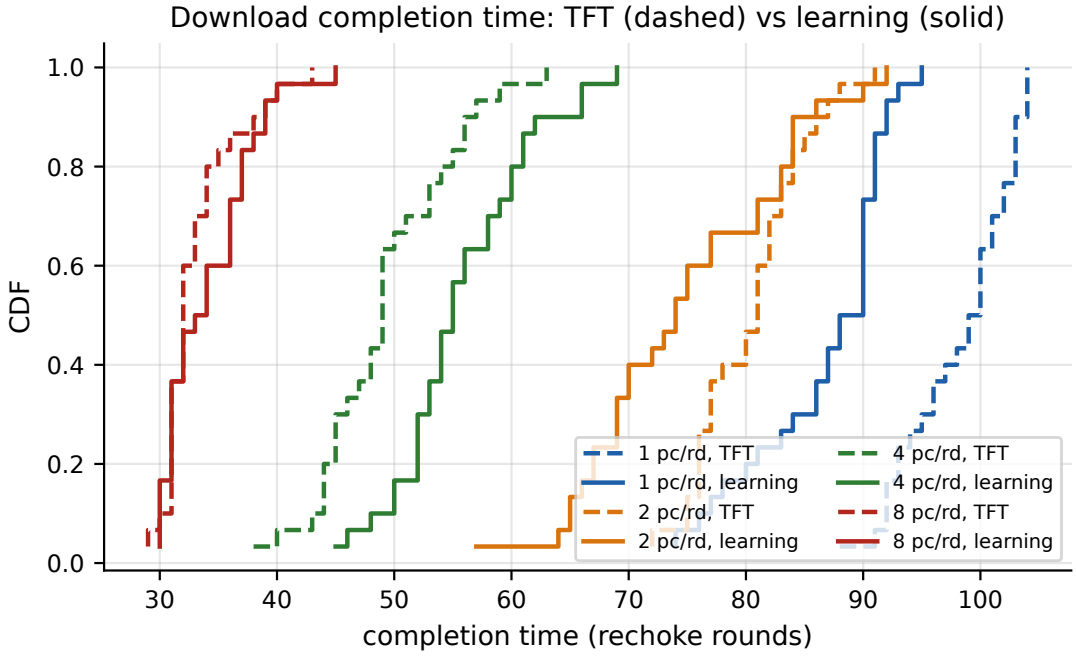


Figure 31.3: Our reproduction: CDF of completion times per capacity class, all 24 leechers using TFT (dashed) or the learning policy (solid), 5 random runs pooled. With learning the slowest class (blue) finishes earlier (mean 86.5 vs. 98.2 rounds) and the swarm empties sooner (last leecher 92.2 vs. 103.4 rounds), but the 4 pc/rd class (green) is slower (55.7 vs. 49.3) and the fastest class (red) is about the same (34.1 vs. 33.1).

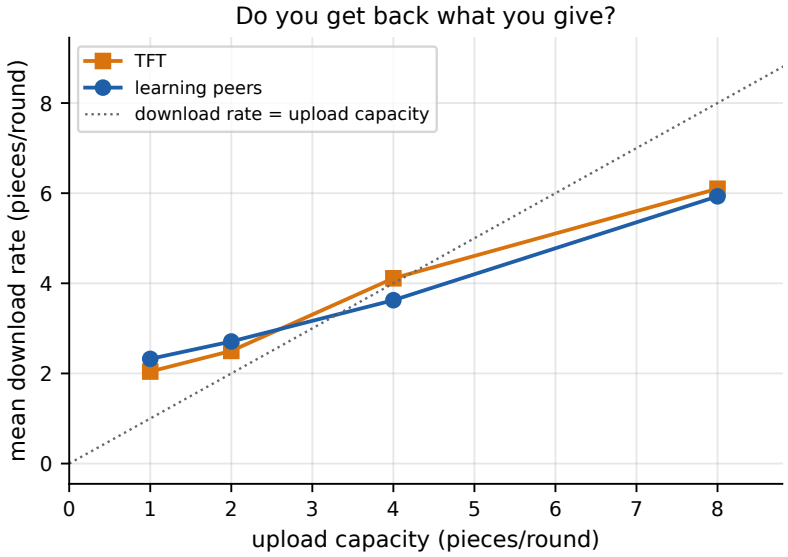


Figure 31.4: Download rate vs. upload capacity (mean over peers and runs). The dotted diagonal is perfect "you get what you give". Both policies give slow peers more than they contribute (they are subsidised by seeds and optimistic unchokes) and fast peers less. In our simulator the learning policy is slightly *flatter* (less proportional) than TFT — the opposite of the fairness effect the paper reports.

57%).

31.4.3 Result 2: one learning peer among TFT peers

In our model a single learner gains nothing (Figure 31.5). This is not too surprising: the learner's download rate is decided by what the *TFT peers* choose to give it, and TFT rewards whoever uploaded most in the last 20 seconds — something the learner does not change, since its upload capacity is fixed and always fully used. In the real client, a peer that keeps stable partners avoids TCP slow-start and connection churn on each switch; we tried a crude "ramp-up" penalty for newly unchoked links in the simulator (the `ramp` option) but did not include it in the figures, because its size is a free parameter we cannot calibrate.

31.4.4 Result 3: free-riders

Free-riding is discouraged in the right direction but much less than reported (Figure 31.6). In our small swarm a large part of the upload to free-riders happens at the very end, when most contributors have left and free-riders are the only interested peers — any policy then uploads to them. Our learning peer also spends its exploration slot on every untried neighbour once, including each free-rider.

Pitfall: read our numbers correctly

Our simulator has no TCP, no network latency, a 200-piece file, a full mesh and a factored MDP. Table 31.1 says which *qualitative* effects of learning-based peer selection appear even in such a toy (stability, some free-rider resistance) and which depend on details we do not model (individual and high-capacity speed-ups). It does *not* say the paper is wrong.

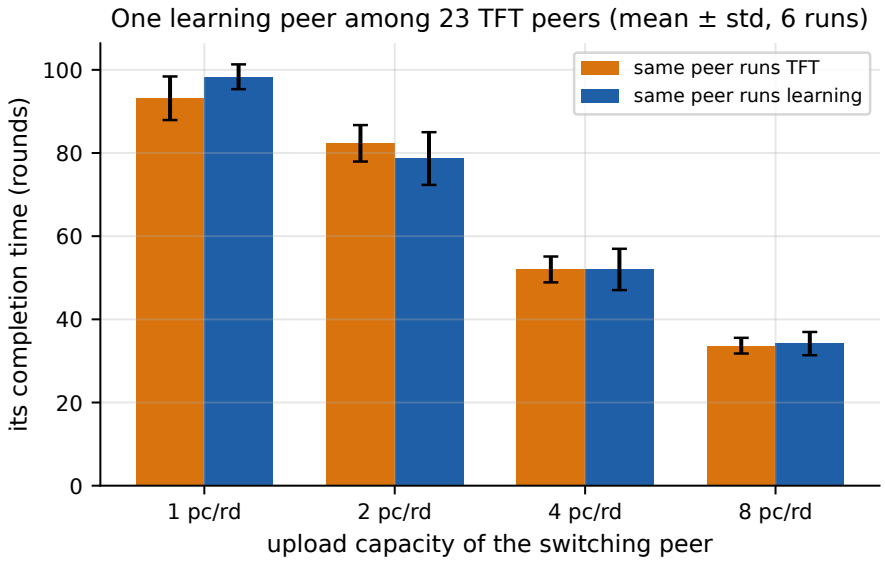


Figure 31.5: Our version of the paper's "single RL leecher" experiment: one peer of the given class switches from TFT to learning while the other 23 stay TFT (6 random runs; mean ± std). The differences are within the run-to-run spread: 93.2 vs. 98.3 rounds (1 pc/rd), 82.3 vs. 78.7 (2), 52.0 vs. 52.0 (4), 33.7 vs. 34.2 (8). We do *not* reproduce the 12–27% individual gain reported by the paper.

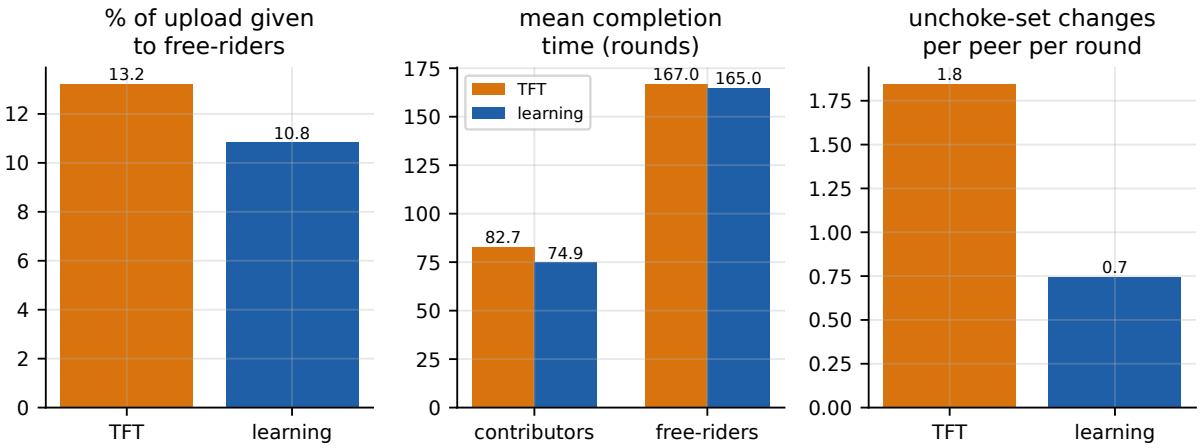


Figure 31.6: Our reproduction with 24 contributors and 6 free-riders (mean of 4 random runs). Left: share of the contributors' upload that goes to free-riders — 13.2% with TFT, 10.8% with learning (an 18% reduction; paper: 64%). Middle: contributors finish faster with learning (74.9 vs. 82.7 rounds) while free-riders are barely affected (165.0 vs. 167.0), because seeds serve them anyway. Right: unchoke-set changes per peer per round fall from 1.85 to 0.75.

Table 31.1: Paper vs. our reproduction (qualitative comparison only; very different scales).

Effect	Paper (PlanetLab, real client)	Ours (24-peer simulator)
Peer-selection stability	57% fewer changes	52% fewer changes (1.43 → 0.69)
Upload to free-riders	64% less	18% less (13.2% → 10.8%)
Free-riders' download time	8–20% longer	about the same
High-capacity peers (all learn)	up to 33% faster	about the same (+3% slower)
Low-capacity peers (all learn)	download rate down by at most 36%	12% faster
Single learner	12–27% faster	no significant change

31.5 Limitations and follow-ups

- **State explosion.** Even with discretised binary states, joint planning is exponential in the number of neighbours, forcing the 7-peer reduction.
- **Predetermined thresholds.** The discretisation threshold θ_j is a design parameter; poor choices make the states uninformative.
- **Non-stationarity.** Every learning peer changes the environment of every other; the empirical model is always slightly out of date.
- **Fairness has a price.** The paper's own results show low-capacity peers slowing down when everyone learns.
- **Scope.** The paper targets file sharing; its conclusion lists delay-sensitive content (multimedia streaming) and RL for seeds with modified rewards as future work.

31.6 Why this matters for wireless networks

Wireless connection: edge caching

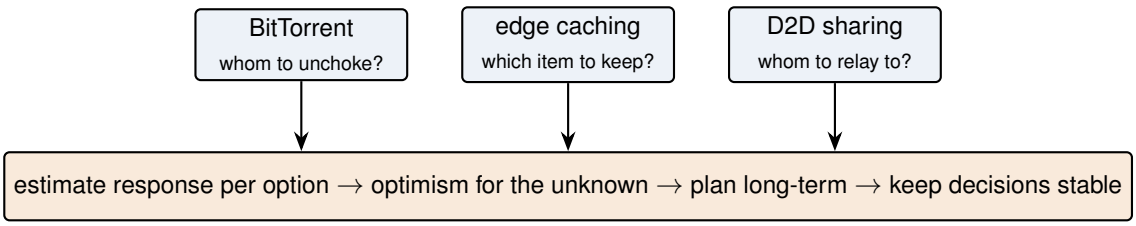
Base stations and access points with storage pre-fetch popular content during off-peak hours and serve it locally. Which content to keep is an online learning problem with delayed, partial feedback: you observe hits only for what you cache, just as the BitTorrent peer observes reciprocation only from peers it unchokes. Bandit and RL formulations of caching use the same ingredients we saw: an estimate per item (popularity instead of reciprocation), optimism for new items, exploration, and a long-term objective.

Wireless connection: D2D content sharing

In device-to-device sharing a phone that has a chunk can relay it to neighbours. Battery and airtime are the upload cost; selfish devices free-ride. A device can learn, per neighbour, "how likely is this phone to relay chunks back to me, and at what rate" — a direct translation of (31.1)–(31.3) — and give its limited transmission opportunities to reliable partners. Stability matters even more than in BitTorrent: every new D2D link costs discovery, synchronisation and possibly beam alignment.

Wireless connection: resource allocation

Schedulers and relay-selection algorithms repeatedly decide whom to serve with incomplete knowledge of how each user or relay will respond. The model-based recipe of this paper — discretise the response, count transitions, plan with value iteration, keep decisions for a few periods to avoid churn — is a lightweight alternative to deep RL when the per-user state is small and decisions must be explainable.



31.7 Recap, exercises and quiz

Chapter recap

- BitTorrent: a swarm of seeds and leechers; a tracker gives each peer a random neighbour set; the file is split into hash-checked pieces fetched rarest-first.
- Peer selection: about 4 unchoked peers, re-decided every 10 s; TFT gives regular slots to the best recent uploaders; one optimistic unchoke rotates every 30 s. Free-riders live on optimistic unchokes and seeds.
- The paper: a repeated stochastic game; each peer estimates neighbours' reciprocation (L_{ij} with weight 0.5), discretises it into states, learns transition frequencies, and maximises the discounted sum ($\gamma = 0.8$) of download rates by value/policy iteration over a reduced set of 7 peers.
- Reported on PlanetLab (104 nodes, Enhanced CTorrent): up to 33% faster for high-capacity peers, 57% fewer selection changes, 64% less upload to free-riders; a single learner gains 12–27%.
- Our toy simulator reproduces the stability gain (52% fewer changes) and a smaller free-rider reduction (18%), but not the speed-ups; learning made our swarm more egalitarian, not more proportional.
- The same "learn the response of each partner, plan, stay stable" pattern applies to edge caching, D2D sharing and resource allocation.

Exercises

1. **(Rarest-first)** Five neighbours hold the following pieces: N1 $\{1,2,3\}$, N2 $\{2,3\}$, N3 $\{3,4\}$, N4 $\{2,3,4\}$, N5 $\{3\}$. You lack pieces 1–4. In which order does rarest-first request them?
2. **(Reciprocation estimate)** With $\gamma_j = 0.5$ and $L_{ij}(0) = 4$, the observed rates in three periods are $2, 0, 0$. Compute L_{ij} after each period. After how many zero periods does it fall below $\theta_j = 0.3$?
3. **(Value iteration)** For the one-neighbour MDP of the worked example, verify $Q(0,1)$ after the second sweep.
4. **(Free-rider share)** A TFT peer has 4 slots, one of which is optimistic and re-drawn uniformly among 29 interested neighbours, 6 of whom are free-riders. Ignoring regular slots, what fraction of its upload goes to free-riders on average?
5. **(Complexity)** With a reduced peer set of $T = 7$ binary-state neighbours and binary choke/unchoke actions, how many joint states and joint actions are there? How many transition probabilities would a full joint model need?
6. **(Conceptual)** Why does planning with $\gamma = 0.8$ reduce peer-selection fluctuations compared with TFT?
7. **(Wireless)** Formulate D2D chunk relaying as in Section 31.2: define state, action and reward for a phone choosing which two neighbours to relay to.

Answers

1. Availability: piece 1 is held by 1 neighbour, piece 4 by 2, piece 2 by 3, piece 3 by 5. Order: 1, 4, 2, 3.
2. $L(1) = 0.5 \cdot 2 + 0.5 \cdot 4 = 3$; $L(2) = 0.5 \cdot 0 + 0.5 \cdot 3 = 1.5$; $L(3) = 0.75$. Then 0.375, 0.1875: it falls below 0.3 after the 4th consecutive zero period (the 5th period overall).
3. After the first sweep $V = (0.2, 2)$. $Q(0,1) = R(0,1) + 0.8\,[\Pr(0 \mid 0,1)V(0) + \Pr(1 \mid 0,1)V(1)] = 0.2 + 0.8\,(0.4 \times 0.2 + 0.6 \times 2) = 0.2 + 0.8 \times 1.28 = 1.224$.
4. One quarter of the bandwidth is optimistic, and a free-rider is chosen with probability 6/29: $\frac{1}{4} \times \frac{6}{29} \approx 5.2\%$.
5. $2^7 = 128$ joint states and $2^7 = 128$ joint actions; a full model needs $128 \times 128 \times 128 \approx 2.1$ million transition probabilities (minus normalisation constraints) — one reason the peer set must be small, and one reason our factored version is so much cheaper.
6. TFT ranks partners on a 20-second window, so random fluctuations (a TCP hiccup, a peer with no interesting piece for a moment) reorder the ranking and trigger switches. The learned values aggregate many periods and the transition model, so a single bad period changes them little; keeping a policy for several periods damps switching further.
7. State: for each neighbour, whether it relayed to me recently above a threshold (and possibly its battery level or link quality bin). Action: the set of two neighbours I transmit chunks to this period. Reward: the number of useful chunks received from neighbours, minus an energy cost per transmission. Estimate transitions from counts and plan with value iteration, as in the paper.

Self-test quiz

1. What is the difference between a seed and a leecher?
   A seed has the whole file and only uploads; a leecher is still downloading and uploads the pieces it has.
2. Why rarest-first?
   It keeps every piece available in the swarm and makes peers interesting to each other.
3. What does the optimistic unchoke do?
   Gives one upload slot to a random neighbour (rotated every 30 s), to bootstrap newcomers and discover better partners.
4. In the paper, what is a peer's state?
   The vector of binary reciprocation levels of its neighbours, $s_{ij} = \mathbb{1}[L_{ij} > \theta_j]$.
5. Which of the paper's three headline effects did our simulator reproduce best?
   The reduction of peer-selection fluctuations (52% vs. 57% reported).

Further reading

- R. Izhak-Ratzin, H. Park, M. van der Schaar, "Online learning in BitTorrent systems", *IEEE Trans. Parallel and Distributed Systems*, 23(12):2280–2288, 2012; and "Reinforcement learning in BitTorrent systems", *IEEE INFOCOM*, 2011.
- B. Cohen, "Incentives build robustness in BitTorrent", Workshop on Economics of Peer-to-Peer Systems, 2003 — the original design rationale for tit-for-tat and optimistic unchoking.

- A. Legout, G. Urvoy-Keller, P. Michiardi, “Rarest first and choke algorithms are enough”, ACM IMC, 2006.
- M. Piatek, T. Isdal, T. Anderson, A. Krishnamurthy, A. Venkataramani, “Do incentives build robustness in BitTorrent?”, USENIX NSDI, 2007 (BitTyrant).
- R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018 — model-based RL and value iteration.

Part VI

Kaggle-Style Labs: Modulation Classification

Chapter 32

Lab A: Feature-Based Modulation Classification in AWGN

This is the first of four *Kaggle-style labs*. In each lab you are handed a dataset of short radio recordings, a metric and a deadline, and your job is to climb a leaderboard. The task never changes: **look at 128 complex baseband samples and say which of eight modulations produced them**. What changes from lab to lab is the *channel* the signal went through and the *tools* you are allowed to use.

In this lab the channel is the friendliest one there is — additive white Gaussian noise (AWGN) plus a random carrier phase, a tiny frequency offset and a random timing offset — and the tools are the classical ones: *hand-crafted features* fed to off-the-shelf machine-learning classifiers. You will derive higher-order *moments* and *cumulants* from scratch, compute the theoretical cumulant table of every constellation (and check it against the classic Swami–Sadler table), understand the beautiful reason why Gaussian noise is "invisible" to cumulants above order two, add amplitude/phase/frequency features in the style of Azzouz and Nandi, and train five classifiers from scikit-learn. The best of them becomes the *baseline* that the deep networks of Labs B–D must beat.

Wireless connection: why modulation classification matters

Automatic modulation classification (AMC) sits between "a signal is present" and "decode it". Spectrum monitoring and regulators use it to identify emitters; cognitive and adaptive radios use it to follow a transmitter that changes its MCS; electronic-warfare and rogue-base-station detectors use it to fingerprint unknown transmitters; test equipment uses it to auto-configure a demodulator. In a 5G receiver the modulation is *signalled* (DCI tells the UE the MCS), so AMC is not needed there — but a passive scanner that has not decoded the control channel faces exactly the problem of this lab.

32.1 The competition

32.1.1 Problem statement

Every example is a vector $\mathbf{r} = (r[0], \ldots, r[127]) \in \mathbb{C}^{128}$ of complex baseband samples, taken at 8 samples per symbol, so one example spans only 16 symbols. The label is one of eight classes:

`BPSK, QPSK, 8PSK, 16QAM, 64QAM, PAM4, GFSK, CPFSK.`

The first six are *linear* modulations: a sequence of symbols a_k from a finite alphabet (the *constellation*) is shaped by a root-raised-cosine (RRC) pulse. The last two are *constant-envelope* frequency-shift keying with continuous phase: the information is in the instantaneous frequency, so every sample has the same amplitude.

32.1.2 How the data were generated

All four labs share one generator, `code/ch32/modsim.py`. For one example it does the following (Figure 32.1):

1. draw random symbols and form the transmitted waveform $s[n]$: for linear classes $s[n] = \sum_k a_k\, p[n - 8k]$ with an RRC pulse p of roll-off 0.35 spanning 8 symbols; for GFSK (Gaussian-filtered, $BT = 0.35$, modulation index $h = 0.5$) and CPFSK (rectangular, $h = 0.75$) $s[n] = e^{j\theta[n]}$ with $\theta[n]$ the running sum of the frequency pulse;
2. start at a random sample offset $0, \ldots, 7$ (*timing offset*);
3. pass s through the channel (identity in this lab; Rayleigh multipath in Lab C; an IIR filter in Lab D);
4. scale to *unit average power*, then rotate by a random carrier phase $\phi \sim \mathcal{U}[0, 2\pi)$ and a small carrier-frequency offset $f_0 \sim \mathcal{U}[-2 \times 10^{-4}, 2 \times 10^{-4}]$ cycles/sample:

$$r[n] = e^{j(2\pi f_0 n + \phi)}\, s[n] + w[n], \qquad w[n] \sim \mathcal{CN}(0, \sigma^2), \quad \mathrm{SNR} = \frac{1}{\sigma^2}; \tag{32.1}$$

5. SNR is drawn from the grid $\{-10, -8, \ldots, +20\}$ dB (16 values), with *the same number of examples for every (class, SNR) pair*.

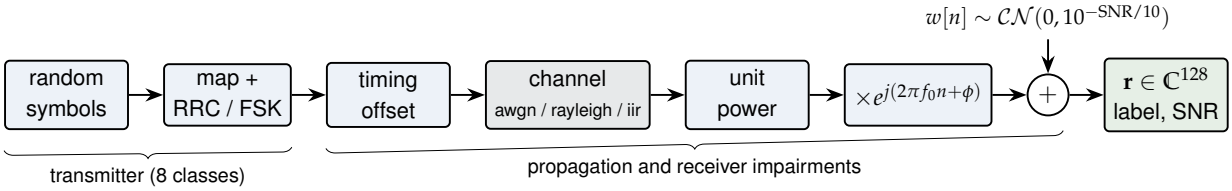


Figure 32.1: How one example of the competition is produced by `modsim.py`. Only the gray channel block changes between Labs A (identity), C (Rayleigh multipath) and D (IIR filter). Because the power is normalised *before* noise is added, the SNR is exactly $1/\sigma^2$.

32.1.3 Files, split and submission

The shared AWGN competition has 150 training and 100 test examples per (class, SNR) pair: $8 \times 16 \times 150 = 19\,200$ training and 12 800 test examples, generated with different random seeds. The script writes Kaggle-style files to `code/ch32/kaggle_awgn/`:

File	Content
`train.npy`	float32 array of shape $(19200, 2, 128)$: row 0 = I, row 1 = Q
`train_labels.csv`	`id,snr,label` (e.g. `17,4,16QAM`)
`train_head.csv`	the first 200 training rows in a flat CSV: `id,snr,label,I0..I127,Q0..Q127`
`test.npy`	float32 array $(12800, 2, 128)$; labels hidden from the competitor
`sample_submission.csv`	`id,label` with a dummy label for every test example

A submission is a CSV with one line per test example, `id,label`. The leaderboard metric is **accuracy**, the fraction of correct labels. Because the test set has every SNR equally represented, the overall accuracy is simply the average of the per-SNR accuracies — and since nobody can classify a -10 dB signal reliably, overall accuracies in the 50–65 % range are *normal*. A good leaderboard therefore always shows the **accuracy-versus-SNR curve** as well.

Pitfall: leaderboard thinking

(i) Never tune on the test set: keep a validation split of the training data. (ii) Look at accuracy vs SNR, not only the single number — two models with equal overall accuracy can differ by 10 points at the SNR you care about. (iii) With 800 test examples per SNR, the standard error of an accuracy near 0.9 is $\sqrt{0.9 \times 0.1/800} \approx 1.1$ points; differences smaller than about two points between models are noise, not progress.

32.1.4 First look at the data

Figure 32.2 shows one noise-free example of each class. You can already see the structure the classifiers must find: BPSK and PAM4 live on a line (Q is a scaled copy of I after the random rotation); QAM amplitudes vary more than PSK amplitudes; the FSK classes are smooth sinusoids of constant envelope. Figure 32.3 plots all samples of 20 examples in the complex plane: at 20 dB the shapes differ clearly, at 4 dB the linear classes are all fuzzy blobs.

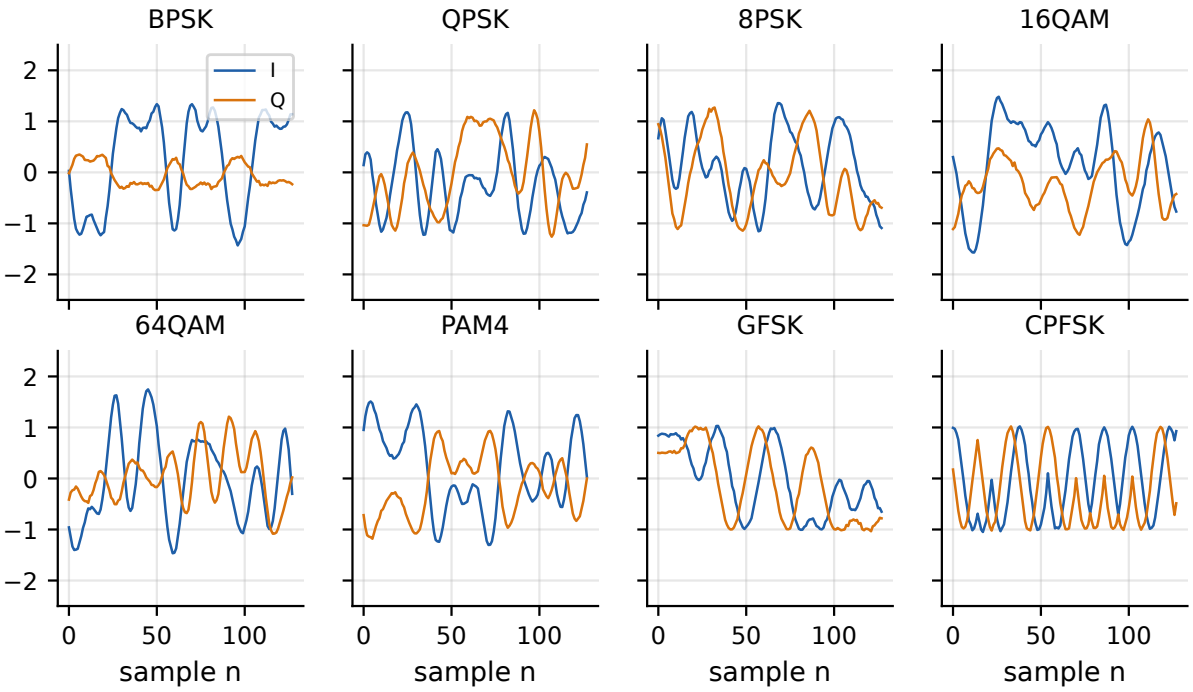


Figure 32.2: One example of each class at 30 dB SNR (I in blue, Q in orange), 128 samples = 16 symbols. Note the constant-envelope FSK classes (bottom right), the "on a line" classes BPSK and PAM4, and that RRC shaping makes the waveform pass smoothly *between* constellation points — most samples are not at symbol instants.

32.2 Two schools: likelihood versus features

Intuition: two ways to recognise a voice

To tell whether a recording is your colleague or a stranger you can either (a) build a precise model of how each person's voice sounds and compute which model explains the recording best — the *likelihood* approach — or (b) measure a few telltale properties (pitch, speed, accent)

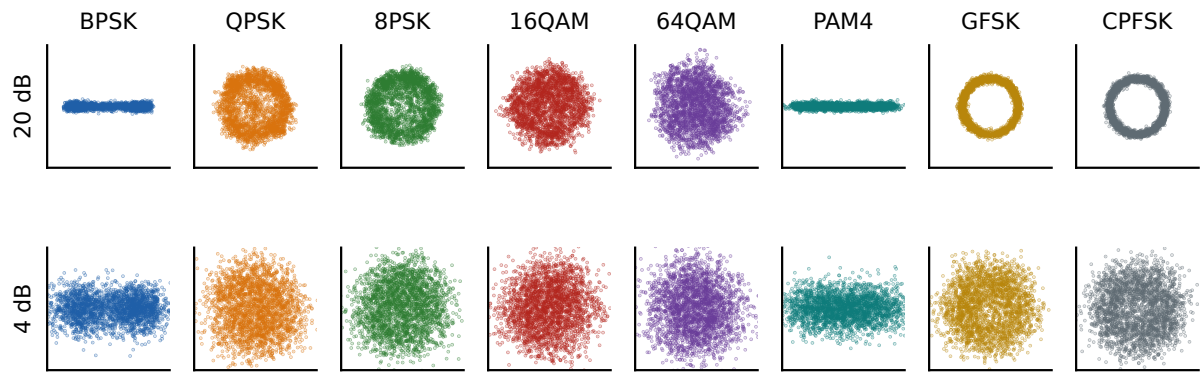


Figure 32.3: All 128 samples of 20 examples per class, at 20 dB (top) and 4 dB (bottom). (For display only, the line-shaped classes BPSK/PAM4 were de-rotated.) Because the samples are taken at 8 per symbol and pass through the RRC pulse, the clouds are *not* the textbook constellations — they are smeared between points. FSK forms a ring. At 4 dB the PSK/QAM clouds look nearly identical to the eye.

and compare them with what you know — the *feature* approach. (a) is optimal if your models are perfect; (b) is cheaper and more forgiving when your models are wrong.

32.2.1 Likelihood-based (LB) classification

Suppose the receiver knew everything except the class and the data symbols: symbol timing, carrier phase, the noise variance, and that it samples once per symbol at the right instants, $r_k = a_k + w_k$. For class i with constellation $\mathcal{A}_i = \{c_1, \dots, c_{M_i}\}$, each sample's density is the average of Gaussian bumps centred on the points:

$$p(r_k \mid i) = \frac{1}{M_i} \sum_{m=1}^{M_i} \frac{1}{\pi\sigma^2} \exp\left(-\frac{|r_k - c_m|^2}{\sigma^2} \right). \tag{32.2}$$

With independent symbols the likelihood of K samples is a product, and the *maximum-likelihood* classifier picks

$$\hat{\imath} = \arg\max_i \sum_{k=1}^{K} \log p(r_k \mid i). \tag{32.3}$$

If the prior over classes is uniform this minimises the probability of error — nothing can beat it. That is why the likelihood approach is the *benchmark* in the theory of AMC.

Worked example: one-sample likelihood, BPSK vs QPSK

Take $r = 0.9 + 0.1j$ and $\sigma^2 = 0.5$ (3 dB). For BPSK the squared distances to ± 1 are 0.02 and 3.62; $e^{-0.02/0.5} = 0.961$, $e^{-3.62/0.5} = 0.0007$; average 0.481, times $1/(\pi \cdot 0.5) = 0.637$: $p(r \mid \text{BPSK}) = 0.306$. For QPSK ($\pm 0.707 \pm 0.707j$) the distances are 0.406, 2.951, 3.234, 0.689, giving exponentials 0.444, 0.003, 0.002, 0.252, average 0.175 and $p(r \mid \text{QPSK}) = 0.112$. This sample favours BPSK by a factor 2.7; the log-likelihood ratio $\log 2.7 \approx 1.0$ is added over all samples.

The catch is in the word "knew". In our data the phase, the frequency offset, the timing and the noise level are unknown, and the samples are taken 8 times per symbol through a pulse. A

proper LB classifier must average (or maximise) the likelihood over all these *nuisance parameters* — an integral with no closed form, expensive to compute and fragile when the model is slightly wrong (a frequency offset or a multipath channel breaks it). Dobre *et al.*'s survey (2007) describes the family of average-, generalised- and hybrid-likelihood tests and their complexity in detail.

32.2.2 Feature-based (FB) classification

The FB approach replaces the full model with a handful of numbers $\mathbf{f}(\mathbf{r}) \in \mathbb{R}^d$ that (1) differ between classes, (2) do not depend on the nuisance parameters (phase, offset, timing), and (3) are robust to noise. A classifier — a decision tree, an SVM, a neural network — is then trained on $(\mathbf{f}, \text{label})$ pairs. FB is sub-optimal in principle, but it is cheap, blind (needs no synchronisation), and degrades gracefully. The rest of this chapter builds the most famous FB features: **higher-order cumulants**.

32.3 Moments and cumulants from scratch

32.3.1 Moments

Intuition: moments as shape descriptors

The mean says where a cloud of points sits, the variance how spread it is. Higher powers emphasise points far from the centre: the 4th moment of a constellation is large if its power is concentrated in a few big points, small if all points have the same radius. Raising a complex number to the p-th power multiplies its *angle* by p: the 4 points of QPSK at $45°, 135°, \dots$ all land on $180°$ after x^4 — so the average of x^4 does not cancel for QPSK, but does cancel for 8PSK. Moments "count symmetries".

Definition: mixed moments of a complex signal

For a zero-mean complex random variable x and integers $p \geq q \geq 0$,

$$M_{pq} = \mathbb{E}\left[x^{p-q}\,(x^*)^q\right], \tag{32.4}$$

where x^* is the complex conjugate. p is the *order*; q says how many factors are conjugated. From N samples the estimate is $\hat{M}_{pq} = \frac{1}{N}\sum_n r[n]^{p-q}\,(r^*[n])^q$.

Examples: $M_{21} = \mathbb{E}[xx^*] = \mathbb{E}\,|x|^2$ is the power; $M_{20} = \mathbb{E}[x^2]$; $M_{42} = \mathbb{E}[x^2(x^*)^2] = \mathbb{E}\,|x|^4$; $M_{40} = \mathbb{E}[x^4]$.

Effect of a phase rotation. If $x \to xe^{j\phi}$ then $x^{p-q}(x^*)^q \to x^{p-q}(x^*)^q e^{j(p-2q)\phi}$. Hence $M_{pq} \to M_{pq}e^{j(p-2q)\phi}$: the *magnitude* of every moment is phase-invariant, and moments with $p = 2q$ (M_{21}, M_{42}, M_{63}) are completely unchanged. That is why we will use $|\cdot|$ of the features.

32.3.2 Why moments are not enough: cumulants

Moments mix information of different orders: $M_{42} = \mathbb{E}\,|x|^4$ of a Gaussian is not zero, it equals $2(\mathbb{E}\,|x|^2)^2$ — it is a consequence of the variance, not new information. *Cumulants* are the combinations of moments that subtract everything already explained by lower orders. Their defining property is *additivity*: for independent x and y, the cumulant of $x + y$ equals the cumulant of x plus the cumulant of y.

Definition: cumulant

For real x, the cumulants κ_k are the Taylor coefficients of the log of the characteristic function: $\log \mathbb{E}[e^{jtx}] = \sum_{k\geq 1} \kappa_k \frac{(jt)^k}{k!}$. For several (possibly complex, conjugated) variables $x_1, \ldots, x_p$ the joint cumulant is given by the *moment-to-cumulant formula*

$$\operatorname{cum}(x_1, \ldots, x_p) = \sum_{\pi} (-1)^{|\pi|-1} (|\pi| - 1)! \prod_{B \in \pi} \mathbb{E}\Big[\prod_{i \in B} x_i\Big], \tag{32.5}$$

where the sum runs over all *partitions* π of $\{1, \ldots, p\}$ into blocks B, and $|\pi|$ is the number of blocks. The complex cumulant C_{pq} is the joint cumulant of $p - q$ copies of x and q copies of x^*.

Why additivity follows. The characteristic function of a sum of independent variables is the product of the characteristic functions; the log of a product is the sum of the logs; so every Taylor coefficient — every cumulant — adds. Moments do not have this property ($\mathbb{E}[(x+y)^4]$ contains cross terms like $6\mathbb{E}[x^2]\mathbb{E}[y^2]$).

32.3.3 Deriving the formulas

For a zero-mean variable all partitions containing a singleton block vanish (they contain $\mathbb{E}[x] = 0$). Applying (32.5):

Order 2. Only the partition $\{\{1,2\}\}$: $C_{20} = M_{20}$, $C_{21} = M_{21}$.

Order 4. The partitions of four elements without singletons are the whole set (coefficient $+1$) and the three pairings $\{12\}\{34\}, \{13\}\{24\}, \{14\}\{23\}$ (coefficient $(-1)^1 1! = -1$). So $\operatorname{cum}(x_1, x_2, x_3, x_4) = \mathbb{E}[x_1x_2x_3x_4] - \mathbb{E}[x_1x_2]\mathbb{E}[x_3x_4] - \mathbb{E}[x_1x_3]\mathbb{E}[x_2x_4] - \mathbb{E}[x_1x_4]\mathbb{E}[x_2x_3]$. Now substitute:

- C_{40}: all four are x; each pair moment is M_{20}: $C_{40} = M_{40} - 3M_{20}^2$.
- C_{41}: (x, x, x, x^*). Pairs among the first three give M_{20}, pairs with x^* give M_{21}; each of the three pairings has one of each: $C_{41} = M_{41} - 3M_{20}M_{21}$.
- C_{42}: (x, x, x^*, x^*). Pairing $\{12\}\{34\}$ gives $\mathbb{E}[x^2]\mathbb{E}[(x^*)^2] = M_{20}M_{20}^* = |M_{20}|^2$; the other two pairings give $\mathbb{E}[xx^*]^2 = M_{21}^2$ each:

Key equation: fourth-order cumulants (zero-mean signal)

$$C_{40} = M_{40} - 3M_{20}^2, \qquad C_{41} = M_{41} - 3M_{20}M_{21}, \qquad C_{42} = M_{42} - |M_{20}|^2 - 2M_{21}^2. \tag{32.6}$$

Order 6. Partitions of six elements without singletons are: the whole set (coefficient 1); a 4-block plus a 2-block ($\binom{6}{2} = 15$ of them, coefficient -1); three pairs (15 of them, coefficient $(-1)^2 2! = 2$). Counting how each type maps onto M's for a given number of conjugates gives, for example, $C_{60} = M_{60} - 15M_{20}M_{40} + 30M_{20}^3$. Doing the same bookkeeping for $q = 1, 2, 3$:

$$\begin{aligned}
C_{60} &= M_{60} - 15M_{20}M_{40} + 30M_{20}^3, \\
C_{61} &= M_{61} - 5M_{21}M_{40} - 10M_{20}M_{41} + 30M_{20}^2M_{21}, \\
C_{62} &= M_{62} - 6M_{20}M_{42} - 8M_{21}M_{41} - M_{22}M_{40} + 6M_{20}^2M_{22} + 24M_{21}^2M_{20}, \\
C_{63} &= M_{63} - 9M_{21}M_{42} + 12M_{21}^3 - 3M_{20}M_{43} - 3M_{22}M_{41} + 18M_{20}M_{21}M_{22}.
\end{aligned} \tag{32.7}$$

(Here $M_{22} = \mathbb{E}[(x^*)^2] = M_{20}^*$ and $M_{43} = \mathbb{E}[x(x^*)^3] = M_{41}^*$.) Do not trust bookkeeping of this size

by eye: our script checks every line of (32.6)–(32.7) against a brute-force implementation of the partition formula (32.5), and they agree to machine precision for all six constellations.

Worked example: cumulants of QPSK by hand

Unit-power QPSK: $x \in \{e^{j\pi/4}, e^{j3\pi/4}, e^{j5\pi/4}, e^{j7\pi/4}\}$, equiprobable. $x^2 \in \{j, -j, j, -j\}$, so $M_{20} = 0$. $x^4 = e^{j\pi} = -1$ for every point, so $M_{40} = -1$. $|x| = 1$, so $M_{21} = 1$ and $M_{42} = 1$. Then $C_{40} = -1 - 3 \cdot 0 = -1$, $C_{42} = 1 - 0 - 2 \cdot 1 = -1$, $C_{41} = M_{41} = \mathbb{E}[x^2 |x|^2] = 0$. For order six: $x^6 = e^{j6(\pi/4 + k\pi/2)} = \mp j$ alternates in sign over the four points and averages to 0, so $C_{60} = 0$; $M_{63} = \mathbb{E}\,|x|^6 = 1$ and $C_{63} = 1 - 9 \cdot 1 \cdot 1 + 12 \cdot 1 = 4$.

32.3.4 Normalisation

A cumulant of order k scales like (amplitude)^k: if $x \to gx$ then $C_{pq} \to g^{p-q}(g^*)^q C_{pq}$. Dividing by $C_{21}^{k/2}$ (power to the $k/2$) removes the unknown gain. Combined with the magnitude (which removes the unknown phase), our features are

$$\tilde{C}_{4q} = \frac{|C_{4q}|}{C_{21}^2}, \qquad \tilde{C}_{6q} = \frac{|C_{6q}|}{C_{21}^3}, \qquad \tilde{C}_{20} = \frac{|C_{20}|}{C_{21}}. \tag{32.8}$$

32.3.5 The theoretical cumulant table

Applying the formulas to each unit-power constellation (equiprobable, independent symbols) gives Table 32.1. We computed every entry numerically from the constellation points with our own code, and the fourth- and sixth-order PSK/QAM values agree with the table of Swami & Sadler (2000) as reproduced in later work by Dobre's group (e.g. $C_{42} = -0.68$ for 16QAM and -0.619 for 64QAM, $C_{63} = 2.08$ and 1.797).

Table 32.1: Theoretical cumulants of unit-power constellations (real-valued for the standard orientation; our features use magnitudes, so the sign is lost but the phase does not matter). $C_{21} = 1$ for all. Computed by `lab_a_explore.py`.

Class	C_{20}	C_{40}	C_{41}	C_{42}	C_{60}	C_{61}	C_{62}	C_{63}
BPSK	1	-2	-2	-2	16	16	16	16
PAM4	1	-1.36	-1.36	-1.36	8.32	8.32	8.32	8.32
QPSK	0	-1	0	-1	0	4	0	4
8PSK	0	0	0	-1	0	0	0	4
16QAM	0	-0.68	0	-0.68	0	2.08	0	2.08
64QAM	0	-0.619	0	-0.619	0	1.797	0	1.797
Gaussian noise	0	0	0	0	0	0	0	0

Read the table as a decision tree — this is precisely the *hierarchical* idea of Swami and Sadler:

- $|C_{20}|$ separates *real* constellations (BPSK, PAM4: $|C_{20}| = 1$) from *circular* ones ($C_{20} = 0$).
- Among real ones, $|C_{42}| = 2$ vs 1.36 separates BPSK from PAM4.
- Among circular ones, $|C_{40}|$ separates 8PSK (0) from QPSK (1) from QAM (≈ 0.6–0.7).
- 16QAM vs 64QAM differ only by 0.68 vs 0.619 in $|C_{42}|$ — a 9% difference that will need very long records to resolve. Remember this; it is the hardest pair of the competition.

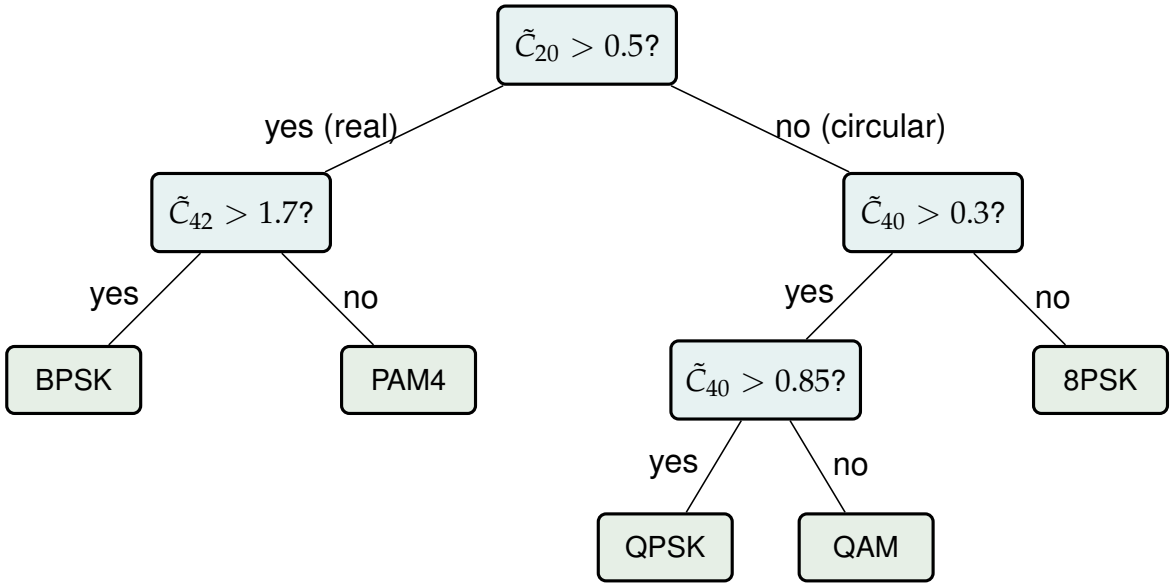


Figure 32.4: The hierarchical idea of Swami and Sadler drawn as a decision tree on the noise-free values of Table 32.1 (thresholds halfway between classes). With noise and $N = 128$ the thresholds must move and the branches blur; a learned tree or forest (Section 32.8) finds them from data. 16QAM vs 64QAM would need one more split on $\tilde{C}_{42}$ (0.68 vs 0.619) that is unreliable with short records.

Pitfall: pulse shaping changes the numbers

Table 32.1 is for *symbols*. Our examples are 8 samples per symbol after RRC shaping, so each sample is a weighted sum $\sum_k a_k p[n - 8k]$ of several symbols. Cumulants are additive and scale with the k-th power of the weights, so at a fixed sampling phase the 4th-order cumulant of the samples is $C_{4q} \sum_k p[n - 8k]^4$. Averaged over the 8 sampling phases this gives $C_{4q} \cdot \kappa_4$ with $\kappa_4 = 8 \sum_n p[n]^4 \approx 0.85$ for our unit-energy RRC pulse. (Pooling all phases adds a small positive term, because the sample power itself ripples with the phase — the signal is *cyclostationary*; pooled, C_{42} gains $+2\,\mathrm{var}(P_m)$, or $+3\,\mathrm{var}(P_m)$ for real constellations, where P_m is the relative power at phase m; since $C_{42} < 0$ this *reduces* $|C_{42}|$, by about 0.05 for QAM and 0.07 for BPSK/PAM4 with our pulse.) Pulse shaping therefore *pulls every constellation towards Gaussian* (by the central-limit theorem, a sum of many symbols looks Gaussian). Our measured long-record values ($|C_{42}| \approx 0.53$ for 16QAM, 0.47 for 64QAM, 1.63 for BPSK, 0.80 for QPSK) match $0.85\times$ the table plus this small correction to two decimals. Classifiers learn the actual values from data, so this does not matter for training — but it matters if you hard-code Swami–Sadler thresholds.

32.4 Why Gaussian noise vanishes above order two

Intuition: noise is the "most boring" distribution

Among all distributions with a given variance, the Gaussian is the one with *no* structure beyond its variance: its log-characteristic function is exactly a quadratic, $\log \mathbb{E}[e^{jtw}] = -\sigma^2 t^2/2$. A quadratic has no Taylor coefficients beyond the second, so every cumulant of order three and higher is *exactly zero*. Constellation points are "lumpy" (non-Gaussian), so their higher-order cumulants are not zero. Since cumulants add for independent signals, adding Gaussian noise leaves the higher-order cumulants of the signal *untouched* — in expectation, cumulants

of order ≥ 3 see straight through the noise.

Formally, for $r = s + w$ with $w \sim \mathcal{CN}(0, \sigma^2)$ independent of s:

$$C_{pq}^{(r)} = C_{pq}^{(s)} + C_{pq}^{(w)}, \qquad C_{pq}^{(w)} = \begin{cases} \sigma^2 & (p,q) = (2,1) \\ 0 & \text{otherwise} \end{cases}. \tag{32.9}$$

For circular complex Gaussian noise even $C_{20}^{(w)} = \mathbb{E}[w^2] = 0$. We checked this numerically on 10^6 noise samples: every normalised cumulant except C_{21} came out below 0.02.

There is one subtlety: we *normalise* by $C_{21}^{(r)} = 1 + \sigma^2$ (unit-power signal plus noise). So the normalised 4th-order feature is

$$\tilde{C}_{42}^{(r)} = \frac{\left|C_{42}^{(s)}\right|}{(1+\sigma^2)^2} = \frac{\left|C_{42}^{(s)}\right|}{(1+1/\mathrm{SNR})^2}. \tag{32.10}$$

At SNR $= 0$ dB the denominator is 4: the feature shrinks by a factor four towards zero, and all classes crowd near the origin. At 10 dB the shrinkage is only $1/1.21$. Check with 16QAM: its noise-free value is 0.53 for long records and 0.61 on average for $N = 128$ (next section). The law predicts 0.13–0.15 at 0 dB and 0.44–0.50 at 10 dB; we measure 0.18 and 0.52. The measurements sit slightly above the prediction because the small-sample bias of the estimator grows when noise inflates the sample moments — the law describes the expectation of the *true* cumulants, not of our $N = 128$ estimates.

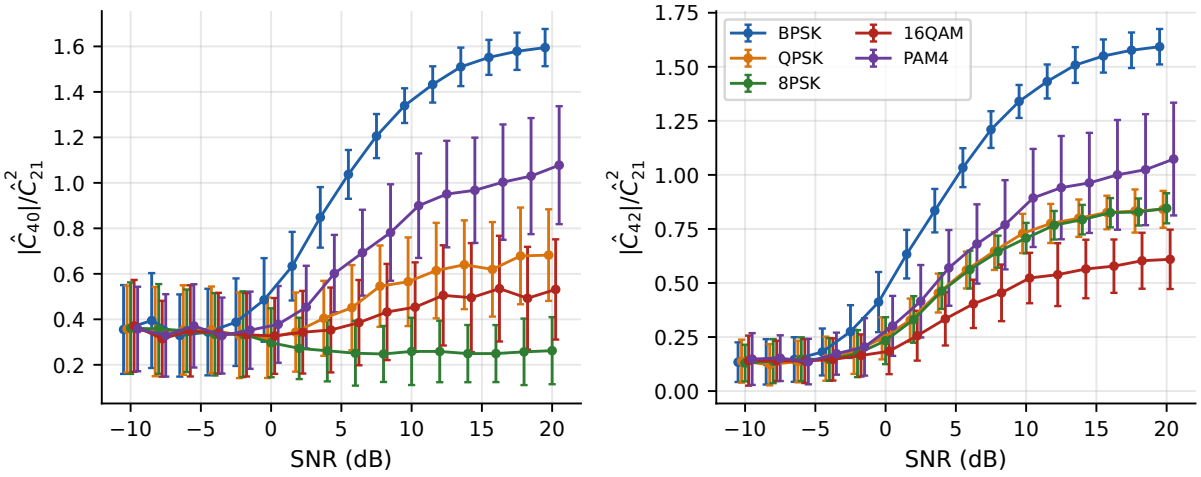


Figure 32.5: Mean $\pm$ one standard deviation of the normalised cumulant estimates over 200 examples ($N = 128$) per point. As SNR falls, the features shrink towards zero by the factor $(1 + 1/\mathrm{SNR})^2$ and the error bars of different classes overlap. Above about 10 dB the classes settle near their (pulse-shaped) theoretical values. Note the large error bars even at 20 dB: that is estimator variance, not noise.

32.5 The enemy at high SNR: estimator variance

The cumulants in Table 32.1 are *expectations*. With only $N = 128$ samples we compute *sample averages*, which fluctuate. A sample mean of N independent terms has a standard deviation proportional to $1/\sqrt{N}$; the samples within one example are correlated (8 per symbol, only 16

independent symbols), which makes it worse. Higher orders are worse still: $|x|^6$ is dominated by the few largest samples.

Figure 32.6 quantifies this without any noise. With $N = 128$ the standard deviation of $\tilde{C}_{42}$ is about 0.13 for 16QAM and 0.15 for 64QAM — larger than the 0.06 gap between their means. The two histograms overlap almost completely: *even with no noise at all*, no threshold on $\tilde{C}_{42}$ can separate 16QAM from 64QAM in 16 symbols. With $N = 1024$ the spread falls to 0.05 and the histograms begin to separate. The $1/\sqrt{N}$ law means that halving the spread costs four times more data.

Notice also that the means at $N = 128$ (0.61 and 0.56) are *higher* than the long-record values (0.53, 0.47): the estimator is biased, because the estimated $\hat{M}_{21}^2$ in the denominator and the product terms in (32.6) are nonlinear functions of noisy averages.

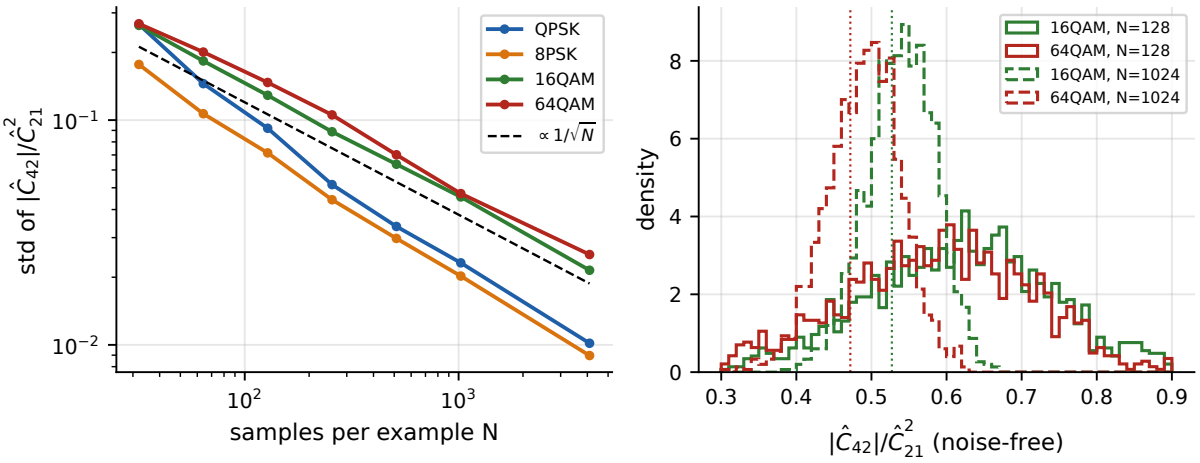


Figure 32.6: Left: standard deviation of $\tilde{C}_{42}$ (noise-free) versus record length N, following the $1/\sqrt{N}$ law (dashed). Right: histograms of $\tilde{C}_{42}$ for 16QAM (green) and 64QAM (red) with $N = 128$ (solid) and $N = 1024$ (dashed); dotted lines are the long-record means. At $N = 128$ the two classes are statistically indistinguishable with this feature.

32.6 Instantaneous and spectral features

Cumulants ignore time order: shuffling the 128 samples does not change them. FSK signals are defined by their *time evolution* (a smoothly changing phase), so we add features computed from the instantaneous amplitude, phase and frequency, following the key features of Azzouz and Nandi (1996). Write $a[n] = |r[n]|$, $\bar{a}$ its mean, and the *normalised-centred amplitude* $a_{cn}[n] = a[n]/\bar{a} - 1$.

Definition: Azzouz–Nandi style key features

- $\gamma_{\max} = \max_k |\mathrm{DFT}(a_{cn})[k]|^2 / N$: peak of the spectrum of the normalised amplitude. Large for signals whose envelope varies (QAM, PAM, BPSK through its zero crossings), near zero for constant-envelope FSK.
- σ_{aa}: standard deviation of $|a_{cn}|$ — amplitude variation again, a robust companion of $\gamma_{\max}$.
- σ_{dp} and σ_{ap}: standard deviation of the *direct* and *absolute* non-linear phase component $\phi_{NL}[n]$ (the unwrapped phase minus its best straight-line fit, which removes the carrier

offset), computed only over "strong" samples with $a[n] > \bar{a}$ because the phase of weak samples is dominated by noise.

- σ_{af}: standard deviation of the absolute normalised-centred instantaneous frequency $\angle(r[n]r^*[n-1])$.

The original work used thresholds on these features inside a hand-built decision tree; we will instead give them to a learned classifier.

Spectral features. The m-th power of a PSK signal contains a pure tone (spectral line) if m is a multiple of the number of phases: x^2 has a line for BPSK/PAM4, x^4 for QPSK and the square QAMs, x^8 for 8PSK. We measure the largest bin of $|\mathrm{DFT}(x^m)|^2$ relative to its total energy for $m = 2, 4, 8$; this is the time-domain view of M_{20}, M_{40} and a cyclic-moment feature in disguise. We also add the spectral flatness, the 90%-power bandwidth, the peak-to-average power ratio (PAPR), the amplitude kurtosis, and the signed real part of C_{42}/C_{21}^2. In total each example becomes a vector of $d = 21$ features (`code/ch32/features.py`).

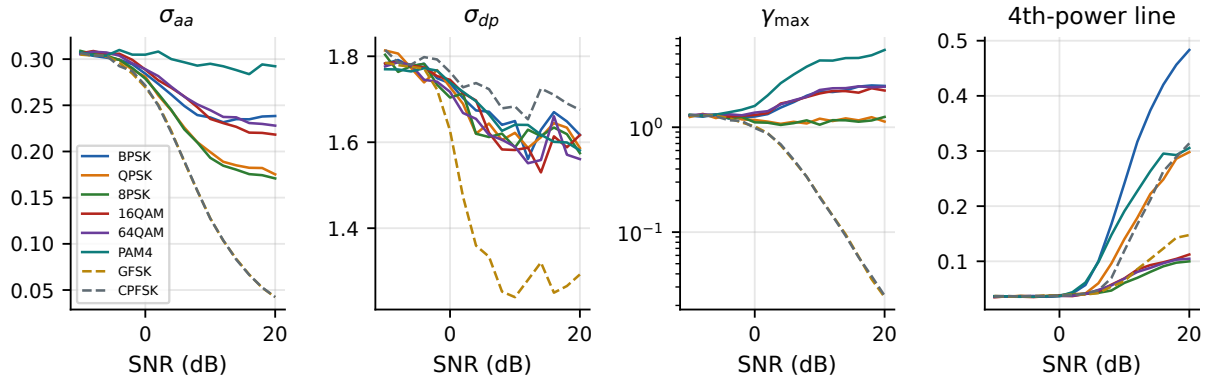


Figure 32.7: Class means of four non-cumulant features versus SNR (120 examples per point; dashed = FSK classes). σ_{aa} and $\gamma_{\max}$ (log scale) collapse for the constant-envelope GFSK/CPFSK as the SNR rises; the 4th-power spectral line grows for QPSK/16QAM/64QAM but not for 8PSK; σ_{dp} singles out CPFSK, whose large phase steps make its non-linear phase very spread.

32.7 Feature extraction code

The heart of `features.py` is short. The moment dictionary is computed once, then each cumulant is one line of (32.6)–(32.7):

```
def cumulants(x):
    """Cumulants C20..C63 from sample moments (formulas derived in Lab A)."""
    M = moments(x)
    M20, M21, M22 = M[2, 0], M[2, 1], M[2, 2]
    M40, M41, M42, M43 = M[4, 0], M[4, 1], M[4, 2], M[4, 3]
    M60, M61, M62, M63 = M[6, 0], M[6, 1], M[6, 2], M[6, 3]
    C = dict(C20=M20, C21=M21)
    C['C40'] = M40 - 3 * M20 ** 2
    C['C41'] = M41 - 3 * M20 * M21
    C['C42'] = M42 - np.abs(M20) ** 2 - 2 * M21 ** 2
    C['C60'] = M60 - 15 * M20 * M40 + 30 * M20 ** 3
    C['C61'] = M61 - 5 * M21 * M40 - 10 * M20 * M41 + 30 * M20 ** 2 * M21
    C['C62'] = M62 - 6 * M20 * M42 - 8 * M21 * M41 - M22 * M40 + 6 * M20 ** 2 *
    M22 + 24 * M21 ** 2 * M20
```

```
    C['C63'] = M63 - 9 * M21 * M42 + 12 * M21 ** 3 - 3 * M20 * M43 - 3 * M22 * M41
     + 18 * M20 * M21 * M22
    return C
```

32.8 Classifiers

We train five scikit-learn classifiers on the 21-dimensional feature vectors. Each was met in earlier chapters; here is the one-line reminder of what each does to the feature space (Figure 32.8).

- **Decision tree** (max depth 12): a sequence of axis-aligned threshold questions, "is $\tilde{C}_{20} > 0.5$?" — exactly the hand-built hierarchical classifiers of the 1990s, but with thresholds learned from data.
- **k-nearest neighbours** ($k = 15$, standardised features): vote among the 15 training examples closest in Euclidean distance. No training, slow prediction.
- **SVM** with an RBF kernel ($C = 10$): maximum-margin boundaries in a kernel-induced feature space; standardisation is essential.
- **Random forest** (200 trees): average of many decorrelated deep trees grown on bootstrap samples; robust and almost tuning-free.
- **Gradient boosting** (histogram GBDT, 300 rounds): trees added one at a time, each fitting the remaining error — usually the strongest tabular-data model, and what O'Shea, Roy and Clancy (2018) used as their higher-order-moment baseline.

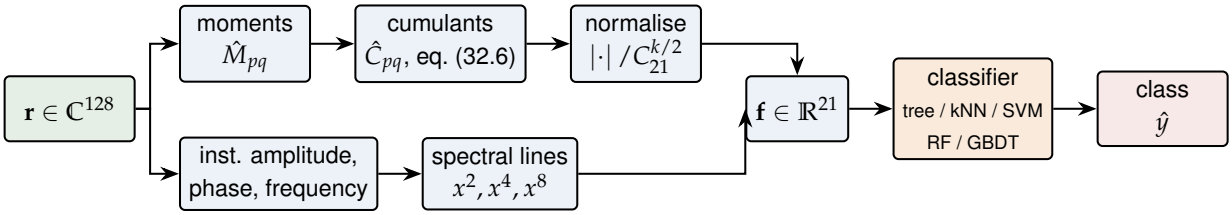


Figure 32.8: The feature-based classifier. Expert knowledge lives in the feature extractor (left); the classifier on the right only has to draw boundaries in a 21-dimensional space. In Lab B the whole left part is replaced by learned convolutional layers.

The training loop is a few lines:

```
models = {
    'Decision tree': DecisionTreeClassifier(max_depth=12, min_samples_leaf=5,
    random_state=0),
    'kNN (k=15)': make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors
    =15)),
    'SVM (RBF)': make_pipeline(StandardScaler(), SVC(C=10, gamma='scale')),
    'Random forest': RandomForestClassifier(n_estimators=200, min_samples_leaf=2,
    n_jobs=1, random_state=0),
    'Gradient boosting': HistGradientBoostingClassifier(max_iter=300,
    learning_rate=0.1, random_state=0),
}
res, preds = {}, {}
for name, m in models.items():
    t = time.time(); m.fit(Ftr, ytr); tfit = time.time() - t
    t = time.time(); p = m.predict(Fte); tpred = (time.time() - t) / len(yte) * 1
    e6
```

```
    preds[name] = p
    res[name] = dict(overall=float(np.mean(p == yte)), by_snr=acc_by_snr(p, yte,
    te['snr']),
                     high=float(np.mean(p[te['snr'] >= 10] == yte[te['snr'] >=
    10])),
                     fit_s=tfit, pred_us=tpred)
    print(f'{name:18s} overall {res[name]["overall"]:.3f}  >=10dB {res[name]["high
    "]:.3f}  ')
```

32.9 Results

32.9.1 Feature space

Figure 32.9 is the most instructive picture of the chapter. At 20 dB the clusters arrange themselves exactly as Table 32.1 predicts, shrunk by the pulse factor: BPSK and PAM4 on the diagonal $|C_{40}| = |C_{42}|$ (real constellations), 8PSK and the FSKs at $|C_{40}| \approx 0$, QPSK at $(0.85, 0.8)$, the QAMs together around $(0.55, 0.55)$. At 10 dB everything has shrunk by $1/1.21$ and the clouds spread. At 0 dB they all collapse into a blob near the origin, as the $(1 + 1/\mathrm{SNR})^{-2}$ law predicts — no classifier can recover what the features no longer contain.

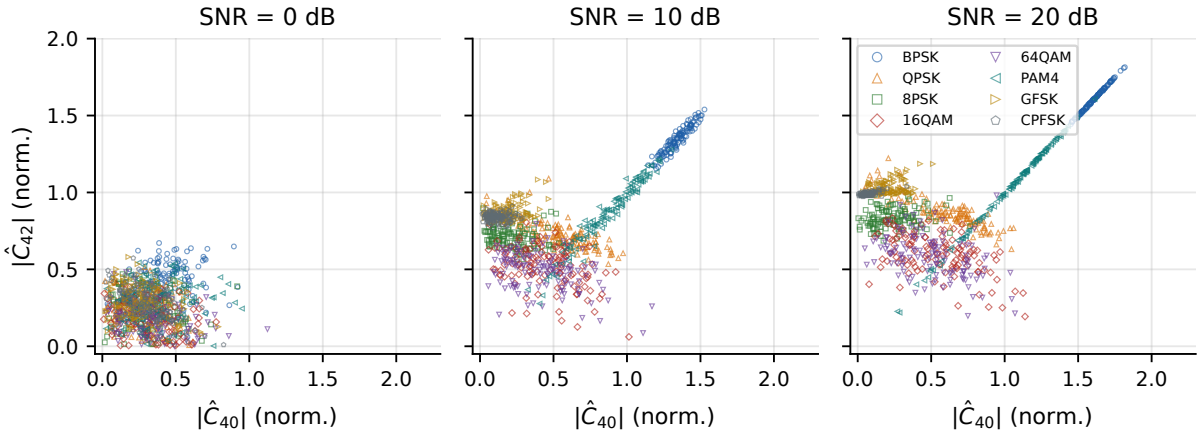


Figure 32.9: Feature scatter $\tilde{C}_{42}$ vs $\tilde{C}_{40}$ (120 examples per class) at 0, 10 and 20 dB. The real constellations (BPSK, PAM4) sit on the diagonal; 8PSK, GFSK and CPFSK on the vertical axis ($C_{40} \approx 0$); QPSK and the two QAMs in between. The 16QAM and 64QAM clouds overlap completely even at 20 dB.

32.9.2 Accuracy versus SNR

Table 32.2 and Figure 32.10 give the leaderboard on the 12 800 test examples.

Four observations:

- **All five classifiers land within five points of each other** (0.536 to 0.588). Once the features are fixed, the choice of classifier matters much less than the features themselves. The SVM wins narrowly, then random forest and gradient boosting; the single tree and kNN trail by about 4–5 points.
- **The time-domain and spectral features matter**: gradient boosting on the 8 cumulant magnitudes alone scores 0.475 overall and 0.752 at high SNR — about 9 and 11 points below

Table 32.2: Lab A leaderboard (AWGN test set, 8 classes, -10 to $+20$ dB). "High SNR" averages the 6 SNRs from 10 to 20 dB. Prediction time is per example on one (shared) CPU thread and excludes feature extraction.

Classifier	overall acc.	acc. (≥ 10 dB)	predict (μs/ex.)
Decision tree (depth 12)	0.536	0.830	2
kNN ($k = 15$)	0.544	0.829	1114
SVM (RBF)	**0.588**	**0.878**	6422
Random forest (200 trees)	0.573	0.863	352
Gradient boosting (300 rounds)	0.567	0.860	1715
Gradient boosting, 8 cumulants only	0.475	0.752	—

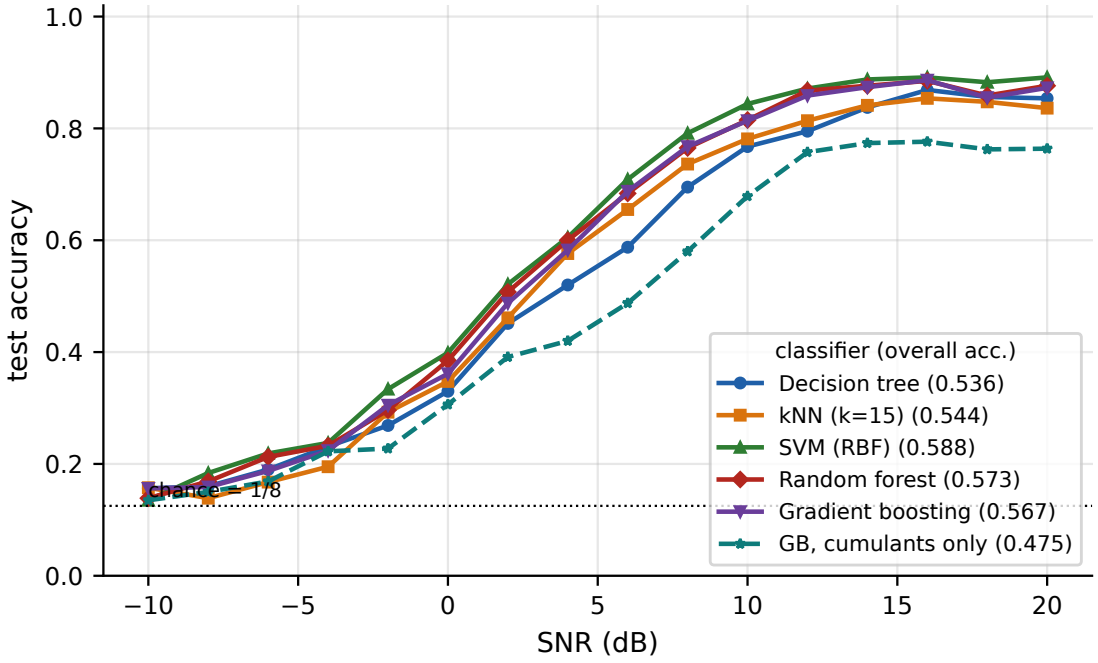


Figure 32.10: Test accuracy versus SNR for the five classifiers on all 21 features, and gradient boosting on the 8 cumulant features only (dashed). All curves rise from chance (1/8) at -10 dB to a plateau above 10 dB. The plateau is below 100% because of the 16QAM/64QAM pair (and some PSK confusion), which is limited by the $N = 128$ estimator variance, not by noise.

the same model on all 21 features. The cumulants cannot see the constant envelope of FSK, nor easily separate QPSK from 8PSK with 16 symbols.

- **The high-SNR plateau is about** 0.88, not 1.0. The curve stops improving above 12 dB: noise is no longer the limit; estimator variance is (Section 32.5).
- **Below 0 dB the curves approach chance**, 0.33 for the SVM at -2 dB and 0.24 at -4 dB. The features have shrunk into the noise-dominated blob of Figure 32.9.

The SVM's high accuracy comes at a price in prediction time (6.4 ms per example on our shared CPU, against 0.35 ms for the forest and 2 μs for the tree), because it compares each test vector against thousands of support vectors.

32.9.3 What is confused with what

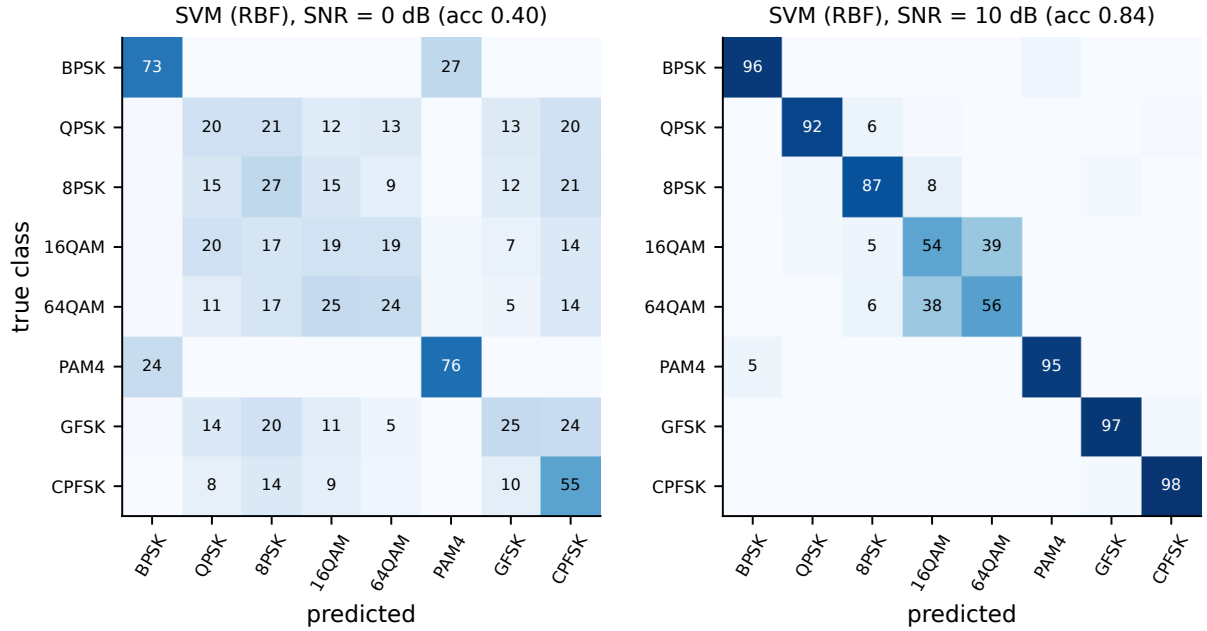


Figure 32.11: Confusion matrices (row-normalised, %) of the best feature-based classifier at 0 dB and 10 dB. Rows are true classes, columns predictions. At 10 dB almost all the error is in the 16QAM/64QAM block; at 0 dB the circular linear classes (QPSK, 8PSK, QAMs) are mixed among themselves while the real (BPSK/PAM4) and constant-envelope (GFSK/CPFSK) groups remain recognisable.

At 10 dB (Figure 32.11, right) BPSK, QPSK, PAM4, GFSK and CPFSK are at 92–98%, 8PSK at 87%, and the 16QAM/64QAM block is nearly a coin flip (54/39 and 38/56). That single 2×2 block costs about 10 points of accuracy at every high SNR. At 0 dB (left) the matrix shows the structure of Table 32.1 melting: BPSK and PAM4 still recognise each other as "real" (73% and 76% correct, the rest mostly swapped), CPFSK is still 55% correct, but QPSK, 8PSK and the QAMs are spread almost uniformly over the circular classes.

32.9.4 Which features matter

The random forest ranks $\tilde{C}_{20}$ first (it splits off the real constellations), followed by $\gamma_{\max}$, σ_{af} and σ_{aa} (constant envelope vs not, and frequency behaviour — the FSK detectors) and the 4th-power spectral line (QPSK and square QAM vs 8PSK). The fourth- and sixth-order cumulants follow

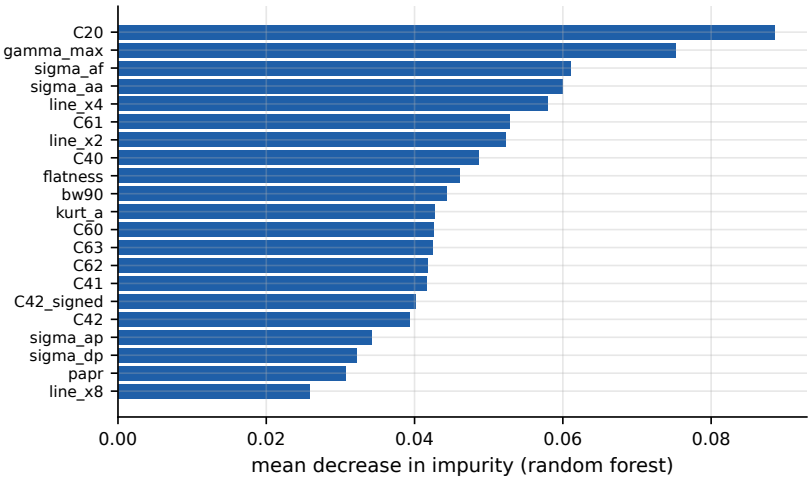


Figure 32.12: Random-forest impurity importance of the 21 features (higher = the feature was used more, at more useful splits). Impurity importance is biased towards continuous, noisy features and splits credit among correlated ones, so read it as a rough ranking.

as a group; their information overlaps heavily, so the forest spreads credit among them. The 8th-power line and PAPR are least used.

32.9.5 The sample-size effect

To isolate the record length we regenerate smaller train/test sets with $N = 128$, 512 and 1024 samples per example and train the same gradient-boosting classifier (Figure 32.13).

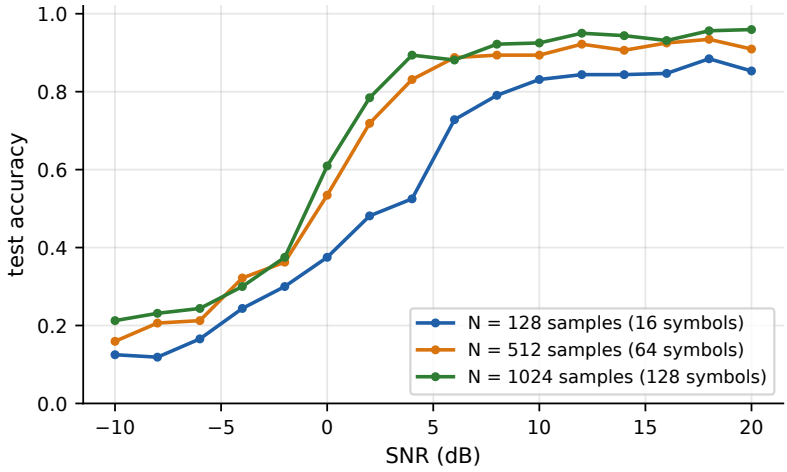


Figure 32.13: Gradient boosting on the 21 features with record lengths $N = 128$, 512 and 1024 (16, 64, 128 symbols), trained on 60 examples per (class, SNR). Longer records shift the whole curve left (lower SNR needed) and raise the plateau, because every feature estimate has a smaller variance.

The effect is large: overall accuracy rises from 0.560 ($N = 128$) to 0.664 ($N = 512$) and 0.695 ($N = 1024$). At 20 dB the accuracy goes from 0.85 to 0.91 and 0.96 — the 16QAM/64QAM wall of Figure 32.6 begins to crumble. The SNR at which the classifier first reaches 0.8 moves from about 9 dB ($N = 128$) to about 3 dB ($N = 1024$): eight times more samples are worth roughly 6 dB of SNR here.

Wireless connection: what this means for a real scanner

Record length is the cheapest "SNR" you can buy. A spectrum monitor that can integrate 1 ms of a 1 Msym/s signal has 1000 symbols, not 16, and cumulant features become very reliable — as long as the channel stays constant over that time. In Lab C we will see that the channel, not the noise, then becomes the limit.

32.10 Kaggle notes for Lab A

- **Feature engineering beats model tuning.** Switching classifiers moved the score by a few points; adding the time-domain features to the cumulants moved it more. Ask "what physical property differs between the confused classes?" and build a feature for it.
- **Normalise for invariances.** Our features use $|\cdot|$ and C_{21} normalisation, so they are invariant to phase and gain by construction. A model never has to learn an invariance that you built in.
- **Use the SNR column carefully.** The training labels include the SNR; the test set does not. You may *estimate* SNR from the data (e.g. from $\hat{M}_{21}$ and $\hat{M}_{42}$) and use it as a feature, but never use the true test SNR.
- **Know your ceiling.** With 16 symbols, 16QAM vs 64QAM is close to a coin flip for any method; Lab B will show that the CNNs hit the same wall.

Chapter recap

- The competition: 128 samples (16 symbols), 8 classes, SNR -10 to 20 dB, metric = accuracy overall and vs SNR.
- Likelihood-based classification is optimal when the model is perfect but needs all nuisance parameters; feature-based classification is blind, cheap and robust.
- Moments $M_{pq} = \mathbb{E}[x^{p-q}(x^*)^q]$; cumulants subtract lower-order structure and are *additive* for independent signals. $C_{40} = M_{40} - 3M_{20}^2$, $C_{42} = M_{42} - |M_{20}|^2 - 2M_{21}^2$; normalise as $|C_{pq}| / C_{21}^{p/2}$.
- Gaussian noise has zero cumulants above order 2, so in expectation higher-order cumulants ignore noise; but normalisation shrinks them by $(1 + 1/\mathrm{SNR})^2$ and the estimator variance ($\propto 1/\sqrt{N}$) dominates at high SNR.
- Pulse shaping multiplies 4th-order cumulants by $\kappa_4 \approx 0.85$ (RRC, $\beta = 0.35$).
- Instantaneous amplitude/phase/frequency features (Azzouz–Nandi) separate the constant-envelope FSK classes; spectral lines of x^m count phase symmetries.
- Best feature-based result: RBF-SVM on 21 features, 0.588 overall and 0.878 at SNR ≥ 10 dB — the baseline for Labs B–D.

Exercises

1. Compute C_{20}, C_{40}, C_{42} for unit-power BPSK by hand.
2. Show that for 8PSK $M_{40} = 0$ and hence $C_{40} = 0$. Which power x^m first gives a non-zero average for 8PSK?

3. Compute C_{42} for unit-power PAM4 (points $\pm 1/\sqrt{5}, \pm 3/\sqrt{5}$) and verify the table value -1.36.
4. A 16QAM signal is received at SNR $= 5\,\text{dB}$. Ignoring estimation error and pulse shaping, what is the expected normalised $\tilde{C}_{42}$? At which SNR would it equal the noise-free 64QAM value 0.619?
5. Prove that $M_{pq} \to M_{pq} e^{j(p-2q)\phi}$ under a phase rotation, and explain why $|C_{40}|$ but not $\operatorname{Re} C_{40}$ is a good feature here.
6. The standard deviation of $\tilde{C}_{42}$ for 16QAM is 0.13 at $N = 128$. Estimate N needed to bring it down to 0.02, and how many symbols that is at 8 samples per symbol.
7. (Conceptual) Why are the instantaneous features computed only on samples with $a[n] > \bar{a}$? What would happen to σ_{dp} at low SNR otherwise?
8. (Coding) Add the feature $\left|\hat{C}_{80}\right|$ (8th order) for 8PSK detection. Does the score improve? Explain using estimator variance.

Answers

1. $x = \pm 1$: $M_{20} = M_{21} = 1$, $M_{40} = M_{42} = 1$. $C_{20} = 1$; $C_{40} = 1 - 3 = -2$; $C_{42} = 1 - 1 - 2 = -2$.
2. $x^4 = e^{j\pi k}$ for $x = e^{j\pi k/4}$, $k = 0..7$: values alternate $+1, -1$, average 0. M_{20}: $x^2 = e^{j\pi k/2}$ cycles through four values, average 0. So $C_{40} = 0 - 0 = 0$. The first non-zero moment of type $\mathbb{E}[x^m]$ is $m = 8$ ($x^8 = 1$ for every point).
3. The points are real, so $x^* = x$ and $M_{20} = M_{21} = (1 + 9 + 1 + 9)/(4 \cdot 5) = 1$, $M_{42} = \mathbb{E}[x^4] = (1 + 81 + 1 + 81)/(4 \cdot 25) = 1.64$. $C_{42} = 1.64 - 1 - 2 = -1.36$. ✓
4. $\text{SNR} = 10^{0.5} = 3.162$; $(1 + 1/3.162)^2 = 1.733$; $\tilde{C}_{42} = 0.68/1.733 = 0.392$. Equal to 0.619 when $(1 + 1/\text{SNR})^2 = 0.68/0.619 = 1.0985$, i.e. $1/\text{SNR} = 0.0481$, $\text{SNR} = 20.8 \approx 13.2\,\text{dB}$. So a 16QAM signal at 13 dB has the same expected $\tilde{C}_{42}$ as noise-free 64QAM — a reason to give the classifier an SNR-sensitive feature as well.
5. $(xe^{j\phi})^{p-q}((xe^{j\phi})^*)^q = x^{p-q}(x^*)^q e^{j(p-q)\phi} e^{-jq\phi}$; take expectations. C_{40} involves only M_{40} and M_{20}^2, both of which rotate by $e^{j4\phi}$, so $C_{40} \to C_{40} e^{j4\phi}$: its magnitude is invariant but its real part depends on the unknown carrier phase.
6. Standard deviation $\propto 1/\sqrt{N}$: $N = 128 \times (0.13/0.02)^2 = 128 \times 42.25 \approx 5400$ samples, i.e. about 680 symbols.
7. The phase of a weak sample is dominated by noise (it is the angle of a small vector plus noise) and is nearly uniform on $(-\pi, \pi]$, which inflates σ_{dp} for every class towards $\pi/\sqrt{3} \approx 1.81$ and destroys its discriminating power. Restricting to strong samples keeps the phase estimates that carry information.
8. Typically the gain is small or zero. The 8th-order estimate is dominated by the largest few samples and its variance at $N = 128$ is huge; furthermore the 8th-power spectral line (`line_x8`) already carries much of the same information. Higher-order statistics need longer records.

Self-test quiz

1. Which cumulant separates real constellations (BPSK, PAM4) from circular ones?
   $|C_{20}|$: *it is 1 for real constellations and 0 for circular ones (QPSK, 8PSK, QAM, FSK).*
2. Why do cumulants of order ≥ 3 of Gaussian noise vanish?
   The log-characteristic function of a Gaussian is exactly quadratic, so all its Taylor coefficients beyond order 2 are zero.
3. What limits 16QAM vs 64QAM accuracy at 20 dB with 128 samples?
   Estimator variance: the spread of $\tilde{C}_{42}$ *(about* 0.13*) exceeds the* 0.06 *gap between the class means.*
4. By what factor does the normalised $\tilde{C}_{42}$ shrink at 0 dB SNR?
   $(1 + 1/\mathrm{SNR})^2 = 4$.
5. Name two features that detect constant-envelope signals.
   $\gamma_{\max}$ *and* σ_{aa} *(also PAPR, amplitude kurtosis).*

Further reading

- A. Swami and B. M. Sadler, "Hierarchical digital modulation classification using cumulants," *IEEE Trans. Communications*, vol. 48, no. 3, pp. 416–429, 2000. The source of the cumulant table and the hierarchical decision idea.
- E. E. Azzouz and A. K. Nandi, *Automatic Modulation Recognition of Communication Signals*, Kluwer Academic Publishers, 1996. Key features $\gamma_{\max}$, σ_{ap}, σ_{dp}, σ_{aa}, σ_{af} and decision-tree/ANN recognisers.
- O. A. Dobre, A. Abdi, Y. Bar-Ness and W. Su, "Survey of automatic modulation classification techniques: classical approaches and new trends," *IET Communications*, vol. 1, no. 2, pp. 137–156, 2007. Likelihood- versus feature-based methods in one place.
- T. J. O'Shea, J. Corgan and T. C. Clancy, "Convolutional radio modulation recognition networks," in *Proc. Engineering Applications of Neural Networks (EANN)*, 2016 (arXiv:1602.04105). The CNN of Lab B.
- T. J. O'Shea, T. Roy and T. C. Clancy, "Over-the-air deep learning based radio signal classification," *IEEE J. Selected Topics in Signal Processing*, vol. 12, no. 1, 2018 (arXiv:1712.04578). Compares deep networks with higher-order-moment features and boosted trees, including multipath.

Chapter 33

Lab B: Deep Learning-Based Modulation Classification in AWGN

Same competition, same files, same metric as Lab A (Chapter 32) — but now you may not hand-craft a single feature. Instead you give the raw 128 complex samples to a neural network and let gradient descent discover what matters. This is the idea that O'Shea, Corgan and Clancy made popular in 2016: a small convolutional network trained on raw I/Q samples matched or beat classifiers built on expert features, most clearly at low SNR.

In this lab you will choose an input representation (I/Q, amplitude/phase, spectrum, constellation image) and normalise it; build three architectures — a VT-CNN2-style baseline, a small 1-D ResNet and a CLDNN (convolution + LSTM) — and train them with cross-entropy, Adam and early stopping; compare them, SNR by SNR, with the best feature-based classifier of Lab A; open the black box with confusion matrices, PCA/t-SNE of the learned embeddings and the first-layer filters; and run ablations on input representation, training-set size and sequence length. We finish with leaderboard tricks: ensembling, stacking and test-time augmentation.

Wireless connection: why learn the features?

Expert features encode what a human understood about a few idealised signals. Real receivers see filtered, distorted, interfering signals whose "ideal" statistics are unknown. A network trained on examples of what the receiver *actually* sees can adapt its features to those impairments. The price: it needs representative training data, and it is only as good as that data — a lesson that Labs C and D will drive home.

33.1 The competition, recapped

Data: $\mathbf{r} \in \mathbb{C}^{128}$ (8 samples/symbol, 16 symbols), 8 classes, SNR $\in \{-10, \ldots, 20\}$ dB, AWGN plus random phase, small frequency offset and timing offset. Training set 19 200 examples, test set 12 800, both balanced over (class, SNR). We hold out 15% of the training set (2 880 examples) for validation and early stopping; the test set is touched once per model. The score to beat is Lab A's best: the RBF-SVM on 21 features, 0.588 overall and 0.878 at SNR ≥ 10 dB.

33.2 Input representations

Intuition: the same signal, four ways

A recording of a voice can be shown as a waveform, as a spectrogram, or as sheet music. All contain (almost) the same information, but some make certain patterns easier to see. For a network with limited depth and data, the right "view" of the signal is a cheap form of prior knowledge.

Definition: four common representations of $\mathbf{r}$

- **I/Q** ($2 \times N$): row $0 = \operatorname{Re} r[n]$, row $1 = \operatorname{Im} r[n]$. Lossless; linear in the signal, so a linear filter in the network is a linear filter on the signal. A phase rotation *mixes* the two rows.
- **Amplitude/phase** ($2 \times N$): $|r[n]|$ and $\angle r[n]/\pi \in (-1, 1]$. A carrier phase becomes a constant offset of the phase row (plus wrapping); constant-envelope signals have a flat amplitude row. Nonlinear: at low SNR the phase of weak samples is pure noise.
- **Spectrum** ($1 \times N$): $\log |\mathrm{DFT}(\mathbf{r})|^2$, standardised. Invariant to phase and timing, but discards the phase of the spectrum and with it most information about the symbol sequence.
- **Constellation image**: a 2-D histogram of the samples in the complex plane, treated as a picture for an image CNN. Discards time order; with only 128 samples per example the image is very sparse.

Normalisation. Neural networks train best when inputs have a predictable scale. Our generator already scales the *signal* to unit power before adding noise, so the total power is $1 + \sigma^2$, between 1.01 and 11. Two choices are possible: (i) leave it, so the network can "see" the SNR in the power; (ii) normalise every example to unit total power, which removes this cue but makes the model robust to an unknown receiver gain. In a real receiver the gain is unknown, so (ii) is the honest choice; in this competition the train and test sets have the same convention, so either works. We use the data as given (signal power 1); the exercise at the end asks you to compare.

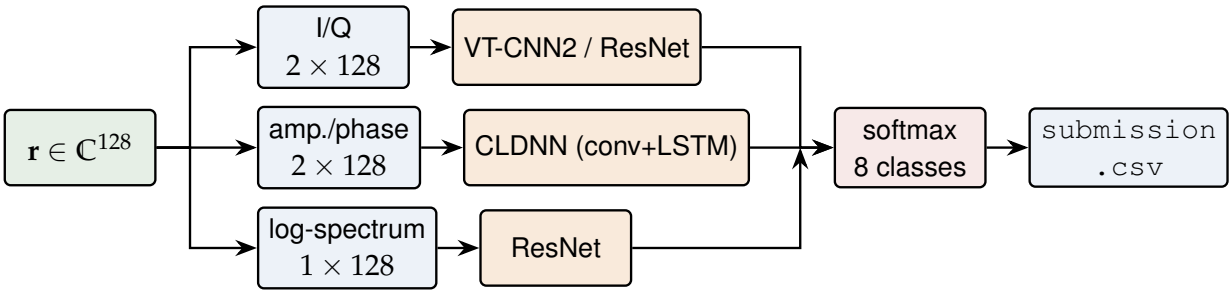


Figure 33.1: The deep-learning pipeline of Lab B. The expert feature extractor of Lab A (Figure 32.8) is gone; the representation box is the only hand-made part left.

33.3 Three architectures

33.3.1 VT-CNN2-style baseline

O'Shea, Corgan and Clancy (2016) treated the 2×128 I/Q array as a tiny two-row image. Their larger "CNN2" had a first convolution with 256 filters of size 1×3 (each row filtered separately), a second with 80 filters of size 2×3 (combining I and Q), a dense layer of 256 units and a softmax output, with ReLU activations and dropout; it was trained with Adam on 11 classes from -20 to $+20$ dB. This architecture became known as "VT-CNN2" in later work. We keep its *shape* but scale it down to train on a shared CPU in minutes: 32 filters $1 \times 3 \to$ 16 filters $2 \times 3 \to$ dense $64 \to 8$, dropout 0.4, 134 872 parameters (Figure 33.2). Almost all parameters are in the first dense layer ($16 \times 128 \times 64$ weights), because the convolutions keep the full length 128 and nothing pools.

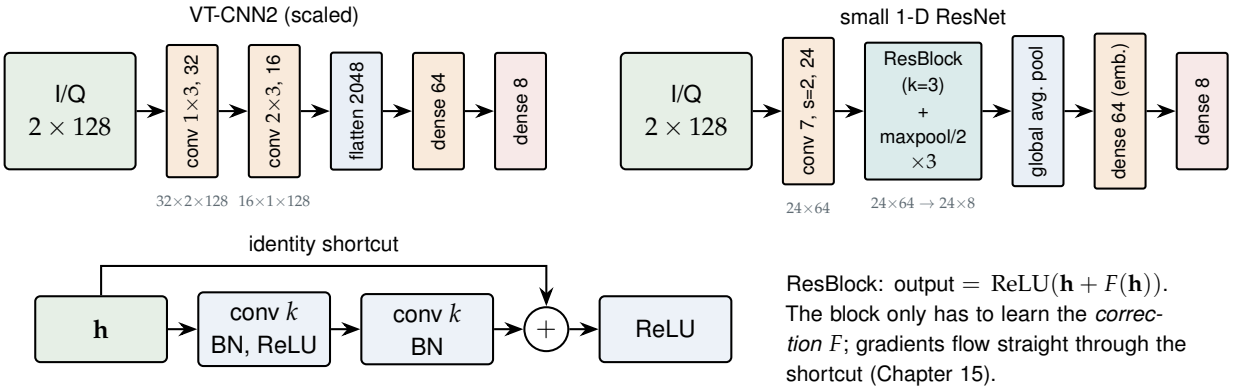


Figure 33.2: Top: the two convolutional models (tensor sizes in gray, channels × length). VT-CNN2 keeps the full length and puts its capacity in a large dense layer; the ResNet down-samples by 16 in total and uses global average pooling, so it has ten times fewer parameters. Bottom: one residual block.

33.3.2 A small 1-D ResNet

Convolutions treat I and Q as two input channels of a 1-D signal — the natural view for an engineer: each first-layer filter is a *complex FIR filter* $h[k] = w_I[k] + jw_Q[k]$ applied to r (more precisely, its output is the real part of a complex correlation). A strided stem convolution (kernel 7, stride 2, 24 channels) is followed by three residual blocks with kernel 3, each followed by max-pooling by 2; global average pooling makes the result independent of where in the window a pattern occurred (useful, because the timing offset is random); a 64-unit dense "embedding" and the 8-way output follow. Total: 13 328 parameters.

Receptive field. Each output of the last block depends on a window of input samples. Stem: 7 samples. Each later conv of kernel 3 at down-sampling factor D adds $2D$ samples; each block has two convs. With $D = 2, 4, 8$ in the three blocks the receptive field is $7 + 2 \cdot 2 \cdot (2 + 4 + 8) = 63$ samples ≈ 8 symbols — half the window. Global pooling then combines all positions.

33.3.3 CLDNN on amplitude/phase

West and O'Shea (2017) studied convolutional–recurrent combinations ("CLDNN": Convolutional, LSTM, Deep Neural Network) and residual variants for this task; amplitude/phase inputs are a common choice for the recurrent part because the LSTM can follow the phase trajectory. Ours: two 1-D convolutions (24 channels, kernels 7 and 5) → max-pool by 4 → one LSTM layer with 48 hidden units over the 32 remaining time steps → dense 64 → 8; 21 128 parameters. The LSTM reads the sequence left to right and its final hidden state summarises it (Chapter 16).

33.4 Training

All models use the same recipe (`code/ch33/dlcommon.py`):

- **Loss**: cross-entropy $L = -\frac{1}{B}\sum_b \log \mathrm{softmax}(\mathbf{z}_b)_{y_b}$ over mini-batches of $B = 256$.
- **Optimiser**: Adam, learning rate 2×10^{-3}, halved when the validation accuracy stops improving for 2 epochs.
- **Early stopping**: at most 15 epochs; stop after 4 epochs without validation improvement and restore the best weights.

- **SNR-balanced batches**: the training set has equal numbers per (class, SNR) and is shuffled every epoch, so every mini-batch is, in expectation, balanced over SNR. This matters: if a model sees mostly high-SNR examples it never learns to handle noise; if mostly low-SNR, the gradient is dominated by hopeless examples. (Curriculum schemes that start at high SNR are an exercise.)
- `torch.set_num_threads(1)`, fixed seeds.

```
def train(model, Xtr, ytr, Xva, yva, epochs=20, bs=256, lr=2e-3, patience=4, seed=0, verbose=True,
          augment=None, max_time=None):
    """Adam + cross-entropy, early stopping on validation accuracy (best weights restored).
    augment: optional function (np batch) -> np batch applied to every training batch."""
    torch.manual_seed(seed); rng = np.random.default_rng(seed)
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    sched = torch.optim.lr_scheduler.ReduceLROnPlateau(opt, factor=0.5, patience=2)
    Xva_t, yva_t = torch.from_numpy(Xva), torch.from_numpy(yva).long()
    best, best_state, bad, hist = -1, None, 0, []
    t0 = time.time()
    for ep in range(epochs):
        model.train(); perm = rng.permutation(len(ytr)); tl = 0
        for i in range(0, len(perm), bs):
            idx = perm[i:i + bs]
            xb = Xtr[idx]
            if augment is not None:
                xb = augment(xb, rng)
            xb = torch.from_numpy(xb); yb = torch.from_numpy(ytr[idx]).long()
            loss = F.cross_entropy(model(xb), yb)
            opt.zero_grad(); loss.backward(); opt.step(); tl += loss.item() * len(idx)
        va = accuracy(model, Xva_t, yva_t)
        sched.step(-va)
        hist.append((tl / len(ytr), va))
        if verbose:
            print(f'  ep {ep:2d} loss {tl / len(ytr):.3f} val {va:.3f} ({time.time() - t0:.0f}s)', flush=True)
        if va > best:
            best, bad = va, 0
            best_state = {k: v.clone() for k, v in model.state_dict().items()}
        else:
            bad += 1
            if bad >= patience:
                break
        if max_time and time.time() - t0 > max_time:
            break
    model.load_state_dict(best_state)
    return hist
```

Pitfall: low-SNR examples are label noise

At -10 dB even the best model is at chance level: those examples are effectively mislabelled noise. They still contribute a quarter of the gradient. Our validation accuracy therefore plateaus around 0.6, and a training loss that keeps falling while validation stalls is the

network memorising noise — early stopping catches this.

33.5 Results

Table 33.1: Lab B leaderboard on the AWGN test set (12 800 examples). Training time on one shared CPU thread (the machine was heavily loaded; on an idle core expect roughly a third of these numbers). Inference is per example in batches of 1024.

Model	params	overall	≥ 10 dB	train (s)	infer (μs)
VT-CNN2 (I/Q, scaled)	134 872	0.505	0.677	664	872
ResNet (I/Q)	13 328	**0.640**	0.806	507	429
CLDNN (amp./phase)	21 128	0.491	0.746	404	544
Ensemble of the three	—	0.631	0.788	—	—
ResNet + TTA (4 rotations)	—	0.657	0.816	—	—
Stack: ResNet + 21 features	—	**0.678**	**0.875**	—	—
Lab A best: SVM on features	—	0.588	0.878	—	6422

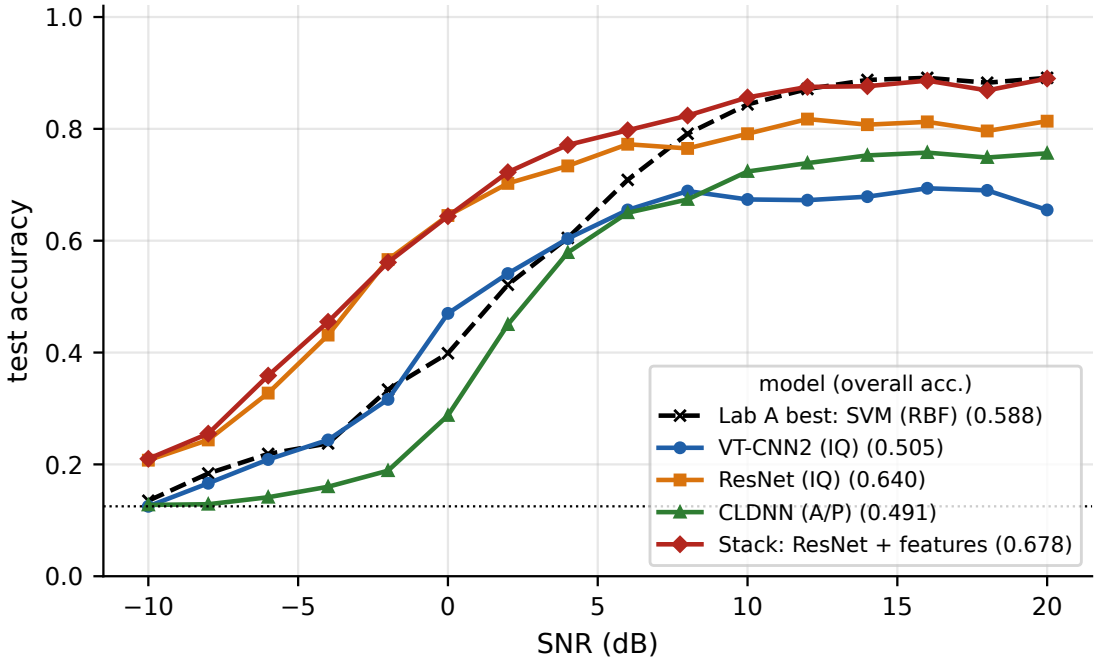


Figure 33.3: Accuracy vs SNR of the three networks, a stacked model (ResNet probabilities + Lab A features), and the best feature-based classifier of Lab A (black dashed). The ResNet dominates below about 4 dB; the feature-based SVM is better on the high-SNR plateau. Stacking combines both.

Read Figure 33.3 from left to right.

- **Low SNR (≤ 0 dB): the ResNet wins clearly.** At -4 dB it scores 0.43 against the SVM's 0.24; at -2 dB, 0.57 against 0.33. The features of Lab A are sample statistics that the noise shrinks towards zero (the $(1 + 1/\mathrm{SNR})^2$ law); the network can instead exploit the whole waveform, effectively learning matched-filter-like averaging. This is the qualitative effect O'Shea *et al.* reported: learned features help most at low SNR.

- **High SNR ($\geq$ 10 dB): the features win.** The SVM's plateau is 0.88, the ResNet's 0.81. Our networks were trained for minutes on 16 000 examples and have to *learn* the phase invariance and the fine amplitude statistics that Lab A builds in by construction.
- **VT-CNN2 is the weakest** (0.68 plateau). It has ten times more parameters than the ResNet, almost all in one dense layer that must learn position-specific patterns — poorly matched to a signal with random timing. The CLDNN on amplitude/phase is useless below 0 dB (the phase of noisy samples is random) but reaches 0.75 at high SNR.

Overall accuracy, which weights all SNRs equally, makes the ResNet the single-model winner (0.640 vs 0.588). The best entry combines both worlds (Section 33.7): 0.678 overall and 0.875 at high SNR.

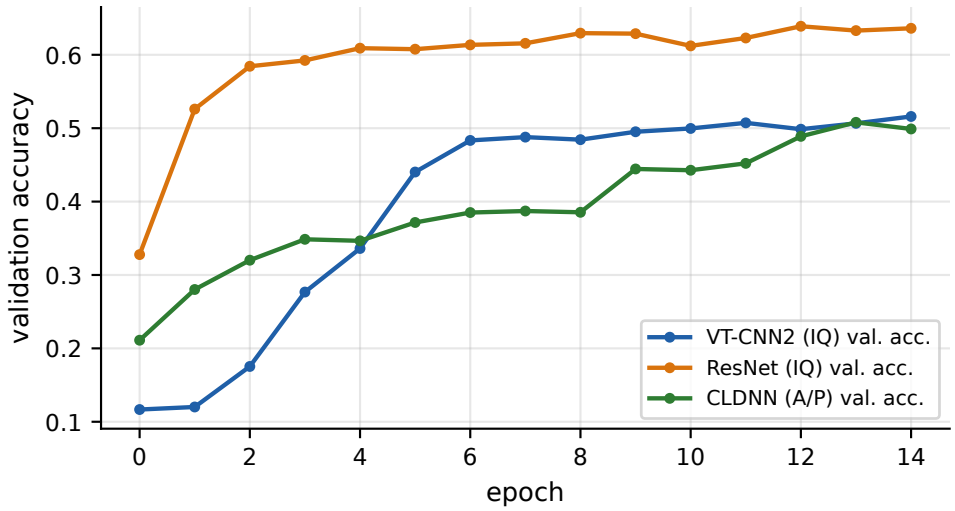


Figure 33.4: Validation accuracy per epoch. The ResNet learns fastest and highest; VT-CNN2 needs several epochs before it "discovers" useful filters (the flat start), a typical sign of a model with a large dense layer and no normalisation.

33.5.1 Confusion at low and high SNR

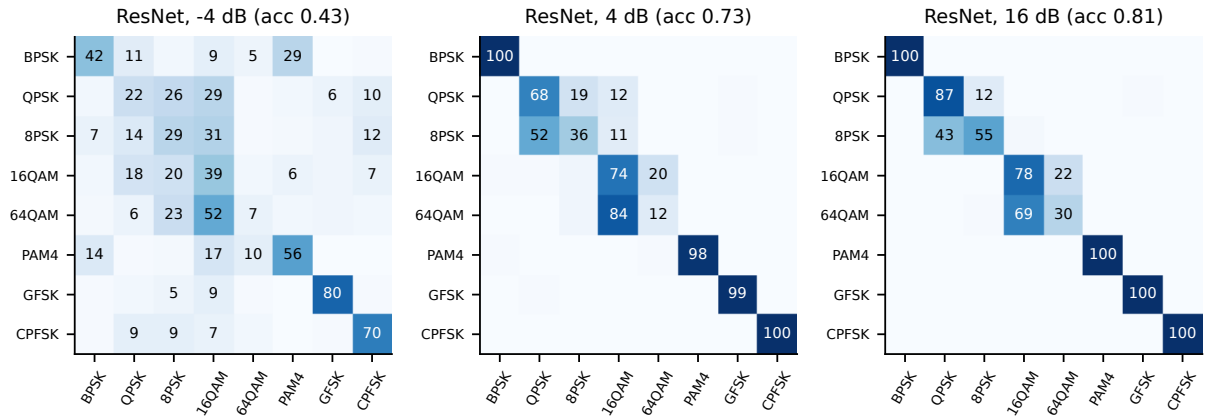


Figure 33.5: ResNet confusion matrices (row-normalised %) at -4, 4 and 16 dB. At -4 dB the model has collapsed most circular classes onto a few "default" answers; at 16 dB the remaining errors are the 16QAM/64QAM block and some QPSK/8PSK leakage.

Figure 33.5 shows how the errors evolve. At 16 dB BPSK, PAM4, GFSK and CPFSK are perfect; QPSK/8PSK (87/12 and 43/55) and 16QAM/64QAM (78/22 and 69/30) are the only confusions — the same two pairs that the cumulant table of Lab A predicts to be the closest. The network is *worse* than the SVM on QPSK vs 8PSK (the SVM has the 4th-power spectral line, a near-perfect QPSK detector at high SNR) and similar on the QAM pair. At 4 dB the PSK pair is mixed more evenly (half of 8PSK is called QPSK) and 84% of 64QAM is called 16QAM; at -4 dB, everything circular collapses together while the real (BPSK, PAM4) and constant-envelope classes remain partly recognisable.

33.5.2 What did the network learn? Embeddings

The 64-dimensional activation before the output layer is the network's *learned feature vector* — the counterpart of Lab A's 21 features. Projecting the embeddings of high-SNR test examples to 2-D (Figure 33.6) shows how the network organises the classes. PCA is a linear projection onto the two directions of largest variance; t-SNE is a nonlinear map that preserves neighbourhoods (distances between far-apart clusters in t-SNE mean little).

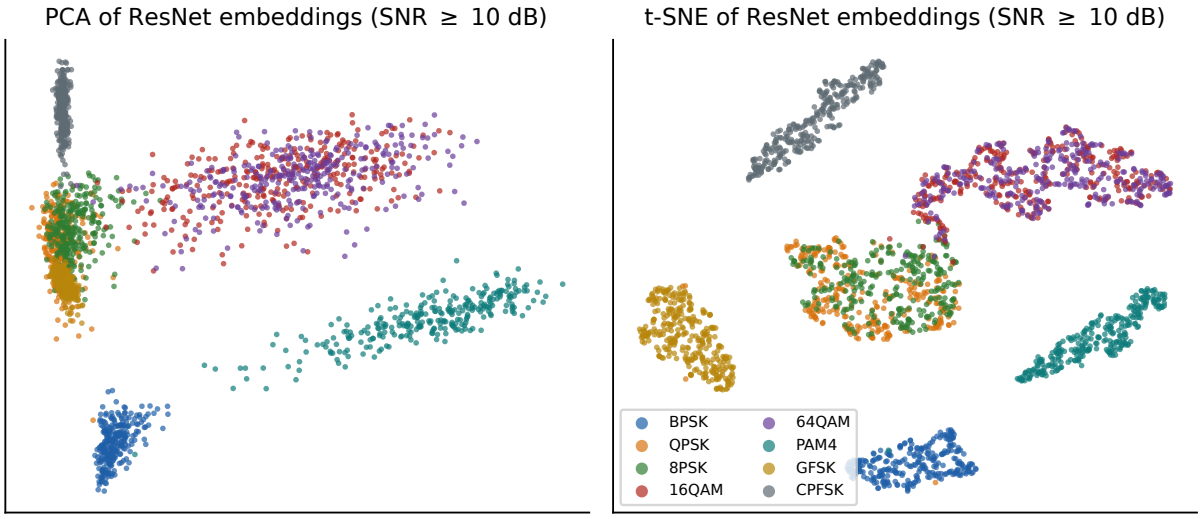


Figure 33.6: PCA (left) and t-SNE (right) of ResNet embeddings for 2400 test examples with SNR ≥ 10 dB. Clean, separate clusters for BPSK, PAM4, GFSK and CPFSK; QPSK and 8PSK adjacent; 16QAM and 64QAM forming one mixed cluster — the network's geometry mirrors exactly the confusions of Lab A's cumulant space.

33.5.3 What did the first layer learn? Filters

Because the ResNet stem treats I and Q as two channels, each of its 24 filters can be read as a 7-tap complex FIR filter $h_k[m] = w_{k,I}[m] + jw_{k,Q}[m]$. Figure 33.7 shows the eight largest-norm filters and the frequency responses of all 24.

Quantitatively: 17 of the 24 filters (71%) have their peak response inside the signal band, and on average 43% of each filter's energy response lies in the band, against 17% for a random (white) filter. Several filters are strongly asymmetric between positive and negative frequencies. Nobody told the network that the signal is band-limited or that I and Q form one complex number; it discovered both.

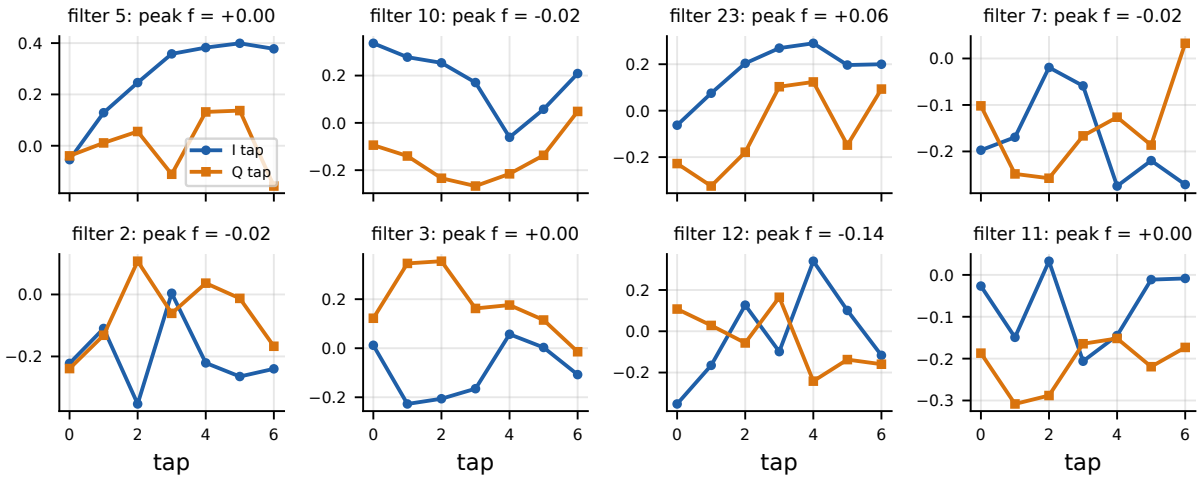


Figure 33.7: The eight largest ResNet first-layer filters: I-channel taps (blue) and Q-channel taps (orange) and the frequency where each filter's complex response peaks. Smooth, slowly varying tap patterns are low-pass/band-pass filters matched to the 1/8-cycle-per-sample signal bandwidth; others are differentiators (sign-alternating patterns) that respond to phase and amplitude changes.

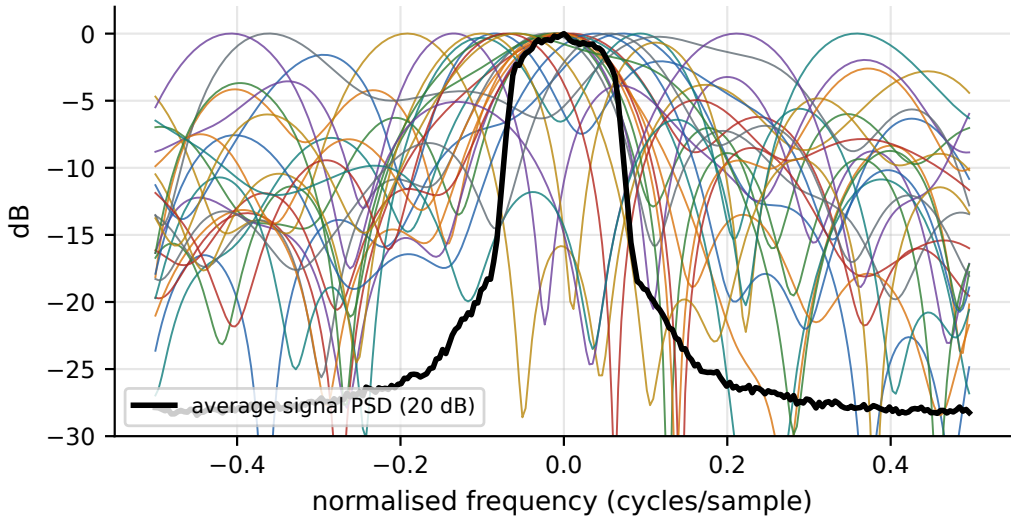


Figure 33.8: Magnitude responses of all 24 stem filters viewed as complex FIR filters (thin coloured lines, each normalised to 0 dB peak), with the average signal power spectrum at 20 dB (thick black). Most filters place their passband inside the signal band ($|f| \lesssim 0.085$ cycles/sample for RRC with roll-off 0.35 at 8 samples/symbol) or on its edges; several are asymmetric in f, i.e. genuinely complex filters that distinguish positive from negative frequencies — something no real-valued filter could do.

33.6 Ablations

An ablation changes one thing and measures the effect. All ablations below retrain the ResNet for at most 8 epochs (slightly shorter than the main run, so compare within the figure only).

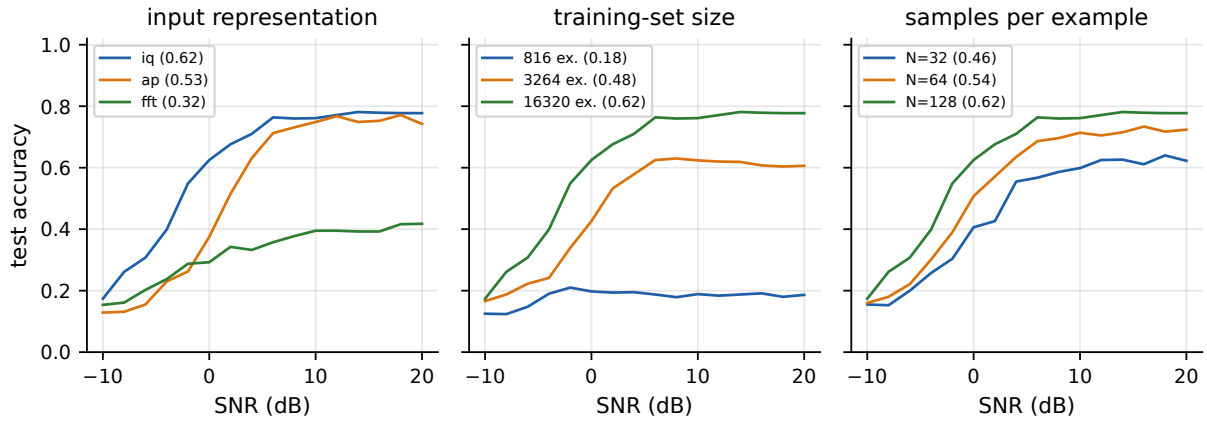


Figure 33.9: ResNet ablations (overall accuracy in brackets). Left: input representation — I/Q, amplitude/phase, log-spectrum. Middle: training-set size. Right: samples per example N (the first N samples of each example; same training-set size).

Representation. I/Q is best (0.617 overall); amplitude/phase reaches a similar plateau (0.755 vs 0.775 at high SNR) but is much worse at low SNR, where phase is noise; the log-spectrum is poor (0.322): it throws away the phase of the spectrum, i.e. the symbol sequence, and keeps essentially the bandwidth and the shape of the pulse, which are similar for all linear classes. (The spectrum still separates FSK from the linear classes, hence its ≈ 0.40 plateau.)

Training-set size. With 816 examples (5%) the ResNet did not learn at all within 8 epochs (0.18, barely above chance — far too few gradient steps), with 3264 (20%) it reaches 0.477, with all 16 320 examples 0.617. Deep models need data; the feature-based classifiers of Lab A reach 0.56 with only 60 examples per (class, SNR) = 7680 (Figure 32.13).

Sequence length. Using only the first 32 or 64 samples (4 or 8 symbols) lowers the score to 0.458 and 0.541 (high SNR 0.62 and 0.72 vs 0.78). The network, like the cumulant estimator, needs symbols to average over; the effect is the same $1/\sqrt{N}$ story as Lab A, seen from the network's side.

33.7 Leaderboard tricks

Intuition: wisdom of crowds

Different models make different mistakes. If their errors are not perfectly correlated, averaging their predicted probabilities cancels part of the errors — the same reason a committee of reasonable people beats most individual members.

Ensembling. Average the softmax outputs $\mathbf{p}^{(m)}$ of M models and take the arg-max: $\hat{y} = \arg\max_c \frac{1}{M}\sum_m p_c^{(m)}$. Our three-model ensemble: 0.631 overall, 0.788 at high SNR — *worse* than the ResNet alone. Averaging only helps if the members are comparably good; here VT-CNN2 and the CLDNN are much weaker and drag the average down. A weighted average (weights tuned on validation data) or an ensemble of several ResNets with different seeds would be the

right move.

Stacking. Train a second-level model on the outputs of first-level models. We fit a logistic regression on $[\log \mathbf{p}^{\text{ResNet}}, \mathbf{f}^{\text{Lab A}}]$ — the ResNet's log-probabilities concatenated with the 21 expert features — using *only the validation split*, which the ResNet never trained on (otherwise the stacker would learn to trust over-confident training predictions). Result: 0.678 overall and 0.875 at high SNR — the best entry on the Lab B leaderboard, combining the ResNet's low-SNR strength with the features' high-SNR precision (Figure 33.3). This is the standard Kaggle recipe: when two model families have *different* error patterns, stacking beats both.

Test-time augmentation (TTA). The label is invariant to a carrier phase rotation $r \to re^{j\theta}$, so we can classify K rotated copies of each test example and average the probabilities. With $K = 4$ rotations by multiples of 90°: 0.657 overall, 0.816 at high SNR, a gain of 1.7 and 1.0 points over the plain ResNet for four times the inference cost. TTA helps when the model has *not* fully learned the invariance; our training data already contain every phase, so the gain is limited. Training-time phase augmentation (random rotation of every batch): 0.611 overall vs 0.617 without (8-epoch runs) — no gain, as expected when the data already contain uniformly random phases.

Pitfall: the Kaggle trap: overfitting the public leaderboard

On a real Kaggle competition the public leaderboard is computed on a small part of the test set. Submitting dozens of variants and keeping the best one "trains" on that part. Keep a private validation set, prefer changes that improve validation *and* make physical sense, and remember that differences below about one point (Section 32.1) are noise.

33.8 Inference cost

A deployed classifier must keep up with the signal. Table 33.1 lists per-example inference times of 0.4–0.9 ms on a shared CPU thread; the feature-based pipeline needs about 1–2 ms per example for feature extraction plus the classifier time of Table 32.2. Rough arithmetic: the ResNet's forward pass is dominated by the residual convolutions, $3 \times 2 \times 24 \times 24 \times 3 \times L$ multiply-accumulates with $L = 64, 32, 16$, i.e. about 0.5 M MACs per example — trivial for an FPGA or DSP. The VT-CNN2 dense layer alone is $2048 \times 64 = 131$ k MACs, but its convolutions at full length cost more.

Chapter recap

- Representations: I/Q (lossless, linear), amplitude/phase (good for LSTMs, noisy phase at low SNR), spectrum (phase/timing invariant but lossy), constellation image (time order lost).
- Architectures: VT-CNN2 (conv $1 \times 3 \to$ conv $2 \times 3 \to$ dense), small 1-D ResNet (residual blocks, pooling, global average), CLDNN (conv + LSTM).
- Training: cross-entropy, Adam, LR halving on plateau, early stopping on a validation split, SNR-balanced batches.
- Results: best single model ResNet (0.640 overall); it beats Lab A's SVM (0.588) by up to 24 points below 0 dB but loses on the high-SNR plateau (0.81 vs 0.88); stacking the ResNet with the features gives the best score, 0.678 / 0.875.
- The learned embedding space reproduces Lab A's cumulant geometry: 16QAM/64QAM remain merged — a data limit (16 symbols), not a model limit.

- Ensembling, stacking (features + network) and TTA by phase rotation are cheap leaderboard gains.

Exercises

1. Count the parameters of the scaled VT-CNN2 layer by layer and check the total 134 872.
2. Compute the receptive field of the ResNet if the three blocks used kernel 5 instead of 3.
3. Show that a phase rotation $r \to re^{j\theta}$ maps the I/Q vector (I, Q) by a 2×2 rotation matrix. Why does a first-layer conv on I/Q not automatically become phase-invariant, while global pooling over time does make the model timing-invariant?
4. The ResNet is better than the SVM at -2 dB but worse at 16 dB. Give two reasons from the material of this lab.
5. A colleague trains on SNR ≥ 0 dB only, "because the low-SNR data is useless". Predict the effect on the test accuracy at -4 dB and at 16 dB.
6. With 800 test examples per SNR, is the difference between 0.80 and 0.82 at 16 dB significant? Compute the standard error.
7. (Coding) Normalise every example to unit *total* power before training. Does accuracy change? Why might it in a real receiver?
8. (Coding) Implement curriculum learning: train 3 epochs on SNR ≥ 10 dB, then on all SNRs. Compare learning curves.

Answers

1. conv1: $32 \times (1 \cdot 3) + 32 = 128$; conv2: $16 \times (32 \cdot 2 \cdot 3) + 16 = 3088$; dense1: $16 \cdot 128 \cdot 64 + 64 = 131\,136$; dense2: $64 \cdot 8 + 8 = 520$. Total $128 + 3088 + 131\,136 + 520 = 134\,872$. ✓
2. Each kernel-5 conv at down-sampling D adds $4D$ samples: $7 + 2 \cdot 4 \cdot (2 + 4 + 8) = 119$ samples, almost the whole window.
3. $(I + jQ)e^{j\theta} = (I\cos\theta - Q\sin\theta) + j(I\sin\theta + Q\cos\theta)$, i.e. $\left(\begin{smallmatrix}\cos\theta & -\sin\theta \\ \sin\theta & \cos\theta\end{smallmatrix}\right)\left(\begin{smallmatrix}I \\ Q\end{smallmatrix}\right)$. A conv filter takes a fixed linear combination of the rows, whose output changes with θ; invariance must be learned from data containing all phases (or built in). Global pooling averages over all time positions, so shifting the pattern in time leaves the pooled value (almost) unchanged.
4. (i) At low SNR the network uses the whole waveform (time structure, matched-filter-like averaging) instead of noisy sample statistics; at high SNR the expert features are near-sufficient statistics and the SVM uses them optimally. (ii) The network was trained for a few minutes on 16 000 examples with 13 000 parameters; it has not fully learned the invariances (phase) and fine amplitude statistics (16QAM vs 64QAM, QPSK vs 8PSK) that the features encode by construction.
5. At -4 dB accuracy drops (the model never saw such noise; its decision regions are tuned to clean data and become overconfident). At 16 dB it may rise slightly, because the capacity and gradient are no longer spent on hopeless examples.
6. SE $= \sqrt{0.81 \cdot 0.19/800} = 0.0139$; the SE of a difference of two independent estimates is $\sqrt{2} \times 0.0139 \approx 0.020$. A 2-point gap is about one SE — not significant.

7. In this competition the total power $1 + 1/\mathrm{SNR}$ is a (weak) SNR cue, so removing it may lower accuracy slightly at low SNR. In a real receiver the AGC sets an unknown gain, so a model that relies on absolute power would fail; per-example normalisation is the robust choice.
8. Typically training is faster initially (clean examples give clear gradients) and the final accuracy is similar; curricula help most for larger models or harder tasks.

Self-test quiz

1. Which input representation is lossless and linear in the signal?
   I/Q.
2. Why does the ResNet have ten times fewer parameters than VT-CNN2?
   It down-samples and uses global average pooling instead of flattening a full-length feature map into a large dense layer.
3. What is a "stacker" trained on, and why must it be held-out data?
   On first-level model outputs (plus features) for data the first-level model did not train on; otherwise it learns to trust over-confident training-set predictions.
4. Which class pair remains confused at high SNR for both features and CNNs, and why?
   16QAM/64QAM: with 16 symbols their statistics are nearly indistinguishable.
5. What does test-time augmentation by phase rotation exploit?
   The invariance of the label to the carrier phase.

Further reading

- T. J. O'Shea, J. Corgan and T. C. Clancy, "Convolutional radio modulation recognition networks," *Proc. EANN*, 2016 (arXiv:1602.04105). The original VT-CNN2-style CNN on raw I/Q.
- T. J. O'Shea, T. Roy and T. C. Clancy, "Over-the-air deep learning based radio signal classification," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, 2018. Deeper ResNet-style models, 24 classes, comparison with moment features + boosted trees.
- N. E. West and T. J. O'Shea, "Deep architectures for modulation recognition," *IEEE DySPAN*, 2017 (arXiv:1703.09197). CLDNN and residual variants.
- A. Swami and B. M. Sadler (2000); E. E. Azzouz and A. K. Nandi (1996); O. A. Dobre *et al.* (2007) — the feature-based background, see Chapter 32.

Chapter 34

Lab C: Deep Learning-Based Modulation Classification in Rayleigh Multipath Channels

The labs so far were played on a friendly field: the signal reached the receiver along one path, with only noise and a rotation added. Real radio signals bounce off buildings, the ground and vehicles, and arrive as a sum of delayed, attenuated, phase-shifted copies of themselves. In this lab the competition data go through a random *Rayleigh multipath* channel before the noise is added — a new channel for every example, sometimes slowly varying in time (Doppler).

You will (i) recap the tapped-delay-line multipath model from first principles; (ii) see with your own eyes how multipath smears constellations and corrupts the cumulants that Lab A relied on; and (iii) run six experiments: the collapse of an AWGN-trained network on multipath data, training on multipath, mixed training, the feature-based classifier under multipath, a blind CMA equaliser in front of the CNN, and architectures with larger receptive fields. The goal is not only a leaderboard number but an understanding of *why* certain classes become confusable.

34.1 The competition

Same eight classes, same 128-sample format, same SNR grid and same files (`rayleigh_train`, `rayleigh_test`; 19 200 and 12 800 examples), same metric. The only change is the gray channel block of Figure 32.1: each example's waveform is convolved with a random time-varying channel before power normalisation and noise. To beat: the Lab B ResNet scored 0.640 overall and 0.806 at SNR $\geq$ 10 dB on AWGN; what does it score here?

34.2 Multipath from first principles

Intuition: echoes in a stairwell

Clap your hands in a stairwell: you hear the direct sound followed by a decaying tail of echoes. Each echo is a delayed, quieter copy. Speech in a stairwell is blurred because each syllable is smeared into the next. A radio symbol in a multipath channel is smeared into the following symbols in exactly the same way — *inter-symbol interference* (ISI). And because the echoes arrive with random phases, they sometimes add and sometimes cancel: at some frequencies the channel is loud, at others almost silent (*frequency-selective fading*).

34.2.1 The tapped delay line

Definition: discrete-time multipath channel

At sample rate $1/T_s$, a linear time-invariant multipath channel is an FIR filter

$$v[n] = \sum_{l=0}^{L-1} g_l \, s[n-l], \tag{34.1}$$

where the complex *tap gain* g_l is the sum of all propagation paths whose delay falls in the l-th sample interval. If the channel changes over time, $g_l \to g_l[n]$.

Why Rayleigh. Each tap collects many paths with independent random phases: $g_l = \sum_i a_i e^{j\theta_i}$. By the central-limit theorem a sum of many independent terms is approximately Gaussian, so $g_l \sim \mathcal{CN}(0, p_l)$. The magnitude of a circular complex Gaussian has the *Rayleigh* density $f(|g|) = \frac{2|g|}{p_l} e^{-|g|^2/p_l}$, and the phase is uniform. Hence "Rayleigh fading". The probability that a tap is 20 dB below its mean power is $1 - e^{-0.01} \approx 1\%$ — deep fades are not rare.

34.2.2 Power-delay profile and delay spread

The *power-delay profile* (PDP) $p_l = \mathbb{E}\,|g_l|^2$ says how much power arrives, on average, at each delay. A common model is **exponential**:

$$p_l = \frac{e^{-l/\tau}}{\sum_{m=0}^{L-1} e^{-m/\tau}}, \qquad l = 0, \dots, L-1, \tag{34.2}$$

normalised so that $\sum_l p_l = 1$ (the channel neither amplifies nor attenuates on average). Its *RMS delay spread* is the standard deviation of the delay weighted by the PDP,

$$\sigma_\tau = \sqrt{\sum_l p_l l^2 - \Big(\sum_l p_l l\Big)^2} \ \text{samples}. \tag{34.3}$$

For a long exponential profile, $\sigma_\tau \approx \tau$.

Worked example: delay spread of a short profile

$\tau = 1$, $L = 4$: unnormalised weights $1, 0.368, 0.135, 0.050$, sum 1.553; $p = (0.644, 0.237, 0.087, 0.032)$. Mean delay $0.237 + 2 \cdot 0.087 + 3 \cdot 0.032 = 0.507$; mean square $0.237 + 4 \cdot 0.087 + 9 \cdot 0.032 = 0.873$; $\sigma_\tau = \sqrt{0.873 - 0.257} = 0.785$ samples, i.e. about 0.1 symbol at 8 samples/symbol.

Coherence bandwidth. The frequency response $G(f) = \sum_l g_l e^{-j2\pi f l}$ varies with f on a scale of roughly $B_c \approx 1/(5\sigma_\tau)$ (a common rule of thumb, for 0.5 correlation). Our signal occupies $|f| \le (1 + 0.35)/(2 \cdot 8) \approx 0.084$ cycles/sample. With $\sigma_\tau = 4$ samples, $B_c \approx 0.05$: the channel varies *within* the signal band — frequency-selective fading. With $\sigma_\tau = 0.5$ samples, $B_c \approx 0.4$: the channel is nearly flat over the band.

34.2.3 Doppler

If the transmitter, receiver or scatterers move at speed v, a path arriving at angle α is shifted in frequency by $f_D \cos\alpha$, with maximum Doppler $f_D = v f_c / c$. Summing many paths with random

angles makes each tap a random process in time. The classic *Clarke/Jakes* model generates it as a sum of sinusoids:

$$g_l[n] = \sqrt{\frac{p_l}{K}} \sum_{i=1}^{K} e^{j(2\pi f_D n \cos \alpha_{l,i} + \varphi_{l,i})}, \qquad \alpha, \varphi \sim \mathcal{U}[0, 2\pi). \tag{34.4}$$

The *coherence time* $T_c \approx 0.423/f_D$ is how long the channel stays roughly constant.

Worked example: how fast does the channel change in one example?

Our generator draws $f_D \sim \mathcal{U}[0, 2 \times 10^{-3}]$ cycles/sample. At the maximum, $T_c \approx 0.423/0.002 = 212$ samples, longer than an example (128 samples): within one example the channel drifts by up to about half a coherence time. In physical terms, with 8 samples per symbol at 1 Msym/s ($T_s = 125$ ns), $f_D = 2 \times 10^{-3}/T_s = 16$ kHz — a fast-moving receiver at mmWave, or an exaggerated value at sub-6 GHz. We use it to make the problem harder, not to model a specific deployment.

34.2.4 The channel in our generator

`modsim.apply_rayleigh` draws, for every example: $\tau \sim \mathcal{U}[0.5, 4]$ samples (i.e. 0.06 to 0.5 symbol), $L = \min(24, \lceil 5\tau \rceil + 1)$ taps, independent $g_l \sim \mathcal{CN}(0, p_l)$, and with Doppler $f_D \sim \mathcal{U}[0, 2 \times 10^{-3}]$ the sum-of-sinusoids model with $K = 8$ per tap (Figure 34.2). After the channel the waveform is normalised to unit power, so the instantaneous fade depth does not change the SNR: we study distortion, not outage.

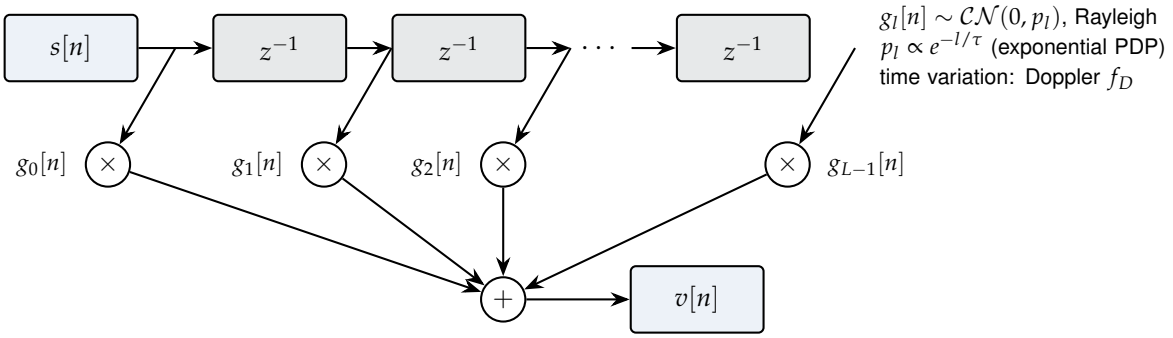


Figure 34.1: The tapped-delay-line Rayleigh channel used in Lab C. Each delay element is one sample (1/8 symbol). Every example gets its own τ, tap gains and Doppler.

34.3 What multipath does to the signal and its features

34.3.1 Smeared constellations

Figure 34.3 shows symbol-rate constellations after different channels (matched filter, best sampling phase, phase-corrected), at 30 dB with long records so noise is negligible. With $\tau = 1.5$ each QPSK point becomes a cluster: the received symbol is $g'_0 a_k + \sum_{m \neq 0} g'_m a_{k-m}$, the desired symbol plus a random combination of neighbours. For 16QAM, whose points are closer together, the clusters merge. With $\tau = 4$ the ISI extends over several symbols, and with Doppler the cloud also rotates and breathes during the record, so a single phase correction cannot align it.

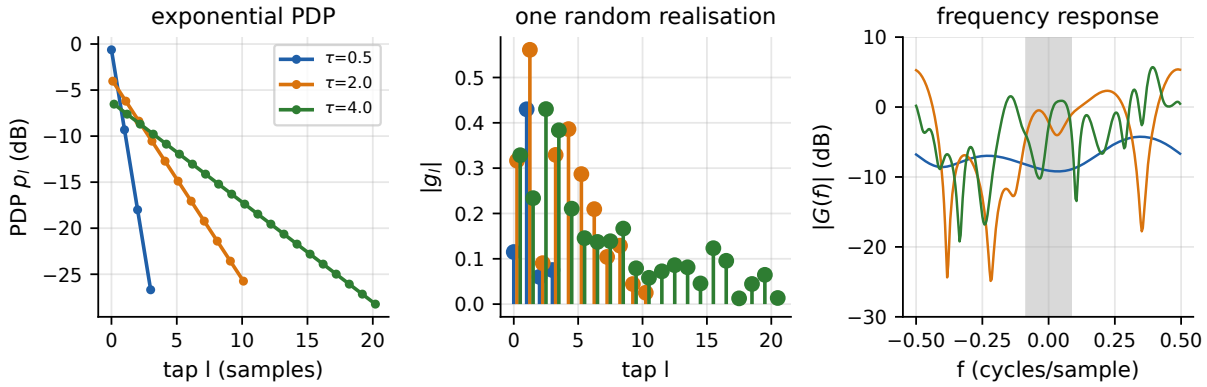


Figure 34.2: Left: exponential PDPs for $\tau = 0.5, 2, 4$ samples. Middle: one random realisation of the tap magnitudes for each. Right: the corresponding frequency responses; the gray band is our signal bandwidth. Short profiles ($\tau = 0.5$, blue) are almost flat across the band: the channel is mostly a random complex gain. Long profiles produce notches of 10–20 dB inside the band.

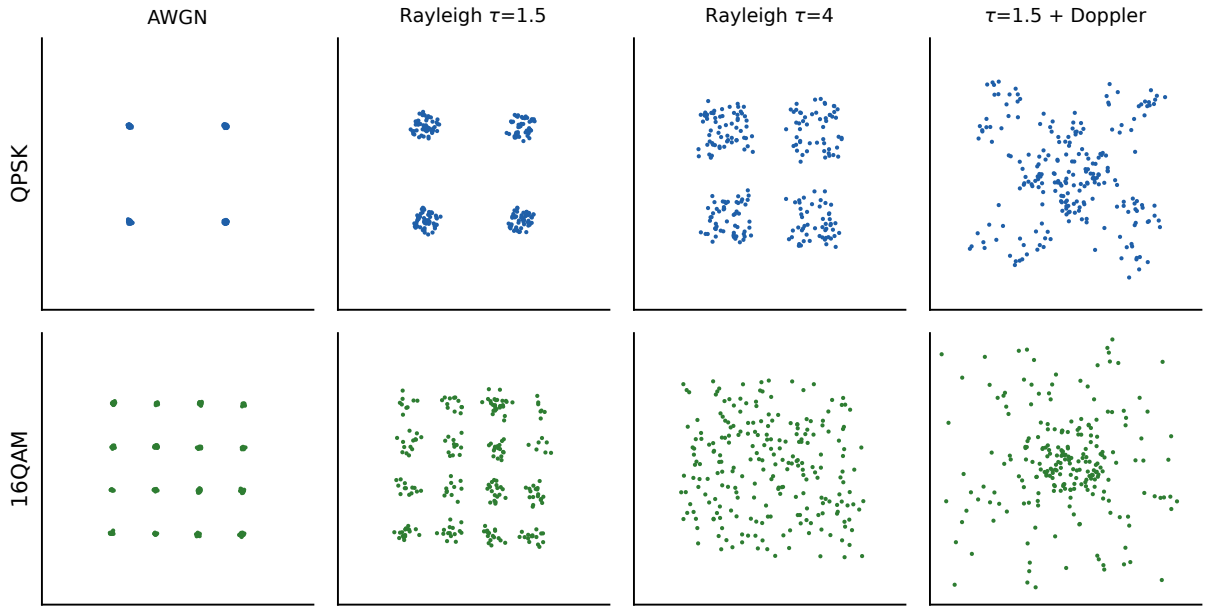


Figure 34.3: Matched-filtered symbol constellations of QPSK (top) and 16QAM (bottom) at 30 dB: no channel, Rayleigh with $\tau = 1.5$ and $\tau = 4$ samples, and $\tau = 1.5$ with Doppler. ISI turns each point into a cluster; Doppler smears the clusters around the circle. The 16QAM structure is gone long before the QPSK structure.

34.3.2 Why cumulants suffer: the central-limit theorem strikes again

Take a symbol-rate view: the received sample is $y_k = \sum_m h_m a_{k-m}$ with independent symbols a. Cumulants are additive over independent terms and $C_{pq}(ha) = h^{p-q}(h^*)^q C_{pq}(a)$, so

$$C_{42}^{(y)} = C_{42}^{(a)} \sum_m |h_m|^4, \qquad C_{21}^{(y)} = C_{21}^{(a)} \sum_m |h_m|^2, \tag{34.5}$$

and the normalised value becomes

Key equation: cumulant shrinkage by a linear channel

$$\tilde{C}_{42}^{(y)} = \tilde{C}_{42}^{(a)} \cdot \rho, \qquad \rho = \frac{\sum_m |h_m|^4}{\left(\sum_m |h_m|^2\right)^2} \le 1. \tag{34.6}$$

$\rho = 1$ only if a single tap is non-zero; with K equal taps, $\rho = 1/K$. Multipath therefore *multiplies all 4th-order features by a random factor* ρ, different for every example. This is exactly the same mechanism as the pulse-shaping factor κ_4 of Lab A, but now random: the signal becomes "more Gaussian" because each sample is a mixture of more symbols. Similarly $\tilde{C}_{40}^{(y)} = \tilde{C}_{40}^{(a)} \cdot \left|\sum_m h_m^4\right| / (\sum_m |h_m|^2)^2$ and $\tilde{C}_{20}^{(y)}$ is multiplied by $\left|\sum_m h_m^2\right| / \sum_m |h_m|^2$ — the phases of the taps now enter, so these features become random even in magnitude.

Worked example: two equal paths

$h = (1, e^{j\psi})/\sqrt{2}$: $\sum |h|^2 = 1, \sum |h|^4 = 1/2$, so $\rho = 0.5$ — QPSK's $\tilde{C}_{42} = 1$ looks like 0.5, below 16QAM's 0.68. For $\tilde{C}_{20}$ of BPSK: $\left|(1 + e^{j2\psi})/2\right| = |\cos\psi|$; with $\psi = 90°$ it is 0 — BPSK looks circular.

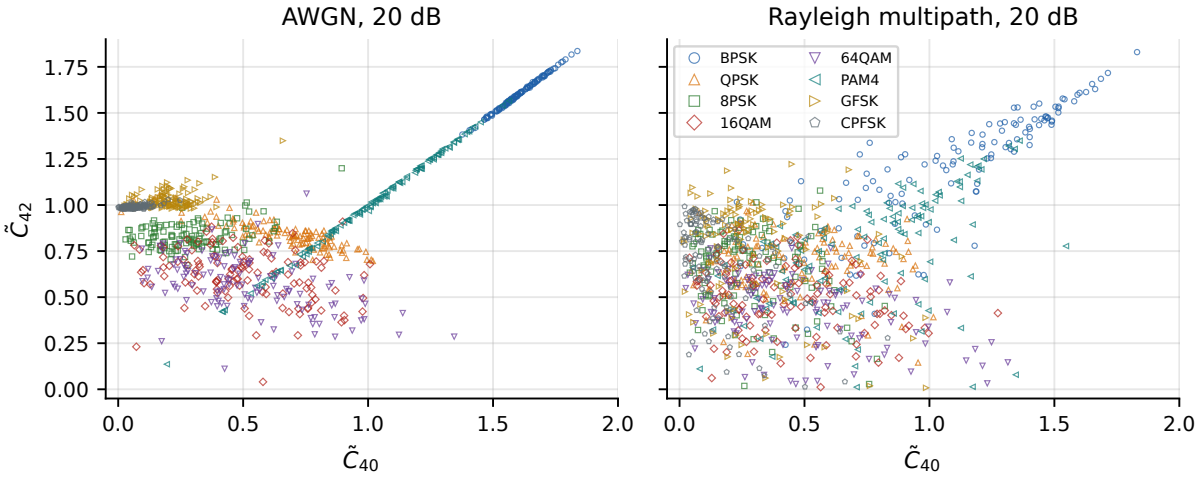


Figure 34.4: The Lab A feature plane $\tilde{C}_{42}$ vs $\tilde{C}_{40}$ at 20 dB for AWGN (left) and Rayleigh multipath (right). Under multipath every cluster is pulled towards the origin by the random factor of eq. (34.6) and stretched along a ray: BPSK and PAM4 slide down the diagonal into the region of QPSK and the QAMs, and QPSK slides into the QAM region.

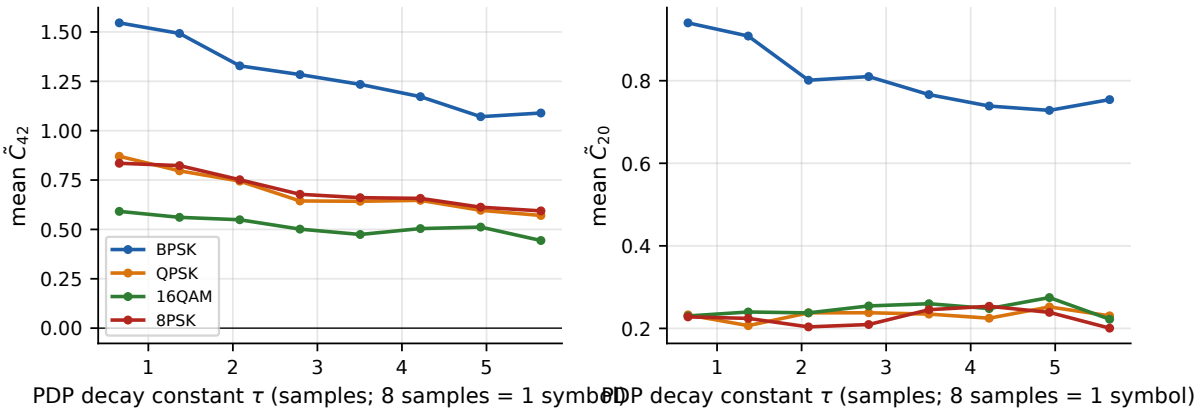


Figure 34.5: Noise-free class means of $\tilde{C}_{42}$ (left) and $\tilde{C}_{20}$ (right) versus the PDP constant τ, 400 channels per class. Longer delay spread means more taps, smaller ρ, and smaller features; BPSK's "realness" $\tilde{C}_{20}$ also falls as tap phases mix I and Q.

34.4 Experiments

All networks are the Lab B ResNet (13 k parameters) unless stated, trained with the Lab B recipe for at most 10 epochs on the 16 320 training examples (15% validation); the full script is `code/ch34/lab_c_train.py`. Figure 34.6 maps the experiments.

34.4.1 (a) Train on AWGN, test on Rayleigh: the collapse

We take the Lab B ResNet, trained only on AWGN, and test it on the Rayleigh test set. The overall accuracy drops from 0.640 (AWGN test set) to 0.354, and the high-SNR plateau from 0.806 to 0.384 (Table 34.1). Worse, the curve in Figure 34.9 is flat above about 0 dB: *more SNR does not help*, because the error is caused by the channel, not by noise. Figure 34.10 (left) shows what the network does with signals it has never seen: at 16 dB it sends more than half of all QPSK, 8PSK, 16QAM and 64QAM examples, and a third of BPSK, into the two QAM classes. That is exactly what eq. (34.6) predicts for any statistic the network may have learned: multipath makes every linear modulation "more Gaussian", and the most Gaussian-looking classes in AWGN were the QAMs. The accuracy also falls steadily with the delay spread (Figure 34.11): from 0.56 for the shortest profiles to 0.27 for the longest.

Pitfall: train/test mismatch

A network learns the distribution it is trained on — nothing more. Its 80% on AWGN is a statement about AWGN data. This is the single most common failure of deep-learning receivers moved from simulation to the field: the simulator's channel was too kind.

34.4.2 (b) Train on Rayleigh

Training the same ResNet on Rayleigh data raises the Rayleigh score to 0.436 overall and 0.517 at high SNR — better than the mismatched model, but far below the AWGN ceiling. Notice a surprise in Table 34.1: tested on *AWGN*, this Rayleigh-trained model scores only 0.487 (high SNR 0.564). Training on the hard channel did not teach it the easy one; the AWGN examples are a different distribution (no ISI) that it never saw. The curve saturates near 0.52 from about 4 dB upward: again a channel-limited plateau.

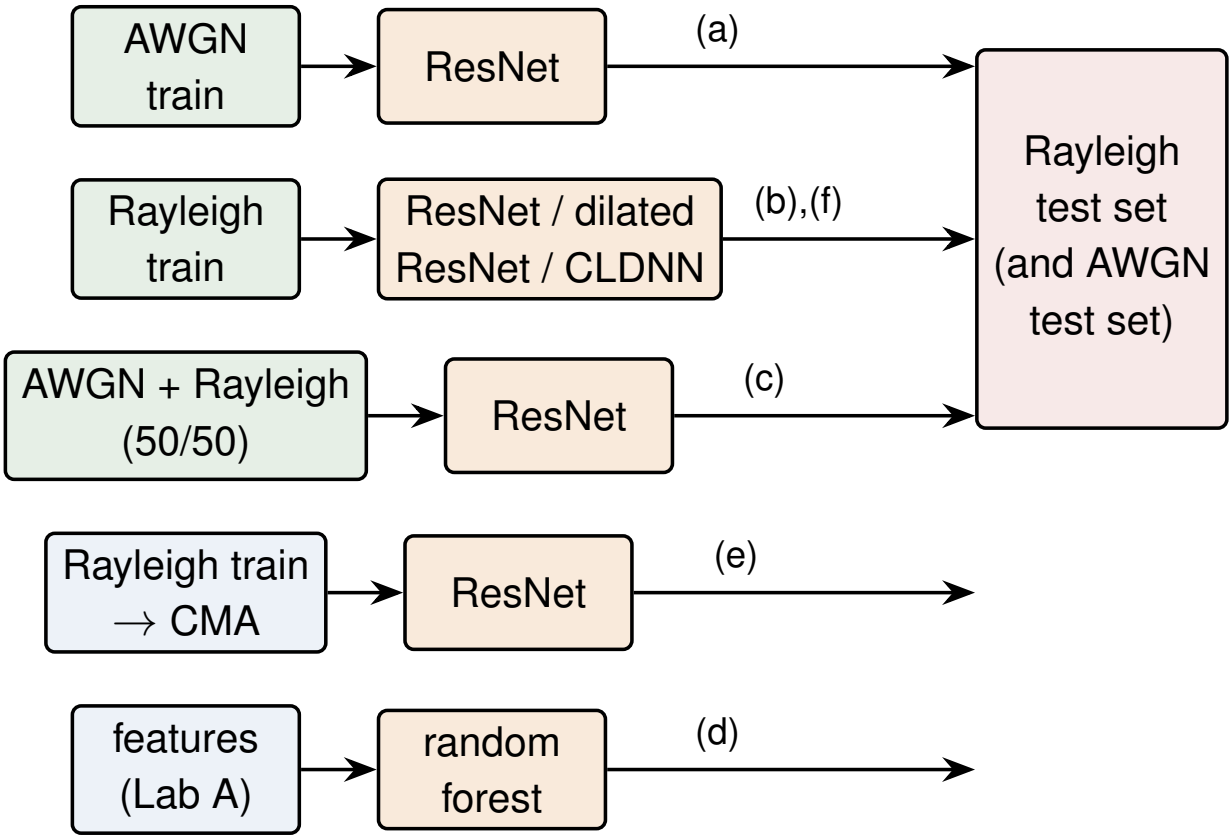


Figure 34.6: The six experiments of Lab C. Every model is scored on the same Rayleigh test set; models (a)–(c) also on the AWGN test set.

34.4.3 (c) Mixed training

Half the training examples from AWGN, half from Rayleigh (same total size). The result is the best deep model of the lab: 0.485 overall and 0.599 at high SNR on Rayleigh, and still 0.731 at high SNR on AWGN (Table 34.1, Figure 34.9). It beats the model trained *only* on Rayleigh, on Rayleigh data. Why? The AWGN half contains the clean "prototypes" of each class — the short-delay-spread end of the Rayleigh family — which makes the class structure easier to learn in a short training budget; in effect AWGN acts as a curriculum and a regulariser. Figure 34.11 confirms that the mixed model gains most on the short profiles ($\tau < 2$) and is no worse on the long ones. The lesson for practice: augment with a *range* of channel conditions, including benign ones, rather than only the worst case.

34.4.4 (d) Feature-based classifier under Rayleigh

A random forest on the 21 Lab A features, (i) trained on AWGN features and (ii) trained on Rayleigh features. The AWGN-trained forest scores 0.351 overall (0.472 at high SNR): it collapses like the network. Retrained on Rayleigh features it reaches 0.419 overall and 0.622 at high SNR — the *best high-SNR result of all single models in this lab*, slightly above the mixed-trained ResNet (0.599). Its curve keeps rising with SNR while the networks saturate at 4–6 dB (Figure 34.9, right); at low SNR, however, the networks are 5–20 points better. The most important features under Rayleigh are still $\tilde{C}_{20}$, σ_{af}, $\gamma_{\max}$ and σ_{aa} (random-forest importance 0.092, 0.066, 0.057, 0.055): realness and envelope variation survive multipath better than the 4th-order magnitudes.

34.4.5 (e) A blind equaliser in front of the CNN

Intuition: undo the echo before you listen

If the channel smeared the symbols together, try to un-smear them first with an adaptive filter. We do not know the symbols, but we know one thing about PSK: every symbol has the same amplitude. A filter that makes the output amplitude as constant as possible is likely to have undone the channel. This is the *constant-modulus algorithm* (CMA), Godard's 1980 idea, still in every modem's toolbox.

Definition: CMA

An FIR equaliser $y_k = \mathbf{w}^{\mathsf{H}}\mathbf{u}_k$ (here written $y_k = \sum_i w_i u_{k,i}$) on the matched-filtered input vector $\mathbf{u}_k$ minimises the *constant-modulus cost*

$$J(\mathbf{w}) = \mathbb{E}\left[\left(|y_k|^2 - R_2\right)^2\right], \qquad R_2 = \frac{\mathbb{E}\,|a|^4}{\mathbb{E}\,|a|^2}. \tag{34.7}$$

The gradient. With $y = \sum_i w_i u_i$, $|y|^2 = yy^*$, the Wirtinger derivative with respect to w_i^* of $(|y|^2 - R_2)^2$ is $2(|y|^2 - R_2)\, y\, u_i^*$. Dropping the expectation (stochastic gradient) gives the CMA update

$$\mathbf{w} \leftarrow \mathbf{w} - \mu\, e_k\, \mathbf{u}_k^*, \qquad e_k = y_k\left(|y_k|^2 - R_2\right). \tag{34.8}$$

For PSK, $R_2 = 1$ and the cost is zero for a perfect equaliser. For QAM, $|a|$ is not constant, but the cost is still minimised (with residual "self-noise") near the ISI-free solution — CMA works for QAM too, more slowly. CMA is blind to the carrier phase (the cost only sees $|y|$), which is fine because our classifier is trained on all phases anyway.

Our implementation (`code/ch34/cma.py`, Figure 34.7) matched-filters the example, picks the sampling phase of largest energy, runs a 17-tap fractionally spaced CMA ($\approx$ 2 symbols) updated once per symbol, and, because 16 symbols are too few to converge, sweeps the same block 20 times with $\mu = 0.01$. On noise-free QPSK it reduces the normalised dispersion $\mathbb{E}(|y|^2 - 1)^2$ at the symbol instants from 0.37 to 0.07 (AWGN: 0.26 $\rightarrow$ 0.01). Figure 34.8 shows the effect on a long 16QAM record.

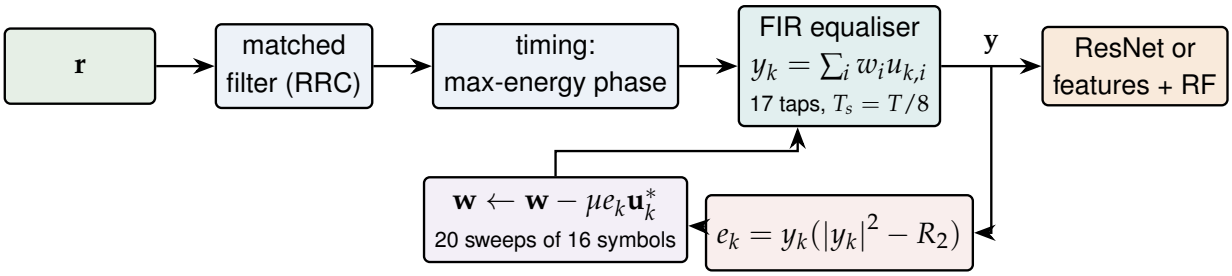


Figure 34.7: Experiment (e): a blind CMA equaliser in front of the classifier. The error e_k needs no known symbols, only the target modulus R_2; the update loop (bottom) adapts the taps once per symbol.

```
    X = matched_filter(np.asarray(X, complex))
    n, N = X.shape
    pad = Lw // 2
    Xp = np.concatenate([np.zeros((n, pad)), X, np.zeros((n, pad))], 1)
    idx = np.arange(N)[:, None] + np.arange(Lw)[None, ::-1]
    U = Xp[:, idx]                                              # (n, N, Lw)
    regressors
```

```
    ph = np.argmax(np.stack([np.mean(np.abs(X[:, p::sps]) ** 2, 1) for p in range(
    sps)], 1), 1)
    w = np.zeros((n, Lw), complex); w[:, pad] = 1.0
    if R2 is None:                                          # Godard constant E|
    a|^4/E|a|^2 ~ 1 for unit power
        R2 = 1.0
    T = np.minimum(ph[:, None] + sps * np.arange(N // sps)[None, :], N - 1)    #
    symbol instants
    rows = np.arange(n)
    J = []
    for _ in range(passes):
        for k in range(T.shape[1]):
            u = U[rows, T[:, k]]
            y = np.sum(w * u, 1)
            e = y * (np.abs(y) ** 2 - R2)
            e = e / np.maximum(1.0, np.abs(e) / 4.0)          # clip large errors
    (avoids divergence)
            w -= mu * e[:, None] * np.conj(u)
        ys = np.einsum('nkl,nl->nk', U[rows[:, None], T], w)
        J.append(np.mean((np.abs(ys) ** 2 / np.mean(np.abs(ys) ** 2, 1, keepdims=
    True) - 1) ** 2))
    Y = np.einsum('nkl,nl->nk', U, w)
    Y = Y / np.sqrt(np.mean(np.abs(Y) ** 2, 1, keepdims=True) + 1e-12)
    if return_info:
        return Y, dict(phase=ph, w=w, J=np.array(J), T=T)
    return Y
```

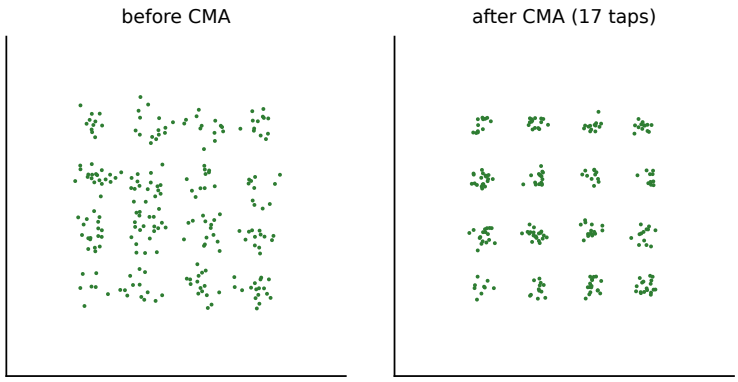


Figure 34.8: A 16QAM signal ($\tau = 2.5$, 30 dB, 2048 samples) before and after the 17-tap CMA, at the symbol instants. The equaliser restores the square grid, although clusters remain wider than in AWGN (residual ISI and CMA self-noise for a non-constant-modulus constellation).

Results (Table 34.1): CMA + ResNet scores 0.432 overall and 0.556 at high SNR, CMA + random forest 0.421 and 0.564. Compared with the same models on un-equalised data (0.436/0.517 and 0.419/0.622), the network gains about 4 points on the high-SNR plateau, while the random forest *loses* 6 points there. This is a real, instructive outcome rather than a failure of the code: (i) with only 16 symbols per example, CMA has not converged for many examples even after 20 sweeps, and its residual misadjustment is a new random distortion; (ii) CMA drives every signal towards constant modulus, so it partly *erases* the amplitude statistics ($\tilde{C}_{42}$, $\gamma_{\max}$, σ_{aa}, PAPR) that distinguish PSK from QAM and BPSK from PAM4 — the classifier's best features; (iii) the matched filter and the energy-based timing choice also change the waveform the classifier sees. Blind equalisers are designed for long records; at 16 symbols, letting the classifier learn the channel directly is as good or better.

34.4.6 (f) Larger receptive field and recurrence

Multipath spreads information over time, so a model that sees a longer context at once might cope better. We try a dilated ResNet (dilations 1, 2, 4 in the three blocks, receptive field $7 + 2 \cdot 2 \cdot (2 \cdot 1 + 4 \cdot 2 + 8 \cdot 4) = 175$ samples, i.e. the whole example) and a CLDNN on I/Q with the LSTM over 64 steps. The dilated ResNet scores 0.451 overall and 0.534 at high SNR: a small gain over the plain ResNet (0.436/0.517), within about one standard error per SNR point but consistent across SNRs. Seeing the whole window at once helps a little; it does not remove the channel-limited plateau. The CLDNN on I/Q failed to train within the 10-epoch budget: it stayed near chance (0.145). We report it rather than hide it — recurrent networks on raw I/Q with a random carrier phase are notoriously slow to start learning (in Lab B the amplitude/phase CLDNN was the slowest model to approach its plateau, Figure 33.4), and a fair comparison would need a longer schedule, gradient clipping and a learning-rate warm-up. This is a useful reminder that "bigger receptive field" is only an advantage if the model can be optimised.

34.5 Results

Table 34.1: Lab C leaderboard. All scores on the Rayleigh test set unless marked; "≥ 10 dB" averages SNR 10–20 dB.

Exp.	Model (training data)	overall	≥ 10 dB
(a)	ResNet (AWGN) on AWGN test	0.640	0.806
(a)	ResNet (AWGN)	0.354	0.384
(b)	ResNet (Rayleigh)	0.436	0.517
(b)	ResNet (Rayleigh) on AWGN test	0.487	0.564
(c)	ResNet (50% AWGN + 50% Rayleigh)	**0.485**	0.599
(c)	ResNet (mixed) on AWGN test	0.591	0.731
(d)	random forest, 21 features (AWGN)	0.351	0.472
(d)	random forest, 21 features (Rayleigh)	0.419	**0.622**
(e)	CMA + ResNet (Rayleigh)	0.432	0.556
(e)	CMA + random forest (Rayleigh)	0.421	0.564
(f)	dilated ResNet (Rayleigh)	0.451	0.534
(f)	CLDNN on I/Q (Rayleigh)	0.145	0.162

34.6 Which classes become confusable, and why

Figure 34.10 (right) shows the Rayleigh-trained ResNet at 16 dB. Three groups emerge:

1. **GFSK and CPFSK stay recognisable** (95% and 96%). Their information is in the instantaneous frequency, and a channel whose coherence bandwidth is comparable to the signal bandwidth mostly changes the *amplitude* of a constant-envelope signal, leaving the frequency trajectory visible. (Their envelope is no longer constant after multipath, so $\gamma_{\max}$ alone would not suffice; the network uses the time structure.)
2. **BPSK and PAM4 merge** (56% of PAM4 called BPSK). Both are real, one-dimensional constellations; $\tilde{C}_{20}$ still says "real" (it is reduced but not destroyed, Figure 34.5), but the amplitude levels that distinguish ± 1 from $\{\pm 1, \pm 3\}$ are exactly what ISI smears: each sample becomes a random sum of neighbours whose amplitude distribution looks the same for both.

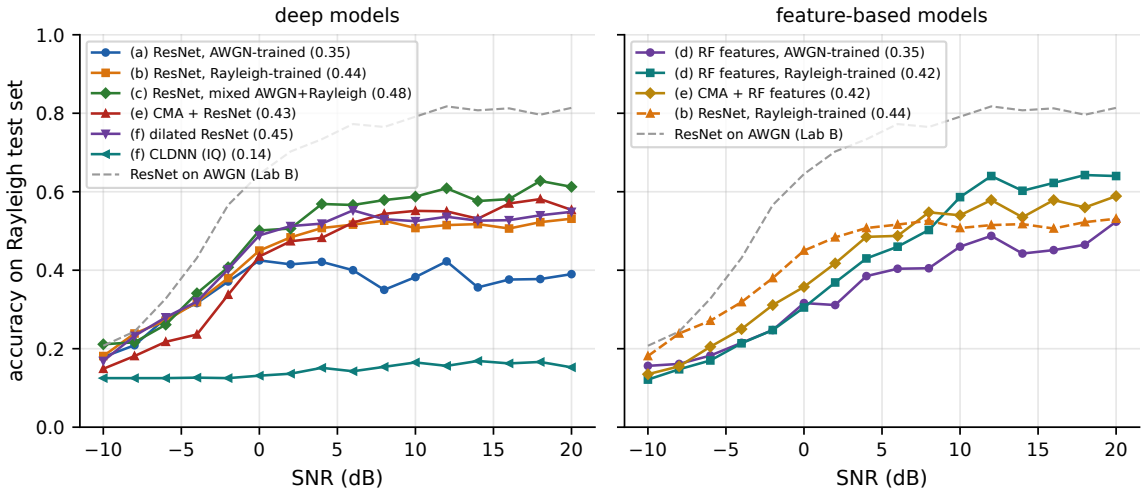


Figure 34.9: Accuracy vs SNR on the Rayleigh test set for experiments (a)–(f). The AWGN-trained network (a) flattens out far below every model trained with multipath. Dashed gray: the same ResNet on the AWGN test set (Lab B), the ceiling that multipath takes away.

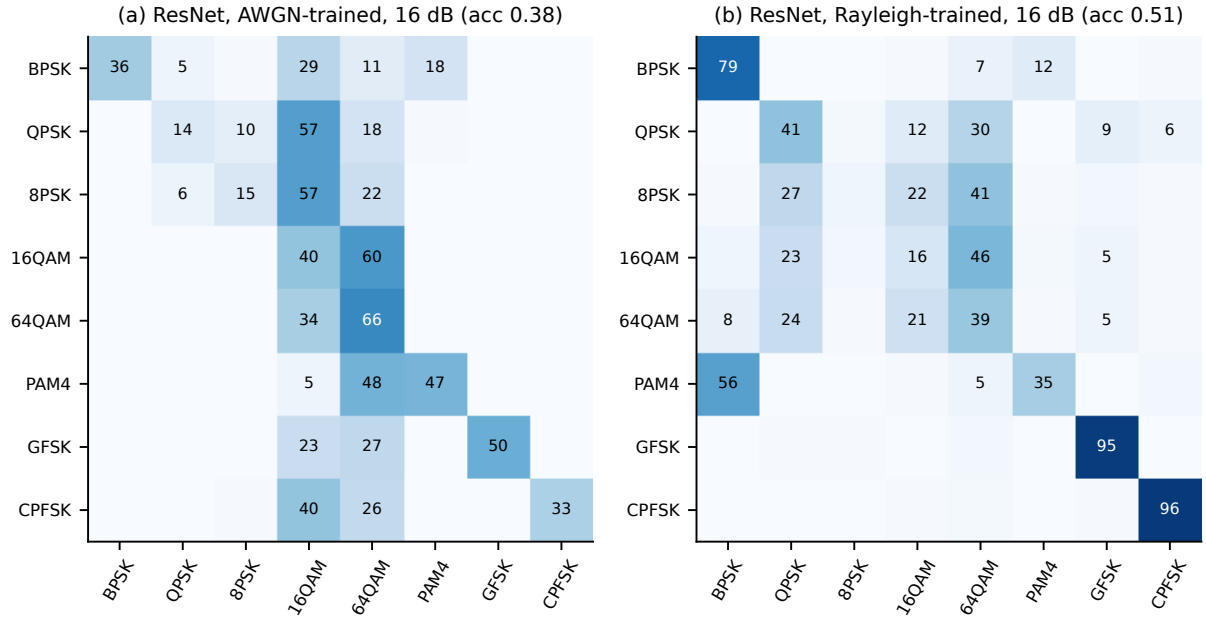


Figure 34.10: Confusion matrices at 16 dB: the AWGN-trained ResNet on Rayleigh data (left) and the Rayleigh-trained ResNet (right). Left: the mismatched model pushes many signals into a few classes. Right: remaining errors form two blocks — the circular linear classes (QPSK, 8PSK, 16QAM, 64QAM) and the "real" pair BPSK/PAM4.

3. **QPSK, 8PSK, 16QAM, 64QAM form one block.** Here eq. (34.6) is the whole story: multipath multiplies the 4th-order cumulants by a random $\rho \in (0, 1]$, so a QPSK signal through a two-path channel ($\rho \approx 0.5$) has the statistics of a QAM signal through a one-path channel. No classifier that sees one 16-symbol example can know which of the two happened. The network spreads its answers over QPSK, 16QAM and 64QAM and almost never says 8PSK.

The accuracy vs delay spread (Figure 34.11) confirms the mechanism: every model loses accuracy as τ grows, because more taps mean smaller ρ and more mixing.

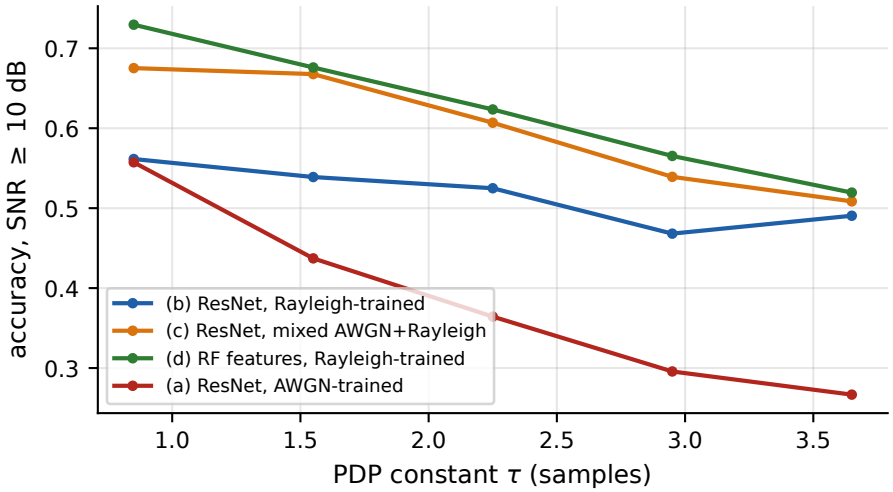


Figure 34.11: Accuracy at SNR ≥ 10 dB versus the PDP constant τ of each test example's channel. The AWGN-trained network (red) falls from 0.56 to 0.27 as the delay spread grows; models trained with multipath degrade less, and the feature-based forest is best for short profiles.

Wireless connection: what real systems do

Practical receivers do not classify raw multipath signals: they synchronise and equalise first (pilots, preambles, or blind algorithms like CMA), or they use features that are invariant to the channel (e.g. cyclostationary features for OFDM). For a passive scanner in 5G NR, the SSB and DMRS give known references that remove most of this problem — but for unknown emitters, the methods in this lab are what you have. O'Shea, Roy and Clancy (2018) report that their deep networks degrade under fading and benefit from training on matching channel models, which is the same qualitative lesson as experiments (a)–(c).

Chapter recap

- Multipath = tapped delay line $v[n] = \sum_l g_l[n]s[n-l]$; Rayleigh taps $g_l \sim \mathcal{CN}(0, p_l)$ from the CLT; exponential PDP; delay spread σ_τ; coherence bandwidth $\sim 1/(5\sigma_\tau)$; Doppler and coherence time $\sim 0.423/f_D$.
- A linear channel multiplies $\tilde{C}_{42}$ by $\rho = \sum |h|^4 / (\sum |h|^2)^2 \leq 1$ (random per example) and scrambles $\tilde{C}_{20}$ and $\tilde{C}_{40}$ through the tap phases: the cumulant clusters smear along rays toward the origin.
- Results: AWGN-trained ResNet collapses from 0.81 to 0.38 at high SNR; training on Rayleigh gives 0.52, mixed AWGN+Rayleigh training 0.60 (best network); a random forest on features retrained on Rayleigh reaches 0.62 at high SNR but is weaker at low

SNR; CMA with 16 symbols helps the network slightly and hurts the features; a dilated ResNet helps slightly; a CLDNN on I/Q failed to train in our budget.

- Lesson: train on the channel you will test on; blind equalisation helps features more than networks; the remaining confusions are among the circular linear classes and BPSK/PAM4.

Exercises

1. Compute σ_τ for an exponential PDP with $\tau = 2$ and $L = 6$ taps.
2. For $h = (0.8,\ 0.6j)$ compute ρ and the resulting $\tilde{C}_{42}$ of QPSK and 16QAM (symbol rate, no noise). Which one now looks like the other?
3. Show that $|C_{20}|$ of BPSK after a two-tap channel $h = (h_0, h_1)$ is $\left|h_0^2 + h_1^2\right| / (|h_0|^2 + |h_1|^2)$. When is it zero?
4. Derive the CMA update from J using $\partial\,|y|^2 / \partial w_i^* = y\,u_i^*$.
5. The maximum Doppler of the generator is 2×10^{-3} cycles/sample. How many coherence times fit in one 128-sample example? In 1024 samples?
6. Why did the CMA front end help the random forest more than the ResNet?
7. (Coding) Retrain experiment (b) with $f_D = 0$ (block fading) and test on the Doppler test set. What changes?
8. (Coding) Replace CMA by a decision-directed LMS after CMA convergence and measure the effect on the 16QAM/64QAM confusion.

Answers

1. Weights $e^{-l/2}$: $1, 0.607, 0.368, 0.223, 0.135, 0.082$; sum 2.415; $p = (0.414, 0.251, 0.152, 0.092, 0.056, 0.034)$. $\mathbb{E}[l] = 0.251 + 0.304 + 0.276 + 0.224 + 0.170 = 1.225$; $\mathbb{E}[l^2] = 0.251 + 0.608 + 0.828 + 0.896 + 0.850 = 3.433$; $\sigma_\tau = \sqrt{3.433 - 1.501} = 1.39$ samples (shorter than $\tau = 2$ because the profile is truncated).
2. $|h|^2 = (0.64, 0.36)$; $\sum |h|^4 = 0.4096 + 0.1296 = 0.539$; $\rho = 0.539$. QPSK: $1 \times 0.539 = 0.539$; 16QAM: $0.68 \times 0.539 = 0.367$. QPSK now has a smaller $\tilde{C}_{42}$ than noise-free 16QAM (0.68) and 64QAM (0.619) — it looks like a QAM.
3. $y = h_0 a_k + h_1 a_{k-1}$ with real independent a: $\mathbb{E}[y^2] = h_0^2 + h_1^2$ (cross terms vanish), $\mathbb{E}\,|y|^2 = |h_0|^2 + |h_1|^2$. It is zero when $h_1^2 = -h_0^2$, i.e. equal magnitudes and a 90° phase difference.
4. $J = \mathbb{E}[(|y|^2 - R_2)^2]$; $\partial J / \partial w_i^* = \mathbb{E}[2(|y|^2 - R_2)\,\partial\,|y|^2 / \partial w_i^*] = 2\mathbb{E}[(|y|^2 - R_2) y u_i^*]$. Steepest descent with step $\mu/2$ and the expectation replaced by the instantaneous value: $w_i \leftarrow w_i - \mu e_k u_{k,i}^*$.
5. $T_c = 0.423/0.002 = 212$ samples: $128/212 = 0.6$ coherence times; $1024/212 \approx 4.8$. Longer records improve feature variance (Lab A) but violate the "static channel" assumption.
6. The random forest's features are the very statistics that ISI corrupts (eq. 34.6); undoing ISI restores them. The ResNet trained on Rayleigh data already learns to classify despite

ISI, and CMA adds its own distortions (self-noise, imperfect convergence on 16 symbols, phase ambiguity) that partially cancel the gain.

7. Typically accuracy on Doppler test data falls somewhat at high SNR: the model never saw a channel that changes within the window. Matching the training channel's time variation matters as much as its delay spread.
8. Decision-directed LMS after CMA usually tightens the QAM clusters, but with only 16 symbols and a 90° phase ambiguity the gain is modest; for 16QAM vs 64QAM the 16-symbol limit of Lab A remains.

Self-test quiz

1. Why are multipath taps modelled as complex Gaussian?
   Each tap is a sum of many paths with random phases; by the CLT the sum is approximately $\mathcal{CN}(0, p_l)$.
2. By what factor does a channel h multiply $\tilde{C}_{42}$?
   $\rho = \sum |h_m|^4 / (\sum |h_m|^2)^2 \leq 1$.
3. What does the CMA cost penalise?
   Deviation of the output's squared magnitude from a constant R_2.
4. Which experiment shows the worst result, and why?
   Training on AWGN and testing on Rayleigh: train/test mismatch.
5. Name the two groups of classes that stay confused under multipath.
   The circular linear classes (QPSK, 8PSK, 16QAM, 64QAM) and the real pair BPSK/PAM4.

Further reading

- T. J. O'Shea, T. Roy and T. C. Clancy, "Over-the-air deep learning based radio signal classification," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, 2018 (arXiv:1712.04578). Studies deep classifiers under impairments including multipath fading.
- O. A. Dobre, A. Abdi, Y. Bar-Ness and W. Su, "Survey of automatic modulation classification techniques," *IET Communications*, vol. 1, no. 2, 2007. Discusses the effect of fading channels on likelihood and feature methods.
- A. Swami and B. M. Sadler, "Hierarchical digital modulation classification using cumulants," *IEEE Trans. Commun.*, vol. 48, no. 3, 2000. Includes an analysis of cumulants under channel effects.
- D. N. Godard, "Self-recovering equalization and carrier tracking in two-dimensional data communication systems," *IEEE Trans. Communications*, vol. 28, no. 11, pp. 1867–1875, 1980. The origin of the constant-modulus idea.
- T. J. O'Shea, J. Corgan and T. C. Clancy, "Convolutional radio modulation recognition networks," *EANN*, 2016. The baseline CNN.

Chapter 35

Lab D: Deep Learning-Based Modulation Classification in IIR-Type Channels

In Lab C the channel was a *finite* sum of echoes: after L samples the past was forgotten. In this last lab the channel is a filter with *feedback*: every output sample depends on previous output samples, so an input sample echoes forever, decaying geometrically. Such *infinite impulse response* (IIR) channels are compact models of resonant behaviour — a few numbers describe an impulse response that is, in principle, infinitely long.

You will learn what an IIR filter is from first principles (difference equation, transfer function, poles and zeros, stability, long tails, resonant frequency responses), where IIR-type models honestly arise in radio, and why they are harder than FIR multipath. Then the competition: the effect of the pole radius on accuracy, the mismatch of training on FIR and testing on IIR, IIR augmentation, architectures with larger receptive fields, and an equaliser front end *learned end-to-end* together with the classifier — both a shared learnable complex FIR layer and a per-example "radio transformer"-style equaliser. We close Part VI with a comparison table of all four labs.

35.1 The competition

Same 8 classes, 128 samples, SNR grid, sizes and files (`iir_train`, `iir_test`), same metric. The gray channel block of Figure 32.1 is now a random stable IIR filter, drawn anew for every example. The generator also records each example's largest pole radius, which we use for analysis (it is *not* available to a competitor at test time).

35.2 IIR filters from first principles

Intuition: a room that rings

Strike a bell: the sound continues long after the hammer has left, fading slowly. A bell is a resonator — energy is fed back into the same vibration again and again, losing a fixed fraction per cycle. An IIR filter is the discrete-time bell: part of each output is fed back into the next one. How long it rings is set by how much is lost per step; at what frequency it rings, by the phase of the feedback.

35.2.1 Difference equation and transfer function

Definition: IIR filter

A linear time-invariant system described by the difference equation

$$v[n] = \sum_{k=0}^{M} b_k \, s[n-k] - \sum_{k=1}^{P} a_k \, v[n-k], \tag{35.1}$$

with $P \geq 1$ feedback (autoregressive) coefficients a_k and $M+1$ feed-forward (moving-average) coefficients b_k, all complex in general.

The z-transform. Define $S(z) = \sum_n s[n] z^{-n}$ (a delay by one sample multiplies the transform by z^{-1}). Transforming both sides of (35.1): $V(z) = \sum_k b_k z^{-k} S(z) - \sum_k a_k z^{-k} V(z)$. Collect $V(z)$:

Key equation: transfer function of an IIR channel

$$H(z) = \frac{V(z)}{S(z)} = \frac{B(z)}{A(z)} = \frac{\sum_{k=0}^{M} b_k z^{-k}}{1 + \sum_{k=1}^{P} a_k z^{-k}} = b_0 \frac{\prod_{i=1}^{M} (1 - z_i z^{-1})}{\prod_{i=1}^{P} (1 - p_i z^{-1})}. \tag{35.2}$$

The roots z_i of B are the *zeros* (frequencies the channel suppresses) and the roots p_i of A are the *poles* (frequencies where it resonates). An FIR multipath channel is the special case $A(z) = 1$: zeros only.

35.2.2 Impulse response of one pole

Take $H(z) = \frac{1}{1 - p z^{-1}}$, i.e. $v[n] = s[n] + p\, v[n-1]$. Feed in an impulse $s[n] = \delta[n]$: $v[0] = 1$, $v[1] = p$, $v[2] = p^2, \ldots$

$$h[n] = p^n, \quad n \geq 0. \tag{35.3}$$

Write $p = r e^{j\omega_0}$. Then $h[n] = r^n e^{j\omega_0 n}$: a complex sinusoid at frequency ω_0 (the *resonance*) with an exponentially decaying envelope r^n. The impulse response is *infinite*, but it decays if $r < 1$.

Definition: stability

A causal IIR filter is bounded-input bounded-output stable if and only if all poles lie strictly inside the unit circle, $|p_i| < 1$. (For one pole: $\sum_n |h[n]| = \sum_n r^n = 1/(1-r)$ is finite iff $r < 1$.)

How long is "infinite"? The energy of the tail beyond n_0 is $\sum_{n \geq n_0} r^{2n} = r^{2n_0}/(1 - r^2)$, a fraction r^{2n_0} of the total. 99% of the energy is contained in the first $n_{99} = \lceil \ln 0.01 / (2 \ln r) \rceil$ samples.

Worked example: memory of a pole

$r = 0.5$: $n_{99} = \lceil -4.605/(2 \cdot -0.693) \rceil = \lceil 3.3 \rceil = 4$ samples — less than a symbol. $r = 0.9$: $\lceil -4.605/(2 \cdot -0.105) \rceil = \lceil 21.9 \rceil = 22$ samples (2.7 symbols). $r = 0.99$: 230 samples — longer than our whole example. Memory grows like $1/(1-r)$ as $r \to 1$.

35.2.3 Frequency response: resonances

On the unit circle $z = e^{j2\pi f}$:

$$\left| H(e^{j2\pi f}) \right|^2 = \frac{1}{\left| 1 - r e^{j(\omega_0 - 2\pi f)} \right|^2} = \frac{1}{1 - 2r\cos(2\pi f - \omega_0) + r^2}. \tag{35.4}$$

At $2\pi f = \omega_0$ this is $1/(1-r)^2$; opposite, $1/(1+r)^2$. The peak-to-valley ratio is $\left(\frac{1+r}{1-r}\right)^2$: for $r = 0.9$, $(1.9/0.1)^2 = 361$, i.e. 25.6 dB. The -3 dB width of the peak is approximately $(1-r)/\pi$ cycles/sample: 0.032 for $r = 0.9$, narrower than our signal band (± 0.084). A pole close to the unit circle near the signal band creates a sharp resonant peak that dominates the received spectrum.

35.2.4 The channel in our generator

`modsim.random_iir` draws $P \in \{1, 2\}$ poles with radius $\mathcal{U}[0.3, 0.9]$ and uniformly random angle, and $M \in \{0, 1, 2\}$ zeros with radius $\mathcal{U}[0, 0.9]$ and random angle (so the filter is also minimum-phase), forms $A(z) = \prod(1 - p_i z^{-1})$ and $B(z) = \prod(1 - z_i z^{-1})$, and scales B so that the impulse response has unit energy, $\sum_n |h[n]|^2 = 1$. To make the infinite memory realistic, 384 extra samples are generated before the 128 kept, so the filter is in steady state. Figure 35.1 shows the structure (direct form I).

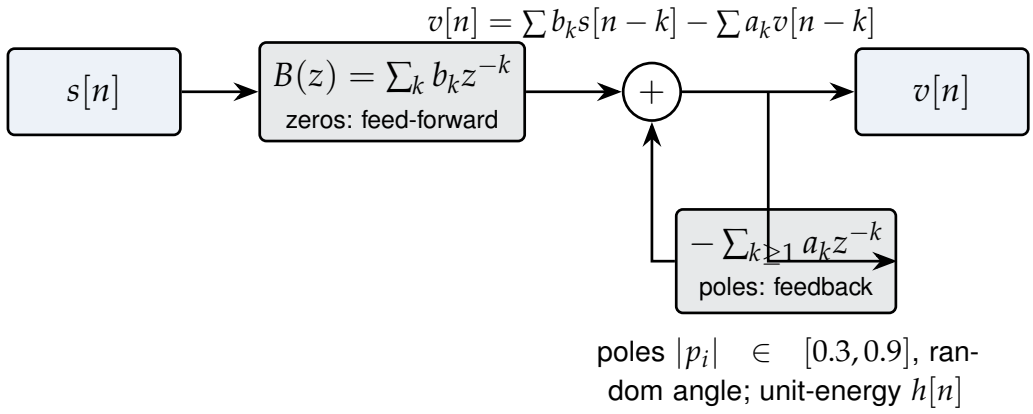


Figure 35.1: An IIR channel: the feed-forward part $B(z)$ (zeros) is an FIR filter like Lab C's multipath; the feedback loop $A(z)$ (poles) makes the impulse response infinitely long. Every example of Lab D has its own random B and A.

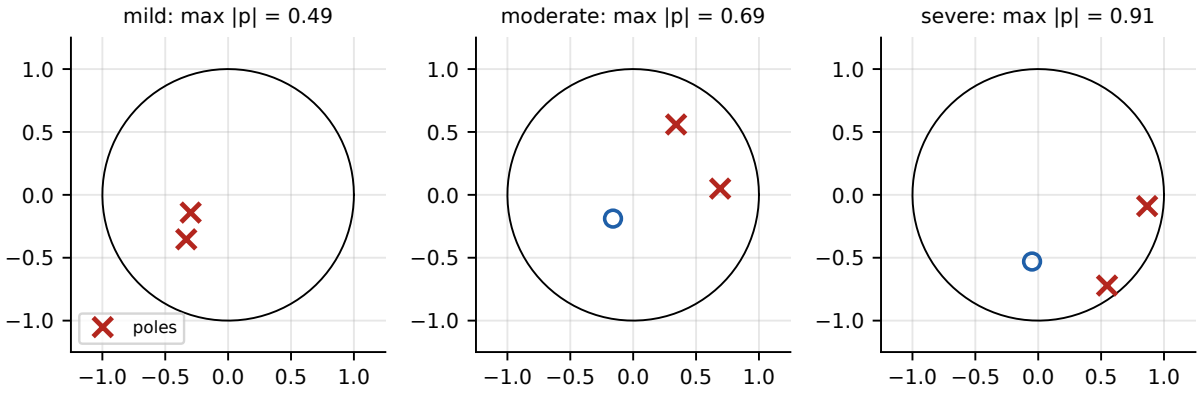


Figure 35.2: Pole–zero plots of three random channels from the generator (two poles, ≤ 1 zero), with the largest pole radius increasing from left to right. The closer a pole sits to the unit circle, the longer the channel rings and the sharper its resonance.

35.3 Where do IIR-type channels arise?

Be careful here: the standard models for *over-the-air* propagation (3GPP TDL/CDL, ITU profiles) are FIR tapped delay lines, because physical propagation paths have finite delays. An IIR model is a *compact approximation* of a long impulse response, or the exact model of a system with feedback. Honest examples:

- **Front-end and IF filters.** Analog filters in the receive chain (LC, SAW, ceramic) are rational transfer functions with poles — exactly IIR. A narrow, high-Q filter whose band edge falls

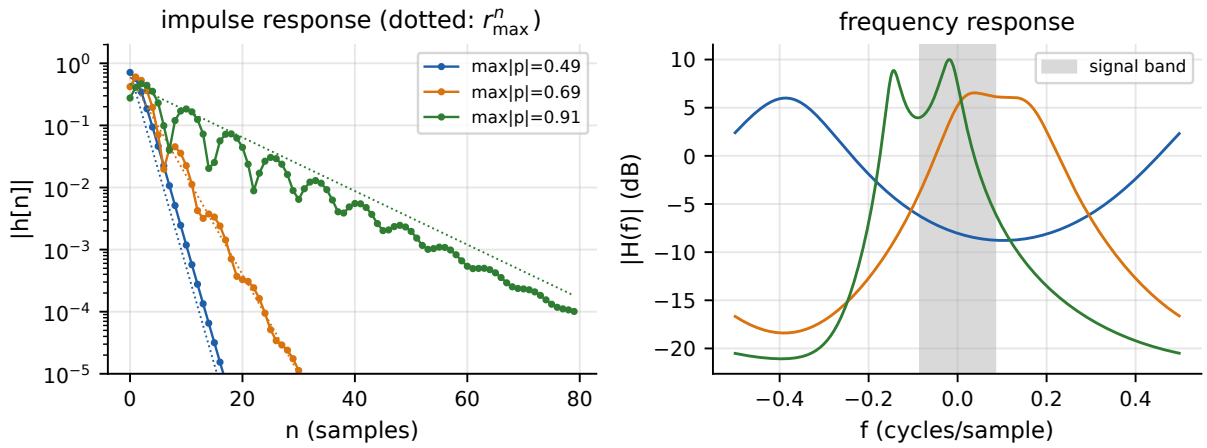


Figure 35.3: The same three channels. Left: $|h[n]|$ on a log scale — straight-line decay with slope set by the largest pole radius (dotted guides $r_{\max}^n$); for $r_{\max} = 0.91$ the response is still 10^{-3} after 80 samples (10 symbols). Right: frequency responses; the gray band is the signal bandwidth. Poles near the unit circle create resonant peaks of 15–20 dB inside or near the band.

on the signal produces ringing and group-delay distortion; a mis-tuned filter is an IIR channel.

- **Feedback and coupling.** On-frequency repeaters with imperfect isolation between their antennas feed part of their output back into their input: $v[n] = s[n] + \beta v[n - D]$, an IIR comb filter with poles of radius $|\beta|^{1/D}$. Self-interference loops and RF coupling in compact devices behave similarly.
- **Resonant or reverberant environments.** Ducts, tunnels, metal enclosures and reverberation chambers produce impulse responses with long exponential tails. Exponential tails are what a pole produces, so a few poles model them far more compactly than hundreds of FIR taps. (Acoustic room models use IIR reverberators for the same reason.) Whether a specific propagation environment is better modelled with poles is an empirical question.
- **Equalisers and channel models in DSP chains** (e.g. AR models of fading processes, cascaded IIR pre-emphasis) introduce poles into the effective end-to-end response.

We use random IIR filters as a *stress test*: a family of channels with long memory and sharp spectral structure, described by few parameters.

35.4 Why IIR is harder than FIR multipath

1. **Infinite memory.** With $r = 0.9$, 99% of the energy needs 22 samples; FIR taps in Lab C rarely exceeded 20 samples, and their PDP decayed with $\tau \leq 4$. More important is the *shape*: a pole near the band produces a resonance whose response mixes symbols over a long window.
2. **Resonance dominates the spectrum.** Unit-energy normalisation means that a strong resonant peak takes most of the power, so the received signal looks like a narrow-band process near ω_0, whatever the modulation was.
3. **Equalisation needs the inverse.** The ideal equaliser $1/H(z) = A(z)/B(z)$ has the poles' polynomial $A(z)$ *in the numerator*: a short FIR filter (e.g. $1 - pz^{-1}$ for one pole)! Paradoxically, a pure-pole channel is easy to invert — *if you know* p. The zeros of $B(z)$ become poles of the

inverse, which an FIR equaliser can only approximate. This motivates the learned equaliser front end of Section 35.5.4.

4. **Mismatch.** Networks trained on short FIR channels have never seen ringing, so their learned invariances do not cover it.

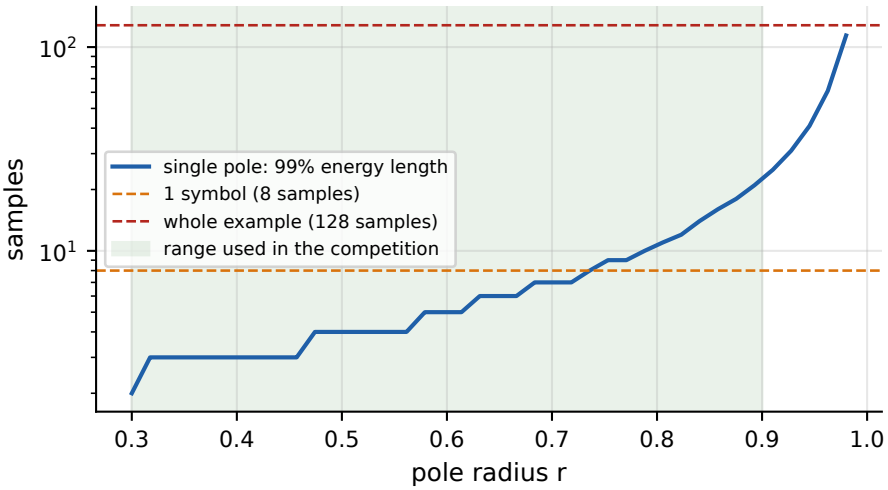


Figure 35.4: Length containing 99% of the impulse-response energy of a single pole versus its radius r. Within the competition's range ($0.3 \le r \le 0.9$, green), the memory goes from less than one symbol to almost three symbols; beyond $r \approx 0.98$ it exceeds the whole 128-sample example.

Derivation of ρ for one pole. With $h[n] = cp^n$, $\sum |h|^2 = |c|^2 / (1 - r^2)$ and $\sum |h|^4 = |c|^4 / (1 - r^4)$, so $\rho = \frac{(1-r^2)^2}{1-r^4} = \frac{1-r^2}{1+r^2}$. This formula is exact for i.i.d. inputs *at the filter's sample rate*; our inputs are RRC-shaped (strongly correlated over 8 samples), so the measured shrinkage in Figure 35.6 is much milder. The figure is an honest warning against applying symbol-rate intuition to oversampled signals.

35.5 Experiments

As in Lab C, every model is trained with the Lab B recipe (at most 10 epochs, 15% validation) on 16 320 IIR training examples; script `code/ch35/lab_d_train.py`.

35.5.1 (a) Mismatch: AWGN- and FIR-trained networks on IIR data

The Lab B ResNet (trained on AWGN) scores 0.524 overall and 0.634 at high SNR on the IIR test set; a ResNet trained on Lab C's FIR Rayleigh data scores *less*, 0.488 and 0.571. Two lessons. First, the collapse is milder than in Lab C (0.384): with our generator, a random IIR channel is on average *less* frequency-selective inside the signal band than a random Rayleigh channel (over 2000 random draws the median spread of $|H(f)|$ across the band is 4.7 dB for IIR versus 8.8 dB for Rayleigh; 18% versus 43% of channels exceed 10 dB). Many IIR draws put their poles away from the band, where they only tilt the spectrum. Second, training on "some other hard channel" is not a substitute for the right one: the FIR-trained model learned invariances to random multipath that do not transfer to ringing resonances, and it lost what it knew about clean signals. The breakdown by pole radius (Figure 35.11) shows the AWGN-trained model falling from 0.70 (largest pole < 0.5) to 0.55 ($0.8 \le r < 0.9$): the closer the poles to the unit circle, the larger the mismatch.

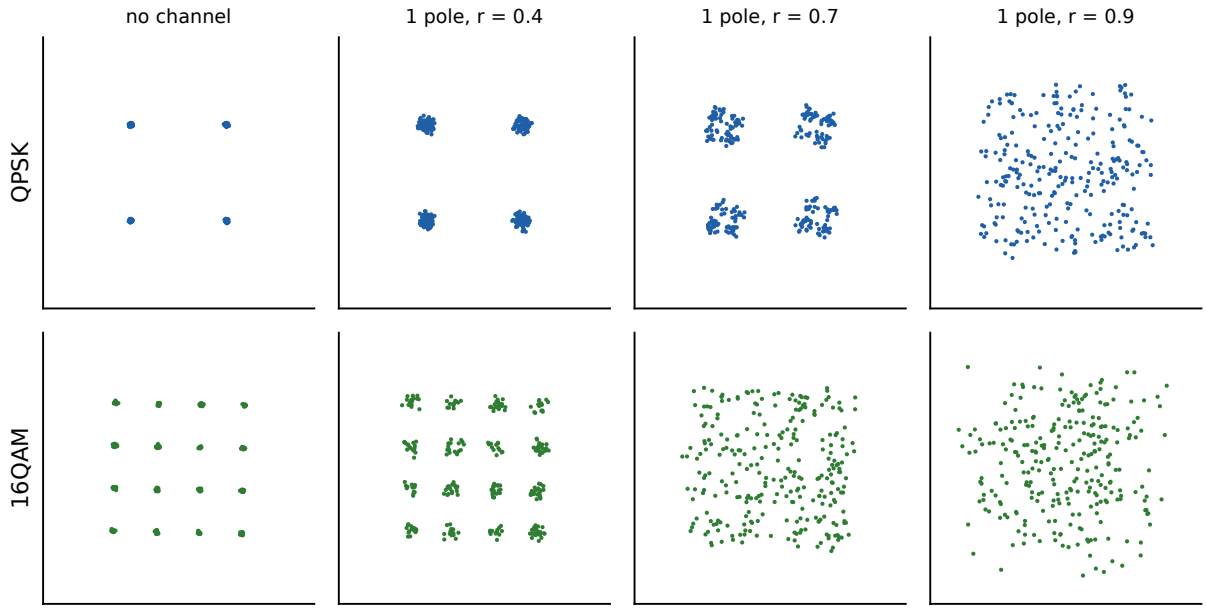


Figure 35.5: Matched-filtered symbol constellations (30 dB, long records) after a single pole at $f = 0.04$ cycles/sample (inside the signal band) with radius 0.4, 0.7 and 0.9. QPSK clusters spread with r and at $r = 0.9$ the four points have merged; 16QAM is unrecognisable already at $r = 0.7$.

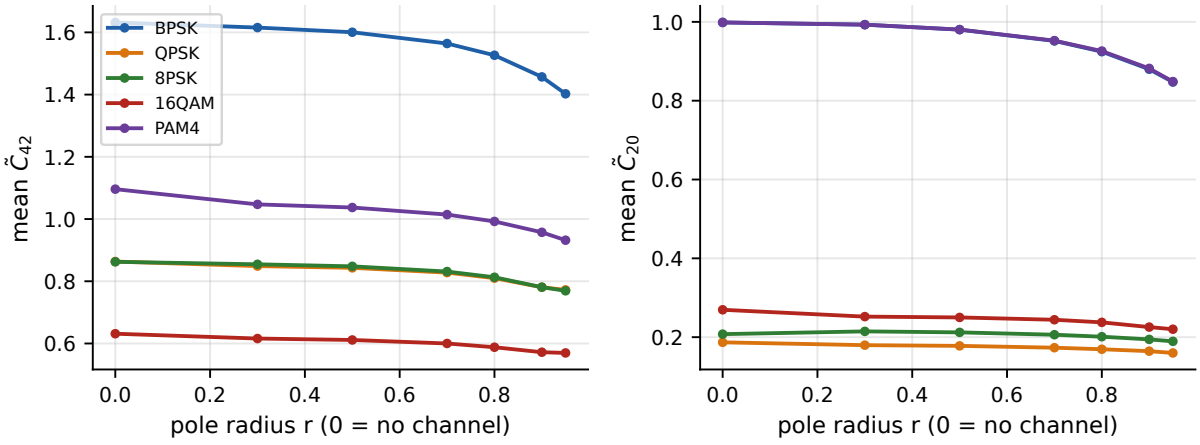


Figure 35.6: Noise-free class means of $\tilde{C}_{42}$ and $\tilde{C}_{20}$ through single-pole channels of random angle, versus the pole radius. The decline is gentle — compare with Lab C's Figure 34.5: with unit-energy normalisation the one-pole filter $h[n] \propto p^n$ gives $\rho = \sum |h|^4 / (\sum |h|^2)^2 = (1 - r^2)/(1 + r^2)$, which is 0.86 at $r = 0.3$ but 0.10 at $r = 0.9$ — yet the measured feature falls far less, because the RRC-shaped signal is already band-limited and the per-sample (not per-symbol) mixing matters.

35.5.2 (b) Training with IIR channels

The same ResNet trained on IIR data reaches 0.590 overall and 0.741 at high SNR — 11 points above the AWGN-trained model on the plateau, and only 6.5 points below the AWGN ceiling of Lab B (0.806). Its accuracy still declines with the pole radius, from 0.78 to 0.68 (Figure 35.11): longer memory and sharper resonances cost about ten points even when the model has seen them in training.

35.5.3 (c) Larger receptive fields: dilated CNN and CLDNN

Figure 35.7 shows why dilation helps in principle.

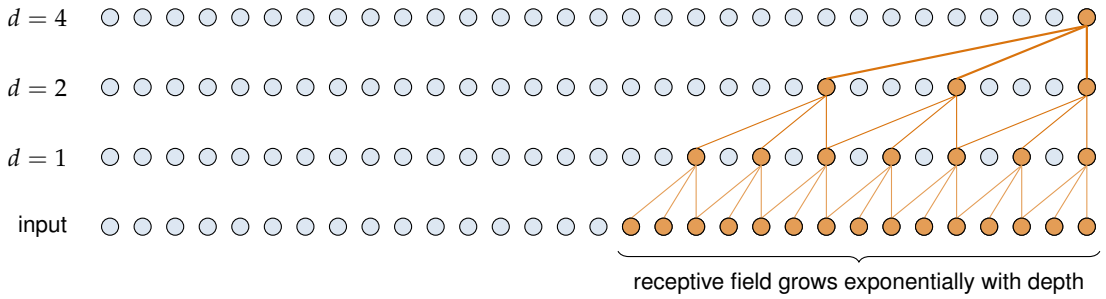


Figure 35.7: Dilated convolutions (causal, kernel 3, dilations 1, 2, 4; orange = everything the top output depends on): each layer skips $d - 1$ samples between taps, so three layers see 15 input samples with the same number of weights that would see only 7 without dilation. Stacking doubling dilations is the cheapest way to give a CNN the long memory that an IIR channel demands.

The dilated ResNet (receptive field covering the whole window) scores 0.585 overall and 0.722 at high SNR: no better than the plain ResNet overall, but its accuracy is *flatter* across pole radii (0.72 to 0.69 versus 0.78 to 0.68) — it is slightly better on the longest-memory channels, as the receptive-field argument predicts, and slightly worse on short ones. The CLDNN on I/Q trained this time (unlike in Lab C) but only to 0.402 overall and 0.481 at high SNR within the 10-epoch budget, the weakest IIR-trained model. A longer receptive field is not automatically a better model: the optimisation, the inductive bias of convolutions and the training budget matter as much.

35.5.4 (d) A learned equaliser front end, trained end-to-end

Intuition: let the classifier tell the equaliser what matters

A classical equaliser minimises a signal-domain cost (MSE, constant modulus) that is only indirectly related to classification. If we put a differentiable equaliser in front of the classifier and backpropagate the *classification* loss into its taps, it will learn whatever filtering makes the classes easiest to tell apart — which may or may not be the channel inverse.

We try two front ends (Figure 35.8):

1. **Shared learnable complex FIR** ($L = 15$ taps, initialised to the identity). Implemented as a 1-D convolution with the 2×2 real block structure of complex multiplication:

$$\begin{pmatrix} y_I \\ y_Q \end{pmatrix} = \begin{pmatrix} h_I & -h_Q \\ h_Q & h_I \end{pmatrix} * \begin{pmatrix} x_I \\ x_Q \end{pmatrix}. \tag{35.5}$$

One filter for all examples: it can only learn an "average" correction (e.g. a band-limiting filter); it cannot invert a channel that differs per example.

2. **RTN-style per-example equaliser.** Inspired by the *radio transformer network* idea of O'Shea and Hoydis (2017), in which a small network estimates the parameters of a known signal-processing transform that is then applied to the input before the main network. Here, a tiny CNN (2 strided convs, 16 channels, global pooling) looks at the received example and outputs the 15 complex taps of an FIR equaliser $\hat{\mathbf{h}}(\mathbf{r})$ for *that example*. The equaliser is applied, the result normalised to unit power and fed to the ResNet. The last layer of the estimator is initialised to zero, so training starts from "no equalisation". Everything is trained end-to-end with cross-entropy; no channel labels are used.

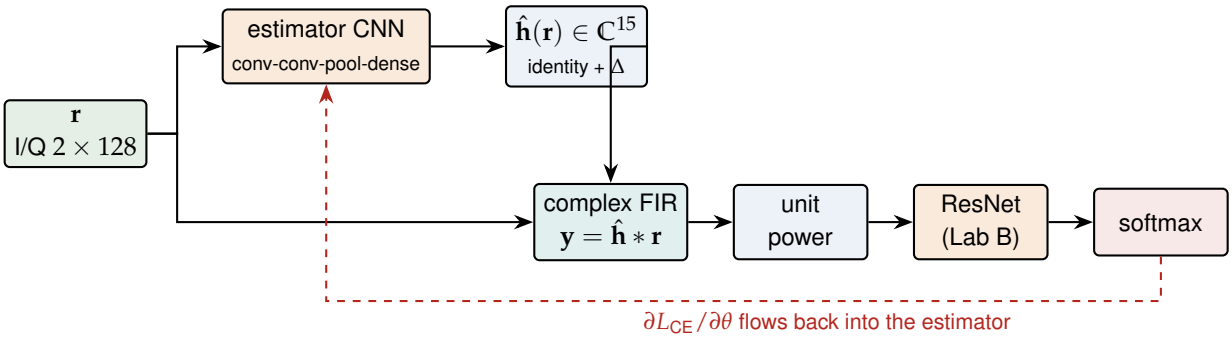


Figure 35.8: The RTN-style learned equaliser. A small estimator network predicts per-example equaliser taps; the equalised signal feeds the classifier; the cross-entropy gradient trains both. The shared-FIR variant replaces the estimator by a single learnable tap vector.

```python
class RTNEqualizer(nn.Module):
    """Radio-transformer-style front end: a small CNN looks at the example and outputs the taps of a
    per-example complex FIR equaliser (L taps), which is applied to the same example. Initialised to
    the identity filter so training starts from 'no equalisation'."""
    def __init__(self, L=15, ch=16):
        super().__init__()
        self.L = L
        self.est = nn.Sequential(nn.Conv1d(2, ch, 7, padding=3, stride=2), nn.ReLU(),
                                 nn.Conv1d(ch, ch, 5, padding=2, stride=2), nn.ReLU(),
                                 nn.AdaptiveAvgPool1d(1), nn.Flatten(), nn.Linear(ch, 2 * L))
        nn.init.zeros_(self.est[-1].weight); nn.init.zeros_(self.est[-1].bias)

    def taps(self, x):
        d = self.est(x).view(-1, 2, self.L)
        base = torch.zeros_like(d); base[:, 0, self.L // 2] = 1.0
        return base + d

    def forward(self, x):
        h = self.taps(x)                                        # (b, 2, L)
        b, _, N = x.shape
        xp = F.pad(x, (self.L // 2, self.L // 2))
        U = xp.unfold(2, self.L, 1)                             # (b, 2, N, L) sliding windows
        hr, hi = h[:, 0].flip(-1)[:, None, :], h[:, 1].flip(-1)[:, None, :]
        ur, ui = U[:, 0], U[:, 1]
```

```
        yr = (ur * hr).sum(-1) - (ui * hi).sum(-1)
        yi = (ur * hi).sum(-1) + (ui * hr).sum(-1)
        y = torch.stack([yr, yi], 1)
        return y / (y.pow(2).sum(1, keepdim=True).mean(-1, keepdim=True).sqrt() +
    1e-6)    # unit power
```

The shared complex FIR front end gives 0.575 overall and 0.725 at high SNR — slightly *below* the plain ResNet. That is expected: one filter for all examples cannot invert channels whose poles are at random angles; at best it learns a band-limiting pre-filter that the ResNet's first layer could learn anyway. The RTN-style per-example equaliser gives the best overall accuracy of the lab among networks, 0.595, a high-SNR accuracy of 0.738 (equal to the plain ResNet within noise), and the best network accuracy on the hardest channels, 0.712 for $0.8 \le r < 0.9$ versus 0.683 for the plain ResNet (Figure 35.11). It also gains 2–5 points at low SNR (-10 to -4 dB, Figure 35.10). The gains are small — of the order of the statistical uncertainty per bin ($\approx \pm 2$ points) — so treat them as suggestive, not conclusive.

What did the estimator learn? Figure 35.9 shows, for three random channels with poles of radius 0.7–0.86, the channel response, the equaliser response predicted by the estimator *for that example*, and their product. The learned equaliser is not a channel inverse: it is a band-pass filter that keeps the signal band, sharply suppresses out-of-band noise and resonances, and partially flattens the channel inside the band. That is exactly what the classification loss rewards — the classifier does not need the symbols back, only a cleaner view of the in-band waveform.

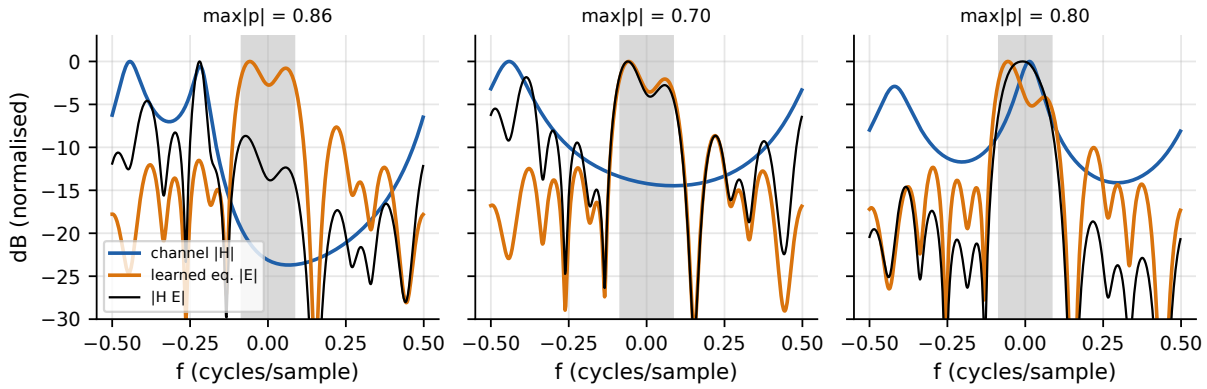


Figure 35.9: For three random IIR channels (QPSK at 20 dB): channel magnitude response $|H|$ (blue), the RTN estimator's per-example equaliser $|E|$ (orange), and the cascade $|HE|$ (black), each normalised to 0 dB peak; gray = signal band. The learned equaliser behaves as an adaptive band-pass filter that suppresses everything outside the band, rather than as an inverse of H.

35.6 Results

Three observations stand out.

1. **Training on the right channel is the biggest lever**: +11 points on the high-SNR plateau compared with AWGN training, more than any architecture change.
2. **The feature-based forest, retrained on IIR features, has the best high-SNR accuracy** (0.805) and is the most robust to the pole radius (0.76 at $0.8 \le r < 0.9$), but it is much weaker below 4 dB (e.g. 0.31 vs 0.52–0.53 at -2 dB). Its overall score (0.542) is therefore below all IIR-trained ResNets. As in Lab A, the expert features shine when the SNR is high enough for their sample statistics to be reliable; the networks shine in noise.

Table 35.1: Lab D leaderboard on the IIR test set. The last column is the accuracy at SNR ≥ 10 dB for the channels with the largest pole radius in $[0.8, 0.9)$.

Model (training data)	params	overall	≥ 10 dB	≥ 10 dB, $r \in [0.8, 0.9)$
ResNet (AWGN)	13 328	0.524	0.634	0.555
ResNet (FIR Rayleigh)	13 328	0.488	0.571	0.563
ResNet (IIR)	13 328	0.590	0.741	0.683
dilated ResNet (IIR)	13 328	0.585	0.722	0.691
CLDNN on I/Q (IIR)	21 128	0.402	0.481	0.442
shared FIR eq. + ResNet (IIR)	13 358	0.575	0.725	0.671
RTN-style eq. + ResNet (IIR)	15 374	**0.595**	0.738	0.712
random forest, 21 features (IIR)	—	0.542	**0.805**	**0.762**

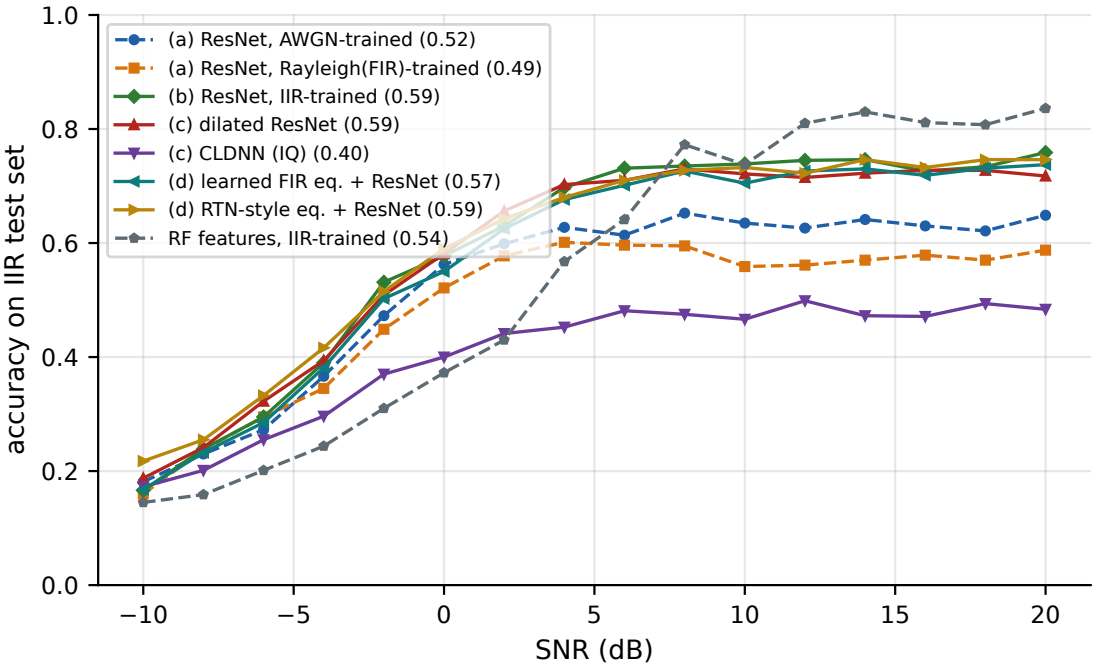


Figure 35.10: Accuracy vs SNR on the IIR test set. Dashed: mismatched models (trained on AWGN or on FIR Rayleigh) and the feature-based random forest; solid: networks trained on IIR data.

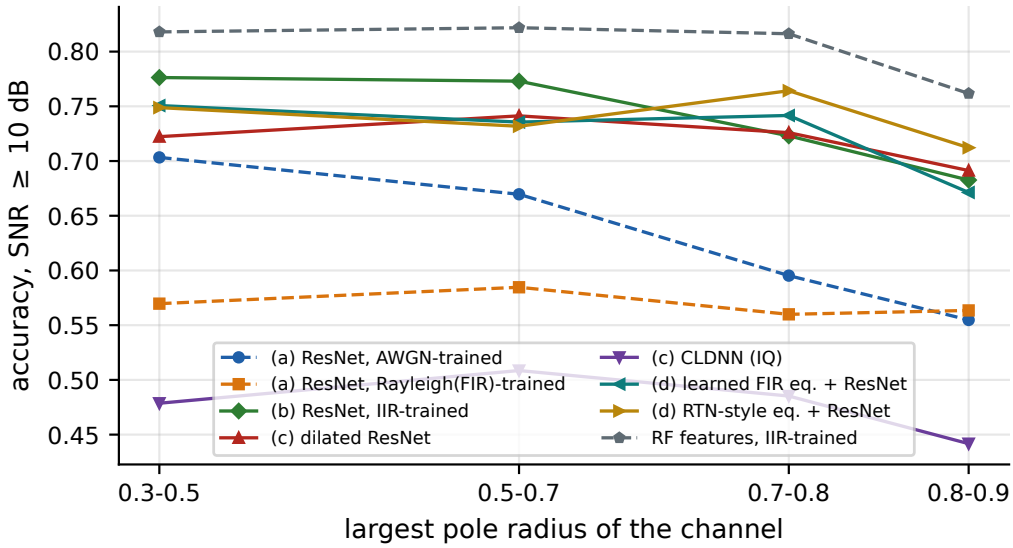


Figure 35.11: Accuracy at SNR $\geq$ 10 dB grouped by the channel's largest pole radius. Every model degrades as the poles approach the unit circle; the mismatched models degrade fastest.

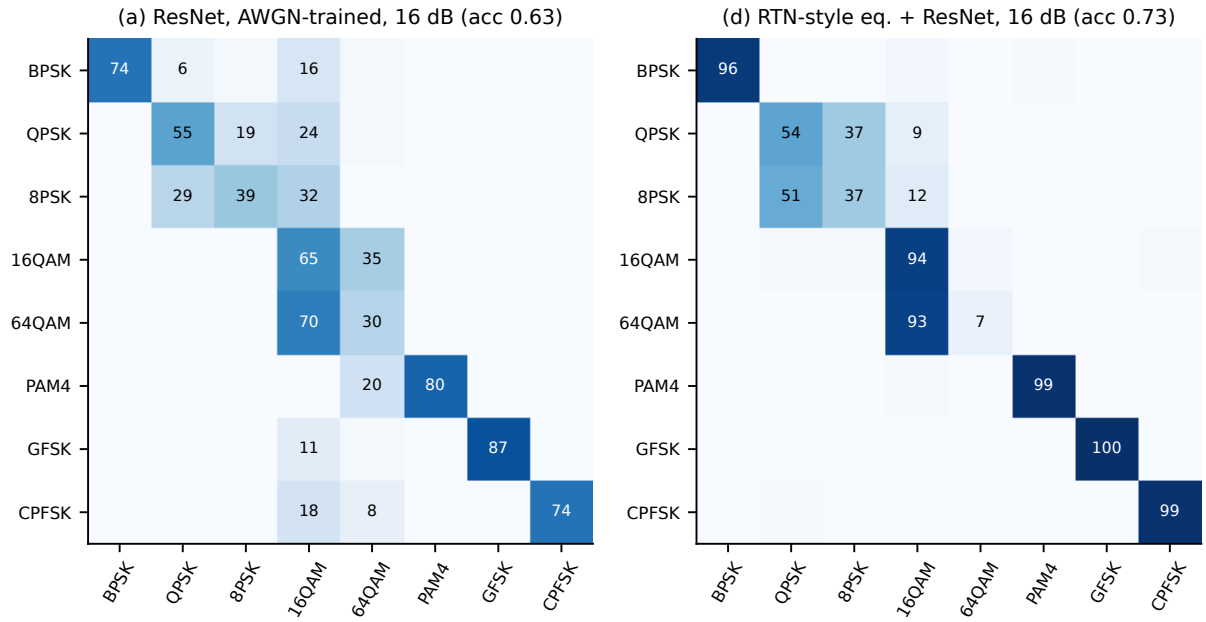


Figure 35.12: Confusion matrices at 16 dB on IIR data: the AWGN-trained ResNet (left) and the best IIR-trained model (right).

3. **The remaining confusions** (Figure 35.12, right) are QPSK/8PSK (each about half and half) and 16QAM/64QAM (almost all called 16QAM); BPSK, PAM4, GFSK and CPFSK are at 96–100%. The AWGN-trained model on the left additionally leaks QPSK/8PSK and FSK into the QAM classes, the same "everything looks more Gaussian" signature as in Lab C. A resonant channel rotates and stretches PSK clusters (Figure 35.5); separating 4 from 8 phases needs the fine phase structure that ringing smears first.

35.7 Part VI finale: four labs side by side

Table 35.2: All four labs. Overall accuracy (all SNRs) / accuracy at SNR ≥ 10 dB, on the test set of each channel. "Matching channel" means trained on data from the same channel family as the test set.

Method	AWGN (Labs A/B)	Rayleigh (Lab C)	IIR (Lab D)
Features + best classifier (retrained per channel)	0.588 / **0.878**	0.419 / **0.622**	0.542 / **0.805**
ResNet, trained on the matching channel	**0.640** / 0.806	0.436 / 0.517	0.590 / 0.741
Best deep variant on that channel	0.640 / 0.806	0.485 / 0.599[a]	**0.595** / 0.738[b]
Stack: ResNet + features	**0.678** / 0.875	—	—
ResNet trained on AWGN only	0.640 / 0.806	0.354 / 0.384	0.524 / 0.634

[a]mixed AWGN+Rayleigh training; [b]RTN-style learned equaliser + ResNet.

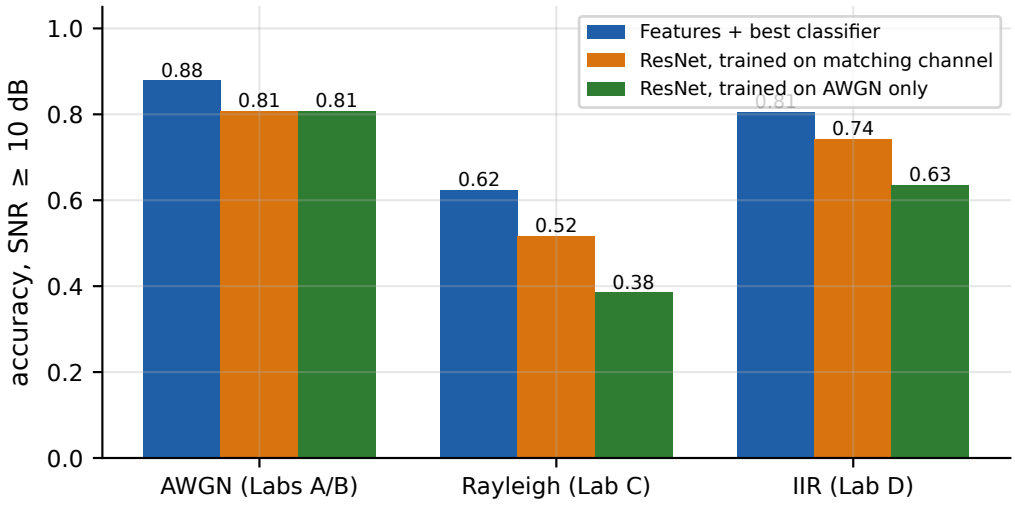


Figure 35.13: High-SNR accuracy (≥ 10 dB) of three approaches on the three channels: feature-based classifiers retrained on each channel (blue), the ResNet trained on the matching channel (orange) and the ResNet trained on AWGN only (green). The features keep the best high-SNR plateau everywhere; the AWGN-only network is fragile on both non-AWGN channels.

What the four labs teach together:

1. **Deep networks win in noise, expert features win at high SNR** — on every channel. The networks dominate below about 4 dB (by up to 24 points in AWGN); the retrained feature classifiers have the best high-SNR plateau on all three channels. Overall accuracy (all SNRs) favours the networks in AWGN and IIR and is close in Rayleigh. In Kaggle terms: the two families make different mistakes, so *stack them* — in AWGN that gave the best entry of all (0.678).

2. **The channel model in training matters more than the architecture.** AWGN-only training loses 42 points at high SNR on Rayleigh and 17 on IIR. Training on the right channel recovers most of it; mixing in benign conditions helped further on Rayleigh. Architecture changes (dilation, LSTM, learned equalisers) moved the scores by a few points at most.
3. **Classical DSP front ends need long records.** CMA, designed for thousands of symbols, gave mixed results with 16. The learned RTN-style equaliser became a per-example band-pass filter, not a channel inverse — it learned what helps classification.
4. **Some confusions are physics, not modelling.** 16QAM vs 64QAM is limited by 16 symbols in every lab; multipath merges all circular linear classes because it multiplies their 4th-order statistics by a random factor; resonances merge QPSK and 8PSK.

In absolute terms our numbers are for small CPU-trained models on a deliberately hard 8-class, 16-symbol task; the published works use far larger models and datasets. The *relative* lessons are the ones to carry into your own projects (Chapter 36).

Chapter recap

- IIR channel: $v[n] = \sum b_k s[n-k] - \sum a_k v[n-k]$, $H(z) = B(z)/A(z)$; poles $p = re^{j\omega_0}$ give $h[n] \propto r^n e^{j\omega_0 n}$ (infinite, decaying, ringing) and resonant peaks of height ratio $((1+r)/(1-r))^2$; stable iff all $|p| < 1$.
- Memory grows like $1/(1-r)$: $n_{99} = \lceil \ln 0.01/(2 \ln r) \rceil$; 4 samples at $r = 0.5$, 22 at 0.9.
- IIR models honestly describe front-end filters, feedback/coupling loops (repeaters), and compactly approximate reverberant environments; over-the-air standards use FIR TDLs.
- The inverse of an all-pole channel is a short FIR filter — if the poles were known; learned equaliser front ends try to estimate it implicitly.
- Results: AWGN-trained ResNet 0.63 at high SNR on IIR data (FIR-trained even worse, 0.57); IIR-trained ResNet 0.74; RTN-style learned equaliser best network overall (0.595) and on the hardest channels; feature-based forest retrained on IIR best at high SNR (0.81) but weak at low SNR. Accuracy falls as poles approach the unit circle for every model.

Exercises

1. Write the difference equation of $H(z) = \frac{1+0.5z^{-1}}{1-0.8z^{-1}}$ and compute $h[0..4]$.
2. For a single pole of radius $r = 0.8$ compute n_{99} and the peak-to-valley ratio of $|H|^2$ in dB.
3. Show that the impulse response of a pole pair p, p^* with $p = re^{j\omega_0}$ is a damped real cosine. (Our generator uses complex poles without conjugate pairs: what does that mean for the frequency response?)
4. Derive $\rho = (1-r^2)/(1+r^2)$ for $h[n] \propto p^n$ and evaluate it at $r = 0.5$ and $r = 0.9$.
5. Why is the ideal equaliser of an all-pole channel FIR? What is the ideal equaliser of $H(z) = 1 - 0.9z^{-1}$, and why is it harder?
6. A repeater with isolation 20 dB and loop delay $D = 10$ samples adds $v[n] = s[n] + 0.1v[n-10]$. Where are its poles and what is their radius?

7. (Coding) Restrict the generator to $r \in [0.9, 0.97]$ and retrain the IIR ResNet. How does the accuracy-vs-radius curve extend?
8. (Coding) Give the RTN estimator an auxiliary loss: predict the channel's largest pole radius (available in training data only). Does the classifier improve?

Answers

1. $v[n] = s[n] + 0.5s[n-1] + 0.8v[n-1]$. With $s = \delta$: $h[0] = 1$; $h[1] = 0.5 + 0.8 = 1.3$; $h[2] = 0.8 \cdot 1.3 = 1.04$; $h[3] = 0.832$; $h[4] = 0.666$.
2. $n_{99} = \lceil -4.605/(2\ln 0.8) \rceil = \lceil -4.605/-0.446 \rceil = \lceil 10.3 \rceil = 11$ samples. Ratio $(1.8/0.2)^2 = 81 = 19.1$ dB.
3. $\frac{1}{(1-pz^{-1})(1-p^*z^{-1})}$: partial fractions give $h[n] = \frac{p^{n+1} - p^{*\,n+1}}{p - p^*} = r^n \frac{\sin((n+1)\omega_0)}{\sin \omega_0}$, a real damped sinusoid. Without conjugate pairs the impulse response is complex and the frequency response is *not* symmetric in f: it can amplify positive frequencies more than negative ones (as in Figure 35.3). This is realistic for a complex-baseband model of a filter whose passband is not centred on the carrier.
4. $\sum |cp^n|^2 = |c|^2 \sum r^{2n} = |c|^2/(1-r^2)$; $\sum |cp^n|^4 = |c|^4/(1-r^4)$; $\rho = \frac{|c|^4/(1-r^4)}{|c|^4/(1-r^2)^2} = \frac{(1-r^2)^2}{(1-r^2)(1+r^2)} = \frac{1-r^2}{1+r^2}$. $r = 0.5$: $0.75/1.25 = 0.6$; $r = 0.9$: $0.19/1.81 = 0.105$.
5. $1/H(z) = A(z)/B(z)$; for $B = b_0$ it is $A(z)/b_0$, a polynomial of degree P — an FIR filter with $P+1$ taps. For $H = 1 - 0.9z^{-1}$ the inverse $1/(1-0.9z^{-1}) = \sum_n 0.9^n z^{-n}$ is IIR with a long tail; an FIR approximation needs about 22 taps for 99% energy, and a zero closer to the unit circle needs more.
6. $1 - 0.1z^{-10} = 0 \Rightarrow z^{10} = 0.1$: ten poles equally spaced on a circle of radius $0.1^{1/10} = 0.794$ — a comb of resonances every 0.1 cycles/sample.
7. Expect accuracy to keep falling with r; the IIR-trained models degrade more gracefully, but as the memory approaches the window length (Figure 35.4) the received signal becomes a narrow-band ringing whose modulation is hard to see in 128 samples.
8. Auxiliary tasks often help a little by shaping the estimator's features; the result must be measured — it can also hurt if the auxiliary loss dominates. Keep its weight small.

Self-test quiz

1. What makes an IIR filter's impulse response infinite?
   Feedback of past outputs (poles); each output contains a fraction of the previous ones.
2. State the stability condition.
   All poles strictly inside the unit circle.
3. Why is an all-pole channel easy to invert in principle?
   Its inverse $A(z)$ is a finite polynomial — an FIR filter.
4. What does an RTN-style front end learn, and from which loss?
   Per-example equaliser taps predicted by a small network, trained only through the classification (cross-entropy) loss.

5. Give one honest physical source of IIR-type channels.
   Analog front-end filters; repeaters with feedback; resonant enclosures with long exponential tails.

Further reading

- T. J. O'Shea and J. Hoydis, "An introduction to deep learning for the physical layer," *IEEE Trans. Cognitive Commun. Netw.*, vol. 3, no. 4, 2017 (arXiv:1702.00832). Introduces radio transformer networks for learned synchronisation and equalisation.
- T. J. O'Shea, T. Roy and T. C. Clancy, "Over-the-air deep learning based radio signal classification," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, 2018. Deep classifiers under channel impairments.
- T. J. O'Shea, J. Corgan and T. C. Clancy, "Convolutional radio modulation recognition networks," *EANN*, 2016.
- A. Swami and B. M. Sadler (2000); E. E. Azzouz and A. K. Nandi (1996); O. A. Dobre *et al.* (2007): the feature-based foundations (Chapter 32).
- A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, Prentice Hall (any edition). The reference for difference equations, z-transforms, poles, zeros and stability.

Part VII

Your Own Model

Chapter 36

Designing, Training, Debugging and Deploying Your Own Model

Up to now every chapter handed you a problem that somebody else had already framed: the data existed, the labels were clean, the architecture was known to work. Real projects begin earlier and end later. They begin with a vague goal ("can a network tell which modulation that emitter is using?") and a pile of raw samples, and they end with a file running inside a receiver under a hard latency budget, being watched for drift. This chapter covers the whole road. You will learn how to *choose* a model from the shape of your data, how to *build* a dataset (including capturing I/Q with a software-defined radio), how to *train* it with a disciplined recipe whose every check we demonstrate with a real experiment, how to *debug* it with a symptom → cause → fix table that includes the bugs peculiar to radio data, how to *evaluate* it honestly, and how to *deploy* it: saving, exporting to ONNX, quantising to 8-bit integers, and fitting it into the slot timing of a 5G NR physical layer. The chapter closes with three capstone projects, each with milestones and pass/fail criteria.

Intuition: a model is a product, not an experiment

Think of building a bridge. The physics (Chapters 2–19) tells you why steel holds; but a bridge also needs a site survey (your data), a design matched to the site (the architecture), load tests at every stage of construction (the sanity checks), an inspection checklist (debugging), certification (evaluation) and maintenance (monitoring). Most failed ML projects did not fail because the maths was wrong; they failed at the survey or the load tests.

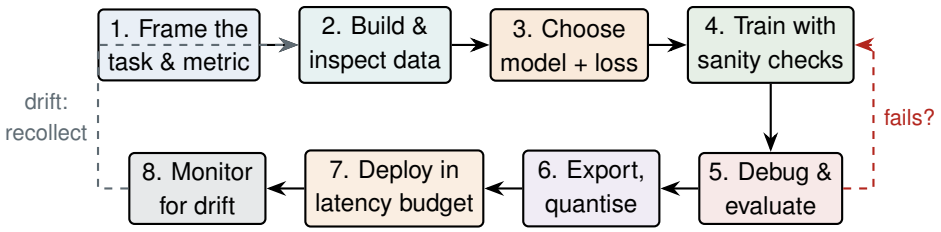


Figure 36.1: The life cycle of your own model. Steps 2–5 form an inner loop you will go round many times; monitoring (step 8) closes an outer loop back to data collection when the radio environment changes.

36.1 Framing the task before touching a network

Before choosing any architecture, write down four sentences: (i) what goes in (the exact tensor: shape, units, sample rate), (ii) what comes out (a class, a number, a bit probability, a sequence), (iii) the metric that decides success (accuracy at a given SNR, BLER at a target, localisation error in metres, inference time in microseconds), and (iv) the *non-ML baseline* you must beat (a cumulant classifier, an LMMSE equaliser, a k-nearest-neighbour fingerprint matcher). If you cannot write sentence (iv), you cannot tell whether the network is good. In wireless, a classical baseline almost

always exists and is often near-optimal in the conditions it was designed for; the network earns its place by being better in the conditions the classical method ignores (hardware impairments, unknown channels, non-Gaussian interference) or by being cheaper.

Wireless connection: a good task statement

"Input: 64 complex baseband samples at one sample per symbol from a Pluto SDR, after coarse frequency correction, normalised to unit average power, shape $(2, 64)$. Output: one of {BPSK, QPSK, 8PSK, 16QAM}. Metric: accuracy $\geq 90\,\%$ at SNR ≥ 10 dB on captures taken on a *different day* from training. Baseline: the cumulant classifier of Chapter 32. Budget: under 100 μs per frame on one ARM core." Every word of this statement will drive a later decision.

36.2 Choosing the model from the data type

The single most useful principle for choosing an architecture is: **match the network's built-in assumptions (its *inductive bias*) to the symmetries of your data.** A convolution assumes that the same local pattern can appear anywhere (translation equivariance); a recurrent network assumes an ordering in time with information carried forward; attention assumes that any element may need to look at any other; a plain multilayer perceptron (MLP) assumes nothing, which is right for a feature vector whose positions have fixed, unrelated meanings.

Definition: inductive bias

The set of assumptions an architecture makes about the function it will learn, before seeing any data. A good inductive bias reduces the amount of data needed, because the network does not have to learn from scratch what the architecture already guarantees.

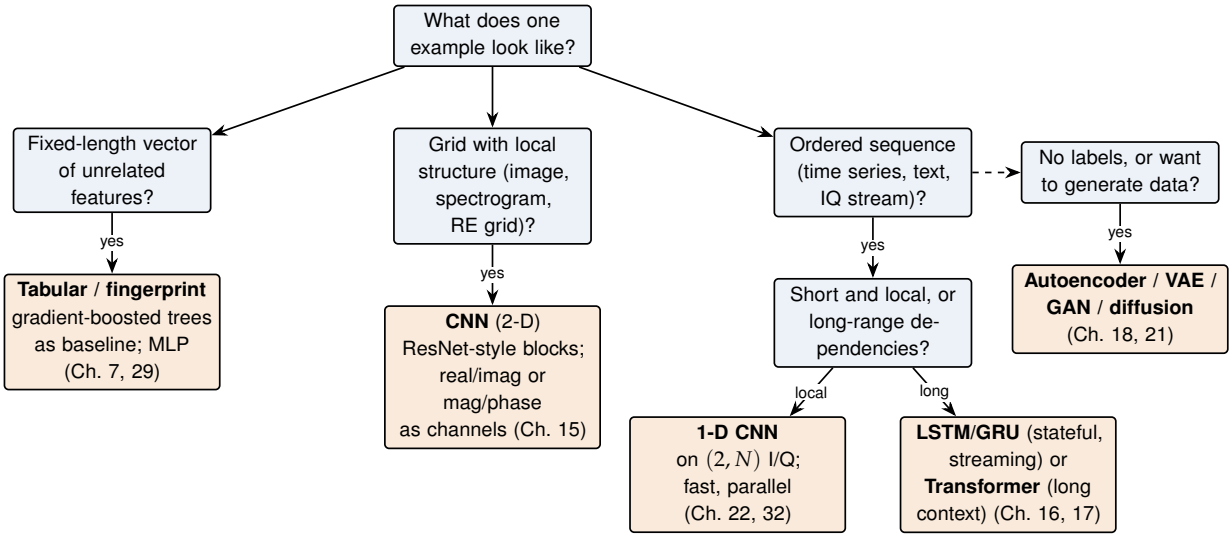


Figure 36.2: Decision tree for a first architecture. Start at the top with the *shape* of one example. The orange leaves are starting points, not final answers; "start with the simplest model that respects the data's structure" is the rule.

36.2.1 Wireless data types and their natural architectures

Raw I/Q frames. A frame of N complex samples becomes a real tensor of shape $(2, N)$: two "channels" (I and Q) along time. A 1-D CNN is the natural first model (Chapters 22, 32–35). If the information is in the *statistics* of the cloud rather than the exact time order (as in modulation recognition at one sample per symbol), end the convolutional stack with *global average pooling*: it makes the output invariant to where in the frame a feature occurred. An LSTM is preferable when the frame is long and the receiver must stream.

OFDM resource grids and CSI tensors. A slot of 14 OFDM symbols by K subcarriers by N_r receive antennas is an image: tensor $(2N_r, 14, K)$ with real and imaginary parts stacked as channels. Neighbouring resource elements are correlated in time (Doppler) and frequency (delay spread), which is exactly the local structure a 2-D CNN exploits. Neural receivers that replace channel estimation, equalisation and demapping are typically residual 2-D CNNs over this grid.

CSI time series. A sequence of channel estimates $\mathbf{H}_1, \mathbf{H}_2, \ldots$ for prediction (Chapter 27): an LSTM/GRU over time with each $\mathbf{H}_t$ flattened (or compressed by a small CNN) as the input vector.

Spectrograms and waterfalls. The magnitude of a short-time Fourier transform is a single-channel image in time–frequency. Use a 2-D CNN; for detecting several emitters at once, an object-detection style output head.

RSS and CSI fingerprints. A vector of received signal strengths from M access points has positions with fixed meanings ("AP number 3"), so an MLP is right (Chapter 29). Per-subcarrier CSI amplitude across a link has frequency order, so a 1-D CNN over subcarriers helps.

Coherent array snapshots. Samples from an M-element array (for example a KrakenSDR) at one instant form a vector whose *phase differences* carry the angle of arrival. Feed the sample covariance $\hat{\mathbf{R}} = \frac{1}{T}\sum_t \mathbf{y}_t \mathbf{y}_t^{\mathsf{H}}$ (real and imaginary parts of its upper triangle) to an MLP, or the raw snapshots to a small CNN across time.

36.2.2 Output head and loss

The last layer and the loss are dictated by *what the output means*, not by the architecture of the body. Table 36.1 is the one to memorise.

Pitfall: softmax twice

`CrossEntropyLoss` and `BCEWithLogitsLoss` apply the softmax/sigmoid internally, in a numerically stable way. Adding `nn.Softmax` at the end of your model and then using `CrossEntropyLoss` still trains, but slower and to a worse optimum, because the gradients are squashed twice. Apply softmax only at inference time, when you need probabilities.

36.3 Building the dataset

36.3.1 Simulation, measurement, or both

Wireless engineers have an unusual luxury: for many problems they can *simulate* unlimited labelled data. Simulation gives perfect labels, any SNR you like, and channel models from standards (3GPP TDL/CDL). Its weakness is the **sim-to-real gap**: real hardware adds carrier

Table 36.1: Output head, loss and the value of the loss at initialisation (a free sanity check, Section 36.4).

Task	Last layer	Loss (PyTorch)	Wireless example / init loss
K-class, one label	K linear logits	`CrossEntropyLoss`	modulation class; $\ln K$
Several independent yes/no	B linear logits	`BCEWithLogitsLoss`	B bits per symbol from a demapper (logits = LLRs); $\ln 2 = 0.693$ per bit
Real value(s)	linear, no activation	`MSELoss` / `HuberLoss`	CSI prediction, (x, y) position; $\approx$ variance of the standardised target (≈ 1)
Value with uncertainty	mean and $\log \sigma^2$	`GaussianNLLLoss`	position with confidence radius
Bounded value in $[0, 1]$	sigmoid	BCE or MSE	occupancy probability of a channel
Reconstruct input	matches input	MSE (or CE for symbols)	autoencoder PHY (Ch. 21)
Action values	$\lvert\mathcal{A}\rvert$ linear outputs	Huber on TD target	DQN channel access (Ch. 30)

frequency offset (CFO), sample-clock offset, I/Q imbalance, DC offset, phase noise, power-amplifier non-linearity, ADC clipping and interference that your simulator either lacks or models imperfectly. A network is a ruthless exploiter of whatever regularities exist in its training data; if every simulated frame starts at symbol boundary 0 with zero phase, it will learn to depend on that.

Intuition: the sim-to-real gap

Learning to drive in a simulator with perfect tarmac, then driving on gravel: the skills mostly transfer, but the car skids in ways the simulator never showed you. The cures are (i) make the simulator rougher than reality (randomise every impairment over a wide range, "domain randomisation"), (ii) fine-tune on a smaller set of real captures, and (iii) always test on real captures.

A practical recipe that works for most PHY problems is: *pre-train on simulation with heavy impairment randomisation; fine-tune on a modest number of real captures; evaluate only on real captures recorded on a different day, at a different place, with a different device.*

36.3.2 Capturing I/Q with an SDR

The **ADALM-PLUTO** (Analog Devices) is a one-transmitter, one-receiver SDR built around the AD9363 transceiver and a Zynq-7010 FPGA; Analog Devices specifies a tuning range of 325 MHz to 3.8 GHz, up to 20 MHz instantaneous bandwidth and 12-bit converters, with Python access through `libiio/pyadi-iio`. Two Plutos (or one in loop-back through a cable and attenuator) let you transmit known waveforms and record them, producing labelled real data. The **KrakenSDR** (KrakenRF) is a five-channel coherent receiver built from five R820T2-based RTL-SDR tuners driven by a common clock, with internal noise-source calibration so the phase relationship between channels can be measured and corrected; KrakenRF quotes a 24–1766 MHz tuning range. It is the cheap way to collect *array* data for angle-of-arrival and localisation models.

Rules for a capture campaign:

1. **Fix the receive gain** (manual gain control) for a whole session and record it. Automatic gain control changes the signal power in a way correlated with whatever you were transmitting.

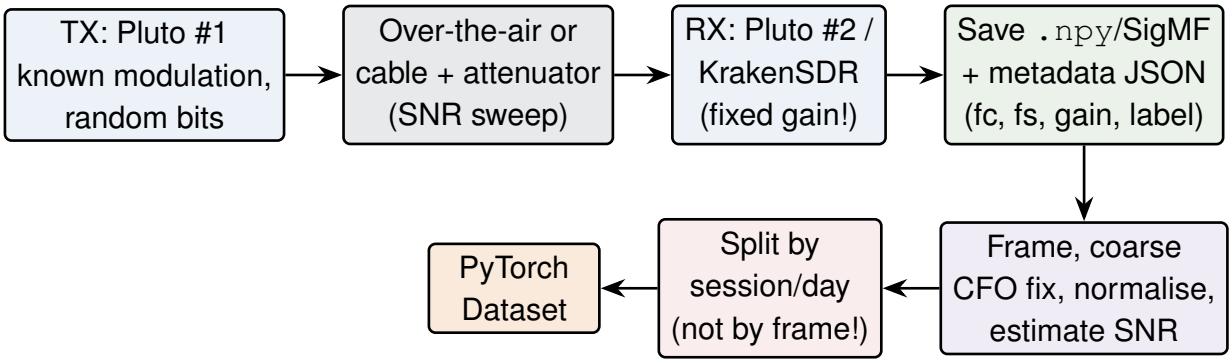


Figure 36.3: A capture pipeline for a self-made radio dataset. The two decisions that most often ruin datasets are marked in red and in the RX block: splitting train/test by individual frame (which leaks session-specific artefacts) and letting automatic gain control change per recording (which leaks the label through power).

2. **Record metadata with every file**: centre frequency, sample rate, gain, transmitter settings, label, date, location, hardware serial numbers. The SigMF format (a `.sigmf-data` binary plus a `.sigmf-meta` JSON) is a widely used open convention.
3. **Vary what you want the model to ignore**: distance, antenna orientation, time of day, the specific transmitter unit, the channel (line-of-sight and not), the frequency.
4. **Sweep the SNR** with a step attenuator rather than by moving the receiver, and estimate the SNR of each frame afterwards (for example, from the error-vector magnitude after decision or from a noise-only capture).
5. **Capture noise-only and interference-only files** as an explicit "no signal" class or for calibration.

Wireless connection: labelling real captures

When you transmit the data yourself, labels are free and exact. When you record signals "in the wild" (for example cellular downlink), the label must come from somewhere: decode the signal with a conventional receiver (an LTE cell search gives you the cell ID; a successful CRC proves the bits) and use that as ground truth. This "teacher" labelling is how neural receivers can be trained on live networks: only frames whose CRC passes are kept, and their decoded bits become labels.

36.3.3 SNR and channel diversity

Train over the full range of SNRs you expect, and *report results per SNR*. A dataset uniformly spread from -10 to $+20$ dB contains a third of examples at negative SNR where even an ideal classifier is close to chance; the network spends capacity on them and the averaged accuracy hides the high-SNR behaviour. Channel diversity matters as much: if the training set was recorded at one location, the network may learn the multipath signature of that room.

36.3.4 A Dataset for tabular data, with train-only normalisation

Fingerprint localisation (Chapter 29) is a tabular problem: each row is a vector of RSS values, the targets are coordinates. The code below is a reusable `Dataset` that reads a CSV file. The crucial design decision is that **normalisation statistics are computed on the training split only**

and then passed to the validation and test splits. Computing the mean and standard deviation on the whole file lets information about the test set leak into training; for small datasets this alone can make results look several percent better than they will be in the field.

```
class TabularDataset(Dataset):
    """Reads a CSV; feature_cols -> inputs, target_cols -> targets.
    Normalisation stats are passed in, so they can come from the TRAIN split only.
    """
    def __init__(self, rows, feature_cols, target_cols, stats=None):
        X = np.array([[float(r[c]) for c in feature_cols] for r in rows], np.
    float32)
        Y = np.array([[float(r[c]) for c in target_cols] for r in rows], np.
    float32)
        if stats is None:                                    # compute ONLY when
    building train set
            stats = {"mean": X.mean(0).tolist(), "std": (X.std(0) + 1e-8).tolist()}

        self.stats = stats
        self.X = torch.from_numpy((X - np.array(stats["mean"], np.float32))
                                  / np.array(stats["std"], np.float32))
        self.Y = torch.from_numpy(Y)
    def __len__(self):
        return len(self.X)
    def __getitem__(self, i):
        return self.X[i], self.Y[i]

def load_splits(path, feature_cols, target_cols, frac=(0.7, 0.15, 0.15), seed=0):
    rows = list(csv.DictReader(open(path)))
    idx = np.random.default_rng(seed).permutation(len(rows))
    n_tr, n_va = int(frac[0] * len(rows)), int(frac[1] * len(rows))
    pick = lambda ii: [rows[i] for i in ii]
    train = TabularDataset(pick(idx[:n_tr]), feature_cols, target_cols)
     # fits stats
    val = TabularDataset(pick(idx[n_tr:n_tr + n_va]), feature_cols, target_cols,
    train.stats)
    test = TabularDataset(pick(idx[n_tr + n_va:]), feature_cols, target_cols,
    train.stats)
    return train, val, test
```

Worked example: standardising with training statistics

Three training rows of one RSS feature are $-60, -70, -80$ dBm. Mean $\mu = (-60 - 70 - 80)/3 = -70$. Population variance $\sigma^2 = \frac{1}{3}[(10)^2 + 0^2 + (-10)^2] = 66.67$, so $\sigma = 8.165$. A *test* reading of -65 dBm becomes $z = (-65 - (-70))/8.165 = 0.612$. If the test file happened to have mean -60, recomputing statistics on it would map -65 to a negative number: the same physical reading would mean different things to the network. The `stats` dictionary is therefore part of the model and must be saved with it (Section 36.7).

In our synthetic version (six access points, log-distance path loss with exponent 3 and 4 dB shadowing, 3000 points in a 20×10 m room; script `tabular_dataset.py`), the input-independent baseline that always predicts the mean training position has a mean error of 5.84 m, while a two-hidden-layer MLP reaches 1.66 m mean (1.54 m median). The baseline number is what makes the 1.66 m meaningful.

36.4 A recipe for training, with every check demonstrated

In 2019 Andrej Karpathy published a widely read blog post, "A Recipe for Training Neural Networks". Its premise is that neural network training fails *silently*: a bug rarely throws an exception, it just makes the network a few percent worse, so you must build confidence step by step with checks that each have a predictable outcome. We condense it into six stages and then run the most important checks on a real model: a small 1-D CNN (2524 parameters; script `common36.py`) classifying frames of 64 symbols into BPSK, QPSK, 8PSK or 16QAM in AWGN.

1. **Become one with the data.** Look at hundreds of examples before writing model code: plot constellations, spectra, histograms of power and SNR, class counts. Search for duplicates, corrupted files, label errors. For radio data, plot I versus Q of random frames from each class and each recording session.
2. **Set up the end-to-end skeleton and get dumb baselines.** Fix the random seed; disable augmentation; verify the loss at initialisation; initialise the last layer sensibly; compute an input-independent baseline; overfit one batch; visualise exactly what enters the network (after all preprocessing); confirm the training loss falls when capacity grows.
3. **Overfit.** Get a model large enough to drive the *training* loss low. Do not be a hero: copy a known architecture. Start with Adam at a learning rate of 3×10^{-4}. Add complexity one piece at a time and check each piece helps.
4. **Regularise.** More real data first; then data augmentation (for radio: random phase, CFO, time shift, SNR); then pre-training, smaller inputs, smaller model, dropout, weight decay, early stopping.
5. **Tune.** Random search over hyperparameters rather than a grid.
6. **Squeeze out the last drop.** Ensembles, and training longer than seems necessary.

36.4.1 Check 1: the loss at initialisation is $\ln K$

A freshly initialised classifier knows nothing, so it should give each of the K classes roughly probability $1/K$. The cross-entropy of the true class is then

$$L_{\text{init}} = -\ln p_{\text{true}} \approx -\ln \frac{1}{K} = \ln K. \tag{36.1}$$

For $K = 4$, $\ln 4 = 1.386$; for $K = 10$, 2.303; for the 24 classes of RadioML 2018.01A, 3.178. If your first printed loss is very different, something is wrong *before* any training: the logits are too large (bad initialisation of the last layer, or inputs not normalised), the labels are out of range, or the loss is being averaged incorrectly.

36.4.2 Check 2: overfit one tiny batch

Take a handful of examples (we use 16 frames, four per class, at 20 dB) and train on them alone, repeatedly. A network with thousands of parameters can memorise 16 examples, so the training loss must go to (nearly) zero. If it does not, the bug is in the plumbing: labels misaligned with inputs, a frozen layer, a wrong loss, a learning rate of zero, a `model.eval()` left on, or gradients not reaching the parameters.

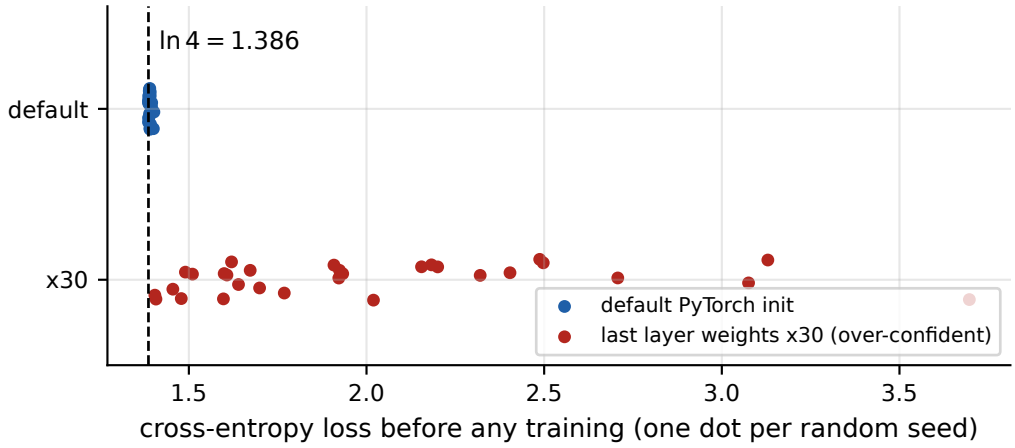


Figure 36.4: Loss before any training for 30 random seeds of our CNN on 1024 frames. With PyTorch's default initialisation every seed lands near $\ln 4$ (mean 1.391). Multiplying the last-layer weights by 30 — a stand-in for un-normalised inputs or a bad init — makes the network confidently wrong at step 0: the mean loss jumps to 2.02 and varies widely from seed to seed. Look for: the blue cluster sitting on the dashed line.

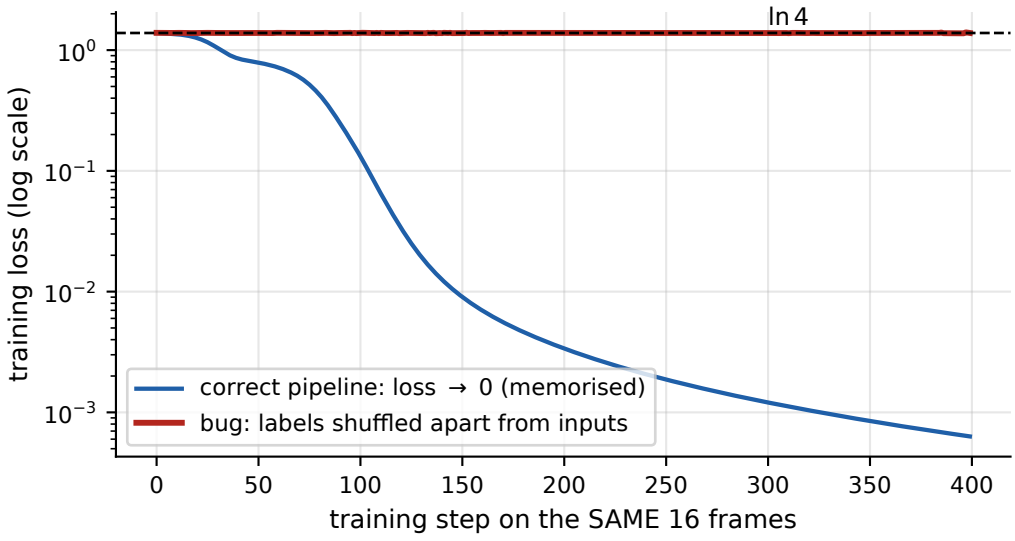


Figure 36.5: Overfitting a single batch of 16 frames (Adam, learning rate 3×10^{-3}). The correct pipeline memorises the batch: loss falls from $\ln 4$ to 6×10^{-4} in 400 steps. In the red run we inserted a classic data-loader bug — the labels are shuffled with a different permutation from the inputs — and the loss stays pinned at $\ln 4 = 1.387$ forever, because no function of the input can predict a label that is random with respect to it. A loss stuck at exactly $\ln K$ is the signature of "labels unrelated to inputs".

36.4.3 Check 3: an input-independent baseline

Train the same model with all inputs replaced by zeros. It can then only learn the class prior, and its loss converges to the entropy of the label distribution ($\ln K$ for balanced classes). Your real model must do clearly better; if it does not, the network is not using its input (inputs disconnected, all-zero after a preprocessing bug, or the information simply is not there).

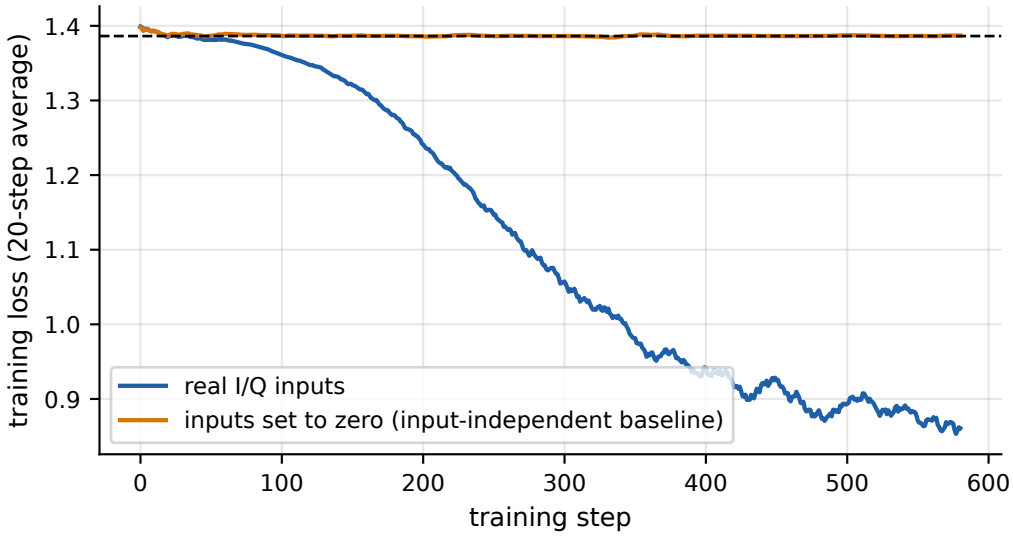


Figure 36.6: Training loss with real I/Q inputs versus inputs set to zero (same model, same 600 steps, Adam 3×10^{-4}). The zero-input model sits at $\ln 4$ (1.387), exactly as theory predicts for balanced classes; the real model descends to 0.86 and is still improving. The gap between the curves is the information the network extracts from the signal.

36.4.4 Check 4: the learning rate, and why 3×10^{-4} is a starting point

Karpathy's suggestion of Adam with 3×10^{-4} is a *safe* default: it rarely diverges and works across many architectures. It is not optimal for every problem. The quick way to find a good value is a **learning-rate sweep**: train for a fixed short budget at log-spaced learning rates and plot the final validation loss.

36.4.5 Check 5: add complexity one step at a time

Change one thing, measure, keep it only if it helps. If you add data augmentation, a deeper network, a new input feature and a learning-rate schedule all at once and the result is worse, you have learned nothing. Keep a log (a spreadsheet is enough): run ID, the one change, training and validation metric. This discipline is the difference between an engineer and a gambler.

Intuition: debugging is bisection

Each check splits the space of possible bugs in half. Loss at init wrong $\Rightarrow$ the bug is in data scaling or initialisation, not in the optimiser. Cannot overfit one batch $\Rightarrow$ the bug is in plumbing, not in generalisation. Overfits but does not generalise $\Rightarrow$ the bug is in data or regularisation, not in plumbing. You never have to search the whole program.

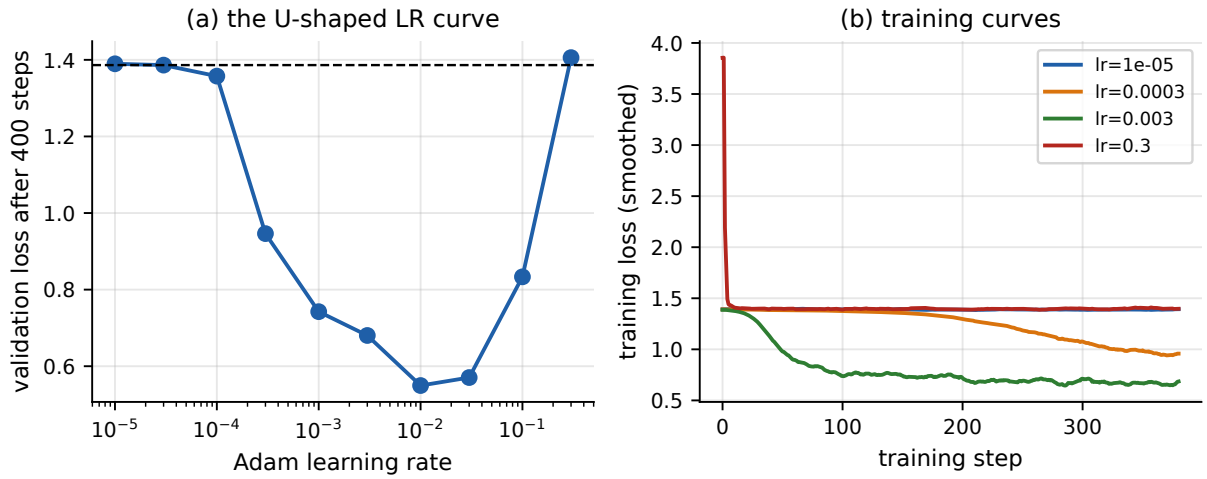


Figure 36.7: Learning-rate sweep for our CNN, 400 Adam steps each. (a) Validation loss is U-shaped in $\log \eta$: below 10^{-4} the network barely moves from $\ln 4$ in the budget; the best value here is around 10^{-2} (validation loss 0.55); at 3×10^{-1} training diverges back to chance. (b) Training curves: too small (blue) is flat, 3×10^{-4} (orange) is slow but steady, 3×10^{-3} (green) is fast, 0.3 (purple) jumps up at the first step and never recovers. For this tiny network a learning rate of 10–30 times the default is best; choose the value a little below the minimum of curve (a) for robustness.

36.5 Debugging: symptoms, causes and fixes

36.5.1 A deliberate wireless bug: label leakage through received power

Suppose each modulation class was recorded in a separate session, and the receiver gain happened to differ between sessions (or AGC settled differently). If frames are not power-normalised, the received power is perfectly correlated with the class. The network will happily learn "loud means 16QAM", reach near-perfect test accuracy on a test split *with the same leak*, and fail completely in the field. We simulated exactly this (script `leakage.py`): class k captured at amplitude gain $0.5, 1.0, 1.5, 2.0$, no normalisation.

Pitfall: too good to be true usually is

If your classifier beats published results by a wide margin, or is excellent at an SNR where a matched-filter-based classical method fails, look for a leak before celebrating. Typical leaks in radio data: power or gain per class, a file-name or ordering correlation, a DC spike present only in some recordings, frames from the same capture in both train and test, and the SNR itself used as an input feature when the SNR range differs between classes.

36.5.2 The debugging table

Table 36.2 collects the bugs you are most likely to meet, general and wireless-specific, with the symptom you will actually observe.

Table 36.2: Symptom → likely cause → fix. Work from the top: the first rows are the cheapest to check.

Symptom	Likely cause	Fix / test
General		

Symptom	Likely cause	Fix / test
Loss at step 0 far from $\ln K$	logits too large: un-normalised input, bad last-layer init, wrong number of classes	standardise inputs; check `out_features`=K; scale last layer down
Loss is `nan/inf`	LR too high; $\log 0$ in a hand-written loss; division by a zero std; exploding RNN gradients	lower LR; use built-in losses with logits; add ϵ; clip gradients
Loss stuck at exactly $\ln K$	labels shuffled apart from inputs; inputs all zero/constant; LR $= 0$; all parameters frozen	overfit one batch; print a batch *after* the loader; check `requires_grad`
Cannot overfit one batch	`optimizer.zero_grad()` missing or misplaced; `model.eval()` on; wrong loss/target dtype; bug in custom layer	re-read the training loop line by line; gradient check (Ch. 13)
Training loss falls, validation rises early	overfitting: too little data, model too big	more data, augmentation, dropout, weight decay, early stopping
Validation loss below training loss	dropout/augmentation active only in training; validation set easier or leaked	expected mildly; if large, check splits for leakage
Training and validation both poor	underfitting: model too small, LR wrong, input lacks information	bigger model; LR sweep; check input-independent baseline gap
Great validation, bad in deployment	train/serve skew: different preprocessing or normalisation at inference	save preprocessing with the model; test the exported file on raw inputs
Accuracy fluctuates wildly between epochs	LR too high; batch too small; BatchNorm with tiny batches	lower LR; larger batch; LayerNorm/GroupNorm
Model always predicts one class	extreme class imbalance; collapsed ReLUs (dead units)	class weights or balanced sampling; check activation statistics
Wireless-specific		
Constellation looks mirrored; QPSK classes swapped; BER ≈ 0.5 on some bits	**I/Q swap or conjugation** (driver returns Q,I; or spectrum inverted by mixing)	plot a known constellation from the capture; swap channels or conjugate
Model fine in simulation, chance on real captures	**power normalisation mismatch**: training frames unit power, real frames raw ADC counts	normalise every frame the same way in training and inference (Fig. 36.9)
Accuracy vs SNR curve shifted by several dB from expectation	**SNR definition mismatch**: E_s/N_0 vs E_b/N_0 vs per-sample SNR with oversampling; noise variance per real dimension	write the SNR formula in code comments; verify measured SNR of generated frames

Symptom	Likely cause	Fix / test
Near-perfect accuracy even at very low SNR	**label leakage through SNR/power**: class-dependent gain, SNR fed as input with class-dependent range	normalise power; randomise gain per frame; split by session (Fig. 36.8)
Works on one recording session, not another	**train/test channel mismatch**; split by frame not by session; device fingerprint learned	split by session/day/device; augment with channels; collect more locations
Accuracy collapses with a small frequency offset	no CFO/phase in training; model memorised absolute phase	augment with random phase and CFO (Fig. 36.9); add coarse CFO correction
Learned demapper LLRs have the wrong sign	LLR convention $\ln \frac{P(0)}{P(1)}$ vs $\ln \frac{P(1)}{P(0)}$; bit-to-label mapping differs	test on a noiseless constellation point; flip sign consistently
Strong spike at 0 Hz dominates	DC offset / LO leakage of zero-IF receiver	remove mean per frame; tune slightly off-centre
Performance varies with frame start	timing offset not represented in training	random time shifts; global pooling; sync before the network
Real captures clip or look square	ADC saturation from too much RX gain	lower gain; reject frames with samples at full scale

36.6 Evaluation that you can trust

36.6.1 Splits

Three splits: *training* (fit parameters), *validation* (choose hyperparameters and when to stop), *test* (touched once, at the end). For radio data the split must be by *recording session* (or day, location, device), never by randomly shuffled frames: frames from the same capture share the channel, the CFO, the gain and the hardware imperfections, so a frame-level split measures memorisation of the session.

36.6.2 Accuracy versus SNR, and robustness to mismatch

One accuracy number averaged over all SNRs says very little. Plot accuracy (or BER/BLER for receivers) against SNR, and plot it under realistic perturbations. Figure 36.9 shows our reference classifier (trained for 4000 steps with Adam, learning rate 3×10^{-3}, cosine decay; script `train_eval.py`) tested in four conditions.

At 15 dB the matched model classifies BPSK, QPSK and 8PSK perfectly (300/300 each), while 52 of 300 16QAM frames are called QPSK and 16 are called 8PSK. This is physically sensible: with only 64 symbols, the inner ring of 16QAM can look like a QPSK cloud. A confusion matrix tells you *which* decision is hard, and therefore what extra information (longer frames, cumulant features) would help.

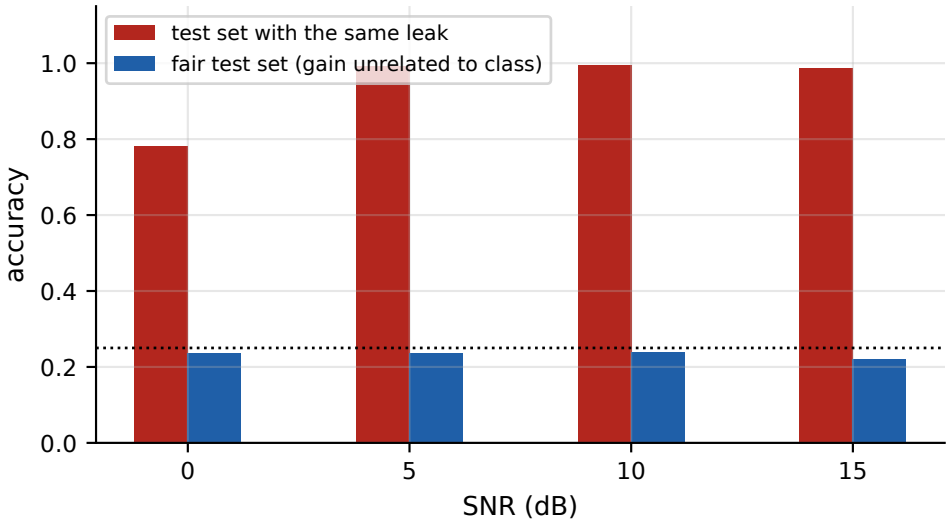


Figure 36.8: Label leakage. Red: test accuracy on frames with the same class-dependent gain as training — about 99 % at 5 dB and above, which looks like a triumph. Blue: the same network on a fair test set where gain is random and unrelated to class — 22–24 %, i.e. chance (dotted line). The network learned the receiver gain, not the modulation. Note also the tell-tale sign: the "leaky" accuracy at 0 dB is *far* better than any legitimate classifier could achieve on 64 noisy symbols.

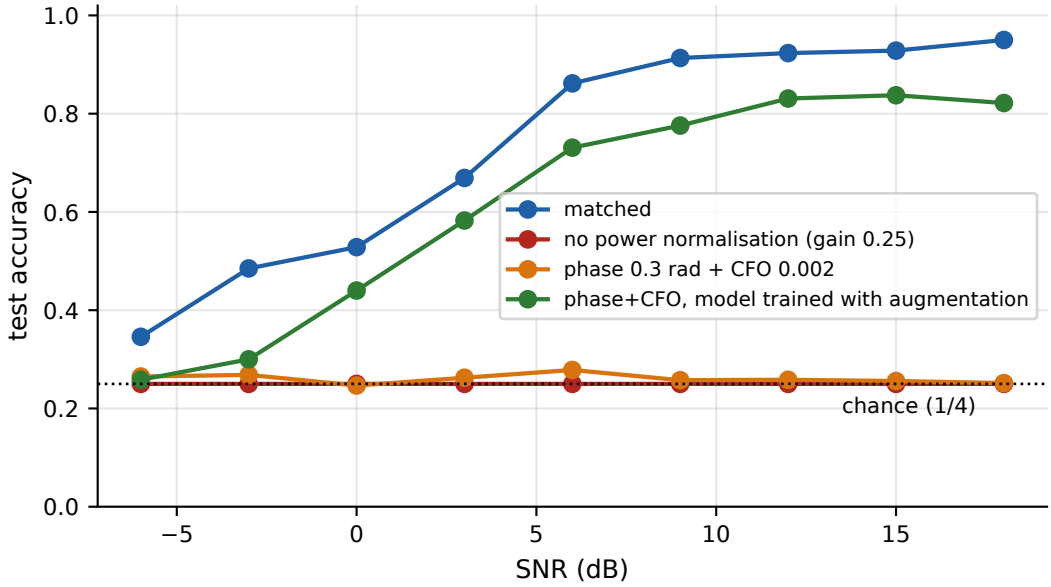


Figure 36.9: Accuracy versus SNR for the same trained network under deployment mismatches. Blue: matched test data (rises from 35 % at −6 dB to 95 % at 18 dB). Red: frames not power-normalised (receiver gain 0.25): chance at every SNR. Orange: a phase rotation of 0.3 rad plus a CFO of 0.002 cycles/sample (about 0.8 rad of extra rotation over a 64-symbol frame): also chance — the model had silently learned absolute phase. Green: a second model trained with random phase and CFO augmentation recovers most of the performance on the impaired data. Two lines of preprocessing and augmentation are worth more than any architecture change.

36.6.3 Receivers: BER and BLER

For demappers, equalisers and decoders, use the communication metrics: bit error rate (BER) and block error rate (BLER), plotted on a logarithmic axis versus E_b/N_0 or SNR, always with the classical baseline (LMMSE + max-log demapper, or the theoretical curve) on the same axes. Simulate enough blocks to observe about 100 errors at each point; at a BLER of 10^{-3} that is 10^5 blocks. A learned receiver that matches the baseline at high SNR and beats it at low SNR is a real result; one that only beats it in a narrow SNR region it was trained at is not.

36.6.4 Calibration: is "90 % sure" right 90 % of the time?

Definition: calibration and ECE

A classifier is *calibrated* if, among all predictions made with confidence c, the fraction that are correct is c. Group predictions into confidence bins B_b; the *expected calibration error* is $\mathrm{ECE} = \sum_b \frac{|B_b|}{N} \, |\mathrm{acc}(B_b) - \mathrm{conf}(B_b)|$.

Calibration matters whenever the probabilities are used downstream: LLRs fed to a channel decoder *are* confidences, and an over-confident demapper hurts the decoder even if its hard decisions are right. Over-confidence is cured cheaply by *temperature scaling*: divide the logits by a scalar $T > 1$ fitted on the validation set.

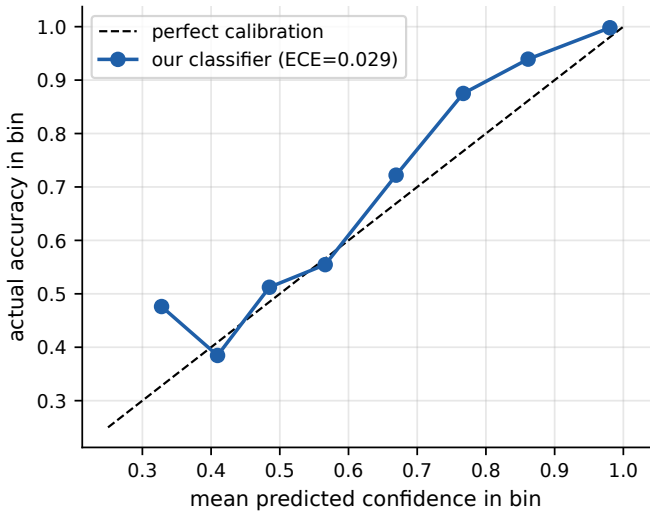


Figure 36.10: Reliability diagram of our classifier on a matched test set (overall accuracy 73.6 % across all SNRs). Points close to the diagonal mean the softmax confidence can be trusted; here ECE $= 0.029$, i.e. confidences are on average within 3 percentage points of the true accuracy. Small, lightly trained networks are often well calibrated; large networks trained to zero training loss tend to be over-confident (points below the diagonal).

36.7 Deployment

36.7.1 Saving: weights plus configuration

Save two files: the `state_dict` (a dictionary of tensors, nothing else) and a small JSON file with everything needed to rebuild the model and its preprocessing: architecture hyperparameters,

class names, input length, normalisation statistics, the SNR definition, the code version. Pickling the whole model object (`torch.save(model)`) ties the file to your exact source code and is fragile.

```
torch.save(model.state_dict(), "modclf_state.pt")
json.dump({"width": 24, "n_sym": 64, "classes": ["BPSK","QPSK","8PSK","16QAM"],
           "norm": "unit average power per frame"}, open("modclf_config.json", "w"))
# --- later, anywhere ---
cfg = json.load(open("modclf_config.json"))
model = SmallCNN(cfg["width"]); model.load_state_dict(torch.load("modclf_state.pt", weights_only=True))
model.eval()                      # disables dropout, BatchNorm uses running statistics
```

36.7.2 ONNX export and ONNX Runtime

ONNX (Open Neural Network Exchange) is a framework-neutral file format describing the computation graph and weights. Exported once, the model can be run by ONNX Runtime on a server, on an ARM board, or converted further for vendor accelerators. Always check that the exported model gives the same outputs as PyTorch.

```
    # ---------- (b) ONNX export ----------
    onnx_path = os.path.join(OUT, name + ".onnx")
    torch.onnx.export(m, X[:1], onnx_path, input_names=["iq"], output_names=["logits"],
                      dynamic_axes={"iq": {0: "batch"}, "logits": {0: "batch"}},
    opset_version=17, dynamo=False)
    def run_ort(path, label):
        so = ort.SessionOptions(); so.intra_op_num_threads = 1
        s = ort.InferenceSession(path, so, providers=["CPUExecutionProvider"])
        xn = X.numpy(); out = s.run(None, {"iq": xn})[0]
        a = (out.argmax(1) == yte.numpy()).mean()
        x1 = xn[:1]
        for _ in range(20): s.run(None, {"iq": x1})
        ts = []
        for _ in range(300):
            t = time.perf_counter(); s.run(None, {"iq": x1}); ts.append(time.perf_counter() - t)
        rows.append((name, label, size_kb(path), float(np.median(ts)) * 1e6, a))
        return out
    ref = run_ort(onnx_path, "ONNX Runtime fp32")
    with torch.no_grad(): print(name, "max |torch - ort| logits:", np.abs(m(X).numpy() - ref).max())
```

For both of our models the maximum absolute difference between PyTorch and ONNX Runtime logits was below 10^{-5} (exactly 0.0 for the MLP).

36.7.3 Quantisation to 8-bit integers

Intuition: quantisation

A 32-bit float can represent numbers from 10^{-38} to 10^{38}; the weights of one layer typically live between about -1 and 1. Quantisation spends 8 bits on that narrow range instead: 256 evenly spaced levels. Weights shrink by $4\times$, integer arithmetic is faster and cheaper in energy, and the rounding error behaves like a small amount of added noise that a trained network usually tolerates.

Key equation: symmetric int8 quantisation

$$s = \frac{\max_i |x_i|}{127}, \qquad q_i = \operatorname{round}\left(\frac{x_i}{s}\right) \in \{-127, \dots, 127\}, \qquad \hat{x}_i = s\, q_i, \qquad |\hat{x}_i - x_i| \le \frac{s}{2}. \tag{36.2}$$

Worked example: quantising four weights by hand

Weights $\mathbf{w} = (0.92, -0.35, 0.7071, 0.1)$. Scale $s = 0.92/127 = 0.007244$. Then $q = (\operatorname{round}(127.0), \operatorname{round}(-48.3), \operatorname{round}(97.6), \operatorname{round}(13.8)) = (127, -48, 98, 14)$. De-quantised: $(0.9200, -0.3477, 0.7099, 0.1014)$. Errors: $(0,\ 0.0023,\ 0.0028,\ 0.0014)$, all below $s/2 = 0.0036$ as promised. Notice that one large weight sets the scale for all: an outlier of 5.0 would make $s = 0.039$ and wipe out the resolution of the small weights. That is why per-channel scales (one s per output channel) are standard.

Two flavours are common. **Dynamic quantisation** stores weights as int8 and computes the activation scale on the fly for each input; it needs no data and suits layers dominated by matrix multiplications (MLPs, LSTMs, Transformers). **Post-training static quantisation** also fixes the activation scales in advance, using a few hundred *calibration* inputs from the training distribution; all arithmetic can then stay in integers, which is what DSPs, NPUs and FPGAs want. (Quantisation-aware training, which simulates rounding during training, is the next step when post-training quantisation loses too much accuracy.)

We measured both on CPU with one thread (script `deploy.py`), for our convolutional classifier and for an MLP classifier (three hidden layers of 512, 512 and 256 units on sorted amplitude and phase features, 0.46 M parameters), using 108 calibration frames drawn separately from the test set. Latency is the median over 300 single-frame calls.

36.7.4 Fixed point for DSPs and FPGAs

Many baseband processors (for example the CEVA and Tensilica DSP families) and all FPGAs compute fastest in fixed point. A **Q***m*.*n* number is an integer interpreted as integer $\times\, 2^{-n}$, with m integer bits and n fractional bits (plus a sign bit). The ubiquitous Q15 (16-bit, $n = 15$) covers $[-1, 1)$ with resolution $2^{-15} \approx 3.05 \times 10^{-5}$; for example $0.7071 \mapsto \operatorname{round}(0.7071 \cdot 2^{15}) = 23170$, which represents 0.707092 (error 7.7×10^{-6}). Deploying a network in fixed point means:

- **Choose formats per tensor** from the observed range on calibration data (exactly the information static quantisation collects): weights often Q15 or int8, activations int8/int16, accumulators 32 or 40 bits to avoid overflow in long dot products (N products of two Q15 numbers need $30 + \lceil \log_2 N \rceil$ bits).

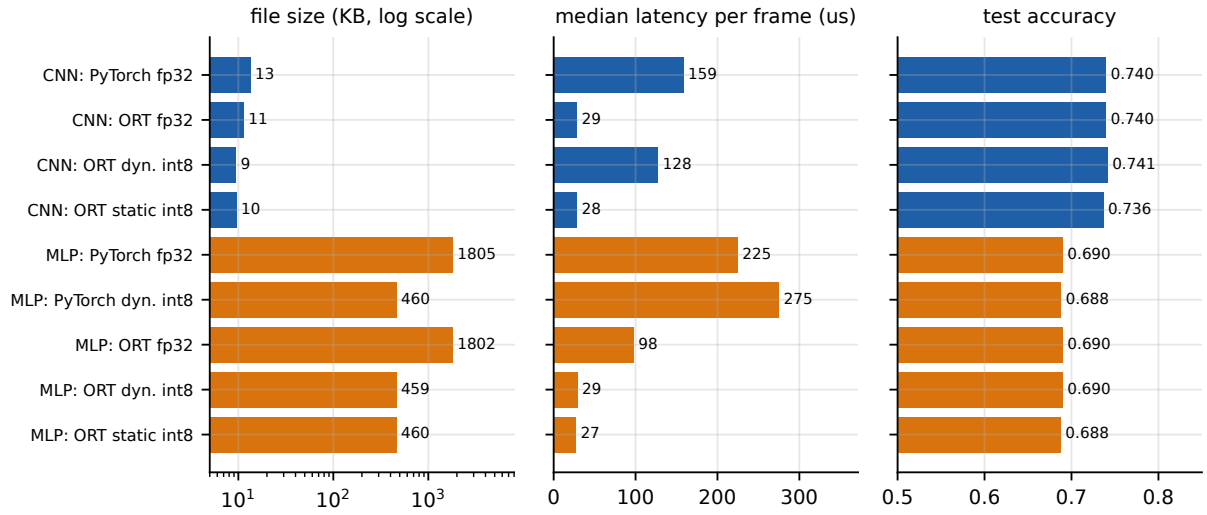


Figure 36.11: Size, single-frame latency and accuracy of each deployment variant, measured on our 2-CPU build machine (one thread; other jobs were running, so absolute latencies are indicative — compare bars within the figure). Int8 shrinks the MLP's 1.8 MB of weights about 4×; accuracy changes by at most 0.4 percentage points for either model. Exporting to ONNX Runtime alone gives a large speed-up over eager PyTorch for single small inputs, because per-call framework overhead dominates such tiny models. For the 2.5 k-parameter CNN, dynamic quantisation is *slower* (quantising activations on every call costs more than it saves), while static quantisation is roughly as fast as float. Quantisation pays off when weights are large; for tiny models, the runtime matters more.

- **Fold BatchNorm into the preceding layer** before quantising: $\gamma(\mathbf{w}^{\mathsf{T}}\mathbf{x} + b - \mu)/\sigma + \beta$ is just another affine map with weights $\gamma\mathbf{w}/\sigma$.
- **Replace expensive non-linearities**: ReLU is free in fixed point; sigmoid, tanh and softmax need lookup tables or piecewise-linear approximations. Remember that for a classifier the *argmax* of the logits needs no softmax at all.
- **Verify bit-exactly**: build a NumPy model that mimics the integer arithmetic (rounding mode, saturation) and compare against the device output on a test vector file, exactly as you would for any DSP kernel.

36.7.5 Latency budgets in a slot-based PHY

In 5G NR the subcarrier spacing is $\Delta f = 15 \cdot 2^{\mu}$ kHz and a slot of 14 OFDM symbols (normal cyclic prefix) lasts $1\,\text{ms}/2^{\mu}$ (3GPP TS 38.211). Table 36.3 lists the values; in Release 15, data channels use $\mu = 0$–3 and $\mu = 4$ is used only for the synchronisation signal block, while Release 17 added $\mu = 5, 6$ (480 and 960 kHz) for the 52.6–71 GHz range.

A neural block inserted into the receive chain must finish within its share of these times, *every* slot, not on average. A typical budget for a PUSCH neural receiver at $\mu = 1$:

Consequences for model design: measure *worst-case*, not mean, latency; batch across PRBs and antennas to use vector units fully; prefer architectures with fixed compute (CNNs) over data-dependent ones; and remember that a model which fits at $\mu = 0$ has half the time at $\mu = 1$ and one-eighth at $\mu = 3$.

Table 36.3: 5G NR numerologies (normal cyclic prefix). The symbol duration is the slot divided by 14, averaged (the first symbol of each half-subframe has a slightly longer CP).

μ	Δf (kHz)	slots/ms	slot duration	avg. symbol	typical use
0	15	1	1 ms	71.4 μs	FR1 (sub-6 GHz)
1	30	2	500 μs	35.7 μs	FR1, most TDD mid-band
2	60	4	250 μs	17.9 μs	FR1 and FR2
3	120	8	125 μs	8.9 μs	FR2 (mmWave)
4	240	16	62.5 μs	4.5 μs	FR2, SSB only
5, 6	480, 960	32, 64	31.25, 15.625 μs	2.2, 1.1 μs	FR2-2 (Rel-17)

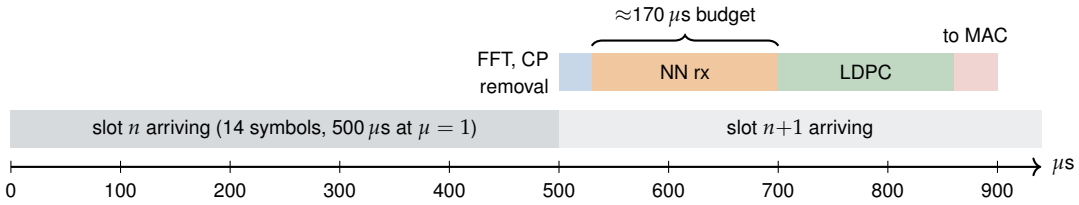


Figure 36.12: An illustrative processing timeline for one uplink slot at $\mu = 1$ (pipeline: while slot n is processed, slot $n{+}1$ is being received). The numbers are an example budget, not a standard: the actual deadline comes from the HARQ timing your scheduler promises and the processing-time capabilities defined in TS 38.214. The point is that the NN must meet its share deterministically, for the worst-case allocation (all PRBs, all layers).

Wireless connection: O-RAN timescales

The O-RAN architecture places learned models at three timescales: the non-real-time RIC (>1 s, e.g. training and policy), the near-real-time RIC (10 ms–1 s, e.g. handover or slicing control through xApps), and the DU/RU (<1 ms, the PHY). The same model-design question — what is my deadline? — decides where a learned function can live.

36.7.6 Monitoring and drift

A deployed model sees data that slowly stops resembling its training set: new interferers, firmware changes in transmitters, seasonal propagation changes (foliage, humidity), a replaced antenna. Monitor without labels where you can: the distribution of input power and estimated SNR, the distribution of maximum softmax confidence (a drop in mean confidence is an early warning), the fraction of frames classified as each class, and for receivers the CRC failure rate, which is a free label. Keep a fixed "canary" test set of real captures and re-run it on every new model and firmware. When drift is detected, go back to step 2 of Figure 36.1: capture, label (with CRCs where possible), fine-tune, re-validate.

36.8 Capstone projects

Choose one. Each is sized for 2–4 weeks of evenings and exercises every stage of this chapter. Write a short report following the task statement template of Section 36.1.

36.8.1 Capstone A: modulation classifier on your own SDR captures

Setup. Two ADALM-PLUTOs (or one in cable loop-back with 30–60 dB of attenuation), f_c in an unlicensed band you are permitted to use (or a cable only), 1 Msample/s, root-raised-cosine pulses, classes {BPSK, QPSK, 8PSK, 16QAM, 64QAM, noise-only}.

1. **M1 Data:** 3 sessions on 3 different days, each with an attenuator sweep in 3 dB steps; fixed RX gain; SigMF metadata. Plot one constellation per class per session.
2. **M2 Baseline:** cumulant classifier (Chapter 32) on the same frames.
3. **M3 Simulation pre-training:** the CNN of this chapter trained on simulated frames with random phase, CFO (± 5 ppm of f_c), timing offset and SNR.
4. **M4 Fine-tune** on sessions 1–2; test on session 3 only.
5. **M5 Deploy:** ONNX + static int8; measure latency on a Raspberry Pi-class board or laptop CPU.

Success criteria: on session 3, accuracy $\geq 90\,\%$ at estimated SNR ≥ 10 dB; beats the cumulant baseline by ≥ 5 points at 5 dB; int8 model within 1 point of float; latency < 1 ms per frame; recipe checks (init loss, overfit one batch, zero-input baseline) documented with plots.

36.8.2 Capstone B: a learned 16-QAM demapper robust to CFO

Setup. Simulated 16-QAM with Gray mapping, frames of 256 symbols, residual CFO uniform in ± 0.002 cycles/symbol, unknown phase, AWGN. Network: a small 1-D CNN or BiGRU over the frame outputting 4 logits per symbol, trained with `BCEWithLogitsLoss` on the bits (Table 36.1).

1. **M1:** baseline = pilot-based phase/CFO estimation (8 pilots) + max-log demapper; BER curve from 0 to 16 dB.
2. **M2:** network with known CFO $= 0$ must match the max-log BER within 0.2 dB (sanity check: it should learn the optimal demapper).
3. **M3:** train with random CFO; compare BER with baseline.
4. **M4:** check LLR calibration: histogram of LLRs conditioned on the true bit; compare with the ideal Gaussian-LLR consistency $p(\ell \mid b{=}0) = e^{\ell} p(-\ell \mid b{=}0)$.
5. **M5:** quantise to int8 and measure the BER loss in dB at BER $= 10^{-3}$.

Success criteria: within 0.5 dB of the genie-CFO max-log receiver at BER 10^{-3}; at least 1 dB better than the 8-pilot baseline; int8 loss ≤ 0.2 dB; sign convention of LLRs documented and tested.

36.8.3 Capstone C: an LSTM CSI predictor

Setup. Simulated Rayleigh fading with Jakes Doppler spectrum (Chapter 27), $f_D T_s \in [0.005, 0.05]$, 8 subcarriers $\times$ 2 antennas, noisy estimates at 10–20 dB. Predict the channel $d = 1..5$ steps ahead from the last 20 estimates.

1. **M1 Baselines:** "hold last value" and an order-p Wiener/AR predictor fitted on training data; NMSE in dB versus horizon d.
2. **M2:** LSTM (one layer, 64 units) with MSE loss on standardised real/imag parts; overfit one batch; LR sweep.

3. **M3:** generalisation test: train at $f_D T_s \leq 0.03$, test at 0.05 (speed mismatch).
4. **M4:** feed the predicted CSI to a precoder (MRT) and report the beamforming gain loss in dB versus perfect CSI.
5. **M5:** export to ONNX; check worst-case latency for one prediction $\ll$ one slot at $\mu = 1$.

Success criteria: NMSE at $d = 3$ at least 3 dB better than "hold last"; no worse than the AR predictor at matched Doppler; degradation under speed mismatch measured and explained; precoding gain loss < 1 dB at $d = 1$.

Chapter recap

- Frame the task first: input tensor, output, metric, classical baseline, latency budget.
- Choose the architecture from the data's structure (Fig. 36.2); choose head and loss from the output's meaning (Table 36.1).
- Build radio datasets with fixed gain, rich metadata, diversity in what should be ignored, and splits by session; compute normalisation statistics on the training split only.
- Recipe checks with predictable outcomes: loss at init $= \ln K$; overfit one batch to ≈ 0; zero-input baseline stays at $\ln K$; LR sweep is U-shaped; change one thing at a time.
- Wireless bugs: I/Q swap, SNR definitions, power normalisation mismatch, label leakage through power/SNR, channel and session mismatch, CFO not in training.
- Evaluate per SNR, under mismatch, with confusion matrices, BER/BLER against baselines, and calibration.
- Deploy as state_dict + config JSON; export to ONNX and verify outputs; quantise (dynamic for big matrices, static with calibration for integer hardware); plan fixed-point formats; meet worst-case slot deadlines; monitor drift.

Exercises

1. A 24-class classifier prints a first-batch loss of 9.7. What value did you expect, and list three possible causes of the discrepancy.
2. You overfit one batch of 32 examples and the loss decreases from 2.30 to 1.90 and stays there. The model has 50 k parameters. What do you conclude, and what are the next two things you check?
3. Weights of a layer are $(0.05, -0.02, 0.4, -1.6)$. Quantise them to symmetric int8 with one per-tensor scale, de-quantise, and give the largest error. Repeat if the -1.6 is clipped to -0.4 before choosing the scale. Discuss.
4. Express 0.3 in Q15 and Q7 (8-bit). What are the representation errors?
5. A neural channel estimator takes 120 μs per slot on your DSP. For which numerologies μ does it fit in a budget of 40 % of a slot?
6. Your classifier is 99 % accurate at -4 dB SNR on 128-sample frames of BPSK vs. 16QAM, but a colleague's matched-filter experiment says ≈ 80 % is the best possible. Give two hypotheses and how you would test each.
7. Explain why splitting radio captures by frame rather than by session overestimates test accuracy. Design a split for a dataset of 6 sessions recorded on 3 days with 2 different

transmitters.

8. Why can dynamic int8 quantisation make a tiny CNN slower, as in Figure 36.11? When would you expect it to help most?

Answers

1. Expected $\ln 24 = 3.18$. Causes: inputs not normalised (huge activations $\Rightarrow$ huge logits), last-layer weights too large, labels outside 0..23 or one-hot labels passed where indices are expected, softmax applied before the loss (would give a value *near* $\ln K$ but trains badly), loss summed rather than averaged over the batch ($9.7/3.18 \approx 3$ would not match a batch size, so likely scaling).

2. A 50 k-parameter network must memorise 32 examples; a plateau at 1.90 (well above 0) indicates plumbing trouble. Check (a) that labels are aligned with inputs (print several input–label pairs after the loader) and (b) that all parameters receive gradients (`p.grad` not `None`/zero; no frozen layers; `zero_grad` placement). Also try a $10\times$ larger LR.

3. $s = 1.6/127 = 0.012598$; $q = (\text{round}(3.97), \text{round}(-1.59), \text{round}(31.75), -127) = (4, -2, 32, -127)$; de-quantised $(0.0504, -0.0252, 0.4031, -1.6)$; largest error 0.0052 (on -0.02, a 26 % relative error). With clipping: $s = 0.4/127 = 0.0031496$, $q = (16, -6, 127, -127)$, de-quantised $(0.0504, -0.0189, 0.4, -0.4)$; small weights now have errors below 0.0016, but the clipped weight has error 1.2. Clipping trades a large error on a rare outlier for better resolution on the common values; calibration methods choose this trade-off (e.g. percentile clipping) or use per-channel scales to avoid it.

4. Q15: $\text{round}(0.3 \cdot 32768) = 9830 \Rightarrow 0.299988$, error 1.2×10^{-5}. Q7: $\text{round}(0.3 \cdot 128) = 38 \Rightarrow 0.296875$, error 3.1×10^{-3}.

5. 40 % of a slot: $\mu = 0$: 400 μs (fits); $\mu = 1$: 200 μs (fits); $\mu = 2$: 100 μs (does not fit); $\mu = 3$: 50 μs (no). So $\mu \in \{0, 1\}$.

6. (i) Label leakage through power or SNR (e.g. one class recorded with a different gain, or frames not normalised): test by normalising every frame to unit power and by evaluating on a set where gain is randomised per frame. (ii) Train/test contamination (the same frames or same capture in both sets, or the noise generator re-seeded identically): test by checking for duplicate frames and re-splitting by session/seed. Also verify that the SNR is defined as you think (E_s/N_0 vs per-sample with oversampling could mean the real SNR is several dB higher).

7. Frames from one session share channel, gain, CFO and hardware quirks; the network can recognise the session and its label rather than the modulation, so the test set is not independent. Split: train on day 1 and day 2 sessions from transmitter A and B except one; validate on one held-out session; test on day 3 sessions, ideally including a transmitter-swapped session; report also a "leave-one-day-out" cross-validation.

8. Dynamic quantisation must compute the activation range and quantise the input at run time for every layer, then de-quantise outputs; for a network whose matrices are tiny, these overheads exceed the savings from int8 multiplications. It helps most when matrix multiplications dominate: large Linear/LSTM layers, Transformers, batch sizes large enough to amortise overhead, and memory-bound inference where loading $4\times$ fewer bytes matters.

Self-test quiz

1. What training loss do you expect for a balanced 8-class problem before training?
   Answer: $\ln 8 \approx 2.08$.
2. Your loss is stuck at exactly $\ln K$ even on a single batch. Most likely bug?
   Answer: labels are not aligned with inputs (or inputs are constant/zero), so no function of the input can predict them.
3. Why must normalisation statistics be computed on the training split only?
   Answer: otherwise information about the test set leaks into training, and the stats must be identical at deployment, so they are part of the model.
4. Which quantisation scheme needs calibration data and why?
   Answer: post-training static quantisation, because activation ranges (scales and zero points) are fixed in advance from representative inputs.
5. How long is a 5G NR slot at 30 kHz subcarrier spacing?
   Answer: 0.5 ms ($\mu = 1$).
6. Name two wireless-specific causes of a model that works in simulation but is at chance on real captures.
   Answer: power normalisation mismatch; CFO/phase not represented in training (also I/Q swap, SNR definition mismatch).

Further reading

- A. Karpathy, "A Recipe for Training Neural Networks," blog post, 25 April 2019, `https://karpathy.github.io/2019/04/25/recipe/`.
- T. J. O'Shea, T. Roy and T. C. Clancy, "Over-the-Air Deep Learning Based Radio Signal Classification," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 168–179, 2018 — the RadioML 2018.01A dataset paper, including over-the-air captures.
- C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, "On Calibration of Modern Neural Networks," ICML 2017 — temperature scaling.
- B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," CVPR 2018.
- ONNX Runtime documentation, "Quantize ONNX models" (`https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html`).
- 3GPP TS 38.211, *NR; Physical channels and modulation*, Section 4 (numerologies and frame structure).

Appendix A

Equation Cheat Sheet

This appendix gathers the formulas of the whole book into compact tables you can keep beside you while reading papers or writing code. Nothing here is new: every line was derived, explained and exercised in an earlier chapter (the right-hand column tells you where). Use the tables to *remember*, not to *learn*; if a line looks unfamiliar, go back to its chapter and redo the worked example.

Intuition: how to use a cheat sheet

A formula is a compressed picture. When you look up a line below, try to replay the picture that goes with it: the gradient as an arrow pointing uphill, softmax as a "soft" winner-takes-all, Adam as a ball with momentum whose step is rescaled per parameter, the Bellman equation as "today's value is reward now plus the discounted value of tomorrow". If no picture comes, that is your cue to revisit the chapter.

Conventions. Scalars x; vectors $\mathbf{x}$ (column vectors in maths, rows in batch code); matrices $\mathbf{W}$; $\odot$ element-wise product; σ the logistic sigmoid; η learning rate; N batch size; K number of classes; log is the natural logarithm unless written $\log_2$ or $\log_{10}$.

A.1 Mathematical foundations

Linear algebra (Chapter 2)

Quantity	Formula / note
Dot product	$\mathbf{a}\cdot\mathbf{b}=\mathbf{a}^{\mathsf{T}}\mathbf{b}=\sum_i a_i b_i=\|\mathbf{a}\|\,\|\mathbf{b}\|\cos\theta$
Euclidean norm	$\|\mathbf{x}\|_2=\sqrt{\sum_i x_i^2}$; $\|\mathbf{x}\|_1=\sum_i \|x_i\|$
Matrix product	$(\mathbf{AB})_{ij}=\sum_k A_{ik}B_{kj}$; $(m\times n)(n\times p)\to(m\times p)$; cost mnp multiply-adds
Transpose rules	$(\mathbf{AB})^{\mathsf{T}}=\mathbf{B}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}$; $(\mathbf{AB})^{\mathsf{H}}=\mathbf{B}^{\mathsf{H}}\mathbf{A}^{\mathsf{H}}$
Inverse	$\mathbf{A}\mathbf{A}^{-1}=\mathbf{I}$; $(\mathbf{AB})^{-1}=\mathbf{B}^{-1}\mathbf{A}^{-1}$; 2×2: $\begin{pmatrix}a&b\\c&d\end{pmatrix}^{-1}=\frac{1}{ad-bc}\begin{pmatrix}d&-b\\-c&a\end{pmatrix}$
Eigen-decomposition	$\mathbf{Av}=\lambda\mathbf{v}$; symmetric $\mathbf{A}=\mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^{\mathsf{T}}$ with orthonormal $\mathbf{Q}$
SVD	$\mathbf{A}=\mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\mathsf{H}}$; singular values $\sigma_i\geq0$ = gains of the matrix along orthogonal directions
Trace, determinant	$\operatorname{tr}\mathbf{A}=\sum_i A_{ii}=\sum_i\lambda_i$; $\det\mathbf{A}=\prod_i\lambda_i$
Frobenius norm	$\|\mathbf{A}\|_F^2=\sum_{ij}\|A_{ij}\|^2=\operatorname{tr}(\mathbf{A}^{\mathsf{H}}\mathbf{A})=\sum_i\sigma_i^2$
Projection onto $\mathbf{u}$	$\operatorname{proj}_{\mathbf{u}}\mathbf{x}=\dfrac{\mathbf{u}^{\mathsf{T}}\mathbf{x}}{\mathbf{u}^{\mathsf{T}}\mathbf{u}}\mathbf{u}$
Least squares	$\hat{\mathbf{w}}=\arg\min\|\mathbf{Xw}-\mathbf{y}\|^2=(\mathbf{X}^{\mathsf{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathsf{T}}\mathbf{y}$

Calculus and the chain rule (Chapter 3)

Rule	Formula
Derivative (definition)	$f'(x) = \lim_{h\to 0} \dfrac{f(x+h)-f(x)}{h}$; numerical check: $\dfrac{f(x+h)-f(x-h)}{2h}$, error $O(h^2)$
Basic derivatives	$(x^n)' = nx^{n-1}$, $(e^x)' = e^x$, $(\ln x)' = 1/x$, $(\sin x)' = \cos x$, $(a^x)' = a^x \ln a$
Product, quotient	$(fg)' = f'g + fg'$; $\quad (f/g)' = (f'g - fg')/g^2$
Chain rule (scalar)	$\dfrac{dy}{dx} = \dfrac{dy}{du}\dfrac{du}{dx}$ for $y = f(u),\ u = g(x)$
Multivariable chain rule	$\frac{\partial L}{\partial x} = \sum_j \frac{\partial L}{\partial u_j}\frac{\partial u_j}{\partial x}$ (sum over every path from x to L)
Gradient	$\nabla f = (\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n})^\top$; points in the direction of steepest increase
Jacobian	$\mathbf{J}_{ij} = \partial y_i / \partial x_j$; $\quad$ vector chain rule $\mathbf{J}_{\mathbf{z}\leftarrow\mathbf{x}} = \mathbf{J}_{\mathbf{z}\leftarrow\mathbf{y}}\mathbf{J}_{\mathbf{y}\leftarrow\mathbf{x}}$
Matrix gradients	$\nabla_{\mathbf{x}}(\mathbf{a}^\top\mathbf{x}) = \mathbf{a}$; $\nabla_{\mathbf{x}}(\mathbf{x}^\top\mathbf{A}\mathbf{x}) = (\mathbf{A}+\mathbf{A}^\top)\mathbf{x}$; $\nabla_{\mathbf{x}}\lVert\mathbf{x}\rVert^2 = 2\mathbf{x}$
Taylor (2nd order)	$f(\mathbf{x}+\boldsymbol{\delta}) \approx f(\mathbf{x}) + \nabla f^\top\boldsymbol{\delta} + \frac{1}{2}\boldsymbol{\delta}^\top\mathbf{H}\boldsymbol{\delta}$, $\mathbf{H}$ = Hessian

Probability and information (Chapter 4)

Quantity	Formula
Expectation, variance	$\mathbb{E}[X] = \sum_x x\,p(x)$; $\operatorname{Var} X = \mathbb{E}[X^2] - \mathbb{E}[X]^2$; $\operatorname{Var}(aX+b) = a^2 \operatorname{Var} X$
Bayes' rule	$p(y \mid x) = \dfrac{p(x \mid y)\,p(y)}{p(x)}$, $\quad p(x) = \sum_y p(x \mid y)p(y)$
Gaussian	$\mathcal{N}(x;\mu,\sigma^2) = \dfrac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/(2\sigma^2)}$
Complex Gaussian	$\mathcal{CN}(0,\sigma^2)$: real and imaginary parts i.i.d. $\mathcal{N}(0,\sigma^2/2)$
Multivariate Gaussian	$\mathcal{N}(\mathbf{x};\boldsymbol{\mu},\boldsymbol{\Sigma}) = (2\pi)^{-d/2}\lvert\boldsymbol{\Sigma}\rvert^{-1/2}\exp(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}))$
Entropy	$H(p) = -\sum_x p(x)\log p(x)$ (nats; use $\log_2$ for bits)
Cross-entropy	$H(p,q) = -\sum_x p(x)\log q(x) = H(p) + D_{\mathrm{KL}}(p\Vert q)$
KL divergence	$D_{\mathrm{KL}}(p\Vert q) = \sum_x p(x)\log\dfrac{p(x)}{q(x)} \geq 0$
KL of Gaussians	$D_{\mathrm{KL}}(\mathcal{N}(\mu,\sigma^2)\Vert\mathcal{N}(0,1)) = \frac{1}{2}(\mu^2+\sigma^2-\ln\sigma^2-1)$
Mutual information	$I(X;Y) = H(Y) - H(Y \mid X) = D_{\mathrm{KL}}(p(x,y)\Vert p(x)p(y))$
Maximum likelihood	$\hat\theta = \arg\max_\theta \sum_i \log p(y_i \mid x_i;\theta) \equiv$ minimise negative log-likelihood

A.2 Neurons, losses and backpropagation

Neuron, layer and activations (Chapters 5–7)

Name	Function $f(z)$	Derivative $f'(z)$
Neuron	$a = f(\mathbf{w}^\top \mathbf{x} + b)$	—
Layer (batch-first)	$\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{b}$, $\mathbf{A} = f(\mathbf{Z})$	—
Sigmoid	$\sigma(z) = \dfrac{1}{1+e^{-z}}$	$\sigma(z)(1-\sigma(z)) \le 0.25$
Tanh	$\tanh z = \dfrac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1$	$1 - \tanh^2 z$
ReLU	$\max(0, z)$	1 if $z > 0$ else 0
Leaky ReLU	$\max(\alpha z, z)$, e.g. $\alpha = 0.01$	1 if $z > 0$ else α
ELU	z if $z > 0$ else $\alpha(e^z - 1)$	1 if $z > 0$ else $f(z) + \alpha$
GELU	$z\,\Phi(z) \approx 0.5z(1 + \tanh[\sqrt{2/\pi}(z + 0.044715z^3)])$	$\Phi(z) + z\,\phi(z)$
SiLU / Swish	$z\,\sigma(z)$	$\sigma(z)(1 + z(1 - \sigma(z)))$
Softplus	$\ln(1 + e^z)$	$\sigma(z)$
Softmax	$p_k = \dfrac{e^{z_k}}{\sum_j e^{z_j}}$ (subtract $\max_j z_j$ first)	$\frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j)$

Losses and their gradients with respect to the network output (Chapter 8)

Loss	Per-example value	Gradient
Mean squared error	$\frac{1}{2}(\hat{y} - y)^2$ (or $(\hat{y} - y)^2$)	$\hat{y} - y$ (or $2(\hat{y} - y)$)
Mean absolute error	$\lvert\hat{y} - y\rvert$	$\operatorname{sign}(\hat{y} - y)$
Huber (δ)	$\frac{1}{2}r^2$ if $\lvert r\rvert \le \delta$, else $\delta(\lvert r\rvert - \frac{1}{2}\delta)$; $r = \hat{y} - y$	r clipped to $[-\delta, \delta]$
Binary cross-entropy	$-[y \log p + (1-y)\log(1-p)]$, $p = \sigma(z)$	$\frac{\partial L}{\partial z} = p - y$
Categorical CE	$-\sum_k y_k \log p_k = -\log p_{\text{true}}$, $\mathbf{p} = \operatorname{softmax}(\mathbf{z})$	$\frac{\partial L}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}$
CE at initialisation	$\approx \ln K$ (uniform guess); $K = 10 \Rightarrow 2.303$	—
Label smoothing ϵ	target $y_k = (1-\epsilon)\mathbb{1}[k = c] + \epsilon/K$	$\mathbf{p} - \mathbf{y}_{\text{smooth}}$
Gaussian NLL	$\frac{1}{2}\ln\sigma^2 + \dfrac{(y-\mu)^2}{2\sigma^2}$	$\frac{\partial L}{\partial \mu} = \dfrac{\mu - y}{\sigma^2}$
Focal loss	$-(1-p_t)^\gamma \log p_t$	down-weights easy examples

Key equation: backpropagation for an L-layer network (Chapter 10)

Forward: $\mathbf{Z}^{(l)} = \mathbf{A}^{(l-1)}\mathbf{W}^{(l)} + \mathbf{b}^{(l)}$, $\mathbf{A}^{(l)} = f(\mathbf{Z}^{(l)})$, $\mathbf{A}^{(0)} = \mathbf{X}$. Define the error signal $\boldsymbol{\Delta}^{(l)} =$

$\frac{\partial L}{\partial \mathbf{Z}^{(l)}}$.

$$\begin{aligned}
\boldsymbol{\Delta}^{(L)} &= \tfrac{1}{N}(\mathbf{P}-\mathbf{Y}) \qquad \text{(softmax+CE, sigmoid+BCE, linear+MSE)}\\
\frac{\partial L}{\partial \mathbf{W}^{(l)}} &= \mathbf{A}^{(l-1)\mathsf{T}}\boldsymbol{\Delta}^{(l)}, \qquad \frac{\partial L}{\partial \mathbf{b}^{(l)}} = \textstyle\sum_{\text{rows}} \boldsymbol{\Delta}^{(l)}\\
\boldsymbol{\Delta}^{(l-1)} &= (\boldsymbol{\Delta}^{(l)}\mathbf{W}^{(l)\mathsf{T}}) \odot f'(\mathbf{Z}^{(l-1)})
\end{aligned}$$

Shapes check: $\partial L/\partial \mathbf{W}$ always has the shape of $\mathbf{W}$. Gradient check: relative error $|g_{\text{num}} - g_{\text{bp}}| \,/\, \max(|g_{\text{num}}|, |g_{\text{bp}}|) < 10^{-6}$ in float64.

A.3 Optimisation and regularisation

Optimisers (Chapters 9 and 11); $\mathbf{g}_t = \nabla_\theta L(\theta_{t-1})$

Method	Update	Typical values
SGD	$\theta_t = \theta_{t-1} - \eta\,\mathbf{g}_t$	$\eta = 0.01$–0.1
Momentum	$\mathbf{v}_t = \mu\mathbf{v}_{t-1} + \mathbf{g}_t$, $\theta_t = \theta_{t-1} - \eta\mathbf{v}_t$	$\mu = 0.9$
Nesterov	$\mathbf{v}_t = \mu\mathbf{v}_{t-1} + \nabla L(\theta_{t-1} - \eta\mu\mathbf{v}_{t-1})$, $\theta_t = \theta_{t-1} - \eta\mathbf{v}_t$	$\mu = 0.9$
AdaGrad	$\mathbf{s}_t = \mathbf{s}_{t-1} + \mathbf{g}_t^2$, $\theta_t = \theta_{t-1} - \eta\,\mathbf{g}_t/(\sqrt{\mathbf{s}_t} + \epsilon)$	$\eta = 0.01$
RMSProp	$\mathbf{s}_t = \rho\mathbf{s}_{t-1} + (1-\rho)\mathbf{g}_t^2$, $\theta_t = \theta_{t-1} - \eta\,\mathbf{g}_t/(\sqrt{\mathbf{s}_t} + \epsilon)$	$\rho = 0.9$–0.99, $\eta = 10^{-3}$
Adam	$\mathbf{m}_t = \beta_1\mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t$; $\mathbf{v}_t = \beta_2\mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2$; $\hat{\mathbf{m}}_t = \mathbf{m}_t/(1-\beta_1^t)$, $\hat{\mathbf{v}}_t = \mathbf{v}_t/(1-\beta_2^t)$; $\theta_t = \theta_{t-1} - \eta\,\hat{\mathbf{m}}_t/(\sqrt{\hat{\mathbf{v}}_t} + \epsilon)$	$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\eta = 3\times 10^{-4}$ to 10^{-3}
AdamW	Adam step, plus decoupled decay $\theta_t \leftarrow \theta_t - \eta\lambda\theta_{t-1}$	$\lambda = 10^{-2}$
L2 in the loss	$L + \frac{\lambda}{2}\|\theta\|^2 \Rightarrow \mathbf{g}_t \leftarrow \mathbf{g}_t + \lambda\theta$ (equals weight decay only for SGD)	$\lambda = 10^{-4}$
Cosine schedule	$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$	with warm-up
Gradient clipping	if $\|\mathbf{g}\| > c$: $\mathbf{g} \leftarrow c\,\mathbf{g}/\|\mathbf{g}\|$	$c = 1$ (RNNs)
Convergence on a quadratic	stable iff $0 < \eta < 2/\lambda_{\max}(\mathbf{H})$; best $\eta = 2/(\lambda_{\max} + \lambda_{\min})$	—

Initialisation (Chapter 11); n_{in}, n_{out} = fan-in, fan-out

Scheme	Variance of weights	Use with
Xavier / Glorot normal	$\operatorname{Var} W = \dfrac{2}{n_{\text{in}} + n_{\text{out}}}$	tanh, sigmoid, linear
Xavier uniform	$W \sim U[-a, a]$, $a = \sqrt{6/(n_{\text{in}} + n_{\text{out}})}$	same

Scheme	Variance of weights	Use with
He / Kaiming normal	$\operatorname{Var} W = \dfrac{2}{n_{\text{in}}}$	ReLU family
He uniform	$a = \sqrt{6/n_{\text{in}}}$	ReLU
LeCun	$\operatorname{Var} W = 1/n_{\text{in}}$	SELU, linear
Biases	$b = 0$ (output bias may be set to the log class prior)	—
Why	keeps $\operatorname{Var}(z^{(l)})$ and $\operatorname{Var}(\delta^{(l)})$ constant across layers	—

Regularisation and normalisation (Chapters 11–12)

Technique	Formula / rule
L2 (ridge)	$L_{\text{tot}} = L + \frac{\lambda}{2}\sum w^2$; gradient adds λw (shrinks weights)
L1 (lasso)	$L_{\text{tot}} = L + \lambda \sum \lvert w\rvert$; gradient adds $\lambda \operatorname{sign}(w)$ (sparse weights)
Inverted dropout	train: $\tilde{a} = a \cdot m/(1-p)$, $m \sim \text{Bernoulli}(1-p)$; test: $\tilde{a} = a$
Early stopping	keep the weights with the best validation metric; stop after "patience" epochs without improvement
BatchNorm (train)	$\mu_B = \frac{1}{N}\sum_i x_i$, $\sigma_B^2 = \frac{1}{N}\sum_i (x_i - \mu_B)^2$, $\hat{x}_i = \dfrac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$, $y_i = \gamma \hat{x}_i + \beta$ (per feature/channel, over the batch)
BatchNorm (test)	use running averages: $\mu \leftarrow (1-m)\mu + m\mu_B$ (PyTorch momentum $m = 0.1$)
LayerNorm	same formula, but statistics over the features of *one* example; identical in train and test
Bias–variance	$\mathbb{E}[(y - \hat{f})^2] = \text{bias}^2 + \text{variance} + \sigma^2_{\text{noise}}$

A.4 Architectures

Convolutional networks (Chapter 15)

Quantity	Formula
2-D convolution (CNN)	$y[i,j] = \sum_c \sum_m \sum_n w_c[m,n]\, x_c[i+m, j+n] + b$ (really cross-correlation)
Output size	$O = \left\lfloor \dfrac{I + 2P - D(k-1) - 1}{S} \right\rfloor + 1$ (input I, kernel k, padding P, stride S, dilation D); for $D = 1$: $\lfloor (I + 2P - k)/S \rfloor + 1$
"Same" padding	$P = (k-1)/2$ for odd k, $S = 1$
Parameters	$(k_h k_w C_{\text{in}} + 1)\, C_{\text{out}}$
Multiply-adds	$k_h k_w C_{\text{in}} C_{\text{out}}\, H_{\text{out}} W_{\text{out}}$

Quantity	Formula
Receptive field	$r_l = r_{l-1} + (k_l - 1)\prod_{i<l} S_i$, $r_0 = 1$
Transposed conv size	$O = (I-1)S - 2P + D(k-1) + P_{\text{out}} + 1$
Max-pool 2×2, $S = 2$	halves H and W; gradient flows only to the arg-max position

Recurrent networks (Chapter 16); $[\mathbf{h}, \mathbf{x}]$ = concatenation

Cell	Equations
Vanilla RNN	$\mathbf{h}_t = \tanh(\mathbf{W}_h\mathbf{h}_{t-1} + \mathbf{W}_x\mathbf{x}_t + \mathbf{b})$, $\hat{\mathbf{y}}_t = \mathbf{W}_y\mathbf{h}_t + \mathbf{c}$
BPTT factor	$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{i=k+1}^{t} \operatorname{diag}(1 - \mathbf{h}_i^2)\mathbf{W}_h$; vanishes/explodes like $\lvert\lambda_{\max}(\mathbf{W}_h)\rvert^{t-k}$
LSTM	$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$, $\mathbf{i}_t = \sigma(\mathbf{W}_i[\cdot] + \mathbf{b}_i)$, $\mathbf{o}_t = \sigma(\mathbf{W}_o[\cdot] + \mathbf{b}_o)$
	$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\cdot] + \mathbf{b}_c)$, $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$, $\mathbf{h}_t = \mathbf{o}_t \odot \tanh \mathbf{c}_t$
GRU	$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t])$, $\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t])$
	$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t])$, $\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$
Parameter count	RNN $H(H + D + 1)$; LSTM $4H(H + D + 1)$; GRU $3H(H + D + 1)$ (PyTorch adds a second bias)

Attention and Transformers (Chapter 17)

Block	Formula
Scaled dot-product attention	$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}$, mask $\mathbf{M}_{ij} = -\infty$ for forbidden pairs
Projections	$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, $\mathbf{V} = \mathbf{X}\mathbf{W}_V$
Multi-head	$\text{MHA} = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}_O$, $d_k = d_{\text{model}}/h$
Sinusoidal position	$PE_{(p,2i)} = \sin(p/10000^{2i/d})$, $PE_{(p,2i+1)} = \cos(p/10000^{2i/d})$
Pre-LN block	$\mathbf{X} \leftarrow \mathbf{X} + \text{MHA}(\text{LN}(\mathbf{X}))$; $\mathbf{X} \leftarrow \mathbf{X} + \text{FFN}(\text{LN}(\mathbf{X}))$
FFN	$\text{FFN}(\mathbf{x}) = \mathbf{W}_2\ \text{GELU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$, inner width $\approx 4d$
Cost	attention $O(T^2 d)$ time, $O(T^2)$ memory per head; parameters per block $\approx 12d^2$
Why $\sqrt{d_k}$	if q_i, k_i have unit variance, $\text{Var}(\mathbf{q}^\top\mathbf{k}) = d_k$; dividing keeps logits $O(1)$

A.5 Generative models and reinforcement learning

Autoencoders, VAEs, GANs and diffusion (Chapter 18)

Model	Key formula
Autoencoder	$\min \ \|\mathbf{x} - g(f(\mathbf{x}))\|^2$
VAE ELBO	$\log p(\mathbf{x}) \geq \mathbb{E}_{q(\mathbf{z}\|\mathbf{x})}[\log p(\mathbf{x} \mid \mathbf{z})] - D_{\mathrm{KL}}(q(\mathbf{z} \mid \mathbf{x}) \| p(\mathbf{z}))$
VAE KL term	$\frac{1}{2}\sum_j(\mu_j^2 + \sigma_j^2 - \ln \sigma_j^2 - 1)$ for $q = \mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}\,\boldsymbol{\sigma}^2)$, $p = \mathcal{N}(0, \mathbf{I})$
Reparameterisation	$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$
GAN minimax	$\min_G \max_D \ \mathbb{E}_{\mathbf{x}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]$
Non-saturating G loss	$\min_G -\mathbb{E}_{\mathbf{z}}[\log D(G(\mathbf{z}))]$
Optimal D	$D^*(\mathbf{x}) = \dfrac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_G(\mathbf{x})}$; then G minimises $2\,\mathrm{JSD} - \ln 4$
DDPM forward	$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\,\mathbf{x}_{t-1}, \beta_t \mathbf{I})$
DDPM closed form	$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$, $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$
DDPM loss	$L_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} \ \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2$
DDPM sampling	$\mathbf{x}_{t-1} = \dfrac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \dfrac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$, $\sigma_t^2 = \beta_t$

Reinforcement learning (Chapter 19)

Concept	Formula
Return	$G_t = \sum_{k \geq 0} \gamma^k r_{t+k+1}$, $0 \leq \gamma < 1$
Value functions	$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$; $\quad Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$
Bellman expectation	$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a)\,[r(s,a,s') + \gamma V^\pi(s')]$
Bellman optimality	$Q^*(s,a) = \sum_{s'} P(s' \mid s, a)\,[r + \gamma \max_{a'} Q^*(s', a')]$
Q-learning	$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s,a)]$
SARSA	$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s', a') - Q(s,a)]$
ϵ-greedy	random action with probability ϵ, else $\arg\max_a Q(s,a)$
DQN target	$y = r + \gamma(1 - \text{done}) \max_{a'} Q_{\theta^-}(s', a')$; loss $(y - Q_\theta(s,a))^2$ or Huber
Double DQN	$y = r + \gamma\, Q_{\theta^-}(s', \arg\max_{a'} Q_\theta(s', a'))$
Policy gradient	$\nabla_\theta J = \mathbb{E}_\pi[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\,(G_t - b(s_t))]$
Advantage	$A(s,a) = Q(s,a) - V(s)$; TD error $\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$
UCB1 (bandits)	choose $\arg\max_a \ \bar{r}_a + c\sqrt{\ln t / n_a}$

A.6 Wireless communications

Signals, SNR and error rates (Chapter 20)

Quantity	Formula
Complex baseband	$s(t) = \mathrm{Re}\{\tilde{s}(t) e^{j2\pi f_c t}\}$, $\tilde{s} = I + jQ$

Quantity	Formula
Euler	$e^{j\phi} = \cos\phi + j\sin\phi$; $\lvert I + jQ\rvert = \sqrt{I^2 + Q^2}$
dB	$x_{\text{dB}} = 10\log_{10} x$ (power); $\text{dBm} = 10\log_{10}(P/1\,\text{mW})$
SNR	$\text{SNR} = P_s/P_n = E_s/N_0$ (one complex sample per symbol, matched filter)
E_b/N_0	$E_b/N_0 = \dfrac{E_s/N_0}{R\log_2 M}$ (R = code rate); in dB: $E_s/N_0 = E_b/N_0 + 10\log_{10}(R\log_2 M)$
AWGN model	$y = x + n$, $n \sim \mathcal{CN}(0, N_0)$; with $\mathbb{E}\lvert x\rvert^2 = 1$: $N_0 = 10^{-\text{SNR}_{\text{dB}}/10}$
Q-function	$Q(x) = \frac{1}{2}\operatorname{erfc}(x/\sqrt{2})$; $Q(x) \approx \frac{1}{x\sqrt{2\pi}} e^{-x^2/2}$ for large x
BPSK / QPSK BER	$P_b = Q(\sqrt{2E_b/N_0})$ (Gray-coded QPSK same per bit)
M-PSK SER (approx.)	$P_s \approx 2Q(\sqrt{2E_s/N_0}\,\sin(\pi/M))$
Square M-QAM SER	$P_s \approx 4(1 - \frac{1}{\sqrt{M}})Q\left(\sqrt{\frac{3}{M-1}\frac{E_s}{N_0}}\right)$; Gray $P_b \approx P_s/\log_2 M$
BPSK in Rayleigh	$P_b = \frac{1}{2}\left(1 - \sqrt{\dfrac{\bar\gamma}{1+\bar\gamma}}\right) \approx \dfrac{1}{4\bar\gamma}$ at high SNR
Diversity order L	$P_b \propto \bar\gamma^{-L}$ (slope $-L$ decades per decade on log–log axes)

Channels, capacity and MIMO (Chapters 20, 26)

Quantity	Formula
Rayleigh fading	$h \sim \mathcal{CN}(0,1)$; $\lvert h\rvert$ Rayleigh, $\lvert h\rvert^2 \sim \text{Exp}(1)$: $p(\gamma) = \frac{1}{\bar\gamma} e^{-\gamma/\bar\gamma}$
Rician K-factor	$h = \sqrt{\frac{K}{K+1}} e^{j\phi} + \sqrt{\frac{1}{K+1}}\,\mathcal{CN}(0,1)$
Path loss (log-distance)	$PL(d) = PL(d_0) + 10n\log_{10}(d/d_0) + X_\sigma$; free space $n = 2$, $\text{FSPL} = 20\log_{10}\dfrac{4\pi d}{\lambda}$
Doppler, coherence	$f_D = v f_c/c$; $T_c \approx 0.423/f_D$; Jakes autocorrelation $J_0(2\pi f_D \tau)$
Delay spread	coherence bandwidth $B_c \approx 1/(5\sigma_\tau)$ (50 % correlation rule of thumb)
Shannon (AWGN)	$C = B\log_2(1 + \text{SNR})$ bit/s; spectral efficiency $\log_2(1 + \text{SNR})$ bit/s/Hz
Ergodic Rayleigh	$\bar C = \mathbb{E}_h[\log_2(1 + \lvert h\rvert^2\,\text{SNR})]$
MIMO capacity	$C = \log_2 \det\left(\mathbf{I}_{N_r} + \frac{\text{SNR}}{N_t}\mathbf{H}\mathbf{H}^{\text{H}}\right) = \sum_i \log_2\left(1 + \frac{\text{SNR}}{N_t}\sigma_i^2\right)$ (no CSIT)
Water-filling (CSIT)	$P_i = (\mu - N_0/\sigma_i^2)^+$, $\sum_i P_i = P$
MRC	$\hat x = \mathbf{h}^{\text{H}}\mathbf{y}/\lVert\mathbf{h}\rVert^2$; output SNR $\lVert\mathbf{h}\rVert^2\,\text{SNR}$
ZF / MMSE	$\mathbf{G}_{\text{ZF}} = (\mathbf{H}^{\text{H}}\mathbf{H})^{-1}\mathbf{H}^{\text{H}}$; $\mathbf{G}_{\text{MMSE}} = (\mathbf{H}^{\text{H}}\mathbf{H} + N_0\mathbf{I})^{-1}\mathbf{H}^{\text{H}}$
Alamouti 2×1	send (x_1, x_2) then $(-x_2^*, x_1^*)$; $\hat x_1 = h_1^* y_1 + h_2 y_2^*$, $\hat x_2 = h_2^* y_1 - h_1 y_2^*$, each with gain $\lvert h_1\rvert^2 + \lvert h_2\rvert^2$ (diversity 2, rate 1)

Detection, features and OFDM (Chapters 20–23, 32–35)

Quantity	Formula
ML detection (AWGN)	$\hat{x} = \arg\min_{x \in \mathcal{X}} \lvert y - hx \rvert^2$
Bit LLR	$\mathrm{LLR}(b) = \ln \dfrac{P(b=0 \mid y)}{P(b=1 \mid y)}$; BPSK in AWGN ($0 \mapsto +1$): $\mathrm{LLR} = \dfrac{4\,\mathrm{Re}\{h^* y\}}{N_0}$
Max-log LLR	$\mathrm{LLR}(b_i) \approx \dfrac{1}{N_0}\left[\min_{x \in \mathcal{X}_i^1} \lvert y - hx \rvert^2 - \min_{x \in \mathcal{X}_i^0} \lvert y - hx \rvert^2\right]$
LLR $\leftrightarrow$ BCE	a demapper trained with BCE on bits outputs logits = LLRs (sign convention!)
Moments	$M_{pq} = \mathbb{E}[y^{p-q}(y^*)^q]$
Cumulants	$C_{20} = M_{20}$, $C_{21} = M_{21}$, $C_{40} = M_{40} - 3M_{20}^2$, $C_{41} = M_{41} - 3M_{20}M_{21}$, $C_{42} = M_{42} - \lvert M_{20} \rvert^2 - 2M_{21}^2$
Normalised C_{42}/C_{21}^2	BPSK -2, QPSK -1, 8PSK -1, 16QAM -0.68, 64QAM -0.619, Gaussian noise 0
Normalised $\lvert C_{40} \rvert / C_{21}^2$	BPSK 2, QPSK 1, 8PSK 0, 16QAM 0.68
OFDM	$x[n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[k] e^{j2\pi kn/N}$; CP length $\geq$ channel memory $\Rightarrow$ $Y[k] = H[k]X[k] + W[k]$
LS channel estimate	$\hat{H}[k] = Y[k]/X_p[k]$ at pilots, interpolate between pilots
5G NR numerology	$\Delta f = 15 \cdot 2^\mu$ kHz; slot $= 1\,\mathrm{ms}/2^\mu$ with 14 OFDM symbols (normal CP); $\mu = 0..3$ data in Rel-15
CFO	$y[n] = x[n] e^{j2\pi\epsilon n/N}$; phase grows linearly with time: rotation of the constellation over a frame

A.7 Evaluation and deployment

Metric	Formula
Accuracy, error	$\mathrm{acc} = \frac{1}{N} \sum_i \mathbb{1}[\hat{y}_i = y_i]$
Precision, recall	$P = \dfrac{TP}{TP+FP}$, $R = \dfrac{TP}{TP+FN}$, $F_1 = \dfrac{2PR}{P+R}$
ROC	$\mathrm{TPR} = TP/(TP+FN)$ vs $\mathrm{FPR} = FP/(FP+TN)$ as the threshold moves; AUC = area under it
Expected calibration error	$\mathrm{ECE} = \sum_b \frac{\lvert B_b \rvert}{N} \lvert \mathrm{acc}(B_b) - \mathrm{conf}(B_b) \rvert$
Temperature scaling	$\mathbf{p} = \mathrm{softmax}(\mathbf{z}/T)$, T fitted on validation NLL
Error-rate confidence	to see n_e errors at rate p, simulate $\approx n_e/p$ trials; rule of thumb $n_e \geq 100$
NMSE	$\mathrm{NMSE} = \mathbb{E}\left\lVert \hat{\mathbf{H}} - \mathbf{H} \right\rVert_F^2 / \mathbb{E}\left\lVert \mathbf{H} \right\rVert_F^2$, reported in dB
Symmetric int8 quantisation	$q = \mathrm{round}(x/s)$, $s = \max \lvert x \rvert / 127$, $\hat{x} = sq$; error $\leq s/2$
Affine (asymmetric)	$q = \mathrm{round}(x/s) + z$, $s = (x_{\max} - x_{\min})/255$, z = zero point

Metric	Formula
Q-format fixed point	Q$m.n$: value $=$ integer $\cdot 2^{-n}$; range $[-2^m, 2^m - 2^{-n}]$ (with sign bit), resolution 2^{-n}

Pitfall: the five sign and scale traps

1. PyTorch's `CrossEntropyLoss` expects *raw logits*; never apply softmax before it.
2. LLR sign: some texts use $\ln P(b{=}1)/P(b{=}0)$. Check before connecting a learned demapper to a decoder.
3. E_b/N_0 vs. E_s/N_0 vs. per-sample SNR differ by $10\log_{10}(R\log_2 M)$ and by the oversampling factor.
4. $\mathcal{CN}(0, N_0)$ means variance $N_0/2$ *per real dimension*.
5. AdamW weight decay is not the same as L2 in the loss when the optimiser is adaptive.

Appendix B

Study Plan, Resources and Paper Index

This appendix turns the book into a plan. It gives a week-by-week schedule with a concrete deliverable for every week, an annotated list of books, courses and software tools, a complete index of the eleven wireless papers taught in Part V with full citations and the chapter where each is explained, and the landmark deep-learning papers behind the techniques of Parts II–IV. Citations were checked against the publishers' or authors' pages and arXiv in October 2026; web addresses change, so if one fails, search for the title.

B.1 A twenty-week study plan

Intuition: why a plan with deliverables

Reading is not learning. Each week below ends with something you *produce* — a derivation on paper, a script that runs, a plot that matches the book's. Producing it is what moves knowledge from "I recognise this" to "I can do this". If a week overruns, do not skip its deliverable; let the schedule slide instead.

The plan assumes 8–10 hours per week: roughly two evenings of reading and one long session of coding. Readers with an engineering background can merge weeks 1–3 into one or two.

Table B.1: Week-by-week schedule.

Week	Chapters	Deliverable (you are done when…)
1	1, 2 Linear algebra	you can multiply a 2×3 by a 3×2 matrix by hand, explain a dot product as a projection, and compute the SVD of a 2×2 matrix with NumPy and interpret it.
2	3 Calculus	you can differentiate a composition of three functions with the chain rule and verify the result with a central-difference numerical derivative.
3	4 Probability & information	you can compute entropy, cross-entropy and KL divergence of two small distributions by hand, and explain why minimising cross-entropy is maximum likelihood.
4	5, 6 Learning from data; the neuron	a single neuron trained by gradient descent separates two Gaussian clouds; you can sketch sigmoid, tanh and ReLU and their derivatives from memory.
5	7, 8 Layers; losses	you derive $\partial L/\partial \mathbf{z} = \mathbf{p} - \mathbf{y}$ for softmax + cross-entropy on paper.
6	9, 10 Gradient descent; backprop	full backpropagation through a 2-2-1 network with numbers, on paper, matching a computer check to 4 decimals.

Week	Chapters	Deliverable
7	11, 12 Init, optimisers, regularisation, normalisation, metrics	a plot of training curves for SGD, momentum and Adam on the same problem, and a confusion matrix with precision/recall computed by hand.
8	13 NumPy network lab	your NumPy MLP passes a gradient check (relative error $< 10^{-6}$) and reaches $> 95\,\%$ on the bundled 8×8 digits.
9	14 PyTorch lab	the same network in PyTorch, with a clean training loop, validation and saved weights.
10	15 CNNs	you compute output sizes and parameter counts of a 3-layer CNN by hand and confirm with `torchinfo` or a print of shapes.
11	16, 17 RNN/LSTM/GRU; Transformers	an LSTM beats a vanilla RNN on a long-memory toy task; a two-layer Transformer encoder trains on a sequence task.
12	18, 19 Generative models; RL	a VAE that generates digits; tabular Q-learning that solves a grid world; one DQN run.
13	20 Wireless primer	BER of BPSK and 16QAM in AWGN and Rayleigh simulated and overlaid on the theoretical curves.
14	21, 22 Autoencoder PHY; modulation recognition	a $(7,4)$ autoencoder whose BLER approaches the Hamming soft-ML curve; a CNN classifier with an accuracy-vs-SNR plot.
15	23, 24, 25 Decoding; model-free E2E; noisy feedback	a neural decoder for a short code with its NVE or BER curve; alternating training without a channel model.
16	26, 27 MIMO; CSI prediction	a 2×2 MIMO autoencoder compared with Alamouti; an LSTM predictor beating "hold last value".
17	28–31 Channel features, localisation, DRL access, BitTorrent	reproduce one figure from each of two of these chapters.
18	32–35 Kaggle labs	a modulation classifier that stays above 80 % at 10 dB in Rayleigh multipath, compared with the cumulant baseline.
19	36 Your own model	a capstone task statement; data collected or simulated; all recipe checks passed and plotted.
20	36 Capstone completion	the capstone success criteria met, model exported to ONNX and quantised, a two-page report written.

Wireless connection: a parallel track for the working PHY engineer

If you work on a baseband platform, run a "shadow project" alongside weeks 13–20: take one block from your own receiver chain (channel estimation on the DMRS, the soft demapper, or PRACH detection), log its inputs and outputs to files, and train a small network to imitate and then improve on it. By week 20 you will have a model, a classical baseline that is *your production code*, and real data — the strongest possible capstone.

B.2 Books

Deep learning

S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. The best modern companion to this book: clear figures, careful maths, chapters on Transformers, diffusion models and RL, and Python notebooks for every chapter. A free PDF is offered by the author at `https://udlbook.github.io/udlbook/`.

C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. A rigorous, probabilistic treatment by the author of the classic *Pattern Recognition and Machine Learning*; excellent on the probabilistic view of losses, normalising flows, VAEs and diffusion.

I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016. The standard reference of the first deep-learning decade; Part I (applied maths and ML basics) and the chapters on optimisation and regularisation remain excellent. Free online at `https://www.deeplearningbook.org`.

M. Nielsen, *Neural Networks and Deep Learning*, Determination Press, 2015. A short, gentle, free online book (`http://neuralnetworksanddeeplearning.com`) that builds backpropagation from scratch with intuition; ideal alongside Chapters 6–10.

A. Zhang, Z. C. Lipton, M. Li and A. J. Smola, *Dive into Deep Learning*, Cambridge University Press, 2023. Code-first: every concept implemented in PyTorch (and other frameworks) in runnable notebooks. Free at `https://d2l.ai`.

R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018. *The* reinforcement-learning textbook, behind Chapters 19 and 30. Free PDF from the authors at `http://incompleteideas.net/book/the-book-2nd.html`.

Wireless communications

D. Tse and P. Viswanath, *Fundamentals of Wireless Communication*, Cambridge University Press, 2005. Fading, diversity, capacity and MIMO with exceptional physical intuition; the best second book after Chapter 20.

J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed., McGraw-Hill, 2008. The encyclopaedic reference for modulation, detection, error probability, equalisation and coding.

A. Goldsmith, *Wireless Communications*, Cambridge University Press, 2005. Engineering-oriented and very readable: path loss and shadowing, fading statistics, adaptive modulation, OFDM, multiuser systems.

B.3 Courses and videos

3Blue1Brown, "Neural networks" series. Grant Sanderson's animated videos (YouTube; `https://www.3blue1brown.com/topics/neural-networks`) on what a network is, gradient descent, backpropagation and its calculus, and, in later videos, Transformers and attention. Watch the first four before Chapter 10; his "Essence of linear algebra" and "Essence of calculus" series pair with Chapters 2–3.

A. Karpathy, "Neural Networks: Zero to Hero". A free video course (`https://karpathy.ai/zero-to-hero.html`) that builds an autograd engine (micrograd), then character-level language models from bigrams through an MLP, BatchNorm, manual backprop and a WaveNet-style model, to a GPT from scratch and its tokenizer. The "becoming a backprop ninja" lecture is the perfect exercise after Chapter 10.

Stanford CS231n, *Deep Learning for Computer Vision*. Lecture notes (`https://cs231n.github.io`) and recorded lectures; the notes on backpropagation, optimisation and "babysitting the learning process" are classics. Course site: `https://cs231n.stanford.edu`.

Stanford CS224n, *Natural Language Processing with Deep Learning*. Word vectors, RNNs, attention, Transformers, pre-training; recorded lectures and assignments at `https://web.stanford.edu/class/cs224n/`. Useful background for Chapter 17.

B.4 Tools and datasets for wireless machine learning

NVIDIA Sionna. An open-source library for communication-systems research (`https://nvlabs.github.io/sionna/`, code at `https://github.com/NVlabs/sionna`) with three components: Sionna PHY (link-level simulation: 5G NR LDPC and polar codes, OFDM, MIMO, 3GPP channel models), Sionna SYS (system-level abstraction) and Sionna RT (a differentiable ray tracer). Every block is differentiable, so learned components can be trained end to end inside a standard-compliant chain. The current documentation (version 2.x) states that PHY and SYS are written in PyTorch; earlier releases used TensorFlow, so check which version a tutorial targets.

MATLAB Deep Learning Toolbox, 5G Toolbox, Communications Toolbox. MathWorks' toolboxes provide standard-compliant NR waveform generation and reception (5G Toolbox), channel models and impairments (Communications Toolbox) and network training/deployment (Deep Learning Toolbox), with many worked AI-for-wireless examples (modulation classification, CSI feedback compression, neural receivers, beam selection).

GNU Radio. The open-source signal-processing framework for SDRs (`https://www.gnuradio.org`), with a graphical flowgraph editor (GNU Radio Companion), drivers for Pluto, USRP and RTL-SDR devices, and Python blocks into which a trained network (for example via ONNX Runtime) can be inserted. The original RadioML datasets were generated with it.

DeepMIMO. A ray-tracing-based dataset generator and database for mmWave and massive-MIMO research (`https://deepmimo.net`; Python package `deepmimo`, now at version 4). Scenarios provide site-specific channels for beam prediction, localisation and CSI tasks. Reference: A. Alkhateeb, "DeepMIMO: A Generic Deep Learning Dataset for Millimeter Wave and Massive MIMO Applications," Proc. ITA Workshop, 2019, arXiv:1902.06435.

RadioML datasets (DeepSig). Distributed at `https://www.deepsig.ai/datasets/` under the CC BY-NC-SA 4.0 licence. **RadioML 2016.10A**: 11 modulations (8 digital, 3 analog), short frames of 128 complex samples, SNR from -20 to $+18$ dB, a Python pickle file; described in T. J. O'Shea and N. West, "Radio Machine Learning Dataset Generation with GNU Radio," *Proc. GNU Radio Conference*, 2016. **RadioML 2018.01A**: 24 modulations, frames of 1024 samples, about 2 million examples in HDF5, including over-the-air-like impairments; described in O'Shea, Roy and Clancy, *IEEE J. Sel. Topics Signal Process.*, 2018 (see Further reading of Chapter 36). DeepSig

itself notes that these historical datasets have known errata and encourages building datasets from real captures — the approach of Chapter 36.

Hardware and file formats. ADALM-PLUTO (Analog Devices; Python via `pyadi-iio`), KrakenSDR (KrakenRF; five coherent channels), SigMF (an open metadata standard for recorded I/Q: `https://sigmf.org`).

B.5 Index of the wireless papers taught in this book

Table B.2 lists the eleven papers of Part V with full citations. Where a paper has an arXiv preprint, its identifier is given; the arXiv version may differ slightly from the published one.

Table B.2: The wireless papers of Part V.

Ch.	Citation
21	T. J. O'Shea and J. Hoydis, "An Introduction to Deep Learning for the Physical Layer," *IEEE Transactions on Cognitive Communications and Networking*, vol. 3, no. 4, pp. 563–575, Dec. 2017. arXiv:1702.00832.
22	N. E. West and T. J. O'Shea, "Deep Architectures for Modulation Recognition," in *Proc. IEEE International Symposium on Dynamic Spectrum Access Networks (DySPAN)*, Baltimore, MD, 2017. arXiv:1703.09197.
23	T. Gruber, S. Cammerer, J. Hoydis and S. ten Brink, "On Deep Learning-Based Channel Decoding," in *Proc. 51st Annual Conference on Information Sciences and Systems (CISS)*, Baltimore, MD, Mar. 2017. arXiv:1701.07738.
24	F. Ait Aoudia and J. Hoydis, "Model-Free Training of End-to-End Communication Systems," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 11, pp. 2503–2516, Nov. 2019. arXiv:1812.05929.
25	M. Goutay, F. Ait Aoudia and J. Hoydis, "Deep Reinforcement Learning Autoencoder with Noisy Feedback," in *Proc. 17th International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt)*, Avignon, France, June 2019. arXiv:1810.05419.
26	T. J. O'Shea, T. Erpek and T. C. Clancy, "Deep Learning Based MIMO Communications," arXiv:1707.07980, July 2017. (A closely related conference version: T. J. O'Shea, T. Erpek and T. C. Clancy, "Physical Layer Deep Learning of Encodings for the MIMO Fading Channel," *Proc. Allerton Conference*, 2017.)
27	C. Luo, J. Ji, Q. Wang, X. Chen and P. Li, "Channel State Information Prediction for 5G Wireless Communications: A Deep Learning Approach," *IEEE Transactions on Network Science and Engineering*, vol. 7, no. 1, pp. 227–236, Jan.–Mar. 2020.
28	S. Navabi, C. Wang, O. Y. Bursalioglu and H. Papadopoulos, "Predicting Wireless Channel Features Using Neural Networks," in *Proc. IEEE International Conference on Communications (ICC)*, Kansas City, MO, May 2018, pp. 1–6. arXiv:1802.00107.
29	L. Zhao, H. Wang, P. Li and J. Liu, "An Improved WiFi Indoor Localization Method Combining Channel State Information and Received Signal Strength," in *Proc. 36th Chinese Control Conference (CCC)*, Dalian, China, 2017, pp. 8964–8969. doi:10.23919/ChiCC.2017.8028783. (Related deep-learning work: C.-H. Hsieh, J.-Y. Chen and B.-H. Nien, *IEEE Access*, vol. 7, 2019.)

Ch.	Citation
30	S. Wang, H. Liu, P. H. Gomes and B. Krishnamachari, "Deep Reinforcement Learning for Dynamic Multichannel Access in Wireless Networks," *IEEE Transactions on Cognitive Communications and Networking*, vol. 4, no. 2, pp. 257–265, June 2018. arXiv:1802.06958.
31	R. Izhak-Ratzin, H. Park and M. van der Schaar, "Online Learning in BitTorrent Systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 23, no. 12, pp. 2280–2288, Dec. 2012.

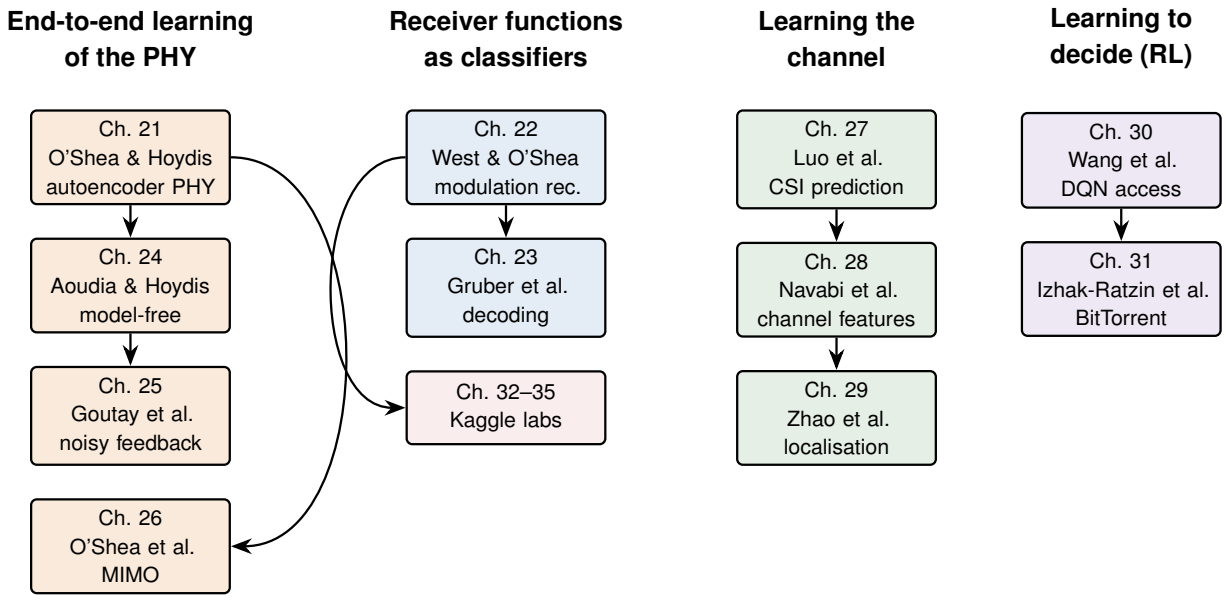


Figure B.1: How the papers of Part V relate. Arrows mean "builds on or is best read after". The autoencoder paper of Chapter 21 is the root of the end-to-end column; the modulation-recognition paper leads directly into the Kaggle labs of Part VI.

B.6 Landmark deep-learning papers

These are the original sources of the main techniques used in this book, in the order the techniques appear. Reading the original paper after the corresponding chapter is an excellent exercise: you will now understand every equation.

Table B.3: Landmark papers behind Parts II–IV.

Topic (chapter)	Citation
Xavier init (11)	X. Glorot and Y. Bengio, "Understanding the Difficulty of Training Deep Feedforward Neural Networks," in *Proc. AISTATS*, PMLR vol. 9, pp. 249–256, 2010.
He init (11)	K. He, X. Zhang, S. Ren and J. Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," in *Proc. ICCV*, 2015. arXiv:1502.01852.

Topic (chapter)	Citation
Dropout (11)	N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.
BatchNorm (12)	S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proc. ICML*, 2015. arXiv:1502.03167.
LayerNorm (12)	J. L. Ba, J. R. Kiros and G. E. Hinton, "Layer Normalization," arXiv:1607.06450, 2016.
Adam (11)	D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," arXiv:1412.6980, 2014; published at ICLR 2015.
AdamW (11)	I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *Proc. ICLR*, 2019. arXiv:1711.05101.
ResNet (15)	K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. CVPR*, 2016. arXiv:1512.03385 (Dec. 2015).
LSTM (16)	S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
Transformer (17)	A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1706.03762.
GELU (6, 17)	D. Hendrycks and K. Gimpel, "Gaussian Error Linear Units (GELUs)," arXiv:1606.08415, 2016.
VAE (18)	D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," arXiv:1312.6114, 2013; published at ICLR 2014.
GAN (18)	I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville and Y. Bengio, "Generative Adversarial Nets," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014. arXiv:1406.2661.
DDPM (18)	J. Ho, A. Jain and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2006.11239.
DQN (19)	V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski et al., "Human-Level Control through Deep Reinforcement Learning," *Nature*, vol. 518, pp. 529–533, 2015.
Training recipe (36)	A. Karpathy, "A Recipe for Training Neural Networks," blog post, 25 Apr. 2019, `https://karpathy.github.io/2019/04/25/recipe/`.

Pitfall: reading papers efficiently

Read in three passes. *Pass 1* (10 minutes): title, abstract, figures and their captions, conclusions — decide whether to continue. *Pass 2* (an hour): the system model and the main figure you want to reproduce; write down every symbol and its meaning in your own notation. *Pass 3* (a few evenings): reproduce one figure at small scale, exactly as the paper chapters of Part V do. A paper you have reproduced is a paper you own.

B.7 Where to go next

- **AI in the 5G-Advanced/6G air interface.** 3GPP's Release 18 study on AI/ML for the NR air interface (CSI feedback, beam management, positioning) and the follow-up normative work described in Chapter 20; the specifications and technical reports (e.g. TR 38.843) are public at `https://www.3gpp.org`.
- **Neural receivers on real hardware.** Open-source stacks such as OpenAirInterface and srsRAN, combined with GPU or CPU inference, make it possible to place a learned block inside a live gNB in a lab.
- **Sensing and localisation.** Joint communication and sensing, angle-of-arrival estimation with coherent arrays, and RF fingerprinting are active areas where the methods of Chapters 15, 29 and 36 apply directly.

Chapter recap: appendix recap

- Twenty weeks, one deliverable per week; let the calendar slip rather than skip a deliverable.
- Prince and Bishop & Bishop for depth; Goodfellow et al. and Nielsen as classics; *Dive into Deep Learning* for code; Sutton & Barto for RL; Tse & Viswanath, Proakis & Salehi and Goldsmith for wireless.
- Sionna, MATLAB, GNU Radio, DeepMIMO and the RadioML datasets are the main tools; build your own captures for anything you will deploy.
- Table B.2 is the map from each Part V paper to its chapter; Table B.3 gives the original sources of the core techniques.

Appendix C

Glossary

This glossary defines the terms used in this book from both worlds, deep learning and wireless communications, in plain words. Each entry is meant to remind you of an idea, not to replace the chapter that teaches it; the chapter number in brackets tells you where to look. Terms are in alphabetical order; acronyms are listed under their letters.

A

Activation function The simple non-linear function a neuron applies to its weighted sum, such as ReLU, sigmoid or tanh. Without it, a stack of layers would collapse into a single linear map (Ch. 6).

Adam An optimiser that keeps running averages of each parameter's gradient (momentum) and squared gradient, and divides one by the square root of the other, so every parameter gets its own step size. A learning rate of 3×10^{-4} to 10^{-3} is a common start (Ch. 11).

AdamW Adam with weight decay applied directly to the weights rather than added to the gradient; the two are not equivalent for adaptive optimisers, and AdamW usually generalises better.

ADC (analogue-to-digital converter) The device that samples a voltage and turns it into integers. Its number of bits sets the dynamic range; driving it too hard causes clipping.

AGC (automatic gain control) A receiver loop that adjusts amplification so the signal fills the ADC range. Convenient in operation, dangerous in dataset capture because it changes power in ways that can leak labels (Ch. 36).

Alamouti code A space–time block code for two transmit antennas that sends two symbols over two time slots in an orthogonal pattern, achieving full diversity with a simple linear receiver (Ch. 20, 26).

Angle of arrival (AoA) The direction from which a wave reaches an antenna array, estimated from phase differences between the elements.

Attention A mechanism that lets each element of a sequence compute a weighted average of all elements, with weights given by how well its query matches their keys (Ch. 17).

Augmentation (data augmentation) Creating extra training examples by transforming existing ones in ways that should not change the label, for radio data random phase, frequency offset, time shift and noise.

Autoencoder A network trained to reproduce its input through a narrow middle layer, forcing it to learn a compact code. Viewing a transmitter–channel–receiver chain as an autoencoder is the key idea of Ch. 21.

Autograd (automatic differentiation) The machinery in frameworks such as PyTorch that records the operations of the forward pass and applies the chain rule backwards to compute every gradient exactly.

AWGN (additive white Gaussian noise) The basic channel model $y = x + n$ with independent complex Gaussian noise of equal power at all frequencies.

B

Backpropagation The algorithm that computes the gradient of the loss with respect to every weight by applying the chain rule from the output layer back to the input, reusing intermediate results (Ch. 10).

Bandit (multi-armed) A simplified reinforcement-learning problem with one state and several actions of unknown average reward, where the agent must balance exploring and exploiting.

Baseband The representation of a radio signal shifted down to be centred at zero frequency, described by complex samples $I + jQ$.

Batch (mini-batch) The group of training examples processed together in one gradient step. Its size trades gradient noise against memory and speed.

Batch normalisation (BatchNorm) A layer that standardises each feature using the mean and variance over the current batch, then rescales with learned parameters; at test time it uses running averages (Ch. 12).

Beamforming Weighting the signals of several antennas so that their contributions add constructively in a chosen direction.

Bellman equation The recursive relation saying the value of a state equals the expected immediate reward plus the discounted value of the next state; the foundation of reinforcement learning (Ch. 19).

BER (bit error rate) The fraction of transmitted bits received in error, usually plotted on a logarithmic axis against SNR.

Bias (parameter) The constant added to a neuron's weighted sum, which shifts its decision boundary away from the origin.

Bias–variance trade-off The tension between models too simple to capture the truth (high bias, underfitting) and models so flexible they fit noise (high variance, overfitting).

BLER (block error rate) The fraction of transmitted blocks (codewords or transport blocks) that fail to decode; the main link metric in LTE and NR.

BPSK Binary phase-shift keying: one bit per symbol, sent as $+1$ or -1.

BPTT (backpropagation through time) Backpropagation applied to a recurrent network unrolled over its time steps.

C

Calibration The property that a model's stated confidence matches its actual accuracy; measured by the expected calibration error and fixed cheaply by temperature scaling (Ch. 36).

Capacity (Shannon) The largest rate at which information can be sent over a channel with arbitrarily small error probability; $B \log_2(1 + \mathrm{SNR})$ for AWGN.

CFO (carrier frequency offset) A mismatch between transmitter and receiver oscillator frequencies, which makes the received constellation rotate steadily over time.

Chain rule The calculus rule that the derivative of a composition is the product of the derivatives of its parts; the mathematical core of backpropagation (Ch. 3).

Channel coding Adding structured redundancy to data so the receiver can detect and correct errors; examples are Hamming, convolutional, LDPC and polar codes.

Channel estimation Measuring the channel's effect (gain and phase per subcarrier and antenna) from known pilot symbols.

Classification A task whose output is one of a finite set of categories, such as a modulation type.

CNN (convolutional neural network) A network built from convolution layers that slide small learned filters across the input, sharing weights across positions (Ch. 15).

Coherence bandwidth The frequency range over which the channel response stays roughly constant; inversely proportional to delay spread.

Coherence time The time over which the channel stays roughly constant; inversely proportional to the Doppler frequency.

Confusion matrix A table counting, for each true class, how often each class was predicted; it shows which mistakes a classifier makes.

Constellation The set of complex points a modulation scheme uses to represent symbols, drawn in the I/Q plane.

Convolution Sliding a small filter over a signal or image and computing a weighted sum at each position. (What deep-learning libraries call convolution is, strictly, cross-correlation.)

Cosine schedule A learning-rate schedule that decreases the rate along half a cosine wave from its maximum to near zero over training.

CP (cyclic prefix) A copy of the end of each OFDM symbol placed at its start; if it is longer than the channel's echoes, the channel acts as simple multiplication on each subcarrier.

Cross-entropy The loss $-\log p_{\text{true}}$ for classification, equal to the expected number of nats needed to encode the true labels with the model's probabilities (Ch. 8).

CSI (channel state information) Knowledge of the channel coefficients, at the receiver (CSIR) or fed back to the transmitter (CSIT).

Cumulant A statistic of a random variable related to its moments; higher-order cumulants of received samples are classic features for modulation classification (Ch. 32).

D

Data leakage Information about the test labels reaching the model during training, through shared recordings, normalisation statistics, or a feature correlated with the label by accident. It makes results look better than they will be.

Dataset split The division of data into training (fit), validation (tune) and test (final check) sets. For radio data split by recording session, not by frame.

dB (decibel) A logarithmic ratio: $10\log_{10}$ of a power ratio. 3 dB is a factor of two, 10 dB a factor of ten.

DDPM (denoising diffusion probabilistic model) A generative model that learns to reverse a gradual noising process step by step, turning random noise into data (Ch. 18).

Delay spread The spread in arrival times of the multipath copies of a signal; it causes frequency selectivity.

Demapper The receiver block that turns received symbols into bit decisions or bit LLRs.

Diversity Sending or receiving the same information over several independently fading paths (time, frequency, antennas) so that a deep fade on one is unlikely to destroy it.

Doppler shift The frequency shift caused by relative motion; the maximum is $f_D = v f_c / c$.

DQN (deep Q-network) Q-learning with a neural network approximating the action-value function, stabilised by experience replay and a target network (Ch. 19, 30).

Dropout A regulariser that randomly zeroes a fraction of activations during training, preventing units from co-adapting.

Drift (data drift) A gradual change in the input distribution after deployment, which slowly degrades a model's accuracy.

E

E_b/N_0 Energy per information bit divided by noise power spectral density; the fair SNR measure for comparing schemes with different rates.

Early stopping Stopping training when the validation metric stops improving and keeping the best weights seen.

ECE (expected calibration error) The average gap between confidence and accuracy over confidence bins.

ELBO (evidence lower bound) The objective maximised by a VAE: reconstruction log-likelihood minus a KL penalty keeping the latent code close to the prior.

Embedding A learned vector representing a discrete item (a word, a message index, a cell ID) so that a network can process it.

End-to-end learning Training all blocks of a system jointly for the final objective, rather than optimising each block separately (Ch. 21).

Entropy The average uncertainty of a random variable, $-\sum p \log p$; the minimum average code length to describe it.

Epoch One pass through the whole training set.

Equalisation Undoing the distortion the channel introduced, for example dividing by the estimated channel on each subcarrier.

Experience replay Storing past transitions and training a reinforcement-learning agent on random samples of them, which breaks correlations between consecutive updates.

Exploding gradient Gradients that grow exponentially as they are propagated back through many layers or time steps; controlled by clipping and careful initialisation.

F

Fading Fluctuation of the received signal strength caused by multipath interference (small-scale) or obstacles (shadowing).

FAPI An interface specification between the MAC and PHY layers of a base station, widely used in small-cell and O-RAN implementations.

Feature One input variable to a model; also a quantity computed from raw data (such as a cumulant) to make learning easier.

Fine-tuning Continuing to train a pre-trained model on a smaller, task-specific dataset, often with a lower learning rate.

Fingerprinting Localisation by matching a measured vector of RSS or CSI values against a database of measurements taken at known positions (Ch. 29).

Fixed point A number format that stores integers with an implied binary point, as in Q15; used by DSPs and FPGAs.

FR1 / FR2 The two frequency ranges of 5G NR: below about 7 GHz (FR1) and millimetre waves above 24 GHz (FR2).

G

GAN (generative adversarial network) A generator and a discriminator trained against each other: one makes fake samples, the other tries to tell them from real ones (Ch. 18).

GELU A smooth activation $z\Phi(z)$ used in Transformers; it behaves like ReLU for large $|z|$ but is smooth near zero.

Generalisation How well a model performs on data it was not trained on.

gNB The 5G NR base station.

Gradient The vector of partial derivatives of a function; it points in the direction of steepest increase.

Gradient check Comparing backpropagated gradients with numerical finite-difference estimates to detect implementation bugs (Ch. 13).

Gradient clipping Rescaling the gradient when its norm exceeds a threshold, to prevent unstable updates.

Gradient descent Repeatedly moving parameters a small step opposite to the gradient of the loss (Ch. 9).

Gray coding Labelling constellation points so neighbours differ in one bit, so the most likely symbol error causes only one bit error.

GRU (gated recurrent unit) A recurrent cell with update and reset gates; a lighter alternative to the LSTM.

H

HARQ (hybrid automatic repeat request) Retransmission with soft combining of failed attempts; its timing sets hard processing deadlines in a base station.

Hidden layer Any layer between the input and the output of a network.

Hyperparameter A setting chosen by the engineer rather than learned, such as learning rate, batch size or number of layers.

I

I/Q (in-phase / quadrature) The real and imaginary parts of a complex baseband sample. A swapped or conjugated I/Q pair mirrors the constellation.

I/Q imbalance Unequal gain or non-90° phase between the I and Q branches of a radio, which distorts the constellation.

Inductive bias The assumptions an architecture builds in about the function to be learned, such as translation equivariance for CNNs.

Inference Running a trained model to make predictions.

Initialisation The choice of starting weights, scaled (Xavier, He) so signals neither vanish nor explode through the layers (Ch. 11).

ISI (inter-symbol interference) Overlap between successive symbols caused by multipath delay; OFDM with a cyclic prefix avoids it.

K–L

KL divergence A measure of how one probability distribution differs from another; zero only when they are equal, and not symmetric.

Label The correct output attached to a training example.

Label smoothing Replacing one-hot targets by slightly softened ones to discourage over-confident predictions.

Latency budget The time a processing block is allowed per slot or frame; in a PHY it must hold in the worst case, every time.

Latent space The space of internal codes produced by an encoder, often lower-dimensional than the data.

Layer normalisation Standardising the features of each example separately; it behaves the same in training and testing and is standard in Transformers.

LDPC code Low-density parity-check code, decoded iteratively by message passing; used for NR data channels.

Learning rate The step size η in gradient descent; the single most important hyperparameter.

Linear regression Fitting a straight line (or hyperplane) to data by minimising squared error.

LLR (log-likelihood ratio) For a bit, $\ln \frac{P(b=0|y)}{P(b=1|y)}$: its sign is the hard decision and its magnitude the confidence; the input of soft channel decoders.

Logistic regression A linear model followed by a sigmoid, trained with binary cross-entropy; a one-neuron classifier.

Logits The raw, unnormalised scores a classifier outputs before softmax or sigmoid.

Loss function A number measuring how wrong the model's predictions are; training minimises it (Ch. 8).

LSTM (long short-term memory) A recurrent cell with a memory cell and input, forget and output gates, which can carry information over long sequences (Ch. 16).

M

MAC layer The medium access control layer above the PHY, responsible for scheduling and HARQ.

Matched filter The receive filter that maximises SNR at the sampling instant for a known pulse shape.

Maximum likelihood (ML) Choosing parameters or decisions that make the observed data most probable.

MIMO Multiple-input multiple-output: using several antennas at both ends for diversity or spatial multiplexing (Ch. 26).

MLP (multilayer perceptron) A network of fully connected layers with non-linear activations; the basic deep network (Ch. 7).

MMSE Minimum mean-squared error: the estimator or equaliser that minimises average squared error, balancing noise and interference.

Modulation Mapping bits onto a carrier's amplitude, phase or frequency; digital schemes include PSK and QAM.

Modulation recognition Identifying the modulation of an unknown signal, a classification task (Ch. 22, 32).

Momentum An optimiser modification that accumulates a running average of gradients, like a ball rolling downhill.

MSE (mean squared error) The average of squared differences between prediction and target; the standard regression loss.

Multipath Reception of several delayed, attenuated copies of a signal after reflection and scattering.

Mutual information The amount of information one random variable gives about another.

N

Neural receiver A neural network that replaces some or all of channel estimation, equalisation and demapping.

Neuron The basic unit of a network: a weighted sum of inputs plus a bias, passed through an activation (Ch. 6).

NMSE Normalised mean squared error, the error power divided by the signal power, often in dB.

Normalisation (input) Rescaling inputs to zero mean and unit variance (or unit power for radio frames) with statistics from the training set.

Numerology In 5G NR, the choice of subcarrier spacing $15 \cdot 2^{\mu}$ kHz, which fixes the slot duration $1/2^{\mu}$ ms.

O

OFDM Orthogonal frequency-division multiplexing: data is sent on many narrow subcarriers in parallel using an inverse FFT; the waveform of LTE, NR and WiFi.

One-hot encoding Representing a class as a vector of zeros with a single one in its position.

ONNX Open Neural Network Exchange, a framework-neutral file format for trained models, run by ONNX Runtime and other engines (Ch. 36).

O-RAN An architecture and set of open interfaces for radio access networks, splitting the base station into RU, DU and CU and adding RAN intelligent controllers.

Optimiser The algorithm that updates the weights from the gradients: SGD, momentum, RMSProp, Adam.

Overfitting Learning the noise and quirks of the training set so that performance on new data suffers.

P

Padding Adding zeros around the input of a convolution to control the output size.

Parameter A number learned by training, a weight or a bias.

Path loss The decrease of received power with distance, modelled as $10n \log_{10} d$ with exponent n.

Phase noise Random jitter of an oscillator's phase, which smears constellation points angularly.

PHY (physical layer) The lowest protocol layer, responsible for turning bits into waveforms and back.

Pilot A known symbol inserted for channel estimation and synchronisation (DMRS in NR).

Policy In reinforcement learning, the rule that maps states to actions or action probabilities.

Policy gradient A family of RL methods that adjust policy parameters along the gradient of expected return (Ch. 19).

Pooling Down-sampling by taking the maximum or average over a window; global average pooling averages over the whole input.

Positional encoding Information added to Transformer inputs to tell the model the order of elements.

PRACH The physical random access channel, used by a device to make first contact with a base station.

PRB (physical resource block) Twelve consecutive subcarriers over one slot; the unit of scheduling in LTE and NR.

Precision and recall Precision is the fraction of positive predictions that are right; recall is the fraction of true positives that are found.

Q

Q-function (Gaussian tail) $Q(x)$, the probability that a standard Gaussian exceeds x; error rates in AWGN are expressed with it.

Q-learning An RL algorithm that learns action values $Q(s, a)$ from experience by bootstrapping towards $r + \gamma \max_{a'} Q(s', a')$.

QAM Quadrature amplitude modulation: points on a square grid in the I/Q plane, e.g. 16QAM carries 4 bits per symbol.

QPSK Quadrature phase-shift keying: four phases, two bits per symbol.

Quantisation Representing weights and activations with fewer bits, typically int8, to shrink and speed up a model (Ch. 36).

R

RadioML Public modulation-recognition datasets released by DeepSig (2016.10A with 11 classes; 2018.01A with 24 classes).

Rayleigh fading Fading where the channel coefficient is complex Gaussian, typical with many scattered paths and no line of sight.

Receptive field The region of the input that influences one output of a convolutional network.

Recurrent neural network (RNN) A network that processes a sequence step by step, carrying a hidden state forward (Ch. 16).

Regression A task whose output is a continuous number or vector.

Regularisation Any technique that reduces overfitting: weight decay, dropout, augmentation, early stopping.

Reinforcement learning (RL) Learning to act by trial and error to maximise cumulative reward (Ch. 19).

ReLU The activation $\max(0, z)$; simple, cheap, and the default in most hidden layers.

Reparameterisation trick Writing a random latent as mean plus standard deviation times fixed noise, so gradients can flow through sampling in a VAE.

Residual connection Adding a layer's input to its output, $\mathbf{x} + F(\mathbf{x})$, which lets very deep networks train (ResNet).

Resource element (RE) One subcarrier during one OFDM symbol; the smallest unit of the NR time–frequency grid.

Reward The scalar feedback an RL agent receives after each action.

Rician fading Fading with a strong line-of-sight component plus scattered paths, characterised by the K-factor.

RMSProp An optimiser that divides each gradient by a running root-mean-square of its recent values.

RSS (received signal strength) The measured received power, often in dBm; a simple localisation feature.

S

SDR (software-defined radio) A radio whose signal processing is done in software on digitised samples, such as the ADALM-PLUTO or KrakenSDR.

Self-attention Attention in which queries, keys and values all come from the same sequence.

SGD (stochastic gradient descent) Gradient descent using the gradient of a random mini-batch instead of the whole dataset.

Shadowing Slow variation of received power caused by large obstacles, usually modelled as log-normal.

Sigmoid The S-shaped function $1/(1+e^{-z})$ that maps any number into $(0,1)$.

Sim-to-real gap The performance drop when a model trained on simulated data meets real hardware and channels.

Slot The basic scheduling interval of NR: 14 OFDM symbols, lasting $1/2^{\mu}$ ms.

SNR (signal-to-noise ratio) Signal power divided by noise power, usually quoted in dB.

Softmax The function that turns a vector of logits into probabilities by exponentiating and normalising.

Spatial multiplexing Sending independent data streams from several antennas at once to increase rate.

Spectrogram A time–frequency image of a signal's power, computed with a short-time Fourier transform.

SSB (synchronisation signal block) The NR block carrying the primary and secondary synchronisation signals and the broadcast channel, used for cell search.

State dict In PyTorch, the dictionary of a model's parameter tensors; the recommended way to save weights.

Stride The step by which a convolution or pooling window moves.

Subcarrier One of the narrowband carriers of an OFDM signal.

Supervised learning Learning from examples that come with correct labels.

SVD (singular value decomposition) Factoring a matrix into rotations and per-direction gains, $\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\mathsf{H}}$; it reveals the parallel sub-channels of a MIMO channel.

T

Tanh The hyperbolic tangent activation, an S-shaped function with outputs in $(-1,1)$.

Target network A slowly updated copy of a DQN used to compute stable training targets.

TDD / FDD Time-division duplex (uplink and downlink share a band at different times) and frequency-division duplex (separate bands).

Temperature scaling Dividing logits by a fitted constant to calibrate predicted probabilities.

Tensor A multi-dimensional array of numbers, the basic data object of deep-learning frameworks.

Test set Data held back and used only once, at the end, to estimate real-world performance.

Transfer learning Reusing a model trained on one task as the starting point for another (Ch. 18).

Transformer An architecture built from self-attention and feed-forward blocks with residual connections and layer normalisation (Ch. 17).

U–V

Underfitting A model too simple, or trained too little, to capture the patterns in the training data.

Unsupervised learning Finding structure in data without labels, e.g. clustering or autoencoders.

Validation set Data used during development to choose hyperparameters and decide when to stop training.

Value function The expected future return from a state (or state–action pair) under a policy.

Vanishing gradient Gradients that shrink towards zero as they flow back through many layers, stalling learning in early layers.

VAE (variational autoencoder) An autoencoder whose code is a probability distribution, trained with the ELBO, which can generate new samples (Ch. 18).

W–Z

Water-filling The optimal allocation of transmit power across parallel channels when the transmitter knows them: more power to stronger channels.

Weight A learned multiplier on a connection between neurons.

Weight decay Shrinking weights a little at every step, equivalent to an L2 penalty for plain SGD.

Xavier (Glorot) initialisation Weight initialisation with variance $2/(n_{\text{in}} + n_{\text{out}})$, suited to tanh and sigmoid.

Zero-forcing (ZF) A linear MIMO detector that inverts the channel matrix, removing interference at the cost of amplifying noise.